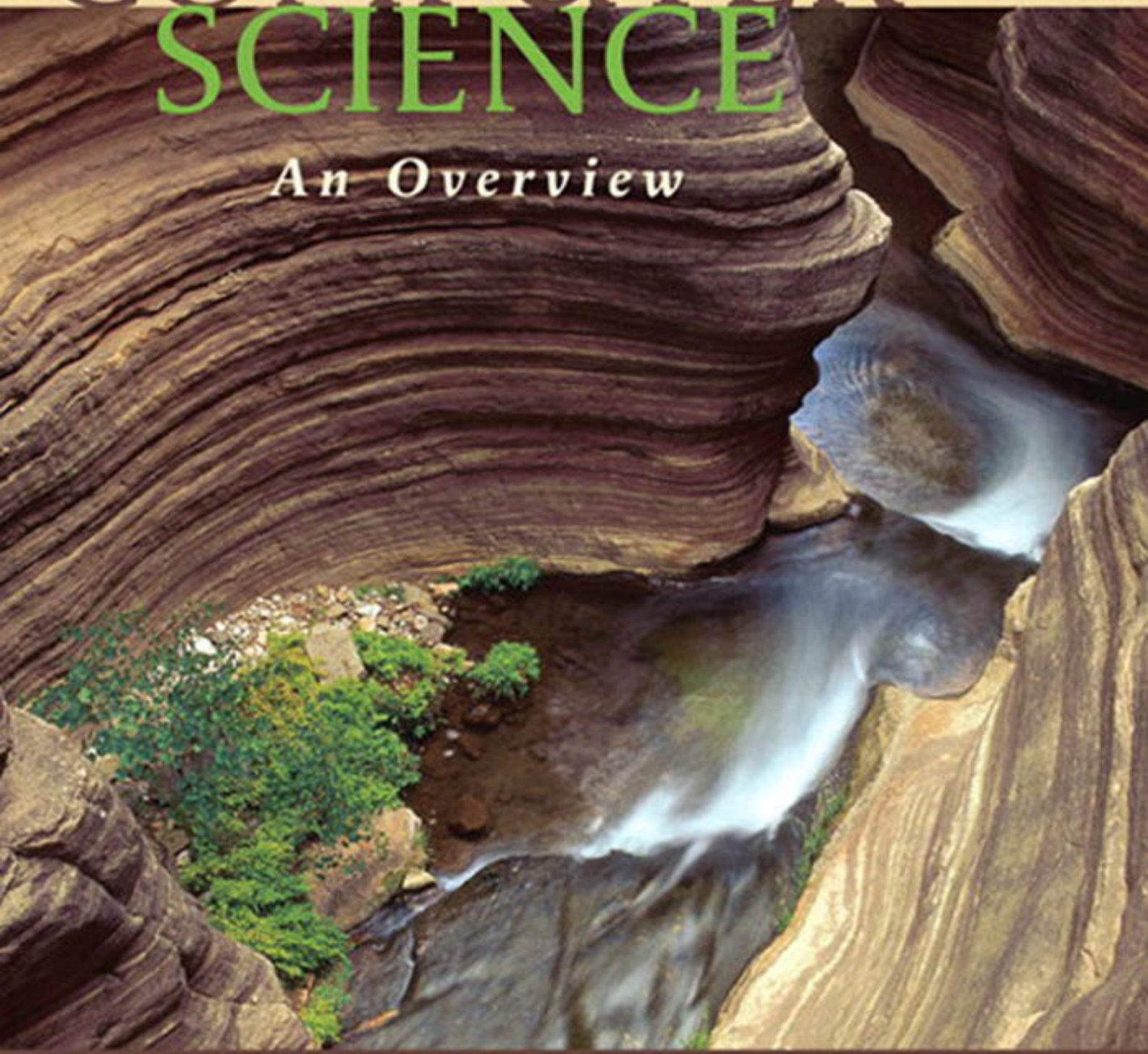
11th Edition

COMPUTER SCIENCE

An Overview



J. Glenn Brookshear

Doclingo https://www.doclingo.ai

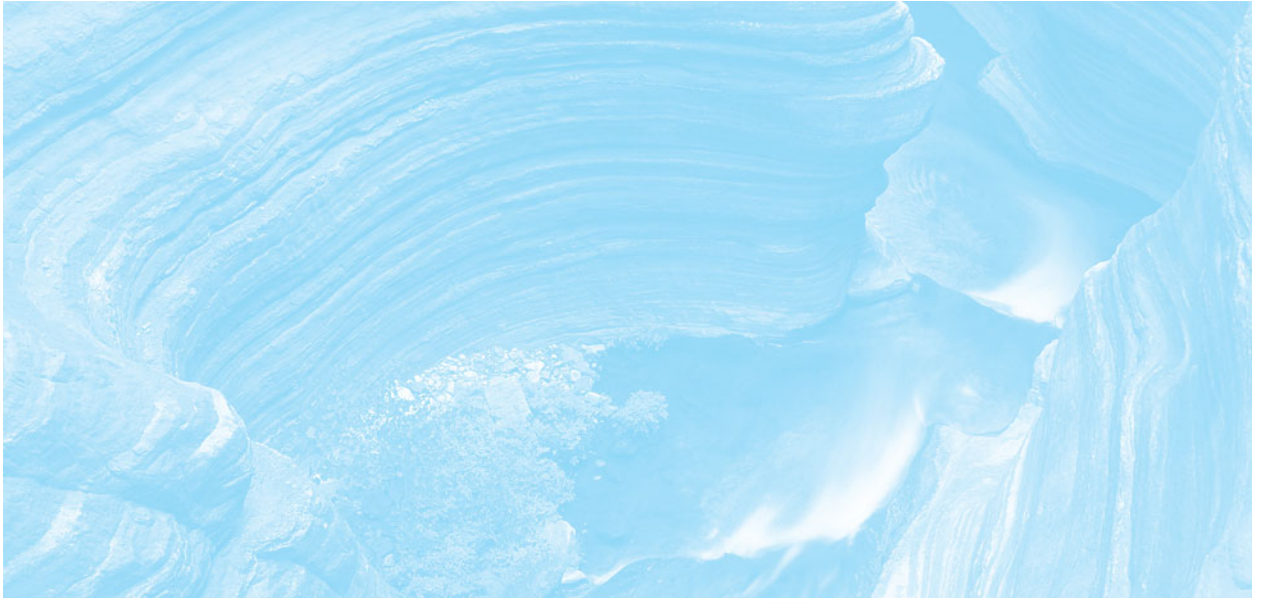
bilgisayar bilimi

BİR GENEL GÖRÜNÜM

Doclingo https://www.doclingo.ai

Bu sayfa kasıtlı olarak boş bırakılmıştır

Doclingo https://www.doclingo.ai



bilgisayar bilimi

BİR GENEL GÖRÜNÜM

11. Baskı

J. Glenn Brookshear

katkılarıyla

David T. Smith

Pennsylvania Üniversitesi

Dennis Brylow

Marquette Üniversitesi

Addison-Wesley

Boston Columbus Indianapolis New York San Francisco Upper Saddle River
Amsterdam Cape Town Dubai Londra Madrid Milano Münih Paris Montréal Toronto
Delhi Mexico City São Paulo Sidney Hong Kong Seul Singapur Taipei Tokyo

Doclingo https://www.doclingo.ai

*Yayın Yönetmeni: Marcia HortonB aş Editör: Michael Hirsch Yayın Asistanı: Stephanie Sellinger Pazarlama Kıdemli Başkan Yardımcısı: Patrice JonesPa zarlama Müdürü:*Yezan Alayan
*Pazarlama Koordinatörü: Kathryn FerrantiÜr etim Kıdemli Başkan Yardımcısı: Vince O'Brie nYönetici Editör:*Jeff Holcomb
*Üretim Proje Yöneticisi:*Kayla Smith-Tarbox
Kıdemli Operasyon Süpervizörü: Lisa McDowellSan at Yönetmenleri: Jayne Conte ve Kristine Carney
*Kapak Tasarımcısı:*Rachael Cronin
Kapak Görseli:"Yarık Kanyon" © gettyimages® Inc.
RF Medya Editörü: Dan Sandin ve Wanda Rockwell
Proje Yönetimi: GEX Yayın HizmetleriSayfa Düzeni ve İllüstrasyon: GEX YayınHizmetl eri
*Yazıcı/Bantlayıcı:*Edwards Brothers
*Kapak Yazıcısı:*Lehigh-Phoenix Renk/Hagerstown

Altyazılar

Şekil 0.3: "Bir abaküs". ©Wayne Chandler. Şekil 0.4: "Mark I bilgisayarı." IBM kurumsal arşivl eri izniyle. İzin alınmadan kullanım yasaktır. Şekil 10.1: "3B grafikler kullanılarak üretilen bir vir al dünya fotoğrafı (Walt Disney/Pixar Animasyon Stüdyoları'ndan *Toy Story*) ©Disney/Pixar. Şek il 10.6: "Dreamworks SKG tarafından *Shrek 2* filminden bir sahne. ©Dreamworks/ Picture Desk Inc./Kobal koleksiyonu. Şekil 11.19: "Bir görüntüdeki pikselleri sınıflandırmak için bir sinir ağı kullanmanın sonuçları." www.actapress.com'dan ilham alınmıştır. Bölüm 11, Tarih Yazan Robotl ar özellik: a. "Bir futbol robotu, 15 Nisan 2010'da Magdeburg, doğu Almanya'da RoboCup Alman Açık 201 0'da bir topa vuruyor." ©Jens Schlueter/AFP/ Getty Images/ Newscom. **b.** "TartanRacing'in "Boss—DARPA tarafından sponsor olunan, araçların kendiliğinden hareket etmesini sağlayan bir yarışma olan Kentsel Mücadele'nin kazananı." ©DARPA. **c.** "NASA'nın bir gezgin robotu—Mars yüzeyini keşfeden bir robot jeolo g." NASA/JPL-Caltech izniyle.

Telif Hakkı© 2012, 2009, 2007, 2005, 2003 Pearson Education, Inc., Addison-Wesley olarak yayın yapmaktadır. Tüm hakları saklıdır. Amerika Birleşik Devletleri'nde üretilmiştir. Bu yayın, Telif Hakkı ile korunmaktadır ve yasaklı çoğaltma, bir geri alma sisteminde depolama veya h erhangi bir formda veya herhangi bir yöntemle, elektronik, mekanik, fotokopi, kayıt veya ben zeri ile iletim öncesinde yayıncıdan izin alınmalıdır. Bu çalışmadan materyal kullanmak için izi n almak üzere, lütfen yazılı bir talep gönderin: Pearson Education, Inc., İzinler Departmanı, 501 Boylston Caddesi, Suite 900, Boston, Massachusetts 02116.

Üreticiler ve satıcılar tarafından ürünlerini ayırt etmek için kullanılan birçok ad, ticari ma rka olarak talep edilmektedir. Bu kitapta bu adlar geçtiğinde ve yayıncı bir ticari marka talebi nden haberdar olduğunda, bu adlar büyük harfle veya tamamen büyük harfle basılmıştır.

Kongre Kütüphanesi Yayın Verileri talep üzerine mevcuttur.

10 9 8 7 6 5 4 3 2 1—EB—14 13 12 11 10

Addison-Wesley
is an imprint of

PEARSON

ISBN 10: 0-13-256903-5
ISBN 13: 978-0-13-256903-3

Doclingo https://www.doclingo.ai

önsöz

Bu kitap, bilgisayar biliminin tanıtıcı bir anketini sunmaktadır. Konunun geni şliğini keşfederken, ilgili konulara dürüst bir takdir kazandıracak kadar derinl ik de içermektedir.

Hedef Kitle

Bu metni bilgisayar bilimi öğrencileri ve diğer disiplinlerden öğrenciler için y azdım. Bilgisayar bilimi öğrencileri için, çoğu çalışmalara bilgisayar biliminin programlama, Web tarayıcısı kullanma ve İnternet dosya paylaşımı olduğu y anılsamasıyla başlar, çünkü gördükleri esasen budur. Ancak bilgisayar bilimi bundan çok daha fazlasıdır. Dolayısıyla, başlangıç düzeyindeki bilgisayar bili mi öğrencilerinin, planladıkları ana dalın genişliği ile tanışmaları gerekmekte dir. Bu tanışmayı sağlamak, bu kitabın temasını oluşturmaktadır. Öğrenciler e bilgisayar bilimi hakkında bir genel bakış sunar—bu, alanındaki gelecekteki derslerin önemini ve karşılıklı ilişkilerini takdir edebilecekleri bir temeldir. B u anket yaklaşımı, aslında doğal bilimlerdeki tanıtım dersleri için kullanılan modeldir.

Bu geniş arka plan, diğer disiplinlerden gelen öğrencilerin yaşadıkları tek nik topluma bağlanabilmeleri için de gereklidir. Bu kitleye yönelik bir bilgisay ar bilimi dersi, yalnızca İnternet kullanımı veya bazı popüler yazılım paketleri nin kullanımıyla ilgili bir tanıtım yerine, tüm alanın pratik ve gerçekçi bir anla yışını sağlamalıdır. Elbette eğitim için uygun bir yer vardır, ancak bu metin e ğitim vermekle ilgilidir.

Bu nedenle, bu metni yazarken, teknik olmayan öğrenciler için erişilebilirliği sağla mak önemli bir hedef olmuştur. Sonuç olarak, önceki baskılar, geniş bir disiplin ve eğiti m seviyesindeki öğrenciler için başarılı bir şekilde kullanılmıştır; bu, liseye kadar uzanan ve lisansüstü dersleri kapsayan bir yelpazeyi içermektedir. Bu on birinci baskı, bu gelen eği sürdürmek için tasarlanmıştır.

On Birinci Baskıda Yenilikler

Bu on birinci baskının geliştirilmesi sırasında temel tema, metni el tipi mobil cihazları, özellikle akıllı telefonları içerecek şekilde güncellemektir.
Bu nedenle, metnin değiştirildiğini ve zaman zaman genişletildiğini göreceksiniz,

Doclingo https://www.doclingo.ai

tartışılan konu ile akıllı telefon teknolojisi arasındaki ilişkiyi sunmak için. Belir li konular şunları içerir:

- Akıllı telefon donanımı
- 3G ve 4G ağları arasındaki fark
- Akıllı telefon işletim sistemleri
- Akıllı telefon yazılım geliştirme
- İnsan/akıllı telefon arayüzü

Bu eklemeler, Bölüm 3 (İşletim Sistemleri) ve4 (Ağ Oluşturma) içinde en belirgin şekilde görülmektedir, ancak Bölüm 6 (Programlama Dilleri)ve 7 (Yazılım Mühendisli ği) içinde de gözlemlenebilir.

Bu baskıya yapılan diğer önemli değişiklikler aşağıdaki konulara güncellemeleri içerm ektedir:

- Yazılım mülkiyeti ve sorumluluğu: Bu konuya ilişkin Bölüm 7 (Yazılım Mühendisliği) içindeki materyal yeniden yazılmış ve güncellenmiştir.
- Yapay sinir ağlarını eğitmek: Bu materyal, Bölüm 11 (Yapay Zeka) içinde modernleştirilmiştir.

Son olarak, metin boyunca materyalin güncellenmiş olduğunu göreceksi nizve bu, en çok Bölüm 0 (Giriş), Bölüm 1 (Veri Depolama) ve Bölüm 2 (Veri Manipülasyonu) içinde belirgindir.

Organizasyon

Bu metin, somuttan soyuta ilerleyen bir konu düzenini takip etmektedir; bu düzen, her konunun bir sonrakine yol açtığı sağlam bir pedagojik sunum son ucunu doğurur. Bilgi kodlaması, veri depolama ve bilgisayar mimarisinin te melleriyle başlar (Bölüm 1 ve 2); işletim sistemleri (Bölüm 3) ve bilgisayar ağl arı (Bölüm 4) üzerine çalışmalara ilerler; algoritmalar, programlama dilleri ve yazılım geliştirme konularını inceler (Bölüm 5'ten 7'ye kadar); bilginin erişile bilirliğini artırma tekniklerini keşfeder (Bölüm 8 ve 9); grafikler (Bölüm 10) ve yapay zeka (Bölüm 11) aracılığıyla bilgisayar teknolojisinin bazı önemli uygul amalarını değerlendirir; ve hesaplama soyut teorisine bir girişle (Bölüm 12) s ona erer.

Metin bu doğal ilerlemeyi takip etse de, bireysel bölümlerve kısımlar şaşı rtıcı bir şekilde bağımsızdır ve genellikle izolebirim olarak okunabilir veya alt ernatif çalışma dizileri oluşturmak için yeniden düzenlenebilir. Gerçekten de, kitapsıklıkla materyali çeşitli sıralarda kapsayan dersler için bir metin olarak kullanılmaktadır. Bu alternatiflerden biri, 5. ve 6. Bölüm (Algoritmalar vePro gramlama Dilleri) ile başlar ve istenildiği gibi daha önceki bölümlere geri dö ner. Aksine, 12. Bölüm'deki hesaplanabilirlik konusuyla başlayan bir dersi bili yorum. Hala diğer durumlarda metin, farklı alanlarda projelere dalmak için y alnızca bir omurga olarak hizmet eden “kıdemli capstone”derslerinde kullanıl mıştır. Daha az teknik odaklı kitleler için dersler, 4. Bölüm (Ağ ve İnternet), 9. Bölüm (Veritabanı Sistemleri),10. Bölüm (Bilgisayar Grafikleri) ve 11. Bölüm (Yapay Zeka) üzerinde yoğunlaşmak isteyebilir.

Her bölümün açılış sayfasında, bazı kısımları isteğe bağlı olarak işaretle mek için yıldızlar kullandım. Bu, daha spesifik ilgi alanlarını kapsayan veya

Doclingo https://www.doclingo.ai

belki de geleneksel konuları daha derinlemesine keşfeden bölümlerdir. Ama cım, metin boyunca alternatif yollar için yalnızca öneriler sunmaktır. Elbette, başkakısa yollar da vardır. Özellikle, hızlı bir okuma arıyorsanız, aşağıdaki sır ayı öneririm:

Bölüm	Konu
1.1–1.4	Veri kodlama ve depolama temelleri
2.1–2.3	Makine mimarisi ve makine dili
3.1–3.3	İşletim sistemleri
4.1–4.3	Ağlar ve İnternet
5.1–5.4	Algoritmalar ve algoritma tasarımı
6.1–6.4	Programlama dilleri
7.1–7.2	Yazılım mühendisliği
8.1–8.3	Veri soyutlamaları
9.1–9.2	Veritabanı sistemleri
10.1–10.2	Bilgisayar grafikleri
11.1–11.3	Yapay zeka
12.1–12.2	Hesaplama teorisi

Metin boyunca birkaç tema işlenmiştir. Bunlardan biri bilgisayar biliminin dinamik olduğudur. Metin, konuları tarihsel bir perspektiften sunmakta, güncel durumu tartışmakta ve araştırma yönlerini belirtmektedir.
Diğer bir tema ise soyutlamanın rolü ve soyut araçların karmaşıklığı kontrol etmek için nasıl kullanıldığıdır. Bu tema Bölüm 0'da tanıtılmakta ve sonrasında işletim sistemi mimarisi, ağ oluşturma, algoritma geliştirme, programlama dili tasarımı, yazılım mühendisliği, veri organizasyonu ve bilgisayar grafikleri bağlamında yankılanmaktadır.

Öğretmenlere

Bu metinde genellikle bir dönem içinde işlenebilecek olandan daha fazla materyal bulunmaktadır, bu nedenle kurs hedeflerinize uymayan konuları atlamaktan veya sıralamayı istediğiniz gibi değiştirmekten çekinmeyin. Metnin bir kurguyu takip etmesine rağmen, konuların büyük ölçüde bağımsız bir şekilde ele alındığını göreceksiniz; bu da istediğiniz gibi seçip almanıza olanak tanır. Kitap, bir kurs kaynağı olarak kullanılmak üzere - bir kurs tanımı olarak değil - tasarlanmıştır. Öğrencileri, kursunuza açıkça dahil edilmeyen materyalleri okumaya teşvik etmenizi öneririm. Her şeyi derste açıklamak zorunda olduğumuzu varsayarsak, öğrencileri küçümsüyoruz. Onların kendi başlarına öğrenmeyi öğrenmelerine yardımcı olmalıyız.

Metnin aşağıdan yukarı, somuttan soyuta organizasyonu hakkında birkaç k elime söylemek zorundayım. Akademisyenler olarak, öğrencilerin bir konudaki bakış açımızı takdir edeceğini çok sık varsayıyoruz - genellikle bu, bir alanda yıll arca çalışarak geliştirdiğimiz bir bakış açısıdır. Öğretmenler olarak, materyali ö ğrencinin perspektifinden sunarak daha iyi bir iş çıkardığımızı düşünüyorum. B u nedenle metin, veri temsili/depolama, makine mimarisi, işletim sistemleri ve ağ bağlantıları ile başlıyor. Bu, öğrencilerin kolayca ilişki kurabileceği konulardı r - muhtemelen JPEG ve MP3 gibi terimleri duymuşlardır; muhtemelen verileri CD'lere ve DVD'lere kaydetmişlerdir; bilgisayar bileşenleri satın almışlardır; bir i şletim sistemi ile etkileşimde bulunmuşlardır; ve İnterneti kullanmışlardır. Kurs a bu konularla başlamak, öğrencilerin yıllardır taşıdıkları birçok "neden" sorusu nun yanıtlarını keşfetmelerini sağlar ve dersi pratik olarak görmeyi öğrenirler.

Doclingo https://www.doclingo.ai

teorik değil. Bu başlangıçtan, algoritmalar, algoritmik yapılar, programlama dill eri, yazılım geliştirme metodolojileri, hesaplanabilirlik ve karmaşıklık gibi daha soyut konulara geçmek doğaldır; bu konular, alandaki bizler tarafından bilimin ana konuları olarak görülmektedir. Daha önce de söylediğim gibi, konular, bu a şağıdan yukarı sıralamayı takip etmenizi zorlamayacak bir şekilde sunulmaktad ır, ancak bunu denemenizi teşvik ediyorum.

Hepimiz öğrencilerin doğrudan öğrettiklerimizden çok daha fazlasını öğrendi ğinin farkındayız veöğrendikleri dersler genellikle açıkça çalışılanlardan daha iyi ö zümlenir. Bu, "öğretme"problem çözme konusunda önemlidir. Öğrenciler, proble m çözme metodolojilerini çalışarakproblem çözücü olmazlar. Problemleri çözerek problem çözücü olurlar—ve sadece dikkatliceformüle edilmiş "ders kitabı problem leri" ile değil. Bu metin birçok problem içermektedir,bunlardan bazıları kasıtlı olar ak belirsizdir—yani mutlaka tek bir doğru yaklaşım veya tek birdoğru cevap yoktu r. Bunları kullanmanızı ve genişletmenizi teşvik ediyorum.

"İçsel öğrenme" kategorisindeki bir diğer konu ise profesyonellik,etik ve sosyal sorumluluktur. Bu materyalin, dersin üzerine eklenmiş izole bir konu olaraksunulması gerektiğine inanmıyorum. Bunun yerine, ilgili olduğunda or taya çıkan kapsamınbütünleyici bir parçası olmalıdır. Bu metinde izlenen yakl aşımdır. Bölüm 3.5, 4.5, 7.8, 9.7,ve 11.7'nin güvenlik, gizlilik, sorumluluk ve s osyal farkındalık gibi konuları işletim sistemleri,\ ağ,database sistemleri, yazı lım mühendisliği ve yapay zeka bağlamında sunduğunu göreceksiniz.Ayrıca, Bölüm 0.6, etik karar verme süreçlerini felsefi olarak sağlam bir temele oturt mayaçalışan daha belirgin teorilerden bazılarını özetleyerek bu temayı tanıt maktadır. Her bölümün,öğrencileri metindeki materyal ile yaşadıkları toplum arasındaki ilişkiyi düşünmeye zorlayan*Sosyal Sorunlar* adında bir soru kolek siyonu içerdiğini de göreceksiniz.

Metnimin kursunuz için dikkate alındığı için teşekkür ederim. Durumunuza uy gun olup olmadığına karar verseniz de vermeseniz de, bunun bilgisayar bilimi eği timi literatürüne bir katkı sağladığını umuyorum.

Eğitimsel Özellikler

Bu metin, yıllarca süren öğretim deneyiminin bir ürünüdür. Sonuç olarak, pedagojik yar dımcılar açısından zengindir. Öğrencinin katılımını artırmak için çok sayıda problem bul unmaktadır—bu on birinci baskıda 1,000'den fazla. Bu problemler, Sorular/Alıştırmalar, Bölüm Gözden Geçirme Problemleri ve Sosyal Sorunlar olarak sınıflandırılmıştır. Sorular /Alıştırmalar, her bölümün sonunda (giriş bölümünün dışında) yer almaktadır.

Bu sorular, yeni tartışılan materyali gözden geçirir, önceki tartışmayı genişletir veya daha sonr a ele alınacak ilgili konulara ipucu verir. Bu soruların cevapları Ek F'de bulunmaktadır.

Bölüm Gözden Geçirme Problemleri, her bölümün sonunda (giriş bölümünün dışında) yer almaktadır. Bu problemler, tüm bölümün materyalini kapsadığı için " ödev" problemleri olarak tasarlanmıştır ve metinde cevapları verilmemiştir.

Ayrıca, her bölümün sonunda Sosyal Sorunlar kategorisindeki sorular bulunmak tadır. Bu sorular düşünme ve tartışma için tasarlanmıştır. Birçoğu, kısa yazılı veya sö zlü raporlarla sonuçlanan araştırma ödevlerini başlatmak için kullanılabilir.

Her bölüm ayrıca, bölümün konusuyla ilgili diğer materyallere atıflar içer en Ek Okuma adlı bir listeyle sona erer. Bu önsözde, metinde ve metnin yan kenarlarında belirtilen web siteleri, ilgili materyalleri bulmak için iyi yerlerdir.

Doclingo https://www.doclingo.ai

Ek Kaynaklar

Bu metin için çeşitli ek materyaller, kitabın Arkadaş Web Sitesi'nde mevcuttu r: www.pearsonhighered.com/brookshear. Aşağıdakiler tüm okuyuculara açı ktır:

- Metindeki konuları genişleten ve ilgili konuları keşfetme fırsatları suna n bölüm bölüm etkinlikleri
- Okuyucuların metinde ele alınan materyali yeniden düşünmelerine yardımcı olan bölüm b ölüm "kendine testler"
- Metinle uyumlu bir pedagojik sırayla Java ve C++'ın temellerini öğrete n kılavuzlar

Ayrıca, aşağıdaki ekler Pearson Education'ın Eğitmen Kaynak Merkezi'nd eki nitelikli eğitmenler için mevcuttur. Lütfen www.pearsonhighered.com ad resini ziyaret edin veya bu ekleri nasıl erişeceğiniz hakkında bilgi için Pearso n satış temsilcinizle iletişime geçin:

- Eğitmen Kılavuzu, Bölüm Gözden Geçirme Problemlerinin cevaplarıyla birlikte
- PowerPoint ders slaytları
- Test bankası

Ayrıca, www.mscs.mu.edu/~glennb adresindeki kişisel web sitemi kontrol e tmek isteyebilirsiniz. Çok resmi değil (ve benim kaprislerime ve mizah anlayışı ma tabidir), ancak orada size yardımcı olabilecek bazı bilgileri tutmaya çalışıyor um. Özellikle, bana bildirilen metindeki hataların düzeltmelerini listeleyen bir h ata sayfası bulacaksınız.

Öğrencilere

Ben biraz uyumsuz biriyim (bazı arkadaşlarım bunun *daha* da ötesinde olduğunu söyler) bu yüzdenbu metni yazmaya başladığımda aldığım tavsiyeleri her zaman t akip etmedim. Özellikle,birçok kişi belirli materyallerin başlangıç düzeyindeki öğre nciler için çok ileri olduğunusavundu. Ama, bir konunun ilgili olduğunu düşünüyo rsam, akademik topluluk onu "ilerikonu" olarak değerlendirse bile, o konu yine de ilgilidir. Bilgisayar bilimininkapsamlı bir resmini sunan bir metni hak ediyorsunuz —sadece giriş düzeyindekiöğrenciler için uygun görülen konuların yapay olarak b asitleştirilmiş sunumlarınıiçeren sulandırılmış bir versiyon değil. Bu nedenle, konu lardan kaçınmadım.Bunun yerine daha iyi açıklamalar aradım. Size bilgisayar bili minin ne hakkındaolduğuna dair dürüst bir resim vermek için yeterli derinliği sağl amaya çalıştım.Bir tarifteki baharatlar gibi, aşağıdaki sayfalardaki bazı konuları atl ayabilirsiniz,bunlar orada, eğer isterseniz tatmanız için—ve bunu yapmanızı teşvik ediyorum.

Ayrıca, teknoloji ile ilgili herhangi bir derste, bugün öğrendiğiniz detayların yarın bilmeniz gereken detaylar olmayabileceğini belirtmeliyim. Alan dinamik—bu, heyecanın bir parçası. Bu kitap size konunun güncel bir resmini ve tarihi bir perspektif sunacak. Bu arka plan ile teknoloji ile birlikte büyümeye hazır olacaksınız. Şimdi bu metnin ötesine geçerek büyüme sürecine başlamanızı teşvik ediyorum. Öğrenmeyi öğrenin.

Kitabımı okumayı seçerek bana duyduğunuz güven için teşekkür ederim. Bir yazar olarak, zamanınıza değer bir el yazması üretme yükümlülüğüm var. Bu yükümlülüğe uygun yaşadığımı umuyorum.

Doclingo https://www.doclingo.ai

Teşekkürler

Bu kitabı önceki baskılarda okuyarak ve kullanarak destekleyen herkese teşekkür ediyorum. Onur duyuyorum.

David T. Smith (Pennsylvania Üniversitesi) ve Dennis Brylow (Marquette Üniversitesi) bu on birinci baskının üretiminde önemli roller oynadılar. David Bölüm 0, 1, 2, 7 ve 11 üzerinde yoğunlaştı; Dennis ise Bölüm 3, 4, 6 ve 10 üzerinde çalıştı. Onların sıkı çalışmaları olmadan bu yeni baskı bugün var olmazdı. Kendilerine içtenlikle teşekkür ediyorum.

Onuncu baskının önsözünde belirtildiği gibi, bu baskının geliştirilmesinde Ed Angel, John Carpinelli, Chris Fox, Jim Kurose, Gary Nutt, Greg Riccardi ve Patrick Henry Winston'a minnettarım. Onların çabalarının sonuçları bu on birinci baskıda görünmeye devam ediyor.

Bu veya önceki baskılara katkıda bulunan diğerleri arasında J. M. Adams, C. M. Allen, D. C. S. Allison, R. Ashmore, B. Auernheimer, P. Bankston, M. Barnard, P. Bender, K. Bowyer, P. W. Brashear, C. M. Brown, H. M Brown, B. Calloni, M. Clancy, R. T. Close, D. H. Cooley, L. D. Cornell, M. J. Crowley, F. Deek, M. Dickerson, M. J. Duncan, S. Ezekiel, S. Fox, N. E. Gibbs, J. D. Harris, D. Hascom, L. Heath, P. B. Henderson, L. Hunt, M. Hutchenreuther, L. A. Jehn, K. K. Kolberg, K. Korb, G. Krenz, J. Liu, T. J. Long, C. May, J. J. McConnell, W. McCown, S. J. Merrill, K. Messersmith, J. C. Moyer, M. Murphy, J. P. Myers, Jr., D. S. Noonan, W. W. Oblitey, S. Olariu, G. Rice, N. Rickert, C. Riedesel, J. B. Rogers, G. Saito, W. Savitch, R. Schlafly, J. C. Schlimmer, S. Sells, G. Sheppard, Z. Shen, J. C. Simms, M. C. Slattery, J. Slimick, J. A. Slomka, D. Smith, J. Solderitsch, R. Steigerwald, L. Steinberg, C. A. Struble, C. L. Struble, W. J. Taffe, J. Talburt, P. Tonellato, P. Tromovitch, E. D. Winter, E. Wright, M. Ziegler ve bir anonim. Bubireylere içten teşekkürlerimi sunuyorum.

Daha önce belirtildiği gibi, metnin Companion Web Sitesi'nde bu dillerin temellerini öğreten Java ve C++ kılavuzları bulacaksınız. Bunlar Diane Christie tarafından yazılmıştır. Teşekkürler Diane.

Bir diğer teşekkür, Companion Web Sitesi'nde bulacağınız bölüm bölüm etkinliklerin arkasındaki yaratıcı güç olan Roger Eastman'a gidiyor.

Ayrıca bu projeye katkıda bulunan Addison-Wesley'deki insanlara da teşekkür ediyorum. Onlarla çalışmak harika bir deneyim ve aynı zamanda iyi arkadaşlar. Eğer bir ders kitabı yazmayı düşünüyorsanız, Addison-Wesley ile yayımlamayı düşünmelisiniz.

Yıllar boyunca büyük bir cesaret kaynağı olan eşim Earlene ve kızım Cheryl'e minnettar olmaya devam ediyorum. Cheryl, elbette, büyüdü ve birkaç yıl önce evden ayrıldı. Ama Earlene hâlâ burada. Şanslı bir adamım. 11 Aralık 1998 sabahı, kalp krizi geçirdim ama o beni zamanında hastaneye götürdü. (Genç nesilden olanlar için açıklamam gerekirse, kalp krizini atlatmak, ödev teslim tarihinin uzatılması gibi bir şey.)

Son olarak, bu kitabın adandığı ebeveynlerime teşekkür ediyorum. Anonim kalacak olan aşağıdaki onayla kapatıyorum: “Oğlumun kitabı gerçekten çok iyi. Herkes okumalı.”

J. G. B.

Doclingo https://www.doclingo.ai

içindekiler

Bölüm 0Giriş1

0.1 Algoritmaların Rolü 2
0.2 Hesaplamanın Tarihi 4
0.3 Algoritmaların Bilimi 10
0.4 Soyutlama 11
0.5 Çalışmamızın Taslağı 12
0.6 Sosyal Sonuçlar 13

Bölüm 1Veri Depolama19

1.1 Bitler ve Depolama 20
1.2 Ana Bellek 26
1.3 Kütle Depolama 29
1.4 Bilgiyi Bit Desenleri Olarak Temsil Etme 35
*1.5 İkili Sistem 42
*1.6 Tam Sayıları Depolama 47
*1.7 Kesirleri Depolama 53
*1.8 Veri Sıkıştırma 58
*1.9 İletişim Hataları 63

Bölüm 2Veri Manipülasyonu73

2.1 Bilgisayar Mimarisi 74
2.2 Makine Dili 77
2.3 Programın Çalıştırılması 83
*2.4 Aritmetik/ Mantık Talimatları 90
*2.5 Diğer Cihazlarla İletişim 94
*2.6 Diğer Mimariler 100

**Yıldızlar isteğe bağlı bölümler için önerileri belirtir.*

Doclingo https://www.doclingo.ai

Bölüm 3İşletim Sistemleri109

3.1 İşletim Sistemlerinin Tarihi 110
3.2 İşletim Sistemi Mimarisi 114
3.3 Makinenin Faaliyetlerini Koordine Etme 122
*3.4 Süreçler Arasındaki Rekabeti Yönetme 125
3.5 Güvenlik 130

Bölüm 4Ağ ve İnternet139

4.1 Ağ Temelleri 140
4.2 İnternet 149
4.3 Dünya Çapında Ağ 158
*4.4 İnternet Protokolleri 167
4.5 Güvenlik 173

Bölüm 5Algoritmalar187

5.1 Algoritma Kavramı 188
5.2 Algoritma Temsili 191
5.3 Algoritma Keşfi 198
5.4 Yinelemeli Yapılar 204
5.5 Rekürsif Yapılar 214
5.6 Verimlilik ve Doğruluk 222

Bölüm 6Programlama Dilleri239

6.1 Tarihsel Perspektif 240
6.2 Geleneksel Programlama Kavramları 248
6.3 Prosedürel Birimler 260
6.4 Dil Uygulaması 268
6.5 Nesne Yönelimli Programlama 276
*6.6 Eşzamanlı Faaliyetleri Programlama 283
*6.7 Bildirimsel Programlama 286

Bölüm 7Yazılım Mühendisliği299

7.1 Yazılım Mühendisliği Disiplini 300
7.2 Yazılım Yaşam Döngüsü 302
7.3 Yazılım Mühendisliği Yöntemleri 306
7.4 Modülerlik 308
7.5 Ticaret Araçları 316
7.6 Kalite Güvencesi 324
7.7 Dokümantasyon 328
7.8 İnsan-Makine Arayüzü 329
7.9 Yazılım Mülkü ve Sorumluluğu 332

Doclingo https://www.doclingo.ai

Bölüm 8Veri Soyutlamaları341

8.1 Temel Veri Yapıları 342
8.2 İlgili Kavramlar 345
8.3 Veri Yapılarını Uygulama 348
8.4 Kısa Bir Vaka Çalışması 362
8.5 Özelleştirilmiş Veri Türleri 367
*8.6 Sınıflar ve Nesneler 371
*8.7 Makine Dilinde Göstergeler 372

Bölüm 9Veritabanı Sistemleri383

9.1 Veritabanı Temelleri 384
9.2 İlişkisel Model 389
*9.3 Nesne Yönelimli Veritabanları 400
*9.4 Veritabanı Bütünlüğünü Koruma 402
*9.5 Geleneksel Dosya Yapıları 406
9.6 Veri Madenciliği 414
9.7 Veritabanı Teknolojisinin Sosyal Etkisi 416

Bölüm 10Bilgisayar Grafikleri425

10.1 Bilgisayar Grafikleri Kapsamı 426
10.2 3B Grafiklere Genel Bakış 428
10.3 Modelleme 430
10.4 Görselleştirme 439
*10.5 Küresel Aydınlatma ile İlgilenme 449
10.6 Animasyon 452

Bölüm 11Yapay Zeka461

11.1 Zeka ve Makineler 462
11.2 Algı 467
11.3 Akıl Yürütme 473
11.4 Ek Araştırma Alanları 484
11.5 Yapay Sinir Ağları 489
11.6 Robotik 497
11.7 Sonuçları Düşünmek 500

Bölüm 12Hesaplama Teorisi509

12.1 Fonksiyonlar ve Hesaplamaları 510
12.2 Turing Makineleri 512
12.3 Evrensel Programlama Dilleri 516
12.4 Hesaplanamaz Bir Fonksiyon 522
12.5 Problemlerin Karmaşıklığı 527
*12.6 Açık Anahtar Kriptografisi 536

Doclingo https://www.doclingo.ai

Ekler545

ASCII 547
B İki'nin Tamamlayıcısını Manipüle Etmek İçin Devreler Temsilleri 548
C Basit Bir Makine Dili 551
D Yüksek Düzey Programlama Dilleri 553
E İteratif ve Rekürsif Yapıların Eşdeğerliliği 555
F Sorular ve Alıştırmalar için Cevaplar 557

İndeks597

Doclingo https://www.doclingo.ai

BÖLÜM

Giriş

0

Bu ön bölümde bilgisayarbiliminin kapsamını ele alıyoruz, ta rihsel bir perspektif geliştiriyoruz ve çalışmamıza başla mak için bir temel oluşturuyoruz.

0.1 Algoritmaların Rolü

0.2 Tarih Hesaplama

0.3 Bilim Algoritmaların

0.4 Soyutlama

0.5 Bir Taslak Çalışmamız

0.6 Toplumsal Sonuçlar

Doclingo https://www.doclingo.ai

Bilgisayar bilimi, bilgisayar tasarımı, bilgisayar programlama, bilgi işleme, so runların algoritmik çözümleri ve algoritmik süreç gibi konular için bilimsel bi r temel oluşturmayı amaçlayan disiplindir. Günümüz bilgisayar uygulamaları nın temellerini sağladığı gibi, yarının bilişim altyapısının da temellerini oluştu rur.

Bu kitap, bu bilime kapsamlı bir giriş sunmaktadır. Bir üniversite bilgisay ar bilimi müfredatını oluşturan konuların çoğu da dahil olmak üzere geniş bi r konu yelpazesini araştıracağız. Alanın tam kapsamını ve dinamiklerini takdi r etmek istiyoruz. Bu nedenle, konuların kendilerine ek olarak, tarihsel gelişi mlerine, mevcut araştırma durumuna ve gelecekteki beklentilere de ilgi duy acağız. Amacımız, bilgisayar bilimi hakkında işlevsel bir anlayış geliştirmektir; bu, bilimi daha özel çalışmalar yapmak isteyenleri destekleyecek ve diğer al anlardaki kişilerin giderek daha teknik bir toplumda başarılı olmalarını sağla yacaktır.

0.1 Algoritmaların Rolü

Bilgisayar biliminin en temel kavramı olan algoritma ile başlıyoruz. Gayri resmi ol arak, bir algoritma, bir görevin nasıl gerçekleştirileceğini tanımlayan bir dizi adım dır. (Bunu daha sonra Bölüm 5'te daha kesin bir şekilde ifade edeceğiz.) Örneğin, yemek pişirmek için (tarifler olarak adlandırılır), garip bir şehirde yol bulmak için (daha yaygın olarak yönler olarak adlandırılır), çamaşır makinelerini çalıştırmak içi n (genellikle makinenin kapağının iç kısmında veya belki bir çamaşırhanenin duva rında gösterilir), müzik çalmak için (nota şeklinde ifade edilir) ve sihir numaraları y apmak için (Şekil 0.1).

Bir bilgisayar gibi bir makinenin bir görevi yerine getirebilmesi için, o gör evi yerine getiren bir algoritmanın keşfedilmesi ve makine ile uyumlu bir biçi mde temsil edilmesi gerekir. Bir algoritmanın temsiline program denir. İnsanların rahatlığı için, bilgisayar programları genellikle kağıda basılır veya bilgisayar ekranlarında görüntülenir. Makinelerin rahatlığı için, programlar makinenin teknolojisi ile uyumlu bir şekilde kodlanır.
Bir program geliştirme, onu makine ile uyumlu bir biçimde kodlama ve bir mak ineye yerleştirme sürecine programlama denir. Programlar ve temsil ettikleri al goritmalar, makinenin kendisi ile karşılaştırıldığında yazılım olarak adlandırılır; makinenin kendisi ise donanım olarak bilinir.

Algoritmaların incelenmesi, matematikte bir konu olarak başladı. Gerçekten de, algoritmaların aranması, günümüz bilgisayarlarının gelişiminden çok önce matemati kçilerin önemli bir etkinliğiydi. Amaç, belirli bir türdeki tüm problemleri çözmek için nasıl bir dizi talimatın izlenmesi gerektiğini tanımlayan tek bir talimat seti bulmaktı. Bu erken araştırmanın en iyi bilinen örneklerinden biri, iki çok basamaklı sayının böl ümünü bulmak için kullanılan uzun bölme algoritmasıdır. Diğer bir örnek ise, antik Y unan matematikçisi Öklid tarafından keşfedilen, iki pozitif tam sayının en büyük orta k bölenini bulmak için kullanılan Öklid algoritmasıdır (Şekil 0.2).

Bir görevi yerine getirmek için bir algoritma bulunduğunda, o görevin ye rine getirilmesi artık algoritmanın dayandığı ilkelerin anlaşılmasını gerektirm ez. Bunun yerine, görevin yerine getirilmesi, yalnızca talimatları izleme sürec ine indirgenir. (Bir bölüm bulmak için uzun bölme algoritmasını veya en büy ük ortak böleni bulmak için Öklid algoritmasını izleyebiliriz, algoritmanın ned en çalıştığını anlamadan.) Bir anlamda, mevcut problemi çözmek için gereke n zeka algoritmada kodlanmıştır.

Doclingo https://www.doclingo.ai

Şekil **0.1** Bir sihir numarası için bir algoritma

Etki: Performans, normal bir deste iskambil kartından bazı kartları yüzü aşağı gelecek şekilde bir masanın üzerine koyar ve bunları masanın üzerinde yayarak iyice karıştırır. Daha sonra, izleyiciler ya kırmızı ya da siyah kartlar talep ettiğinde, performans istenen renkl erdeki kartları çevirir.

Sır ve Sohbet:

Adım 1. Normal bir deste karttan on kırmızı kart ve on siyah kart seçin. Bu kartları renge göre masanın üze rinde iki deste halinde yüzü yukarı gelecek şekilde dağıtın.

Adım 2. Biraz kırmızı kart ve biraz siyah kart seçtiğinizi duyurun.

Adım 3. Kırmızı kartları alın. Küçük bir deste oluşturma bahanesiyle, bunları yüzü aşağı gelecek şekilde sol elinizde tutun ve sağ elinizin baş parmağı ve işaret parmağı ile desteğin her iki ucuna geri çekerek her kartın hafifçe geri kıvrılmasını sağlayın. Ardından, "İşte bu d estedeki kırmızı kartlar." diyerek kırmızı kartları yüzü aşağı gelecek şekilde masanın üz erine koyun.

Adım 4. Siyah kartları alın. Adım 3'teki gibi, bu kartlara hafif bir ileri kıvrım verin. Ardından, "V e işte bu destedeki siyah kartlar." diyerek bu kartları yüzü aşağı gelecek şekilde mas anın üzerine geri koyun.

Adım 5. Siyah kartları masanın üzerine geri koyduktan hemen sonra, her iki elinizi kullanarak kır mızı v e siyah kartları (hala yüzü aşağı) karıştırın ve masanın üzerinde yayarak dağıtın. Ka rtları iyice karıştırdığınızı açıklayın.

Adım 6. Masada aşağıya bakar durumda kartlar olduğu sürece, aşağı daki adımları tekrar edin:

6.1. İzleyiciden ya kırmızı ya da siyah kart talep etmesini isteyin.

6.2. Eğer talep edilen renk kırmızıysa ve aşağıya bakar durumda bir kart varsa , böyle bir kartı çevirin ve "İşte bir kırmızı kart." deyin.

6.3. Eğer talep edilen renk siyahsa ve aşağıya bakar durumda bir kart varsa, b öyle bir kartı çevirin ve "İşte bir siyah kart." deyin.

6.4. Aksi takdirde, talep edilen renkten başka kart kalmadığını belirtin ve iddianızı kanıt lamak için kalan kartları çevirin.

Şekil **0.2**İki pozitif tam sayının en büyük ortak bölenini bulmak için Öklid alg oritması

Açıklama: Bu algoritma, girdisinin iki pozitif tam sayıdan oluştuğunu varsayar ve bu iki değerin en büyük ortak bölenini hesaplamaya devam eder.

Prosedür:

Adım 1. M ve N'ye, sırasıyla iki girdi değerinin daha büyük ve daha küçük olanını atayın.

Adım 2. M'yi N'ye bölün ve kalanı R olarak adlandırın.

Adım 3. Eğer R 0 değilse, M'ye N'nin değerini atayın, N'ye R'nin değerini atayın ve adım 2'ye geri dönün; aksi takdirde, en büyük ortak bölen şu anda N'ye atanan değerdir.

Doclingo https://www.doclingo.ai

Bu yetenek sayesinde, algoritmalar aracılığıyla zeka (veya en azından zek i davranış) yakalayıp iletebildiğimiz için, faydalı görevleri yerine getiren maki neler inşa edebiliyoruz. Sonuç olarak, makinelerin sergilediği zeka seviyesi, a lgoritmalar aracılığıyla iletilebilen zeka ile sınırlıdır.

Bir görevi yerine getirmek için bir makine inşa edebiliriz yalnızca o görevi yerine geti ren bir algoritma mevcutsa. Dolayısıyla, bir problemi çözmek için hiçbir algoritma yo ksa, o problemin çözümü makinelerin yeteneklerinin ötesindedir.

Algoritmik yeteneklerin sınırlamalarını tanımlamak, 1930'larda Kurt Göd el'in eksiklik teoreminin yayımlanmasıyla matematikte bir konu olarak pekişt i. Bu teorem, esasen, geleneksel aritmetik sistemimizi kapsayan herhangi bir matematiksel teoride, algoritmik yollarla doğruluğu veya yanlışlığı belirlene meyen ifadelerin bulunduğunu belirtir. Kısacası, aritmetik sistemimizin tam bir çalışması, algoritmik faaliyetlerin yeteneklerinin ötesindedir.

Bu farkındalık, matematiğin temellerini sarstı ve ardından gelen algoritm ik yeteneklerin incelenmesi, günümüzde bilgisayar bilimi olarak bilinen alanı n başlangıcıydı. Gerçekten de, algoritmaların incelenmesi bilgisayar biliminin özünü oluşturur.

0.2 Bilgisayar Tarihi

Bugünün bilgisayarları geniş bir soy ağacına sahiptir. Daha önceki bilgisayarcihazl arından biri abaküstür. Tarih, bunun muhtemelen antikÇin'de köklerinin olduğun u ve erken Yunan ve Roma medeniyetlerinde kullanıldığınısöyler. Makine oldukça basittir; çubuklar üzerine dizilmiş boncuklardan oluşurve bu çubuklar dikdörtgen bir çerçeveye monte edilmiştir (Şekil 0.3). Boncuklarçubuklar üzerinde ileri geri ha reket ettikçe, konumları saklanan değerleritemsil eder. Bu “bilgisayar”ın veri tems il ettiği ve depoladığı yer, boncuklarınkonumlarıdır. Bir algoritmanın yürütülmesin in kontrolü için makine, insanoperatöre dayanır. Böylece abaküs tek başına yalnız ca bir veri depolamasistemidir; tam bir hesaplama makinesi oluşturmak için bir in sanla birleştirilmelidir.

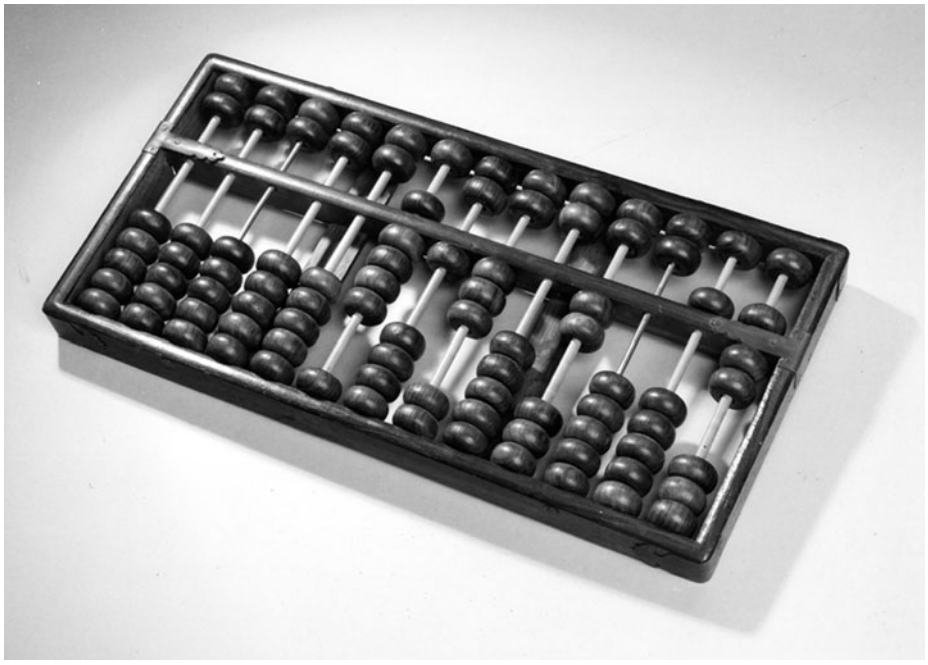
Orta Çağ'dan sonraki ve Modern Çağ'dan önceki dönemde daha sofistikebi lgisayar makineleri arayışı başlamıştır. Birkaç mucit dişli teknolojisiyledenemele r yapmaya başladı. Bunlar arasında Fransa'dan Blaise Pascal(1623–1662), Alma nya'dan Gottfried Wilhelm Leibniz (1646–1716) veİngiltere'den Charles Babbag e (1792–1871) bulunmaktadır. Bu makineler,dişli konumlandırması yoluyla veril eri temsil ediyordu; veriler, başlangıçdişli konumları belirlenerek mekanik olara k giriliyordu. Pascal ve Leibniz'inmakinelerinden çıkan sonuçlar, son dişli konu mlarını gözlemleyerek eldeediliyordu. Babbage ise, hesaplamaların sonuçlarını kağıda yazdıracakmakineleri hayal ediyordu, böylece transkripsiyon hatası olas ılığıeliminin mümkün olacağını düşünüyordu.

Bir algoritmayı takip etme yeteneği açısından, bu makinelerde bir esneklik ilerlemesi görüyoruz. Pascal'ın makinesi yalnızca toplama yapmak için inşa edilmiştir.

Sonuç olarak, uygun adım dizisi makinenin yapısına gömülmüştü.Benzer bir şe kilde, Leibniz'in makinesi algoritmalarını mimarisine sıkı bir şekilde gömmüştü, bununla birlikte operatörün seçebileceği çeşitli aritmetik işlemler sunuyordu. B abbage'ın Fark Motoru (sadece bir gösterim modeli inşa edilmiştir)çeşitli hesap lamalar yapmak için değiştirilebilirken, Analitik Motoru (inşası için hiç fon alma dı)kağıt kartlarda delikler şeklinde talimatları okumak için tasarlanmıştı.

Doclingo https://www.doclingo.ai

Şekil **0.3**Bir abaküs (Wayne Chandler tarafından fotoğraflanmıştır)



Böylece Babbage'ın Analitik Motoru programlanabilir hale gelmişti. Aslında, Augusta Ada Byron (Ada Lovelace), Babbage'ın Analitik Motoru'nun çeşitli he saplamalar yapmak için nasıl programlanabileceğini gösteren bir makale yay ınlamış,bugün sıklıkla dünyanın ilk programcısı olarak tanımlanmaktadır.

Kağıt üzerindeki delikler aracılığıyla bir algoritmayı iletme fikri Babbage t arafından ortaya atılmamıştır.Bu fikri, 1801'de dokuma sürecinde gerçekleşti rilecek adımların büyük kalın kartlardaki delik desenleriyle belirlendiği bir do kuma tezgahı geliştiren Joseph Jacquard'dan (1752–1834) almıştır.Bu şekilde, tezgahın izlediği algoritma, farklı dokuma tasarımları üretmek için kolayca d eğiştirilebiliyordu. Jacquard'ın fikrinden yararlanan bir diğer kişi isebilgiyi ka ğıt kartlarda delikler olarak temsil etme konseptini 1890 ABD nüfus sayımınd a tablo oluşturma sürecini hızlandırmak için uygulayan Herman Hollerith'tir.(Hollerith'in bu çalışması, IBM'in kuruluşuna yol açmıştır.) Bu tür kartlar nihay etinde delikli kartlar olarak bilinir hale gelmiş ve 1970'lerin sonlarına kadar b ilgisayarlarla iletişim kurmanın popüler bir yolu olarak kalmıştır.

Gerçekten de, bu teknik günümüzde de yaşamaya devam ediyor; 2000 ABD başkanlık seçimlerinde ortaya çıkan oy verme sorunları bunun bir kanıtıdır.

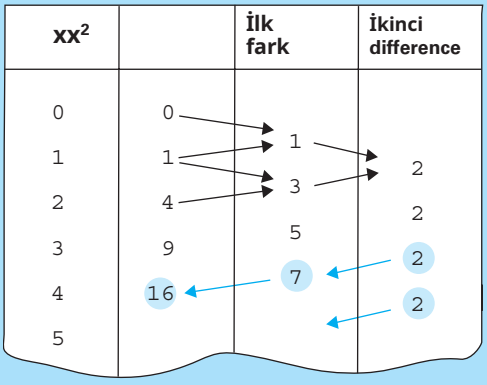
O dönemin teknolojisi, Pascal, Leibniz ve Babbage'ın karmaşık dişli meka nizmalarını mali açıdan uygulanabilir bir şekilde üretemiyordu. Ancak, 1900'l erin başındaki elektronik alanındaki ilerlemelerle bu engel aşıldı.

Bu ilerlemenin örnekleri arasında George Stibitz'in 1940'ta Bell Laboratuvarları'nda t amamladığı elektromekanik makine ve 1944'te tamamlanan Mark I bulunmaktadır.

Doclingo https://www.doclingo.ai

Babbage'ın Fark Motoru

Charles Babbage tarafından tasarlanan makineler, modern bilgisayar tasarım ının gerçekten de öncüleriydi. Eğer teknoloji, onun makinelerini ekonomik ola rak uygulanabilir bir şekilde üretebilseydi ve ticaret ile hükümetin veri işleme talepleri, bugünkü gereksinimlerin ölçeğinde olsaydı, Babbage'ın fikirleri 180 0'lerde bir bilgisayar devrimine yol açabilirdi. Ancak, onun yaşamında yalnızca Fark Motoru'nun bir gösterim modeli inşa edildi. Bu makine, "ardışık farklar" hesaplayarak sayısal değerleri belirliyordu. Bu tekniğe, tam sayıların karelerin i hesaplama problemini düşünerek bir bakış açısı kazanabiliriz.
0'ın karesinin 0, 1'in karesinin 1, 2'nin karesinin 4 ve 3'ün karesinin 9 olduğ unu bilerek başlıyoruz. Bununla, 4'ün karesini aşağıdaki şekilde belirleyebili riz (aşağıdaki diyagrama bakın). Öncelikle, zaten bildiğimiz karelerin farkları nı hesaplıyoruz: 1 2 0 2 1, 2 2 1 2 3 ve 3 2 2 2 5.
Sonra bu sonuçların farklarını hesaplıyoruz: 3 − 1 = 2, ve 5 − 3 = 2. Not bu farkların her ikisinin de 2 olduğunu. Bu tutarlılığın devam ettiğini varsayarsak (ma tematik bunun böyle olduğunu gösterebilir) değerler arasındaki farkın (4 2 3 2) ve (3 2 2 2) de 2 olması gerektiğini sonucuna varıyoruz. Dolayısıyla (4 2 3 2) (3 2 2 2)'den 2 daha büyük olmalıdır, bu nedenle 4 2 3 2 7 ve böylece 4 2 3 2 + 716. Artık 4'ün karesi ni bildiğimize göre, 1 2, 2 2, 3 2 ve 4 2 değerlerine dayanarak 5'in karesini hesaplama k için prosedürümüze devam edebiliriz. (Ardışık farkların daha derin bir tartışması m evcut çalışmamızın kapsamının ötesinde olsa da, kalkülüs öğrencileri, önceki örneğin yx 2'nin türevine dayandığını gözlemlemek isteyebilirler; bu, 2 eğimine sahip bir doğ ruyu temsil eder.)



Harvard Üniversitesi'nde Howard Aiken ve bir grup IBM mühendisi tarafından (Şekil 0.4). Bu makineler, elektronik kontrollü mekanik röleleri yoğun bir şekilde kullandı. Bu anlamda, inşa edilir edilmez neredeyse modası geçmişti, çünkü diğer araştırmacıl ar, tamamen elektronik bilgisayarlar inşa etmek için vakum tüpleri teknolojisini uy guluyorlardı. Bu makinelerden ilki, 1937 ile 1941 yılları arasında Iowa Eyalet Koleji' nde (şimdi Iowa Eyalet Üniversitesi) John Atanasoff ve asistanı Clifford Berry tarafı ndan inşa edilen Atanasoff-Berry makinesi olarak görünmektedir. Diğer bir makin e ise Tommy'nin yönetiminde inşa edilen Colossus adında bir makinedir.

Doclingo https://www.doclingo.ai

Şekil **0.4** Mark I bilgisayarı (IBM arşivleri izniyle. İzin alınmadan kullanılamaz.)



İkinci Dünya Savaşı'nın son dönemlerinde Alman mesajlarını çözmek için İngiltere 'deki çiçekler. (Aslında, bu makinelerden en az on tanesinin yapıldığı görünüyordu , ancak askeri gizlilik ve ulusal güvenlik sorunları, varlıklarının "bilgisayar aile ağac ı"nın bir parçası haline gelmesini engelledi.) John Mauchly ve J. Presper Eckert tara fından Pennsylvania Üniversitesi'ndeki Moore Elektrik Mühendisliği Okulu'nda gel iştirilen ENIAC (elektronik sayısal entegre edici ve hesap makinesi) gibi daha esne k makineler kısa süre sonra ortaya çıktı.

O noktadan itibaren, bilgisayar makinelerinin tarihi, transistörlerin icadıy la (fizikçiler William Shockley, John Bardeen ve Walter Brattain'a Nobel Ödülü kazandıran) ve tek birim olarak inşa edilen tam devrelerin geliştirilmesiyle y akından bağlantılı hale geldi; bu devreler entegre devreler olarak adlandırılır (Jack Kilby'nin de fizik alanında Nobel Ödülü kazandığı). Bu gelişmelerle birli kte, 1940'ların oda boyutundaki makineleri, on yıllar içinde tek dolap boyutu na indirildi. Aynı zamanda, bilgisayar makinelerinin işlem gücü her iki yılda b ir iki katına çıkmaya başladı (bu eğilim günümüze kadar devam etti). Entegre devreler üzerindeki çalışmalar ilerledikçe, bir bilgisayar içindeki birçok devre , oyuncak boyutundaki plastik bloklar içinde yer alan entegre devreler olarak açık pazarda kolayca bulunabilir hale geldi.

Bilgisayarın popülerleşmesi yönünde atılan büyük bir adım, masaüstü bil gisayarların geliştirilmesiydi. Bu makinelerin kökenleri, ev yapımı bilgisayarla r inşa eden bilgisayar meraklılarına kadar uzanıyor. Steve Jobs ve Stephen W ozniak, bu "yeraltı" hobi faaliyetleri içinde ticari olarak uygulanabilir bir ev bil gisayarı inşa ettiler ve 1976'da ürünlerini üretmek ve pazarlamak için Apple Computer, Inc. (şimdi Apple Inc.)'i kurdular. Benzer ürünleri pazarlayan diğe r şirketler Commodore, Heathkit ve Radio Shack'tı. Bu ürünler bilgisayar mer aklıları arasında popüler olmasına rağmen,

Doclingo https://www.doclingo.ai

Augusta Ada Byron

Augusta Ada Byron, Lovelace Kontesi, bilgisayar topluluğunda çokça yorum konusu olmuştur. 37 yıl (1815–1852) kadar kısa bir yaşam sürmüştür; bu yaşam, kötü sağlık durumu ve kadınların profesyonel rolünü sınırlayan bir toplumda bir uyumsuz olması nedeniyle karmaşık hale gelmiştir. Geniş bir bilim yelpazesine ilgi duysa da, çalışmalarını matematik üzerine yoğunlaştırmıştır. "Bilgisayar bilimi"ne olan ilgisi, 1833'te Charles Babbage'ın Fark Motoru prototipinin bir gösteriminde makinelerine hayran kalmasıyla başladı. Bilgisayar bilimine katkısı, Babbage'ın Analitik Motor için tasarımlarını tartışan bir makalenin Fransızcadan İngilizceye çevirisinden kaynaklanmaktadır. Bu çeviriye, Babbage ona motorun uygulamalarını tanımlayan ve motorun çeşitli görevleri yerine getirmek için nasıl programlanabileceğine dair örnekler içeren bir ek eklemesini teşvik etti. Babbage'ın Ada Byron'ın çalışmasına olan heyecanı, muhtemelen bu çalışmanın yayınlanmasının Analitik Motor'un inşası için finansal destek sağlayacağı umuduyla motive olmuştur. (Lord Byron'ın kızı olan Ada Byron, potansiyel olarak önemli finansal bağlantılara sahip bir ünlü statüsüne sahipti.) Bu destek asla gerçekleşmedi, ancak Ada Byron'ın ekleri hayatta kaldı ve ilk bilgisayar programı örneklerini içerdiği kabul edilmektedir. Babbage'ın Ada Byron'ın çalışmasını ne ölçüde etkilediği tarihçiler tarafından tartışılmaktadır. Bazıları Babbage'ın büyük katkılarda bulunduğunu savunurken, diğerleri onun daha çok bir engel olduğunu iddia etmektedir.Yine de, Augusta Ada Byron bugün dünyanın ilk programcısı olarak tanınmaktadır; bu statü, ABD Savunma Bakanlığı tarafından onun onuruna adlandırılan önde gelen bir programlama dili (Ada) ile tescillenmiştir.

iş dünyası tarafından geniş çapta kabul edilmedi; bu topluluk, çoğu bilgisayar ihtiyacı için köklü IBM'e yönelmeye devam etti.

1981'de IBM, kişisel bilgisayar olarak adlandırılan ilk masaüstü bilgisayarını tanıttı; bu bilgisayarın temel yazılımı, Microsoft olarak bilinen yeni kurulan bir şirket tarafından geliştirildi. PC, anında bir başarı elde etti ve masaüstü bilgisayarı iş dünyası gözünde yerleşik bir mal olarak meşrulaştırdı. Bugün, *PC* terimi, IBM'in ilk masaüstü bilgisayarından evrim geçiren (çeşitli üreticilerden) tüm bu makineleri ifade etmek için yaygın olarak kullanılmaktadır; bunların çoğu, Microsoft'tan yazılımlarla pazarlanmaya devam etmektedir. Ancak, bazen *PC* terimi, genel terimler olan *masaüstü* veya *dizüstü* ile birbirinin yerine kullanılmaktadır.

Yirminci yüzyıl sona ererken, bireysel bilgisayarları dünya çapında bir sistem olan **Internet** ile bağlama yeteneği iletişimi devrim niteliğinde değiştiriyordu. Bu bağlamda, Tim Berners-Lee (bir Britanyalı bilim insanı), Internet üzerindeki bilgisayarlarda depolanan belgelerin birbirine bağlanabileceği bir sistem önerdi ve bu da **World Wide Web** (genellikle "Web" olarak kısaltılır) adı verilen bağlı bilgi labirentini üretti. Web'deki bilgileri erişilebilir kılmak için, arama motorları olarak adlandırılan yazılım sistemleri geliştirildi; bu sistemler Web'i "tarayıp", bulgularını "kategorize edip" ardından sonuçları belirli konularda araştırma yapan kullanıcılara yardımcı olmak için kullandılar. Bu alandaki büyük oyuncular Google, Yahoo ve Microsoft'tur. Bu şirketler, geleneksel düşünce biçimimizi zorlayan yönlerde, Web ile ilgili faaliyetlerini genişletmeye devam ediyorlar.

Doclingo https://www.doclingo.ai

Evlerde masaüstü bilgisayarların (ve daha yeni mobil dizüstü bilgisayarla rın) kabul edilip kullanılmaya başlandığı dönemde, bilgisayar makinelerinin miniaturizasyonu devam etti. Bugün, çeşitli cihazların içinde küçük bilgisayar lar yer almaktadır. Örneğin, otomobiller artık motorun işlevini izleyen ve ara cın ses ve telefon iletişim sistemlerini kontrol etmek için sesli komut hizmetl eri sunan Küresel Konumlama Sistemleri (GPS) çalıştıran küçük bilgisayarlar i çermektedir.

Belki de bilgisayar miniaturizasyonunun en potansiyel olarak devrim niteliğin deki uygulaması, taşınabilir telefonların genişleyen yeteneklerinde bulunuyor. Ge rçekten de, yakın zamanda sadece bir telefon olan şey, telefonculuğun yalnızca bi rçok uygulamadan biri olduğu küçük, elde taşınabilir genel amaçlı bir bilgisayar ol an akıllı telefona evrildi. Bu "telefonlar", kameralar, mikrofonlar, pusulalar, dokun matik ekranlar, telefonun yönünü ve hareketini algılamak için ivmeölçerler ve diğ er akıllı telefonlar ve bilgisayarlarla iletişim kurmak için bir dizi kablosuz teknoloji dahil olmak üzere zengin bir sensör ve arayüz dizisi ile donatılmıştır.

Potansiyel muazzam. Gerçekten de, birçok kişi akıllı telefonun toplum üzerinde PC'den daha b üyük bir etkiye sahip olacağını savunuyor.

Bilgisayarların miniaturizasyonu ve genişleyen yetenekleri, bilgisayar teknolojisi ni günümüz toplumunun ön saflarına getirmiştir. Bilgisayar teknolojisi o kadar yaygı n hale geldi ki, bununla tanışıklık modern toplumun bir üyesi olmanın temel bir unsu ru haline gelmiştir. Hesaplama teknolojisi, hükümetlerin kontrol uygulama yeteneği ni değiştirmiş; küresel ekonomiler üzerinde büyük bir etki yaratmış; bilimsel araştır malarda çarpıcı ilerlemelere yol açmış; veri toplama, depolama ve uygulamaların rol ünü devrim niteliğinde değiştirmiş; insanların iletişim kurma ve etkileşimde bulunm a biçimlerine yeni yollar sunmuş; ve sürekli olarak toplumun mevcut durumunu sorg ulamıştır. Sonuç olarak, bilgisayar bilimi etrafında birçok konu ortaya çıkmış ve her b iri kendi başına önemli bir çalışma alanı haline gelmiştir. Dahası, mekanik mühendisl ik ve fizik gibi, bu alanlar arasında bir sınır çizmek genellikle zordur.

Google

1998 yılında kurulan Google Inc., dünyanın en tanınmış teknoloji şirketlerinden biri halin e gelmiştir. Temel hizmeti olan Google arama motoru, milyonlarca insan tarafından Dün ya Çapında Ağ'da belgeleri bulmak için kullanılmaktadır. Ayrıca, Google elektronik posta hizmeti (Gmail olarak adlandırılır), internet tabanlı bir video paylaşım hizmeti (YouTube olarak adlandırılır) ve Google Haritalar, Google Takvim, Google Earth, Google Kitaplar ve Google Çeviri gibi birçok diğer internet hizmeti sunmaktadır.

Ancak, girişimci ruhun mükemmel bir örneği olmanın yanı sıra,Google, genişleyen teknolojinin toplumu nasıl zorladığına dair örnekler de sunma ktadır. Örneğin, Google'ın arama motoru, uluslararası bir şirketin bireysel hükümetlerin isteklerine ne ölçüde uyması gerektiği konusunda sorulara y ol açmıştır; YouTube, bir şirketin hizmetleri aracılığıyla başkaları tarafından dağıtılan bilgiler için ne ölçüde sorumlu olması gerektiği ve şirketin bu bil gilere sahiplik iddiasında bulunma derecesi konusunda sorular ortaya çıka rmıştır; Google Books, fikri mülkiyet haklarının kapsamı ve sınırlamaları ile ilgili endişeler yaratmıştır; ve Google Maps, gizlilik haklarını ihlal etmekle s uçlanmıştır.

Doclingo https://www.doclingo.ai

bilgisayar bilimi kendisi. Bu nedenle, doğru bir perspektif kazanmak için, çalı şmamız yalnızca bilgisayar biliminin merkezine ilişkin konuları kapsamayaca k, aynı zamanda bilimin uygulamaları ve sonuçlarıyla ilgili çeşitli disiplinleri d e keşfedecektir. Gerçekten de, bilgisayar bilimine giriş, disiplinler arası bir gir işimdir.

0.3 Algoritmaların Bilimi

Sınırlı veri depolama yetenekleri ve karmaşık, zaman alıcı programlama prosed ürleri, erken hesaplama makinelerinde kullanılan algoritmaların karmaşıklığını kısıtlamıştır. Ancak, bu sınırlamalar ortadan kalkmaya başladıkça, makineler gi derek daha büyük ve daha karmaşık görevler için uygulanmaya başlanmıştır. B u görevlerin bileşimini algoritmik biçimde ifade etme girişimleri insan zihninin yeteneklerini zorlamaya başladıkça, daha fazla araştırma çabası algoritmaların ve programlama sürecinin incelenmesine yönlendirilmiştir.

Bu bağlamda, matematikçilerin teorik çalışmaları meyve vermeye başlad ı.Gödel'in eksiklik teoreminin bir sonucu olarak, matematikçiler, gelişen tekn olojinin şimdi gündeme getirdiği algoritmik süreçlerle ilgili soruları araştırm aya başlamışlardı. Böylece, bilgisayar bilimi olarak bilinen yeni bir disiplinin o rtaya çıkması için zemin hazırlanmış oldu.

Bugün, bilgisayar bilimi algoritmaların bilimi olarak kendini kanıtlamıştır. B u bilimin kapsamı geniştir ve matematik, mühendislik, psikoloji, biyoloji, işletm e yönetimi ve dilbilim gibi çeşitli konulardan beslenmektedir. Gerçekten de, bil gisayar biliminin farklı dallarındaki araştırmacılar, bilimi çok farklı tanımlara sa hip olabilirler. Örneğin, bilgisayar mimarisi alanındaki bir araştırmacı, devreleri küçültme görevine odaklanabilir ve bu nedenle bilgisayar bilimini teknolojinin i lerlemesi ve uygulanması olarak görebilir. Ancak, veritabanı sistemleri alanında ki bir araştırmacı, bilgisayar bilimini bilgi sistemlerini daha kullanışlı hale getir me yollarını aramak olarak görebilir. Ve yapay zeka alanındaki bir araştırmacı, bilgisayar bilimini zeka ve zeki davranışın incelenmesi olarak değerlendirebilir.

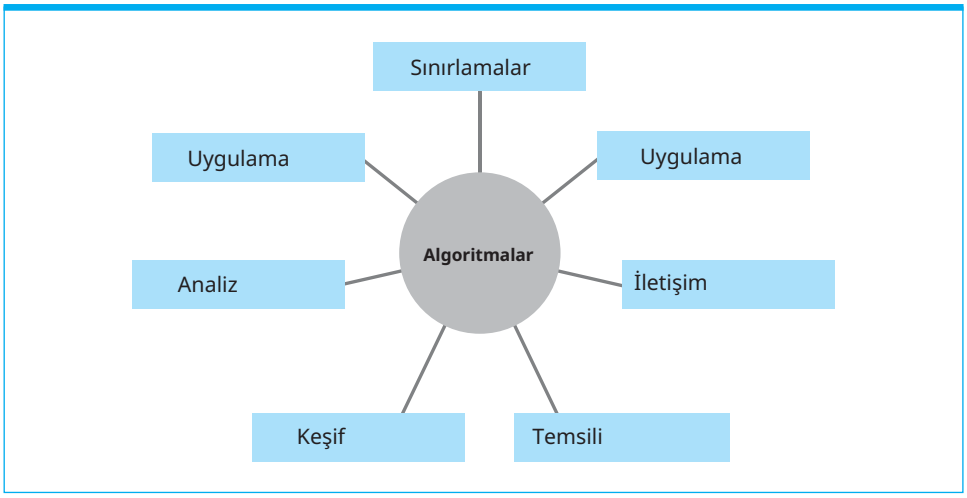
Böylece, bilgisayar bilimine bir giriş, çeşitli konuları içermelidir,
bu da takip eden bölümlerde ele alacağımız bir görevdir. Her durumda, amacımız konudaki merkezi fikirleri, mevcut araştırma konularını ve alandaki bilgiyi ilerletmek için uygulanan bazı teknikleri tanıtmaktır. Bu kadar çeşitli konularla, genel resmi kaybetmek kolaydır.
Bu nedenle, çalışmanın odaklanması için bazı soruları belirleyerek düşüncelerimiz i toplamak için duruyoruz.

- Algoritmik süreçler tarafından hangi problemler çözülebilir?
- Algoritmaların keşfi nasıl kolaylaştırılabilir?
- Algoritmaları temsil etme ve iletme teknikleri nasıl geliştirilebilir ?
- Farklı algoritmaların özellikleri nasıl analiz edilebilir ve karşılaş tırılabilir?
- Algoritmalar bilgiyi manipüle etmek için nasıl kullanılabilir?
- Algoritmalar zeki davranış üretmek için nasıl uygulanabilir?
- Algoritmaların uygulanması toplum üzerinde nasıl bir etki yapar?

Tüm bu soruların ortak temasının algoritmaların incelenmesi olduğunu unutmayın (Şekil 0.5).

Doclingo https://www.doclingo.ai

Şekil **0.5** Algoritmaların bilgisayar bilimindeki merkezi rolü



0.4 Soyutlama

Soyutlama kavramı, bilgisayar bilimi çalışmaları ve bilgisayar sistemlerinin ta sarımı üzerinde o kadar derin bir etkiye sahiptir ki, bu ön bölümde ele alınm ası gerekmektedir. Burada kullandığımız soyutlama, bir varlığın dışsal özelli kleri ile varlığın iç yapısının detayları arasındaki ayrımı ifade eder. Soyutlama, bir bilgisayar, otomobil veya mikrodalga fırın gibi karmaşık bir cihazın iç det aylarını göz ardı etmemizi ve onu tek bir, anlaşılır bir birim olarak kullanma mızı sağlar. Dahası, bu tür karmaşık sistemlerin tasarlanması ve üretilmesi d e soyutlama aracılığıyla gerçekleşir. Bilgisayarlar, otomobiller ve mikrodalga fırınlar, her biri daha küçük bileşenlerden oluşan bileşenlerden inşa edilmişti r. Her bileşen, bileşenin iç yapısının detaylarından izole edilmiş bir bileşen ku llanım seviyesini temsil eder.

Abstraksiyonu uygulayarak, büyük, karmaşık bilgisayar sistemlerini inşa edebi liyor, analiz edebiliyor ve yönetebiliyoruz; bu sistemler, detaylı bir seviyede tama mı göz önüne alındığında bunaltıcı olurdu. Her bir soyutlama seviyesinde, sistemi, iç bileşimini göz ardı ettiğimiz soyut araçlar olarak adlandırılan bileşenler açısınd an inceliyoruz. Bu, her bir bileşenin aynı seviyedeki diğer bileşenlerle nasıl etkileşi mde bulunduğuna ve topluluğun bir bütün olarak nasıl daha yüksek seviyeli bir bi leşen oluşturduğuna odaklanmamıza olanak tanır. Böylece, detaylar denizinde ka ybolmak yerine, mevcut görevle ilgili sistemin parçasını anlamış oluruz.

Abstraksiyonun bilim ve teknoloji ile sınırlı olmadığını vurguluyoruz. Bu, t oplumumuzun aksi takdirde imkansız olacak bir yaşam tarzı yarattığı önemli bir basitleştirme tekniğidir. Günlük yaşamın çeşitli kolaylıklarının aslında nası l uygulandığını pek azımız anlıyor. Kendimiz üretemediğimiz yiyecekler yiyor uz ve giysiler giyiyoruz. Temel teknolojiyi anlamadan elektrikli cihazlar ve ilet işim sistemleri kullanıyoruz. Diğerlerinin hizmetlerini, mesleklerinin detayları nı bilmeden kullanıyoruz. Her yeni ilerleme ile, toplumun küçük bir kısmı uyg ulamasında uzmanlaşmayı seçerken, geri kalanımız sonuçları soyut araçlar o larak kullanmayı öğreniyor. Bu şekilde, toplumun soyut araçlar deposu geniş ler ve toplumun ilerleme yeteneği artar.

Doclingo https://www.doclingo.ai

Soyutlama, çalışmamızda tekrarlayan bir temadır. Bilgisayar ekipmanının soyut araçların seviyelerinde inşa edildiğini öğreneceğiz. Ayrıca, büyük yazılım sistemlerinin, her modülün daha büyük modüllerde bir soyut araç olarak kullanıldığı modüler bir şekilde geliştirildiğini göreceğiz. Dahası, soyutlama, bilgisayar biliminin kendisini ilerletme görevinde önemli bir rol oynarak araştırmacıların karmaşık bir alandaki belirli alanlara odaklanmalarını sağlar. Aslında, bu metnin organizasyonu bilimin bu özelliğini yansıtır. Bilimin belirli bir alanına odaklanan her bölüm, genellikle diğerlerinden şaşırtıcı bir şekilde bağımsızdır, ancak birlikte bölümler geniş bir çalışma alanının kapsamlı bir genel görünümünü oluşturur.

0.5 Çalışmamızın Taslağı

Bu metin, bilgisayar biliminin incelenmesine alt-üst bir yaklaşım izleyerek, bilgi sayar donanımı gibi uygulamalı konularla başlayıp, algoritma karmaşıklığı ve h esaplanabilirlik gibi daha soyut konulara yönelmektedir. Sonuç olarak, çalışma mız, konuyu anlama düzeyimiz arttıkça daha büyük ve daha soyut araçlar inşa etme desenini takip etmektedir.

Algoritmaları yürütmek için makinelerin tasarımı ve inşasıyla ilgili konula rı ele alarak başlıyoruz. Bölüm 1'de (Veri Depolama) bilgilerin modern bilgisa yarlarda nasıl kodlandığını ve depolandığını inceliyoruz ve Bölüm 2'de (Veri Manipülasyonu) basit bir bilgisayarın temel iç işleyişini araştırıyoruz. Bu çalış manın bir kısmı teknoloji ile ilgili olsa da, genel tema teknoloji bağımsızdır. Y ani, dijital devre tasarımı, veri kodlama ve sıkıştırma sistemleri ile bilgisayar mimarisi gibi konular, geniş bir teknoloji yelpazesinde geçerlidir ve gelecekt eki teknolojinin yönüne bakılmaksızın geçerliliğini koruyacağına söz veriyor.

Bölüm 3'te (İşletim Sistemleri) bir bilgisayarın genel işleyişini kontrol ede n yazılımı inceliyoruz. Bu yazılıma işletim sistemi denir. Bir bilgisayarın işleti m sistemi, makine ile dış dünya arasındaki arayüzü kontrol eder, makineyi ve içindeki verileri yetkisiz erişimden korur, bir bilgisayar kullanıcısının çeşitli pr ogramların yürütülmesini talep etmesine olanak tanır ve kullanıcının talepler ini yerine getirmek için gereken iç faaliyetleri koordine eder.

Bölüm 4'te (Ağ Oluşturma ve İnternet) bilgisayarların birbirine nasıl bağl andığını ve ağların nasıl internete bağlandığını inceliyoruz. Bu çalışma, ağ pr otokolleri, İnternet'in yapısı ve iç işleyişi, Dünya Çapında Ağ ve güvenlik ile il gili birçok konuyu kapsar.

Bölüm 5 (Algoritmalar), algoritmaların daha resmi bir perspektiften incel enmesini tanıtmaktadır. Algoritmaların nasıl keşfedildiğini araştırıyor, birkaç temel algoritmik yapıyı tanımlıyor, algoritmaları temsil etmek için temel tekn ikler geliştiriyor ve algoritma verimliliği ve doğruluğu konularını tanıtıyoruz.

Bölüm 6 (Programlama Dilleri) algoritma temsilini ve program geliştirme sürecini ele alıyoruz. Burada, daha iyi programlama teknikleri arayışının, her birinin kendi programlama dilleri setine sahip çeşitli programlama metodol ojileri veya paradigmaları ile sonuçlandığını görüyoruz.
Bu paradigmaları ve dilleri araştırıyor ve dilbilgisi ve dil çevirisi konularını da ele alıyoruz.

Doclingo https://www.doclingo.ai

Bölüm 7 (Yazılım Mühendisliği), büyük yazılım sistemleri geliştirirken kar şılaşılan sorunlarla ilgilenen yazılım mühendisliği olarak bilinen bilgisayar bili mi dalını tanıtmaktadır. Temel tema, büyük yazılım sistemlerinin tasarımının, geleneksel mühendisliğin ötesinde sorunları kapsayan karmaşık bir görev ol duğudur. Bu nedenle, yazılım mühendisliği konusu, mühendislik, proje yöne timi, personel yönetimi, programlama dili tasarımı ve hatta mimarlık gibi çeş itli alanlardan yararlanarak bilgisayar bilimi içinde önemli bir araştırma alanı haline gelmiştir.

Sonraki iki bölümde verilerin bir bilgisayar sisteminde nasıl organize edilebilece ğine bakacağız. 8. Bölüm (Veri Soyutlamaları) bilgisayarın ana belleğinde verileri org anize etmek için geleneksel olarak kullanılan teknikleri tanıtır ve ardından veri soyutl amasının evrimini, ilkel kavramlardan günümüzün nesne yönelimli tekniklerine kada r izleriz. 9. Bölüm (Veritabanı Sistemleri) ise bilgisayarın kütle depolamasında verileri organize etmek için geleneksel olarak kullanılan yöntemleri ele alır ve son derece bü yük ve karmaşık veritabanı sistemlerinin nasıl uygulandığını araştırırız.

10. Bölüm (Bilgisayar Grafikleri) grafik ve animasyon konusunu keşfederi z; bu alan sanal dünyalar yaratma ve fotoğraflama ile ilgilidir.
Daha geleneksel bilgisayar bilimi alanlarındaki (makine mimarisi, algoritma t asarımı, veri yapıları ve yazılım mühendisliği gibi) ilerlemelere dayanarak, gr afik ve animasyon disiplini önemli bir ilerleme kaydetmiş ve şimdi heyecan v erici, dinamik bir konu haline gelmiştir. Ayrıca, bu alan, bilgisayar biliminin ç eşitli bileşenlerinin fizik, sanat ve fotoğrafçılık gibi diğer disiplinlerle nasıl birl eştiğini ve çarpıcı sonuçlar ürettiğini örneklemektedir.

11. Bölüm (Yapay Zeka) bölümünde, daha kullanışlı makineler geliştirme k için bilgisayar biliminin insan zekasını incelemeye yöneldiğini öğreniyoruz. Umut, kendi zihinlerimizin nasıl akıl yürüttüğünü ve algıladığını anlayarak, ar aştırmacıların bu süreçleri taklit eden algoritmalar tasarlayabilmeleridir ve b öylece bu yetenekleri makinelere aktarabilmektir. Sonuç, psikoloji, biyoloji ve dilbilim gibi alanlardaki araştırmalara büyük ölçüde dayanan yapay zeka ola rak bilinen bilgisayar bilimi alanıdır.

Çalışmamızı, algoritmaların (ve dolayısıyla makinelerin) sınırlamalarını an lamamıza olanak tanıyan bir konu olan bilgisayar biliminin teorik temellerini araştırarak Bölüm 12 (Hesaplama Teorisi) ile kapatıyoruz. Burada, algoritmik olarak çözülemeyen bazı problemleri (ve dolayısıyla makinelerin yetenekleri nin ötesinde olanları) tanımlıyoruz ve birçok diğer problemin çözümlerinin o kadar büyük zaman veya alan gerektirdiğini öğreniyoruz ki, bunlar pratik bir perspektiften de çözülemez. Böylece, bu çalışma sayesinde algoritmik sistem lerin kapsamını ve sınırlamalarını kavrayabiliyoruz.

Her bölümdeki amacımız, konunun gerçek bir anlayışa yol açacak derinli ğe inmek. Bilgisayar bilimi hakkında, yaşadığınız teknik toplumu anlamanızı sağlayacak ve bilim ve teknolojinin ilerlemesiyle kendi başınıza öğrenebilece ğiniz bir temel oluşturacak bir çalışma bilgisi geliştirmek istiyoruz.

0.6 Sosyal Sonuçlar

Bilgisayar bilimindeki ilerleme, toplumumuzun geçmişte kararlarını dayandırdı ğı birçok ayrımı bulanıklaştırıyor ve toplumun uzun süredir sahip olduğu birçok ilkeyi sorguluyor. Hukukta, fikri mülkiyetin ne ölçüde sahip olunabileceği ve bu nunla birlikte gelen haklar ve yükümlülükler hakkında sorular ortaya çıkarıyor.

Doclingo https://www.doclingo.ai

sahiplik. Etikte, sosyal davranışın dayandığı geleneksel ilkelere meydan okuyan birçok seçenek üretir. Hükümette, bilgisayar teknolojisi ve uygulamalarının ne ölçüde düzenlenmesi gerektiği konusunda tartışmalar yaratır. Felsefede, zeki davranışın varlığı ile zekanın kendisinin varlığı arasında bir çatışma yaratır. Ve toplum genelinde, yeni uygulamaların yeni özgürlükler mi yoksa yeni kontroller mi temsil ettiği konusunda anlaşmazlıklar doğurur.

Bilgisayar biliminin kendisinin bir parçası olmasa da, bu tür konular, bilgisayar veya bilgisayarla ilgili alanlarda kariyer düşünenler için önemlidir. Bilimdeki ifşalar bazen tartışmalı uygulamalar bulmuş, bu da ilgili araştırmacılar için ciddi bir hoşnutsuzluk yaratmıştır. Dahası, aksi takdirde başarılı bir kariyer, bir etik hata ile hızla raydan çıkabilir.

Gelişen bilgisayar teknolojisinin ortaya çıkardığı ikilemlerle başa çıkabilme yeteneği, onun hemen dışında olanlar için de önemlidir. Gerçekten de, teknoloji topluma o kadar hızlı sızıyor ki, etkilerinden bağımsız olan çok az kişi, varsa, bulunmaktadır.

Bu metin, bilgisayar biliminin ürettiği ikilemleri rasyonel bir şekilde ele almak için gereken teknik arka planı sağlar. Ancak, bilimin teknik bilgisi tek başına, ilgili tüm sorulara çözümler sunmaz. Bunu göz önünde bulundurarak, bu metin sosyal, etik ve hukuki konulara adanmış birkaç bölüm içermektedir. Bunlar arasında güvenlik endişeleri, yazılım sahipliği ve sorumluluğu konuları, veritabanı teknolojisinin sosyal etkisi ve yapay zeka alanındaki ilerlemelerin sonuçları bulunmaktadır.

Ayrıca, bir probleme genellikle kesin bir doğru cevap yoktur ve birçok geçerli çözüm, karşıt (ve belki de eşit derecede geçerli) görüşler arasında bir uzlaşmadır. Bu tür durumlarda çözümler bulmak, dinleme yeteneğini, diğer bakış açılarını tanıma yeteneğini, mantıklı bir tartışma yürütme yeteneğini ve yeni içgörüler elde edildikçe kendi görüşünü değiştirme yeteneğini gerektirir. Bu nedenle, bu metnin her bölümü, bilgisayar bilimi ile toplum arasındaki ilişkiyi araştıran “Sosyal Sorunlar” başlığı altında bir dizi soru ile sona ermektedir. Bunlar mutlaka cevaplanması gereken sorular değildir. Bunun yerine, düşünülmesi gereken sorulardır. Birçok durumda, ilk başta açık görünebilecek bir cevap, alternatifleri keşfettikçe sizi tatmin etmeyi bırakacaktır. Kısacası, bu soruların amacı sizi “doğru” bir cevaba yönlendirmek değil, farkındalığınızı artırmaktır; bu, bir sorundaki çeşitli paydaşlara olan farkındalığınızı, alternatiflere olan farkındalığınızı ve bu alternatiflerin hem kısa hem de uzun vadeli sonuçlarına olan farkındalığınızı içerir.

Bu bölümü, kararları ve davranışları yönlendiren ilkeler için temel teoriler arayışında filozoflar tarafından önerilen bazı etik yaklaşımlarını tanıtarak kapatıyoruz. Bu teorilerin çoğu, sonuç temelli etik, görev temelli etik, sözleşme temelli etik ve karakter temelli etik başlıkları altında sınıflandırılabilir. Bu teorileri, metinde sunulan etik sorunlara yaklaşmanın bir yolu olarak kullanmak isteyebilirsiniz.
Özellikle, farklı teorilerin çelişkili sonuçlara yol açtığını ve böylece gizli alternatifleri ortaya çıkardığını görebilirsiniz.

Sonuç temelli etik, çeşitli seçeneklerin sonuçlarına dayanarak sorunları analiz etmeye çalışır. Önde gelen bir örnek, toplumun en büyük kesimi için en büyük iyiliğe yol açan “doğru” kararın veya eylemin ne olduğunu öneren faydacılıktır. İlk bakışta faydacılık, etik ikilemleri çözmenin adil bir yolu gibi görünmektedir. Ancak, niteliksiz formunda faydacılık

Doclingo https://www.doclingo.ai

birçok kabul edilemez sonuca yol açar. Örneğin, bir toplumun çoğunluğunun küç ük bir azınlığı köleleştirmesine izin verir. Dahası, birçok kişi, sonuçlara vurgu yapa n etik teorilerine dayalı yaklaşımların, insanı yalnızca bir amaca ulaşmak için bir ar aç olarak görme eğiliminde olduğunu savunur; bu da insanı değerli bir birey olara k görmemektedir. Bu, devam ederler, tüm sonuç temelli etik teorilerinde temel bi r kusuru oluşturmaktadır.

Sonuç temelli etik ile karşılaştırıldığında, görev temelli etik, kararların ve eylemlerin sonuçlarını dikkate almaz; bunun yerine, bir toplumun üyelerinin etik soruların çözülmesi için temel oluşturan belirli içsel görevleri veya yükü mlülükleri olduğunu önerir. Örneğin, başkalarının haklarına saygı gösterme yükümlülüğünü kabul edersek, sonuçlarına bakılmaksızın köleliği reddetmeli yiz. Öte yandan, görev temelli etik karşıtları, çelişkili görevlerle ilgili sorunlar a çözümler sunmadığını savunur. Bir meslektaşın güvenini yok etse bile gerç eği söylemeli misiniz? Bir ulus, savaşta kendini savunmalı mı, yoksa ardından gelen savaşlar birçok vatandaşının ölümüne yol açacaksa?

Sözleşmeye dayalı etik teorisi, tamamen etik temeli olmayan bir toplumu h ayal ederek başlar. Bu "doğa durumu" ortamında, her şey serbesttir—bireyleri n kendilerini savunmak zorunda olduğu ve sürekli olarak diğerlerinden gelen s aldırılara karşı dikkatli olmaları gereken bir durumdur. Bu koşullar altında, sözl eşmeye dayalı etik teori, toplumun üyelerinin birbirleriyle "sözleşmeler" geliştir eceğini öne sürer. Örneğin, sen benden çalmayacaksan, ben de senden çalmay acağım. Bu "sözleşmeler" daha sonra etik davranışı belirlemenin temeli haline gelecektir. Sözleşmeye dayalı etik teorisinin etik davranış için bir motivasyon sa ğladığını unutmayın—"etik sözleşmelere" uymalıyız çünkü aksi takdirde hoş ol mayan bir yaşam süreriz. Ancak, sözleşmeye dayalı etik teorisinin karşıtları, etik ikilemleri çözmek için yeterince geniş bir temel sağlamadığını savunurlar çünk ü bu teori yalnızca sözleşmelerin kurulduğu durumlarda rehberlik sağlar. (Mev cut bir sözleşmeyle kapsanmayan durumlarda istediğim gibi davranabilirim.) Ö zellikle, yeni teknolojiler mevcut etik sözleşmelerin geçerli olmayabileceği keşfe dilmemiş alanlar sunabilir.

Karakter temelli etik (bazen erdem etiği olarak adlandırılır), Platon ve Ari stoteles tarafından teşvik edilmiştir ve "iyi davranışın" tanımlanabilir kuralları n uygulanmasının sonucu değil, "iyi karakterin" doğal bir sonucu olduğunu s avunur. Sonuç temelli etik, görev temelli etik ve sözleşmeye dayalı etik, bir ki şinin etik bir ikilemi çözmesini "Sonuçlar nelerdir?"; "Görevlerim nelerdir?"; ve ya "Hangi sözleşmelere sahibim?" diye sorarak önerirken, karakter temelli eti k, ikilemlerin "Kim olmak istiyorum?" diye sorarak çözülmesini önerir. Böylec e, iyi davranış, genellikle sağlam bir yetiştirme ve erdemli alışkanlıkların geliş tirilmesinin sonucu olan iyi karakter inşa edilerek elde edilir.

Etik öğretiminde genellikle benimsenen yaklaşım, karakter temelli etik a nlayışına dayanmaktadır. Belirli etik teorileri sunmak yerine, profesyonelleri n uzmanlık alanlarındaki çeşitli etik soruları ortaya koyan vaka çalışmalarını t anıtmak amaçlanmaktadır. Ardından, bu vakalardaki artı ve eksileri tartışara k, profesyoneller kendi mesleki yaşamlarında gizli tehlikelerin farkına varara k daha bilinçli, anlayışlı ve duyarlı hale gelirler ve böylece karakter olarak geli şirler. Her bölümün sonunda sosyal konularla ilgili soruların sunulma ruhu b udur.

Doclingo https://www.doclingo.ai

Sosyal Sorunlar

Aşağıdaki sorular, bilgisayar alanıyla ilgili etik/sosyal/hukuki sorunlara rehberlik etmek amacıyla hazırlanmıştır. Amaç, bu soruları yalnızca yanıtlamak değildir. Ayrıca, neden böyle yanıt verdiğinizi ve gerekçelerinizin bir sorudan diğerine tutarlı olup olmadığını da düşünmelisiniz.

1. Toplumumuzun bilgisayar devrimi olmadan olacağı halden farklı olduğu önermesi genel olarak kabul edilmektedir. Toplumumuz, devrim olmadan olacağı halden daha mı iyi? Toplumumuz daha mı kötü? Toplum içindeki konumunuz farklı olsaydı, yanıtınız değişir miydi?
2. Bugünün teknik toplumuna katılmak, o teknolojinin temellerini anlamak için çaba göstermeden kabul edilebilir mi? Örneğin, oyları genellikle teknolojinin nasıl destekleneceğini ve kullanılacağını belirleyen bir demokrasinin üyelerinin, o teknolojiyi anlamaya çalışmak gibi bir yükümlülüğü var mı? Cevabınız hangi teknolojinin değerlendirildiğine bağlı mı? Örneğin, nükleer teknoloji ile bilgisayar teknolojisini değerlendirirken cevabınız aynı mı?
3. Nakitsiz finansal işlemlerde, bireyler geleneksel olarak finansal işlerini hizmet ücretleri olmadan yönetme seçeneğine sahipti. Ancak, ekonomimizin daha fazla otomatik hale gelmesiyle birlikte, finansal kurumlar bu otomatik sistemlere erişim için hizmet ücretleri uygulamaya başlıyor. Bu ücretlerin bireyin ekonomiye erişimini haksız bir şekilde kısıtladığı bir nokta var mı? Örneğin, bir işveren çalışanlara yalnızca çekle ödeme yapıyorsa ve tüm finansal kurumlar çek bozdurma ve yatırma işlemleri için bir hizmet ücreti alıyorsa, çalışanlar haksız bir şekilde mi muamele görüyor? Peki, bir işveren yalnızca doğrudan yatırma ile ödeme yapmayı ısrarla talep ederse ne olur?
4. İnteraktif televizyon bağlamında, bir şirketin çocuklardan bilgi almasına (belki de interaktif bir oyun formatı aracılığıyla) ne ölçüde izin verilmelidir? Örneğin, bir şirketin bir çocuğun ebeveynlerinin satın alma alışkanlıkları hakkında rapor almasına izin verilmeli mi? Çocuk hakkında bilgi almak ne olacak?
5. Bir hükümetin bilgisayar teknolojisini ve uygulamalarını ne ölçüde düzenlemesi gerekir? Örneğin, 3. ve 4. Sorularda bahsedilen konuları düşünün. Hükümet düzenlemesini ne haklı çıkarır?
6. Teknoloji ile ilgili, genel olarak ve özellikle bilgisayar teknolojisi ile ilgili alacağımız kararlar, gelecek nesilleri ne ölçüde etkileyecek?
7. Teknoloji ilerledikçe, eğitim sistemimiz sürekli olarak konuların sunulduğu soyutlama seviyesini yeniden düşünmeye zorlanıyor. Birçok soru, bir becerinin hala gerekli olup olmadığı veya öğrencilerin soyut bir araca güvenmelerine izin verilmesi gerekip gerekmediği biçiminde ortaya çıkıyor. Trigonometri öğrencilerine artık trigonometrik fonksiyonların değerlerini tablolar kullanarak bulmaları öğretilmiyor. Bunun yerine, bu değerleri bulmak için soyut araçlar olarak hesap makineleri kullanıyorlar. Bazıları uzun bölmenin de soyutlamaya yer vermesi gerektiğini savunuyor. Benzer tartışmalarla ilgili başka hangi konular var? Modern kelime işlemciler, yazım becerilerini geliştirme ihtiyacını ortadan kaldırıyor mu? Video teknolojisinin kullanımı bir gün okumaya olan ihtiyacı ortadan kaldıracak mı?

Doclingo https://www.doclingo.ai

8. Kamusal kütüphanelerin kavramı, büyük ölçüde bir demokrasideki tüm vatandaşların bilgiye erişim hakkına sahip olması gerektiği öncülüne dayanmaktadır. Bilgi bilgisayar teknolojisi aracılığıyla daha fazla depolandıkça ve yayıldıkça, bu teknolojiye erişim her bireyin hakkı haline mi geliyor? Eğer öyleyse, kamusal kütüphaneler bu erişimin sağlandığı kanal olmalı mı?
9. Soyut araçların kullanımına dayanan bir toplumda hangi etik endişeler ortaya çıkmaktadır? Bir ürün veya hizmetin nasıl çalıştığını anlamadan kullanmanın etik olmadığı durumlar var mı? Nasıl üretildiğini bilmeden? Ya da, kullanımının yan ürünlerini anlamadan?
10. Toplumumuz daha otomatik hale geldikçe, hükümetlerin vatandaşlarının faaliyetlerini izlemeleri daha kolay hale geliyor. Bu iyi mi yoksa kötü mü?
11. George Orwell'in (Eric Blair) romanı *1984* da hayal ettiği hangi teknolojiler gerçeğe dönüştü? Bunlar Orwell'in öngördüğü şekilde mi kullanılıyor?
12. Eğer bir zaman makineniz olsaydı, hangi tarih döneminde yaşamak isterdiniz? Yanınıza almak istediğiniz mevcut teknolojiler var mı? Seçtiğiniz teknolojiler diğerlerini almadan yanınıza alınabilir mi? Bir teknolojiyi diğerinden ne ölçüde ayırmak mümkündür? Küresel ısınmaya karşı protesto etmekle modern tıbbi tedaviyi kabul etmek tutarlı mı?
13. İşiniz başka bir kültürde yaşamanızı gerektiriyorsa, kendi kültürünüzün etiklerini mi uygulamaya devam etmelisiniz yoksa ev sahibi kültürün etiklerini mi benimsemelisiniz? Cevabınız, konunun giyim kuralları mı yoksa insan hakları mı ile ilgili olduğuna bağlı mı? Kendi kültürünüzde yaşamaya devam ederken, yabancı bir kültürle iş yapıyorsanız hangi etik standartların geçerli olması gerekir?
14. Toplum, ticaret, iletişim veya sosyal etkileşimler için bilgisayar uygulamalarına çok mu bağımlı hale geldi? Örneğin, internet ve/veya cep telefonu hizmetinde uzun süreli bir kesintinin sonuçları ne olurdu?
15. Çoğu akıllı telefon, GPS aracılığıyla telefonun konumunu belirleyebilir. Bu, uygulamaların telefonun mevcut konumuna dayalı olarak yerel haberler, yerel hava durumu veya çevredeki işletmelerin varlığı gibi konuma özgü bilgiler sağlamasına olanak tanır. Ancak, bu tür GPS yetenekleri, diğer uygulamaların telefonun konumunu diğer taraflara iletmesine de olanak tanıyabilir. Bu iyi mi? Telefonun konumunun (dolayısıyla sizin konumunuzun) kötüye kullanılması nasıl olabilir?
16. Önceki sorulara verdiğiniz ilk yanıtlara dayanarak, Bölüm 0.6'da sunulan hangi etik teorisine abone olma eğilimindesiniz?

Ek Okuma

Goldstine, J. J. *Pascal'dan von Neumann'a Bilgisayar*. Princeton: Princeton Üniversitesi Yayınları, 1972.

Kizza, J. M. *Bilgi Çağında Etik ve Sosyal Sorunlar*, 3. baskı. Londra: Springer-Verlag, 2007.

Doclingo https://www.doclingo.ai

Mollenhoff, C. R. *Atanasoff: Unutulmuş Bilgisayar Babası*. Ames: Iowa Eyalet Üniversitesi Yayınları, 1988.

Neumann, P. G. *Bilgisayar İle İlgili Riskler*. Boston, MA: Addison-Wesley, 1995.

Ni, L. *Akıllı Telefon ve Gelecek Nesil Mobil Bilgisayar.*San Francisco: Morgan Kaufmann, 2006.

Quinn, M. J. *Bilgi Çağı için Etik*, 2. baskı. Boston, MA: Addison-Wesley, 2006.

Randell, B. *Dijital Bilgisayarların Kökenleri*, 3. baskı. New York: Springer-Verlag, 1982.

Spinello, R. A. ve H. T. Tavani. *Siberetik Okumalar*, 2. baskı. Sudbury, MA: Jones ve Bartlett, 2004.

Swade, D. *Fark Motoru*. New York: Viking, 2000.

Tavani, H. T. *Etik ve Teknoloji: Bilgi ve İletişim Teknolojisi Çağında Etik Soru nlar*, 3. baskı. New York: Wiley, 2011.

Woolley, B. *Bilimin Gelini, Romantizm, Akıl ve Byron'un Kızı*. New York: McGraw-Hill, 1999.

Doclingo https://www.doclingo.ai

BÖLÜM

Veri Depolama

1

Bu bölümde, veri temsili ve bir bilgisayar içindeki veri depo lama ile ilgili konuları ele alıyoruz. Dikkate alacağımız veri t ürleri metin, sayısal değerler, görüntüler, ses ve video olac aktır. Bu bölümdeki bilgilerin çoğu, dijital fotoğrafçılık, ses /video kaydı ve yeniden üretimi, ve uzun mesafe iletişimi g ibi geleneksel bilgisayar biliminin ötesindeki alanlar için de geçerlidir.

1.1 Bitler ve Depolama

Boolean İşlemleri
Kapılar ve Flip-Flop'lar
Onaltılık Notasyon

1.2 Ana Bellek

Bellek Organizasyonu
Bellek Kapasitesinin Ölçülmesi

1.3 Kitlesel Depolama

Manyetik SistemlerO
ptik SistemlerFlas
h Sürücüler
Dosya Depolama ve Geri Alma

1.4 Temsil Etme

Bilgiyi Bit Desenleri Olarak Temsil Etme
Metni Temsil Etme
Sayısal Değerleri Temsil Etme
Görüntüleri Temsil Etme
Ses Temsili

*1.5İkili Sistem

İkili Notasyon
İkili Toplama
İkili Kesirler

*1.6Tam Sayıları Saklama

İki'nin Tamamlayanı Notasyonu
Aşırı Notasyon

*1.7Kesirleri Saklama

Kaydırmalı Nokta Notasyonu
Kısaltma Hataları

*1.8Veri Sıkıştırma

Genel Veri Sıkıştırma Teknikler
Görüntü Sıkıştırma
Ses ve Video Sıkıştırma

*1.9İletişim Hataları

Parite Bitleri
Hata Düzeltme Kodları

**Yıldızlar, isteğe bağlı bölümler için önerileri belirtir.*

Doclingo https://www.doclingo.ai

Bilgisayar bilimleri çalışmalarımıza, bilgilerin bilgisayarlar içinde nasıl kodlandığı v e depolandığına bakarak başlıyoruz. İlk adımımız, bir bilgisayarın veri depolama a ygıtlarının temellerini tartışmak ve ardından bilgilerin bu sistemlerde depolama iç in nasıl kodlandığını düşünmektir. Bugünün veri depolama sistemlerinin sonuçları nı ve veri sıkıştırma ve hata yönetimi gibi tekniklerin bu sistemlerin eksikliklerini a şmak için nasıl kullanıldığını keşfedeceğiz.

1.1 Bitler ve Depolamaları

Günümüz bilgisayarlarında bilgi, 0 ve 1'lerin desenleri olarak kodlanır. Bu ra kamlara bit (ikili rakamların kısaltması) denir. Bitleri sayısal değerlerle ilişkile ndirmeye eğilimli olsanız da, aslında bunlar yalnızca anlamı uygulamaya bağ lı olan sembollerdir. Bazen bit desenleri sayısal değerleri temsil etmek için k ullanılır; bazen bir alfabedeki karakterleri ve noktalama işaretlerini temsil ed er; bazen görüntüleri temsil eder; ve bazen sesleri temsil eder.

Boolean İşlemleri

Bireysel bitlerin bir bilgisayar içinde nasıl depolandığını ve işlendiğini anlama k için, bit 0'ın *yanlış* değerini ve bit 1'in *doğru* değerini temsil ettiğini hayal etmek uygundur çünkü bu, bitleri doğru/yanlış değerleri olarak manipüle et meyi düşünmemizi sağlar. Doğru/yanlış değerleri manipüle eden işlemlere, matematikçi George Boole (1815–1864) onuruna Boolean işlemleri denir; Bo ole, mantık adı verilen matematik alanında bir öncüdür. Temel Boolean işle mlerinden üçü AND, OR ve XOR (özel veya) olarak özetlenmiştir. Bu işlemler, bir çift değeri (işlemin girişi) bir üçüncü değer (çıktı) üretmek için birleştirdikl eri için çarpma ve toplama aritmetik işlemlerine benzer. Ancak aritmetik işle mlerinin aksine, Boolean işlemleri sayısal değerler yerine doğru/yanlış değer lerini birleştirir.

Boolean işlemi AND, iki daha küçük veya daha basit ifadeyi ve bağlacı ile birleştirerek oluşturulan bir ifadenin doğruluğunu veya yanlışlığını yansıtaca k şekilde tasarlanmıştır. Bu tür ifadelerin genel formu

$$P \text{ VE } Q$$

burada P bir ifadeyi ve Q başka bir ifadeyi temsil eder—örneğin,

Kermit bir kurbağa ve Miss Piggy bir aktris.

AND işleminin girdileri, bileşik ifadenin bileşenlerinin doğru veya yanlış olduğunu temsi l eder; çıktı, bileşik ifadenin kendisinin doğru veya yanlış olduğunu temsil eder. $P \text{AND} Q$ biçimindeki bir ifadenin yalnızca her iki bileşeni de doğru olduğunda doğru olduğunu bi ldiğimizden, 1 AND 1'in 1 olması gerektiği sonucuna varıyoruz, diğer tüm durumların is e 0 çıktısı vermesi gerektiği, Şekil 1.1 ile uyumludur.

Benzer bir şekilde, OR işlemi, aşağıdaki biçimdeki bileşik ifadeler üzerine ku ruludur

$$P \text{ VEYA } Q$$

Doclingo https://www.doclingo.ai

burada, tekrar, P bir ifadeyi ve Q başka bir ifadeyi temsil eder. Bu tür ifadeler , bileşenlerinden en az biri doğru olduğunda doğrudur; bu, Şekil 1.1'de göst erilen OR işlemi ile uyumludur.

İngilizce dilinde XOR işleminin anlamını yakalayan tek bir bağlaç yoktur. XOR, girişlerinden biri 1 (doğru) ve diğeri 0 (yanlış) olduğunda 1 (doğru) çıktı sı üretir. Örneğin, PXOR Q biçimindeki bir ifade "ya P ya da Q ama ikisi birden değil." anlamına gelir. (Kısacası, XOR işlemi, girişleri farklı olduğunda 1 çıktıs ı üretir.)

NOT işlemi, başka bir Boolean işlemdir. AND,OR ve XOR'dan farklıdır çün kü yalnızca bir girişi vardır. Çıkışı, ogirişin tersidir; eğer NOT işleminin girişi d oğruysa, çıkış yanlış olur vetersi de geçerlidir. Böylece, NOT işleminin girişi, if adenin doğruluğu veya yanlışlığı ise

Fozzie bir ayıdır.

o zaman çıkış, ifadenin doğruluğunu veya yanlışlığını temsil eder

Fozzie bir ayı değildir.

Kapılar ve Flip-Flop'lar

Bir Boolean işleminin çıkışını, işlemin giriş değerleri verildiğinde üreten bir ci haza kapı denir. Kapılar, dişliler, röleler ve optik cihazlar gibi çeşitli teknolojil erden inşa edilebilir. Günümüz bilgisayarlarının içinde,kapılar genellikle 0 ve 1'in voltaj seviyeleri olarak temsil edildiği küçük elektronik devreler olarak uy gulanır. Ancak, böyle detaylarla ilgilenmemize gerek yok. Amacımız için, kapı ları sembolik

Şekil **1.1** Boolean işlemleri AND, OR ve XOR (özel veya)

VE işlemi

0 VE 0 0	0 VE 1 0	1 VE 0 0	1 VE 1 1

VE işlemi

0 VE 0 0	0 VE 1 1	1 VE 0 1	1 VE 1 1

XOR işlemi

0 ÖZGÜR 0 0	0 ÖZGÜR 1 1	1 ÖZGÜR 0 1	1 ÖZGÜR 1 0

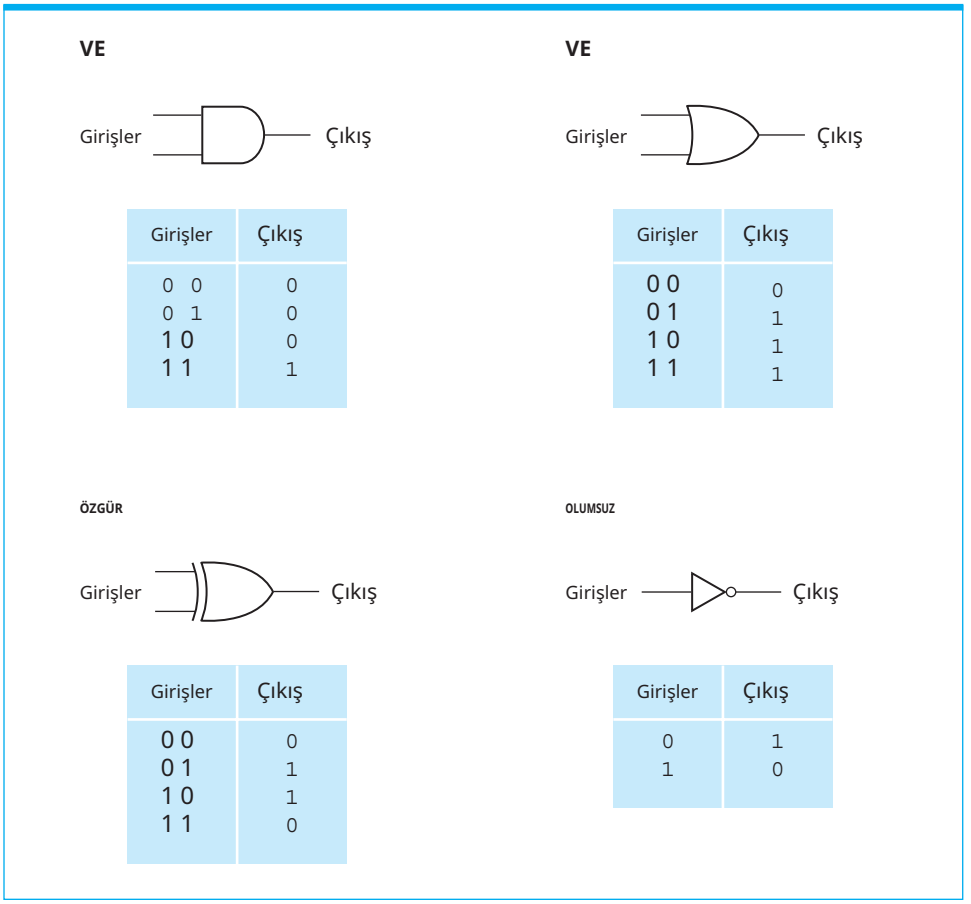
Doclingo https://www.doclingo.ai

şekilde, Şekil 1.2'de gösterildiği gibi. AND, OR, XOR ve NOT kapılarının belirgin şekilde şekillendirilmiş sembollerle temsil edildiğini unutmayın; giriş değerleri bir taraftan girerken, çıkış diğer taraftan çıkmaktadır.

Kapılar, bilgisayarların inşa edildiği yapı taşlarını sağlar. Bu yöndeki önemli bir adım, Şekil 1.3'teki devrede gösterilmektedir. Bu, flip-flop olarak bilinen devreler koleksiyonundan özel bir örnektir. Flip-flop, 0 veya 1 değerini üreten bir devredir ve başka bir devreden gelen bir darbe (0'a d önen geçici bir 1 değişikliği) onu diğer değere kaydırana kadar sabit kalır. Di ğer bir deyişle, çıkış, dış uyarıcıların kontrolü altında iki değer arasında değiş ecektir. Şekil 1.3'teki devredeki her iki giriş 0 kaldığı sürece, çıkış (0 veya 1 ol up olmadığı) değişmeyecektir. Ancak, üst girişe geçici olarak 1 yerleştirmek ç ıkışı 1 yapacakken, alt girişe geçici olarak 1 yerleştirmek çıkışı 0 yapacaktır.

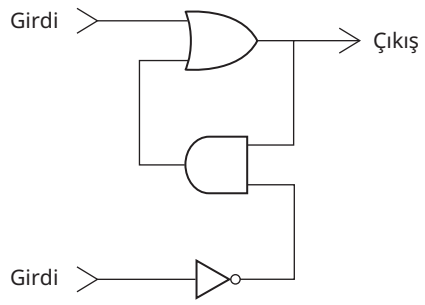
Bu iddiayı daha ayrıntılı olarak ele alalım. Şekil 1.3'teki devrenin mevcut ç ıkışını bilmeden, üst girişin 1'e değiştirildiğini ve alt girişin 0 kaldığını varsaya lım (Şekil 1.4a). Bu, OR kapısının çıkışını 1 yapacaktır, bu kapıya gelen diğer g irişten bağımsız olarak. Dolayısıyla, AND kapısına gelen her iki giriş de artık 1 olacaktır, çünkü bu kapıya gelen diğer giriş zaten 1'dir (flip-flop'un alt girişi 0 olduğunda NOT kapısı tarafından üretilen çıkış). AND kapısının çıkışı 1 olaca k ve bu da ikinci girişin

Şekil **1.2**AND, OR, XOR ve NOT kapılarının yanı sıra giriş ve çıkış değerlerinin resims el temsili



Doclingo https://www.doclingo.ai

Şekil **1.3** Basit bir flip-flop devresi

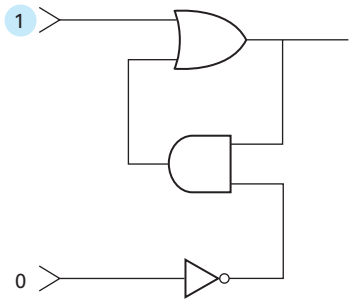


OR kapısı artık 1 olacak (Şekil 1.4b). Bu, flip-flop'un çıkışının 1 olarak kalması nı garanti eder, üst giriş 0'a geri döndüğünde bile (Şekil 1.4c). Özetle, flip-flo p'un çıkışı 1 olmuştur ve bu çıkış değeri, üst giriş 0'a döndüğünde bile kalaca ktır.

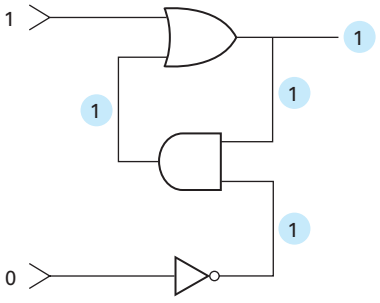
Benzer bir şekilde, alt girişe geçici olarak 1 değeri yerleştirmek, flip-flop' un çıkışını 0 yapacak ve bu çıkış, giriş değeri 0'a döndüğünde devam edecekt ir.

Şekil **1.4** Bir flip-flop'un çıkışını 1'e ayarlama

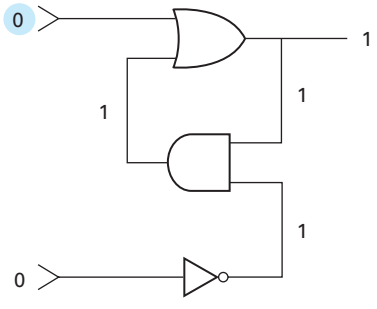
a. Üst girişe 1 yerleştirilir.



b. Bu, OR kapısının çıkışının 1 olmasına ve dolayısıyla AND kapısının çıkışının 1 olmasına neden olur.



c. AND kapısından gelen 1, üst giriş 0'a döndüğünd e OR kapısının değişmesini engeller.



Doclingo https://www.doclingo.ai

Şekil 1.3 ve 1.4'teki flip-flop devresini tanıtma amacımız üç yönlüdür. İlk ola rak, cihazların kapılardan nasıl inşa edilebileceğini gösterir, bu süreç dijital devr e tasarımı olarak bilinir ve bilgisayar mühendisliğinde önemli bir konudur. Gerç ekten de, flip-flop, bilgisayar mühendisliğinde temel araçlar olan birçok devred en sadece biridir.

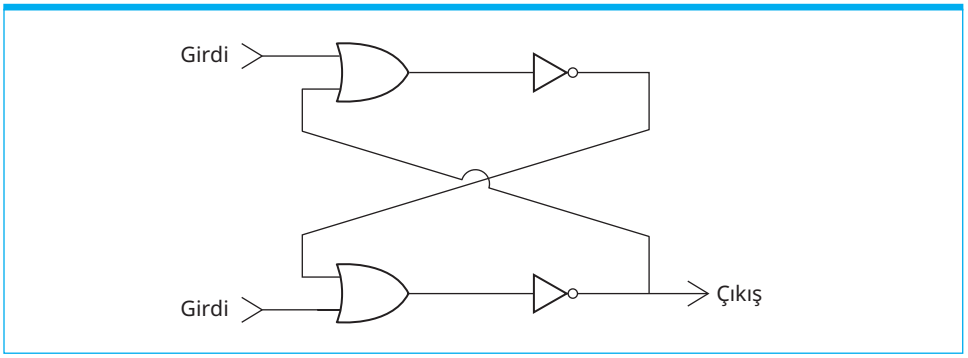
İkincisi, flip-flop kavramı, soyutlama ve soyut araçların kullanımına bir ör nek sağlar. Aslında, flip-flop inşa etmenin başka yolları da vardır. Bir alternati f Şekil 1.5'te gösterilmiştir. Bu devre ile deneme yaparsanız, farklı bir iç yapıy a sahip olmasına rağmen, dış özelliklerinin Şekil 1.3'teki ile aynı olduğunu gö receksiniz. Bir bilgisayar mühendisi, flip-flop içinde hangi devrenin kullanıldı ğını bilmek zorunda değildir. Bunun yerine, flip-flop'un dış özelliklerini anla mak, onu soyut bir araç olarak kullanmak için yeterlidir. Flip-flop, diğer iyi ta nımlanmış devrelerle birlikte, bir mühendisinin daha karmaşık devreler inşa edebileceği bir yapı taşı seti oluşturur. Bu arada, bilgisayar devrelerinin tasar ımı hiyerarşik bir yapı alır; her seviyede, alt seviye bileşenler soyut araçlar ola rak kullanılır.

Flip-flop'un tanıtılmasının üçüncü amacı, modern bir bilgisayar içinde bir biti depolamanın bir yolu olmasıdır. Daha kesin olarak, bir flip-flop, çıkış değerinin 0 v eya 1 olmasını sağlamak için ayarlanabilir. Diğer devreler, flip-flop'un girişlerine d arbeler göndererek bu değeri ayarlayabilir ve diğer devreler, depolanan değere y anıt vererek flip-flop'un çıkışını girişleri olarak kullanabilir. Böylece, çok küçük elek trik devreleri olarak inşa edilen birçok flip-flop, 0 ve 1'lerin desenleri olarak kodlan mış bilgileri kaydetmek için bir bilgisayar içinde kullanılabilir. Gerçekten de, milyo nlarca elektrik bileşeninin bir wafer (çip olarak adlandırılan) üzerinde inşa edilmes ini sağlayan çok büyük ölçekli entegrasyon (VLSI) olarak bilinen teknoloji, milyonla rca flip-flop ile birlikte kontrol devrelerini içeren mini cihazlar oluşturmak için kull anılır. Bu çipler, bilgisayar sistemlerinin inşasında soyut araçlar olarak kullanılır. A slında, bazı durumlarda VLSI, tek bir çip üzerinde bir bütün bilgisayar sistemi oluş turmak için kullanılır.

Onaltılık Notasyon

Bir bilgisayarın iç aktivitelerini düşünürken, bazıları oldukça uzun olabilen bit desenleriyle, yani bit dizileriyle ilgilenmemiz gerekir. Uzun bir bit dizisine ge nellikle bir akış denir. Ne yazık ki, akışlar insan zihni için anlaşılması zor bir y apıdır. 101101010011 desenini yalnızca yazmak bile zahmetli ve hata yapma ya açıktır. Bu nedenle, böyle bit desenlerinin temsilini basitleştirmek için gen ellikle onaltılık olarak adlandırılan bir kısayol notasyonu kullanırız.

Şekil **1.5** Flip-flop'un başka bir yapım şekli



Doclingo https://www.doclingo.ai

Şekil **1.6** Onaltılık kodlama sistemi

Bit pattern	Hexadecimal representation
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

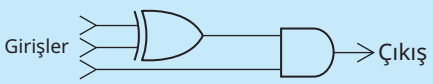
notasyon, makine içindeki bit desenlerinin genellikle dört kat uzunluklarda olm a eğiliminde olduğunu göz önünde bulundurarak. Özellikle, onaltılık notasyon dört bitlik bir deseni temsil etmek için tek bir sembol kullanır. Örneğin, on iki bi tlik bir dizi üç onaltılık sembol ile temsil edilebilir.

Şekil 1.6, onaltılık kodlama sistemini sunmaktadır. Sol sütun, dört uzunlu ğundaki tüm olası bit desenlerini gösterir; sağ sütun ise solundaki bit deseni ni temsil etmek için onaltılık notasyonda kullanılan sembolü gösterir. Bu sist emi kullanarak, 10110101 bit deseni B5 olarak temsil edilir. Bu, bit desenini d ört uzunlukta alt dizelere ayırarak ve ardından her alt diziyi onaltılık karşılığı i le temsil ederek elde edilir—1011 B ile, 0101 ise 5 ile temsil edilir. Bu şekilde, 16 bitlik 1010010011001000 deseni daha hoş bir forma A4C8 olarak indirge nebilir.

Bir sonraki bölümde onaltılık notasyonu kapsamlı bir şekilde kullanacağız. Or ada verimliliğini takdir edeceksiniz.

Sorular & Alıştırmalar

1. Aşağıdaki devrenin 1 çıkışı üretmesi için hangi giriş bit desenleri ger eklidir?

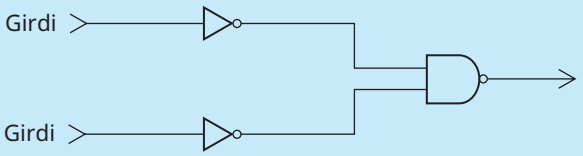


2. Metinde, Şekil 1.3'te flip-flop'un alt girişine 1 yerleştirmenin(üst girişi 0'da tutarken) flip-flop'un çıkışını 0 yapacağını iddia ettik.Bu durum da flip-flop içinde gerçekleşen olayların sırasını tanımlayın.

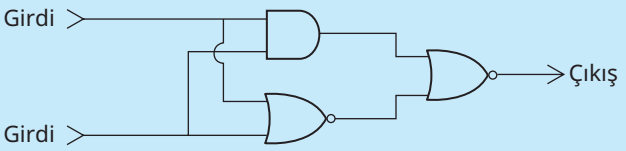
Doclingo https://www.doclingo.ai

3. Şekil 1.5'teki flip-flop'un her iki girişi 0 varsayılarak, üst giriş geçici olarak 1'e ayarlandığında gerçekleşen olayların sırasını tanımlayın.

4. a. Bir AND kapısının çıkışı bir NOT kapısından geçirilirse, bu kombinasyonNAND adı verilen Boolean işlemini hesaplar; bu işlemin çıkışı yalnızca her iki girişi 1 olduğunda 0'dır. Bir NAND kapısının sembolü, çıkışında bir daire bulunan bir AND kapısının sembolüyle aynıdır. Aşağıdaki, bir NAND kapısı içeren bir devredir. Devre hangi Boolean işlemini hesaplar?



b. Bir OR kapısının çıktısı bir NOT kapısından geçirilirse, kombinasyon, her iki girişi de 0 olduğunda yalnızca 1 çıktısı veren NOR adı verilen Boolean işlemini hesaplar. NOR kapısının sembolü, çıktısında bir daire bulunan OR kapısıyla aynıdır. Aşağıda bir AND kapısı ve iki NOR kapısı içeren bir devre bulunmaktadır. Bu devre hangi Boolean işlemini hesaplar?



5. Aşağıdaki bit desenlerini temsil etmek için onaltılık notasyon kullanın:

a. 01101010 11110010 b. 111010000 1010101000010111
c. 01001000

6. Aşağıdaki onaltılık desenler tarafından temsil edilen bit desenleri nelerdir?

a. 5FD97 b. 610A c. ABCD d. 0100

1.2 Ana Bellek

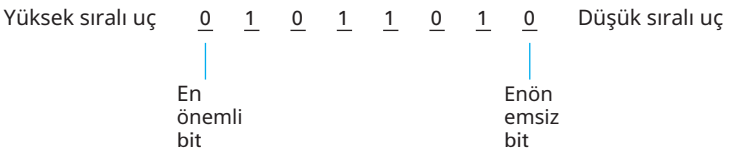
Veri depolama amacıyla, bir bilgisayar, her biri tek bir biti depolayabilen büyük bir devre koleksiyonuna(söz konusu flip-flop'lar gibi) sahiptir. Bu bit rezervuarı, makinenin ana belleği olarak bilinir.

Bellek Organizasyonu

Bir bilgisayarın ana belleği, hücreler olarak adlandırılan yönetilebilir birimlerde düzenlenmiştir;tipik bir hücre boyutu sekiz bittir. (Sekiz bitlik bir dizi byte olarak adlandırılır. Dolayısıyla, tipik bir bellek hücresinin kapasitesi bir byte'dır.) Mikrodalga fırınlar gibi ev aletlerinde kullanılan küçük bilgisayarlar,ana bellekleri yalnızca birkaç yüz hücreden oluşabilirken, büyük bilgisayarların ana belleklerinde milyarlarca hücre bulunabilir.

Doclingo https://www.doclingo.ai

Şekil **1.7** Bir bayt boyutunda bellek hücresinin organizasyonu



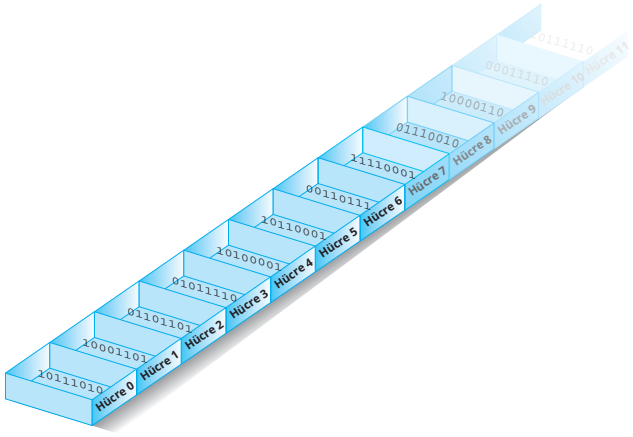
Bir bilgisayar içinde sol veya sağ yoktur, ancak genellikle bir bellek hücresi ndeki bitleri bir sırada düzenlenmiş olarak hayal ederiz. Bu sıranın sol ucu yüks ek sıralı uç olarak adlandırılır ve sağ ucu düşük sıralı uç olarak adlandırılır. En s oldaki bit, hücrenin içeriği bir sayısal değeri temsil edecek şekilde yorumlandığı nda, bu bit sayının en önemli basamağı olacağından yüksek sıralı bit veya en ö nemli bit olarak adlandırılır. Benzer şekilde, en sağdaki bit düşük sıralı bit veya en az önemli bit olarak adlandırılır. Böylece, bir bayt boyutundaki bellek hücre sinin içeriğini Şekil 1.7'de gösterildiği gibi temsil edebiliriz.

Bir bilgisayarın ana belleğindeki bireysel hücreleri tanımlamak için, her hücre yebenzersiz bir "isim" atanır, buna adres denir. Sistem, bir şehirdeki evleri adresl erle tanımlama tekniğine benzer.Ancak bellek hücreleri söz konusu olduğunda, k ullanılan adresler tamamen sayısaldır. Daha kesin olmak gerekirse, tüm hücreleri n tek bir sıraya yerleştirildiğini ve bu sıranın sıfır değerinden başlayarak numarala ndırıldığını hayal ediyoruz. Böyle bir adresleme sistemi, her hücreyi benzersiz bir şekilde tanımlamanın yanı sıra hücrelere bir sıra da atar (Şekil 1.8), bize "bir sonra ki hücre" veya "önceki hücre" gibi ifadeler verir.

Ana bellek hücrelerine ve her hücre içindeki bitlere bir sıra atamanın ön emli bir sonucu, bir bilgisayarın ana belleğindeki bitlerin tamamının esasen tek bir uzun sırada sıralanmış olmasıdır. Bu uzun sıranın parçaları, dolayısıyl a, bir hücrenin uzunluğundan daha uzun bit desenlerini depolamak için kull anılabilir. Özellikle, yalnızca iki ardışık bellek hücresini kullanarak 16 bitlik bi r dizi depolayabiliriz.

Bir bilgisayarın ana belleğini tamamlamak için, bitleri gerçekten tutan d evre, diğer devrelerin

Şekil **1.8** Adreslere göre düzenlenmiş bellek hücreleri



Doclingo https://www.doclingo.ai

verileri bellek hücrelerinden depolamak ve almak. Bu şekilde, diğer devreler belirli bir adresin içeriğini elektronik olarak talep ederek bellekten veri alabilirler (buna okuma işlemi denir) veya belirli bir bit deseninin belirli bir adresteki hücreye yerleştirilmesini isteyerek bellek içinde bilgi kaydedebilirler (buna yazma işlemi denir).

Bir bilgisayarın ana belleği, bireysel, adreslenebilir hücreler olarak düzenlendiğinden, hücrelere gerektiği gibi bağımsız olarak erişilebilir. Hücrelere herhangi bir sırayla erişim yeteneğini yansıtmak için, bir bilgisayarın ana belleğine genellikle rastgele erişim belleği (RAM) denir. Ana belleğin bu rastgele erişim özelliği, uzun bit dizilerinin birleştirilmiş bloklar olarak işlendiği kütle depolama sistemleriyle keskin bir tezat oluşturur; bunları bir sonraki bölümde tartışacağız.

Bitleri depolamak için terlikleri tanıtmış olsak da, modern bilgisayarların çoğundaki RAM, daha büyük miniaturizasyon ve daha hızlı yanıt süresi sağlayan diğer teknolojiler kullanılarak inşa edilmiştir. Bu teknolojilerin birçoğu, hızla dağılabilen küçük elektrik yükleri olarak bitleri depolar. Bu nedenle, bu cihazlar, yükleri her saniye birçok kez yenileyen bir yenileme devresi olarak bilinen ek devreler gerektirir. Bu değişkenliği tanıyarak, bu tür teknolojilerden inşa edilen bilgisayar belleğine genellikle dinamik bellek denir ve bu da Dinamik RAM anlamına gelen DRAM (DEE–ram olarak telaffuz edilir) terimine yol açar. Veya bazen, bellek hücrelerinden içerikleri almak için gereken süreyi azaltmak için ek teknikler uygulayan DRAM'a atıfta bulunmak için Senkron DRAM anlamına gelen SDRAM (ES-DEE-ram olarak telaffuz edilir) terimi kullanılır.

Bellek Kapasitesini Ölçme

Bir sonraki bölümde öğreneceğimiz gibi, ana bellek sistemlerini tasarlamak için toplam hücre sayısının iki sayısının bir kuvveti olması uygundur. Erken bilgisayarlardaki belleklerin boyutu genellikle 1024 (bu 2¹⁰) hücre birimi cinsinden ölçülmüştür. 1024, 1000 değerine yakın olduğundan, bilgisayar topluluğu bu birime atıfta bulunmak için kilo ön ekini benimsemiştir. Yani, kilobayt (kısaca KB) terimi 1024 bayta atıfta bulunmak için kullanılmıştır. Böylece, 4096 bellek hücresine sahip bir makinenin 4KB belleğe sahip olduğu söylenmiştir (4096 = 4 × 1024). Bellekler büyüdükçe, bu terminoloji MB (megabayt), GB (gigabayt) ve TB (terabayt) gibi terimleri de içerecek şekilde genişlemiştir. Ne yazık ki, kilo-, mega- gibi ön eklerin bu şekilde kullanımı, bu terimlerin zaten binin kuvvetlerine atıfta bulunmak için diğer alanlarda kullanılması nedeniyle bir terminoloji yanlış kullanımıdır. Örneğin, mesafe ölçerken, *kilometre* 1000 metreye, radyo frekanslarını ölçerken ise *megahertz* 1.000.000 hertz'e karşılık gelir. Bu nedenle, bu terminolojiyi kullanırken dikkatli olunmalıdır. Genel bir kural olarak, kilo-, *mega-* gibi terimler, bir bilgisayarın belleği bağlamında kullanıldığında iki katlarının kuvvetleri ne, diğer bağlamlarda ise binin kuvvetlerine karşılık gelir.

Sorular & Alıştırmalar

1. Eğer 5 adresli bellek hücresi 8 değerini içeriyorsa, hücre numarası 6'ya 5 değerini yazmak ile hücre numarası 5'in içeriğini hücre numarası 6'ya taşımak arasındaki fark nedir?
2. Bellek hücrelerinde 2 ve 3'te saklanan değerleri değiştirmek istiyorsanız, aşağıdaki adım dizisinde ne yanlış?

   Adım 1. Hücre numarası 2'nin içeriğini hücre numarası 3'e taşıyın.

   Adım 2. Hücre numarası 3'ün içeriğini hücre numarası 2'ye taşıyın.

Doclingo https://www.doclingo.ai

Bu hücrelerin içeriklerini doğru bir şekilde değiştiren bir adım dizisi t asarlayın. Gerekirse, ek hücreler kullanabilirsiniz.

3. 4KB belleğe sahip bir bilgisayarın belleğinde kaç bit bulunur?

1.3 Kütle Depolama

Bir bilgisayarın ana belleğinin volatilitesi ve sınırlı boyutu nedeniyle, çoğu bilgisayarın kütle de polama (veya ikincil depolama) sistemleri olarak adlandırılan ek bellek cihazları vardır; bunlar manyetik diskler, CD'ler, DVD'ler, manyetik bantlar ve flash sürücülerdir (bunların hepsini kısa süre içinde tartışacağız). Kütle depolama sistemlerinin ana belleğe göre avantajları arasında d aha az volatilite, büyük depolama kapasiteleri, düşük maliyet ve birçok durumda, depolama or tamını arşiv amaçları için makineden çıkarabilme yeteneği bulunmaktadır.

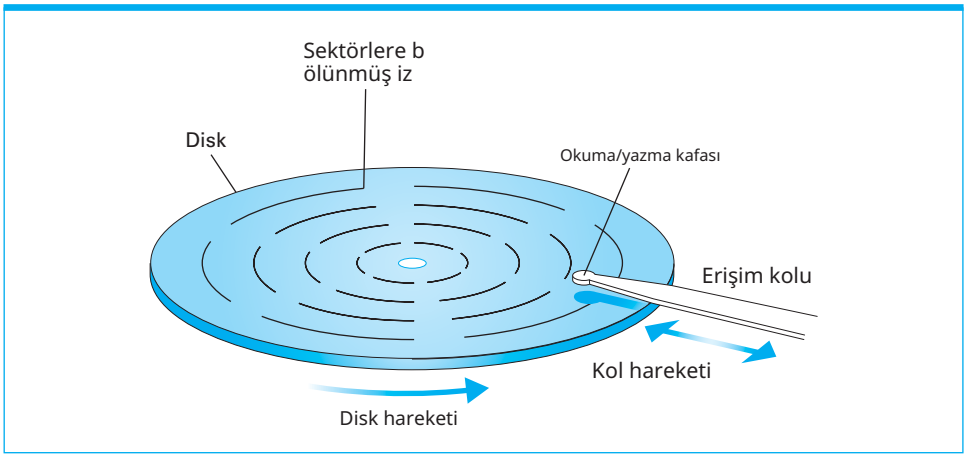
Terimler *çevrimiçi* ve *çevrimdışı* genellikle bir makineyebağlı veya bağlı olmayan cihazları tanımlamak için kullanılır. Çevrimiçi, cihazın veyabilgilerin makineye bağlı olduğu ve insan müdahalesi olmadan makineye hazır olduğu anlamına gelir. Çevrimdışı, cihazın veya bilgilere makine tarafından erişilme den önce insan müdahalesinin gerekli olduğu anlamına gelir—belki de cihazı n açılması gerektiği veya bilgileri tutan ortamın bir mekanizmaya yerleştirilm esi gerektiği için.

Kütle depolama sistemlerinin büyük bir dezavantajı, genellikle mekanik hareket gerektirmeleri ve bu nedenle verileri depolamak ve almak için makinenin ana belleği nden çok daha fazla zaman gerektirmeleridir; burada tüm işlemler elektronik olarak gerçekleştirilir.

Manyetik Sistemler

Yıllardır, manyetik teknoloji kütle depolama alanında hakim olmuştur. Bugün k ullanılan en yaygın örnek, verileri tutmak için ince bir dönen disk ile manyetik k aplama kullanılan manyetik disk tir (Şekil 1.9). Okuma/yazma başlıkları, disk dö nerken her başlığın bir çemberi, iz olarak adlandırılan bir daireyi geçmesini sa ğlamak için diskin üstünde ve/veya altında yerleştirilmiştir. Okuma/yazma başlı klarını yeniden konumlandırarak, farklı konsantrik izlere erişilebilir. Birçok duru mda, bir disk depolama sistemi, okuma/yazma başlıklarının tabakalar arasında kayabilmesi için yeterli alanla, üst üste yerleştirilmiş birkaç diskten oluşur.

Şekil **1.9** Bir disk depolama sistemi



Doclingo https://www.doclingo.ai

okuma/yazma kafaları eşzamanlı olarak hareket eder. Okuma/yazma kafaları her yeniden kon umlandırıldığında, erişilebilir hale gelen yeni bir iz seti—ki buna silindir denir—oluşur.

Bir iz, normalde bir seferde manipüle etmek istediğimizden daha fazla bi lgi içerebileceğinden, her iz, bilginin sürekli bir bit dizisi olarak kaydedildiği s ektörler adı verilen küçük yaylara bölünmüştür. Bir diskteki tüm sektörler ay nı sayıda bit içerir (tipik kapasiteler 512 bayt ile birkaç KB arasında değişir) ve en basit disk depolama sistemlerinde her iz aynı sayıda sektöre sahiptir. Böy lece, diskin dış kenarına yakın bir izdeki bitler, iç kısımdaki izlere göre daha a z yoğun bir şekilde depolanır, çünkü dış izler iç olanlardan daha uzundur. Asl ında, yüksek kapasiteli disk depolama sistemlerinde, dış kenara yakın izler, merkeze yakın olanlardan önemli ölçüde daha fazla sektör içerebilir ve bu ye tenek genellikle zoned-bit recording adı verilen bir teknikle kullanılır. Zoned-bit recording kullanarak, birkaç bitişik iz, tipik bir diskte yaklaşık on zona sahi p olan bölgeler olarak bilinir. Bir bölgede bulunan tüm izler aynı sayıda sektö re sahiptir, ancak her bölge, içindeki bölgeden daha fazla sektör içerir.Bu şek ilde, diskin tüm yüzeyinin verimli kullanımı sağlanır. Detaylardan bağımsız ol arak, bir disk depolama sistemi, her biri bağımsız bir bit dizisi olarak erişilebil en birçok bireysel sektörden oluşur.

Yolların ve sektörlerin konumu, bir diskin fiziksel yapısının kalıcı bir parça sı değildir. Bunun yerine, diski biçimlendirme (veya başlatma) adı verilen bir süreçle manyetik olarak işaretlenir. Bu işlem genellikle diskin üreticisi tarafın dan gerçekleştirilir ve biçimlendirilmiş diskler olarak bilinen sonuçlar doğuru r. Çoğu bilgisayar sistemi de bu görevi yerine getirebilir. Dolayısıyla, bir diski n biçim bilgisi hasar görürse, disk yeniden biçimlendirilebilir, ancak bu işlem daha önce diske kaydedilmiş tüm bilgileri yok eder.

Bir disk depolama sisteminin kapasitesi, kullanılan plaka sayısına ve yolla rın ve sektörlerin yerleştirildiği yoğunluğa bağlıdır. Daha düşük kapasiteli sis temler tek bir plakadan oluşabilir. Birçok gigabayt veya hatta terabayt tutabil en yüksek kapasiteli disk sistemleri, muhtemelen ortak bir mil üzerine mont e edilmiş üç ila altı plakadan oluşur. Ayrıca, veriler her plakanın üst ve alt yüz eylerinde depolanabilir.

Bir disk sisteminin performansını değerlendirmek için birkaç ölçüm kulla nılır: (1) arama süresi (okuma/yazma kafalarının bir yoldan diğerine hareket etmesi için gereken süre); (2) döngü gecikmesi veya gecikme süresi (diskin ta m bir dönüş yapması için gereken sürenin yarısı, bu, kafanın istenen yolun ü zerine yerleştirildiği anda istenen verinin okuma/yazma kafasına ulaşması içi n gereken ortalama süre); (3) erişim süresi (arama süresi ve döngü gecikmes inin toplamı); ve (4) transfer hızı (verilerin diske veya diskten transfer edilebil eceği hız). (Bölge-bit kaydı durumunda, bir disk dönüşünde bir okuma/yazm a kafasından geçen veri miktarının dış bölgelerde iç bölgelere göre daha fazl a olduğunu ve bu nedenle veri transfer hızının kullanılan diskin bölümüne ba ğlı olarak değiştiğini unutmayın.)

Erişim süresi ve transfer hızını sınırlayan bir faktör, bir disk sisteminin dö ndüğü hızdır. Hızlı döngü hızlarını kolaylaştırmak için, bu sistemlerdeki oku ma/yazma başlıkları diske dokunmaz, bunun yerine yüzeyin hemen üzerinde "yüzer". Boşluk o kadar yakındır ki, tek bir toz tanesi bile başlık ile disk yüzey i arasında sıkışabilir ve her ikisini de yok edebilir (bu olaya başlık çökmesi de nir). Bu nedenle, disk sistemleri genellikle fabrikada kapatılan kasalarda yer alır. Bu yapı ile disk sistemleri,

Doclingo https://www.doclingo.ai

birkaç bin kez saniyede dönebilir ve transfer hızları MB cinsinden ölçülür
.

Disk sistemleri, çalışmaları için fiziksel harekete ihtiyaç duyduğundan, bu sistemler elektronik devrelerdeki hızlarla karşılaştırıldığında dezavantajlıdır. Elektronik bir devredeki gecikme süreleri, nanosecond (saniyenin milyarda biri) veya daha az birimlerle ölçülürken, disk sistemlerinin arama süreleri, gecikme süreleri ve erişim süreleri milisaniye (saniyenin binde biri) cinsinden ölçülmektedir. Bu nedenle, bir disk sistemden bilgi almak için gereken süre, bir sonucu bekleyen bir elektronik devre için sonsuz bir zaman gibi görünebilir.

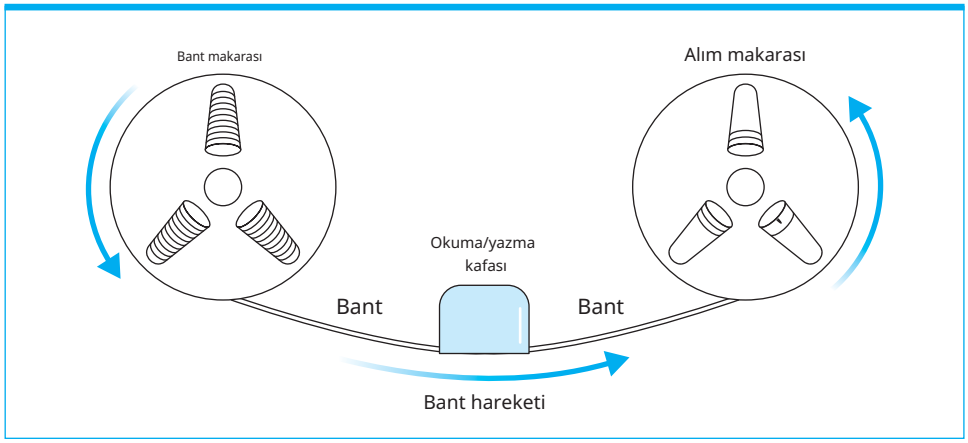
Disk depolama sistemleri, manyetik teknolojiyi uygulayan tek kütle depolama cihazları değildir. Manyetik teknolojiyi kullanan daha eski bir kütle depolama biçimi, manyetik banttır (Şekil 1.10). Bu sistemlerde, bilgi, depolama için bir makaraya sarılmış ince bir plastik bantın manyetik kaplamasına kaydedilir. Verilere erişmek için bant, genellikle bilgisayarın kontrolü altında bantı okuyabilen, yazabilen ve geri sarabilen bir bant sürücüsüne takılır. Bant sürücüleri, stereo sistemlerinde görülen bantlara benzer bant kullanan, akış bant üniteleri olarak adlandırılan küçük kartuş birimlerinden, daha eski büyük makaralı ünitelerine kadar değişen boyutlarda olabilir. Bu cihazların kapasitesi kullanılan formata bağlı olsa da, çoğu birçok GB depolayabilir.

Bantın büyük bir dezavantajı, bir bant üzerindeki farklı konumlar arasında hareket etmenin, makaralar arasında taşınması gereken bant miktarı nedeniyle çok zaman alıcı olabilmesidir. Bu nedenle, bant sistemlerinin veri erişim süreleri, farklı sektörlerin okuma/yazma kafasının kısa hareketleriyle erişilebildiği manyetik disk sistemlerine göre çok daha uzundur. Bu da bant sistemlerinin çevrimiçi veri depolama için popüler olmamasına neden olmaktadır. Bunun yerine, manyetik bant teknolojisi, yüksek kapasitesi, güvenilirliği ve maliyet etkinliği nedeniyle çevrimdışı arşiv veri depolama uygulamaları için ayrılmıştır, ancak DVD'ler ve flash sürücüler gibi alternatiflerdeki ilerlemeler, manyetik bantın bu son kalıntısını hızla tehdit etmektedir.

Optik Sistemler

Bir diğer kütle depolama sistemi sınıfı optik teknolojiyi uygular. Bir örnek, kompakt disk (CD)'dir. Bu diskler 12 santimetre (yaklaşık 5 inç) çapındadır ve şeffaf bir koruyucu kaplama ile kaplanmış yansıtıcı bir malzemeden oluşur. Bilgi, yansıtıcı yüzeylerinde değişiklikler oluşturarak kaydedilir.

Şekil 1.10 Manyetik bant depolama mekanizması



Doclingo https://www.doclingo.ai

Bu bilgiler, CD dönerken yansıtıcı yüzeydeki düzensizlikleri tespit eden bir laz er ışını aracılığıyla geri alınabilir.

CD teknolojisi, başlangıçta CD-DA (kompakt disk-dijital ses) olarak bilinen bir k ayıt formatı kullanarak ses kayıtlarına uygulanmıştır ve günümüzde bilgisayar veri depolama için kullanılan CD'ler esasen aynı formatı kullanmaktadır. Özellikle, bu CD'lerdeki bilgiler, eski tarz bir kayıtta olduğu gibi CD etrafında spiral şeklinde dö nen tek bir yolda depolanır; ancak, eski tarz kayıtların aksine, bir CD'deki yol içten dışa doğru spiral şekildedir (Şekil 1.11). Bu yol, her birinin kendine özgü tanımlayı cı işaretleri ve 2KB veri kapasitesine sahip olan sektörler adı verilen birimlere bölü nmüştür; bu, ses kayıtları durumunda 1/75 saniye müziğe eşdeğerdir.

Spiral yolun etrafındaki mesafenin, diskin iç kısmında olduğundan dış ke narında daha büyük olduğunu unutmayın. Bir CD'nin kapasitesini maksimize etmek için, bilgiler tüm spiral yol boyunca uniform bir lineer yoğunlukta dep olanır; bu, spiralın dış kısmında bir döngü etrafında daha fazla bilgi depolan dığı anlamına gelir. Bu durumda, lazer ışını spiral yolun dış kısmını tararken, diskin tek bir devrinde daha fazla sektör okunacaktır; iç kısmı tararken olduğ undan daha fazla sektör okunacaktır. Böylece, veri transferinin uniform bir h ızını elde etmek için, CD-DA oynatıcıları lazer ışınının konumuna bağlı olarak döndürme hızını değiştirecek şekilde tasarlanmıştır.

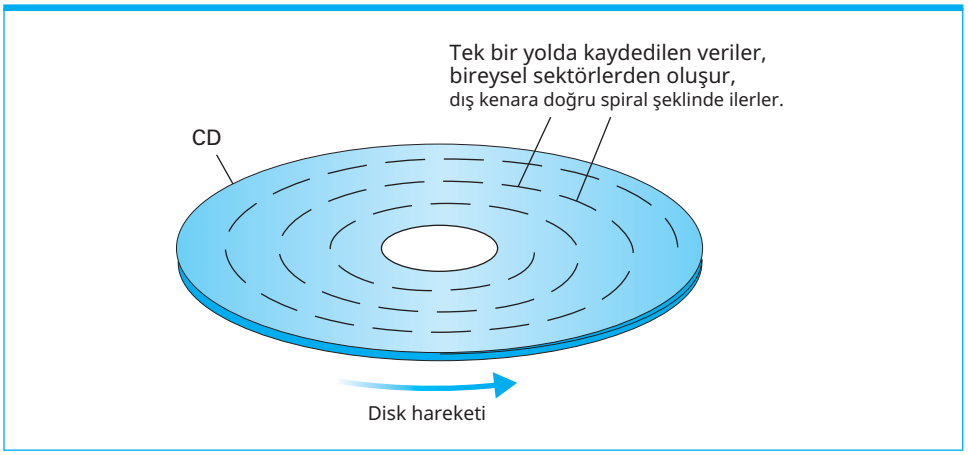
Ancak, bilgisayar veri depolama için kullanılan çoğu CD sistemi daha hızlı, sabit bir hızda d öner ve bu nedenle veri transfer hızlarındaki değişiklikleri karşılamak zorundadır.

Bu tür tasarım kararlarının bir sonucu olarak, CD depolama sistemleri en iyi uzun, s ürekli veri dizileri ile başa çıktıklarında performans gösterir, müzik yeniden üretimi gibi. Buna karşılık, bir uygulama veriye rastgele erişim gerektirdiğinde, manyetik disk depola ma (bireysel, konsantrik yolların bireysel olarak erişilebilir sektörlere bölündüğü) yaklaşı mı, CD'lerde kullanılan spiral yaklaşımından daha iyi performans gösterir.

Geleneksel CD'lerin kapasitesi 600 ile 700MB arasında değişmektedir. Ancak, DVD'ler (Dijital Çok Amaçlı Diskler), tam olarak odaklanmış bir lazerle görüntülend iğinde farklı yüzeyler olarak hizmet eden birden fazla, yarı saydam katmandan olu şur ve birkaç GB depolama kapasitesi sağlar. Bu tür diskler, tüm hareketli resimle ri içeren uzun multimedya sunumlarını depolama kapasitesine sahiptir.

Son olarak, mavi-mor ışık spektrumunda bir lazer kullanan Blu-ray teknolojis i (kırmızı yerine), lazer ışınını çok ince bir hassasiyetle odaklayabilir. Bir

Şekil **1.11** CD depolama formatı



Doclingo https://www.doclingo.ai

sonuç, BDs (Blu-ray Diskleri) bir DVD'nin kapasitesinin beş katından fazlasını sağlar. Bu görünüşte büyük depolama miktarı, yüksek tanımlı video taleplerini karşılama k için gereklidir.

Flash Sürücüler

Manyetik veya optik teknolojiye dayanan kütle depolama sistemlerinin ortak bir özelliği, verileri depolamak ve geri almak için fiziksel hareketin, örneğin dönen diskler, hareket eden okuma/yazma kafaları ve lazer ışınlarını hedefleme gerektirmesidir. Bu, veri depol ama ve geri almanın, elektronik devrelerin hızına kıyasla yavaş olduğu anlamına gelir. Flash bellek teknolojisi bu dezavantajı hafifletme potansiyeline sahiptir. Bir flash b ellek sisteminde, bitler, elektronik sinyalleri doğrudan depolama ortamına gönder erek depolanır; bu, elektronların silikon dioksit küçük odacıklarında hapsolmasına neden olur ve böylece küçük elektronik devrelerin özelliklerini değiştirir. Bu odacı klar, hapsolmuş elektronlarını yıllarca tutabildiğinden, bu teknoloji veri için çevrim dışı depolama için uygundur.

Flash bellek sistemlerinde depolanan verilere, RAM uygulamalarındaki gibi küçük byte boyutlu birimlerde erişilebilse de, mevcut teknoloji, depolanan veril erin büyük bloklar halinde silinmesini gerektirir. Ayrıca, tekrar eden silme işlem leri silikon dioksit odacıklarına yavaşça zarar verir; bu da mevcut flash bellek te knolojisinin, içeriğinin saniyede birçok kez değiştirilebileceği genel ana bellek u ygulamaları için uygun olmadığı anlamına gelir. Ancak, değişikliklerin makul bir seviyede kontrol edilebildiği uygulamalarda, örneğin dijital kameralar, cep tele fonları ve el tipi PDA'lar gibi, flash bellek tercih edilen kütle depolama teknoloji si haline gelmiştir. Gerçekten de, flash bellek fiziksel şoka duyarlı olmadığından (manyetik ve optik sistemlerin aksine), taşınabilir uygulamalardaki potansiyeli cazip hale gelmektedir.

Flash bellek cihazları, genel kütle depolama uygulamaları için mevcut olan flash sürücüler, birkaç yüz GB'a kadar kapasitelere sahiptir. Bu birimler, sürücü nün çevrimdışı olduğu zaman birimin elektrik bağlantısını korumak için bir ucu nda çıkarılabilir bir kapak bulunan yaklaşık üç inç uzunluğunda küçük plastik ku tularda paketlenmiştir. Bu taşınabilir birimlerin yüksek kapasitesi ve bilgisayara kolayca bağlanıp ayrılabilmesi, onları çevrimdışı veri depolama için ideal hale g etirir. Ancak, küçük depolama odacıklarının hassasiyeti, bunların gerçekten uzu n vadeli uygulamalar için optik diskler kadar güvenilir olmadığını belirler.

Flash teknolojisinin bir diğer uygulaması, SD (Güvenli Dijital) bellek kartları (vey a sadece SD Kart) olarak bulunur. Bu kartlar, iki GB'a kadar depolama sağlar ve bir p osta pulu boyutunda plastik bir yapı içinde paketlenmiştir (SD kartlar ayrıca daha küç ük mini ve mikro boyutlarda da mevcuttur), SDHC (Yüksek Kapasite) bellek kartları 3 2 GB'a kadar depolama sağlayabilir ve bir sonraki nesil SDXC (Genişletilmiş Kapasite) bellek kartları bir TB'yi aşabilir. Kompakt fiziksel boyutları göz önüne alındığında, b u kartlar küçük elektronik cihazların yuvalarına rahatça sığar. Böylece, dijital kameral ar, akıllı telefonlar, müzik çalarlar, araç navigasyon sistemleri ve diğer birçok elektro nik cihaz için idealdir.

Dosya Depolama ve Geri Alma

Bir kütle depolama sisteminde saklanan bilgiler, kavramsal olarak büyükbirimlere, yani do syalara gruplandırılır. Tipik bir dosya, tam bir metin belgesi, bir fotoğraf,bir program, bir müzik kaydı veya bir şirketteki çalışanlar hakkında veri koleksiyonuşeklinde olabilir.

Doclingo https://www.doclingo.ai

Kütle depolama cihazlarının, bu dosyaların daha küçük, çoklu bayt birimlerinde saklanmasını ve geri alınmasını zorunlu kıldığını gördük. Örneğin, manyetik bir diskte saklanan bir dosya, her biri sabit bir önceden belirlenmiş boyutta olan sektörler aracılığıyla işlenmelidir. Bir depolama cihazının belirli özelliklerine uyan bir veri bloğuna *fiziksel kayıt* denir. Böylece, kütle depolamada saklanan büyük bir dosya, genellikle birçok fiziksel kayıttan oluşur.

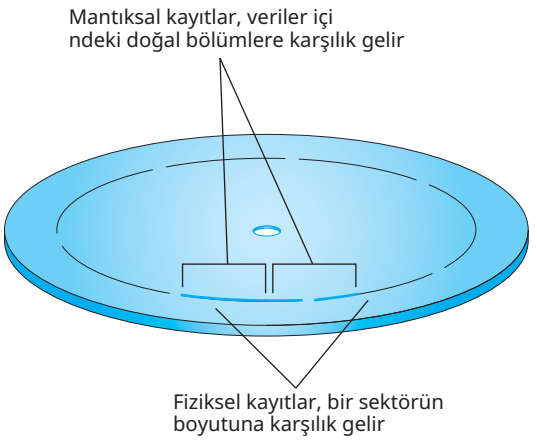
Bu fiziksel kayıtlara yapılan bu bölünmenin aksine, bir dosya genellikle temsil edilen bilgi tarafından belirlenen doğal bölümlere sahiptir. Örneğin, bir şirketin çalışanlarıyla ilgili bilgileri içeren bir dosya, her biri bir çalışana ait bilgileri içeren birden fazla birimden oluşur. Veya, bir metin belgesi içeren bir dosya, paragraflardan veya sayfalardan oluşur. Bu doğal olarak oluşan veri bloklarına *mantıksal kayıtlar* denir.

Mantıksal kayıtlar genellikle *alanlar* olarak adlandırılan daha küçük birimle rdenoluşur. Örneğin, bir çalışana ait bilgileri içeren bir mantıksal kayıt muhtem elenisim, adres, çalışan kimlik numarası gibi alanlardan oluşur. Bazen bir dosya içindekiher mantıksal kayıt, kayıttaki belirli bir alan aracılığıyla benzersiz bir şe kilde tanımlanır(belki bir çalışanın kimlik numarası, bir parça numarası veya bir katalog öğesi numarası).Böyle bir tanımlayıcı alana *anahtar alan* denir. Bir ana htar alanında tutulan değere *anahtar* denir.

Mantıksal kayıt boyutları, bir kütledepolama cihazı tarafından belirlenen fizi ksel kayıt boyutlarıyla nadiren eşleşir. Bunun sonucunda, bir fiziksel kayıtta bir den fazla mantıksal kaydın bulunması veya belki de bir mantıksal kaydın iki vey a daha fazla fiziksel kayda bölünmesi mümkündür (Şekil 1.12). Sonuç olarak, kü tle depolama sistemlerinden veri alırken belirli bir miktar çözme işlemi ile ilişkili dir. Bu sorunun yaygın bir çözümü, birkaç fiziksel kaydı tutacak kadar büyük bir ana bellek alanı ayırmak ve bu bellek alanını yeniden gruplama alanı olarak kul lanmaktır. Yani, fiziksel kayıtlara uyumlu veri blokları, bu ana bellek alanı ile küt le depolama sistemi arasında aktarılabilirken, ana bellek alanında bulunan veril er mantıksal kayıtlar açısından referans alınabilir.

Bu şekilde kullanılan bir bellek alanına *tampon* denir. Genel olarak, bir t ampon, verileri geçici olarak tutmak için kullanılan bir depolama alanıdır; ge nellikle bir cihazdan diğerine aktarım sürecinde kullanılır. Örneğin, modern

Şekil **1.12** Diskte mantıksal kayıtlar ile fiziksel kayıtlar arasındaki karşılaştırma



Doclingo https://www.doclingo.ai

Yazıcılar, kendi bellek devrelerine sahiptir ve bunların büyük bir kısmı, yazıcı tarafınd an alınan ancak henüz yazdırılmamış bir belgenin bölümlerini tutmak için birönbelle k olarak kullanılır.

Sorular & Alıştırmalar

1. Bir disk veya CD'nin döndürme hızını artırmanın ne gibi faydaları vardır?
2. Birden fazla disk depolama sisteminde veri kaydederken, başka bir yüzeye b aşlamadan önce birkomple disk yüzeyini mi doldurmalıyız, yoksa başka bir s ilindire başlamadan önce birsilindiri mi tamamen doldurmalıyız?
3. Güncellenen bir rezervasyon sistemindeki verilerin neden bir CD vey a DVD yerine manyetik bir diskte saklanması gerekir?
4. Bazen, bir kelime işlemci ile bir belgeyi değiştirirken, metin eklemek dosyanın kütük depolamada görünür boyutunu artırmaz, ancak diğe r zamanlarda tek bir sembol eklenmesi dosyanın görünür boyutunu birkaç yüz bayt artırabilir. Neden?
5. Flash sürücülerin bu bölümde tanıtılan diğer kütük depolama sistemlerin e göre ne avantajı vardır?
6. Önbellek nedir?

1.4 Bilgiyi Bit Desenleri Olarak Temsil Etme

Bitleri depolama tekniklerini göz önünde bulundurduktan sonra, bilgilerin bit desenleri olarak nasıl kodlanabileceğini ele alıyoruz. Çalışmamız, metin, sayısal veriler, görüntüler ve ses kodlama için popüler yöntemlere odaklanmaktadır. Bu sistemlerin her birinin, ti pik bir bilgisayar kullanıcısı tarafından genellikle görülebilen sonuçları vardır. Amacımız, bu teknikler hakkında yeterince bilgi edinmek ve sonuçlarını ne oldukları için tanıyabil mektir.

Metni Temsil Etme

Metin biçimindeki bilgiler, metindeki farklı sembollerin (alfabenin harfleri ve noktalama işaretleri gibi) her birine benzersiz bir bit deseni atandığı bir kod aracılığıyla genellikle temsil edilir. Metin, ardından orijinal metindeki ardışık sembolleri temsil eden ardışık desenlerin bulunduğu uzun bir bit dizisi olara k temsil edilir.

1940'lar ve 1950'lerde, bu tür birçok kod, farklı ekipmanlarla bağlantılı ol arak tasarlandı ve kullanıldı ve buna bağlı olarak iletişim sorunlarının çoğalm asına neden oldu. Bu durumu hafifletmek için, Amerikan

Ulusal Standartlar Enstitüsü (ANSI, “AN–see” olarak telaffuz edilir) Amerikan Bilgi Değişim Standart Kodunu (ASCII, “A S–kee” olarak telaffuz edilir) benimsedi. Bu kod, İngiliz alfabesinin büyük ve küçük harflerini, noktalama sembollerini, 0'dan 9'a kadar olan rakamları ve satır sonları, taşıyıcı dönüşleri ve sekmeler gibi belirli kontrol bilgilerini temsil etme k için yedi bit uzunluğunda bit desenleri kullanır. ASCII, her bir yedi bitlik desenin en anlamlı ucuna 0 ekleyerek sekiz bitlik bir sembol formatına genişletilir. Bu teknik, yalnızca her desenin tipik bir bayt boyutundaki bellek hücresine uy gun bir şekilde sığmasını sağlamakla kalmaz, aynı zamanda İngiliz alfabesi ve ilgili noktalama işaretlerinin ötesindeki sembolleri temsil etmek için kullanılabilecek 128 ek bit deseni (ek bitin 1 değerini aldığı desenler) sağlar.

Doclingo https://www.doclingo.ai

Her desenin tipik bir bayt boyutundaki bellek hücresine uygun bir şekilde sığm asını sağlamakla kalmaz, aynı zamanda İngiliz alfabesi ve ilgili noktalama işaret lerinin ötesindeki sembolleri temsil etmek için kullanılabilecek 128 ek bit desen i (ek bitin 1 değerini aldığı desenler) sağlar.

ASCII'nin sekiz bitlik sembol formatındaki bir kısmı Ek A'da gösterilmiştir. Bu eki referans alarak, bit desenini çözebiliriz

```
01001000  01100101  01101100  01101100  01101111  00101110
```

“Merhaba.” mesajı olarak, Şekil 1.13'te gösterildiği gibi.

Uluslararası Standartlar Örgütü (ISO olarak da bilinir, Yunan kelimesi iso s'tan, eşit anlamına gelir) birçok ASCII uzantısı geliştirmiştir; bunların her biri büyük bir dil grubunu karşılamak üzere tasarlanmıştır. Örneğin, bir standart , çoğu Batı Avrupa dillerinin metnini ifade etmek için gereken sembolleri sağ lar. 128 ek deseni arasında, İngiliz sterlini ve Almanca ünlüler olan ä, ö ve ü i çin semboller bulunmaktadır.

ISO genişletilmiş ASCII standartları, dünyanın çok dilli iletişimini destekle me konusunda büyük bir ilerleme kaydetti; ancak iki büyük engel ortaya çıktı . İlk olarak, genişletilmiş ASCII'de mevcut olan ek bit desenlerinin sayısı, birç ok Asya ve bazı Doğu Avrupa dillerinin alfabesini karşılamak için yetersizdir. İ kincisi, belirli bir belge yalnızca seçilen bir standartta semboller kullanmakla sınırlı olduğundan, farklı dil gruplarından dillerin metnini içeren belgeler des teklenememektedir.

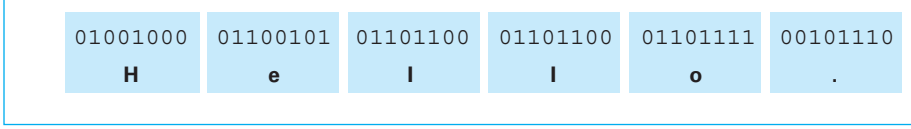
Her ikisi de uluslararası kullanım için önemli bir engel teşkil etti. Bu eksikliği gidermek için, Unicode, donanım ve yazılımın önde gelen birkaç üreticisinin i şbirliği ile geliştirilmiş ve hızla bilgisayar topluluğunda destek kazanmıştır. B u kod, her sembolü temsil etmek için 16 bitlik benzersiz bir desen kullanır. S onuç olarak, Unicode 65,536 farklı bit deseninden oluşur—Çince, Japonca ve İbranice gibi dillerde yazılmış metinlerin temsil edilmesine yetecek kadar.

ASCII veya Unicode kullanılarak kodlanmış uzun bir sembol dizisinden oluşan bir d osyaya genellikle bir metin dosyası denir. Basit metin dosyaları ile metin editörleri (veya genellikle sadece editörler) olarak adlandırılan yardımcı programlar tarafından işlenen dosyalar ile Microsoft’un Word gibi kelime işlemcileri tarafından üretilen daha karmaşık dosyalar arasında ayrım yapmak önemlidir. Her ikisi de metinsel materyal içerir. Ancak, bir metin dosyası yalnızca metnin karakter karakter kodlamasını içerirken, bir kelime işl emci tarafından üretilen dosya, yazı tiplerindeki değişiklikleri, hizalama bilgilerini vb. te msil eden birçok özel kod içerir.

Sayısal Değerleri Temsil Etme

Bilgiyi kodlanmış karakterler cinsinden depolamak, kaydedilen bilgi tamamen sayısal ol duğunda verimsizdir. Nedenini görmek için, 25 değerini depolama sorununu düşüneli m. Eğer bunu ASCII'de her sembol için bir bayt kullanarak kodlanmış semboller olarak d epolamakta ısrar edersek, toplamda 16 bite ihtiyacımız var. Dahası, en büyük sayı

Şekil **1.13** ASCII'de "Merhaba." mesajı



Doclingo https://www.doclingo.ai

Amerikan Ulusal Standartlar Enstitüsü

Amerikan Ulusal Standartlar Enstitüsü (ANSI), 1918 yılında mühendislik dernekleri ve devlet aja nslarından oluşan küçük bir konsorsiyum tarafından, özel sektörde gönüllü standartların gelişti rilmesini koordine etmek amacıyla kar amacı gütmeyen bir federasyon olarak kurulmuştur. Bugün, ANSI üyeliği 1300'den fazla işletme, meslek kuruluşu, ticaret derneği ve de vlet ajansını içermektedir. ANSI'nin merkezi New York'tadır ve Amerika Birleşik Dev letleri'ni ISO'da bir üye kuruluş olarak temsil etmektedir. Amerikan Ulusal Standart lar Enstitüsü'nün web sitesi http://www.ansi.org adresindedir.

Diğer ülkelerdeki benzer kuruluşlar arasında Avustralya Standartları (Avu stralya), Kanada Standartlar Konseyi (Kanada), Çin Devlet Kalite ve Teknik Den etim Bürosu (Çin), Alman Standartlar Enstitüsü (Almanya), Japon Sanayi Stand artları Komitesi (Japonya), Meksika Genel Standartlar Dairesi (Meksika), Rusya Federasyonu Devlet Standartları ve Metrologi Komitesi (Rusya), İsviçre Stand artlar Derneği (İsviçre) ve Britanya Standartlar Enstitüsü (Birleşik Krallık) bulu nmaktadır.

16 bit kullanarak depolayabileceğimiz maksimum değer 99'dur. Ancak, kısa süre içinde göreceğimiz gibi, kullanarak ikili notasyon bu 16 bit içinde 0 ile 65535 arasındaki herhangi bir tam sayıyı saklayabiliriz. Bu nedenle, ikili notasyon (veya onun varyasyonları) bilgisayar depolaması için kodlanmış sayısal veriler için yaygın olarak kullanılmaktadır.

İkili notasyon, sayısal değerleri yalnızca 0 ve 1 rakamlarını kullanarak temsil e tmenin bir yoludur,klasik ondalık veya on tabanlı sistemde olduğu gibi 0, 1, 2, 3, 4, 5, 6, 7, 8 ve 9 rakamlarınıkullanmak yerine. İkili sistemi daha ayrıntılı olarak Bölü m 1.5'te inceleyeceğiz.Şu anda, sistemin temel bir anlayışına ihtiyacımız var. Bu amaçla, yalnızca 0 ve 1 rakamlarını içeren eski tarz bir otomobil kilometre sayacını düşünün.Kilometre sayacı, tüm 0'larla başlayan bir okumayla başlar ve araba ilk birkaç milsürülürken, en sağdaki tekerlek 0'dan 1'e döner. Sonr a, o 1 tekrar 0'a dönerken,solundaki 1'in görünmesine neden olur ve 10 dese nini üretir. Sağdaki 0 daha sonra 1'e döner,11'i üretir. Şimdi en sağdaki teker lek 1'den 0'a döner, bu da solundaki 1'in de 0'adönmesine neden olur. Bu da üçüncü sütunda başka bir 1'in görünmesine neden olur ve 100desenini üret ir. Kısacası, arabayı sürerken kilometre sayacının okumalarının şu dizisini gör ürüz:

```
0000
0001
0010
0011
0100
0101
0110
0111
1000
```

Bu dizilim, sıfırdan sekize kadar olan tam sayıların ikili temsillerinden olu şmaktadır.Sıkıcı olsa da, bu sayma tekniğini genişleterek on altı 1'den oluşan bit deseninin 65535 değerini temsil ettiğini keşfedebiliriz,

Doclingo https://www.doclingo.ai

bu da 0 ile 65535 arasındaki herhangi bir tam sayının 16 bit kullanılarak kodlanabileceği idd iamızı doğrulamaktadır.

Bu verimlilik nedeniyle, sayısal bilgilerin kodlanmış semboller yerine ikili notasyon biçiminde saklanması yaygındır. "İkili notasyon biçimi" diyoruz çün kü daha önce tanımlanan basit ikili sistemi, makinelerde kullanılan birkaç sa yısal depolama tekniğinin yalnızca temelidir. Bu ikili sistemin bazı varyasyonl arı bu bölümde daha sonra tartışılacaktır. Şu anda, tam sayıları saklamak için yaygın olarak kullanılan iki'nin tamamlayanı notasyonu (bkz. Bölüm 1.6) adı nda bir sistemin, negatif sayıları pozitif sayılarla temsil etmek için uygun bir yöntem sağladığını yalnızca belirtelim. Kesirli kısımlara sahip sayıları temsil e tmek için ise kayan nokta notasyonu (bkz. Bölüm 1.7) adında başka bir tekni k kullanılmaktadır.

Görüntüleri Temsil Etme

Bir görüntüyü temsil etmenin bir yolu, görüntüyü her biri piksel olarak adlandırıl an noktalar topluluğu olarak yorumlamaktır. Her pikselin görünümü daha sonra k odlanır ve tüm görüntü bu kodlanmış piksellerin bir koleksiyonu olarak temsil edil ir. Böyle bir koleksiyona bit haritası denir. Bu yaklaşım, yazıcılar ve ekranlar gibi b irçok görüntüleme cihazının piksel kavramı üzerinde çalışması nedeniyle popüler dir. Dolayısıyla, bit haritası biçimindeki görüntüler kolayca görüntüleme için biçim lendirilir.

Piksellerin bir bit haritasında kodlama yöntemi uygulamalara göre değişiklik gösterir. Basit bir siyah beyaz görüntü durumunda, her piksel, karşılık gelen pikselin siy ah mı yoksa beyaz mı olduğuna bağlı olarak değeri değişen tek bir bit ile temsil edilebilir. Bu, çoğu faks makinesi tarafından kullanılan yaklaşımdır. Daha karm aşık siyah beyaz fotoğraflar için, her piksel, genellikle sekiz bitlik bir koleksiyon ile temsil edilebilir; bu, çeşitli gri tonlarının temsil edilmesine olanak tanır.

Renkli görüntüler durumunda, her piksel daha karmaşık bir sistemle kodlanır. İki yaklaşım yaygındır. Birinde, RGB kodlaması olarak adlandıracağımız yönt emde, her piksel üç renk bileşeni olarak temsil edilir - bir kırmızı bileşen, bir yeşil bileşen ve bir mavi bileşen - bu da ışığın üç ana rengini karşılar. Her ren k bileşeninin yoğunluğunu temsil etmek için genellikle bir bayt kullanılır. Dol ayısıyla, orijinal görüntüdeki tek bir pikseli temsil etmek için üç bayt depola ma gereklidir.

ISO—Uluslararası Standardizasyon Örgütü

Uluslararası Standardizasyon Örgütü (daha yaygın olarak ISO olarak adlandırılır), 1947 yılında h er ülkeden bir standardizasyon kuruluşunun yer aldığı dünya çapında bir federasyon olarak kur ulmuştur. Bugün, İsviçre'nin Cenevre şehrinde merkezi bulunmaktadır ve daha fazla 100 üye kuruluşlar ve çok sayıda muhabir üye. (Bir muhabir üye genellikle ulusal olarak tanınan bir standardizasyon kuruluşuna sahip olmayan bir ülke den bir standardizasyon kuruluşudur. Bu tür üyeler standartların geliştirilmesine doğrudan katılamazlar ancak ISO faaliyetleri hakkında bilgilendirilirler.) ISO, http://www.iso.org adresi nde bir web sitesi bulundurmaktadır.

Doclingo https://www.doclingo.ai

Basit RGB kodlamasına bir alternatif, bir “parlaklık” bileşenive iki renk bil eşeni kullanmaktır. Bu durumda “parlaklık” bileşeni, pikselinluminansı olarak adlandırılır ve esasen kırmızı, yeşil ve mavibileşenlerin toplamıdır. (Aslında, pikseldeki beyaz ışık miktarı olarakkabul edilir, ancak bu detaylar burada bizi ilgilendirmemektedir.) Diğer iki bileşen,mavi krominans ve kırmızı krominan s olarak adlandırılır, pikselin luminansı ilepikseldeki mavi veya kırmızı ışık mi ktarı arasındaki farkı hesaplayarak belirlenir.Birlikte bu üç bileşen, pikseli ye niden üretmek için gereken bilgiyi içerir.

Görüntüleri aydınlatma ve renk bileşenleri kullanarak kodlama popülarit esi, bu yaklaşımın eski siyah-beyaz televizyon alıcılarıyla da uyumlu renkli gö rüntüleri kodlama imkanı sağlaması nedeniyle renkli televizyon yayıncılığı al anında ortaya çıkmıştır. Gerçekten de, bir görüntünün gri tonlamalı bir versi yonu, kodlanmış renkli görüntünün yalnızca aydınlatma bileşenleri kullanılar ak üretilebilir.

Bir görüntüyü bit haritaları olarak temsil etmenin dezavantajı, bir görüntünün herh angi bir rastgele boyuta kolayca yeniden ölçeklendirilememesidir. Temelde, görüntüyü büyütmenin tek yolu pikselleri daha büyük hale getirmektir, bu da tahriş edici bir görün ümle sonuçlanır. (Bu, dijital kameralar tarafından kullanılan “dijital zoom” tekniğidir; “op tik zoom” ise kamera lensini ayarlayarak elde edilir.)

Bu ölçeklendirme sorununu aşan alternatif bir görüntü temsil etme yolu, görünt üyü analitik geometri teknikleri kullanılarak kodlanabilen, çizgiler ve eğriler gibi geo metrik yapıların bir koleksiyonu olarak tanımlamaktır. Böyle bir tanım, nihayetinde g örüntüyü görüntüleyen cihazın geometrik yapıların nasıl görüntüleneceğine karar ve rmesine olanak tanır; cihazın belirli bir piksel desenini yeniden üretmesini zorunlu kıl mak yerine. Bu, günümüz kelime işleme sistemleri aracılığıyla mevcut olan ölçeklene bilir yazı tiplerini üretmek için kullanılan yaklaşımdır. Örneğin, TrueType (Microsoft v e Apple tarafından geliştirilmiştir) metin sembollerini geometrik olarak tanımlamak i çin bir sistemdir. Benzer şekilde, PostScript (Adobe Systems tarafından geliştirilmiştir) karakterleri ve daha genel resim verilerini tanımlamak için bir araç sağlar. Bu geom etrik görüntü temsil etme yöntemi, üç boyutlu nesnelerin çizimlerinin bilgisayar ekra nlarında görüntülendiği ve manipüle edildiği bilgisayar destekli tasarım (CAD) sistem lerinde de popülerdir.

Bir görüntüyü geometrik yapılar şeklinde temsil etme ile bit haritaları şe klinde temsil etme arasındaki ayrım, kullanıcıların Microsoft'un Paint aracı gi bi birçok çizim yazılımı sisteminde belirgindir. Bu sistemler, kullanıcının dikd örtgenler, ovalar ve temel eğriler gibi önceden belirlenmiş şekillerden oluşa n resimler çizmesine olanak tanır. Kullanıcı, menüden istenen geometrik şekl i seçer ve ardından o şeklin çizimini bir fare ile yönlendirir. Çizim süreci boyu nca yazılım, çizilen şeklin geometrik tanımını korur. Fare ile verilen yönlendir melerle, iç geometrik temsil değiştirilir, bit haritası biçimine yeniden dönüşt ürülür ve görüntülenir. Bu, görüntünün kolayca ölçeklenmesini ve şekillendir ilmesini sağlar. Ancak, çizim süreci tamamlandığında, temel geometrik tanı m atılır ve yalnızca bit haritası korunur; bu da ek değişikliklerin zahmetli bir p iksel-piksel modifikasyon süreci gerektirdiği anlamına gelir. Öte yandan, bazı çizim sistemleri tanımı geometrik şekiller olarak korur, bu da daha sonra de ğiştirilebileceği anlamına gelir. Bu sistemlerle, şekiller kolayca yeniden boyut landırılabilir ve her boyutta net bir görüntü sağlanır.

Doclingo https://www.doclingo.ai

Ses Temsili

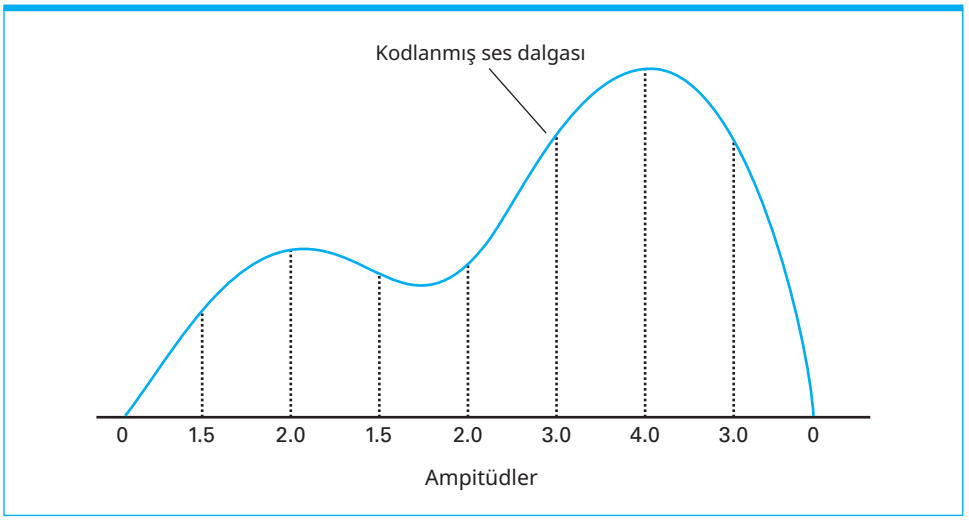
Bilgisayar depolama ve manipülasyonu için ses bilgilerini kodlamanın en gen el yöntemi, ses dalgasının genliğini düzenli aralıklarla örneklemektir ve elde edilen değerler serisini kaydetmektir. Örneğin, 0, 1.5, 2.0, 1.5, 2.0, 3.0, 4.0, 3. 0, 0 serisi, genliğin arttığı, kısa bir süre düştüğü, daha yüksek bir seviyeye yü kseldiği ve ardından 0'a geri düştüğü bir ses dalgasını temsil eder (Şekil 1.14) . Bu teknik, saniyede 8000 örnekleme hızı kullanarak uzun mesafeli sesli telef on iletişiminde yıllardır kullanılmaktadır. İletişimin bir ucundaki ses, her seki z bin saniyede bir sesin genliğini temsil eden sayısal değerler olarak kodlanır . Bu sayısal değerler daha sonra iletişim hattı üzerinden alıcı uca iletilir ve bu rada sesin yeniden üretilmesinde kullanılır.

8000 örnek/saniye hızı hızlı bir oran gibi görünse de, yüksek kaliteli müzi k kayıtları için yeterli değildir. Günümüz müzik CD'lerinde elde edilen kaliteli ses yeniden üretimi için saniyede 44.100 örnek hızı kullanılır. Her örnekten el de edilen veriler 16 bit (stereo kayıtlar için 32 bit) olarak temsil edilir. Sonuç o larak, stereo olarak kaydedilen her saniye müzik, bir milyondan fazla bit gere ktirir.

Müzik Aleti Dijital Arayüzü (MIDI, “MID-ee” olarak telaffuz edilir) olarak bilin en alternatif bir kodlama sistemi, elektronik klavyelerde bulunan müzik sentezl eyicilerinde, video oyunu seslerinde ve web sitelerini eşlik eden ses efektlerind e yaygın olarak kullanılmaktadır. MIDI, müziği sentezleyicide üretmek için yönl endirmeleri kodlayarak, sesin kendisini kodlamaktan kaçınır ve bu da örneklem e tekniğinin büyük depolama gereksinimlerini ortadan kaldırır. Daha kesin olar ak, MIDI hangi enstrümanın hangi notayı ne kadar süre çalacağını kodlar, bu d a D notasını iki saniye çalan bir klarnetin, 44.100 örnek/saniye hızında örneklen mesi durumunda iki milyondan fazla bit yerine üç bayt olarak kodlanabileceği a nlamına gelir.

Kısacası, MIDI, bir sanatçı tarafından okunan nota sayfasını kodlama yön temi olarak düşünülebilir, performansı değil ve dolayısıyla, bir MIDI “kaydı” f arklı sentezleyicilerde çalındığında önemli ölçüde farklı sesler çıkarabilir.

Şekil **1.14** Dizi tarafından temsil edilen ses dalgası 0, 1.5, 2.0, 1.5, 2.0, 3.0, 4.0, 3.0, 0



Doclingo https://www.doclingo.ai

Sorular & Alıştırmalar

1. İşte 8 bitlik sembol kullanılarak ASCII'de kodlanmış bir mesaj. Ne diyor? (*Bkz* Ek A)

```
01000011  01101111  01101101  01110000  01110101  01110100
01100101  01110010  00100000  01010011  01100011  01101001
01100101  01101110  01100011  01100101
```

2. ASCII kodunda, büyük harf ile aynı harfin küçük harfi arasındaki ilişki nedir? (*Bkz* Ek A.)
3. Bu cümleleri ASCII'de kodlayın:

   a. "Dur!" Cheryl bağırdı. b. $2 + 3 = 5?$

4. Günlük hayattan, iki durumda olabilen bir cihazı tanımlayın,örneğin bir direk üzerindeki bayrağın yukarıda veya aşağıda olduğu gibi. Bir duruma 1tanımlayın ve diğerine 0, ve böyle bitlerle saklandığında b h arfinin ASCII temsili nasıl görüneceğini gösterin.
5. Aşağıdaki ikili temsillerin her birini eşdeğeronluk biçimine dönüştürü n:

   a. `0101` b. `1001` c. `1011`
   d. `0110` ve. `10000` f. `10010`

6. Aşağıdaki onluk temsillerin her birini eşdeğerikili biçimine dönüştürü n:

   a. 6 b. 13 c. 11
   d. 18 ve. 27 f. 4

7. Üç bayt ile temsil edilebilecek en büyük sayısal değer nedirher bir rakam bir bayt başına bir ASCII deseni kullanılarak kodlanırsa? Eğerikili notasyon kullanılsaydı ne olurdu?
8. Bit desenlerini temsil etmek için onaltılık notasyona bir alternatif noktalı ondalık notasyon her bir baytın desen içinde ondalık karşılığı i le temsil edildiği bir sistemdir.Bu bayt temsilleri, noktalarla ayrılır. Ör neğin, 12.5 deseni0000110000000101'i temsil eder (bayt 00001100 1 2 ile, 00000101 ise 5 ile temsil edilir) ve desen 1000100000010000000 00111 ise 136.16.7 ile temsil edilir. Aşağıdaki bit desenlerinden her bi rini noktalı ondalık notasyonla temsil edin.

   a. `0000111100001111` b. `00110011000000010000000`
   c. `0000101010100000`

9. Geometrik yapılar aracılığıyla görüntüleri temsil etmenin, bit haritalarına kıyasl a ne gibi bir avantajı vardır? Bit harita teknikleri ile geometrik yapılar arasındak i durum nedir?
10. Bir saatlik müziğin stereo kaydının, metinde tartışıldığı gibi, saniyede 44,1 00 örnekleme hızı kullanılarak kodlandığını varsayalım. Kodlanmış versiyo nun boyutu, bir CD'nin depolama kapasitesi ile nasıl karşılaştırılır?

Doclingo https://www.doclingo.ai

1.5 İkili Sistem

Bölüm 1.4'te, ikili notasyonun yalnızca 0 ve 1 rakamlarını kullanarak sayısal değerleri temsil etmenin bir yolu olduğunu gördük; bu, daha yaygın olan onluk notasyon sistemde kullanılan 0'dan 9'a kadar on rakam yerine geçer. Şimdi ikili notasyona daha ayrıntılı bir şekilde bakalım.

İkili Notasyon

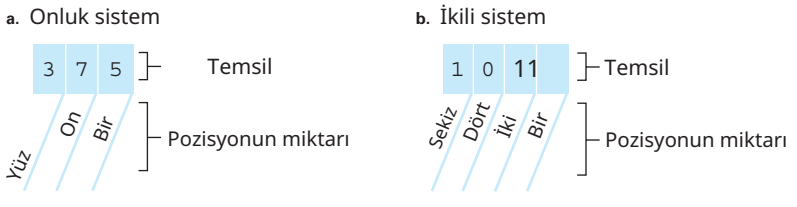
Onluk sistemde, bir temsil içindeki her pozisyon bir miktar ile ilişkilidir. 375 temsilinde, 5 bir ile ilişkilendirilen pozisyondadır, 7 on ile ilişkilendirilen pozisyondadır ve 3 ise yüz ile ilişkilendirilen pozisyondadır (Şekil 1.15a). Her miktar, sağındaki miktarın on katıdır. Tüm ifadenin temsil ettiği değer, her rakamın değerini o rakamın pozisyonu ile ilişkilendirilen miktar ile çarparak ve ardından bu çarpımları toplayarak elde edilir. Örneğin, 375 deseni (3 $\times$ yüz) $+(7\times$ on$)+(5\times$ bir) olarak temsil edilir; bu, daha teknik notasyonda $(3\times10^2)+(7\times10^1)+(5\times10^0)$ şeklindedir.

Her bir basamağın ikili gösterimdeki konumu, bir miktar ile ilişkilidir; ancak her konumla ilişkili miktar, sağındaki konumla ilişkili miktarın iki katıdır. Daha kesin olarak, bir ikili temsilin en sağdaki basamağı, bir (2^0) ile ilişkili miktarla ilişkilidir, soldaki bir sonraki konum iki (2^1) ile, bir sonraki dört (2^2) ile, bir sonraki sekiz (2^3) ile ve bu şekilde devam eder. Örneğin, 1011 ikili temsilinde, en sağdaki 1, bir ile ilişkili konumda, yanındaki 1 iki ile ilişkili konumda, 0 dört ile ilişkili konumda ve en soldaki 1 sekiz ile ilişkili konumdadır (Şekil 1.15b).

Bir ikili temsilin temsil ettiği değeri çıkarmak için, onluk tabanda olduğu gibi aynı prosedürü izleriz—her basamağın değerini, konumuyla ilişkili miktar ile çarpar ve sonuçları toplarız. Örneğin, 100101'in temsil ettiği değer 37'dir, bu Şekil 1.16'da gösterilmiştir. İkili gösterim yalnızca 0 ve 1 basamaklarını kullandığı için, bu çarpma ve toplama işlemi, yalnızca 1'lerin bulunduğu konumlarla ilişkili miktarları toplamakla sınırlıdır. Böylece, 1011 ikili deseni, 1'lerin bir, iki ve sekiz ile ilişkili konumlarda bulunduğu için on bir değerini temsil eder.

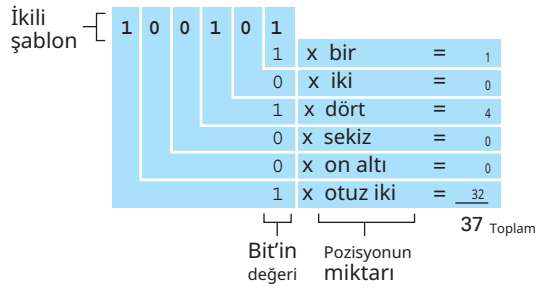
1.4. Bölümde ikili notasyonda nasıl sayılacağını öğrendik, bu da bize küçük tam sayıları kodlama imkanı sağladı. Büyük değerlerin ikili temsillerini bulmak için, Şekil 1.17'deki algoritmanın tanımladığı yaklaşımı tercih edebilirsiniz. Bu algoritmayı on (Şekil 1.18) değerine uygulayalım. Öncelikle on'u ikiye böleriz,

Şekil 1.15 Onluk ve ikilik sistemler



Doclingo https://www.doclingo.ai

Şekil **1.16** İkili temsilin çözülmesi 100101



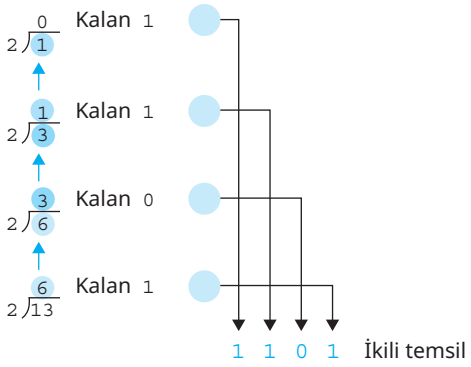
Şekil **1.17** Pozitif bir tam sayının ikili temsilini bulmak için bir algoritma

Adım 1. Değeri ikiye bölün ve kalanı kaydedin.

Adım 2. Elimize geçen bölüm sıfır olmadığı sürece, en yeni bölümü ikiy e bölmeye devam edinve kalanı kaydedin.

Adım 3. Artık sıfır bölümü elde edildiğine göre, orijinal değerin ikil itemsilinin kalanı, kaydedildiği sırayla sağdan sola doğru listele nmiştir.

Şekil **1.18** Algoritmayı Şekil 1.17 kullanarak on üçün ikili temsilini elde etme



altı ve bir kalan elde ederek. Bölüm 2, bölümün sıfır olmadığını belirttiği için, bölüm (altı) ikiye bölünür, yeni bir bölüm elde edilirüç ve sıfır kalan. En yeni b ölüm hala sıfır değil, bu yüzden ikiye bölüyoruz, bir bölüm ve bir kalan elde e diyoruz. Yine,en yeni bölümü (bir) ikiye bölüyoruz, bu sefer sıfır bölüm ve bir kalan elde ediyoruz. Artık sıfır bölüm elde ettiğimiz için, Bölüm 3'e geçiyoruz, burada orijinaldeğerin (on üç) ikili temsilinin 1101 olduğunu öğreniyoruz, ka lanlar listesinden elde edilmiştir.

Doclingo https://www.doclingo.ai

İkili Toplama

İkili olarak temsil edilen iki tam sayının toplama sürecini anlamak için,öncelik le geleneksel onluk taban notasyonunda temsil edilen değerlerin toplama sü recini hatırlayalım. Örneğin, aşağıdaki problemi düşünelim:

```
  58
27
```

En sağdaki sütunda 8 ve 7'yi toplayarak 15 toplamını elde ediyoruz.
Bu sütunun altına 5'i kaydediyoruz ve 1'i bir sonraki sütuna taşıyoruz,
bu da

```
   1
  58
27
   5
```

Bir sonraki sütundaki 5 ve 2'yi, taşınan 1 ile birlikte toplayarak 8 toplamını el de ediyoruz; bu toplamı sütunun altına kaydediyoruz. Sonuçşu şekildedir:

```
  58
+ 27
  85
```

Kısacası, prosedür, her sütundaki rakamları toplarken sağdan sola doğru iler lemek,toplamın en az anlamlı rakamını sütunun altına yazmak ve toplamın d aha anlamlı rakamını (varsa) bir sonraki sütuna taşımaktır.

İkili notasyonla temsil edilen iki tam sayıyı toplamak için, aynı prosedürü izl iyoruz,bununla birlikte tüm toplamlar, ilkokulda öğrendiğiniz geleneksel onluk taban bilgileri yerine Şekil 1.19'da gösterilen toplama bilgileri kullanılarak hesa planmaktadır. Örneğin, problemi çözmek için

```
  111010
+  11011
```

en sağdaki 0 ve 1'i toplayarak başlıyoruz; 1 elde ediyoruz, bunu sütunun altına yazıyoruz. Şimdi bir sonraki sütundaki 1 ve 1'i toplayarak 10 elde ediyoruz. Bu 1 0'dan 0'ı sütunun altına yazıyoruz ve 1'i bir sonraki sütunun üstüne taşıyoruz. B u noktada, çözümümüz şöyle görünüyor:

```
    1
  111010
+  11011
    01
```

Şekil **1.19** İkili toplama gerçekleri

```
  0     1     0     1
+0    +0    +1    +1
  0     1     1    10
```

Doclingo https://www.doclingo.ai

Bir sonraki sütundaki 1, 0 ve 0'ı topluyoruz, 1 elde ediyoruz ve bu sütün altına 1 yazıyoruz. Bir sonraki sütundaki 1 ve 1 toplamı 10'dur; 0'ı sütunun altına yazıyoruz ve 1'i bir sonraki sütuna taşıyoruz. Şimdi çözümümüz şöyle görünüyor:

```
  1
 111010
11011
  0101
```

Bir sonraki sütundaki 1, 1 ve 1 toplamı 11'dir (üç değerinin ikili gösterimi);alt sırad aki 1'i sütunun altına yazıyoruz ve diğer 1'i bir sonraki sütunun üstüne taşıyoruz. O 1'i, o sütunda zaten bulunan 1'e ekleyerek 10 elde ediyoruz. Yine,alt sıradaki 0'ı kaydediyoruz ve 1'i bir sonraki sütuna taşıyoruz. Şimdi şunları elde ettik:

```
 1
  111010
11011
  010101
```

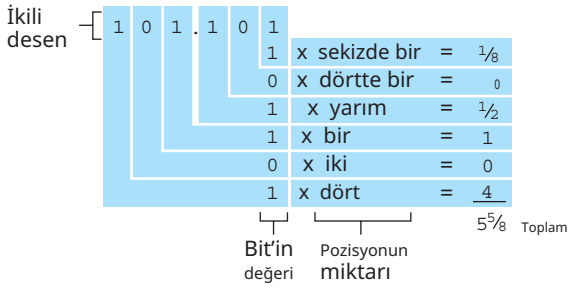
Bir sonraki sütundaki tek giriş, önceki sütundan taşıdığımız 1'dir, bu yüzden cevaba kaydediyoruz. Nihai çözümümüz şudur:

```
  111010
11011
 1010101
```

İkili Kesirler

Kesirli değerleri karşılamak için ikili gösterimi genişletmek için, ondalık göste rimdeki ondalıknoktasıyla aynı rolü üstlenen bir taban noktası kullanıyoruz. Yani, noktanın solundaki basamaklar,değerin tam kısmını (bütün kısmını) te msil eder ve daha önce tartışılan ikili sistemde olduğu gibi yorumlanır. Sağın daki basamaklar ise,değerin kesirli kısmını temsil eder ve diğer bitler gibi yor umlanır, ancak pozisyonları kesirli miktarlarla atanır. Yani, tabanın sağındaki ilk pozisyona,şu miktar atanır:
$^1/_2$ (yani 2^1), bir sonraki pozisyon miktarı $^1/_4$ (yani 2^2), bir sonraki $^1/_8$ (yani 2^3), ve devam ediyor. Bu, daha önce belirtilen kuralın yalnızca bir deva mıdır:Her pozisyona, sağındaki pozisyonun büyüklüğünün iki katı kadar bir miktar atanır. Bu miktarlar bit pozisyonlarına atandıktan sonra, bir ondalık n okta içeren ikili bir temsili çözmek, ondalık nokta olmadan kullanılan aynı pr osedürü gerektirir. Daha kesin olarak, her bit değerini, temsildeki o bitin poz isyonuna atanan miktarla çarparız. Örneğin, ikili temsil 101.101, Şekil 1.20'de gösterildiği gibi $5^5/_8$ olarak çözülür.

Şekil **1.20** İkili temsili çözme 101.101



Doclingo https://www.doclingo.ai

Analog ve Dijital

Yirmi birinci yüzyıldan önce, birçok araştırmacı dijital ve analog teknolojinin avantajları ve d ezavantajları üzerine tartıştı. Dijital bir sistemde, bir değer bir dizi rakam olarak kodlanır ve ardından her biri rakamlardan birini temsil eden birkaç cihaz kullanılarak depolanır. Analog bir sistemde ise, her değer sürekli bir aralık içinde herhangi bir değeri temsil edebilen tek bi r cihazda depolanır.

İki yaklaşımı su kovaları kullanarak depolama cihazları olarak karşılaştıralı m. Dijital bir sistemi simüle etmek için, boş bir kovayı rakamı 0 temsil edecek şekilde ve dolu bir kovayı rakamı 1 temsil edecek şekilde kabul edebiliriz. Ardı ndan, bir dizi kova içinde bir sayısal değeri kayan nokta notasyonu kullanarak depolayabiliriz (bkz. Bölüm 1.7). Buna karşılık, bir analog sistemi, su seviyesin in temsil edilen sayısal değeri temsil ettiği noktaya kadar tek bir kovayı kısme n doldurarak simüle edebiliriz. İlk bakışta, analog sistem daha doğru görüneb ilir çünkü dijital sistemdeki kesme hatalarından etkilenmez (yine bkz. Bölüm 1 .7). Ancak, analog sistemde kovanın herhangi bir hareketi su seviyesini algıla mada hatalara neden olabilirken, dijital sistemde dolu bir kova ile boş bir kov a arasındaki ayrımın bulanıklaşması için önemli miktarda dalgalanma olması gerekir. Böylece dijital sistem, analog sistemden daha az hata hassasiyetine s ahip olacaktır.Bu dayanıklılık, başlangıçta analog teknolojiye dayanan birçok uygulamanın (telefon iletişimi, ses kayıtları ve televizyon gibi) dijit al teknolojiye geçmesinin başlıca nedenlerinden biridir.

Toplama için, onluk sistemde uygulanan teknikler ikili sistemde de geçerlid ir. Yani, ondalık noktaları olan iki ikili temsili toplamak için, yalnızca ondalık nok taları hizalarız ve daha önceki toplama işlemini uygularız. Örneğin, 10.011 ile 1 00.11 toplandığında 111.001 elde edilir, burada gösterildiği gibi:

```
  10.011
+ 100.110
---------
 111.001
```

Sorular & Alıştırmalar

1. Aşağıdaki ikili temsillerin her birini eşdeğeronluk biçimine dönüştürü n:

   a. 101010 **b.** 100001 **c.** 10111 **d.** 0110 **Ve.** 11111

2. Aşağıdaki onluk temsillerin her birini eşdeğerikili biçimine dönüştürü n:

   a. 32 **b.** 64 **c.** 96 **d.** 15 **Ve.** 27

3. Aşağıdaki ikili temsillerin her birini eşdeğeronluk biçimine dönüştürü n:

   a. 11.01 **b.** 101.111 **c.** 10.1 **d.** 110.011 **Ve.** 0.101

4. Aşağıdaki değerleri ikili notasyonda ifade edin:

   a. $4\frac{1}{2}$ **b.** $2\frac{3}{4}$ **c.** $1\frac{1}{8}$ **d.** $\frac{5}{16}$ **Ve.** $5\frac{5}{8}$

Doclingo https://www.doclingo.ai

5. Aşağıdaki toplama işlemlerini ikili notasyonda gerçekleştirin:

a.
```
 11011
+ 1100
```

b.
```
 1010.001
+   1.101
```

c.
```
 11111
+ 0001
```

d.
```
 111.11
+ 00.01
```

1.6 Tam Sayıları Saklama

Matematikçiler uzun zamandır sayısal notasyon sistemleriyle ilgilenmektedir ve birçok fikirleri dijital devre tasarımıyla çok uyumlu çıkmıştır. Bu bölümde, tam sayı değerlerini bilgisayar ekipmanında temsil etmek için kullanılan iki notasyon sistemini, iki'nin tamamlayanı notasyonu ve fazlalık notasyonunu ele alacağız. Bu sistemler ikili sisteme dayanmaktadır ancak bilgisayar tasarımıyla daha uyumlu hale getiren ek özelliklere sahiptir. Ancak bu avantajlarla birlikte dezavantajlar da gelmektedir.

Amacımız bu özellikleri anlamak ve bunların bilgisayar kullanımını nasıl etkilediğiniöğrenmektir.

İki'nin Tamamlayanı Notasyonu

Günümüz bilgisayarlarında tam sayıları temsil etmek için en popüler sistem, iki'nin tamamlayanı notasyonudur. Bu sistem, sistemdeki her bir değeri temsil etmek için sabit bir bit sayısı kullanır. Günümüz ekipmanlarında, her bir değerin 32 bitlik bir şablonla temsil edildiği bir iki'nin tamamlayanı sistemi kullanmak yaygındır. Bu kadar büyük bir sistem, geniş bir sayı aralığının temsil edilmesine olanak tanır ancak gösterim amaçları için rahatsız edicidir. Bu nedenle, iki'nin tamamlayanı sistemlerinin özelliklerini incelemek için daha küçük sistemlere odaklanacağız.

Şekil 1.21, biri üç bit uzunluğundaki bit desenlerine, diğeri ise dört bit uzunluğundaki bit desenlerine dayanan iki tam iki'nin tamamlayıcı sistemini göstermektedir. Böyle bir

Şekil **1.21**İki'nin tamamlayıcısı notasyon sistemleri

a. Üç bit uzunluğundaki desenleri kullanarak

Bit pattern	Value represented
011	3
010	2
001	1
000	0
111	1
110	2
101	3
100	4

b. Dört bit uzunluğundaki desenleri kullanarak

Bit pattern	Value represented
0111	7
0110	6
0101	5
0100	4
0011	3
0010	2
0001	1
0000	0
1111	1
1110	2
1101	3
1100	4
1011	5
1010	6
1001	7
1000	8

Doclingo https://www.doclingo.ai

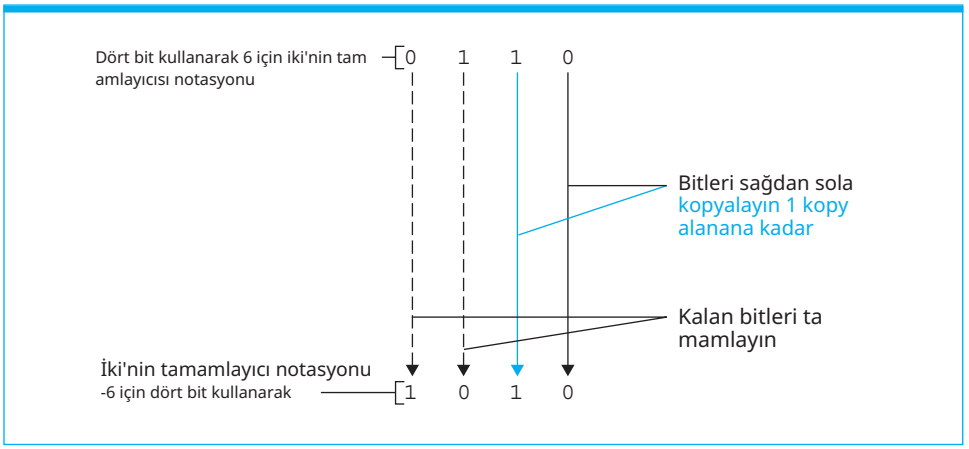
sistem, uygun uzunlukta 0'ların bir dizisi ile başlayarak inşa edilir ve ardında nbir 0'ın ardından 1'lerin geldiği desene ulaşana kadar ikili olarak sayılır. Bu desenler 0, 1, 2, 3, . . . değerlerini temsil eder.Negatif değerleri temsil eden d esenler, uygun uzunlukta 1'lerin bir dizisi ile başlayarak ve ardındanbir 1'in a rdından 0'ların geldiği desene ulaşana kadar ikili olarak geriye doğru sayılar ak elde edilir. Bu desenler 1,2, 3, . . . değerlerini temsil eder. (Eğer ikili olarak −geriye doğru saymak zor geliyorsa, sadecetablonun en altından, bir 1'in ar dından 0'ların geldiği desenle başlayın ve tüm 1'lerin olduğu desene kadar s ayın.)

İki'nin tamamlayıcısı sisteminde, bir bit deseninin en soldaki biti temsil edilen de ğerin işaretini gösterir.Bu nedenle, en soldaki bit genellikle işaret biti olarak adlandır ılır. İki'nin tamamlayıcısı sisteminde, negatif değerler 1 olan işaret bitlerine sahip des enlerle temsil edilir;negatif olmayan değerler ise 0 olan işaret bitlerine sahip desenle rle temsil edilir.

İki'nin tamamlayıcısı sisteminde, aynı büyüklükteki pozitif ve negatif değ erleri temsil eden desenler arasında kullanışlı bir ilişki vardır.
Sağdan sola okunduğunda, ilk 1'e kadar ve dahil olmak üzere aynıdırlar.
Bundan sonra, desenler birbirinin tamamlayıcısıdır. (Bir desenin tamamlayıcı sı, tüm 0'ları 1'e ve tüm 1'leri 0'a değiştirerek elde edilen desendir; 0110 ve 1 001 tamamlayıcılardır.) Örneğin, Şekil 1.21'deki 4 bitlik sistemde 2'yi temsil e den desenler 2 ve 2 her ikisi de 10 ile biter, ancak 2'yi temsil eden desen 00 il e başlarken, 2'yi temsil eden desen 11 ile başlar. Bu gözlem, aynı büyüklükte ki pozitif ve negatif değerleri temsil eden bit desenleri arasında ileri geri dön üşüm için bir algoritmaya yol açar. Sadece sağdan sola orijinal deseni kopyal arız, bir 1 kopyalanana kadar, ardından kalan bitleri son bit desenine aktarır ken tamamlarız (Şekil 1.22).

İki'nin tamamlayıcısı sistemlerinin bu temel özelliklerini anlamak, aynı zam anda iki'nin tamamlayıcısı temsillerini çözmek için bir algoritmaya da yol açar. Çözülmesi gereken desenin işaret biti 0 ise, değeri sadece okumamız yeterlidir.

Şekil **1.22** Değeri 6 iki'nin tamamlayıcı notasyonunda 4 bit kullanarak kodlama



Doclingo https://www.doclingo.ai

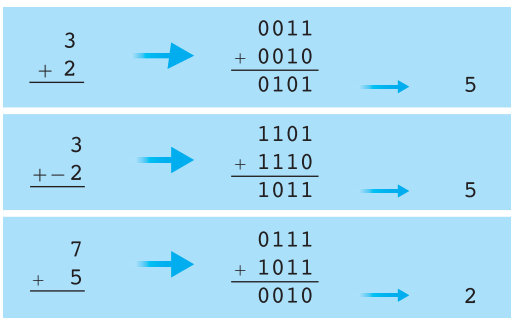
desenin bir ikili temsil olduğunu varsayalım. Örneğin, 0110 değeri 6'yı temsil eder, çünkü 110 6'nın ikili karşılığıdır. Eğer çözülecek desenin işaret biti 1 ise, temsil edilen değerin negatif olduğunu biliriz ve geriye kalan tek şey değerin büyüklüğünü bulmaktır. Bunu Şekil 1.22'deki "kopyala ve tamamla" prosedürünü uygulayarak ve ardından elde edilen deseni sanki basit bir ikili temsil gibi çözerek yaparız. Örneğin, 1010 desenini çözmek için önce işaret bitinin 1 olduğunu tanırız, bu da temsil edilen değerin negatif olduğu anlamına gelir. Bu nedenle, "kopyala ve tamamla" prosedürünü uygulayarak 0110 desenini elde ederiz, bunun 6'nın ikili temsili olduğunu tanırız ve orijinal desenin 6'yı temsil ettiğini sonucuna varırız.

İki'nin Tamamlayıcı Notasyonunda Toplama İki'nin tamamlayıcı notasyonunda temsil edilen değerleri toplamak için, ikili toplama için kullandığımız aynı algoritmayı uygularız, tek farkla ki tüm bit desenleri, cevap dahil, aynı uzunluktadır. Bu, iki'nin tamamlayıcı sisteminde toplarken, son taşıma ile cevabın sol tarafında üretilen herhangi bir ekstra bitin kesilmesi gerektiği anlamına gelir. Böylece "toplama" 0101 ve 0010, 0111 üretir ve "toplama" 0111 ve 1011, 0010 (0111 + 1011 10010, bu da = 0010'a kesilir) sonucunu verir.

Bu anlayışla, Şekil 1.23'teki üç toplama problemini göz önünde bulundurun. Her durumda, problemi iki'nin tamamlayıcısı notasyonuna çevirdik (dört uzunlukta bit desenleri kullanarak), daha önce tanımlanan toplama işlemini gerçekleştirdik ve sonucu tekrar alışıldık onluk notasyona çözdük.

Şekil 1.23'teki üçüncü problemin, pozitif bir sayının negatif bir sayıya eklenmesini içerdiğini gözlemleyin; bu, iki'nin tamamlayıcısı notasyonunun büyük bir faydasını gösterir: Herhangi bir imzalı sayı kombinasyonunun toplanması, aynı algoritma ve dolayısıyla aynı devre kullanılarak gerçekleştirilebilir. Bu, insanların gelenekselolarak aritmetik hesaplamaları nasıl yaptığıyla keskin bir tezat oluşturur. İlkokul çocukları önce toplama yapmayı öğrenirken, daha sonra çıkarma yapmayı öğrenirler; iki'nin tamamlayıcısı notasyonunu kullanan bir makinenin yalnızca toplama yapmayı bilmesi gerekir.

Şekil **1.23** İki'nin tamamlayıcısı notasyonuna dönüştürülen toplama problemleri



Doclingo https://www.doclingo.ai

Örneğin, 7 çıkarma problemi − 5, toplama problemiyle aynıdır-lem 7 + (5). Sonuç olarak, bir makineden 7'den 5'i (0101 olarak saklanan) çıkarması istendiğinde, önce 5'i 5'e (1011 olarak temsil edilen) dönüştürecek ve ardından 0111 + 1011 toplama işlemini gerçekleştirerek 0010 elde edecektir. Bu da 2'yi temsil eder, aşağıdaki gibi:

```
  7              0111            0111
 −5      →    − 0101      →    + 1011
                                0010      →      2
```

O halde, iki'nin tamamlayıcı notasyonu sayısal değerleri temsil etmek için kullanıldığında, bir toplama devresi ile bir değeri negatif hale getiren bir devrenin, hem toplama hem de çıkarma problemlerini çözmek için yeterli olduğunu görüyoruz. (Bu tür devreler Ek B 'de gösterilmiş ve açıklanmıştır.)

Taşma Problemi Önceki örneklerde kaçındığımız bir problem, herhangi bir iki' nin tamamlayıcı sisteminde temsil edilebilecek değerlerin boyutunda bir sınır olduğudur. 4 bitlik desenlerle iki'nin tamamlayıcısını kullanırken, temsil edilebilecek en büyük pozitif tam sayı 7'dir ve en negatif tam sayı 8'dir. Özellikle, 9 değeri temsil edilemez, bu da 5 + 4 probleminin doğru cevabını elde etme umudumuzun olamayacağı anlamına gelir. Aslında, sonuç 7 olarak görünecektir. Bu olguya taşma denir. Yani, taşma, bir hesaplamanın temsil edilebilecek değerler aralığının dışına düşen bir değer üretmesi durumunda meydana gelen problemdir. İki'nin tamamlayıcı notasyonu kullanıldığında, bu, iki pozitif değeri toplarken veya iki negatif değeri toplarken meydana gelebilir. Her iki durumda da, durumun, cevabın işaret bitini kontrol ederek tespit edilebileceği anlamına gelir. İki pozitif değerin toplamı negatif bir değerin deseni ile sonuçlanırsa veya iki negatif değerin toplamı pozitif görünüyorsa, bir taşma olduğu belirtilir.

Elbette, çünkü çoğu bilgisayar, örneklerimizde kullandığımızdan daha uzun bit desenlerine sahip iki'nin tamamı sistemlerini kullanır, bu nedenle daha büyük değerler taşma olmadan işlenebilir. Bugün, iki'nin tamamı notasyonunda değerleri depolamak için 32 bit desenleri kullanmak yaygındır ve bu, taşma meydana gelmeden önce 2.147.483.647 kadar büyük pozitif değerlerin birikmesine olanak tanır. Daha büyük değerlere ihtiyaç varsa, daha uzun bit desenleri kullanılabilir veya belki de ölçü birimleri değiştirilebilir. Örneğin, inç yerine mil cinsinden bir çözüm bulmak, daha küçük sayıların kullanılmasına neden olur ve yine de gereken doğruluğu sağlayabilir.

Önemli olan, bilgisayarların hata yapabilmesidir. Bu nedenle, makineyi kullanan kişinin ilgili tehlikelerin farkında olması gerekir. Bir sorun, bilgisayar programcıları ve kullanıcılarının kendilerini rahat hissetmeleri ve küçük değerlerin büyük sayılar üretmek için birikme yapabileceğini göz ardı etmeleridir. Örneğin, geçmişte iki'nin tamamı notasyonunda değerleri temsil etmek için 16 bit desenleri kullanmak yaygındı, bu da 32.768 veya daha büyük değerlere ulaşıldığında taşma olacağı anlamına geliyordu. = 32.768 veya daha büyük değerlere ulaşıldığında. 19 Eylül 1989'da, yıllarca güvenilir hizmet verdikten sonra bir hastane bilgisayar sistemi arızalandı. Yakından incelendiğinde, bu tarihin 1 Ocak 1900'den 32.768 gün sonra olduğu ve makinenin bu başlangıç tarihine dayalı olarak tarihler hesaplamak üzere programlandığı ortaya çıktı. Böylece, taşma nedeniyle 19 Eylül 1989, negatif bir değer üretti - bilgisayarın programının başa çıkmak üzere tasarlanmadığı bir fenomen.

Doclingo https://www.doclingo.ai

Aşırı Notasyon

Tam sayı değerlerini temsil etmenin bir diğer yöntemi aşırı notasyondur. İki'nin tamamlayıcısı notasyonunda olduğu gibi, bir aşırı notasyon sistemindeki her değer, aynı uzunlukta bir bit deseni ile temsil edilir. Bir aşırı sistem oluşturmak için, önce kullanılacak desen uzunluğunu seçeriz, ardından o uzunluktaki tüm farklı bit desenlerini, ikili sayma yapıyormuşuz gibi sıralarız. Sonra, en önemli biti 1 olan ilk desenin listedeki yerinin yaklaşık olarak ortada olduğunu gözlemleriz. Bu deseni sıfırı temsil etmek için seçeriz; bunun ardındaki desenler 1, 2, 3, . . .; ve bunun önündeki desenler 1, 2, 3, . . . için kullanılır. Dört uzunluğundaki desenler kullanıldığında elde edilen kod, Şekil 1.24'te gösterilmiştir. Orada 5 değerinin 1101 deseni ile ve 5'in 0011 ile temsil edildiğini görüyoruz. (Aşırı bir sistem ile iki'nin tamamlayıcısı sistemi arasındaki fark, işaret bitlerinin ters çevrilmiş olmasıdır.)

Şekil 1.24'te temsil edilen sistem aşırı sekiz notasyonu olarak bilinir. Nedenini anlamak için, önce koddaki her bir deseni geleneksel ikili sistem kullanarak yorumlayın ve ardından bu sonuçları aşırı notasyonda temsil edilen değerlerle karşılaştırın. Her durumda, ikili yorumlamanın aşırı notasyon yorumlamasını 8 değeri kadar aştığını bulacaksınız. Örneğin, ikili notasyondaki 1100 deseni 12 değerini temsil eder, ancak bizim aşırı sistemimizde 4'ü temsil eder; ikili notasyondaki 0000, 0'ı temsil eder, ancak aşırı sistemde -8'i temsil eder. Benzer bir şekilde, beş uzunluğundaki desenlere dayanan bir aşırı sistem aşırı 16 notasyonu olarak adlandırılacaktır.

Şekil **1.24** Aşırı sekiz dönüşüm tablosu

Bit pattern	Value represented
1111	7
1110	6
1101	5
1100	4
1011	3
1010	2
1001	1
1000	0
0111	−1
0110	−2
0101	−3
0100	−4
0011	−5
0010	−6
0001	−7
0000	−8

Doclingo https://www.doclingo.ai

Şekil **1.25** Üç uzunlukta bit desenleri kullanan bir fazlalık notasyon sistemi

Bit pattern	Value represented
111	3
110	2
101	1
100	0
011	−1
010	−2
001	−3
000	−4

çünkü örneğin 10000 deseni, genellikle 16 olan değerini temsil etmek yerine sıfırı temsil etmek için kullanılacaktır. Benzer şekilde, üç bit fazlalık sistemini n fazlalık dört notasyonu olarak bilineceğini doğrulamak isteyebilirsiniz (Şekil 1.25).

Sorular & Alıştırmalar

1. Aşağıdaki iki tamamlayıcı temsilinin her birini eşdeğer onluk forma d önüştürün:

   a. 00011 **b.** 01111 **c.** 11100
   d. 11010 **e.** 00000 **f.** 10000

2. Aşağıdaki onluk temsilinin her birini 8 bitlik desenler kullanarak eşde ğer iki tamamlayıcı forma dönüştürün:

   a. 6 **b.** −6 **c.** −17
   d. 13 **ve.** −1 **f.** 0

3. Aşağıdaki bit desenlerinin iki tamamlayıcı notasyonunda saklanan değe rleri temsil ettiğini varsayalım. Her değerin negatifinin iki tamamlayıcı t emsilini bulun:

   a. 00000001 **b.** 01010101 **c.** 11111100
   d. 11111110 **e.** 00000000 **f.** 01111111

4. Bir makinenin sayıları iki tamamlayıcı notasyonunda sakladığını varsayalı m. Makine aşağıdaki uzunluklarda bit desenleri kullanıyorsa saklanabilece k en büyük ve en küçük sayılar nelerdir?

   a. dört **b.** altı **c.** sekiz

5. Aşağıdaki problemlerde, her bit deseni iki'nin tamamlayıcısı notasyonunda s aklanan bir değeri temsil eder. Her problemin cevabını iki'nin tamamlayıcısı notasyonunda, aşağıda açıklanan toplama işlemini gerçekleştirerek bulun.

Doclingo https://www.doclingo.ai

metni. Ardından, probleminizi ve cevabınızı onluk tabanda notasyonuna çevirerek çalışmanızı kontrol edin.

a. 0101 + 0010 **b.** 0011 + 0001 **c.** 0101 + 1010 **d.** 1110 + 0011 **ve.** 1010 + 1110

6. Aşağıdaki her bir problemi iki'nin tamamlayıcısı notasyonunda çözün, ancak bu sefer taşmayı izleyin ve bu fenomen nedeniyle hangi cevapların yanlış olduğunu belirtin.

a. 0100 + 0011 **b.** 0101 + 0110 **c.** 1010 + 1010 **d.** 1010 + 0111 **ve.** 0111 + 0001

7. Aşağıdaki her bir problemi ondalık gösterimden iki'nin tamamı gösterimine çevirin ve ardından her problemi eşdeğer bir toplama problemine dönüştürün (bir makinenin yapabileceği gibi) ve toplama işlemini gerçekleştirin. Cevaplarınızı ondalık gösterime geri çevirerek kontrol edin.

a. $6 + -(-1)$ **b.** $3 + -2$ **c.** $4 + -6$ **d.** $2 + -(-4)$ **ve.** $1 + -5$

8. İki'nin tamamı gösteriminde bir değer pozitif ve diğeri negatif olduğunda değerler toplandığında taşma (overflow) durumu hiç meydana gelebilir mi? Cevabınızı açıklayın.

9. Aşağıdaki her bir fazla sekiz gösterimini, metindeki tabloya atıfta bulunmadan eşdeğer ondalık forma çevirin:

a. 1110 **b.** 0111 **c.** 1000
d. 0010 **e.** 0000 **f.** 1001

10. Aşağıdaki her bir ondalık gösterimi, metindeki tabloya atıfta bulunmadan eşdeğer fazla sekiz formuna çevirin:

a. 5 **b.** −5 **c.** 3
d. 0 **ve.** 7 **f.** −8

11. 9 değeri fazla sekiz gösteriminde temsil edilebilir mi? Peki ya 6'nın fazla dört gösteriminde temsil edilmesi? Cevabınızı açıklayın.

1.7 Kesirleri Saklama

Tam sayıların depolanmasının aksine, kesirli bir parçaya sahip bir değerin depolanması sadece onun ikili temsilini temsil eden 0 ve 1'lerin desenini değil, aynı zamanda kayıt noktasının konumunu da depolamayı gerektirir. Bunu yapmanın popüler bir yolu, bilimsel notasyona dayanmaktadır ve kayan nokta notasyonu olarak adlandırılır.

Kayan Nokta Notasyonu

Kayan nokta notasyonunu yalnızca bir bayt depolama kullanarak bir örnekle açıklayalım. Makineler genellikle çok daha uzun desenler kullansa da, bu 8 bitlik format gerçek sistemleri temsil eder ve uzun bit desenlerinin karmaşası olmadan önemli kavramları göstermeye hizmet eder.

Öncelikle, baytın yüksek sıralı bitini işaret bit olarak belirliyoruz. Yine, işaret bitinde 0 olması, depolanan değerin negatif olmadığını, 1 olması ise değerin negatif olduğunu anlamına gelir. Sonra, baytın geri kalan 7 bitini iki gruba veya alana ayırıyoruz:

Doclingo https://www.doclingo.ai

üstel alan ve mantissa alanı. İşaret bitinden sonraki 3 biti üstel alan olarak ve kalan 4 biti mantissa alanı olarak belirleyelim. Şekil 1.26, baytın nasıl bölündüğünü göstermektedir.

Alanların anlamını aşağıdaki örneği dikkate alarak açıklayabiliriz. Diyelim ki bir bayt,bit deseni 01101011'den oluşuyor. Bu deseni önceki formatla analiz ett iğimizde,işaret bitinin 0, üstelin 110 ve mantissanın 1011 olduğunu görüyoruz. Baytı çözmek için,öncelikle mantissayı çıkarıyoruz ve sol tarafına bir kayıt nokta sı koyarak elde ediyoruz:

```
.1011
```

Sonra, üstel alanın içeriğini (110) çıkarıyoruz ve bunu 3 bitlik fazlalık yöntemi kullana rak saklanan bir tam sayı olarak yorumluyoruz (bkz. tekrar Şekil 1.25). Böylece, örne ğimizdeki üstel alandaki desen pozitif 2'yi temsil ediyor. Bu, çözümümüzdeki ondalık noktayı 2 bit sağa kaydırmamız gerektiğini söylüyor. (Negatif bir üstel, ondalık nokta yı sola kaydırmak anlamına gelir.) Sonuç olarak, elde ediyoruz

```
10.11
```

bu, $2\frac{3}{4}$ için ikili temsildir. Sonraki adımda, örneğimizdeki işaret bitinin 0 olduğu nu not ediyoruz; dolayısıyla temsil edilen değer negatif değildir. 01101011 bayt ının $2\frac{3}{4}$'ü temsil ettiğini sonucuna varıyoruz. Desen 11101011 olsaydı (bu, işare t bitinden başka her şeyle aynı), temsil edilen değer $2\frac{3}{4}$ olurdu.

Başka bir örnek olarak, 00111100 baytını ele alalım. Mantissayı çıkarıyoruz ve elde ediyoruz

```
.1100
```

ve üstel alan (011) 1 değerini temsil ettiğinden ondalık noktayı 1 bit sola kaydırı yoruz. Dolayısıyla, elimizde

```
.01100
```

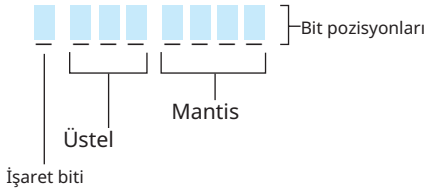
$\frac{3}{8}$'yi temsil ediyor. Orijinal desendeki işaret bitinin 0 olduğunu bildiğimiz için, saklanan değ er negatif değildir. 0011110 0 deseninin $\frac{3}{8}$'yi temsil ettiğini sonucuna varıyoruz.

Bir değeri kayan nokta notasyonu kullanarak saklamak için, önceki süreci t ersine çeviriyoruz. Örneğin, $1\frac{1}{8}$'yi kodlamak için, önce bunu ikili notasyonda ifa de ediyoruz ve 1.001 elde ediyoruz. Sonra, bit desenini mantissa alanına soldan sağa, ikili temsildeki en soldaki 1 ile başlayarak kopyalıyoruz. Bu noktada, bayt şöyle görünüyor:

```
_ _ _ _ 1 0 0 1
```

Şimdi üstel alanı doldurmamız gerekiyor. Bu amaçla, mantissa alanının içeriği ni solunda bir ondalık nokta ile hayal ediyoruz ve orijinal ikili sayıyı elde etmek için bit sayısını ve ondalığın hangi yöne hareket etmesi gerektiğini belirliyoruz.

Şekil **1.26** Kayan nokta notasyonu bileşenleri



Doclingo https://www.doclingo.ai

Örneğimizde, .1001'deki ondalığın 1 bit sağa hareket ettirilmesi gerektiğini görüyoru z, böylece 1.001 elde ediliyor. Bu nedenle, üstel pozitif olmalıdır, bu yüzden üstel ala na 101 (Şekil 1.25'te gösterildiği gibi, artı dört notasyonunda pozitif bir) yerleştiriyor uz. Son olarak, depolanan değerin negatif olmadığından dolayı işaret bitini 0 ile dold uruyoruz. Tamamlanmış byte şöyle görünüyor:

0 1 0 1 1 0 0 1

Mantissa alanını doldururken kaçırmış olabileceğiniz ince bir nokta var. Kural, ikili temsilin soldan sağa doğru, en soldaki 1 ile başlayarak bit desenin i kopyalamaktır. Açıklığa kavuşturmak için, değeri $^{3}/_{8}$ olan, ikili notasyonda .0 11 olan süreci düşünün. Bu durumda mantissa şöyle olacaktır:

_ _ _ _ 1 1 0 0

Olmayacak

_ _ _ _ 0 1 1 0

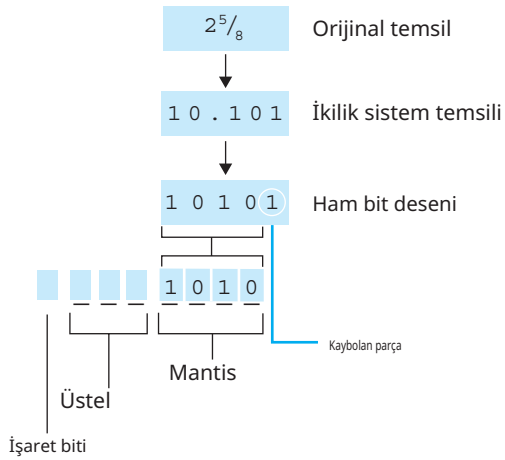
Bu, mantissa alanını *en soldaki 1* ile doldurduğumuz için ikili temsilin içinde görünen ilk 1 ile başlar. Bu kurala uyan temsiller normalleştirilmiş formda olarak adlandırılır.

Normalleştirilmiş form kullanmak, aynı değer için birden fazla temsil olasılığın ı ortadan kaldırır.Örneğin, hem 0011110 0 hem de 01000110, $^{3}/_{8}$ değerine çözüml enecektir, ancak yalnızca ilk desen normalleştirilmiş formdadır. Normalleştirilmiş f orma uymak, tüm sıfır olmayan değerler için temsilin 1 ile başlayan bir mantissa i çereceği anlamına gelir. Ancak, sıfır değeri özel bir durumdur; onun kayan nokta t emsili tamamen 0'lardan oluşan bir bit desenidir.

Kısaltma Hataları

$2^{5}/8$ değerini bir baytlık kayan nokta sistemimizle saklamaya çalıştığımızda ortaya çıkan sinir bozucu sorunu ele alalım. Öncelikle $2^{5}/_{8}$ değerini ikili olarak yazarız, bu da bize 10.101 verir. Ancak bunu mantissa alanına kopyaladığımızda yer kalmaz v e en sağdaki 1 (son $^{1}/_{8}$'yi temsil eden) kaybolur (Şekil 1.27). Eğer göz ardı edersek

Şekil 1.27 Değeri kodlama $2^{5}/_{8}$



Doclingo https://www.doclingo.ai

bu problemi şimdilik çözmek ve üstel alanı ve işaret bitini doldurarak devam etmek, $2^{1}/_{2}$ yerine 01101010 bit desenine ulaşmamıza neden olur $2^{5}/_{8}$. Gerçekleşen şeye kesme hatası veya yuvarlama hatası denir—bu, saklanan değerin bir kısmının kaybolduğu anlamına gelir çünkü mantissa alanıyeterince büyük değildir.

Bu tür hataların önemi, daha uzun bir mantissa alanı kullanılarak azaltılabilir.Aslında, günümüzde üretilen çoğu bilgisayar, burada kullandığımız 8 bit yerine, değerleri kayan nokta notasyonunda saklamak için en az 32 bit kullanmaktadır. Bu, aynı zamanda daha uzun bir üstel alanın kullanılmasına da olanak tanır. Ancak, bu daha uzun formatlarla bile, daha fazla doğruluğun gerektiği zamanlar hala vardır.

Kesme hatalarının bir diğer kaynağı, onluk notasyonda zaten alışkın olduğunuz bir fenomen olan: durmayan genişlemeler problemidir; bu, $^{1}/_{3}$'ü ondalık biçimde ifade etmeye çalışırken karşılaşılan durumlardır. Bazı değerler, kaç basamak kullanırsak kullanalım, doğru bir şekilde ifade edilemez. Geleneksel onluk notasyonumuz ile ikili notasyon arasındaki fark, ikili notasyonda daha fazla değerin durmayan temsillere sahip olmasıdır. Örneğin, ondalık biçimde ifade edildiğinde bir ondalık kesir durmayan bir değerdir. Bu durum, doları ve centleri saklamak ve işlemek için kayan nokta notasyonu kullanan dikkatsiz bir kişi için ne tür sorunlar yaratabileceğini hayal edin. Özellikle, dolar ölçü birimi olarak kullanıldığında, bir dime'ın değeri doğru bir şekilde saklanamaz. Bu durumda bir çözüm, tüm değerlerin tam sayılar olduğu ve iki'nin tamamlayanı gibi bir yöntem kullanılarak doğru bir şekilde saklanabileceği için verileri kuruş birimlerinde işlemektir.

Kesme hataları ve bunlarla ilgili problemler, sayısal analiz alanında çalışan insanlar için günlük bir endişe kaynağıdır. Bu matematik dalı, genellikle büyük olan ve önemli bir doğruluk gerektiren gerçek hesaplamalar yaparken karşılaşılan sorunlarla ilgilenir.

Aşağıda, herhangi bir sayısal analistin kalbini ısıtacak bir örnek bulunmaktadır. Varsayalım ki, daha önce tanımlanan bir baytlık kayan nokta notasyonumuzu kullanarak aşağıdaki üç değeri toplamak isteniyor:

$$2^{1}/_{2} \; {}^{1}/_{8} \; {}^{1}/_{8}$$

Tek Hassasiyetli Kayan Nokta

Bu bölümde tanıtılan kayan nokta notasyonu (Bölüm 1.7), gerçek bir bilgisayarda kullanılmak için çok basit kalmaktadır. Sonuçta, sadece 8 bit ile, tüm reel sayıların kümesinden yalnızca 256 sayı ifade edilebilir. Tartışmamız, örnekleri basit tutmak için 8 bit kullanılmıştır, ancak yine de önemli temel kavramları kapsamaktadır.

Günümüz bilgisayarlarının çoğu, 32 bitlik bu notasyonun Tek Hassasiyetli Kayan Nokta olarak adlandırılan bir formunu desteklemektedir. Bu format, işaret için 1 bit, üstel için 8 bit (bir fazlalık notasyonu içinde) ve mantissa için 23 bit kullanır. Böylece, tek hassasiyetli kayan nokta, çok büyük sayıları (yaklaşık 10^{38}) çok küçük sayılara (yaklaşık 10^{37}) kadar 7 ondalık basamak hassasiyeti ile ifade edebilir. Yani, belirli bir ondalık sayının ilk 7 basamağı oldukça iyi bir doğrulukla saklanabilir (küçük bir hata hala mevcut olabilir). İlk 7 basamaktan sonraki herhangi bir basamak kesinlikle kesme hatası nedeniyle kaybolacaktır (ancak sayının büyüklüğü korunur).

Başka bir form, **Double Precision Floating Point** olarak adlandırılan, 64 bit kullanır ve 15 ondalık basamağın hassasiyetini sağlar.

Doclingo https://www.doclingo.ai

Değerleri belirtilen sırayla toplarsak, önce $2\frac{1}{2}$ ile $\frac{1}{8}$'i toplarız ve $2\frac{5}{8}$ elde ederiz, bunun ikili karşılığı 10.101'dir. Ne yazık ki, bu değer doğru bir şekilde saklanam az(önceki örnekte görüldüğü gibi), bu nedenle ilk adımımızın sonucu $2\frac{1}{2}$ olarak saklanır(bu, topladığımız değerlerden birinin aynısıdır). Sonraki adım, bu sonu cu son $\frac{1}{8}$ ile toplamaktır. Burada da bir kesme hatası meydana gelir ve nihaiso nucumuz yanlış cevap olan $2\frac{1}{2}$ olur.

Şimdi değerleri ters sırayla toplayalım. Önce $\frac{1}{8}$'i $\frac{1}{8}$ ile toplarız ve $\frac{1}{4}$ elde ederiz. Bunun ikili karşılığı .01'dir; bu nedenle ilk adımımızın sonucu bir baytta 00111000 olarak saklanır, bu da doğrudur. Şimdi bu $\frac{1}{4}$'ü listedeki bir sonraki değer olan $2\frac{1}{2}$ ile toplarızve $2\frac{3}{4}$ elde ederiz, bu da bir baytta 01101011 olarak doğru bir şekilde saklanabilir. Bu seferki sonuçdoğru cevaptır.

Özetlemek gerekirse, ondalık gösterimle temsil edilen sayısal değerleri t oplarken, toplama sırası önemli olabilir. Sorun, eğerçok büyük bir sayı çok kü çük bir sayıya eklenirse, küçük sayının kesilebilmesidir. Bu nedenle, birden fa zla değeri toplarken genel kural,küçük değerleri önce toplamak ve bunların büyük değerlere eklendiğinde anlamlı bir değere ulaşmasını ummaktır. Bu, önceki örnekte deneyimlenen bir olguydu.

Bugünün ticari yazılım paketlerinin tasarımcıları, eğitimsiz kullanıcıyı bu tür soru nlardan koruma konusunda iyi bir iş çıkarıyor. Tipik bir elektronik tablo sisteminde, e klenen değerler 10^{16} veya daha fazla bir faktörle farklı olmadıkça doğru cevaplar eld e edilecektir. Bu nedenle, bir değere bir eklemeniz gerektiğini düşünüyorsanız

```
10,000,000,000,000,000
```

cevabı alabilirsiniz

```
10,000,000,000,000,000
```

bunun yerine

```
10,000,000,000,000,001
```

Bu tür sorunlar, küçük hataların ek hesaplamalarda birikerek önemli sonuçla r doğurabileceği uygulamalarda (navigasyon sistemleri gibi) önemlidir, anca k tipik bir PC kullanıcısı için çoğu ticari yazılımın sunduğu doğruluk derecesi yeterlidir.

Sorular & Alıştırmalar

1. Aşağıdaki bit desenlerini metinde tartışılan kayan nokta formatını kul lanarak çözün:

   a. 01001010 **b.** 01101101 **c.** 00111001 **d.** 11011100 **e.** 10101011

2. Aşağıdaki değerleri metinde tartışılan kayan nokta formatına kodlayı n. Kısaltma hatalarının meydana geldiğini belirtin.

   a. $2\frac{3}{4}$ **b.** $5\frac{1}{4}$ **c.** $\frac{3}{4}$ **d.** $-3\frac{1}{2}$ **ve.** $-4\frac{3}{8}$

3. Metinde tartışılan kayan nokta formatı açısından, 01001001 ve 0011110 1 desenlerinden hangisi daha büyük bir değeri temsil ediyor? Açıklayın.

Doclingo https://www.doclingo.ai

iki desenin hangisinin daha büyük değeri temsil ettiğini belirlemek için basit bir prosedür.

4. Metinde tartışılan kayan nokta formatını kullanırken, temsil edilebilecek en büyük değer nedir? Temsil edilebilecek en küçük pozitif değer nedir?

1.8 Veri Sıkıştırma

Veri depolamak veya aktarmak amacıyla, genellikle verilerin boyutunu azaltmak (ve bazen zorunlu olarak) ve temel bilgiyi korumak faydalıdır. Bunu başarmak için kullanılan teknik veri sıkıştırma olarak adlandırılır.
Bu bölüme bazı genel veri sıkıştırma yöntemlerini ele alarak başlıyoruz ve ardından belirli uygulamalar için tasarlanmış bazı yaklaşımlara bakıyoruz.

Genel Veri Sıkıştırma Teknikleri

Veri sıkıştırma şemaları iki kategoriye ayrılır. Bazıları kayıpsız, diğerleri iseka yıplıdır. Kayıpsız şemalar, sıkıştırma sürecinde bilgi kaybetmeyenlerdir. Kayıplı şemalar ise bilgi kaybına yol açabilenlerdir.
Kayıplı teknikler genellikle kayıpsız olanlardan daha fazla sıkıştırma sağlar ve bu nedenle küçük hataların tolere edilebileceği ortamlarda, örneğin görüntüler ve ses durumunda, popülerdir.

Sıkıştırılan verilerin aynı değerin uzun dizilerinden oluştuğu durumlarda, run-length encoding adı verilen sıkıştırma tekniği, kayıpsız bir yöntem olarak popülerdir. Bu, aynı veri öğelerinin dizilerini, tekrar eden öğeyi ve dizide kaç kez bulunduğunu belirten bir kod ile değiştirme işlemidir. Örneğin, bir bit deseninin 253 bir, ardından 118 sıfır, ardından 87 bir içerdiğini belirtmek için gereken alan, tüm 458 biti listelemekten daha azdır.

Bir diğer kayıpsız veri sıkıştırma tekniği frekansa bağlı kodlama dır; bu sistemde, bir veri öğesini temsil etmek için kullanılan bit deseninin uzunluğu, öğenin kullanım sıklığı ile ters orantılıdır. Bu tür kodlar, değişken uzunlukta kodların örnekleridir; bu, öğelerin farklı uzunluktaki desenlerle temsil edildiği anlamına gelir, Unicode gibi kodların aksine, burada tüm semboller 16 bit ile temsil edilir. David Huffman, frekansa bağlı kodlar geliştirmek için yaygın olarak kullanılan bir algoritmayı keşfetmekle tanınır ve bu şekilde geliştirilen kodlara Huffman kodları denmesi yaygın bir uygulamadır. Dolayısıyla, günümüzde kullanılan çoğu frekansa bağlı kod Huffman kodlarıdır.

Frekansa bağlı kodlamanın bir örneği olarak, kodlanmış İngilizce metin görevini düşünün. İngilizce dilinde e, t, a, ve iharfleri z, q, ve x'ten daha sık kullanılır. Bu nedenle, İngilizce dilindeki metin için bir kod oluştururken, önceki harfleri temsil etmek için kısa bit desenleri ve sonrakileri temsil etmek için daha uzun bit desenleri kullanarak alan tasarrufu sağlanabilir. Sonuç, İngilizce metnin, uniform uzunlukta kodlarla elde edilecek olandan daha kısa temsillere sahip olduğu bir kod olacaktır.

Bazı durumlarda, sıkıştırılacak veri akışı, her biri bir öncekinden yalnızca hafifçe farklı olan birimlerden oluşur. Bir örnek, bir sinema filminin ardışık kareleri olabilir. Bu durumlarda, göreceli

Doclingo https://www.doclingo.ai

kodlama, aynı zamanda diferansiyel kodlama olarak da bilinir, faydalıdır. Bu teknikler, ardışık veri birimleri arasındaki farkları kaydeder, tüm birimleri değil; yani, her birim, önceki birimle olan ilişkisi açısından kodlanır.
Göreceli kodlama, ardışık veri birimleri arasındaki farkların tam olarak mı yoksa yaklaşık olarak mı kodlandığına bağlı olarak kayıpsız veya kayıplı biçimde uygulanabilir.

Diğer popüler sıkıştırma sistemleri, sözlük kodlama tekniklerine dayanmaktadır. Burada sözlük terimi, sıkıştırılan mesajın inşa edildiği yapı taşlarının bir koleksiyonunu ifade eder ve mesajın kendisi, sözlüğe yapılan referansların bir dizisi olarak kodlanır. Sözlük kodlama sistemlerini genellikle kayıpsız sistemler olarak düşünürüz, ancak görüntü sıkıştırma konusundaki tartışmamızda göreceğimiz gibi, sözlükteki girişlerin doğru veri öğelerinin yalnızca yaklaşıkları olduğu zamanlar vardır ve bu da kayıplı bir sıkıştırma sistemine yol açar.

Sözlük kodlaması, kelime işlemcileri tarafından metin belgelerini sıkıştırmak için kullanılabilir çünkü bu işlemcilerdeki yazım denetimi amacıyla zaten bulunan sözlükler mükemmel sıkıştırma sözlükleri oluşturur. Özellikle, bir kelime, bu sözlüğe tek bir referans olarak kodlanabilir, bu da onu ASCII veya Unicode gibi bir sistem kullanarak kodlanmış bireysel karakterlerin dizisi olarak kodlamaktan daha verimlidir. Bir kelime işlemcisindeki tipik bir sözlük yaklaşık 25.000 giriş içerir, bu da bireysel bir girişin 0 ile 24.999 arasındaki bir tam sayı ile tanımlanabileceği anlamına gelir. Bu, sözlükteki belirli bir girişin yalnızca 15 bitlik bir desenle tanımlanabileceği anlamına gelir. Buna karşılık, referans alınan kelime altı harften oluşuyorsa, karakter bazında kodlaması 8 bitlik ASCII kullanarak 48 bit veya Unicode kullanarak 96 bit gerektirecektir.

Bir sözlük kodlaması varyasyonu, uyarlanabilir sözlük kodlaması (aynı zamandadinamik sözlük kodlaması olarak da bilinir) olarak adlandırılır. Uyarlanabilir bir sözlük kodlamasisteminde, sözlük kodlama süreci sırasında değişmesine izin verilir. Popüler birörnek, Lempel-Ziv-Welsh (LZW) kodlaması (yaratıcıları Abraham Lempel, Jacob Ziv ve Terry Welsh'in adını taşır). LZW kullanarak bir mesajı kodlamak için,birisi, mesajın inşa edildiği temel yapı taşlarını içeren bir sözlükle başlar, ancak mesajda daha büyük birimler bulunduğunda, bunlar sözlüğe eklenir—bu, o birimlerin gelecekteki tekrarlarının tek bir, çoklu sözlük referansları yerine kodlanabileceği anlamına gelir. Örneğin,ingilizce metni kodlarken, bireysel karakterler, rakamlar ve noktalama işaretlerini içeren bir sözlükle başlanabilir. Ancak mesajdaki kelimelerbelirlendikçe, bunlar sözlüğe eklenebilir. Böylece, sözlük mesaj kodlandıkça büyür ve sözlük büyüdükçe, mesajdaki daha fazla kelime (veyatekrarlayan kelime kalıpları) tek referanslar olarak kodlanabilir.

Sonuç, o belirli mesaja özgü oldukça büyük bir sözlük açısından kodlanmış bir mesaj olacaktır. Ancak bu büyük sözlüğün,mesajı çözmek için mevcut olması gerekmez. Sadece orijinal küçük sözlük gereklidir. Gerçekten de, çözme süreci,kodlama sürecinin başladığı aynı küçük sözlükle başlayabilir. Ardından, çözme süreci devam ettikçe,kodlama sürecinde bulunan aynı birimlerle karşılaşacak ve böylece bunları gelecekteki referanslar için sözlüğe ekleyebilecektir; tıpkı kodlama sürecinde olduğu gibi.

Mesajı netleştirmek için LZW kodlamasını uygulamayı düşünün

```
xyx xyx xyx xyx
```

Doclingo https://www.doclingo.ai

üç girişten oluşan bir sözlükle başlayarak, birincisi x, ikincisi y,ve üçüncüsü b ir boşluk. xyx'ı 121 olarak kodlamaya başlayacağız, bu da mesajınilk sözlük gi rişini, ardından ikincisini, ardından birincisini içeren bir desenle başladığı anl amına geliyor. Sonra boşluk, 1213 üretmek için kodlanır. Ancak, bir boşluğa ulaştığımızda, önceki karakter dizisinin bir kelime oluşturduğunu biliyoruz, b u yüzden deseni xyx dördüncü giriş olarak sözlüğe ekliyoruz. Bu şekilde deva m ederek, tüm mesaj 121343434 olarak kodlanır.

Eğer şimdi bu mesajı çözmemiz istenseydi, orijinal üç girişli sözlükle başl ayarak, ilk dizeyi 1213 olarak çözmeye başlardık ve bu xyx ardından bir boşlu k. Bu noktada xyx dizisinin bir kelime oluşturduğunu tanıyacak ve onu dördü ncü giriş olarak sözlüğe ekleyecektik, tıpkı kodlama sürecinde yaptığımız gibi . Mesajı çözmeye devam ederken, mesajdaki 4'ün bu yeni dördüncü girişe atı fta bulunduğunu tanıyacak ve onu xyx olarak çözeceğiz, desen üreterek

```
xyx xyx
```

Bu şekilde devam edersek, nihayetinde 121343434 dizisini şu şekilde çözeceğiz

```
xyx xyx xyx xyx
```

bu da orijinal mesajdır.

Görüntüleri Sıkıştırma

Bölüm 1.4'te, görüntülerin bit haritası teknikleri kullanılarak nasıl kodlandığını gördük. Ne yazık ki, üretilen bit haritaları genellikle çok büyüktür. Bunun sonucunda, görü ntü temsilleri için özel olarak geliştirilmiş birçok sıkıştırma şeması bulunmaktadır.

GIF olarak bilinen bir sistem (Grafik Değişim Formatı'nın kısaltması ve bazıl arı tarafından "Giff", diğerleri tarafından "Jiff" olarak telaffuz edilir) CompuServe tarafından geliştirilen bir sözlük kodlama sistemidir. Sıkıştırma sorununa, bir p iksele atanabilecek renk sayısını yalnızca 256 ile sınırlayarak yaklaşır. Bu renkler in her biri için kırmızı-yeşil-mavi kombinasyonu üç bayt kullanılarak kodlanır ve bu 256 kodlama, palet adı verilen bir tabloda (bir sözlükte) saklanır.

Bir görüntüdeki her piksel, değeri pikselin rengini temsil eden 256 palet girişinden h angisinin temsil ettiğini gösteren tek bir bayt ile temsil edilebilir. (Tek bir baytın 256 f arklı bit deseninden herhangi birini içerebileceğini unutmayın.) GIF'in, paletteki renkl erin orijinal görüntüdeki renklerle aynı olmayabileceğinden, rastgele görüntülere uy gulandığında kayıplı bir sıkıştırma sistemi olduğunu unutmayın.

GIF, bu basit sözlük sistemini LZW tekniklerini kullanarak uyarlanabilir bi r sözlük sistemine genişleterek ek sıkıştırma elde edebilir.Özellikle, kodlama süreci sırasında piksel desenleriyle karşılaşıldıkça, bunlar sözlüğe eklenir, bö ylece bu desenlerin gelecekteki oluşumları daha verimli bir şekilde kodlanabi lir.Böylece, nihai sözlük, orijinal paletin ve bir dizi piksel deseninin birleşimin den oluşur.

GIF paletindeki renklerden biri genellikle "şeffaf" değerine atanır, bu da ark a planın bu "renge" atanmış her bölgeden görünmesine izin verildiği anlamına gelir.Bu seçenek, GIF sisteminin göreceli basitliği ile birleştiğinde, birden fazla görüntünün bir bilgisayar ekranında hareket etmesi gereken basit animasyon uygulamalarında GIF'i mantıklı bir seçim haline getirir.Öte yandan, yalnızca 256 rengi kodlama yeteneği, fotoğrafçılık alanında olduğu gibi daha yüksek hassas iyetin gerektiği uygulamalar için uygun olmamasını sağlar.

Doclingo https://www.doclingo.ai

Görüntüler için bir başka popüler sıkıştırma sistemi **JPEG** (okunuşu "JAY-peg") 'dir. Bu, ISO içinde Ortak Fotoğraf Uzmanları Grubu tarafından geliştirilen bir sta ndarttır (bu nedenle standardın adı). JPEG, renkli fotoğrafları sıkıştırmak için etkili bir standart olduğunu kanıtlamış ve fotoğraf endüstrisinde yaygın olarak kullanıl maktadır; çoğu dijital kameranın JPEG'i varsayılan sıkıştırma tekniği olarak kullan ması bu durumu göstermektedir.

JPEG standardı aslında her biri kendi hedeflerine sahip birkaç görüntü sı kıştırma yöntemini kapsamaktadır. En yüksek hassasiyetin gerektiği durumla rda, JPEG kayıpsız bir mod sunar. Ancak, JPEG'in kayıpsız modu, diğer JPEG se çenekleriyle karşılaştırıldığında yüksek sıkıştırma seviyeleri üretmez. Ayrıca, diğer JPEG seçenekleri oldukça başarılı olmuştur, bu da JPEG'in kayıpsız modunun n adiren kullanıldığı anlamına gelir. Bunun yerine, JPEG'in temel standardı olarak bilinen seç enek (aynı zamanda JPEG'in kayıplı ardışık modu olarak da bilinir) birçok uygulamada tercih edilen standart haline gelmiştir.

JPEG temel standardını kullanarak görüntü sıkıştırma, bazıları insan gözü nün sınırlamalarından yararlanmak üzere tasarlanmış bir dizi adım gerektirir . Özellikle, insan gözü parlaklık değişikliklerine renk değişikliklerinden daha duyarlıdır. Bu nedenle, luminans ve krominans bileşenleri açısından kodlan mış bir görüntüden başlayarak, ilk adım ikiye iki piksel kareler üzerindeki kro minans değerlerini ortalamaktır. Bu, tüm orijinal parlaklık bilgilerini korurke n krominans bilgisinin boyutunu dört kat azaltır. Sonuç, görüntü kalitesinde fark edilebilir bir kayıp olmadan önemli bir sıkıştırma derecesidir.

Sonraki adım, resmi sekiz-by-sekiz piksel bloklarına ayırmak ve her bloğu n içindeki bilgiyi birim olarak sıkıştırmaktır. Bu, ayrıntılarına burada girmemi z gerekmeyen ayrık kosinüs dönüşümü olarak bilinen matematiksel bir tekni k uygulanarak yapılır. Önemli nokta, bu dönüşümün orijinal sekiz-by-sekiz bl oğu, piksel değerleri yerine orijinal bloktaki piksellerin birbirleriyle nasıl ilişkil i olduğunu yansıtan başka bir bloğa dönüştürmesidir. Bu yeni blok içinde, ön ceden belirlenmiş bir eşik değerinin altındaki değerler sıfırlarla değiştirilir; b u, bu değerlerin temsil ettiği değişikliklerin insan gözü tarafından algılanam ayacak kadar ince olduğunu yansıtır. Örneğin, orijinal blok bir dama tahtası deseni içeriyorsa, yeni blok tek tip bir ortalama rengi yansıtabilir. (Tipik bir se kiz-by-sekiz piksel bloğu, görüntü içinde çok küçük bir kareyi temsil edeceğin den, insan gözü dama tahtası görünümünü zaten tanımlayamaz.)

Bu noktada, ek sıkıştırma elde etmek için daha geleneksel koşu uzunluğu k odlama, göreli kodlama ve değişken uzunlukta kodlama teknikleri uygulanır. Bir araya getirildiğinde, JPEG'in temel standardı genellikle renkli görüntüleri en az 10 kat, ç oğu zaman da 30 kat kadar sıkıştırır ve bu, belirgin bir kalite kaybı olmadan gerçekleşir.

Görüntülerle ilişkili bir başka veri sıkıştırma sistemi **TIFF** (Tagged Image File Format'ın kısaltması)dır. Ancak, TIFF'in en popüler kullanımı veri sıkıştırm a aracı olarak değil, bunun yerine tarih, saat ve kamera ayarları gibi ilgili bilg ileri içeren fotoğrafları depolamak için standart bir format olarakdır. Bu bağl amda, görüntü genellikle sıkıştırılmadan kırmızı, yeşil ve mavi piksel bileşenl eri olarak depolanır.

TIFF standartları koleksiyonu, çoğu metin belgelerinin faks uygulamalarınd a görüntülerini sıkıştırmak için tasarlanmış veri sıkıştırma tekniklerini içermekt edir. Bu teknikler, metin belgelerinin uzun beyaz piksel dizilerinden oluştuğu g erçeğinden yararlanmak için run-length encoding varyasyonlarını kullanır.

Doclingo https://www.doclingo.ai

TIFF standartlarında yer alan renkli görüntü sıkıştırma seçeneği, GIF tarafından kullanılan t ekniklere benzer tekniklere dayanmaktadır ve bu nedenle fotoğrafçılık topluluğunda yaygı n olarak kullanılmamaktadır.

Ses ve Video Sıkıştırma

Ses ve video kodlama ve sıkıştırma için en yaygın kullanılan standartlar, ISO l iderliğinde **Motion Picture Experts Group (MPEG)** tarafından geliştirilmişt ir. Bu standartlar kendileri MPEG olarak adlandırılmaktadır.

MPEG, farklı uygulamalar için çeşitli standartları kapsamaktadır. Örneğin , yüksek çözünürlüklü televizyon (HDTV) yayın talepleri, yayın sinyalinin sınırlı yeteneklere sahip çeşitli iletişim yollarından geçmesi gereken video konfera ns taleplerinden farklıdır. Ayrıca, bu iki uygulama, videonun bölümlerin tekra r oynatılabileceği veya atlanabileceği bir şekilde depolanmasından da farklıdı r.

MPEG tarafından kullanılan teknikler, bu metnin kapsamının çok ötesindedir, bununla birlikte, genel olarak video sıkıştırma teknikleri, videonun, filmdeki hareketli resimlerin kaydedildiği şekilde, bir dizi resim olarak oluşturulmasına dayanmaktadır. Bu tür dizileri sıkıştırmak için, yalnızca bazı resimler, I-çerikleri olarak adlandırılan, tamamen kodlanır. I-çeriklerinin arasındaki resimler, göreli kodlama teknikleri kullanılarak kodlanır. Yani, tüm resmi kodlamak yerine, yalnızca önceki görüntüden olan farklılıkları kaydedilir. I-çerikleri genellikle JPEG'e benzer tekniklerle sıkıştırılır.

Ses sıkıştırma için en iyi bilinen sistem MP3'tür, MPEG standartları içinde geliştirilmiştir. Aslında, MP3 kısaltması, MPEG katmanı 3'ün kısaltmasıdır.
Diğer sıkıştırma teknikleri arasında, MP3 insan kulağının özelliklerindenfaydalanarak , insan kulağının algılayamayacağı detayları ortadan kaldırır. Buözelliklerden biri, geç ici maskeleme olarak adlandırılan, yüksek bir sesinardından kısa bir süre boyunca, in san kulağının başka türlü duyulabilirolan daha yumuşak sesleri algılayamamasıdır. D iğeri, frekans maskelemesiolarak adlandırılan, bir frekansta bir sesin, yakın frekanst a daha yumuşaksesleri maskeleme eğiliminde olmasıdır. Bu tür özelliklerden faydala narak,MP3, sesin CD kalitesine yakın bir ses kalitesini korurken önemli birses sıkıştır ması elde etmek için kullanılabilir.

MPEG ve MP3 sıkıştırma tekniklerini kullanarak, video kameraları 128MB depolama alanında bir saatin video kaydını yapabilir ve taşınabilir müzik çala rlar tek bir GB'de 400 popüler şarkı depolayabilir. Ancak, diğer ortamlardaki sıkıştırma hedeflerinin aksine, ses ve video sıkıştırmanın amacı mutlaka dep olama alanı tasarrufu sağlamak değildir. Bilgilerin günümüz iletişim sistemle ri üzerinden zamanında sunum sağlamak için yeterince hızlı bir şekilde iletil mesini sağlayan kodlamalar elde etme hedefi de aynı derecede önemlidir. H er video karesi 1 MB depolama alanı gerektiriyorsa ve kareler yalnızca saniye de 1 KB iletme kapasitesine sahip bir iletişim yolu üzerinden iletilmek zorund aysa, başarılı bir video konferans için hiçbir umut kalmaz. Bu nedenle, ses ve video sıkıştırma sistemleri genellikle, zamanında veri iletişimi için gereken il etim hızları açısından da değerlendirilir. Bu hızlar genellikle bit/saniye (bps) cinsinden ölçülür. Yaygın birimler **Kbps** (kilo-bps, bin bps'e eşit), **Mbps** (mega -bps, bir milyon bps'e eşit) ve **Gbps** (giga-bps, bir milyar bps'e eşit) içerir.MPE G tekniklerini kullanarak, video sunumları 40 Mbps transfer hızları sağlayan i letişim yolları üzerinden başarıyla iletilebilir. MP3 kayıtları genellikle 64 Kbps' den fazla transfer hızları gerektirmez.

Doclingo https://www.doclingo.ai

Sorular & Alıştırmalar

1. Dört genel sıkıştırma tekniğini listeleyin.
2. Mesajın kodlanmış versiyonu ne olurdu

   xy x y xy xyx xyxy xyxy

   eğer LZW sıkıştırması, metinde açıklandığı gibi x,y ve bir boşluk içeren sözlükle başlatılırsa?
3. Renkli çizgi filmleri kodlarken neden GIF, JPEG'den daha iyi olurdu?
4. Başka gezegenlere seyahat edecek ve fotoğraflar gönderecek bir uzay aracı tas arlayan bir ekibin parçası olduğunuzu varsayalım. Görüntüleri depolamak ve il etmek için gereken kaynakları azaltmak amacıyla fotoğrafları GIF veya JPEG'in t emel standardını kullanarak sıkıştırmak iyi bir fikir olur mu?
5. JPEG'in temel standardı insan gözünün hangi özelliğinden yararlanır ?
6. MP3 insan kulağının hangi özelliğinden yararlanır?
7. Sayısal bilgileri, görüntüleri ve sesi bit desenleri olarak kodlarken yay gın olan bir sorunlu olguyu tanımlayın.

1.9 İletişim Hataları

Bilgiler, bir bilgisayarın çeşitli parçaları arasında geri ve ileri aktarılırken, ya da Dünya'dan aya ve geri iletilirken, ya da bu konuda, yalnızca depoda bırakıldığın da, geri alınan bit deseninin nihayetinde orijinal olanla aynı olmama şansı vardı r. Manyetik bir kayıt yüzeyinde bulunan kir veya yağ parçacıkları ya da arızalı bi r devre, verilerin yanlış bir şekilde kaydedilmesine veya okunmasına neden ola bilir. Bir iletim yolundaki statik, verilerin bazı kısımlarını bozabilir.
Ve bazı teknolojilerin durumunda, normal arka plan radyasyonu, bir makinenin ana belleğinde depolanan desenleri değiştirebilir.

Bu tür sorunları çözmek için, hataların tespit edilmesine ve hatta düzeltil mesine olanak tanıyan çeşitli kodlama teknikleri geliştirilmiştir. Bugün, bu te knikler büyük ölçüde bir bilgisayar sisteminin iç bileşenlerine entegre edildiğ inden, makineyi kullanan personel için belirgin değildir. Yine de, varlıkları ön emlidir ve bilimsel araştırmalara önemli bir katkı sağlar. Bu nedenle, günüm üz ekipmanlarının güvenilirliğinin arkasında yatan bu tekniklerden bazılarını incelemek uygun olacaktır.

Parite Bitleri

Hataları tespit etmenin basit bir yöntemi, her bir bit deseninin işlenmesi sıra sında tek sayıda 1'e sahip olması ve çift sayıda 1'e sahip bir desenle karşılaşıl dığında bir hatanın meydana gelmiş olması ilkesine dayanmaktadır. Bu ilked en yararlanmak için, her desenin tek sayıda 1 içerdiği bir kodlama sistemine ihtiyacımız var. Bu, mevcut bir kodlama sistemine (belki yüksek sıralı uçta) ek bir bit, parite biti, ekleyerek kolayca elde edilir. Her durumda, bu yeni bite 1 veya 0 değeri atarız.

Doclingo https://www.doclingo.ai

sonuçta oluşan tüm desenin tek sayıda 1'e sahip olması için. Kodlama sistem imiz bu şekilde değiştirildikten sonra, çift sayıda 1'e sahip bir desen, bir hata meydana geldiğini ve işlenen desenin yanlış olduğunu gösterir.

Şekil 1.28, parite bitlerinin A ve F harflerinin ASCII kodlarına nasıl eklenec eğini göstermektedir.A'nın kodunun 101000001 (parite bit 1) haline geldiğini ve F'nin ASCII'sinin 001000110 (parite bit 0) haline geldiğini not edin. A'nın o rijinal 8 bitlik deseni çift sayıda 1'e sahipken, F'nin orijinal 8 bitlik deseni tek s ayıda 1'e sahiptir, ancak her iki 9 bitlik desen de tek sayıda 1'e sahiptir. Bu te knik, tüm 8 bitlik ASCII desenlerine uygulanırsa, hata, çift sayıda 1'e sahip he rhangi bir 9 bitlik desenle gösterileceği bir 9 bitlik kodlama sistemi elde ederi z.

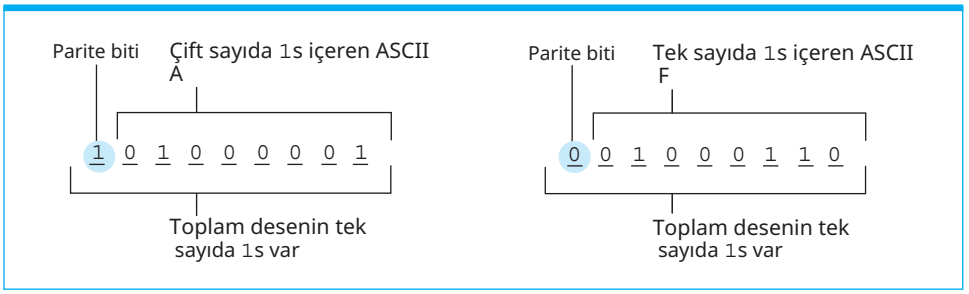
Tanımlanan parite sistemi tek parite olarak adlandırılır, çünkü sistemimizi her doğru desenin tek sayıda 1 içerecek şekilde tasarladık. Diğer bir teknik ise çift parite olarak adlandırılır. Çift parite sisteminde, her desenin çift sayıda 1 içe recek şekilde tasarlandığı ve dolayısıyla bir hata, tek sayıda 1 içeren bir desenin ortaya çıkmasıyla sinyal verildiği anlamına gelir.

Bugün, bir bilgisayarın ana belleğinde parite bitlerinin kullanılması alışılma dık değildir. Bu makinelerin 8 bit kapasiteli bellek hücrelerine sahip olduğunu h ayal etsek de, gerçekte her biri 9 bit kapasiteye sahiptir; bunun 1 biti parite biti olarak kullanılır. Her seferinde 8 bitlik bir desen belleğe depolanmak üzere veril diğinde, devre bir parite biti ekler ve ortaya çıkan 9 bitlik deseni depolar. Desen daha sonra geri alındığında, devre 9 bitlik desenin paritesini kontrol eder. Eğer bu bir hata göstermiyorsa, bellek parite bitini kaldırır ve kalan 8 bitlik deseni g üvenle geri döndürür. Aksi takdirde, bellek 8 veri bitini geri döndürür ve geri d önen desenin, başlangıçta belleğe emanet edilen desenle aynı olmayabileceğin e dair bir uyarı verir.

Parite bitlerinin basit kullanımı basit olsa da, sınırlamaları vardır. Eğer bir desen başlangıçta tek sayıda 1'e sahipse ve iki hata alırsa, hala tek sayıda 1' e sahip olacaktır ve dolayısıyla parite sistemi hataları tespit edemeyecektir. Aslında, parite bitlerinin basit uygulamaları, bir desendeki herhangi bir çift sayı da hatayı tespit edemez.

Bu sorunu en aza indirmenin bir yolu, bazen manyetik bir diskteki bir sektörde kaydedilen bit dizisi gibi uzun bit desenlerin e uygulanır. Bu durumda desen, desendeki bitlerin belirli bir koleksiyonu ile ilişkili olan bir dizi parite bitini içeren bir kontrol baytı ile birlikte gelir. Örneğin, bir parite biti, desendeki her sekizinci bit ile ilişkilendirilebilir ve bu ilk bitten başlayabilirken, başka bir p arite biti ikinci bitten başlayarak her sekizinci bit ile ilişkilendirilebilir. Bu şekilde, orijinal desendeki bir alanda yoğunlaşmış hata k oleksiyonlarının tespit edilme olasılığı daha yüksektir, çünkü bu, birkaç parite bitinin kapsamındadır. Bu kontrol baytı kavramının varyasyonları, kontrol toplamları ve döngüsel fazlalık olarak bilinen hata tespit şemalarına yol açar.

Şekil **1.28**A ve F harfleri için ayarlanmış ASCII kodları



Doclingo https://www.doclingo.ai

ilk bitten başlayarak, diğer bir parite biti ikinci bitten başlayarak her sekizinci bi t ile ilişkilendirilebilir. Bu şekilde, orijinal desendeki bir alanda yoğunlaşmış hat a koleksiyonlarının tespit edilme olasılığı daha yüksektir, çünkü bu, birkaç parit e bitinin kapsamındadır. Bu kontrol baytı kavramının varyasyonları, kontrol topl amları ve döngüsel fazlalık olarak bilinen hata tespit şemalarına yol açar. **kontroller (CRC).**

Hata Düzeltme Kodları

Bir parite bitinin kullanımı bir hatanın tespit edilmesini sağlasa da, hatayı dü zeltmek için gereken bilgiyi sağlamaz. Birçok insan, hata düzeltme kodlarının hataların yalnızca tespit edilmekle kalmayıp aynı zamanda düzeltilebileceği şekilde tasarlanabileceğine şaşırır. Sonuçta, sezgi, bir mesajda bulunan bilgi yi zaten bilmiyorsak, alınan bir mesajdaki hataları düzeltemeyeceğimizi söyle r. Ancak, böyle bir düzeltici özelliğe sahip basit bir kod Şekil 1.29'da sunulmu ştur.

Bu kodun nasıl çalıştığını anlamak için önce R. W. Hamming'in adını taşıy an Hamming mesafesi terimini tanımlıyoruz; Hamming, 1940'ların erken röle makinelerinin güvenilirliğindeki eksiklikten hayal kırıklığına uğradıktan sonr a hata düzeltme kodları arayışında öncülük etmiştir. İki bit deseni arasındaki Hamming mesafesi, desenlerin farklı olduğu bit sayısıdır. Örneğin, Şekil 1.29' daki A ve B'yi temsil eden desenler arasındaki Hamming mesafesi dört, B ve C arasındaki Hamming mesafesi ise üçtür. Şekil 1.29'daki kodun önemli özelli ği, herhangi iki desenin en az üç Hamming mesafesi ile ayrılmasıdır.

Şekil 1.29'daki bir desende tek bir bit değiştirildiğinde, hata tespit edilebilir çünk ü sonuç yasal bir desen olmayacaktır. (Herhangi bir desenin başka bir yasal desene b enzemesi için en az 3 bit değiştirmemiz gerekir.) Dahası, orijinal desenin ne olduğun u da anlayabiliriz. Sonuçta, değiştirilmiş desen, orijinal formundan yalnızca bir Ham ming mesafesi uzaklıkta olacak, ancak diğer yasal desenlerden en az iki Hamming m esafesi uzaklıkta olacaktır.

Böylece, başlangıçta Şekil 1.29 kullanılarak kodlanmış bir mesajı çözmek için, alınan her deseni koddaki desenlerle karşılaştırıyoruz ve alınan desene bir mesafede olan biri ni bulana kadar devam ediyoruz. Bunu, çözümleme için doğru sembol olarak kabul ediy oruz. Örneğin, eğer bit desenini aldık ve bu deseni koddaki desenlerle karşılaştırdıysak, 010100 bu deseni elde ederiz.

Şekil **1.29** Hata düzeltme kodu

Sembol	Kod
A	000000
B	001111
C	010011
D	011100
E	100110
F	101001
G	110101
H	111010

Doclingo https://www.doclingo.ai

Şekil **1.30** Deseni çözme 010100 Şekil 1.29 kullanarak.

Karakter	Kod	Desen alındı	Alınan desen ile kod arasındaki mesafe
A	0 0 0 0 0 0	0 1 0 1 0 0	2
B	0 0 1 1 1 1	0 **1 0** 1 **0 0**	4
C	0 1 0 0 1 1	0 1 0 **1 0 0**	3
D	0 1 1 1 0 0	0 1 **0** 1 0 0	**1** — En küçük distans
E	1 0 0 1 1 0	**0 1** 0 1 **0** 0	3
F	1 0 1 0 0 1	**0 1 0 1** 0 0	5
G	1 1 0 1 0 1	**0** 1 0 1 0 **0**	2
H	1 1 1 0 1 0	**0** 1 **0 1 0** 0	4

Şekil 1.30'daki tabloyu. Böylece, iletilen karakterin bir D olması gerektiği son ucuna varırız çünkü bu en yakın eşleşmedir.

Bu tekniği Şekil 1.29'daki kodla kullanmanın, aslında her desende iki hat ayı tespit etmemizi ve bir hatayı düzeltmemizi sağladığını gözlemleyeceksiniz. Eğer kodu, her desenin diğerlerinden en az beş Hamming mesafesinde olacak şekilde tasarlasaydık, her desende dört hatayı tespit edebilir ve iki hatayı düzeltebilirdik. Elbette, büyük Hamming mesafeleri ile ilişkili verimli kodların tasarımı basit bir görev değildir. Aslında, bu, cebirsel kodlama teorisi adı verilen matematik dalının bir parçasını oluşturur; bu da lineer cebir ve matris teorisi alanları içinde bir konudur.

Hata düzeltme teknikleri, bilgisayar ekipmanının güvenilirliğini artırmak için yaygın olarak kullanılmaktadır.Örneğin, manyetik yüzeydeki kusurların verileri bozma olasılığını azaltmak için genellikle yüksek kapasiteli manyetik disk sürücülerinde kullanılırlar. Ayrıca, ses diskleri için kullanılan orijinal CD formatı ile bilgisayar veri depolama için kullanılan sonraki format arasındaki önemli bir ayrım, hata düzeltme derecesidir. CD-DA formatı, hata oranını yalnızca iki CD'de bir hata olacak şekilde azaltan hata düzeltme özelliklerini içerir. Bu, ses kayıtları için oldukça yeterlidir, ancak yazılımları müşterilere sağlamak için CD kullanan bir şirket, disklerin yüzde 50'sindeki kusurların kabul edilemez olduğunu bulacaktır.

Böylece, veri depolama için kullanılan CD'lerde ek hata düzeltme özellikleri uygulanmakta, hata olasılığını 20.000 disk başına bir hata olacak şekilde azaltmaktadır.

Sorular & Alıştırmalar

1. Aşağıdaki baytlar başlangıçta tek parite kullanılarak kodlanmıştır. Hangisinde bir hata meydana geldiğini biliyorsunuz?

   a. 100101101 **b.**000000000 **c.**000000000
   d. 111000000 **e.**111111111

2. Soru 1'de bir baytta hatalar meydana gelmiş olabilir mi, bunu bilmeden? Cevabınızı açıklayın.

Doclingo https://www.doclingo.ai

3. Eğer tek parite yerine çift paritenin kullanıldığını söyleselerdi, Soru 1 ve 2'ye verdiğiniz cevaplar nasıl değişirdi?
4. Bu cümleleri, her karakter kodunun yüksek sıralı ucuna bir parite biti ekleyerek tek parite kullanarak ASCII'ye kodlayın:

   a. "Dur!" Cheryl bağırdı. b. $2 + 3 = 5?$

5. Şekil 1.29'da sunulan hata düzeltme kodunu kullanarak,aşağıdaki me sajları çözün:

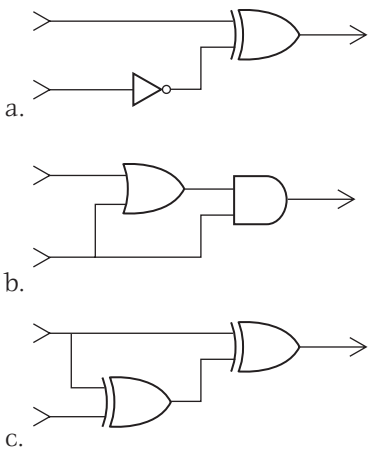
   a. `001111 100100 001100` b. `010001 000000 001011`
   c. `011010 110110 100000 011100`

6. A, B, C ve D karakterleri için, Hamming mesafesininherhangi iki desen arasın da en az üç olacak şekilde,uzunluğu beş olan bit desenleri kullanarak bir kod oluşturun.

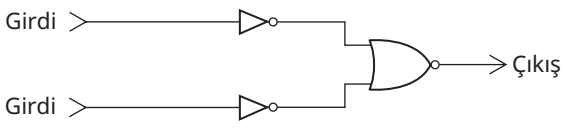
Bölüm Gözden Geçirme Problemleri

(Yıldızlı problemler isteğe bağlı bölümlerle ilişkilidir.)

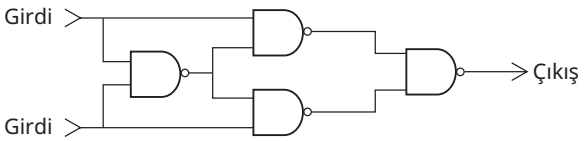
1. Aşağıdaki devrelerin her birinin çıktısını beli rleyin,üst girişin 1 ve alt girişin 0 olduğu nu varsayarak. Üst giriş 0ve alt giriş 1 old uğunda çıkış ne olur?



2. a. Devre hangi Boolean işlemini hesaplıyor?



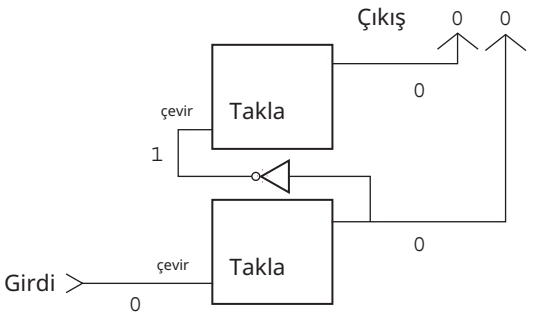
b. Devre hangi Boolean işlemini hesaplıyor?



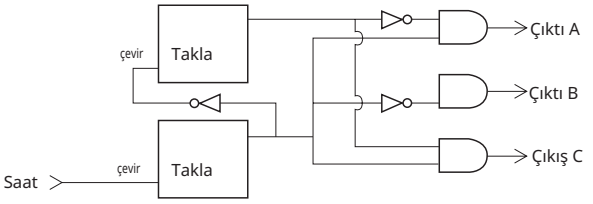
*3. a. Eğer bir elektronik bileşen mağazasından bir f lip-flop devresi satın alırsak,bunun flip adın da ek bir girişi olduğunu görebiliriz. Bu giriş 0'dan 1'e değiştiğinde, çıkış du rumu değişir (eğer 0 ise şimdi 1'dir vete rsi de geçerlidir). Ancak, flip girişi 1'de n 0'a değiştiğinde, hiçbir şey olmaz. Bu davranışı gerçekleştirmek için gereke n devre detaylarını bilmesek de, bu cihazı diğer devrelerde soyut bir araç ola rak kullanabiliriz. Aşağıdaki iki flip-flop kullanan devreyi düşünün.
Eğer devrenin girişine bir darbe gönderilirse, alt flip-flop durumu değiştirecektir. Ancak, ikinci flip-flop değişmeyecek, çünkü girişi (NOT kapısından alınan) 1' den 0'a geçti.
Sonuç olarak, bu devre artık 0 ve 1 çıktıl arı üretecektir. İkinci bir darbe,

Doclingo https://www.doclingo.ai

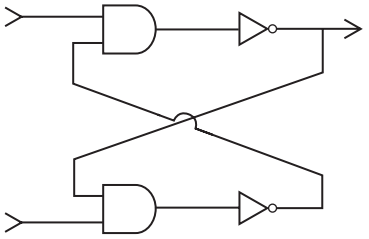
her iki flip-flop'un durumunu değiştirerek 1 ve 0 çıktısı üretecektir. Üçüncü bir darbeden sonra çıkış ne olur? Dördüncü bir darbeden sonra?



b. Bir bilgisayar içindeki çeşitli bileşenlerin aktivitelerini koordine etmek genellikle gereklidir. Bu, bir puls eden sinyali (bir *saat* olarak adlandırılan) a bölümüne benzer devreye bağlayarak gerçekleştirilir. Ek kapılar (görüldüğü gibi) daha sonra diğer bağlı devrelere koordine bir şekilde sinyaller gönderecektir. Bu devreyi incelediğinizde, saat 1st, 5th, 9th . . . darbelerinde A çıkışında 1 gönderileceğini doğrulayabilmelisiniz. Saatin hangi darbelerinde B çıkışında 1 gönderilecektir? Saatin hangi darbelerinde C çıkışında 1 gönderilecektir? Saatin 4th darbesinde hangi çıkışta 1 gönderilmektedir?



4. Her iki girişin de 1 olduğunu varsayın. Üst girişin geçici olarak 0'a değiştirilmesi durumunda ne olacağını açıklayın. Alt girişin geçici olarak 0'a değiştirilmesi durumunda ne olacağını açıklayın. Devreyi NAND kapıları kullanarak yeniden çizin.



5. Aşağıdaki tablo, bir makinenin ana belleğindeki hücrelerin adreslerini ve içeriklerini (onaltılık notasyon kullanarak) temsil etmektedir.

Bir makinenin ana belleğindeki bazı hücrelerin içeriklerini temsil etmektedir.

Bu bellek düzeni ile başlayarak, talimatların sırasını izleyin ve bu bellek hücrelerinin her birinin son içeriklerini kaydedin:

Adres	İçindekiler
00	AB
01	53
02	D6
03	02

Adım 1. Adresi 03 olan hücrenin içeriğini, adresi 00 olan hücreye taşıyın.

Adım 2. Değer 01'i adresi 02 olan hücreye taşıyın.

Adım 3. Adresi 01'de saklanan değeri, adresi 03 olan hücreye taşıyın.

6. Her hücrenin adresi iki onaltılık basamakla temsil edilebiliyorsa, bir bilgisayarın ana belleğinde kaç hücre olabilir? Dört onaltılık basamak kullanıldığında ne olur?

7. Aşağıdaki onaltılık notasyonlar tarafından temsil edilen bit desenleri nelerdir?

. CD b. 67 c. 9A
d. FF Ve. 10

8. Aşağıdaki onaltılık notasyonlarla temsil edilen bit desenlerinde en anlamlı bitin değeri nedir?

a. 8F b. FF
c. 6F d. 1F

9. Aşağıdaki bit desenlerini onaltılık notasyonda ifade edin:

a. 101000001010
b. 110001111011
c. 000010111110

10. Diyelim ki bir dijital kameranın depolama kapasitesi 256MB'dır. Her biri 1024 piksel satır ve 1024 piksel sütun içeren fotoğraflar kamerada kaç tane depolanabilir? Her pikselin üç bayt depolama gerektirdiğini varsayalım.

11. Bir resmin, 1024 sütun ve 768 satırdan oluşan dikdörtgen bir dizi ile bir ekran üzerinde temsil edildiğini varsayalım. Her piksel için, rengi kodlamak için 8 bit ve yoğunluğu kodlamak için başka bir 8 bit gerektiğinde, tüm resmi tutmak için kaç bayt boyutunda bellek hücresine ihtiyaç vardır?

Doclingo https://www.doclingo.ai

12. a. Ana belleğin manyetik disk depolama üzerindeki iki avantajını belirleyin.
b. Manyetik disk depolamanın ana bellek üzerindeki iki avantajını belirleyin.

13. Varsayalım ki kişisel bilgisayarınızın 120GB'lık sabit disk sürücüsünün yalnızca 50GB'ı boştur. Tüm verilerinizi yedeklemek için CD'leri kullanmak makul olur mu? Peki ya DVD'ler?

14. Eğer bir manyetik disk üzerindeki her sektör 1024 bayt içeriyorsa, her karakterin Unicode ile temsil edildiği varsayıldığında, tek bir metin sayfasını (belki 100 karakterden oluşan 50 satır) depolamak için kaç sektöre ihtiyaç vardır?

15. ASCII kullanıldığında, her sayfasında 3500 karakter bulunan 400 sayfalık bir romanı depolamak için kaç bayt depolama alanına ihtiyaç duyulacaktır? Unicode kullanıldığında kaç bayt gerekecektir?

16. Tipik bir hard disk sürücüsünün 360 devir/dakika hızında dönerken gecikme süresi ne kadardır?

17. 360 devir/dakika hızında dönen bir hard disk için ortalama erişim süresi nedir? Arama süresi 10 milisaniye olarak varsayıldığında?

18. Bir daktilocu her gün sürekli olarak dakikada 60 kelime yazabiliyorsa, 640MB kapasitesine sahip bir CD'yi doldurması ne kadar sürer? Bir kelimenin beş karakter olduğunu ve her karakterin bir bayt depolama gerektirdiğini varsayın.

19. İşte ASCII formatında bir mesaj. Ne diyor?

```
01010111 01101000 01100001 01110100
00100000 01100100 01101111 01100101
01110011 00100000 01101001 01110100
00100000 01110011 01100001 01111001
00111111
```

20. Aşağıdaki, her karakter için bir bayt kullanılarak ASCII formatında kodlanmış ve ardından onaltılık notasyonda temsil edilmiş bir mesajdır. Mesaj nedir?

```
68657861646563696D616C
```

21. Aşağıdaki cümleleri ASCII formatında, her karakter için bir bayt kullanarak kodlayın.
a. 100 / 5 eder mi? $= 20?$
b. Toplam maliyet $7.25'tir.

22. Önceki problemin cevaplarını onaltılık notasyonda ifade edin.

23. 8 ile 18 arasındaki tam sayıların ikili temsillerini listeleyin.

24. a. 2 ve 3'ü ASCII ile temsil ederek 23 sayısını yazın.
b. 23 sayısını ikili temsilde yazın.

25. Hangi değerlerin yalnızca bir bitinin 1 olduğu ikili temsilleri vardır? Bu özelliğe sahip en küçük altı değerin ikili temsillerini listeleyin.

***26.** Aşağıdaki ikili temsillerin her birini onluk tabandaki karşılığına dönüştürün:

a. `1111`	b. `0001`	c. `10101`
d. `1000`	ve. `10011`	f. `000000`
g. `1001`	h. `10001`	ben. `100001`
j. `11001`	k. `11010`	l. `11011`

***27.** Aşağıdaki onluk sayı sistemindeki temsilleri eşdeğer ikilik sayı sistemine dönüştürün:

a. 7	b. 11	c. 16
d. 17	ve. 31	

***28.** Aşağıdaki fazla 16 temsilini eşdeğer onluk temsiline dönüştürün:

a. `10001`	b. `10101`	c. `01101`
d. `01111`	ve. `11111`	

***29.** Aşağıdaki onluk temsilini eşdeğer fazla dört temsiline dönüştürün:

a. 0	b. 3	c. -2
d. -1	e. 2	

***30.** Aşağıdaki iki'nin tamamlayıcı temsillerini eşdeğer onluk temsiline dönüştürün:

a. `01111`	b. `10100`	c. `01100`
d. `10000`	ve. `10110`	

***31.** Aşağıdaki onluk temsilini eşdeğer iki'nin tamamlayıcı temsiline dönüştürün; her değer 7 bit olarak temsil edilmiştir:

a. 13	b. -13	c. -1
d. 0	ve. 16	

***32.** Aşağıdaki toplama işlemlerini gerçekleştirinizbit dizilerinin iki'nin tamamı notasyonunda değerleri temsil ettiğini varsayarak. Her birini tanımlayın

Doclingo https://www.doclingo.ai

sonucun taşma nedeniyle yanlış ol duğu durumlar.

a. 00101 + 01000
b. 11111 + 00001
c. 01111 + 00001
d. 10111 + 11010
e. 11111 + 11111
f. 00111 + 01100

***33.** Aşağıdaki her bir problemi, değerleri iki'nin tamam ı notasyonuna çevirerek (5 bitlik desenler kullan arak), herhangi bir çıkarma problemini eşdeğer bir toplama problemine dönüştürerek ve o topla ma işlemini gerçekleştirerek çözün. Cevabınızı on aylamak için, cevabınızı onluk tabana çevirin. (Ta şma durumuna dikkat edin.)

a. 5 − 1
b. 5 − −1
c. 12 − −5
d. 8 − −7
Ve. 12 − 5
f. 5 − −11

***34.** Aşağıdaki ikili temsilleri, eşdeğer on luk taban temsiline çevirin:

a. 11.11
b. 100.0101
c. 0.1101
d. 1.0
Ve. 10.01

***35.** Aşağıdaki her bir değeri ikili notasyon da ifade edin:

a. $5\frac{3}{4}$
b. $15\frac{15}{16}$
c. $5\frac{3}{8}$
d. $1\frac{1}{4}$
Ve. $6\frac{5}{8}$

***36.** Aşağıdaki bit desenlerini Şekil 1.26'da tanımlanan kayan nokta formatını kullanarak çözün:

a. 01011001
b. 11001000
c. 10101100
d. 00111001

***37.** Aşağıdaki değerleri Şekil 1.26'da tanımlanan 8-bit kayan nokta formatını kullanarak kodlayın. Herhangi bir kesme hatasının meydana geldiği d urumu belirtin.

a. $-7\frac{1}{2}$
b. $\frac{1}{2}$
c. $-3\frac{3}{4}$
d. $\frac{7}{32}$
Ve. $\frac{31}{32}$

***38.** Normalleştirilmiş form kullanma kısıtlamanız yok sa,Şekil 1.26'da tanımlanan kayan nokta formatın ı kullanarak$3/8$ değerini temsil etmek için kull anılabilecek tüm bit desenlerini listeleyin.

***39.** Şekil 1.26'da tanımlanan 8-bit kayan nokta formatında ifade edilebilecek en iyi yaklaşım nedir?
Bu yaklaşım bir makine tarafından bukayan nokta f ormatını kullanarak kare alındığındagerçekte hangi değer elde edilir?

***40.** Bir ondan birinin değerine en iyi yaklaşım, Ş ekil 1.26'da tanımlanan 8 bitlik kayan no kta formatında nasıl temsil edilebilir?

***41.** Metre sistemi kullanılarak yapılan ölçümlerin kayan nokta notasyonunda kaydedilmesi sırasında ha taların nasıl oluşabileceğini açıklayın. Örneğin, 1 10 cm metre cinsinden kaydedilirse ne olur?

***42.** 01011 ve 11011 bit desenlerinden biri, fazla 1 6 notasyonunda saklanan bir değeri temsil e derken, diğeri aynı değeri iki'nin tamamlaya nı notasyonunda saklar.
a. Bu ortak değer hakkında ne belirlenebi lir?
b. İki'nin tamamlayıcı notasyonunda saklan an bir değeri temsil eden bir desen ile aynı değeri aşırı notasyonda temsil ed en desen arasındaki ilişki nedir? Her i ki sistem de aynı bit desen uzunluğunu kullandığında?

***43.** Üç bit deseni 10000010, 01101000 ve 000 00010, Şekil 1.26'da sunulan iki'nin tamaml ayıcısı, aşırı ve 8-bit kayan nokta formatın da aynı değerin temsilleridir, ancak b u sırayla olmayabilir.
Ortak değer nedir ve hangi desen hangi notasyondadır?

***44.** Aşağıdaki değerlerden hangisi Şekil 1.26'da tanıtılan kayan nokta formatında doğru bi r şekilde temsil edilemez?

a. $6\frac{1}{2}$
b. $\frac{13}{16}$
c. 9
d. $\frac{17}{32}$
Ve. $\frac{15}{16}$

***45.** Tam sayıları ikili olarak temsil etmek için kull anılan bit dizilerinin uzunluğunu 4 bitten 6 bite değiştirirseniz, temsil edebileceğiniz e n büyük tam sayının değerinde ne değişiklik olur? İki'nin tamamlayıcı notasyonu kulla nıyorsanız ne olur?

***46.** Her hücrenin bir bayt kapasitesine sahip ol duğu 4MB'lık bir bellekteki en büyük bellek adresinin onaltılık temsili ne olurdu?

***47.** Mesajın kodlanmış versiyonu ne olur du?

xxy yyx xxy xxy yyx

LZW sıkıştırması, 1.8 Bölümde açıklandığı gibi x,y ve bir boşluk içeren sözlükle başla tılırsa, kullanılır mı?

Doclingo https://www.doclingo.ai

***48.** Aşağıdaki mesaj, ilk, ikinci ve üçüncü girişler i sırasıyla x,y ve boşluk olan bir sözlük k ullanılarak LZW sıkıştırması ile sıkıştırıl mıştır. Sıkıştırılmamış mesaj nedir?

```
22123113431213536
```

***49.** Eğer mesaj

```
xxy yyx xxy xxyy
```

LZW kullanılarak sıkıştırılmış ve başlangıç sözlüğünün birinci, ikinci ve üçüncü öğeleri sırasıyla x,y ve boşluk ise, son sözlükteki öğeler ne olurdu?

***50.** Bir sonraki bölümde öğreneceğimiz gibi, gelenekselTelefon sistemleri üzerinden bi t iletmenin biryolu, bit desenlerini sese dö nüştürmek, sesitelefon hatları üzerinden ilet mek ve ardındansesi tekrar bit desenlerine dönüştürmektir. Buteknikler 57.6 Kbps'lik transfer hızlarıylasınırlıdır. Video MPEG kull anılarak sıkıştırıldığında,telekonferans için bu yeterli midir?

***51.** Aşağıdaki cümleleri ASCII formatında eşit parite k ullanarak bir parite biti ekleyerek kodlayın her karakterin yüksek sıralı ucunda kodu:

a. 100/5 yapar mı $= 20?$

b. Toplam maliyet $7.25'tir.

***52.** Aşağıdaki mesaj, her kısa bit dizisinde tek parite ile başlangıçta iletilmiştir. Hangi dizilerde kesinlikle hatalar oluşmuştur?

```
11001 11011 10110 00000 11111 10001
10101 00100 01110
```

***53.** 24 bitlik bir kod, her sembolü ASCII temsil inin üç ardışık kopyasıyla temsil edere k üretilirse (örneğin, A sembolü bit dizisi 010000010100000101000001 ile temsil edilir). Bu yeni kodun hangi hata düzeltme özellikleri vardır?

***54.** Şekil 1.30'da tanımlanan hata düzeltme kodunu kullanarak, aşağıdaki kelimeleri çözün:

a. `111010 110110`

b. `101000 100110 001100`

c. `011101 000110 000000 010100`

d. `010010 001000 001110 101111 000000 110111 100110`

ve. `010011 000000 101001 100110`

Sosyal Sorunlar

Aşağıdaki sorular, bilgisayar alanıyla ilgili etik/sosyal/hukuki sorunlara rehberlik etmek amacıyla hazırlanmıştır. Amaç, bu soruları yalnızca yanıtlamak değildir. Ayrıca, neden böyle yanıt verdiğinizi ve gerekçelerinizin bir sorudan diğerine tutarlı olup olmadığını da düşünmelisiniz.

1. Bir kesme hatası kritik bir durumda meydana geldi ve geniş çapta hasar ve can kaybına neden oldu. Eğer birisi sorumluysa, kimdir? Donanımın tasarımcısı mı? Yazılımın tasarımcısı mı? O programın o kısmını gerçekten yazan programcı mı? O yazılımı belirli bir uygulamada kullanmaya karar veren kişi mi? Yazılım, ilk geliştiren şirket tarafından düzeltilmiş olsaydı, ancak bu güncelleme kritik uygulamada satın alınmamış ve uygulanmamış olsaydı ne olurdu? Yazılım korsan olarak mı elde edilmişti?
2. Bireyin kendi uygulamalarını geliştirirken kesme hatalarının ve sonuçlarının olasılığını göz ardı etmesi kabul edilebilir mi?
3. 1970'lerde yılı temsil etmek için yalnızca iki rakam kullanarak yazılım geliştirmek etik miydi (örneğin, 1976 yılını temsil etmek için 76 kullanmak), yüzyılın dönüşü yaklaşırken yazılımın hatalı olacağını göz ardı ederek? Bugün yılı temsil etmek için yalnızca üç rakam kullanmak (örneğin, 1982 için 982 ve 2015 için 015) etik mi? Sadece dört rakam kullanmak ne olacak?

Doclingo https://www.doclingo.ai

4. Birçok kişi, bilgilerin kodlanmasının genellikle bilgiyi sulandırdığını veya başka bir şekilde çarpıttığını savunuyor, çünkü bu esasen bilgiyi nicelleştirmeye zorlar. Bir anketin, katılımcıların görüşlerini birden beşe kadar bir ölçekle yanıtlamaları gerektiği durumun doğası gereği hatalı olduğunu iddia ediyorlar. Bilgi ne ölçüde nicelleştirilebilir? Atık bertaraf tesisi için farklı yerlerin artı ve eksileri nicelleştirilebilir mi? Nükleer enerji ve nükleer atık konusundaki tartışma nicelleştirilebilir mi? Kararları ortalamalar ve diğer istatistiksel analizler üzerine kurmak tehlikeli mi? Haber ajanslarının anket sonuçlarını, soruların tam metnini dahil etmeden raporlaması etik mi? Bir insan hayatının değerini nicelleştirmek mümkün mü? Bir şirketin, ek yatırımın ürünün kullanımına bağlı bir ölüm olasılığını azaltabileceği halde, bir ürünün iyileştirilmesine yatırım yapmayı durdurması kabul edilebilir mi?
5. Verilerin toplanması ve yayılması hakkı, verinin biçimine bağlı olarak ayrım yapılmalı mı? Yani, fotoğraf, ses veya video toplama ve yayma hakkı, metin toplama ve yayma hakkıyla aynı mı olmalı?
6. İster kasıtlı olsun ister olmasın, bir gazeteci tarafından sunulan bir rapor genellikle bu gazetecinin önyargısını yansıtır. Genellikle sadece birkaç kelime değiştirerek, bir hikayeya olumlu ya da olumsuz bir anlam kazanabilir. (Karşılaştırın, "Anket yapılanların çoğunluğu referandumakarşıydı." ile "Anket yapılanların önemli bir kısmı referandumu destekledi.") Bir hikayeyi değiştirmekle(belirli noktaları atlayarak veya kelimeleri dikkatlice seçerek) bir fotoğrafı değiştirmek arasındabir fark var mı?
7. Bir veri sıkıştırma sisteminin kullanılması, ince ama önemli bilgi unsurlarının kaybına yol açıyorsa, hangi sorumluluk sorunları ortaya çıkabilir? Bunlar nasıl çözülmelidir?

Ek Okuma

Drew, M. ve Z. Li. *Multimedyanın Temelleri.*Upper Saddle River, NJ: Prentice-Hall, 2004.

Halsall, F. *Multimedya İletişimi.*Boston, MA: Addison-Wesley, 2001.

Hamacher, V. C., Z. G. Vranesic ve S. G. Zaky. *Bilgisayar Organizasyonu,*5. baskı. New York: McGraw-Hill, 2002.

Knuth, D. E. *Bilgisayar Programlamanın Sanatı,*Cilt 2, 3. baskı. Boston, MA: Addison-Wesley, 1998.

Long, B. *Tam Dijital Fotoğrafçılık,* 3. baskı. Hingham, MA: Charles River Media, 2005.

Miano, J. *Sıkıştırılmış Görüntü Dosya Biçimleri*. New York: ACM Press, 1999.

Petzold, C. *KOD: Bilgisayar Donanımı ve Yazılımının Gizli Dili*. Redman, WA: Microsoft Press, 2000.

Salomon, D. *Veri Sıkıştırma: Tam Referans,*4. baskı. New York: Springer, 2007.

Sayood, K. *Veri Sıkıştırmaya Giriş,* 3. baskı. San Francisco: Morgan Kaufmann, 2005.

Doclingo https://www.doclingo.ai

BÖLÜM

Veri Manipülasyonu 2

Bu bölümde bir bilgisayarın verileri nasıl işlediğini ve yazıcı lar ve klavyeler gibi çevresel cihazlarla nasıl iletişim kurdu ğunu öğreneceğiz. Bunu yaparken, bilgisayar mimarisinin temellerini keşfedecek ve bilgisayarların makine dili talima tları olarak adlandırılan kodlanmış talimatlar aracılığıyla n asıl programlandığını öğreneceğiz.

2.1 Bilgisayar Mimarisi
CPU Temelleri
Depolanan Program Kavramı

2.2 Makine Dili
Talimat Repertuvarı
Örnek Bir Makine Dili

2.3 Programın Çalıştırılması
ProgramınÇalıştırılmasına Bir Örnek
Programlar ve Veri

***2.4Aritmetik/ Mantık Talimatları**
Mantık İşlemleri
Döndürme ve Kaydırma İşlemleri
Aritmetik İşlemler

***2.5Diğer Cihazlarla İletişim**
Kontrolörlerin RolüDoğr udan Bellek ErişimiElleri n Tokalaşması
Popüler İletişim Ortamları
İletişim Hızları

***2.6Diğer Mimariler**
Pipeline'lama
Çok İşlemcili Makineler

**Yıldızlar, isteğe bağlı bölümler için önerileri belirtir.*

Doclingo https://www.doclingo.ai

Bölüm 1'de, bir bilgisayar içindeki verilerin depolanmasıyla ilgili konuları inceledik. Bu bölümde, bir bilgisayarın bu verileri nasıl manipüle ettiğini göreceğiz. Bu manipül asyon,verileri bir yerden başka bir yere taşımak ve aritmetik hesaplamalar, metin dü zenleme ve görüntü manipülasyonu gibi işlemleri gerçekleştirmekten oluşur. Bilgisayar mimarisini veri depolama sistemlerinin ötesine genişleterek anla mamıza başlıyoruz.

2.1 Bilgisayar Mimarisi

Verilerin manipülasyonunu kontrol eden bir bilgisayardaki devreye merkezi i şlem birimi, veya**CPU** (genellikle sadece işlemci olarak adlandırılır) denir. Yir minci yüzyılın ortalarındaki makinelerde, CPU'lar, birimlerin önemini yansıta n birkaç raf elektronik devre içeren büyük birimlerdi. Ancak, teknoloji bu ciha zları önemli ölçüde küçültmüştür. Günümüz masaüstü bilgisayarlarında ve di züstü bilgisayarlarda bulunan CPU'lar, makinenin ana devre kartına (ana kar t olarak adlandırılan) takılan bağlantı pinleriyle birlikte küçük düz kareler (ya klaşık iki inç'e iki inç) olarak paketlenmiştir.

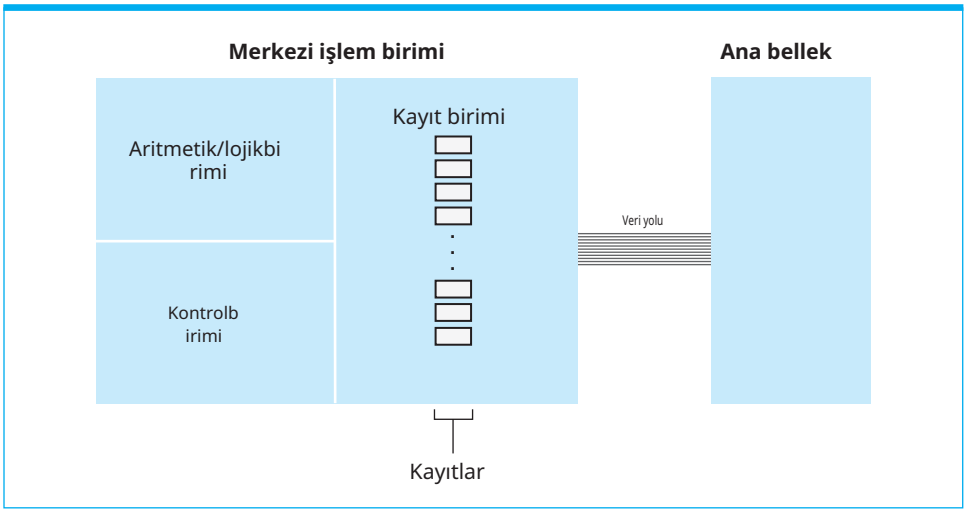
Akıllı telefonlarda, mini dizüstü bilgisayarlarda ve diğer Mobil İnternet Cihazlarında (MID), CPU'lar bir pulun boyutunun yarısı kadar. Küçük boyutları nedeniyle, bu işlemcilere mikroişlemciler denir.

CPU Temelleri

Bir CPU üç bölümden oluşur (Şekil 2.1): aritmetik/lojik birim, veriler üzerinde işlemler gerçekleştiren devreleri (toplama veçıkarma gibi) içeren; kontrol bi rimi, makinenin aktivitelerini koordine eden devreleri içeren; ve kayıt birimi, CPU içinde bilgilerin geçici olarak saklanması için kullanılan, ana bellek hücre lerine benzer veri depolama hücrelerini (kayıtlar olarak adlandırılır) içeren.

Kayıt birimi içindeki bazı kayıtlar genel amaçlı kayıtlar olarak kabul edilir ken, diğerleri özel amaçlı kayıtlar olarak kabul edilir. Bazılarını tartışacağız.

Şekil **2.1** CPU ve ana bellek bir veri yolu aracılığıyla bağlı



Doclingo https://www.doclingo.ai

Özel amaçlı kayıtlar Bölüm 2.3'te. Şu anda, yalnızca genel amaçlı kayıtlara od aklanıyoruz.

Genel amaçlı kayıtlar, CPU tarafından işlenen veriler için geçici depolama alanı olarak hizmet eder. Bu kayıtlar, aritmetik/lojik birimin devrelerine girişl eri tutar ve o birim tarafından üretilen sonuçlar için depolama alanı sağlar. A na bellek içinde depolanan verilere bir işlem gerçekleştirmek için, kontrol bir imi verileri bellekten genel amaçlı kayıtlara aktarır, aritmetik/lojik birime han gi kayıtların verileri tuttuğunu bildirir, aritmetik/lojik birim içindeki uygun de vreleri etkinleştirir ve aritmetik/lojik birime hangi kaydın sonucu alması gere ktiğini söyler.

Bit desenlerini aktarmak amacıyla, bir makinenin CPU'su ve ana belleği bir **bus** (bakınız tekrar Şekil 2.1) olarak adlandırılan bir dizi tel ile bağlanmıştır. Bu bus aracılığıyla, CPU ilgili bellek hücresinin adresini sağlayarak ve bellek devrelerine belirtilen hücredeki veriyi geri alması gerektiğini belirten bir elektronik sinyal göndererek ana bellekten veri çıkarır (okur). Benzer bir şekilde, CPU verileri belleğe yerleştirirken, hedef hücrenin adresini ve saklanacak veriyi uygun elektronik sinyal ile birlikte sağlar ve ana belleğe kendisine gönderilen veriyi saklaması gerektiğini bildirir.

Bu tasarıma dayanarak, ana bellekte saklanan iki değeri toplama görevi,t amamen toplama işleminin gerçekleştirilmesinden daha fazlasını içerir. Veril erana bellekten CPU içindeki kayıtlara aktarılmalı, değerler toplanmalı veson uç bir kayıtta yer almalıdır; ardından sonuç bir bellek hücresinesaklanmalıdır . Tüm süreç, Şekil 2.2'de listelenen beş adım ile özetlenmiştir.

Depolanan Program Kavramı

Erken bilgisayarlar esneklikleri ile tanınmıyordu—her cihazın uyguladığı adımlar, makinenin bir parçası olarak kontrol birimine entegre edilmişti. Daha fazla esneklik kazanmak için, bazı erken elektronik bilgisayarlar CPU'nun kolayca yeniden bağlanabilmesi için tasarlandı. Bu esneklik, eski telefon santrallerine benzer bir delikli pano düzeni aracılığıyla sağlandı; burada atlama tellerinin uçları deliklere takılıyordu.

Şekil **2.2** Bellekte saklanan değerleri ekleme

Adım 1. Bellekten eklenmesi gereken de
ğerlerden birini al ve onu bir
kaydediciye yerleştir.

Adım 2. Bellekten eklenmesi gereken
diğer değeri al ve onu
başka bir kaydediciye yerleştir.

Adım 3. Eklemeyi sağlayan devreyi Adım 1
ve 2'de kullanılan kayıtlarla
giriş olarak etkinleştir ve son
ucu tutmak için belirlenen baş
ka bir kaydedici kullan.

Adım 4. Sonucu belleğe kaydet.

Adım 5. Durdur.

Doclingo https://www.doclingo.ai

Önbellek Belleği

Bir bilgisayardaki bellek olanaklarını işlevselliği ile karşılaştırmak öğreticidir . Kayıtlar, mevcut işlemler için hemen uygulanabilir verileri tutmak için kull anılır; ana bellek, yakın gelecekte ihtiyaç duyulacak verileri tutmak için kulla nılır; ve kütle depolama, muhtemelen yakın gelecekte ihtiyaç duyulmayacak verileri tutmak için kullanılır. Birçok makine, önbellek belleği olarak adlandı rılan ek bir bellek seviyesi ile tasarlanmıştır. Önbellek belleği, CPU'nun ken disi içinde bulunan yüksek hızlı belleğin bir kısmıdır (belki birkaç yüz KB). B u özel bellek alanında, makine, şu anda ilgi alanında olan ana belleğin o kıs mının bir kopyasını tutmaya çalışır. Bu ayarda, normalde kayıtlar ve ana bel lek arasında yapılacak veri transferleri, kayıtlar ve önbellek belleği arasında yapılır. Önbellek belleğinde yapılan herhangi bir değişiklik, daha uygun bir zamanda ana belleğe topluca aktarılır. Sonuç, ana bellek iletişimi tarafında n geciktirilmediği için makine döngüsünü daha hızlı bir şekilde gerçekleştirebilen bir CPU'dur.

Bir atılım (görünüşe göre yanlış bir şekilde John von Neumann'a atfedile n) programın, veriler gibi, ana bellekte kodlanabileceği ve saklanabileceği g erçeği ile geldi. Kontrol birimi, programı bellekten çıkarmak, talimatları çözm ek ve yürütmek için tasarlanmışsa, makinenin izlediği program, yalnızca bilgi sayarın belleğinin içeriğini değiştirerek değiştirilebilir; CPU'yu yeniden kablol amak yerine.

Bir bilgisayarın programını ana belleğinde saklama fikri, saklanan-progra m kavramı olarak adlandırılır ve günümüzde kullanılan standart yaklaşım halin e gelmiştir—o kadar standart ki, bu durum açık görünüyor. Başlangıçta zorlaştı ran şey, herkesin programları ve verileri farklı varlıklar olarak düşünmesiydi: Veriler bellekte saklandı; programlar CPU'nun bir parçasıydı. Sonuç, ağaçları görememek için ormanı görmemek konusunda birinci sınıf bir örnekti. Böyle tuzaklara yakalanmak kolaydır ve bilgisayar biliminin gelişimi, bugün hala bi rçok tuzağın içinde olabiliriz, bunu bilmeden. Gerçekten de, bilimin heyecanı nın bir kısmı, yeni içgörülerin sürekli olarak yeni teorilere ve uygulamalara k apılar açmasıdır.

Sorular & Alıştırmalar

1. Bir bilgisayardaki bir bellek hücresinin içeriğini başka bir bellek hücresine taşımak için hangi olaylar dizisinin gerekli olduğunu düşünüyorsunuz?
2. Bir bellek hücresine bir değer yazmak için CPU'nun ana bellek devresine h angi bilgileri sağlaması gerekir?
3. Kitlesel depolama, ana bellek ve genel amaçlı kayıtlar hepsi depolam a sistemleridir. Kullanımlarındaki fark nedir?

Doclingo https://www.doclingo.ai

2.2 Makine Dili

Depolanan program kavramını uygulamak için, CPU'lar bit desenleri olarak kod lanmış talimatları tanıyacak şekilde tasarlanmıştır. Bu talimatlar koleksiyonu ve kodlama sistemi makine dili olarak adlandırılır. Bu dilde ifade edilen bir talima t, makine düzeyinde bir talimat veya daha yaygın olarak bir makine talimatı ola rak adlandırılır.

Talimat Repertuvarı

Tipik bir CPU'nun çözmesi ve yürütmesi gereken makine talimatları listesi oldu kça kısadır. Aslında, bir makine belirli temel ama iyi seçilmiş görevleri yerine ge tirebildiğinde, daha fazla özellik eklemek makinenin teorik yeteneklerini artırm az. Diğer bir deyişle, belirli bir noktadan sonra, ek özellikler yalnızca kullanım k olaylığı gibi şeyleri artırabilir, ancak makinenin temel yeteneklerine hiçbir şey e klemez.

Makine tasarımlarının bu durumdan ne ölçüde yararlanması gerektiği, CPU m imarisinin iki felsefesine yol açmıştır. Birincisi, bir CPU'nun minimum bir makine t alimatı setini yürütmek üzere tasarlanması gerektiğidir. Bu yaklaşım, azaltılmış ta limat seti bilgisayarı (RISC) olarak adlandırılan bir duruma yol açar. RISC mimarisi nin lehine yapılan argüman, böyle bir makinenin verimli, hızlı ve üretimi daha ucu z olduğudur. Öte yandan, diğerleri, birçok talimatın teknik olarak gereksiz olması na rağmen, karmaşık bir çok talimatı yürütme yeteneğine sahip CPU'lar lehine ar gümanlar öne sürmektedir. Bu yaklaşımın sonucu, karmaşık talimat seti bilgisaya rı (CISC) olarak bilinir. CISC mimarisinin lehine yapılan argüman, daha karmaşık C PU'nun sürekli artan karmaşıklıklarla daha iyi başa çıkabileceğidir.

Ne Kimin İcadı?

Bir icat için tek bir bireye kredi vermek her zaman şüpheli bir girişimdir. Thomas Edison, akkor lambayı icat etmekle suçlanıyor, ancak diğeraraştırm acılar benzer lambalar geliştiriyordu ve bir anlamda Edison'unpatenti alma sı için şanslı olduğu söylenebilir. Wright kardeşler, uçağı icat etmeklesuçlan ıyor, ancak birçok çağdaşıyla rekabet ediyorlardı ve onların çalışmalarından faydalandılar; hepsi de bir dereceye kadar, on beşinci yüzyılda uçan makine lerleoynayan Leonardo da Vinci tarafından geçilmişti. Hatta Leonardo'nun t asarımlarıönceki fikirlere dayanıyordu. Elbette, bu durumlarda belirlenen ic atçı halaverilen krediye meşru taleplere sahiptir. Diğer durumlarda, tarih u ygun olmayan birşekilde kredi vermiş gibi görünüyor—örneğin, saklanan p rogram kavramı. Şüphesiz,John von Neumann, birçok katkısı için kredi alma yı hak eden parlak bir bilim insanıydı.Ancak popüler tarihin ona kredi verm eyi seçtiği katkılardan biri, saklanan programkavramı, görünüşe göre Penn sylvania Üniversitesi'ndeki Moore Elektrik MühendisliğiOkulu'nda J. P. Ecker t liderliğindeki araştırmacılar tarafından geliştirilmiştir.John von Neumann, yalnızca bu fikri rapor eden çalışmayı yayınlayan ilk kişioldu ve bu nedenle hesaplama bilgisi onu icatçı olarak seçmiştir.

Doclingo https://www.doclingo.ai

bugünün yazılımlarının. CISC ile programlar, RISC tasarımında çoklu talimat dizisi gerektiren birçok talimatın güçlü ve zengin bir setinden yararlanabilir.

1990'larda ve binyılda, ticari olarak mevcut CISC ve RISC işlemcileri masaüstü bilgisayarlarda hakimiyet için aktif olarak rekabet ediyordu. PC'lerde kullanılan Intel işlemcileri, CISC mimarisinin örnekleridir; PowerPC işlemcileri (Apple, IBM ve Motorola arasında bir ittifak tarafından geliştirilen) RISC mimarisinin örnekleridir ve Apple Macintosh'ta kullanılmıştır. Zamanla, CISC'in üretim maliyeti önemli ölçüde düşmüştür; bu nedenle Intel'in işlemcileri (veya AMD - Advanced Micro Devices, Inc.'den eşdeğerleri) artık neredeyse tüm masaüstü ve dizüstü bilgisayarlarda bulunmaktadır (hatta Apple artık Intel ürünlerine dayalı bilgisayarlar üretmektedir).

CISC masaüstü bilgisayarlardaki yerini güvence altına alırken, elektrik gücüne doymak bilmeyen bir açlığı vardır. Buna karşılık, Advanced RISC Machine (ARM) şirketi, düşük güç tüketimi için özel olarak bir RISC mimarisi tasarlamıştır. (Advanced RISC Machine, başlangıçta Acorn Computers'tı ve şimdi ARM Holdings'dir.) Böylece, Qualcomm ve Texas Instruments gibi birçok satıcı tarafından üretilen ARM tabanlı işlemciler, oyun kumandalarında, dijital TV'lerde, navigasyon sistemlerinde, otomotiv modüllerinde, cep telefonlarında, akıllı telefonlarda ve diğer tüketici elektroniği ürünlerinde kolayca bulunabilir.

RISC ve CISC arasındaki seçimden bağımsız olarak, bir makinenin talimatlarıüç gruba ayrılabilir: (1) veri transfer grubu, (2) aritmetik/lojik grup ve (3) kontrol grubu.

Veri Transferi Veri transfer grubu, verilerin bir yerden başka bir yere hareketini talep eden talimatlardan oluşur. Şekil 2.2'deki 1, 2 ve 4. adımlar bu kategoriye girer. Bu talimat grubunu tanımlamak için *transfer* veya*move* gibi terimlerin kullanılmasının aslında yanlış bir adlandırma olduğunu belirtmeliyiz. Aktarılan verinin orijinal konumundan silinmesi nadirdir. Bir transfer talimatında yer alan süreç, daha çok veriyi kopyalamak gibidir, taşımaktan ziyade. Bu nedenle, bu talimat grubunun *hareketlerini daha iyi tanımlayan terimler copy veya clone dır.*

Terminoloji konusunu ele alırken, CPU ile ana bellek arasındaki veri transferine atıfta bulunurken özel terimlerin kullanıldığını belirtmeliyiz. Genel amaçlı bir kayıt boşluğunu bir

Değişken Uzunlukta Talimatlar

Metindeki açıklamaları basitleştirmek için, bu bölümdeki örneklerde kullanılan makine dili (ve Ek C'de tanımlanan) tüm talimatlar için sabit bir boyut (iki bayt) kullanır. Bu nedenle, bir talimat almak için CPU her zaman iki ardışık bellek hücresinin içeriğini alır ve program sayacını iki artırır. Bu tutarlılık, talimatları alma görevini kolaylaştırır ve RISC makinelerine özgüdür.

CISC makineleri, ancak, talimatlarının uzunluğunun değiştiği makine dillerine sahiptir. Örneğin, günümüz Intel işlemcileri, tek baytlık talimatlardan, talimatın kesin kullanımına bağlı olarak değişen çoklu baytlık talimatlara kadar uzanan talimatlara sahiptir. Bu tür makine dillerine sahip CPU'lar, gelen talimatın uzunluğunu talimatın op-kodu ile belirler. Yani, CPU önce talimatın op-kodunu alır ve ardından alınan bit desenine dayanarak, talimatın geri kalanını elde etmek için bellekten ne kadar daha fazla bayt alması gerektiğini bilir.

Doclingo https://www.doclingo.ai

Hafıza hücresi genellikle bir YÜK talimatı olarak adlandırılır; tersine, bir kaydı n içeriğini bir hafıza hücresine aktarma isteği bir DEPOLAMA talimatı olarak adlandırılır. Şekil 2.2'de, Adım 1 ve 2 YÜK talimatlarıdır ve Adım 4 bir DEPOLA MA talimatıdır.

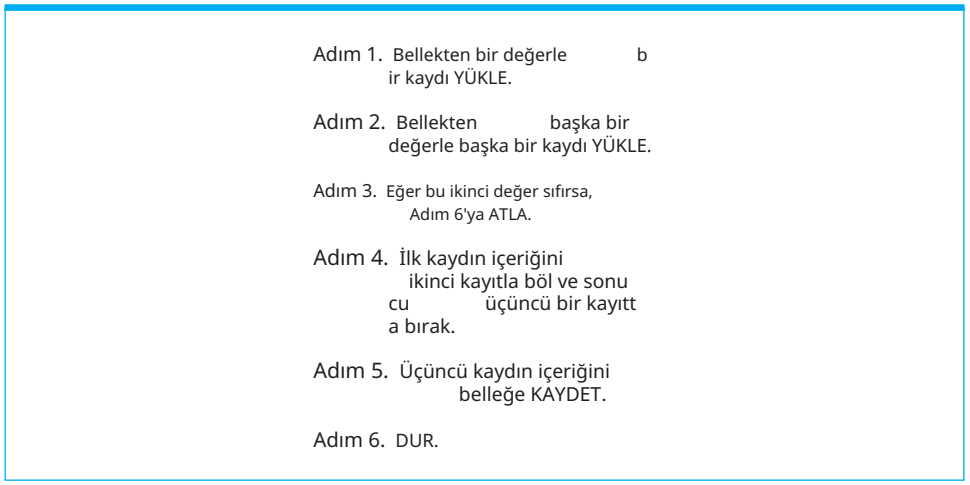
Veri transferi kategorisindeki önemli bir talimat grubu, CPU-ana bellek b ağlamının dışındaki cihazlarla iletişim kurmak için komutları içerir (yazıcılar, klavyeler, ekranlar, disk sürücüleri vb.). Bu talimatlar, makinenin giriş/çıkış (I/ O) aktivitelerini yönettiğinden, G/Ç talimatları olarak adlandırılır ve bazen ke ndi başlarına bir kategori olarak kabul edilirler. Öte yandan, Bölüm 2.5, bu G /Ç aktivitelerinin CPU ile ana bellek arasında veri transferi talep eden aynı tal imatlar tarafından nasıl yönetilebileceğini açıklar. Böylece, G/Ç talimatlarını v eri transfer grubu içinde bir parça olarak değerlendireceğiz.

Aritmetik/ Mantık Aritmetik/mantık grubu, kontrol birimine aritmetik/mantık birimi içinde bir etkinlik talep etmesini söyleyen talimatlardan oluşur. Şekil 2. 2'deki Adım 3 bu gruba girer. Adından da anlaşılacağı gibi, aritmetik/mantık birimi temel aritmetik işlemler dışında işlemler gerçekleştirme yeteneğine sa hiptir. Bu ek işlemlerden bazıları, Bölüm 1'de tanıtılan VE, VEYA ve XOR Boole an işlemleridir; bunları bu bölümde daha ayrıntılı olarak tartışacağız.

Diğer bir işlem grubu, çoğu aritmetik/mantık biriminde mevcut olan, kay ıtların içeriğinin kayıtta sağa veya sola taşınmasına olanak tanır. Bu işlemler, bitlerin kaydın ucundan "düştüğü" durumlarda yalnızca atıldığı (SHIFT) veya diğer uçta bırakılan boşlukları doldurmak için kullanıldığı (ROTATE) durumlar a bağlı olarak ya SHIFT ya da ROTATE işlemleri olarak bilinir.

Kontrol Kontrol grubu, programın yürütülmesini yönlendiren talimatlardan o luşur, veri manipülasyonundan ziyade. Şekil 2.2'deki Adım 5 bu kategoriye gi rer, ancak son derece temel bir örnektir. Bu grup, bir makinenin repertuarın daki daha ilginç talimatların çoğunu içerir; örneğin, CPU'yu listedeki bir sonr aki talimattan başka bir talimatı yürütmeye yönlendirmek için kullanılan JUM P (veya BRANCH) talimatları ailesi. Bu JUMP talimatları iki çeşitte görünür: ko şulsuz atlamalar ve koşullu atlamalar.

Şekil **2.3** Bellekte saklanan değerlerin bölünmesi



Doclingo https://www.doclingo.ai

Öncekilerin bir örneği “Adım 5'e atla” talimatı olurdu; sonrakilerin bir örneği i se, “Elde edilen değer 0 ise, o zaman Adım 5'e atla.”dır. Ayrım, koşullu bir atla manın yalnızca belirli bir koşul sağlandığında “yer değişikliği” ile sonuçlanma sıdır. Örneğin, Şekil 2.3'teki talimat dizisi, Adım 3'ün sıfıra bölme olasılığına k arşı koruma sağlayan bir koşullu atlama olduğu iki değeri bölme algoritması nı temsil eder.

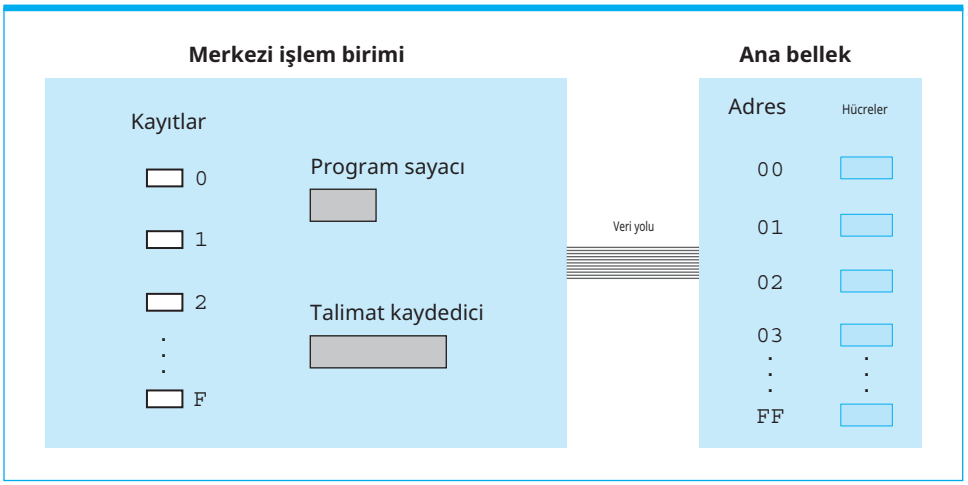
Öğretici Bir Makine Dili

Şimdi tipik bir bilgisayarın talimatlarının nasıl kodlandığını düşünelim. Görüşmemiz için kullanacağımız makine, Ek C'de tanımlanmış ve Şekil 2.4'te öz etlenmiştir. 16 genel amaçlı kayıt ve her biri 8 bit kapasiteye sahip 256 ana bell ek hücresi vardır. Referans amaçları için, kayıtları 0'dan 15'e kadar olan değerle rle etiketliyoruz ve bellek hücrelerini 0'dan 255'e kadar olan değerlerle adresliy oruz. Kolaylık olması açısından, bu etiketleri ve adresleri iki tabanda temsil edil en değerler olarak düşünüyoruz ve elde edilen bit desenlerini onaltılık notasyo n kullanarak sıkıştırıyoruz. Böylece, kayıtlar 0'dan F'ye kadar etiketleniyor ve bel lek hücreleri 00'dan FF'ye kadar adresleniyor.

Bir makine talimatının kodlanmış versiyonu iki bölümden oluşur: op-kod u(operasyon kodunun kısaltması) alanı ve **operand** alanı. Op-kodu alanında görünen bit deseni, talimatın hangi temel işlemleri, örneğin STORE, SHIFT, X OR ve JUMP, talep ettiğini gösterir. Operand alanında bulunan bit desenleri, op-kod tarafından belirtilen işlem hakkında daha ayrıntılı bilgi sağlar. Örneği n, bir STORE işlemi durumunda, operand alanındaki bilgi hangi kaydın sakla nacak veriyi içerdiğini ve hangi bellek hücresinin veriyi alacağını gösterir.

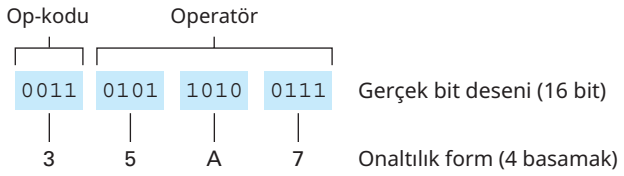
Örnek makinemizin (Ek C) tüm makine dili yalnızca on iki temel talimatta n oluşur. Bu talimatların her biri toplamda 16 bit kullanılarak kodlanmıştır ve dört onaltılık basamakla temsil edilir (Şekil 2.5). Her talimatın op-kodu ilk 4 b itten veya eşdeğer olarak ilk onaltılık basamaktan oluşur. Bu op-kodların 1'd en C'ye kadar olan onaltılık basamaklarla temsil edildiğini unutmayın (Ek C). Özellikle, Ek C'deki tablo

Şekil **2.4** Ek C'de tanımlanan makinenin mimarisi



Doclingo https://www.doclingo.ai

Şekil **2.5**Ek C'deki makine için bir talimatın bileşimi

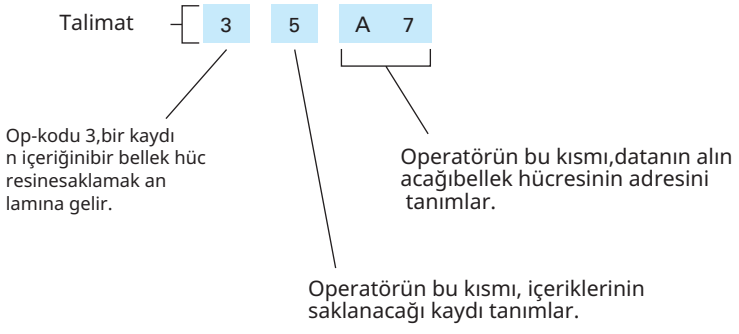


bize, 3 onaltılık basamağı ile başlayan bir talimatın bir STOREtalimatına, A on altılık basamağı ile başlayan bir talimatın ise birROTATE talimatına atıfta bulu nduğunu bildirir.

Örnek makinemizdeki her talimatın operand alanı üç onaltılık basamakta n(12 bit) oluşur ve her durumda (HALT talimatı hariç, bu talimatın daha fazla ayrıntıya ihtiyacı yoktur) op-kod tarafından verilen genel talimatı netleştirir.Ö rneğin (Şekil 2.6), bir talimatın ilk onaltılık basamağı 3 (bir kaydedicininiçeriği ni saklamak için op-kod) olursa, talimatın bir sonraki onaltılıkbasamağı hangi kaydedicinin saklanacağını belirtir ve son iki onaltılıkbasamak hangi bellek h ücresinin veriyi alacağını gösterir. Böylece 35A7(tam onaltılık) talimatı "5 nu maralı kaydedicide bulunan bit desenini A7adresine sahip bellek hücresine S TORE et" ifadesine çevrilir. (Onaltılıknotasyonun kullanımının tartışmamızı na sıl basitleştirdiğine dikkat edin.Gerçekte, 35A7 talimatı bit deseni 001101011 0100111'dir.)

(35A7 talimatı, ana bellek kapasitelerinin neden iki kuvveti cinsindenölçüld üğüne dair açık bir örnek de sunar. Bu talimatta 8 bit, bu talimatile kullanılan b ellek hücresini belirtmek için ayrılmıştır, bu nedenle tamolarak 2^8 farklı bellek h ücresine atıfta bulunmak mümkündür. Bu nedenle,bu kadar hücreye sahip ana bellek inşa etmemiz gerekir—0'dan 255'e kadaradreslenmiş. Eğer ana bellek d aha fazla hücreye sahip olsaydı, bunlararasındaki ayrımı belirten talimatlar yaz amazdık; eğer ana bellek daha azhücreye sahip olsaydı, var olmayan hücrelere atıfta bulunan talimatlaryazabilirdik.)

Şekil **2.6**Talimatın çözülmesi 35A7



Doclingo https://www.doclingo.ai

Op-kodu tarafından verilen genel talimatı netleştirmek için operand alan ının nasıl kullanıldığına dair başka bir örnek olarak, iki kaydın OR edilmesini t alep eden 7 (onaltılık) op-koduna sahip bir talimatı düşünün. (İki kaydı OR et menin ne anlama geldiğini Bölüm 2.4'te göreceğiz. Şu anda yalnızca talimatl arın nasıl kodlandığıyla ilgileniyoruz.) Bu durumda, bir sonraki onaltılık raka m, sonucun yerleştirilmesi gereken kaydı belirtirken, son iki onaltılık rakam h angi iki kaydın OR edileceğini gösterir. Böylece talimat 70C5, "C kaydının içer iğini 5 kaydının içeriği ile OR et ve sonucu 0 kaydında bırak" ifadesine çevrilir .

Makinemizin iki LOAD talimatı arasında ince bir ayrım vardır. Burada, op-kodu 1'in (onaltılık) bir bellek hücresinin içeriği ile bir kayıt yükley en bir talimatı tanımladığını, op-kodu 2'nin (onaltılık) ise belirli bir değerle bir kayıt yükleyen bir talimatı tanımladığını görüyoruz. Fark, birinci türdeki bir talimatın operand alanının bir adres içermesi, ikinci t ürde ise operand alanının yüklenmesi gereken gerçek bit desenini içermesid ir.

Makinenin iki ADD talimatı olduğunu unutmayın: biri iki'nin tamamlayanı temsillerini toplamak için ve diğeri kayan nokta temsillerini toplamak için. B u ayrım, iki'nin tamamlayanı notasyonunda kodlanmış değerleri temsil eden bit desenlerini toplamanın, kayan nokta notasyonunda kodlanmış değerleri t oplamaktan farklı etkinlikler gerektirmesinin bir sonucudur.

Bu bölümü, Şekil 2.2'deki talimatların kodlanmış bir versiyonunu içeren Ş ekil 2.7 ile kapatıyoruz. Toplanacak değerlerin 6C ve 6D bellek adreslerinde i ki'nin tamamlayanı notasyonunda saklandığını ve toplamın 6E adresindeki b ellek hücresine yerleştirileceğini varsaydık.

Şekil **2.7**Şekil 2'deki talimatların kodlanmış bir versiyonu.2

Şifrelenmiş talimatlar	Çeviri
156C	Kayıt 5'i, 6C adresindeki bellek hücre sinde bulunan bit deseni ile y ükleyin.
166D	6. kayıtçıyı, 6D adresindeki bellek hü cresinde bulunan bit desenin e yükleyin.
5056	Register 5 ve 6'nın içeriklerini, san ki bunlar iki'nin tamamlayıcı temsiliymiş gibi ekleyin ve sonu cu register 0'da bırakın.
306E	Register 0'ın içeriğini 6E adresindeki bellek hücresine kaydedin.
C000	Dur.

Doclingo https://www.doclingo.ai

Sorular & Alıştırmalar

1. Terim *hareket* verilerin bir makinedeki bir yerden diğerine t aşınması işlemi için yanlış bir ad olarak neden kabul edilebilir?
2. Metinde, JUMP talimatları, JUMP talimatı içinde varış yerinin adını (ve ya adım numarasını) belirterek açıkça tanımlanarak ifade edilmiştir (örneğin, "Adım 6'ya Atlama"). Bu tekniğin bir dezavantajı, eğer bir tali mat adı (numarası) daha sonra değiştirilirse, o talimata yapılan tüm a tlamaları bulmamız ve o adı da değiştirmemiz gerektiğidir. Varış yeri nin adının açıkça belirtilmediği başka bir JUMP talimatı ifade etme yol unu tanımlayın.
3. "Eğer 0, 0'a eşitse, o zaman Adım 7'ye atla" talimatı koşullu mu yoksa koş ulsuz mu bir atlama? Cevabınızı açıklayın.
4. Şekil 2.7'deki örnek programı gerçek bit desenlerinde yazın.
5. Aşağıdakiler, Ek C'de tanımlanan makine dilinde yazılmış talimatlardır. Onları İngilizceye yeniden yazın.

   a. 368A **b.** BADE **c.** 803C **d.** 40F4

6. Ek C'deki makine dilinde 15AB ve 25AB talimatları arasındaki fark ne dir?
7. İşte İngilizce bazı talimatlar. Her birini Ek C'deki makine diline çevirin.

   a. Kayıt numarası 3'ü 56 onaltılık değeri ile yükleyin.
   b. Kayıt numarası 5'i sağa üç bit döndürün.
   c. Kayıt A'nın içeriğini kayıt 5'in içeriği ile VEleyin ve sonucu kayıt 0'd a bırakın.

2.3 Programın Çalıştırılması

Bir bilgisayar, belleğinde saklanan bir programı, gerektiğinde talimatları bell ekten CPU'ya kopyalayarak takip eder.CPU'ya ulaştığında, her talimat çözülü r ve yerine getirilir. Talimatların bellekteki sırası,bellekte saklanan talimatları n sırasına karşılık gelir; aksi belirtilmedikçe, bu sıralama bir JUMP talimatı ile değiştirilir.

Genel yürütme sürecinin nasıl gerçekleştiğini anlamak için, CPU içindeki i ki özel amaçlı kaydı dikkate almak gereklidir: talimat kaydı ve program saya cı (bkz. tekrar Şekil 2.4). Talimat kaydı, yürütülen talimatı tutmak için kullanılı r. Program sayacı, yürütülecek bir sonraki talimatın adresini içerir ve böylece makinenin programda nerede olduğunu takip etme yolunu sağlar.

CPU, makine döngüsü olarak bilinen üç aşamalı bir süreçte rehberlik ede n bir algoritmayı sürekli olarak tekrarlayarak işini yapar.Aşamalar

Doclingo https://www.doclingo.ai

makine döngüleri, alma, çözme ve yürütme (Şekil 2.8) adımlarından oluşur. A lma adımında,CPU, program sayacının gösterdiği adreste saklanan talimatı a na bellekten talep eder. Her talimatın iki bayt uzunluğunda olduğu için,alma işlemi, ana bellekten iki bellek hücresinin içeriğini almakla ilgilidir. CPU, belle kten aldığı talimatıtalimat kaydına yerleştirir ve ardından program sayacını ik i artırarak, sayacın bellek içinde saklanan bir sonraki talimatınadresini içerm esini sağlar. Böylece program sayacı bir sonraki alma işlemi için hazır olacakt ır.

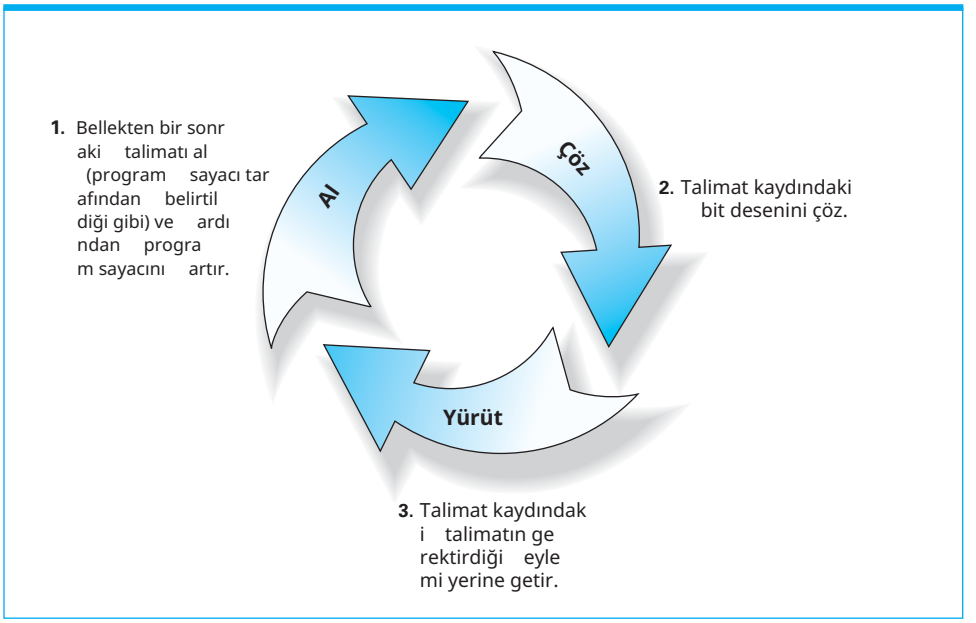
Artık talimat talimat kaydında olduğuna göre, CPU talimatı çözer; bu, tali matın op-kodu temelindeoperand alanını uygun bileşenlerine ayırmayı içerir.

CPU daha sonra talimatı, istenen görevi yerine getirmek için uygun devr eleri etkinleştirerek yürütür. Örneğin, talimat bir yükleme ise,CPU ana belleğ e uygun sinyalleri gönderir, ana belleğin veriyi göndermesini bekler ve ardın dan veriyi istenen kayıt içine yerleştirir;eğer talimat bir aritmetik işlem içinse, CPU, doğru kayıtları girdi olarak kullanarak aritmetik/lojik birim içindeki uyg un devreleri etkinleştirirve aritmetik/lojik birimin cevabı hesaplamasını ve uy gun kayıtta yerleştirmesini bekler.

Talimat kaydındaki talimat yürütüldükten sonra, CPUmakine döngüsüne fetch adımıyla yeniden başlar. Önceki fetch'in sonunda program sayacının ar tırıldığını gözlemleyin, bu nedenle CPU'ya doğru adresi tekrar sağlar.

Özel bir durum, JUMP talimatının yürütülmesidir. Örneğin, 2.9. Şekil'deki B258 talimatını düşünün; bu, "Eğer 2. kayıt içeriği 0. kayıt içeriğiyle aynıysa, 5 8 (onaltılık) adresindeki talimata JUMP yap" anlamına gelir. Bu durumda, ma kine döngüsünün yürütme adımı, 2 ve 0 kayıtlarının karşılaştırılmasıyla başla r. Eğer farklı bit desenleri içeriyorlarsa, yürütme adımı

Şekil **2.8**Makine döngüsü



Doclingo https://www.doclingo.ai

Bilgisayar Gücünü Karşılaştırma

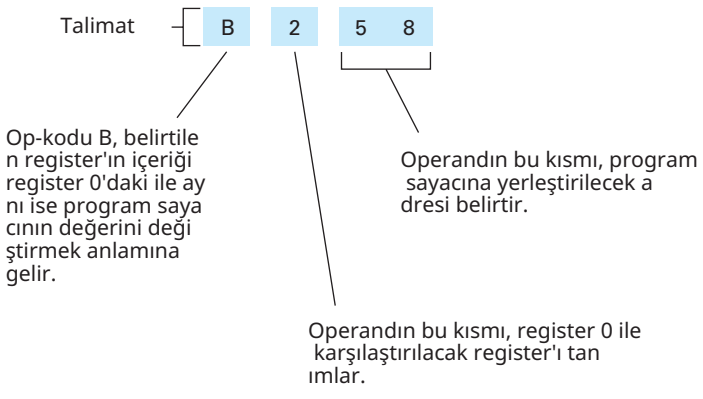
Kişisel bir bilgisayar satın alırken, saat hızlarının genellikle makineleri karşılaştırmak için kullanıldığını göreceksiniz. Bir bilgisayarın saati, makinenin aktivitelerini koordin e etmek için kullanılan, osilatör adı verilen bir devredir; bu osilatör devresi ne kadar hızlı darbeler üretirse, makine o kadar hızlı makine döngüsünü gerçekleştirir. Saat hı zları hertz (kısaca Hz) cinsinden ölçülür; bir Hz, bir saniyede bir döngü (veya darbe) e şittir. Masaüstü bilgisayarlardaki tipik saat hızları birkaç yüz MHz (eski modeller) ile b irkaç GHz arasında değişmektedir. (MHz, bir milyon Hz'yi ifade eden mega-hertz'in kı saltmasıdır. GHz, 1000 MHz'ye eşit olan gigahertz'in kısaltmasıdır.)

Ne yazık ki, farklı CPU tasarımları bir saat döngüsünde farklı miktarlarda iş yap abilir ve bu nedenle saat hızı, farklı CPU'lara sahip makineleri karşılaştırmada tek b aşına geçerli değildir. Eğer bir Intel işlemciye dayalı bir makineyi ARM tabanlı bir m akine ile karşılaştırıyorsanız, performansı karşılaştırmak için benchmark testleri ku llanmak daha anlamlı olacaktır; bu, farklı makinelerin aynı programı çalıştırırken p erformansını karşılaştırma sürecidir ve bu programa benchmark denir. Farklı uygu lama türlerini temsil eden benchmark'lar seçerek, çeşitli pazar segmentleri için anl amlı karşılaştırmalar elde edersiniz.

sona erer ve bir sonraki makine döngüsü başlar. Ancak, bu kayıtların içeriği eşitse, makine yürütme adımında program sayacına 58 (onaltılık) değerini ye rleştirir. Bu durumda, bir sonraki fetch adımı program sayacında 58 bulur, b u nedenle o adresteki talimat, alınacak ve yürütülecek bir sonraki talimat ola caktır.

Not edin ki, eğer talimat B058 olsaydı, program sayacının değiştirilip değ iştirilmeyeceği, register 0'ın içeriğinin register 0'ın içeriğine eşit olup olmam asına bağlı olacaktı. Ancak bunlar aynı register'lar ve dolayısıyla eşit içeriğe s ahip olmalıdır. Dolayısıyla, B0XY biçimindeki herhangi bir talimat, register 0'ı n içeriğine bakılmaksızın XY bellek konumuna atlama yapılmasına neden ola caktır.

Şekil **2.9** Talimat B'nin çözülmesi258



Doclingo https://www.doclingo.ai

Program Yürütme Örneği

Şekil 2.7'de sunulan programa uygulanan makine döngüsünü takip edelim,b u döngü ana bellekten iki değer alır, bunların toplamını hesaplar vebu topla mı bir ana bellek hücresine kaydeder. Öncelikle programıbellekte bir yere ye rleştirmemiz gerekiyor. Örneğimiz için, programınA0 (onaltılık) adresinden b aşlayarak ardışık adreslerde saklandığını varsayalım.Program bu şekilde sakl andığında, makinenin bunu yürütmesini sağlamak içinilk komutun adresini (A0) program sayacına yerleştirip makineyi başlatabiliriz(Şekil 2.10).

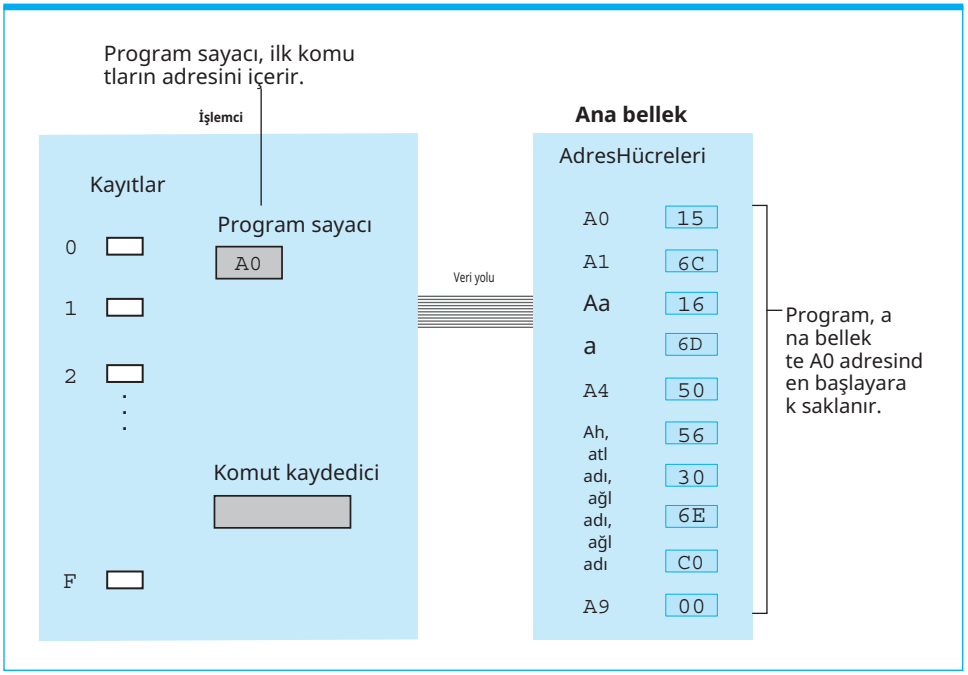
CPU, makine döngüsünün getirme adımına, ana bellekten A0 konumunda saklanan komutu çıkararak başlar ve bu komutu (156C) komut kaydedicisine yerleştirir (Şekil 2.11a). Dikkat edin ki, makinemizde, komutlar 16 bit (iki bayt) uzunluğundadır. Bu nedenle, alınacak tüm komut, hem A0 hem de A1 adreslerindeki bellek hücrelerini kaplar. CPU, bunu dikkate alacak şekilde tasarlanmıştır, bu yüzden her iki hücrenin içeriğini alır ve alınan bit desenlerini 16 bit uzunluğundaki komut kaydedicisine yerleştirir. CPU daha sonra program sayacına iki ekler, böylece bu kayıt, sonraki komutun adresini içerir (Şekil 2.11b). İlk makine döngüsünün getirme adımının sonunda, program sayacı ve komut kaydedici aşağıdaki verileri içerir:

Program Sayacı: A2
Komut Kaydedici: 156C

Sonra, CPU komut kaydedicisindeki komutu analiz eder ve 6C adresindeki b ellek hücresinin içeriği ile 5 numaralı kaydı yüklemesi gerektiğine karar verir. B u yükleme işlemi, makine döngüsünün yürütme adımında gerçekleştirilir ve CP U ardından bir sonraki döngüye başlar.

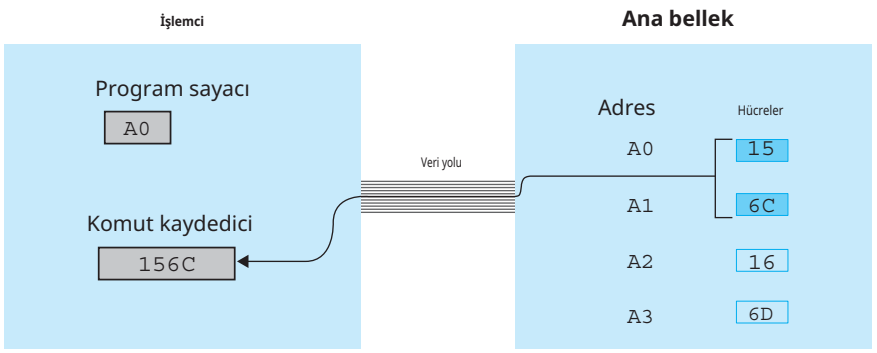
Bu döngü, A2 adresinden başlayarak iki bellek hücresinden 166D komut unu getirmekle başlar. CPU bu komutu komut kaydedicisine yerleştirir.

Şekil **2.10** Şekil 2.7 deki program, yürütme için ana bellekte saklanmıştır.

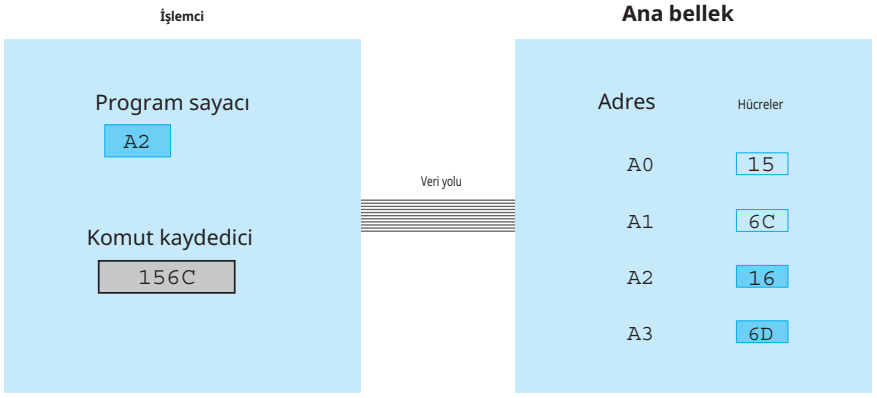


Doclingo https://www.doclingo.ai

Şekil **2.11** Makine döngüsünün getirme adımını gerçekleştirme



a. Getirme adımının başında, `A0` adresinden başlayan komut, bellekten alınır ve komut kaydedicisine yerleştirilir.



b. Ardından, program sayacı artırılır, böylece bir sonraki komuta işaret eder.

kayıt ve program sayacını A4'e artırır. Program sayacı ve komut kaydedicisin deki değerler şu şekilde olur:

Program Sayacı: A4
Komut Kaydedici: 166D

Şimdi CPU, 166D komutunu çözümler ve bunun, bellek adresi 6D'nin içer iğii ile 6 numaralı kaydı yüklemesi gerektiğini belirler. Ardından komutu yürü tür. Bu esnada 6 numaralı kayıt gerçekten yüklenir.

Program sayacı artık A4 içerdiğinden, CPU bu adresten başlayarak bir so nraki komutu çıkarır. Sonuç olarak, 5056 komut kaydedicisine yerleştirilir ve program sayacı A6'ya artırılır. CPU şimdi komut kaydedicisinin içeriğini çözü mler ve 5 ve 6 numaralı kayıtların girişleri ile iki'nin tamamlayıcı toplama dev resini etkinleştirerek yürütür.

Bu yürütme adımında, aritmetik/lojik birim istenen toplama işlemini gerçek leştirir, sonucu 0 numaralı kayıtta bırakır (kontrol birimi tarafından istenildiği gi bi) ve kontrol birimine tamamladığını bildirir. CPU daha sonra başka bir makine döngüsüne başlar. Yine, program sayacının yardımıyla, komutu getirir.

Doclingo https://www.doclingo.ai

İki bellek hücresinden (306E) sonraki komut, A6 bellek konumundan başlayarak al ınır ve program sayacı A8'e artırılır. Bu komut daha sonra çözümleme ve yürütme adımına geçilir. Bu noktada, toplam 6E bellek konumuna yerleştirilir.

Sonraki komut, A8 bellek konumundan başlayarak alınır ve program say acı AA'ya artırılır. Komut kaydedicisindeki (C000) içerikler artık durdurma ko mutu olarak çözümleme yapılır. Sonuç olarak, makine makine döngüsünün y ürütme adımında durur ve program tamamlanır.

Özetle, bellekte saklanan bir programın yürütülmesinin, sizin ve benim a yrıntılı bir talimat listesine uymamız gerektiğinde kullanabileceğimiz aynı sür eci içerdiğini görüyoruz. Biz talimatları yerine getirirken yerimizi işaretleyere k koruyabiliriz, CPU ise program sayacını kullanarak yerini korur. Hangi kom utun bir sonraki olarak yürütüleceğini belirledikten sonra, komutu okur ve a nlamını çıkarırız. Ardından, istenen görevi yerine getirir ve CPU'nun komut k aydedicisindeki komutu yürüttüğü ve ardından başka bir getirme adımına ge çtiği şekilde, bir sonraki komut için listeye geri döneriz.

Programlar ve Veriler

Bir bilgisayarın ana belleğinde birçok program aynı anda saklanabilir, yeter k i farklı konumları işgal etsinler. Makine başlatıldığında hangi programın çalış tırılacağı, program sayacının uygun şekilde ayarlanmasıyla belirlenebilir.

Ancak, verilerin de ana bellekte bulunduğunu ve 0'lar ve 1'ler cinsinden k odlandığını akılda tutmak gerekir; makinenin tek başına verinin ne olduğunu ve programın ne olduğunu bilmesi mümkün değildir. Program sayacı, isten en programın adresi yerine verinin adresine atanırsa, CPU, daha iyi bilmediğ i için, verinin bit desenlerini komutlar gibi alır ve yürütür. Nihai sonuç, ilgili v erilere bağlı olacaktır.

Ancak, programların ve verilerin bir makinenin belleğinde ortak bir görünüm e sahip olmasının kötü olduğunu sonucuna varmamamız gerekir. Aslında, bu, bir programın diğer programları (veya hatta kendisini) verilerle aynı şekilde manipüle etmesine olanak tanıdığı için faydalı bir özellik olarak kanıtlanmıştır. Örneğin, çev resiyle etkileşimine yanıt olarak kendini değiştiren ve böylece öğrenme yeteneği s ergileyen bir programı veya kendisine sunulan sorunları çözmek için diğer progra mları yazıp yürüten bir programı hayal edin.

Sorular & Alıştırmalar

1. Aşağıdaki tabloda verilen (onaltılık) bit desenlerinin, Ek C'de tanımlanan makinedeki 00 ile 05 adresleri arasındaki bellek hücrelerinde bulunduğ unu varsayalım:

Adres	İçerik
00	14
01	02
02	34
03	17
04	C0
05	00

Doclingo https://www.doclingo.ai

Makine program sayacının 00 değerini içerdiği durumda, makine durdu ğunda adresi onaltılık 17 olan bellek hücresinde hangi bit deseni bulun maktadır?

2. Aşağıdaki tabloda verilen (onaltılık) bit desenlerinin, Ek C'de tanımlanan makinedeki B0 ile B8 adresleri arasındaki bellek hücrelerinde bulundu ğunu varsayalım:

Adres	İçerik
B0	13
B1	B8
B2	A3
B3	02
B4	33
B5	B8
B6	C0
B7	00
B8	0F

   a. Program sayacı B0'dan başlarsa, ilk komut yürütüldükten sonra 3 numaralı kayıtta hangi bit deseni bulunmaktadır?
   b. Durma komutu yürütüldüğünde B8 bellek hücresinde hangi bit deseni bulu nmaktadır?

3. Aşağıdaki tabloda verilen (onaltılık) bit desenlerinin, Ek C'de tanımlanan makinedeki A4 ile B1 adresleri arasındaki bellek hücrelerinde bulundu ğunu varsayalım:

Adres	İçerik
A4	20
A5	00
A6	21
A7	03
A8	22
A9	01
AA	B1
AB	B0
klima	50
reklam	02
ANCAK	B0
İLE İLGİLİ	AA
B0	C0
B1	00

Aşağıdaki soruları yanıtlarken, makinenin program sayacının A4 ile b aşlatıldığını varsayın.

   a. Adres AA'deki komut yürütüldüğünde, kayıt 0'da ilk seferde ne var ?
   b. Adres AA'deki komut yürütüldüğünde, kayıt 0'da ikinci seferde ne var?
   c. Makine durmadan önce adres AA'deki komut kaç kez yürütülüyor ?

Doclingo https://www.doclingo.ai

4. F0 ile F9 adreslerindeki bellek hücrelerinin, Ek C'de tanımlanan makin ede, aşağıdaki tabloda tanımlanan (onaltılık) bit desenlerini içerdiğini varsayalım:

Adres	İçerik
F0	20
F1	C0
F2	30
F3	F8
F4	20
F5	00
F6	30
F7	F9
F8	FF
F9	FF

Makine program sayacı F0 değerini içeriyorsa, makine F8 adresindeki komutu ulaştığında ne yapar?

2.4 Aritmetik/Logic Talimatları

Daha önce belirtildiği gibi, aritmetik/logic talimatları grubu, aritmetik, mantı k ve kaydırma işlemlerini talep eden talimatlardan oluşur. Bu bölümde, bu işl emlere daha yakından bakıyoruz.

Mantık İşlemleri

1. Bölümde, iki giriş bitini birleştirerek tek bir çıkış biti üreten işlemler olarak AND, OR ve XOR (özel veya) mantık işlemlerini tanıttık. Bu işlemler, temel işle mi bireysel sütunlara uygulayarak iki bit dizisini birleştiren işlemlere genişlet ilebilir. Örneğin, 10011010 ve 11001001 desenlerinin AND'lanması sonucu

```
  10011010
VE 11001001
  10001000
```

her sütundaki 2 bitin AND'lanmasının sonucunu sütunun altına yazmış olduk . Benzer şekilde, bu desenlerin OR'lanması ve XOR'lanması da

```
  10011010
VE 11001001
  11011011
```

```
  10011010
XOR 11001001
  01010011
```

AND işleminin en önemli kullanımlarından biri, bir bit deseninin bir kısmı nda 0'ları yerleştirirken diğer kısmını etkilememektir. Örneğin, 00001111 bay tının bir AND işleminin ilk operandi olduğunu düşünelim. İkinci operandın iç eriğini bilmeden, sonucun en yüksek dört bitinin 0 olacağını kesin olarak söy leyebiliriz. Dahası, sonucun en düşük dört biti,

Doclingo https://www.doclingo.ai

ikinci operandın o kısmının bir kopyası olacaktır; aşağıdaki örnekte gösterildi ği gibi:

```
  00001111
VE 10101010
  00001010
```

AND işleminin bu kullanımı, maskeleme olarak adlandırılan sürecin bir örneğidir. Burada bir operand, maske olarak adlandırılır, diğer operandın hangi kısmının sonucu etkileyeceğini belirler. AND işlemi durumunda, maskeleme, operandlar dan birinin kısmi bir kopyasını üreten bir sonuç verir; 0'lar, kopyalanmamış poz isyonları kaplar.

Bu tür bir işlem, her bir bitin belirli bir nesnenin varlığını veya yokluğunu te msil ettiği bir bit haritasını manipüle ederken faydalıdır. Zaten, her bir bitin bir piksel ile ilişkilendirildiği görüntüleri temsil etme bağlamında bit haritaları ile k arşılaştık. Başka bir örnek olarak, her bir bitin belirli bir oyun kartı ile ilişkilendir ildiği 52 bitlik bir dizi, eldeki kartlarla ilişkili 5 bit için 1'ler ve diğer tüm bitler içi n 0'lar atayarak bir poker elini temsil etmek için kullanılabilir. Benzer şekilde, o n üçü 1 olan 52 bitlik bir bit haritası, bir köprü elini temsil etmek için kullanılabil ir veya 32 bitlik bir bit haritası, otuz iki dondurma çeşidinin hangilerinin mevcut olduğunu temsil etmek için kullanılabilir.

Diyelim ki, bir bellek hücresindeki 8 bit bir bit haritası olarak kullanılıyor ve yüksek sıralı uçtan üçüncü bit ile ilişkili nesnenin mevcut olup olmadığını öğrenmek istiyoruz. Yalnızca tüm baytı 00100000 maskesi ile AND'lamamız g erekiyor; bu, yalnızca bit haritasının yüksek sıralı ucundan üçüncü bitinin ken disi 0 ise tüm 0'lardan oluşan bir bayt üretir. Bir program, AND işlemini bir k oşullu dalga komutuyla takip ederek buna göre hareket edebilir. Ayrıca, bit h aritasının yüksek sıralı ucundan üçüncü bit 1 ise ve diğer bitleri rahatsız etm eden 0'a değiştirmek istiyorsak, bit haritasını maske ile AND'layabiliriz.

11011111 ve ardından sonucu orijinal bit haritasının yerinde saklayın.

VE işleminin bir bit dizisinin bir kısmını kopyalamak için kullanılabileceği yerlerde ,0'ları kopyalanmamış kısma yerleştirirken, VEYA işlemi bir dizinin bir kısmını kopyala mak içinkullanılabilirken, kopyalanmamış kısma 1'leri yerleştirir. Bunun için yine bir maske kullanıyoruz,bu sefer kopyalanacak bit pozisyonlarını 0'larla ve kopyalanmam ış pozisyonları belirtmek için 1'lerle gösteriyoruz. Örneğin, herhangi bir baytı 11110000 ile VEYA yapmak, en önemli 4 bitinde 1'ler bulunan bir sonuç üretirke n,geri kalan bitleri diğer operandın en az önemli 4 bitinin bir kopyasıdır; aşağıd aki örnekle gösterildiği gibi:

```
  11110000
VEYA 10101010
  11111010
```

Sonuç olarak, maske 11011111, bir baytın yüksek sıralı ucundaki üçüncü bitin 0 olmasını sağlamak için VE işlemi ile kullanılabilirken, maske 00100000 bu pozisyonda 1 olmasını sağlamak için VEYA işlemi ile kullanılabilir.

XOR işleminin önemli bir kullanımı, bir bit dizisinin tamamlayıcısını oluştur maktır.Herhangi bir baytı tüm 1'lerden oluşan bir maske ile XORlamak, baytın t amamlayıcısını üretir. Örneğin,aşağıdaki örnekte ikinci operand ile sonuç arası ndaki ilişkiye dikkat edin:

```
  11111111
XOR 10101010
  01010101
```

Doclingo https://www.doclingo.ai

Ek C'de tanımlanan makine dilinde, 7, 8 ve 9 numaralı op-kodlar sırasıyla mantık işlemleri VEYA, VE ve XOR için kullanılır. Her biri, iki belirlenen kaydını çerikleri arasında ilgili mantık işleminin gerçekleştirilmesini ve sonucun başk a birbelirlenen kayıtta saklanmasını talep eder. Örneğin, 7ABC talimatı, B ve Ckayıtlarının içeriklerinin VEYA yapılmasının sonucunun A kaydına yerleştiril mesini talep eder.

Dönme ve Kaydırma İşlemleri

Dönme ve kaydırma işlemleri sınıfındaki işlemler, bir kayıttaki bitleri hareket ettirmek için bir yol sağlar ve genellikle hizalama problemlerini çözmede kull anılır. Bu işlemler, hareket yönüne (sağa veya sola) ve sürecin dairesel olup o lmadığına göre sınıflandırılır. Bu sınıflandırma kılavuzları içinde karışık termi noloji ile birçok varyasyon bulunmaktadır. İlgili fikirleri hızlıca gözden geçirel im.

Bir bayt bit içeren bir kaydı düşünelim. İçeriğini 1 bit sağa kaydırdığımızd a, en sağdaki bitin kenardan düştüğünü ve en solda bir boşluk oluştuğunu h ayal ederiz. Bu ekstra bit ve boşluk, çeşitli kaydırma işlemleri arasındaki ayırt edici özelliktir. Bir teknik, sağ uçtan düşen biti en soldaki boşluğa yerleştirm ektir. Sonuç, dairesel kaydırma, ayrıca döndürme olarak da adlandırılır. Böy lece, bir bayt boyutundaki bit deseninde sağa dairesel kaydırma işlemi sekiz kez yaparsak, başladığımız aynı bit desenini elde ederiz.

Diğer bir teknik, kenardan düşen biti atmak ve her zaman boşluğu 0 ile doldu rmaktır. Bu işlemleri ifade etmek için genellikle mantıksal kaydırma terimi kullanıl ır. Bu tür kaydırmalar sola, iki'nin tamamlayıcı temsillerini iki ile çarpmak için kulla nılabilir. Sonuçta, ikili rakamları sola kaydırmak, iki ile çarpmaya karşılık gelir; tıpkı ondalık rakamların benzer bir kaydırmasının on ile çarpmaya karşılık gelmesi gibi . Ayrıca, ikiye bölme işlemi, ikili diziyi sağa kaydırarak gerçekleştirilebilir. Her iki ka ydırmada da, belirli notasyon sistemleri kullanılırken işaret bitinin korunmasına di kkat edilmelidir. Bu nedenle, genellikle işaret bitinin konumunda oluşan boşluğu her zaman orijinal değeri ile dolduran sağa kaydırmalar buluruz.

İşaret bitini değiştirmeyen kaydırmalar bazen aritmetik kaydırmalar olarak adlandırılır.

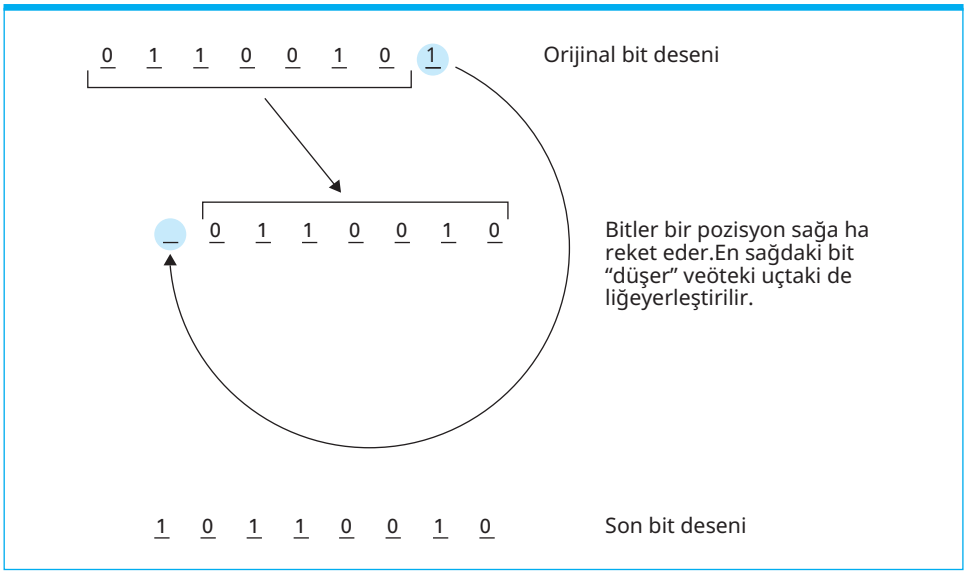
Çeşitli kaydırma ve döndürme talimatları arasında, Ek C'de tanımlanan m akinedili yalnızca A kodu ile belirtilen sağa dairesel kaydırma içerir. Bu duru mda, operandınilk onaltılık basamağı döndürülecek kaydı belirtirken, operan dın geri kalanıkaydırılacak bit sayısını belirtir. Böylece A501 talimatı “5 numar alı kaydıniçeriğini sağa 1 bit döndür” anlamına gelir. Özellikle, eğer 5 numar alı kayıtbaşlangıçta 65 (onaltılık) bit desenini içeriyorsa, bu talimat yürütüldü kten sonraB2 içerecektir (Şekil 2.12). (Ek C'deki makine dilinde sağa ve sola k aydırmatalimatlarının nasıl üretilebileceğini denemek isteyebilirsiniz. Örneği n, bir kayıt8 bit uzunluğunda olduğundan, 3 bit sağa dairesel kaydırma, 5 bit soladairesel kaydırma ile aynı sonucu üretir.)

Aritmetik İşlemler

Toplama, çıkarma, çarpma ve bölme aritmetik işlemlerini daha önce belirtmiş olsak da, birkaç eksik bağlantı hala yapılmalıdır. Öncelikle, çıkarma işleminin toplama ve olumsuzlama yoluyla simüle edilebileceğini zaten gördük. Ayrıca, çarpma yalnızca tekrarlanan toplama ve bölme tekrarlanan çıkarma işlemidir. (Altı, ikiye bölündüğünde üç eder çünkü üç tane iki vardır.)

Doclingo https://www.doclingo.ai

Şekil **2.12**Bit desenini 65 (onaltılık) bir bit sağa döndürme



altıdan çıkarılmıştır.) Bu nedenle, bazı küçük CPU'lar yalnızca toplama veya belki d e yalnızca toplama ve çıkarma komutları ile tasarlanmıştır.

Ayrıca, her aritmetik işlem için birçok varyasyonun mevcut olduğunu da belirtmeliyiz.Bunu, Ek C'de makinemizde mevcut olan toplama işlemleri ile il gili olarak zatenbelirtmiştik. Örneğin, toplama durumunda, eklenecek değerl er iki'nin tamamlayıcınotasyonunda saklanıyorsa, toplama işlemi basit bir sü tun sütun toplama olarakgerçekleştirilmelidir. Ancak, eğer operandlar kayan nokta değerleri olarak saklanıyorsa,toplama işlemi her birinin mantissasını ç ıkarmalı, üstel alanlara göre sağa veya solakaydırmalı, işaret bitlerini kontrol etmeli, toplama işlemini gerçekleştirmeli vesonucu kayan nokta notasyonun a çevirmelidir. Böylece, her iki işlem de toplama olarakkabul edilse de, makin enin eylemi aynı değildir.

Sorular & Alıştırmalar

1. Belirtilen işlemleri gerçekleştirin.

a. 01001011 VE 10101011	**b.** 10000011 VE 11101100	**c.** 11111111 VE 00101101
d. 01001011 VE 10101011	**Ve.** 10000011 VE 11101100	**f.** 11111111 VE 00101101
g. 01001011 ÜCRETSİZ 10101011	**h.** 10000011 ÜCRETSİZ 11101100	**Ben.** 11111111 ÜCRETSİZ 00101101

2. Bir baytın ortadaki 4 bitini izole etmek istiyorsanız, diğer 4 bite 0 yerl eştirerek ortadaki 4 bite dokunmadan. Hangi maskeyi ve hangi işlem i kullanmalısınız?

Doclingo https://www.doclingo.ai

3. Bir baytın ortadaki 4 bitini tamamlamak istiyorsanız, diğer 4 bite dok unmadan. Hangi maskeyi ve hangi işlemi kullanmalısınız?

4. a. Bir bit dizisinin ilk 2 bitini XOR'layıp ardından dizinin her sonucunu sırayla dizideki bir sonraki bit ile XOR'layarak devam ederseniz, so nucunuz dizideki 1'lerin sayısıyla nasıl ilişkilidir?

   b. Bu problem, bir mesajı kodlarken uygun parite bitinin ne olması g erektiğini belirlemekle nasıl ilişkilidir?

5. Sayısal birinin yerine mantıksal bir işlem kullanmak genellikle uygun dur.Örneğin, mantıksal AND işlemi, 2 biti çarpma ile aynı şekilde birle ştirir. Hangi mantıksal işlem, 2 bit eklemeye neredeyse eşdeğerdir ve bu durumda ne yanlış gider?

6. Küçük harflerin ASCII kodlarını büyük harflere dönüştürmek için hangi mantıksal işle mi ve hangi maskeyi kullanabilirsiniz? Peki ya büyük harfleri küçük harflere dönüştür mek için?

7. Aşağıdaki bit dizileri üzerinde 3 bit sağa döngüsel kaydırma işlemi gerçekleştirmenin sonucu nedir:

   a. `01101010` b. `00001111` c. `11111111`

8. Aşağıdaki onaltılık notasyonda temsil edilen baytlar üzerinde 1 bit sola döng üsel kaydırma işlemi gerçekleştirmenin sonucu nedir? Cevabınızı onaltılık biç imde verin.

   a. AB b. 5C c. B7 d. 35

9. 8 bitlik bir dizide 3 bit sağa döngüsel kaydırma, kaç bit sola döngüsel kaydırmaya eşdeğerdir?

10. 01101010 ve 11001100'ün toplamını temsil eden bit deseni nedir, eğer desenler ik i'nin tamamı notasyonunda saklanan değerleri temsil ediyorsa? Eğer desenler, Bö lüm 1'de tartışılan kayan nokta formatında saklanan değerleri temsil ediyorsa ne olur?

11. Ek C'nin makine dilini kullanarak, A7 adresine sahip bellek hücresinin en anlamlı bitine 1 yerleştiren bir program yazın ve hücredeki diğer bitleri değiştirmeyin.

12. C Ekinde verilen makine dilini kullanarak, bellek hücresi E0'dan ortad aki 4 biti kopyalayan bir program yazın ve E1 konumundaki hücrenin en yüksek anlamlı 4 bitine 0'lar yerleştirin.

2.5 Diğer Cihazlarla İletişim

Ana bellek ve CPU, bir bilgisayarın çekirdeğini oluşturur. Bu bölümde, bilgisaya r olarak adlandıracağımız bu çekirdeğin, kütle depolama sistemleri, yazıcılar, kl avyeler, fareler, ekranlar, dijital kameralar ve hatta diğer bilgisayarlar gibi çevr esel cihazlarla nasıl iletişim kurduğunu araştırıyoruz.

Doclingo https://www.doclingo.ai

Kontrolörlerin Rolü

Bir bilgisayar ile diğer cihazlar arasındaki iletişim genellikle kontrolör olarak bilinen bir aracı cihaz aracılığıyla gerçekleştirilir. Kişisel bir bilgisayar durumu nda, bir kontrolör, bilgisayarın anakartına kalıcı olarak monte edilmiş devrel erden oluşabilir veya esneklik için anakarttaki bir yuvaya takılan bir devre ka rtı şeklinde olabilir. Her iki durumda da, kontrolör, bilgisayar kasasındaki çev resel cihazlara veya belki de dış cihazların bağlanabileceği bilgisayarın arkası ndaki bir konektöre, yani bir porta, kablolar aracılığıyla bağlanır. Bu kontrolö rler bazen kendileri de küçük bilgisayarlardır; her biri kendi bellek devrelerin e ve kontrolörün faaliyetlerini yönlendiren basit bir CPU'ya sahiptir.

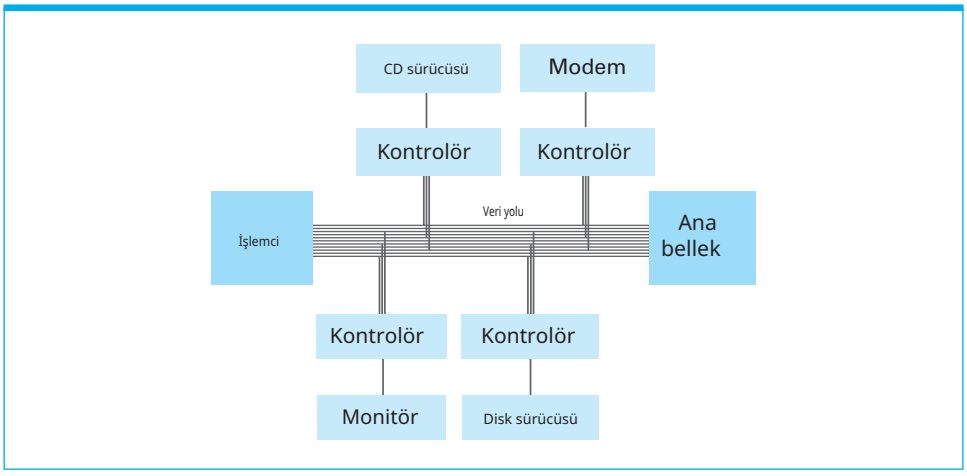
Bir kontrolör, mesajları ve verileri, bilgisayarın iç özellikleri ile bağlı olduğ u çevresel cihazın özellikleriyle uyumlu formlar arasında çevirir. Başlangıçta, her kontrolör belirli bir cihaz türü için tasarlanmıştı; bu nedenle, yeni bir çevr esel cihaz satın almak genellikle yeni bir kontrolör satın almayı da gerektiriy ordu.

Son zamanlarda, kişisel bilgisayar alanında, tek bir denetleyicinin çeşitli cih azları yönetebilmesi için evrensel seri veri yolu (USB) ve FireWire gibi standartla rın geliştirilmesi yönünde adımlar atılmıştır. Örneğin, tek bir USB denetleyicisi, bir bilgisayar ile USB uyumlu cihazların herhangi bir koleksiyonu arasında aray üz olarak kullanılabilir. Bugün piyasada bir USB denetleyicisi ile iletişim kurabil en cihazlar arasında fareler, yazıcılar, tarayıcılar, kütle depolama cihazları, dijita l kameralar ve akıllı telefonlar bulunmaktadır.

Her denetleyici, bilgisayarın CPU'su ve ana belleğini bağlayan aynı veri y oluna bağlantılar aracılığıyla bilgisayar ile iletişim kurar (Şekil 2.13). Bu konu mdan, CPU ile ana bellek arasında gönderilen sinyalleri izleyebilir ve kendi si nyallerini veri yoluna enjekte edebilir.

Bu düzenleme ile CPU, veri yoluna bağlı denetleyicilerle, ana bellekle ileti şim kurduğu şekilde iletişim kurabilir. Bir denetleyiciye bir bit desenini gönd ermek için, bit deseni önce CPU'nun genel amaçlı kayıtlarından birinde oluşt urulur. Ardından, CPU tarafından bit desenini denetleyicide "saklamak" için S TORE komutuna benzer bir komut yürütülür.

Şekil **2.13** Bir makinenin veri yoluna bağlı kontrolörler



Doclingo https://www.doclingo.ai

Aynı şekilde, bir kontrolörden bit desenini almak için, birLOAD komutuna benzer bir komut kullanılır.

Bazı bilgisayar tasarımlarında, kontrolörlere veri transferi,ağırlıklı olarak ana bellek ile iletişim için sağlanan aynı LOAD ve STORE op-kodlarıyla yönlendirilir. Bu durumlarda, her kontrolör, ana belleğin bu konumlara yapılan referansları göz ardı edecek şekilde tasarlanmışken, benzersiz bir adres setine yapılan referanslara yanıt vermek üzere tasarlanmıştır. Böylece, CPU bir kontrolöre atanmış bir bellek konumunda bir bit desenini depolamak için veri yolunda bir mesaj gönderdiğinde, bit deseni aslında ana bellek yerine kontrolörde "depolanır." Aynı şekilde, CPU böyle bir bellek konumundan veri okumaya çalıştığında, bir LOAD komutunda olduğu gibi, bellekten değil, kontrolörden bir bit deseni alır. Bu tür bir iletişim sistemine bellek eşlemeli G/Ç denir çünkü bilgisayarın giriş/çıkış cihazları çeşitli bellek konumlarında görünür (Şekil 2.14).

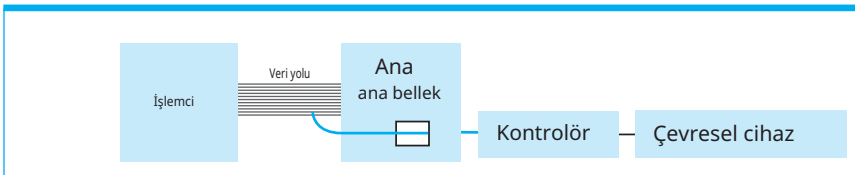
Belirli bir alternatif olarak bellek eşlemeli G/Ç yerine, kontrolörlere veri transferlerini yönlendirmek için makine dilinde özel op-kodlar sağlanabilir. Bu op-kodlara sahip komutlara G/Ç komutları denir. Örneğin, Ek C'de tanımlanan dil bu yaklaşımı izleseydi, "5 numaralı kaydın içeriğini A3 bit deseni ile tanımlanan kontrolörde STORE et" anlamına gelen F5A3 gibi bir komut içerebilir.

Doğrudan Bellek Erişimi

Bir denetleyici bir bilgisayarın veri yoluna bağlı olduğundan, CPU'nun veri yolunu kullanmadığı nanosekizlerde ana bellek ile kendi iletişimini sürdürebilir. Bir denetleyicinin ana belleğe erişim yeteneği doğrudan bellek erişimi (DMA) olarak bilinir ve bu, bir bilgisayarın performansı için önemli bir avantajdır. Örneğin, bir diskin bir sektöründen veri almak için CPU, diske bağlı denetleyiciye sektörün okunmasını ve verilerin ana belleğin belirli bir alanına yerleştirilmesini isteyen bit desenleri olarak kodlanmış istekler gönderebilir. CPU, denetleyici okuma işlemini gerçekleştirirken ve verileri DMA aracılığıyla ana belleğe yerleştirirken diğer görevlerle devam edebilir. Böylece iki faaliyet aynı anda gerçekleştirilecektir. CPU bir programı yürütürken, denetleyici disk ile ana bellek arasındaki veri transferini denetleyecektir. Bu şekilde, CPU'nun hesaplama kaynakları, nispeten yavaş veri transferi sırasında israf edilmemiş olur.

DMA'nın kullanımı, bir bilgisayarın veri yolunda gerçekleşen iletişimi karmaşıklaştırma gibi olumsuz bir etkiye de sahiptir. Bit desenleri CPU ile ana bellek arasında, CPU ile her denetleyici arasında ve her denetleyici ile ana bellek arasında hareket etmelidir. Veri yolundaki tüm bu etkinliklerin koordinasyonu büyük bir tasarım sorunudur. Mükemmel tasarımlara sahip olunsa bile, merkezi veri yolu bir

Şekil **2.14** Bellek haritalı G/Ç'nin kavramsal temsili



Doclingo https://www.doclingo.ai

USB ve FireWire

Ev bilgisayarlarına yeni çevresel cihazlar ekleme sürecini basitleştiren stan dartlaştırılmış seri iletişim sistemleri olan evrensel seri veri yolu (USB) ve Fi reWire, Intel'in liderliğinde geliştirilmiştir. FireWire'ın geliştirilmesi ise Appl e tarafından yönlendirilmiştir. Her iki durumda da temel tema, çeşitli çevre sel cihazların bağlanabileceği dış portlar sağlayan tek bir denetleyicidir. Bu bağlamda, denetleyici bilgisayarın iç sinyal özelliklerini uygun USB veya Fir eWire standart sinyallerine dönüştürür. Ardından, denetleyiciye bağlı her c ihaz, iç farklılıklarını aynı USB veya FireWire standardına dönüştürerek den etleyici ile iletişim kurar. Sonuç olarak, bir PC'ye yeni bir cihaz eklemek, yen i bir denetleyici takmayı gerektirmez. Bunun yerine, herhangi bir USB uyu mlu cihazı bir USB portuna veya bir FireWire uyumlu cihazı bir FireWire por tuna takmak yeterlidir.

İkisi arasında, FireWire daha hızlı bir transfer hızı sağlarken, USB teknolojisinin daha düşü k maliyeti onu düşük maliyetli kitle pazarında lider yapmıştır. Bugün piyasada bulunan USB uyu mlu cihazlar arasında fareler, klavyeler, yazıcılar, tarayıcılar, dijital kameralar, akıllı telefonlar ve yedekleme uygulamaları için tasarlanmış kütle depolama sistemleri bulunmaktadır. FireWire uy gulamaları genellikle video kaydediciler ve çevrimiçi kütle depolama sistemleri gibi daha yüksek transfer hızları gerektiren cihazlara odaklanmaktadır.

CPU ve denetleyicilerin veri yolu erişimi için rekabet etmesi nedeniyle bir eng el. Bu engel, von Neumann darboğazı olarak bilinir çünkü bir CPU'nun komu tlarını merkezi bir veri yolu üzerinden bellekten almasının sonucudur.

El sıkışma

İki bilgisayar bileşeni arasındaki veri transferi nadiren tek yönlü bir olaydır. B ir yazıcıyı veri alan bir cihaz olarak düşünsek de, gerçekte bir yazıcı da bilgisa yara veri gönderir. Sonuçta, bir bilgisayar, bir yazıcının yazdığı karakterlerde n çok daha hızlı bir şekilde karakter üretebilir ve gönderebilir. Eğer bir bilgis ayar kör bir şekilde verileri bir yazıcıya gönderirse, yazıcı hızla geride kalır ve veri kaybına yol açar. Bu nedenle, bir belgeyi yazdırma gibi bir süreç, bilgisay ar ve çevresel cihazın cihazın durumu hakkında bilgi alışverişinde bulunduğu ve faaliyetlerini koordine ettiği sürekli iki yönlü bir diyalog olan el sıkışma ile ilgilidir.

El sıkışma genellikle çevresel cihaz tarafından üretilen ve denetleyiciye g önderilen bir durum kelimesi içerir. Durum kelimesi, cihazın koşullarını yans ıtan bit desenidir. Örneğin, bir yazıcı durum kelimesinin en az anlamlı bitinin değeri, yazıcının kağıdının bitip bitmediğini gösterebilirken, bir sonraki bit ya zıcının ek veri için hazır olup olmadığını gösterebilir. Başka bir bit ise kağıt sı kışmasının varlığını göstermek için kullanılabilir. Sisteme bağlı olarak, denetl eyici bu durum bilgisine kendisi yanıt verebilir veya bunu CPU'ya sunabilir. H er iki durumda da, durum kelimesi, bir çevresel cihazla iletişimin koordine ed ilmesini sağlayan mekanizmayı sağlar.

Doclingo https://www.doclingo.ai

Popüler İletişim Medyaları

Bilgisayar cihazları arasındaki iletişim, iki tür yol üzerinden gerçekleştirilir: paralel ve seri. Bu terimler, sinyallerin birbirine göre nasıl iletildiğini ifade ed er. Paralel iletişim durumunda, birden fazla sinyal aynı anda, her biri ayrı bir "hat" üzerinde iletilir. Bu teknik, verilerin hızlı bir şekilde iletilmesini sağlarke n, nispeten karmaşık bir iletişim yolu gerektirir. Örnekler arasında, büyük ve ri bloklarının ve diğer sinyallerin aynı anda iletilmesine olanak tanıyan bir bil gisayarın iç veri yolu yer alır.

Öte yandan, seri iletişim, sinyallerin birer birer tek bir hat üzerinden iletil mesine dayanır. Bu nedenle, seri iletişim, paralel iletişimden daha basit bir v eri yolu gerektirir; bu da onun popülaritesinin nedenidir.
USB ve FireWire, yalnızca birkaç metre mesafede nispeten yüksek hızlı veri tr ansferi sunan seri iletişim sistemlerine örneklerdir. Biraz daha uzun mesafel er (bir ev veya ofis binası içinde) için, kablo veya radyo yayını ile Ethernet bağ lantıları üzerinden seri iletişim popülerdir (Bölüm 4.1).

Daha uzun mesafelerde iletişim için, geleneksel ses telefon hatları, kişise l bilgisayar alanında yıllarca baskın olmuştur. Bu iletişim yolları, tonların bire r birer iletildiği tek bir telden oluşan, doğası gereği seri sistemlerdir. Bu hatl ar üzerinden dijital verilerin iletimi, önce bit desenlerinin bir modem aracılığı yla işitilebilir tonlara dönüştürülmesi, bu tonların telefon sistemi üzerinden s eri olarak iletilmesi ve ardından varış noktasındaki başka bir modemle tonlar ın tekrar bitlere dönüştürülmesi ile gerçekleştirilir.

Daha hızlı uzun mesafe iletişimi için geleneksel telefon hatları üzerinden, telefon şirketleri, mevcut telefon hatlarının, geleneksel ses iletişimi için kulla nılan frekans aralığından daha geniş bir frekans aralığınıişleyebildiği gerçeği nden yararlanan DSL (Dijital Abone Hattı) olarak bilinen bir hizmet sunmakt adır.Daha kesin olarak, DSL, dijital verileri iletmek için işitme aralığının üzeri ndeki frekansları kullanırken,alt frekans spektrumunu ses iletişimi için bırakı r. DSL oldukça başarılı olmasına rağmen,telefon şirketleri, dijital iletişimi gel eneksel telefon hatlarından daha iyi destekleyen fiber optik hatlarahızla geçi ş yapmaktadır.

DSL ve fiber optiklerle rekabet eden diğer teknolojiler,kablo, kablo televizy on sistemlerinde kullanılan, ve yüksek frekanslı radyo yayınları aracılığıyla uydu bağlantılarıdır.

İletişim Hızları

Bir bilgisayar bileşeninden diğerine bitlerin transfer edildiği hız, bit/saniye (bps) cinsinden ölçülmektedir. Yaygın birimler Kbps(kilo-bps, bir bin bps'e eşi t), **Mbps** (mega-bps, bir milyon bps'e eşit) ve **Gbps** (giga-bps, bir milyar bps'e eşit) içerir. (Bitler ve baytlar arasındaki ayrımı not edin—yani, 8 Kbps, saniye de 1 KB'ye eşittir. Kısaltmalarda, küçük harfle yazılmış b genellikle *bit* anlamı na gelirken, büyük harfle yazılmış B *byte* anlamına gelir.)

Kısa mesafe iletişimi için, USB ve FireWire, çoğu multimedya uygulaması için yeterli olan birkaç yüz Mbps transfer hızı sağlar.

Doclingo https://www.doclingo.ai

Bu, kullanım kolaylığı ve nispeten düşük maliyetleri ile birleştiğinde, ev bilgisayarl arı ile yazıcılar, harici disk sürücüleri ve kameralar gibi yerel çevre birimleri arasın daki iletişim için popüler olmalarının nedenidir.

Veri iletim yollarının birden fazla yolu hizmet etmesi için verilerin kodlan ması veya iç içe geçirilmesi (multiplexing) ve veri sıkıştırma tekniklerini birleş tirerek, geleneksel ses telefon sistemleri 57.6 Kbps transfer hızlarını destekle yebildi; bu, günümüzün multimedya ve internet uygulamalarının, örneğin Yo uTube ve Facebook'un ihtiyaçlarının gerisindedir. MP3 müzik kayıtlarını çalm ak yaklaşık 64 Kbps transfer hızı gerektirirken, düşük kaliteli video klipleri çal mak bile Mbps cinsinden ölçülen transfer hızları gerektirir. Bu nedenle, Mbp s aralığında transfer hızları sağlayan DSL, kablo ve uydu bağlantıları gibi alte rnatifler, geleneksel ses telefon sistemlerinin yerini almıştır. (Örneğin, DSL y aklaşık 54 Mbps transfer hızları sunar.)

Belirli bir ortamda mevcut olan maksimum hız, iletişim yolunun türüne ve uygulamasında kullanılan teknolojiye bağlıdır. Bu maksimum hız genellikle ileti şim yolunun bant genişliği ile gevşek bir şekilde eşitlenir, ancak bant genişliği t erimi aynı zamanda transfer hızından ziyade kapasite anlamına da gelir. Yani, b ir iletişim yolunun yüksek bant genişliğine sahip olduğunu (veya geniş bant hiz meti sağladığını) söylemek, iletişim yolunun yüksek bir hızda bitleri iletme yete neğine sahip olduğu kadar, aynı anda büyük miktarda bilgi taşıma kapasitesine de sahip olduğu anlamına gelir.

Sorular & Alıştırmalar

1. Ek C'de tanımlanan makinenin bellek haritalı G/Ç kullandığını ve B5 a dresinin yazıcı portundaki verilerin gönderileceği yeri temsil ettiğini v arsayalım.
   a. Eğer 7 numaralı kayıt A harfinin ASCII kodunu içeriyorsa, o harfin yazıcıda basılmasını sağlamak için hangi makine dili komutu kulla nılmalıdır?
   b. Eğer makine saniyede bir milyon komut yürütüyorsa, bu karakter bir saniyede yazıcıya kaç kez gönderilebilir?
   c. Eğer yazıcı dakikada beş geleneksel sayfa metin basma kapasitesi ne sahipse, (b) kısmında gönderilen karakterlerle başa çıkabilir mi ?
2. Kişisel bilgisayarınızdaki sabit diskin dakikada 3000 devir döndüğünü , her izde 16 sektör bulunduğunu ve her sektörün 1024 bayt içerdiği ni varsayalım. Disk sürücüsünden dönen diskten okunan bitleri alaca ksa, disk sürücüsü ile disk denetleyicisi arasında yaklaşık ne kadar bir iletişim hızı gereklidir?
3. 54 Mbps transfer hızında Unicode ile kodlanmış 300 sayfalık bir romanı aktarm anın ne kadar süreceğini tahmin edin.

Doclingo https://www.doclingo.ai

2.6 Diğer Mimariler

Bakış açımızı genişletmek için, şimdiye kadar tartıştığımız geleneksel makine mimari sine bazı alternatifleri göz önünde bulunduralım.

Pipelineleme

Elektrik darbeleri, ışık hızından daha hızlı bir şekilde bir tel üzerinden geçemez. Işık, bir nanosekonda (bir milyarda bir saniye) yaklaşık 1 fit yol alır, bu nedenle CPU'nun 1 fit uzaklıktaki bir bellek hücresinden bir komut alması en az 2 nanosekondan az sür mez. (Okuma isteği belleğe gönderilmeli, bu da en az 1 nanosekondan fazla zaman a lır ve komut CPU'ya geri gönderilmelidir, bu da en az bir nanosekondan fazla zaman alır.) Sonuç olarak, böyle bir makinede bir komutu almak ve yürütmek birkaç nanose kondan fazla zaman alır - bu da bir makinenin yürütme hızını artırmanın nihayetinde bir miniaturizasyon problemi haline geldiği anlamına gelir.

Ancak, yürütme hızını artırmak, bir bilgisayarın performansını artırmanın tek yolu değildir. Gerçek hedef, makinenin verimliliğini artırmaktır; bu, maki nenin belirli bir zaman diliminde gerçekleştirebileceği toplam iş miktarını ifa de eder.

Bir bilgisayarın verimliliğinin artırılabileceği bir örnek, yürütme hızında bi r artış gerektirmeden pipelineleme tekniğidir; bu, makine döngüsündeki ad ımların üst üste binmesine izin verme tekniğidir. Özellikle, bir komut yürütül ürken, bir sonraki komut alınabilir; bu, "boruda" birden fazla komutun aynı a nda, her biri farklı bir işleme aşamasında olabileceği anlamına gelir. Böylece, her bir bireysel komutu almak ve yürütmek için gereken zaman aynı kalsa bi le, makinenin toplam verimliliği artar. (Elbette, bir JUMP komutuna ulaşıldığı nda, önceden alma ile elde edilebilecek herhangi bir kazanç gerçekleşmez çü nkü "boruda" bulunan komutlar aslında gerekenler değildir.)

Modern makine tasarımları, boru hattı kavramını basit örneğimizin ötesi ne taşır. Genellikle aynı anda birkaç komut getirme ve bu komutların birbirin e bağımlı olmadığı durumlarda aynı anda birden fazla komutu yürütme yete neğine sahiptirler.

Çok Çekirdekli CPU

Teknoloji, bir silikon çip üzerine giderek daha fazla devre yerleştirme yolları s ağladıkça, bir bilgisayarın bileşenleri arasındaki fiziksel ayrım azalır. Örneğin, tek bir çip bir CPU ve ana bellek içerebilir. Bu, "çip üzerindeki sistem" yaklaşı mının bir örneğidir; burada amaç, daha yüksek düzey tasarımlarda soyut bir a raç olarak kullanılabilecek tek bir cihazda tam bir aparat sağlamaktır. Diğer d urumlarda, tek bir cihaz içinde aynı devrenin birden fazla kopyası sağlanır. Bu son taktik, başlangıçta birkaç bağımsız kapı veya belki de birden fazla flip-flo p içeren çipler şeklinde ortaya çıkmıştır. Günümüzün en son teknolojisi, tek bi r çip üzerinde birden fazla tam CPU yerleştirilmesine olanak tanır. Bu, aynı çip üzerinde bulunan iki veya daha fazla CPU'dan oluşan çok çekirdekli CPU'lar ol arak bilinen cihazların temel mimarisidir ve paylaşılan önbellek belleği ile birli kte gelir. (İki işlem birimi içeren çok çekirdekli CPU'lar genellikle çift çekirdekli CPU olarak adlandırılır.) Bu tür cihazlar, MIMD sistemlerinin inşasını basitleşti rir ve ev bilgisayarlarında kullanılmak üzere kolayca temin edilebilir.

Doclingo https://www.doclingo.ai

Çok İşlemcili Makineler

Pipeline, paralel işleme doğru bir ilk adım olarak görülebilir; bu, aynı anda birkaç etkinliğin gerçekleştirilmesidir. Ancak, gerçek paralel işleme, birden fazla işlem birimi gerektirir ve bu da çok işlemcili makineler olarak bilinen bilgisayarlara yol açar.

Bugün birçok bilgisayar bu fikirle tasarlanmıştır. Bir strateji, her biri tek işlemcili bir makinedeki CPU'ya benzeyen birkaç işlem birimini aynı ana belleğe bağlamaktır. Bu yapılandırmada, işlemciler bağımsız olarak ilerleyebilir, ancak ortak bellek hücrelerinde birbirlerine mesaj bırakarak çabalarını koordine edebilirler. Örneğin, bir işlemci büyük bir görevle karşılaştığında, o görevin bir kısmı için bir programı ortak bellekte saklayabilir ve ardından başka bir işlemciden bunu yürütmesini isteyebilir. Sonuç, farklı veri setleri üzerinde farklı komut dizilerinin gerçekleştirildiği bir makinedir; bu, daha geleneksel SISD (tek komut akışı, tek veri akışı) mimarisinin aksine, MIMD (çoklu komut akışı, çoklu veri akışı) mimarisi olarak adlandırılır.

Çoklu işlemci mimarisinin bir varyasyonu, işlemcileri bir araya bağlayarak aynı komut dizisini eş zamanlı olarak yürütmelerini sağlamak ve her birinin kendi veri setine sahip olmasıdır. Bu, **SIMD** (tek komut akışı, çoklu veri akışı) mimarisine yol açar. Bu tür makineler, aynı görevin büyük bir veri bloğu için deki her benzer öğe setine uygulanması gereken uygulamalarda faydalıdır.

Paralel işleme için bir diğer yaklaşım, her birinin kendi belleği ve CPU'su olan daha küçük makinelerin birleşimi olarak büyük bilgisayarlar inşa etmektir. Böyle bir mimaride, küçük makinelerin her biri komşularıyla bağlantılıdır, böylece tüm sisteme atanan görevler bireysel makineler arasında bölünebilir. Böylece, iç makinelerden birine atanan bir görev bağımsız alt görevlere ayrılabiliyorsa, o makine komşularından bu alt görevleri eşzamanlı olarak yerine getirmelerini isteyebilir. Böylece, orijinal görev, tek işlemcili bir makinenin gerektireceğinden çok daha az sürede tamamlanabilir.

Sorular & Alıştırmalar

1. Bölüm 2.3'teki Soru 3'e geri dönersek, eğer makine metinde tartışılan boru hattı tekniğini kullanıyorsa, AA adresindeki komut yürütüldüğünde “boruda” ne olacak? Bu noktada boru hattının faydalı olmaması için hangi koşullar geçerli olmalıdır?
2. Soru 4'teki programı bir boru hattı makinesinde çalıştırırken hangi çatışmaların çözülmesi gerekir?
3. Diyelim ki aynı belleğe bağlı iki “merkezi” işlem birimi var ve farklı programlar yürütüyorlar. Ayrıca, bu işlemcilerden birinin, diğerinin aynı hücreden bir çıkarması gerektiği sırada, bir bellek hücresinin içeriğine bir eklemesi gerektiğini varsayalım. (Net etki, hücrenin başladığı değerde kalması olmalıdır.)

   a. Bu aktivitelerin bir dizisini tanımlayın, böylece hücre başlangıç değerinden bir eksik bir değerle sonuçlanır.
   b. Bu aktivitelerin bir dizisini tanımlayın, böylece hücre başlangıç değerinden bir fazla bir değerle sonuçlanır.

Doclingo https://www.doclingo.ai

Bölüm Gözden Geçirme Problemleri

(Yıldızlı problemler isteğe bağlı bölümlerle ilişkilidir.)

1. a. Genel amaçlı kayıtlarve ana bellek hücrele ri hangi yönlerden benzerdir?
   b. Genel amaçlı kayıtlarve ana bellek hücrel eri hangi yönlerden farklıdır?
2. Aşağıdaki soruları, Ek C'de tanımlanan makine dili açısından yanıtlayın.
   a. 2304 (onaltılık) komutunu16 bitlik bir dizi olarak yazın.
   b. B2A5 (onaltılık) komutunun op-kodunu 4 bitlik bir dizi olarak yazın.
   c. B2A5 (onaltılık) komutunun operand alanını 12 bitlik bir dizi olarak yazın.
3. Bir veri bloğunun,

   Ek C'den 98 adresinden A2'ye kadar, dahil. Bu blokta kaç bellek hücresi var? Adreslerini listeleyin.
4. Ek C'de tanımlanan makinede B0CD kom utunu yürüttükten hemen sonra program sa yacının değeri nedir?
5. Ek C'de tanımlanan makinedeki 00 ile 05 adreslerindeki bellek hücrelerinin aş ağıdaki bit desenlerini içerdiğini varsayalım:

Adres	İçerik
00	22
01	11
02	32
03	02
04	C0
05	00

   Program sayacının başlangıçta 00 içerdiğ ini varsayarak, makine döngüsünün her getirme aşamasının sonunda program saya cının, komut kaydedicisinin ve adres 02'deki bellek hücresinin içeriklerini kaydedin.
6. Üç değer x, y ve z'nin bir makinenin belleğind e saklandığını varsayalım. Bellekten kayıtları yükleme, bellek içinde değerleri kaydetme vb . olayların sırasını tanımlayın; bu, x + y + z'ni n hesaplanmasına yol açar. Peki ya (2 x) + y?
7. Aşağıdakiler, Ek C'de tanımlanan makine d ilinde yazılmış komutlardır. Onları İngilizceye çevirin.

   a. 7123 b. 40E1 c. A304
   d. B100 e. 2BCD
8. Bir makine dili, 4 bitlik bir op-kod alanı ile ta sarlandığını varsayalım. Dil kaç farklı komut türü içerebilir? Op-code alanı 6 bite çıkarılırsa ne olur?
9. Aşağıdaki talimatlarıİngilizceden Ek C'd e tanımlananmakine diline çevirin.

   a. 6 numaralı kaydı onaltılık77 değeri ile YÜKLE.
   b. 7 numaralı kaydı bellek hücresi77'nin içeriğ i ile YÜKLE.
   c. 0 numaralı kaydın içeriği A numaralı kayttaki değere eşitse, 24 numaralı bellek konumundaki komuta ATLA.
   d. 4 numaralı kaydı sağa üç bit DÖNDÜR.
   e. VE kayıtlar E ve 2'nin içeriğini 1 numaralı kayıtta sonucu bırakarak birleştir.
10. Şekil 2.7'deki programı, eklenmesi gereken değer lerin iki'nin tamamlayıcı notasyonu yerine kayan no kta notasyonu kullanılarak kodlandığını varsayarak yeniden yazın.
11. Aşağıdaki talimatların her birini (Ek C'deki m akine dilinde) 3B konumundaki bellek h ücresinin içeriğini değiştirip değiştirmediğ i, 3C konumundaki bellek hücresinin içeriğ ini alıp almadığı veya 3C konumundaki bell ek hücresinin içeriğinden bağımsız olup olm adığı açısından sınıflandırın.
    a. 353C b. 253C c. 153C
    d. 3C3C e. 403C
12. Varsayalım ki, Ek C'de tanımlanan makine deki 00 ile 03 adreslerindeki bellek hüc releri aşağıdaki bit desenlerini içermektedir:

Adres	İçerik
00	26
01	55
02	C0
03	00

    a. İlk talimatı İngilizceye çevirin.
    b. Makine, program sayacında 00 ile başla tıldığında, makine durduğunda 6 numa ralı kayıtta hangi bit deseni vardır?

Doclingo https://www.doclingo.ai

13. Makine, Ek C'de tanımlanan adreslerde 00'dan 02'ye kadar olan bellek hücrel erinde aşağıdaki bit desenlerini içeriyorsa:

Adres	İçerik
00	12
01	21
02	34

a. Makineyi program sayacında 00 ile ba şlattığımızda ilk yürütülecek komut n e olurdu?
b. Makineyi program sayacında 01 ile ba şlattığımızda ilk yürütülecek komut n e olurdu?

14. 00ile 05 arasındaki bellek hücrelerinin, C Ekinde tanımlanan makinede aşağı daki bit desenlerini içerdiğini varsayalım:

Adres	İçerik
00	12
01	02
02	32
03	42
04	C0
05	00

Aşağıdaki soruları yanıtlarken,makineni n program sayacının 00'a eşit olduğunu v arsayın.
a. Yürütülen komutları İngilizceye çevirin .
b. Makine durduğunda 42 adresindeki bellek hücresinde hangi bit deseni vardır?
c. Makine durduğunda program sayacında hangi bit deseni vardır?

15. 00ile 09 arasındaki bellek hücrelerinin, C Ekinde tanımlanan makinede aşağı daki bit desenlerini içerdiğini varsayalım:

Adres	İçerik
00	1C
01	03
02	2B
03	03
04	5A
05	M.Ö.
06	3A
07	00
08	C0
09	00

Makinenin program sayacının 00 içerdiğin i varsayın.
a. Makine durduğunda 00 adresindeki belle k hücresinde ne olacak?
b. Makine durduğunda program sayacın da hangi bit deseni olacak?

16. 00ile 07 arasındaki bellek hücrelerinin, C Ekinde tanımlanan makinede aşağı daki bit desenlerini içerdiğini varsayalım:

Adres	İçerik
00	2B
01	07
02	3B
03	06
04	C0
05	00
06	00
07	23

a. Program sayacının 00 değerini içerdiği dur umda makineyi başlattığımızda yürütülece k programı içeren bellek hücrelerinin adreslerini listeleyin.
b. Veri tutmak için kullanılan bellek hücrelerinin adreslerini listeleyin.

17. Appendix C'de tanımlanan makinedeki 00 i le 0D adreslerindeki bellek hücrelerinin aşağıdaki bit desenlerini içerdiğini varsayalım:

Adres	İçerik
00	20
01	04
02	21
03	01
04	40
05	12
06	51
07	12
08	B1
09	0C
0A	B0
0B	06
0C	C0
0D	00

Makinenin program sayacının 00 içerdiğin i varsayın.
a. Makine durduğunda kayıt 0'da hangi bit deseni olacak?
b. Makine durduğunda kayıt 1'de hangi bit deseni olacak?
c. Makine durduğunda program sayacında hangi bit deseni vardır?

Doclingo https://www.doclingo.ai

18. F0 ile FD adreslerindeki bellek hücreleri nin, Ek C'de tanımlanan makinede aşa ğıdaki (onaltılık) bit desenlerini içerdiğini v arsayalım:

Adres	İçerik
F0	20
F1	00
F2	22
F3	02
F4	23
F5	04
F6	B3
F7	FC
F8	50
F9	02
FA	B0
Facebook	F6
FC	C0
FD	00

Makineyi program sayacının F0 değerini içe rerek başlatırsak, makine nihayetinde FC konu mundaki durdurma komutunu yürüttüğünde 0 n umaralı kayıtta hangi değer bulunur?

19. Ek C'deki makine her mikro saniyede (bi r milyonuncu saniye) bir komut yürütüyors a, Problem 18'deki programı tamamlamak ne kadar sürer?

20. 20 ile 28 adreslerindeki bellek hücreleri nin, Ek C'de tanımlanan makinede aş ağıdaki bit desenlerini içerdiğini varsayalım:

Adres	İçerik
20	12
21	20
22	32
23	30
24	B0
25	21
26	24
27	C0
28	00

Makinenin program sayacının 20 değerini içe rerek başladığını varsayın.

a. Makine durduğunda 0, 1 ve 2 numaralı kayıtla rda hangi bit desenleri olacak?

b. Makine durduğunda 30 adresindeki bellek hücresinde hangi bit deseni olacak?

c. Makine durduğunda B0 adresindeki bellek hücresinde hangi bit deseni olacak?

21. Aşağıdaki bit desenlerinin, Ek C'de tanımla nan makinedeki AFile B1 adreslerindeki bellek hücrelerinde bulunduğunu varsayalım:

Adres	İçerik
İLE İLGİLİ	B0
B0	B0
B1	İLE İLGİLİ

Makineyi AF içeren program sayacı ile başla tırsak ne olur?

22. Ek C'de tanımlanan makinedeki 00 ile 05 adreslerindeki bellek hücrelerinin aş ağıdaki (onaltılık) bit desenlerini içerdiğini varsayalım:

Adres	İçerik
00	25
01	B0
02	35
03	04
04	C0
05	00

Makineyi 00 içeren program sayacı ile başlat ırsak, makine ne zaman durur?

23. Aşağıdaki her durumda, Ek C'de tanımlanan makine dilinde istenen etkinlikleri gerçekleştir mek için kısa bir program yazın. Programları nızın her birinin bellek içinde 00 adresinde n başlayarak yerleştirildiğini varsayın.

a. D8 bellek konumundaki değeri B3 bellek konumuna taşıyın.

b. D8 ve B3 bellek konumlarında saklanan d eğerleri değiştirin.

c. 44 bellek konumunda saklanan değer 00 ise , 46 bellek konumuna 01 değerini yerleştiri n; aksi takdirde, 46 bellek konumuna FF değerini koyun.

24. Bir zamanlar bilgisayar meraklıları arasınd a popüler olan bir oyun core wars - savaş gemisi varyasyonu. (Kavram *core*, 0 ve 1'le rin küçük manyetik malzeme halkalarında manyetik alanlar olarak temsil edildiği erk en bellek teknolojisinden gelmektedir. H alkalara çekirdek denir.) Oyun, aynı bilgisaya rın belleğinin farklı konumlarında saklana n iki karşıt program arasında oynanır. Bilgisayarın iki program arasında alternatif olarak geçiş yaptığı, birinden bir komut yürütüp ardından diğerinden bir komut yürüttüğü varsayılmaktadır. Her programın amacı

Doclingo https://www.doclingo.ai

diğerinin arızalanmasına neden olmaktır; an cak, her ikiprogram da diğerinin konumun u bilmemektedir.

a. Ek C'deki makine dilinde, oyunamümkün olduğunca küçük bir şekildesavunmacı bir yaklaşım sergileyen bir program yaz ın.

b. Ek C dilinde, karşıt programdan herha ngi bir saldırıyıönlemeye çalışan ve farklı kon umlarahareket eden bir program yazın. Dah akesin olarak, 00 konumundan başlayarak,bi r program yazın ki bu program kendini70 k onumuna kopyalayacak ve ardından70 ko numuna atlayacaktır.

c. (b) bölümündeki programı, yeni bellek konumlarına yeniden y erleştirmeye devam edecek şekilde genişletin. Özellikle, prog ramınızın 70 konumuna, ardından E0 (70 + 70) konumuna, a rdından 60 (70 + konumuna geçmesini sağlayın. 70 + 70), vb.

25. Appendix C'deki makine dilinde, A0, A1, A2 ve A3 bellek konumlarında saklanan kayan nokta değerlerinin toplamını hesaplamak için bir program yazın. Programınız toplamı A4 bellek konumunda saklamalıdır.

26. Ek C'de tanımlanan makinedeki 00 ile 05 adreslerindeki bellek hücrelerinin aş ağıdaki (onaltılık) bit desenlerini içerdiğini varsayalım:

Adres	İçerik
00	20
01	C0
02	30
03	04
04	00
05	00

Program sayacı 00 içeren bir makine ile başl arsak ne olur?

27. Makine C Ekinde tanımlanan bellek hücreleri 08 ve 09 adreslerinde sırasıyla B 0 ve 08 bit desenlerini içeriyorsa ve makin e program sayacında 08 değeri ile başlatılıy orsa ne olur?

28. C Ekinde tanımlanan makine dilinde yazılmı ş aşağıdaki program, ana bellekte 30 adresi nden (onaltılık) başlamaktadır. Program çal ıştırıldığında hangi görevi yerine getirecekti r?

```
2003
2101
```

220023101400341052215331323933338488038500

29. C Ekinde tanımlanan makine B op-kod una sahip bir komut yürütürken yer alan adımları özetleyin. Cevabınızı, CPU'ya ne y apması gerektiğini söyler gibi bir dizi tali mat olarak ifade edin.

***30.** C Ekinde tanımlanan makine 5 op-kod una sahip bir komut yürütürken yer alan adımları özetleyin. Cevabınızı, CPU'ya ne yapması gerektiğini söyler gibi bir dizi tal imat olarak ifade edin.

***31.** C Ekinde tanımlanan makine 6 op-kod una sahip bir komut yürütürken yer alan adımları özetleyin. Cevabınızı, CPU'ya ne yapması gerektiğini söyler gibi bir dizi tal imat olarak ifade edin.

***32.** C Ekinde tanımlanan makinedeki 4 ve 5 nu maralı kayıtlar sırasıyla 3A ve C8 bit desen lerini içeriyorsa, aşağıdaki her bir komut yürütüldükten sonra 0 numaralı kayıtta h angi bit deseni kalır:

a. 5045 b. 6045 c. 7045
d. 8045 Ve. 9045

***33.** Açıklanan makine dilini kullanarak Ek C'de, aşağıdaki her bir görevi yerine getirmek için programlar yazın:

a. Bellek konumu 44'te saklanan bit desenini bellek konumu AA'ya kopyalayın.

b. Bellek hücresi 34'teki en az anlamlı 4 b iti 0'açevirirken diğer bitleri değiştir meyin.

c. Bellek konumu A5'ten en az anlamlı 4 bitibellek konumu A6'nın en az anla mlı 4 bitine kopyalayın,A6'daki diğer bitl eri değiştirmeden.

d. Bellek konumu A5'ten en az anlaml ı 4 bitiA5'in en çok anlamlı 4 bitine kopy alayın. (Böylece,A5'teki ilk 4 bit, son 4 b it ile aynı olacaktır.)

Doclingo https://www.doclingo.ai

***34.** Belirtilen işlemleri gerçekleştirin:

a. 111001
VE 101001

b. 000101
VE 101010

c. 001110
VE 010101

d. 111011
VE 110111

Ve. 111001
VEYA 101001

f. 010100
VEYA 101010

g. 000100
VEYA 010101

h. 101010
VEYA 110101

Ben. 111001
ÜCRETSİZ 101001

j. 000111
ÜCRETSİZ 101010

k. 010000
ÜCRETSİZ 010101

l. 111111
ÜCRETSİZ 110101

***35.** Her birinin gerçekleştirilmesi için gereken maske ve mantıksal işlemi tanımlayın:

a. Diğer bitleri rahatsız etmeden 8 bitlik bir desende üst 4 bite 1 koyun.
b. Diğer bitleri değiştirmeden 8 bitlik bir desenin en anlamlı bitinin tamamlayıcısını alın.
c. 8 bitlik bir desenin tamamlayıcısını alın.
d. Diğer bitleri rahatsız etmeden 8 bitlik bir desende en az anlamlı bite 0 koyun.
e. En anlamlı biti bozmadan, 8 bitlik bir desende en anlamlı bit hariç tüm bitler e 1 koyun.

***36.** Giriş dizesi 10000001 olduğunda, 8 bitlik bir giriş dizesine uygulandığında tüm 0'ları üreten bir mantıksal işlemi (ve ilgili maskeyi) tanımlayın.

***37.** Giriş dizesi hem 1 ile başlıyor hem de 1 ile bitiyorsa, 8 bitlik bir giriş dizesine uygulandığında tüm 0'ları üreten mantıksal işlemler dizisini (ve ilgili maskelerini) tanımlayın. Aksi takdirde, çıktı en az bir 1 içermelidir.

***38.** Aşağıdaki bit desenlerine 4 bitlik bir sola döngüsel kaydırma uygulamanın sonucu ne olur?

a. 10101 b. 1111 0000 c. 001
d. 101000 Ve. 00001

***39.** Aşağıdaki baytlara 2 bitlik bir sağa döngüsel kaydırma uygulamanın sonucu ne olur? (Cevaplarınızı onaltılık notasyonda verin.)

a. 3F b. 0D
c. FF d. 77

***40.** a. Ek C'nin makine dilinde hangi tek komut kullanılabilir kayıt B'nin 5 bitlik sağ döngü kaydırmasınıbaşar abilir misin?

b. Ek C'deki makine dilinde hangi tek komut, B kaydının 2 bitlik sola dairesel kaydırılmasını gerçekleştirmek için kullanılabilir?

***41.** Bellek hücresinin içeriğini tersine çeviren bir program yazınEk C'nin makine dilinde, adres 8C'de. (Yani, soldan sağa okunduğunda adres 8C'deki sonbit deseni, sağdan sola okunduğunda orijinal deseneuygun olmalıdır.)

***42.** A1'de saklanan değeri A2'de saklanan değerden çıkaran ve sonucu A0 adresine yerleştiren bir program yazın. Değerlerin iki'nin tamamı notasyonunda kodlandığını varsayın.

***43.** Yüksek tanımlı video, her birinin 1920 × 1080 piksel çözünürlüğe sahip olduğu 30 kare/saniye (fps) hızında iletilebilir. Bu formatta sıkıştırılmamış bir video akışı USB 1.1 seri portu üzerinden gönderilebilir mi? USB 2.0 seri portu üzerinden? USB 3.0 seri portu mu? (Not: USB 1.1, USB 2.0 ve USB 3.0 seri portlarının maksimumhızları sırasıyla 12Mbps, 480Mbps ve 5 Gbps'dir.)

***44.** Bir kişinin bir klavye üzerinde dakikada kırk kelime yazdığını varsayalım. (Bir kelime beş karakter olarak kabul edilir.) Eğer bir makine her mikro saniyede (milyonda bir saniye) 500 komut yürütüyorsa, iki ardışık karakterin yazılması arasındaki süre için de makine kaç komut yürütür?

***45.** Bir klavye, dakikada kırkkelime yazan bir yazara ayak uydurmak içinsaniyede kaç bit iletmelidir? (Her karakterinASCII ile kodlandığını ve her kelimeninaltı karakterden oluştuğunu varsayın.)

***46.** Ek C'de tanımlanan makinenin, bellek haritalı G/Ç tekniğini kullanarak bir yazıcı ile iletişim kurduğunu varsayalım. Ayrıca, FF adresinin karakterleri yazıcıya göndermek için kullanıldığını ve FE adresinin yazıcının durumu hakkında bilgi almak için kullanıldığını varsayalım. Özellikle, adres FE'deki en az anlamlı bitin printer'ın başka bir karakter almak için hazır olup olmadığınıbelirttiğini varsayalım (0 "hazır değil" ve 1 "hazır"olarak gösterir). Adres 00'dan başlayarak, bekleyen bir makine dili rutini yazın.

Doclingo https://www.doclingo.ai

yazıcı bir başka karakter için hazır ve sonra 5 numaralı kayıttaki bit desenini temsil eden karakteri yazıcıya gönderir.

*47. Makine dilinde bir program yazın Ek C'de tanımlandığı gibi, A0'dan C0'a kadar olan tüm bellek hücrelerine 0 yerleştirir ancak 00'dan 13'e kadar (onaltılık) bellek hücrelerine sığacak kadar küçüktür.

*48. Bir makinenin 200 GB depolama alanı olan bir sabit diski olduğunu ve 15 Mbps hızında bir geniş bant bağlantısı üzerinden veri aldığını varsayalım. Bu hızda, mevcut depolama alanını doldurmak ne kadar zaman alır?

*49. Bir uydu sisteminin 250 Kbps hızında seri bir veri akışını almak için kullanıldığını varsayalım. Eğer bir atmosferik parazit patlaması 6.96 saniye sürerse, kaç veri biti etkilenir?

*50. Size her biri iki çok basamaklı sayının toplamınıbir milyonuncu saniyede bulabilen 32 işlemciverildiğini varsayalım. Paralel işleme teknik lerinin64 sayının toplamını sadece altı milyonuncusaniyede bulmak için nasıl uygulanabileceğinianlatın. Tek bir işlemcinin bu aynı toplamıbulması ne kadar zaman alır?

*51. CISC mimarisi ile RISC mimarisi arasındaki farkı özetleyin.

*52. Verimliliği artırmanın iki yolunu belirleyin.

*53. Birden fazla işlemci makinesi ile tek işlemci makinesi arasında bir sayı koleksiyonunun ortalamasının daha hızlı nasıl hesaplanabileceğini anlatın.

Sosyal Sorunlar

Aşağıdaki sorular, bilgisayar alanıyla ilişkili etik/sosyal/hukuki sorunlara bir rehber olarak tasarlanmıştır.Amaç, bu soruları yalnızca yanıtlamak değildir. Ayrıca, neden böyle yanıt verdiğinizi ve gerekçelerinizin bir sorudan diğerine tutarlı olup olmadığını da düşünmelisiniz.

1. Bir bilgisayar üreticisi yeni bir makine mimarisi geliştirirse, şirketin o mimariye sahip olmasına ne ölçüde izin verilmelidir? Toplum için en iyi politika nedir?
2. Bir anlamda, 1923 yılı, birçok kişinin şimdi planlı eskime olarak adlandırdığı şeyin doğuşunu işaret etti. Bu yıl, Alfred Sloan liderliğindeki General Motors, otomobil endüstrisini model yılları kavramıyla tanıştırdı. Amaç, daha iyi bir otomobil sunmak yerine stil değişiklikleri yaparak satışları artırmaktı. Sloan, "Sizi mevcut arabanızdan memnuniyetsiz hale getirmek istiyoruz ki yeni bir tane satın alasınız." şeklinde bir alıntı yapmıştır. Bu pazarlama taktiği günümüzde bilgisayar endüstrisinde ne ölçüde kullanılmaktadır?
3. Sıklıkla bilgisayar teknolojisinin toplumumuzu nasıl değiştirdiği açısından düşünürüz. Ancak birçok kişi, bu teknolojinin eski sistemlerin hayatta kalmasına izin vererek ve bazı durumlarda daha köklü hale gelmesine neden olarak değişimlerin gerçekleşmesini sıklıkla engellediğini savunuyor. Örneğin, merkezi bir hükümetin toplumdaki rolü bilgisayar teknolojisi olmadan hayatta kalır mıydı? Bilgisayar teknolojisi olmasaydı, merkezi otorite bugün ne ölçüde var olurdu? Bilgisayar teknolojisi olmadan ne ölçüde daha iyi veya daha kötü durumda olurduk?
4. Bireyin, bir makinenin iç detayları hakkında hiçbir şey bilmesine gerek olmadığını düşünmesi etik midir? Çünkü başka biri onu inşa edecek, bakımını yapacak ve ortaya çıkan sorunları çözecektir. Cevabınız, makinenin bir bilgisayar, otomobil, nükleer santral veya tost makinesi olup olmamasına bağlı mı?
5. Bir üreticinin bir bilgisayar çipi ürettiğini ve daha sonra tasarımında bir hata keşfettiğini varsayalım. Üreticinin gelecekteki üretimde hatayı düzelttiğini ancak orijinal hatayı gizli tutmaya karar verdiğini ve bunu yapmadığını varsayalım.

Doclingo https://www.doclingo.ai

gönderilen çipleri geri çağırmak, zaten kullanımda olan çiplerin hatanın s onuçlarının olacağı bir uygulamada kullanılmadığını düşünerek. Üreticini n kararından dolayı kimse zarar görüyor mu? Eğer kimse zarar görmüyor sa ve bu karar üreticinin para kaybetmesini engelliyorsa ve muhtemelen çalışanları işten çıkarmasını gerektirmiyorsa, üreticinin kararı haklı mı?

6. Gelişen teknoloji kalp hastalıklarına çareler mi sunuyor yoksa kalp hastalıklarına ka tkıda bulunan hareketsiz bir yaşam tarzının kaynağı mı?
7. Finansal veya navigasyonel felaketlerin, taşma ve kesme problemleri nede niyle meydana gelebilecek aritmetik hatalardan kaynaklanabileceğini hayal etmek kolaydır. Görüntü depolama sistemlerindeki hatalardan kaynaklana n sonuçlar (belki de keşif veya tıbbi teşhis gibi alanlarda) ne olabilir?
8. ARM Holdings, çeşitli tüketici elektronik cihazları için işlemcileri tasarlaya n küçük bir şirkettir. İşlemcilerin hiçbirini üretmez; bunun yerine tasarıml ar, üretilen her birim için telif ücreti ödeyen yarı iletken satıcılarına (Qual comm, Samsung ve Texas Instruments gibi) lisanslanır. Bu iş modeli, bilg isayar işlemcilerinin araştırma ve geliştirme yüksek maliyetini tüm tüketi ci elektronik pazarına yayar.

   Bugün, tüm cep telefonlarının (sadece akıllı telefonlar değil) %95'inden fazlası, tüm diji tal kameraların %40'ından fazlası ve Dijital TV'lerin %25'i ARM işlemcisi kullanmaktadır. Dahası, ARM işlemcileri mini dizüstü bilgisayarlarda, MP3 çalarlarda, oyun kumandala rında, elektronik kitap okuyucularda, navigasyon sistemlerinde ve liste devam etmekt edir. Bunu göz önünde bulundurarak, bu şirketi bir tekel olarak mı görüyorsunuz? Ne den veya neden olmasın? Tüketici cihazları, günümüz toplumunda giderek artan bir ro l oynarken, bu az bilinen şirkete bağımlılık iyi mi, yoksa endişeleri mi artırıyor?

Ek Okuma

Carpinelli, J. D. *Bilgisayar Sistemleri Organizasyonu ve Mimarisi*. Boston, MA: Addison-Wesley, 2001.

Comer, D. E. *Bilgisayar Mimarisi Temelleri*. Upper Saddle River, NJ: Prentice- Hall, 2005.

Dandamudi, S P. *Programcılar ve Mühendisler için RISC İşlemcileri Rehberi*. New York: Springer, 2005.

Furber, S. *ARM Sistem-on-Chip Mimarisi*, 2. baskı. Boston, MA: Addison-Wesley, 2000.

Hamacher, V. C., Z. G. Vranesic ve S. G. Zaky. *Bilgisayar Organizasyonu,* 5. baskı. New York: McGraw-Hill, 2002.

Knuth, D. E. *Bilgisayar Programlamanın Sanatı,* Cilt 1, 3. baskı. Boston, MA: Addison-Wesley, 1998.

Murdocca, M. J. ve V. P. Heuring. *Bilgisayar Mimarisi ve Organizasyonu: Entegre Bir Yaklaşım.*New York: Wiley, 2007.

Stallings, W. *Bilgisayar Organizasyonu ve Mimarisi,* 7. baskı. Upper Saddle River, NJ: Prentice-Hall, 2006.

Tanenbaum, A. S. *Yapılandırılmış Bilgisayar Organizasyonu*, 5. baskı. Upper Saddle River, NJ: Prentice-Hall, 2006.

Doclingo https://www.doclingo.ai

BÖLÜM

İşletimSistemleri

3

Bu bölümde, bir bilgisayarın iç aktivitelerini koordine eden ve dış dünya ile iletişimini denetleyen yazılım paketleri olan işletim sistemlerini inceliyoruz. Bir bilgisayarın işletim sistemi, bilgisayar donanımını faydalı bir araca dönüştüren unsurdur. Amacımız, işletim sistemlerinin ne yaptığını ve bunu nasıl yaptığını anlamaktır. Böyle bir arka plan, aydınlanmış bir bilgisayar kullanıcısı olmanın merkezidir.

3.1 İşletim Sistemlerinin Tarihi

İşletim Sistemleri

3.2 İşletim Sistemi Mimari

Bir Yazılım Anketi
Bir İşletim Sisteminin Bileşenleri
Başlatmak

3.3 Makinenin

Faaliyetlerini Koordine Etmek

Bir Süreç Kavramı
Süreç Yönetimi

***3.4Rekabetin Yönetimi Süreçler Arasında**

Sinyaller
Kilitleme

3.5 Güvenlik

Dışarıdan Gelen Saldırılar
İçeriden Gelen Saldırılar

**Yıldız işaretleri, isteğe bağlı bölümler için önerileri belirtir.*

Doclingo https://www.doclingo.ai

Bir işletim sistemi, bir bilgisayarın genel işletimini kontrol eden yazılımdır.Kullanıcının dosyaları depolayıp almasını sağlayan araçları sunar, kullanıcının programların yürütülmesini talep edebileceği arayüzü sağlar ve talep edilen programları yürütmek için gerekli ortamı sağlar.

Belki de en iyi bilinen işletim sistemi örneği Windows'tur; bu, Microsoft tarafından birçok versiyonda sağlanmakta ve PC alanında yaygın olarak kullanılmaktadır. Diğer bir köklü örnek ise UNIX'tir; bu, daha büyük bilgisayar sistemleri ve PC'ler için popüler bir tercihtir. Aslında, UNIX, Mac OS ve Solaris gibi iki diğer popüler işletim sisteminin temelini oluşturur; Mac OS, Apple tarafından Mac makineleri için sağlanan işletim sistemidir ve Solaris, Sun Microsystems (şu anda Oracle'a ait) tarafından geliştirilmiştir. Hem büyük hem de küçük makinelerde bulunan bir diğer işletim sistemi örneği ise, başlangıçta ticari olmayan bir şekilde bilgisayar meraklıları tarafından geliştirilen ve şimdi IBM de dahil olmak üzere birçok ticari kaynaktan temin edilebilen Linux'tur.

Gündelik bilgisayar kullanıcıları için, işletim sistemleri arasındaki farklar büyük ölçüde kozmetiktir. Bilgisayar profesyonelleri için ise, farklı işletim sistemleri, çalıştıkları araçlarda veya işlerini yayma ve sürdürme felsefelerinde önemli değişiklikler temsil edebilir. Yine de, temelinde tüm ana akım işletim sistemleri, bilgisayar uzmanlarının elli yılı aşkın bir süredir karşılaştığı aynı türden sorunları ele almaktadır.

3.1 İşletim Sistemlerinin Tarihi

Bugünün işletim sistemleri, mütevazı başlangıçlardan büyüyen büyük, karmaşık yazılım paketleridir. 1940'lar ve 1950'lerin bilgisayarları çok esnek veya verimli değildi. Makineler tüm odaları kaplıyordu. Program yürütme, manyetik bantların monte edilmesi, delikli kartların kart okuyuculara yerleştirilmesi, anahtarların ayarlanması gibi ekipmanın önemli bir hazırlığını gerektiriyordu. Her programın yürütülmesi, bir iş olarak adlandırılıyordu ve izole bir etkinlik olarak ele alınıyordu—makine programı yürütmek için hazırlanıyordu, program yürütülüyordu ve ardından bir sonraki program hazırlığına başlanmadan önce tüm bantlar, delikli kartlar vb. geri alınmalıydı. Birden fazla kullanıcının bir makineyi paylaşması gerektiğinde, kullanıcıların makineyi zaman dilimleri için rezerve edebilmeleri için kayıt defterleri sağlanıyordu. Bir kullanıcıya tahsis edilen süre boyunca, makine tamamen o kullanıcının kontrolündeydi. Oturum genellikle program kurulumu ile başlar, ardından kısa program yürütme süreleri gelir. Genellikle, bir sonraki kullanıcının sabırsızca kurulum yapmaya başladığı sırada “Sadece bir dakika alacak” diyerek bir şey daha yapma çabasıyla aceleyle tamamlanıyordu.

Bu tür bir ortamda, işletim sistemleri program kurulumunu basitleştirmek ve işler arasında geçişi kolaylaştırmak için sistemler olarak ortaya çıktı. Erken bir gelişme, kullanıcılar ile ekipmanın ayrılmasıydı; bu, insanların bilgisayar odasına girip çıkma fiziksel geçişini ortadan kaldırdı. Bu amaçla, makineyi çalıştırmak için bir bilgisayar operatörü işe alındı. Bir program çalıştırmak isteyen herkes, programın gereksinimleri hakkında gerekli veriler ve özel talimatlarla birlikte programı operatöre sunmak zorundaydı ve sonuçlar için daha sonra geri dönmeliydi. Operatör, bu malzemeleri makinenin kütle depolama alanına yükledi; burada işletim sistemi adı verilen bir program bunları okuyup çalıştırabiliyordu.

Doclingo https://www.doclingo.ai

Onları birer birer çalıştırdı. Bu, toplu işleme—işlerin tek bir toplu halde topla narak, daha sonra kullanıcı ile daha fazla etkileşim olmadan yürütülmesi—ba şlangıcıydı.

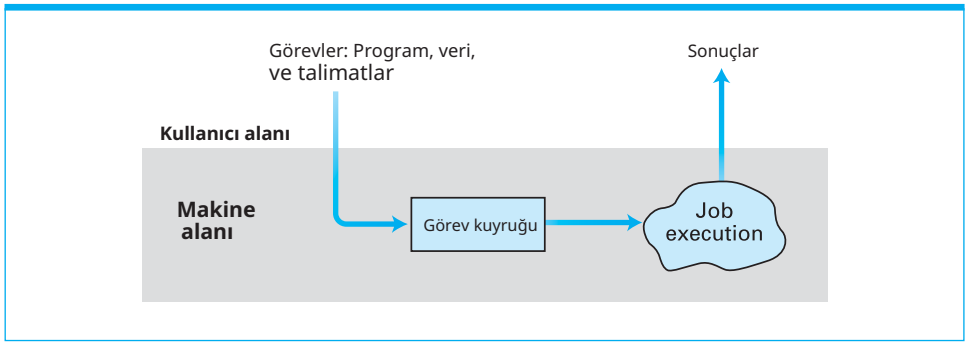
Toplu işleme sistemlerinde, kütle depolamada bulunan işler, iş kuyruğu (Şekil 3.1) içinde yürütülmeyi bekler. Kuyruk, nesnelerin (bu durumda, işleri n) ilk giren, ilk çıkar (kısaca FIFO ve "FI-foe" olarak telaffuz edilir) düzeninde s ıralandığı bir depolama organizasyonudur. Yani, nesneler kuyruğa geldikleri sıraya göre çıkarılır. Gerçekte, çoğu iş kuyruğu FIFO yapısını katı bir şekilde t akip etmez; çünkü çoğu işletim sistemi iş önceliklerinin dikkate alınmasını sa ğlar. Sonuç olarak, iş kuyruğunda bekleyen bir iş, daha yüksek öncelikli bir iş tarafından geçilebilir.

Erken toplu işleme sistemlerinde, her iş, makineyi o belirli iş için hazırlam ak üzere gereken adımları açıklayan bir dizi talimatla birlikte geliyordu. Bu ta limatlar, iş kontrol dili (JCL) olarak bilinen bir sistem kullanılarak kodlanmış v e iş ile birlikte iş kuyruğuna depolanmıştı. İş yürütme için seçildiğinde, işleti m sistemi bu talimatları bir yazıcıda yazdırarak bilgisayar operatörü tarafınd an okunup takip edilebilecek hale getiriyordu. İşletim sistemi ile bilgisayar o peratörü arasındaki bu iletişim, günümüzde de görülmektedir; örneğin, "dis k sürücüsü erişilemez" ve "yazıcı yanıt vermiyor" gibi hataları rapor eden PC i şletim sistemleri buna tanıklık etmektedir.

Bir bilgisayar ile kullanıcıları arasında bir aracı olarak bilgisayar operatörün ün kullanılmasının büyük bir dezavantajı, kullanıcıların işlerini operatöre teslim ettikten sonra onlarla etkileşimde bulunamamalarıdır. Bu yaklaşım, verilerin ve tüm işleme kararlarının önceden belirlendiği bordro işleme gibi bazı uygulama lar için kabul edilebilir. Ancak, kullanıcının bir programla yürütme sırasında etki leşimde bulunması gerektiğinde kabul edilemez. Örnekler arasında, rezervasyo nların ve iptallerin meydana geldikçe bildirilmesi gereken rezervasyon sistemle ri; belgelerin dinamik bir yazma ve yeniden yazma biçiminde geliştirildiği kelim e işleme sistemleri; ve makine ile etkileşimin oyunun merkezi özelliği olduğu bil gisayar oyunları bulunmaktadır.

Bu ihtiyaçları karşılamak için, kullanıcı ile programın yürütülmesi sırasınd a diyalog kurmasına izin veren yeni işletim sistemleri geliştirildi.

Şekil **3.1** Toplu işleme



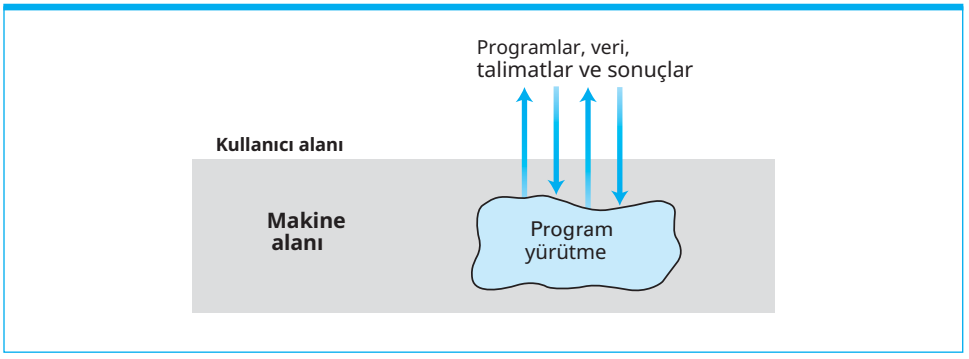
Doclingo https://www.doclingo.ai

uzaktan terminaller— etkileşimli işleme olarak bilinen bir özellik (Şekil 3.2). (Bir terminal, kullanıcının girdi yazabileceği ve bilgisayarın kağıda basılan yanıtını okuyabileceği bir elektronik daktilodan fazlasını içermiyordu. Bugün terminaller, iş istasyonları olarak adlandırılan daha sofistike cihazlara ve istenildiğinde bağımsız bilgisayarlar olarak işlev görebilen tam PC'lere evrimleşmiştir.)Başarılı etkileşimli işlemenin en önemli unsuru, bilgisayarın kullanıcı ihtiyaçlarıyla uyumlu bir şekilde yeterince hızlı hareket etmesidir; bu, kullanıcının makinenin zaman çizelgesine uymak zorunda kalmaması anlamına gelir. (Maaş işleme görevi, bilgisayarın gerektirdiği süreye uymak üzere planlanabilir, ancak bir kelime işlemci kullanmak, makine karakterler yazıldıkça hemen yanıt vermezse sinir bozucu olur.) Bir anlamda, bilgisayar, gerçek zamanlı işleme olarak bilinen bir süreçte son tarih altında görevleri yerine getirmeye zorlanır; bu süreçte gerçekleştirilen eylemlerin gerçek zamanlı olarak gerçekleştiği söylenir. Yani, bir bilgisayarın bir görevi gerçek zamanlı olarak yerine getirmesi, bilgisayarın (dış gerçek dünya) ortamındaki son tarihlere uygun olarak görevi yerine getirmesi anlamına gelir.

Etkileşimli sistemlerin yalnızca bir kullanıcıya hizmet etmesi gerekiyorsa, gerçek zamanlı işleme bir sorun olmazdı. Ancak 1960'lar ve 1970'lerdeki bilgisayarlar pahalıydı, bu nedenle her makine birden fazla kullanıcıya hizmet etmek zorundaydı. Bu nedenle, uzaktaki terminallerde çalışan birkaç kullanıcının aynı anda bir makineden etkileşimli hizmet talep etmesi yaygındı ve gerçek zamanlı düşünceler engeller oluşturuyordu. İşletim sistemi yalnızca bir işi aynı anda yürütmekte ısrar ederse, sadece bir kullanıcı tatmin edici gerçek zamanlı hizmet alırdı.

Bu sorunun çözümü, aynı anda birden fazla kullanıcıya hizmet veren işletim sistemleri tasarlamaktı: zaman paylaşımı olarak adlandırılan bir özellik. Zaman paylaşımını uygulamanın bir yolu, zamanın aralıklara bölündüğü ve ardından her işin yürütülmesinin sadece bir aralıkla sınırlı olduğu çok programlama tekniğini uygulamaktır. Her aralığın sonunda, mevcut iş geçici olarak bir kenara bırakılır ve bir diğeri bir sonraki aralıkta yürütülmesine izin verilir. Bu şekilde işleri hızlı bir şekilde ileri geri karıştırarak, birden fazla işin aynı anda yürütülmesi yanılsaması yaratılır. Yürütülen işlerin türüne bağlı olarak, erken zaman paylaşım sistemleri, aynı anda 30 kullanıcıya kadar kabul edilebilir gerçek zamanlı işleme sağlayabiliyordu. Bugün, çok programlama teknikleri hem tek kullanıcı hem de çok kullanıcı sistemlerinde kullanılmaktadır, bununla birlikte, ilki genellikle çoklu görev olarak adlandırılır. Yani, zaman paylaşımı, birden fazla kullanıcının ortak bir bilgisayara erişimi paylaşmasını ifade ederken, çoklu görev, bir kullanıcının aynı anda birçok görevi yürütmesini ifade eder.

Şekil **3.2**Etkileşimli işleme



Doclingo https://www.doclingo.ai

Çok kullanıcılı, zaman paylaşımına dayalı işletim sistemlerinin gelişimiyle birlikte, tipik bir bilgisayar kurulumu, birçok iş istasyonuna bağlı büyük bir merkezi bilgisaya r olarak yapılandırılmıştır. Bu iş istasyonlarından, kullanıcılar bilgisayar odasının dışı ndan bilgisayarla doğrudan iletişim kurabilirler, böylece bir bilgisayar operatörüne t alep göndermek zorunda kalmazlar. Yaygın olarak kullanılan programlar, makinenin kütle depolama aygıtlarında saklanır ve işletim sistemleri, iş istasyonlarından talep e dildiği gibi bu programları yürütmek üzere tasarlanmıştır. Bu durumda, kullanıcılar il e bilgisayar arasında bir aracı olarak bilgisayar operatörünün rolü azalmaya başlar.

Bugün, bilgisayar operatörünün varlığı esasen ortadan kalkmıştır, özellikle de bilgisayar kullanıcısının bilgisayar işletimi ile ilgili tüm sorumlulukları üstlendiği ki şisel bilgisayarlar alanında. Hatta çoğu büyük bilgisayar kurulumu esasen deneti msiz çalışmaktadır. Gerçekten de, bilgisayar operatörlüğü işi, bilgisayar sistemini yöneten bir sistem yöneticisi rolüne dönüşmüştür; yeni ekipman ve yazılımın edin ilmesi ve kurulumu, yeni hesapların verilmesi gibi yerel düzenlemelerin uygulanm ası ve çeşitli kullanıcılar için kütle depolama alanı sınırlarının belirlenmesi gibi gör evleri yerine getirirken, makineleri doğrudan işletmekten ziyade sistemde ortaya çıkan sorunları çözmek için çaba koordine etmektedir.

Kısacası, işletim sistemleri, programları birer birer alıp yürüten basit program lardan, zaman paylaşımını koordine eden, makinenin kütle depolama aygıtlarında programları ve veri dosyalarını saklayan ve bilgisayar kullanıcılarının taleplerine d oğrudan yanıt veren karmaşık sistemlere dönüşmüştür.

Ama işletim sistemlerinin evrimi devam etmektedir. Çok işlemcili makineler in geliştirilmesi, farklı görevleri farklı işlemcilere atayarak ve her bir işlemcinin z amanını paylaşarak zaman paylaşımı/multitasking yetenekleri sunan işletim sis temlerine yol açmıştır. Bu işletim sistemleri, tüm işlemcilerin verimli bir şekilde kullanılmasını sağlamak için çeşitli işlemcilere dinamik olarak görevler atamak gibi yük dengeleme sorunlarıyla ve mevcut işlemci sayısıyla uyumlu alt görevle re bölme gibi ölçeklendirme sorunlarıyla başa çıkmak zorundadır.

Ayrıca, birçok makinenin büyük mesafeler üzerinden bağlı olduğu bilgisa yar ağlarının ortaya çıkması, ağın faaliyetlerini koordine etmek için yazılım si stemlerinin oluşturulmasına yol açmıştır. Böylece, ağ alanı (4. Bölümde incel eyeceğimiz) birçok açıdan işletim sistemleri konusunun bir uzantısıdır.

Akıllı telefonda neler var?

Cep telefonları daha güçlü hale geldikçe, yalnızca sesli aramaları işlemekten çok daha fazlasını sunmaları mümkün hale geldi. Tipik bir akıllı telefon artık mesaj g öndermek, web'de gezinmek, yönlendirme sağlamak, multimedya içeriği görünt ülemek için kullanılabilir; kısacası, geleneksel bir PC ile aynı hizmetlerin çoğunu sunmak için kullanılabilir. Bu nedenle, akıllı telefonlar, yalnızca akıllı telefon don anımının sınırlı kaynaklarını yönetmek için değil, aynı zamanda hızla genişleyen akıllı telefon uygulama yazılımı koleksiyonunu destekleyen özellikler sağlamak iç in tam teşekküllü işletim sistemlerine ihtiyaç duyar. Akıllı telefon işletim sistemi pazarında hakimiyet mücadelesinin şiddetli olması ve hangi sistemin en yaratıcı özellikleri en iyi fiyatla sunabileceği temelinde çözüme kavuşması muhtemeldir. Akıllı telefon işletim sistemi arenasındaki rakipler arasında Apple'ın iPhone OS'u, Research In Motion'ın BlackBerry OS'u, Microsoft'un Windows Phone'u, Nokia'n ın Symbian OS'u ve Google'ın Android'i bulunmaktadır.

Doclingo https://www.doclingo.ai

sistemler—amaç, birçok kullanıcı arasında birçokmakinada kaynakları yönet mek, tek bir, izole bir bilgisayar yerine.

İşletim sistemlerinde bir diğer araştırma yönü, tıbbi cihazlar, araç elektroniği, ev aletleri, cep telefonları veya diğer taşınabilir bilgisayarlar gibi belirli görevlere a danmış cihazlara odaklanmaktadır. Bu cihazlarda bulunan bilgisayar sistemleri gö mülü sistemler olarak bilinir. Gömülü işletim sistemlerinin genellikle pil gücünü k oruması, zorlu gerçek zamanlı son tarihlere uyması veya az veya hiç insan gözeti mi olmadan sürekli çalışması beklenir. Bu çabada başarılar, Wind River Systems ta rafından geliştirilen ve Spirit ve Opportunity adlı Mars Keşif Araçları'nda kullanılan VxWORKS gibi sistemlerle işaretlenmiştir; Microsoft tarafından geliştirilen Windo ws CE (Pocket PC olarak da bilinir); ve özellikle taşınabilir cihazlarda kullanılmak üz ere PalmSource, Inc. tarafından geliştirilen Palm OS ile.

Sorular & Alıştırmalar

1. Kuyruk örneklerini belirleyin. Her durumda, FIFO yapısını ihlal eden her hangi bir durumu belirtin.
2. Aşağıdaki faaliyetlerden hangileri gerçek zamanlı işleme gerektirir?
   a. Posta etiketleri yazdırma
   b. Bir bilgisayar oyunu oynama
   c. Bir akıllı telefon ekranında numaraların arama yapıldıkça görüntülenmesi
   d. Gelecek yılın ekonomisinin durumunu tahmin eden bir programı yürütme
   ve. MP3 kaydını oynatma
3. Gömülü sistemler ile PC'ler arasındaki fark nedir?
4. Zaman paylaşımı ile çoklu görev arasındaki fark nedir?

3.2 İşletim Sistemi Mimarisi

Tipik bir işletim sisteminin bileşimini anlamak için, önce tipik bir bilgisayar si steminde bulunan yazılımın tamamını göz önünde bulundurmalıyız. Daha sonra işletim sisteminin kendisine odaklanacağız.

Bir Yazılım Anketi

Tipik bir bilgisayar sisteminde bulunan yazılımları incelemeye, yazılımları sını flandırmak için bir şema sunarak yaklaşacağız. Bu tür sınıflandırma şemaları, benzer yazılım birimlerini, zaman dilimlerinin atamasının yakın toplulukların saatlerini bir saat arayla ayarlamasını zorunlu kılması gibi, farklı sınıflara yer leştirir; bu durumda, gün doğumu ve gün batımı arasındaki önemli bir fark y oktur. Ayrıca, yazılım sınıflandırması durumunda, konunun dinamikleri ve ke sin bir otoritenin olmaması çelişkili terminolojiye yol açar. Örneğin, Microsof t'un Windows işletim sistemlerinin kullanıcıları, uygulama ve yardımcı progra m sınıfları olarak adlandıracağımız yazılımları içeren "Aksesuarlar" ve "Yöneti m Araçları" olarak adlandırılan program grupları bulacaklardır. Aşağıdaki

Doclingo https://www.doclingo.ai

taksonomi, evrensel olarak kabul edilen bir gerçek beyanı olarak değil, geniş , dinamik bir konuya ayak basmanın bir aracı olarak görülmelidir.

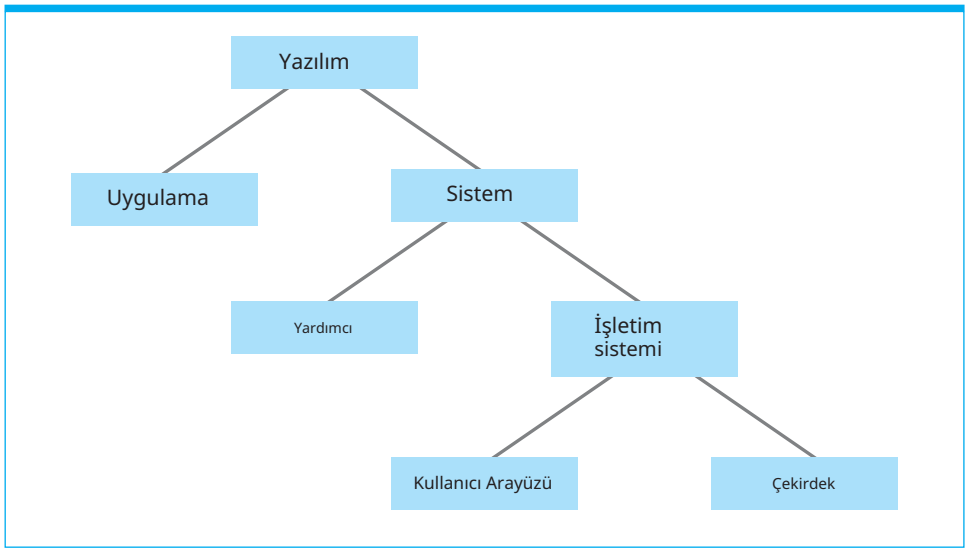
Bir makinenin yazılımını iki geniş kategoriye ayırarak başlayalım: uygulama yazılımı ve sistem yazılımı (Şekil 3.3). Uygulama yazılımımakinenin kullanımına özgü görevleri yerine getiren programlardan oluşur. Bir üretim şirketinin envanterinikorumak için kullanılan bir makine, bir elektrik mühend isinin kullandığı bir makineden farklı uygulama yazılımlarınasahip olacaktır. Uygulama yazılımı örnekleri arasında hesap tabloları,database sistemleri, m asaüstü yayıncılık sistemleri, muhasebe sistemleri, programgeliştirme yazılı mları ve oyunlar bulunmaktadır.

Uygulama yazılımının aksine, sistem yazılımı genel olarak bilgisayar siste mlerine özgü olan görevleri yerine getirir. Bir anlamda, sistem yazılımı,uygul ama yazılımının ihtiyaç duyduğu altyapıyı sağlar; bu, bir ulusun altyapısının (hükümet, yollar, kamu hizmetleri, finansalkurumlar vb.) vatandaşlarının bire ysel yaşam tarzları için güvendiği temeli sağladığı gibi.

Sistem yazılımı sınıfı içinde iki kategori bulunmaktadır: biri işletim sistemi kendisi, diğeri ise toplucautilite yazılımı olarak bilinen yazılım birimleridir. Bi r kurulumun utilite yazılımının çoğunluğu,operasyon sistemine dahil olmaya n ancak bilgisayar kurulumları için temel olan faaliyetleri yerine getiren prog ramlardan oluşur. Bir anlamda, utilite yazılımı,operasyon sisteminin yetenekl erini genişleten (veya belki özelleştiren) yazılım birimlerinden oluşur. Örneği n, manyetik bir diski biçimlendirme veya bir dosyayı manyetik bir diskten bir CD'ye kopyalama yeteneği,genellikle işletim sisteminin kendisi içinde uygula nmaz, bunun yerine bir utilite programı aracılığıyla sağlanır. Diğer utilite yazı lımı örnekleri arasında verileri sıkıştırma ve açma yazılımı,multimedya sunu mlarını oynatma yazılımı ve ağ iletişimini yönetme yazılımı bulunmaktadır.

Belirli etkinliklerin yardımcı yazılım olarak uygulanması, sistem yazılımının belirli bir kurulumun ihtiyaçlarına daha kolay bir şekilde özelleştirilmesine olanak tanır.

Şekil **3.3** Yazılım sınıflandırması



Doclingo https://www.doclingo.ai

Linux

Bir işletim sistemi ile iç bileşenleri denemek isteyen bilgisayar meraklıları içi n Linux mevcuttur. Linux, Helsinki Üniversitesi'nde öğrenci iken Linus Torva lds tarafından tasarlanan bir işletim sistemidir. Tescillenmemiş bir ürün olu p, kaynak kodu (bkz. Bölüm 6) ve belgeleri ile birlikte ücretsiz olarak mevcu ttur. Kaynak kodu biçiminde serbestçe mevcut olduğu için, bilgisayar hobil eri, işletim sistemleri öğrencileri ve genel olarak programcılar arasında pop üler hale gelmiştir. Ayrıca, Linux günümüzde mevcut olan daha güvenilir işl etim sistemlerinden biri olarak tanınmaktadır. Bu nedenle, birçok şirket şim di Linux'un kolay kullanılabilir bir biçimde paketlenmiş ve pazarlanan sürü mlerini sunmaktadır ve bu ürünler artık piyasadaki köklü ticari işletim siste mlerine meydan okumaktadır. Linux hakkında daha fazla bilgiye http://ww w.linux.org adresinden ulaşabilirsiniz.

İşletim sistemine dahil edilselerdi, bu durum daha yaygın olurdu. Gerçekten de , makinelerinin işletim sistemi ile birlikte sağlanan yardımcı yazılımı değiştiren veya ekleyen şirketler veya bireyler bulmak yaygındır.

Ne yazık ki, uygulama yazılımı ile yardımcı yazılım arasındaki ayrım belirsiz ola bilir. Bizim bakış açımıza göre, fark, paketin bilgisayarın "yazılım altyapısının" bir p arçası olup olmadığıdır. Böylece yeni bir uygulama, temel bir araç haline gelirse y ardımcı yazılım statüsüne evrilebilir. Hala bir araştırma projesi iken, İnternet üzeri nden iletişim kurmak için yazılım uygulama yazılımı olarak kabul ediliyordu; bugü n bu tür araçlar, çoğu PC kullanımında temel hale gelmiştir ve bu nedenle yardım cı yazılım olarak sınıflandırılacaktır.

Yardımcı yazılım ile işletim sistemi arasındaki ayrım da aynı derecede belirsizdir. Özellikle, Amerika Birleşik Devletleri ve Avrupa'daki anti-trust davaları, tarayıcılar ve medya oynatıcılar gibi birimlerin Microsoft'un işletim sistemlerinin bileşenleri mi yok sa Microsoft'un rekabeti bastırmak için yalnızca dahil ettiği yardımcı yazılımlar mı old uğu konusundaki sorulara dayanmıştır.

Bir İşletim Sisteminin Bileşenleri

Şimdi bir işletim sistemi alanındaki bileşenlere odaklanalım. Bilgisayarın kullanı cıları tarafından talep edilen eylemleri gerçekleştirmek için, bir işletim sistemini n bu kullanıcılarla iletişim kurabilmesi gerekir. Bu iletişimi yöneten işletim siste mi kısmına genellikle kullanıcı arayüzü denir. Daha eski kullanıcı arayüzleri, kab uk (shell) olarak adlandırılan, kullanıcılarla klavye ve monitör ekranı aracılığıyla metin mesajları kullanarak iletişim kuruyordu. Daha modern sistemler, dosyala r ve programlar gibi manipüle edilecek nesnelerin ekranda simgeler olarak res medildiği grafik kullanıcı arayüzü (GUI - "GOO-ee" olarak telaffuz edilir) aracılığı yla bu görevi yerine getirir. Bu sistemler, kullanıcıların birkaç yaygın giriş cihazı ndan birini kullanarak komut vermesine olanak tanır.

Örneğin, bir veya daha fazla düğmesi olan bir bilgisayar faresi, ekrandaki simgeleri tıklama k veya sürüklemek için kullanılabilir. Fare yerine, grafik sanatçılar veya çeşitli taşınabilir cih az türlerinde genellikle özel amaçlı işaretleme cihazları veya kalemler kullanılır. Son zamanl arda, ince dokunmatik ekranlardaki gelişmeler, kullanıcıların simgeleri doğrudan parmakla rıyla manipüle etmelerine olanak tanımaktadır. Günümüzün GUI'leri iki boyutlu görüntü pr ojeksiyon sistemleri kullanırken, insan kullanıcıların bilgisayarlarla 3D projeksiyon aracılığıy la iletişim kurmasına olanak tanıyan üç boyutlu arayüzler de mevcuttur.

Doclingo https://www.doclingo.ai

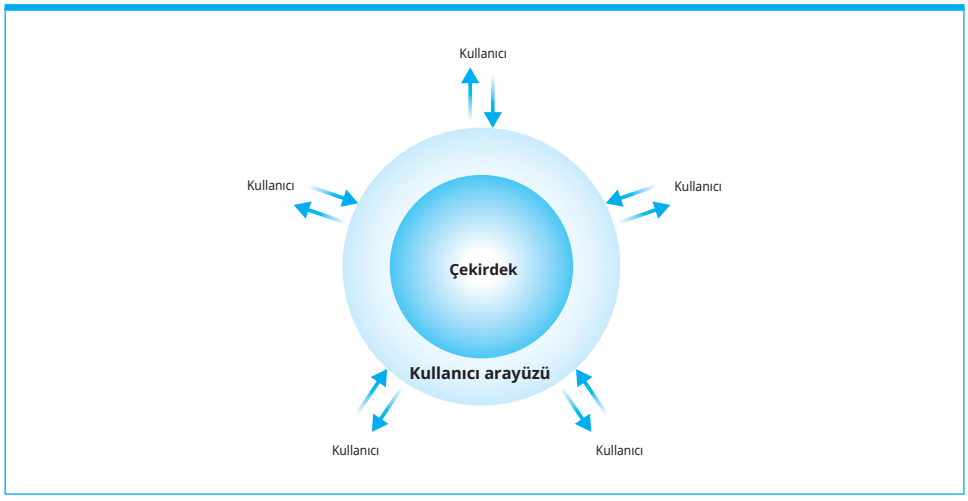
Sistemler, dokunsal duyusal cihazlar ve surround ses audio üretim sistemleri güncel araştırma konularıdır.

Bir işletim sisteminin kullanıcı arayüzü, bir makinenin işlevselliğini belirlemede önemli bir rol oynasa da, bu çerçeve yalnızca bilgisayarın kullanıcısı ile işletim sisteminin gerçek kalbi arasında bir aracı olarak işlev görmektedir (Şekil 3.4). Kullanıcı arayüzü ile işletim sisteminin iç bileşenleri arasındaki bu ayrım, bazı işletim sistemlerinin kullanıcının en rahat etkileşimi elde etmesi için farklı arayüzler seçmesine izin vermesiyle vurgulanmaktadır. UNIX işletim sistemi kullanıcıları, örneğin, Bourne shell, C shell ve Korn shell dahil olmak üzere çeşitli shell'ler arasında seçim yapabilirler; ayrıca X11 adında bir GUI de mevcuttur. Microsoft Windows'un en eski sürümleri, MS-DOS işletim sisteminin komut shell'inden yüklenebilen bir GUI uygulama programıydı. DOS cmd.exe shell'i, en son Windows sürümlerinde bir yardımcı program olarak hala bulunmaktadır, ancak bu arayüz, sıradan kullanıcılar tarafından neredeyse hiç gereksinim duyulmamaktadır. Benzer şekilde, Apple'ın OS X'i, o sistemin UNIX atalarına geri dönen bir Terminal yardımcı programı shell'ini korumaktadır.

Bugünün GUI kabukları içindeki önemli bir bileşen, ekran üzerinde pencereler olarak adlandırılan alan bloklarını tahsis eden ve her pencereyle ilişkili uygulamayı takip eden pencere yöneticisidir.Bir uygulama ekranda bir şey göstermek istediğinde, pencere yöneticisine bildirimde bulunur ve pencere yöneticisi istenen görüntüyü uygulamaya atanan pencerede yerleştirir. Sırasıyla, bir fare düğmesine tıklandığında, fare konumunu ekranda hesaplayan ve fare eylemini uygun uygulamaya bildiren pencere yöneticisidir. Pencere yöneticileri, genellikle bir GUI'nin "stili" olarak adlandırılan şeyden sorumludur ve çoğu yönetici bir dizi yapılandırılabilir seçenek sunar. Linux kullanıcıları, popüler seçenekler arasında KDE ve Gnome'un bulunduğu bir pencere yöneticisi için bir dizi seçeneğe sahiptir.

Bir işletim sisteminin kullanıcı arayüzünün aksine, bir işletim sisteminin iç kısmı çekirdek olarak adlandırılır. Bir işletim sisteminin çekirdeği, bilgisayar kurulumunun gerektirdiği en temel işlevleri yerine getiren yazılım bileşenlerini içerir. Bu tür bir birim, makinenin kütle depolama tesislerinin kullanımını koordine etmekle görevli dosya yöneticisidir.

Şekil **3.4** Kullanıcı arayüzü, kullanıcılar ile işletim sisteminin çekirdeği arasında bir aracı olarak işlev görür.



Doclingo https://www.doclingo.ai

Tüm dosyaların kayıtlarını tutar, bunlar arasında her dosyanın nerede bulundu ğu, hangi kullanıcıların çeşitli dosyalara erişim izni olduğu ve hangi kısımların y eni dosyalar veya mevcut dosyaların uzantıları için kullanılabilir olduğu yer alır. Bu kayıtlar, ilgili dosyaları içeren bireysel depolama ortamında tutulur, böyle ce ortam çevrimiçi her alındığında, dosya yöneticisi bunları alabilir ve böylec e o belirli ortamda neyin depolandığını bilebilir.

Makinenin kullanıcılarının rahatlığı için, çoğu dosya yöneticisi dosyaların bir dizi n veya klasör olarak adlandırılan bir paket halinde gruplanmasına izin verir. Bu yakla şım, bir kullanıcının ilgili dosyaları aynı dizine yerleştirerek dosyalarını amaçlarına gö re organize etmesine olanak tanır. Ayrıca, dizinlerin diğer dizinleri, alt dizinler olarak adlandırılan, içermesine izin vererek hiyerarşik bir organizasyon oluşturulabilir. Örneğin, bir kullanıcı `MyRecords` adında bir dizin oluşturabilir ve bu dizin, Fina ncialRecords, MedicalRecords ve HouseHold-Records adında alt dizinler içerebil ir. Bu alt dizinlerin her birinde o belirli kategoriye giren dosyalar bulunabilir. (W indows işletim sistemi kullanıcıları, dosya yöneticisinden mevcut klasör koleksiy onunu görüntülemesini istemek için Windows Explorer yardımcı programını çal ıştırabilir.)

Bir dizinler zinciri, dizin yolu olarak adlandırılır. Yollar, genellikle, yol boy unca dizinleri listeleyerek ve bunları eğik çizgilerle ayırarak ifade edilir. Örne ğin, `animals/prehistoric/dinosaurs`, animals adlı dizinden başlayarak, prehistoric adlı alt dizininden geçerek ve `dinosaurs` adlı alt-alt dizinde sona eren yolu temsil eder. (Windows kullanıcıları için, böyle bir yol ifadesindeki e ğik çizgiler, animalsprehistoric\dinosaurs şeklinde ters çevrilmiştir.)

Diğer yazılım birimleri tarafından bir dosyaya erişim, dosya yöneticisinin takdirine bağlı olarak elde edilir. Prosedür, dosya yöneticisinden dosyaya eri şim izni talep ederek başlar. Eğer dosya yöneticisi talep edilen erişimi onayla rsa, dosyayı bulmak ve üzerinde işlem yapmak için gereken bilgileri sağlar.

Çekirdek bileşenlerinden bir diğeri, yazılım birimlerinin, makineye bağlı ç evresel cihazlar üzerinde işlemler gerçekleştirmek için denetleyicilerle (veya bazen doğrudan çevresel cihazlarla) iletişim kuran cihaz sürücüleri koleksiyo nundan oluşur. Her cihaz sürücüsü, belirli bir cihaz türü (örneğin, bir yazıcı, d isk sürücüsü veya monitör) için özel olarak tasarlanmıştır ve genel talepleri, o sürücüye atanmış cihazın gerektirdiği daha teknik adımlara çevirir. Örneği n, bir yazıcı için bir cihaz sürücüsü, o yazıcının durum kelimesini okumak ve ç özümlemek için yazılımı içerir ve ayrıca tüm diğer el sıkışma detaylarını da iç erir. Böylece, diğer yazılım bileşenleri bir dosyayı yazdırmak için bu teknik de taylarla uğraşmak zorunda kalmaz. Bunun yerine, diğer bileşenler, dosyayı y azdırmak için yalnızca cihaz sürücüsü yazılımına güvenebilir ve detayları ciha z sürücüsüne bırakabilir. Bu şekilde, diğer yazılım birimlerinin tasarımı, belirl i cihazların benzersiz özelliklerinden bağımsız olabilir.Sonuç, yalnızca uygun cihaz sürücülerini yükleyerek belirli çevresel cihazlar için özelleştirilebilen ge nel bir işletim sistemidir.

Bir işletim sisteminin çekirdeğinin bir diğer bileşeni, makinenin ana belleğini kullan ma görevini koordine etmekle sorumlu olan bellek yöneticisidir. Bilgisayarın yalnızca bi r görevi aynı anda yerine getirmesi istendiği bir ortamda, bu görevler minimaldir. Bu du rumlarda, mevcut görevi yerine getiren program, ana bellekte önceden belirlenmiş bir konuma yerleştirilir, yürütülür ve ardından bir sonraki görevi yerine getiren programla değiştirilir. Ancak, bilgisayarın aynı anda birçok ihtiyacı karşılaması gereken çok kullanıc ılı veya çok görevli ortamlarda, bellek yöneticisinin görevleri kapsamlıdır.

Doclingo https://www.doclingo.ai

Bu durumlarda, birçok program ve veri bloğu ana bellekte eşzamanlı olarak bulunmalıdır. Bu nedenle, bellek yöneticisi bu ihtiyaçlar için bellek alanı bulmalı ve atamalıdır ve her programın eylemlerinin programın tahsis edilen alanıyla sınırlı olmasını sağlamalıdır. Ayrıca, farklı etkinliklerin ihtiyaçları gelip gittiği için, bellek yöneticisi artık işgal edilmeyen bellek alanlarını takip etmelidir.

Bellek yöneticisinin görevi, gerekli toplam ana bellek alanı bilgisayarda mevcut olan alanı aştığında daha da karmaşık hale gelir. Bu durumda, bellek yöneticisi, programları ve verileri ana bellek ile kütle depolama arasında ileri geri döndürerek ek bellek alanı illüzyonu yaratabilir (bu tekniğe sayfalama denir). Örneğin, 8GB ana bellek gerektiğini varsayalım, ancak bilgisayarın yalnızca 4GB'ı var. Daha büyük bellek alanı illüzyonunu yaratmak için, bellek yöneticisi bir manyetik diskte 4GB depolama alanı ayırır. Orada, ana bellekte 8 GB'lık gerçek bir kapasite olsaydı depolanacak bit desenlerini kaydeder. Bu veri, genellikle birkaç KB boyutunda olan sayfalar olarak adlandırılan eşit boyutlu birimlere bölünür. Ardından bellek yöneticisi, ihtiyaç duyulan sayfaların herhangi bir zamanda gerçekten 4GB ana bellekte mevcut olmasını sağlamak için bu sayfaları ana bellek ile kütle depolama arasında ileri geri karıştırır. Sonuç olarak, bilgisayar aslında 8GB ana belleğe sahipmiş gibi işlev görebilir. Sayfalama ile oluşturulan bu büyük “hayali” bellek alanına sanal bellek denir.

Bir işletim sisteminin çekirdeğindeki iki ek bileşen, zamanlayıcı ve dağıtıcıdır; bunları bir sonraki bölümde inceleyeceğiz. Şimdilik, çoklu programlama sisteminde zamanlayıcının hangi aktivitelerin yürütülmek üzere dikkate alınacağını belirlediğini ve dağıtıcıların bu aktivitelere zaman tahsisatını kontrol ettiğini not ediyoruz.

Başlatmak

Bir işletim sisteminin diğer yazılım birimleri tarafından gereken yazılım altyapısını sağladığını gördük, ancak işletim sisteminin nasıl başlatıldığını düşünmedik. Bu,

Donanım Yazılımı

Önyükleme yükleyicisine ek olarak, bir PC'nin ROM'u, klavyeden bilgi alma, bilgisayar ekranında mesaj görüntüleme ve kütle depolamadan veri okuma gibi temel giriş/çıktı aktivitelerini gerçekleştirmek için bir dizi yazılım rutinini içerir. FlashROM gibi kalıcı olmayan belleklerde depolandığı için, bu yazılım makinenin silikonuna—donanımına—kalıcı olarak kazınmamıştır, ancak kütle depolamadaki diğer programlar kadar kolay değiştirilebilir de değildir. Bu orta alanı tanımlamak için "firmware" terimi türetilmiştir. Firmware rutinleri, işletim sistemi işlevsel hale gelmeden önce I/O aktivitelerini gerçekleştirmek için önyükleme yükleyicisi tarafından kullanılabilir. Örneğin, önyükleme süreci gerçekten başlamadan önce bilgisayar kullanıcısıyla iletişim kurmak ve önyükleme sırasında hataları raporlamak için kullanılırlar. Yaygın olarak kullanılan firmware sistemleri arasında "PC'lerde" uzun süredir kullanılan BIOS (Temel Girdi/Çıktı Sistemi), daha yeni EFI (Genişletilebilir Firmware Arayüzü), Sun'ın Open Firmware'ı (şimdi Oracle'ın bir ürünü) ve birçok gömülü cihazda kullanılan CFE (Ortak Firmware Ortamı) bulunmaktadır.

Doclingo https://www.doclingo.ai

Bir bilgisayarın her açılışında gerçekleştirilen boot strapping (genellikle **booting** olarak kısaltılır). Bu prosedür, işletim sisteminikalıcı olarak depoland ığı kütük depolama alanından ana belleğe (makine ilk açıldığındaesas olarak boş olan) aktarır. Boot strap sürecini ve neden gerekli olduğunu anlamak içi n,bilgisayarın CPU'sunu göz önünde bulundurarak başlıyoruz.

Bir CPU, her açıldığında program sayacının belirli bir önceden belirlenmi ş adresle başlaması için tasarlanmıştır.CPU'nun, yürütülecek programın başl angıcını bulmasını beklediği yer burasıdır. Kavramsal olarak,bu nedenle, tek gereken şey işletim sistemini bu konumda depolamaktır. Ancak,teknik nede nlerden dolayı, bir bilgisayarın ana belleği genellikleuçucu teknolojilerden in şa edilir; bu da belleğin, bilgisayar kapatıldığında içindeki verileri kaybettiği a nlamına gelir. Bu nedenle, ana belleğin içeriği,bilgisayar her yeniden başlatıl dığında yenilenmelidir.

Kısacası, bilgisayar ilk açıldığında ana bellekte bir programın (tercihen işl etim sistemi) bulunması gerekiyor,bununla birlikte bilgisayarın uçucu belleği her kapatıldığında silinir. Bu ikilemi çözmek için,CPU'nun başlangıç program ını bulmasını beklediği bilgisayarın ana belleğinin küçük bir kısmı,özel uçucu olmayan bellek hücrelerinden inşa edilmiştir.

Bu bellek, içeriği okunabilir ancak değiştirilemez olduğu için salt okunur bellek (ROM) olarak bilinir. Bir benzetme olarak, ROM'da bit desenlerini depolamayı, küçük sigortaları (bazıları açık - birler - ve bazıları kapalı - sıfırlar - olan) patlatm ak olarak düşünebilirsiniz, ancak kullanılan teknoloji daha gelişmiştir. Daha kes in olarak, günümüz PC'lerindeki çoğu ROM, flash bellek teknolojisi ile inşa edil miştir (bu, özel koşullar altında değiştirilebileceği için katı bir ROM olmadığı anl amına gelir).

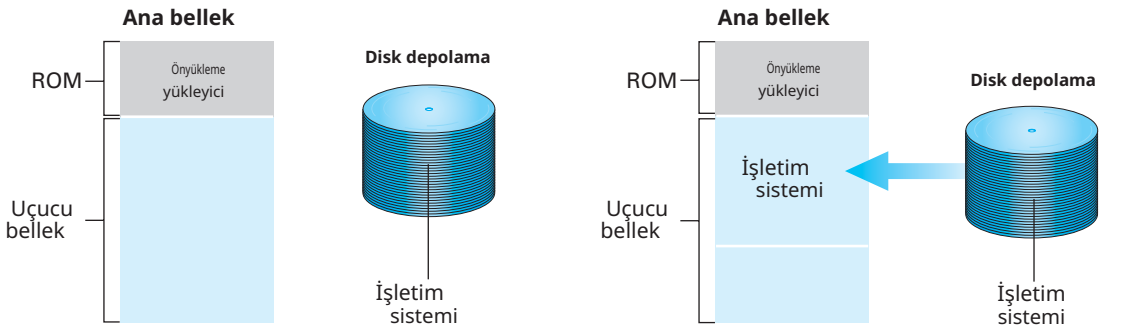
Genel amaçlı bir bilgisayarda, önyükleme yükleyici adı verilen bir progra m makinenin ROM'unda kalıcı olarak depolanır. Bu, makine açıldığında ilk ol arak yürütülen programdır. Önyükleme yükleyicindeki talimatlar, CPU'yu işle tim sistemini önceden belirlenmiş bir konumdan ana bellek (Şekil 3.5) içinde ki geçici alana aktarması için yönlendirir. Modern önyükleme yükleyicileri, işl etim sistemini çeşitli konumlardan ana belleğe kopyalayabilir. Örneğin, akıllı telefonlar gibi gömülü sistemlerde, işletim sistemi özel flash (kalıcı olmayan) bellekten kopyalanır; büyük şirketler veya üniversitelerdeki küçük iş istasyonl arı durumunda, işletim sistemi uzak bir makineden bir ağ üzerinden kopyala nabilir. İşletim sistemi ana belleğe yerleştirildikten sonra, önyükleme yükleyi ci CPU'yu o bellek alanına atlama talimatı yürütmesi için yönlendirir. Bu nokt ada, işletim sistemi devralır ve makinenin faaliyetlerini kontrol etmeye başlar . Önyükleme yükleyicisini yürütme ve böylece işletim sistemini başlatma sür ecine bilgisayarı önyükleme denir.

Bir masaüstü bilgisayarın, kütükleme işleminin gerekli olmadığı şekilde tü m işletim sistemini tutacak kadar yeterli ROM ile donatılmadığını sormuş olabili rsiniz. Küçük işletim sistemlerine sahip gömülü sistemler için bu mümkün olsa da, genel amaçlı bilgisayarlarda büyük blokları kalıcı depolama için ana belleğe ayırmak, günümüz teknolojisi ile verimli değildir. Dahası, bilgisayar işletim sist emleri güvenliği sağlamak ve en son donanım için yeni ve geliştirilmiş cihaz sür ücülerine ayak uydurmak amacıyla sık sık güncellemeler geçirir.

ROM'da depolanan işletim sistemlerini ve önyükleme yükleyicilerini güncellemek mümkün olsa da,

Doclingo https://www.doclingo.ai

Şekil **3.5** Önyükleme süreci



Adım 1: Makine, bellekte zaten bulunan önyükleme yükleyici programını çalıştırarak başlar. İşletim sistemi, kütük depolamada saklanır.

Adım 2: Önyükleme yükleyici programı, işletim sistemi nin ana belleğe aktarımını yönlendirir ve a rdından kontrolü ona devreder.

(genellikle firmware güncellemesi olarak adlandırılır) teknolojik sınırlamalar, kütükleme işlemi için da ha geleneksel bilgisayar sistemleri için en yaygın seçeneği kütük depolama yapar.

Sonuç olarak, önyükleme sürecini ve işletim sistemi, yardımcı yazılım ve uygulama yazılımı arasındaki ayrımları anlamanın, çoğu genel amaçlı bilgisa yar sisteminin çalıştığı genel metodolojiyi kavramamıza yardımcı olduğunu b elirtmeliyiz. Böyle bir makine ilk kez açıldığında, önyükleme yükleyici işletim sistemini yükler ve etkinleştirir. Kullanıcı daha sonra yürütülecek yardımcı ve ya uygulama programlarıyla ilgili işletim sistemine taleplerde bulunur. Her y ardımcı veya uygulama sona erdiğinde, kullanıcı tekrar işletim sistemi ile ba ğlantı kurar ve bu noktada kullanıcı ek taleplerde bulunabilir. Bu nedenle, bö yle bir sistemi kullanmayı öğrenmek iki katmanlı bir süreçtir.

Belirli bir yardımcı program veya uygulamanın ayrıntılarını öğrenmenin yanı sıra,makin anın işletim sistemi hakkında yeterince bilgi edinmek gerekir ki,uygulamalar arasında g ezinebilirsiniz.

Sorular & Alıştırmalar

1. Tipik bir işletim sisteminin bileşenlerini listeleyin veher birinin rolünü tek bir cümleyle özetleyin.
2. Uygulama yazılımı ile yardımcı yazılım arasındaki fark nedir?
3. Sanal bellek nedir?
4. Başlatma prosedürünü özetleyin.

Doclingo https://www.doclingo.ai

3.3 Makinenin Faaliyetlerini Koordine Etme

Bu bölümde, bir işletim sisteminin uygulama yazılımlarının, yardımcı yazılımları n ve işletim sisteminin içindeki birimlerin yürütülmesini nasıl koordine ettiğini e le alıyoruz. Süreç kavramı ile başlıyoruz.

Süreç Kavramı

Modern işletim sistemlerinin en temel kavramlarından biri, bir program ile bi r programın yürütülmesi arasındaki ayrımdır. İlki, statik bir talimat setidir, iki ncisi ise zaman ilerledikçe özellikleri değişen dinamik bir aktivitedir. (Bu ayrı m, bir müzik parçasının, raftaki bir kitapta hareketsiz durması ile o parçayı ça lan bir müzisyenin, nota kağıdında tarif edilen eylemleri gerçekleştirmesi ara sında bir analojidir.) İşletim sistemi kontrolü altında bir programı yürütme ak tivitesine süreç denir. Bir süreçle ilişkili olan, aktivitenin mevcut durumu, sür eç durumu olarak adlandırılır. Bu durum, yürütülen programdaki mevcut ko numu (program sayacının değeri) ve diğer CPU kayıtlarındaki değerler ile iliş kili bellek hücrelerini içerir. Kabaca, süreç durumu, makinenin belirli bir zam andaki anlık görüntüsüdür. Bir programın yürütülmesi sırasında (bir süreçte ki farklı zamanlarda) farklı anlık görüntüler (farklı süreç durumları) gözlemle necektir.

Müzisyenlerin genellikle bir seferde yalnızca bir müzik parçası çalmaya çalış tığı gibi, tipik zaman paylaşımı/multitasking bilgisayarları birçok işlemi çalıştırm aktadır; bunların hepsi bilgisayarın kaynakları için rekabet etmektedir. İşletim s isteminin görevi, her işlemin ihtiyaç duyduğu kaynaklara (çevresel cihazlar, ana bellek alanı, dosyalara erişim ve CPU'ya erişim) sahip olmasını sağlamak, bağı msız işlemlerin birbirine müdahale etmemesini sağlamak ve bilgi alışverişi yap ması gereken işlemlerin bunu yapabilmesini sağlamaktır.

İşlem Yönetimi

İşlemlerin yürütülmesini koordine etme ile ilgili görevler, işletim sisteminin ç ekirdeğindeki zamanlayıcı ve dağıtıcı tarafından yönetilmektedir. Zamanlayıc ı, bilgisayar sisteminde mevcut olan işlemlerin kaydını tutar, bu havuza yeni i şlemler ekler ve tamamlanan işlemleri havuzdan çıkarır. Böylece bir kullanıcı bir uygulamanın yürütülmesini talep ettiğinde, o uygulamanın yürütülmesini mevcut işlemler havuzuna ekleyen zamanlayıcıdır.

Tüm işlemleri takip etmek için zamanlayıcı, ana bellek içinde işlem tablo su olarak adlandırılan bir bilgi bloğu tutar. Bir programın yürütülmesi talep e dildiğinde, zamanlayıcı o işlem için işlem tablosunda yeni bir giriş oluşturur. Bu giriş, işlemin atanmış bellek alanı (bellek yöneticisinden alınan), işlemin ö nceliği ve işlemin hazır mı yoksa beklemede mi olduğu gibi bilgileri içerir. Bir işlem hazır ise, ilerlemesinin devam edebileceği bir durumda demektir; bek lemede ise, ilerlemesi şu anda bazı dış olaylar gerçekleşene kadar gecikmiş d emektir; bu dış olaylar, bir kütle depolama işleminin tamamlanması, klavyed e bir tuşa basılması veya başka bir işlemden bir mesajın gelmesi gibi duruml ar olabilir.

Dispatcher, planlanan süreçlerin yürütülmesini denetleyen çekirdek bileş enidir. Zaman paylaşımı/multitasking sisteminde bu

Doclingo https://www.doclingo.ai

görev, çoklu programlama ile gerçekleştirilir; yani, zamanı kısa parçalara ayırarak, her birine zaman dilimi (genellikle milisaniye veya mikro saniye cinsinden ölçülür) denir ve ardından CPU'nun dikkatini süreçler arasında, her birinin bir zaman dilimi boyunca yürütülmesine izin verildiğinde, değiştirilir (Şekil 3.6). Bir süreçten diğerine geçiş yapma prosedürü süreç değişimi (veya bağlam değişimi) olarak adlandırılır.

Dispatcher her seferinde bir sürece bir zaman dilimi verdiğinde, dilimin sonunu belirten bir sinyal üreterek bir zamanlayıcı devresi başlatır, bu sinyale kesme denir. CPU, bu kesme sinyaline, bir görevden kesildiğinizdeki gibi tepki verir. Ne yaptığınızı durdurur, görevde nerede kaldığınızı kaydedersiniz (böylece daha sonra geri dönebilirsiniz) ve kesen varlıkla ilgilenirsiniz. CPU bir kesme sinyali aldığında, mevcut makine döngüsünü tamamlar, mevcut süreçteki konumunu kaydeder ve ana bellek içinde önceden belirlenmiş bir konumda saklanan kesme işleyicisi adlı bir programı yürütmeye başlar. Bu kesme işleyicisi, dispatcher'ın kesme sinyaline nasıl yanıt vermesi gerektiğini tanımlar ve dispatcher'ın bir parçasıdır.

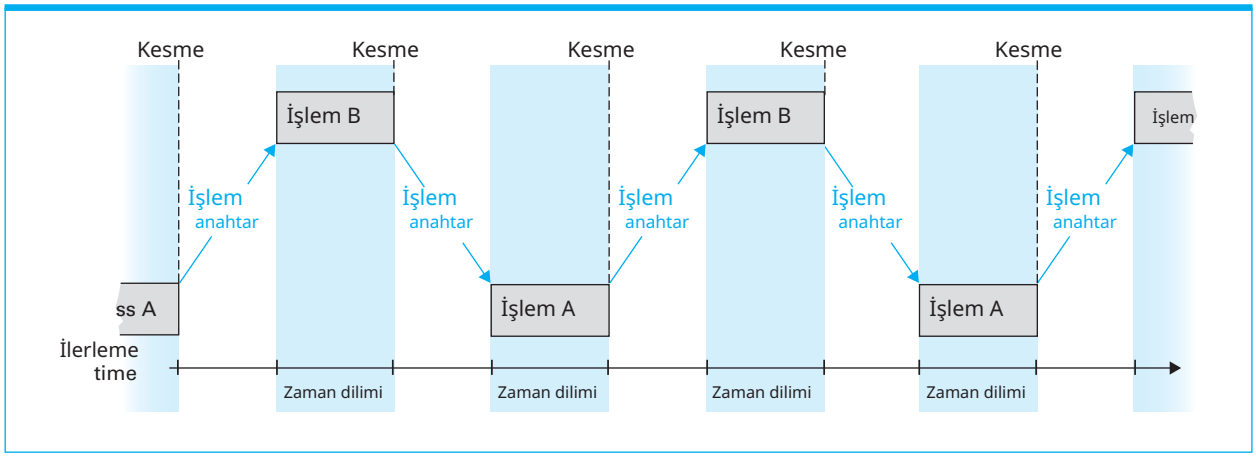
Bu nedenle, kesme sinyalinin etkisi mevcut süreci önceden kesmek ve kontrolü tekrar dağıtıcıya aktarmaktır. Bu noktada, dağıtıcı, hazır süreçler arasında en yüksek önceliğe sahip olan süreci (planlayıcı tarafından belirlenmiş) süreç tablosundan seçer, zamanlayıcı devresini yeniden başlatır ve seçilen sürecin zaman dilimine başlamasına izin verir.

Bir çoklu programlama sisteminin başarısı için en önemli olan, bir süreci durdurma ve daha sonra yeniden başlatma yeteneğidir. Bir kitabı okurken kesildiyseniz, daha sonra okumaya devam etme yeteneğiniz, kitabın içindeki konumunuzu hatırlama yeteneğinize ve o noktaya kadar biriktirdiğiniz bilgilere bağlıdır. Kısacası, kesilmeden hemen önce mevcut olan ortamı yeniden oluşturabilmelisiniz.

Bir süreç durumunda, yeniden oluşturulması gereken ortam, daha önce bahsedildiği gibi, program sayacının değeri ile birlikte kayıtların ve ilgili bellek hücrelerinin içeriğini içeren süreç durumudur.

Çoklu programlama sistemleri için tasarlanmış CPU'lar, bu bilgiyi kesme sinyaline yanıt olarak CPU'nun tepkisinin bir parçası olarak kaydetme görevini içerir. Bu CPU'lar ayrıca

Şekil **3.6** İşlem A ve işlem B arasında çoklu programlama



Doclingo https://www.doclingo.ai

Kesintiler

Metinde açıklanan zaman dilimlerini sonlandırmak için kesintilerin kullanımı, bir bilgi sayarın kesinti sisteminin birçok uygulamasından sadece biridir. Kesinti sinyali üretil en birçok durum vardır ve her birinin kendi kesinti rutini vardır. Gerçekten de, kesinti ler bir bilgisayarın eylemlerini çevresiyle koordine etmek için önemli bir araç sağlar. Örneğin, hem bir fareye tıklamak hem de klavyede bir tuşa basmak, CPU'nun mevcut etkinliğini bir kenara bırakmasına ve kesintinin nedenine yönelmesine neden olan k esinti sinyalleri üretir.

Gelen kesintileri tanıma ve yanıt verme görevini yönetmek için, çeşitli k esinti sinyallerine öncelikler atanır, böylece daha önemli görevler önce hall edilebilir. En yüksek öncelikli kesinti genellikle bir güç kesintisi ile ilişkilidir. Bilgisayarın gücü beklenmedik bir şekilde kesilirse, böyle bir kesinti sinyali üretilir. İlgili kesinti rutini, voltaj seviyesinin operasyonel seviyenin altına dü şmeden önceki milisaniyeler boyunca CPU'yu bir dizi "temizlik" işini yapmas ı için yönlendirir.

makine dili komutlarının daha önce kaydedilmiş bir durumu yeniden yüklemek içinbulunm a eğiliminde olduğunu belirtmek gerekir. Bu tür özellikler, birişlem anahtarını gerçekleştiri rken dağıtıcı için görevi basitleştirir ve modern CPU'larınbugünün işletim sistemlerinin ihtiy açlarından nasıl etkilendiğini örnekler.

Sonuç olarak, çoklu programlamanın bir makinenin genel verimliliğini artır dığıbelirtilmelidir. Bu, çoklu programlamanın gerektirdiği işlemlerin karıştırılm asınınbir yük getirmesi nedeniyle biraz sezgisel olarak ters bir durumdur. Anca k, çokluprogramlama olmadan her işlem, bir sonraki işlem başlamadan önce ta mamlanır,yani bir işlemin çevresel cihazların görevleri tamamlamasını beklerke n veya birkullanıcının bir sonraki isteği yapmasını beklerken harcanan zaman k aybolur.Çoklu programlama, bu kaybolan zamanın başka bir işleme verilmesin e olanak tanır.Örneğin, bir işlem, manyetik bir diskten veri almak gibi bir I/O ist eğigerçekleştirirse, zamanlayıcı, işlemin bir dış olayı beklediğini yansıtacak şekil deişlem tablosunu güncelleyecektir. Bunun sonucunda, dağıtıcı o işleme zama n dilimleriverme işlemini durduracaktır. Daha sonra (belki birkaç yüz milisaniye sonra), I/Oisteği tamamlandığında, zamanlayıcı işlem tablosunu güncelleyerek işlemin hazırolduğunu gösterecek ve böylece o işlem tekrar zaman dilimleri içi n rekabet edecektir.Kısacası, I/O isteği gerçekleştirilirken diğer görevlerde ilerl eme kaydedilecektir ve böylece tüm görevler, ardışık bir şekilde yürütüldüğünd e harcanacak olandan daha kısa bir sürede tamamlanacaktır.

Sorular & Alıştırmalar

1. Bir program ile bir süreç arasındaki farkı özetleyin.
2. Bir kesinti meydana geldiğinde CPU tarafından gerçekleştirilen adımları özetleyin.
3. Bir çoklu programlama sisteminde, yüksek öncelikli süreçlerin diğerlerinden daha hızlı ç alışmasına nasıl izin verilebilir?

Doclingo https://www.doclingo.ai

4. Eğer çoklu programlama sistemindeki her zaman dilimi 50 milisaniye ise ve her bağlam değişikliği en fazla bir mikro saniye gerektiriyorsa, makine bir saniyede kaç işlemi hizmet verebilir?
5. Eğer her işlem, Soru 4'teki makinede zaman diliminin tamamını kullanıyorsa, makinenin zamanının ne kadar kısmı gerçekten işlemleri gerçekleştirmekle har canmaktadır? Eğer her işlem zaman diliminin sadece bir mikro saniyesinden so nra bir G/Ç isteği gerçekleştirirse, bu kesir ne olurdu?

3.4 Süreçler Arasında Rekabetin Yönetimi

Bir işletim sisteminin önemli bir görevi, makineninkaynaklarını sistemdeki sü reçlere tahsis etmektir. Burada *kaynak* teriminimakinenin çevresel cihazları nı ve makinenin kendisindeki özellikleri de kapsayan geniş bir anlamda kulla nıyoruz. Dosya yöneticisi dosyalara erişim tahsis eder ve yeni dosyaların oluş turulması için kütle depolama alanı tahsis eder; bellek yöneticisi bellek alanı tahsis eder; zamanlayıcı süreç tablosunda alan tahsis eder; ve dağıtıcı zaman dilimleri tahsis eder. Bilgisayar sistemlerindeki birçok sorun gibi, bu tahsis g örevi ilk bakışta basit görünebilir. Ancak yüzeyin altında, kötü tasarlanmış bir sistemde arızalara yol açabilecek birkaç incelik bulunmaktadır. Unutmayın, bir makine kendi başına düşünmez; sadece talimatları takip eder. Bu nedenl e, güvenilir işletim sistemleri oluşturmak için, her olası durumu kapsayan alg oritmalar geliştirmeliyiz, ne kadar önemsiz görünse de.

Sinyaller

Tek bir yazıcıyı kontrol eden bir zaman paylaşımı/multitasking işletim sistemi ni düşünelim. Eğer bir süreç sonuçlarını yazdırmak isterse, işletim sistemind en yazıcının cihaz sürücüsüne erişim talep etmesi gerekir. Bu noktada, işleti m sistemi, yazıcının başka bir süreç tarafından kullanılıp kullanılmadığına ba ğlı olarak bu talebi kabul edip etmeyeceğine karar vermelidir. Eğer kullanılmı yorsa, işletim sistemi talebi kabul etmeli ve sürecin devam etmesine izin ver melidir; aksi takdirde, işletim sistemi talebi reddetmelidir.

Microsoft'un Görev Yöneticisi

Microsoft Windows işletim sisteminin bazı iç aktivitelerine, Görev Yöneticisi a dlı yardımcı programı çalıştırarak göz atabilirsiniz. (Ctrl, Alt ve Delete tuşlarına aynı anda basın.) Özellikle, Görev Yöneticisi penceresindeki İşlemler sekmesi ni seçerek, işlem tablosunu görüntüleyebilirsiniz. İşte yapabileceğiniz bir den ey: Herhangi bir uygulama programını etkinleştirmeden önce işlem tablosun a bakın. (Tabloda bu kadar çok işlemin olduğunu görmek sizi şaşırtabilir. Bunl ar sistemin temel işleyişi için gereklidir.) Şimdi bir uygulamayı etkinleştirin ve ek bir işlemin tabloya girdiğini doğrulayın. Ayrıca, işleme ne kadar bellek alanı tahsis edildiğini de görebileceksiniz.

Doclingo https://www.doclingo.ai

ve belki de süreci yazıcı mevcut olana kadar bekleyen bir süreç olarak sınıfla ndırmak. Sonuçta, eğer iki süreç bilgisayarın yazıcısına eşzamanlı erişim veril irse, sonuçlar her ikisi için de değersiz olur.

Yazıcıya erişimi kontrol etmek için işletim sistemi, yazıcının tahsis edilip e dilmediğini takip etmelidir. Bu görevi yerine getirmenin bir yolu, bu bağlamd a bellek içinde durumları genellikle *set* ve *clear* olarak adlandırılan bir bit kull anmaktır; 1 ve 0 yerine. Temiz bir bayrak (değer 0), yazıcının mevcut olduğun u gösterirken, ayarlanmış bir bayrak (değer 1) yazıcının şu anda tahsis edildi ğini gösterir. Yüzeyde, bu yaklaşım mantıklı görünüyor. İşletim sistemi, her y azıcı erişim talebi yapıldığında bayrağı kontrol eder. Eğer bayrak temizse, tal ep kabul edilir ve işletim sistemi bayrağı ayarlar. Eğer bayrak ayarlanmışsa, i şletim sistemi talep eden süreci bekletir. Her seferinde bir süreç yazıcı ile işin i bitirdiğinde, işletim sistemi ya yazıcıyı bekleyen bir sürece tahsis eder ya da eğer bekleyen bir süreç yoksa, sadece bayrağı temizler.

Ancak, bu basit bayrak sisteminin bir sorunu var. Bayrağı test etme ve m uhtemelen ayarlama görevi birkaç makine talimatı gerektirebilir. (Bayrağın d eğeri ana bellekten alınmalı, CPU içinde işlenmeli ve nihayetinde belleğe geri kaydedilmelidir.) Bu nedenle, bir görev, temiz bir bayrak tespit edildikten so nra ama bayrak ayarlanmadan önce kesintiye uğrayabilir.
Özellikle, yazıcının şu anda mevcut olduğunu ve bir işlemin onun kullanımını talep ettiğini varsayalım. Bayrak ana bellekten alınır ve temiz olduğu bulunur , bu da yazıcının mevcut olduğunu gösterir. Ancak, bu noktada işlem kesilir v e başka bir işlem zaman dilimini almaya başlar. O da yazıcının kullanımını tal ep eder. Yine, bayrak ana bellekten alınır ve hala temiz olduğu bulunur çünk ü önceki işlem, işletim sisteminin bayrağı ana bellekte ayarlamak için zaman bulmadan kesilmiştir. Sonuç olarak, işletim sistemi ikinci işlemin yazıcıyı kull anmasına izin verir. Daha sonra, orijinal işlem, işletim sisteminin bayrağın te miz olduğunu bulduğu yerden, yani hemen sonrasında yürütmeye devam ed er. Böylece işletim sistemi, ana bellek içinde bayrağı ayarlayarak orijinal işle me yazıcıya erişim izni verir. Artık iki işlem aynı yazıcıyı kullanmaktadır.

Bu sorunun çözümü, bayrağı test etme ve muhtemelen ayarlama görevini kesint isiz tamamlamayı zorunlu kılmaktır. Bir yaklaşım, çoğu makine dilinde sağlanan kesi ntiyi devre dışı bırakma ve kesintiyi etkinleştirme talimatlarını kullanmaktır. Uygulan dığında, bir kesintiyi devre dışı bırakma talimatı, gelecekteki kesintilerin engellenme sine neden olurken, bir kesintiyi etkinleştirme talimatı CPU'nun kesinti sinyallerine y anıt vermeye devam etmesini sağlar. Böylece, işletim sistemi bayrak test etme rutini ne bir kesintiyi devre dışı bırakma talimatıyla başlarsa ve bir kesintiyi etkinleştirme ta limatıyla bitirirse, rutin başladığında başka bir etkinlik onu kesintiye uğratamaz.

Başka bir yaklaşım, birçok makine dilinde mevcut olan **test-and-set** komutu nu kullanmaktır. Bu komut, CPU'ya bir bayrağın değerini almasını, alınan değer i not etmesini ve ardından bayrağı ayarlamasını yönlendirir—tüm bunlar tek bi r makine komutu içinde gerçekleşir. Buradaki avantaj, CPU'nun her zaman bir k omutu tamamlamadan bir kesintiyi tanımadığı için, bayrağı test etme ve ayarla ma görevinin tek bir komut olarak uygulandığında bölünememesidir.

Yukarıda açıklandığı gibi düzgün bir şekilde uygulanmış bir bayrak, demiryolu sinyallerine atıfta bulunarak semafor olarak adlandırılır; bu sinyaller, ray kesimle rine erişimi kontrol etmek için kullanılır. Aslında, semaforlar yazılım sistemlerinde, demiryolu sistemlerinde olduğu gibi benzer şekilde kullanılmaktadır. Sadece

Doclingo https://www.doclingo.ai

bir tren bir seferde yalnızca bir işlem tarafından yürütülmesi gereken bir komut dizisidir. Bu tür bir komut dizisine kritik bölge denir.
Bir seferde yalnızca bir işlemin kritik bir bölgeyi yürütmesine izin verilmesi gerekliliği, karşılıklı dışlama olarak bilinir. Özetle, kritik bir bölgeye karşılıklı dışlama sağlamak için yaygın bir yol, kritik bölgeyi bir semafor ile korumaktır. Kritik bölgeye girmek için bir işlem, semaforu boş bulmalı ve ardından kritik bölgeye girmeden önce semaforu ayarlamalıdır; kritik bölgeden çıkarken ise işlem semaforu temizlemelidir. Semafor ayarlı durumda bulunursa, kritik bölgeye girmeye çalışan işlem, semafor temizlenene kadar beklemek zorundadır.

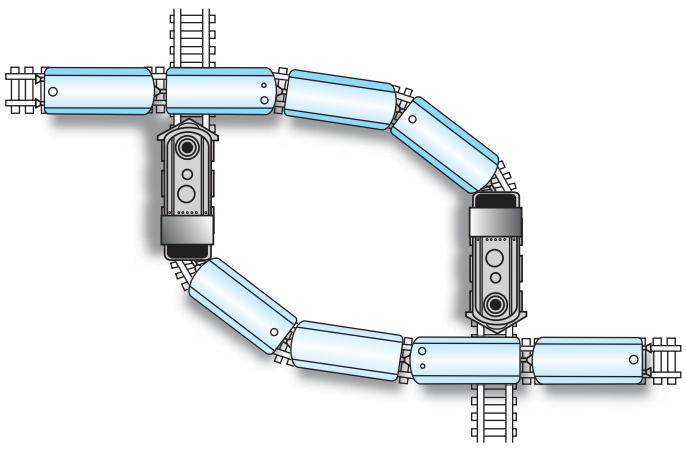
Ölü kilit

Kaynak tahsisi sırasında ortaya çıkabilecek bir diğer sorun, ölü kilit durumudur; bu, iki veya daha fazla işlemin, her birinin başka birine tahsis edilmiş bir kaynağı beklemesi nedeniyle ilerleyemediği durumdur. Örneğin, bir işlem bilgisayarın yazıcısına erişime sahip olabilir ancak bilgisayarın CD çalarına erişim bekliyor olabilirken, diğer bir işlem CD çalarına erişime sahip olabilir ancak yazıcıyı bekliyor olabilir. Diğer bir örnek, işlemlerin yeni işlemler oluşturmasına izin verilen sistemlerde (UNIX jargonunda fork etmek olarak adlandırılan bir eylem) alt görevleri yerine getirmek için ortaya çıkar. Eğer zamanlayıcıda işlem tablosunda yer kalmamışsa ve sistemdeki her işlem, görevini tamamlamadan önce ek bir işlem oluşturmak zorundaysa, o zaman hiçbir işlem devam edemez. Bu tür koşullar, diğer ortamlardaki gibi (Şekil 3.7), bir sistemin performansını ciddi şekilde düşürebilir.

Ölü kilidin analizi, aşağıdaki üç koşulun hepsinin sağlanmadığı sürece meydana gelemeyeceğini ortaya koymuştur:

1. Paylaşılamayan kaynaklar için rekabet vardır.
2. Kaynaklar kısmi olarak talep edilir; yani, bazı kaynaklar alındıktan sonra, bir işlem daha fazla talep etmek için daha sonra geri dönecektir.
3. Bir kaynak tahsis edildikten sonra, zorla geri alınamaz.

Şekil **3.7** Paylaşılamayan demiryolu kesişimleri için rekabetten kaynaklanan bir deadlock



Doclingo https://www.doclingo.ai

Bu koşulları izole etmenin amacı, deadlock probleminin üç koşuldan herhangi birine saldırarak ortadan kaldırılabileceğidir. Üçüncü koşula saldıran teknikler, deadlock tespiti ve düzeltme şemaları olarak bilinen kategoriye girer. Bu durumlarda, deadlock'un meydana gelmesi o kadar uzak bir ihtimal olarak kabul edilir ki, problemi önlemek için hiçbir çaba gösterilmez. Bunun yerine, yaklaşım, meydana gelirse tespit etmek ve ardından bazı tahsis edilmiş kaynakları zorla geri alarak düzeltmektir. Tam bir işlem tablosu örneğimiz bu sınıfa girebilir. Eğer deadlock, dolu bir tablo nedeniyle meydana gelirse, işletim sistemi içindeki rutinler (veya belki de "süper kullanıcı" olarak yetkilerini kullanan bir insan yöneticisi) bazı süreçleri kaldırabilir (teknik terim **kill**). Bu, işlem tablosunda alan açar, deadlock'u kırar ve kalan süreçlerin görevlerine devam etmesine izin verir.

İlk iki koşula saldıran teknikler, deadlock önleme şemaları olarak bilinir. Örneğin, biri, her sürecin tüm kaynaklarını bir seferde talep etmesini gerektirerek ikinci koşula saldırır. Diğer bir şema, rekabeti doğrudan ortadan kaldırmak yerine, paylaşılamayan kaynakları paylaşılabilir hale dönüştürerek birinci koşula saldırır. Örneğin, söz konusu kaynak bir yazıcı ise ve çeşitli süreçler onun kullanımını gerektiriyorsa.

Her seferinde bir işlem yazıcıyı talep ettiğinde, işletim sistemi talebi yerine getirebilir.Ancak, işlemi yazıcının aygıt sürücüsüne bağlamak yerine, işletim sistemi onu yazdırılacak bilgileri kütük depolama alanında saklayan bir aygıt sürücüsüne bağlayacaktır.

Böylece her işlem, yazıcıya erişimi olduğunu düşünerek normal bir şekilde çalışabilir. Daha sonra, yazıcı mevcut olduğunda, işletim sistemi verileri kütük depolama alanından yazıcıya aktarabilir. Bu şekilde, işletim sistemi, paylaşılmayan bir kaynağı, birden fazla yazıcı illüzyonu yaratarak paylaşılabilir gibi gösterir. Verileri daha sonra ama daha uygun bir zamanda çıktı için tutma tekniğine spooling denir.

Spooling'i, birkaç işleme ortak bir kaynağa erişim sağlama tekniği olarak tanıttık - bu tema birçok varyasyona sahiptir. Örneğin,

Çok Çekirdekli İşletim Sistemleri

Geleneksel zaman paylaşımı/multitasking sistemleri, insanın algılayabileceğinden daha hızlı zaman dilimleri arasında hızlıca geçiş yaparak birçok işlemi aynı anda yürütme illüzyonunu verir. Modern sistemler bu şekilde çoklu görev yapmaya devam eder, ancak ayrıca, en son çok çekirdekli CPU'lar gerçekten iki, dört veya daha fazla işlemi aynı anda çalıştırma kapasitesine sahiptir. Tek çekirdekli bilgisayarların birlikte çalıştığı bir gruptan farklı olarak, çok çekirdekli bir makine, bilgisayarın çevresel birimlerini, belleğini ve diğer kaynaklarını paylaşan birden fazla bağımsız işlemci (bu durumda çekirdekler olarak adlandırılır) içerir. Çok çekirdekli bir işletim sistemi için bu, dağıtıcı ve zamanlayıcının her çekirdekte hangi işlemlerin yürütüleceğini dikkate alması gerektiği anlamına gelir. Farklı çekirdekler de farklı işlemler çalıştığında, işlemler arasındaki rekabeti yönetmek daha zor hale gelir çünkü bir çekirdek kritik bir bölgeye girmek istediğinde tüm çekirdekler de kesintileri devre dışı bırakmak son derece verimsiz olacaktır. Bilgisayar bilimi, yeni çok çekirdekli dünyaya daha uygun işletim sistemi mekanizmaları inşa etmeye yönelik birçok aktif araştırma alanına sahiptir.

Doclingo https://www.doclingo.ai

Dosya yöneticisi, süreçler yalnızca dosyadan veri okuyorsa, aynı dosyaya birden fazla sü recin erişimini sağlayabilir, ancak birden fazla süreç aynı anda bir dosyayı değiştirmeye çalışırsa çatışmalar meydana gelebilir. Bu nedenle, bir dosya yöneticisi, süreçlerin ihtiya çlarına göre dosya erişimini tahsis edebilir, birden fazla sürecin okuma erişimine sahip olmasına izin verirken yalnızca birinin yazma erişimine sahip olmasına izin verebilir. Diğ er sistemler, dosyayı parçalara ayırarak farklı süreçlerin dosyanın farklı kısımlarını aynı anda değiştirmesine olanak tanıyabilir. Ancak, bu tekniklerin her birinin güvenilir bir sis tem elde etmek için çözülmesi gereken incelikleri vardır. Örneğin, yalnızca bir dosyaya okuma erişimi olan süreçler, yazma erişimine sahip bir sürecin dosyayı değiştirdiğinde nasıl bilgilendirilmelidir?

Sorular & Alıştırmalar

1. A süreci ve B süreci aynı makinede zaman paylaşıyorsa ve her biri kıs a süreler için aynı paylaşılamayan kaynağa ihtiyaç duyuyorsa (örneği n, her süreç bağımsız, kısa raporlar dizisi yazdırıyor olabilir), her süre ç kaynağı tekrar tekrar alabilir, serbest bırakabilir ve daha sonra tekr ar talep edebilir. Kaynağa erişimi aşağıdaki şekilde kontrol etmenin b ir dezavantajı nedir:

   Bir bayrağa 0 değeri atayarak başlayın. Eğer işlem A kaynağı talep ederse v e bayrak 0 ise, talebi onaylayın. Aksi takdirde, işlem A'nın beklemesini sağla yın. Eğer işlem B kaynağı talep ederse ve bayrak 1 ise, talebi onaylayın. Aksi takdirde, işlem B'nin beklemesini sağlayın. İşlem A kaynağı bitirdiğinde, ba yrağı 1'e değiştirin. İşlem B kaynağı bitirdiğinde, bayrağı 0'a değiştirin.

2. İki şeritli bir yolun bir tünele girmek için tek şeride birleştiğini varsayalım. Tünelin kullanımını koordine etmek için aşağıdaki sinyal sistemi kuru lmuştur:

   Tünele giren bir araç, tünel girişlerinin üzerindeki kırmızı ışıkların yanm asına neden olur. Araç tünelden çıktığında, ışıklar söner. Eğer yaklaşan bir araç kırmızı ışık bulursa, tünele girmeden önce ışığın sönmesini bek ler.

   Bu sistemdeki hata nedir?

3. İki aracın karşılaştığı tek şeritli bir köprüde meydana gelen çıkmazı ortadan kaldırmak için aşağıdaki çözümlerin önerildiğini varsayalım. Metinde verilen çıkmaz koşulunun her bir çözümle hangi koşulun ortadan kaldırıldığını belirl eyin.

   a. Köprü boş olana kadar bir aracı köprüye almayın.

   b. Eğer araçlar karşılaşırsa, bunlardan birinin geri gitmesini sağlayın.

   c. Köprüye ikinci bir şerit ekleyin.

4. Her bir süreci çoklu programlama sisteminde bir nokta ile temsil ettiğimizi v e ilk nokta tarafından temsil edilen sürecin ikinci nokta tarafından kullanılan (paylaşılamayan) bir kaynağı beklediği durumlarda bir noktadan diğerine o k çizeceğimizi varsayalım. Matematikçiler, ortaya çıkan resmi yönlendirilmiş graf olarak adlandırır. Yönlendirilmiş grafın hangi özelliği sistemdeki kilitlenmeye eşdeğerdir?

Doclingo https://www.doclingo.ai

3.5 Güvenlik

İşletim sistemi bir bilgisayardaki etkinlikleri denetlediğinden, güvenliği sağlama konusu nda da önemli bir rol oynaması doğaldır. Geniş anlamda, bu sorumluluk birden fazla biç imde kendini gösterir, bunlardan biri güvenilirliktir. Dosya yöneticisindeki bir hatanın bi r dosyanın bir kısmının kaybına neden olması durumunda, dosya güvenli değildi. Dağıtı cıdaki bir hatanın sistem arızasına (genellikle sistem çökmesi olarak adlandırılır) yol açar ak bir saatin yazımını kaybetmemize neden olması durumunda, çalışmamızın güvenli ol madığını savunuruz. Bu nedenle, bir bilgisayar sisteminin güvenliği iyi tasarlanmış, güv enilir bir işletim sistemi gerektirir.

Güvenilir yazılım geliştirme, işletim sistemleri ile sınırlı bir konu değildir. Yazılım geliştirme spektrumunun tamamına yayılır ve yazılım mühendisliği ol arak bilinen bilgisayar bilimi alanını oluşturur, bu konuyu 7. Bölümde inceley eceğiz. Bu bölümde, işletim sistemlerinin özellikleriyle daha yakından ilişkili güvenlik sorunlarına odaklanıyoruz.

Dışarıdan Gelen Saldırılar

İşletim sistemlerinin gerçekleştirdiği önemli bir görev, bilgisayarınkaynakları nı yetkisiz kişilerin erişiminden korumaktır. Birden fazla kişi tarafındankullanı lan bilgisayarlarda, bu genellikle çeşitli yetkili kullanıcılar için"hesaplar" oluşt urma yoluyla ele alınır—bir hesap, esasen kullanıcının adı,şifresi ve o kullanıc ıya verilecek ayrıcalıklar gibi girişleri içeren işletimsistemindeki bir kayıttır. İşl etim sistemi, bu bilgiyi her giriş prosedürüsırasında (kullanıcının bir bilgisay arın işletim sistemi ile ilk temasıkurduğu işlem dizisi) sistemi kontrol etmek i çin kullanabilir.

Hesaplar, süper kullanıcı veya yönetici olarak bilinen bir kişi tarafından oluşturulur. Bu kişi, giriş prosedürü sırasında kendisini yönetici olarak belirterek (genellikle ad ve şifre ile) işletim sistemine yüksek ayrıcalıklı erişim kazanır. Bu temas kurulduktan sonra, yönetici işletim sistemi içindeki ayarları değiştirebilir, kritik yazılım paketlerini düzenleyebilir, diğer kullanıcılara verilen ayrıcalıkları ayarlayabilir ve normal kullanıcılara verilmeyen çeşitli bakım faaliyetlerini gerçekleştirebilir.

Bu "yüksek yerden", yönetici, bilgisayar sistemindeki etkinliği izleyerek yıkıc ı davranışları, ister kötü niyetli ister kazara olsun, tespit etme çabası içinde olab ilir. Bu konuda yardımcı olmak için, bilgisayar sisteminde gerçekleşen etkinlikle ri kaydeden ve ardından analiz eden birçok yazılım aracı, denetim yazılımları ol arak adlandırılan, geliştirilmiştir. Özellikle, denetim yazılımları, yanlış şifreler ku llanarak giriş yapma girişimlerinin bir selini açığa çıkarabilir; bu, yetkisiz bir kull anıcının bilgisayara erişim sağlamaya çalıştığını gösterebilir.

Denetim yazılımları ayrıca, bir kullanıcının hesabındaki, o kullanıcının geçmiş davranışlarıyl a uyumlu olmayan etkinlikleri de tanımlayabilir; bu, yetkisiz bir kullanıcının o hesaba erişim sağladığını gösterebilir. (Geleneksel olarak yalnızca kelime işlem ve hesap tablosu yazılıml arı kullanan bir kullanıcının, aniden yüksek teknik yazılım uygulamalarına erişmeye başlam ası veya o kullanıcının yetkilerinin dışında kalan yardımcı program paketlerini çalıştırmaya çalışması olası değildir.)

Denetim sistemlerinin tespit etmek için tasarlandığı bir diğer suçlu, bilgisayarda çalışmaya bırakıldığında etkinlikleri kaydeden ve daha sonra bunları potansiyel bir saldırgana rapor eden, sniffer yazılımının varlığıdır.

Doclingo https://www.doclingo.ai

Eski, iyi bilinen bir örnek, işletim sisteminin giriş prosedürünü simüle eden bi r programdır. Böyle bir program, yetkili kullanıcıları, işletim sistemiyle iletişi m kuruyormuş gibi düşünmeye kandırmak için kullanılabilir; oysa aslında isi mlerini ve şifrelerini bir sahtekâre veriyorlar.

Bilgisayar güvenliği ile ilgili tüm teknik karmaşıklıklar göz önüne alındığınd a, bilgisayar sistemlerinin güvenliğine yönelik en büyük engellerden birinin kull anıcıların dikkatsizliği olduğu birçok kişi için şaşırtıcıdır. Kullanıcılar, tahmin edil mesi görece kolay olan (isimler ve tarihler gibi) şifreler seçerler, şifrelerini arka daşlarıyla paylaşırlar, şifrelerini zamanında değiştirmeyi ihmal ederler, çevrimd ışı kütle depolama cihazlarını makineler arasında ileri geri transfer ederek pota nsiyel bozulmalara maruz bırakırlar ve sistemin güvenliğini tehlikeye atabilecek onaylanmamış yazılımları sisteme aktarırlar. Bu tür sorunlar için, büyük bilgisa yar kurulumlarına sahip çoğu kurum, kullanıcıların gereksinimlerini ve sorumlu luklarını kataloglayan politikalar benimser ve uygular.

İçten Gelen Saldırılar

Bir saldırgan (veya belki de kötü niyetli bir yetkili kullanıcı) bir bilgisayar siste mine erişim sağladığında, bir sonraki adım genellikle bilgi aramak veya yıkıcı yazılımları yerleştirmek için yerler keşfetmektir. Eğer saldırgan, yöneticinin h esabına erişim sağladıysa, bu basit bir süreçtir; bu nedenle yöneticinin şifresi sıkı bir şekilde korunur. Ancak, erişim genel bir kullanıcının hesabı üzerinde n sağlanıyorsa, saldırganın o kullanıcıya verilen ayrıcalıkların ötesine geçmes ine izin vermek için işletim sistemini kandırması gerekir. Örneğin, saldırgan, bir sürecin kendisine tahsis edilen alanın dışındaki ana bellek hücrelerine eri şmesine izin vermek için bellek yöneticisini kandırmaya çalışabilir veya saldır gan, erişimin reddedilmesi gereken dosyaları almak için dosya yöneticisini k andırmaya çalışabilir.

Günümüz CPU'ları, bu tür girişimleri engellemek için tasarlanmış özellikl erle geliştirilmiştir.Örneğin, bir sürecin bellek yöneticisi tarafından kendisine atanan ana bellek alanıyla sınırlı kalması gerekliliğini düşünün. Bu tür kısıtla malar olmadan, bir süreç işletim sistemini ana bellekten silebilir ve bilgisayar ın kontrolünü ele alabilir. Bu tür girişimlere karşı koymak için, çoklu program lama sistemleri için tasarlanmış CPU'lar genellikle işletim sisteminin bir süre cin tahsis edilen bellek alanının üst ve alt sınırlarını depolayabileceği özel am açlı kayıtlar içerir. Daha sonra, süreci gerçekleştirirken, CPU her bellek refera nsını bu kayıtlara karşılaştırarak referansın belirlenen sınırlar içinde olduğun dan emin olur. Eğer referans, sürecin belirlenen alanının dışında bulunursa, CPU otomatik olarak kontrolü işletim sistemine geri aktarır (bir kesme dizisi gerçekleştirerek) böylece işletim sistemi uygun eylemi alabilir.

Bu illüstrasyonda, ince ama önemli bir sorun gömülüdür. Daha fazla güv enlik özelliği olmadan, bir süreç, bellek sınırlarını içeren özel amaçlı kayıtları değiştirerek, belirlenen alanının dışındaki bellek hücrelerine erişim sağlayabi lir. Yani, ek belleğe erişim isteyen bir süreç, üst bellek sınırını içeren kayıttaki değeri artırarak, işletim sisteminden onay almadan ek bellek alanını kullanm aya devam edebilir.

Doclingo https://www.doclingo.ai

Bu tür eylemlere karşı korunmak için, çoklu programlama sistemleri için CPU'lar iki ayrı ayrıcalık seviyesinde çalışacak şekilde tasarlanmıştır; birine "ayrıca ayrıcalıklı mod" diğerine ise "ayrıca ayrıcalıksız mod" diyeceğiz. Ayrıcalıklı modda iken, CPU tüm makine dilindeki komutları yürütme yeteneğine sahiptir. Ancak, ayrıcalıksız modda iken, kabul edilebilir komutların listesi sınırlıdır. Sadece ayrıcalıklı modda mevcut olan komutlara ayrıcalıklı komutlar denir. (Ayrıcalıklı komutların tipik örnekleri, bellek limit kayıtlarının içeriğini değiştiren komutlar ve CPU'nun mevcut ayrıcalık modunu değiştiren komutlardır.) CPU ayrıcalıksız modda iken bir ayrıcalıklı komutu yürütme girişimi bir kesintiye neden olur. Bu kesinti, CPU'yu ayrıcalıklı moda dönüştürür ve kontrolü işletim sistemindeki bir kesinti işleyicisine aktarır.

İlk açıldığında, CPU ayrıcalıklı moddadır. Dolayısıyla, işletim sistemi önyükleme sürecinin sonunda başladığında, tüm komutlar yürütülebilir. Ancak, işletim sistemi her seferinde bir sürecin bir zaman dilimi başlatmasına izin verdiğinde, CPU'yu "ayrıca ayrıcalık modunu değiştir" komutunu yürütmek sureti yle ayrıcalıksız moda geçirir. Bunun sonucunda, eğer süreç bir ayrıcalıklı komutu yürütmeye çalışırsa işletim sistemi bilgilendirilecektir ve böylece işletim sistemi bilgisayar sisteminin bütünlüğünü koruma pozisyonunda olacaktır.

Ayrıcalıklı komutlar ve ayrıcalık seviyelerinin kontrolü, işletim sistemlerinin güvenliği sağlamak için sahip olduğu ana araçtır. Ancak, bu araçların kullanımı, bir işletim sisteminin tasarımının karmaşık bir bileşenidir ve mevcut sistemlerde hatalar bulunmaya devam etmektedir. Ayrıcalık seviyesi kontrolündeki tek bir hata, kötü niyetli programcılar veya istemeden yapılan programlama hataları nedeniyle felakete kapı açabilir. Eğer bir süreç, sistemin çoklu programlama sistemini kontrol eden zamanlayıcıyı değiştirmeye izin verilirse, o süreç zaman dilimini uzatabilir ve makineyi domine edebilir. Eğer bir süreç, çevresel cihazlara doğrudan erişim izni alırsa, sistemin dosya yöneticisi tarafından denetlenmeden dosyaları okuyabilir. Eğer bir süreç, kendisine tahsis edilen alanın dışındaki bellek hücrelerine erişim izni alırsa, diğer süreçler tarafından kullanılan verileri okuyabilir ve hatta değiştirebilir. Bu nedenle, güvenliği sağlamak, bir yöneticinin önemli bir görevi olmaya devam etmekte ve işletim sistemi tasarımında bir hedef olarak kalmaktadır.

Sorular & Alıştırmalar

1. Şifreler için kötü seçim örnekleri verin ve neden kötü seçimler olabileceklerini açıklayın.
2. Intel'in Pentium serisindeki işlemciler, dört ayrıcalık seviyesi sunar. CPU tasarımcıları neden üç veya beş seviye yerine dört seviye sağlamayı tercih etsinler?
3. Eğer bir çoklu programlama sisteminde bir süreç, kendisine tahsis edilen alanın dışındaki bellek hücrelerine erişebilirse, makinenin kontrolünü nasıl ele geçirebilir?

Doclingo https://www.doclingo.ai

Bölüm Gözden Geçirme Problemleri

(Yıldızlı problemler isteğe bağlı bölümlerle ilişkilidir.)

1. Tipik bir işletim sisteminin dört etkinliğini listeleyin.
2. Toplu işleme ile etkileşimli işleme arasındaki farkı özetleyin.
3. R, S ve T adlı üç öğe, bu sırayla bir kuyruğa yerleştirildiğini varsayalım. Daha sonra, dördüncü bir öğe olan X, kuyrukta yer almadan önce kuyruktan bir öğe çıkarılır. Ardından, kuyruktan bir öğe çıkarılır, Y ve Z öğeleri kuyruğa yerleştirilir ve ardından kuyruk, bir seferde bir öğe çıkarılarak boşaltılır. Çıkarılan tüm öğeleri, çıkarıldıkları sıraya göre listeleyin.
4. Gömülü sistemler ile PC'ler arasındaki fark nedir?
5. Çoklu görevli bir işletim sistemi nedir?
6. Bir PC'niz varsa, çoklu görev yeteneklerinden yararlanabileceğiniz bazı durumları tanımlayın.
7. Aşina olduğunuz bir bilgisayar sistemi temelinde, iki uygulama yazılımı birimi ve iki yardımcı yazılım birimi tanımlayın. Ardından, bunları neden bu şekilde sınıflandırdığınızı açıklayın.
8. a. Bir işletim sisteminin kullanıcı arayüzünün rolü nedir?
   b. Bir işletim sisteminin çekirdeğinin rolü nedir?
9. X/Y/Z yoluyla tanımlanan dizin yapısı nedir?
10. "Süreç" terimini işletim sistemleri bağlamında tanımlayın.
11. Bir işletim sistemindeki süreç tablosunda hangi bilgiler bulunmaktadır?
12. Hazır bir süreç ile bekleyen bir süreç arasındaki fark nedir?
13. Sanal bellek ile ana bellek arasındaki fark nedir?
14. Bir bilgisayarın 512MB (MiB) ana belleğe sahip olduğunu ve bir işletim sisteminin bu boyutun iki katı büyüklüğünde 2KB (KiB) sayfalar kullanarak sanal bellek oluşturması gerektiğini varsayalım. Kaç sayfa gerekecektir?
15. İki sürecin aynı anda aynı dosyaya erişim talep etmesi durumunda bir zaman paylaşımı/multitasking sisteminde hangi komplikasyonlar ortaya çıkabilir? Dosya yöneticisinin böyle talepleri onaylaması gerekenden durumlar var mı? Dosya yöneticisinin böyle talepleri reddetmesi gereken durumlar var mı?
16. Uygulama yazılımı ile sistem yazılımı arasındaki ayrım nedir? Her birine bir örnek verin.
17. Çok işlemcili mimariler bağlamında yük dengelemeyi ve ölçeklendirmeyi tanımlayın.
18. Başlatma sürecini özetleyin.
19. Başlatma süreci neden gereklidir?
20. Eğer bir PC'niz varsa, onu açtığınızda gözlemleyebileceğiniz ardışık etkinlikleri kaydedin.Sonra, önyükleme işlemi gerçekten başlamadan önce bilgisayar ekranında hangi mesajların göründüğünü belirleyin.Bu mesajları hangi yazılım yazıyor?
21. Varsayalım ki çoklu programlama işletim sistemi10 milisaniyelik zaman dilimleri ayırıyor ve makineortalama olarak saniyede beş komut yürütüyor.Bir zaman diliminde kaç komut yürütülebilir?
22. Bir dakikada altmış kelime yazan bir daktilocu(ki bir kelime beş karakter olarak kabul edilir)her bir karakteri yazarken ne kadar zaman geçer?Eğer çoklu programlama işletim sistemi zaman dilimlerini 10 milisaniye birimlerinde ayırıyorsa ve işlem geçişleri için gereken zamanı göz ardı edersek, yazılan karakterler arasında kaç zaman dilimi ayrılabilir?
23. Varsayalım ki çoklu programlama işletim sistemi 50 milisaniyelik zaman dilimleri ayırıyor. Eğer bir diskin okuma/yazma kafasını istenen şeridin üzerine yerleştirmek 8 milisaniye sürüyorsa ve istenen verinin okuma/yazma kafasının üzerine dönmesi için başka 17 milisaniye gerekiyorsa, bir programın zaman diliminden ne kadar süre okuma işlemi için beklemek üzere harcanabilir? Eğer makine her nanosekonda on komut yürütme kapasitesine sahipse, bu bekleme süresi boyunca kaç komut yürütülebilir?

Doclingo https://www.doclingo.ai

(Bu nedenle, bir işlem bir çevresel cihazla bir işlem gerçekleştirdiğinde, çoklu p rogramlamasistemi o işlemin zaman dilimini sonlandırırve ilk işlemin çevresel cihazın hizmetlerinibeklerken başka bir işlemin çalı şmasına izin verir.)

24. Çoklu görevli bir işletim sisteminin erişimi koor dine etmesi gereken beş kaynağı listeleyin.

25. Bir sürecin I/O-bağımlı olduğu söylenir eğer çok sayıda I/O işlemi gerektiriyorsa, oysa CPU/bellek sisteminde çoğunlukla hesaplamalardan olu şan bir süreç hesaplama-bağımlı olarak adlan dırılır. Hem bir hesaplama-bağımlı süreç hem d e bir I/O-bağımlı süreç bir zaman dilimi bekliy orsa, hangisine öncelik verilmelidir? Neden?

26. İki işlemi çoklu programlama ortamında ç alıştıran bir sistemin daha yüksek bir veri m elde edip edemeyeceği, eğer her iki işlem de G/Ç bağlıysa (Problem 25'e bakın) veya biri G/Ç bağlı ve diğeri hesaplama bağlıysa mıdır? Neden?

27. Bir işletim sisteminin dağıtımcısına ne yapması g erektiğini söyleyen bir dizi talimat yazın. işlem süresi dilimi sona erdi.

28. Bir sürecin durumunda hangi bilgiler bulunm aktadır?

29. Çoklu programlama sisteminde, bir işlemin ke ndisine tahsis edilen zaman diliminin tamamını t üketmediği bir durumu tanımlayın.

30. Bir sürecin kesildiğinde meydana gelen ö nemli olayları kronolojik sırayla listeleyin.

31. Aşağıdakilerin her birine, kullandığınız bir işletim sistemi açısından cevap verin:

a. İşletim sistemine bir dosyayı bir yerden ba şka bir yere kopyalamasını nasıl istersiniz?
b. İşletim sistemine bir diskteki dizini göstermesini nasıl istersiniz?
c. İşletim sistemine bir programıçalıştırma sını nasıl istersiniz?

32. Aşağıdakilerin her birine, kullandığınız bir işletim sistemi açısından cevap verin:

a. İşletim sistemi, yalnızca onaylı kullanıcıların erişimini nasıl kısıtlar?
b. İşletim sistemine şu anda işlem tablos undahangi süreçlerin bulunduğunu nas ıl sorarsınız?
c. İşletim sistemine, makinenin diğer kullanıcılarının belgelerinize erişmesini istemediğinizi nasıl belirtirsiniz?

***33.** Birçok makine dilinde bulunan test ve ayarla komutunun önemli bir kullanımını açıklayın. Test ve ayarla işleminin tek bir komut olarak uygulanmasının önemi nedir?

***34.** Sadece 100.000 $ olan bir bankacı, ikimüşteriye 50 .000 $ kredi verir. Daha sonra,both customers re turn with the story that before they can repay their loans they must each borrow another $10,000 to complete the business deals in which their previous loans are involved. Bankacı ,bu çıkmazı başka bir kaynaktan ek fonlar borçlanarak ve bu krediyi (faiz oranında b ir artışla)iki müşteriye aktararak çözer. Bank acı hangi üççıkmaz koşulundan birini kaldır mıştır?

***35.** Yerel üniversitede Model Demiryolu II'yekayı t olmak isteyen öğrencilerin,instrüktörd en izin alması ve bir laboratuvarücreti öde mesi gerekmektedir. İki gereklilik,herhangi bir sırayla ve kampüsün farklınoktaları nda bağımsız olarak yerine getirilir.Kayıt, yirmi öğrenci ile sınırlıdır; bu sınır,izin ver ecek olan instrüktör ve laboratuvarücreti ni ödemesine izin verecek olanfinans ofis i tarafından korunmaktadır. Bukayıt siste mi, on dokuz öğrencininkurs için başarıyla k ayıt yaptırmasına nedenolmuştur, ancak son yer, sadece instrüktördenizin alan bir öğr enci ve sadece ücreti ödeyenbir öğrenci ta rafından talep edilmiştir. Aşağıdakiçözü mlerden her biri, hangi çıkmaz gereklili ğinikaldırmaktadır?

a. Her iki öğrenci de derse kabul edilir.
b. Sınıf mevcudu on dokuz ile sınırlandırıl mıştır, bu nedenleiki öğrenciden hiçbi ri derse kayıt olamaz.
c. Rekabet eden öğrencilerin her ikisi de sınıfa girişi reddedilir ve üçüncü bir öğrenciye yirminci yer verilir.
d. Derse girişin tek gerekliliğininücretin öd enmesi olduğu kararlaştırılmıştır.Böylece ücreti ödeyen öğrenci dersekabul edilir ve diğer öğrenciye giriş reddedilir.

Doclingo https://www.doclingo.ai

*36. Bir bilgisayarın ekranındaki her alan aynı anda yalnızca bir işlem tarafından kullanılabileceğinden (aksi takdirde ekrandaki görüntü okunamaz hale gelir), bu alanlar paylaşılamayan kaynaklardır ve pencere yönetici tarafından tahsis edilir. Pencere yönetici, deadlock'u önlemek için hangi üç şarttan birini ortadan kaldırır?

*37. Bir bilgisayar sistemindeki her paylaşılamayankaynağın seviye 1, seviye 2 veya seviye 3kaynağı olarak sınıfla ndırıldığını varsayalım.Ayrıca, sistemdeki her işlem in bu sınıflandırmayagöre ihtiyaç duyduğu kaynakl arı talep etmesigerektiğini varsayalım. Yani, seviye 1 k aynaklarınınhepsini bir seferde talep etmelidir, ardınd anherhangi bir seviye 2 kaynağı talep edebilir.Seviye 1 kaynaklarını aldıktan sonra, gerekliolan tüm seviye 2 kaynaklarını talep edebilir,ve bu şekilde devam ede r. Böyle bir sistemdedeadlock oluşabilir mi? Neden v eya nedenolmasın?

*38. İki robot kolunun her biri, bir konveyör bantta n montajları alacak, toleranslarını test edecek v e test sonuçlarına bağlı olarak bunları iki kutu dan birine yerleştirecek şekilde programlanmıştı r. Montajlar, aralarında yeterli bir zaman aralığı i le birer birer gelir. Her iki kolun da aynı mo ntajı almaya çalışmasını önlemek için, kolları ko ntrol eden bilgisayarlar ortak bir bellek hücresini paylaşır. Eğer bir kol, bir montaj yaklaşırken mevcutsa, kontrol bilgisayarı ortak hücrenin de ğerini okur. Eğer değer sıfırdan farklıysa, kol montajın geçmesine izin verir. Aksi takdirde, kontrol bilgisayarı bellek hücresine sıfırdan farkl ı bir değer koyar, kolu montajı almak üzere y önlendirir ve işlem tamamlandıktan sonra bel lek hücresine 0 değerini geri koyar. İki kol ar asında bir çekişme yaşanmasına neden olabile cek olaylar dizisi nedir?

*39. Bir kuyruğun, bir yazıcıya çıktı dökme sürecin deki kullanımını tanımlayın.

*40. Bir zaman dilimini bekleyen bir süreç, asla bir zaman dil imi verilmediğinde açlık çekiyor olarak kabul edilir.

a. Bir kavşaktaki kaldırım, araçların kavşağa yaklaşırken rekabet ettiği paylaşılmaya n bir kaynak olarak düşünülebilir. Kayn ağın tahsisini kontrol etmek için bir trafi k ışığı, bir işletim sistemi yerine kullanılır. Eğer ışık, gelen trafik miktarını algılayabi liyorsa her yönden gelir ve daha ağır trafiğe ye şil ışık vermek için programlanmıştır, d aha hafif trafik açlık çekebilir. Açlık nasıl önlenir?

b. Dispatcher her zaman her sürecin önceliğ inin sabit kaldığı bir öncelik sistemine gö re zaman dilimleri atıyorsa, bir süreç açlı k çekebilir mi? (*İpucu:* Zaman dilimini y eni tamamlayan sürecin önceliği, bek leyen süreçlerle karşılaştırıldığında ne dir ve dolayısıyla hangi rutin bir so nraki zaman dilimini alır?) Birçok işleti m sisteminin bu sorunu nasıl önledi ğini tahmin edebilir misiniz?

*41. Deadlock ile açlık arasındaki benzerlik nedi r? (Problem 40'a bakın.) Deadlock ile açlık a rasındaki fark nedir?

*42. Aşağıda, E. W. Dijkstra tarafından ilk olarak önerilen v e şimdi bilgisayar bilimi folklorünün bir parçası olan "y emek yiyen filozoflar" problemi bulunmaktadır.

Beş filozof yuvarlak bir masada oturuyor. Her birinin önünde bir tabak spagetti var. Masada , her tabak arasında bir tane olmak üzere beş çata l var. Her filozof düşünmek ve yemek yemek ar asında geçiş yapmak istiyor. Yemek yemek için, bir fi lozofun, filozofun tabağının yanındaki her iki çatalı da elinde bulundurması gerekiyor.

Ölü kilitlenme ve açlık olasılıklarını belirleyi n (bkz. Problem 40) yemek felsefecileri probl eminde mevcut olan.

*43. Çoklu programlama sisteminde zaman d ilimlerinin uzunlukları giderek kısaldıkça han gi problem ortaya çıkar? Uzunlukları gi derek uzadıkça ne olur?

*44. Bilgisayar bilimi geliştikçe, makine dilleri özel komutlar sağlamak için genişletilmiştir. Bölüm 3.4'te tanıtılan üç makine komutu, iş letim sistemleri tarafından yaygın olarak k ullanılmaktadır. Bu komutlar nelerdir?

45. Bir işletim sisteminin yöneticisi tarafından gerçekleştirilebilec ek ancak tipik bir kullanıcı tarafından gerçekleştirilemeyecek iki e tkinliği tanımlayın.

46. Bir işletim sistemi, bir sürecin başka bir sür ecin bellek alanına erişmesini nasıl engeller?

47. Diyelim ki bir şifre, İngiliz alfabesinden oluş an dokuz karakterlik bir dizeden oluşuyor.

Doclingo https://www.doclingo.ai

(yirmi altı karakter). Her olası şifre bir milisaniyede test edilebiliyorsa, tüm olası şifreleri test etmek ne kadar sürer?

48. Çoklu görevli işletim sistemleri için tasarlanmış CPU'lar neden farklı ayrıcalık seviyelerinde çalışabilme yeteneğine sahiptir?
49. Ayrıcalıklı komutlar tarafından genellikle talep edilen iki etkinliği tanımlayın.
50. Bir işletim sistemi tarafından engellenmediği takdirde, bir sürecin bir bilgisayar sisteminin güvenliğini tehdit edebileceği üç yolu tanımlayın.
51. Çok çekirdekli bir işletim sistemi nedir?
52. Bir firmware güncellemesi ile bir işletim sistemi güncellemesi arasındaki fark nedir?
53. Pencere yöneticisi işletim sistemi ile nasıl ilişkilidir?
54. Internet Explorer, Microsoft'un Windows işletim sisteminin bir parçası mıdır?
55. Gömülü bir işletim sisteminin ele alabileceği özel sorunlar nelerdir?

Sosyal Sorunlar

Aşağıdaki sorular, bilgisayar alanıyla ilişkili etik/sosyal/hukuki sorunlara bir rehber olarak tasarlanmıştır.Amaç, bu soruları yalnızca yanıtlamak değildir. Ayrıca, neden böyle yanıt verdiğinizi ve gerekçelerinizin bir sorudan diğerine tutarlı olup olmadığını da düşünmelisiniz.

1. Diyelim ki, diğer kullanıcıların dosyalarının adlarını görüntülemenize ve bu dosyaların koruma altında olmayan içeriklerini görüntülemenize izin veren çoklu kullanıcı işletim sistemi kullanıyorsunuz. Böyle bir bilgiyi izinsiz görüntülemek, birinin kilitsiz evinde izinsiz dolaşmaya benzer mi, yoksa bir doktorun bekleme odası gibi ortak bir salonda bulunan materyalleri okumaya mı daha çok benzer?
2. Çok kullanıcılı bir bilgisayar sistemine erişiminiz olduğunda, şifrenizi seçerken hangi sorumluluklara sahip olursunuz?
3. Bir işletim sisteminin güvenliğindeki bir açığın, kötü niyetli bir programcının hassas verilere yetkisiz erişim sağlamasına izin vermesi durumunda, işletim sisteminin geliştiricisi ne ölçüde sorumlu tutulmalıdır?
4. Evini, davet edilmediği sürece hırsızların giremeyeceği şekilde kilitlemek senin sorumluluğun mu, yoksa halkın senin evine girmemesi mi? Bir işletim sisteminin bir bilgisayara ve içeriğine erişimi koruma sorumluluğu var mı, yoksa hackerların makineye dokunmamaları mı gerekir?
5. Walden'da, Henry David Thoreau, araçlarımızın araçları haline geldiğimizi savunuyor; yani, sahip olduğumuz araçlardan faydalanmak yerine, araçlarımızı elde etmek ve sürdürmek için zaman harcıyoruz. Bu durum bilgisayar kullanımı açısından ne ölçüde doğrudur? Örneğin, bir kişisel bilgisayara sahipseniz, onu ödemek için ne kadar zaman harcıyorsunuz, işletim sistemini kullanmayı öğrenmek için, yardımcı ve uygulama yazılımlarını kullanmayı öğrenmek için, bakımını yapmak için ve yazılımına güncellemeler indirmek için ne kadar zaman harcıyorsunuz? Onu kullandığınızda, zamanınız iyi mi harcanıyor? Kişisel bilgisayarınızla mı yoksa onsuz mu daha sosyal bir şekilde aktif oluyorsunuz?

Doclingo https://www.doclingo.ai

Ek Okuma

Bishop, M. *Bilgisayar Güvenliğine Giriş.*Boston, MA: Addison-Wesley, 2005.

Davis, W. S. ve T. M. Rajkumar. *İşletim Sistemleri*: *Sistematik Bir Bakış*, 6. baskı. Boston, MA: Addison-Wesley, 2005.

Deitel, H. M., P. J. Deitel ve D. R. Choffnes. *İşletim Sistemleri*, 3. baskı. Upper Saddle River, NJ: Prentice-Hall, 2005.

Nutt, G. *İşletim Sistemleri: Modern Bir Yaklaşım*, 3. baskı. Boston, MA: Addison-Wesley, 2004.

Rosenoer, J. *Siber Hukuk, İnternet Hukuku*. New York: Springer, 1997.

Silberschatz, A., P. B. Galvin ve G. Gagne. *İşletim Sistemi Kavramları*, 8. baskı, New York: Wiley, 2008.

Stallings, W. *İşletim Sistemleri*, 5. baskı. Upper Saddle River, NJ: Prentice-Hall, 2006.

Tanenbaum, A. S. *Modern İşletim Sistemleri*, 3. baskı. Upper Saddle River, NJ: Prentice-Hall, 2008.

Doclingo https://www.doclingo.ai

Bu sayfa kasıtlı olarak boş bırakılmıştır

Doclingo https://www.doclingo.ai

BÖLÜM

Ağ Oluşturma ve İnternet

4

Bu bölümde, bilgisayarların bilgi ve kaynakları paylaşmak için nasıl birbirine bağlanabileceğini inceleyen ağ oluşturma olarak bilinen bilgisayar bilimi alanını tartışıyoruz. Çalışmamız, ağların inşası ve işletilmesi, ağların uygulamaları ve güvenlik sorunlarını içerecektir. Önemli bir konu, İnternet olarak bilinen belirli bir dünya çapında ağlar ağı olacaktır.

4.1 Ağ Temelleri
Ağ SınıflandırmalarıProtokoller
Ağların Birleştirilmesi
İşlem Yöntemleri
İletişim
Dağıtık Sistemler

4.2 İnternet
İnternet Mimarisi
İnternet Adresleme
İnternet Uygulamaları

4.3 Dünya Çapında Ağ
Ağ Uygulaması
HTML
XML
Müşteri Tarafı ve Sunucu Tarafı Etkinlikler

***4.4İnternet Protokolleri**
İnternet Yazılımına Yönelik Katmanlı Yaklaşı
mTCP/IP Protokol Takımı

4.5 Güvenlik
Saldırı Biçimleri
Koruma ve Tedaviler
Şifreleme
Ağ Güvenliğine Yasal Yaklaşımlar

**Yıldızlar, isteğe bağlı bölümler için önerileri belirtmektedir.*

Doclingo https://www.doclingo.ai

Farklı bilgisayarlar arasında bilgi ve kaynak paylaşma ihtiyacı, makineler arasın da veri aktarımının yapılabilmesi için bilgisayarların bağlı olduğu ağlar olarak a dlandırılan bağlantılı bilgisayar sistemlerine yol açmıştır. Bu ağlarda, bilgisayar kullanıcıları mesaj alışverişinde bulunabilir ve sistem genelinde dağılmış olan y azıcı yetenekleri, yazılım paketleri ve veri depolama tesisleri gibi kaynakları payl aşabilirler. Bu tür uygulamaları desteklemek için gereken temel yazılım, basit y ardımcı program paketlerinden, karmaşık bir ağ genelinde altyapı sağlayan ge nişleyen bir ağ yazılımı sistemine dönüşmüştür. Bir anlamda, ağ yazılımı, ağ ge nelinde bir işletim sistemine evrim geçiriyor. Bu bölümde, bilgisayar biliminin b u genişleyen alanını keşfedeceğiz.

4.1Ağ Temelleri

Ağları incelemeye, çeşitli temel ağ kavramlarını tanıtarak başlıyoruz.

Ağ Sınıflandırmaları

Bir bilgisayar ağı genellikle ya bir yerel alan ağı (LAN), ya bir metropol alan a ğı (MAN) ya da bir geniş alan ağı (WAN) olarak sınıflandırılır. Bir LAN genellikl e tek bir bina veya bina kompleksi içindeki bilgisayarların bir koleksiyonunda n oluşur. Örneğin, bir üniversite kampusundaki bilgisayarlar veya bir üretim tesisindeki bilgisayarlar bir LAN ile bağlanabilir. Bir MAN, yerel bir topluluğu kapsayan orta boyutlu bir ağdır. Bir WAN, makineleri daha büyük bir mesafe de—belki komşu şehirlerde veya dünyanın zıt taraflarında—bağlar.

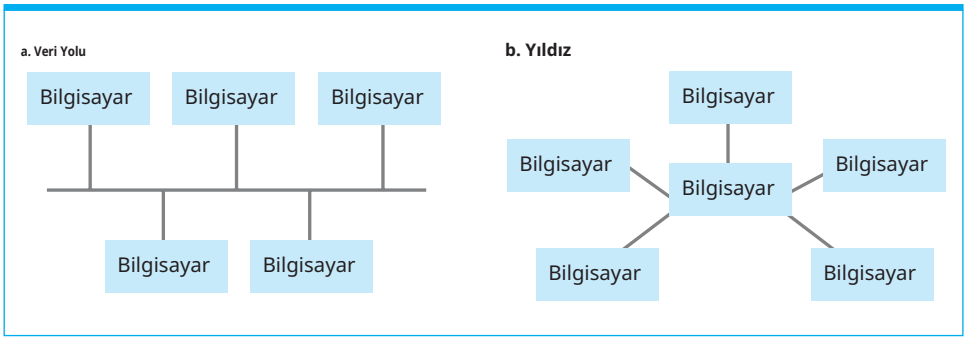
Ağları sınıflandırmanın bir diğer yolu, ağın iç işleyişinin kamuya açık tasa rımlara mı yoksa belirli bir varlık, örneğin bir birey veya bir şirket tarafından sahip olunan ve kontrol edilen yeniliklere mi dayandığına dayanmaktadır. İlk türdeki bir ağa açık ağ denir; ikinci türdeki bir ağa ise kapalı veya bazen özel ağ denir. Açık ağ tasarımları serbestçe dolaşır ve genellikle, lisans ücretleri v e sözleşme koşullarıyla kısıtlanan özel yaklaşımlara karşı nihayetinde üstünlü k sağladıkları noktaya kadar popülarite kazanır.

İnternet (bu bölümde inceleyeceğimiz popüler dünya çapında bir ağlar a ğı) açık bir sistemdir. Özellikle, İnternet üzerindeki iletişim, TCP/IP protokol p aketi olarak bilinen açık bir standartlar koleksiyonu tarafından yönetilmekte dir; bu konu Bölüm 4.4'te ele alınmaktadır. Herkes bu standartları ücret öde meden veya lisans sözleşmeleri imzalamadan kullanma özgürlüğüne sahipti r. Buna karşılık, Novell Inc. gibi bir şirket, sahiplik haklarını korumayı seçtiği özel sistemler geliştirebilir ve bu ürünleri satmaktan veya kiralamaktan gelir elde edebilir.

Ağları sınıflandırmanın bir başka yolu, makinelerin bağlandığı deseni ifa de eden ağ topolojisine dayanmaktadır. Daha popüler iki topoloji, makineleri n hepsinin bir veri yolu (Şekil 4.1a) olarak adlandırılan ortak bir iletişim hattın a bağlı olduğu veri yolu ve bir makinenin diğerlerinin bağlandığı merkezi bir odak noktası olarak hizmet ettiği yıldız topolojisidir (Şekil 4.1b). Veri yolu top olojisi, 1990'larda Ethernet olarak bilinen bir dizi standart altında uygulandığ ında popüler hale geldi ve Ethernet ağları, günümüzde kullanılan en popüler ağ sistemlerinden biri olmaya devam etmektedir.

Doclingo https://www.doclingo.ai

Şekil **4.1** İki popüler ağ topolojisi



Yıldız topolojisinin kökleri 1970'lere kadar uzanmaktadır. Birçok kullanıcıya hizmet eden büyük bir merkezi bilgisayar paradigmasından evrimleşmiştir. Bu kullanıcılar tarafından kullanılan basit terminaller küçük bilgisayarlara dönüştüğünde, bir yıldız ağı ortaya çıkmıştır. Bugün, yıldız konfigürasyonu, iletişimin radyo yayını yoluyla gerçekleştirildiği kablosuz ağlarda popülerdir ve merkezi makine, erişim noktası (AP) olarak adlandırılır ve tüm iletişimin koordine edildiği bir odak noktası olarak hizmet eder.

Bir veri yolu ağı ile bir yıldız ağı arasındaki fark, ekipmanın fiziksel düzenlemesiyle her zaman belirgin değildir. Ayrım, ağdaki makinelerin kendilerini ortak bir veri yolu üzerinden doğrudan birbirleriyle mi yoksa dolaylı olarak bir aracı merkezi makine aracılığıyla mı iletişim kuruyor olarak gördüğüdür. Örneğin, bir veri yolu ağı, bilgisayarların kısa bağlantılar üzerinden bağlandığı uzun bir veri yolu olarak görünmeyebilir. Bunun yerine, bireysel makinelere uzun bağlantılarla birlikte çok kısa bir veri yolu olabilir, bu da ağın daha çok bir yıldız gibi görünmesine neden olur. Gerçekten de, bazen bir veri yolu ağı, her bilgisayardan merkezi bir konuma bağlantılar çalıştırarak oluşturulur ve burada bir hub adı verilen bir cihaza bağlanır. Bu hub, çok kısa bir veri yolundan başka bir şey değildir. Yaptığı tek şey, aldığı sinyali (belki biraz amplifikasyon ile) bağlı olduğu tüm makinelere geri iletmektir. Sonuç, bir yıldız ağı gibi görünen ancak bir veri yolu ağı gibi çalışan bir ağdır.

Protokoller

Bir ağın güvenilir bir şekilde çalışabilmesi için, aktivitelerin yürütüldüğü kuralların belirlenmesi önemlidir. Bu tür kurallara protokoller denir. Protokol standartları geliştirip benimseyerek, satıcılar ağ uygulamaları için diğer satıcıların ürünleriyle uyumlu ürünler üretebilirler. Böylece, protokol standartlarının geliştirilmesi, ağ teknolojilerinin geliştirilmesinde vazgeçilmez bir süreçtir.

Protokol kavramına bir giriş olarak, bir ağdaki bilgisayarlar arasında mesaj iletimini koordine etme sorununu ele alalım. Bu iletişimi yöneten kurallar olmadan, tüm bilgisayarlar mesajları aynı anda iletmekte ısrar edebilir veya diğer makinelerin ihtiyaç duyduğunda yardım etmemekte başarısız olabilirler.

Ethernet standartlarına dayanan bir veri yolu ağında, mesaj iletim hakkı Taşıyıcı Duyma, Çoklu Erişim (CSMA/CD) olarak bilinen protokol tarafından kontrol edilir.

Doclingo https://www.doclingo.ai

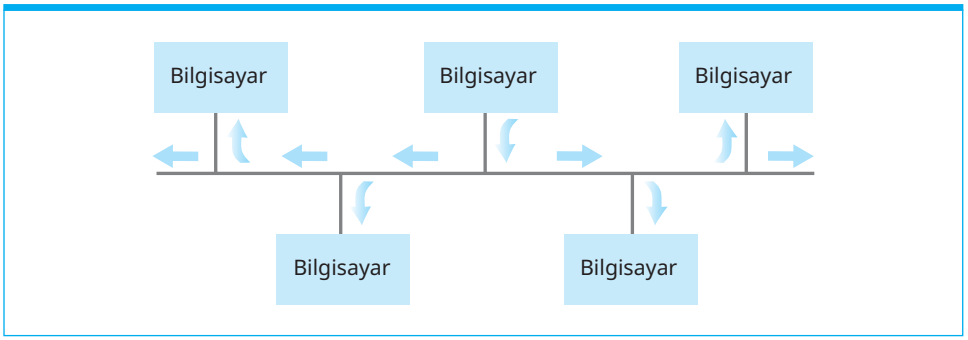
Çarpışma Tespiti ile (CSMA/CD). Bu protokol, her mesajın veri yolundaki tüm m akinelerle yayınlanmasını zorunlu kılar (Şekil 4.2). Her makine tüm mesajları izl er ancak yalnızca kendisine adreslenmiş olanları saklar. Bir mesaj iletmek için, bir makine veri yolu sessiz olana kadar bekler ve bu sırada iletmeye başlar, aynı zamanda veri yolunu izlemeye devam eder. Eğer başka bir makine de iletmeye başlarsa, her iki makine de çarpışmayı tespit eder ve tekrar iletmeye çalışmada n önce kısa, bağımsız rastgele bir süre duraklar. Sonuç, bir grup insanın bir kon uşma sırasında kullandığı sisteme benzer. Eğer iki kişi aynı anda konuşmaya ba şlarsa, ikisi de durur. Fark, insanların "Üzgünüm, ne demek istiyordun?" veya "H ayır, hayır. Önce sen konuş" gibi bir dizi geçirebileceğidir; oysa CSMA/CD proto kolü altında her makine sadece daha sonra tekrar denemeye çalışır.

CSMA/CD'nin, tüm makinelerin merkezi bir AP aracılığıyla iletişim kurduğu ka blosuz yıldız ağlarıyla uyumlu olmadığını unutmayın. Bunun nedeni, bir makineni n iletimlerinin başka bir makinenin iletimleriyle çakıştığını tespit edememesidir. Ö rneğin, makine diğerini duymayabilir çünkü kendi sinyali diğer makinenin sinyalin i boğmaktadır. Diğer bir neden, farklı makinelerden gelen sinyallerin, merkezi AP i le iletişim kurabilseler bile, nesneler veya mesafe tarafından birbirlerinden engell enmesidir (bu duruma gizli terminal problemi denir, Şekil 4.3). Sonuç olarak, kabl osuz ağlar çakışmaları *önlemeye* çalışmak yerine *tespit etmeye* çalışmayı tercih e den bir politika benimser. Bu tür politikalar, Taşıyıcı Algılama, Çarpışma Önleme il e Çoklu Erişim (CSMA/CA) olarak sınıflandırılır ve bunların birçoğu IEEE tarafından standartlaştırılmıştır (bkz. Bölüm 7'de "Elektrik ve Elektronik Mühendisleri Enstitüs ü" yan çubuğu) IEEE 802.11'de tanımlanan protokoller içinde ve genellikle WiFi ol arak adlandırılır. Çarpışma önleme protokollerinin çarpışmaları önlemek için tasar landığını ve bunları tamamen ortadan kaldırmayabileceğini vurgulamak isteriz.Ça rpışmalar meydana geldiğinde, mesajlar yeniden iletilmelidir.

Çarpışma önleme için en yaygın yaklaşım, iletim fırsatını bekleyen makin elere avantaj vermeye dayanmaktadır.

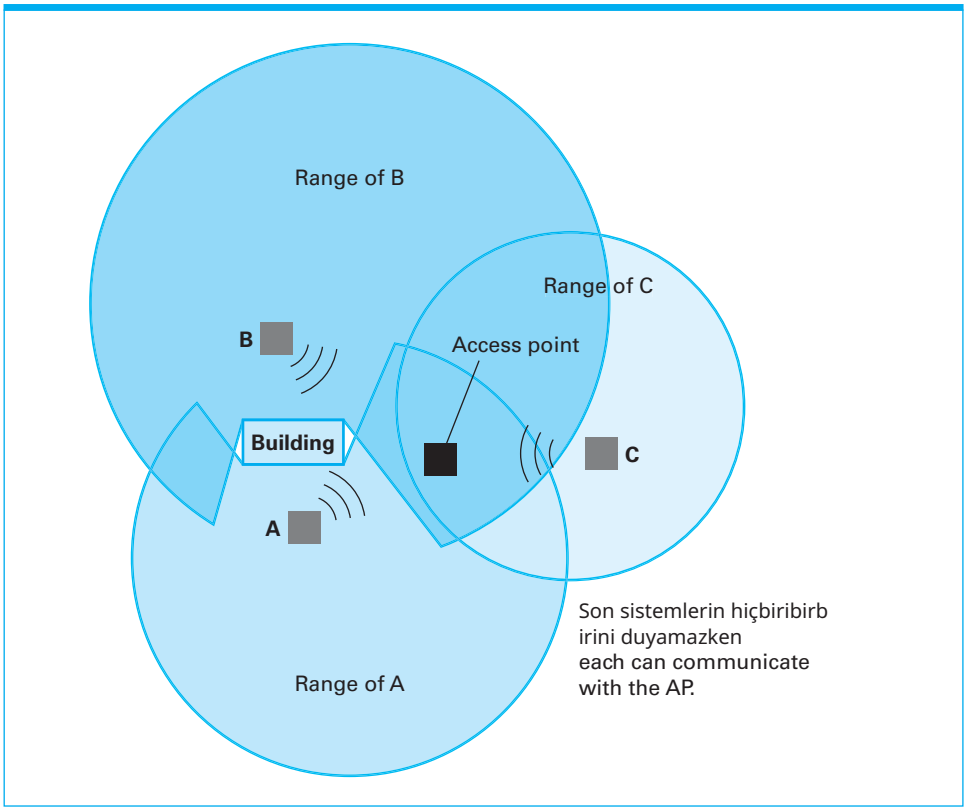
Kullanılan protokol, Ethernet'in CSMA/CD'sine benzer. Temel fark, bir makine nin ilk kez bir mesaj iletmesi gerektiğinde ve iletişim kanalını sessiz bulduğu nda, hemen iletmeye başlamamasıdır. Bunun yerine, kısa bir süre bekler ve yalnızca bu süre boyunca kanal sessiz kalmışsa iletmeye başlar. Bu süreçte meşgul bir kanal yaşanırsa, makine tekrar denemeden önce rastgele belirlen miş bir süre bekler. Bu süre dolduğunda, makinenin tereddüt etmeden sessi z bir kanalı talep etmesine izin verilir. Bu, "yeni gelenler" ile zaten bekleyenle r arasındaki çarpışmaların önlendiği anlamına gelir çünkü bir "yeni gelenin"

Şekil **4.2** Bir veri yolu ağı üzerinden iletişim



Doclingo https://www.doclingo.ai

Şekil **4.3** Gizli terminal problemi



herhangi bir makinenin bekleme fırsatı verilmeden sessiz bir kanalı talep et mesine izin verilmez.

Bu protokol, ancak gizli terminal sorununu çözmez. Sonuçta, sessiz veya meşgul bir kanal arasında ayrım yapmaya dayanan herhangi bir protokol, he r bir istasyonun diğerlerini duyabilmesini gerektirir. Bu sorunu çözmek için b azı WiFi ağları, her makinenin AP'ye kısa bir "istek" mesajı göndermesini ve t üm mesajı iletmeden önce AP'nin bu isteği onaylamasını beklemesini gerekti rir. AP, "gizli terminal" ile ilgilendiği için meşgulse, isteği görmezden gelir ve istekte bulunan makine beklemesi gerektiğini anlar. Aksi takdirde, AP isteği onaylar ve makine iletiminin güvenli olduğunu bilir. Ağdaki tüm makinelerin AP'den gönderilen tüm onayları duyacağını ve böylece AP'nin herhangi bir a nda meşgul olup olmadığı hakkında iyi bir fikre sahip olacağını unutmayın, h er ne kadar iletimlerin gerçekleştiğini duyamayabilirler.

Ağları Birleştirme

Bazen mevcut ağları birleştirerek genişletilmiş bir iletişim sistemi oluşturmak ger eklidir. Bu, ağları aynı "tip" ağın daha büyük bir versiyonunu oluşturacak şekilde b irleştirerek yapılabilir. Örneğin, Ethernet protokollerine dayanan veri yolu ağları d urumunda, genellikle veri yollarını birleştirerek tek bir uzun veri yolu oluşturmak mümkündür. Bu, tekrarlayıcılar, köprüler ve anahtarlar olarak bilinen farklı cihazl ar aracılığıyla yapılır; bunların arasındaki farklılıklar ince ama bilgilendiricidir.

Doclingo https://www.doclingo.ai

Ethernet

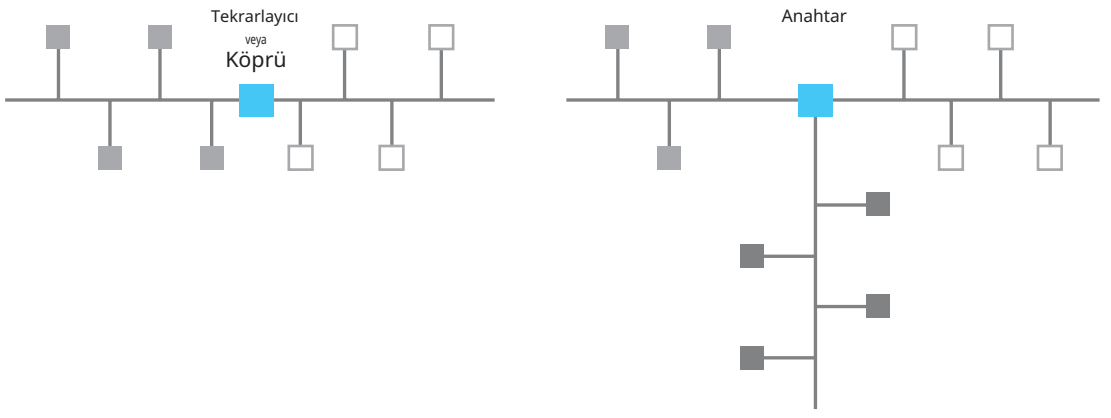
Ethernet, bir veri yolu topolojisi ile LAN uygulamak için bir dizi standarttır. Adı, makin elerin bir koaksiyel kablo olan eter ile bağlandığı orijinal Ethernet tasarımından türetil miştir. İlk olarak 1970'lerde geliştirilmiş ve şimdi IEEE tarafından IEEE 802 standartları ailesinin bir parçası olarak standartlaştırılmıştır, Ethernet, PC'leri ağlamak için en yay gın yöntemlerden biridir. Gerçekten de, Ethernet denetleyicileri, mevcut perakende p azarında bulunan PC'lerde standart bir bileşen haline gelmiştir.

Bugün aslında, teknolojideki ilerlemeleri ve daha yüksek transfer hızları nı yansıtan birkaç Ethernet versiyonu bulunmaktadır. Ancak hepsi, Ethernet ailesini karakterize eden ortak özelliklere sahiptir. Bunlar arasında verilerin iletim için paketlendiği format, bitlerin gerçek iletimi için Manchester kodla ması (bir 0'ın azalan bir sinyal ile ve bir 1'ın artan bir sinyal ile temsil edildiği bir yöntem) kullanımı ve iletim hakkını kontrol etmek için CSMA/CD kullanı mı bulunmaktadır.

Bunların en basiti tekrarlayıcıdır, bu, iki orijinal veri yolu arasında sinyalleri sadece ileriye ve geriye ileten bir cihazdan başka bir şey değildir (genellikle bazı şekillerde amplifikasyon ile) sinyallerin anlamını dikkate almadan (Şekil 4.4a).

Bir köprü, bir tekrarlayıcıya benzer, ancak daha karmaşıktır. Bir tekrarlayı cı gibi, iki veri yolunu bağlar, ancak bağlantı üzerinden tüm mesajları iletmek zorunda değildir. Bunun yerine, her mesajla birlikte gelen hedef adresine ba kar ve yalnızca o mesaj diğer taraftaki bir bilgisayara yönlendirilmişse bağla ntı üzerinden bir mesaj iletir. Böylece, köprünün aynı tarafında bulunan iki m akine, diğer tarafta gerçekleşen iletişimi etkilemeden mesaj alışverişi yapabil ir. Bir köprü, bir tekrarlayıcı tarafından üretilenden daha verimli bir sistem ür etir.

Şekil **4.4** Küçük ağlardan büyük bir veri yolu ağı oluşturma



a. İki veri yolunu bağlayan bir tekrarlayıcı veya köprü

b. Birden fazla veri yolunu bağlayan bir anahtar

Doclingo https://www.doclingo.ai

A anahtar esasen birden fazla bağlantıya sahip bir köprüdür ve bu sayede birden fazla veri yolunu birbirine bağlayabilir, sadece iki değil. Böylece, bir anahtar, anahtardan çıkan tekerlek jantı gibi birkaç veri yolundan oluşan bir ağ üretir (Şekil 4.4b). Bir köprü durumunda olduğu gibi, bir anahtar tüm mesajların hedef adreslerini dikkate alır ve yalnızca diğer jantlara gidecek olan mesajları iletir. Dahası, iletilen her mesaj yalnızca uygun janta yönlendirilir, böylece her janttaki trafiği en aza indirir.

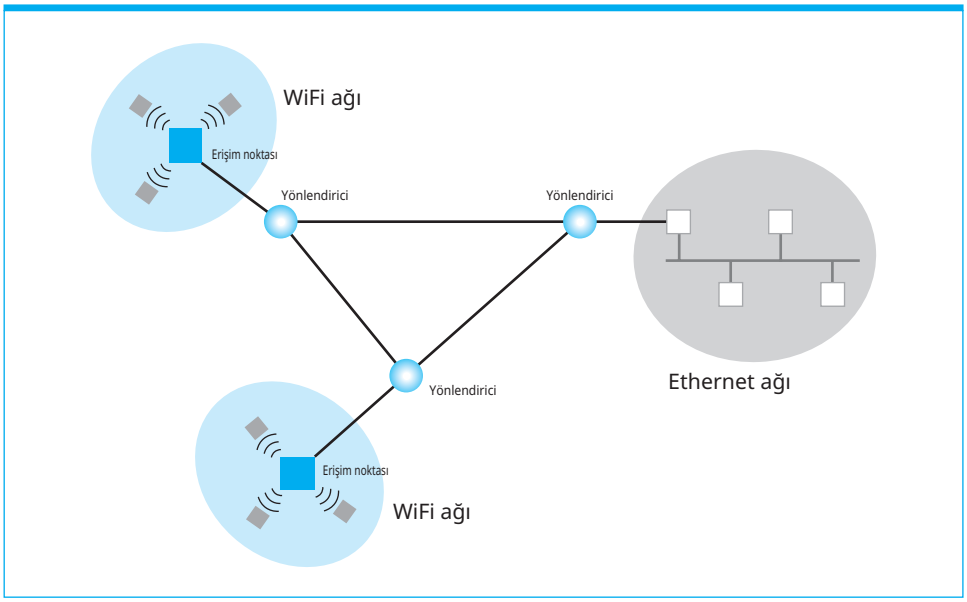
Ağların tekrarlayıcılar, köprüler ve anahtarlar aracılığıyla bağlandığında,s onuç tek bir büyük ağdır. Tüm sistem, orijinal daha küçük ağların her biriylea ynı şekilde (aynı protokolleri kullanarak) çalışır.

Bazen, bağlanacak ağların uyumsuz özellikleri olabilir. Örneğin, bir WiFi ağınınözellikleri, bir Ethernet ağıyla kolayca uyumlu değildir. Bu durumlarda, ağlar,özgünlüklerini koruyarak ve otonom ağlar olarak işlev görmeye deva m ederek,'network of networks' (ağlar ağı) olarak bilinen bir şekilde bağlanm alıdır.(Dikkat edin ki, genel terim *internet*, the Internet teriminden farklıdır.B üyük harfle yazılan Internet, bu bölümde daha sonra inceleyeceğimizbelirli, dünya çapında bir interneti ifade eder.)

Birçok başka internet örneği vardır. Gerçekten de, geleneksel telefon iletişimi, İ nternet popülerleşmeden çok önce dünya çapında internet sistemleri tarafında n yönetiliyordu.)

Ağlar arasında bir internet oluşturmak için bağlantı, mesajları iletmek için kull anılan özel amaçlı bilgisayarlar olan yönlendiriciler (router) olarak bilinen cihazlar tarafından sağlanır. Bir yönlendiricinin görevinin, tekrar ediciler, köprüler ve anah tarların görevinden farklı olduğunu unutmayın; çünkü yönlendiriciler, her ağın ke ndine özgü iç özelliklerini korurken ağlar arasında bağlantılar sağlar. Örneğin, Şe kil 4.5, iki WiFi yıldız ağını ve bir Ethernet veri yolunu göstermektedir.

Şekil 4.5İki WiFi ağını ve bir Ethernet ağını bir internet oluşturmak için bağlayan yönle ndiriciler



Doclingo https://www.doclingo.ai

Yönlendiricilerle bağlı ağ. WiFi ağlarından birindeki bir makine, Ethernet ağın daki bir makineye mesaj göndermek istediğinde, önce mesajı kendi ağındaki AP'ye gönderir. Oradan, AP mesajı ilişkili yönlendiricisine gönderir ve bu yön lendirici mesajı Ethernet'teki yönlendiriciye iletir. Orada mesaj, veri yolundak i bir makineye verilir ve o makine mesajı Ethernet'teki nihai hedefine iletir.

Router'ların bu şekilde adlandırılmasının nedeni, amaçlarının mesajları d oğru yönlere iletmek olmasıdır. Bu iletim süreci, bir internet genelinde adres leme sistemine dayanır; bu sistemde bir internet içindeki tüm cihazlar (orijin al ağlardaki makineler ve router'lar dahil) benzersiz adresler atanmıştır. (Bu nedenle, orijinal ağlardan birindeki her makinenin iki adresi vardır: kendi ağındaki orijinal "yerel" adresi ve internet adresi.) Uzak bir ağdaki bir makin eye mesaj göndermek isteyen bir makine, mesajın üzerine hedefin internet a dresini ekler ve mesajı yerel router'ına yönlendirir. Oradan, mesaj doğru yön e iletilir. Bu iletim amacı için, her router, mesajların hedef adreslerine bağlı o larak hangi yöne gönderilmesi gerektiği hakkında router'ın bilgilerini içeren bir iletim tablosu tutar.

Bir ağın internete bağlandığı "nokta" genellikle bir geçit olarak adlandırıl ır çünkü bu, ağ ile dış dünya arasında bir geçit işlevi görür. Geçitler çeşitli biçi mlerde bulunabilir ve bu nedenle terim oldukça gevşek bir şekilde kullanılır. Birçok durumda, bir ağın geçidi, internetin geri kalanıyla iletişim kurduğu ro uter'dır. Diğer durumlarda, geçit terimi yalnızca bir router'dan daha fazlasın ı ifade etmek için kullanılabilir. Örneğin, internete bağlı çoğu konut WiFi ağın da, geçit terimi, genellikle bir bütün olarak ağın AP'si ve AP'ye bağlı router'ı i fade eder çünkü bu iki cihaz genellikle tek bir birim içinde paketlenmiştir.

İşlem İletişim Yöntemleri

Ağ içindeki farklı bilgisayarlarda (veya zaman paylaşımı/multitasking yoluyla aynı makinede) çalışan çeşitli aktivitelerin (veya süreçlerin) genellikle birbirle riyle iletişim kurması gerekir; bu, eylemlerini koordine etmek ve belirlenen g örevlerini yerine getirmek içindir. Süreçler arasındaki bu iletişime işlem arası iletişim denir.

İşlem arası iletişim için kullanılan popüler bir kavram, istemci/sunucu mo delidir. Bu model, süreçlerin ya diğer süreçlerden isteklerde bulunan bir iste mci ya da istemcilerin yaptığı istekleri karşılayan bir sunucu olarak oynadığı t emel rolleri tanımlar.

İstemci/sunucu modelinin erken bir uygulaması, ofisler kümesindeki tü m bilgisayarları bağlayan ağlarda ortaya çıktı. Bu durumda, tek bir yüksek ka liteli yazıcı ağa bağlandı ve ağdaki tüm makineler için kullanılabilir hale geldi. Bu durumda yazıcı, bir sunucu rolü oynadı (genellikle bir yazıcı sunucusu ola rak adlandırılır) ve diğer makineler, yazıcı sunucusuna yazdırma istekleri gön deren istemciler olarak programlandı.

İstemci/sunucu modelinin bir başka erken uygulaması, manyetik disk de polama maliyetini azaltmak ve kayıtların kopyalarının gereksiz olmasını sağla mak için kullanıldı. Burada, bir ağdaki bir makine, bir organizasyonun tüm ka yıtlarını içeren yüksek kapasiteli bir kütle depolama sistemi (genellikle bir ma nyetik disk) ile donatıldı. Ağdaki diğer makineler, ihtiyaç duydukça kayıtlara e rişim talep etti. Böylece, kayıtları gerçekten içeren makine

Doclingo https://www.doclingo.ai

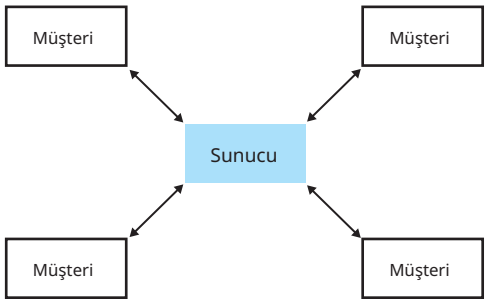
bir sunucunun (bir dosya sunucusu olarak adlandırıldığı) rolü ve diğer makinelerin dosya s unucusunda saklanan dosyalara erişim talep eden istemciler olarak rol oynadığıdır.

Bugün istemci/sunucu modeli ağ uygulamalarında yaygın olarak kullanıl maktadır,bu bölümde daha sonra göreceğimiz gibi. Ancak, istemci/sunucu modelitek başına süreçler arası iletişim için bir yol değildir. Diğer bir model eşler arası(sıklıkla **P2P** olarak kısaltılır) modelidir. İstemci/sunucu modeli,bir sürecin (sunucu) birçok diğerine (istemciler) hizmet sağladığı bir durumu içe rirken,eşler arası model, birbirine hizmet sağlayan ve hizmet alan süreçleri iç erir(Şekil 4.6). Dahası, bir sunucunun, istemcilerine her an hizmet vermeye h azır olması için sürekli olarak çalışması gerekirken,eşler arası model genellikl e geçici olarak çalışan süreçleri içerir. Örneğin,eşler arası modelin uygulamal arı, insanların İnternet üzerinden yazılı bir konuşma yaptığı anlık mesajlaşm ayı veinsanların rekabetçi etkileşimli oyunlar oynadığı durumları içerir.

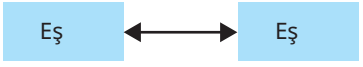
Eşler arası model, müzik kayıtları ve sinema filmleri gibi dosyaların İnter net üzerinden dağıtımında da popüler bir yöntemdir. Bu durumda, bir eşbaş ka birinden bir dosya alabilir ve ardından o dosyayı diğer eşlere sağlayabilir. Bu tür bir dağıtıma katılan eşlerin topluluğuna bazenbir sürü denir. Dosya d ağıtımında sürü yaklaşımı, istemci/sunucu modelini uygulayanmerkezi bir da ğıtım merkezi (sunucu) kurarak istemcilerin dosyaları indirdiği (veya en azınd anbu dosyalar için kaynak bulduğu) önceki yaklaşımlara zıttır.

Bir neden, P2P modelinin dosya paylaşımı için istemci/sunucu modelinin ye rini almasıdır; çünkü hizmet görevini birçok eş üzerinde dağıtır, tek bir sunucud a yoğunlaştırmak yerine. Bu merkezi bir operasyon tabanının olmaması, daha v erimli bir sisteme yol açar. Ne yazık ki, dosya popülaritesinin bir diğer nedeni

Şekil **4.6**İstemci/sunucu modeli ile eşler arası modelin karşılaştırılması



a. Sunucu, her zaman birden fazla istemciye hizmet vermeye hazır olmalıdır.



b. Eşitler olarak bire bir iletişim kurarlar.

Doclingo https://www.doclingo.ai

P2P modeline dayanan dağıtım sistemlerinin, hukuki belirsizlik durumlarında merkezi bir sunucunun olmaması nedeniyle telif hakkı yasalarını uygulamak için hukuki çabaları daha zor hale getirdiği durumlar vardır. Ancak, "zor"un "imkansız" anlamına gelmediğini keşfeden bireylerin, telif hakkı ihlali nedeniyle önemli yükümlülüklerle karşılaştıkları birçok durum vardır.

Sıklıkla *eşler arası ağ* terimini okuyabilir veya duyabilirsiniz; bu, teknik terimlerin teknik olmayan topluluk tarafından benimsenmesiyle nasıl yanlış kullanımın evrilebileceğine dair bir örnektir. *Eşler arası* terimi, iki sürecin bir ağ (veya internet) üzerinden iletişim kurduğu bir sistemi ifade eder. Bu, ağın (veya internetin) bir özelliği değildir. Bir süreç, başka bir süreçle iletişim kurmak için eşler arası modeli kullanabilir ve daha sonra aynı ağ üzerinden başka bir süreçle iletişim kurmak için istemci/sunucu modelini kullanabilir. Bu nedenle, eşler arası model aracılığıyla iletişim kurmaktan bahsetmek, eşler arası bir ağ üzerinden iletişim kurmaktan daha doğru olur.

Dağıtık Sistemler

Ağ teknolojisinin başarısıyla birlikte, bilgisayarlar arasındaki etkileşimağlar üzerinden yaygın ve çok yönlü hale gelmiştir. Küresel bilgialma sistemleri, şirket genelindeki muhasebe ve envanter sistemleri,bilgisayar oyunları ve hatta bir ağın altyapısını kontrol eden yazılımlar gibi birçok modern yazılım sistemi, farklı bilgisayarlarda süreçler olarakçalışan yazılım birimlerinden oluşan dağıtık sistemler olarak tasarlanmıştır.

Erken dağıtık sistemler sıfırdan bağımsız olarak geliştirilmiştir. Ancak bugün, bu sistemler arasında iletişim ve güvenlik sistemleri gibi ortak bir altyapının çalıştığını ortaya koyan araştırmalar yapılmaktadır. Bununla birlikte, bu temel altyapıyı sağlayan ve dolayısıyla dağıtık uygulamaların uygulamaya özgü olan kısmını geliştirerek inşa edilmesini sağlayan prefabrik sistemler üretmek için çabalar sarf edilmiştir.

Şu anda birkaç tür dağıtık hesaplama sistemi yaygındır. Kümehesaplama, birçok bağımsız bilgisayarın bir arada çalışarak çok daha büyükbir makineye benzer hesaplama veya hizmetler sağladığı bir dağıtık sistemitanımlar. Bu bireysel makinelerin maliyeti, bunları bağlamak için gerekenhızlı ağ ile birlikte, daha yüksek fiyatlı bir süper bilgisayardan daha azolabilir, ancak daha yüksek güvenilirlik ve daha düşük bakım maliyetleriyle.Bu tür dağıtık sistemler, en az bir küme üyesinin bir isteği yanıtlayabilmeolasılığının daha yüksek olması nedeniyle yüksek kullanılabilirlik sağlamakve yük dengelemesi yapmak için kullanılır; çünkü iş yükü, çok fazla işiolan küme üyelerinden, daha az işi olanlara otomatik olarak kaydırılabilir.Grid hesaplama, kümelerden daha gevşek bir şekilde bağlı olan ancak yine debüyük görevleri başarmak için birlikte çalışan dağıtık sistemleri ifade eder.Grid hesaplama, bir gridde yer alan makinelere veri ve algoritmaları dağıtmayıkolaylaştırmak için özel yazılımlar içerebilir.Örnekler arasında Wisconsin Üniversitesi'nin Condor sistemi veyaBerkeley'nin Ağ Hesaplama için Açık Altyapısı (BOINC) bulunmaktadır. Busistemler genellikle iş yerinde veya evde başka amaçlar için kullanılanbilgisayarlara kurulur ve bu bilgisayarlar kullanılmadığında grid'ehesaplama gücü gönüllü olarak sağlayabilir. İnternetin artan bağlantıolanakları sayesinde, bu tür gönüllü, dağıtık grid hesaplama, milyonlarcaev bilgisayarının çalışmasını sağlamıştır.

Doclingo https://www.doclingo.ai

son derece karmaşık matematiksel ve bilimsel problemler üzerinde. Bulut bilişim,paylaşıla n devasa bilgisayar havuzlarının ağda ihtiyaç duyulduğundamüşteriler tarafından kullanılm ak üzere tahsis edilebildiği en son trend,dağıtık sistemlerde. Tıpkı yirminci yüzyılın başların daki büyükşehir elektrik şebekelerinin,bireysel fabrikaların ve işletmelerin kendi jeneratörl erini sürdürme ihtiyacını ortadan kaldırması gibi,İnternet, varlıkların verilerini ve hesaplam alarını "Bulut"a emanet etmelerini sağlıyor,bu durumda ağda zaten mevcut olan devasa he saplama kaynaklarını ifade ediyor. Amazon'un Elastic Compute Cloud gibi hizmetler,müşter ilerin sanal bilgisayarları saatlik olarak kiralamasına olanak tanır,bilgisayar donanımının ge rçekten nerede bulunduğu konusunda endişelenmeden. Google Docs ve Google Apps, kull anıcıların bilgileri üzerinde işbirliği yapmalarına veya Web hizmetleri oluşturmalarına olana k tanır,problemin üzerinde kaç bilgisayarın çalıştığını veya ilgili verilerin nerede saklandığın ı bilmeye gerek kalmadan.Bulutbilişim hizmetleri, güvenilirlik ve ölçeklenebilirlik konusund a makul garantiler sağlasa da,bizim kullandığımız bilgisayarların kimin sahip olduğu ve işle ttiği konusunda artık bilmediğimiz bir dünyada,mahremiyet ve güvenlik endişelerini de gü ndeme getiriyor.

Sorular & Alıştırmalar

1. Açık bir ağ nedir?
2. Bir köprü ile bir anahtar arasındaki farkı özetleyin.
3. Bir yönlendirici nedir?
4. Müşteri/sunucu modeline uyan toplumdaki bazı ilişkileri tanımlayın.
5. Toplumda kullanılan bazı protokolleri tanımlayın.
6. Küme hesaplama ile grid hesaplama arasındaki farkı özetleyin.

4.2 İnternet

Bir internetin en dikkat çekici örneği, 1960'ların başlarına kadar uzanan araşt ırma projelerinden kaynaklanan İnternet (büyük harf *I* ile dikkat edin)dir.Am aç, çeşitli bilgisayar ağlarını bağlama yeteneğini geliştirmekti, böylece yerel f elaketlerden etkilenmeyen bağlı bir sistem olarak işlev görebilirlerdi.
Bu çalışmaların çoğu, ABD hükümeti tarafından Savunma İleri Araştırma Proj eleri Ajansı (DARPA - "DAR-pa" olarak telaffuz edilir) aracılığıyla desteklenmiş tir. Yıllar içinde, İnternet'in geliştirilmesi, hükümet destekli bir projeden akad emik bir araştırma projesine kaydı ve bugün büyük ölçüde milyonlarca bilgis ayarı içeren dünya çapında LAN, MAN ve WAN kombinasyonlarını bağlayan ti cari bir girişimdir.

İnternet Mimarisi

Daha önce de belirttiğimiz gibi, İnternet bağlı ağların bir koleksiyonudur.

Genel olarak, bu ağlar, İnternet Servis Sağlayıcıları (ISP'ler) olarak adlandırılan kuruluşlar tarafından inşa edilir ve bakım yapılır.

Ayrıca, ağların kendilerine atıfta bulunurken ISP terimini kullanmak da yaygı ndır. Bu nedenle, bir ISP'ye bağlanmaktan bahsederken, aslında bir ISP taraf ından sağlanan ağa bağlanmaktan kastettiğimizi söyleyeceğiz.

Doclingo https://www.doclingo.ai

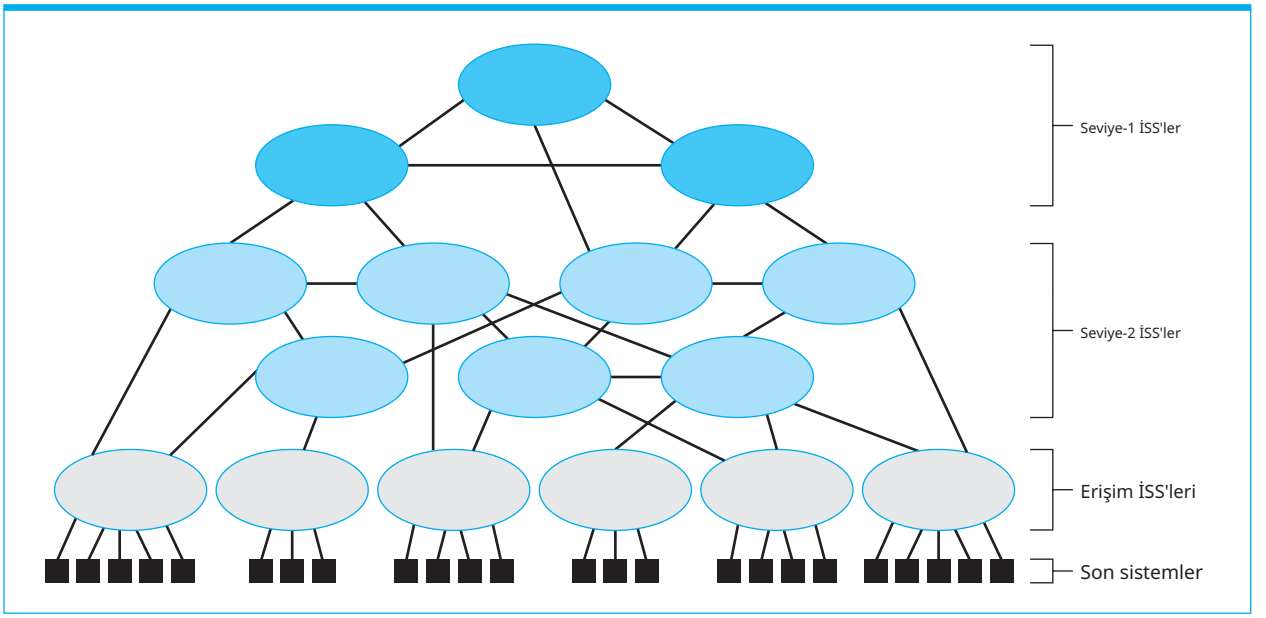
İnternet Servis Sağlayıcıları (ISP'ler) tarafından işletilen ağlar, genel İnternet yapısındaki rolüne göre bir hiyerarşi içinde sınıflandırılabilir (Şekil 4.7). Bu hiyerarşinin en üstünde, çok yüksek hızlı, yüksek kapasiteli, uluslararası WAN'lardan oluşan nispeten az sayıda tier-1 ISP bulunmaktadır. Bu ağlar, İnternet'in omurgası olarak düşünülmektedir. Genellikle iletişim işinde olan büyük şirketler tarafından işletilmektedir. Bir örnek, geleneksel bir telefon şirketi olarak başlayan ve kapsamını diğer iletişim hizmetlerini sağlamaya genişleten bir şirket olabilir.

Tier-1 ISP'lere bağlanan tier-2 ISP'ler, genellikle daha bölgesel bir kapsamda ve yetenekleri açısından daha az güçlüdür. (Tier-1 ve tier-2 ISP'ler arasındaki ayrım genellikle bir görüş meselesidir.) Yine, bu ağlar genellikle iletişim işinde olan şirketler tarafından işletilmektedir.

Tier-1 ve tier-2 ISP'ler, kolektif olarak İnternet'in iletişim altyapısını sağlayan yönlendirici ağlarıdır. Bu nedenle, İnternet'in çekirdeği olarak düşünülebilirler. Bu çekirdeğe erişim genellikle bir erişim ISP'si adı verilen bir aracılık tarafından sağlanır. Bir erişim ISP'si, bireysel kullanıcılara İnternet erişimi sağlama işinde olan tek bir otorite tarafından işletilen bağımsız bir internet, bazen intranet olarak adlandırılır. Örnekler arasında AOL, Microsoft gibi şirketler ve hizmetleri için ücret alan yerel kablo ve telefon şirketleri ile kendi organizasyonlarındaki bireylere İnternet erişimi sağlama görevini üstlenen üniversiteler veya şirketler gibi kuruluşlar bulunmaktadır.

Bireysel kullanıcıların erişim İSS'lerine bağlandığı cihazlar, son sistemler veya ana bilgisayarlar olarak bilinir. Bu son sistemler, geleneksel anlamda bilgisayar olmak zorunda değildir. Telefonlar, video kameralar, otomobiller ve ev aletleri gibi çok çeşitli cihazları kapsamaktadır. Sonuçta, İnternet esasen bir iletişim sistemidir ve bu nedenle diğer cihazlarla iletişim kurmaktan fayda sağlayacak herhangi bir cihaz potansiyel bir son sistemdir.

Şekil **4.7** İnternet bileşimi



Doclingo https://www.doclingo.ai

Son sistemlerin erişim İSS'lerine bağlandığı teknoloji de çeşitlidir. Belki de en hızlı büyüyenler, WiFi teknolojisine dayanan kablosuz bağlantılardır. Strateji, AP'yi bir erişim İSS'sine bağlamak ve böylece AP'nin yayın alanındaki son sistem lere o İSS üzerinden İnternet erişimi sağlamaktır. AP'nin kapsama alanı genellikle bir sıc ak nokta olarak adlandırılır. Sıcak noktalar ve sıcak nokta grupları, bireysel konutlardan, otel ve ofis binalarına, küçük işletmelere, parklara ve bazı durumlarda tüm şehirlere ka dar oldukça yaygın hale gelmektedir. Benzer bir teknoloji, sıcak noktaların hücreler olar ak bilindiği cep telefonu endüstrisi tarafından kullanılmakta ve hücreleri oluşturan "yönl endiriciler" sürekli hizmet sağlamak için koordine edilmektedir, böylece bir son sistem b ir hücreden diğerine geçerken kesintisiz hizmet sunulmaktadır.

Diğer popüler teknikler, erişim İSS'sine bağlanmak için telefon hatları veya kablo/uydu sistemleri kullanmaktadır. Bu teknolojiler, bir son sisteme veya birden fazla son sistemin bağlı olduğu bir müşterinin yönlendiricisine doğrudan bağlantı sağlamak için kullanılabilir. Bu son taktik, mevcut kablo veya telefon hatları üzerinden bir erişim İSS'sine bağlı bir yönlendirici/AP tarafından oluşturulan yerel bir hot spot için bireysel konutlarda giderek daha popüler hale gelmektedir.

Mevcut kablo ve uydu bağlantıları, geleneksel telefon hatlarına göre yüksek hızda veri transferi için doğası gereği daha uyumludur; çünkü bu hatlar ilk olarak ses iletişimi düşünülerek kurulmuştur. Ancak, dijital verilerin iletimini sağlamak için bu ses bağlantılarını uzatmak amacıyla birkaç akıllı şema geliştirilmiştir. Bu, aktarılacak dijital veriyi, kullanılan iletim ortamıyla uyumlu bir forma dönüştüren modemler (modülatör/demodülatör kısaltması) adı verilen cihazları kullanır. Bir örnek, 4 KHz (4,000 kilohertz) altındaki frekans aralığının geleneksel ses iletişimi için ayrıldığı ve daha yüksek frekansların dijital verilerin aktarımında kullanıldığı DSL (dijital abone hattı)'dır. Daha eski bir yaklaşım ise dijital veriyi sese dönüştürmek ve bunu sesle aynı şekilde iletmektir. Bu son uygulama, geçici bağlantılar için kullanıldığı gerçeğine atıfta bulunarak dial-up erişim olarak adlandırılmaktadır; bu durumda kullanıcı, bir erişim İSS'sinin yönlendiricisine telefonla geleneksel bir arama yapar ve ardından telefonunu kullanılacak son sisteme bağlar.

Uygun fiyatlı ve yaygın olarak bulunmasına rağmen, dial-up'ın nispeten yavaş veri transferi hızı, giderek artan bir şekilde, gerçek zamanlı video iletişimi ve büyük veri bloklarının iletimi gibi günümüz İnternet uygulamalarını karşılamakta yetersiz kalmaktadır. Bu nedenle, giderek daha fazla ev ve küçük işletme, erişimİSS'lerine, kablo te levizyon bağlantıları, özel telefon veri hatları, uydu antenleri ve hatta fiber o ptik kablolar gibi geniş bant teknolojileri aracılığıyla bağlanmaktadır.

İnternet2

İnternet, bir araştırma projesinden bir ev eşyası haline geldiğinden beri,araştırma topluluğu, yalnızca akademik bir sistem olarak tasarlanan Internet2adlı bir projeye geçmiştir. Internet2, sanayi ve hükümetle işbirliği içinde çalışançok sayıda üniversi teyi içermektedir. Amaç, uzaktan erişim ve kontrol gibi yüksek bant genişliği iletişi mi gerektiren İnternet uygulamalarında araştırma yapmaktır.Maliyetli, son teknolo ji ekipmanlar, teleskoplar ve tıbbi tanı cihazları gibi. Mevcut araştırmalara bir örne k,uzaktan cerrahi uygulamasıdır; bu, uzaktaki bir cerrahın hastayı video aracılığıyla izleyerek, robotik ellerle gerçekleştirdiği bir işlemdir. Internet2 hakkında daha fazl a bilgiyehttp://www.internet2.org adresinden ulaşabilirsiniz.

Doclingo https://www.doclingo.ai

İnternet Adresleme

4.1. Bölümde öğrendiğimiz gibi, bir internetin, sistemdeki her bilgisayara benzersiz bir t anımlayıcı adres atayan internet genelinde bir adresleme sistemine ihtiyacı vardır. İnternet'te bu adresler IP adresleri olarak bilinir. *(IP* terimi, "İnternet Protok olü"nü ifade eder, bu terimi 4.4. Bölümde daha fazla öğreneceğiz.)Başlangıçt a, her IP adresi 32 bitlik bir desendi, ancak daha büyük bir adres seti sağlam ak için 128 bitlik adreslere dönüştürme süreci şu anda devam etmektedir(IPv 6'nın 4.4. Bölümdeki tartışmasına bakın). Ardışık numaralandırılmış IP adresi blokları, İnternet'in işletimini koordine etmek için kurulmuş kar amacı gütme yen bir kuruluş olan İnternet Atanmış İsimler ve Numaralar Kurumu (ICANN) tarafından İSS'lere verilmektedir. İSS'ler daha sonra, kendilerine tahsis edil en bloklar içindeki adresleri yetki alanlarındaki makinelere tahsis etme iznin e sahiptir. Böylece, İnternet genelindeki makineler benzersiz IP adresleri alır.

IP adresleri geleneksel olarak noktalarla ayrılmış ondalık notasyon şeklinde y azılır; bu notasyonda adresin baytları noktalarla ayrılır ve her bayt, geleneksel onl uk notasyonda temsil edilen bir tam sayı olarak ifade edilir. Örneğin, noktalarla ay rılmış ondalık notasyon kullanarak, 5.2 deseni iki baytlık bit desenini temsil eder. 0000010100000010, 5 olarak temsil edilen 00000101 baytınıve 2 olarak temsi l edilen 00000010 baytını içermektedir ve 17.12.25 deseni,bayt 00010001 (ikil i notasyonda 17 olarak yazılır) ile başlayan,bayt 00001100 (ikili olarak 12 yazıl ır) ile devam eden,bayt 00011001 (ikili olarak 25 yazılır) ile biten üç baytlık bit desenini temsil eder.Özetle, 32 bitlik bir IP adresi, noktalı ondalık notasyond a 192.207.177.133 olarakgörünebilir.

Bit-deseni biçimindeki adresler (noktalı ondalık notasyon kullanılarak sıkıştırılsa bile) insan tüketimi için nadiren uygundur. Bu nedenle İnternet, makinelerin mnemonik adlarla tanımlandığı alternatif bir adresleme sistemine sahiptir. Bu adresleme sistemi, tek bir otorite tarafından işletilen İnternetin bir "bölgesi" olarak düşünülebilecek bir alan kavramına dayanmaktadır; bu otorite bir üniversite, kulüp, şirket veya devlet ajansı olabilir. (Bölge kelimesi burada alıntı içinde çünkü, yakında göreceğimiz gibi, böyle bir bölge İnternetin fiziksel bir alanına karşılık gelmeyebilir.) Her alan, ICANN ile kaydedilmelidir—bu işlem, ICANN tarafından bu rolü üstlenmiş olan kayıt şirketleri tarafından yürütülmektedir. Bu kayıt sürecinin bir parçası olarak, alana, tüm İnternet üzerindeki alan adları arasında benzersiz olan bir mnemonik alan adı atanmaktadır. Alan adları genellikle alanı kaydeden organizasyonu tanımlayıcıdır, bu da insanların kullanımını artırır.

Örnek olarak, Addison-Wesley yayınevi'nin alan adıaw.com'dur. Noktada n sonraki eki not edin. Bu, alanınsınıflandırmasını yansıtmak için kullanılır; b u durumda, `com` eki ile gösterildiği gibi "ticari"dır.

Bu ekler üst düzey alan adları (TLD'ler) olarak adlandırılır. Diğer TLD'ler arasında `edu` eğitim kurumları için, `gov` ABD hükümet kurumları için, `org` kar amacı gütmeyen kuruluşlar için, `museum` müzeler için, `info` sınırsız kullanım için ve `net`, ilk olarak İSS'ler için tasarlanmış ancak şimdi çok daha geniş bir ölçekte kullanılan bir ektir. Bu genel TLD'lere ek olarak, belirli ülkeler için iki harfli TLD'ler de vardır (bunlara ülke kodu TLD'leri denir) örneğin `au` Avustralya için ve `ca` Kanada için.

Bir alanın mnemonik adı kaydedildiğinde, adı kaydeden kuruluş,alan içinde ki bireysel öğeler için mnemonik tanımlayıcılar elde etmek üzere adı genişletm e özgürlüğüne sahiptir. Örneğin, Addison-Wesley içindeki bireysel bir makine s senterprise.aw.com olarak tanımlanabilir. Alanadlarının sola doğru genişletildi ğini ve bir nokta ile ayrıldığını not edin. Bazı durumlarda birden fazla

Doclingo https://www.doclingo.ai

uzantı, alt alanlar olarak adlandırılır ve bir alan içindeki adları düzenlemenin bir yolu olarak kullanılır. Bu alt alanlar genellikle alanın yargı yetkisi içindeki farklı ağları temsil eder. Örneğin, Nowhere Üniversitesi'ne `nowhereu.edu` al an adı verilmişse, o zaman Nowhere Üniversitesi'ndeki bireysel bir bilgisayar `r2d2.compsc.nowhereu.edu` gibi bir adı olabilir; bu, `r2d2` bilgisayarının `compsc` alt alanında olduğunu ve `nowhereu` alanında, `edu` TLD'si içinde oldu ğunu ifade eder. (Mnemonik adreslerde kullanılan noktalı notasyonun, bit de seni biçiminde adresleri temsil etmek için kullanılan noktalı ondalık notasyon la ilgili olmadığını vurgulamalıyız.)

Mnemonic adresler insanlar için kullanışlı olsa da, mesajlar her zaman IP adresleri aracılığıyla İnternet üzerinden iletilir. Bu nedenle, bir insan uzak bi r makineye mesaj göndermek istediğinde ve hedefi bir mnemonic adresle ta nımladığında, kullanılan yazılımın o adresi bir IP adresine dönüştürebilmesi gerekir. Bu dönüşüm, adres çeviri hizmetleri sunan dizinler olan birçok sunu cunun yardımıyla gerçekleştirilir. Toplu olarak, bu ad sunucuları, alan adı sis temi (DNS) olarak bilinen İnternet genelinde bir dizin sistemi olarak kullanılır . DNS kullanarak bir çeviri gerçekleştirme sürecine DNS sorgulaması denir.

Bu nedenle, bir makinenin mnemonic alan adı aracılığıyla erişilebilir olm ası için, o adın DNS içindeki bir ad sunucusunda temsil edilmesi gerekir. Alan ı kuran varlığın kaynakları olduğunda, kendi alanındaki tüm adları içeren ken di ad sunucusunu kurabilir ve sürdürebilir.

Gerçekten de, bu alan sistemi başlangıçta bu modele dayanıyordu. Her kayıtl ı alan, bir şirket, üniversite veya devlet kurumu gibi yerel bir otorite tarafınd an işletilen İnternet'in fiziksel bir bölgesini temsil ediyordu. Bu otorite, İnter net'e bağlı kendi intraneti aracılığıyla üyelerine İnternet erişimi sağlayan bir erişim İSS'siydi. Bu sistemin bir parçası olarak, organizasyon kendi alanında kullanılan tüm adlar için çeviri hizmetleri sunan kendi ad sunucusunu sürdür üyordu.

Bu model bugün hala yaygındır. Ancak, birçok birey veya küçük kuruluş, gerekli kaynakları taahhüt etmeden İnternet'te bir alan varlığı oluşturmak ist emektedir. Örneğin, yerel bir satranç kulübünün `KingsandQueens.org` olar ak İnternet'te bir varlığa sahip olması faydalı olabilir, ancak kulübün kendi a ğını kurmak, bu ağdan İnternet'e bir bağlantı sürdürmek ve kendi ad sunucu sunu uygulamak için muhtemelen kaynakları yoktur. Bu durumda, kulüp, ISP ile bir sözleşme yaparak ISP tarafından zaten oluşturulmuş kaynakları kullan arak kayıtlı bir alan görünümü oluşturabilir.

Genellikle, kulüp, belki de ISP'nin yardımıyla, kulübün seçtiği ismi kaydeder v e bu ismin ISP'nin ad sunucusuna dahil edilmesi için ISP ile sözleşme yapar. Bu, yeni alan adıyla ilgili tüm DNS sorgularının ISP'nin ad sunucusuna yönlen dirileceği anlamına gelir; buradan doğru çeviri elde edilecektir. Bu şekilde, bi rçok kayıtlı alan tek bir ISP içinde yer alabilir ve her biri genellikle tek bir bilgi sayarın yalnızca küçük bir kısmını kaplar.

İnternet Uygulamaları

Bu alt bölümde, üç *geleneksel* uygulama ile başlayarak İnternet'in bazı uygu lamalarını tartışıyoruz. Ancak, bu "geleneksel" uygulamalar, bugünün İntern et'inin heyecanını yakalamakta yetersiz kalmaktadır. Gerçekten de, ayrım

Doclingo https://www.doclingo.ai

bir bilgisayar ile diğer elektronik cihazlar arasındaki sınırlar giderek belirsizleşiyor. Telef onlar, televizyonlar, ses sistemleri, hırsız alarm sistemleri, mikrodalga fırınlar ve video k ameralar potansiyel "İnternet cihazları"dır. Bunun sonucunda, İnternet'in geleneksel uy gulamaları, anlık mesajlaşma, video konferans, İnternet telefonculuğu ve İnternet rady osu gibi yeni kullanım alanlarının genişleyen akışı tarafından gölgede bırakılmaktadır. Sonuçta, İnternet yalnızca verilerin aktarılabileceği bir iletişim sistemidir. Tek noloji, bu sistemin aktarım hızlarını artırmaya devam ettikçe, aktarılan veriler in içeriği yalnızca birinin hayal gücüyle sınırlı olacaktır. Bu nedenle, günümüz ün ortaya çıkan İnterneti ile ilgili bazı sorunları göstermek için iki yeni İntern et uygulaması, telefonculuk ve radyo yayını dahil edeceğiz; bunlar arasında e k protokol standartlarına ihtiyaç, İnterneti diğer iletişim sistemleriyle bağlam a ihtiyacı ve İnternet yönlendiricilerinin işlevselliğini genişletme ihtiyacı bulu nmaktadır.

Elektronik Posta İnternet'in en popüler kullanım alanlarından biri e-posta (elektronik posta kısaltması) olup, mesajların İnternet kullanıcıları arasında aktarılmasını sağlay an bir sistemdir. E-posta hizmeti sağlamak amacıyla, bir alanın yerel otoritesi, alanı iç inde bir posta sunucusu rolünü üstlenmesi için belirli bir makineyi atayabilir. Genelli kle, posta sunucuları, kendi alanlarında kullanıcılarına posta hizmeti sağlamak amacı yla erişim İSS'leri tarafından işletilen alanlarda kurulmaktadır.
Kullanıcı, yerel makinesinden e-posta gönderdiğinde, öncelikle kullanıcının e -posta sunucusuna aktarılır. Orada, alıcı e-posta sunucusuyla iletişime geçip biriken e-postaları görüntülemek istediğinde, hedef e-posta sunucusuna ileti lir.

E-posta sunucuları arasında e-posta transferi yapmak ve yazarın yerel makinesinden y azarın e-posta sunucusuna yeni bir mesaj göndermek için kullanılan protokol SMTP'dir. (Basit E-posta Transfer Protokolü). SMTP, başlangıçta ASCII ile kodlanmış me tin mesajlarını aktarmak için tasarlandığından, ASCII dışı verileri SMTP uyuml u forma dönüştürmek için MIME (Çok Amaçlı İnternet E-posta Uzantıları) gibi ek protokoller geliştirilmiştir.

Kullanıcının e-posta sunucusuna ulaşan ve biriken e-postalara erişmek için k ullanılabilecek iki popüler protokol vardır. Bunlar POP3 (Posta Ofisi Protokolü sür üm 3) ve IMAP (İnternet E-posta Erişim Protokolü)dir. POP3 (üçüncü “pop-THREE” olarak telaffuz edilir) ikisinden daha basit olanıdır. POP3 kullanarak, bir kullanıcı mesajları yerel bilgisayarına aktarır (indirir) ve burada okunabilir, çeşitli klasörler de saklanabilir, düzenlenebilir ve kullanıcı istediği gibi manipüle edilebilir.
Bu, kullanıcının yerel makinesinde yerel makinenin kütle depolama alanı kullanılarak yapılır. IMAP (okunuşu “EYE-map”), bir kullanıcının mesajlarıve ilgili materyalleri aynı ma kinede, posta sunucusuyla birlikte saklamasına ve manipüle etmesine olanak tan ır. Bu şekilde, farklı bilgisayarlardan e-postalarına erişmesi gereken bir kullanıcı, erişim sağlayabileceği herhangi bir uzaktan bilgisayardan erişilebilen posta sunu cusunda kayıtları sürdürebilir.

Bir mail sunucusunun rolü göz önünde bulundurulduğunda, bir bireyin e-p osta adresinin yapısını anlamak kolaydır.Bu, bireyi tanımlayan bir sembol dizisi nden (bazen hesap adı olarak adlandırılır) oluşur, ardından @ sembolü gelir (ok unuşu “at”), ardından da e-postayı alması gereken mail sunucusunu nihayetind e tanımlayan mnemonik dizi gelir.(Aslında bu dizi genellikle yalnızca hedef alan ı tanımlar ve alanın mail sunucusu nihayetinde bir DNS sorgulaması yoluyla tan ımlanır.) Böylece Addison-Wesley Inc.'deki bir bireyin e-posta adresi `shakespeare@aw.com` olarak görünebilir. Diğer bir deyişle, bir mesaj

Doclingo https://www.doclingo.ai

bu adrese gönderilen, `aw.com` alanındaki posta sunucusuna gitmekte ve sembol dizisi shakespeare ile tanımlanan kişi için tutulmalıdır.

Dosya Transfer Protokolü Dosyaları (belgeler, fotoğraflar veya diğer kodlanmış bilg iler gibi) aktarmanın bir yolu, bunları e-posta mesajlarına eklemektir. Ancak, daha verimli bir yol, Dosya Transfer Protokolü(FTP)'nden yararlanmaktır, bu, dosyaları İnternet üzerinden aktarmak için bir istemci/sunucu protokolüdür. FTP kullanarak bir dosya aktarmak için, İnternet'teki bir bilgisayardaki bir kullanıcı, FTP'yi uygula yan bir yazılım paketi kullanarak başka bir bilgisayarla iletişim kurar. (Orijinal bilgi sayar bir istemci rolünü oynar. İletişim kurduğu bilgisayar ise genellikle FTP sunu cusu olarak adlandırılan bir sunucu rolünü oynar.) Bu bağlantı kurulduktan sonra, dosyalar iki bilgisayar arasında her iki yönde aktarılabilir.

FTP, Internet üzerinden verilere sınırlı erişim sağlamak için popüler bir yol haline gelmiştir. Örneğin, belirli kişilerin bir dosyayı almasına izin vermek, anca k diğerlerinin erişimini engellemek istiyorsanız, dosyayı FTP sunucu olan bir ma kineye yerleştirmeniz ve dosyaya erişimi bir şifre ile korumanız yeterlidir. O zaman, şifreyi bilen kişiler FTP aracılığıyla dosyaya erişim sağlayabilirken, diğerleri engellenecektir. Bu şekilde kullanılan bir Internet makinesine bazen FTP sitesi denir çünkü bu, dosyaların FTP aracılığıyla mevcut olduğu bir Inter net konumunu oluşturur.

FTP siteleri, dosyalara sınırsız erişim sağlamak için de kullanılır. Bunu başarmak i çin, FTP sunucuları evrensel bir giriş adı olarak *anonim* terimini kullanır. Bu tür sitele re genellikle anonim FTP siteleri denir ve kendi denetimleri altındaki dosyalara sınırs ız erişim sağlarlar.

FTP istemcileri ve sunucuları geniş ölçüde mevcut olmaya devam etse de, çoğu kullanıcı artık dos ya transfer ihtiyaçlarını HTTP kullanan Web tarayıcıları aracılığıyla karşılamaktadır (bir sonraki bölümd e tartışılacaktır).

Telnet ve Güvenli Shell İnternetin erken kullanımlarından biri, bilgisayar kullanıcılarının u zak mesafelerden bilgisayarlara erişim sağlamasıydı. **Telnet**, bu amaç için oluşturulmuş bir protokol sistemidir. Telnet kullanarak, bir kullanıcı (telnet istemci yazılımı çalıştıran) uzak bir bilgisayardaki telnet sunucusuna ulaşabilir ve ardından o işletim sisteminin giri ş prosedürünü takip ederek uzak makineye erişim sağlayabilir. Böylece, telnet aracılığıyl a, uzak bir kullanıcı, yerel bir kullanıcının sahip olduğu uygulamalara ve yardımcı progra mlara aynı erişime sahip olur.

İnternetin gelişiminin erken dönemlerinde tasarlanmış olan telnet'inbirç ok eksikliği vardır. Bunlardan daha kritik olanlardan biri, telnet üzerindenileti şimin şifrelenmemiş olmasıdır. Bu, iletişimin konusu hassas olmasa bileöne mlidir çünkü kullanıcının şifresi, oturum açma sürecinde iletişimin bir parçası dır.Bu nedenle telnet kullanımı, bir dinleyicinin bir şifreyi ele geçirme ve dah a sonrabu kritik bilgiyi kötüye kullanma olasılığını açar. Güvenli Kabuk (SSH), busoruna bir çözüm sunan ve hızla telnet'in yerini alan bir alternatiftir. SSH' nınözellikleri arasında, aktarılan verilerin şifrelenmesini sağlaması ve iki taraf ıniletişimde bulunduğunun gerçekten kim olduğunu doğrulama süreci olan kimlikdoğrulama (Bölüm 4.5) yer almaktadır.

VoIP Daha yeni bir Internet uygulaması örneği olarak, VoIP (Internet Protokolü üzerinden Ses) dikkate alınabilir; burada Internet altyapısı, geleneksel telefon sistemlerine benzer ses iletişimi sağlamak için kullanılmaktadır. En

Doclingo https://www.doclingo.ai

basit haliyle, VoIP, farklı makinelerde ses verilerini P2P modeli aracılığıyla aktaran iki süreçten oluşmaktadır; bu süreç kendisi önemli bir sorun oluşturmamaktadır. Ancak, arama başlatma ve alma, VoIP'yi geleneksel telefon sistemleriyle bağlama ve acil durum 911 iletişimi gibi hizmetler sağlama gibi görevler, geleneksel Internet uygulamalarının ötesine geçen sorunlardır. Ayrıca, kendi ülkelerinin geleneksel telefon şirketlerine sahip olan hükümetler, VoIP'yi b ir tehdit olarak görmekte ve ya ağır vergilendirmekte ya da tamamen yasaklamaktadır.

Mevcut VoIP sistemleri, popülarite için rekabet eden dört farklı biçimde gelmektedi r. VoIP yumuşak telefonlar, iki veya daha fazla PC'nin sadece bir hoparlör ve bir mikrof on gibi özel donanım olmaksızın bir arama paylaşmasına olanak tanıyan P2P yazılımınd an oluşur. VoIP yumuşak telefon sistemine bir örnek Skype'tır; bu sistem ayrıca müşteril erine geleneksel telefon iletişim sistemine bağlantılar sunmaktadır. Skype'ın bir dezava ntajı, özel bir sistem olmasıdır; bu nedenle, operasyonel yapısının büyük bir kısmı kamu ya açık değildir. Bu, Skype kullanıcılarının, üçüncü taraf doğrulaması olmaksızın Skype y azılımının bütünlüğüne güvenmeleri gerektiği anlamına gelir. Örneğin, arama almak içi n bir Skype kullanıcısının bilgisayarını internete bağlı ve Skype sistemine erişilebilir duru mda bırakması gerekir; bu, bilgisayarın bazı kaynaklarının, bilgisayar sahibinin farkında olmaksızın diğer Skype iletişimlerini desteklemek için kullanılabileceği anlamına gelir— bu özellik bazı direnişler yaratmıştır.

VoIP'nin ikinci bir biçimi, kullanıcıların geleneksel telefonlarını bir erişim ISP t arafından sağlanan telefon hizmetine bağlamalarına olanak tanıyan analog telefo n adaptörleridir. Bu seçenek genellikle geleneksel internet hizmeti ve/veya dijital t elevizyon hizmeti ile birlikte sunulmaktadır.

VoIP'nin üçüncü türü, geleneksel bir telefonu doğrudan bir TCP/IP ağına bağlı eşdeğer bir el cihazıyla değiştiren gömülü VoIP telefonları şeklindedir. Gömülü VoIP telefonları, maliyetleri azaltmak ve özellikleri artırmak için gele neksel iç bakır tel telefon sistemlerini VoIP üzerinden Ethernet ile değiştiren büyük organizasyonlar arasında giderek daha yaygın hale gelmektedir.

Son olarak, bir sonraki nesil akıllı telefonların VoIP teknolojisini kullanması bekleniyor. Yani, önceki nesil kablosuz telefonlar yalnızca telefon şirketinin ağında o şirk etin protokollerini kullanarak iletişim kuruyordu. İnternete erişim

Kablosuz Telefonların Nesilleri

Son on yılda mobil telefon teknolojisi, basit, tek amaçlı, taşınabilir cihazlard an karmaşık, çok işlevli el bilgisayarlarına evrildi. İlk nesil kablosuz telefon a ğı, geleneksel telefonlar gibi havadan analog ses sinyalleri iletti, ancak duv ardan geçen bakır tel olmadan. Geriye dönüp baktığımızda, bu erken telefo n sistemlerine "1G" veya birinci nesil ağlar diyoruz. İkinci nesil, sesi kodlam ak için dijital sinyaller kullandı ve hava dalgalarının daha etkili kullanımını s ağladı ve metin mesajlaşması gibi diğer dijital veri türlerinin iletimini sağla dı. Üçüncü nesil ("**3G**") telefon ağı, mobil video aramaları ve diğer bant geni şliği yoğun etkinlikler için daha yüksek veri transfer hızları sağlar. 4G ağının hedefleri, daha yüksek veri transfer hızları ve IP protokolünü kullanan tam amen paket anahtarlamalı bir ağdır; bu, en yeni akıllı telefon nesline şu and a yalnızca geniş bant destekli PC'lerde mevcut olan yetenekleri sağlay acaktır.

Doclingo https://www.doclingo.ai

şirketin ağı ile İnternet arasındaki geçitler aracılığıyla elde edildi, bu noktada sinyaller TCP/IP sistemine dönüştürüldü. Ancak, yeni 4Gtelefon ağı, tamame n IP tabanlı bir ağ olarak tasarlanmıştır, bu da bir 4G telefonunun esasen kür esel İnternet'teki başka bir ana bilgisayar olduğu anlamına gelir.

İnternet Radyosu Diğer bir son zamanlarda ortaya çıkan İnternet uygulaması, rady o istasyonu programlarının iletimidir—sinyallerin "hava üzerinden" değil, İnternet a racılığıyla iletildiği için *webcasting* olarak adlandırılan bir süreçtir.Daha kesin olarak, İnternet radyosu, gerçek zamanlı olarak ses verilerinin transferini ifade eden strea ming audio'nun özel bir örneğidir.

Yüzeyde, İnternet radyosunun özel bir dikkate ihtiyaç duymadığı görünmey ebilir. Bir istasyonun, program mesajlarını talep eden her bir istemciye gönder ecek bir sunucu kurabileceğini tahmin edebiliriz. Bu teknik N-unicast olarak bili nir. (Daha kesin olarak, *unicast* bir göndericinin bir alıcıya mesaj göndermesini i fade ederken, *N-unicast* birden fazla unicast ile ilgili tek bir göndericiyi ifade ede r.) N-unicast yaklaşımı uygulanmıştır ancak istasyonun sunucusu ve sunucunun hemen yanındaki İnternet komşuları üzerinde önemli bir yük oluşturma dezav antajına sahiptir. Gerçekten de, N-unicast sunucunun her bir istemcisine gerçe k zamanlı olarak bireysel mesajlar göndermesini zorlar ve bu mesajların hepsi sunucunun komşuları tarafından iletilmelidir.

Çoğu N-unicast alternatifleri, bu sorunu hafifletme girişimlerini temsil eder. Birisi, dosya paylaşım sistemlerini anımsatan bir şekilde P2P modelini uygular. Yani, bir eş veri aldığında, bu veriyi hala bekleyen diğer eşlere dağıtmaya ba şlar; bu da dağıtım probleminin büyük bir kısmının verinin kaynağından eşle re aktarılması anlamına gelir.

Diğer bir alternatif, çoklu yayım (multicast) olarak adlandırılan, dağıtım pro blemini İnternet yönlendiricilerine aktarır. Çoklu yayım kullanarak, bir sunucu t ek bir adres aracılığıyla birden fazla istemciye bir mesaj iletir ve yönlendiricileri n bu adresin önemini tanımasını ve mesajın kopyalarını uygun hedeflere üretip iletmesini bekler. Çoklu yayımda kullanılan tek adres, grup adresi olarak adlan dırılır ve belirli bir başlangıç bit deseni ile tanımlanır. Kalan bitler, çoklu yayım t erminolojisinde grup olarak adlandırılan yayın istasyonunu tanımlamak için kul lanılır. Bir istemci belirli bir istasyondan mesaj almak istediğinde (belirli bir gru ba abone olmak istediğinde), en yakın yönlendiricisine bu isteğini bildirir. O yö nlendirici, bu isteği İnternet üzerinden geri ileterek diğer yönlendiricilerin o gr up adresi ile tüm gelecekteki mesajları istemciye doğru iletmeye başlamasını s ağlar.Kısacası, çoklu yayım kullanıldığında, sunucu dinleyen kaç istemci olursa olsun yalnızca bir program kopyası iletir ve bu mesajların kopyalarını gerektiği gibi yapmak ve uygun hedeflere yönlendirmek yönlendiricilerin sorumluluğun dadır. Bu nedenle, çoklu yayım kullanan uygulamaların, İnternet yönlendiriciler inin orijinal görevlerinin ötesinde işlevselliğinin genişletilmesini gerektirdiğini u nutmayın. Bu süreç şu anda devam etmektedir.

O halde, VoIP gibi internet radyosunun da popülaritesinin arttığını ve temel lerini aradığını görüyoruz. Geleceğin ne getireceği kesin değil. Ancak, internet altyapısının yetenekleri genişlemeye devam ettikçe, web yayıncılığı uygulamala rının da onunla birlikte gelişeceği kesindir.

Gömülü cihazlar ve ev bilgisayarları artık internet üzerinden talep üzerin e yüksek tanımlı video akışı yapabiliyor. Geniş bir televizyon, DVD/Blu-ray oy natıcı ve oyun konsolu sınıfı artık TCP/IP ağına doğrudan bağlanarak hem üc retsiz hem de abonelik sunucularından görüntülenebilir içerik seçebiliyor.

Doclingo https://www.doclingo.ai

Sorular & Alıştırmalar

1. Tier-1 ve tier-2 ISS'lerin amacı nedir? Erişim ISS'lerinin amacı nedir?
2. DNS nedir?
3. Dotted decimal notasyonunda 3.6.9 tarafından temsil edilen bit deseni nedir? Bit desenini 0001010100011100, dotted decimal notasyonu kullanarak ifade edin.
4. Bir bilgisayarın İnternet üzerindeki mnemonik adres yapısı (örneğin `r2d2.compsc.nowhereu.edu`) geleneksel bir posta adresine nasıl b enzer? Bu aynı yapı IP adreslerinde de mevcut mu?
5. İnternette bulunan üç sunucu türünü adlandırın ve her birinin ne yaptığını s öyleyin.
6. SSH neden telnet'ten üstün olarak kabul edilir?
7. P2P ve çoklu yayın (multicast) yaklaşımlarının internet radyosu yayıncılı ğı, N-unicast'tan nasıl farklıdır?
8. VoIP'in dört türünden birini seçerken hangi kriterler dikkate alınmalı dır?

4.3 Dünya Çapında Ağ

Bu bölümde, çoklu ortam bilgilerini internet üzerinden dağıtan bir internet u ygulamasına odaklanıyoruz. Bu, bağlantılar içeren metin belgelerini tanımlay an bir terim olan hiper metin kavramına dayanmaktadır. Bugün, hiper metin , görüntüleri, sesi ve videoyu kapsayacak şekilde genişletilmiştir ve bu genişl etilmiş kapsam nedeniyle bazen hiper medya olarak adlandırılmaktadır.

Bir GUI kullanırken, bir hiper metin belgesinin okuyucusu, fare ile işaretl eyip tıklayarak ona bağlı hiper bağlantıları takip edebilir. Örneğin, "Orkestra' nın Maurice Ravel'in 'Bolero' eserinin performansı olağanüstüydü" cümlesi bi r hiper metin belgesinde yer alıyorsa ve isim Maurice Ravel başka bir belgey e bağlantılıysa—belki de besteci hakkında bilgi veriyordur. Bir okuyucu, fare ile isme Maurice Ravel'i işaretleyip fare düğmesine tıklayarak o ilgili materyal i görüntülemeyi seçebilir. Dahası, uygun hiper bağlantılar kuruluysa, okuyuc u isme Bolero tıklayarak konserin ses kaydını dinleyebilir.

Bu şekilde, hiper metin belgelerinin bir okuyucusu, ilgili belgeleri keşfed ebilir veya belgeler arasında bir düşünce akışını takip edebilir. Çeşitli belgele rin bölümleri diğer belgelere bağlandığında, ilgili bilgilerin iç içe geçmiş bir a ğı oluşur. Bir bilgisayar ağı üzerinde uygulandığında, böyle bir ağ içindeki be lgeler farklı makinelerde bulunabilir ve ağ genelinde bir web oluşturur. İnter net üzerinde gelişen web, tüm dünyayı kapsar ve **World Wide Web** (aynı za manda **WWW, W3,** veya **Web** olarak da anılır) olarak bilinir.

World Wide Web üzerindeki bir hiper metin belgesine genellikle Web sayfası denir. Yakından ilişkili Web sayfalarının bir koleksiyonuna Web sitesi denir.

World Wide Web'in kökenleri, Tim Berners-Lee'nin bağlantılı belge kavra mını internet teknolojisi ile birleştirmenin potansiyelini fark etmesi ve Aralık 1990'da WWW'yi uygulamak için ilk yazılımı üretmesiyle ortaya çıkmıştır.

Doclingo https://www.doclingo.ai

Web Uygulaması

Kullanıcıların internette hiper metne erişmesine olanak tanıyan yazılım paket leri iki kategoriye ayrılır: istemci rolü üstlenen paketler ve sunucu rolü üstlen en paketler. İstemci paketi, kullanıcının bilgisayarında bulunur ve kullanıcını n talep ettiği materyalleri elde etme ve bu materyalleri düzenli bir şekilde kul lanıcıya sunma görevini üstlenir. Kullanıcının web içinde gezinmesini sağlaya n kullanıcı arayüzünü sağlayan istemcidir. Bu nedenle istemci genellikle bir t arayıcı veya bazen bir Web tarayıcısı olarak adlandırılır.
Sunucu paketi (genellikle bir Web sunucusu olarak adlandırılır), erişilmesi gere ken hiper metin belgelerini içeren bir bilgisayarda bulunur. Görevi, istemcilerin talep ettiği belgeleri kontrolü altındaki belgelerle erişim sağlamaktır. Özetle, bi r kullanıcı, kullanıcının bilgisayarında bulunan bir tarayıcı aracılığıyla hiper meti n belgelerine erişim sağlar. Bu tarayıcı, istemci rolünü üstlenerek, internet gen elinde dağılmış Web sunucularının hizmetlerini talep ederek belgeleri elde eder . Hiper metin belgeleri genellikle tarayıcılar ve Web sunucuları arasında Hiper Metin Transfer Protokolü olarak bilinen bir protokol kullanılarak aktarılır. **(HTTP).**

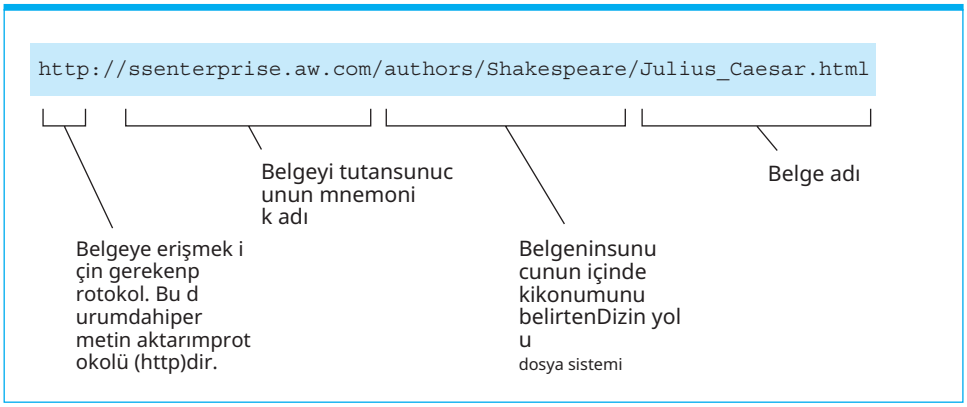
Dünya Çapında Ağ'da belgeleri bulmak ve almak için, her belgeye Tekdü zen Kaynak Bulucu (URL) adı verilen benzersiz bir adres verilir.
Her URL, bir tarayıcının uygun sunucuya ulaşmak ve istenen belgeyi talep et mek için ihtiyaç duyduğu bilgileri içerir. Böylece bir Web sayfasını görüntüle mek için, bir kişi önce tarayıcısına istenen belgenin URL'sini sağlar ve ardınd an tarayıcıya belgeyi alıp görüntülemesi talimatını verir.

Tipik bir URL, Şekil 4.8'de sunulmuştur. Dört bölümden oluşur:
belgeye erişimi kontrol eden sunucu ile iletişim kurmak için kullanılacak prot okol, sunucuyu barındıran makinenin mnemonik adresi, sunucunun belgeyi i çeren dizini bulması için gereken dizin yolu ve belgenin kendisinin adı. Kısac ası, Şekil 4.8'deki URL, bir tarayıcıya, HTTP protokolünü kullanarak

`ssenterprise.aw.com` bilgisayarındaki Web sunucusuna ulaşmasını ve Sh akespeare alt dizininde bulunan `Julius_Caesar.html` adlı belgeyi almasını söyler.

Bazen bir URL, Şekil 4.8'de gösterilen tüm bölümleri açıkça içermeyebilir. Örneğin, eğer sunucunun belgeye ulaşmak için bir dizin yolunu takip etmesi gerekmiyorsa, URL'de hiçbir dizin yolu görünmeyecektir. Ayrıca,

Şekil **4.8** Tipik bir URL



Doclingo https://www.doclingo.ai

Bazen bir URL yalnızca bir protokol ve bir
bilgisayarın mnemonik adresinden oluşur. Bu durumlarda, o bilgisayardaki Web sunucusu önceden belirlenmiş bir belge döndürür, genellikle ana sayfa olarak adlandırılır ve bu genellikle o Web sitesinde mevcut olan bilgileri tanımlar. Bu tür kısaltılmış URL'ler, kuruluşlarla iletişim kurmanın basit bir yolunu sağlar. Örneğin, URL `http://www.google.com` Google'ın ana sayfasına yönlendirecek, bu sayfa şirketle ilgili hizmetlere, ürünlere ve belgelere bağlantılar içerir.

Web sitelerini bulmayı daha da basitleştirmek için, birçok tarayıcı,eğer bir protokol tanımlanmamışsa HTTPprotokolünün kullanılacağını varsayar. Bu tarayıcılar,www.google.com'dan oluşan "URL" verildiğinde doğru bir şekildeGoogle ana sayfasını alır.

HTML'

Geleneksel bir hiper metin belgesi, metninin bir sistem (ASCII veya Unicode gibi) kullanılarak karakter karakter kodlandığı için bir metin dosyasına benzer. Fark, bir hiper metin belgesinin ayrıca belgenin bir ekran üzerinde nasıl görünmesi gerektiğini, hangi çoklu ortam kaynaklarının (görüntüler gibi) belgeye eşlik etmesi gerektiğini ve belgedeki hangi öğelerin diğer belgelere bağlı olduğunu tanımlayan özel semboller, yani etiketler içermesidir. Bu etiketler sistemi, Hiper Metin İşaretleme Dili (HTML) olarak bilinir.

Böylece, bir Web sayfasının yazarı, bir tarayıcının sayfayı kullanıcının ekranında sunmak için ihtiyaç duyduğu bilgileri ve mevcut sayfa tarafından referans verilen ilgili belgeleri bulmak için HTML açısından tanımlar. Bu süreç, bir düz yazılı metne (belki kırmızı bir kalem kullanarak) dizgi talimatları eklemeye benzer, böylece bir dizgici materyalin nihai biçimde nasıl görünmesi gerektiğini bilecektir. Hiper metin durumunda, kırmızı işaretler HTML etiketleriyle değiştirilir ve bir tarayıcı nihayetinde dizgicinin rolünü oynar, HTML etiketlerini okuyarak metnin bilgisayar ekranında nasıl sunulacağını öğrenir.

Son derece basit bir Web sayfasının HTML kodlu versiyonu (kaynak versiyonu olarak adlandırılır) Şekil 4.9a'da gösterilmektedir. Etiketlerin < ve > sembolleriyle belirlendiğine dikkat edin. HTML kaynak belgesi iki bölümden oluşur - bir başlık (< başlık > ve < /başlık > etiketleriyle çevrelenmiş) ve bir gövde (< gövde > ve < /gövde > etiketleriyle çevrelenmiş).
Bir Web sayfasının başlığı ve gövdesi arasındaki ayrım, bir iç ofis notunun başlığı ve gövdesi arasındaki ayrım ile benzerdir. Her ikisinde de

Dünya Çapında Ağ Konsorsiyumu

Dünya Çapında Ağ Konsorsiyumu (W3C), 1994 yılında Dünya Çapında Ağı teşvik etmek amacıyla protokol standartları (W3C standartları olarak bilinir) geliştirmek için kurulmuştur. W3C, Cenevre, İsviçre'deki yüksek enerjili parçacık fiziği laboratuvarı CERN'de merkezi bulunmaktadır. CERN, orijinal HTML işaretleme dilinin ve HTML belgelerinin İnternet üzerinden aktarımı için HTTP protokolünün geliştirildiği yerdir. Bugün W3C, geniş bir İnternet ürünleri yelpazesinde uyumluluğa yol açan birçok standardın (XML ve sayısız multimedya uygulamaları için standartlar dahil) kaynağıdır. W3C hakkında daha fazla bilgiye http://www.w3c.org adresinden ulaşabilirsiniz.

Doclingo https://www.doclingo.ai

Durumlarda, başlık belge hakkında ön bilgi içerir (tarih, alt-konu vb. bir not için). Gövde, belgenin esasını içerir, Web sayfası durumunda, sayfa görüntülendiğinde bilgisayar ekranında sunulacak malzemedir.

Şekil 4.9a'da görüntülenen Web sayfasının başlığı yalnızca belgenin titresini içerir (“title” etiketleriyle çevrelenmiştir). Bu başlık yalnızca belgelendirme amaçları içindir; bu, bilgisayar ekranında görüntülenecek sayfanın bir parçası değildir. Ekranda görüntülenen malzeme belgenin gövdesinde yer alır.

Şekil 4.9a'daki belgenin gövdesindeki ilk giriş, “My WebPage” metnini içeren bir seviye bir başlıktır (`<h1>` ve`</h1>` etiketleriyle çevrelenmiştir). Seviye bir başlık olması, tarayıcının bu metniekranda belirgin bir şekilde görüntülemesi gerektiği anlamına gelir. Gövdedeki bir sonraki giriş,“Click here for another page ” metnini içeren bir metin paragrafıdır(`<p>` ve`</p>` etiketleriyle çevrelenmiştir) . Şekil 4.9b, sayfanın birtarayıcı tarafından bilgisayar ekranında nasıl sunulacağını göstermektedir.

Şekil **4.9** Basit bir Web sayfası

a. HTML kullanılarak kodlanmış sayfa.

Dökümanınbaşlangıcınıbelirten etiket

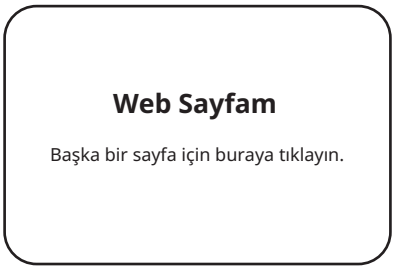
Ön Hazırlıklar

Tarayıcı tarafından görüntülenecek olandökümanın kısmı

Dökümanın sonunubelirten etiket

```
<html>
<kafa>
<title>gösterim sayfası</title>
</head>
<gövde>
<h1>Web Sayfam</h1>
<p>Başka bir sayfa için buraya tıklayın.</p>
</body>
</html>
```

b. Sayfanın bir bilgisayar ekranında görüneceği gibi.



Doclingo https://www.doclingo.ai

Mevcut haliyle, Şekil 4.9'daki sayfa, izleyici kelimeye tıkladığında hiçbir şeyin o lmayacağı anlamında tam olarak işlevsel değildir. *burada*, sayfanın böyle bir eyle min tarayıcının başka bir sayfayı görüntülemesine neden olacağını ima etmesine r ağmen. Uygun eylemi gerçekleştirmek için, kelimeyi *burada* başka bir belgeye ba ğlamalıyız.

Farz edelim ki, kelimeye tıklandığında *burada*, tarayıcının URL'deki sayfayı almasını ve görüntülemesini istiyoruz. `http://crafty.com/demo.html`. Bun u yapmak için, sayfanın kaynak versiyonunda kelimeyi *burada* `<a>` ve`</a>` eti ketleriyle çevrelemeliyiz; bunlara bağlantı etiketleri denir. Açılış bağlantı etiketi nin içine parametreyi ekliyoruz

```
href = http://crafty.com/demo.html
```

(Şekil 4.10a'da gösterildiği gibi) etiketle ilişkili hiper metin referansının (`href`) e şitlik işaretinden sonraki URL olduğunu belirtir (http://crafty.com/demo.html). Bağlantı etiketlerini ekledikten sonra, Web sayfası artık Şekil 4.10b'de gösterildi ği gibi bir bilgisayar ekranında görünecektir. Bu, kelimenin *burada* başka bir W eb belgesine bağlantı olduğunu gösteren bir renk ile vurgulandığı dışında Şekil 4.9b ile aynıdır. Böyle vurgulanan terimlere tıklamak, tarayıcının ilişkili Web bel gesini almasını ve görüntülemesini sağlayacaktır. Böylece, Web belgeleri birbiri ne bağlantı etiketleri aracılığıyla bağlanır.

Son olarak, basit bir Web sayfamıza bir görüntünün nasıl dahil edilebilec eğini belirtmeliyiz. Bu amaçla, dahil etmek istediğimiz görüntünün JPEG kodl amasının `OurPic.jpg` adlı dosya olarak dizinde saklandığını varsayalım. Görüntüler Images.com adresinde bulunmaktadır ve o konumda Web sunucusu aracılığıyla mevcuttur. Bu koşullar altında, bir tarayıcıya Web sayfasının en üstünde resmi görüntül emesi için resim etiketini`<img src = “http://Images.com/Images/OurPic.jpg” >` HTML kaynak belgesindeki `<body>` etiketinin hemen sonrasına yerleştirer ek talimat verebiliriz. Bu, tarayıcıya OurPic.jpg adlı resmin belgenin başında görüntülenmesi gerektiğini söyler. (src terimi “kaynak” kelimesinin kısaltmas ıdır, bu da eşittir işaretinden sonraki bilginin görüntülenecek resmin kaynağı nı belirttiği anlamına gelir.) Tarayıcı butag'i bulduğunda, `Images.com` adresi ndeki HTTP sunucusuna `OurPic.jpg` adlı resmi talep eden bir mesaj gönder ir ve ardından resmi uygun şekilde görüntüler.

Resim etiketini belgenin sonuna, tam olarak `</body>` etiketi, ardından tarayıcı resmi Web sayfasının en altında görüntül er. Elbette, bir resmi Web sayfasında konumlandırmak için daha sofistike tek nikler vardır, ancak bunlar şu anda bizi ilgilendirmiyor.

XML

HTML esasen bir metin belgesinin yanı sıra belgenin görünümünün basit bir metin dosyası olarak kodlanabileceği bir notasyon sistemidir. Benzer bir şeki lde, metin dışı materyalleri de metin dosyaları olarak kodlayabiliriz—bir örne k nota. İlk bakışta, müziğin geleneksel olarak temsil edildiği nota çizgileri, ölç ü çubukları ve notaların deseni, metin dosyaları tarafından dayatılan karakte r-karakter formatına uymamaktadır. Ancak, bu sorunu alternatif bir notasyo n sistemi geliştirerek aşabiliriz. Daha kesin olarak, bir nota çizgisinin başlang ıcını `<nota anahtarı = “soprano”>` ile temsil etmeyi kabul edebiliriz, sonu

Doclingo https://www.doclingo.ai

Şekil **4.10**Geliştirilmiş basit Web sayfası

a. HTML kullanılarak kodlanmış sayfa.

```
<html>
<kafa>
<title>gösterim sayfası</title>
</head>
<gövde>
<h1>Web Sayfam</h1>
<p>Tıkla
    <a href="http://crafty.com/demo.html">
burada
    </a>
başka bir sayfa için.</p>
</body>
</html>
```

Anahtar etik etiparametre içerenKap

atmaan ahtar etiketi

b. Sayfanın bir bilgisayar ekranında görüneceği gibi.



personel tarafından </personel>, <time>2/4</zaman> biçiminde bir zama n imzası, bir ölçünün başlangıcı ve sonu <ölçü> ve</ölçü>, sırasıyla, C notası nda bir sekizlik nota gibi <notalar> sekizlik C</notalar>, ve devamı. Sonra m etin

<staff clef = "treble"> <key>C minor</key>
<time> 2/4 </time>
<measure> <rest> uzunluk </rest> <notes> uzunluk G,
eg uzunluk G, uzunluk G </notes></measure>
<ölçü> <notlar> hlf E </notlar></ölçü>
</personel>

Doclingo https://www.doclingo.ai

Şekil 4.11'de gösterilen müziği kodlamak için kullanılabilir. Böyle bir notasyo n kullanarak,nota, değiştirilebilir, saklanabilir ve İnternet üzerinden metin do syaları olarak aktarılabilir. Ayrıca, bu tür dosyaların içeriğinigeleneksel nota biçiminde sunmak veya müziği birsentezleyicide çalmak için yazılım yazılabili r.

Nota sistemimizin, HTML tarafından kullanılan aynı tarzı kapsadığını unu tmayın. Bileşenleri tanımlayan etiketlerisembollerle< ve> ile ayırmayı seçtik. Yapıların (bir nota, nota dizisi veya ölçü gibi) başlangıcını ve sonunuaynı isim deki etiketlerle belirtmeyi seçtik—bitirme etiketi bir eğik çizgi ile belirtilmiştir (bir <ölçü>, </ölçü> etiketi ile sonlandırılmıştır). Ve etiketler içinde özel niteli kleri belirtmek için`clef = "treble"` gibi ifadeleri kullanmayı seçtik. Bu ay nı tarz, matematiksel ifadeler ve grafikler gibi diğer formatları temsil etmek i çin sistemler geliştirmek için de kullanılabilir.

Genişletilebilir İşaretleme Dili (XML), verileri metin dosyaları olarak temsi l etmek için notasyon sistemleri tasarlamak için standartlaştırılmış bir stil (m üzik örneğimize benzer)dir. (Aslında, XML, Standart Genel İşaretleme Dili ola rak bilinen daha eski bir standart setinin basitleştirilmiş bir türevidir, daha iyi bilinen adıyla SGML.) XML standardını takip ederek, matematik, çoklu ortam sunumları ve müzik temsil etmek için işaretleme dilleri olarak adlandırılan n otasyon sistemleri geliştirilmiştir. Aslında, HTML, Web sayfalarını temsil etm ek için geliştirilen XML standardına dayanan işaretleme dilidir. (Aslında, HTM L'nin orijinal versiyonu, XML standardı sağlamlaştırılmadan önce geliştirilmiş tir ve bu nedenle HTML'nin bazı özellikleri XML'ye tam olarak uymamaktadır. Bu yüzden, XML'ye titizlikle uyan HTML sürümü olan XHTML'e atıflar görebili rsiniz.)

XML, standartların geniş kapsamlı uygulamalara sahip olacak şekilde nas ıl tasarlandığına dair iyi bir örnek sunar. Çeşitli belge türlerini kodlamak için bireysel, ilgisiz işaretleme dilleri tasarlamak yerine, XML tarafından temsil ed ilen yaklaşım, genel olarak işaretleme dilleri için bir standart geliştirmektir. B u standart ile, çeşitli uygulamalar için işaretleme dilleri geliştirilebilir. Bu şekil de geliştirilen işaretleme dilleri, müzik notaları ve matematiksel ifadeler içer en metin belgeleri gibi karmaşık uygulamalar için işaretleme dilleri elde etm ek üzere birleştirilmelerine olanak tanıyan bir tutarlılığa sahiptir.

Son olarak, XML'in HTML'den farklı olarak görünüm yerine anlamı vurgul ayan yeni işaretleme dilleri geliştirilmesine olanak tanıdığını belirtmeliyiz. Ör neğin, HTML ile bir tarifteki malzemeler, her bir malzemenin bir liste içinde y er aldığı şekilde işaretlenebilir.

Şekil **4.11** Beethoven'ın Beşinci Senfonisi'nin ilk iki çubuğu



Doclingo https://www.doclingo.ai

ayrı bir satır. Ama eğer anlam odaklı etiketler kullansaydık, bir tarifteki malzemeler malzeme olarak işaretlenebilir (belki de etiketleri kullanarak <malzeme> ve </ingredient>) yalnızca bir listedeki maddeler yerine. Fark inceama önem li. Anlamsal yaklaşım, arama motorlarının (kullanıcıların ilgi alanlarına ilişkin Web materyalini bulmalarına yardımcı olan Web siteleri)belirli malzemeleri iç eren veya içermeyen tarifleri tanımlamasına olanak tanır ki bu,belirli kelimel eri içeren veya içermeyen tariflerin izole edilebildiği mevcut durumdanönem li bir iyileşme olacaktır. Daha kesin olarak, eğer anlamsal etiketler kullanılıyo rsa,bir arama motoru ıspanak içermeyen lazanya tariflerini tanımlayabilir, oy sa yalnızcakelime içeriğine dayanan benzer bir arama, "Bu lazanya ıspanak iç ermez." ifadesiylebaşlayan bir tarifi atlayacaktır. Bunun sonucunda, belgeleri görünüm yerine anlamlarınagöre işaretlemek için Internet genelinde bir sta ndart kullanarak, bugün sahip olduğumuzDünya Çapında *Anlamsal* Web yer ine, Dünya Çapında *Sentaktik* Web oluşturulacaktır.

Müşteri Tarafı ve Sunucu Tarafı Faaliyetleri

Şimdi, bir tarayıcının Şekil 4.10'da gösterilen basit Web sayfasını almak ve taray ıcının bilgisayar ekranında görüntülemek için gereken adımları düşünelim. Önc elikle, bir müşteri rolünü üstlenen tarayıcı, bir URL'deki bilgileri (belki de tarayıc ıyı kullanan kişiden alınmış) kullanarak sayfaya erişimi kontrol eden Web sunuc usuyla iletişime geçer ve sayfanın bir kopyasının kendisine aktarılmasını ister. S unucu, Şekil 4.10a'da gösterilen metin belgesini tarayıcıya göndererek yanıt ver ir. Tarayıcı, belgedeki HTML etiketlerini yorumlayarak sayfanın nasıl görüntülen mesi gerektiğini belirler ve belgeyi bilgisayar ekranında buna göre sunar. Taray ıcıyı kullanan kişi, Şekil 4.10b'de tasvir edilen bir görüntü görür. Kullanıcı daha s onra *burada* kelimesinin üzerine fare ile tıkladığında, tarayıcı, ilişkili bağlantı et iketindeki URL'yi kullanarak uygun sunucu ile iletişime geçer ve başka bir Web sayfasını almak ve görüntülemek için gerekli işlemi yapar. Özetle, süreç, tarayıc ının yalnızca kullanıcının yönlendirmesiyle Web sayfalarını alması ve görüntüle mesi ile ilgilidir.

Ama animasyon içeren bir Web sayfası veya bir müşterinin bir sipariş formunu doldur up siparişi göndermesine izin veren bir sayfa istiyorsak ne olur? Bu ihtiyaçlar, ya tarayıcıda n ya da Web sunucusundan ek faaliyetler gerektirecektir. Bu tür faaliyetler, bir müşteri (örn eğin bir tarayıcı) tarafından gerçekleştiriliyorsa müşteri tarafı faaliyetleri veya bir sunucu (örneğin bir Web sunucusu) tarafından gerçekleştiriliyorsa sunucu tarafı faaliyetleri olarak adlandırılır.

Örnek olarak, bir seyahat acentesinin müşterilerin istedikleri destinasyonla rı ve seyahat tarihlerini belirleyebilmelerini istediğini varsayalım; bu durumda a cente, müşteriye yalnızca o müşterinin ihtiyaçlarına uygun bilgileri içeren özelle ştirilmiş bir Web sayfası sunacaktır. Bu durumda seyahat acentesinin Web sites i, önce müşteriye mevcut destinasyonları sunan bir Web sayfası sağlayacaktır. Bu bilgilere dayanarak, müşteri ilgi duyduğu destinasyonları ve istediği seya hat tarihlerini belirtecektir (bir istemci tarafı etkinliği). Bu bilgiler daha sonra acentenin sunucusuna geri aktarılacak ve burada uygun özelleştirilmiş Web sayfasını oluşturmak için kullanılacaktır (bir sunucu tarafı etkinliği) ve ardınd an müşterinin tarayıcısına gönderilecektir.

Doclingo https://www.doclingo.ai

Bir başka örnek, bir arama motorunun hizmetlerini kullanırken meydana g elir. Bu durumda istemcideki bir kullanıcı ilgi duyduğu bir konuyu belirtir (bir is temci tarafı etkinliği) ve bu bilgi arama motoruna aktarılır; burada olası ilgi çek ebilecek belgeleri tanımlayan özelleştirilmiş bir Web sayfası oluşturulur (bir sun ucu tarafı etkinliği) ve istemciye geri gönderilir. Bir başka örnek ise Web mail'dir — bilgisayar kullanıcılarının Web tarayıcıları aracılığıyla e-postalarına eri şebildikleri giderek daha popüler bir yöntemdir. Bu durumda, Web sunucusu istemci ile istemcinin mail sunucusu arasında bir aracı görevi görmektedir. Temelde, Web s unucusu mail sunucusundan bilgi içeren Web sayfaları oluşturur (bir sunucu tarafı et kinliği) ve bu sayfaları istemciye gönderir; burada istemcinin tarayıcısı bunları görünt üler (bir istemci tarafı etkinliği). Tersine, tarayıcı kullanıcıya mesajlar oluşturma imka nı tanır (bir istemci tarafı etkinliği) ve bu bilgiyi Web sunucusuna gönderir; ardından Web sunucusu mesajları mail sunucusuna iletir (bir sunucu tarafı etkinliği) ve gönder im için hazırlar.

Müşteri ve sunucu tarafı etkinliklerini gerçekleştirmek için birçok sistem bu lunmaktadır; her biri diğerleriyle öne çıkmak için rekabet etmektedir. Müşteri t arafı etkinliklerini kontrol etmenin erken ve hala popüler bir yolu, HTML kaynak belgesine JavaScript (Netscape Communications, Inc. tarafından geliştirilen bir dil) ile yazılmış programları dahil etmektir. Buradan bir tarayıcı programları çık arabilir ve gerektiğinde takip edebilir. Diğer bir yaklaşım (Sun Microsystems tar afından geliştirilmiştir) önce bir Web sayfasını bir tarayıcıya aktarmak ve ardınd an HTML kaynak belgesinde talep edilen şekilde tarayıcıya Java dilinde yazılmış applet adı verilen ek program birimlerini aktarmaktır. Diğer bir yaklaşım ise, ge niş multimedya müşteri tarafı sunumlarının uygulanabileceği Flash sistemidir (Macromedia tarafından geliştirilmiştir).

Sunucu tarafı etkinliklerini kontrol etmenin erken bir yolu, CGI (Common Gatew ay Interface) adı verilen bir standart setini kullanmaktı; bu sayede müşteriler sunucu da depolanan programların yürütülmesini talep edebiliyordu. Bu yaklaşımın bir vary asyonu (Sun Microsystems tarafından geliştirilmiştir) müşterilerin sunucu tarafında s ervlet adı verilen program birimlerinin yürütülmesine neden olmalarına izin vermekt ir. İstenen sunucu tarafı etkinliği özelleştirilmiş bir Web sayfasının oluşturulması old uğunda, servlet yaklaşımının basitleştirilmiş bir versiyonu uygulanabilir; bu, seyahat acentesi örneğimizde olduğu gibidir. Bu durumda, JavaServer Pages (JSP) adı verilen Web sayfası şablonları Web sunucusunda depolanır ve bir müşteriden alınan bilgilerl e tamamlanır. Özelleştirilmiş Web sayfalarının oluşturulduğu şablonlar Microsoft tar afından Active Server Pages (ASP) olarak adlandırılmaktadır. Bu özel sistemlerin aksi ne, PHP (ilk başta Kişisel Ana Sayfa anlamına geliyordu, ancak şimdi PHP Hiper Metin İşlemcisi olarak kabul edilmektedir) sunucu tarafı işlevselliğini uygulamak için açık k aynak bir sistemdir.

Son olarak, müşterilerin ve sunucuların birbirlerinin makinelerinde programlar ç alıştırmalarına izin vermenin getirdiği güvenlik ve etik sorunlarını tanımadığımız tak dirde eksik kalırız. Web sunucularının rutin olarak programları müşterilere aktarması ve bunların çalıştırılması, sunucu tarafında etik sorulara ve istemci tarafında güvenli k sorularına yol açar. İstemci, bir Web sunucusu tarafından kendisine gönderilen her hangi bir programı körü körüne çalıştırırsa, sunucu tarafından kötü niyetli faaliyetler e maruz kalır. Benzer şekilde, istemcilerin sunucuda programların çalıştırılmasına ne den olabilmesi, istemci tarafında etik sorulara ve sunucu tarafında güvenlik soruların a yol açar. Sunucu, bir istemci tarafından kendisine gönderilen herhangi bir program ı körü körüne çalıştırırsa, sunucuda güvenlik ihlalleri ve potansiyel zararlar meydana gelebilir.

Doclingo https://www.doclingo.ai

Sorular & Alıştırmalar

1. URL nedir? Tarayıcı nedir?
2. İşaretleme dili nedir?
3. HTML ile XML arasındaki fark nedir?
4. Aşağıdaki HTML etiketlerinin her birinin amacı nedir?
   a. html
   b. head
   c. /p
   d. /a
5. Terimler *istemci tarafı* ve *sunucu tarafı* neyi ifade ediyor?

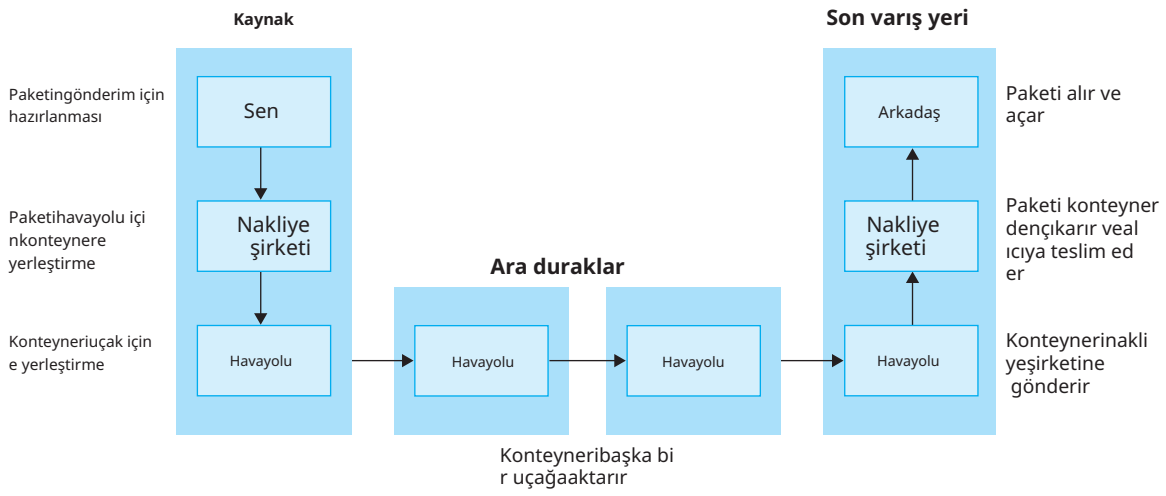
4.4 İnternet Protokolleri

Bu bölümde, mesajların İnternet üzerinden nasıl iletildiğini araştırıyoruz. Bu transfer süreci, sistemdeki tüm bilgisayarların işbirliğini gerektirir ve bu n edenle bu süreci kontrol eden yazılım, İnternet'teki her bilgisayarda bulunur. Bu yazılımın genel yapısını inceleyerek başlıyoruz.

İnternet Yazılımına Katmanlı Yaklaşım

Ağ yazılımının temel görevlerinden biri, mesajları bir makineden diğerine iletm ek için gereken altyapıyı sağlamaktır. İnternet'te, bu mesaj iletim etkinliği, yazılı m birimlerinin hiyerarşisi aracılığıyla gerçekleştirilir; bu birimler, eğer bir hediy eyi Amerika Birleşik Devletleri'nin Batı Kıyısı'ndan Doğu Kıyısı'ndaki bir arkadaşı nıza gönderecek olsaydınız, yerine getireceğiniz görevlere benzer görevleri yeri ne getirir (Şekil 4.12). Öncelikle hediyeyi bir paket olarak sararsınız

Şekil **4.12**Paket gönderim örneği



Doclingo https://www.doclingo.ai

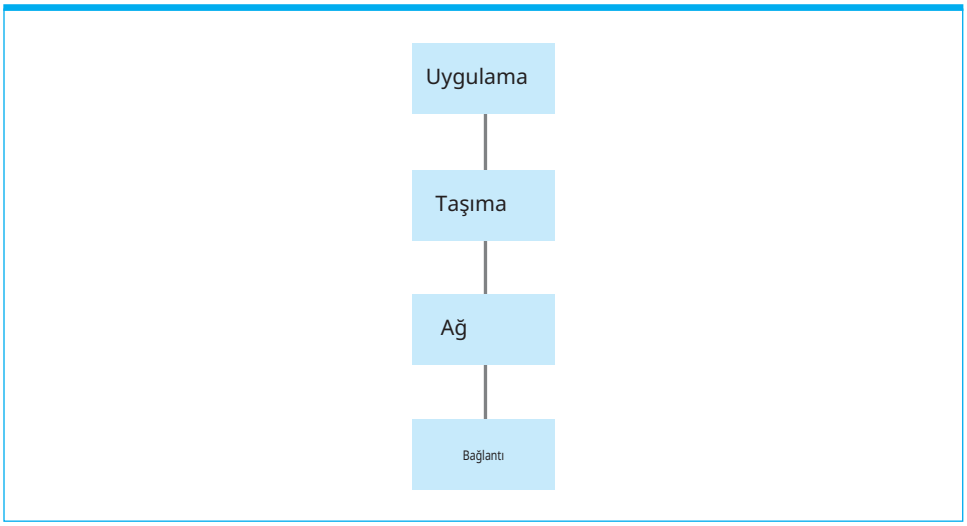
ve paketin dışına uygun adresi yazarsınız. Ardından, paketi U.S. Postal Servic e gibi bir kargo şirketine götürürsünüz.
Nakliye şirketi, paketi diğerleriyle birlikte büyük birkonteynere yerleştirip, hiz metlerini sözleşmeli olarak kullandığı bir havayoluşirketine teslim edebilir. H avayolu şirketi, konteyneri bir uçağa yerleştirip,hedef şehirdeki varış noktası na, belki de yol boyunca ara duraklarla birlikte,aktarıp götürecektir. Nihai var ış noktasında, havayolu şirketi konteyneri uçaktançıkaracak ve onu hedefteki nakliye şirketinin ofisine verecektir. Bununardından, nakliye şirketi paketiniz i konteynerden çıkaracak ve alıcıyaulaştıracaktır.

Kısacası, hediyenin taşınması üç seviyeli bir hiyerarşi tarafından gerçekleştirilecektir: (1) kullanıcı seviyesi (sizin ve arkadaşınızdan oluşur), (2) nakliye şirketi ve (3) havayolu şirketi. Her seviye, bir sonraki alt seviyeyi soyut bir araç olarak kullanır. (Nakliye şirketinin detaylarıyla ilgilenmiyorsunuz ve nakliye şirketi de havayolu şirketinin iç operasyonlarıyla ilgilenmiyor.) Hiyerarşideki her seviyenin hem başlangıçta hem de varış noktasında temsilcileri vardır ve varış noktasındaki temsilciler, başlangıçtaki temsilcilerin tersini yapma eğilimindedir.

İnternet üzerinden iletişimi kontrol eden yazılımlar için de durum böyledir, ancak İnternet yazılımları üç yerine dört katmandan oluşur ve her biri insanlar ve şirketler yerine bir dizi yazılım rutininden oluşur.
Dört katman, uygulama katmanı, taşıma katmanı, ağ katmanı ve bağlantı kat manı (Şekil 4.13) olarak bilinir. Bir mesaj genellikle uygulama katmanında ba şlar. Oradan, iletim için hazırlanırken taşıma ve ağ katmanlarından aşağıya d oğru geçer ve nihayetinde bağlantı katmanı tarafından iletilir. Mesaj, varış n oktasındaki bağlantı katmanı tarafından alınır ve mesajın varış noktasındaki uygulama katmanına teslim edilene kadar hiyerarşide yukarı doğru iletilir.

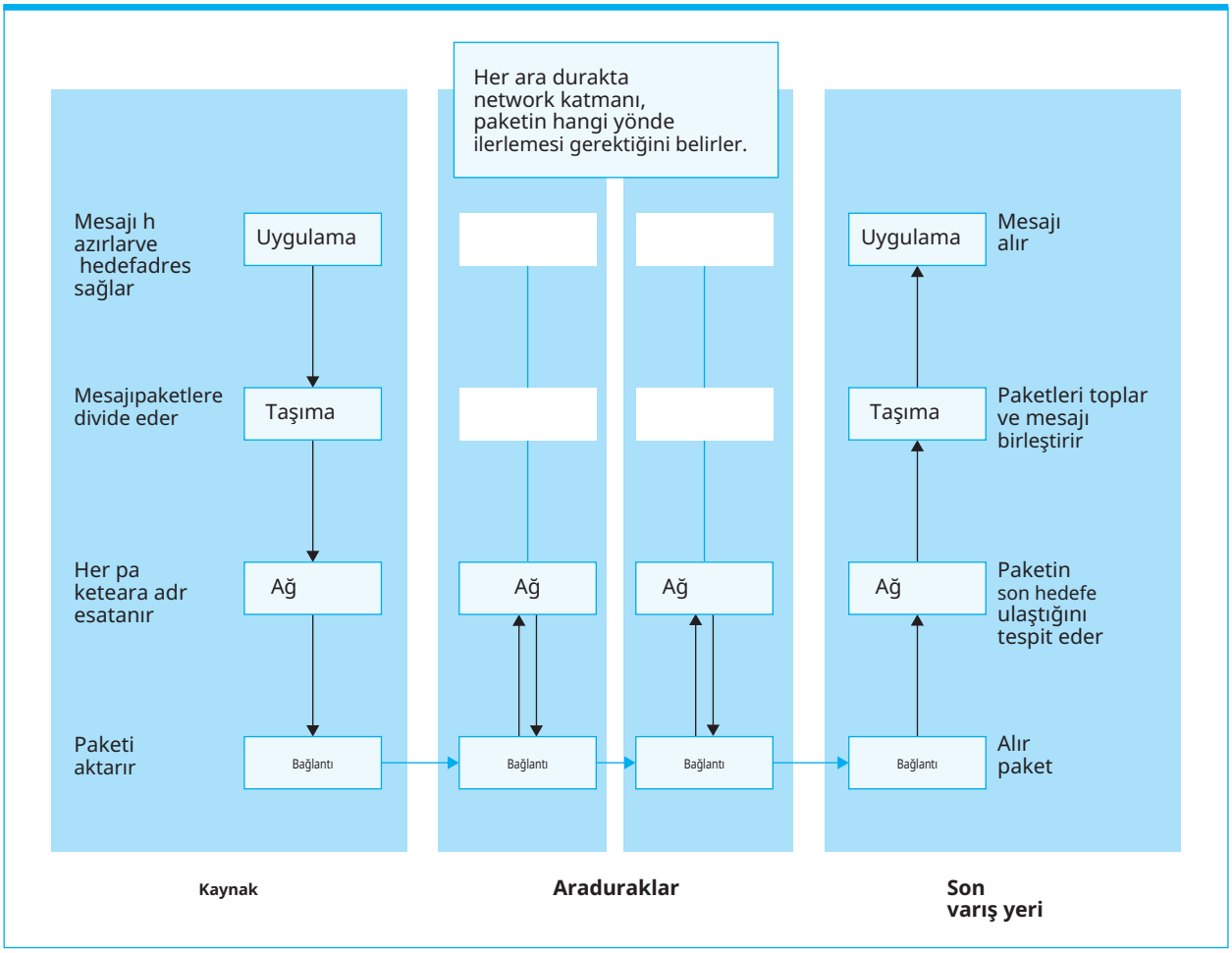
Bu süreci daha ayrıntılı olarak inceleyelim ve bir mesajın sistemde nasıl y ol aldığını izleyelim (Şekil 4.14). Yolculuğumuza uygulama katmanıyla başlıyo ruz.

Şekil 4.13 İnternet yazılım katmanları



Doclingo https://www.doclingo.ai

Şekil **4.14**Bir mesajın İnternet üzerinden izlenmesi



Uygulama katmanı, görevlerini yerine getirmek için Internet iletişimini k ullanan istemciler ve sunucular gibi yazılım birimlerinden oluşur. İsimler ben zer olsa da, bu katman, Bölüm 3.2'de sunulan uygulama sınıflandırmasındak i yazılımlarla sınırlı değildir, aynı zamanda birçok yardımcı paketi de içerir. Örneğin, FTP kullanarak dosya transferi yapan veya SSH kullanarak uzaktan giriş yet enekleri sağlayan yazılımlar o kadar yaygın hale gelmiştir ki, genellikle yardımcı yazılı m olarak kabul edilirler.

Uygulama katmanı, mesajları İnternet üzerinden göndermek ve almak iç in taşıma katmanını kullanır; tıpkı bir kargo şirketini kullanarak paketleri gön dermek ve almak gibi. Kargo şirketinin spesifikasyonlarıyla uyumlu bir adres sağlamak sizin sorumluluğunuzsa, uygulama katmanının da İnternet altyapı sıyla uyumlu bir adres sağlaması gerekmektedir. Bu ihtiyacı karşılamak için, uygulama katmanı, insanların kullandığı mnemonik adresleri İnternet uyuml u IP adreslerine çevirmek için İnternet içindeki ad sunucularının hizmetlerini kullanabilir.

Taşıma katmanının önemli bir görevi, uygulama katmanından gelen mesajlar ı kabul etmek ve mesajların doğru bir şekilde biçimlendirildiğinden emin olmaktır.

Doclingo https://www.doclingo.ai

İnternet üzerinden iletim. Bu son hedefe doğru, taşıma katmanı uzun mesajları küçük parçalara böler, bu parçalar İnternet üzerinden bireysel birimler olarak iletilir. Bu bölme, tek bir uzun mesajın birçok mesajın kesiştiği İnternet yönlendiricilerinde diğer mesajların akışını engelleyebileceği için gereklidir. Gerçekten de, mesajların küçük parçaları bu noktalarda iç içe geçebilirken, uzun bir mesaj diğerlerinin geçmesi için beklemesine neden olur (bir trenin geçmesini bekleyen arabalar gibi).

Taşıma katmanı, ürettiği küçük parçalara sıralama numaraları ekler, böylece parçalar mesajın varış yerinde yeniden bir araya getirilebilir. Daha sonra bu parçalara, paketler denir, ağ katmanına teslim eder. Bunokta dan itibaren, paketler, nihai varış yerlerinde taşıma katmanınaulaşana kadar bireysel, ilgisiz mesajlar olarak ele alınır. Ortak bir mesajaait paketlerin İnter net üzerinden farklı yollar izleyebilmesi oldukçamümkündür.

Ağ katmanının görevi, bir paketin İnternet üzerindeki yolunda her adımda hangi yöne gönderileceğine karar vermektir. Aslında, ağ katmanı ve altındaki bağlantı katmanının birleşimi, İnternet yönlendiricilerinde bulunan yazılımı oluşturur. Ağ katmanı, yönlendiricinin yönlendirme tablosunu korumaktan ve bu tabloyu kullanarak paketleri hangi yöne ileteceğini belirlemekten sorumludur. Yönlendiricinin bağlantı katmanı, paketleri almaktan ve iletmekten sorumludur.

Bu nedenle, bir paketin kaynağındaki ağ katmanı, paketi taşıma katmanından al dığında, paketin nereye gönderileceğini belirlemek için yönlendirme tablosunu kulla nır ve yolculuğuna başlaması için paketi yönlendirir. Doğru yön belirlendikten sonra, ağ katmanı paketi gerçek iletim için bağlantı katmanına teslim eder.

Bağlantı katmanı, paketin iletiminden sorumludur. Bu nedenle, bağlantı katmanı, bilgisayarın bulunduğu bireysel ağın iletişim detaylarıyla ilgilenmek zorundadır. Örneğin, eğer bu ağ bir Ethernet ise, bağlantı katmanı CSMA/C D uygular. Eğer ağ bir WiFi ağı ise, bağlantı katmanı CSMA/CA uygular.

Bir paket iletildiğinde, bağlantının diğer ucundaki bağlantı katmanı tarafı ndan alınır. Orada, bağlantı katmanı paketi ağ katmanına iletir; burada pake tin son varışı, ağ katmanının yönlendirme tablosuyla karşılaştırılır ve paketin bir sonraki adımının yönü belirlenir. Bu karar verildikten sonra, ağ katmanı p aketi bağlantı katmanına geri gönderir ve yoluna devam etmesi için iletilir. B u şekilde, her paket son varışına ulaşana kadar makineden makineye atlar.

Bu yolculuk sırasında yalnızca bağlantı ve ağ katmanlarının ara duraklarda ye r aldığını unutmayın (bkz. tekrar Şekil 4.14) ve dolayısıyla bunlar daha önce belirtil diği gibi yönlendiricilerde bulunan tek katmanlardır. Ayrıca, bu ara "duraklardaki" gecikmeyi en aza indirmek için, bir yönlendiricinin içindeki ağ katmanının yönlend irme rolü, bağlantı katmanıyla yakından entegre edilmiştir. Modern bir yönlendiri cinin bir paketi iletmesi için gereken süre, milyonda bir saniye cinsinden ölçülmek tedir.

Bir paketin nihai varış noktasında, paketin yolculuğunun tamamlandığını tanıyan katman ağ katmanıdır.Bu durumda, ağ katmanı paketi iletmek yerin e onu taşıma katmanına teslim eder. Taşıma katmanı, ağ katmanından paket leri aldıkça, temel mesaj segmentlerini çıkarır ve mesajın kökeninde taşıma k atmanı tarafından sağlanan sıra numaralarına göre orijinal mesajı yeniden i nşa eder. Mesaj

Doclingo https://www.doclingo.ai

birleştirildiğinde, taşıma katmanı onu uygulama katmanındaki uygun birime teslim eder—böylece mesaj iletim süreci tamamlanır.

Uygulama katmanında hangi birimin gelen bir mesajı alması gerektiğini bel irlemek, taşıma katmanının önemli bir görevidir. Bu, çeşitli birimlere benzersiz port numaraları atayarak (Bölüm 2'de tartışılan G/Ç portlarıyla ilgili olmayan) v e mesajın yolculuğuna başlamadan önce uygun port numarasının mesajın adre sine eklenmesini gerektirerek gerçekleştirilir. Daha sonra, mesaj varış noktasın daki taşıma katmanı tarafından alındığında, taşıma katmanı mesajı yalnızca bel irlenen port numarasındaki uygulama katmanı yazılımına teslim eder.

İnternet kullanıcıları, yaygın uygulamaların evrensel olarak kabul edilen port nu maralarına sahip olması nedeniyle, port numaralarıyla nadiren ilgilenmek zorundadı r. Örneğin, bir Web tarayıcısından URL'si http://www.zoo.org/animals/frog.html olan belgeyi alması istendiğinde, tarayıcı bunun HTTP sunucusuna `www.zoo.org` üzerind en 80 numaralı port aracılığıyla ulaşması gerektiğini varsayar. Benzer şekilde, bir dos ya aktarırken, bir FTP istemcisi FTP sunucusuyla 20 ve 21 numaralı portlar aracılığıyla iletişim kurması gerektiğini varsayar.

Özetle, İnternet üzerinden iletişim, dört yazılım katmanının etkileşimini i çerir. Uygulama katmanı, uygulamanın bakış açısından gelen mesajlarla ilgil enir. Taşıma katmanı, bu mesajları İnternet ile uyumlu segmentlere dönüştü rür ve alınan mesajları uygun uygulamaya teslim etmeden önce yeniden bir araya getirir. Ağ katmanı, segmentlerin İnternet üzerinden yönlendirilmesiyl e ilgilenir. Bağlantı katmanı, segmentlerin bir makineden diğerine gerçek ilet imini yönetir. Tüm bu etkinliklerle, İnternet'in yanıt süresinin milisaniyelerle ölçülmesi oldukça şaşırtıcıdır, bu nedenle birçok işlem anında gerçekleşiyor muş gibi görünmektedir.

TCP/IP Protokol Takımı

Açık ağlara olan talep, üreticilerin diğer satıcıların ürünleriyle düzgün bir şeki lde çalışan ekipman ve yazılım sağlayabileceği yayımlanmış standartlara ihtiy aç doğurmuştur. Ortaya çıkan bir standart, Açık Sistemler Arası Bağlantı (OSI) referans modeli, Uluslararası Standartlar Örgütü tarafından üretilmiştir. Bu standart, daha önce tanımladığımız dört seviyeli hiy erarşinin aksine yedi seviyeli bir hiyerarşeye dayanmaktadır. Uluslararası bir ku ruluşun otoritesini taşıdığı için sıkça alıntı yapılan bir modeldir, ancak dört sevi yeli bakış açısını değiştirmekte yavaş kalmıştır; bunun başlıca nedeni, dört seviy eli hiyerarşinin zaten Internet için de facto standart haline gelmesinden sonra kurulmuş olmasıdır.

TCP/IP protokol yelpazesi, İnternet tarafından uygulanan dört seviyeli ile tişim hiyerarşisini gerçekleştirmek için kullanılan protokol standartlarının bir koleksiyonudur. Aslında, İletim Kontrol Protokolü (TCP) ve İnternet Protokol ü (IP) bu geniş koleksiyondaki yalnızca iki protokolün adıdır—bu nedenle, tü m koleksiyona TCP/IP protokol yelpazesi denmesi oldukça yanıltıcıdır. Daha kesin olarak, TCP, taşıma katmanının bir versiyonunu tanımlar. *versiyon* de memizin nedeni, TCP/IP protokol yelpazesinin taşıma katmanını uygulamanı n birden fazla yolunu sağlamasıdır; diğer seçeneklerden biri Kullanıcı Datagr am Protokolü (UDP) tarafından tanımlanmıştır. Bu çeşitlilik, bir paketi gönde rirken farklı nakliye şirketleri seçme seçeneğinizin olmasıyla benzerlik göster ir; her biri aynı temel hizmeti sunar ancak kendi benzersiz özellikleriyle.

Doclingo https://www.doclingo.ai

karakteristikler. Bu nedenle, gereken belirli hizmet kalitesine bağlı olarak, uygulama katmanındaki bir birim, verileri taşıma katmanının TCP veya UDP sürümü aracılığıyla göndermeyi seçebilir (Figure 4.15).

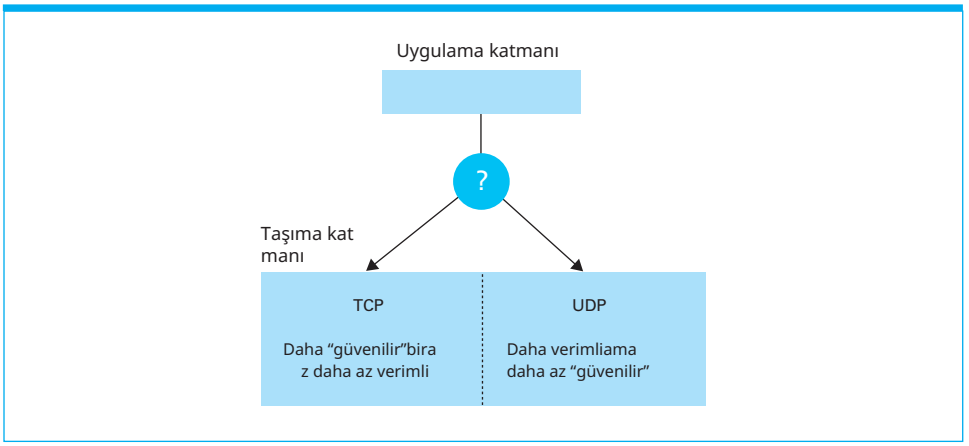
TCP ve UDP arasında birkaç fark vardır. Birincisi, uygulama katmanı taraf ından istenen bir mesajı göndermeden önce, TCP tabanlı bir taşıma katmanı, hedefteki taşıma katmanına bir mesaj gönderir ve bir mesajın gönderileceği ni bildirir. Daha sonra, uygulama katmanının mesajını göndermeye başlama dan önce bu mesajın onaylanmasını bekler. Bu şekilde, TCPtaşıma katmanını n bir mesaj göndermeden önce bir bağlantı kurduğu söylenir. UDP tabanlı bi r taşıma katmanı ise,bir mesaj göndermeden önce böyle bir bağlantı kurmaz . Sadece verilen adrese mesajı gönderir ve onu unutur.Bildiği kadarıyla, hede f bilgisayar çalışmıyor bile olabilir. Bu nedenle, UDP bağlantısız bir protokol olarak adlandırılır.

TCP ve UDP arasındaki bir diğer fark, köken ve hedefteki TCP taşıma kat manlarının,mesajın tüm segmentlerinin başarıyla hedefe iletildiğinden emin olmak için onaylar ve paket yeniden iletimleri aracılığıyla birlikte çalışmasıdır . Bu nedenle TCP güvenilir bir protokol olarak adlandırılır,UDP ise böyle bir y eniden iletim hizmeti sunmadığı için güvenilmez bir protokol olarak adlandır ılır.

TCP ve UDP arasındaki bir diğer ayrım, TCP'nin hem akış kontrolü sağladı ğı, yani bir mesajın kaynağındaki TCP taşıma katmanının, hedefteki karşı tara fı bunaltmamak için segmentleri iletme hızını azaltabileceği, hem de yoğunlu k kontrolü sağladığı, yani bir mesajın kaynağındaki TCP taşıma katmanının, il etim hızını ayarlayarak kendisi ile mesajın hedefi arasındaki yoğunluğu hafifl etebileceğidir.

Bütün bunlar, UDP'nin kötü bir seçim olduğu anlamına gelmez. Sonuçta, UDP'ye da yalı bir taşıma katmanı, TCP'ye dayalı bir katmandan daha akıcıdır ve dolayısıyla bir uyg ulama, UDP'nin potansiyel sonuçlarını yönetmeye hazırsa, bu seçenek daha iyi bir tercih olabilir. Örneğin, UDP'nin verimliliği, onu DNS sorguları ve VoIP için tercih edilen proto kol haline getirir. Ancak, e-posta zaman açısından daha az hassas olduğundan, posta su nucuları e-posta iletimi için TCP kullanır.

Şekil **4.15**TCP ve UDP arasında seçim yapmak



Doclingo https://www.doclingo.ai

IP, ağ katmanına atanan görevleri yerine getirmek için İnternet'in standartı dır. Bu görevin, İnternet üzerinden paketlerin iletilmesini içeren iletim ve katm anın iletim tablosunu değişen koşulları yansıtacak şekilde güncellemeyi içeren yönlendirme olduğunu zaten gözlemledik. Örneğin, bir yönlendirici arızalanabil ir, bu da trafiğin artık onun yönüne iletilmemesi gerektiği anlamına gelir veya İ nternet'in bir bölümü yoğunlaşabilir, bu da trafiğin tıkanıklığın etrafında yönle ndirilmesi gerektiği anlamına gelir. Yönlendirme ile ilgili IP standardının büyük bir kısmı, komşu ağ katmanları arasında yönlendirme bilgilerini değiş tokuş ed erken kullanılan protokollerle ilgilidir.

İleri yönlendirme ile ilişkili ilginç bir özellik, bir mesajın kaynağındaki IP a ğ katmanı her paket hazırladığında, o pakete bir hop sayısı veya yaşam süres i olarak adlandırılan bir değer eklemesidir. Bu değer, paketin İnternet'te yolu nu bulmaya çalışırken yönlendirilmesi gereken maksimum sayıyı belirler. Bir IP ağ katmanı her paketi yönlendirdiğinde, o paketin hop sayısını bir azaltır. Bu bilgi ile ağ katmanı, İnternet'i sistem içinde sonsuz döngüde dolaşan pak etlerden koruyabilir. İnternet her gün büyümeye devam etse de, 64'lük bir b aşlangıç hop sayısı, günümüzün İnternet Servis Sağlayıcıları içindeki yönlend iriciler labirentinde bir paketin yolunu bulması için yeterlidir.

Yıllardır, IPv4 (IP versiyon dört) olarak bilinen bir IP versiyonu, İnternet'te ki ağ katmanını uygulamak için kullanılmaktadır. Ancak, İnternet hızla IPv4 t arafından belirlenen 32 bitlik internet adresleme sistemini aşmaktadır. Bu so runu çözmek ve çoklu yayın gibi diğer iyileştirmeleri uygulamak için, 128 bitli k internet adresleri kullanan IPv6 olarak bilinen yeni bir IP versiyonu oluştur ulmuştur. IPv4'ten IPv6'ya geçiş süreci şu anda devam etmektedir—bu, Bölü m 4.2'de internet adreslerinin tanıtımında bahsedilen dönüşümdür—ve 2025 yılına kadar İnternet'te 32 bitlik adreslerin kullanımının sona ereceği beklen mektedir.

Sorular & Alıştırmalar

1. Bir yönlendiricide hangi İnternet yazılım hiyerarşisi katmanları gerekli de ğildir?
2. TCP protokolüne dayanan bir taşıma katmanı ile UDP protokolüne dayanan bir taşıma katmanı arasındaki bazı farklar nelerdir?
3. Taşıma katmanı, uygulama katmanındaki hangi birimin gelen bir me sajı alması gerektiğini nasıl belirler?
4. Bir bilgisayarın İnternet'te, üzerinden geçen tüm mesajların kopyalarını kaydetmesini ne engeller?

4.5 Güvenlik

Bir bilgisayar bir ağa bağlandığında, yetkisiz erişim ve vandalizme maruz kalı r. Bu bölümde bu sorunlarla ilişkili konuları ele alıyoruz.

Doclingo https://www.doclingo.ai

Bilgisayar Acil Durum Müdahale Ekibi

Kasım 1988'de internete salınan bir solucan, hizmette önemli kesintilere neden oldu. Son uç olarak, ABD Savunma İleri Araştırma Projeleri Ajansı (DARPA—"DAR–pa" olarak telaffu z edilir) Bilgisayar Acil Durum Müdahale Ekibini (CERT—"SERT" olarak telaffuz edilir) oluşt urdu; bu ekip Carnegie-Mellon Üniversitesi'ndeki CERT Koordinasyon Merkezi'nde bulun maktadır. CERT, internetin güvenlik "gözcüsü"dür. Görevleri arasında güvenlik sorunları nın araştırılması, güvenlik uyarılarının yayınlanması ve internet güvenliğini artırmak için kamu bilinci kampanyalarının uygulanması yer almaktadır. CERT Koordinasyon Merkezi, faaliyetlerinin bildirimlerini yayınladığı http://www.cert.org adresinde bir web sitesi sürd ürmektedir.

Saldırı Biçimleri

Bir bilgisayar sistemi ve içeriği, ağ bağlantıları aracılığıyla birçok şekilde saldı rıya uğrayabilir. Bunların çoğu, kötü amaçlı yazılım (toplu olarak kötü amaçlı yazılım olarak adlandırılır) kullanımını içerir. Bu tür yazılımlar, bilgisayara akt arılabilir ve bilgisayarın kendisinde çalıştırılabilir veya uzaktan bilgisayara sal dırabilir. Saldırı altındaki bilgisayara aktarılan ve çalıştırılan yazılımlara örnek ler arasında virüsler, solucanlar, Truva atları ve casus yazılımlar bulunur; bu yazılımların isimleri, yazılımın temel özelliğini yansıtır.

Bir virüs, kendisini makinede zaten bulunan programlara yerleştirerek bir bilg isayarı enfekte eden yazılımdır. Daha sonra, "ana" program çalıştırıldığında, virüs de çalıştırılır. Çalıştırıldığında, birçok virüs, kendilerini bilgisayardaki diğer progra mlara aktarmaktan başka pek bir şey yapmaz. Ancak bazı virüsler, işletim sistemi nin bazı bölümlerini degrade etme, büyük veri depolama alanlarını silme veya ver ileri ve diğer programları bozulma gibi yıkıcı eylemler gerçekleştirir.

A solucan, bir ağa üzerinden kendini transfer eden, bilgisayarlara yerleşen ve kendisinin kopyalarını diğer bilgisayarlara ileten otonom bir programdır.Bir virüs durumunda olduğu gibi, bir solucan yalnızca kendini çoğaltmak veya dah a aşırı vandalizm yapmak için tasarlanabilir. Bir solucanın karakteristik bir sonu cu, meşru uygulamaların performansını düşüren ve nihayetinde tüm bir ağı ve ya interneti aşırı yükleyebilen solucanın çoğaltılmış kopyalarının patlamasıdır.

A Truva atı, istenen bir program, örneğin bir oyun veya yararlı bir yardım cı program paketi gibi, bir bilgisayar sistemine gizlenmiş olarak giren bir pro gramdır ve bu program kurban tarafından isteyerek içe aktarılır. Ancak bilgis ayara girdikten sonra, Truva atı zararlı etkileri olabilecek ek faaliyetler gerçek leştirir. Bazen bu ek faaliyetler hemen başlar. Diğer durumlarda, Truva atı be lirli bir olay tarafından tetiklenene kadar bekleyebilir, örneğin önceden belirl enmiş bir tarihin gerçekleşmesi gibi. Truva atları genellikle çekici e-posta me sajlarına ek olarak gelir. Ek açıldığında (yani, alıcı ekin görüntülenmesini iste diğinde), Truva atının kötü niyetli eylemleri etkinleştirilir.
Böylece, bilinmeyen kaynaklardan gelen e-posta ekleri asla açılmamalıdır.

Diğer bir kötü amaçlı yazılım türü, **spyware** (bazen sniffingsoftware olarak adlandırılır) olup, bulunduğu bilgisayardaki etkinlikler hakkında bilgi toplayan v e bu bilgiyi saldırının düzenleyicisine geri raporlayan bir yazılımdır. Bazı şirketle r, müşteri profilleri oluşturmak amacıyla spyware kullanmaktadır ve bu bağlam da etik açıdan sorgulanabilir bir değere sahiptir. Diğer durumlarda, spyware

Doclingo https://www.doclingo.ai

parola veya kredi kartı numaralarını aramak amacıyla bilgisayarın klavyesinde yaz ılan sembol dizilerini kaydetmek gibi açıkça kötü niyetli amaçlar için kullanılır. Casus yazılımlar aracılığıyla gizlice bilgi edinmenin aksine, **phishing**, bilgiyi açıkça talep ederek elde etme tekniğidir. Phishing terimi, sürecin birçok "ip" atarak birinin "yemi" almasını umduğu için *fishing* kelimesinin bir oyunudur. Phishing genellikle e-posta yoluyla gerçekleşti rilir ve bu biçimde, eski bir telefon dolandırıcılığından başka bir şey değildir. Fail, bir finans kurumu, bir devlet dairesi veya belki de bir kolluk kuvveti gibi davranarak e-posta mesajları gönderir. E-posta, potansiyel kurbandan, meşr u amaçlar için gerekli olduğu iddia edilen bilgileri talep eder. Ancak, elde edil en bilgiler fail tarafından düşmanca amaçlar için kullanılır.

Virüsler ve casus yazılımlar gibi içsel enfeksiyonlardan muzdarip olmanın aksine,bir ağdaki bir bilgisayar, sistemdeki diğer bilgisayarlarda yürütülen y azılımlar tarafından da saldırıya uğrayabilir. Bir örnek, bir bilgisayarı mesajla rla aşırı yükleme süreci olan**denial of service (DoS)** saldırısıdır. Denial of ser vicesaldırıları, bir şirketin işini kesintiye uğratmak için İnternet'teki büyük tic ari Web sunucularına karşı başlatılmıştır ve bazı durumlarda şirketin ticari fa aliyetlerini durma noktasına getirmiştir.

Hizmet reddi saldırısı, kısa bir süre içinde büyük sayıda mesaj üretilmesi ni gerektirir. Bunu başarmak için, bir saldırgan genellikle birçok habersiz bilg isayara yazılım yerleştirir ve bu yazılım bir sinyal verildiğinde mesajlar üretir. Ardından, sinyal verildiğinde, bu bilgisayarların hepsi hedefi mesajlarla boğa r. Hizmet reddi saldırılarında, habersiz bilgisayarların suç ortağı olarak kulla nılabilirliği doğaldır. Bu nedenle, tüm PC kullanıcılarının bilgisayarlarını kulla nılmadığında İnternet'e bağlı bırakmamaları teşvik edilmektedir.
Bir PC'nin İnternet'e bağlandığında, en az bir saldırganın varlığını 20 dakika için de istismar etmeye çalışacağı tahmin edilmektedir. Bu durumda, korunmasız bi r PC, İnternet'in bütünlüğü için önemli bir tehdit oluşturmaktadır.

İstenmeyen mesajların bolluğu ile ilişkili bir diğer sorun, istenmeyen gerek siz e-postaların, yani spam'in çoğalmasıdır. Ancak, hizmet reddi saldırısının aksi ne, spam hacmi genellikle bilgisayar sistemini aşacak kadar yeterli değildir. Bu nun yerine, spam'in etkisi, spam alan kişiyi bunaltmaktır. Bu sorun, daha önce gördüğümüz gibi, spam'in kimlik avı ve virüsler ile diğer zararlı yazılımları yaya bilecek Truva atlarını kışkırtmak için yaygın olarak benimsenmiş bir ortam olma sı gerçeğiyle daha da karmaşık hale gelmektedir.

Koruma ve Tedaviler

"Bir ons önlem, bir pound tedaviye bedeldir" eski atasözü, ağ bağlantıları üze rinden vandalizmi kontrol etme bağlamında kesinlikle doğrudur.Birincil önle me tekniği, genellikle bir güvenlik duvarı olarak adlandırılan bir programla, ağdaki bir noktadan geçen trafiği filtrelemektir.Örneğin, bir güvenlik duvarı, bir kuruluşun intranetinin kapısında, bölgeden giren ve çıkan mesajları filtrel emek için kurulabilir.Bu tür güvenlik duvarları, belirli hedef adreslere sahip g iden mesajları engellemek veya sorun kaynakları olarak bilinen yerlerden gel en gelen mesajları engellemek üzere tasarlanmış olabilir.Bu son işlev, saldır gan bilgisayarlardan gelen trafiği engelleme aracı olarak bir hizmet reddi sal dırısını sonlandırmak için bir araçtır.Bir güvenlik duvarının kapıdaki diğer yay gın rolü, kapıdan erişilen bölgedeki köken adreslerine sahip tüm gelen mesa jları engellemektir çünkü böyle bir mesaj, bir dış kişinin

Doclingo https://www.doclingo.ai

iç bölgenin bir üyesiymiş gibi davrandığını gösterir. Başka birinin kimliğine b ürünmek, sahtecilik olarak bilinir.

Güvenlik duvarları, tüm ağlar veya alanlar yerine bireysel bilgisayarları korumak içi n de kullanılır. Örneğin, bir bilgisayar bir Web sunucusu, bir ad sunucusu veya bir e-pos ta sunucusu olarak kullanılmıyorsa, o bilgisayarda bu tür uygulamalara yönlendirilmiş t üm gelen trafiği engellemek için bir güvenlik duvarı kurulmalıdır. Gerçekten de, bir sald ırganın bir bilgisayara girmesinin bir yolu, var olmayan bir sunucu tarafından bırakılan bir "delik" aracılığıyla iletişim kurmaktır. Özellikle, casus yazılım tarafından toplanan bilg ileri geri almak için bir yöntem, enfekte olmuş bilgisayarda kötü niyetli istemcilerin casu s yazılımın bulgularını alabileceği gizli bir sunucu kurmaktır. Doğru bir şekilde kurulmuş bir güvenlik duvarı, bu kötü niyetli istemcilerden gelen mesajları engelleyebilir.

Bazı güvenlik duvarı çeşitleri belirli amaçlar için tasarlanmıştır—örneğin, ist enmeyen e-postaları engellemek için tasarlanmış güvenlik duvarları olan spam filtreleri. Birçok spam filtresi, istenen e-posta ile spam arasında ayrım yapmak i çin oldukça sofistike teknikler kullanır. Bazıları, kullanıcının spam öğelerini tanı mladığı bir eğitim süreci aracılığıyla bu ayrımı yapmayı öğrenir, böylece filtre k endi başına karar vermek için yeterli örnek edinir. Bu filtreler, çeşitli konu alanl arının (olasılık teorisi, yapay zeka vb.) diğer alanlardaki gelişmelere nasıl katkıd a bulunabileceğinin örnekleridir.

Filtreleme çağrışımları olan bir diğer önleyici araç ise proxy sunucusudur. Bir proxy sunucusu, bir istemci ile bir sunucu arasında aracılık yapan bir yazılım birimidir ve amacı istemciyi sunucunun olumsuz eylemlerinden korumaktır. Bi r proxy sunucusu olmadan, bir istemci doğrudan bir sunucu ile iletişim kurar, b u da sunucunun istemci hakkında belirli bir miktar bilgi edinme fırsatına sahip olduğu anlamına gelir. Zamanla, bir organizasyonun intranetindeki birçok iste mci uzak bir sunucu ile işlem yaptıkça, o sunucu intranetin iç yapısı hakkında ço k sayıda bilgi toplayabilir—bu bilgi daha sonra kötü niyetli faaliyetler için kullan ılabilir. Bunu karşılamak için, bir organizasyon belirli bir tür hizmet (FTP, HTTP, telnet vb.) için bir proxy sunucusu kurabilir. O zaman, intranet içindeki her iste mci o tür bir sunucu ile iletişim kurmaya çalıştığında, istemci aslında proxy sun ucusu ile iletişime geçer. Ardından, proxy sunucusu, bir istemci rolünü üstlener ek gerçek sunucu ile iletişim kurar. O zamandan sonra proxy sunucusu, gerçek istemci ile gerçek sunucu arasında mesajları ileterek aracılık rolünü üstlenir.Bu düzenlemenin ilk avantajı, gerçek sunucunun proxy sunucusunun gerçek istem ci olmadığını bilme şansının olmamasıdır ve aslında, gerçek istemcinin varlığın dan asla haberdar olmaz. Bununla birlikte, gerçek sunucunun intranetin iç özell ikleri hakkında bilgi edinme şansı yoktur. İkinci avantaj ise proxy sunucusunun, sunucudan istemciye gönderilen tüm mesajları filtreleme pozisyonunda olmas ıdır. Örneğin, bir FTP proxy sunucusu, gelen tüm dosyaları bilinen virüslerin var lığı açısından kontrol edebilir ve tüm enfekte dosyaları engelleyebilir.

Ağ ortamında sorunları önlemek için bir diğer araç, işletim sistemi güven liği (Bölüm 3.5) konusundaki tartışmamızda öğrendiğimiz denetim yazılımına benzer denetim yazılımıdır. Ağ denetim yazılımı kullanarak, bir sistem yöneti cisi, yöneticinin alanındaki çeşitli yerlerde mesaj trafiğinde ani bir artışı tespit edebilir, sistemin güvenlik duvarlarının aktivitelerini izleyebilir ve bireysel bil gisayarlar tarafından yapılan isteklerin desenini analiz ederek anormallikleri tespit edebilir. Aslında, denetim yazılımı, sorunları kontrolden çıkmadan önc e tanımlamak için bir yöneticinin birincil aracıdır.

Ağ bağlantıları aracılığıyla yapılan saldırılara karşı bir diğer savunma aracı, bilinen virüslerin ve diğer enfe ksiyonların varlığını tespit etmek ve kaldırmak için kullanılan antivirus yazılımı olarak adlandırılan yazılımdır.

Doclingo https://www.doclingo.ai

Antivirus yazılımı, her biri belirli bir enfeksiyon türünü tespit etmek ve kaldır mak için tasarlanmış geniş bir yazılım ürünleri sınıfını temsil eder. Örneğin, b irçok ürün virüs kontrolüne odaklanırken, diğerleri spyware korumasına oda klanmaktadır. Bu paketlerin kullanıcılarının, biyolojik sistemlerde olduğu gibi , sürekli olarak ortaya çıkan yeni bilgisayar enfeksiyonlarının güncellenmiş aş ılar gerektirdiğini anlamaları önemlidir.
Bu nedenle, antivirüs yazılımı, yazılımın satıcısından güncellemeler indirilerek d üzenli olarak bakım yapılmalıdır.Ancak bu bile bir bilgisayarın güvenliğini garan ti etmez. Sonuçta, yeni bir virüsün keşfedilmesi ve bir aşı üretilmesi için önce b azı bilgisayarlara bulaşması gerekir. Bu nedenle, akıllı bir bilgisayar kullanıcısı, t anımadığı kaynaklardan gelen e-posta eklerini asla açmaz, yazılımı güvenilirliği ni doğrulamadan indirmez, açılır pencerelere yanıt vermez ve böyle bir bağlant ı gerekli olmadığında bir PC'yi internete bağlı bırakmaz.

Şifreleme

Bazı durumlarda ağ vandalizminin amacı sistemi bozmak (hizmet reddi saldı rıları gibi), ancak diğer durumlarda nihai hedef bilgiye erişim sağlamaktır. Bil giyi korumanın geleneksel yolu, erişimini şifreler kullanarak kontrol etmektir . Ancak, şifreler tehlikeye atılabilir ve veriler bilinmeyen varlıklar tarafından il etildiğinde ağlar ve internetler üzerinden aktarıldığında pek bir değer taşıma z. Bu durumlarda, veriler kötü niyetli ellere geçse bile, şifrelenmiş bilginin giz li kalmasını sağlamak için şifreleme kullanılabilir. Bugün, birçok geleneksel İ nternet uygulaması, şifreleme tekniklerini içerecek şekilde değiştirilmiştir ve bu uygulamaların “güvenli sürümleri” olarak adlandırılan versiyonları üretilm iştir. Örnekler arasında, FTP'nin güvenli bir sürümü olan FTPS ve bölüm 4.2'd e telnet için güvenli bir alternatif olarak tanıttığımız SSH bulunmaktadır.

Bir diğer örnek, HTTPS olarak bilinen güvenli HTTP sürümüdür, ki bu da çoğu finansal kurum tarafından müşterilere hesaplarına güvenli İnternet erişimi sağlamak için kullanılmaktadır. HTTPS'nin belkemiği, Web istemcileri ile sunucular arasında güvenli iletişim bağlantıları sağlamak için Netscape tarafından ilk olarak geliştirilen Güvenli Yuva Katmanı (SSL) adlı protokol sistemidir. Çoğu tarayıcı, bilgisayar ekranında küçük bir kilit simgesi göstererek SSL kullanımını belirtir. (Bazıları, SSL'nin kullanılıp kullanılmadığını belirtmek için simgenin varlığını vey a yokluğunu kullanır;diğerleri ise kilidi ya kilitli ya da kilitsiz pozisyonda g österir.) Şifreleme alanındaki daha ilginç konulardan biri, açık anahtar şifrelemesidir, bu da şifreleme sistemlerinin tasarlandığı teknikleri içerir; böy lece mesajların nasıl şifrelendiği hakkında bilgi sahibi olmak,mesajları şifrele rini çözmeye olanak tanımaz. Bu özellik biraz sezgisel değildir.
Sonuçta, sezgi, bir kişinin mesajların nasıl şifrelendiğini bilmesi durumunda,o kişi nin şifreleme sürecini tersine çevirebileceğini ve böylece mesajları çözebileceğini öne sürer. Ancak açık anahtar şifreleme sistemleri,bu sezgisel mantığı çiğner.

Açık anahtar şifreleme sistemi, anahtarlar olarak adlandırılan iki değerin kullanımını içerir. Bir anahtar, açık anahtar olarak bilinir, mesajları şifrelemek için kullanılır; di ğer anahtar, özel anahtar olarak bilinir, mesajları şifre çözmek için gereklidir. Sistemi kullanmak için,açık anahtar öncelikle belirli bir hedefe mesaj gönder mesi gerekenlere dağıtılır. Özel anahtar bu hedefte gizli tutulur.
Daha sonra, bir mesajın oluşturucusu mesajı açık anahtarı kullanarak şifreleyebilir ve mesajı içeriğinin güvende olduğuna dair güvenle hedefe gönderebilir,

Doclingo https://www.doclingo.ai

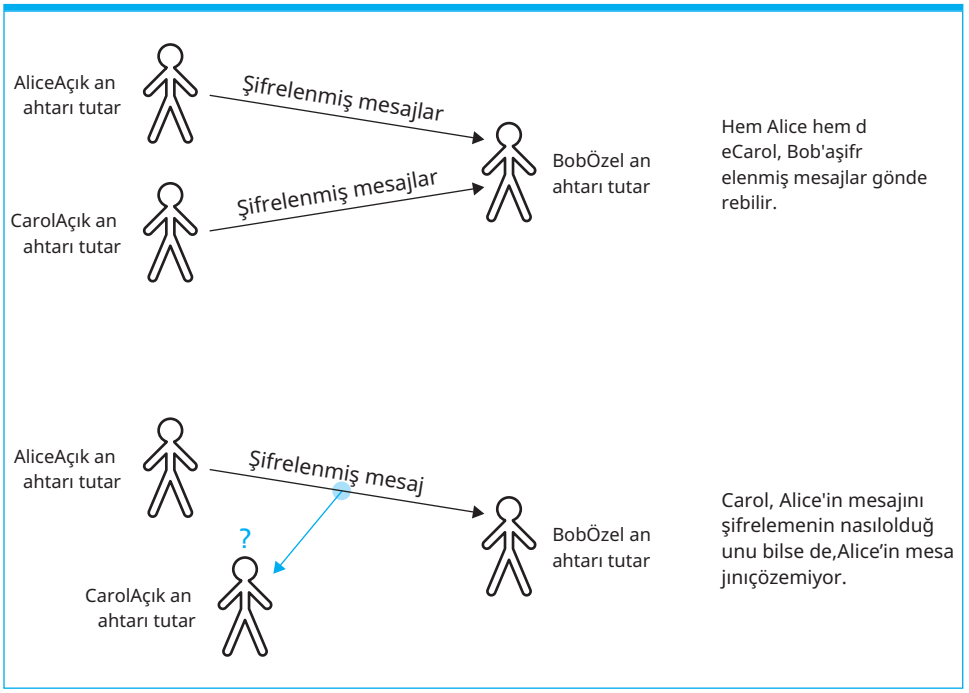
Oldukça İyi Gizlilik

İnternette kullanılan en popüler açık anahtar şifreleme sistemlerinden bazıları,Ron Rives t, Adi Shamir ve Len Adleman'ın adını taşıyan RSA algoritmasınadayanmaktadır. Bunu Bö lüm 12'nin sonunda detaylı olarak tartışacağız. RSA teknikleri(başka yöntemlerin yanı sır a) PGP Corporation tarafından üretilen bir dizi yazılım paketindekullanılmaktadır. PGP, P retty Good Privacy'nin kısaltmasıdır. Bu paketler, PC'lerde kullanılançoğu e-posta yazılım ıyla uyumludur ve kişisel, ticari olmayan kullanım içinhttp://www.pgp.com adresinden üc retsiz olarak temin edilebilir. PGP yazılımını kullanarak,bir birey açık ve özel anahtarlar ol uşturabilir, mesajları açık anahtarlarla şifreleyebilirve özel anahtarlarla mesajları çözebili r.

aracılar tarafından da işleniyor olsa bile, açık anahtarı bilen. Gerçekten de,m esajı şifre çözebilen tek taraf, mesajın hedefindeki ve özel anahtarı tutan tara fdır. Böylece Bob bir açık anahtar şifreleme sistemi oluştursave hem Alice' e hem de Carol'a açık anahtarı verirse, o zaman hem Alice hem de Carol Bob' amesajlar şifreleyebilir, ancak diğerinin iletişimini gözetleyemezler.
Gerçekten de, Carol Alice'ten bir mesajı keserse, Alice'in nasıl şifrelediğini bilse bile onu şifre çözemeyecektir (Şekil 4.16).

Elbette, açık anahtar sistemlerinde gizli sorunlar vardır.
Bunlardan biri, kullanılan açık anahtarın, aslında, hedef taraf için doğru anah tar olduğunu sağlamaktır. Örneğin, eğer bankanızla iletişim kuruyorsanız,

Şekil **4.16**Açık anahtar şifrelemesi



Doclingo https://www.doclingo.ai

kullanmakta olduğunuz kamu anahtarının bankaya ait olduğundan ve bir sahtekarın anaht ar olmadığından emin olmak istiyorsunuz. Eğer bir sahtekar kendisini banka olarak tanıtırs a (bir sahtecilik örneği) ve size kendi kamu anahtarını verirse, şifrelediğiniz ve "banka"ya gö nderdiğiniz mesajlar sahtekara anlamlı olur ve sizin bankanıza değil. Bu nedenle, kamu an ahtarlarını doğru taraflarla ilişkilendirme görevi önemlidir.

Bu sorunu çözmenin bir yaklaşımı, güvenilir internet siteleri kurmaktır,sertifik a otoriteleri olarak adlandırılan, görevleri tarafların ve onların açık anahtarlarının doğru listelerinikorumaktır. Bu otoriteler, sunucu olarak hareket ederek, müşteril erinesertifika olarak bilinen paketler halinde güvenilir açık anahtar bilgileri sağlar. Bir sertifika, bir tarafın adını ve o tarafın açık anahtarını içeren bir pakettir. Şu anda internette birçok ticari sertifika otoritesi mevcuttur,ancak organizas yonların kendi sertifika otoritelerini sürdürmesi de yaygındır,organizasyonu n iletişiminin güvenliği üzerinde daha sıkı kontrol sağlamak için.

Son olarak, kamu anahtarı şifreleme sistemlerininkimlik doğrulama soru nlarını çözmedeki rolü hakkında yorum yapmalıyız—bir mesajın yazarının,ge rçekten iddia ettiği taraf olup olmadığını sağlamak. Buradaki kritik nokta,baz ı kamu anahtarı şifreleme sistemlerinde, şifreleme ve çözme anahtarlarınınr ollerinin tersine çevrilebileceğidir. Yani, metin özel anahtar ile şifrelenebilir v esadece bir taraf o anahtara erişebildiğinden, böyle şifrelenmiş herhangi bir metinmutlaka o taraftan gelmiş olmalıdır. Bu şekilde, özel anahtarın sahibi,ü retmeyi yalnızca o tarafın bildiği bir bit deseni, yani dijital imza, üretebilir.O i mzayı bir mesaja ekleyerek, gönderici mesajı otantik olarak işaretleyebilir.Dij ital imza, mesajın kendisinin şifrelenmiş versiyonu kadar basit olabilir.Gönde ricinin yapması gereken tek şey, iletilen mesajı kendi özel anahtarıylaşifrele mektir (genellikle çözme için kullanılan anahtar). Mesaj alındığında,almacı, g öndericinin açık anahtarını kullanarak imzayı çözer.Açığa çıkan mesajın otant ik olması garanti edilir çünkü yalnızca özel anahtarın sahibi,şifrelenmiş versiy onu üretebilirdi.

Ağ Güvenliği için Hukuki Yaklaşımlar

Bilgisayar ağ sistemlerinin güvenliğini artırmanın bir diğer yolu hukuki çözü mler uygulamaktır. Ancak, bu yaklaşımın iki engeli vardır. Birincisi, bir eylemi yasadışı hale getirmenin o eylemi engellememesidir. Yalnızca hukuki bir baş vuru sağlar. İkincisi, ağların uluslararası doğası nedeniyle, başvuru elde etm enin genellikle çok zor olmasıdır. Bir ülkede yasadışı olan bir şey, başka bir ül kede yasal olabilir. Sonuç olarak, ağ güvenliğini hukuki yollarla artırmak ulus lararası bir projedir ve bu nedenle uluslararası hukuki kuruluşlar tarafından ele alınmalıdır; potansiyel bir oyuncu, Lahey'deki Uluslararası Adalet Divanı o labilir.

Bu feragatnameleri yaptıktan sonra, hukuki güçlerin mükemmel olmaktan uzak olmasına rağmen hala büyük bir etkiye sahip olduğunu kabul etmeliyiz ve bu nedenle ağ alanındaki çatışmaları çözmek için atılan bazı hukuki adımları ke şfetmek bizim için önemlidir. Bu amaçla, Amerika Birleşik Devletleri'nin federal yasalarından örnekler kullanıyoruz. Benzer örnekler, Avrupa Birliği gibi diğer h ükümet organlarından da alınabilir.

Malware'in yayılması ile başlıyoruz. Amerika Birleşik Devletleri'nde bu sorun, ilk ola rak kabul edilen Bilgisayar Dolandırıcılığı ve Kötüye Kullanım Yasası ile ele alınmaktadır.

Doclingo https://www.doclingo.ai

1984, birkaç kez değişikliğe uğramış olmasına rağmen. Bu yasa altında, solu canların ve virüslerin tanıtımını içeren çoğu dava kovuşturulmuştur. Kısacası, yasa, sanığın bilerek bir programın veya verinin iletimine neden olduğunu k anıtlamayı gerektirir; bu da kasıtlı olarak zarar vermiştir.

Bilgisayar Dolandırıcılığı ve Kötüye Kullanım Yasası, bilgi hırsızlığını içeren dav aları da kapsamaktadır. Özellikle, yasa, bir bilgisayara yetkisiz erişim yoluyla değe rli bir şey elde etmeyi yasaklamaktadır. Mahkemeler, "değerli bir şey" ifadesine ge niş bir yorum getirmeye eğilimli olmuşlardır ve bu nedenle Bilgisayar Dolandırıcılı ğı ve Kötüye Kullanım Yasası, bilgi hırsızlığından daha fazlasına uygulanmıştır. Ör neğin, mahkemeler, bir bilgisayarın basit kullanımının "değerli bir şey" olarak kab ul edilebileceğine hükmetmişlerdir. Gizlilik hakkı, belki de hukuki topluluğun k arşılaştığı en tartışmalı ağ sorunlarından biridir. Bir işverenin çalışanların iletişiml erini izleme hakkı ve bir internet servis sağlayıcısının müşterileri tarafından iletile n bilgilere erişim yetkisi konusundaki sorular üzerinde önemli düşünceler yapılmı ştır. Amerika Birleşik Devletleri'nde, bu soruların birçoğu 1986 tarihli Elektronik İl etişim Gizliliği Yasası (ECPA) ile ele alınmaktadır; bu yasa, dinlemeyi kontrol etmek için çıkarılan yasaların kökenine sahiptir. Yasa uzun olmasına rağmen, amacı birk aç kısa alıntıda özetlenmiştir. Özellikle, şunu belirtmektedir ki

> Bu bölümde aksi belirtilmedikçe, herhangi bir kişi, kasıtlı olarak herhangi bir tel, sözlü veya elektronik iletişimi kesmeye, kesmeye çalışmaya veya başka bir kişiyi kesmeye veya kesmeye çalışmaya teşvik ederse, . . . (4) numaralı alt maddeye göre cezalandırılacak veya (5) numaralı alt maddeye göre dava açılacaktır.

ve

> . . . kamuya elektronik iletişim hizmeti sağlayan herhangi bir kişi veya kuruluş, kasıtlı olarak bu hizmetteki herhangi bir iletişimin içeriğini, . . . yalnızca bu iletişimin alıcı veya hedef alıcısı olan herhangi bir kişi veya kuruluş dışında, bu alıcının veya hedef alıcısının bir temsilcisine ifşa etmeyecektir.

Kısacası, ECPA, bireylerin özel iletişim hakkını onaylamaktadır—bir İnternet h izmetsağlayıcısının, müşterilerinin iletişimi hakkında bilgi vermesi yasaktır ve yetkisizkişilerin başkalarının iletişimini dinlemesi yasaktır. Ancak ECPA tartış ma içinalan bırakmaktadır. Örneğin, bir işverenin çalışanların iletişimini izle me hakkı,izin verme meselesi haline gelir; mahkemeler, iletişim işverenin eki pmanını kullanarakgerçekleştirildiğinde genellikle işverenlere izin verme eğil imindedir.

Ayrıca, bu yasa bazı hükümet ajanslarına belirli kısıtlamalar altında elektr onik iletişimi izleme yetkisi vermektedir. Bu hükümler çok tartışmaya neden olmuştur. Örneğin, 2000 yılında FBI, bir mahkeme tarafından belirlenen hed efin yanı sıra bir internet servis sağlayıcısının tüm abonelerinin iletişimini rap orlayan Carnivore adlı sisteminin varlığını açıkladı ve 2001 yılında Dünya Tica ret Merkezi'ne yapılan terör saldırısının ardından, kongre, hükümet ajansları nın faaliyet göstermesi gereken kısıtlamaları değiştiren tartışmalı USA PATRI OT (Amerika'yı Terörizmi Önlemek için Gerekli Araçlarla Birleştirme ve Güçle ndirme Yasası) Yasası'nı kabul etti.

Bu gelişmelerin yarattığı hukuki ve etik tartışmalara ek olarak, izleme ha kları sağlamak bazı teknik sorunlar da ortaya çıkarmaktadır.

Doclingo https://www.doclingo.ai

Bu sorunlar, iletişim sisteminin iletişimlerin izlenebilmesi için inşa edilmesi v e programlanması gerektiğidir. Bu tür yetenekleri sağlamak, İletişim Yardımı için Hukuk Uygulama Yasası'nın (CALEA) amacıdır. Bu yasa, telekomünikasyo n taşıyıcılarının, hukuk uygulama dinlemeleri için ekipmanlarını değiştirmele rini gerektirmektedir - bu gereklilik karşılamak karmaşık ve pahalı olmuştur.

Başka bir tartışmalı konu, hükümetin iletişimleri izleme hakkı ile halkın şi freleme kullanma hakkı arasındaki çatışmadır. İzlenen mesajlar iyi bir şekilde şifrelenmişse, iletişimi dinlemek, kolluk kuvvetleri için sınırlı bir değer taşır. Amerika Birleşik Devletleri, Kanada ve Avrupa'daki hükümetler, şifreleme an ahtarlarının kaydını gerektiren sistemleri düşünmektedir, ancak bu tür talepl er şirketler tarafından karşı çıkılmaktadır. Sonuçta, kurumsal casusluk neden iyle, şifreleme anahtarlarının kaydını gerektirmenin birçok yasalara uyan şirk eti ve vatandaşı rahatsız edeceği anlaşılabilir. Kayıt sistemi ne kadar güvenli olabilir?

Son olarak, İnternet etrafındaki hukuki sorunların kapsamını tanımak am acıyla, 1999 yılında çıkarılan Anticybersquatting Tüketici Koruma Yasası'nı an ıyoruz; bu yasa, başka birinin taklit alan adlarını kurabilecek sahtecilere karşı kuruluşları korumak için tasarlanmıştır (bu uygulama siber işgal olarak bilini r). Yasa, başka birinin ticari markası veya "ortak hukuk markası" ile aynı veya karıştırıcı şekilde benzer alan adlarının kullanımını yasaklamaktadır. Bir etkisi , yasa alan adı spekülasyonunu (potansiyel olarak arzu edilen alan adlarını ka ydetme ve daha sonra o ismin haklarını satma süreci) yasaklamasa da, uygul amayı genel alan adları ile sınırlamasıdır. Böylece, bir alan adı spekülatörü, `GreatUsedCars.com` gibi genel bir ismi yasal olarak kaydedebilir, ancak Big Al zaten ikinci el araba işindeyse `BigAlUsedCars.com` isminin haklarını tale p edemeyebilir. Bu tür ayrımlar, Anticybersquatting Tüketici Koruma Yasası'n a dayanan davalarda sıkça tartışma konusu olmaktadır.

Sorular & Alıştırmalar

1. Ne demektir *kimlik avı*? Bilgisayarlar buna karşı nasıl korunur?
2. Bir alanın geçidinde yerleştirilebilecek güvenlik duvarı türleri ile alan içindeki bireysel bir ana bilgisayar arasında ne gibi bir ayrım vardır?
3. Teknik olarak, *veri*, bilginin temsillerini ifade ederken, *bilgi*, temel an lamı ifade eder. Parolaların kullanımı veriyi mi yoksa bilgiyi mi korur? Şifrelemenin kullanımı veriyi mi yoksa bilgiyi mi korur?
4. Kamu anahtarı şifrelemesinin daha geleneksel şifreleme tekniklerine göre n e gibi bir avantajı vardır?
5. Ağ güvenliği sorunlarına karşı koruma sağlamak için yasal girişimlerle ilişkili hangi sorunlar vardır?

Doclingo https://www.doclingo.ai

Bölüm Gözden Geçirme Problemleri

(Yıldızlı problemler isteğe bağlı bölümlerle ilişkilidir.)

1. Protokol nedir? Bu bölümde tanıtılan üç p rotokolü belirleyin ve her birinin amacını açıklayın.
2. İstemci/sunucu modelini tanımlayın.
3. Eşler arası modeli tanımlayın.
4. Dağıtık hesaplama sistemlerinin üç türün ü tanımlayın.
5. Açık bir ağ ile kapalı bir ağ arasındaki fark nedir?
6. CSMA/CD protokolü neden kablosuz bir ağda geçerli değildir?
7. CSMA/CD protokolünü kullanarak bir mesajı bir ağda iletmek isteyen bir makinenin izlediği adımları tanımlayın.
8. Gizli terminal problemi nedir?
   Bunu çözmek için bir teknik tanımlayın.
9. Bir hub, bir repeater'dan nasıl farklıdır?
10. Bir router, repeater'lar, köprüler ve anahtarla r gibi cihazlardan nasıl farklıdır?
11. Bir ağ ile internet arasındaki fark nedir?
12. Ağda bir mesaj iletme hakkını kontrol etme k için iki protokol tanımlayın.
13. 32-bit Internet adreslerinin başlangıçta geni şleme için yeterli alan sağlayacağı düşünülm üştü, ancak bu varsayım doğru çıkmıyor. IPv 6, 128-bit adresleme kullanıyor. Bu yeterli olacak mı? Cevabınızı gerekçelendirin. (Örneğin, mümkün olan adres sayısını düny a nüfusuyla karşılaştırabilirsiniz.)
14. Aşağıdaki bit desenlerinin her birini noktalı ond alık notasyon kullanarak kodlayın.
    a. 00000101000010010001000011
    b. 1000000000100000
    c. 0011000000011000
15. Aşağıdaki noktalı ondalık desenlerin her biri tarafı ndan temsil edilen bit deseni nedir?
    A. 0,0
    b. 26.19.1
    c. 8.12.20.13
16. Bir uç sistemin Internet üzerindeki adresi 1 34.48.4.122 olarak belirtilmişse, 32 bit adres in onaltılık notasyonu nedir?
17. DNS sorgulaması nedir?
18. Eğer bir bilgisayarın mnemonik Internet adresi

    Batman.bat mağarası.metropolis.gov

    makineyi içeren alanın yapısı hakkında ne va rsayabilirsiniz?
19. email adresinin bileşenlerini açıklayın

    `kermit@animals.com`
20. VoIP bağlamında, bir analog telefon adapt örü ileentegre bir telefon arasındaki fark n edir?
21. Bir mail sunucusunun rolü nedir?
22. N-unicast ile multicast arasındaki ayrım ned ir?
23. Aşağıdakilerin her birini tanımlayın:
    a. İsim sunucusu
    b. Erişim ISP
    c. Geçit
    d. Uç sistem
24. Aşağıdakilerin her birini tanımlayın:
    a. Hiper metin
    b. HTML
    c. Tarayıcı
25. İnternetin birçok "sıradan kullanıcısı"terimleri *İnternet* ve *Dünya Çapında Ağ* olarakbirbirinin yerine kullanmaktad ır. Her bir terimneye doğru bir şekilde atıfta bulunur?
26. Basit bir Web belgesini görüntülerken, tarayıcınızdan belgenin kaynak sürümünü göstermesini isteyin. Ardından belgenin temel yapısını tanımlayın. Özellikle, belgenin başlığını ve gövdesini tanımlayın ve her birinde bulduğunuz bazı ifadeleri listeleyin.
27. Beş HTML etiketini listeleyin ve anlamlarını tanımlayın.
28. Aşağıdaki HTML belgesini değiştirin, böy lece"Rover" kelimesi http://animals.org/ pets/dogs.html URL'sine bağlı olsun.

Doclingo https://www.doclingo.ai

html
head
<title>Örnek</title>
</head>
body
<h1>Evcil Hayvanım Köpeğim</h1>
p>Köpeğimin adı Rover.</p>
</body>
/html

29. Aşağıdaki HTML belgesinin bir bilgisayar ekranında nasıl görüneceğini gösteren bir taslak çizin.

html
head
<title>Örnek</title>
</head>
body
<h1>Evcil Hayvanım Köpeğim</h1>
<img src = "Rover.jpg">
</body>
/html

30. Metinde sunulan gayri resmi XML stilini kullanarak, basit cebirsel ifadeleri metin dosyaları olarak temsil etmek için bir işaretleme dili tasarlayın.

31. Metinde sunulan gayri resmi XML stilini kullanarak, bir kelime işlemcisinin temel metni işaretlemek için kullanabileceği bir dizi etiket tasarlayın. Örneğin, bir kelime işlemcisi hangi metnin kalın, italik, altı çizili vb. olması gerektiğini nasıl gösterebilir?

32. Metinde sunulan gayri resmi XML stilini kullanarak, metin öğelerinin basılı sayfada nasıl görünmesi gerektiğine göre film eleştirilerini işaretlemek için kullanılabilecek bir dizi etiket tasarlayın. Ardından, metindeki öğeleri n anlamına göre eleştirileri işaretlemek için kullanılabilecek bir dizi etiket tasarlayın.

33. Metinde sunulan gayri resmi XML stilini kullanarak, metin öğelerinin basılı sayfada nasıl görünmesi gerektiğine göre spor etkinlikleri hakkında makaleleri işaretlemek için kullanılabilecek bir dizi etiket tasarlayın. Ardından, metindeki öğelerin anlamına göre makaleleri işaretlemek için kullanılabilecek bir dizi etiket tasarlayın.

34. Aşağıdaki URL'nin bileşenlerini tanımlayın ve her birinin anlamını açıklayın.
http://lifeforms.com/animals/moviestars/kermit.html

35. Aşağıdaki kısaltılmış URL'lerin her birinin bileşenlerini tanımlayın.
a. http://www.farmtools.org/windmills.html
b. http://castles.org/
c. www.coolstuff.com

36. Bir tarayıcının eylemi, URL'deki "belgeyi bul" demeniz durumunda nasıl farklılaşır? http://stargazer.universe.org karşısında https://stargazer.universe.org?

37. Web'deki istemci tarafı etkinliklerine iki örnek verin. Web'deki sunucu tarafı etkinliklerine iki örnek verin.

***38.** OSI referans modeli nedir?

***39.** Veri yolu topolojisine dayanan bir ağda, veri yolu, makinelerin mesaj iletmek için rekabet etmesi gereken paylaşılmayan bir kaynaktır. Bu bağlamda kilitlenme (bkz. isteğe bağlı Bölüm 3.4) nasıl kontrol edilir?

***40.** İnternet yazılım hiyerarşisindeki dört katmanı listeleyin ve her katmanın gerçekleştirdiği bir görevi tanımlayın.

***41.** Taşıma katmanı neden büyük mesajları küçük paketlere böler?

***42.** Bir uygulama, taşıma katmanından bir mesaj iletmek için TCP kullanmasını istediğinde, taşıma katmanı uygulama katmanının isteğini yerine getirmek için hangi ek mesajları gönderecektir?

***43.** TCP, taşıma katmanını uygulamak için UDP'den daha iyi bir protokol olarak nasıl değerlendirilebilir? UDP, TCP'den daha iyi olarak nasıl değerlendirilebilir?

***44.** UDP'nin bağlantısız bir protokol olduğunu söylemek ne anlama geliyor?

***45.** TCP/IP protokol hiyerarşisinde bir güvenlik duvarı, gelen trafiği filtrelemek için hangi katmanda yer alabilir
a. Mesaj içeriği
b. Kaynak adresi
c. Uygulama türü

46. Bir güvenlik duvarı kurmak istiyorsanız, e-posta mesajlarını filtrelemek için

Doclingo https://www.doclingo.ai

belirli terimler ve ifadeler içeren. Bugüvenli k duvarı, alan adınızın geçidindemi yoksa alan adını n e-posta sunucusunda mıyer alacak? Cevabınızı açı klayın.

47. Proxy sunucusu nedir ve faydaları ne lerdir?
48. Açık anahtar şifrelemesinin ilkelerini özetleyin.
49. Korumasız bir bekleme durumundaki PC, İnternet için hangi şekilde bir tehlike oluşturur?
50. İnternetin küresel doğası, İnternetsorunl arına yasal çözümleri nasıl sınırlar?

Sosyal Sorunlar

Aşağıdaki sorular, bilgisayar alanıyla ilişkili etik/sosyal/hukuki sorunlara bir r ehber olarak tasarlanmıştır.Amaç, bu soruları yalnızca yanıtlamak değildir. A yrıca, neden böyle yanıt verdiğinizi ve gerekçelerinizin bir sorudan diğerine t utarlı olup olmadığını da düşünmelisiniz.

1. Ağlar aracılığıyla bilgisayarları bağlama yeteneği,evde çalışma kavramını popüler hale getirmiştir. Bu hareketin bazı artıları ve eksileri nelerdir? Do ğal kaynakların tüketimini etkileyecek mi? Aileleri güçlendirecek mi? "Ofis politikalarını" azaltacak mı? Evde çalışanlar, yerinde çalışanlarla aynı kari yerilerleme fırsatlarına sahip olacak mı? Topluluk bağları zayıflayacak mı? Eşitler ile azalan kişisel temasın olumlu veya olumsuz bir etkisi olacak mı ?
2. İnternet üzerinden mal siparişi vermek, "elle tutulur" alışverişe alternatif olmaya başlamaktadır. Bu alışveriş alışkanlıklarındaki değişimin topluluklar üzerinde ne gibi bir etkisi olacak? Alışveriş m erkezleri hakkında ne düşünüyorsunuz? Satın almadan gezmeyi sevdiğiniz küçük dükkanlar, kitap çılar ve giyim mağazaları hakkında ne düşünüyorsunuz? En düşük fiyatla satın almanın ne kadar i yi veya kötü olduğu ne ölçüde geçerlidir? Yerel bir işletmeyi desteklemek için bir ürün için daha fa zla ödeme yapma ahlaki yükümlülüğü var mı? Yerel bir mağazada ürünleri karşılaştırıp ardından seçiminizi daha düşük bir fiyatla İnternet üzerinden sipariş etmenin etik olup olmadığını düşünüy or musunuz? Bu tür davranışların uzun vadeli sonuçları nelerdir?
3. Bir hükümet, vatandaşlarının İnternete (veya herhangi bir uluslararası a ğa) erişimini ne ölçüde kontrol etmelidir? Ulusal güvenlik ile ilgili konular ne olacak? Hangi güvenlik sorunları ortaya çıkabilir?
4. Elektronik ilan panoları, ağ kullanıcılarının mesajlar yayınlamasına (genelli kleanonim olarak) ve diğerleri tarafından yayınlanan mesajları okumasına olanak tanır. Böyle bir ilan panosunun yöneticisi, içeriğinden sorumlu tutul malı mı? Bir telefon şirketi, telefon görüşmelerinin içeriğinden sorumlu tut ulmalı mı? Bir bakkal dükkanının yöneticisi, dükkanda bulunan bir topluluk ilan panosunun içeriğinden sorumlu tutulmalı mı?
5. İnternetin kullanımı izlenmeli mi? Düzenlenmeli mi? Eğer öyleyse, kim ta rafından ve ne ölçüde?
6. İnterneti kullanmak için ne kadar zaman harcıyorsunuz? Bu zaman iyi bir şekilde harcanıyor mu? İnternet erişimi sosyal aktivitelerinizi değiştirdi mi? İnternetten insanlarla yüz yüze ko nuşmaktan daha kolay mı konuşuyorsunuz?
7. Bir kişisel bilgisayar için bir yazılım paketi satın aldığınızda, geliştirici genelli kle sizi geliştirici ile kaydolmanızı ister, böylece size bildirimde bulunabilir.

Doclingo https://www.doclingo.ai

gelecek güncellemeler. Bu kayıt süreci giderek daha fazla İnternet üzerin den yürütülmektedir. Genellikle adınız, adresiniz ve belki de ürünü nasıl öğrendiğiniz gibi bilgileri vermeniz istenir ve ardından geliştiricinin yazılı mı bu verileri otomatik olarak geliştiriciye aktarır. Eğer geliştirici kayıt yaz ılımını, kayıt süreci sırasında geliştiriciye ek bilgiler gönderecek şekilde ta sarlarsa, hangi etik sorunlar ortaya çıkabilir?
Örneğin, yazılım sisteminizin içeriğini tarayabilir ve bulunan diğer yazılı m paketlerini raporlayabilir.

8. Bir web sitesini ziyaret ettiğinizde, o site bilgisayarınızda ziyaret ettiğinizi belir ten verileri, çerezler olarak adlandırılan verileri kaydetme yeteneğine sahiptir. Bu çerezler, geri dönen ziyaretçileri tanımlamak ve önceki etkinliklerini kaydetm ek için kullanılabilir, böylece siteye gelecekteki ziyaretler daha verimli bir şekilde yönetilebilir. Bilgisayarınızdaki çerezler ayrıca ziyaret ettiğiniz sitelerin bir kaydın ı sağlar. Bir web sitesinin bilgisayarınızda çerez kaydetme yeteneğine sahip olm ası gerekir mi? Bir web sitesinin bilgisayarınızda çerez kaydetmesine izniniz olma dan izin verilmeli mi? Çerezlerin olası faydaları nelerdir? Çerezlerin kullanımıyla hangi sorunlar ortaya çıkabilir?
9. Eğer şirketlerin şifreleme anahtarlarını bir devlet kurumuna kaydetmeleri ger ekiyorsa, güvende olacaklar mı?
10. Genel olarak, görgü kuralları, bir arkadaşınızı kişisel veya sosyal konular için, örneğin bir hafta sonu gezisi düzenlemek gibi, iş yerinde aramaktan kaçınma mız gerektiğini söyler. Benzer şekilde, çoğumuz bir müşteriyi evinde yeni bir ürünü tanıtmak için aramaktan çekiniriz. Benzer bir şekilde, düğün davetiyele rini misafirlerin ikamet adreslerine gönderirken, iş konferanslarının duyurular ını katılımcıların iş adreslerine göndeririz. Bir arkadaşınıza, arkadaşınızın iş ye rindeki mail sunucusu aracılığıyla kişisel e-posta göndermek uygun mu?
11. Bir PC sahibi, PC'yi İnternete bağlı bırakırsa ve bu PC başka bir taraf taraf ından hizmet reddi saldırısı gerçekleştirmek için kullanılırsa, PC sahibinin ne kadar sorumlu olması gerekir? Cevabınız, sahibin uygun güvenlik duv arları kurup kurmadığına bağlı mı?
12. Şeker veya oyuncak üreten şirketlerin, çocukları eğlendirirken şirketin ür ünlerini tanıtan oyunlar sunmasının etik olup olmadığı nedir? Oyun çocu klardan bilgi toplamak için tasarlandıysa ne olur? Eğlence, reklamcılık ve sömürü arasındaki sınırlar nelerdir?

Ek Okuma

Antoniou, G. ve F. van Harmelem. *Bir Semantik Web Giriş*. Cambridge, MA: MIT Pr ess, 2004.

Bishop, M. *Bilgisayar Güvenliğine Giriş*. Boston, MA: Addison-Wesley, 2005.

Comer, D. E. *Bilgisayar Ağları ve İnternetler,* 5. baskı. Upper Saddle River, NJ: Prentice-Hall, 2009.

Comer, D. E. *TCP/IP ile Ağlar Arası İletişim,* Cilt 1, 5. baskı. Upper Saddle River, NJ: Prentice-Hall, 2006.

Doclingo https://www.doclingo.ai

Goldfarb, C. F. ve P. Prescod. *XML El Kitabı,* 5. baskı. Upper Saddle River, NJ: Prentice-Hall, 2004.

Halsal, F. *Bilgisayar Ağları ve İnternet.*Boston, MA: Addison-Wesley, 2005.

Harrington, J. L. *Ağ Güvenliği: Pratik Bir Yaklaşım.*San Francisco: Morgan Kaufmann, 2005.

Kurose, J. F. ve K. W. Ross. *Bilgisayar Ağları: İnterneti İçeren Üstten Aşağıya Ya klaşım,* 4. baskı. Boston, MA: Addison-Wesley, 2008.

Peterson, L. L. ve B. S. Davie. *Bilgisayar Ağları: Sistem Yaklaşımı,* 3. baskı. San Francisco: Morgan Kaufmann, 2003.

Rosenoer, J. *Siber Hukuk, İnternetin Hukuku*. New York: Springer, 1997.

Spinello, R. A. ve H. T. Tavani. *Siber Etik Üzerine Okumalar*. Sudbury, MA: Jones ve Bartlett, 2001.

Stallings, W. *Kriptografi ve Ağ Güvenliği,* 4. baskı. Upper Saddle River, NJ: Prentice-Hall, 2006.

Stevens, W. R. *TCP/IP Açıklamalı,* Cilt 1. Boston, MA: Addison-Wesley, 1994.

Doclingo https://www.doclingo.ai

BÖLÜM

Algoritmalar 5

Giriş bölümünde, bilgisayar biliminin merkezi temasının al goritmaların incelenmesi olduğunu öğrendik. Şimdi bu tem el konuya odaklanma zamanı. Amacımız, bilgisayar bilimini n gerçek anlamda anlaşılmasını ve takdir edilmesini sağla mak için yeterince bu temel materyali keşfetmektir.

5.1 Algoritma Kavramı
Resmi Olmayan Bi
r İncelemeAlgoritmanın Resmi Tanı
mıAlgoritmaların Soyut Doğası

5.2 Algoritma Temsili
Primitifler
Sözdekod

5.3 Algoritma Keşfi
Problemi Çözme Sanatı
Kapıyı Açmak

5.4 Yinelemeli Yapılar
Sıralı Arama AlgoritmasıDöng
ü Kontrolü
Ekleme Sıralaması Algoritması

5.5 Özyinelemeli Yapılar
İkili Arama AlgoritmasıÖzyi
neleme Kontrolü

5.6 Verimlilik ve Doğruluk
Algoritma Verimliliği
Yazılım Doğrulaması

Doclingo https://www.doclingo.ai

Bir bilgisayarın bir görevi yerine getirebilmesi için, ona ne yapması gerektiği ni tam olarak söyleyen bir algoritma verilmesi gerektiğini gördük; dolayısıyla , algoritmaların incelenmesi bilgisayar biliminin temel taşını oluşturur. Bu bö lümde, algoritma keşfi ve temsili ile birlikte yineleme ve özyineleme gibi ana kontrol kavramlarını içeren bu çalışmanın birçok temel kavramını tanıtıyoruz . Bu bağlamda, arama ve sıralama için birkaç iyi bilinen algoritmayı da sunuy oruz. Algoritma kavramını gözden geçirerek başlıyoruz.

5.1 Algoritma Kavramı

Giriş bölümünde, bir algoritmayı bir görevin nasıl yerine getirileceğini tanımlayan adımlar k ümesi olarak gayri resmi bir şekilde tanımlamıştık. Bu bölümde, bu temel kavrama daha ya kından bakıyoruz.

Resmi Olmayan Bir İnceleme

Çalışmamızda birçok algoritma ile karşılaştık. Sayısal temsilleri bir formdan d iğerine dönüştüren, verilerdeki hataları tespit edip düzelten, veri dosyalarını sıkıştıran ve açan, çoklu görev ortamında çoklu programlamayı kontrol eden ve daha birçok algoritma bulduk. Ayrıca, bir CPU tarafından izlenen makine ç evriminin basit bir algoritmadan başka bir şey olmadığını gördük.

Durdurma talimatı yürütülmediği sürece aşağıdaki adımları uygula
maya devam edin:
a. Bir talimat alın.
b. Talimatı çözümleyin.
c. Talimatı yürütün.

Şekil 0.1'deki bir sihir numarasını tanımlayan algoritmanın gösterdiği gibi, algori tmalar teknik faaliyetlerle sınırlı değildir. Gerçekten de, bezelye ayıklamak gibi sırad an faaliyetlerin bile temelini oluştururlar:

Kabuklu bezelye dolu bir sepet ve boş bir kase alın.
Sepette kabuklu bezelye olduğu sürece aşağıdaki adımları uygulamaya d evam edin:
a. Sepetten bir bezelye alın.
b. Bezelye kabuğunu açın.
c. Bezelyeleri kabuktan kaseye dökün.
d. Kabuğu atın.

Aslında, birçok araştırmacı insan zihninin her aktivitesinin,hayal gücü, yaratıcılı k ve karar verme dahil, aslında algoritma yürütülmesinin bir sonucu olduğunu düşünmektedir—bu, yapay zeka çalışmamızda yeniden ele alacağımız bir varsa yımdır (Bölüm 11).

Ancak daha ileri gitmeden önce, bir algoritmanın resmi tanımını göz önünde bulun duralım.

Bir Algoritmanın Resmi Tanımı

Resmi olarak tanımlanmamış, gevşek tanımlı kavramlar günlük hayatta kabul edilebilir ve yayg ındır, ancak bir bilim iyi tanımlanmış terminolojiye dayanmalıdır. O halde, Şekil 5.1'de belirtilen bir algoritmanın resmi tanımını göz önünde bulundurun.

Tanımın, bir algoritmadaki adımların sıralı olmasını gerektirdiğini unutm ayın. Bu, bir algoritmadaki adımların yürütme sırası açısından iyi belirlenmiş bir yapıya sahip olması gerektiği anlamına gelir. Ancak bu, adımların birinci a dım, ardından ikinci adım ve devamı şeklinde bir sırayla yürütülmesi gerektiğ i anlamına gelmez. Paralel algoritmalar olarak bilinen bazı algoritmalar,

Doclingo https://www.doclingo.ai

Şekil **5.1** Bir algoritmanın tanımı

> Bir algoritma, sonlandırıcı bir sür
> eci tanımlayan,belirsiz olmayan, yür
> ütülebilir adımların sıralı bir setidir.

birden fazla adım dizisi içerir; her biri çoklu işlemci makinesinde farklı işlemcile r tarafından yürütülmek üzere tasarlanmıştır. Bu tür durumlarda, genel algorit ma, birinci adım, ikinci adım senaryosuna uyan tek bir adım dizisine sahip değil dir. Bunun yerine, algoritmanın yapısı, farklı işlemcilerin genel görevin farklı kıs ımlarını yerine getirmesiyle dallanan ve yeniden bağlanan birden fazla iplikten oluşur. (Bu kavrama Bölüm 6'da tekrar döneceğiz.) Diğer örnekler, her kapının genel algoritmanın tek bir adımını gerçekleştirdiği, Bölüm 1'deki flip-flop gibi d evreler tarafından yürütülen algoritmalardır. Burada adımlar, her kapının eyle minin devre boyunca yayıldığı neden-sonuç ilişkisiyle sıralanmıştır.

Sonra, bir algoritmanın yürütülebiliradımlardan oluşması gerektiği gereksinimini düşü nün. Bu koşulu takdir etmek için, talimatı göz önünde bulundurun

Tüm pozitif tam sayıların bir listesini yapın

bu, sonsuz sayıda pozitif tam sayı olduğu için gerçekleştirilemez. Bu nedenle, b u talimatı içeren herhangi bir talimat seti bir algoritma olamaz. Bilgisayar bilim cileri, yürütülebilir olma kavramını yakalamak için *etkili* terimini kullanır. Yani, bir adımın etkili olduğunu söylemek, onun yapılabilir olduğu anlamına gelir.

Şekil 5.1'deki tanım tarafından dayatılan bir diğer gereklilik, bir algoritmad akiadımların belirsiz olmamasıdır. Bu, bir algoritmanın yürütülmesi sırasında,p rosesin durumundaki bilginin, her adımın gerektirdiği eylemleri benzersiz ve ta m olarak belirlemek için yeterli olması gerektiği anlamına gelir. Diğer bir deyişl e,bir algoritmadaki her adımın yürütülmesi yaratıcı beceriler gerektirmez. Tam tersine, yalnızca talimatları takip etme yeteneğini gerektirir. (Bölüm 12'de,"al goritmalar" olarak adlandırılan, bu kısıtlamaya uymayan belirsiz algoritmaların ön emli bir araştırma konusu olduğunu öğreneceğiz.)

Şekil 5.1'deki tanım ayrıca bir algoritmanın sonlandırıcı bir süreci tanımla masını gerektirir; bu, bir algoritmanın yürütülmesinin bir sona ulaşması gere ktiği anlamına gelir. Bu gerekliliğin kökeni teorik bilgisayar bilimindedir; bur ada amaç, "Algoritmaların ve makinelerin nihai sınırlamaları nelerdir?" gibi s orulara yanıt vermektir. Burada bilgisayar bilimi, algoritmik olarak elde edile bilen sorunlar ile algoritmik sistemlerin yeteneklerinin ötesinde kalan sorunl ar arasında ayrım yapmaya çalışır. Bu bağlamda, bir yanıtla sonuçlanan süre çler ile yalnızca sonsuza kadar devam eden ve sonuç üretmeyen süreçler ara sında bir çizgi çekilir.

Ancak, sonlandırmayan süreçler için anlamlı uygulamalar vardır,hastane hast asının hayati belirtilerini izlemek ve bir uçağınuçuş sırasında irtifasını korumak gib i. Bazıları, bu uygulamaların yalnızcaalgoritmaların tekrarıyla ilgili olduğunu, her b irinin bir sona ulaştığını ve ardından otomatik olarak tekrar ettiğini savunur. Diğe rleri ise, bu tür argümanların yalnızca aşırı kısıtlayıcı bir resmi tanıma tutunma ça baları olduğunu ileri sürer. Her durumda, sonuç olarak, algoritma terimi genellikl e uygulamalı veya gayri resmi ortamlarda, sonlandırıcı süreçleri tanımlamayan ad ım setlerine atıfta bulunmak için kullanılır. Bir örnek, 1'i 3'e bölmek için sonlandırı cı bir süreç tanımlamayan uzun bölme "algoritması"dır. Teknik olarak, bu tür duru mlar terimin yanlış kullanımlarını temsil eder.

Doclingo https://www.doclingo.ai

Algoritmaların Soyut Doğası

Bir algoritma ile onun temsili arasındaki ayrımı vurgulamak önemlidir—bu, b ir hikaye ile bir kitap arasındaki ayrıma benzer. Bir hikaye soyut veya kavram saldır; bir kitap, bir hikayenin fiziksel temsilidir. Bir kitap başka bir dile çevrild iğinde veya farklı bir formatta yeniden yayımlandığında, yalnızca hikayenin t emsili değişir—hikaye kendisi aynı kalır.

Aynı şekilde, bir algoritma soyut ve temsilinden ayrıdır. Tek bir algoritma birçok şekilde temsil edilebilir. Örneğin, Celsius'tan Fahrenheit'a sıcaklık oku malarını dönüştüren algoritma geleneksel olarak cebirsel formül olarak tem sil edilir.

$$F = (9/5)C + 32$$

Ancak, Celsius'taki sıcaklık okumalarını 9/5 il

e çarp ve ardından ürüne 32 ekle talimatı ile de temsil edilebilir.

ve hatta bir elektronik devre biçiminde bile. Her durumda, temel algoritma a ynıdır; yalnızca temsiller farklıdır.

Bir algoritma ile onun temsili arasındaki ayrım, algoritmaları iletmeye çalışt ığımızda bir sorun ortaya çıkarır. Yaygın bir örnek, bir algoritmanın ne kadar ay rıntılı bir şekilde tanımlanması gerektiği ile ilgilidir. Meteorologlar arasında, "Ce lsius okumayı Fahrenheit eşdeğerine dönüştür" talimatı yeterlidir, ancak daha ayrıntılı bir tanıma ihtiyaç duyan bir sıradan insan, talimatın belirsiz olduğunu s avunabilir. Ancak sorun, temel algoritmada değil, algoritmanın sıradan insan iç in yeterince ayrıntılı bir şekilde temsil edilmemesindedir. Bir sonraki bölümde, primitlerin kavramının bir algoritmanın temsilindeki bu tür belirsizlik sorunları nı ortadan kaldırmak için nasıl kullanılabileceğini göreceğiz.

Son olarak, algoritmalar ve onların temsilleri konusunu ele alırken, iki di ğer ilgili kavram - programlar ve süreçler - arasındaki ayrımı netleştirmeliyiz. Bir program, bir algoritmanın temsilidir. (Burada, birçok programın sonlandı rılmayan "algoritmaların" temsilleri olduğu anlamında, algoritma terimini da ha az resmi bir anlamda kullanıyoruz.) Aslında, bilgisayar topluluğunda prog ram terimi genellikle bilgisayar uygulaması için tasarlanmış bir algoritmanın resmi temsilini ifade eder. Bölüm 3'te bir süreci, bir programı yürütme etkinli ği olarak tanımladık. Ancak, bir programı yürütmek, program tarafından tem sil edilen algoritmayı yürütmek olduğundan, bir süreç eşit şekilde bir algorit mayı yürütme etkinliği olarak tanımlanabilir. Programlar, algoritmalar ve sür eçler ayrı, ancak ilişkili varlıklardır. Bir program, bir algoritmanın temsilidir; o ysa bir süreç, bir algoritmayı yürütme etkinliğidir.

Sorular ve Alıştırmalar

1. Bir süreç, bir algoritma ve bir program arasındaki ayrımları özetleyin.
2. Tanıdığınız bazı algoritma örnekleri verin. Bunlar gerçekten kesin anl amda algoritmalar mı?
3. Giriş bölümünün 0.1. kısmında tanıtılan bir algoritmanın gayri resmi tanımında belirsizlik noktalarını belirleyin.

Doclingo https://www.doclingo.ai

4. Aşağıdaki talimatlar listesinin tanımladığı adımlar, hangi anlamda bir algoritma oluşturmayı başaramaz?

   Adım 1. Cebinizden bir madeni para çıkarın ve masanın üzerine koyun.

   Adım 2. Adım 1'e geri dönün.

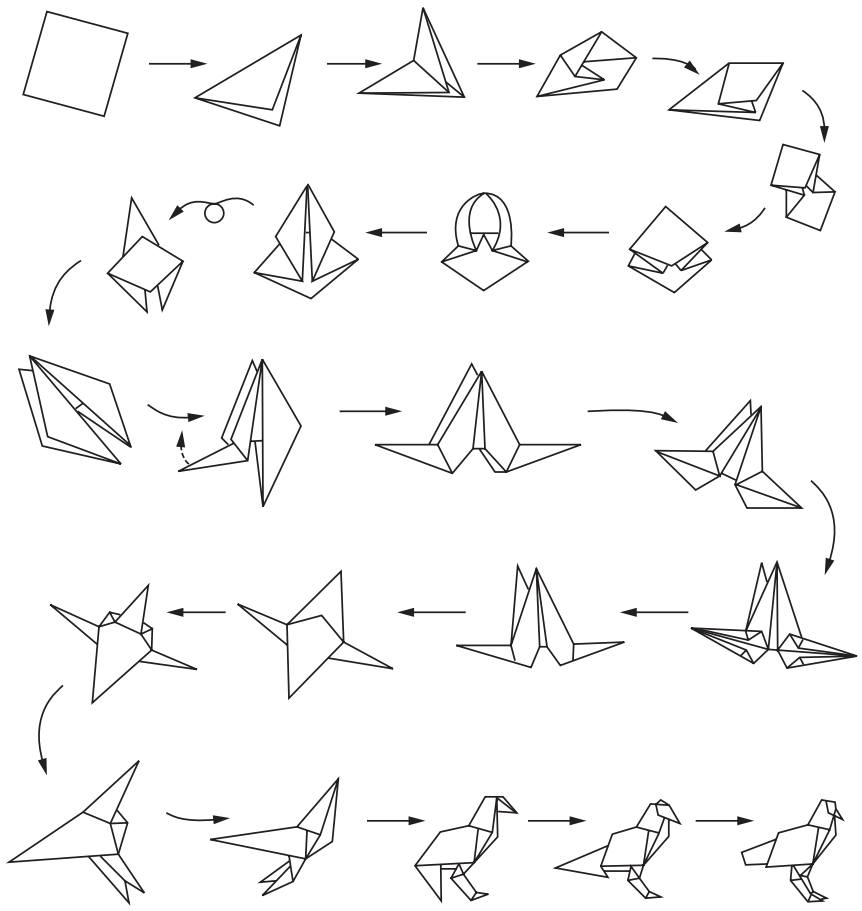
5.2 Algoritma Temsili

Bu bölümde, bir algoritmanın temsili ile ilgili konuları ele alıyoruz. Amacımız, temel kavramları tanıtmak ve kendi kullanımımız için bir temsil sistemi oluşturmaktır.

Temeller

Bir algoritmanın temsili, bir tür dil gerektirir. İnsanlar için bu, geleneksel bir doğal dil (İngilizce, İspanyolca, Rusça, Japonca) veya belki de resimlerin dili olabilir; bu, Şekil 5.2'de gösterilmiştir,

Şekil **5.2**Bir kare kağıttan bir kuş katlama



Doclingo https://www.doclingo.ai

bir kare kağıttan bir kuş katlama algoritmasını tanımlar.
Ancak, genellikle, bu tür doğal iletişim kanalları yanlış anlamalara yol açar; b azen kullanılan terminolojinin birden fazla anlamı olduğu için. (“Torunları ziy aret etmek sinir bozucu olabilir” cümlesi, torunların ziyaret ettiğinde sorun çı kardığı ya da onlara gitmenin sorunlu olduğu anlamına gelebilir.) Gerekli det ay seviyesi ile ilgili yanlış anlamalar nedeniyle de sorunlar ortaya çıkar. Az sa yıda okuyucu, Şekil 5.2'de verilen talimatlarla bir kuşu başarıyla katlayabilirk en, bir origami öğrencisi muhtemelen hiç zorluk çekmeyecektir. Kısacası, bir algoritmanın temsili için kullanılan dil tam olarak tanımlanmadığında veya bi lgi yeterli detayda verilmediğinde iletişim sorunları ortaya çıkar.

Bilgisayar bilimi, algoritma temsillerinin inşa edilebileceği iyi tanımlanmı ş bir yapı taşları seti oluşturarak bu sorunlara yaklaşır. Böyle bir yapı taşı pri mitive olarak adlandırılır. Bu primitife kesin tanımlar atamak, belirsizlik soru nlarının çoğunu ortadan kaldırır ve algoritmaların bu primitife göre tanımlan masını gerektirmek, uniform bir detay seviyesini oluşturur. Primitiflerin bir k oleksiyonu ile bu primitiflerin daha karmaşık fikirleri temsil etmek için nasıl b irleştirileceğini belirten bir kural koleksiyonu, programlama dili oluşturur.

Her primitife kendi sözdizimi ve anlamsal yapısı vardır. Sözdizimi, primitife ait sembolik temsili ifade eder; anlamsal yapı ise primitife ait anlamı ifade eder. hava sözdizimi üç sembolden oluşur, oysa anlamsal yapısı dünyayı çevreleye n gaz halindeki bir maddeyi ifade eder. Örneğin, Şekil 5.3, origami'de kullanıl an bazı primitifi sunmaktadır.

Algoritmaları bilgisayar yürütmesi için temsil etmek amacıyla kullanılacak bir primitf koleksiyonu elde etmek için, makinenin yürütmek üzere tasarlandığı birey sel talimatlara başvurabiliriz. Eğer bir algoritma bu detay seviyesinde ifade edilirs e, kesinlikle makine yürütmesi için uygun bir program elde ederiz. Ancak, algorit maları bu seviyede ifade etmek zahmetlidir, bu nedenle genellikle makinenin dilin de sağlanan daha düşük seviyeli primitiflerden inşa edilmiş her biri soyut bir araç olan “daha yüksek seviyeli” primitiflerin bir koleksiyonu kullanılır. Sonuç, algoritm aların makine dilinden kavramsal olarak daha yüksek bir seviyede ifade edilebilec eği resmi bir programlama dilidir. Bu tür programlama dillerini bir sonraki bölüm de tartışacağız.

Sözdekod

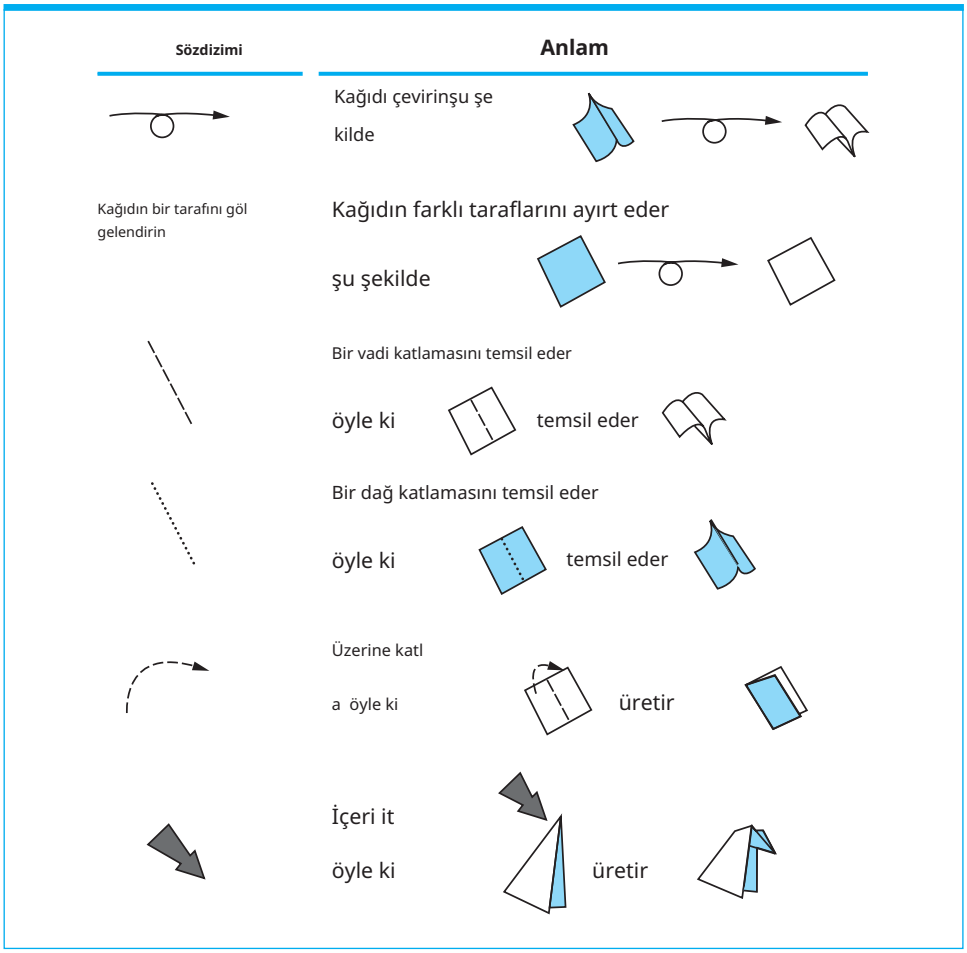
Şu anda, bir resmi programlama dilinin tanıtımını, sözdekod olarak bilinen dah a az resmi, daha sezgisel bir notasyon sistemine tercih ediyoruz. Genel olarak, bir sözdekod, algoritma geliştirme sürecinde fikirlerin gayri resmi olarak ifade edilebileceği bir notasyon sistemidir.

Bir sözdekod elde etmenin bir yolu, algoritmanın nihai versiyonunun ifad e edileceği resmi dilin kurallarını gevşetmektir. Bu yaklaşım, hedef programl ama dilinin önceden bilindiği durumlarda yaygın olarak kullanılır. Burada, pr ogram geliştirme sürecinin erken aşamalarında kullanılan sözdekod, hedef p rogramlama dilinde kullanılanlardan benzer, ancak daha az resmi olan sözdi zimsel-anlamsal yapılardan oluşur.

Ancak, amacımız, tartışmamızı belirli bir programlama diliyle sınırlamadan algoritma g eliştirme ve temsil etme konularını ele almaktır. Bu nedenle, sözdekod yaklaşımımız, tekrar layan anlamsal yapıların temsil edilmesi için tutarlı, özlü bir notasyon geliştirmektir.

Doclingo https://www.doclingo.ai

Şekil **5.3** Origami temel unsurları



Bu yapıların, gelecekteki fikirleri ifade etmeye çalıştığımız temel unsurlar hali ne gelecektir.

Bu tür tekrarlayan anlamsal yapılardan biri, hesaplanan bir değerin saklanmasıdır. Örneğin, kontrol ve tasarruf hesaplarımızın bakiyelerinin toplamını hesapladıysak, sonucu sakl amak isteyebiliriz, böylece daha sonra buna atıfta bulunabiliriz. Bu tür durumlarda, aşağıdaki f ormu kullanacağız:

isim←`expression`

burada *isim*, sonucu referans alacağımız isimdir ve expression, kaydedilece k sonucun hesaplamasını tanımlar. Bu ifadeleri " *isim* değerini *expression* ol arak ata," şeklinde okuyacağız ve bu tür ifadeleri atama ifadeleri olarak adla ndıracağız. Örneğin, ifade

KalanFonlar←KontrolBakiyesi+TasarrufBakiyesi

KontrolBakiyesi ve TasarrufBakiyesi toplamını KalanFonlar ismine atayan bir atama ifadesidir. Böylece, KalanFonlar terimi, gelecekteki ifadelerde o toplamı referans al mak için kullanılabilir.

Doclingo https://www.doclingo.ai

Algoritma Tasarımı Sırasında Algoritma Temsili

Karmaşık bir algoritma tasarlama görevi, tasarımcının birçok birbiriyle ilişkili kavramı takip etmesini gerektirir - bu, insan zihninin yeteneklerini aşabilir. Bu nedenle karma şık algoritmaların tasarımcısının, konsantrasyonunun gerektirdiği şekilde gelişen bir algoritmanın bölümlerini kaydetme ve hatırlama yoluna ihtiyacı vardır.

1950'ler ve 1960'larda, akış şemaları (algoritmaların oklarla bağlı geom etrik şekillerle temsil edildiği) en son tasarım aracıydı.
Ancak, akış şemaları genellikle, temel algoritmanın yapısını anlamayı zorlaştıran çapraz okların karmaşık ağları haline geliyordu. Bu nedenle, akış şemalarının ta sarım araçları olarak kullanımı, diğer temsil tekniklerine yerini bıraktı. Bu metin de kullanılan sözdekod gibi bir örnek, algoritmaların iyi tanımlanmış metin yapıl arıyla temsil edildiği bir yöntemdir. Akış şemaları, hedef sunum olduğunda hala faydalıdır. Örneğin, Şekil 5.8 ve 5.9, popüler kontrol ifadeleri tarafından temsil e dilen algoritmik yapıyı göstermek için akış şeması notasyonunu uygular.

Daha iyi tasarım notasyonları arayışı devam eden bir süreçtir. Bölüm 7'de, büyük yazılı m sistemlerinin küresel tasarımında yardımcı olmak için grafik tekniklerinin kullanılma eğili mini göreceğiz, oysa sözdekod, bir sistem içindeki daha küçük prosedürel bileşenlerin tasar ımı için popülerliğini korumaktadır.

Bir diğer tekrar eden anlamsal yapı, bazı koşulların doğruluğuna veya ya nlışlığına bağlı olarak iki olasıfaaliyetten birinin seçilmesidir. Örneklerşunları içerir:

> Gayri safi yurtiçi hasıla arttıysa, adi hisse senedi al; aksi takdirde, adi hisse s enedi sat.
> Gayri safi yurtiçi hasıla arttıysa hisse senedi satın alın, aksi takdirde satın.
>
> Gayri safi yurtiçi hasıla artmış veya azalmışsa, hisse senedi al veya sat.

Bu ifadelerin her biri, yapıya uymak için yeniden yazılabilir.

```
eğer ( koşul)  o zaman ( faaliyet)
                        devam et ( faaliyet)
```

burada ana yapı içindeki farklı alt yapıları duyurmak için `eğer`, `o zaman` ve `de vam et` anahtar kelimelerini kullandık ve bu alt yapıların sınırlarını belirlemek için parantezler kullandık. Sözdekodumuz için bu sözdizimsel yapıyı benimse yerek, bu ortak anlamsal yapıyı ifade etmenin tek tip bir yolunu elde ediyoru z. Böylece,

> Yılın artık yıl olup olmamasına bağlı olarak, toplamı sırasıyla 366 vey a 365'e bölün.

daha yaratıcı bir edebi tarza sahip olabilir, biz her zaman basit olanı tercih ed eceğiz.

```
eğer (yıl artık yıl ise)
    o zaman(günlük toplam ←toplam 366'ya bölünmüş)
    devam et(günlük toplam ←toplam 365'e bölünmüş)
```

Doclingo https://www.doclingo.ai

Ayrıca daha kısa sözdizimini benimsemiş bulunuyoruz

eğer (koşul) o zaman (etkinlik)

bir else etkinliği içermeyen durumlar için. Bu notasyonu kullanarak, "Satışları

n azaldığı durumunda, fiyatı %5 düşürün." ifadesi yazılabilir.

şuna indirgenecek

eğer(satışlar azaldıysa) o zaman(fiyatı %5 düşür)

Bir diğer yaygın anlamsal yapı, bir durum ifadesinin veya ifadeler dizisinin, bazı koşullar doğru kaldığı sürece tekrar tekrar yürütülmesidir. Gayri resmi örn ekler şunları içerir:

Satılacak biletler olduğu sürece, bilet satmaya devam et.

ve

Satılacak biletler olduğu sürece, bilet satmaya devam et.

Bu tür durumlar için, standart bir desen benimseriz

iken (*koşul*) yap (*faaliyet*)

sözde kodumuz için. Kısacası, böyle bir ifade, *koşulu kontrol et* ve eğer doğruysa, *faaliyeti* gerçekleştir ve tekrar *koşulu* kontrol et anlamına gelir. Ancak, eğer *koşul* yanlış bulunursa, `while` yapısını takip eden bir sonraki tali mata geç.

satışa kalmış(biletler varken) **do**(bir bilet sat)

Girinti, bir programın okunabilirliğini genellikle artırır. Örneğin, beyan

```
eğer (yağmur yağmıyorsa)t
    hen ( eğer (sıcaklık = sıcak)then(yüzmeye git)else(
                golf oyna)


          )
    else(televizyon izle)
```

anlamak, aksi takdirde eşdeğer olandan daha kolaydır

eğer(yağmur yağmıyorsa) **then** (eğer(sıcaklık = sıcak) **then**(yüzmeye git)**else**(golf oyna)) **else**(televizyon izle)

Böylece, sözdekodumuzda girintiyi kullanmayı benimseyeceğiz. (Bir kapanış parantezini , eşleştiği parantezin hemen altına hizalamak için girintiyi kullanabileceğimizi unutmayı n; bu, ifadelerin veya cümlelerin kapsamını tanımlamayı kolaylaştırır.)

Sözdekodumuzu, diğer uygulamalarda soyut araçlar olarak kullanılabilec ek etkinlikleri tanımlamak için kullanmak istiyoruz. Bilgisayar bilimi, bu tür pr ogram birimleri için alt program, alt rutin, prosedür, modül ve fonksiyon gibi çeşitli terimlere sahiptir; her birinin kendi anlamı vardır. Biz, sözdekodumuz için prosedür terimini benimseyeceğiz ve bu terimi, sözdekod biriminin bilinec eği başlığı duyurmak için kullanacağız. Daha kesin olarak, bir sözdekod birimini, aşağıdaki biçimde bir ifade ile başlatacağız:

prosedür\adı

Doclingo https://www.doclingo.ai

burada *name* birimin belirli adıdır. Bu tanıtıcı ifadeyi, birimin eylemini tanımla yan ifadelerle takip edeceğiz. Örneğin, Şekil 5.4, "Merhaba" mesajını üç kez y azdıran Greetings adlı bir prosedürün sözdekod temsilidir.

Bir prosedürün gerçekleştirdiği görev, sözdekodumuzda başka bir yerde g erektiğinde, yalnızca adını isteyeceğiz. Örneğin, iki prosedür `ProcessLoan` ve `RejectApplication` olarak adlandırılmışsa, bu durumda, hizmetlerini bir `if-then-else` yapısı içinde talep edebiliriz ve şöyle yazabiliriz:

```
eğer(. . .)   o zaman(İşlem Yapma Prosedürünü Uygula)
              aksi takdirde(Uygulama Reddetme Prosedürünü Uygula)
```

bu, test edilenşart doğruysa İşlem Yapma Prosedürünün uygulanması veya şar tyanlışsa Uygulama Reddetme Prosedürünün uygulanması ile sonuçlanacaktır.

Eğer prosedürler farklı durumlarda kullanılacaksa, mümkün olduğunca genel şekilde tasarlanmalıdır. İsim listelerini sıralamak için bir prosedür, herhangi bir listeyi sıralamak üzere tasarlanmalı—belirli bir liste değil—bu nedenle, sıralanacak listenin prosedürün kendisinde belirtilmemesi gerekir. Bunun yerine, liste, prosedürün temsilinde genel bir isimle anılmalıdır.

Pseudocode'umuzda, bu genelisimleri (ki bunlara parametreler denir) pr osedürün adını belirttiğimizaynı satırda parantez içinde listeleme kuralını be nimseyeceğiz. Özellikle,herhangi bir isim listesini sıralamak üzere tasarlanmı ş Sırala adlı bir prosedür,şu ifadeyle başlayacaktır:prosedürSırala (Liste)

Sıralanacak listeye atıf gereken temsilin ilerleyen kısımlarında,genel isim List e kullanılacaktır. Sırasıyla, Sırala hizmetlerineihtiyaç duyulduğunda, prosedür de Liste için hangi listeninyerine geçeceğini belirteceğiz.
Sırala. Bu nedenle şöyle bir şey yazacağız

```
Kuruluşun üye listesine Sırala prosedürünü uygulayın
```

ve

```
Düğün davetli listesine Sırala prosedürünü uygulayın
```

ihtiyaçlarımıza bağlı olarak.

Pseudocode'umuzun amacının, algoritmaları okunabilir, gayri resmi bir şekilde t emsil etmenin bir yolunu sağlamak olduğunu unutmayın. Bir notasyon istiyoruz

Şekil 5.4 Pseudocode'daki Selamlar prosedürü

```
prosedür  Selamlar
Say← 3;
iken  (Say > 0)  yap
     (mesajı "Merhaba" yazdır ve
          Say←  Say –1)
```

Doclingo https://www.doclingo.ai

Programlarda Öğeleri Adlandırma

Doğal bir dilde, öğelerin genellikle "bir widget üretmenin maliyeti" veya "tahmini varış za manı" gibi çok kelimeli adları vardır. Deneyimler, algoritmanın tanımında bu tür çok keli meli adların kullanılmasının algoritmanın tanımını karmaşıklaştırabileceğini göstermiştir . Her öğenin tek bir kesintisiz metin bloğu ile tanımlanması daha iyidir. Yıllar içinde, prog ramlardaki öğeler için tanımlayıcı adlar elde etmek amacıyla birden fazla kelimeyi tek bir leksikal birime sıkıştırmak için birçok teknik kullanılmıştır. Bunlardan biri, kelimeleri bağ lamak için alt çizgiler kullanmak ve tahmini_varış_zamanı *gibi adlar üretmektir.*
Diğer bir yöntem ise, okuyucunun sıkıştırılmış çok kelimeli bir adı anlamasına yardımcı o lmak için büyük harfler kullanmaktır. Örneğin, her kelimeyi büyük harfle başlatarak TahminiVarışZamanı gibi adlar elde edilebilir. Bu teknik Pascal casing olarak adlandırılır, çü nkü Pascal programlama dilinin kullanıcıları tarafından popüler hale getirilmiştir. Pascal casing'in bir varyasyonu camel casing olarak adlandırılır; bu, ilk harfin küçük harf olarak kaldığı tahminiVarışZamanı gibi Pascal casing ile aynıdır. Bu metinde Pascal casing'e eğ ilim gösteriyoruz, ancak seçim büyük ölçüde bir zevk meselesidir.

fikirlerimizi ifade etmemize yardımcı olacak bir sistem—bizi katı, resmi kurallar a köleleştirmeyecek. Bu nedenle, ihtiyaç duyduğumuzda pseudocode'umuzu g enişletmek veya değiştirmek için özgür hissedeceğiz. Özellikle, bir parantez için deki ifadeler kendileri parantezli ifadeleri içeriyorsa, açılış ve kapanış parantezl erini görsel olarak eşleştirmek zorlaşabilir. Bu durumlarda, birçok insan, kapanı ş parantezini takip eden kısa bir yorumun, hangi ifade veya ifadenin sonlandığı nı açıklamakta yardımcı olduğunu bulur. Özellikle, bir `while` ifadesindeki son p arantezi `end while` kelimeleriyle takip etmek, şöyle bir ifade üretir

```
while (...) do
 (.
  .
  .
 )end while
```

ya da belki

```
while (...) do
 (if (...)
    then (.
           .
           .
         )end if
 )end while
```

her iki `if` ve`while` ifadelerinin sonunu belirttiğimiz yer.

Önemli olan, bir algoritmayı okunabilir bir biçimde ifade etmeye çalışmamızdır, ve bu nedenle bu amaca ulaşmak için zaman zaman görsel yardımcılar (girintiler, yorumlar vb.) getiriyoruz. Ayrıca, eğer henüz sözde kodumuza dahil edilmemiş bir tekrar eden tema ile karşılaşırsak, yeni kavramı temsil etmek için tutarlı bir sözdizimi benimseyerek sözde kodumuzu genişletmeyi seçebiliriz.

Doclingo https://www.doclingo.ai

Sorular ve Alıştırmalar

1. Bir bağlamda bir ilke, başka bir bağlamda ilke bileşeni olabilir. Örneğ in, bizim `while` ifademiz, sözdekodumuzda bir ilke iken, nihayetinde makine-dili talimatlarının bir bileşeni olarak uygulanmaktadır. Bu olg unun bilgisayar dışındaki bir ortamda iki örneğini verin.

2. Prosedürlerin inşası, ilke inşası açısından ne anlama geliyor?

3. Öklid algoritması, iki pozitif tam sayının *X* ve*Y* en büyük ortak bölenin i aşağıdaki süreçle bulur:

   X veyaY değerlerinden hiçbiri sıfır olmadığı sürece, daha büyük olan değ eri daha küçük olanla bölmeye devam edin ve X veY'ye sırasıyla bölüm v e kalanı atayın. (Nihai X değeri en büyük ortak bölen olacaktır.)

   Bu algoritmayı sözdekodumuzda ifade edin.

4. Bilgisayar programlaması dışındaki bir konuda kullanılan ilke koleksi yonunu tanımlayın.

5.3 Algoritma Keşfi

Bir programın geliştirilmesi, iki etkinlikten oluşur—temel algoritmanın keşfi ve bu algoritmanın bir program olarak temsil edilmesi. Şu ana kadar, algorit ma temsilinin sorunlarıyla ilgileniyorduk, algoritmaların ilk etapta nasıl keşfe dildiği sorusunu dikkate almadan. Ancak, algoritma keşfi genellikle yazılım g eliştirme sürecindeki daha zorlu adımdır. Sonuçta, bir problemi çözmek için bir algoritma keşfetmek, o problemi çözmenin bir yöntemini bulmayı gerekti rir. Dolayısıyla, algoritmaların nasıl keşfedildiğini anlamak, problem çözme s ürecini anlamak demektir.

Problem Çözme Sanatı

Problem çözme teknikleri ve bunlar hakkında daha fazla bilgi edinme ihtiyacı, bilg isayar bilimlerine özgü değildir; aksine, neredeyse her alana ilişkin konulardır. Alg oritma keşfi süreci ile genel problem çözme süreci arasındaki yakın ilişki, bilgisaya r bilimcilerinin diğer disiplinlerden olanlarla daha iyi problem çözme teknikleri ara yışında bir araya gelmesine neden olmuştur. Nihayetinde, problem çözme sürecin i kendisi bir algoritmaya indirmek istenir, ancak bunun imkansız olduğu gösterilm iştir. (Bu, Bölüm 12'deki materyalin bir sonucudur; burada, algoritmik çözümleri o lmayan problemler olduğunu göstereceğiz.) Dolayısıyla, problem çözme yeteneği, öğrenilmesi gereken kesin bir bilimden ziyade, geliştirilmesi gereken daha çok sa natsal bir beceri olarak kalmaktadır.

Problemi çözmenin kaçınılmaz, sanatsal doğasının bir kanıtı olarak, matematikçi G. Polya tarafından 1945 yılında sunulan aşağıdakigevşek tanımlı problem çözme aşamala rı, günümüzde problem çözme becerilerini öğretmeye yönelik birçok girişimin temel ilk eleri olmaya devam etmektedir.

Aşama 1. Problemi anla.

Aşama 2. Problemi çözmek için bir plan hazırla.

Doclingo https://www.doclingo.ai

Aşama 3. Planı uygulayın.

Aşama 4. Çözümü doğruluk açısından ve diğer sorunları çözmek için bir araç olarak potansi yeli açısından değerlendirin.

Program geliştirme bağlamında, bu aşamalar şunlar haline gelir

Aşama 1. Sorunu anlayın.

Aşama 2. Bir algoritmik prosedürün sorunu nasıl çözebileceğine dair bir fikir edinin.

Aşama 3. Algoritmayı formüle edin ve onu bir program olarak temsil edin.

Aşama 4. Programı doğruluk açısından ve diğer sorunları çözmek için bir araç olarak potans iyeli açısından değerlendirin.

Polya'nın listesini sunduktan sonra, bu aşamaların bir problemi çözmeye çalışı rken izlenmesi gereken adımlar değil, çözüm süreci boyunca bir noktada tamaml anacak aşamalar olduğunu vurgulamalıyız. Buradaki anahtar kelime *izlenmiştir*. Problemleri izleyerek çözemezsiniz. Daha ziyade, bir problemi çözmek için in isiyatif almalı ve liderlik etmelisiniz. "Artık Aşama 1'i bitirdim, Aşama 2'ye geç me zamanı" şeklindeki bir zihniyetle bir problemi çözme görevine yaklaşırsa nız, başarılı olma olasılığınız düşüktür. Ancak, probleme dahil olursanız ve ni hayetinde onu çözerseniz, muhtemelen yaptıklarınıza geri dönüp Polya'nın a şamalarını gerçekleştirdiğinizi fark edebilirsiniz.

Bir diğer önemli gözlem, Polya'nın aşamalarının mutlaka sırayla tamamla nmadığıdır. Başarılı problem çözücüleri genellikle bir problemi çözme stratej ileri geliştirmeye (Aşama 2) problemi tamamen anlamadan (Aşama 1) başlarl ar. Sonra, eğer bu stratejiler başarısız olursa (Aşama 3 veya 4 sırasında), pota nsiyel problem çözücüsü problemin karmaşıklıklarını daha derinlemesine anl ar ve bu daha derin anlayışla, başka ve umarım daha başarılı stratejiler gelişt irmek için geri dönebilir.

Unutmayın ki, sorunların nasıl çözüldüğünü tartışıyoruz - nasıl çözülmesini istediğimizi değil. İdeal olarak, az önce tanımlanan deneme-yanılma sürecinde var olan israfı ortadan kaldırmak isteriz. Büyük yazılım sistemleri geliştirirken, y anlış anlamaların Faz 4'te keşfedilmesi, kaynaklar açısından büyük bir kayıp anl amına gelebilir. Böyle felaketlerden kaçınmak, yazılım mühendislerinin (Bölüm 7) ana hedeflerinden biridir; bu mühendisler, bir çözümle ilerlemeden önce bir sorunun kapsamlı bir şekilde anlaşılmasını ısrarla savunmuşlardır. Ancak, bir s orunun gerçek bir anlayışının, bir çözüm bulunduğunda elde edildiği iddia edil ebilir. Bir sorunun çözülmemiş olması, bir anlayış eksikliğini ima eder. Bu nede nle, herhangi bir çözüm önermeden önce sorunun tam bir anlayışına ısrar etm ek, bir bakıma idealist bir tutumdur.

Bir örnek olarak, aşağıdaki sorunu düşünün:

> Kişi A, kişi B'nin üç çocuğunun yaşlarını belirleme görevini üstlenmiştir. B, A'ya çocukl arın yaşlarının çarpımının 36 olduğunu söyler. Bu ipucunu değerlendirdikten sonra, A başka bir ipucu gerektiğini yanıtlar, bu yüzden B, A'ya çocukların yaşlarının toplamını söyler. Yine, A başka bir ipucu gerektiğini yanıtlar, bu yüzden B, A'ya en büyük çocuğu n piyano çaldığını söyler. Bu ipucunu duyduktan sonra, A, B'ye üç çocuğun yaşlarını s öyler.
>
> Üç çocuk kaç yaşında?

İlk bakışta son ipucu problemin tamamen alakasız görünüyor, ancak görünüşe göre bu ipucu A'nın çocukların yaşlarını nihayet belirlemesine olanak tanıyor. Bu nasıl olabilir? Bir saldırı planı oluşturarak ve bu planı takip ederek ilerleyel im, hala problem hakkında birçok sorumuz olmasına rağmen. Planımız

Doclingo https://www.doclingo.ai

problem ifadesinde tanımlanan adımları izlemek olacak ve hikaye ilerledikçe A'nın elindeki bilgileri takip edeceğiz.

A'ya verilen ilk ipucu, çocukların yaşlarının çarpımının 36 olduğudur. Bu, üç yaşın temsil ettiği üçlünün Şekil 5.5(a)'da listelenenlerden biri olduğu anlamına gelir. Bir sonraki ipucu, istenen üçlünün toplamıdır. Bu toplamın ne olduğunu bilmiyoruz, ancak A'nın doğru üçlüyü izole etmek için bu bilginin yeterli olmadığını biliyoruz; bu nedenle istenen üçlü, Şekil 5.5(b) tablosunda toplamı en az iki kez görünen bir üçlü olmalıdır. Ancak Şekil 5.5(b)'de aynı toplamları olan tek üçlüler (1,6,6) ve (2,2,9) olup, her ikisi de 13 toplamını üretmektedir. Bu, son ipucu verildiğinde A'nın elindeki bilgidir. Son ipucunun önemini nihayet bu noktada anlıyoruz. Piyano çalmakla hiçbir ilgisi yok; daha ziyade en büyük çocuğun olması gerçeğidir. Bu, (1,6,6) üçlüsünü dışlar ve böylece çocukların yaşlarının 2, 2 ve 9 olduğunu sonucuna varmamıza olanak tanır.

Bu durumda, o zaman, sorunu çözme planımızı uygulamaya çalıştığımızda (Aşama 3) sorunun tam bir anlayışını kazanırız (Aşama 1). Aşama 1'i tamamlamaya ısrar etseydik, muhtemelen çocukların yaşlarını asla bulamazdık. Sorun çözme sürecindeki bu tür düzensizlikler, sistematik sorun çözme yaklaşımlarını geliştirmekteki zorlukların temelini oluşturur.

Bir diğer düzensizlik, görünüşte başarısız bir şekilde bir sorun üzerinde çalıştıktan sonra, potansiyel bir sorun çözücüsünün başka bir görev yaparken birdenbire çözümü görmesidir. Bu fenomen, H. von Helmholtz tarafından 1896'da tanımlanmış ve matematikçi Henri Poincaré tarafından Paris'teki Psikolojik Dernekte bir konferansta tartışılmıştır. Orada, Poincaré, bir sorunu bir kenara bıraktıktan ve başka projelere başladıktan sonra, o sorunun çözümünü fark etme deneyimlerini anlatmıştır. Bu fenomen, zihnin bilinçaltı bir kısmının çalışmaya devam ediyormuş gibi göründüğü ve başarılı olursa çözümü bilinçli zihne zorla soktuğu bir süreci yansıtır. Bugün, bir sorun üzerinde bilinçli çalışma ile ani ilham arasındaki dönem, bir inkübasyon dönemi olarak bilinir ve bu konudaki anlayış, güncel araştırmaların bir hedefi olmaya devam etmektedir.

Kapıyı Aralamak

Biraz felsefi bir bakış açısıyla problem çözmeyi tartışıyorduk ve bir problemi nasıl çözmeye çalışmamız gerektiği sorusuyla doğrudan yüzleşmekten kaçınıyorduk. Elbette, her biri belirli durumlarda başarılı olabilen sayısız problem çözme yaklaşımı vardır.

Şekil 5.5

a. 36'nın çarpımı olan üçlüler

(1,1,36)	(1,6,6)
(1,2,18)	(2,2,9)
(1,3,12)	(2,3,6)
(1,4,9)	(3,3,4)

b. Üçlülerin toplamları (a) bölümünden

1 + 1 + 36 = 38	1 + 6 + 6 = 13
1 + 2 + 18 = 21	2 + 2 + 9 = 13
1 + 3 + 12 = 16	2 + 3 + 6 = 11
1 + 4 + 9 = 14	3 + 3 + 4 = 10

Doclingo https://www.doclingo.ai

ayarlar. Bunların bazılarını kısa süre içinde tanımlayacağız. Şu anda, bu tekni kler arasında bir ortak iplik olduğunu not ediyoruz, ki bu da basitçe "kapıdan içeri girmektir." Örnek olarak, aşağıdaki basit problemi ele alalım:

> A, B, C ve D bir yarışa girmeden önce aşağıdaki tahminleri yaptılar:
>
> A'nın B'yi kazanacağını tahmin etti.
> B, D'nin son olacağını tahmin etti.
> C, A'nın üçüncü olacağını tahmin etti.
> D, A'nın tahmininin doğru olacağını tahmin etti.
>
> Bu tahminlerden sadece biri doğruydu ve bu, kazanan tarafından yapılan ta hmindi. A, B, C ve D yarışı hangi sırayla bitirdi?

Problemi okuduktan ve verileri analiz ettikten sonra, A ve D'nin tahminlerinin e şdeğer olduğunu ve sadece bir tahminin doğru olduğunu fark etmek uzun sür memelidir; bu nedenle hem A'nın hem de D'nin tahminleri yanlış olmalıdır. Böy lece ne A ne de D kazananlardı. Bu noktada kapıyı aralamış olduk ve problemi mizin tam çözümünü elde etmek, buradan bilgimizi genişletmek meselesidir. E ğer A'nın tahmini yanlışsa, o zaman B de kazanmadı. Kazanan için tek kalan seç enek C'dir. Böylece, C yarışı kazandı ve C'nin tahmini doğruydu.
Sonuç olarak, A'nın üçüncü geldiğini biliyoruz. Bu, bitişsırasının ya CBAD ya da CDAB olduğu anlamına geliyor. Ancak, B'nin tahmininyanlış olması gerektiğin den, ilk seçenek eleniyor. Bu nedenle bitiş sırası CDAB'dır.

Elbette, kapıyı aralamamız gerektiği söylenmesi, bunu nasıl yapacağımızınsöy lenmesiyle aynı şey değildir. Bu ilk adımı elde etmek ve bu başlangıçhamlesini sor unun tam çözümüne nasıl genişleteceğimizi anlamak, potansiyelsorun çözücüsün den yaratıcı bir girdi gerektirir. Ancak, Polya ve diğerleri tarafından önerilen, kapıy ı aralamak için nasıl bir yol izlenebileceğine dairbirkaç genel yaklaşım vardır. Bir t anesi, problemi geriye doğru çözmeyeçalışmaktır. Örneğin, problem belirli bir gir di ile belirli bir çıktıyıüretmenin bir yolunu bulmaksa, o çıktıyla başlayabilir ve veril en girdiylegeriye doğru gitmeye çalışabilirsiniz. Bu yaklaşım, önceki bölümdeki ku ş-katlama algoritmasını keşfetmeye çalışan insanların tipik bir davranışıdır.Tama mlanmış bir kuşu açmaya çalışarak nasıl inşa edildiğini görmeyeçalışırlar.

Bir diğer genel problem çözme yaklaşımı, ya daha kolay çözülebilen ya daönc eden çözülmüş bir ilgili problemi aramak ve ardından onun çözümünümevcut pro bleme uygulamaya çalışmaktır. Bu teknik, program geliştirmebağlamında özellikl e değerlidir. Genel olarak, program geliştirme, belirlibir problem örneğini çözme süreci değil, daha çok problemin tüm örnekleriniçözmek için kullanılabilecek gene l bir algoritma bulma sürecidir. Dahakesin olarak, eğer isim listelerini alfabetik sır aya dizmek için bir programgeliştirme göreviyle karşı karşıya kalsaydık, görevimiz belirli bir listeyisıralamak değil, herhangi bir isim listesini sıralamak için kullanılab ilecekgeneral bir algoritma bulmak olurdu. Böylece, talimatlar

> David ve Alice'in isimlerini değiştirin.
> Carol'un ismini Alice ve David'in arasına taşıyın.
> Bob'un ismini Alice ve Carol'un arasına taşıyın.

David, Alice, Carol ve Bob listesini doğru bir şekilde sıralamak, istediğimiz genel ama çlı algoritmayı oluşturmaz. İhtiyacımız olan, bu listeyi sıralayabilen bir algoritmadır

Doclingo https://www.doclingo.ai

ve karşılaşabileceğimiz diğer listeleri de sıralayabilen bir algoritmadır. Bu, belirli b ir listeyi sıralama çözümümüzün, genel amaçlı bir algoritma arayışımızda tamame n değersiz olduğu anlamına gelmez. Örneğin, genel ilkeleri bulma çabasında böyl e özel durumları dikkate alarak kapıyı aralayabiliriz; bu ilkeler daha sonra istenen genel amaçlı algoritmayı geliştirmek için kullanılabilir. Bu durumda, çözümümüz, i lgili problemler topluluğunu çözme tekniği ile elde edilir.

Kapıyı aralamanın bir başka yaklaşımı, adım adım iyileştirme uygulamakt ır; bu, esasen tüm bir görevi (tüm detaylarıyla) bir anda fethetmeye çalışma mak tekniğidir. Aksine, adım adım iyileştirme, mevcut problemi birkaç alt pr oblem açısından görmeyi önerir. Amaç, orijinal problemi alt problemlere ayır arak, genel çözümü, her biri orijinal problemin tamamından daha kolay çözü lebilen adımlar açısından yaklaşmaktır. Bununla birlikte, adım adım iyileştir me, bu adımların daha küçük adımlara ayrılmasını ve bu daha küçük adımlar ın hala daha küçük olanlara bölünmesini önerir; ta ki tüm problem, kolayca ç özülebilen alt problemlerin bir koleksiyonuna indirgenene kadar.

Bu ışıkta, adım adım iyileştirme, genel bir durumdan özel bir duruma ilerleye n bir yukarıdan aşağıya metodolojidir. Aksine, aşağıdan yukarıya bir metodolojiö zelden genele doğru ilerler. Teorik olarak zıt olsalar da, bu iki yaklaşım yaratıcı pr oblem çözmede sıklıkla birbirini tamamlar. Adım adım iyileştirmenin yukarıdan aş ağıya metodolojisi tarafından önerilen bir problemin parçalanması, genellikle pro blem çözücünün sezgisi tarafından yönlendirilir; bu sezgi aşağıdan yukarıya bir m odda çalışıyor olabilir.

Adım adım iyileştirmenin yukarıdan aşağıya metodolojisi, esasen bu org anizasyonun sonuçları olan problem çözme özelliklerine sahip bir organizasy on aracıdır. Uzun zamandır veri işleme topluluğunda önemli bir tasarım met odolojisi olmuştur; burada büyük yazılım sistemlerinin geliştirilmesi önemli b ir organizasyon bileşenini kapsamaktadır. Ancak, 7. Bölümde öğreneceğimiz gibi, büyük yazılım sistemleri giderek önceden üretilmiş bileşenlerin birleştiri lmesiyle inşa edilmektedir—bu yaklaşım doğası gereği aşağıdan yukarıdır. Böylece, hem yukarıdan aşağıya hem de aşağıdan yukarıya metodolojileri bilgisayar biliminde önemli araçlar ol maya devam etmektedir.

Böyle geniş bir perspektifi korumanın önemi, önceden belirlenmiş kavra mlar ve seçilmiş araçların problem çözme görevine getirilmesinin bazen bir problemin basitliğini maskeleyebileceği gerçeğiyle örneklendirilmektedir. Bu bölümde daha önce tartışılan çocukların yaşları problemi, bu fenomenin mü kemmel bir örneğidir. Cebir öğrencileri, problemi genellikle bir dizi eşzamanl ı denklem olarak ele alır; bu yaklaşım bir çıkmaza yol açar ve çoğu zaman pot ansiyel problem çözücüyü verilen bilginin problemi çözmek için yeterli olma dığına inandırır.

Bir diğer örnek aşağıdaki gibidir:

> Bir iskeleden bir kayığa adım attığınızda, şapkanız suya düşer, siz farkında olmadan. Nehrin akışı saatte 2.5 mil olduğundan, şapkanız aşağı akıntıya doğru yüzmeye b aşlar. Bu arada, kayıkta suya göre saatte 4.75 mil hızla yukarı akıntıya doğru seya hat etmeye başlarsınız. 10 dakika sonra şapkanızın kaybolduğunu fark eder, kayı ğı döndürür ve şapkanızı nehirde yakalamaya başlarsınız. Şapkanızı yakalamak n e kadar zaman alacak?

Çoğu cebir öğrencisi ve hesap makinesi meraklısı bu probleme, önce kayığın 10 dakikada ne kadar yukarı akıntıya gideceğini belirleyerek yaklaşır.

Doclingo https://www.doclingo.ai

Aynı zamanda şapkanın o süre zarfında ne kadar aşağı akıntıya gideceğini de belirlerler. Daha sonra, kayığın bu konuma ulaşmak için aşağı akıntıya gitmesinin ne kadar zaman alacağını hesaplarlar. Ancak, kayık bu konuma ulaştığında, şapka daha da aşağı akıntıya doğru yüzmüş olacaktır. Böylece, problem çözücüsü ya kalkülüs tekniklerini uygulamaya başlar ya da kayığın şapkanın bulunduğu yere gittiği her seferde şapkanın nerede olacağını hesaplama döngüsünde sıkışıp kalır.

Ancak, problem çok daha basittir. Püf noktasının formüller yazmaya ve hesaplamalar yapmaya başlama isteğine karşı koymak olduğunu anlamaktır. Bunun yerine, bu becerileri bir kenara bırakmalı ve bakış açımızı ayarlamalıyız. Tüm problem nehirde gerçekleşmektedir. Suyun kıyıya göre hareket etmesi önemsizdir.

Bir nehir yerine büyük bir konveyör bandında aynı problemi düşünün. Öncelikle,bant dururken problemi çözün. Bantta dururken şapkanızı ayaklarınızın yanına koyarsanızve sonra şapkanızdan 10 dakika uzaklaşırsanız, şapkanıza geri dönmek 10 dakika alacaktır. Şimdi konveyör bandını açın. Bu,\ manzaranın bantın yanından geçmeye başlayacağı anlamına gelir, ancak bantta olduğunuz için,bu, bantla veya şapkanızla olan ilişkinizi değiştirmez. Şapkanıza geri dönmek yine 10 dakika alacaktır.

Algoritma keşfinin, iyi tanımlanmış metodolojilerden oluşan bir konu olarak öğretilmek yerine, zamanla geliştirilmesi gereken zorlu bir sanat olduğunu sonucuna varıyoruz. Gerçekten de, bir potansiyel problem çözücüyü belirli metodolojileri takip etmesi için eğitmek, bu yaratıcı becerileri bastırmak anlamına gelir; bu becerilerin yerine beslenmesi gerekir.

Sorular ve Alıştırmalar

1. **a.** Aşağıdaki problemi çözmek için bir algoritma bulun: Verilen bir pozitif tam sayı n, toplamı n olan tüm pozitif tam sayı listeleri arasında yer alan en büyük çarpıma sahip pozitif tam sayıların listesini bulun. Örneğin, *eğer n 4 ise, istenen liste 2, 2'dir çünkü 2 × 2, 1 × 1 × 1 × 1'den daha büyüktür,* $2 \times 1 \times 1$ ve 3×1. Eğer$n$ 5 ise, istenen liste 2, 3'tür.

b. İstenilen liste nedir eğer $n = 2001$?

c. Kapıyı nasıl açtığını açıklayın.

2. **a.** Bir pozitif tam sayı n için, 2n *satır ve 2*n sütundan oluşan bir dama tahtası ve her biri tahtada tam olarak üç kareyi kaplayabilen L şeklinde tuğlalar içeren bir kutu verildiğini varsayalım. Eğer tahtadan herhangi bir tek kare kesilirse, kalan tahtayı tuğlalarla kaplayabilir miyiz, böylece tuğlalar üst üste binmez veya tahtanın kenarından sarkmaz?

b. (a) için bulduğunuz çözümün, $2^{2n}1$'in tüm pozitif tam sayılar n için 3'e bölünebilir olduğunu göstermek için nasıl kullanılabileceğini açıklayın.

c. (a) ve (b) Polya'nın problem çözme aşamalarıyla nasıl ilişkilidir?

3. Aşağıdaki mesajı çözün, ardından kapıyı nasıl açtığınızı açıklayın. *Pdeo eo pda yknnayp wjosan.*

4. Bir resim bulmacasını sadece parçaları bir masanın üzerine dökerek ve onları bir araya getirmeye çalışarak çözmeye çalışırsanız, yukarıdan aşağıya bir metodoloji izliyor olur musunuz? Tüm resmin nasıl görünmesi gerektiğini görmek için bulmaca kutusuna bakarsanız cevabınız değişir mi?

Doclingo https://www.doclingo.ai

5.4 Yinelemeli Yapılar

Şu anda amacımız, algoritmik süreçleri tanımlamada kullanılan bazı yinelem eli yapıları incelemektir. Bu bölümde, bir dizi talimatın döngüsel bir şekilde t ekrarlandığı yinelemeli yapılar üzerinde duracağız. Bir sonraki bölümde, özy ineleme tekniğini tanıtacağız. Yan etki olarak, bazı popüler algoritmaları—sır alı arama, ikili arama ve ekleme sıralaması—tanıtacağız. Sıralı arama algorit masını tanıtarak başlayalım.

Sıralı Arama Algoritması

Bir listedeki belirli bir hedef değerin varlığını arama problemini düşünelim. Bu değerin listede olup olmadığını belirleyen bir algoritma geliştirmek istiy oruz. Değer listede varsa, aramayı başarılı kabul ederiz; aksi takdirde başarı sız kabul ederiz. Listenin, girişlerini sıralamak için bazı kurallara göre sıralan dığını varsayıyoruz. Örneğin, liste bir isimler listesi ise, isimlerin alfabetik sır ayla göründüğünü varsayıyoruz; ya da liste sayısal değerlerden oluşuyorsa, girişlerin artan büyüklük sırasına göre göründüğünü varsayıyoruz.

Kapıyı aralamak için, belki 20 girişten oluşan bir misafir listesinde belirli bir ismi nasıl arayabileceğimizi hayal ediyoruz. Bu durumda, listenin başınd an başlayarak her girişi hedef isimle karşılaştırarak listeyi tarayabiliriz. Hede f ismi bulursak, arama başarıyla sona erer. Ancak, hedef değeri bulmadan li stenin sonuna ulaşırsak, arama başarısızlıkla sona erer. Aslında, hedefi bul madan hedef isimden (alfabetik olarak) daha büyük bir isme ulaşırsak, aram a başarısızlıkla sona erer. (Unutmayın, liste alfabetik sıraya göre düzenlenm iştir, bu nedenle hedef isimden daha büyük bir isme ulaşmak, hedefin listed e görünmediğini gösterir.) Özetle, kaba fikrimiz, araştırılacak daha fazla isi m olduğu sürece ve hedef ismin, şu anda değerlendirilen isimden daha büy ük olduğu sürece listeyi aramaya devam etmektir.

Bizim sözdekodumuzda, bu süreç şu şekilde temsil edilebilir

```
Listede ilk girişi TestEntry olarak seçin.
iken(TargetValue > TestEntry ve
          değerlendirilecek girişler kalmışken)
   yap(Listede bir sonraki girişi TestEntry olarak seçin)
```

Bu iken yapısını sonlandırdığımızda, iki durumdan biri doğru olacaktır: ya hedef değer bulunmuştur ya da hedef değer listede yoktur. Her iki duru mda da test girişini hedef değerle karşılaştırarak başarılı bir arama tespit e debiliriz. Eğer eşitlerse, arama başarılı olmuştur. Böylece şunu ekliyoruz

```
eğer(TargetValue = TestEntry)
   o zaman(Aramayı başarılı olarak ilan et.)
   aksi(Aramayı başarısız olarak ilan et.)
```

sözdekod rutinimizin sonuna.

Son olarak, rutinimizdeki ilk ifadenin, listede test girişi olarak ilk girişi seçti ğini gözlemliyoruz ve bu, söz konusu listenin en az bir giriş içerdiği varsayımına dayanmaktadır. Bunun güvenli bir tahmin olduğunu düşünebiliriz, ancak emin olmak için, rutinimizi ifadenin aksi seçeneği olarak konumlandırabiliriz

Doclingo https://www.doclingo.ai

eğer (Liste boş)
o zaman(Başarısız bir arama olarak bildir.)
aksi takdirde(. . .)

Bu, Şekil 5.6'da gösterilen prosedürü üretir. Bu prosedürünbaşka prosedürler için de, aşağıdaki gibi ifadeler kullanılarak kullanılabileceğini unutmayın

Yolcu listesine Darrel Baker'ı hedef değer olarak kullanarak A
rama prosedürünü uygulayın.

Darrel Baker'ın bir yolcu olup olmadığını öğrenmek için

Malzemeler listesine muskatı hedef değer olarak kullanarak Arama
prosedürünü uygulayın.

muskatın malzemeler listesinde yer alıp almadığını öğrenmek için.

Özetle, Şekil 5.6'da temsil edilen algoritma, listedeki girişlerisırasıyla dikk ate alır. Bu nedenle, algoritmaya sıralı arama algoritması denir. Basitliği ned eniyle,kısa listeler için veya diğer endişelerin kullanımını gerektirdiği duruml arda sıklıkla kullanılır. Ancak,uzun listelerde, sıralı aramalar diğer teknikler k adar verimli değildir (bunu yakında göreceğiz).

Döngü Kontrolü

Bir talimatın veya talimat dizisinin tekrarlı kullanımı, önemli biralgoritmik kav ramdır. Böyle bir tekrarı uygulamanın bir yöntemi, döngü olarak bilinen yin elemeli yapıdır; burada,talimatlar topluluğu, döngünün gövdesi olarak adlan dırılır, bazı kontrol süreçlerinin yönlendirmesi altında tekrarlı bir şekilde yürü tülür. Tipik bir örnek, Şekil 5.6'da temsil edilen sıralı arama algoritmasında b ulunur.Burada, `while` ifadesini, List'teki bir sonraki girişi TestEntry olarak seç ifa desinin tekrarlanmasını kontrol etmek için kullanıyoruz.
Gerçekten, `while` ifadesi

`while` (*koşul*) **`do`** (*body*)

Şekil **5.6** Sıralı arama algoritması sözdekodda

```
prosedür  Arama (Liste, HedefDeğer)
if  (Liste boş)
    o zaman
      (arama başarısız olarak ilan edilir)
    aksi takdirde
      (Liste'deki ilk girişi TestEntry olarak seç;
       while  (HedefDeğer > TestEntry ve
                          dikkate alınacak girişler kalmışsa)
            do  (Liste'deki bir sonraki girişi TestEntry olarak seç.);
       if  (HedefDeğer = TestEntry)
            sonra  (Aramayı başarılı olarak ilan et.)aksi  (
            Aramayı başarısız olarak ilan et.)      ) eğe
      r son
```

Doclingo https://www.doclingo.ai

bir döngü yapısının kavramını örnekler çünkü yürütmesi döngüselşemayı izl er

```
şartı kontrol et.
gövdeyi çalıştır.
şartı kontrol et.
gövdeyi çalıştır.
   .
   .
   .
şartı kontrol et.
```

şart başarısız olana kadar.

Genel bir kural olarak, bir döngü yapısının kullanımı, gövdeyi birkaç kez açıkça yazarak elde edilebilecek olandan daha yüksek bir esneklik derecesi ü retir. Örneğin, ifadeyi yürütmek için

```
Bir damla sülfürik asit ekleyin.
```

üç kez yazabiliriz:

```
Bir damla sülfürik asit ekleyin.
Bir damla sülfürik asit ekleyin.
Bir damla sülfürik asit ekleyin.
```

Ancak döngü yapısına eşdeğer benzer bir diziyi üretemeyiz

```
iken(pH seviyesi 4'ten büyük) yap
  (bir damla sülfürik asit ekle)
```

çünkü kaç damla asidin gerekli olacağını önceden bilmiyoruz.

Şimdi döngü kontrolünün bileşimine daha yakından bakalım. Bu döngü y apısının bu kısmını önemsiz olarak görmeye teşvik edilebilirsiniz. Sonuçta, genellikle döngünün gövdesi, mevcut görevi (örneğin, asit damlalar ını eklemek) yerine getiren kısımdır—kontrol faaliyetleri, gövdeyi tekrarlayan bir şekilde yürütmeyi seçtiğimiz için yalnızca üst yük olarak görünmektedir. Ancak, deneyimler göstermiştir ki, bir döngünün kontrolü yapının daha hata yapmaya yatkın kısmıdır ve bu nedenle dikkatimize değer.

Bir döngünün kontrolü, başarılı döngü kontrolü için her birinin varlığının gere kli olduğu üç aktiviteden oluşur: başlatma, test etme ve değiştirme (Şekil 5.7). Test aktivitesi, döngüleme sürecinin sona ermesini sağlamakla yükümlüdür; bu, sona ermenin gerçekleşmesi gerektiğini gösteren bir durumu izleyerek yapılır. Bu durum, sona erme durumu olarak bilinir. Bu test aktivitesinin amacı doğ rultusunda, her `while` ifadesinde bir koşul sağlarız. Ancak, `while` ifadesinde belirtilen koşul, döngünün gövdesinin yürütülmesi gereken koşuldur—sona erme durumu, while yapısında görünen koşulun olumsuzluğudur. Böylece, if adede

```
iken(pH seviyesi 4'ten büyük) yap
  (bir damla sülfürik asit ekle)
```

Doclingo https://www.doclingo.ai

Şekil **5.7**Tekrarlayıcı kontrol bileşenleri

Başlat:	Sonlandırma koşusuna doğru değiştirilecek bir başlangıç d urumu oluşturun
Test:	Mevcut durumu sonlandırma koşusuyla karşılaştırın ve eşitse tekrarı sonlandırın
Değiştir:	Durumu, sonlandırma koşuluna doğru hareket edecek şekilde değiştirin

sonlandırma koşulu "pH seviyesi 4'ten büyük değildir" ve Şekil 5.6'daki while ifadesinde, sonlandırma koşulu şöyle ifade edilebilir

(HedefDeğer TestGirişi) veya (dikkate alınacak daha fazla giriş yok)D

öngü kontrolündeki diğer iki etkinlik, sonlandırmakoşulunun nihayetind e gerçekleşmesini sağlar. Başlatma adımı bir başlangıçkoşulu belirler ve değiştirme adımı bu koşulu sonlandırmakoşuluna doğru hareket ettirir. Örneğin, Şekil 5.6'da, başlatma ifadesi `while` ifadesinden önceki ifadede gerçekleşir, burada mevcut test girişiilk liste girişi olarak belirlenir. Bu durumda değiştirme adımı aslında döngü gövdesi içinde gerçekleştirilir, burada ilgi alanımız (test girişi ilebelirtilen) listenin sonuna doğru hareket ettirilir. Böylece, başlatma adımınıuyguladıktan sonra, değiştirme adımının tekrarlı uygulanması,sonlandırma koşuluna ulaşmamızı sağlar. (Ya hedef değere eşit veyabüyük bir test girişine ulaşacağız ya da nihayetinde listenin sonuna ulaşacağız.)

Başlatma ve değiştirme adımlarının uygun sonlandırma koşulunaulaşmasını gerektiğini vurgulamalıyız. Bu özellik, doğru döngü kontrolü için kritikönemesahiptir, bu nedenle bir döngü yapısı tasarlarken her zamanvarlığını iki kez kontrol etmek gerekir. Böyle bir değerlendirmeninyapılmaması, en basit durumlarda bile hatalara yol açabilir. Tipik bir örnek,ifadelerde bulunur

```
Numara←1 ;
iken(Numara≠6 ) do
  (Numara←Numara + 2)
```

Burada sonlandırma koşulu "Numara = 6." Ancak, Numara değeri 1 olarak başlatılır ve ardından değişiklik adımında 2 artırılır. Böylece, döngü dönerken, Numara'ya atanan değerler 1, 3, 5, 7, 9, vb. olacak, ancak asla 6 değeri olmayacaktır. Dolayısıyla, döngü asla sonlanmayacaktır.

Döngü kontrol bileşenlerinin yürütülme sırası ince sonuçlar doğurabilir. Aslında, yalnızca bu açıdan farklılık gösteren iki yaygın döngü yapısı vardır. İlki, sözdekod ifademizle örneklendirilmiştir.

iken (*koşul*) yap (*faaliyet*)

Doclingo https://www.doclingo.ai

anlamlarının Şekil 5.8'de bir akış diyagramı biçiminde temsil edildiği (Bu tür diyagramlar, bireysel adımları temsil etmek için çeşitli şekiller kullanır ve adımların sırasını belirtmek için oklar kullanır. Şekiller arasındaki ayrım, il gili adımda yer alan eylem türünü gösterir. Bir elmas, bir kararı ve bir dikdör tgen, keyfi bir ifadeyi veya ifadeler dizisini gösterir.) `while` yapısındaki sonla ndırma testi, döngünün gövdesi çalıştırılmadan önce gerçekleşir.

Buna karşılık, Şekil 5.9'daki yapı, döngünün gövdesininsonlandırma testi yapılmadan önce yürütülmesini talep eder. Bu durumda, döngününgövdesi her zaman en az bir kez gerçekleştirilirken, while yapısında,gövde, sonlandır ma koşulu ilk kez test edildiğinde sağlanıyorsa asla yürütülmez.

Sentaktik formu kullanacağız

tekrar (*aktivite*) sonra (*koşul*)

Şekil 5.9'da gösterilen yapıyı temsil etmek için sözdekodumuzda. Böylece, ifade

tekrar(cebinden bir madeni para al)
sonra(cebinde hiç madeni para kalmadığında)

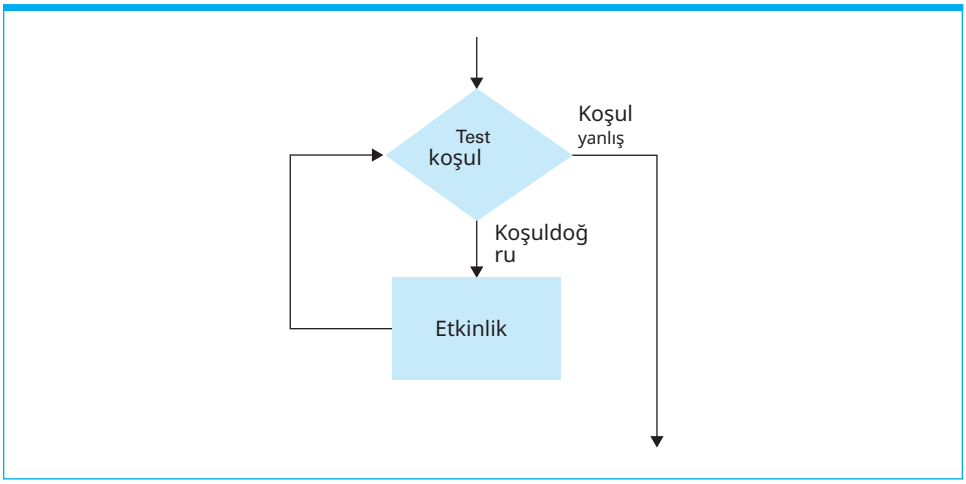
başlangıçta cebinde bir madeni para olduğunu varsayıyor, ancak

`while`(cebinde bir madeni para varken) **`do`**
(cebinden bir madeni para al)

bunu yapmaz.

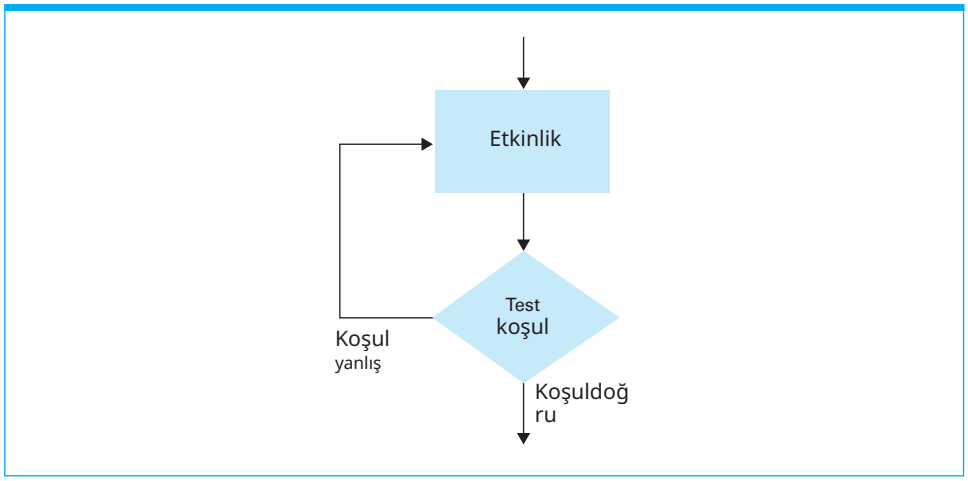
Sözdekodumuzun terminolojisini takip ederek, buyapılara genellikle `while` döngü yapısı veya `repeat` döngü yapısı olarak atıfta bulunacağız. Dahagenel bi r bağlamda, `while` döngü yapısının ön test döngüsü (çünkü sonlandırma testi, gövde yürütülmeden önce yapılır) olarak adlandırıldığını ve `repeat` döngü yapı sının son test döngüsü (çünkü sonlandırma testi,gövde yürütüldükten sonra y apılır) olarak adlandırıldığını duyabilirsiniz.

Şekil **5.8** While döngü yapısı



Doclingo https://www.doclingo.ai

Şekil **5.9**Tekrar döngüsü yapısı



Ekleme Sıralaması Algoritması

Yinelemeli yapıların bir ek örneği olarak, isimler listesini alfabetik sıraya koy ma problemini ele alalım. Ancak ilerlemeden önce, çalışacağımız kısıtlamalar ı belirlemeliyiz. Basitçe ifade etmek gerekirse, amacımız listeyi "kendi içinde" sıralamaktır. Diğer bir deyişle, listeyi başka bir yere taşımak yerine, girişlerini karıştırarak sıralamak istiyoruz. Durumumuz, girişlerin ayrı indeks kartları ü zerinde kaydedildiği ve kalabalık bir masaüstünde yayıldığı bir listeyi sıralam a problemiyle benzerlik göstermektedir. Kartlar için yeterince alan açtık ama daha fazla alan yaratmak için ek malzemeleri geri itmemize izin verilmiyor. B u kısıtlama, bilgisayar uygulamalarında tipiktir; çünkü makine içindeki çalışm a alanı mutlaka kalabalık olduğu için değil, sadece mevcut depolama alanını verimli bir şekilde kullanmak istediğimiz için.

Masaüstündeki isimleri nasıl sıralayabileceğimizi düşünerek kapıyı aralay alım. İsimler listesini düşünün

Fred
Alex
Diana
Byron
Carol

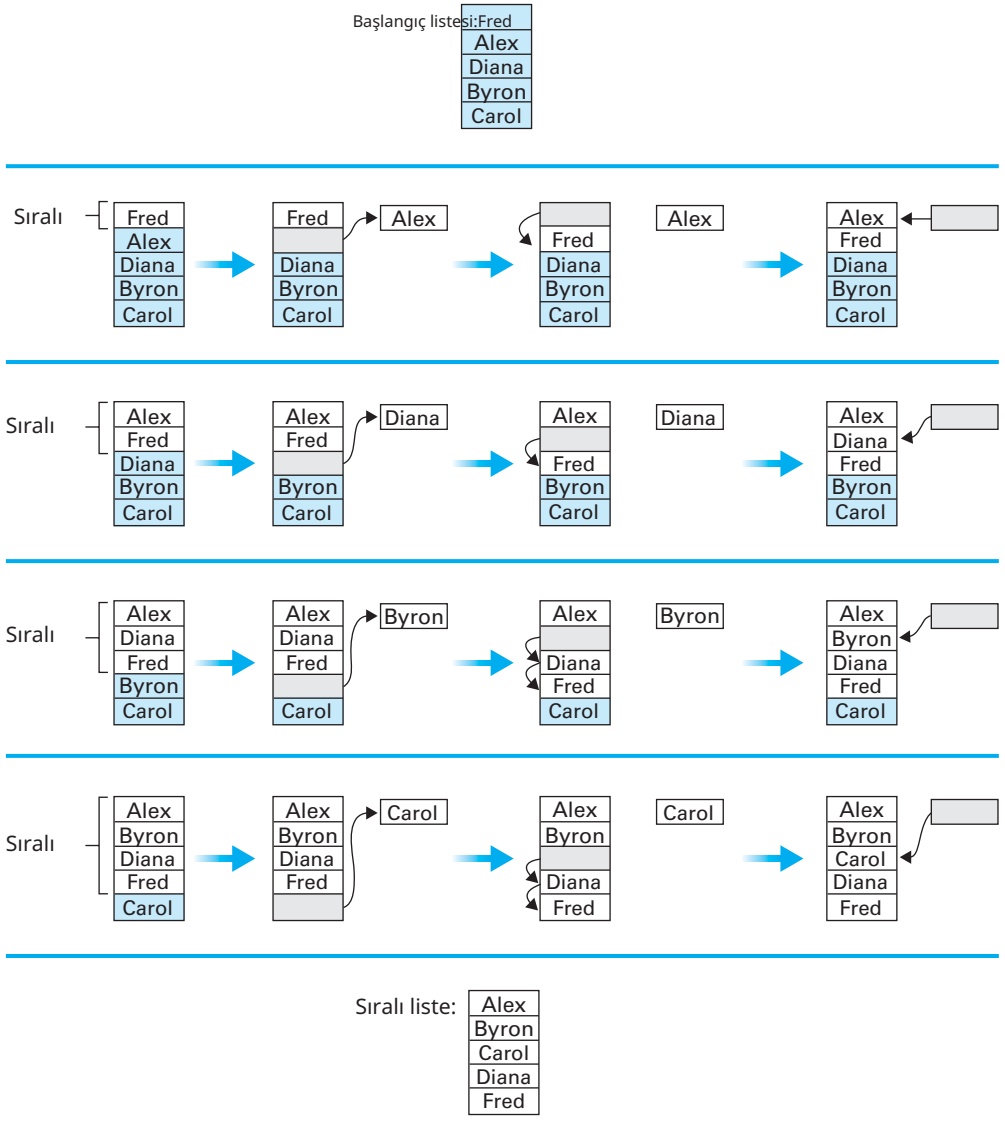
Bu listeyi sıralamanın bir yolu, yalnızca en üstteki isim olan Fred'den oluşan alt list enin sıralı olduğunu, ancak en üstteki iki isim olan Fred ve Alex'ten oluşan alt liste nin sıralı olmadığını not etmektir. Böylece Alex ismini içeren kartı alabilir, Fred ism ini Alex'in bulunduğu yere kaydırabilir ve ardından Alex ismini listenin en üstünde ki boşluğa yerleştirebiliriz; bu, Şekil 5.10'daki ilk satırda temsil edilmektedir. Bu no ktada listemiz şöyle olacaktır:

Alex
Fred\i
nDiana
\Byron
Carol

Doclingo https://www.doclingo.ai

Artık en üstteki iki isim sıralı bir alt liste oluşturuyor, ancak en üstteki üç isi m oluşturmuyor. Bu nedenleüçüncü ismi, Diana'yı alabiliriz, Fred ismini Diana'n ın olduğu yere kaydırabiliriz ve ardından Diana'yı Fred'in bıraktığı yere yerleştir ebiliriz, bu daŞekil 5.10'un ikinci satırında özetlenmiştir. Listede en üstteki üç gi riş artıksıralı olacaktır. Bu şekilde devam ederek, dördüncü ismi, Byron'u alarak , Fredve Diana isimlerini aşağı kaydırarak ve ardından Byron'u boşluğa yerleştir erek sıralı dörtgiriş elde edebiliriz (Şekil 5.10'un üçüncü satırına bakın). Son olar ak, Carol'ı alarak,Fred ve Diana'yı aşağı kaydırarak ve ardından Carol'ı kalan bo şluğa yerleştirerek sıralama işlemini tamamlayabiliriz(Şekil 5.10'un dördüncü s atırına bakın).

Şekil **5.10**Fred, Alex, Diana, Byron ve Carol'ı alfabetik olarak sıralama



Doclingo https://www.doclingo.ai

Belirli bir listeyi sıralama sürecini analiz ettikten sonra, şimdi görevimiz b usüreci genelleştirerek genel listeleri sıralamak için bir algoritma elde etmek tir. Buamaçla, Şekil 5.10'un her satırının aynı genel süreci temsil ettiğini gözl emliyoruz: Sıralanmamışlistenin ilk ismini al, çıkarılan ismin üstündeki isimle ri aşağı kaydır ve çıkarılan ismiboşluğun olduğu yere geri yerleştir. Eğer çıkar ılan ismi pivot girişi olarak tanımlarsak,bu süreç sözdekodumuzda şu şekilde ifade edilebilir

Pivot girişini geçici bir konuma taşıyarak List'te bir boşluk bırakın;

while(boşluğun üzerinde bir isim varsa ve
bu isim pivot'tan büyükse) **do**
(boşluğun üzerindeki ismi aşağıya boşluğa taşıyın
ismin üzerinde bir boşluk bırakarak)
Pivot girişini List'teki boşluğa taşıyın.

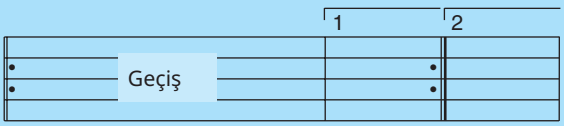
Sonra, bu işlemin tekrar tekrar gerçekleştirilmesi gerektiğini gözlemliyor uz. Sıralama işlemine başlamak için,pivot listedeki ikinci giriş olmalı ve ardın dan,her ek yürütmeden önce, pivot seçimi listedeki bir giriş daha aşağıda ol malıdırta ki son giriş konumlandırılana kadar. Yani, öncekirutin tekrarlandıkç a, pivot girişinin başlangıç konumu ikinci girişten üçüncüye,dördüncüye vb. il erlemelidir, ta ki rutin listedeki son girişi konumlandırana kadar. Bu yönde, g ereklitekrarı kontrol edebiliriz.

Müzikte Yinelemeli Yapılar

Müzisyenler, bilgisayar bilimcilerinden yüzyıllar önce yinelemeli yapıları kullanıyor ve programlıyordu. Gerçekten de, bir şarkının yapısı (birçok dizeden oluşması,here bir koro ile takip edilmesi) while ifadesiyle örneklendirilmektedir.

while (bir dize kalmışsa) do
(bir sonraki dizeyi s
öyle;koro söyle)

Ayrıca, notasyon



yalnızca bir bestecinin yapıyı ifade etme yoludur

```
N ← 1;
while (N < 3) do
 (play the passage;
  play the Nth ending;
  N ← N + 1)
```

Doclingo https://www.doclingo.ai

```
N←2;
while (N'nin değeri List'in uzunluğunu aşmadığı sürece) do
  (List'teki N'inci girişi pivot girişi olarak seçin;
     .
     .
     .
   N ←N + 1)
```

burada N pivot girişi için kullanılacak konumu temsil eder, List'in uzunluğu listedeki giriş sayısını ifade eder ve noktalar, önceki rutin yerleştirilmesi gereken yeri belirtilmektedir.

Tam pseudocode programımız Şekil 5.11'de gösterilmiştir. Kısacası, program bir listeyi tekrar tekrar bir girişi çıkararak ve onu uygun yerine yerleştirerek sıralar. Bu tekrar eden ekleme süreci nedeniyle, temel algoritmaya ekleme sıralaması denir.

Şekil 5.11'in yapısının, dış döngünün ilk `while` ifadesiyle ifade edildiği ve iç döngünün ikinci `while` ifadesiyle temsil edildiği bir döngü içinde döngü olduğunu unutmayın. Dış döngünün gövdesinin her bir yürütülmesi, iç döngünün başlatılmasına ve sonlandırma koşulu elde edilene kadar yürütülmesine neden olur. Böylece, dış döngünün gövdesinin tek bir yürütülmesi, iç döngünün gövdesinin birkaç yürütülmesiyle sonuçlanır.

Dış döngünün kontrolünün başlatma bileşeni, N'nin başlangıç değerini belirlemekten oluşur.

```
N ← 2;
```

Değiştirme bileşeni, döngünün gövdesinin sonunda N'nin değerini artırarak işlenir.

```
N ← N + 1
```

Sonlandırma koşulu, N'nin listenin uzunluğunu aşması durumunda gerçekleşir.

İç döngünün kontrolü, listeye pivot girişini kaldırarak başlatılır ve böylece bir delik oluşturulur. Döngünün değiştirme adımı, girişlerin deliğe doğru kaydırılmasıyla gerçekleştirilir, bu da deliğin yukarı hareket etmesine neden olur. Sonlandırma koşulu, deliğin pivot değerinden büyük olmayan bir ismin hemen altında olması veya deliğin listenin en üstüne ulaşmasıdır.

Şekil **5.11** Ekleme sıralaması algoritması sözdekodda ifade edilmiştir

```
prosedür  Sırala (Liste)
N ← 2;
iken  (N'nin değeri Listenin uzunluğunu aşmaz)  yap
        (Listede N'inci girişi pivot girişi olarak seç;
     Pivot girişini geçici bir yere taşıyarak Listede bir delik bırak;
     iken  (delik üzerinde bir isim varsa ve o isim pivot'tan büyükse)
          yap  (delik üzerindeki ismi aşağıya delik içine taşı          is
               min üzerinde bir delik bırakarak)
     Pivot girişini Listenin deliğine yerleştir;
        N ← N + 1
     )
```

Doclingo https://www.doclingo.ai

Sorular ve Alıştırmalar

1. Şekil 5.6'daki sıralı arama prosedürünü, sıralanmamış listelere izin verece k şekilde değiştirin.
2. Sözdekod rutinini dönüştür

```
Z←0;
X←1;
while(X < 6) do
 (Z←Z + X;
  X←X + 1)
```

eşdeğer bir rutine `repeat` ifadesi kullanarak dönüştür.

3. Bugünün popüler programlama dillerinden bazıları,

yaparken (. . .) yaparken (. . .)

bir ön test döngüsünü temsil etmek için ve

yapmak (. . .) yaparken (. . .)

bir son test döngüsünü temsil etmek için sözdizimini kullanır. Tasarım açısından zarif olmasına rağmen, böyle benzerliklerden hangi sorunlarortaya çıkabilir?

4. Şekil 5.11'de sunulan ekleme sıralaması, Gene, Cheryl, Alice ve Brend a listesini uygulandığını varsayalım. Dış `while` yapısının gövdesinin h er yürütülmesinin sonunda listenin organizasyonunu tanımlayın.
5. Şekil 5.11'deki while ifadesindeki "büyüktür" ifadesini "büyüktür veya e şittir" olarak değiştirmek istememizin nedeni nedir?
6. Ekleme sıralama algoritmasının bir varyasyonu seçim sıralamasıdır. Bu, listede en küçük girişi seçerek ve onu ön tarafa taşıyarak başlar. Ardından, listede kalan girişlerden en küçük girişi seçer ve onu listen in ikinci konumuna taşır. Listede kalan kısmından en küçük girişi tekr ar tekrar seçerek ve o girişi öne taşıyarak, sıralı listenin versiyonu list enin önünden büyürken, geri kalan kısmı ise sıralanmamış girişlerde n oluşan kısım küçülür. Seçim sıralama algoritmasını kullanarak bir li steyi sıralamak için Şekil 5.11'dekiyle benzer bir prosedürü ifade etm ek için sözdekodumuzu kullanın.
7. Diğer bir iyi bilinen sıralama algoritması balon sıralamasıdır. Bu, iki bitişik ismi sürekli olarak karşılaştırma ve eğer doğru sırada değillers e yer değiştirme sürecine dayanır.
   Varsayalım ki söz konusu listedeki n girdi var. Balon sıralaması, n ve$n$ – 1 konumlarındaki girdileri karşılaştırarak (ve muhtemelen yer değiştire rek) başlayacaktır. Ardından, n – 1 ve n – 2 konumlarındaki girdileri dikk ate alacak ve listenin başına kadar ilerlemeye devam edecektir; böylece listenin birinci ve ikinci girdileri karşılaştırılmış (ve muhtemelen yer de ğiştirmiş) olacaktır. Bu geçişin, en küçük girdiyi listenin önüne çektiğini gözlemleyin. Benzer şekilde, bir başka geçiş, en küçük girdinin ikinci ko numa çekilmesini sağlayacaktır. Böylece, toplamda n – 1 geçiş yaparak, tüm liste sıralanmış olacaktır. (Algoritmayı çalışırken izleyen biri, küçük girdilerin listenin üstüne baloncuk gibi çıktığını görür—bu gözlem, alg oritmanın adını aldığı gözlemdir.) Balon sıralama algoritmasını kullanar ak bir listeyi sıralamak için Şekil 5.11'dekiyle benzer bir prosedürü ifade etmek için sözdekodumuzu kullanın.

Doclingo https://www.doclingo.ai

5.5 Özyinelemeli Yapılar

Özyinelemeli yapılar, aktivitelerin tekrarı için döngü paradigmasına alternatif sağlar. Bir döngü, bir dizi talimatı tamamlayıp ardından tekrar eden bir şekil de tekrarlarken, özyineleme, talimatlar setini kendisinin bir alt görevi olarak tekrarlamayı içerir. Bir benzetme olarak, çağrı bekleme özelliği ile telefon gö rüşmeleri yapma sürecini düşünelim. Burada, tamamlanmamış bir telefon g örüşmesi bir kenara bırakılırken başka bir gelen çağrı işlenir. Sonuç olarak, ik i görüşme gerçekleşir. Ancak, bunlar bir döngü yapısında olduğu gibi ardışık olarak gerçekleştirilmez, bunun yerine biri diğerinin içinde gerçekleştirilir.

İkili Arama Algoritması

Özyinelemeyi tanıtmanın bir yolu olarak, belirli bir kaydın sıralı bir listede olup olmadığını arama sorununu tekrar ele alalım, ancak bu sefer sözlükte arama ya parken izlediğimiz prosedürü dikkate alarak kapıyı aralıyoruz. Bu durumda, ard ışık olarak her bir girişi veya hatta sayfa sayfa bir prosedür gerçekleştirmiyoruz . Bunun yerine, hedef kaydın bulunduğunu düşündüğümüz alandaki bir sayfayı açarak başlıyoruz. Eğer şanslıysak, hedef değeri orada bulacağız; aksi takdirde, aramaya devam etmemiz gerekecek. Ancak bu noktada aramamızı önemli ölçü de daraltmış olacağız.

Elbette, bir sözlükte kelime ararken, kelimelerin nerede bulunabileceğin e dair önceden bir bilgimiz var.Eğer somnambulism kelimesini arıyorsak, söz lüğün son kısmını açarak başlamalıyız. Ancak genel listelerde bu avantajımız yok, bu yüzden aramamıza her zaman listenin “orta” girişinden başlamayı ka bul edelim. Burada kelimeyi *orta* tırnak içinde yazıyoruz çünkü liste çift sayıd a girişe sahip olabilir ve bu nedenle tam anlamıyla bir orta giriş olmayabilir. Bu durumda, “orta” girişinin listenin ikinci yarısındaki ilk girişe atıfta bulundu ğunu kabul edelim.

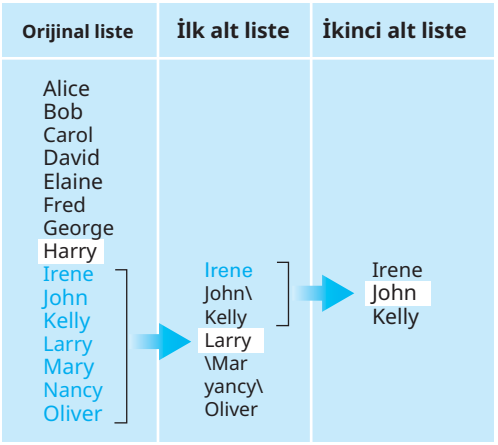
Listede orta giriş hedef değer ise, aramayı başarılı olarak ilan edebiliriz. Aksi takdirde, hedef değerin, dikkate aldığımız girişten daha az veya daha fa zla olup olmadığına bağlı olarak arama sürecini listenin ilk veya son yarısıyla sınırlayabiliriz. (Listenin sıralı olduğunu unutmayın.)

Listenin kalan kısmını aramak için sıralı arama uygulayabiliriz, ancak bun un yerine bu kısmı, tüm liste için kullandığımız aynı yaklaşımı uygulayarak in celeyelim. Yani, listenin kalan kısmındaki orta girişi bir sonraki dikkate alacağ ımız giriş olarak seçiyoruz. Önceki gibi, eğer o giriş hedef değer ise, işimiz bit ti. Aksi takdirde, aramamızı listenin daha da küçük bir kısmıyla sınırlayabiliriz .

Bu arama sürecine yaklaşım, Şekil 5.12'de özetlenmiştir; buradaşeklin solu ndaki listedeki John girişini arama görevini ele alıyoruz. Öncelikle ortadaki gi riş Harry'i göz önünde bulunduruyoruz. Hedefimiz bu girişten sonra geldiği i çin, arama orijinal listenin alt yarısını dikkate alarak devam ediyor.
Bu alt listenin ortası Larry olarak bulunmuştur. Hedefimiz Larry'den önce gelm esi gerektiğinden, mevcut alt listenin ilk yarısına dikkatimizi yönlendiriyoruz. O ikincil alt listenin ortasını sorguladığımızda, hedefimiz John'u buluyoruz ve ara mayı başarılı olarak ilan ediyoruz. Kısacası, stratejimiz, hedef bulunana kadar v eya arama boş bir parçaya daralana kadar söz konusu listeyi ardışık olarak dah a küçük parçalara bölmektir.

Doclingo https://www.doclingo.ai

Şekil **5.12** Stratejimizi John kaydını aramak için uygulamak



Bu son noktayı vurgulamamız gerekiyor. Hedef değer orijinal listede yok sa, listeyi daha küçük parçalara bölerek arama sürecimiz devam edecektir; b u noktada dikkate alınan parça boş olduğunda algoritmamız aramanın başar ısız olduğunu tanımalıdır.

Şekil 5.13, sözdekodumuzu kullanarak düşüncelerimizin ilk taslağıdır. Bize aram aya başlamak için listenin boş olup olmadığını test etmemizi yönlendirir. Eğer öyleys e, aramanın başarısız olduğunu bildirmemiz gerektiği söylenir. Aksi takdirde, listede ortadaki girişi dikkate almamız gerektiği belirtilir. Bu giriş hedef değer değilse, listen in ön yarısını veya arka yarısını aramamız gerektiği söylenir. Bu iki olasılık da ikincil b ir arama gerektirir. Bu aramaları soyut bir aracın hizmetlerini çağırarak gerçekleştir mek güzel olurdu. Özellikle, yaklaşımımız adında bir prosedür uygulamaktır Arama bu ikincil aramaları gerçekleştirmek için. Programımızı tamamlamak iç in, böyle bir prosedür sağlamalıyız.

Şekil **5.13** İkili arama tekniğinin ilk taslağı

```
eğer (Liste boşsa)
öyleyse
 (Aramanın başarısız olduğunu bildir.)
aksi takdirde
 [TestEntry olarak Listede "orta" girişi seç;
    Aşağıdaki talimatlar bloğunu çalıştır
   uygun durumla ilişkili olan.
    durum 1: TargetValue = TestEntry
                (Aramanın başarılı olduğunu bildir
    .)      durum 2: TargetValue < TestE
            ntry          (TargetValue için TestEntry'den önceki Listen
                in kısmını ara          ve o aramanın sonucunu bildir.)
           durum 3: TargetValue > TestEnt
            ry          (TargetValue için TestEntry'den sonraki Listeni
                n kısmını ara          ve o aramanın sonucunu bildir.)
 ] eğer sonu
```

Doclingo https://www.doclingo.ai

Arama ve Sıralama

Sıralı ve ikili arama algoritmaları, arama sürecini gerçekleştirmek için birçok algoritmadan yalnızca ikisidir. Benzer şekilde, ekleme sıralaması da birçok sıralama algoritmasından yalnızca biridir. Sıralama için diğer klasik algoritmalar arasında birleştirme sıralaması (Bölüm 12'de tartışılmıştır), seçim sıralaması (Bölüm 5.4'teki Soru/Alıştırma 6), balon sıralaması (Bölüm 5.4'teki Soru/Alıştırma 7), hızlı sıralama (sıralama sürecine böl ve fethet yaklaşımını uygulayan) ve yığın sıralaması (listede ileriye taşınması gereken girişleri bulmak için akıllı bir teknik kullanan) bulunmaktadır. Bu algoritmalar hakkında tartışmaları, bu bölümün sonunda Ek Okuma altında listelenen kitaplarda bulacaksınız.

Ancak bu prosedür, daha önce yazdığımız sözdekodda ifade edilen aynı görevi yerine getirmelidir. Öncelikle kendisine verilen listenin boş olup olmadığını kontrol etmelidir; eğer boş değilse, o listenin orta girişini dikkate alarak devam etmelidir. Böylece, ihtiyaç duyduğumuz prosedürü yalnızca mevcut rutini `Search` olarak tanımlayarak ve ikincil aramaların gerektiği yerlerde o prosedüre referanslar ekleyerek sağlayabiliriz. Sonuç Şekil 5.14'te gösterilmiştir.

Bu prosedürün kendisine bir referans içerdiğini unutmayın. Eğer bu prosedürü izliyorsak ve talimata geldiğimizde

```
Prosedürü Uygula Arama . . .
```

aynı prosedürü, orijinal listeye uyguladığımız daha küçük listeye uygularız. Eğer bu arama başarılı olursa, orijinal aramamızı başarılı olarak ilan etmek için geri döneriz; eğer bu ikincil arama başarısız olursa, orijinal aramamızı başarısız olarak ilan ederiz.

Şekil **5.14** İkili arama algoritması sözdekodda

```
prosedür  Arama (Liste, HedefDeğer)
eğer  (Liste boş)
   o zaman
     (Aramanın başarısız olduğunu bildir.)
   aksi takdirde
     [Liste'deki "orta" girişi TestEntry olarak seç;
          Aşağıdaki talimatlar bloğunu çalıştır
               uygun durumla ilişkilidir.
        durum 1: HedefDeğer = TestEntry
                    (Aramanın başarılı olduğunu bildir.)
        durum 2: HedefDeğer < TestEntry
                    (HedefDeğer'in TestEntry'den önceki Liste kısmında
                          olup olmadığını görmek için Arama prosedürünü uygul
                    a,                ve o aramanın sonucunu bildir.)
        durum 3: HedefDeğer > TestEntry
                  (HedefDeğer'in TestEntry'den sonraki Liste kısmında                o
                     lup olmadığını görmek için Arama prosedürünü uygula,
                          ve o aramanın sonucunu bildir.)
     ]  son if
```

Doclingo https://www.doclingo.ai

Şekil 5.14'teki prosedürün görevini nasıl yerine getirdiğini görmek için, h edef değer Bill için Alice, Bill, Carol, David, Evelyn, Fred ve George listesini ar arken onu takip edelim. Armamız, test girişi olarak David'i (orta giriş) seçerek başlar. Hedef değer (Bill) bu test girişinden daha küçük olduğundan, David'i n önündeki girişler listesini—yani Alice, Bill ve Carol listesini—arama prosed ürü `Search`'ü uygulamamız için talimat alıyoruz. Bunu yaparken, arama pros edürünün ikinci bir kopyasını oluşturuyor ve bu ikincil göreve atıyoruz.

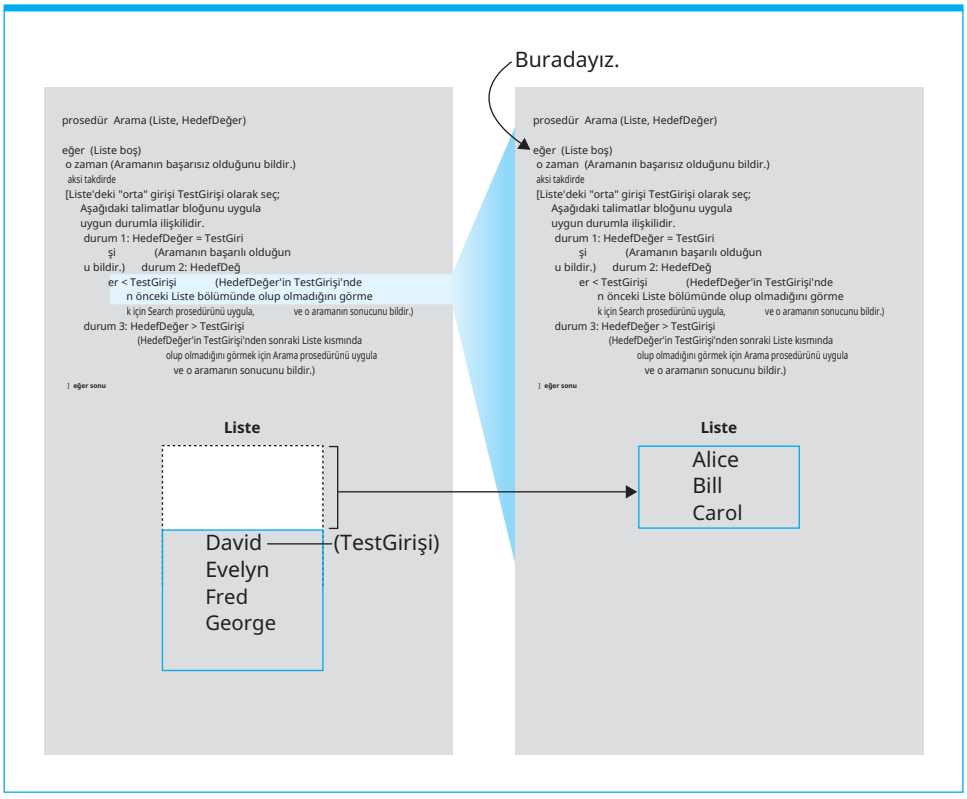
Artık arama prosedürümüzün iki kopyası çalıştırılıyor, Şekil 5.15'te özetle ndiği gibi. Orijinal kopyadaki ilerleme, talimatın üzerinde geçici olarak askıya alındı.

HedefDeğer'in TestGirişi'nden önceki Liste bölümünde olup olma dığını görmek için Search prosedürünü uygulayın.

İkinci kopyayı Alice, Bill ve Carol listesini arama görevine uygularken. Bu ikin cil aramayı tamamladığımızda, prosedürün ikinci kopyasını atacağız, bulgula rını orijinal kopyaya bildireceğiz ve orijinaldeki ilerlemeye devam edeceğiz. B u şekilde, prosedürün ikinci kopyası, orijinalin bir alt birimi olarak çalışır, oriji nal modül tarafından talep edilen görevi yerine getirir ve ardından kaybolur.

İkincil arama, test girişi olarak Bill'i seçer çünkü bu, Alice, Bill ve Carol list esindeki ortada yer alan giriş. Bu, hedef değerle aynı olduğu için, aramasını başarılı olarak ilan eder ve sonlandırır.

Şekil 5.15



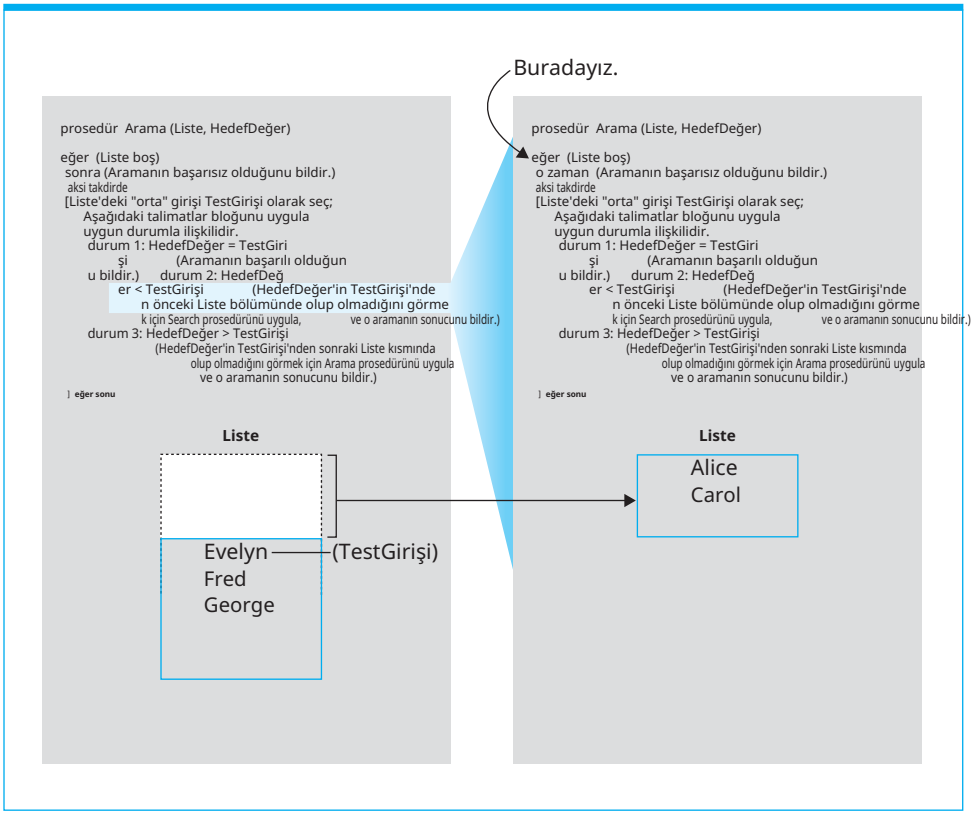
Doclingo https://www.doclingo.ai

Bu noktada, prosedürün orijinal kopyası tarafından talep edilen ikincil ar amayı tamamladık,bu nedenle o orijinal kopyanın yürütülmesine devam ede biliyoruz. Burada, ikincil aramanın sonucununorijinal aramanın sonucu olara k bildirilmesi gerektiği söyleniyor. Böylece orijinalarama başarılı olmuştur di ye bildiriyoruz. Sürecimiz, Bill'in Alice, Bill, Carol, David, Evelyn, Fred ve Geor ge listesininbir üyesi olduğunu doğru bir şekilde belirlemiştir.

Şimdi, Şekil 5.14'teki prosedürü David girişi için Alice, Carol, Evelyn, Fred ve George listesiniarama yapması için sorduğumuzda ne olacağını düşüneli m. Bu sefer prosedürün orijinal kopyasıEvelyn'i test girişi olarak seçer ve hed ef değerin listenin önceki kısmında bulunması gerektiğinesonuçlanır. Bu ned enle, Evelyn'in önünde yer alan girişler listesini araması için başka birprosed ür kopyası talep eder—yani, Alice ve Carol'dan oluşan iki girişli listeyi. Bu aşa madadurumumuz Şekil 5.16'da gösterildiği gibidir.

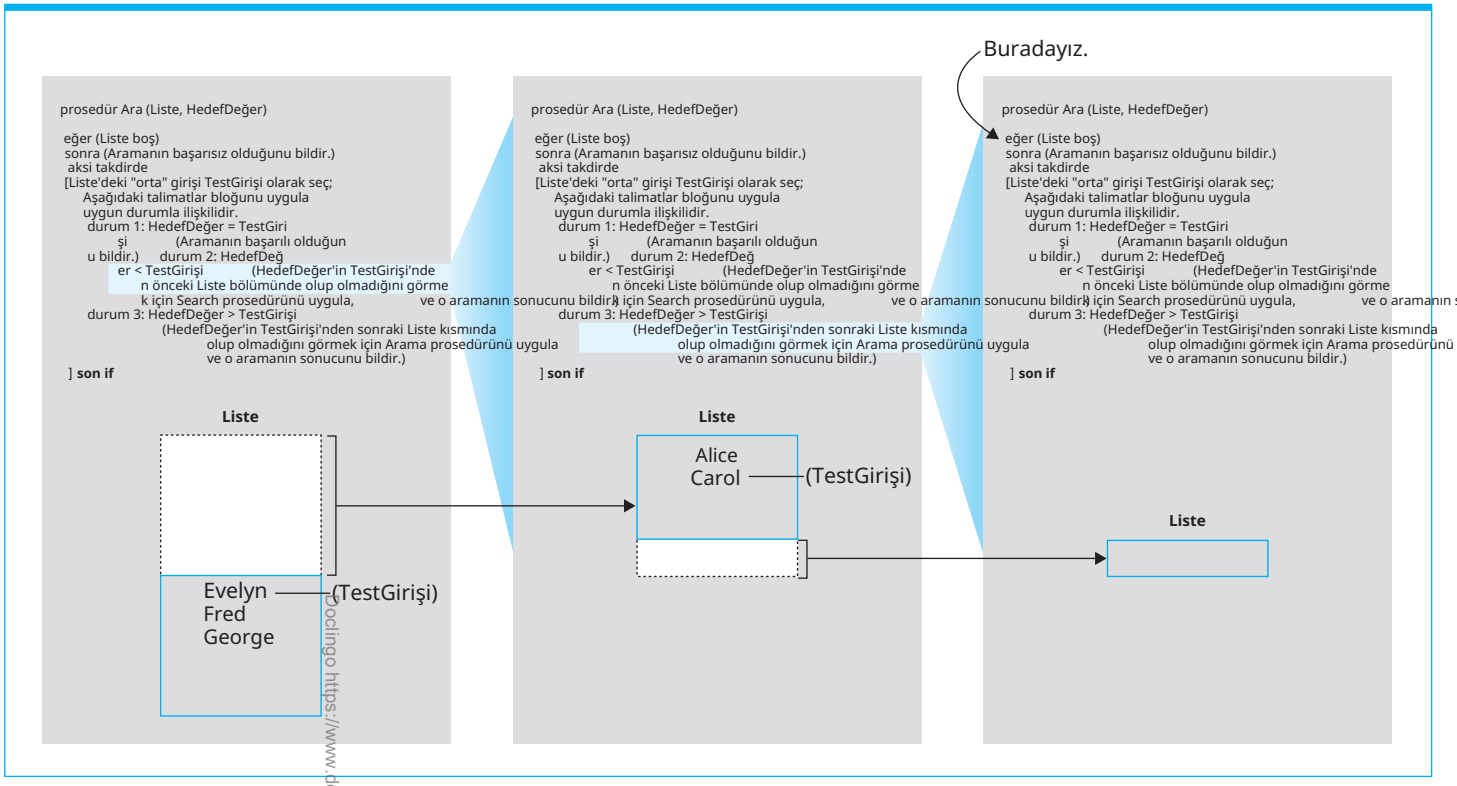
İkinci kopya, Carol'u mevcut girişi olarak seçer ve hedef değerin listesini n son kısmında yer alması gerektiğine karar verir. Ardından, Carol'dan sonra ki isimler listesini aramak için prosedürün üçüncü bir kopyasını talep eder. B u alt liste boştur, bu nedenle prosedürün üçüncü kopyası, hedef değer David 'i bulmak için boş listeyi arama görevine sahiptir. Bu noktadaki durumumuz Şekil 5.17 ile temsil edilmektedir. Prosedürün orijinal kopyası, test girişi Evely n olan Alice, Carol, Evelyn, Fred ve George listesini arama görevini üstlenmiş tir; ikinci kopya ise görevlendirilmiştir.

Şekil5.16



Doclingo https://www.doclingo.ai

Şekil5.17



Doclingo https://www.doclingo.ai

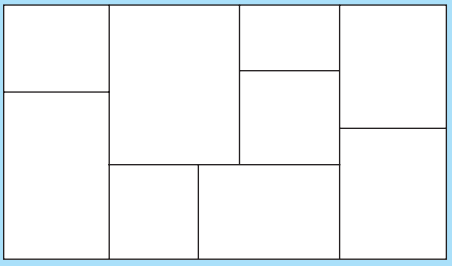
listeyi Alice ve Carol ile ararken, test girişi Carol; ve üçüncü kopya boş listeyi aramaya başlamak üzere.

Elbette, prosedürün üçüncü kopyası aramasını hızla bir başarısızlık olarak ilan eder ve sonlanır. Üçüncü kopyanın görevini tamamlaması, ikinci kopyanın görevine devam etmesine olanak tanır. İstediği aramanın başarısız olduğunu not eder, kendi görevini bir başarısızlık olarak ilan eder ve sonlanır. Bu rapor, prosedürün orijinal kopyasının beklediği şeydir, böylece şimdi devam edebilir. İstediği arama başarısız olduğu için, kendi aramasının da başarısız olduğunu ilan eder ve sonlanır. Rutini, David'in Alice, Carol, Evelyn, Fred ve George listesinin içinde olmadığını doğru bir şekilde sonuçlandırmıştır.

Özetle, önceki örneklere geri dönecek olursak, Şekil 5.14'te temsil edilen algoritmanın kullandığı sürecin, söz konusu listeyi sürekli olarak iki daha küçük parçaya ayırmak olduğunu görebiliriz; böylece kalan arama yalnızca bu parçalardan birine kısıtlanabilir. Bu ikiye ayırma yaklaşımı, algoritmanın ikili arama olarak bilinmesinin nedenidir.

Sanatta Özyinelemeli Yapılar

Aşağıdaki özyinelemeli prosedür, Hollandalı ressam Piet Mondrian'ın (1872–1944) tarzında çizimler üretmek için dikdörtgen bir tuvale uygulanabilir. Mondrian, dikdörtgen tuvali giderek daha küçük dikdörtgenlere bölen resimler üretmiştir. Kendiniz bu prosedürü takip ederek gösterilen benzer çizimleri üretmeyi deneyin. Çalıştığınız tuvali temsil eden bir dikdörtgene prosedürü uygulayarak başlayın. (Bu prosedürle temsil edilen algoritmanın Bölüm 5.1'deki tanıma göre bir algoritma olup olmadığını merak ediyorsanız, şüpheleriniz yerindedir. Bu, aslında, prosedürü takip eden kişi veya makinenin "yaratıcı" kararlar vermesi gereken yerler olduğu için bir belirsiz algoritma örneğidir. Belki de bu yüzden Mondrian'ın sonuçları sanat olarak kabul edilirken, bizimkiler edilmez.)



```
prosedür Mondrian (Dikdörtgen)
eğer (Dikdörtgenin boyutu sanatsal zevkiniz için çok büyükse)
   then (Dikdörtgeni iki daha küçük dikdörtgene ayırın;
         Mondrian prosedürünü daha küçük dikdörtgenlerden birine uygulayın;
         Mondrian prosedürünü diğer daha küçük dikdörtgene uygulayın)
```

Doclingo https://www.doclingo.ai

Özyineleme Kontrolü

İkili arama algoritması, her algoritmanın tekrarlayıcı bir sürecin yürütülmesi ni talep etmesi açısından sıralı arama ile benzerdir. Ancak, bu tekrarlamanın uygulanması önemli ölçüde farklıdır. Sıralı arama, döngüsel bir tekrarlama bi çimini içerirken, ikili arama tekrarlamanın her aşamasını bir önceki aşamanı n alt görevi olarak yürütür. Bu teknik özyineleme olarak bilinir.

Gördüğümüz gibi, bir özyinelemeli prosedürün yürütülmesiyle yaratılan illü zyon, her biri prosedürün bir aktivasyonu olarak adlandırılan, prosedürün bird en fazla kopyasının varlığıdır. Bu aktivasyonlar, teleskopik bir şekilde dinamik o larak oluşturulur ve nihayetinde algoritma ilerledikçe kaybolur. Herhangi bir an da var olan bu aktivasyonlardan yalnızca biri aktif olarak ilerlemektedir. Diğerle ri etkili bir şekilde belirsizlikte, her biri başka bir aktivasyonun sona ermesini be klemektedir.

Tekrarlayıcı bir süreç olması nedeniyle, özyinelemeli sistemler döngü yap ıları kadar doğru kontrol gerektirir. Döngü kontrolünde olduğu gibi, özyinele meli sistemler bir sonlandırma koşulunu test etmeye ve bu koşulun ulaşılma sını sağlayan bir tasarıma bağımlıdır. Aslında, doğru özyineleme kontrolü, dö ngü kontrolünde gereken başlangıç, değişiklik ve sonlandırma testi olmak üz ere üç bileşeni içerir.

Genel olarak, özyinelemeli bir prosedür, daha fazla aktivasyon talep etmeden önce sonlandırma koşulunu (genellikle temel durum veya dejeneratif durum olar ak adlandırılır) test etmek için tasarlanmıştır. Eğer sonlandırma koşulu sağlanmaz sa, rutin prosedürün başka bir aktivasyonunu oluşturur ve ona mevcut aktivasyon a atanan problemden sonlandırma koşuluna daha yakın olan revize edilmiş bir pr oblemi çözme görevini verir. Ancak, eğer sonlandırma koşulu sağlanırsa, mevcut aktivasyonun ek aktivasyonlar oluşturmadan sonlanmasını sağlayan bir yol izlenir .

Şimdi, Şekil 5.14'teki ikili arama prosedürümüzde tekrarlayıcı kontrolün başlatm a ve değiştirme aşamalarının nasıl uygulandığını görelim. Bu durumda, hedef değer bulunduğunda veya görev boş bir listeyi arama görevine indirildiğinde ek aktivasyon ların oluşturulması sonlandırılır. Süreç, başlangıç listesi ve bir hedef değeri verilerek örtük olarak başlatılır. Bu başlangıç konfigürasyonundan prosedür, atanan görevini daha küçük bir listeyi arama görevine değiştirir. Orijinal listenin sonlu bir uzunluğa s ahip olduğu ve her değiştirme adımının söz konusu listenin uzunluğunu azalttığı göz önüne alındığında, hedef değerin nihayetinde bulunacağı veya görevin boş listeyi ar ama görevine indirileceği konusunda güvence altındayız. Bu nedenle, tekrarlayıcı sür ecin sona ereceğini kesin olarak söyleyebiliriz.

Son olarak, hem döngü hem de özyinelemeli kontrol yapıları, bir dizi tali matın tekrarlanmasını sağlama yolları olduğundan, bunların güç açısından e şdeğer olup olmadığını sorabiliriz. Yani, bir algoritma bir döngü yapısı kullan ılarak tasarlandığında, aynı problemi çözecek şekilde yalnızca özyinelemeli te knikler kullanan başka bir algoritma tasarlanabilir mi ve tam tersi? Bu tür sor ular, bilgisayar biliminde önemlidir çünkü cevapları, en güçlü programlama s istemini elde etmek için bir programlama dilinde hangi özelliklerin sağlanma sı gerektiğini bize söyler. Bu fikirlere, bilgisayar biliminin daha teorik yönleri ni ve matematiksel temellerini ele aldığımız Bölüm 12'de döneceğiz. Bu arka planla, Ek E'de yinelemeli ve özyinelemeli yapıların eşdeğerliğini kanıtlayabil eceğiz.

Doclingo https://www.doclingo.ai

Sorular ve Alıştırmalar

1. İkili arama (Şekil 5.14) sırasında Joe ismini ararken hangi isimler sorg ulanır? Alice, Brenda, Carol, Duane, Evelyn, Fred, George, Henry, Iren e, Joe, Karl, Larry, Mary, Nancy ve Oliver listesinde?
2. İkili arama uygulandığında 200 girişlik bir liste için sorgulanması ger eken maksimum giriş sayısı nedir? Peki ya 100.000 girişlik bir liste içi n?
3. Aşağıdaki özyinelemeli
   prosedür, N'ye 1 değeri atandığında hangi sayı dizisini yazdırır?

```
prosedürEgzersiz (N)
N'nin değerini yazdır;
eğer(N < 3) ise(prosedür Egzersiz'i N + 1 değerine uygula);

N'nin değerini yazdır.
```

4. Soru/Egzersiz 3'teki özyinelemeli prosedürdeki sonlandırma koşulu n edir?

5.6 Verimlilik ve Doğruluk

Bu bölümde bilgisayar bilimi içinde önemli araştırma alanlarını oluşturan iki konuyu tanıtıyoruz. Bunlardan ilki algoritma verimliliği, ikincisi ise algoritma doğruluğudur.

Algoritma Verimliliği

Günümüz makineleri her saniye milyonlarca talimatı yerine getirebilse de, veri mlilik algoritma tasarımında önemli bir endişe olmaya devam etmektedir. Gen ellikle verimli ve verimsiz algoritmalar arasındaki seçim, bir probleme pratik bir çözüm ile pratik olmayan bir çözüm arasında fark yaratabilir.

Bir üniversite kayıt memurunun öğrenci kayıtlarını geri alma ve güncelle me görevini üstlendiği problemi ele alalım. Üniversitenin herhangi bir döne mde yaklaşık 10.000 öğrencilik gerçek bir kaydı olmasına rağmen, "mevcut ö ğrenci" dosyası, son birkaç yıl içinde en az bir derse kayıtlı olan ancak bir der ece tamamlamamış 30.000'den fazla öğrencinin kayıtlarını içermektedir. Şim dilik, bu kayıtların kayıt memurunun bilgisayarında öğrenci kimlik numaralar ına göre sıralanmış bir liste olarak saklandığını varsayalım. Bu nedenle, herh angi bir öğrenci kaydını bulmak için kayıt memuru bu listede belirli bir kimlik numarasını arayacaktır.

Bu tür bir listeyi aramak için iki algoritma sunduk: sıralı arama ve ikili arama. Şimdi sorumuz, bu iki algoritma arasındaki seçimin kaydedici durumunda herhangi bir fark yaratıp yaratmadığıdır. Öncelikle sıralı aramayı ele alıyoruz.

Bir öğrenci kimlik numarası verildiğinde, sıralı arama algoritmasılistenin başından başlar ve bulunan girişleri istenen kimlik numarasıyla karşılaştırır. Hedef değerin kaynağı hakkında hiçbir şey bilmediğimizde, biz

Doclingo https://www.doclingo.ai

bu aramanın listenin ne kadar derinine gitmesi gerektiğini kesin olarak söyleye meyiz. Ancak, birçok aramadan sonra, aramaların ortalama derinliğinin listenin ortasında olmasını bekliyoruz; bazıları daha kısa, bazıları ise daha uzun olacakt ır. Bu nedenle, bir süre boyunca, sıralı aramanın her aramada yaklaşık 15.000 k aydı inceleyeceğini tahmin ediyoruz. Her kaydın kimlik numarasını almak ve ko ntrol etmek 10 milisaniye (saniyenin on binde biri) gerektiriyorsa, böyle bir ara ma ortalama 150 saniye veya 2.5 dakika sürecektir—bu, kaydın bir bilgisayar ek ranında görünmesi için kaydedicinin beklemesi için dayanılmaz bir süre. Her ka ydın alınması ve kontrol edilmesi için gereken süre sadece 1 milisaniyeye düşür ülse bile, arama yine de ortalama 15 saniye sürecektir ki bu da beklemek için h ala uzun bir süredir.

Buna karşılık, ikili arama, hedef değeri listedeki orta girişle karşılaştırara k ilerler. Eğer bu istenen giriş değilse, o zaman en azından kalan arama, oriji nal listenin yalnızca yarısıyla sınırlıdır. Böylece, 30.000 öğrenci kaydının listes inde orta girişi sorguladıktan sonra, ikili arama en fazla 15.000 kaydı hala gö z önünde bulunduracaktır. İkinci sorgulama sonrasında en fazla 7.500 kayıt k alır ve üçüncü alım sonrasında, söz konusu liste en fazla 3.750 girişe düşmüş tür. Bu şekilde devam ederek, hedef kaydın 30.000 kayıttan en fazla 15 giriş alındıktan sonra bulunacağını görüyoruz. Böylece, bu alımların her biri 10 mi lisaniyede gerçekleştirilebiliyorsa, belirli bir kaydı arama süreci yalnızca 0.15 saniye gerektirir—bu da kaydın kayıttan sorumlu kişinin bakış açısından anlı k olarak erişilebilir görüneceği anlamına gelir. Sonuç olarak, sıralı arama alg oritması ile ikili arama algoritması arasındaki seçim, bu uygulamada önemli bir etki yaratacaktır.

Bu örnek, algoritmaların gerektirdiği kaynakların, örneğin zaman veya dep olama alanı gibi, incelendiği algoritma analizi olarak bilinen bilgisayar bilimi ala nının önemini göstermektedir. Bu tür çalışmaların önemli bir uygulaması, alter natif algoritmaların göreceli faydalarının değerlendirilmesidir.

Algoritma analizi genellikle en iyi durum, en kötü durum ve ortalama durums enaryolarını içerir. Örneğimizde, 30.000 girişten oluşan bir listeyi aramak için gere ken süreyi tahmin etmek amacıyla sıralı arama algoritmasının ortalama durum an alizini ve ikili arama algoritmasının en kötü durum analizini gerçekleştirdik. Genel olarak, bu tür analiz daha genel bir bağlamda yapılır. Yani, listeleri aramak için alg oritmaları değerlendirirken, belirli bir uzunluktaki bir listeye odaklanmıyoruz, bun un yerine algoritmanın rastgele uzunluktaki listeler için performansını gösterecek bir formül tanımlamaya çalışıyoruz. Önceki akıl yürütmemizi rastgele uzunluktaki listelere genelleştirmek zor değildir. Özellikle, n girişine sahip bir listeye uygulan dığında, sıralı arama algoritması ortalama olarak n½ giriş sorgularken, ikili arama algoritması en kötü durum senaryosunda en fazla lg n giriş sorgulayacaktır. (lg n, n'nin taban iki logaritmasını temsil eder.)

Ekleme sıralama algoritmasını (Şekil 5.11'de özetlenmiştir) benzer bir şekilde analiz edelim. Bu algoritmanın, pivot giriş olarak adlandırılan bir liste girişini seçm eyi, bu girişi önündeki girişlerle karşılaştırmayı ve ardından pivot için uygun yeri b ulduğunda pivot girişini bu yere yerleştirmeyi içerdiğini hatırlayın. İki girişi karşıla ştırma etkinliği algoritmayı domine ettiğinden, yaklaşımımız, uzunluğu n olan bir l isteyi sıralarken gerçekleştirilen bu tür karşılaştırmaların sayısını saymak olacaktır .

Algoritma, pivot olarak seçilecek ikinci liste girişini seçerek başlar. Ardınd an, listenin sonuna ulaşana kadar ardışık girişleri pivot olarak seçerek ilerler.

Doclingo https://www.doclingo.ai

En iyi durumda, her pivot zaten doğru yerindedir ve bu keşfedilene kadar yal nızca bir girişle karşılaştırılması gerekir.
Bu nedenle, en iyi durumda, n girişe sahip bir listeye ekleme sıralaması uygulamak n1 k arşılaştırma gerektirir. (İkinci giriş bir girişle, üçüncü giriş bir girişle karşılaştırılır ve bu ş ekilde devam eder.)

Karşıt olarak, en kötü senaryo, her pivotun doğru konumu bulunmadan önce tüm önceki girişlerle karşılaştırılması gerektiğidir. Bu, orijinal listenin te rs sırada olması durumunda gerçekleşir. Bu durumda, ilk pivot (ikinci liste gir işi) bir girişle, ikinci pivot (üçüncü liste girişi) iki girişle karşılaştırılır ve bu şeki lde devam eder (Şekil 5.18). Böylece, n girişe sahip bir listeyi sıralarken topla m karşılaştırma sayısı $1 + 2 + 3 + \ldots + (n - 1)$, ile eşdeğerdir $(1/2)(n^2 - n)$. Özellikle, liste 10 giriş içeriyorsa, ekleme sıralama algoritmasının en kötü senaryosu 45 karşılaştırm a gerektirecektir.

Ekleme sıralamasının ortalama durumunda, her pivotunönceki girişlerin yarısı ile karşı laştırılmasını bekleriz. Bu, en kötü durumda yapılan karşılaştırmaların yarısı kadar karşılaştı rma yapılmasına yol açar veya $(1/4)(n^2 n)$ karşılaştırma ile n girişten oluşan bir listeyi sıralama k için gereklidir. Örneğin, ekleme sıralamasını uzunluğu 10 olan çeşitli listeleri sıralamak içi n kullanırsak, her sıralama başına ortalama karşılaştırma sayısının 22.5 olmasını bekleriz.

Bu sonuçların önemi, ekleme sıralama algoritmasının yürütülmesi sırasın da yapılan karşılaştırma sayısının, algoritmanın yürütülmesi için gereken za manın bir tahminini vermesidir. Bu tahmini kullanarak, Şekil 5.19, ekleme sır alama algoritmasını yürütmek için gereken zamanın liste uzunluğu arttıkça n asıl arttığını gösteren bir grafik sunmaktadır. Bu grafik, algoritmanın en kötü durum analizine dayanmaktadır; burada, uzunluğu n olan bir listeyi sıralam anın en fazla $(1/2)(n^2 n)$ karşılaştırma gerektireceğini sonucuna vardık. Grafikt e, birkaç liste uzunluğunu işaretledik ve her durumda gereken zamanı belirtt ik. Liste uzunlukları eşit artışlarla arttıkça, listeyi sıralamak için gereken zama nın giderek daha fazla arttığını unutmayın. Böylece, algoritma liste boyutu ar ttıkça daha az verimli hale gelir.

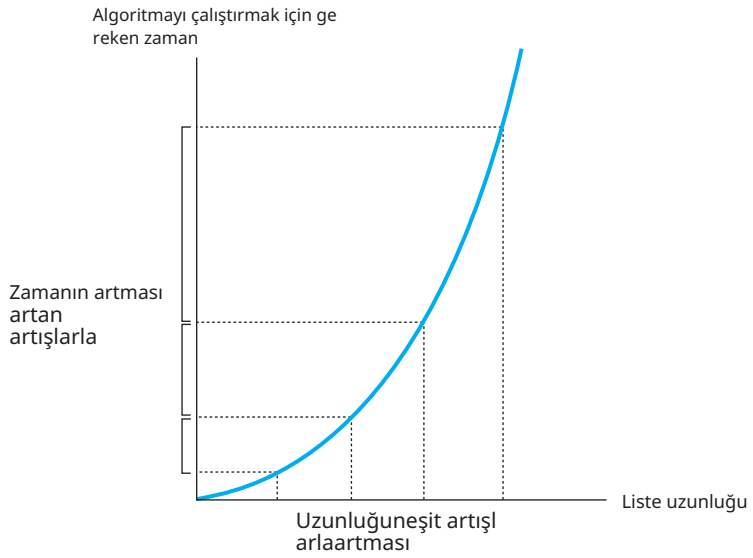
İkili arama algoritmasına benzer bir analiz uygulayalım. Bu algoritmayı k ullanarak n girişe sahip bir listeyi aramanın en fazla $\lg n$ girişi sorgulamayı g erektireceğini ve bunun da çeşitli liste boyutları için algoritmanın çalıştırılma sı gereken süreye bir yaklaşık değer verdiğini hatırlayın. Şekil 5.20, bu analiz e dayanan ve birkaç listeyi tekrar işaretlediğimiz bir grafiği göstermektedir.

Şekil **5.18** Ekleme sıralamasının en kötü durum senaryosunda uygulanması

Başlangıç listesi	Her pivot için yapılan karşılaştırmalar				Sıralılı ste
	1. pivot	2. pivot	3. pivot	4. pivot	
Elaine David Carol Barbara Alfred	1 Elaine David Carol Barbara Alfred	3 David 2 Elaine Carol Barbara Alfred	6 Carol 5 David 4 Elaine Barbara Alfred	10 Barbara 9 Carol 8 David 7 Elaine Alfred	Alfred Barbara Carol David Elaine

Doclingo https://www.doclingo.ai

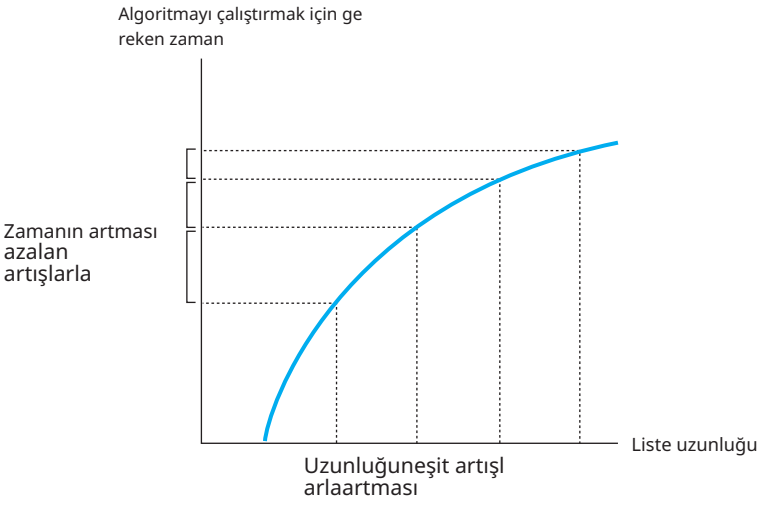
Şekil **5.19** Ekleme sıralama algoritmasının en kötü durum analizinin grafiği



Uniform olarak artan boyutların uzunlukları ve her durumda algoritmanın g erektirdiği zamanı belirledik. Algoritmanın gerektirdiği zamanın azalan artışl arla arttığını unutmayın. Yani, ikili arama algoritması, listenin boyutu arttıkça daha verimli hale gelir.

Şekil 5.19 ile 5.20 arasındaki ayırt edici faktör, ilgili grafiklerin genel şekli dir. Bu genel şekil, bir algoritmanın daha büyük ve daha büyük girdiler için n e kadar iyi performans göstermesi gerektiğini ortaya koyar. Ayrıca, bir grafiğ in genel şekli, temsil edilen ifadenin türü tarafından belirlenir.

Şekil **5.20** İkili arama algoritmasının en kötü durum analizinin grafiği



Doclingo https://www.doclingo.ai

ifadenin ayrıntılarından daha fazlası—tüm lineer ifadeler düzbir çizgi üretir; t üm ikinci dereceden ifadeler parabolik bir eğri üretir; tüm logaritmik ifadeler Şekil 5.20'de gösterilen logaritmik şekli üretir. Bu şekli üreten en basit ifadeyl ebir şekli tanımlamak adettendir. Özellikle, parabolik şekli n^2 ifadesiyle ve log aritmikşekli $lg\ n$ ifadesiyle tanımlarız.

Bir algoritmanın görevini yerine getirmek için gereken zamanınveri girişi b oyutuyla karşılaştırılarak elde edilen grafiğin şekli, algoritmanın verimliliközelli klerini yansıttığı için, algoritmaların bu grafiklerin şekillerine göresınıflandırılm ası yaygındır—genellikle algoritmanın en kötü durum analizine dayanarak. Bu sınıfları tanımlamak için kullanılan notasyon bazenbüyük-teta notasyonu olarak adlandırılır. Parabol şekline sahip grafiklere sahip olan tüm algoritmalar,eklem e sıralama gibi, $\Theta(n^2)$ (okunuşu "büyükteta n kare") ile temsil edilen sınıfa konur ; logaritmikifade şekline sahip grafiklere sahip olan tüm algoritmalar, ikili aram a gibi, $\Theta(lg\ n)$("büyük teta $log\ n$" olarak okunur) ile temsil edilen sınıfa düşer. Bel irli bir algoritmanın hangi sınıfa düştüğünü bilmek,performansını tahmin etme mizi ve aynı problemi çözen diğer algoritmalarla karşılaştırmamızı sağlar.$\Theta(n^2)$ i çindeki iki algoritma, girdi boyutları arttıkça benzerzaman gereksinimi değişikli kleri gösterecektir.Ayrıca, bir algoritmanın zaman gereksinimleri $\Theta(lg\ n)$ şeklind e, bir algoritmanın zaman gereksinimlerinden daha hızlı genişlemeyecek $\Theta(n^2$).

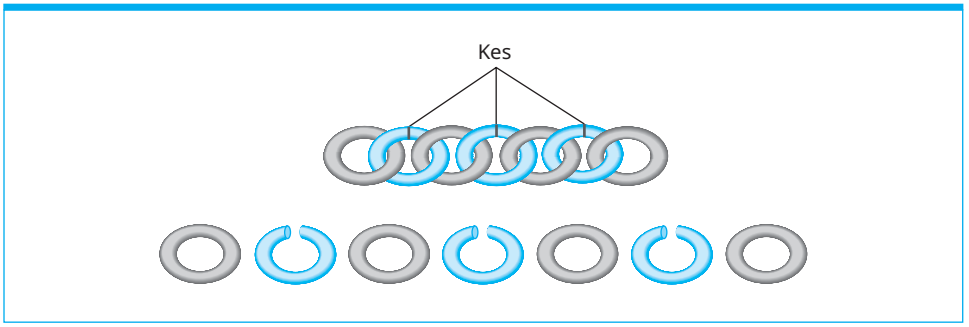
Yazılım Doğrulaması

Polya'nın problem çözme analizindeki dördüncü aşamanın (Bölüm 5.3) çözümün doğruluğunu ve diğer problemleri çözmek için bir araç olarak potansiyelini değerl endirmek olduğunu unutmayın. Bu aşamanın ilk kısmının önemi, aşağıdaki örnekl e gösterilmektedir:

> Yedi halkalı bir altın zincire sahip bir gezginin yedigece boyunca izole bir otelde kalması ge rekiyor. Her geceki kira, zincirin bir halkasından oluşuyor. Gezginin her sabah oteli ödeyeb ilmesi için en az kaç halkayı kesmesi gerekiyor, böylece konaklama için önceden ödeme ya pmadan her sabah bir halkayı otel sahibine verebilsin?

Bu problemi çözmek için önce zincirdeki her bağlantının kesilmesi gerekmediğini fa rk ediyoruz.Eğer sadece ikinci bağlantıyı kesersek, hem birinci hem de ikinci bağlantıyı diğer beş bağlantıdankurtarabiliriz. Bu içgörüden yola çıkarak, zincirde yalnızca ikinci, d ördüncü ve altıncı bağlantıları kesmeçözümüne ulaşıyoruz; bu işlem her bağlantıyı serb est bırakırken yalnızca üç bağlantıyı kesiyor (Şekil 5.21).Ayrıca, daha az kesim yapmak,

Şekil **5.21**Zinciri yalnızca üç kesimle ayırma



Doclingo https://www.doclingo.ai

iki bağlantı birbirine bağlı, bu yüzden sorunumuzun doğru cevabının üç olduğunu sö yleyebiliriz.

Ancak, problemi yeniden değerlendirirken, zincirde yalnızca üçüncü bağl antı kesildiğinde, bir, iki ve dört uzunluklarında üç zincir parçası elde ettiğimi zi gözlemleyebiliriz (Şekil 5.22). Bu parçalarla şu şekilde devam edebiliriz:

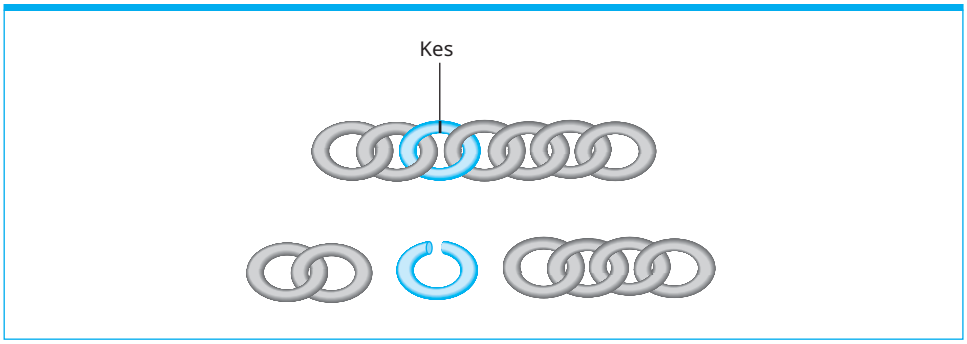
İlk sabah: Otele tek bağlantıyı verin.
İkinci sabah: Tek bağlantıyı alın ve otele iki bağlantılı parçayı verin.
Üçüncü sabah: Otele tek bağlantıyı verin.
Dördüncü sabah: Otele ait üç bağlantıyı alın ve otele dört bağlantılı par çayı verin.
Beşinci sabah: Otele tek bağlantıyı verin.
Altıncı sabah: Tek bağlantıyı alın ve otele çift bağlantılı parçayı verin.
Yedinci sabah: Otele tek bağlantıyı verin.

Sonuç olarak, doğru olduğunu düşündüğümüz ilk cevabımız yanlıştır. O halde, yeni çözüm ümüzün doğru olduğundan nasıl emin olabiliriz? Aşağıdaki gibi argüman geliştirebiliriz: İlk sabah otele tek bir bağlantı verilmesi gerektiğinden, zincirin en az bir bağlantısını n kesilmesi gerekir ve yeni çözümümüz yalnızca bir kesim gerektirdiğinden, bu en o ptimal olmalıdır.

Programlama ortamına çevrildiğinde, bu örnek, doğru olduğu düşünüle n bir program ile gerçekten doğru olan bir program arasındaki ayrımı vurgul ar. İkisi mutlaka aynı değildir. Veri işleme topluluğu, "doğru" olduğu bilinen y azılımların, bazı öngörülemeyen durumlar nedeniyle kritik bir anda başarısız olduğu korku hikayeleriyle doludur.
Yazılımın doğrulanması bu nedenle önemli bir girişimdir ve verimli doğrulama tekniklerinin araştırılması bilgisayar bilimlerinde aktif bir araştırma alanını oluş turmaktadır.

Bu alandaki önemli bir araştırma hattı, bir programın doğruluğunu kanıtl amak içinresmi mantık tekniklerini uygulamaya çalışmaktadır. Yani, amaç, bi r program tarafından temsil edilen algoritmanın, amaçlandığı gibi çalıştığını kanıtlamak için resmi mantığı uygulamaktır. Temel tez, doğrulama sürecini r esmi bir prosedüre indirerek, sezgisel argümanlarla ilişkili olabilecek yanlış s onuçlardan korunmaktır; bu, altın zincir problemi durumunda olduğu gibi. P rogram doğrulama yaklaşımını daha ayrıntılı olarak ele alalım.

Şekil **5.22** Sadece bir kesimle problemi çözme



Doclingo https://www.doclingo.ai

Yazılım Doğrulamasının Ötesinde

Metinde tartışıldığı gibi, doğrulama problemleri yazılıma özgü değildir. Eşit derec edeönemli olan, bir programı yürüten donanımın kusurlardan arınmış olduğunuo naylama problemidir. Bu, devre tasarımlarının yanı sıra makine inşasınındoğrulan masını içerir. Yine, en son teknoloji büyük ölçüde testlere dayanır; bu, yazılımörne ğinde olduğu gibi, ince hataların bitmiş ürünlere sızabileceği anlamına gelir. Kayıtl ar,1940'larda Harvard Üniversitesi'nde inşa edilen Mark I'in, yıllarca tespit edileme yenkablolama hataları içerdiğini göstermektedir. Daha yakın bir örnek, erken Pen tium mikroişlemcilerininkayan nokta bölümündeki bir hatadır. Bu iki durumda da, hata ciddi sonuçlarortaya çıkmadan önce tespit edilmiştir.

Resmi bir matematiksel kanıt, aksiyomlara dayanıyorsa (geometrik kanıtlar genellikle Öklidyen geometri aksiyomlarına dayanırken, diğer kanıtlar küme te orisi aksiyomlarına dayanır), bir programın doğruluğuna dair resmi bir kanıt, pr ogramın tasarlandığı spesifikasyonlara dayanır. Bir programın isim listelerini d oğru bir şekilde sıraladığını kanıtlamak için, programın girdi olarak bir isim liste si aldığı varsayımını yapmamıza izin verilir; ya da program bir veya daha fazla p ozitif sayının ortalamasını hesaplamak üzere tasarlandıysa, girdinin aslında bir veya daha fazla pozitif sayıdan oluştuğunu varsayarız. Kısacası, doğruluk kanıtı, belirli koşulların, ön koşullar olarak adlandırılan, programın yürütülmesinin b aşlangıcında sağlandığı varsayımıyla başlar.

Doğruluk kanıtındaki bir sonraki adım, bu ön koşulların program boyunc a nasıl yayıldığını düşünmektir. Bu amaçla, araştırmacılar, bir yapının yürütül meden önce doğru olduğu bilinen bir ifadenin, yapının yürütülmesiyle nasıl etkilendiğini belirlemek için çeşitli program yapılarını analiz etmiştir.
Basit bir örnek olarak, eğer Y değerine dair belirli bir ifadenin, talimatı yürütmede n önce geçerli olduğu biliniyorsa

$$X \leftarrow Y$$

o zaman aynı ifade, talimat yürütüldükten sonra X hakkında da geçerli olabili r. Daha kesin olarak, eğer Y'nin değeri talimat yürütülmeden önce 0 değilse, o zaman X'in değeri talimat yürütüldükten sonra 0 olmayacaktır.

Daha karmaşık bir örnek, aşağıdaki gibi bir if-then-else yapısında ortaya çıkar

eğer (*koşul*) hen (*instruction A*)
aksi (*instruction B*)

Burada, eğer yapının yürütülmesinden önce bazı ifadelerin geçerli olduğu biliniyo rsa, o zaman talimat A'yı yürütmeden hemen önce, hem o ifadenin hem de test e dilen koşulun doğru olduğunu biliyoruz; oysa talimat B yürütülecekse, ifadenin ve test edilen koşulun olumsuzlamasının geçerli olması gerektiğini biliyoruz.

Bu tür kurallar izlenerek, bir doğruluk kanıtı, programın çeşitli noktalarında belirlenebilen iddialar olarak adlandırılan ifadeleri tanımlayarak ilerler. Sonuç, her biri

Doclingo https://www.doclingo.ai

programın ön koşulları ve iddianın belirlendiği noktaya giden talimatların sırası. Eğer programın sonunda belirlenen bu iddia, istenen çıktı spesifikasyonlarına karşılık geliyorsa (bunlara son koşullar denir), programın doğru olduğunu sonuçlandırırız.

Örnek olarak, Şekil 5.23'te temsil edilen tipik bir `while` döngü yapısını dü şünün. A noktasında verilen ön koşulların bir sonucu olarak, her seferinde so nlandırma testi yapıldığında (B noktası) belirli bir ifadenin doğru olduğunu b elirleyebileceğimizi varsayalım. (Bir döngüde bir noktada, o noktaya ulaşıldığ ında her seferinde doğru olan bir ifade, döngü değişmezliği olarak bilinir.) O zaman, eğer tekrarlar bir noktada sona ererse, yürütme C noktasına geçer; burada hem döngü değişmezliğinin hem de sonlandırma koşulunun geçerli olduğunu sonuçlandırabiliriz. (Döngü değişmezliği hala geçerlidir çünkü sonl andırma testi programdaki hiçbir değeri değiştirmez ve sonlandırma koşulu geçerlidir çünkü aksi takdirde döngü sona ermez.) Eğer bu birleşik ifadeler is tenen son koşulları ima ediyorsa, doğruluk kanıtımızı yalnızca döngünün baş latma ve değiştirme bileşenlerinin nihayetinde sonlandırma koşuluna yol açt ığını göstererek tamamlayabiliriz.

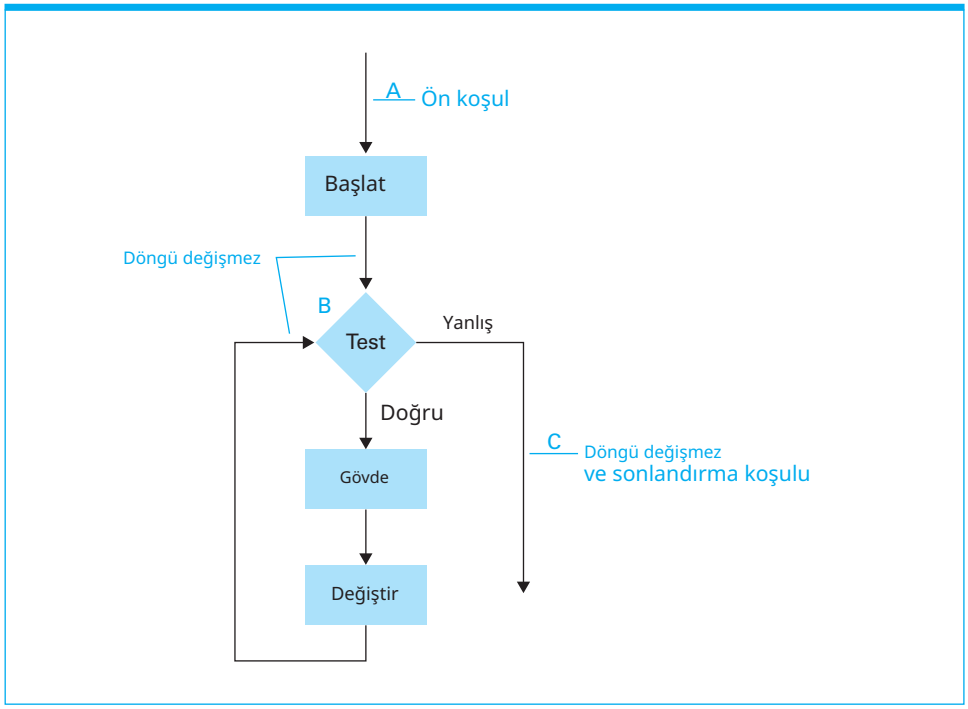
Bu analizi, Şekil 5.11'de gösterilen ekleme sıralama örneğimizle karşılaştır malısınız. O programdaki dış döngü, döngü değişmezliğine dayanmaktadır

> . Her seferinde sonlandırma testi yapıldığında, listedeki girişler 1. po zisyondan N1. pozisyona kadar sıralanmıştır.

ve sonlandırma koşulu

> N değeri listenin uzunluğundan büyüktür.

Şekil **5.23** Tipik bir while yapısıyla ilişkili ifadeler



Doclingo https://www.doclingo.ai

Bu nedenle, eğer döngü bir şekilde sonlanırsa, her iki koşulun da sağlanması gerektiğin i biliyoruz, bu da tüm listenin sıralı olacağı anlamına gelir.

Program doğrulama tekniklerinin geliştirilmesindeki ilerleme hala zorlu olmaya devam ediyor. Ancak, önemli gelişmeler kaydedilmektedir. Bunlarda n biri, daha popüler olan Ada diline yakın olan SPARK programlama dilinde b ulunmaktadır. (Ada, bir sonraki bölümde örnekler alacağımız dillerden biridir .) SPARK, programların sözde kodumuz gibi yüksek seviyeli bir biçimde ifade edilmesine izin vermenin yanı sıra, program içinde ön koşullar, son koşullar ve döngü değişmezleri gibi ifadeleri dahil etme imkanı sunar. Böylece, SPAR K'ta yazılmış bir program yalnızca uygulanacak algoritmayı değil, aynı zaman da resmi doğruluk kanıtı tekniklerinin uygulanması için gereken bilgileri de i çerir. Bugüne kadar, SPARK, ABD Ulusal Güvenlik Ajansı için güvenli yazılımla r, Lockheed Martin'in C130J Hercules uçağında kullanılan iç kontrol yazılımlar ı ve kritik demir yolu taşımacılığı kontrol sistemleri de dahil olmak üzere, kriti k yazılım uygulamalarını içeren birçok yazılım geliştirme projesinde başarıyla kullanılmıştır.

SPARK gibi başarılarına rağmen, resmi program doğrulama teknikleri henüz yaygın bir kullanım bulmamıştır ve bu nedenle günümüz yazılımlarının çoğu "doğrulanmış" test ile - en iyi ihtimalle sarsak bir süreç. Sonuçta, test ile doğrulama programın test edildiği durumlar için doğru çalıştığını kanıtlamaktan başka bir şey ispatlamaz. Ekstra sonuçlar yalnızca projeksiyonlardır.

Bir programda bulunan hatalar genellikle test sırasında ve geliştirme sırasında kolayca gözden kaçan ince dikkatsizliklerin sonuçlarıdır. Sonuç olarak bir programdaki hatalar, tıpkı altın zincir problemi üzerindeki hatamız gibi, tespit edilemeyebilir ve sıklıkla da tespit edilmez, bu hatalardan kaçınmak için önemli çaba harcansa bile. AT&T'de dramatik bir örnek yaşandı: 114 anahtarlama istasyonunu kontrol e den yazılımdabir hata, yazılımın Aralık 1989'daki kurulumundan 15 Ocak 199 0'a kadar tespit edilmedi; bu tarihteeşsiz bir dizi koşul, yaklaşık beş milyon ça ğrının dokuz saatlik bir süre boyunca gereksiz yere engellenmesineneden ol du.

Sorular ve Alıştırmalar

1. Diyelim ki, ekleme sıralama algoritmamızla programlanmış bir makine nin 100 isimden oluşan bir listeyi sıralamasıortalama bir saniye alıyor. 1000 isimden oluşan bir listeyi sıralamanın ne kadar sürdüğünü tahmin ediyorsun uz? Peki ya10,000 isim?
2. Aşağıdaki sınıflardan her biri için bir algoritma örneği verin: $\Theta(lg\ n)$, $\Theta(n)$ ve $\Theta(n^2)$.
3. Sınıfları $\Theta(n^2)$,$\Theta(lg\ n)$,$\Theta(n)$ ve $\Theta(n^3)$ verimlilik açısından azalan sırayla list eleyin.
4. Aşağıdaki problemi ve önerilen yanıtı düşünün. Önerilen yanıt doğru mu? Neden veya neden değil?

Doclingo https://www.doclingo.ai

Problem: Bir kutuda üç kart olduğunu varsayın. Üç karttan biri her iki tarafı da siyah, biri her iki tarafı da kırmızı ve üçüncüsü bir tarafı kırmızı, diğer tarafı siyah olarak boyanmıştır. Kutudan bir kart çekiliyor ve onun bir şeyini görebiliyorsunuz. Kartın diğer tarafının gördüğünüz tarafla aynı renkte olma olasılığı nedir?
Önerilen yanıt: Yarım. Gördüğünüz kartın tarafının kırmızı olduğunu varsayın. (Kartın tarafı siyah olsaydı, argüman simetrik olurdu.) Üç karttan yalnızca ikisi kırmızı bir tarafa sahiptir. Dolayısıyla gördüğünüz kart bu ikisinden biri olmalıdır. Bu iki karttan biri diğer tarafta kırmızı, diğeri ise siyah. Dolayısıyla gördüğünüz kartın diğer tarafının kırmızı olma olasılığı, siyah olma olasılığı kadar yüksektir.

5. Aşağıdaki program segmenti, iki pozitif tam sayının (bir pay ve bir payda) bölümünü (kalanı unutarak) hesaplamaya yönelik bir denemedir. Paydanın, paydanın altına düşmeden önce kaç kez paydanın paydan çıkarılabileceğini sayarak. Örneğin, ⅔, 3'ün 7'den iki kez çıkarılabileceği için 2 üretmelidir.
Program doğru mu? Cevabınızı gerekçelendirin.

```
Sayım←0;
Kalan←Bölünecek;
tekrarla(Kalan←Kalan – Bölücü;
            Sayım←Sayım + 1)
kadar(Kalan < Bölücü)
Bölüm←Sayım.
```

6. Aşağıdaki program segmenti, iki pozitif tam sayının X veY çarpımını h esaplamak için tasarlanmıştır Y kopyalarının toplamını biriktirerek Y— yani, 3 ile 4'ün çarpımı, üç tane 4'ün toplamını biriktirerek hesaplanır. Prog ram doğru mu? Cevabınızı gerekçelendirin.

```
Çarpan←Y;
Sayım←1;
iken(Sayım < X) yap
  (Çarpan←Çarpan + Y;
    Sayım←Sayım + 1)
```

7. N ile ilişkili değerin pozitif bir tam sayı olduğu ön koşulunu varsayara k, aşağıdaki rutin sona ererse, Toplam değerinin 0 +1+. . . +N olduğ u sonucuna götüren bir döngü değişmezliği oluşturun.

```
Toplam←0;
K←0;
while(K < N) do
  (K←K + 1;
    Toplam ←Toplam + K)
```

Rutinin gerçekten sona erdiği etkisinde bir argüman sağlayın.

8. Hem bir programın hem de onu yürüten donanımındoğru olduğunun resmi olarak doğrulandığını varsayalım. Bu, doğruluğu garanti eder mi?

Doclingo https://www.doclingo.ai

Bölüm Gözden Geçirme Problemleri

(Yıldızlı problemler isteğe bağlı bölümlerle ilişkilidir.)

1. Bölüm 5.1'in ilk paragrafında verilen ga yri resmi algoritma tanımına uyan bir diz i adım örneği verin, ancak Şekil 5.1'de verilen resmi tanıma uymamaktadır.

2. Önerilen bir algoritmadaki belirsizlik ile bir algoritmanın temsilindeki belirsizlik arasın daki farkı açıklayın.

3. Temel yapıların kullanımının bir algoritmanın te msilindeki belirsizlikleri nasıl ortadan kaldır dığını tanımlayın.

4. Tanıdık olduğunuz bir konu seçin ve o konuda yönlendirme vermek için bir sözdekod tasarlayı n. Özellikle, kullanacağınız temel yapıları ve bunları temsil etmek için kullanacağınız sözdizi mini tanımlayın. (Bir konu düşünmekte zor lanıyorsanız, sporlar, sanatlar veya el san atları deneyin.)

5. Aşağıdaki program, katı anlamda bir algoritmayı t emsil ediyor mu? Neden veya neden olmasın?

   Sayım←0;
   Sayım 5 değilken(yap)
     (Sayım←Sayım + 2)

6. Aşağıdaki üç adım, bir algoritmayı neden ol uşturmuyor?

   Adım 1: Dikdörtgen koordinatları (2,5) ve (6, 11) olan noktalar arasında düz bir doğru pa rçası çizin.

   Adım 2: Dikdörtgen koordinatları (1,3) ve (3, 6) olan noktalar arasında düz bir doğru par çası çizin.

   Adım 3: Önceki doğru parçalarının kesişi minde merkezi olan ve yarıçapı iki olan bir d aire çizin.

7. Aşağıdaki program segmentini bir tekrar yapısı kullanarak yeniden yazın, `while` yapısı yerine. Y eni versiyonun, orijinal ile aynı değerleri yazdırd ığından emin olun.

   Sayı←2;
   while(Sayı < 7) **do**
     (Sayıya atanan değeri yazdır ve
     Sayı←Sayı + 1)

8. Aşağıdaki program segmentiniawhile yapısı k ullanarak yeniden yazın,repeat yapısı yerine. Yeni versiyonun,orijinal ile aynı değerleri yazdı rdığından emin olun.

   Sayı←1;
   repeat
     (Sayıya atanan değeri yazdır ve
     Sayı←Sayı + 1)
   until(Sayı = 5)

9. Bir posttest döngüsünü, şu şekilde ifade edilen

   tekrar(. . .) **until**(. . .)

   eşdeğer bir posttest döngüsüne çevirmek için,şu şekilde ifa de edilen

   do(. . .) **while**(. . .)

10. 0, 1, 2, 3, 4, 5, 6, 7,8, 9 rakamlarının bir düzenle mesi verildiğinde,bu rakamları yeniden düzenleyen bir algoritma tasarlayın,yeni düzenlemenin bu r akamlarla temsil edilebilecekbir sonraki daha büyük değeri temsil etmesini sağlasın(veya daha bü yük bir değer üreten herhangi bir düzenleme yoksa,böyle bir düzenlemenin mevcut olmadığını bi ldirsin). Böylece 5647382901 şunu üretecektir: 5647382910.

11. Pozitif bir sayının tüm çarpanlarını bulmak i çin bir algoritma tasarlayın.Örneğin, 12 say ısı durumunda,algoritmanız 1, 2, 3, 4, 6 ve 12 değerlerinirapor etmelidir.

12. 1 Ocak 1700'den itibaren herhangi bir tarihin hafta nın gününü belirlemek için bir algoritma tasarlayın. Örneğin, 17 Ağustos 2001 bir Cuma idi.

13. Resmi bir programlama dili ile bir sözdeko d arasındaki fark nedir?

14. Söz dizimi ile anlam arasındaki fark nedi r?

15. Aşağıda geleneksel onluk notasyonda bir t oplama problemi bulunmaktadır. Her harf

Doclingo https://www.doclingo.ai

farklı bir rakamı temsil etmektedir. Her harf hangi rakamı temsil ediyor? Kapıyı nasıl araladın?

```
  XYZ
+ YWY
 ZYZW
```

16. Aşağıda geleneksel onluk notasyonda bir çarpma problemi bulunmaktadır. Her harff arklı bir rakamı temsil etmektedir. Her harf hangirakamı temsil ediyor? Kapıyı nasıl araladın?

```
   XY
 × YX
 ----
   XY
  YZ
 ----
  WVY
```

17. Aşağıda ikili notasyonda bir toplama problemi bulunmaktadır. Her harf benzersizbir ikili rakamı temsil etmektedir. Hangi harf 1'i vehangi harf 0'ı temsil ediyor? Bu tür problemleriçözmek için bir algoritma tasarlayın.

```
  YXX
+ XYX
 ----
 XYYY
```

18. Sadece bir feneri olan dört madenci bir madenşaftından geçmek zorundadır. En fazla iki madenci birlikte seyahat edebilir ve şafttta bulunan herhangi bir madenci fenerle birlikte olmalıdır. Andrews,Blake, Johnson ve Kelly adındaki madenciler sırasıyla bir dakika,d iki dakikada, dört dakikada ve sekiz dakikada şafttan geçebilirler. İki madenci birlikte yürüdüğünde, daha yavaş olan madencinin hızında hareket ederler. Dört madencinin dmaden şaftından sadece15 dakikada nasıl geçebileceğini düşünün? Bu problemi çözdükten sonra, kapıyı nasıl araladığını açıklayın.

19. Büyük bir şarap kadehi ve küçük bir şarap kadehi ile başlayın, küçük kadehi şarapla doldurun ve ardından o şarabı büyük kadehe dökün. Sonra, küçük kadehi suyla doldurun ve o sudan birazını büyük kadehe dökün. Büyük kadehin içeriğini karıştırın ve ardından karışımı küçük kadehe geri dökün, küçük kadeh dolana kadar. Büyük kadehte küçük kadehteki şaraptan daha fazla su olacak mı? Bu problemi çözdükten sonra, kapıyı nasıl açtığınızı açıklayın.

20. Romeo ve Juliet adında iki arı, farklı kovanlarda yaşıyor ama tanışıp aşık oldular. Rüzgarsız bir bahar sabahında, her biri kendi kovanından çıkıp birbirlerini ziyaret etmek için yola çıkıyorlar. Rotaları, en yakın kovanın 50 metre uzağında bir noktada kesişiyor, ancak birbirlerini göremiyorlar ve hedeflerine doğru devam ediyorlar. Hedeflerine vardıklarında, diğerinin evde olmadığını keşfetmek için aynı süre harcıyorlar ve geri dönüş yolculuklarına başlıyorlar. Geri dönüş yolculuklarında, en yakın kovanın 20 metre uzağında bir noktada karşılaşıyorlar. Bu sefer birbirlerini görüyorlar ve eve dönmeden önce piknik yapıyorlar. İki kovan ne kadar uzakta? Bu problemi çözdükten sonra, kapıyı nasıl açtığınızı açıklayın.

21. İki karakter dizisi verildiğinde, ilk dizinin ikinci dizide bir alt dizi olarak görünüp görünmediğini test eden bir algoritma tasarlayın.

22. Aşağıdaki algoritma, Fibonacci dizisi olarak bilinen dizinin başlangıcını yazdırmak için tasarlanmıştır. Döngünün gövdesini belirleyin. Döngü kontrolü için başlangıç adımı nerede? Değiştirme adımı? Test adımı? Hangi sayı listesi üretiliyor?

```
Son←0;
Mevcut←1;
while(Mevcut < 100) do
 (Mevcut'a atanan değeri yazdır;
   Geçici←Son;
   Son←Mevcut; ve
   Mevcut←Son + Geçici)
```

23. Aşağıdaki algoritma, 0 ve 1 giriş değerleriyle başlatıldığında hangi sayı dizisini yazdırır?

```
prosedürGizemYaz (Son, Mevcut)
if(Mevcut < 100) then
(Mevcut'a atanan değeri yazdır;
  Geçici←Mevcut + Son;
  GizemYaz'ı Mevcut ve Geçici değerlerine uygula
       )
```

24. Önceki problemdeki `MysteryWrite` prosedürünü değiştirin, böylece değerlerters sırayla yazdırılsın.

Doclingo https://www.doclingo.ai

25. J değerini ararken A, B, C, D, E, F, G, H, I, J, K, L, M, N, O listesininüzerinde ikili aram a (Şekil 5.14) uygulandığında hangi harfler sorgulanır? Z değerini ararken ne olacak?

26. 6,000 girişten oluşan bir listede birçok sıralı arama yaptıktan sonra,hedef değerin bir li ste girişiyle ortalama kaç kez karşılaştırılmas ını beklersiniz? Arama algoritması ikili ara ma olsaydı ne olurdu?

27. Aşağıdaki yinelemeli ifadelerin her birinde s onlandırma koşulunu belirleyin.

a. **while**(Sayı < 5) **do**( )

b. tekrar( )
kadar(Sayı = 1)

c. **while**((Sayı < 5) ve (Toplam < 56))
do( )

28. Aşağıdaki döngü yapısının gövdesini belirl eyin vekaç kez çalıştırılacağını sayın. Test "(Sayı 6 değil)"şeklinde değiştirilirse ne olur?

```
Sayı←1;
while(Sayı 7 değil) do
   (Sayıya atanan değeri yazdır ve
    Sayı←Sayı + 3)
```

29. Aşağıdaki program bir bilgisayarda uygul anırsa hangisorunların ortaya çıkması nı beklersiniz? (*İpucu:* Kayan noktaaritmet iği ile ilişkili yuvarlama hataları sorununu h atırlayın.)

```
Sayım←bir-on;
tekrarla
   (Sayım'a atanan değeri yazdır ve
    Sayım←Sayım + bir-on)
kadar(Sayım 1'e eşit olana kadar)
```

30. Öklid algoritmasının özyinelemeli bir versiyo nunu tasarlayın(Soru 3, Bölüm 5.2).

31. Girdi değeri 1'e hem `Test1` hem de Test 2(aşağıda tanımlanmıştır) uyguladığımızı va rsayalım. İkirutinin yazdırdığı çıktılardaki fark nedir?

```
prosedürTest1 (Sayım)
eğer(Sayım 5 değilse)
   o zaman (Sayım'a atanan değeri yazdır;
            Sayım + 1 değerine Test1 uygu
            layın)

prosedürTest2 (Sayım)
eğer(Sayım 5 değilse)
   o zaman(Test2'yi Sayım + 1 değerine uy
           gula;
           değeri yazdır
           Count)
```

32. Önceki problemin rutinlerindeki kontrol me kanizmasının önemli bileşenlerini belirleyin. Özellikle, sürecin sona ermesine neden ola n koşul nedir? Sürecin bu sona erme koşulu na doğru durumu nerede değiştirilir? Kont rol sürecinin durumu nerede başlatılır?

33. Aşağıdaki özyinelemeli prosedürde sona e rme koşulunu belirleyin.

```
prosedür XXX (N)
eğer(N = 5) o zaman (prosedür XXX'i N + 1 de
                     ğerine uygula)
```

34. Prosedürü `MysteryPrint` (aşağıda tanımlanmı ştır) 3 değerine uygulayın ve yazdırılan değ erleri kaydedin.

```
prosedürMysteryPrint (N)eğer (N
 > 0)  o zaman (N'nin değerini yazdır ve prosed
               ürü uygula
               MysteryPrint
               değeri N – 2)
N + 1 değerini yazdır.
```

35. Uygulama prosedürü `MysteryPrint` (aşağıda tanımlanmıştır)değeri 2'ye ve yazdırılan değ erleri kaydedin.

```
prosedürMysteryPrint (N)
 eğer(N > 0)
     o zaman (N'nin değerini yazdır ve pr
              osedürü MysteryPrint'i N - 2 değerine
                uygula)
      aksi takdirde(N'nin değerini yazdır ve
              eğer(N > –1)
                  o zaman (prosedürü uygula
                           MysteryPrint
                           değeri N + 1'e))
```

36. Pozitif tam sayıların (artan sırada) dizisiniür etmek için bir algoritma tasarlayıntek asal bölenleri 2 ve 3 olan; yani,programınız 2,3, 4, 6, 8, 9, 12, 16, 18, 24, 27, ... dizisini üretmelidir. Programınızkatı anlamda bir a lgoritmayı temsil ediyor mu?

Doclingo https://www.doclingo.ai

37. Aşağıdaki soruları liste açısından yanıtlayın: Alice, Byron, Carol, Duane, Elaine, Floyd, Gene, Henry, Iris.

a. Hangi arama algoritması (sıralı veyai kili) Gene adını dahahızlı bulacaktır?

b. Hangi arama algoritması (sıralı veyai kili) Alice adını dahahızlı bulacaktır?

c. Hangi arama algoritması (sıralı veya ikili) Bruce adının yokluğunu daha hızlı tespit edecektir?

d. Hangi arama algoritması (sıralı veya ikili) Sue adının yokluğunu daha hızlı tespit edecektir?

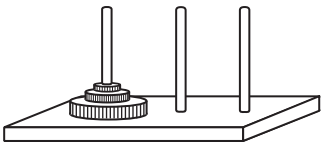
e. Sıralı arama kullanıldığında Elaine adı nıaramak için kaç giriş sorgulanacaktır?İ kili arama kullanıldığında kaçsorgulan acaktır?

38. 0'ın faktöriyeli 1 olarak tanımlanmıştır. Pozitif birtam sayının faktöriyeli, o tam sayınınbir sonraki daha küçük pozitif tam sayınınfakt öriyeli ile çarpımı olarak tanımlanır. Biz,$n!$ n otasyonunu kullanarak tam sayınınfaktöriye lini ifade ederiz.Böylece 3'ün faktöriyeli (3!) $3 \times (2!) = 3 \times (2 \times (1!)) = 3 \times (2 \times (1 \times (0!))) = 3 \times (2 \times (1 \times (1)))$ = 6'dır. Verilen bir değerin faktör iyelinihesaplayan özyinelemeli bir algo-rithma tasa rlayın.

39. a. Beş ismin sıralanması gerektiğinivarsayal ım ve dört ismin sıralanması içinzate n bir algoritma tasarlamışsınız. Daha önc etasarlanan algoritmadan yararlanarak beşismin sıralanması için bir algori tma tasarlayın.

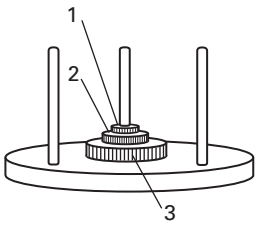
b. Belirli bir teknik kullanarak isimlerin keyfi list elerini sıralamak için özyinelemeli bir algoritma tasarlayın.

40. Hanoi Kuleleri olarak adlandırılan bulmaca, üç çubuktan oluşur; bunlardan biri, alt kısımdan üst kısma doğru azalan çapta sıralanmış birkaç halkayı içerir. Problem, halka yığınını başka bir ç ubuğa taşımaktır. Aynı anda yalnızca bir halkayı h areket ettirmenize izin verilir ve hiçbir zaman da ha küçük bir halkanın üstüne bir halka yerleştiril memelidir. Eğer bulmaca yalnızca bir halka i çeriyorsa, son derece kolay olacağını gözlemleyin. Dahası, bu problemle karşılaştığınızda birkaç halkayı hareket ettirmek, en büyük halk ayı başka bir çubuğa taşımak mümkünse, en b üyük halka üçüncü çubuğa yerleştirilebilir v e ardından sorun, kalan halkaları onun üzeri ne taşımak olacaktır. Bu gözlemi kullanarak, H anoi Kuleleri bulmacasını çözmek için özyinele meli bir algoritma geliştirin. rastgele sayıda halka.



41. Hanoi kuleleri bulmacasını çözmenin bir di ğer yaklaşımı, direklerin 4, 8 ve 12 saat pozisy onlarında bir direk ile dairesel bir stand üzeri nde düzenlendiğini hayal etmektir. Halka, en küçük halka 1 olacak şekilde 1, 2, 3 vb. numar alandırılmıştır. Tek numaralı halkalar, bir yı ğının üstünde olduğunda, saat yönünde bir s onraki direğe hareket etmelerine izin verilir; benzer şekilde, çift numaralı halkaların saat yönünün tersine hareket etmelerine izin verilir (bu hareket, bir halkayı daha küçük bir halkanı n üzerine koymadığı sürece).

Bu kısıtlama altında, her zaman hareket ettirile bilen en büyük numaralı halkayı hareket ettirin. Bu gözlemden yola çıkarak, Hanoi Kuleleri bulma casını çözmek için özyinelemeli olmayan bir algo ritma geliştirin.



42. Bir döngü yapısına dayanan ve diğerinin özyine lemeli bir yapıya dayandığı iki algoritma gelişti rin; her gün bir önceki günün maaşının iki katı (i lk günün çalışması için bir kuruşla başlayar ak) ödenen bir işçinin günlük maaşını yazdır mak için 30 günlük bir süre boyunca. Çözüml erinizi gerçek bir makinede uyguladığınızda han gi sayı depolama ile ilgili sorunlarla karşılaşmayı bekliyorsunuz?

Doclingo https://www.doclingo.ai

43. Bir pozitif sayının karekökünü bulmak için b ir algoritma tasarlayın; sayının kendisini il k tahmin olarak alarak ve önceki tahmin ile orijinal sayıyı önceki tahmine bölerek elde edilen sonucu ortalayarak sürekli yeni bir ta hmin üreterek başlayın. Bu tekrarlayıcı sür ecin kontrolünü analiz edin. Özellikle, tek rarı sonlandıracak koşul nedir?

44. Beş farklı karakterden oluşan bir dizedeki semb ollerin tüm olası yeniden düzenlemelerini listeleyen bir algoritma tasarlayın.

45. Bir isimler listesi verildiğinde, listedeki e n uzun ismi bulan bir algoritma tasarlayın. Çözümünüzün listede birkaç "en uzun" isi m varsa ne yaptığını belirleyin. Özellikle, tü m isimler aynı uzunluktaysa algoritmanız ne yapar?

46. Beş veya daha fazla sayının verildiği bir liste içi n, tüm listeyi sıralamadan listedeki beş en küçü k ve beş en büyük sayıyı bulan bir algoritma ta sarlayın.

47. Brenda, Doris, Raymond, Steve, Timothy v e William isimlerini, ekleme sıralama algori tması ile sıralandığında en az karşılaştır ma gerektiren bir sıraya yerleştirin (Şekil 5.11).

48. 4000 isimlik bir listeye ikili arama algoritma sı (Şekil 5.14) uygulandığında sorgulana n girişlerin en büyük sayısı nedir? Bu , sıralı arama (Şekil 5.6) ile nasıl karşılaştırılır ?

49. Toplama ve çarpma için geleneksel ilkokul algoritmalarını sınıflandırmak için büyük-te ta notasyonunu kullanın. Yani, her biri *n* ba samağa sahip iki sayıyı toplamanız iste ndiğinde, kaç bireysel toplama işlemi ya pılması gerekir? İki *n* basamaklı sayıyı çarpm anız istendiğinde, kaç bireysel çarpma işle mi gereklidir?

50. Bazen bir problemin küçük bir değişikliği, çözümü nün şeklini önemli ölçüde değiştirebilir. Örneğin, aş ağıdaki problemi çözmek için basit bir algoritma b ulun ve bunu büyük-teta notasyonu kullanarak sınıfl andırın:

    Bir grup insanı, iki ayrık alt gruba (arbitrary boyutta) ayırın, böylece iki alt grubu n üyelerinin toplam yaşları arasındaki fark mümkün olduğunca büyük olsun.

    Şimdi problemi, istenendifference'in mümkün olduğunca küçük olması için değiştirin veprobl eme yaklaşımınızı sınıflandırın.

51. Aşağıdaki listeden, toplamı 3165 olan birsa yılar koleksiyonu çıkarın. Probleminize yakl aşımınız ne kadar verimli?

    26, 39, 104, 195, 403, 504, 793, 995, 1156, 1677

52. Aşağıdaki rutin döngüsü sonlanıyor mu?Ce vabınızı açıklayın. Bu rutin bir bilgisaya r tarafındangerçekten çalıştırılırsa ne ola bileceğini açıklayın (Bölüm 1.7'ye bakın).

```
X←1;
Y←1/2;
while(X sıfıra eşit değil) do
  (X←X – Y;
   Y←Y ÷ 2)
```

53. Aşağıdaki program segmenti, iki pozitif olm ayantam sayının X veY çarpımını hesaplam ak için tasarlanmıştır;yani, 3 ile 4'ün çarpı mı, üç 4'ün toplamını biriktirerek hesapl anır.
    Program segmenti doğru mu? Cevabınız ı açıklayın.

```
Çarpan←0;
Sayım←0;
tekrar(Product←Product + Y,
        Sayım←Sayım + 1)
sonra(Sayım = X)
```

54. Aşağıdaki program segmenti, pozitif tam sayılardan h angisinin X veY daha büyük olduğunuraporlamak için t asarlanmıştır. Program segmenti doğru mu? Cevabınız ıaçıklayın.

```
Fark←X – Y;
Eğer(Fark pozitifse)
    O zaman(yazdır "X, Y'den büyüktür")
    Aksi takdirde(yazdır "Y, X'ten büyüktür")
```

55. Aşağıdaki program segmenti, boş olmayan birta m sayılar listesindeki en büyük girişi bulma k için tasarlanmıştır. Doğru mu? Cevabınızıaçı klayın.

```
TestValue←ilk liste girişi;CurrentEnt
ry←ilk liste girişi;while (CurrentEntry son
giriş değilse)do

  (if(CurrentEntry > TestValue)
       sonra(TestValue←CurrentEntry)
   CurrentEntry← bir sonraki liste girişi)
```

Doclingo https://www.doclingo.ai

56. a. Şekil 5.6'da gösterildiği gibi sıralı arama için ön koşulları belirleyin.
Programdaki whileyapısı için bir döngü değişmezliği oluşturun; bu, sonlandırma koşuluyla birleştirildiğinde,döngü sona erdiğinde algoritmanın başarı veya başarısızlıkraporlayacağını doğru bir şekilde ima eder.

b. Şekil 5.6'daki while döngüsünün gerçekten sona erdiğini gösteren bir argüman verin.

57. X veY içinnegatif olmayan tam sayılar atandığı ön koşullara dayanarak, bir döngü değişmezleri için aşağıdaki iken yapı-, sonlandırma koşuluyla birleştirildiğinde, değerin
döngü sonlandırıldığında Z ile ilişkilidir şu olmalıdır X – Y.

```
Z←X;
J←0;
while(J < Y) do
   (Z←Z – 1;
    J←J + 1)
```

Sosyal Sorunlar

Aşağıdaki sorular, bilgisayar alanıyla ilişkili etik/sosyal/hukuki sorunlara bir rehber olarak tasarlanmıştır. Amaç, bu soruları yalnızca yanıtlamak değildir. Ayrıca, neden böyle yanıt verdiğinizi ve gerekçelerinizin bir sorudan diğerine tutarlı olup olmadığını da düşünmelisiniz.

1. Karmaşık programların doğruluğunu tamamen doğrulamak şu anda imkansız olduğuna göre, bir programın yaratıcısı hatalardan hangi koşullarda, varsa, sorumlu olmalıdır?
2. Bir fikriniz olduğunu ve bunu birçok insanın kullanabileceği bir ürüne dönüştürdüğünüzü varsayalım. Dahası, fikrinizi genel halk için faydalı bir forma dönüştürmek bir yıl süren bir çalışma ve 50.000 $'lık bir yatırım gerektirmiştir. Ancak nihai formunda, ürün çoğu insan tarafından sizden hiçbir şey satın almadan kullanılabilir. Tazminat talep etme hakkınız nedir? Bilgisayar yazılımlarını korsanlamak etik midir? Peki ya müzik ve sinema filmleri?
3. Bir yazılım paketi o kadar pahalıdır ki tamamen bütçenizin dışındadır. Kendi kullanımınız için kopyalamak etik midir? (Sonuçta, paketi satın almayacağınız için tedarikçiyi bir satıştan mahrum bırakmıyorsunuz.)
4. Nehirlerin, ormanların, okyanusların vb. mülkiyeti uzun zamandır tartışma konusu olmuştur. Bir algoritmanın mülkiyeti açısından birine veya bir kuruma mülkiyet verilmesi gerektiği anlamı nedir?
5. Bazı insanlar yeni algoritmaların keşfedildiğini, bazıları ise yeni algoritmaların yaratıldığını düşünüyor. Hangi felsefeye katılıyorsunuz?
Farklı bakış açıları, algoritmaların mülkiyeti ve mülkiyet hakları ile ilgili farklı sonuçlara yol açar mı?
6. Bir yasa dışı eylemi gerçekleştirmek için bir algoritma tasarlamak etik midir? Algoritmanın hiç çalıştırılıp çalıştırılmadığı önemli mi? Böyle bir algoritmayı yaratan kişinin o algoritma üzerinde mülkiyet hakları olmalı mı? Eğer öyleyse, bu haklar ne olmalı? Algoritma mülkiyet hakları, algoritmanın amacına bağlı mı olmalı? Güvenliği kırma tekniklerini tanıtmak ve yaymak etik midir? Ne tür bir şeye girildiği önemli mi?

Doclingo https://www.doclingo.ai

7. Bir yazar, hikaye genellikle film versiyonunda değişse bile, bir romanın sine ma hakları için ödeme alır. Bir hikayenin farklı bir hikaye haline gelmesi için ne kadar değişmesi gerekir? Bir algoritmanın farklı bir algoritma haline gel mesi için hangi değişiklikler yapılmalıdır?
8. Eğitim yazılımları artık 18 ay veya daha küçük yaştaki çocuklar için pazarlanmaktadı r. Destekleyenler, bu tür yazılımların birçok çocuğa başka türlü ulaşamayacakları g örüntü ve sesler sağladığını savunuyor. Karşıt görüşte olanlar ise bunun kişisel ebe veyn/çocuk etkileşimi için kötü bir ikame olduğunu iddia ediyor. Sizin görüşünüz ne dir? Yazılım hakkında daha fazla bilgi edinmeden görüşünüze dayanarak herhangi bir eylemde bulunmalı mısınız? Eğer öyleyse, hangi eylem?

Ek Okuma

Aho, A. V., J. E. Hopcroft ve J. D. Ullman. *Bilgisayar Algoritmalarının Tasarımı ve Analizi.*Boston, MA: Addison-Wesley, 1974.

Baase, S. *Bilgisayar Algoritmaları: Tasarım ve Analize Giriş*, 3. baskı. Boston, MA : Addison-Wesley, 2000.

Barnes, J. *Yüksek Bütünlükte Yazılım: Güvenlik ve Emniyet için SPARK Yaklaşımı.* Boston, MA: Addison-Wesley, 2003.

Gries, D. *Programlamanın Bilimi*. New York: Springer-Verlag, 1998.

Harbin, R. *Origami—Kağıt Katlama Sanatı*. Londra: Hodder Paperbacks, 1973.

Johnsonbaugh, R. ve M. Schaefer. *Algoritmalar*. Upper Saddle River, NJ: Prentice-Hall, 2004.

Kleinberg, J. ve E. Tardos. *Algoritma Tasarımı*. Boston, MA: Addision-Wesley, 2006.

Knuth, D. E. *Bilgisayar Programlamanın Sanatı*, Cilt 3, 3. baskı. Boston, MA: Addison-Wesley, 1998.

Levitin, A. V. *Algoritmaların Tasarımı ve Analizi Giriş*, 2. baskı. Boston, MA: Addison-Wesley, 2007.

Polya, G. *Nasıl Çözülür*. Princeton, NJ: Princeton University Press, 1973.

Roberts, E. S. *Özyinelemeli Düşünme*. New York: Wiley, 1986.

Doclingo https://www.doclingo.ai

BÖLÜM

Programlama Dilleri

6

Bu bölümde programlama dillerini inceliyoruz. Amacımız belirli bir dili öğrenmek değil. Daha ziyade programlama dilleri hakkında bilgi edinmektir. Programlama dilleri ve b unlara bağlı metodolojiler arasındaki ortaklık ve çeşitliliği takdir etmek istiyoruz.

6.1 Tarihsel Perspektif
Erken Nesiller
Makine Bağımsızlığı ve Ötesi
Programlama Paradigmaları

6.2 Geleneksel Programlama Kavramları
Değişkenler ve Veri Türleri
Veri Yapısı
Sabitler ve Literaller
Atama İfadeleri
Kontrol İfadeleri
Yorumlar

6.3 Prosedürel Birimler
Prosedürler
Parametreler
Fonksiyonlar

6.4 Dil Uygulama
Çeviri Süreci
Yazılım Geliştirme Paketleri

6.5 Nesne Yönelimli Programlama
Sınıflar ve Nesneler
Yapıcılar
Ek Özellikler

***6.6Programlama Eşzamanlı Aktiviteler**

***6.7Açıklayıcı Programlama**
Mantıksal Çıkarım
Prolog

**Yıldızlar, isteğe bağlı bölümler için önerileri belirtir.*

Doclingo https://www.doclingo.ai

İşletim sistemleri, ağ yazılımları ve günümüzde mevcut olan geniş uygulama yazılımları gibi karmaşık yazılım sistemlerinin geliştirilmesi, insanların makine dilinde program yazmaya zorlanması durumunda muhtemelen imkansız olurdu. Bu tür dillerle ilişkili karmaşık detaylarla başa çıkmak ve karmaşık sistemleri organize etmeye çalışmak, en azından yorucu bir deneyim olurdu. Sonuç olarak, sözdekodumuza benzer programlama dilleri geliştirilmiştir ki bu diller algoritmaların hem insanlar için anlaşılır bir biçimde hem de makine dili talimatlarına kolayca dönüştürülebilir bir formda ifade edilmesine olanak tanır. Bu bölümdeki amacımız, bu dillerin tasarımı ve uygulanmasıyla ilgilenen bilgisayar bilimi alanını keşfetmektir.

6.1 Tarihsel Perspektif

Programlama dillerinin tarihsel gelişimini izleyerek çalışmamıza başlıyoruz.

Erken Nesiller

2. Bölümde öğrendiğimiz gibi, modern bilgisayarlar için programlar, sayısal rakamlar olarak kodlanmış talimatların dizilerinden oluşur. Bu tür bir kodlama sistemi makine dili olarak bilinir. Ne yazık ki, makine dilinde program yazmak, genellikle bulunup düzeltilmesi gereken hatalara yol açan sıkıcı bir iştir (bu işleme hata ayıklama denir) ve iş tamamlanmadan önce yapılması gerekir.

1940'larda, araştırmacılar talimatların sayısal form yerine anımsatıcı formda temsil edilebileceği notasyon sistemleri geliştirerek programlama sürecini basitleştirdiler. Örneğin, talimat

> Register 5'in içeriğini register 6'ya taşımak

şu şekilde ifade edilir

```
4056
```

2. Bölümde tanıtılan makine dilini kullanarak, bir mnemonik sisteminde şöyle görünebilir

```
MOV R5, R6
```

Daha kapsamlı bir örnek olarak, makine dili rutini

```
156C
166D
5056
306E
C000
```

6C ve 6D bellek hücrelerinin içeriklerini toplayıp sonucu 6E konumuna kaydeden (2. Bölüm, Şekil 2.7) şu şekilde ifade edilebilir

```
LD R5,Fiyat
LD R6,KargoÜcreti
ADDI R0,R5 R6
ST R0,ToplamMaliyet
HLT
```

Doclingo https://www.doclingo.ai

mnemonikleri kullanarak. (Burada LD, ADDI, ST ve HLT'yi temsil etmek için kullandık *yükle*, *ekle*, *depola* ve *durdur*. Ayrıca, 6C,6D ve 6E konumlarındaki bellek hücrelerine atıfta bulunmak için Fiyat,GönderimÜcreti, ve ToplamMaliyet gibi tanımlayıcı isimler kullandık. Bu tür tanımlayıcı isimlere genellikle tanımlayıcılar denir.)Mnemonik formun, hala eksik olmasına rağmen, rutin anlamını sayısal formdan daha iyi temsil ettiğini unutmayın.

Bu tür bir mnemonik sistem kurulduğunda, mnemonik ifadeleri makine dilit alimatlarına dönüştüren derleyiciler adı verilen programlar geliştirildi. Böylece, bir insanın doğrudan makine dilinde bir program geliştirmek zorunda kalmak yerine, mnemonik formda bir program geliştirebileceği ve ardından bir derleyici aracılığıyla makine diline dönüştürebileceği anlamına geliyordu.

Programları temsil etmek için kullanılan bir mnemonik sistemi topluca montaj dili olarak adlandırılır. Montaj dilleri ilk geliştirildiğinde, daha iyi programlama teknikleri arayışında dev bir adım temsil ediyordu. Aslında, montaj dilleri o kadar devrimciydi ki, makine dilleri kendileri ilk nesil iken, ikinci nesil diller olarak bilinir hale geldiler.

Makine dili karşılıklarına göre montaj dillerinin birçok avantajı olmasına rağmen, nihai programlama ortamını sağlama konusunda hala yetersiz kalmaktadırlar. Sonuçta, bir montaj dilinde kullanılan ilkel, ilgili makine dilinde bulunanlarla esasen aynıdır. Fark, onları temsil etmek için kullanılan sözdiziminedir. Bu nedenle, montaj dilinde yazılmış bir program doğası gereği makine bağımlıdır; yani, program içindeki talimatlar belirli bir makinenin özellikleri açısından ifade edilir. Dolayısıyla, montaj dilinde yazılmış bir program başka bir bilgisayar tasarımına kolayca taşınamaz çünkü yeni bilgisayarın kayıt yapılandırmasına ve talimat setine uymak için yeniden yazılması gerekir.

Montaj dilinin bir diğer dezavantajı, bir programcının, talimatları sayısal biçimde kodlaması gerekmese de, yine de makinenin dilinin küçük, artımlı adımları açısından düşünmeye zorlanmasıdır. Durum, bir evi tahtalar, çiviler, tuğlalar vb. açısından tasarlamaya benzer. Gerçekten de, evin inşası nihayetinde bu temel parçalar temelinde bir tanım gerektirir, ancak tasarım süreci, odalar, pencereler, kapılar gibi daha büyük birimler açısından düşünürsek daha kolaydır.

Kısacası, bir ürünün nihayetinde inşa edilmesi gereken temel primitfler, ürünün tasarımı sırasında kullanılacak primitfler olmak zorunda değildir. Tasarım süreci, her biri ürünün ana özellikleriyle ilişkili bir kavramı temsil eden yüksek seviyeli primitflerin kullanılmasına daha uygundur. Tasarım tamamlandığında, bu primitfler uygulamanın detaylarıyla ilgili daha düşük seviyeli kavramlara dönüştürülebilir.

Bu felsefeyi takip ederek, bilgisayar bilimcileri yazılım geliştirmeye daha uygun olan programlama dilleri geliştirmeye başladılar; bu diller, düşük seviyeli montaj dillerinden daha elverişliydi. Sonuç olarak, önceki nesillerden farklı olan üçüncü nesil programlama dilleri ortaya çıktı; bu dillerin primitfleri hem daha yüksek seviyedeydi (büyük artışlarla talimatları ifade ettikleri için) hem de makine bağımsızdı (belirli bir makinenin özelliklerine dayanmadıkları için). En iyi bilinen erken örnekler, bilimsel ve mühendislik uygulamaları için geliştirilen FORTRAN (FORMül Çevirici) ve iş uygulamaları için ABD Donanması tarafından geliştirilen COBOL (Ortak İşletme Yönelimli Dil)dir.

Doclingo https://www.doclingo.ai

Genel olarak, üçüncü nesil programlama dillerine yaklaşım, yazılımın geli
ştirilebileceği yüksek seviyeli bir dizi ilkenin (temelde 5. Bölümde sözdekodu
muzu geliştirdiğimiz ruhla) tanımlanmasıydı. Bu ilkelerin her biri, makine dill
erinde mevcut olan düşük seviyeli ilkelerin bir dizisi olarak uygulanabilecek ş
ekilde tasarlandı. Örneğin, ifade

```
assignTotalCostthe valuePrice + ShippingCharge
```

belirli bir makinenin görevi nasıl yerine getireceğine atıfta bulunmadan yüks
ek seviyeli bir etkinliği ifade eder, ancak daha önce tartışılan makine talimatl
arı dizisi ile uygulanabilir. Böylece, sözdekod yapımız

tanımla←ifadesi

potansiyel bir yüksek seviyeli ilke.

Bu yüksek seviyeli ilkeler dizisi belirlendikten sonra, bu yüksek seviyeli ilk
elerde ifade edilen programları makine dili programlarına çeviren bir progra
m, çevirici olarak adlandırıldı. Böyle bir çevirici, ikinci nesil montajcılara benz
erdi, ancak genellikle tek bir yüksek seviyeli ilkenin talep ettiği etkinliği simül
e etmek için birkaç makine talimatını kısa dizilere derlemek zorundaydı. Böyl
ece, bu çeviri programlarına genellikle derleyiciler denirdi.

Bir alternatif olarak, yorumlayıcılar, üçüncü nesil dilleri uygulamanın baş
ka bir yolu olarak ortaya çıktı. Bu programlar, talimatları gelecekte kullanılm
ak üzere çevrilmiş versiyonunu kaydetmek yerine, çevrildikleri anda yürütme
leri açısından çevirmenlere benziyordu. Yani, daha sonra yürütülecek bir ma
kine dili kopyası üretmek yerine, bir yorumlayıcı aslında bir programı yüksek
seviyeli biçiminden yürütüyordu.

Yan bir konu olarak, üçüncü nesil programlama dillerini teşvik etme göre
vinin, hayal edilebileceği kadar kolay olmadığını belirtmeliyiz. Doğal bir dile b
enzer bir biçimde program yazma düşüncesi o kadar devrimciydi ki, yönetici
pozisyonundaki birçok kişi başlangıçta bu fikre karşı çıktı. İlk derleyicinin geli
ştiricisi olarak tanınan Grace Hopper, Almanca terimlerin kullanıldığı bir üçü
ncü nesil dil için bir çevirmeni gösterdiği hikayesini sık sık anlatırdı. Buradaki
nokta, programlama dilinin, çevirmen üzerinde yalnızca basit değişikliklerle i
fade edilebilecek çeşitli doğal dillerde ifade edilebilen küçük bir primitif seti e
trafında inşa edilmiş olmasıydı. Ancak, izleyiciler arasında birçok kişinin, II. D
ünya Savaşı yıllarında bir bilgisayara Almanca "anlamayı" öğreteceği için şok
olduğunu görmekten şaşırmıştı. Bugün, doğal bir dili anlamanın, birkaç titizli
kle tanımlanmış primitif ile yanıt vermekten çok daha fazlasını içerdiğini biliy
oruz.Gerçekten de, doğal diller (İngilizce, Almanca ve Latince gibi) biçimsel
dillerden (programlama dilleri gibi) ayırt edilir; çünkü ikincileri, gramerler tar
afından kesin bir şekilde tanımlanırken (Bölüm 6.4), ilki zamanla resmi gram
er analizi olmadan evrimleşmiştir.

Makine Bağımsızlığı ve Ötesi

Üçüncü nesil dillerin gelişimi ile birlikte, makine bağımsızlığı hedefi büyük ölçüde gerçekleş
tirilmiştir. Üçüncü nesil bir dildeki ifadeler, belirli bir makinenin özelliklerine atıfta bulunma
dığı için, bir makine için olduğu kadar diğer bir makine için de kolayca derlenebilir.

Doclingo https://www.doclingo.ai

Çapraz Platform Yazılımı

Tipik bir uygulama programı, birçok görevini yerine getirmek için işletim sistemine g üvenmek zorundadır. Bilgisayar kullanıcısıyla iletişim kurmak için pencere yöneticisin in hizmetlerine ihtiyaç duyabilir veya veri almak için dosya yöneticisini kullanabilir. Ne yazık ki, farklı işletim sistemleri bu hizmetler için taleplerin farklı şekillerde yapılmasını zorunlu kılar. Bu nedenle, programların farklı makine tasarımları ve farklı işletim sistemleri içeren ağlar ve internette taşınması ve çalıştırılması için, programların hem işletim sistemi bağımsız hem de makine bağımsız olm ası gerekir. Bu ek bağımsızlık seviyesini yansıtmak için çapraz platform terimi kullanılır. Yani, çapraz platform yazılımı, bir işletim sisteminin tasarımına ve m akinenin donanım tasarımına bağımsız olan yazılımdır ve bu nedenle bir ağ b oyunca çalıştırılabilir.

Teorik olarak, üçüncü nesil bir dilde yazılmış bir program, uygun derleyici uy gulandığında herhangi bir makinede kullanılabilir.

Ancak gerçeklik bu kadar basit olmamıştır. Bir derleyici tasarlandığında, bazen temel makinenin belirli özellikleri, çevrilen dil üzerinde koşullar olarak yansıtılmaktadır. Örneğin, makinelerin G/Ç işlemlerini ele alma şekilleri tarih sel olarak "aynı" dilin farklı makinelerde farklı özelliklere veya lehçelere sahip olmasına neden olmuştur. Sonuç olarak, bir programı bir makineden diğeri ne taşımak için genellikle en azından küçük değişiklikler yapmak gerekmekte dir.

Taşınabilirlik sorununu daha da karmaşık hale getiren, bazı durumlarda belirli bir dilin doğru tanımının ne olduğu konusunda bir anlaşmazlığın olma sıdır. Bu konuda yardımcı olmak için, Amerikan Ulusal Standartlar Enstitüsü ve Uluslararası Standartlar Örgütü, birçok popüler dil için standartlar benims emiş ve yayımlamıştır. Diğer durumlarda, belirli bir dilin lehçesinin popülarit esi ve diğer derleyici yazarlarının uyumlu ürünler üretme isteği nedeniyle ga yri resmi standartlar gelişmiştir. Ancak, yüksek derecede standartlaştırılmış dillerde bile, derleyici tasarımcıları genellikle standart sürümün parçası olma yan, bazen dil uzantıları olarak adlandırılan özellikler sunmaktadır. Bir progr amcı bu özelliklerden yararlanırsa, üretilen program farklı bir satıcıdan gelen bir derleyici kullanan ortamlarla uyumlu olmayacaktır.

Programlama dillerinin genel tarihindeki üçüncü nesil dillerin gerçek ma kine bağımsızlığında yetersiz kalması, aslında iki nedenle pek de önemli değ ildir. İlk olarak, makine bağımsızlığına yeterince yakın oldukları için yazılımlar bir makineden diğerine görece kolaylıkla taşınabiliyordu. İkinci olarak, maki ne bağımsızlığı hedefi, daha iddialı hedefler için yalnızca bir tohum haline ge ldi. Gerçekten de, makinelerin

yüksek seviyeli ifadelerle, örneğinassign`TotalCost`**`the value`**Price + Shipping

Chargeşeklinde yanıt verebileceğini anlamak, bilgisayar bilimcilerini insanların makinelerle soyut k avramlar açısından iletişim kurmalarını sağlayacak programlama ortamları hayal etmeye yönlendi rdi; bu da bu kavramları makine uyumlu forma çevirmelerini zorunlu kılmaktan kaçınmayı sağladı.

Doclingo https://www.doclingo.ai

Ayrıca, bilgisayar bilimcileri, yalnızca algoritma yürütmesi değil, aynı zamand a algoritma keşif sürecinin çoğunu gerçekleştirebilen makineler istiyorlardı. Sonuç, nesiller açısından net bir sınıflandırmayı zorlayan sürekli genişleyen b ir programlama dilleri yelpazesi olmuştur.

Programlama Paradigmaları

Programlama dillerini sınıflandırma yaklaşımı, bir dilin konumunun, dilin kull anıcısının bilgisayar jargonundan ne ölçüde kurtulduğu ve çözülen probleml e ilişkili terimlerle düşünmesine izin verildiği derecelere göre belirlendiği bir doğrusal ölçeğe (Şekil 6.1) dayanmaktadır. Gerçekte, programlama dillerinin gelişimi bu şekilde ilerlememiştir, ancak programlama sürecine alternatif ya klaşımlar (programlama paradigmaları olarak adlandırılan) ortaya çıktıkça fa rklı yollar boyunca gelişmiştir. Sonuç olarak, programlama dillerinin tarihsel gelişimi, farklı paradigmaların farklı yollarının ortaya çıkıp bağımsız olarak ile rlediğini gösteren çoklu bir yol diyagramı ile daha iyi temsil edilmektedir. Öz ellikle, şekil, işlevsel, nesne yönelimli, zorlayıcı ve deklaratif paradigmaları te msil eden dört yolu sunmakta ve her paradigmaya bağlı çeşitli dillerin, diğer dillere göre doğumlarını gösteren bir şekilde konumlandırılmaktadır. (Bu, bir dilin mutlaka bir öncekinden evrimleştiği anlamına gelmez.)

Şunu belirtmeliyiz ki, Şekil 6.2'de tanımlanan paradigmalar programlama paradigmaları olarak adlandırılsa da, bu alternatiflerin programlama sürecinin ötesinde sonuçları vardır. Problemlere çözümler üretme konusunda temelde f arklı yaklaşımları temsil ederler ve bu nedenle tüm yazılım geliştirme sürecini e tkilerler. Bu anlamda, programlama paradigmaları terimi yanıltıcıdır. Daha ge rçekçi bir terim yazılım geliştirme paradigmaları olacaktır.

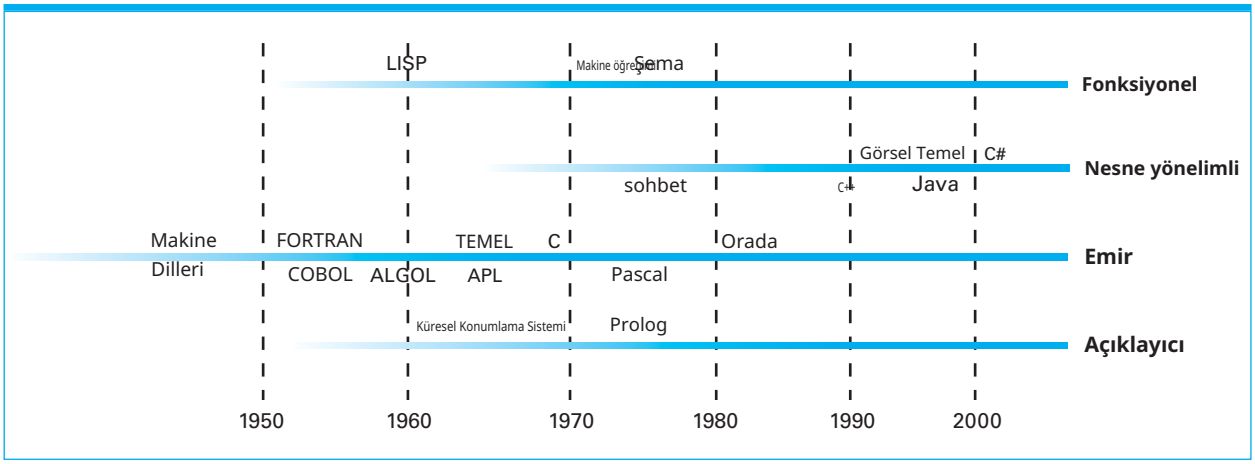
İkna edici paradigma, prosedürel paradigma olarak da bilinir, programla ma sürecine geleneksel yaklaşımı temsil eder. Bu, Bölüm 5'teki sözdekodumuz un ve Bölüm 2'de tartışılan makine dilinin temelini oluşturduğu paradigmadır. İ simden de anlaşılacağı gibi, ikna edici paradigma programlama sürecini, takip edildiğinde verileri manipüle ederek istenen sonucu üreten bir komut dizisinin geliştirilmesi olarak tanımlar. Böylece ikna edici paradigma, programlama süre cine yaklaşmamız gerektiğini, mevcut problemi çözmek için bir algoritma bulm amız ve ardından o algoritmayı bir komut dizisi olarak ifade etmemiz gerektiği ni söyler.

Şekil **6.1** Programlama dillerinin kuşakları



Doclingo https://www.doclingo.ai

Şekil **6.2** Programlama paradigmalarının evrimi



İhtiyacı olan bir algoritma yerine, çözülmesi gereken problemi tanımlam asını isteyen deklaratif paradigma, zorunlu paradigmanın zıttıdır. Daha kesin olarak, bir deklaratif programlama sistemi, kendisine sunulan problemleri ç özmek için önceden belirlenmiş genel amaçlı bir problem çözme algoritması nı uygular. Böyle bir ortamda, bir programcının görevi, problemi çözmek için bir algoritma tanımlamak yerine, problemin kesin bir ifadesini geliştirmek h aline gelir.

Deklaratif paradigma temelinde programlama sistemleri geliştirmede büyük bir engel, altında yatan bir problem çözme algoritmasının gerekliliğidir. Bu nede nle, erken deklaratif programlama dilleri genellikle belirli uygulamalarda kullanıl mak üzere tasarlanmış özel amaçlı olma eğilimindeydi. Örneğin, deklaratif yaklaş ım, hipotezleri test etmek veya tahminler elde etmek amacıyla bir sistemi (politik, ekonomik, çevresel vb.) simüle etmek için uzun yıllar boyunca kullanılmıştır. Bu ortamlarda, temel algoritma esasen, daha önce hesaplanmış değerlere day anarak parametrelerin (gayri safi yurtiçi hasıla, ticaret açığı vb.) değerlerini tekr ar tekrar yeniden hesaplayarak zamanın geçişini simüle etme sürecidir. Bu ned enle, böyle simülasyonlar için bir deklaratif dil uygulamak, öncelikle bu tekrarla yıcı prosedürü gerçekleştiren bir algoritmanın uygulanmasını gerektirir. O zaman sistemin bir programcıdan beklenen tek görev, simüle edilecek durumu tanımlamaktır. Bu şekilde, bir hava durumu tahmincisi hava durumunu tahmin e tmek için bir algoritma geliştirmek zorunda kalmaz, sadece mevcut hava durumu durumunu tanımlar ve bu da temel simülasyon algoritmasının yakın gelecekte h ava tahminleri üretmesine olanak tanır.

Matematikteki formal mantığın konusu, genel amaçlı bir deklaratif progr amlama sisteminde kullanılmak üzere uygun basit bir problem çözme algoritması sağladığı keşfi ile deklaratif paradigma büyük bir ivme kazandı. Sonuç o larak, deklaratif paradigmaya artan bir ilgi ve Bölüm 6.7'de tartışılan mantık programlamanın ortaya çıkması olmuştur.

Bir diğer programlama paradigması ise işlevsel paradıgmadır. Bu paradigmada bir program, girdileri kabul eden ve çıktılar üreten bir varlık olarak görülür. Matemat ikçiler bu tür varlıklara fonksiyonlar derler, bu nedenle bu yaklaşım işlevsel paradig ma olarak adlandırılır. Bu paradigmada bir program, daha küçük önceden tanımlan mış program birimlerini (önceden tanımlanmış) birleştirerek oluşturulur.

Doclingo https://www.doclingo.ai

fonksiyonlar) böylece her bir birimin çıktıları, istenen genel girdi-çıktı ilişkisini eld e edecek şekilde başka bir birimin girdisi olarak kullanılır. Kısacası, işlevsel paradi gma altında programlama süreci, daha basit fonksiyonların iç içe geçmiş karmaşı k yapıları olarak fonksiyonlar inşa etmektir.

Örnek olarak, Şekil 6.3, kontrol defterinizi dengelemek için bir fonksiyonun iki daha basit f onksiyondan nasıl oluşturulabileceğini göstermektedir. Bunlardan biri, Find_sum olarak adlan dırılan, değerleri girdi olarak kabul eder ve bu değerlerin toplamını çıktı olarak üretir. Diğeri is e, Find_diff olarak adlandırılan, iki girdi değerini kabul eder ve bunların farkını hesaplar. Şekil 6.3'te gösterilen yapı, LISP programlama dilinde (önemli bir işlevsel pro gramlama dili) aşağıdaki ifade ile temsil edilebilir

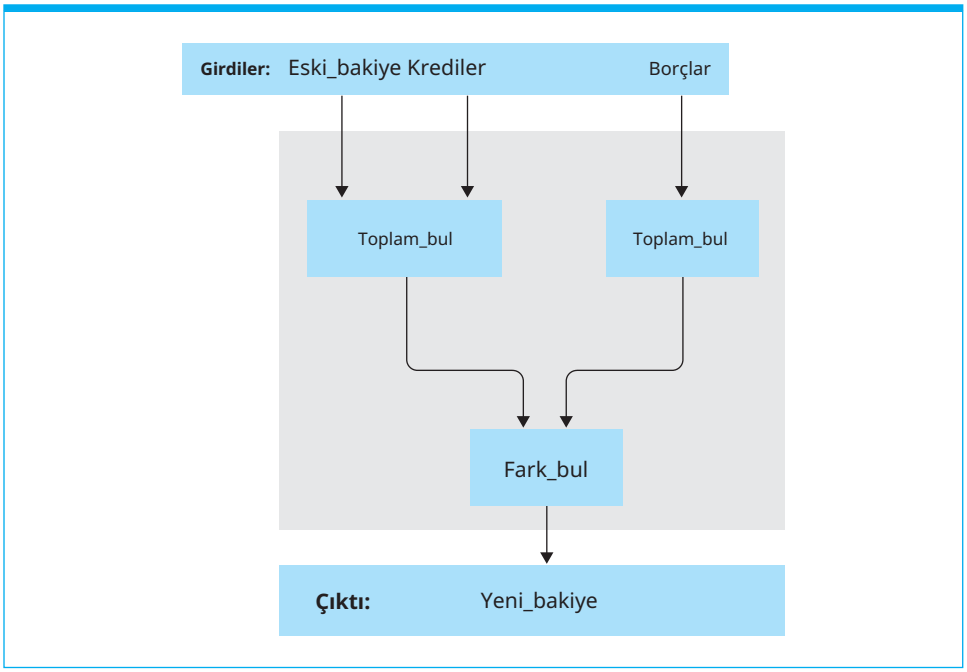
(Find_diff (Find_sum Eski_bakiye Krediler) (Find_sum Borçlar))

Bu ifadenin iç içe yapısı (parantezler tarafından gösterildiği gibi), `Find_diff` fonksiyon una girdi olarak sağlanan değerlerin, `Find_sum` fonksiyonunun iki uygulaması tarafınd an üretildiğini yansıtır. `Find_sum` fonksiyonunun ilk uygulaması, tüm Krediler ile Eski_ bakiye'yi toplamanın sonucunu üretir. `Find_sum` fonksiyonunun ikinci uygulaması ise t üm Borçlar'ın toplamını hesaplar. Ardından, Find_diff fonksiyonu bu sonuçları kullanar ak yeni kontrol defteri bakiyesini elde eder.

Fonksiyonel ve zorlayıcı paradigmalar arasındaki farkı daha iyi anlamak i çin, bir kontrol defterini dengelemek için fonksiyonel programı, zorlayıcı par adigmadan elde edilen aşağıdaki sözdekod programı ile karşılaştıralım:

```
Toplam_krediler← tüm Kredilerin toplamı
Geçici_bakiye← Eski_bakiye + Toplam_krediler
Toplam_borçlar← tüm Borçların toplamı
Bakiye← Geçici_bakiye - Toplam_borçlar
```

Şekil **6.3** Daha basit fonksiyonlardan oluşturulmuş bir çek defteri dengeleme fonksiyonu



Doclingo https://www.doclingo.ai

Bu zorunlu programın birden fazla ifadesi vardır; her biri bir hesaplamanın y apılmasını ve sonucun daha sonra kullanılmak üzere saklanmasını talep eder . Buna karşılık, fonksiyonel program, her hesaplamanın sonucunun hemen b ir sonraki hesaplamaya yönlendirildiği tek bir ifadeden oluşur. Bir anlamda, z orunlu program, her biri ham maddelerini depolarda saklanan ürünlere dön üştüren bir dizi fabrikaya benzer. Bu depolardan, ürünler daha sonra ihtiyaç duyuldukça diğer fabrikalara gönderilir.
Ancak fonksiyonel program, her birinin yalnızca diğer fabrikalar tarafından si pariş edilen ürünleri ürettiği ve bu ürünleri ara depolama olmaksızın hemen varış noktalarına gönderdiği koordine edilmiş bir fabrika koleksiyonuna ben zer. Bu verimlilik, fonksiyonel paradigmanın savunucuları tarafından öne sür ülen faydalardan biridir.

Bir diğer programlama paradigması (ve günümüz yazılım geliştirmede e n belirgin olanı) nesne yönelimli paradır, bu da nesne yönelimli programlam a (OOP) olarak adlandırılan programlama süreci ile ilişkilidir.
Bu paradigmayı takip ederek, bir yazılım sistemi, her biri kendisiyle doğrudan ilişkili eyleml eri gerçekleştirme yeteneğine sahip olan nesneler olarak adlandırılan birimlerin bir koleksi yonu olarak görülür ve diğer nesnelerin eylemlerini talep edebilir. Bu nesneler birlikte, me vcut sorunu çözmek için etkileşimde bulunurlar.

Nesne yönelimli yaklaşımın işleyişine bir örnek olarak, grafiksel bir kullan ıcı arayüzü geliştirme görevini düşünün. Nesne yönelimli bir ortamda, ekran da görünen simgeler nesneler olarak uygulanır. Bu nesnelerin her biri, o nes nenin fare düğmesine tıklanarak seçilmesi veya fare ile ekranda sürüklenme si gibi çeşitli olayların meydana gelmesine nasıl yanıt vereceğini tanımlayan bir dizi prosedürü (nesne yönelimli dilde yöntemler olarak adlandırılır) kapsa r. Böylece, tüm sistem, her biri kendisiyle ilgili olaylara nasıl yanıt vereceğini bilen nesnelerin bir koleksiyonu olarak inşa edilir.

Nesne yönelimli paradigmayı zorlayıcı paradigma ile karşılaştırmak için, bir isimler listesi içeren bir programı düşünün. Geleneksel zorlayıcı paradig mada, bu liste yalnızca bir veri koleksiyonu olurdu. Listeye erişen herhangi bi r program birimi, gerekli manipülasyonları gerçekleştirmek için algoritmaları içermek zorundaydı. Ancak nesne yönelimli yaklaşımda, liste, listeyi ve listeyi manipüle etmek için bir dizi yöntemi içeren bir nesne olarak inşa edilir. (Bu, l isteye yeni bir giriş eklemek, listeden bir girişi silmek, listenin boş olup olma dığını tespit etmek ve listeyi sıralamak için prosedürleri içerebilir.) Bununla b irlikte, listeyi manipüle etmesi gereken başka bir program birimi, ilgili görevl eri gerçekleştirmek için algoritmaları içermeyecektir. Bunun yerine, nesnede sağlanan prosedürleri kullanacaktır. Bir anlamda, zorlayıcı paradigmada old uğu gibi listeyi sıralamak yerine, program birimi listeye kendisini sıralamasın ı isteyecektir.

Objeye yönelik paradigmayı daha ayrıntılı olarak 6.5. Bölümde tartışacağız, ancak günümüz yazılım geliştirme alanındaki önemi, bu tanıma bir sınıf kavra mını dahil etmemizi gerektiriyor. Bu amaçla, bir nesnenin, veri (örneğin bir isim ler listesi) ile birlikte etkinlikleri gerçekleştirmek için yöntemler koleksiyonunda n (örneğin listeye yeni isimler eklemek gibi) oluşabileceğini hatırlayın.
Bu özellikler, yazılı programda ifadelerle tanımlanmalıdır. Nesnenin özellikler inin bu tanımına sınıf denir. Bir sınıf oluşturulduktan sonra, bu özelliklere sa hip bir nesne gerektiğinde her zaman uygulanabilir. Böylece, birkaç nesne a ynı sınıftan (yani, aynı sınıftan inşa edilmiş) oluşturulabilir.

Doclingo https://www.doclingo.ai

İkiz kardeşler gibi, bu nesneler farklı varlıklar olacaktır, ancak aynı şablondan (aynı sınıftan) inşa edildikleri için aynı özelliklere sahip olacaklardır. (Belirli bi r sınıfa dayanan bir nesne, o sınıfın bir örneği olarak adlandırılır.)

Nesnelerin iyi tanımlanmış birimler olması ve tanımlarının yeniden kullanılabilir sını flarda izole edilmesi nedeniyle, nesneye yönelik paradigma popülerlik kazanmıştır. Gerçekten de, nesneye yönelik programlamanın savunucuları, nesneye yöne lik paradigmanın yazılım geliştirmeye "yapı taşı" yaklaşımı için doğal bir orta m sağladığını savunuyorlar. Yeni yazılım sistemlerinin, birçok geleneksel ürü nün raflardan alınan bileşenlerden inşa edildiği şekilde, önceden tanımlanmı ş sınıflardan oluşan yazılım kütüphaneleri hayal ediyorlar.
Bu tür kütüphanelerin inşası ve genişletilmesi sürekli bir süreçtir, bunuBölüm 7'de öğre neceğiz.

Sonuç olarak, bir nesne içindeki yöntemlerin esasen küçük zorlayıcı prog ram birimleri olduğunu belirtmeliyiz. Bu, nesne yönelimli paradigma temelin de olan çoğu programlama dilinin zorlayıcı dillerde bulunan birçok özelliği iç erdiği anlamına gelir. Örneğin, popüler nesne yönelimli dil C++, zorlayıcı dil o larak bilinen C'ye nesne yönelimli özellikler eklenerek geliştirilmiştir. Ayrıca, J ava ve C# C++'ın türevleri olduğundan, bu zorlayıcı çekirdeği de miras almışl ardır. Bölüm 6.2 ve 6.3'te bu zorlayıcı özelliklerin çoğunu keşfedeceğiz ve bu süreçte, günümüzün nesne yönelimli yazılımlarının büyük bir kısmını etkiley en kavramları tartışacağız. Ardından, Bölüm 6.5'te nesne yönelimli paradigm aya özgü özellikleri ele alacağız.

Sorular ve Alıştırmalar

1. Üçüncü nesil bir dildeki bir program ne anlamda makine bağımsızdır ? Hala ne anlamda makine bağımlıdır?
2. Bir montajcı ile bir derleyici arasındaki fark nedir?
3. Zorlayıcı programlama paradigmasını, mevcut sorunun çözümüne g ötüren bir süreci tanımlamaya vurgu yaptığını söyleyerek özetleyebili riz. Benzer şekilde, deklaratif, fonksiyonel ve nesne yönelimli paradig maların özetini verin.
4. Üçüncü nesil programlama dilleri, önceki nesillerden hangi anlamda daha yüksek bir seviyededir?

6.2 Geleneksel Programlama Kavramları

Bu bölümde, hem zorlayıcı hem de nesne yönelimli programlama dillerinde bulunan bazı kavramları ele alacağız. Bu amaçla, Ada, C, C++, C#, FORTRAN v e Java dillerinden örnekler alacağız. Amacımız, belirli bir dilin detaylarına takı lmak değil, yalnızca yaygın dil özelliklerinin gerçek dillerde nasıl göründüğün ü göstermektir. Bu nedenle, dil koleksiyonumuz, manzarayı temsil edecek şe kilde seçilmiştir. C, üçüncü nesil zorlayıcı bir dildir. C++, C dilinin bir uzantısı o larak geliştirilmiş bir nesne yönelimli dildir.

Doclingo https://www.doclingo.ai

Java ve C#, C++'dan türetilmiş nesne yönelimli dillerdir. (Java, daha sonra Oracle tarafından satın alınan Sun Microsystems'te geliştirilmiştir, C# ise Microsoft'un bir ürünüdür.) FORTRAN ve Ada, başlangıçta üçüncü nesil zorlayıcı diller olarak tasarlanmışlardır, ancak daha yeni sürümleri çoğu nesne yönelimli paradigmayı kapsayacak şekilde genişlemiştir. Ek D, bu dillerin her biri hakkında kısa bir arka plan içermektedir.

C++, Java ve C# gibi nesne yönelimli dilleri örnek dillerimiz arasında bulundursak da, bu bölümü zorlayıcı paradigma içinde bir program yazıyormuşuz gibi ele alacağız, çünkü nesne yönelimli bir program içindeki birçok birim (bir nesnenin dışsal bir uyarana nasıl tepki vermesi gerektiğini tanımlayan prosedürler gibi) esasen kısa zorlayıcı programlardır. Daha sonra, 6.5. bölümde, nesne yönelimli paradigmaya özgü özelliklere odaklanacağız.

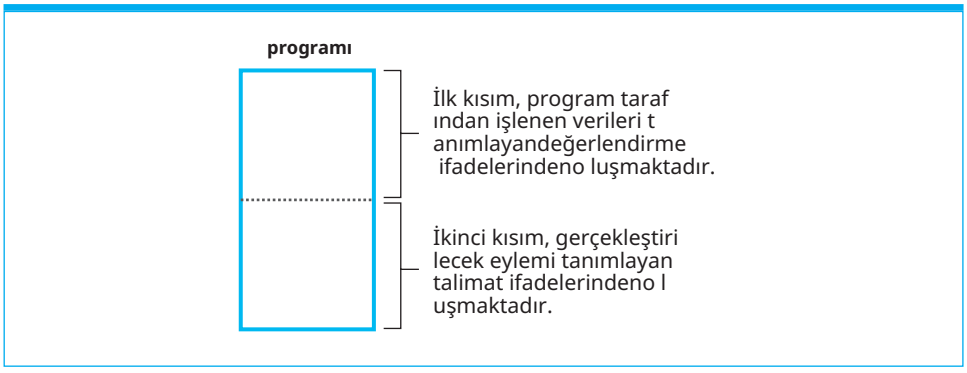
Genel olarak, bir program, üç kategoriye düşme eğiliminde olan bir dizi ifadeden oluşur: deklaratif ifadeler, zorlayıcı ifadeler ve yorumlar. Deklaratif ifadeler, programda daha sonra kullanılacak özel terimleri tanımlar, örneğin veri öğelerini referans almak için kullanılan isimler; zorlayıcı ifadeler, temel algoritmalardaki adımları açıklar; ve yorumlar, programın daha insan uyumlu bir biçimde esrarengiz özelliklerini açıklayarak okunabilirliğini artırır. Normalde, bir zorlayıcı program (veya nesne yönelimli bir program içindeki bir zorlayıcı program birimi), Şekil 6.4'te gösterilen yapıya sahip olduğu düşünülebilir. Programın manipüle edeceği verileri tanımlayan bir dizi deklaratif ifade ile başlar. Bu ön materyali, yürütülecek algoritmayı tanımlayan zorlayıcı ifadeler takip eder. Birçok dil artık deklaratif ve zorlayıcı ifadelerin serbestçe iç içe geçmesine izin veriyor, ancak kavramsal ayrım devam ediyor. Yorum ifadeleri, programı netleştirmek için gerektiği kadar dağılmıştır.

Bu öncülün ardından, programlama kavramlarını incelemeye, bir programda karşılaşabileceğimiz ifade kategorilerini, deklaratif ifadelerle ilişkili kavramlarla başlayarak, sırayla ele alarak yaklaşırız.

Değişkenler ve Veri Türleri

6.1. Bölümde önerildiği gibi, yüksek seviyeli programlama dilleri, ana bellekteki konumların sayısal adresler yerine tanımlayıcı adlarla referans alınmasına izin verir. Böyle bir ad, konumda saklanan değerin değişmesiyle birlikte, adla ilişkili değerin program çalıştıkça değiştiği gerçeğini tanıdığı için değişken olarak bilinir.

Şekil **6.4** Tipik bir zorlayıcı programın veya program biriminin bileşimi



Doclingo https://www.doclingo.ai

Betik Dilleri

İmperatif programlama dillerinin bir alt kümesi, betik dilleri olarak bilinen dillerin koleksiyonudur.Bu diller genellikle karmaşık programlar geliştirmek yerine idari görevleri yerine getirmek için kullanılır. Böyle bir görevin ifade sine betik denir, bu da "betik dili" terimini açıklar. Örneğin, bir bilgisayar si steminin yöneticisi, her akşam gerçekleştirilmesi gereken bir kayıt tutma et kinliği dizisini tanımlamak için bir betik yazabilir veya bir PC kullanıcısı, bir d ijital kameradan resimleri okumak, resimleri tarihe göre dizinlemek ve bunl arın kopyalarını arşivleme depolama sisteminde saklamak için gereken pro gramların bir dizisinin yürütülmesini yönlendirmek için bir betik yazabilir. B etik dillerinin kökeni, 1960'larda toplu işleme görevlerinin zamanlamasında bir işletim sistemini yönlendirmek için kullanılan iş kontrol dillerine kadar i zlenebilir (bkz. Bölüm 3.1). Bugün bile, birçok kişi betik dillerini diğer progra mların yürütülmesini yönlendirmek için diller olarak görmektedir, bu da me vcut betik dillerinin oldukça kısıtlayıcı bir görüşüdür. Betik dilleri örnekleri a rasında, sunucu tarafı Web uygulamalarını kontrol etmede popüler olan Pe rl ve PHP bulunmaktadır (bkz. Bölüm 4.3). Ayrıca, Microsoft tarafından geliş tirilen ve
Windows'a özgü durumlarda kullanılan Visual Basic'in bir lehçesi olan VBScript de vardır.

Değeri saklayan konumda değişiklik yaparak, adla ilişkili değer program çalış tıkça değişir. Örnek dillerimiz, değişkenlerin programın başka yerlerinde kull anılmadan önce bir tanımlayıcı ifade ile tanımlanmasını gerektirir. Bu tanıml ayıcı ifadeler ayrıca programcının, değişkenle ilişkili bellek konumunda sakla nacak veri türünü tanımlamasını gerektirir.

Böyle bir tür, veri türü olarak bilinir ve veri öğesinin nasıl kodlandığını ve o ve riye uygulanabilecek işlemleri kapsar. Örneğin, tam sayı türü, muhtemelen iki'nin tamamlayıcı notasyonu kullanılarak saklanan tam sayılardan oluşan sayısal veriyi ifade eder. Tam sayı verileri üzerinde gerçekleştirilebilecek işlemler, geleneksel ar itmetik işlemler ve bir değerin diğerinden daha büyük olup olmadığını belirleme g ibi karşılaştırmaları içerir. **float** (bazen gerçek olarak adlandırılır) türü, muhtemel en kayan nokta notasyonu ile saklanan tam sayılardan farklı değerler içerebilecek sayısal veriyi ifade eder. float türündeki veriler üzerinde gerçekleştirilen işlemler, t am sayı türündeki veriler üzerinde gerçekleştirilenlere benzer. Ancak, iki float tür ündeki öğeyi toplamak için gereken etkinliğin, iki tam sayı türündeki öğeyi toplam ak için gereken etkinlikten farklı olduğunu unutmayın.

O halde, `WeightLimit` değişkenini, iki'nin tamamlayıcı notasyonunda ko dlanmış bir sayısal değeri içeren ana bellekteki bir alanı ifade etmek için bir programda kullanmak istediğimizi varsayalım. C, C++, Java ve C# dillerinde ni yetimizi belirtmek için aşağıdaki ifadeyi ekleriz

```
int AğırlıkSınırı;
```

programın başında. Bu ifade "İsim
AğırlıkSınırı, programda daha sonra iki'nin tamamlayıcı notasyonunda saklanan bir değ eri içeren bir bellek alanına atıfta bulunmak için kullanılacaktır." Birden fazla değişkenin

Doclingo https://www.doclingo.ai

aynı tür genellikle aynı bildirim ifadesinde bildirilebilir. Örneğin,

```
int Yükseklik, Genişlik;
```

ifadesi hem Yükseklik hem de Genişlik değişkenlerini tam sayı türünde değişkenler olarak bildirir. Dahası, çoğu dil, bir değişkenin bildirildiğinde başlangıç değeri almasına izin verir. Böylece,

```
int WeightLimit = 100;
```

tartım sınırını tam sayı türünde bir değişken olarak bildirmekle kalmaz, aynı zamanda ona başlangıç değeri 100'ü de atar.

Diğer yaygın veri türleri karakter ve Boolean'dır. Karakter türü, muhtemelen ASCII veya Unicode kullanılarak depolanan sembollerden oluşan veriyi ifade eder. Bu tür veriler üzerinde gerçekleştirilen işlemler, bir sembolün alfabetik sırada diğerinden önce gelip gelmediğini belirleme, bir sembol dizisinin başka bir dizinin içinde görünüp görünmediğini test etme ve bir sembol dizisini başka bir dizinin sonuna ekleyerek tek bir uzun dize oluşturma gibi karşılaştırmaları içerir.
ifadesi

```
char Harf, Rakam;
```

C, C++, C# ve Java dillerinde Harf ve Rakam değişkenlerini karakter türünde bildirmek için kullanılabilir.

Boolean türü, yalnızca doğru veya yanlış değerlerini alabilen veri öğelerini ifade eder. Boolean türündeki veriler üzerindeki işlemler, mevcut değerin doğru mu yoksa yanlış mı olduğunu sorgulamayı içerir. Örneğin, eğer LimitAşımı değişkeni Boolean türünde bildirildiyse, o zaman aşağıdaki biçimde bir ifade

```
eğer (LimitExceeded) ise (...) yoksa (...)
```

makul olurdu.

Bir programlama dilinde yer alan ve ilkel veri türleri olarak adlandırılan veri türleri, örneğin `int` tam sayı için ve `char` karakter için, ilkel veri türleri olarak adlandırılır. Öğrendiğimiz gibi, tam sayı, float, karakter ve Boolean türleri yaygınilkel türlerdir. Henüz yaygınlaşmamış diğer veri türleriresimler, ses, video ve hiper metin gibi türlerdir. Ancak, GIF, JPEGve HTML gibi türler yakında tam sayı ve float kadar yaygın hale gelebilir. Daha sonra (Bölümler6.5 ve 8.4) nesne yönelimli paradigmanın bir programcınınbir dilde sağlanan ilkel türlerin ötesinde mevcut veri türleri repertuarını genişletmesine nasıl olanak tanıdığını öğreneceğiz. Gerçekten de, bu yetenek nesne yönelimli paradigmanınkutlanan bir özelliğidir.

Özetle, C dilinde ve onun türevleri C++, C# ve Java'da ifade edilen aşağıdaki program segmenti,`Length` ve`Width` değişkenlerini float türünde, `Price`, `Tax` ve `Total` değişkenlerini tam sayı türünde,ve `Symbol` değişkenini karakter türünde olarak tanımlar.

```
kayan nokta Uzunluk, Genişlik;
int         Fiyat, Vergi, Toplam;
karakter Sembolü;
```

Bölüm 6.4'te bir çevirmenin, yüksek seviyeli bir dilden makine diline bir programı çevirmek içinbu tür beyan ifadelerinden topladığı bilgiyi nasıl kullandığını göreceğiz. Şimdilik, bu tür bilgilerin kullanılabileceğini not ediyoruz.

Doclingo https://www.doclingo.ai

hataları tanımlamak için. Örneğin, bir çevirmen daha önce Boolean türünde olduğun u beyan eden iki değişkenin toplanmasını talep eden bir ifade bulursa, bu ifadeyi mu htemelen hata olarak değerlendirmeli ve bu bulguyu kullanıcıya bildirmelidir.

Veri Yapısı

Veri türüne ek olarak, bir programdaki değişkenler genellikle veri yapısı ile ili şkilendirilir, bu da verinin kavramsal şekli veya düzenidir. Örneğin, metin ge nellikle uzun bir karakter dizisi olarak görülürken, satış kayıtları sayısal değer lerin dikdörtgen bir tablosu olarak hayal edilebilir; burada her satır belirli bir çalışanın yaptığı satışları ve her sütun belirli bir günde yapılan satışları temsil eder.

Yaygın bir veri yapısı, aynı türdeki elemanların bir bloğu olan dizidir; bu, bir boyutlu bir liste, satır ve sütunlara sahip iki boyutlu bir tablo veya daha y üksek boyutlara sahip tablolar olabilir. Bir programda böyle bir diziyi oluştur mak için birçok programlama dili, dizinin adını beyan eden beyan ifadesinin dizinin her boyutunun uzunluğunu da belirtmesini gerektirir. Örneğin, Şekil 6.5, beyan ifadesiyle tanımlanan kavramsal yapıyı göstermektedir.

```
int Puanlar[2][9];
```

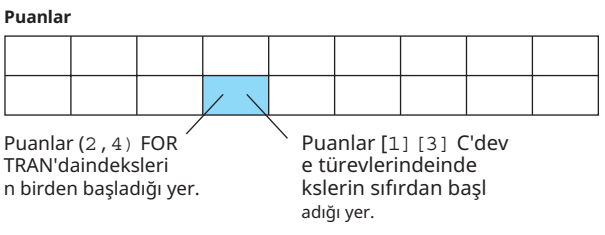
C dilinde, "Değişken `Scores`, aşağıdaki program biriminde iki boyutlu bir ta m sayı dizisine atıfta bulunmak için kullanılacaktır.İki satır ve dokuz sütun içe ren." Aynı ifade FORTRAN'daşu şekilde yazılacaktır

```
TAM SAYI Puanları(2,9)
```

Bir dizi tanımlandıktan sonra, programın başka yerlerinde adıyla veya bireys el bir eleman, satırı, sütunu vb. belirten indeksler olarak adlandırılan tam sa yı değerleri aracılığıyla tanımlanabilir. Ancak, bu indekslerin aralığı dilden dil e değişir. Örneğin, C'de (ve türevleri C++, Java ve C#) indeksler 0'dan başlar, bu da `Scores` (yukarıda tanımlandığı gibi) adlı dizinin ikinci satırındaki ve dör düncü sütunundaki girişin `Scores[1][3]` ile referans alınacağı anlamına ge lir ve birinci satır ve birinci sütundaki giriş `Scores[0][0]` olacaktır. Buna ka rşılık, FORTRAN programında indeksler 1'den başlar, bu nedenle ikinci satır v e dördüncü sütundaki giriş `Scores(2,4)` ile referans alınacaktır (bkz. tekrar Şekil 6.5).

Tüm veri öğelerinin aynı türde olduğu bir dizinin aksine, bir toplu tür (ay nı zamanda yapı, bir kayıt veya bazen heterojen dizi olarak da adlandırılır)

Şekil **6.5** İki satır ve dokuz sütunlu iki boyutlu bir dizi



Doclingo https://www.doclingo.ai

farklı elemanların farklı türlere sahip olabileceği bir veri bloğudur. Örneğin, bir çalışana atıfta bulunan bir veri bloğu,
karakter türünde bir İsim, tam sayı türünde bir Yaş ve
float türünde bir YetenekDeğeri içeren bir girişten oluşabilir. Böyle bir toplu tür, C'deşu ifadeyle t anımlanır:

```
struct {karakter Adı[25];
        int     Yaş;
        float YetenekDeğeri;}
       Çalışan;
```

değişkenin `Employee`, üç bileşenden oluşan bir yapıyı (kısacastruct) ifade et mesi gerektiğini belirtir: `Name` (25 karakterlik bir dize),Age ve `SkillRating` (Şekil 6.6). Böyle bir toplam tanımlandıktan sonra, birprogramcı yapı adını (`Employee`) kullanarak tüm toplamı ifade edebilir veyabileşen adını takip ede n bir nokta ve alan adı ile toplam içindeki bireysel alanları referans alabilir (ö rneğin `Employee.Age`).

8. Bölümde, diziler gibi kavramsal yapıların bir bilgisayar içinde nasıl uygulandığını göreceğiz. Özellikle, bir dizide bulunan verilerin ana belleğin veya kütle depolamanın geniş bir alanına dağılmış olabileceğini öğreneceğiz. Bu, veri yapısını verilerin *kavramsal* şekli veya düzeni olarak adlandırmamızın nedenidir. Gerçekten de, bilgisayarın depolama sistemindeki gerçek düzen, kavramsal düzeninden çok farklı olabilir.

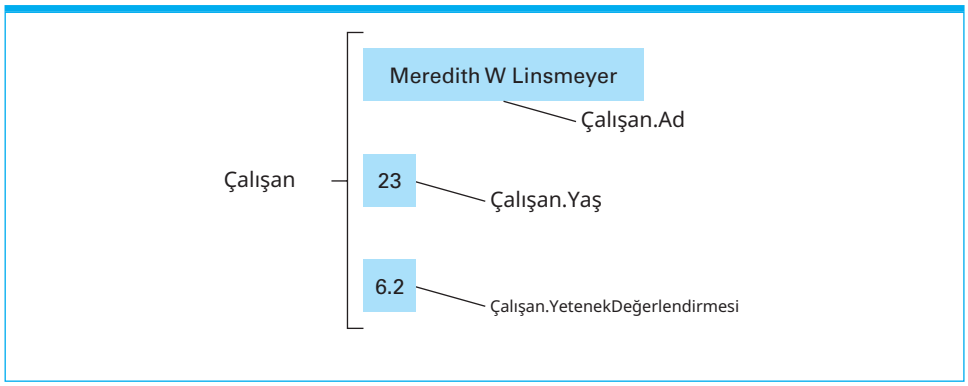
Sabitler ve Literaller

Bazen bir programda sabit, önceden belirlenmiş bir değer kullanılır. Örneğin, belirli bir havaalanının çevresindeki hava trafiğini kontrol eden bir program, o havaalanının deniz seviyesinden yüksekliğine birçok atıfta bulunabilir. Böyle bir program yazarken, bu değ eri, örneğin 645 feet, gerektiği her seferinde kelime anlamıyla dahil edebilirsiniz. Bir de ğerin bu şekilde açıkça görünmesine literal denir. Literallerin kullanımı, aşağıdaki gibi p rogram ifadelerine yol açar:

```
EffectiveAlt ← Altimeter + 645
```

burada `EffectiveAlt` ve`Altimeter` değişkenler olarak varsayılmakta ve `645` birlite ral olarak kabul edilmektedir. Böylece, bu ifade `EffectiveAlt` değişkenine, `Altimeter` değişkenine atanan değere 645 eklemenin sonucunu atamasını istemektedir.

Şekil 6.6Çalışan yapısının kavramsal düzeni



Doclingo https://www.doclingo.ai

Çoğu programlama dilinde, metin içeren literaller, diğer program bileşenlerinden ayırt etmek için tırnak işaretleriyle belirlenir. Örneğin, aşağıdaki ifade:

```
SonAd←"Smith"
```

metni "Smith" değişkenine atamak için kullanılabilir SonAd, oysa ifade

```
SonAd←Smith
```

değişkenin değerini `Smith` değişkenine atamak için kullanılacaktır.

Sıklıkla, literal kullanımı iyi bir programlama pratiği değildir çünkü literal'lar bulundukları ifadelerin anlamını maskeleyebilir. Örneğin, bir okuyucu bu ifadenin 645 değerinin neyi temsil ettiğini nasıl bilebilir?

```
EffectiveAlt ← Altimeter + 645
```

Ayrıca, literal'lar programı değiştirme görevini zorlaştırabilir eğer bu gerekli olursa. Eğer hava trafiği programımız başka bir havaalanına taşınırsa, havaalanının yüksekliğine yapılan tüm referansların değiştirilmesi gerekir. Eğer 645 literal'ı her referansta kullanılıyorsa, program boyunca her bir referansın bulunup değiştirilmesi gerekir. Problem, 645 literal'ının havaalanının yüksekliğinden başka bir miktara referans olarak da geçmesi durumunda daha da karmaşık hale gelir.

Hangi 645 geçişlerini değiştireceğimizi ve hangilerini olduğu gibi bırakacağımızı nasıl biliyoruz?

Bu sorunları çözmek için, programlama dilleri belirli, değiştirilemez değerlere tanımlayıcı isimler atanmasına izin verir. Böyle bir isme sabit denir.

Örnek olarak, C++ ve C#'da, deklaratif ifade

```
const int AirportAlt = 645;
```

tanımlayıcı `AirportAlt`'ı sabit değer 645 ile ilişkilendirir (bu, tam sayı türünde kabul edilir). Java'daki benzer kavram ise

```
final int AirportAlt = 645;
```

Böyle deklarasyonlardan sonra, tanımlayıcı isim `AirportAlt` literal 645 yerine kullanılabilir. Pseudocode'umuzda böyle bir sabit kullanarak, ifade

```
EffectiveAlt ← Altimeter + 645
```

şu şekilde yeniden yazılabilir

```
EffectiveAlt ← Altimeter + AirportAlt
```

ifade anlamını daha iyi temsil eder. Ayrıca, eğer böyle sabitler, literallerin yerine kullanılırsa ve program başka bir havaalanına taşınırsa, bu havaalanının yüksekliği 267 feet olan bir yere, sabitin tanımlandığı tek bildirimli ifadeyi değiştirmek, havaalanının yüksekliğine yapılan tüm referansları yeni değere dönüştürmek için yeterlidir.

Atama İfadeleri

Bir programda kullanılacak özel terminoloji (değişkenler ve sabitler gibi) ilan edildikten sonra, bir programcı ilgili algoritmaları tanımlamaya başlayabilir. Bu, zorlayıcı ifadeler aracılığıyla yapılır. En temel zorlayıcı ifade, bir değerin

Doclingo https://www.doclingo.ai

bir değişkene atanmasını (veya daha doğru bir şekilde, değişken tarafından tan ımlanan bellek alanında saklanmasını) talep eden atama ifadesidir. Böyle bir ifa de genellikle bir değişkenin, ardından atama işlemini temsil eden bir sembolün ve ardından atanacak değeri belirten bir ifadenin yer aldığı sözdizimsel biçimi alır. Böyle bir ifadenin anlamı, ifadenin değerlendirilmesi ve sonucun değişkeni n değeri olarak saklanmasıdır. Örneğin,

```
Z = X + Y;
```

C, C++, C# ve Java'da, X ve Y'nin toplamının değişkene atanmasını talep eder Z. Diğer bazı dillerde (örneğin Ada) eşdeğer ifade şöyle görünür

```
Z := X + Y;
```

Bu ifadelerin yalnızca atama operatörünün sözdiziminde farklılık gösterdiğini unutm ayın,C, C++, C# ve Java'da yalnızca bir eşittir işareti iken, Ada'da bir iki nokta üst üste ve ardından bir eşittir işareti gelir. Belki de atama operatörü için daha iyi bir notasyo n,1962'de Kenneth E. Iverson tarafından tasarlanan APL dilinde bulunur. (APLBir Pro gramlama Dili'nin kısaltmasıdır.) Atamayı temsil etmek için bir ok kullanır.
Böylece, önceki atama şöyle ifade edilecektir

```
Z ← X + Y
```

APL'de (5. Bölümdeki sözdekodumuzda olduğu gibi).

Atama ifadelerinin gücünün büyük bir kısmı, ifadenin sağ tarafında görü nebilecek ifadelerin kapsamından gelir. Genel olarak, herhangi bir cebirselif ade kullanılabilir; toplama, çıkarma,çarpma ve bölme aritmetik işlemleri gen ellikle sırasıyla +,, *, ve / sembolleriyle temsil edilir. Bazı dillerde ** kombina syonu üslü sayıları temsil etmek için kullanılır. Örneğin, Ada'da ifade

```
x ** 2
```

x^2'yi temsil eder. Ancak diller, cebirselifadelerin nasıl yorumlandığı konusund a farklılık gösterir. Örneğin, 2 * 4 + 6 / 2 ifadesi,eğer sağdan sola değerlendi rilirse 14 değerini, soldan sağa değerlendirilirse 7 değerini üretebilir. Bu beli rsizlikler genellikle operatör kurallarıyla çözülür.
öncelik, belirli işlemlerin diğerlerine göre öncelik taşıdığı anlamına gelir. Algebra'nın geleneksel kuralları, çarpma ve bölmenin toplama ve çıkarmaya göre önceliğe sahip olduğunu belirtir. Yani, çarpma ve bölme işlemleri, topla ma ve çıkarma işlemlerinden önce gerçekleştirilir. Bu kuralı takip ederek, ön ceki ifade 11 değerini üretir. Çoğu dilde, parantezler dilin operatör önceliğini geçersiz kılmak için kullanılabilir. Böylece 2 * (4 + 6) / 2 10 değerini üretir.

Pek çok programlama dili, bir sembolün birden fazla işlemi temsil etmesi ne izin verir. Bu durumlarda sembolün anlamı, operandların veri türü tarafın dan belirlenir. Örneğin, sembol + geleneksel olarak sayısal operandlar oldu ğunda toplama işlemini gösterir, ancak bazı dillerde, örneğin Java'da, sembo l karakter dizileri operandları olduğunda birleştirme işlemini gösterir. Yani, if adenin sonucu

```
"abra" + "cadabra"
```

abracadabra'dır. Bir işlem sembolünün bu şekilde çoklu kullanımı aşırı yükleme olarak adlandırılır.
Pek çok dil, birkaç yaygın operatör için yerleşik aşırı yükleme sağlar,

Doclingo https://www.doclingo.ai

ancak Ada, C++ ve C# gibi diğerleri, programcıların ek aşırı yüklenmiş anlamlar tanımla masına veya hatta ek operatörler eklemesine izin verebilir.

Kontrol İfadeleri

A kontrol ifadesi programın yürütme sırasını değiştirir. Tümprogramlama ya pıları arasında, bu gruptan olanlar muhtemelen en fazlailgi görmüş ve en ço k tartışma yaratmıştır. Ana kötü karakter, en basitkontrol ifadesi olan `goto` if adesidir. Bu, yürütme sırasınıbu amaçla bir ad veya numara ile etiketlenmiş başka bir yere yönlendirmearacı sağlar. Bu nedenle, makine düzeyindeki JU MP talimatının doğrudanuygulamasından başka bir şey değildir. Yüksek sevi yeli bir programlama dilindeböyle bir özelliğin sorunu, programcıların bir far e yuvası gibi yazmalarınaolanak tanımasıdır.

```
        40'a git
20 Uygula prosedürü Kaçın
        goto 70
40 eğer (KryptoniteLevel < LethalDose) ise goto 60
        goto 20
60 Uygula prosedürü KurtarKız
70   ...
```

tek bir ifade gibi

```
eğer (KryptoniteLevel < LethalDose)
        ise (uygula prosedürü KurtarKız)
        aksi takdirde (uygula prosedürü Kaçın)
```

işi yapar.

Böyle karmaşıklıkları önlemek için, modern diller, tek birsözdizimsel yapı içinde tüm bir dallanma deseninin ifade edilmesine olanaksağlayan kontrol i fadeleri ile tasarlanmıştır. Bir dilde hangi kontrolifadelerinin yer alacağına ka rar vermek bir tasarım kararıdır. Amaç,dizilerin okunabilir bir biçimde ifade e dilmesine olanak tanıyan veprogramcının böyle bir okunabilirlik elde etmesi ne yardımcı olan bir dilsağlamaktır. Bu, tarihsel olarak dağınık programlama ya yol açanözelliklerin kullanımını kısıtlayarak ve daha iyi tasarlanmışözellikle rin kullanımını teşvik ederek yapılır. Sonuç, yapılandırılmışprogramlama olar ak bilinen uygulamadır, bu, organize bir tasarım metodolojisiniyerine getire n ve dilin kontrol ifadelerinin uygun kullanımını birleştirir.Amaç, kolayca anla şılabilen ve spesifikasyonlarını karşıladığı gösterilebilenbir program üretmek tir.

Artık, 5. Bölümdeki sözdekodumuzda, `if-then-else` ve`while` ifadeleriy le temsil edilen iki popüler dallanma yapısıyla tanıştık. Bu yapılar, neredeyse tüm zorlayıcı, işlevsel veya nesne yönelimli dillerde mevcuttur. Daha kesin ol arak, sözdekod ifadeleri

```
if  (koşul)
    then (ifadeA)
    else  (ifadeB)
```

ve

```
while  (koşul) do
     (döngü gövdesi)
```

Doclingo https://www.doclingo.ai

Programlama Dili Kültürleri

Doğal dillerde olduğu gibi, farklı programlama dillerinin kullanıcıları kültürel far klılıklar geliştirme eğilimindedir ve genellikle bakış açılarını tartışırlar. Bazen bu f arklılıklar, farklı programlama paradigmalarının söz konusu olduğu durumlarda olduğu gibi, önemli olabilir. Diğer durumlarda, ayrımlar ince olabilir. Örneğin, m etin prosedürler ve fonksiyonlar arasında ayrım yaparken (Bölüm 6.3), C progra mcıları her ikisini de fonksiyon olarak adlandırır. Bunun nedeni, bir C programın daki prosedürün bir değer döndürmeyen bir fonksiyon olarak düşünülmesidir. Benzer bir örnek, C++ programcılarının bir nesne içindeki bir prosedürü üye fon ksiyonu olarak adlandırmasıdır, oysa bunun genel terimi yöntemdir. Bu tutarsızl ık, C++'nın C'nin bir uzantısı olarak geliştirilmiş olmasından kaynaklanmaktadır. Bir diğer kültürel fark, Ada'daki programların genellikle rezerve edilmiş kelimele rin büyük harfle veya kalın yazılmasıyla yazılmasıdır - bu, C, C++, C#, FORTRAN veya Java kullanıcıları tarafından yaygın olarak uygulanmayan bir gelenektir.

Bu kitap dil açısından tarafsızdır ve genel terminoloji kullanmaktadır, anc ak her özel örnek, ilgili dilin tarzıyla uyumlu bir biçimde sunulmaktadır. Bu ör neklerle karşılaştığınızda, bunların belirli bir dilin detaylarını öğretmek amacıy la değil, genel fikirlerin gerçek dillerde nasıl göründüğüne dair örnekler olara k sunulduğunu aklınızda bulundurmalısınız. Ağaçlara değil, ormana bakmaya çalışın.

şu şekilde yazılacaktır

```
if (koşul) ifadesielt ifadesi;
```

ve

```
while (koşul)
    {döngü gövdesi}
```

C, C++, C# ve Java dillerinde. Bu ifadelerin dört dilde de aynı olması, C++, C# ve Java'nın zorlayıcı dil C'nin nesne yönelimli uzantıları olmasından kaynakla nmaktadır. Buna karşılık, karşılık gelen ifadeler şu şekilde yazılacaktır

```
IF koşulTHEN
   ifadeA;
ELSE
   ifade;\aND IF
```

ve

```
WHILE koşulu LOOP\dön
   gü gövdesi
SON DÖNGÜ;
```

Ada dilinde.

Başka bir yaygın dallanma yapısı genellikle bir `switch` veyacase ifadesi il e temsil edilir. Bu, belirli bir değişkene atanan değere bağlı olarak birkaç seç enek arasından bir ifade dizisini seçme imkanı sağlar. Örneğin, ifade

Doclingo https://www.doclingo.ai

```
switch (değişken) {
   case 'A': ifadeA; break;
   case 'B': ifadeB; break;
   case 'C': ifadeC; break;
   default: ifadeD}
```

C, C++, C# ve Java'da, değişken'in mevcut değeri sırasıyla A, B veya C olduğu na bağlı olarak ifadeA, ifadeB veya ifadeC'nin yürütülmesini talep eder vey a değişken'in değeri başka bir şeyse ifadeD'nin yürütülmesini talep eder. A ynı yapı şu şekilde ifade edilir:

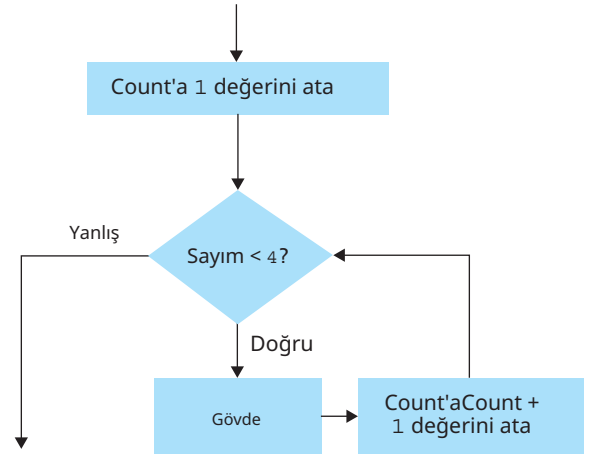
```
CASE değişken DIR
   WHEN 'A'=> ifadeA;
   WHEN 'B'=> ifadeB;
   WHEN 'C'=> ifadeC;
   WHEN DİĞERLERİ=> ifadeD;
END CASE
```

Ada'da.

Yine başka bir yaygın kontrol yapısı, genellikle for yapısı olarak adlandırıl an, Şekil 6.7'de C++, C# ve Java'daki temsili ile birlikte gösterilmektedir. Bu, s özdekodumuzdaki `while` ifadesine benzer bir döngü yapısıdır. Fark, döngün ün tüm başlatma, değiştirme ve sonlandırma işlemlerinin tek bir ifadeye dah il edilmesidir. Böyle bir ifade, döngü gövdesinin belirli bir aralıktaki her değe r için bir kez gerçekleştirilmesi gerektiğinde kullanışlıdır.

Özellikle, Şekil 6.7'deki ifadeler döngü gövdesinin tekrar tekrar gerçekleştirilmesini yönlendirir—ilk olarak `Count` değeri 1 iken, sonra `Count` değeri 2 iken ve tekrar `Count` değeri 3 iken.

Şekil **6.7** for döngüsü yapısı ve C++, C# ve Java'daki temsili




```
for (int Count = 1; Count < 4; Count++)
     gövde ;
```

Doclingo https://www.doclingo.ai

Verdiğimiz örneklerden çıkarılacak nokta, yaygındalgalanma yapıların, k üçük varyasyonlarla, zorunlu ve nesne yönelimli programlama dilleri arasınd agörünmesidir. Teorik bilgisayar biliminden biraz şaşırtıcı bir sonuç, bu yapıl arın sadece birkaçınınbir programlama dilinin herhangi bir algoritmik çözüm ü olan bir probleme çözüm ifade etme aracını sağlamak için gerekli olduğud ur. Bu iddiayı Bölüm 12'de inceleyeceğiz. Şimdilik, bir programlama dilini öğr enmenin farklı kontrol ifadelerini öğrenmekten sonsuz bir görev olmadığını sadece belirtmekteyiz. Günümüz programlama dillerinde bulunan kontrol ya pılarının çoğu, burada tanımladığımız yapıların temelde varyasyonlarıdır.

Yorumlar

Bir programlama dilinin ne kadar iyi tasarlandığı ve dilin özelliklerinin bir pro gramda ne kadar iyi uygulandığı önemli değildir; bir insan programı okumay a ve anlamaya çalıştığında genellikle ek bilgi yararlı veya zorunludur.
Bu nedenle, programlama dilleri, bir program içinde açıklayıcı ifadeler eklem enin yollarını sağlar; bunlara yorumlar denir. Bu ifadeler bir çevirmen tarafı ndan göz ardı edilir ve bu nedenle varlıkları veya yoklukları, makinenin bakış açısından programı etkilemez. Bir çevirmen tarafından üretilen programın m akine dili versiyonu, yorumlar olsun ya da olmasın aynı olacaktır, ancak bu if adeler tarafından sağlanan bilgi, bir insanın bakış açısından programın öne mli bir parçasını oluşturur. Böyle bir belgeleme olmadan, büyük ve karmaşık programlar, bir insan programcısının anlayışını kolayca engelleyebilir.

Bir program içinde yorum eklemenin iki yaygın yolu vardır. Biri, tüm yorum u özel işaretleyicilerle çevrelemektir; bir işaretleyici yorumun başında ve diğeri sonunda bulunur. Diğeri ise yalnızca yorumun başlangıcını işaretlemek ve yoru mun işaretleyicinin sağındaki satırın geri kalanını kaplamasına izin vermektir. B u tekniklerin her ikisinin örneklerini C++, C# ve Java'da buluyoruz. /* ve */ ile y orumların parantez içine alınmasına izin verirken, aynı zamanda bir yorumun // ile başlamasına ve satırın geri kalanına uzanmasına da izin verirler.

Böylece her ikisi de

```
/* This is a comment. */
```

ve

```
// This is a comment.
```

geçerli yorum ifadeleridir.

Anlamlı bir yorumun neyi oluşturduğuna dair birkaç kelime söylemek gerekir. Yeni başlayan programcılar, iç belgeleme için yorum kullanmaları söylendiği nde, genellikle

```
ApproachAngle = SlipAngle + HyperSpaceIncline;
```

“HyperSpaceIncline ve SlipAngle ekleyerek ApproachAngle'ı hesapla” gibi bir yoruml a bir program ifadesini takip etme eğilimindedirler. Bu tür bir gereksizlik, bir progra mın netliğinden çok uzunluğunu artırır. Bir yorumun amacı programı açıklamaktır, t ekrar etmek değil. Bu durumda daha uygun bir yorum, ApproachAngle'ın neden hes aplandığını açıklamak olabilir (eğer bu açık değilse). Örneğin, “ApproachAngle daha sonra ForceFieldJettisonVelocity'yi hesaplamak için kullanılır ve bundan sonra gerek mez” yorumu, önceki yorumdan daha faydalıdır.

Doclingo https://www.doclingo.ai

Ayrıca, bir programın ifadeleri arasında dağılmış yorumlar, bazen bir ins anın programın akışını takip etme yeteneğini engelleyebilir ve böylece progr amı anlamayı, hiç yorum eklenmemiş olmasından daha zor hale getirebilir. İ yi bir yaklaşım, bir program birimine ait olan yorumları tek bir yerde toplama k, belki de birimin başında yapmaktır. Bu, program biriminin okuyucusunun açıklamalar için bakabileceği merkezi bir yer sağlar. Ayrıca, program birimini n amacı ve genel özelliklerinin tanımlanabileceği bir konum sağlar. Bu forma t tüm program birimleri için benimsenirse, tüm program, her birimin açıklayı cı ifadeler bloğu ile takip eden program biriminin resmi sunumundan oluştu ğu bir uniformite derecesi kazanır.

Bir programdaki bu tür bir tutarlılık, okunabilirliğini artırır.

Sorular ve Alıştırmalar

1. Sabitlerin kullanımı, bir literal kullanımından neden daha iyi bir programlama stili olarak kabul edilir?
2. Bir deklaratif ifade ile bir imperatif ifade arasındaki fark nedir?
3. Bazı yaygın veri türlerini listeleyin.
4. Imperatif ve nesne yönelimli programlama dillerinde bulunan bazı y aygın kontrol yapıları nelerdir?
5. Bir dizi ile bir toplu tür arasındaki fark nedir?

6.3 Prosedürel Birimler

Önceki bölümlerde büyük programları yönetilebilir birimlere ayırmanın avan tajlarını gördük. Bu bölümde, bir prosedür kavramına odaklanıyoruz; bu, bir imperatif dilde bir programın modüler bir temsilini elde etmenin ana tekniği dir. Ayrıca, nesne yönelimli dillerde, programcılar nesnelerin çeşitli uyarıcılar a nasıl yanıt vermesi gerektiğini prosedürler aracılığıyla belirtirler.

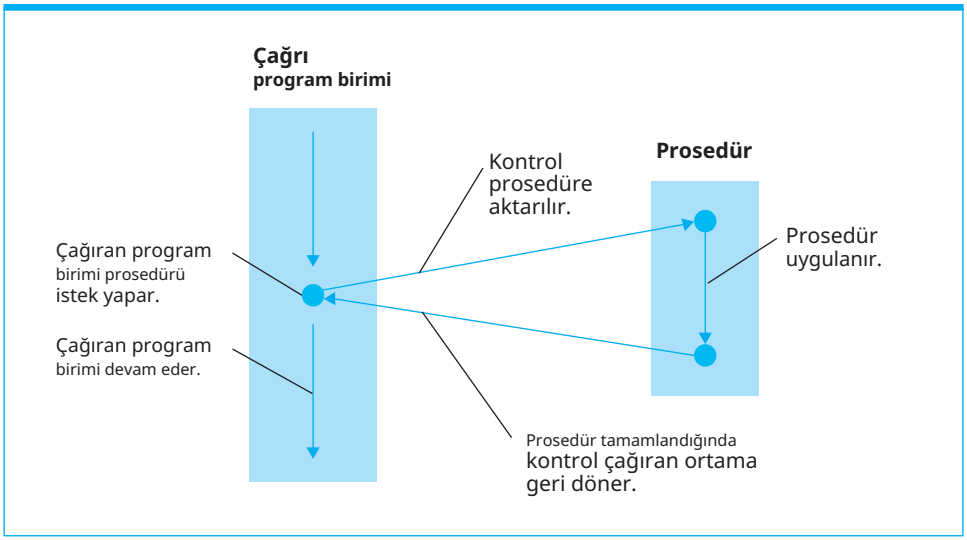
Prosedürler

Bir prosedür, genel anlamda, bir görevi yerine getirmek için bir dizi talimattır ve d iğer program birimleri tarafından soyut bir araç olarak kullanılabilir. Kontrol, hizm etlerinin gerektiği anda prosedüre aktarılır ve prosedür tamamlandıktan sonra ori jinal program birimine geri döner (Şekil 6.8). Kontrolün bir prosedüre aktarılması süreci genellikle prosedürü çağırma veya yürütme olarak adlandırılır. Bir prosedü rün yürütülmesini talep eden program birimine çağıran birim denir.

5. Bölümdeki sözdekodumuzda olduğu gibi, prosedürler genellikle birey sel program birimleri olarak yazılır. Birim, prosedürün adını tanımlayan, diğe r şeylerin yanı sıra, prosedürün başlığı olarak bilinen bir ifadeyle başlar. Bu b aşlıktan sonra, prosedürün ayrıntılarını tanımlayan ifadeler gelir. Bu ifadeler, geleneksel bir

Doclingo https://www.doclingo.ai

Şekil **6.8** Bir prosedürü içeren kontrol akışı



emirsel programda olduğu gibi, prosedürde kullanılan değişkenleri tanımlay an bildirim ifadeleriyle başlayıp, prosedür çalıştırıldığında gerçekleştirilecek adımları tanımlayan emirsel ifadelerle düzenlenir.

Genel bir kural olarak, bir prosedür içinde tanımlanan bir değişken, yalnızca o prosedür içinde referans alınabilen bir yerel değişkendir. Bu, bağımsız olarak yazı lmış iki prosedürün aynı isimde değişkenler kullanması durumunda ortaya çıkabil ecek herhangi bir karışıklığı ortadan kaldırır. (Bir değişkenin referans alınabileceği programın kısmına değişkenin kapsamı denir. Dolayısıyla, bir yerel değişkenin ka psamı, tanımlandığı prosedürdür. Kapsamları belirli bir program bölümüne kısıtla nmamış değişkenler ise küresel değişkenler olarak adlandırılır. Çoğu programlam a dili, bir değişkenin yerel mi yoksa küresel mi olacağını belirtmek için bir yol sa ğlar.) 5. Bölümdeki sözdekodumuza karşılık gelen bir ifadeyle, bir prosedürün çalıştırılmasını talep ettiğimizde, "Prosedürü uygula" gibi bir ifade kullanmıştık. `DeactivateKrypton`," olarak belirtildiği gibi, modern programlama dillerinin ço ğuprosedürlerin yalnızca prosedürün adını belirterek çağrılmasına izin verir. Örne ğin,`GetNames`,`SortNames` ve `WriteNames`, bir isimler listesini edinmek,sıralama k ve yazdırmak için prosedürlerin adlarıysa, o zaman bir program,listenin alınması , sıralanması ve yazdırılması için şöyle yazılabilir:

```
GetNames;
Sıralama İsimleri;
İsimleri Yaz;
```

yerine

```
Prosedür GetNames'i uygula.
Prosedür SortNames'i uygula.
Prosedür WriteNames'i uygula.
```

Her prosedüre, prosedürün gerçekleştirdiği eylemi belirten bir ad atayarak,b u yoğunlaştırılmış form, programın anlamını yansıtan bir komut dizisi olarak görünür.

Doclingo https://www.doclingo.ai

Parametreler

Prosedürler genellikle, prosedür uygulandığında belirli hale getirilen genel teri mler kullanılarak yazılır. Örneğin, önceki bölümün Şekil 5.11'i, belirli bir liste yer ine genel bir liste terimleriyle ifade edilmiştir. Sözdekodumuzda, prosedürün b aşlığında bu tür genel terimleri parantez içinde tanımlamayı kabul ettik. Böylec e, Şekil 5.11'deki prosedür başlıkla başlar

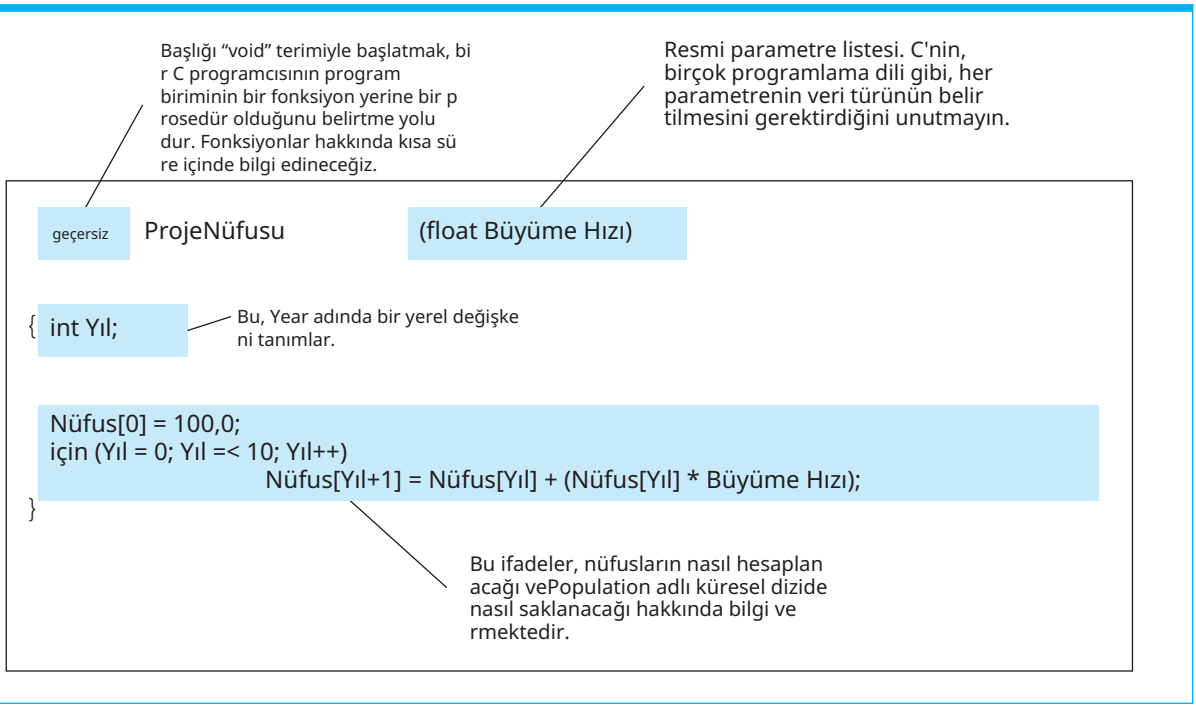
prosedürSırala (Liste)

ve ardından sıralama sürecini tanımlamak için Liste terimini kullanarak sıralanan list eye atıfta bulunur. Eğer bir düğün davetli listesini sıralamak için prosedürü uygulam ak istiyorsak, prosedürdeki talimatları izlememiz yeterlidir; genel terim Liste düğün davetli listesine atıfta bulunuyormuş gibi varsayıyoruz. Ancak, bir üyelik listesini sıral amak istiyorsak, genel terim Liste'yi üyelik listesine atıfta bulunacak şekilde yorumla mamız yeterlidir.

Prosedür içindeki bu tür genel terimlere parametreler denir. Daha kesin olarak, prosedürde kullanılan terimlere biçimsel parametreler denir ve pros edür uygulandığında bu biçimsel parametrelere atanan kesin anlamlara ger çek parametreler denir. Bir anlamda, biçimsel parametreler, prosedür talep edildiğinde gerçek parametrelerin yerleştirildiği prosedürdeki boşlukları tem sil eder.

Pseudocode'umuzda olduğu gibi, çoğu programlama dili, bir prosedür ta nımlarken biçimsel parametrelerin prosedürün başlığında parantez içinde lis telenmesini gerektirir. Örnek olarak, Şekil 6.9, programlama dilinde yazılabil eceği gibi ProjeNüfusu adında bir prosedürün tanımını sunmaktadır.

Şekil **6.9** C programlama dilinde yazılmış ProjectPopulation prosedürü



Doclingo https://www.doclingo.ai

C programlama dilinde. Prosedür çağrıldığında belirli bir yıllık büyüme oranı almasını bekler. Bu orana dayanarak, prosedür, 100'lük bir başlangıç nüfusu varsayarak,gelecek 10 yıl için bir türün tahmin edilen nüfusunu hesaplar ve bu değerleri `Population` adlı küresel dizide saklar.

Çoğu programlama dili, bir prosedür çağrıldığında gerçek parametreleri tanımlamak için parantezli notasyon kullanır. Yani, bir prosedürün yürütülm esini talep eden ifade, prosedür adının ardından parantez içinde gerçek para metrelerin bir listesini içerir. Böylece,

ProjeNüfusu'nu 0.03'lük bir büyüme oranı kullanarakuygulayın, bu da bizim sözdekodumuzda

kullandığımız ifadedir:

```
ProjectPopulation(0.03);
```

0.03'lük bir büyüme oranı kullanarak Şekil 6.9'daki `ProjectPopulation` prosedürünü çağırm ak için bir C programında kullanılacaktır.

Birden fazla parametre söz konusu olduğunda, gerçek parametreler, pro sedürünbaşlığında listelenen resmi parametrelerle, giriş giriş eşleşmesiyle ili şkilendirilir—ilk gerçek parametre, ilk resmi parametre ile ilişkilendirilir,ve b u şekilde devam eder. Ardından, gerçek parametrelerin değerleri, ilgili resmi parametrelerine etkili bir şekilde aktarılırve prosedür yürütülür.

Bu noktayı vurgulamak için, `PrintCheck` prosedürünün aşağıdaki gibi bir başlıkla tanı mlandığını varsayalım:

```
prosedür PrintCheck(Payee, Amount)
```

burada `Payee` ve`Amount`, çekin ödenecek kişi ve çekin miktarınıbelirtmek içi n prosedür içinde kullanılan resmi parametrelerdir. Ardından, aşağıdaki ifad eyle prosedürü çağırmak

```
PrintCheck("John Doe", 150)
```

prosedürün, resmi parametre Payee'nin gerçek parametre John Doe ile ilişki lendirilmesi ve resmi parametreAmount'ün 150 değeri ile ilişkilendirilmesi ile yürütülmesine neden olacaktır. Ancak, prosedürü aşağıdaki ifadeyle çağırm ak

```
PrintCheck(150, "John Doe")
```

değer 150'nin resmi parametreye Alacaklı atanmasına veisim John Doe'nun resmi parametreye Tutar atanmasına neden olacağını,bu da hatalı sonuçlar a yol açacaktır.

Veri transferi, gerçek ve biçimsel parametreler arasında, farklı programla ma dilleri tarafından çeşitli şekillerde ele alınmaktadır. Bazı dillerde, gerçek p arametreler tarafından temsil edilen verinin bir kopyası üretilir ve prosedüre verilir. Bu yaklaşımı kullanarak, prosedür tarafından yapılan veri değişiklikle ri yalnızca kopyada yansıtılır - çağıran program birimindeki veriler asla değiş tirilmez. Bu tür parametrelerin değerle geçirildiğini sıkça söyleriz. Değerle pa rametre geçişinin, çağıran birim içindeki verilerin kötü tasarlanmış bir prose dür tarafından yanlışlıkla değiştirilmesini koruduğunu unutmayın. Örneğin, ç ağıran birim bir çalışanın adını bir prosedüre geçerse, prosedürün o adı değiş tirmesini istemeyebilir.

Ne yazık ki, parametreleri değerle geçmek, parametreler büyük veri blok larını temsil ettiğinde verimsizdir. Daha verimli bir teknik,

Doclingo https://www.doclingo.ai

gerçek parametrelere doğrudan erişim sağlamak için çağıran program biriminin gerçek parametrelerinin adreslerini ona söyleyerek. Bu durumda parametrelerin referansla geçirildiğini söyleriz. Parametrelerin referansla geçirilmesinin, prosedürün çağıran ortamda bulunan verileri değiştirmesine izin verdiğini unutmayın. Böyle bir yaklaşım, bir listeyi sıralamak için bir prosedür söz konusu olduğunda arzu edilir çünkü böyle bir prosedürü çağırmanın amacı listenin değişiklikler yapmasını sağlamaktır.

Örnek olarak, prosedürün `Demo` olarak tanımlandığını varsayalım

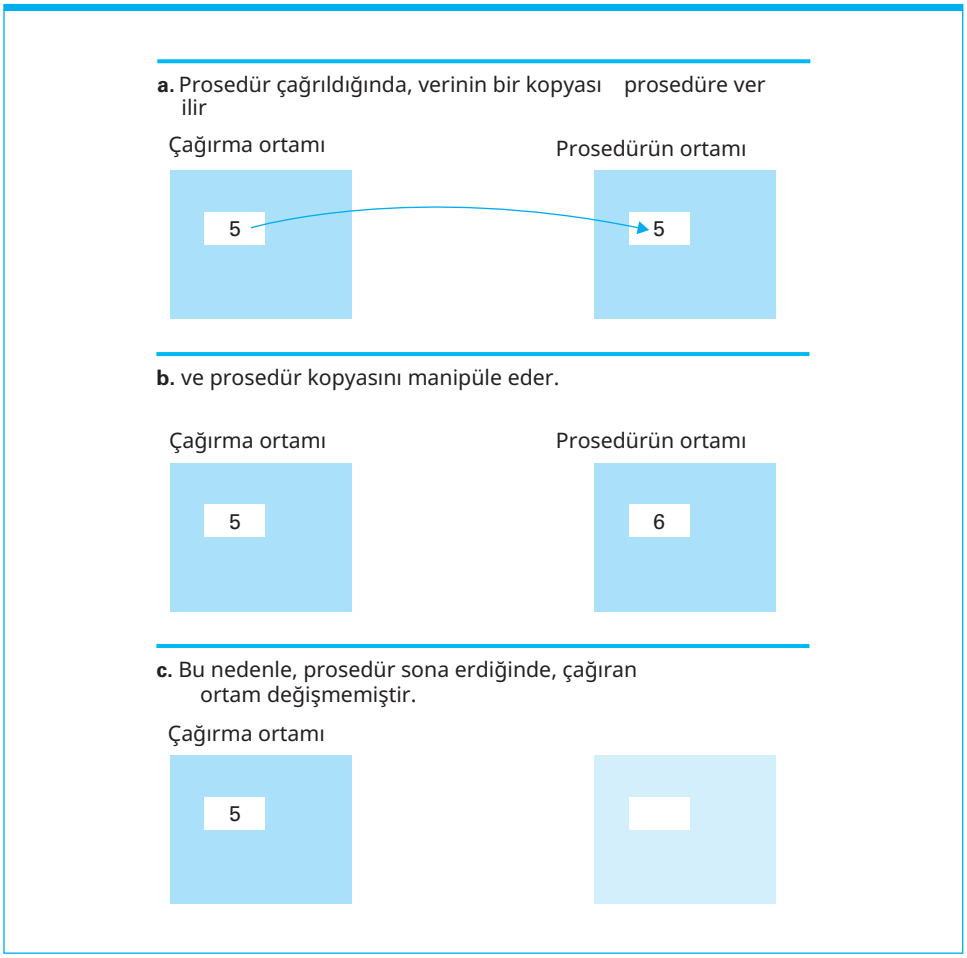
```
prosedür Demo (Formal)
Biçimsel← Biçimsel + 1;
```

Ayrıca, `Actual` değişkeninin 5 değerine atandığını ve `Demo` çağrıldığını varsayalım

```
Demo(Gerçek)
```

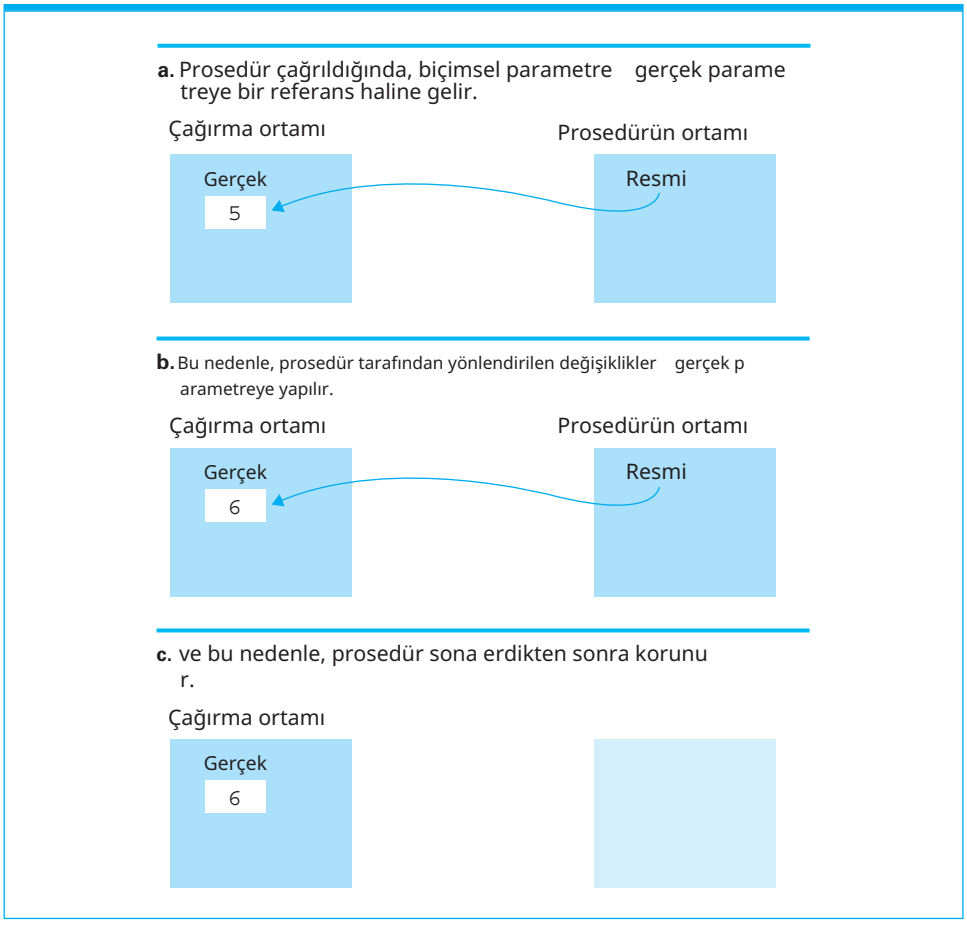
O zaman, eğer parametreler değer ile geçiliyorsa, prosedürdeki `Formal` değişkeni `Actual` değişkeninde yansıtılmayacaktır (Şekil 6.10). Ancak, eğer parametrelerreferans ile geçiliyorsa, `Actual` değeri birarttırılacaktır (Şekil 6.11).

Şekil **6.10** Prosedür Demo'yu çalıştırma ve parametreleri değerle geçirme



Doclingo https://www.doclingo.ai

Şekil **6.11** Prosedür Demo'yu çalıştırma ve parametreleri referansla geçirme



Görsel Temel

Visual Basic, Microsoft tarafından geliştirilen nesne yönelimli bir programlama dili dir ve Microsoft'un Windows işletim sistemi kullanıcılarının kendi GUI uygulamaları nı geliştirebilecekleri bir araçtır. Aslında, Visual Basic bir dilden daha fazlasıdır—ta mamen önceden tanımlanmış bileşenlerden (butonlar, onay kutuları, metin kutula rı, kaydırma çubukları vb. gibi) uygulamalar inşa etmeyi sağlayan bir yazılım gelişti rme paketidir ve bu bileşenlerin çeşitli olaylara nasıl tepki vermesi gerektiğini tanı mlayarak özelleştirilmesine olanak tanır. Örneğin, bir buton durumunda, program cı o butona tıklandığında ne olacağını tanımlayacaktır. Bölüm 7, önceden tanımlan mış bileşenlerden yazılım inşa etme stratejisinin yazılım geliştirme tekniklerindeki mevcut eğilimi temsil ettiğini öğreneceğiz.

Windows işletim sisteminin popülaritesi ile Visual Basic geliştirme paketinin sağladığı kolaylık, Visual Basic'i yaygın olarak kullanılan bir programlama dili hal ine getirmiştir. Microsoft'un C#'ı tanıttığı göz önüne alındığında, bu önemin dev am edip etmeyeceği henüz belli değil.

Doclingo https://www.doclingo.ai

Farklı programlama dilleri, farklı parametre geçiş teknikleri sağlar, ancak tüm durumlarda parametrelerin kullanımı, bir prosedürün genel bir anlamda yazılmas ına ve uygun zamanda belirli verilere uygulanmasına olanak tanır.

Fonksiyonlar

Biraz farklı bir prosedür kavramını düşünmek için duralım ki bu, birçok progr amlama dilinde bulunur. Bazen bir prosedürün amacı bir eylem gerçekleştir mek yerine bir değer üretmektir. (Bir prosedürün amacının satılacak widget sayısını tahmin etmek ile basit bir oyunu oynamak arasındaki ince ayrımı düş ünün—ilkinde vurgu bir değer üretmekte, ikincisinde ise bir eylem gerçekleş tirmekte.) Amaç bir değer üretmekse, "prosedür" bir fonksiyon olarak uygula nabilir. Burada terim fonksiyon, bir değer çağıran program birimine "fonksiy onun değeri" olarak geri aktarılacak şekilde bir prosedüre benzer bir progra m birimini ifade eder. Yani, fonksiyonun yürütülmesi sonucunda bir değer h esaplanacak ve çağıran program birimine geri gönderilecektir. Bu değer dah a sonra bir değişkende saklanabilir veya hemen bir hesaplamada kullanılabil ir. Örneğin, bir C, C++, Java veya C# programcısı şunu yazabilir

```
ProjectedJanSales = EstimatedSales(January);
```

değişkeninin `ProjectedJanSales` değerinin, Ocak ayında satılması beklene n widget sayısını belirlemek için `EstimatedSales` fonksiyonunun uygulanm asının sonucu olarak atanmasını istemek için.

```
if (LastJanSales < EstimatedSales(January)) ...
   else ...
```

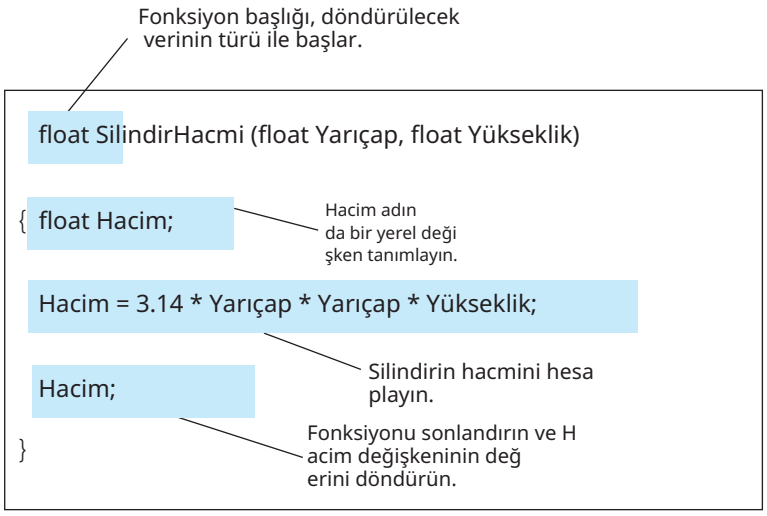
bu Ocak ayındaki satışların geçen Ocak ayındakilerden daha iyi olup olmayacağın a bağlı olarak farklı eylemlerin gerçekleştirilmesini sağlamak için. İkinci durumda, fonksiyon tarafından hesaplanan değer hangi dalın alınması gerektiğini belirleme k için kullanılır, ancak asla saklanmaz.

Olay Tabanlı Yazılım Sistemleri

Metinde, prosedürlerin programın başka yerlerinde açıkça prosedürü çağı ran ifadelerin sonucu olarak etkinleştirildiği durumları ele aldık. Ancak, pr osedürlerin bir olayın meydana gelmesiyle örtük olarak etkinleştirildiği du rumlar da vardır. Örnekler, bir düğmeye tıklandığında ne olacağını tanıml ayan prosedürün başka bir program biriminden bir çağrı ile değil, düğmey e tıklanması sonucu etkinleştirildiği GUI'lerde bulunur. Olaylar tarafından etkinleştirilen prosedürlerin bulunduğu yazılım sistemlerine olay tabanlı si stemler denir. Kısacası, bir olay tabanlı yazılım sistemi, çeşitli olayların son ucu olarak ne olacağını tanımlayan prosedürlerden oluşur. Sistem çalıştırıl dığında, bu prosedürler ilgili olay meydana gelene kadar uyku halindedir - sonra aktif hale gelir, görevlerini yerine getirir ve tekrar uyku haline döne r.

Doclingo https://www.doclingo.ai

Şekil **6.12** Programlama dili C'de yazılmış SilindirHacmi fonksiyonu



Fonksiyonlar, bir program içinde prosedürler ile aynı şekilde tanımlanır. Fark, bir fonksiyon başlığının genellikle döndürülecek değerin veri türünü belirterek başlaması ve fonksiyon tanımının genellikle döndürülecek değerin belirtildiği bir return ifadesi ile sona ermesidir.

Şekil 6.12, C dilinde yazılabileceği şekilde `CylinderVolume` adında bir fonksiyonun tanımını sunmaktadır. (Aslında, bir C programcısı daha öz bir form kullanır, ancak biz pedagojik nedenlerle bu biraz ayrıntılı versiyonu kullanacağız.) Çağrıldığında, fonksiyon `Radius` ve `Height` için resmi parametreler için belirli değerler alır ve bu boyutlara sahip bir silindirin hacmini hesaplamanın sonucunu döndürür. Böylece fonksiyon, programda aşağıdaki gibi bir ifadede başka yerlerde kullanılabilir.

```
Cost = CostPerVolUnit * CylinderVolume(3.45, 12.7);
```

3.45 yarıçapa ve 12.7 yüksekliğe sahip bir silindirin içeriğinin maliyetini belirlemek için.

Sorular ve Alıştırmalar

1. Bir değişkenin "kapsamı" ile ne kastedilmektedir?
2. Bir prosedür ile bir fonksiyon arasındaki fark nedir?
3. Pek çok programlama dili, I/O işlemlerini neden prosedür çağrılarıymış gibi uygular?
4. Resmi bir parametre ile gerçek bir parametre arasındaki fark nedir?
5. Modern programlama dillerinde yazarken, programcılar prosedürlerin adları için fiil ve fonksiyonların adları için isim kullanma eğilimindedir. Neden?

Doclingo https://www.doclingo.ai

6.4 Dil Uygulaması

Bu bölümde, yüksek seviyeli bir dilde yazılmış bir programın makine tarafından çalıştırılabilir bir forma dönüştürülmesi sürecini araştırıyoruz.

Çeviri Süreci

Bir programı bir dilden diğerine dönüştürme sürecine çeviri denir. Orijinal formundaki program kaynak programdır; çevrilen versiyon ise nesne programıdır. Çeviri süreci, çevirmen içinde leksikal analiz, ayrıştırma ve kod üretimi olarak bilinen birimlerin gerçekleştirdiği üç etkinlikten oluşur (Şekil 6.13).

Leksikal analiz, kaynak programdan hangi sembol dizilerinin tek bir varlığı veya token'ı temsil ettiğini tanıma sürecidir. Örneğin, üç sembol 153, bir 1, bir 5 ve bir 3 olarak yorumlanmamalıdır, bunun yerine tek bir sayısal değeri temsil ettiği tanınmalıdır. Benzer şekilde, programda yer alan bir kelime, bir eysel sembollerden oluşmasına rağmen, tek bir birim olarak yorumlanmalıdır. Çoğu insan, leksikal analizi çok az bilinçli çaba ile gerçekleştirir. Sesli okumaları istendiğinde, kelimeleri bireysel karakterler yerine telaffuz ederiz.

Bu nedenle, leksikal analizör kaynak programı sembol sembol okur, hangi sembol gruplarının token'ları temsil ettiğini belirler ve bu token'ları sayısal değerler, kelimeler, aritmetik operatörler vb. olup olmadığına göre sınıflandırır. Leksikal analizör her token'ı sınıflandırmasıyla kodlar ve bunları ayrıştırıcıya iletir. Bu süreçte, leksikal analizör tüm yorum ifadelerini atlar.

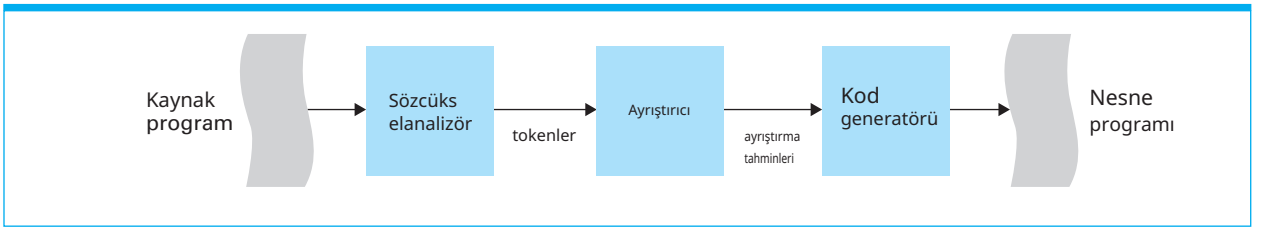
Böylece, ayrıştırıcı programı leksikal birimler (token'lar) açısından, bireysel semboller yerine görür. Ayrıştırıcının görevi bu birimleri ifadeler halinde gruplamaktır. Gerçekten de, ayrıştırma, programın dilbilgisel yapısını tanımlama ve her bileşenin rolünü tanıma sürecidir. Cümleyi okurken tereddüt etmenize neden olan ayrıştırmanın teknik detaylarıdır.

Yarışı kazanan atın fırlattığı adam yaralanmadı.

(Bunu deneyin: “O o değildir. O o değildir değilse o o değildir.”!)

Ayrıştırma sürecini basitleştirmek için, erken programlama dilleri her program ifadesinin basılı sayfada belirli bir şekilde konumlandırılmasını zorunlu kıldı. Böyle diller sabit format dilleri olarak biliniyordu. Bugün, çoğu programlama dili serbest format dilleri olup,

Şekil **6.13** Çeviri süreci



Doclingo https://www.doclingo.ai

Piton

Python, Guido van Rossum tarafından geç dönemlerde oluşturulmuş bir programlama dilidir. 1980s. Bugün, Web uygulamaları geliştirmede, bilimsel hesaplamalarda ve öğre nciler için bir tanıtım dili olarak popülerdir. Python, okunabilirliğe vurgu yapar v e zorunlu, nesne yönelimli ve fonksiyonel programlama paradigmalarının unsur larını içerir. Python, sabit biçimlendirme kullanan modern bir dil örneğidir. Prog ram bloklarını belirtmek için noktalama işaretleri veya ayrılmış kelimeler yerine girintiyi kullanır.

ifadelerin kritik olmaması. Serbest formatlı dillerin avantajı, bir programcı-'nın yaz ılı programı insanın bakış açısından okunabilirliği artıracak şekilde düzenleme yet eneğindedir. Bu durumlarda, bir okuyucunun bir ifadenin yapısını kavramasına ya rdımcı olmak için girinti kullanmak yaygındır. Yazmak yerine

```
eğerMaliyet < NakitElimde öyleysenakit ile ödeme aksikredi k
artı kullan
```

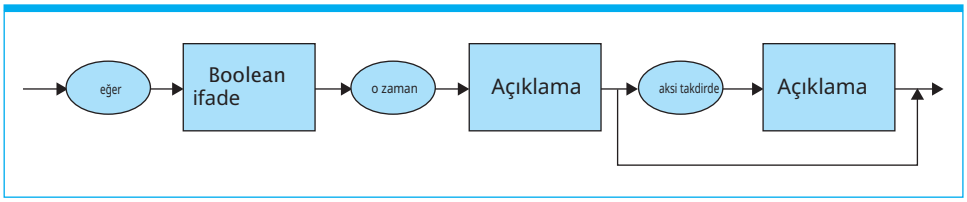
bir programcı şöyle yazabilir

```
eğerNakit < NakitMevcut
    o zamannakit ile ödeme yap
    aksi takdirdekredi kartı kullan
```

Bir makinenin serbest formatta yazılmış bir programı ayrıştırabilmesi için, dilin sözdizi miprogramın yapısının kaynak programda kullanılan boşluklardan bağımsız olarak tanımla nabilmesi için tasarlanmalıdır. Bu amaçla, çoğu serbest format dilleri, ifadelerin sonlarını iş aretlemek için noktalama işaretleri, örneğin noktalı virgüller kullanır ve ayrıca anahtar keli meler olarak eğer, o zaman ve aksi takdirde gibi ifadelerin başlangıcını işaretlemek için kull anılır. Bu anahtar kelimeler genellikle ayrılmış kelimeler olarak adlandırılır, bu da program cı tarafından program içinde başka amaçlar için kullanılamayacakları anlamına gelir.

Ayrıştırma süreci, programlama dilinin sözdizimini tanımlayan bir dizi kural a dayanır. Bu kurallar topluca dil bilgisi olarak adlandırılır. Bu kuralları ifade et menin bir yolu sözdizim diyagramları kullanmaktır; bu, bir dilin dil bilgisel yapıs ının resimsel temsilleridir. Şekil 6.14, Bölüm 5'teki sözde kodumuzdan eğer-o za man-aksi takdirde ifadesinin bir sözdizim diyagramını göstermektedir.
Bu diyagram, bir `if-then-else` yapısının
`if` kelimesiyle başladığını, ardından bir *Boolean ifadesi*, ardından `then` kelimesi, ardından a *İfade* geldiğini göstermektedir. Bu kombinasyon, `else`
ve bir *İfade* kelimesiyle takip edilebilir veya edilmeyebilir. Gerçekten bir `if-then-else`

Şekil **6.14** if-then-else sözde kod ifademizin bir sözdizim diyagramı



Doclingo https://www.doclingo.ai

ifadeler oval içinde yer alırken, daha fazla açıklama gerektiren terimler,örne ğin *Boolean ifadesi* ve *İfade*, dikdörtgenler içinde yer alır. Daha fazla açıkla ma gerektiren terimlere (dikdörtgenlerdeki olanlar) nonterminal denir; oval içinde yer alan terimlere ise terminal denir. Bir dilin sözdiziminin tam tanımı nda nonterminal terimler ek diyagramlarla açıklanır.

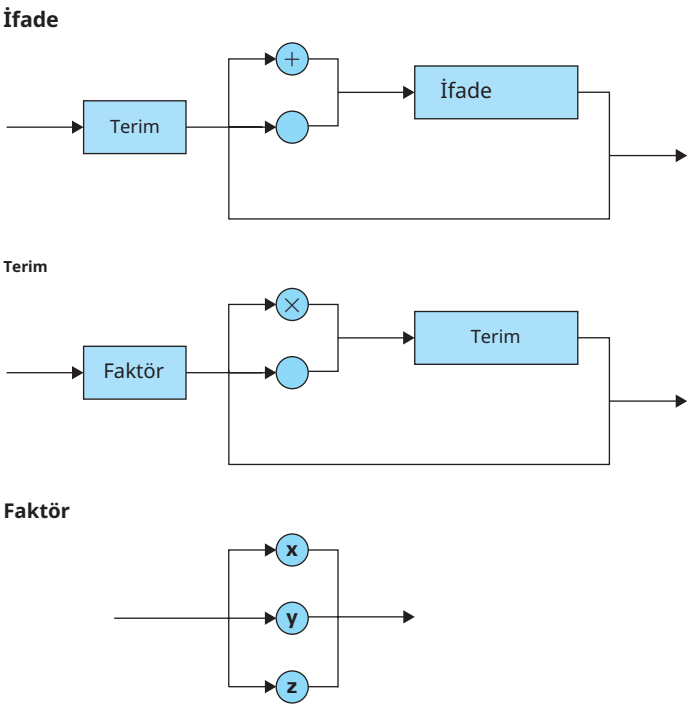
Daha kapsamlı bir örnek olarak, Şekil 6.15, *İfade* olarak adlandırılan bir yapın ın sözdizimini tanımlayan bir dizi sözdizim diyagramı sunmaktadır.Bu yapı, basit a ritmetik ifadelerin yapısı olarak tasarlanmıştır. İlk diyagram birini tanımlar. *İfade, ya bir Terim ile ya da bir + veya sembol ile takip edilebilecek başka bir İfa de içeren bir Terim olarak tanımlanır. İkinci diyagram, bir Terim'in ya tek bir Fak tör ya da bir Faktör ile takip edilen bir × veya sembol içeren bir Terim olarak tanı mlandığını açıklar. Son olarak, son diyagram bir Faktör'ü* x,y *veya* z *sembollerind en biri olarak tanımlar.*

Belirli bir dizenin bir dizi sözdizim diyagramına nasıl uyduğunu, bir ayrışt ırma ağacı ile resimsel bir biçimde temsil edilebilir; bu, dizenin ayrıştırma ağ acını sunan Şekil 6.16'da gösterilmiştir.

x + y × z

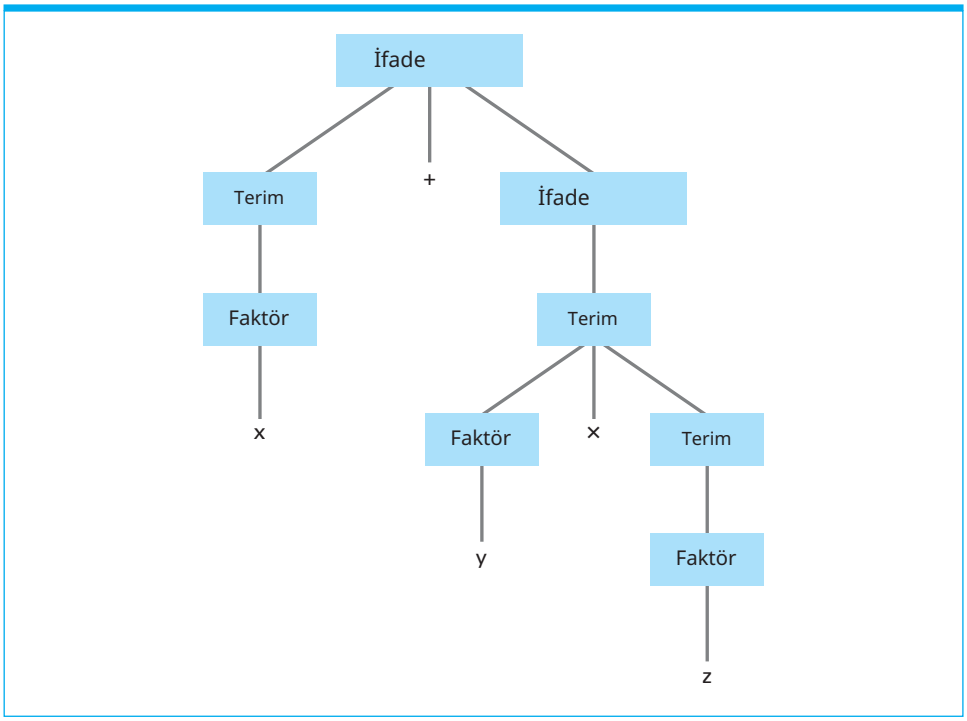
Şekil 6.15'teki diyagramlar setine dayanmaktadır. Ağacın en üstten başladığı nı unutmayınve her seviyede o seviyedeki nonterminal ifadelerin nasıl ayrıştı rıldığını gösterirve sembollerin kendisinin elde edildiği ana kadar devam ede r. Özellikle, şekil (Şekil 6.15'teki ilk diyagrama göre) bir *İfade*, bir *Terim* olarak ayrıştırılabilir, ardından + sembolü gelir, ardından

Şekil **6.15** Basit bir cebirsel ifadenin yapısını tanımlayan sözdizim diyagramları



Doclingo https://www.doclingo.ai

Şekil **6.16** x + y + z dizesi için ayrıştırma ağacı, Şekil 6.15'teki sözdizim diyagramlarına dayanarak



Bir ifadenin Terim olarak ayrıştırılması (Şekil 6.15'teki ikinci diyagramı kullanarak) Faktör (bu da x sembolü olarak ortaya çıkar) olarak yapılabilir ve nihai İfade (Şekil 6.15'teki üçüncü diyagramı kullanarak) Terim (bu da y × z olarak ortaya çıkar) olarak ayrıştırılabilir.

Java VE C# Uygulaması

Bazı durumlarda, animasyonlu bir Web sayfasının kontrolü gibi, yazılımın İnternet üzerinden aktarılması ve uzak makinelerde çalıştırılması gerekir. Bu yazılım kaynak program biçiminde sağlanıyorsa, varış noktasında ek gecikmeler meydana gelecektir çünkü yazılımın çalıştırılmadan önce uygun makine diline çevrilmesi gerekecektir. Ancak, yazılımın makine dili biçiminde sağlanması, uzak bilgisayarın kullandığı makine diline bağlı olarak yazılımın farklı bir versiyonunun sağlanması gerektiği anlamına gelecektir.

Sun Microsystems ve Microsoft, kaynak programların çevrilebileceği "evrensel makine dilleri" (Java için bytecode ve C# için .NET Ortak Ara Dil olarak adlandırılır) tasarlayarak bu sorunu çözmüştür. Bu diller gerçekten makine dilleri olmasa da, hızlı bir şekilde çevrilebilecek şekilde tasarlanmıştır. Böylece Java veya C# ile yazılmış bir yazılım uygun "evrensel makine diline" çevrildiğinde, bu yazılım diğer makinelerde verimli bir şekilde çalıştırılmak üzere İnternet'te aktarılabilir. Bazı durumlarda bu yürütme bir yorumlayıcı tarafından gerçekleştirilir. Diğer durumlarda, yürütmeden önce hızlı bir şekilde çevrilir; bu işleme ise anında derleme denir.

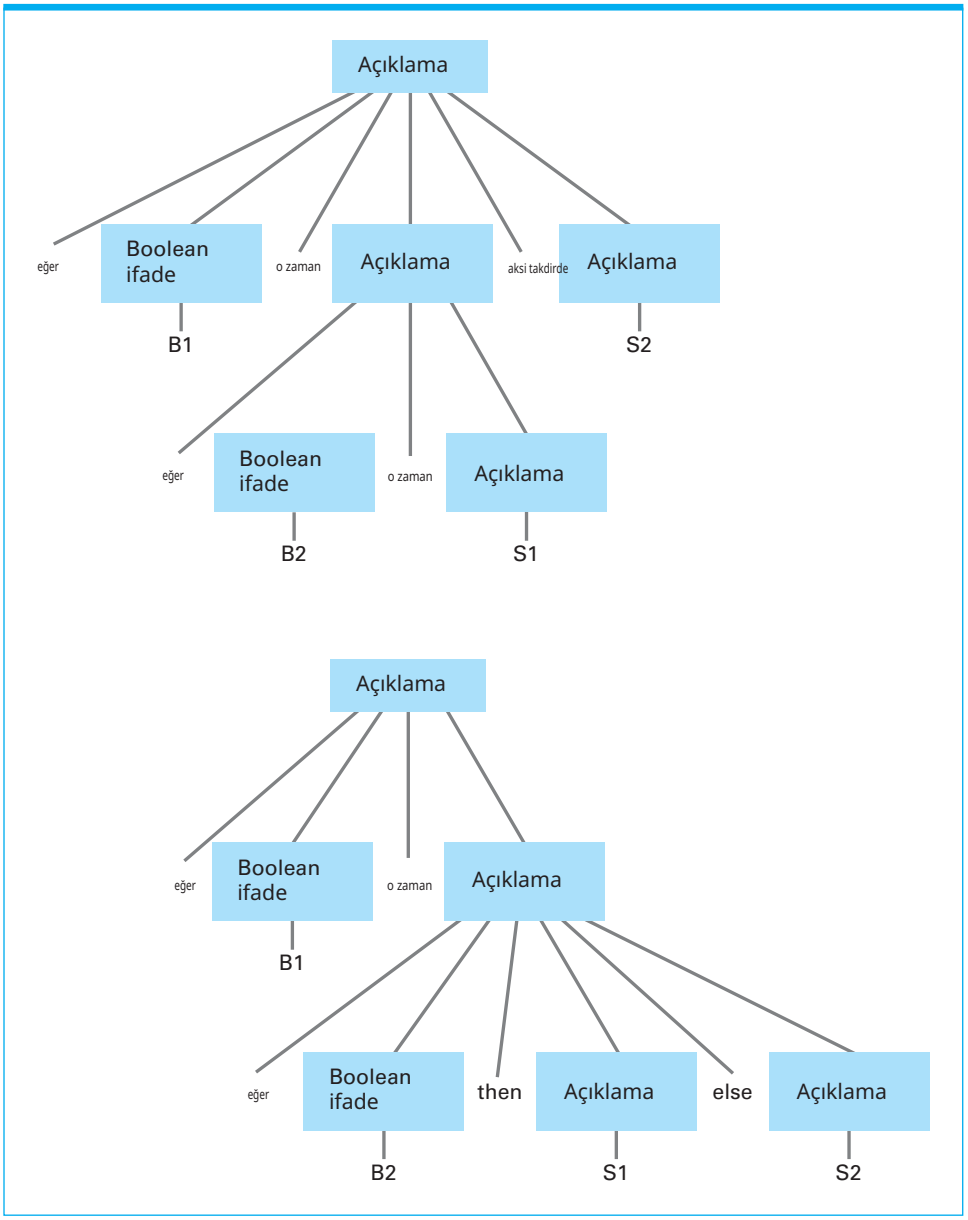
Doclingo https://www.doclingo.ai

Bir programı ayrıştırma süreci esasen kaynak program için bir ayrıştırma ağacı i nşa etme işlemidir. Gerçekten de, bir ayrıştırma ağacı, ayrıştırıcının programın dilbilg isel bileşimini yorumlamasını temsil eder. Bu nedenle, bir programın dilbilgisel yapıs ını tanımlayan sözdizimi kuralları, bir dize için iki farklı ayrıştırma ağacına izin verme melidir, çünkü bu durum ayrıştırıcı içinde belirsizliklere yol açar. Bir dize için iki farklı ayrıştırma ağacına izin veren bir dilbilgisi, belirsiz bir dilbilgisi olarak adlandırılır.

Gramerlerdeki belirsizlikler oldukça ince olabilir. Aslında, Şekil 6.14'teki kural böyle bir hat ayı içermektedir. Bu, tek bir ifade için Şekil 6.17'deki her iki ayrıştırma ağacına da izin verir.

eğer*B1***ise eğer***B2***ise***S1***değilse***S2*

Şekil **6.17** if B1 ise if B2 ise S1 değilse S2 için iki farklı ayrıştırma ağacı



Doclingo https://www.doclingo.ai

Bu yorumların önemli ölçüde farklı olduğunu unutmayın. İlk yorum, *B1* yanlışsa *S2* i fadesinin yürütüleceğini ima eder; ikincisi ise *B1* doğru ve *B2* yanlışsa yalnızca *S2* if adesinin yürütüleceğini ima eder.

Resmi programlama dillerinin sözdizimi tanımları, böyle belirsizlikleri önlemek iç in tasarlanmıştır. Sözdekodumuzda, bu tür sorunlardan kaçınmak için parantezler ku llanıyoruz. Özellikle, şöyle yazabiliriz:

```
eğerB1
   sonra ( EğerB2sonraS1 )
   aksiS2
```

ve

```
eğerB1
   sonra ( EğerB2sonraS1
                 aksiS2 )
```

iki olası yorum arasında ayırt etmek için.

Bir ayrıştırıcı bir programın dilbilgisel yapısını analiz ederken, bireysel ifa deleri tanımlayabilir ve bildirimsel ifadeler ile emirsel ifadeler arasında ayırt edebilir. Bildirimsel ifadeleri tanıdıkça, bildirilen bilgiyi sembol tablosu adı v erilen bir tabloda kaydeder. Böylece sembol tablosu, programda görünen de ğişkenlerin adları ile bu değişkenlerle ilişkili veri türleri ve veri yapıları gibi bil gileri içerir. Ayrıştırıcı daha sonra bu bilgiyi, aşağıdaki gibi emirsel ifadeleri a naliz ederken kullanır.

```
z ← x + y;
```

Özellikle, sembolün anlamını belirlemek için `+`, ayrıştırıcının `x` ve `y` ile ilişkili ver i türünü bilmesi gerekir. Eğer `x` float türündeyse ve `y` karakter türündeyse, o za man `x` ve `y` toplamak pek anlamlı değildir ve bir hata olarak bildirilmelidir. Eğer `x` ve `y` her ikisi de tam sayı türündeyse, o zaman ayrıştırıcı, kod üretecinin maki nenin tam sayı toplama op-kodunu kullanarak bir makine dili talimatı oluşturm asını isteyecektir; eğer her ikisi de float türündeyse, ayrıştırıcı, float toplama op -kodunun kullanılmasını isteyecektir; ya da eğer her ikisi de karakter türündeys e, ayrıştırıcı, birleştirme işlemini gerçekleştirmek için gereken makine dili talim atları dizisini oluşturmasını isteyebilir.

Özellikle, `x` tam sayı türündeyse ve `y` float türündeyse, biraz özel bir durum ortaya çıkar. O zaman toplama kavramı geçerlidir ancak değerler uyumlu biçimlerde kodlanmamıştır . Bu durumda ayrıştırıcı, kod üretecinin bir değeri diğer türüne dönüştürmek için talima tlar oluşturmasını ve ardından toplama işlemini gerçekleştirmesini seçebilir. Bu tür türl er arasında yapılan örtük dönüşüm coercion olarak adlandırılır.

Coercion, birçok dil tasarımcısı tarafından hoş karşılanmaz, çünkü örtük tür d önüşümü bir veri öğesinin değerini değiştirebilir ve ince program hatalarına yol a çabilir. Onlar, coercion ihtiyacının genellikle programın tasarımında bir kusuru gö sterdiğini ve bu nedenle ayrıştırıcı tarafından karşılanmaması gerektiğini savunurl ar. Sonuç olarak, modern dillerin çoğu strongly typed olup, bu da bir program tar afından talep edilen tüm faaliyetlerin uyumlu türlerde veri içermesi gerektiği anla mına gelir. Java gibi bazı diller, eğer bu bir type promotion ise, coercion'a izin veri r; bu da düşük hassasiyetli bir değerin daha yüksek hassasiyetli bir değere dönüşt ürülmesini içerir. Değeri değiştirebilecek örtük coercion'lar hata olarak bildirilir. Ç oğu durumda, bir programcı bu tür dönüşümleri açık bir type cast yaparak talep edebilir; bu, derleyiciye programcının bir tür dönüşümünün uygulanacağını bildiğ ini bildirir.

Doclingo https://www.doclingo.ai

Çeviri sürecindeki son etkinlik, kod üretimidir, bu, parsing tarafından tanınan ifadeleri uygulamak için makine dili talimatlarını oluşturma sürecidir. Bu süreç, programların verimli makine dili sürümlerinin üretilmesi gibi birçok sorunu içerir. Örneğin, iki ifadeden oluşan dizinin çevirisini düşünün.

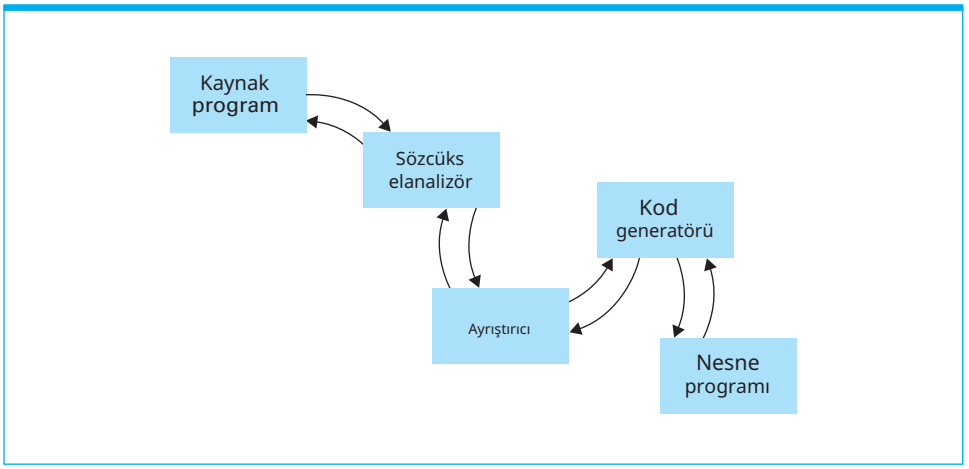
```
x←y + z;
w←x + z;
```

Eğer bu ifadeler bireysel ifadeler olarak çevrilirse, her biri belirtilen toplama işlemi gerçekleşmeden önce verilerin ana bellekten CPU'ya aktarılmasını gerektirecektir. Ancak, ilk ifadenin yürütüldükten sonra `x` ve`z` değerlerinin CPU'nun genel amaçlı kayıtlarında zaten bulunacağını kabul ederek verimlilik kazanılabilir ve bu nedenle ikinci toplama işlemi gerçekleştirilmeden önce bellekten yüklenmeleri gerekmez. Bu tür içgörülerin uygulanmasına kod optimizasyonu denir ve kod üreticisinin önemli bir görevidir.

Son olarak, sözdizimsel analiz, parsing ve kod üretimi adımlarının katı bir sıralı düzen içinde gerçekleştirilmediğini belirtmeliyiz. Bunun yerine, bu etkinlikler iç içe geçmiş durumdadır. Sözdizimsel analizci, kaynak programdan karakterleri okumaya başlayarak ilk token'ı tanımlar. Bu token'ı parser'a iletir. Her seferinde parser, sözdizimsel analizciden bir token aldığında, okunan dil bilgisel yapıyı analiz eder. Bu noktada, sözdizimsel analizciye başka bir token talep edebilir veya parser, tamamlanmış bir ifade veya cümlenin okunduğunu tanırsa, uygun makine talimatlarını üretmesi için kod üreticisini çağırır. Her böyle talep, kod üreticisinin nesne programına eklenen makine talimatlarını oluşturmasına neden olur.

Bir programı bir dilden diğerine çevirmek görevi, nesne yönelimli paradigma ile doğal olarak uyum sağlar. Kaynak program, sözcük analizi, ayrıştırıcı, kod üreteci ve nesne programı, her nesnenin görevini yerine getirirken mesajlar göndererek etkileşimde bulunduğu nesnelerdir (Şekil 6.18).

Şekil 6.18 Çeviri sürecine nesne yönelimli bir yaklaşım



Doclingo https://www.doclingo.ai

Yazılım Geliştirme Paketleri

Yazılım geliştirme sürecinde kullanılan editörler ve çeviriciler gibi yazılım ara çları, genellikle bir bütünleşik yazılım geliştirme sistemi olarak işlev gören bir pakette gruplanır. Böyle bir sistem, Bölüm 3.2'nin sınıflandırma şemasında uygulama yazılımı olarak sınıflandırılacaktır. Bu uygulama paketini kullanara k, bir programcı program yazmak için bir editöre, programları makine diline dönüştürmek için bir çeviriciye ve bir arızalı programın çalışmasını izleyerek nerede yanlış gittiğini keşfetmesine olanak tanıyan çeşitli hata ayıklama araç larına kolay erişim sağlar.

Böyle bir bütünleşik sistemi kullanmanın birçok avantajı vardır. Belki de e n belirgin olanı, bir programcının programda değişiklikler yapıldıkça ve test edildikçe editör ve hata ayıklama araçları arasında kolayca geçiş yapabilmesi dir. Dahası, birçok yazılım geliştirme paketi, geliştirilmekte olan ilgili progra m birimlerinin, ilgili birimlere erişimi basitleştirecek şekilde bağlanmasına ol anak tanır. Bazı paketler, bir grup ilgili birim içindeki hangi program birimler inin son benchmark'tan bu yana değiştirildiğine dair kayıtlar tutar. Bu tür yet enekler, birçok birbiriyle ilişkili birimin farklı programcılar tarafından geliştiril diği büyük yazılım sistemlerinin geliştirilmesinde oldukça avantajlıdır.

Küçük ölçekli olarak, yazılım geliştirme paketlerindeki editörler genellikle kullanılan programlama diline göre özelleştirilmiştir. Böyle bir editör genellikle hedef dil için de facto standart olan otomatik satır girintisi sağlar ve bazı durumlarda programcı yalnızca ilk birkaç karakteri yazdıktan sonra anahtar kelimeleri otomatik olarak tanıyıp tamamlayabilir. Dahası, editör kaynak programlar içindeki anahtar kelimeleri (belki de renk ile) vurgulayarak öne çıkarabilir, bu da programların okunmasını kolaylaştırır.

Bir sonraki bölümde, yazılım geliştiricilerin yeni yazılım sistemlerinin önceden üretilmiş bloklardan, bileşenler olarak adlandırılan, nasıl inşa edilebileceğini araştırdıklarını öğreneceğiz—bu da bileşen mimarisi olarak adlandırılan yeni bir yazılım geliştirme modeline yol açmaktadır. Bileşen mimarisi modeline dayanan yazılım geliştirme paketleri genellikle bileşenlerin ekranda simgeler olarak temsil edilebileceği grafiksel arayüzler kullanır. Bu ortamda bir programcı (veya bileşen montajcısı) istenen bileşenleri fare ile seçer. Seçilen bir bileşen, paketin editörü aracılığıyla özelleştirilebilir ve ardından fare ile tıklayarak başka bileşenlere eklenebilir. Böyle paketler, daha iyi yazılım geliştirme araçları arama sürecinde önemli bir adımı temsil eder.

Sorular ve Alıştırmalar

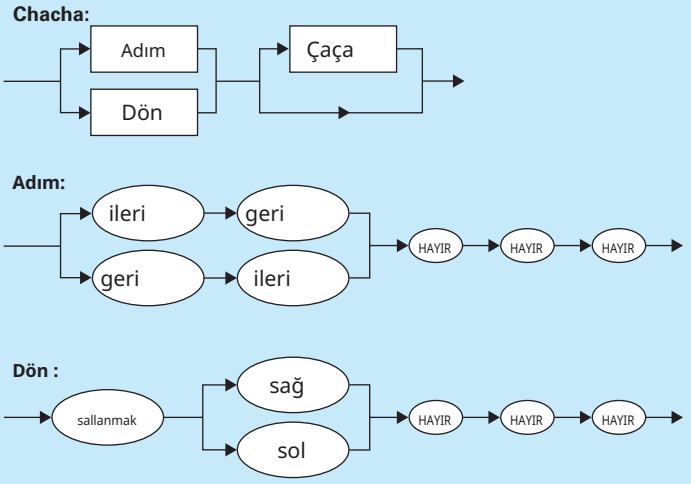
1. Çeviri sürecindeki üç ana adımı tanımlayın.
2. Sembol tablosu nedir?
3. Terminal ile nonterminal arasındaki fark nedir?
4. İfadenin ayrıştırma ağacını çizin

$$x \times y + x + z$$

Şekil 6.15'teki sözdizim diyagramlarına dayanarak.

Doclingo https://www.doclingo.ai

5. Aşağıdaki sözdizim diyagramlarına göre Chacha yapısına uyan dizeleri tanımlayın.



6.5 Nesne Yönelimli Programlama

Bölüm 6.1'de, nesne yönelimli paradigmanın, her biri çeşitli uyarıcılara nasıl yanıt vermesi gerektiğini tanımlayan prosedürler içeren aktif program birimleri olan nesnelerin geliştirilmesini içerdiğini öğrendik. Bir probleme nesne yönelimli yaklaşım, ilgili nesneleri tanımlamak ve bunları kendine yeterli birim olarak tanımlamaktır. Bunun sonucunda, nesne yönelimli programlama dilleri, nesneleri ve davranışlarını tanımlamak için ifadeler sağlar. Bu bölümde, C++, Java ve C# dillerinde görülen bu ifadelerden bazılarını tanıtacağız; bunlar günümüzde kullanılan daha belirgin nesne yönelimli dillerden üçüdür.

Sınıflar ve Nesneler

Oyuncunun, yüksek güçlü lazerlerle düşen meteorları vurarak Dünya'yı korumasi gereken basit bir bilgisayar oyunu geliştirme görevini düşünün. Her lazer, her ateşlendiğinde kısmen tüketilen sonlu bir iç güç kaynağı içerir. Bu kaynak tükendiğinde, lazer işe yaramaz hale gelir. Her lazer, daha sağa nişan alma, daha sola nişan alma ve lazer ışınını ateşleme komutlarına yanıt verebilmelidir.

Nesne yönelimli paradigmada, bilgisayar oyunundaki her lazer, kalan gücünün kaydını ve nişan alma ile lazer ışınını ateşleme prosedürlerini içeren bir nesne olarak uygulanır. Tüm lazer nesneleri aynı özelliklere sahip olduğundan, bunlar ortak bir şablon aracılığıyla tanımlanabilir. Nesne yönelimli paradigmada, bir nesne koleksiyonu için bir şablona sınıf denir.

Bölüm 8'de, sınıflar ve veri türleri arasındaki benzerlikleri keşfedeceğiz. Şimdilik, bir sınıfın, 1, 5 ve 82 gibi sayıların ortak özelliklerini kapsayan ilkel veri türü tam sayısı kavramı ile aynı şekilde, bir nesne koleksiyonunun ortak özelliklerini tanımladığını sadece not ediyoruz. Bir programcı, bir programda bir sınıfın tanımını dahil ettiğinde, o şablon, ilkel tür tam sayısının tam sayı türündeki "nesneleri" manipüle etmesine izin verdiği şekilde, o "türden" nesneleri oluşturmak ve manipüle etmek için kullanılabilir.

Doclingo https://www.doclingo.ai

C++, Java ve C# dillerinde bir sınıf, şu biçimde bir ifade ile tanımlanır

```
class Name
{
     .
     .
     .
}
```

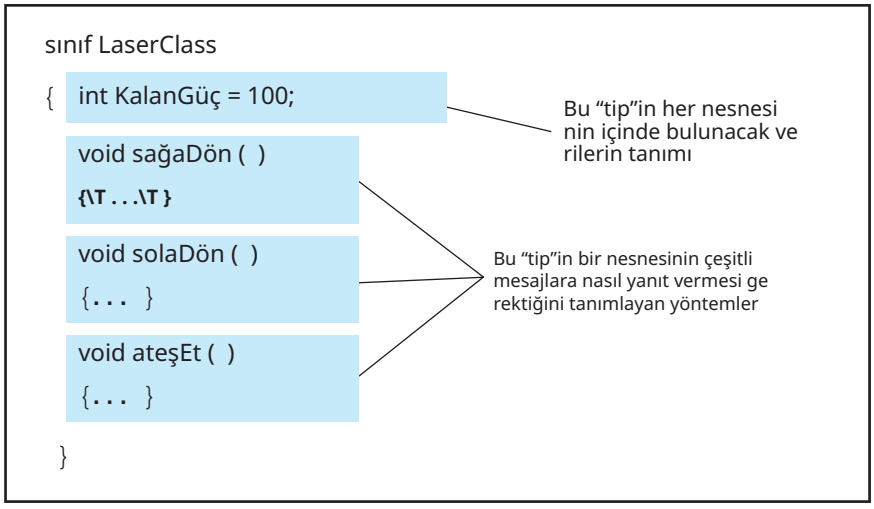
burada İsim, sınıfın programın başka yerlerinde referans alınabileceği isimd ir. Sınıfın özelliklerinin tanımlandığı yer süslü parantezler içindedir. Özellikle, bilgisayar oyunumuzda bir lazerin yapısını tanımlayan `LaserClass` adlı bir sınıf Şekil 6.19'da özetlenmiştir. Sınıf, `RemainingPower` (tamsayı tür ünde) adlı bir değişkenin beyanı ve `turnRight`,`turnLeft` ve `fire` adlı üç pr osedürden oluşmaktadır. Bu prosedürler, ilgili eylemi gerçekleştirmek için ya pılması gereken rutinleri tanımlar. Böylece bu şablondan oluşturulan herhan gi bir nesne, `RemainingPower` adlı bir değişken ve `turnRight`,`turnLeft` ve `fire` adlı üç prosedüre sahip olacaktır.

Bir nesne içinde bulunan bir değişken, örneğin `RemainingPower`, örnek değişkeni olarak adlandırılır ve bir nesne içindeki prosedürler yöntemler (ve ya C++ terimlerinde üye fonksiyonları) olarak adlandırılır. Şekil 6.19'da `RemainingPower` örnek değişkeninin, Bölüm 6.2'de tartışılanlara benzer bir beyan ifadesi kullanılarak tanımlandığını ve yöntemlerin, Bölüm 6.3'te tartışıl an prosedürler ve fonksiyonlara benzer bir biçimde tanımlandığını unutmayı n. Sonuçta, örnek değişkenlerinin beyanları ve yöntemlerin tanımları temeld e zorunlu programlama kavramlarıdır.

Oyun programımızda `LaserClass` sınıfını tanımladıktan sonra, "tip"Lase rClass olan üç değişken Laser1, Laser2 ve Laser3 bildirebiliriz.

```
Lazer Sınıfı Lazer 1, Lazer 2, Lazer 3;
```

Şekil **6.19** Bilgisayar oyununda bir lazer silahını tanımlayan bir sınıfın yapısı



Doclingo https://www.doclingo.ai

Bu, üç değişkeni tanımlamak için kullanılacak aynı formattır

```
int x, y, z;
```

türünde x, y ve z adında üç değişkeni tanımlamak için kullanılacak,Bölüm 6. 2'de öğrendiğimiz gibi. Her ikisi de bir "tip"in adını ve ardındanbelirtilen değişkenlerin listesini içerir. Fark, son ifadenin x, y ve z değişkenlerininprogram da tamsayı türündeki (bu bir ilkel türdür) öğelere atıfta bulunmak için kullanılacağınısöylemesidir; oysa önceki ifade Laser1,Laser2 ve Laser3 değişkenlerininprogramda "tip" LaserClass (bu program içinde tanımlanmış bir "tip" dir) öğelerineatıfta bulunmak için kullanılacağını belirtmektedir.

Değişkenleri Laser1,Laser2 ve Laser3 "tip"LaserClass olarak tanımladıktan sonra, onlara değer atayabiliriz. Bu durumda değerler, "tip" LaserClass ile uyumlu nesneler olmalıdır. Bu atamalar, atama ifadeleriyle yapılabilir, ancak genellikle değişkenleri tanımlamak için kullanılan aynı tanım ifadeleri içinde başlangıç değerleri atamak daha uygundur. C++ dilinde tanımlamalar durumunda bu tür başlangıç atamaları otomatik olarak yapılır. Yani, ifade

```
LaserClass Laser1, Laser2, Laser3;
```

hem Laser1,Laser2 ve Laser3 değişkenlerini oluşturur, hem de her bir değişkenin değeri olarak "tip"LaserClass olan üç nesne yaratır. Java ve C# dillerinde, bu tür başlangıç atamaları, ilkel türlerin değişkenlerine yapılan başlangıç atamalarıyla çok benzer bir şekilde başlatılır. Özellikle, ifade

```
int x = 3;
```

hem x değişkenini tam sayı türünde bir değişken olarak tanımlar, hem de değişkene 3 değerini atar, ifade

```
LaserClass Laser1 = new LaserClass();
```

Laser1 değişkenini "tip"LaserClass olarak tanımlar ve ayrıca LaserClass şablonunu kullanarak yeni bir nesne oluşturur ve bu nesneyi Laser1 için başlangıç değeri olarak atar.

Bu noktada, bir sınıf ile bir nesne arasındaki ayrımı vurgulamak için durmalıyız. Bir sınıf, nesnelerin inşa edildiği bir şablondur. Bir sınıf, çok sayıda nesne oluşturmak için kullanılabilir. Bir nesneye, oluşturulduğu sınıfın bir örneği olarak sıkça atıfta bulunuruz. Böylece, bilgisayar oyunumuzda Laser1, Laser2, veLaser3, LaserClass sınıfının örnekleri olan değişkenlerdir.

Değişkenleri Laser1,Laser2, veLaser3 oluşturmak için deklaratif ifadeler kullandıktan sonra, bu nesneler içindeki uygun yöntemleri etkinleştiren zorlayıcı ifadeler yazarak oyun programımıza devam edebiliriz (nesne yönelimli terimlerle, bu nesnelere mesaj göndermek olarak adlandırılır). Özellikle, Laser1 değişkenine atanan nesnenin fire yöntemini çalıştırmasını sağlayabiliriz.

```
Laser1.fire();
```

Ya da Laser2'ye atanan nesnenin turnLeft yöntemini çalıştırmasını sağlayabiliriz.

```
Laser2.turnLeft();
```

Doclingo https://www.doclingo.ai

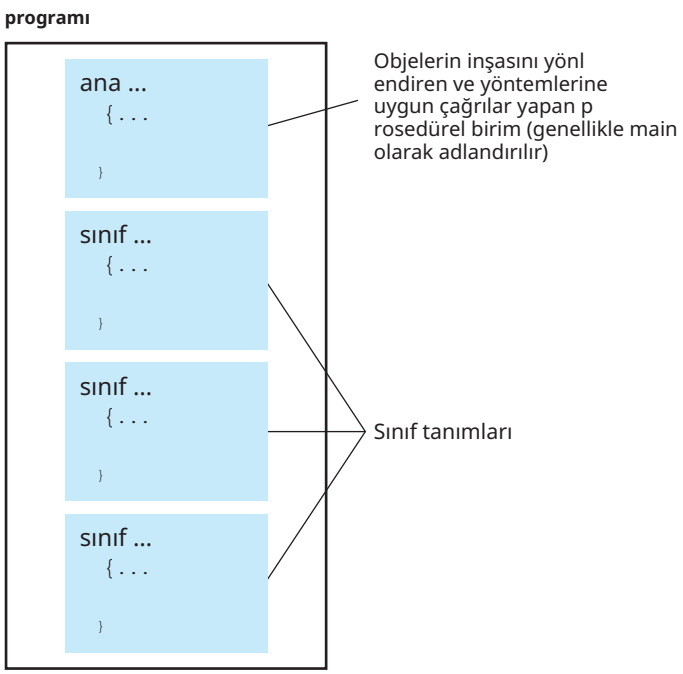
Bunlar aslında prosedür çağrılarından başka bir şey değildir. Gerçekten de, ö nceki ifade, `Laser1` değişkenine atanan nesne içindeki `fire` prosedürüne (y öntemine) bir çağrıdır ve sonuncu ifade, `Laser2` değişkenine atanan nesne i çindeki `turnLeft` prosedürüne bir çağrıdır.

Bu aşamada meteor oyunu örneğimiz, tipik bir nesne yönelimli programı n genel yapısını kavramamız için arka planı sağlamıştır (Şekil 6.20). Program, Şekil 6.19'a benzer çeşitli sınıf tanımları içerecektir; her biri programda kulla nılan bir veya daha fazla nesnenin yapısını tanımlamaktadır. Ayrıca, program , program çalıştırıldığında başlangıçta gerçekleştirilmesi gereken adımların s ırasını içeren bir zorunlu program segmenti (genellikle "main" adıyla ilişkilen dirilir) içerecektir. Bu segment, programda kullanılan nesneleri oluşturmak i çin lazer tanımlarımıza benzer bildirim ifadeleri ile bu nesneler içindeki yönte mlerin yürütülmesini talep eden zorunlu ifadeleri içerecektir.

Yapıcılar

Bir nesne oluşturulduğunda, genellikle bazı özelleştirme faaliyetlerinin gerçe kleştirilmesi gerekir. Örneğin, meteor bilgisayar oyunumuzda farklı lazerleri n farklı başlangıç güç ayarlarına sahip olmasını isteyebiliriz; bu da çeşitli nes nelerdeki KalanGüç adlı örnek değişkenlerin farklı başlangıç değerleri almas ı gerektiği anlamına gelir. Bu tür başlatma ihtiyaçları

Şekil **6.20** Tipik bir nesne yönelimli programın yapısı



Doclingo https://www.doclingo.ai

uygun sınıf içinde yapıcılar olarak adlandırılan özel yöntemleri tanımlama. Yapıcılar, bir nesne sınıftan oluşturulduğunda otomatik olarak çalıştırılır. Bir yapıcı, sınıf tanımında sınıfla aynı adı taşıyan bir yöntem olmasıyla tanımlanır .

Şekil 6.21, Şekil 6.19'da başlangıçta gösterilen `LaserClass` tanımının bir uzantısını sunmaktadır. Bu yöntemin `LaserClass` adında bir yöntem biçimi nde bir yapıcı içerdiğini unutmayın. Bu yöntem, örnek değişkeni Remaining-Power'ı parametre olarak aldığı değere atar. Böylece, bu sınıftan bir nesne ol uşturulduğunda, bu yöntem çalıştırılacak ve `RemainingPower` uygun ayarda başlatılacaktır.

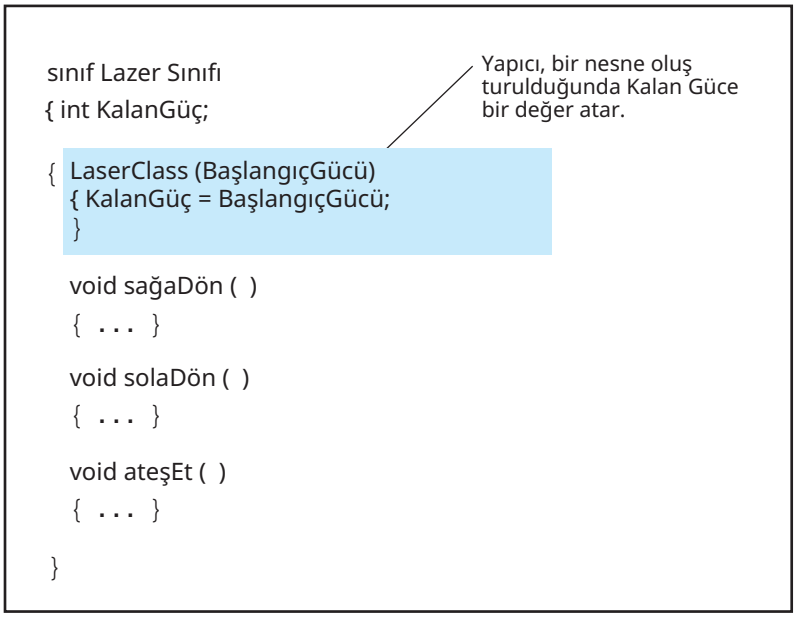
Bir yapıcı tarafından kullanılacak gerçek parametreler, nesnenin oluşturu lmasına neden olan ifadede bir parametre listesinde tanımlanır. Böylece, Şek il 6.21'deki sınıf tanımına dayanarak, bir C++ programcısı

```
LaserClass Laser1(50), Laser2(100);
```

tipi `LaserClass` olan iki nesne oluşturmak için - biri `Laser1` olarak bilinen ve başlangıç ta 50 güç rezervine sahip olan, diğeri ise `Laser2` olarak bilinen ve başlangıçta 100 güç r ezervine sahip olan. Java ve C# programcıları aynı görevi aşağıdaki ifadelerle gerçekleşti receklerdir.

```
Lazer Sınıfı Lazer1 = yeni Lazer Sınıfı(50);
Lazer Sınıfı Lazer2 = yeni Lazer Sınıfı (100);
```

Şekil **6.21** Bir yapıcıya sahip bir sınıf



Doclingo https://www.doclingo.ai

Ek Özellikler

Şimdi, meteor bilgisayar oyunumuzu geliştirmek istediğimizi varsayalım, böylece belirli bir skora ulaşan bir oyuncu, lazerlerin bazılarını orijinal güç ayarına yeniden şarj ederek ödüllendirilecektir. Bu lazerler, diğer lazerlerle aynı özelliklere sahip olacak, tek farkları yeniden şarj edilebilir olmalarıdır.

Benzer ancak farklı özelliklere sahip nesnelerin tanımını basitleştirmek için, nesne yönelimli diller, bir sınıfın başka bir sınıfın özelliklerini kapsamasına olanak tanıyan bir teknik olan kalıtım kullanır. Örneğin, oyunumuzu geliştirmek için Java kullandığımızı varsayalım. Öncelikle, programdaki tüm lazerler için ortak olan özellikleri tanımlayan `LaserClass` adında bir sınıf tanımlamak için daha önce tanımlanan sınıf ifadesini kullanabiliriz. Ardından, bu ifadeyi kullanabiliriz

```
sınıf RechargeableLaser, LaserClass'ı genişletir
{
        .
        .
        .
}
```

başka bir sınıfı tanımlamak için `RechargeableLaser`. (C++ ve C# programcıları kelimeyi `extends` ile bir iki nokta üst üste ile değiştireceklerdir.) Burada extendsclauses, bu sınıfın Laser-Class sınıfının özelliklerini miras alacağını ve ayrıca süslü parantezler içinde yer alan özellikleri içereceğini belirtir. Bizim durumumuzda, bu süslü parantezler, örneğin `recharge` adında yeni bir yöntemi içerecek ve bu yöntem, örnek değişken Remaining Power'ı orijinal değerine sıfırlamak için gereken adımları tanımlayacaktır. Bu sınıflar tanımlandıktan sonra, ifadeyi kullanabiliriz

```
Lazer Sınıfı Lazer 1, Lazer 2;
```

Laser1 ve Laser2'yi geleneksel lazerlere atıfta bulunan değişkenler olarak tanımlamak için bu ifadeyi kullanın.

```
Şarj Edilebilir Lazer Lazer 3, Lazer 4;
```

`Laser3` ve`Laser4`'ü `RechargeableLaser` sınıfında tanımlanan ek özelliklere sahip lazerlere atıfta bulunan değişkenler olarak tanımlamak için kullanın.

Kalıtımın kullanımı, benzer ancak farklı özelliklere sahip çeşitli nesnelerin varlığına yol açar; bu da, Bölüm 6.2'de karşılaştığımız aşırı yüklenmeyi andıran bir olguya yol açar. (Aşırı yüklenmenin, + gibi tek bir sembolün, operandlarının türüne bağlı olarak farklı işlemleri temsil etmek için kullanılması anlamına geldiğini hatırlayın.) Nesne yönelimli bir grafik paketinin, her biri bir şekli (daire, dikdörtgen, üçgen vb.) temsil eden çeşitli nesnelerden oluştuğunu varsayalım. Belirli bir görüntü, bu nesnelerin bir koleksiyonunu içerebilir. Her nesne, boyutunu, konumunu ve rengini "bilir" ve örneğin, yeni bir konuma taşınması gerektiğini bildiren mesajlara nasıl yanıt vereceğini de bilir.

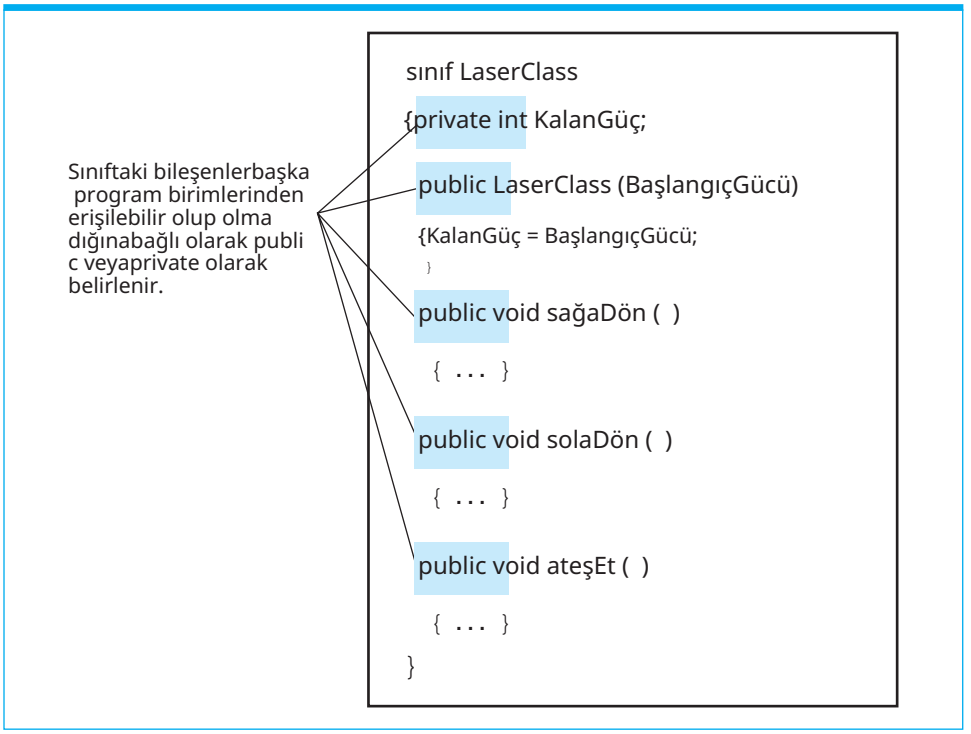
Doclingo https://www.doclingo.ai

Bir görüntüyü çizmek için, görüntüdeki her nesneye “kendini çiz” mesajı gön deririz. Ancak, bir nesneyi çizmek için kullanılan rutin, nesnenin şekline göre değişir—bir kare çizmek, bir daire çizmeyle aynı süreç değildir. Bu mesajın ö zelleştirilmiş yorumu polymorphism olarak bilinir; mesajın polymorfik olduğ u söylenir.

Nesne yönelimli programlamayla ilişkili bir diğer özellik, bir nesnenin iç ö zelliklerine erişimi kısıtlama anlamına gelen kapsüllemektir. Bir nesnenin bel irli özelliklerinin kapsüllendiğini söylemek, yalnızca nesnenin kendisinin bu ö zelliklere erişebileceği anlamına gelir. Kapsüllenen özellikler özel olarak adla ndırılır. Nesneden dışarıdan erişilebilen özellikler ise genel olarak adlandırılır .

Örnek olarak, Şekil 6.19'da ilk olarak tanımlanan LaserClass'ımıza geri dö nelim. Bunun, RemainingPower adında bir örnek değişkeni ve turnRight, tur nLeft ve fire adında üç yöntemi tanımladığını hatırlayın. Bu yöntemler, Laser Class'ın bir örneğinin uygun eylemi gerçekleştirmesi için diğer program biri mleri tarafından erişilecektir. Ancak RemainingPower'ın değeri yalnızca örne ğin iç yöntemleri tarafından değiştirilmelidir. Diğer hiçbir program birimi bu değere doğrudan erişebilmelidir. Bu kuralları uygulamak için RemainingPow er'ı özel ve turnRight, turnLeft ve fire'ı genel olarak tanımlamak yeterlidir; bu , Şekil 6.22'de gösterildiği gibi. Bu tanımlamalar eklendiğinde, bulunduğu ne sneden dışarıdan RemainingPower'ın değerine erişim girişimi, program çevr ildiğinde bir hata olarak tanımlanacak ve programcının ilerlemeden önce sor unu düzeltmesini zorunlu kılacaktır.

Şekil **6.22** Kapsülleme kullanarak LaserClass tanımımızın Java veya C#programında nasıl görün eceği



Doclingo https://www.doclingo.ai

Sorular ve Alıştırmalar

1. Bir nesne ile bir sınıf arasındaki fark nedir?
2. Bu bölümde kullanılan bilgisayar oyunu örneğinde başka hangi nesne sınıfları Lazer Sınıfı bulunabilir? `RemainingPower` dışında `LaserClass` sınıfında hangi örnek değişke nler bulunabilir?
3. PartTimeEmployee ve FullTimeEmployee sınıflarının Employee sı nıfının özelliklerini miras aldığını varsayalım. Her sınıfta bulmayı bekl eyebileceğiniz bazı özellikler nelerdir?
4. Bir yapıcı nedir?
5. Bir sınıf içindeki bazı öğelerin neden özel olarak belirlendiği?

6.6 Eşzamanlı Aktiviteleri Programlama

Diyelim ki, birden fazla saldıran düşman uzay gemisini içeren bir aksiyon bilgisayar oyunu için animasyon üretecek bir program tasarlamamız istendi. Bir yaklaşım, tüm animasyon ekranını kontrol edecek tek bir program tasarlamak olabilir. Böyle bir program, her bir uzay gemisini çizmekle yükümlü olacaktır, ki (animasyon gerçekçi görünmesi gerekiyorsa) bu, programın çok sayıda uzay aracının bireysel özellikleriyle uyumlu kalması gerektiği anlamına gelir. Alternatif bir yaklaşım, programın yürütülmesinin başında atanan parametrelerle belirlenen bir tek uzay gemisinin animasyonunu kontrol edecek bir program tasarlamak olabilir. Ardından, bu programın her biri kendi parametre setine sahip birden fazla aktivasyonu oluşturarak animasyon inşa edilebilir. Bu aktivasyonları eşzamanlı olarak yürütmekle, ekranda aynı anda birçok bireysel uzay gemisinin hızla geçişi illüzyonunu elde edebiliriz.

Birden fazla aktivasyonun eşzamanlı yürütülmesi paralelişlem veya eşza manlı işlem olarak adlandırılır. Gerçek paralel işlem, her aktivasyonu yürütm ek için birden fazla CPU gerektirir. Sadece bir CPU mevcut olduğunda, parale l işlem yanılsaması, aktivasyonların tek işlemcinin zamanını paylaşmasına izi n verilerek elde edilir; bu, çoklu programlama sistemleri tarafından uygulana n bir yaklaşıma benzer (Bölüm 3).

Birçok modern bilgisayar uygulaması, tek bir talimat dizisi içeren daha gele neksel bir bağlamda çözülmekten daha kolay bir şekilde paralel işlem bağlamın da çözülmektedir. Bunun sonucunda, daha yeni programlama dilleri, paralel he saplamalarda yer alan anlamsal yapıları ifade etmek için sözdizimi sağlar. Böyle bir dilin tasarımı, bu anlamsal yapıların tanımlanmasını ve bunları temsil etme k için bir sözdizimi geliştirilmesini gerektirir.

Her programlama dili, paralel işlem paradigmasına kendi bakış açısıyla yaklaş ma eğilimindedir ve bu da farklı terminolojilere yol açar. Örneğin, gayri resmi olar ak aktivasyon olarak adlandırdığımız şey, Ada dilinde bir görev ve Java'da bir iplik olarak adlandırılır. Yani, bir Ada programında, eşzamanlı eylemler birden fazla gö rev oluşturarak gerçekleştirilirken, Java'da birden fazla iplik oluşturulur.
Her iki durumda da, sonuç, çoklu görevli bir işletim sistemi kontrolü altında s üreçler gibi oluşturulan ve yürütülen çoklu etkinliklerdir. Java terminolojisini benimseyeceğiz ve bu tür "süreçleri" iş parçacıkları olarak adlandıracağız.

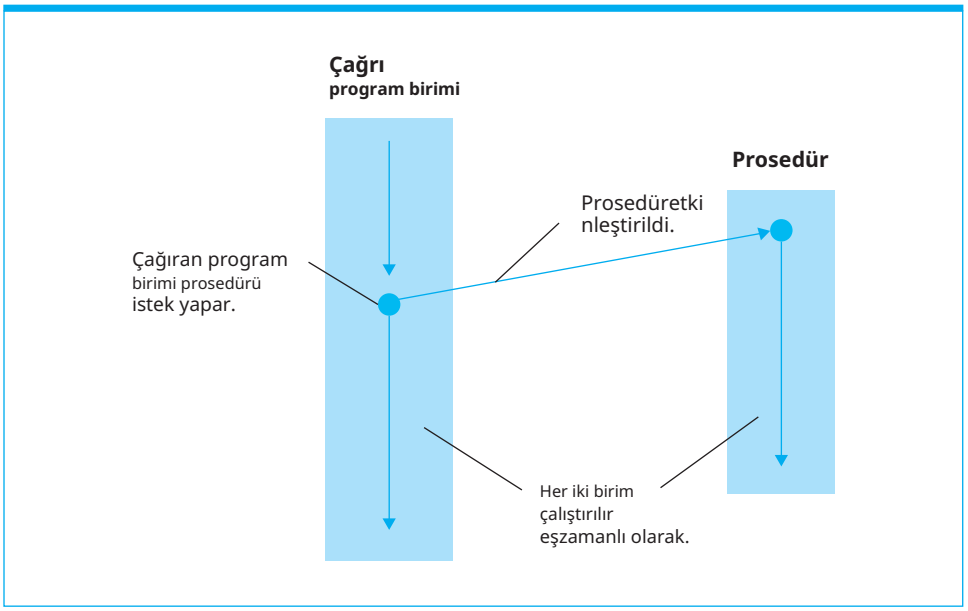
Doclingo https://www.doclingo.ai

Paralel işleme dahil olan bir programda ifade edilmesi gereken en temel eylemlerden biri yeni iş parçacıkları oluşturmaktır. Uzay gemisi programının birden fazla etkinleştirilmesinin aynı anda yürütülmesini istiyorsak, bunu beli rtmek için bir sözdizimine ihtiyacımız var. Yeni iş parçacıklarının oluşturulma sı, geleneksel bir prosedürün yürütülmesini istemeye benzer. Fark, geleneks el ortamda, bir prosedürün etkinleştirilmesini talep eden program birimi, tal ep edilen prosedür sona erene kadar daha fazla ilerlemez (Şekil 6.8'i hatırlayı n), oysa paralel bağlamda talep eden program birimi, talep edilen prosedür görevini yerine getirirken yürütmeye devam eder (Şekil 6.23). Bu nedenle, ek randa hızla giden birden fazla uzay gemisi oluşturmak için, uzay gemisinin a yırt edici özelliklerini tanımlayan parametrelerle sağlanan, uzay gemisi progr amının birden fazla etkinleştirmesini basitçe üreten bir ana program yazardı k.

Daha karmaşık bir paralel işleme ile ilgili sorun, iş parçacıkları arasındaki iletişimi yönetmektir. Örneğin, uzay gemisi örneğimizde, farklı uzay gemilerini temsil eden iş parçacıkları, faaliyetlerini koordine etmek için birbirleriyle konumlarını iletişime geçmeleri gerekebilir. Diğer durumlarda, bir iş parçacığı, diğerinin belirli bir hesaplama noktasına ulaşmasını beklemek zorunda kalabilir veya bir iş parçacığı, ilk iş parçacığı belirli bir görevi tamamlayana kadar diğerini durdurmak zorunda kalabilir.

Bu tür iletişim ihtiyaçları uzun zamandır bilgisayar bilimcileri arasında birçalış ma konusu olmuştur ve birçok yeni programlama dili, iş parçacığı etkileşimsorunl arına çeşitli yaklaşımları yansıtmaktadır. Örneğin, iki iş parçacığı aynıveri üzerinde işlem yaparken karşılaşılan iletişim sorunlarını ele alalım. (Buörnek, isteğe bağlı B ölüm 3.4'te daha ayrıntılı olarak sunulmaktadır.) Eğereşzamanlı olarak çalışan iki i ş parçacığının her biri, ortak bir veri öğesine üçdeğerini eklemek zorundaysa, bir i ş parçacığının işlemini tamamlamasına izinverilmeden diğerinin görevini yerine g etirmesini sağlamak için bir yöntemgereklidir. Aksi takdirde, her ikisi de aynı başl angıç değeriyle bireyselhesaplamalarına başlayabilir ki bu da anlamına gelir ki

Şekil **6.23** İş parçacıkları oluşturma



Doclingo https://www.doclingo.ai

Akıllı Telefon Programlama

Taşınabilir, mobil ve gömülü cihazlar için yazılım genellikle diğer bağlamlarda kullanılan aynı genel amaçlı programlama dilleri kullanılarak geliştirilir. Daha büyük bir klavye ve ekstra sabır ile bazı akıllı telefon uygulamaları akıllı telefonun kendisi kullanılarak yazılabilir. Ancak, çoğu durumda akıllı telefon yazılımı, akıllı telefon yazılımını düzenleme, çevirme ve test etme araçları sağlayan özel yazılım sistemleri kullanılarak masaüstü bilgisayarlarda geliştirilir. Basit uygulamalar genellikle Java, C++ ve C# dillerinde yazılır. Ancak, daha karmaşık uygulamalar veya çekirdek sistem yazılımları yazmak, eşzamanlı ve olay odaklı programlama için ek destek gerektirebilir.

Sonuç yalnızca üç ile artırılacaktır, altı değil. Sadece bir iş parçacığı tarafından erişilebilen verilere karşılıklı olarak dışlayıcı erişim denir.

Karşılıklı olarak dışlayıcı erişimi uygulamanın bir yolu, paylaşılan verileri kullanan bir iş parçacığı olduğunda, diğer iş parçacıklarının bu verilere erişimini engelleyecek şekilde ilgili iş parçacıklarını tanımlayan programbirimlerini yazmaktır. (Bu, paylaşılan verilere erişen bir sürecin bölümünü kritik bölge olarak tanımladığımız isteğe bağlı Bölüm 3.4'te açıklanan yaklaşımdır.) Deneyimler, bu yaklaşımın karşılıklı dışlamayı sağlama görevini programın çeşitli bölümlerine dağıtmanın dezavantajına sahip olduğunu göstermiştir—verilere erişen her programbirimi karşılıklı dışlamayı sağlamak için uygun şekilde tasarlanmalıdır ve bu nedenle tek bir segmentteki bir hata tüm sistemi bozabilir. Bu nedenle, birçok kişi daha iyi bir çözümün veriyi kendisine erişimi kontrol etme yeteneği ile donatmak olduğunu savunmaktadır. Kısacası, verilere erişen iş parçacıklarının çoklu erişime karşı koruma sağlamasına güvenmek yerine, veri öğesine bu sorumluluk atanır. Sonuç olarak, erişim kontrolü programda tek bir noktada yoğunlaşır, birçok program birimi arasında dağılmak yerine. Kendisine erişimi kontrol etme yeteneği ile artırılmış bir veri öğesine genellikle bir *monitör* denir.

Paralel işlem için programlama dillerinin tasarımının, iş parçacıklarının oluşturulması, iş parçacıklarının duraklatılması ve yeniden başlatılması, kritik bölgelerin tanımlanması ve izleyicilerin bileşimi gibi şeyleri ifade etmenin yollarını geliştirmeyi içerdiğini sonucuna varıyoruz.

Sonuç olarak, animasyonun paralel hesaplama sorunlarını keşfetmek için ilginç bir ortam sağladığını belirtmeliyiz, ancak bu, paralel işleme tekniklerinden faydalanan birçok alandan sadece biridir. Diğer alanlar arasında hava durumu tahmini, hava trafiği kontrolü, karmaşık sistemlerin simülasyonu (nükleer reaksiyonlardan yaya trafiğine kadar), bilgisayar ağları ve veritabanı bakımı bulunmaktadır.

Sorular ve Alıştırmalar

1. Eşzamanlı işleme için bir programlama dilinde bulunacak bazı özellikler, daha geleneksel bir dilde bulunmayacak neler olabilir?
2. Verilere karşılıklı olarak dışlayıcı erişimi sağlamak için iki yöntemi tanımlayın.
3. Paralel hesaplamanın faydalı olduğu animasyon dışındaki bazı ortamları tanımlayın.

Doclingo https://www.doclingo.ai

6.7 Deklaratif Programlama

Bölüm 6.1'de, biçimsel mantığın, bir deklaratif programlama sisteminin inşa edilebileceği genel bir problem çözme algoritması sağladığını iddia etmiştik. Bu bölümde, bu iddiayı, önce algoritmanın temel unsurlarını tanıtarak ve ard ından buna dayanan bir deklaratif programlama diline kısaca bakarak araştır ıyoruz.

Mantıksal Çıkarım

Diyelim ki Kermit'in sahnede olduğu ya da Kermit'in hasta olduğu bilgisini biliyoruz ve Kermit'in sa hnede olmadığı söyleniyor. O zaman Kermit'in hasta olması gerektiğini sonucuna varabiliriz. Bu, çözümleme adı verilen bir çıkarımsal akıl yürütme ilkesinin bir örneğidir. Çözü mleme, bir dizi ifadenin sonucunu çıkarmak için kullanılan birçok teknikten biridir ve bu tekniklere çıkarım kuralları denir.

Çözümlemeyi daha iyi anlamak için, önce basit durum ifadelerini tek harf lerle temsil etmeyi ve bir ifadenin olumsuzluğunu ¬ sembolü ile göstermeyi kabul edelim. Örneğin, "Kermit bir prens" ifadesini A ile ve "Miss Piggy bir ak tris" ifadesini B ile temsil edebiliriz. O zaman, ifade

A VEYA B

"Kermit bir prens ya da Miss Piggy bir aktris" anlamına gelecektir ve

B AND $\neg A$

"Miss Piggy bir aktris ve Kermit bir prens değildir." anlamına gelecektir. "İma eder" a nlamını belirtmek için birok kullanacağız. Örneğin, ifade

$A \rightarrow B$

"Kermit bir prens ise, Miss Piggy bir aktris demektir."Genel fo rmunda, çözümleme ilkesi iki ifadeninşu biçimde olduğunu belirtir

P VEYA Q

ve

R VEYA $\neg Q$

ifadesinden, şu sonucu çıkarabiliriz:

P VEYA R

Bu durumda, iki orijinal ifadenin üçüncüifade olan çözümlenen ifade oluşturacak şekild e çözüldüğünü söyleriz. Çözümlenen ifadenin, orijinal ifadelerin mantıksal bir sonucu ol duğunu gözlemlemek önemlidir. Yani, eğer orijinalifadeler doğruysa, çözümlenen ifade de doğru olmalıdır. (Eğer Q doğruysa, o zaman R de doğru olmalıdır; ancak eğer Q yanlı şsa, o zaman P doğru olmalıdır. Böylece Q'nun doğruluğu veya yanlışlığına bakılmaksızı n, ya P ya da R doğru olmalıdır.)

İki ifadenin çözümlemesini, Şekil 6.24'te gösterildiği gibi resimsel olarak temsil edeceğiz; burada orijinal ifadeleri, çözümlenen ifadelerine doğru uzan an çizgilerle yazarız. Çözümlemenin yalnızca kloz biçimi olan ifade çiftlerine uygulanabileceğini unutmayın; yani, temel bileşenleri Mantıksal VEYA işlemi i le bağlı olan ifadelere. Böylece

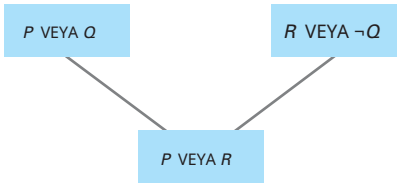
P VEYA Q

kloz biçimindedir, oysa

$P \rightarrow Q$

Doclingo https://www.doclingo.ai

Şekil **6.24** İfadeleri çözmek (*P* VEYA ¬ *Q*) ve (*R* VEYA *Q*) ile (*P* VEYA *R*) üretmek



kloz biçiminde değildir. Bu potansiyel sorunun ciddi bir endişe yaratmaması, matematiksel mantıkta herhangi bir ifadenin birinci dereceden predikat mantığı (geniş ifade gücüne sahip ifadeleri temsil etme sistemi) ile ifade edilebileceğini belirten bir teoremin sonucudur. Bu önemli teoremi burada incelemeyeceğiz, ancak gelecekte referans olması açısından, ifadenin

$$P \rightarrow Q$$

kloz biçimi ifadesine eşdeğerdir

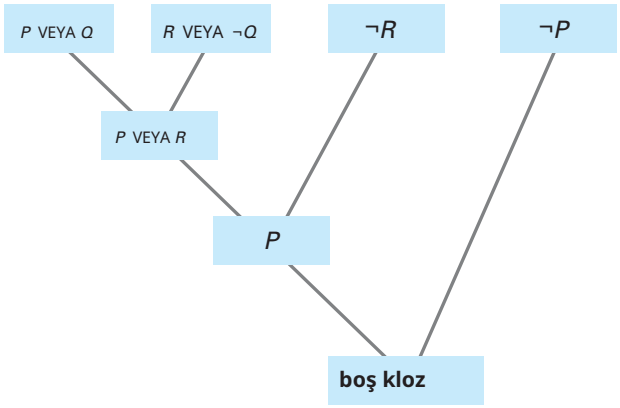
$$Q \text{ OR } \neg P$$

Bir ifade kümesinin, tüm ifadelerin aynı anda doğru olması imkânsızsa tutarsız olduğu söylenir. Diğer bir deyişle, tutarsız bir ifade kümesi, kendisiyle çelişen ifadelerden oluşan bir kümedir. Basit bir örnek, P ifadesini ve $\neg P$ ifadesini içeren bir kümedir. Mantıkçılar, tekrar eden çözümlemenin tutarsız bir kloz kümesinin tutarsızlığını doğrulamanın sistematik bir yöntemi olduğunu göstermiştir. Kural, eğer tekrar eden çözümleme uygulaması boş klozu (kloz biçiminde P ifadesi ile $\neg P$ ifadesinin çözülmesi sonucu) üretiyorsa, o zaman orijinal ifade kümesi tutarsız olmalıdır. Örnek olarak, Şekil 6.25, ifadeler kümesinin

$$P \text{ OR } Q \qquad R \text{ OR } \neg Q \qquad \neg R \qquad \neg P$$

tutarsız olduğunu göstermektedir.

Şekil **6.25** İfadeleri çözmek (*P* VEYA *Q*), (*R* VEYA ¬ *Q*), ¬ *R* ve ¬ *P*



Doclingo https://www.doclingo.ai

Şimdi, bir dizi ifadenin ifadesi P'yi ima ettiğini doğrulamak istediğimizi va rsayalım. İfadesi P'yi ima etmek, ifadesi $\neg P$ ile çelişmekle aynıdır. Bu nedenle, orijinal ifadelerin ifadesi P'yi ima ettiğini göstermek için, yapmamız gereken tek şey, orijinal ifadelere ifadesi $\neg P$'yi uygulamak ve boş bir kloz oluşana kad ar devam etmektir. Boş bir kloz elde ettiğimizde, ifadesi $\neg P$'nın orijinal ifadel erle tutarsız olduğunu ve dolayısıyla orijinal ifadelerin P'yi ima etmesi gerekti ğini sonucuna varabiliriz.

Bir son nokta, çözümlemeyi gerçek bir programlama ortamında uygulamay a hazır olmadan önce kalıyor. Diyelim ki elimizde iki ifade var

```
(Mary is at X) → (Mary's lamb is at X)
```

(burada X herhangi bir yeri temsil eder) ve

Mary evde

Kloz biçiminde iki ifade şunlardır

```
(Mary's lamb is at X) OR ¬(Mary is at X)
```

ve

```
(Mary is at home)
```

ilk bakışta çözülebilecek bileşenlere sahip gibi görünmüyorlar. Öte yandan, b ileşenler (Mary evde) ve ¬(Mary X'te)birbirinin zıttı olmaya oldukça yakın. Sor un, Mary X'te ifadesinin, genel olarak yerler hakkında bir ifade olması neden iyle, özellikle *ev* hakkında bir durum ifadesi olduğunu tanımaktır. Böylece yu karıdaki ilk ifadenin özel bir durumu

```
(Mary's lamb is at home) OR ¬(Mary is at home)
```

şu ifadeyle çözülebilir

(Mary evde)

şu ifadeyi üretmek için

```
(Mary's lamb is at home)
```

Değişkenlere değer atama süreci (örneğin, X'e ev değerini atamak) çözümle me yapılabilmesi için birleştirme olarak adlandırılır. Bu süreç, genel ifadeleri n bir çıkarım sisteminde belirli uygulamalara uygulanmasını sağlar.

Prolog

Prolog programlama dili (LOGic'te PROgramming'in kısaltması), tekrar eden çözümleme temelli bir problem çözme algoritmasına dayanan bir deklaratif programlama dilidir. Bu tür dillere mantıksal programlama dilleri denir. Prol og'daki bir program, temel ifadelerden oluşur ve bu ifadelere temel algoritm a çıkarımsal akıl yürütmesini uygular. Bu ifadelerin inşa edildiği bileşenlere p redikatlar denir.

Bir predikat, bir predikat tanımlayıcısı ve ardından predikatın argümanlarını listeleyen p arantez içi bir durum ifadesi içerir. Tek bir predikat, argümanları hakkında bir gerçeği te msil eder ve tanımlayıcısı genellikle bu temel durumu yansıtacak şekilde seçilir.

Doclingo https://www.doclingo.ai

anlam. Bu nedenle, Bill'in Mary'nin ebeveyni olduğunu ifade etmek istiyorsak, predik at biçimini kullanabiliriz

ebe(bill, mary)

Bu predikattaki argümanların küçük harfle başladığını unutmayın, bu argüm anlar özel isimleri temsil etse bile. Bunun nedeni, Prolog'un sabit olan argü manlar ile değişken olan argümanları ayırt etmesidir; sabitler küçük harfle, d eğişkenler ise büyük harfle başlamalıdır.
(Burada prolog kültürünün terminolojisini kullandık, burada terim *constant*, daha genel terim olan *literal* yerine kullanılır. Daha kesin olarak, terim`bill` [küçük harfle dikkat edin] prologda, daha genel bir notasyonda "`Bill`" olarak temsil edilebilecek olan literal'ı temsil etmek için kullanılır. `Bill` terimi [büyük harfle dikkat edin] prologda bir değişkene atıfta bulunmak için kullanılır.)

Bir Prolog programındaki ifadeler ya gerçekler ya da kurallardır ve her biri bir nokta ile sonlanır. Bir gerçek, tek bir predikattan oluşur. Örneğin, bir kaplumbağa nın bir sümüklüböcekten daha hızlı olduğu gerçeği, Prolog ifadesi ile temsil edileb ilir.

```
faster(turtle, snail).
```

ve bir tavşanın bir kaplumbağadan daha hızlı olduğu gerçeği

hızlı(tavşan, kaplumbağa).

Bir Prolog kuralı bir "çıkarım yapar" ifadesidir. Ancak, böyle bir ifadeyi X → Y biçi minde yazmak yerine, bir Prolog programcısı "Y eğer X" şeklinde yazar, tek farkla ki i fadesinin yerine :- (bir iki nokta üst üste ve ardından bir tire) sembolü kullanılır. Böyl ece "X yaşlıdır çıkarım yapar X bilgedir" ifadesi bir mantıkçı tarafından

```
old(X) → wise(X)
```

ama Prolog'da şu şekilde ifade edilir

bilge(X) :- yaşlı(X).

Başka bir örnek olarak, kural

(hızlı(X, Y) VE hızlı(Y, Z)) → hızlı(X, Z)

Prolog'da şu şekilde ifade edilir

```
faster(X, Z) :- faster(X, Y), faster(Y, Z).
```

(hızlı(X, Y) ve hızlı(Y, Z) ayıran virgül, AND birleşimini temsil eder.) Bu tür kura llar kloz biçiminde olmasa da, Prolog'da kolayca kloz biçimine dönüştürülebil eceği için kabul edilir.

Prolog sisteminin bir programdaki öntanımların anlamını bilmediğini unut mayın; sadece çözümleme çıkarım kuralına göre tamamen sembolik bir şekilde ifadeleri manipüle eder. Bu nedenle, programcının bir öntanımın tüm ilgili özel liklerini gerçekler ve kurallar açısından tanımlaması gerekmektedir. Bu bağlam da, Prolog gerçekleri bir öntanımın belirli örneklerini tanımlamak için kullanılır ken, kurallar genel ilkeleri tanımlamak için kullanılır. Bu, hızlı öntanımıyla ilgili önceki ifadelerin izlediği yaklaşımdır. İki gerçek "hızlılık"ın belirli örneklerini tanı mlarken, kural genel bir özelliği tanımlar. Bir tavşanın bir sümüklüböceğinden daha hızlı olduğu gerçeğinin, açıkça belirtilmemiş olmasına rağmen, iki gerçeği n kural ile birleştirilmesinin bir sonucu olduğunu unutmayın.

Doclingo https://www.doclingo.ai

Prolog kullanarak yazılım geliştirirken, bir programcının görevi,bilgiyi tan ımlayan gerçekler ve kurallar topluluğunu geliştirmektir.Bu gerçekler ve kur allar, çıkarımsal sistemde kullanılacak olan başlangıç ifadeleri kümesini oluşt urur.Bu ifadeler topluluğu oluşturulduktan sonra, sisteme önerilebilecek var sayımlar (Prolog terminolojisinde hedefler olarak adlandırılır)genellikle bir bi lgisayarın klavyesine yazarak sunulur. Böyle bir hedef Prolog sistemine sunu lduğunda,sistem, hedefin başlangıç ifadelerinin bir sonucu olduğunu doğrul amaya çalışmak için çözümleme uygular.İlişkiyi tanımlayan ifadeler topluluğ umuza dayanarak daha hızlı, her bir hedef

daha_hızlı(turtle, sümüklü böcek).
hızlı(tavşan, kaplumbağa).
daha_hızlı(tavşan, sümüklü böcek).

her biri başlangıç ifadelerinin mantıksal bir sonucu olduğu için böyle doğrulanabilir. İlk iki, başlangıç ifadelerinde yer alan gerçeklerle aynıdır,
üçüncüsü ise sistem tarafından belirli bir çıkarım derecesi gerektirir.

Daha ilginç örnekler, argümanları sabitler yerine değişkenler olan hedefler sa ğladığımızda elde edilir.Bu durumlarda Prolog, başlangıç ifadelerinden hedefi tür etmeye çalışırken,bunu yapmak için gereken birleştirmeleri takip eder.Sonra, eğe r hedef elde edilirse, Prolog bu birleştirmeleri rapor eder.Örneğin, aşağıdaki hede fi düşünün

daha_hızlı(W, salyangoz).

Buna yanıt olarak, Prolog bildirir

daha_hızlı(tavşan, salyangoz).

Gerçekten de, bu, başlangıç ifadelerinin bir sonucudur ve birleştirme yoluyla hedefle uyumludur. Dahası, Prolog'dan daha fazla bilgi istemiş olsaydık, bulur ve sonucu bildirir.

```
faster(rabbit, snail).
```

Aksine, Prolog'dan bir tavşandan daha yavaş olan hayvanların örneklerini bulmasını isteyebiliriz, hedefi önererek.

```
faster(rabbit, W).
```

Aslında, eğer hedefle başlasaydık

```
faster(V, W).
```

Prolog, nihayetinde başlangıç ifadelerinden türetilebilecek tüm daha hızlı ilişkileri arama yoluna gidecektir. Bu, tek bir Prolog programının belirli bir hayvanın diğerinden daha hızlı olduğunu doğrulamak, belirli bir hayvandan daha hızlı olan hayvanları bulmak, belirli bir hayvandan daha yavaş olan hayvanları bulmak veya tüm daha hızlı ilişkileri bulmak için kullanılabileceği anlamına gelir.

Bu potansiyel çok yönlülük, bilgisayar bilimcilerinin hayal gücünü çeken özelliklerden biridir. Ne yazık ki, bir Prolog sisteminde uygulandığında, çözümleme prosedürü, teorik formunda mevcut olmayan sınırlamaları miras alır ve bu nedenle Prolog programları beklenen esnekliğe ulaşamayabilir.
Ne demek istediğimizi anlamak için, önce Şekil 6.25'teki diyagramın sadece mevcut görevle ilgili olan çözümlemeleri gösterdiğini not edin. Çözümleme sürecinin izleyebileceği başka yönler de vardır. Örneğin, en soldaki ve en sağdaki klozlar, çözümlenen ifade Q'yu üretmek için çözülebilir. Böylece, bir uygulamada yer alan temel gerçekleri ve kuralları tanımlayan ifadelerin yanı sıra, bir Prolog

Doclingo https://www.doclingo.ai

Program genellikle çözümleme sürecini doğru yönde yönlendirmek amacıyla ek ifadeler içermelidir. Bu nedenle, gerçek Prolog programları önceki örneği mizin önerdiği çoklu amaçları yakalayamayabilir.

Sorular & Alıştırmalar

1. Hangi ifadeler $R, S, T, U,$ ve V mantıksal sonuçlardır ifadeler kümesi ($\neg R$ VEYA T VEYA S), ($\neg S$ VEYA V), ($\neg V$ VEYA R), (U VEYA $\neg S$), (T VEYA U), ve (S VEYA V)?
2. Aşağıdaki ifadeler kümesi tutarlı mı? Cevabınızı açıklayın.

   P OR Q OR R $\quad$ $\neg R$ OR Q $\quad$ R OR $\neg P$ $\quad$ $\neg Q$
3. Aşağıdaki Prolog programının sonundaki iki kuralı tamamlayın, böylece predikat `mother(X, Y)` "X, Y'nin annesidir" anlamına gelsin ve predikat `father(X, Y)` "X, Y'nin babasıdır" anlamına gelsin.

```
kadın(carol).
kadın(dava).
erkek(fatura).
erkek(john).
ebe(john, carol).
ebe(sue, carol).
anne(X,Y) :-
baba(X,Y) :-
```

4. Soru 3'teki Prolog programı bağlamında, aşağıdaki kural X'in Y'nin kardeşi olduğunu ifade etmek için tasarlanmıştır, eğer `X` ve `Y` ortak bir ebeveynleri varsa.

   kardeş(X, Y) :- ebeveyn(Z, X), ebeveyn(Z, Y).

   Bu kardeş ilişkisi tanımının Prolog'un hangi beklenmedik sonuca ulaşmasına izin verecektir?

Bölüm Gözden Geçirme Problemleri

(Yıldızlı problemler isteğe bağlı bölümlerle ilişkilidir.)

1. Bir programlama dilinin makine bağımsız olduğunu söylemek ne anlama geliyor?
2. Aşağıdaki sahte kod programınıEk C'de tanımlanan makine diline çevirin.

```
x ← 0;
while(x < 3) do
   (x←x + 1)
```

3. İfadeyi çevirin

   Yarısı← Uzunluk + Genişlikmak

   ale C'nin makine diline,Uzunluk, Genişlik ve Yarısı'nınhepsinin kayan nokta olarak temsil edildiğini varsayarak notasyonu.
4. Yüksek seviyeli ifadeyi çevirin

```
if(X eşittir 0)
   sonraZ←Y + W
   else Z ← Y + X
```

Doclingo https://www.doclingo.ai

makale C'nin makine diline,W, X, Y ve Z'nı n hepsinin değerler olduğunu varsayarak iki'nin tamamlayıcı notasyonunda,her bir i bir bayt bellek kullanarak.

5. Değişkenlerle ilişkili verilerin türünü belirle mek neden gerekliydi?Problem 4'te ifadel eri çevirmek için?
Neden birçok yüksek seviyeli programlam a dili,programcının bir programın başında her birdeğişkenin türünü tanımlamasını ger ektirir?

6. Dört farklı programlama paradigmasını adland ırın ve tanımlayın.

7. Fonksiyon f iki sayısal değeri girdi olarak bekliyor ve bu iki değerden daha küçüğünü çıktı değeri olarak döndürüyor. Eğer w,x,y, ve z sayısal değerleri temsil ediyorsa, f(f(w,x), f(y,z)) ifadesinin döndürdüğü sonuç nedir?

8. Fonksiyon f verilen sembol dizisinin tersini almaktadır ve g verilen iki dizenin birleştiril mesinidöndüren bir fonksiyondur. Eğer x diz isi *abcd* ise,$g(f(x),x)$ ifadesinin döndürdüğ ü sonuç nedir?

9. Finansal kayıtlarınızı tutmak için nesne yönelimlibir program yazmayı planlıyorsanı z, kontrolhesabınızı temsil eden nesne içind e hangi verilersaklanmalıdır? O nes ne hangi mesajlara yanıtverebilmelidir? Pro gramda kullanılabilecek diğernesneler n elerdir?

10. Bir makine dili ile bir montaj dili arasındaki farkı özetleyin.

11. Ek C'de tanımlanan makine için bir montaj dili tasarımı yapın.

12. John Programmer, bir program içinde sabitlerdecl are etme yeteneğinin gerekli olmadığını,değişkenlerin bunun yerine kullanılabileceğinisavunmaktadır. Örneğin, `AirportAlt` örneğimiz Bölüm 6. 2'de, AirportAlt'ı bir değişken olarak tan ımlayarak ve ardından programın başında g erekli değeri atayarak işlenebilir. Neden b u, bir sabit kullanmaktan daha iyi değil?

13. Deklaratif ifadeler ile emir ifadeleri arasın daki farkı özetleyin.

14. Bir literal, birsabit ve bir değişken arasın daki farkları açıklayın.

15. a. Operatör önceliği nedir?
b. Operatör önceliğine bağlı olarak, ifade $6 + 2 \times 3$ ile ilişkilendirilebilecek değerler neler olabilir?

16. Yapılandırılmış programlama nedir?

17. "eşittir" sembolünün ifadedeki a nlamı arasındaki fark nedir

```
eğer (X = 5) ise ( . . . )
atama ifadesinin aksine
X = 2 + Y
```

18. Aşağıdaki ifadeyi temsil eden bir akış diyagramı çizin.

```
for (int x = 2; x < 8; ++x)
{ . . . }
```

19. Aşağıdaki for ifadesini, 5. Bölümdeki sahte kodumuz da eşdeğer bir program segmentine çevirin. while ifadesini kullanarak.

```
for (int x = 2; x < 8; ++x)
{ . . . }
```

20. Yazılı müziğe aşina iseniz, müzik notasyonu nu bir programlama dili olarak analiz edin. Kontrol yapıları nelerdir? Program yoruml arını eklemek için sözdizimi nedir? Şekil 6.7' deki for ifadesine benzer anlamlara sahip müzik notasyonu nedir?

21. Aşağıdaki ifadeyi temsil eden bir akış diya gramı çizin.

```
geçiş (takım elbise)
  {case "kulüpler": bid(1);
   case "elmaslar": bid(2);
   case "kalpler": bid(3);
   case "maçlar": bid(4);
  }
```

22. Aşağıdaki program segmentini iç içe `if-then-else` ifadeleri yerine tek bir case ifadesi kullanarak yeniden yazın.

```
eğer(W = 5)
  sonra(Z←7)
  else(if(W = 6)
         then(Y←7)
         else(if(W = 7)
                then(X←7)
              )
         )
```

Doclingo https://www.doclingo.ai

23. Aşağıdaki fare yuvası rutinini tek bir `if-then-else` ifadesi ile özetleyin:

```
    eğerX > 5 sonragoto 80
    X = X + 1
    goto 90
80  X = X + 2
90 dur
```

24. İkna edici ve nesne yönelimli programlama dillerinde bulunan temel kontrol yapılarını aşağıdaki her bir etkinliği gerçekleştirmek için özetleyin:

a. Hangi komutun bir sonraki olarak yürütülmesi gerektiğini belirleme

b. Bir dizi komutu tekrarlama

c. Bir değişkenin değerini değiştirme

25. Bir çevirmen ile bir yorumlayıcı arasındaki farkı özetleyin.

26. Bir programda `X` değişkeninin tam sayı türünde olarak tanımlandığını varsayalım. Program ifadesini yürütürken hangi hata meydana gelecektir

`X`← 2,5

27. Bir programlama dilinin güçlü bir şekilde tiplenmiş olduğunu söylemek ne anlama gelir?

28. Büyük bir dizinin muhtemelen bir prosedüre değer olarak neden iletilmeyeceğini açıklayın?

29. Varsayalım ki `Modify` prosedürü, 5. Bölümdeki sahte kodumuzda şu şekilde tanımlanmıştır

```
prosedürDeğiştir (Y)
  Y←7;
  Y'nin değerini yazdır.
```

Parametreler değer ile geçiliyorsa, aşağıdaki program parçası çalıştırıldığında ne yazdırılacak?Parametreler referans ile geçiliyorsa ne olacak?

```
X←5;
X'e Modifiye prosedürünü uygula;
X'in değerini yazdır;
```

30. Varsayalım ki `Modify` prosedürü, 5. Bölümdeki sahte kodumuzda şu şekilde tanımlanmıştır

```
prosedürDeğiştir (Y)
  Y←9;
  X'in değerini yazdır;
  Y'nin değerini yazdır.
```

Ayrıca X'in bir global değişken olduğunu varsayalım. Eğerparametreler değer ile geçiliyorsa, ne olacak aşağıdaki program segmenti çalıştırıldığında ne yazdırılır?Parametreler referansla geçirilirse ne olur?

```
X←5;
X'e Modifiye prosedürünü uygula;
X'in değerini yazdır;
```

31. Bazen bir gerçek parametre, prosedür tarafından kullanılmak üzere bir kopya üretilerek bir prosedüre geçirilir (parametre değerle geçirildiğinde olduğu gibi), ancak prosedür tamamlandığında, prosedürün kopyasındaki değer, çağıran prosedür devam etmeden önce gerçek parametreye aktarılır. Bu tür durumlarda, parametrenin değer-sonuç olarak geçirildiği söylenir. Parametreler değer-sonuç olarak geçirilseydi, Problem 30'daki program segmenti ne yazdırırdı?

32. a. Parametrelerin değeriyle geçirilmesinin, referansla geçirilmesine göre ne gibi bir avantajı vardır?

b. Parametrelerin referansla geçirilmesinin, değeriyle geçirilmesine göre ne gibi bir avantajı vardır?

33. İfadede hangi belirsizlikler vardır

`X`←3 + 2 ×5

34. Diyelim ki küçük bir şirketin beş çalışanı var ve sayıyı altıya çıkarmayı planlıyor. Ayrıca, şirketin bir programının aşağıdaki atamaifadelerini içerdiğini varsayalım.

```
GünlükMaaş = ToplamMaaş/5;
OrtalamaMaaş = ToplamMaaş/5;
GünlükSatışlar = ToplamSatışlar/5;
OrtalamaSatışlar = ToplamSatışlar/5;
```

Programın güncellenmesi görevinasıl basitleştirilirdi eğer programbaşlangıçta Num-berOf Emp ve`WorkWeek` (her ikisi de5 değerine ayarlanmış) adında sabitler kullanılarakyazılmış olsaydı, böylece atama ifadelerişu şekilde ifade edilebilirdi

```
GünlükMaaş = ToplamMaaş/GünSayısı;
OrtalamaMaaş = ToplamMaaş/ÇalışanSayısı;
GünlükSatışlar = ToplamSatışlar/GünSayısı;
OrtalamaSatışlar = ToplamSatışlar/ÇalışanSayısı;
```

35. a. Biçimsel bir dil ile doğal bir dil arasındakifark nedir?

b. Her birine bir örnek verin.

Doclingo https://www.doclingo.ai

36. Bölüm 5'teki sahte kodda `while` ifadesinin yapısını temsil eden bir sözdizim diyagramı çizin.

37. Telefon numaralarının sözdizimini tanımla mak için bir dizi sözdizim diyagramı tasarl ayın.Örneğin, Amerika Birleşik Devletleri'nde te lefonnumaraları bir alan kodu, ardından bir bölgesel kod,sonra da (444) 555–1234 g ibi dört haneli bir numaraile oluşur.

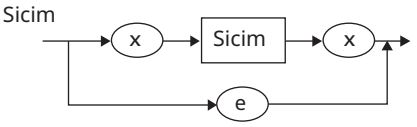
38. Kendi ana dilinizdeki basit cümleleri tanımlam ak için bir dizi sözdizim diyagramı tasarlayın.

39. Farklı tarihleri temsil etmenin yollarını tanımlamak için bir dizi sözdizim diyagramı tasarlayın, örneğin *ay/gün/yıl* veya *ay gün, yıl*.

40. "evet" kelimesinin tekrarları ile oluşan "cümle lerin" dilbilgisel yapısını tanımlayan bir dizi s özdizim diyagramı tasarlayın; ardından a ynı sayıda "hayır" kelimesi gelmektedir. Örneğin, "evet evet hayır hayır" böyle bir cümle ol urken, "hayır evet," "evet hayır hayır" ve "evet ha yır evet" böyle değildir.

41. "evet" kelimesinin tekrarları ile oluşan "cü mlelerin" dilbilgisel yapısını tanımlayan bir dizi sözdizim diyagramı tasarlanamayac ağına dair bir argüman verin; ardından a ynı sayıda "hayır" kelimesi gelmektedir, ar dından aynı sayıda "belki" kelimesi gelmekt edir.
Örneğin, "evet hayır belki" ve "evet evet hayır hayır belki belki" böyle cümleler o lurken, "evet belki," "evet hayır hayır belki belki" ve "belki hayır" böyle değildir.

42. Aşağıdaki sözdizim diyagramı ile tanımlanan bir dize yapısını tanımlayan bir cümle yazı n. Ardından, xxyxx dizesi için ayrıştırma ağ acını çizin.



43. Soru 5'teki sözdizim diyagramlarına ekleyin. Bölüm 6.4'te, Dans yapısının ya bir Chacha ya da bir Va ls olmasını tanımlayan bir dizi diyagram elde et mek için, bir Vals'ın bir veya daha fazla kopyasınd an oluştuğu bir desen tanımlayın.

ileri çapraz kapalıvey
a
geri çapraz kapalı

44. İfade için ayrıştırma ağacını çizin

`x×y+y  x`

Şekil 6.15'teki sözdizimi diyagramlarına dayanarak.

45. Bir kod oluşturucu, aşağıdaki ifadeyi temsil e den makine kodunu oluştururken hang i kod optimizasyonunu gerçekleştirebilir

```
eğer (X = 5)  sonra (Z←X + 2)
              aksi takdirde (Z←X + 4)
```

46. Aşağıdaki program segmentini basitleştirin

```
Y ← 5;
eğer (Y = 7)
  sonra (Z←8)
  aksi takdirde (Z←9)
```

47. Aşağıdaki program segmentini basitleştirin

```
iken(X 5'e eşit değil) yap
  (X ← 5)
```

48. Bir nesne yönelimli programlama ortamın da, türler ve sınıflar nasıl benzer? Nasıl farklı dırlar?

49. Farklı türde binaları tanımlayan sınıfları geli ştirmek için mirasın nasıl kullanılabileceğini açıklayın.

50. Bir sınıfın kamu ve özel kısımları arasındaki fark nedir?

51. a. Bir örnek verin; bir örnek değiş kenin özel olması gereken bir duru m.
b. Bir örnek verin; bir örnek değiş kenin kamu olması gereken bir dur um.
c. Bir örnek verin; bir metodun özel olması gereken bir durum.
d. Bir örnek verin; bir metodun kamu olm ası gereken bir durum.

52. Bir otel lobisinde yaya trafiğini simüle eden bir programda bulunabilecek bazı nesneleri tanımlayın. Bazı nesnelerin gerçekleştire bileceği eylemlerin açıklamalarını da dahil edin.

***53.** "monitor" terimi bir programlama dili bağ lamında ne anlama gelir?

Doclingo https://www.doclingo.ai

***54.** Eşzamanlı işlemenin hangi özellikleri bir eşzamanlılık desteğine sahip bir programlama dili kullanmayı arzu edilir hale getirir?

***55.** Bir diyagram çizin (Şekil 6.25'e benzer) çözümlemeleri temsil eden,ifadelerin kümesinin (Q VEYA $\neg R$),(T VEYAR), $\neg P$, (P VEYA $\neg T$) ve (P VEYA $\neg Q$) tutarsız olduğunu göstermek için.

***56.** İfadeler kümesi $\neg R$, (T VEYAR),(P VEYA $\neg Q$), (Q VEYA $\neg T$), ve (R VEYA $\neg P$) tutarlı mı?\ Cevabınızı açıklayın.

***57.** Prolog programını Bölüm 6.7'deki Soru 3 ve 4'te belirtilen şekilde, amca, hala, büyükanne ve kuzen gibi ek aile ilişkilerini de içerecek şekilde genişletin. Ayrıca, X ve Y'nin Z'yi tanımladığı bir kural ekleyin.

YZ'nin ebeveynleridir.

***58.** Aşağıdaki Prolog programındaki ilk ifadenin anlamının "Alice sporları sever," programın son iki ifadesini çevir. Ardından, bu programa dayanarak Prolog'un Alice'in sevdiği şeyleri listelemesini sağla.
Listeyi açıkla.

```
seviyor(alice, spor).
likes(alice, müzik).
likes(carol, müzik).
likes(david, X) :- likes(X, sporlar).
seviyor(alice, X):- seviyor(david, X).
```

***59.** Aşağıdaki program segmenti, 1.7. Bölümde tanımlanan sekiz bitlik kayan nokta formatında değerlerin temsil edildiği bir bilgisayarda çalıştırıldığında hangi sorunla karşılaşılacaktır?

```
X← 0,01;
while(X 1.00'e eşit değil) do
   (X'in değerini yazdır;
    X←X + 0,01)
```

Sosyal Sorunlar

Aşağıdaki sorular, bilgisayar alanıyla ilişkili etik/sosyal/hukuki sorunlara bir rehber olarak tasarlanmıştır. Amaç yalnızca bu soruları yanıtlamak değildir. Ayrıca neden böyle yanıt verdiğinizi ve gerekçelerinizin bir sorudan diğerine tutarlı olup olmadığını da düşünmelisiniz.

1. Genel olarak, telif hakkı yasaları bir fikrin ifadesiyle ilişkili mülkiyet haklarını destekler, ancak fikrin kendisi için değil. Sonuç olarak, bir kitapta bir paragraf telif hakkına tabi olabilir, ancak paragrafta ifade edilen fikirler telif hakkına tabi değildir. Bu hak, kaynak programlara ve ifade ettikleri algoritmalara nasıl uzanmalıdır? Bir ticari yazılım paketinde kullanılan algoritmaları bilen bir kişinin, bu algoritmaları ifade eden kendi programını yazmasına ve bu yazılımın sürümünü pazarlamasına ne ölçüde izin verilmelidir?
2. Yüksek seviyeli bir programlama dili kullanarak bir programcı, if,$then$ ve $while$ gibi kelimeleri kullanarak algoritmaları ifade edebilir. Bilgisayar bu kelimelerin anlamını ne ölçüde anlıyor? Kelimelerin doğru bir şekilde kullanılmasına yanıt verme yeteneği, kelimeleri anladığını mı ima eder? Başka bir kişinin söylediklerinizi anladığını nasıl bilirsiniz?
3. Yeni ve faydalı bir programlama dili geliştiren bir kişinin, o dilin kullanımından kar elde etme hakkına sahip olması gerekir mi? Eğer öyleyse, bu hak nasıl korunabilir? Bir dil ne ölçüde sahiplenilebilir? Bir şirket, çalışanlarının yaratıcı, entelektüel başarılarını sahiplenme hakkına ne ölçüde sahip olmalıdır?

Doclingo https://www.doclingo.ai

4. Bir son tarih yaklaşırken, bir programcının zamanında bir program çalıştırma k için yorum ifadeleri aracılığıyla belgeleri göz ardı etmesi kabul edilebilir mi? (Yeni başlayan öğrenciler, profesyonel yazılım geliştiricileri arasında belgelendirme nin ne kadar önemli olduğunu öğrenince genellikle şaşırırlar.)
5. Programlama dilleri üzerine yapılan araştırmaların çoğu, programcıların in san tarafından kolayca okunabilen ve anlaşılabilen programlar yazmalarına olanak tanıyan diller geliştirmeye yönelik olmuştur. Bir programcının bu tü r yetenekleri kullanması ne kadar gereklidir? Yani, programın insan perspe ktifinden iyi yazılmamış olsa bile doğru bir şekilde çalışması ne kadar yeterl idir?
6. Farz edelim ki bir amatör programcı, kendi kullanımı için bir program yaz ıyor ve bu süreçte programın yapısında dikkatsiz davranıyor. Program, d aha okunabilir hale getirecek programlama dili özelliklerini kullanmıyor, verimli değil ve programcının programı kullanmayı planladığı belirli duru mu avantaja çeken kısayollar içeriyor.
   Zamanla programcı, programı kendisi kullanmak isteyen arkadaşlarına kopyalarını verir ve bu arkadaşlar da bunu kendi arkadaşlarına verir. Programcının, meydana g elebilecek sorunlar için ne ölçüde sorumlu olduğu nedir?
7. Bir bilgisayar profesyonelinin çeşitli programlama paradigmaları hakkında ne ölçüde bilgi sahibi olması gerekir? Bazı şirketler, o şirkette geliştirilen tüm ya zılımların aynı, önceden belirlenmiş programlama dilinde yazılmasını ısrarla t alep eder. Profesyonelin böyle bir şirkette çalışması durumunda, orijinal soru ya verdiğiniz cevap değişir mi?

Ek Okuma

Aho, A. V., M. S. Lam, R. Sethi ve J. D. Ullman. *Derleyiciler: İlkeler, Teknikler v e Araçlar,* 2. baskı. Boston, MA: Addison-Wesley, 2007.

Barnes, J. *Ada 2005'te Programlama*. Boston, MA: Addison-Wesley, 2006.

Clocksin, W. F. ve C. S. Mellish. *Prolog'da Programlama,* 5. baskı. New York: Springer-Verlag, 2003.

Friedman, D. P. ve M. Felleisen. *Küçük Şemacı,* 4. baskı. Cambridge, MA: MIT Press, 1995.

Hamburger, H. ve D. Richards. *Bilgisayar Bilimleri için Mantık ve Dil Modelleri*. Upper Saddle River, NJ: Prentice-Hall, 2002.

Kernighan, B.W., ve D.M. Ritchie. *C Programlama Dili*, 2. baskı. Engle- wood Cliffs, NJ: Prentice Hall, 1988.

Metcalf, M., ve J. Reid. *Fortran 90/95 Açıklandı,* 2. baskı. Oxford, İngiltere: Oxford University Press, 1999.

Pratt, T. W. ve M. V. Zelkowitz. *Programlama Dilleri, Tasarım ve Uygulama,* 4. baskı. Upper Saddle River, NJ: Prentice-Hall, 2001.

Doclingo https://www.doclingo.ai

Savitch, W. *Absolute C++*, 3. baskı. Boston, MA: Addison-Wesley, 2008.

Savitch, W. *Absolute Java,* 3. baskı. Boston, MA: Addison-Wesley, 2008.

Savitch, W. *Problem Solving with C++* 6. baskı. Boston, MA: Addison-Wesley, 2008.

Scott, M. L. *Programming Language Pragmatics,* 3. baskı. New York: Morgan Kaufmann, 2009.

Sebesta, R. W. *Concepts of Programming Languages,* 9. baskı. Boston, MA: Addison-Wesley, 2009.

Wu, C. T. *An Introduction to Object-Oriented Programming with Java* , 3. baskı. Burr Ridge, IL: McGraw-Hill, 2008.

Doclingo https://www.doclingo.ai

Bu sayfa kasıtlı olarak boş bırakılmıştır

Doclingo https://www.doclingo.ai

BÖLÜM

YazılımMühendisliği

7

Bu bölümde, büyük, karmaşık yazılım sistemlerinin geliştirilmesi sırasında karşılaşılan sorunları inceliyoruz. Konu, yazılım geliştirme bir mühendislik süreci olduğu için yazılım mühendisliği olarak adlandırılmaktadır. Yazılım mühendisliğindeki araştırmacıların amacı, yazılım geliştirme sürecine rehberlik eden ve verimli, güvenilir yazılım ürünlerine yol açan ilkeleri bulmaktır.

7.1 Yazılım Mühendislik Disiplini

7.2 Yazılımın Yaşamı

Döngüsü

Döngü Bütün Olarak

Geleneksel Geliştirme

Aşaması

7.3 Yazılım Mühendisliği Metodolojileri

7.4 Modülerlik

Modüler Uygulama

Bağlantı

Bağlılık

Bilgi Gizleme

Bileşenler

7.5 Ticaret Araçları

Bazı Eski Arkadaşlar

Birleşik Modelleme Dili

Tasarım Kalıpları

7.6 Kalite Güvencesi

Kalite Güvencesinin Kapsamı

Yazılım Testi

7.7 Dokümantasyon

7.8 İnsan-Makine Arayüz

7.9 Yazılım Sahipliği ve Sorumluluk

Doclingo https://www.doclingo.ai

Yazılım mühendisliği, büyük, karmaşık yazılım sistemlerinin geliştirilmesine r ehberlik edecek ilkeleri arayan bilgisayar bilimi dalıdır. Bu tür sistemlerin geli ştirilmesi sırasında karşılaşılan problemler, küçük programlar yazarken karşıl aşılan problemlerin büyütülmüş versiyonlarından daha fazladır. Örneğin, bu tür sistemlerin geliştirilmesi, önerilen sistemin gereksinimlerinin değişebilec eği ve projeye atanan personelin değişebileceği uzun bir süre boyunca birde n fazla kişinin çabasını gerektirir. Sonuç olarak, yazılım mühendisliği, bilgisay ar bilimi ile daha çok iş yönetimi ile ilişkilendirilen personel ve proje yönetimi gibi konuları içerir. Ancak, biz bilgisayar bilimi ile doğrudan ilişkili konulara odaklanacağız.

7.1 Yazılım Mühendisliği Disiplini

Yazılım mühendisliğinde yer alan problemleri takdir etmek için, büyük karmaşık b ir cihaz (bir otomobil, çok katlı bir ofis binası veya belki bir katedral) seçmek ve bu nun tasarımını yapmanız ve ardından inşaatını denetlemeniz istendiğini hayal et mek faydalıdır. Projeyi tamamlamak için zaman, para ve diğer kaynaklar açısında n maliyeti nasıl tahmin edebilirsiniz? Projeyi yönetilebilir parçalara nasıl bölebilirsi niz? Üretilen parçaların uyumlu olmasını nasıl sağlayabilirsiniz? Farklı parçalar üze rinde çalışanlar nasıl iletişim kurabilir? İlerlemeyi nasıl ölçebilirsiniz? Detayların ge niş yelpazesiyle (kapı kollarının seçimi, gargoyle'ların tasarımı, vitray pencereler iç in mavi camın temin edilebilirliği, sütunların dayanıklılığı, ısıtma sistemi için haval andırma kanallarının tasarımı) nasıl başa çıkabilirsiniz? Aynı kapsamda sorular, bü yük bir yazılım sisteminin geliştirilmesi sırasında yanıtlanmalıdır.

mühendislik iyi kurulmuş bir alan olduğu için, böyle soruları yanıtlamakt a faydalı olabilecek daha önce geliştirilmiş mühendislik tekniklerinin bolca ol duğunu düşünebilirsiniz. Bu akıl yürütme kısmen doğrudur, ancak yazılımın özellikleri ile diğer mühendislik alanlarının özellikleri arasındaki temel farklılı kları göz ardı etmektedir. Bu ayrımlar, yazılım mühendisliği projelerini zorla mış, maliyet aşımına, ürünlerin geç teslimine ve memnuniyetsiz müşterilere yol açmıştır. Bu ayrımları belirlemek, yazılım mühendisliği disiplinini ilerletm enin ilk adımı olmuştur.

Bu ayrımlardan biri, sistemleri genel prefabrik bileşenlerden inşa etme yet eneğini içerir. Geleneksel mühendislik alanları, karmaşık cihazlar inşa ederken "raf ürünleri" bileşenlerini yapı taşları olarak kullanma yeteneğinden uzun zam andır faydalanmaktadır. Yeni bir otomobilin tasarımcısı, yeni bir motor veya şa nzıman tasarlamak zorunda değildir; bunun yerine, bu bileşenlerin daha önce t asarlanmış versiyonlarını kullanır. Ancak, yazılım mühendisliği bu konuda geri kalmaktadır. Geçmişte, daha önce tasarlanmış yazılım bileşenleri alan spesifikti ; yani, iç tasarımları belirli bir uygulamaya dayanıyordu ve bu nedenle genel bil eşenler olarak kullanımları sınırlıydı. Sonuç olarak, karmaşık yazılım sistemleri t arihsel olarak sıfırdan inşa edilmiştir. Bu bölümde göreceğimiz gibi, bu konuda önemli ilerlemeler kaydedilmektedir, ancak daha yapılacak iş vardır.

Yazılım mühendisliği ile diğer mühendislik disiplinleri arasındaki bir diğer a yrım, yazılımın özelliklerini ölçmek için kullanılan niceliksel tekniklerin, yani me triklerin, eksikliğidir. Örneğin, bir yazılım sisteminin geliştirme maliyetini projek te etmek için, önerilen ürünün karmaşıklığını tahmin etmek istenir, ancak yazılı mın “karmaşıklığını” ölçmek için yöntemler belirsizdir. Benzer şekilde, bir yazılı m ürününün kalitesini değerlendirmek zordur. Mekanik cihazlar söz konusu ol duğunda, kalitenin önemli bir ölçüsü, arızalar arasındaki ortalama süredir; bu, esasen bir cihazın aşınmaya ne kadar iyi dayandığının bir ölçümüdür.

Doclingo https://www.doclingo.ai

Buna karşılık, yazılım aşınmaz, bu nedenle kaliteyi ölçme yöntemi yazılım mühend isliğinde o kadar uygulanabilir değildir.

Yazılım özelliklerini niceliksel bir şekilde ölçme konusundaki zorluklar, ya zılım mühendisliğinin mekanik ve elektrik mühendisliği ile aynı anlamda sağl am bir temel bulmakta zorlanmasının nedenlerinden biridir.
Bu son konular, fizik biliminin yerleşik temellerine dayanırken, yazılım mühe ndisliği köklerini aramaya devam etmektedir.

Böylece yazılım mühendisliğindeki araştırmalar şu anda iki düzeyde ilerlemektedir: Bazı araştırmacılar, bazen uygulayıcılar olarak adlandırılan, hemen uygulanabilir t eknikler geliştirmeye çalışırken, diğerleri, teorisyenler olarak adlandırılan, daha k ararlı tekniklerin bir gün inşa edilebileceği temel ilkeleri ve teorileri aramaktadır. Öznel bir temele dayandığı için, geçmişte uygulayıcılar tarafından geliştirilen ve te şvik edilen birçok metodoloji, zamanla kendileri de geçersiz hale gelebilecek diğer yaklaşımlarla değiştirilmiştir. Bu arada, teorisyenlerin ilerlemesi yavaş olmaya de vam etmektedir.

Uygulayıcılar ve teorisyenler tarafından ilerleme ihtiyacı büyüktür. Toplumumuz bilgisayar sistemlerine ve bunlarla ilişkili yazılımlara bağımlı hale gel miştir. Ekonomimiz, sağlık hizmetlerimiz, hükümetimiz, hukuk uygulamamız, ulaşı mımız ve savunmamız büyük yazılım sistemlerine bağlıdır. Ancak bu sistemlerin g üvenilirliği ile ilgili büyük sorunlar devam etmektedir. Yazılım hataları, yükselen ay ın nükleer saldırı olarak yorumlanması, New York Bankası'nın bir günde 5 milyon dolar kaybetmesi, uzay sondalarının kaybı, ölüme ve felçlere neden olan radyasyo n aşırı dozları ve büyük bölgelerde telefon iletişiminin aynı anda kesintiye uğrama sı gibi felaketler ve yakın felaketlere yol açmıştır.

Bu, durumun tamamen karamsar olduğu anlamına gelmiyor. Prefabrik bileşenler ve metrikler gibi sorunların üstesinden gelinmesinde önemli ilerlemeler kaydedilmektedir. Ayrıca, yazılım geliştirme sürecine bilgisayar teknolojisinin uygulanması, bilg isayar destekli yazılım mühendisliği (CASE) olarak adlandırılan bir sonuç doğ urarak yazılım geliştirme sürecini daha da kolaylaştırmakta ve basitleştirmek tedir. CASE, maliyet tahmini, proje planlaması ve personel tahsisi gibi konula rda yardımcı olan proje planlama sistemleri, geliştirme projesinin ilerlemesin i izlemeye yardımcı olan proje yönetim sistemleri, belgeleri yazma ve düzenl eme konusunda yardımcı olan dokümantasyon araçları, prototiplerin geliştir ilmesine yardımcı olan prototipleme ve simülasyon sistemleri, GUI'lerin geliş tirilmesine yardımcı olan arayüz tasarım sistemleri ve program yazma ve hat a ayıklama konusunda yardımcı olan programlama sistemleri gibi çeşitli bilgi sayarlı sistemlerin geliştirilmesine yol açmıştır. Bu araçların bazıları pek fazla

Bilgisayar Mühendisliği Derneği

Bilgisayar Mühendisliği Derneği (ACM), 1947 yılında bilgi teknolojisinin sanatl arını, bilimlerini ve uygulamalarını ilerletmeye adanmış uluslararası bir bilims el ve eğitim kuruluşu olarak kurulmuştur. Merkezi New York'tadır ve bilgisaya r mimarisi, yapay zeka, biyomedikal hesaplama, bilgisayarlar ve toplum, bilgis ayar bilimi eğitimi, bilgisayar grafikleri, hiper metin/hiper medya, işletim siste mleri, programlama dilleri, simülasyon ve modelleme ile yazılım mühendisliği gibi konulara odaklanan birçok özel ilgi grubunu (SIG) içermektedir.
ACM'nin web sitesi `http://www.acm.org` adresindedir. Etik ve Mesleki Davranışlar Kodu `http://www.acm.org/constitution/code.html` adresinde bulunabilir.

Doclingo https://www.doclingo.ai

kelime işlemciler, hesap tablosu yazılımları ve yazılım mühendisleri tarafından benimsenen genel kullanım için başlangıçta geliştirilen e-posta iletişim sisteml erinden daha fazlasıdır. Diğerleri, esasen yazılım mühendisliği ortamı için tasar lanmış oldukça sofistike paketlerdir. Gerçekten de, entegre geliştirme ortamları (IDEs) yazılım geliştirmek için araçları (metin düzenleyicileri, derleyiciler, hata ayıklama araçları vb.) tek bir entegre paket içinde birleştirir. Bu tür sistemler in başlıca örnekleri, akıllı telefonlar için uygulama geliştirenlerdir. Bu sistemler, ya zılımı yazmak ve hata ayıklamak için gerekli programlama araçlarını sağlamakla k almaz, aynı zamanda grafiksel ekranlar aracılığıyla, geliştirilen yazılımın bir telefo nda nasıl performans göstereceğini görmeyi sağlayan simülatörler de sunar.

Araştırmacıların çabalarının yanı sıra, ISO, Bilgisayar Makineleri Derneği (ACM) ve Elektrik ve Elektronik Mühendisleri Enstitüsü (IEEE) gibi profesyonel ve standartlaştırma kuruluşları, yazılım mühendisliğinin durumunu iyileştirm ek için mücadeleye katılmıştır. Bu çabalar, yazılım geliştiricilerin profesyonell iğini artıran ve her bireyin sorumluluklarına karşı kayıtsız tutumları tersine ç eviren mesleki davranış ve etik kodlarının benimsenmesinden, yazılım gelişti rme kuruluşlarının kalitesini ölçmek için standartlar belirlemeye ve bu kurulu şların durumlarını iyileştirmelerine yardımcı olacak kılavuzlar sağlamaya kad ar uzanmaktadır.

Bu bölümün geri kalanında, yazılım mühendisliğinin bazı temel ilkelerini (yazılım yaşam döngüsü ve modülerlik gibi) tartışacağız, yazılım mühendisliğ inin yöneldiği bazı alanlara (tasarım kalıplarının tanımlanması ve uygulanma sı ile yeniden kullanılabilir yazılım bileşenlerinin ortaya çıkması gibi) bakacağı z ve nesne yönelimli paradigmanın alanda yarattığı etkileri gözlemleyeceğiz.

Sorular & Alıştırmalar

1. Bir programdaki satır sayısı neden programın karmaşıklığını ölçmek i çin iyi bir ölçüt olmasın?
2. Yazılım kalitesini ölçmek için bir metrik önerin. Önerdiğiniz metrikin zayıf yönleri nelerdir?
3. Bir yazılım biriminde kaç hata olduğunu belirlemek için hangi teknik kul lanılabilir?
4. Yazılım mühendisliği alanının iyileştirmelere doğru ilerlediği veya şu an da ilerlemekte olduğu iki bağlamı tanımlayın.

7.2 Yazılım Yaşam Döngüsü

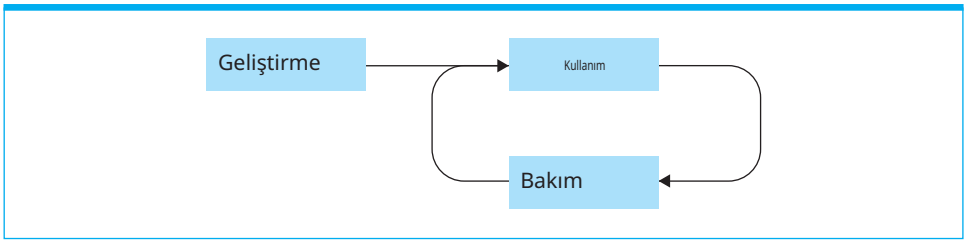
Yazılım mühendisliğindeki en temel kavram yazılım yaşam döngüsüdür.

Döngü Bütün Olarak

Yazılım yaşam döngüsü Şekil 7.1'de gösterilmektedir. Bu şekil, yazılım geliştir ildiğinde, kullanılıp bakımının yapıldığı bir döngüye girdiğini temsil eder; bu döngü yazılımın yaşamı boyunca devam eder. Bu tür bir model, birçok üretil en ürün için de yaygındır. Fark, diğer ürünlerin bakım aşamasının genellikle bir onarım süreci olmasıdır, oysa

Doclingo https://www.doclingo.ai

Şekil **7.1**Yazılım yaşam döngüsü



Yazılım durumunda, bakım aşaması genellikle düzeltme veya güncelleme ile ilgilid ir. Gerçekten de, yazılım bakım aşamasına geçer çünkü hatalar keşfedilir, yazılımı n uygulamasında değişiklikler meydana gelir ve bu da yazılımda karşılık gelen değ işiklikler gerektirir veya önceki bir değişiklik sırasında yapılan değişikliklerin yazılı mın başka yerlerinde sorunlara yol açtığı bulunur.

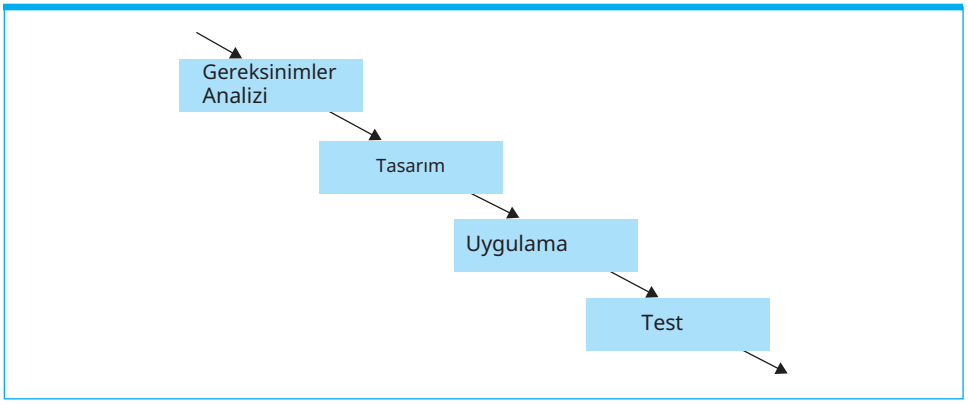
Yazılımın bakım aşamasına girmesinin nedenine bakılmaksızın, süreç, bir kişinin (genellikle orijinal yazar olmayan) temel programı ve belgelerini incel emesini gerektirir, ta ki program veya en azından programın ilgili kısmı anlaş ılana kadar. Aksi takdirde, herhangi bir değişiklik, çözdüğünden daha fazla s orun getirebilir. Bu anlayışı edinmek zor bir görev olabilir, yazılım iyi tasarlan mış ve belgelenmiş olsa bile. Aslında, genellikle bu aşamada bir yazılım parç ası, mevcut paketi başarılı bir şekilde değiştirmektense sıfırdan yeni bir siste m geliştirmenin daha kolay olduğu bahanesiyle (çok sık doğru) atılır.

Deneyimler, yazılım geliştirme sırasında biraz çabanın, değişiklikler gere ktiğinde büyük bir fark yaratabileceğini göstermiştir. Örneğin, Bölüm 6'da ve ri tanım ifadeleri üzerine yaptığımız tartışmada, sabitlerin yerine literal kulla nmanın gelecekteki ayarlamaları büyük ölçüde basitleştirebileceğini gördük. Sırasıyla, yazılım mühendisliği alanındaki araştırmaların çoğu, bu çaba-karşısında- fayda avantajından yararlanma amacıyla yazılım yaşam döngüsünün geliştirme aş amasına odaklanmaktadır.

Geleneksel Geliştirme Aşaması

Geleneksel yazılım geliştirme yaşam döngüsündeki ana adımlar gereksinim analizi, tasarım, uygulama ve testtir (Şekil 7.2).

Şekil **7.2**Yazılım yaşam döngüsünün geleneksel geliştirme aşaması



Doclingo https://www.doclingo.ai

Gereksinim Analizi Yazılım yaşam döngüsü, gereksinim analizi ile başlar—bunun amacı, önerilen sistemin hangi hizmetleri sağlayacağını belirtmek, hizmetler üzerindeki herhangi bir koşulu (zaman kısıtlamaları, güvenlik vb.) tanımlamak ve dış dünyanın sistemle nasıl etkileşime gireceğini tanımlamaktır.

Gereksinim analizi, önerilen sistemin paydaşlarından(gelecek kullanıcılar ve yasal veya finansal çıkarları olan diğer bağları olanlar)önemli bir girdi gerektirir. Aslında, nihai kullanıcının bir varlık olduğu durumlarda,örneğin bir şirket veya yazılım projesinin gerçek icrası için bir yazılım geliştiriciistediği bir devlet kurumu, gereksinim analizi, yalnızca kullanıcı tarafından yapılanbir fizibilite çalışması ile başlayabilir. Diğer durumlarda, yazılım geliştirici, ticari hazır ürünler

(COTS) kitle pazarı için yazılım, muhtemelen perakende mağazalarında satılacak veya İnternet üzerinden indirilecektir. Bu bağlamda kullanıcı daha az kesin tanımlanmış bir varlıktır ve gereksinim analizi yazılım geliştiricisi tarafından bir pazar araştırması ile başlayabilir.

Her durumda, gereksinim analizi süreci yazılım kullanıcısının ihtiyaçlarını derlemek ve analiz etmek; projenin paydaşlarıyla istekler, ihtiyaçlar, maliyetler ve uygulanabilirlik arasındaki denge üzerinde müzakere etmek; ve nihayetinde tamamlanmış yazılım sisteminin sahip olması gereken özellikleri ve hizmetleri tanımlayan bir gereksinim seti geliştirmekten oluşur. Bu gereksinimler, yazılım gereksinimleri spesifikasyonu olarak adlandırılan bir belgede kaydedilir. Bir anlamda, bu belge, yazılımın geliştirilmesini yönlendirmek ve geliştirme sürecinde ortaya çıkabilecek anlaşmazlıkları çözmek için bir araç sağlamak amacıyla tüm ilgili taraflar arasında yazılı bir anlaşmadır. Yazılım gereksinimleri spesifikasyonunun önemi, IEEE gibi profesyonel kuruluşların ve ABD Savunma Bakanlığı gibi büyük yazılım müşterilerinin onun bileşimi için standartlar benimsemiş olmasıyla gösterilmektedir.

Yazılım geliştiricisinin perspektifinden, yazılım gereksinimleri spesifikasyonu, yazılımın geliştirilmesinin ilerleyebileceği sağlam bir hedef tanımlamalıdır. Ancak, çoğu zaman bu belge bu istikrarı sağlamayı başaramaz.
Gerçekten de, yazılım mühendisliği alanındaki çoğu uygulayıcı, kötü iletişim ve değişen gereksinimlerin yazılım mühendisliği endüstrisinde maliyet aşımının ve geç ürün teslimatının başlıca nedenleri olduğunu savunmaktadır. Birçok müşteri, temeli inşa edildikten sonra bir binanın kat planında büyük değişiklikler talep etmezken, yazılım sisteminin inşası başladıktan sonra istenen yeteneklerin genişletildiği veya başka şekilde değiştirildiği organizasyonların sayısı oldukça fazladır. Bu, bir şirketin başlangıçta yalnızca bir yan kuruluş için geliştirilen sistemin aslında tüm kuruluşa uygulanması gerektiğine karar vermesi veya teknolojideki ilerlemelerin başlangıç gereksinim analizinde mevcut olan yetenekleri geçersiz kılması nedeniyle olabilir. Her durumda, yazılım mühendisleri, projenin paydaşlarıyla açık ve sık iletişimin zorunlu olduğunu bulmuşlardır.

Tasarım Gereksinim analizi önerilen yazılım ürününün bir tanımını sağlarken, tasarım önerilen sistemin inşası için bir plan oluşturmayı içerir. Bir anlamda, gereksinim analizi çözülmesi gereken problemi tanımlamakla ilgilidir, oysa tasarım problemi çözmek için bir çözüm geliştirmekle ilgilidir.
Bir sıradan insanın bakış açısından, gereksinim analizi genellikle bir yazılım sisteminin ne yapması gerektiği ile eşitlenirken, tasarım bunun nasıl yapılacağı ile eşitlenir. Bu tanım aydınlatıcı olsa da, birçok yazılım mühendisi bunun hatalı olduğunu savunuyor çünkü gerçekte, çok fazla

Doclingo https://www.doclingo.ai

gereksinim analizi sırasında nasıl dikkate alındı ve tasarım sırasında çok *ne* d ikkate alındı.

Yazılım sisteminin iç yapısının belirlendiği aşama tasarım aşamasıdır.Tas arım aşamasının sonucu, yazılım sisteminin yapısının ayrıntılı bir tanımıdır ve bu tanım programlara dönüştürülebilir.

Eğer proje bir yazılım sistemi yerine bir ofis binası inşa etmeyi amaçlıyors a, tasarım aşaması belirlenen gereksinimleri karşılayan bir bina için ayrıntılı yapısal planların geliştirilmesinden oluşacaktır. Örneğin, bu tür planlar, öner ilen binayı çeşitli ayrıntı seviyelerinde tanımlayan bir dizi mavi baskı içerecekt ir. Gerçek bina bu belgelerden inşa edilecektir. Bu planların geliştirilmesi için kullanılan teknikler yıllar içinde evrim geçirmiştir ve standartlaştırılmış notas yon sistemleri ile birçok modelleme ve diyagramlama metodolojisini içermek tedir.

Aynı şekilde, diyagramlama ve modelleme yazılım tasarımında önemli roller oynamaktadır. Ancak, yazılım mühendisleri tarafından kullanılan metodolojiler ve notasyon sistemleri, mimarlık alanındaki kadar istikrarlı değildir. Mimarlık disiplininin iyi kurulmuş yapısıyla karşılaştırıldığında, yazılım mühendisliği uygulaması çok dinamik görünmektedir, çünkü araştırmacılar yazılım geliştirme sürecine daha iyi yaklaşımlar bulmak için çaba sarf etmektedirler. Bu değişen alanı Bölüm 7.3'te keşfedeceğiz ve bazı mevcut notasyon sistemlerini ve bunların ilişkili diyagramlama/modelleme metodolojilerini Bölüm 7.5'te inceleyeceğiz.

Uygulama Uygulama, programların gerçek yazımını, veri dosyalarının oluştur ulmasınıve veritabanlarının geliştirilmesini içerir. Uygulama aşamasında, bir yazılımanalisti (bazen sistem analisti olarak adlandırılır) ile bir programcıara sındaki ayrımı görürüz. İlk kişi, tüm geliştirme süreciyle ilgilenen bir kişidir,b elki de gereksinim analizi ve tasarım adımlarına vurgu yaparak. İkincisi isees as olarak uygulama adımıyla ilgilenen bir kişidir. En dar anlamıyla, bir progra mcı,yazılım analisti tarafından üretilen tasarımı uygulayan programları yazm aklasorumludur. Bu ayrımı yaptıktan sonra, tekrar belirtmeliyiz ki, bu konud a hiçbir

Elektrik ve Elektronik Mühendisleri Enstitüsü

Elektrik ve Elektronik Mühendisleri Enstitüsü (IEEE, “i-triple-e” olarak telaff uz edilir), elektrik, elektronik ve imalat mühendislerinden oluşan bir organ izasyondur. 1963 yılında, yirmi beş elektrik mühendisinin (Thomas Edison dahil) 1884 yılında kurduğu Amerikan Elektrik Mühendisleri Enstitüsü ile 1 912 yılında kurulan Radyo Mühendisleri Enstitüsü'nün birleşmesi sonucu o luşmuştur. Bugün, IEEE'nin operasyon merkezi New Jersey, Piscataway'de bulunmaktadır. Enstitü, Havacılık ve Elektronik Sistemleri Derneği, Lazerle r ve Elektro-Optik Derneği, Robotik ve Otomasyon Derneği, Araç Teknoloji si Derneği ve Bilgisayar Derneği gibi birçok teknik topluluğu içermektedir. IEEE'nin faaliyetleri arasında standartların geliştirilmesi de bulunmaktadır . Örneğin, IEEE'nin çabaları, günümüz bilgisayarlarının çoğunda kullanılan tek hassasiyetli kayan nokta ve çift hassasiyetli kayan nokta standartlarını n (Bölüm 1'de tanıtılan) ortaya çıkmasına yol açmıştır.

IEEE'nin web sayfasını `http://www.ieee.org`, IEEE Bilgisayar Derneği 'nin web sayfasını `http://www.computer.org` ve IEEE'nin Etik Kuralları'nı `http://www.ieee.org/about/whatis/code.html` adresinde bulabilirsiniz.

Doclingo https://www.doclingo.ai

terminolojinin bilgisayar topluluğu genelinde kullanımını kontrol eden merkezi otorite. Yazılım analisti unvanını taşıyan birçok kişi esasen pro-gramcıdır ve pro gramcı (ya da belki kıdemli programcı) unvanına sahip olanların birçoğu, terimi n tam anlamıyla yazılım analistidir. Bu terminoloji bulanıklığı, günümüzde yazılı m geliştirme sürecindeki adımların genellikle iç içe geçmiş olmasından kaynakl anmaktadır, bunu yakında göreceğiz.

Test Etme Geçmişteki geleneksel geliştirme aşamasında, test etme esasen pr ogramların hata ayıklama süreciyle ve nihai yazılım ürününün yazılım gereks inimleri spesifikasyonu ile uyumlu olduğunu doğrulamakla eşitleniyordu. Ancak, bugün bu test etme vizyonu çok dar olarak kabul edilmektedir. Progr amlar, yazılım geliştirme sürecinde test edilen tek nesneler değildir. Gerçekten de, tüm geliştirme sürecindeki her ara adımın sonucu "doğruluk" açı sından "test edilmelidir." Dahası, Bölüm 7.6'da göreceğimiz gibi, test etme artık kalite güvencesi için genel mücadelede yalnızca bir segment olarak tanınmakt adır; bu, tüm yazılım yaşam döngüsünü kapsayan bir hedeftir. Bu nedenle, birç ok yazılım mühendisi test etmenin artık yazılım geliştirmede ayrı bir adım olara k görülmemesi gerektiğini, bunun yerine, ve birçok tezahürüyle birlikte, diğer a dımlara dahil edilmesi gerektiğini savunmaktadır; bu, "gereksinim analizi ve on ayı," "tasarım *ve doğrulama*," ve "uygulama *ve test etme*" gibi isimlere sahip ola bilecek üç aşamalı bir geliştirme süreci üretmektedir.Ne yazık ki, modern k alite güvencesi tekniklerine rağmen, büyük yazılım sistemleri önemli testler den sonra bile hatalar içermeye devam etmektedir. Bu hataların birçoğu siste min ömrü boyunca tespit edilemeyebilir, ancak diğerleri büyük arızalara neden olabilir. Bu tür hataların ortadan kaldırılması, yazılım mühendisliğinin hedefleri nden biridir. Hala yaygın olmaları, yapılacak çok fazla araştırma olduğunu göst ermektedir.

Sorular & Alıştırmalar

1. Yazılım yaşam döngüsünün geliştirme aşaması, bakım aşamasını na sıl etkiler?
2. Yazılım yaşam döngüsünün geliştirme aşamasındaki dört aşamayı (g ereksinim analizi, tasarım, uygulama ve test) özetleyin.
3. Yazılım gereksinimleri spesifikasyonunun rolü nedir?

7.3 Yazılım Mühendisliği Yöntemleri

Yazılım mühendisliğine yönelik erken yaklaşımlar, gereksinim analizi, tasarım, uygul ama ve testin kesin bir sıralama içinde gerçekleştirilmesi gerektiğini savunuyordu. Büyük bir yazılım sisteminin geliştirilmesi sırasında çok fazla risk olduğu inancı, d eğişikliklere izin vermemeyi gerektiriyordu. Sonuç olarak, yazılım mühendisleri, ta sarıma başlamadan önce sistemin tüm gereksinim spesifikasyonunun tamamlan masını ve benzer şekilde, uygulamaya başlamadan önce tasarımın tamamlanması nı ısrarla talep ettiler. Sonuç olarak, geliştirme süreci artık şelale modeli olarak adl andırılmaktadır; bu, geliştirme sürecinin yalnızca bir yönde akmasına izin verildiği gerçeğine bir benzetmedir.

Doclingo https://www.doclingo.ai

Son yıllarda, yazılım mühendisliği teknikleri, şelale modelinin dayattığı s on derece yapılandırılmış ortam ile yaratıcı problem çözme için genellikle ha yati olan "serbest" deneme-yanılma süreci arasındaki çelişkiyi yansıtacak şe kilde değişmiştir. Bu, yazılım geliştirme için artımlı modelin ortaya çıkmasıyl a gösterilmektedir. Bu modele göre, istenen yazılım sistemi, ilk olarak sınırlı işlevselliğe sahip nihai ürünün basitleştirilmiş bir versiyonu olan artımlarla i nşa edilir. Bu versiyon test edildikten ve belki de gelecekteki kullanıcı tarafın dan değerlendirildikten sonra, daha fazla özellik eklenir ve sistem tamamla nana kadar artımlı bir şekilde test edilir. Örneğin, geliştirilen sistem bir hast ane için hasta kayıtları sistemi ise, ilk artım yalnızca tüm kayıt sisteminin küç ük bir örneğinden hasta kayıtlarını görüntüleme yeteneğini içerebilir.
O sürüm çalışmaya başladığında, kayıt ekleme ve güncelleme gibi ek özellikl er adım adım eklenecektir.

Katı bir şekilde şelale modeline bağlılıktan uzaklaşmayı temsil eden bir d iğer model, yinelemeli modeldir; bu model, aslında bazen artımlı modelle eş itlenmesine rağmen, ikisi de farklıdır. Artımlı model, bir ürünün her ön versi yonunu daha büyük bir versiyona genişletme kavramını taşırken, yinelemeli model her versiyonu rafine etme kavramını kapsamaktadır. Gerçekte, artıml ı model altında yatan bir yinelemeli süreç içerir ve yinelemeli model, özellikl eri artımlı olarak ekleyebilir.

Yinelemeli tekniklerin önemli bir örneği, Rational Software Corporation tar afından oluşturulan ve şu anda IBM'in bir bölümü olan rational unified process (RUP, "cup" ile kafiyeli) modelidir. RUP, yazılım yaşam döngüsünün geliştirme a şamasındaki adımları yeniden tanımlayan ve bu adımları gerçekleştirmek için kı lavuzlar sağlayan bir yazılım geliştirme paradigmasıdır. Bu kılavuzlar, onları de steklemek için CASE araçlarıyla birlikte IBM tarafından pazarlanmaktadır. Bugü n, RUP yazılım endüstrisinde yaygın olarak uygulanmaktadır. Aslında, popülarit esi, ticari olmayan bir temel üzerinde mevcut olan unified process adlı bir mülki yetsiz versiyonun geliştirilmesine yol açmıştır.

Artımlı ve yinelemeli modeller, yazılım geliştirmedeki prototipleme eğili minden yararlanır; burada, önerilen sistemin tamamlanmamış versiyonları, prototipler olarak adlandırılır, inşa edilir ve değerlendirilir. Artımlı model du rumunda, bu prototipler, tamamlanmış, nihai sisteme evrilir - bu işleme evri msel prototipleme denir. Daha yinelemeli bir durumda, prototipler, nihai ta sarımın yeni bir uygulaması lehine atılabilir. Bu yaklaşım, atıl prototipleme olarak bilinir. Genellikle bu atıl kategoriye giren bir örnek, hızlı prototipleme dir; burada, önerilen sistemin basit bir örneği, geliştirme sürecinin erken aş amalarında hızlı bir şekilde inşa edilir. Böyle bir prototip, sistemin kullanıcıla rıyla nasıl etkileşimde bulunacağını ve hangi yeteneklere sahip olacağını gö steren sadece birkaç ekran görüntüsünden oluşabilir. Amaç, ürünün çalışan bir versiyonunu üretmek değil, yazılım geliştirme sürecinde yer alan taraflar arasında iletişimi netleştirmek için kullanılabilecek bir gösterim aracı elde e tmektir.Örneğin, hızlı prototipler, gereksinim analizi sırasında sistem gereks inimlerini netleştirmede veya potansiyel müşterilere satış sunumları sırasın da yardımcı olarak avantajlı olduğunu kanıtlamıştır.

Daha az resmi bir biçimi, yıllardır bilgisayar meraklıları/hobici ler tarafında n kullanılan artımlı ve yinelemeli fikirlerin, açık kaynak geliştirme olarak bilinir. Bu, günümüzün ücretsiz yazılımlarının büyük bir kısmının üretildiği yöntemdir. Belki de en belirgin örnek, açık kaynak geliştirmesinin başlangıçta Linus Tor valds tarafından yönetildiği Linux işletim sistemidir. Açık kaynak

Doclingo https://www.doclingo.ai

bir yazılım paketinin geliştirilmesi şu şekilde ilerler: Tek bir yazar, yazılımın (gen ellikle kendi ihtiyaçlarını karşılamak için) ilk sürümünü yazar ve kaynak kodunu ve belgelerini İnternet'te yayınlar. Buradan indirilip başkaları tarafından ücretsi z olarak kullanılabilir. Bu diğer kullanıcılar kaynak koduna ve belgelere sahip ol dukları için, yazılımı kendi ihtiyaçlarına uyacak şekilde değiştirebilir veya buldu kları hataları düzeltebilirler. Bu değişiklikleri orijinal yazara bildirirler, yazar da bunları yazılımın yayınlanan sürümüne dahil eder ve bu genişletilmiş sürümü d aha fazla değişiklik için kullanılabilir hale getirir. Pratikte, bir yazılım paketinin t ek bir hafta içinde birkaç uzantı ile evrim geçirmesi mümkündür.

Belki de şelale modelinden en belirgin kayma, her birinin artımlı bir tem elde erken ve hızlı uygulama, değişen gereksinimlere yanıt verme ve titiz ge reksinim analizi ve tasarımına daha az vurgu önerdiği çevik yöntemler olara k bilinen metodolojilerin topluluğu tarafından temsil edilmektedir. Çevik bir yöntem örneği, aşırı programlama (XP) dir.

XP modeline göre, yazılım, fikirleri serbestçe paylaştıkları ve geliştirme proje sinde birbirlerine yardımcı oldukları bir ortak çalışma alanında çalışan onda n az bireyden oluşan bir ekip tarafından geliştirilir. Yazılım, günlük döngüler halinde gayri resmi gereksinim analizi, tasarım, uygulama ve test etme yolu yla kademeli olarak geliştirilir. Böylece, yazılım paketinin yeni genişletilmiş s ürümleri düzenli olarak ortaya çıkar ve her biri projenin paydaşları tarafında n değerlendirilebilir ve daha fazla artışa işaret etmek için kullanılabilir. Özetl e, çevik yöntemler esneklik ile karakterize edilir; bu, yöneticilerin ve progra mcıların bireysel ofislerde çalıştığı ve genel yazılım geliştirme görevinin iyi t anımlanmış bölümlerini katı bir şekilde yerine getirdiği şelale modeline zıt b ir görüntü çizer.

Şelale modeli ile XP'yi karşılaştırarak tasvir edilen zıtlıklar, yazılım geliştirme sürecine uygulanan metodolojilerin genişliğini ortaya koyar; bu, güvenilir yazılı mları verimli bir şekilde inşa etmenin daha iyi yollarını bulma umuduyla yapılm aktadır. Bu alandaki araştırmalar devam eden bir süreçtir. İlerleme kaydedilme ktedir, ancak yapılacak çok iş vardır.

Sorular & Alıştırmalar

1. Geleneksel şelale modeli ile daha yeni kademeli ve yinelemeli paradi gmalar arasındaki ayrımı özetleyin.
2. Üç geliştirme paradigmasını tanımlayın, bu paradigmalardan biri katı bir şekilde şelale modeline bağlılıktan uzaklaşmayı temsil etmektedir.
3. Geleneksel evrimsel prototipleme ile açık kaynak geliştirme arasınd aki fark nedir?
4. Açık kaynak metodolojisiyle geliştirilen yazılımların sahiplik hakları açısında n hangi potansiyel sorunların ortaya çıkabileceğini düşünüyorsunuz?

7.4 Modülerlik

7.2. Bölümdeki önemli bir nokta, yazılımı değiştirmek için programı veya en azından programın ilgili kısımlarını anlamanın gerekliliğidir. Böyle bir anlayışa sahip olmak, k üçük programlar söz konusu olduğunda genellikle yeterince zorlayıcıdır ve bu da

Doclingo https://www.doclingo.ai

büyük yazılım sistemleriyle uğraşırken neredeyse imkansızdır eğer modülerlik—yani, yazılımın yönetilebilir birimlere, genel olarakmodüllerolarak adlandırılan parçalara bölünmesi, her birinin yazılımın genel sorumluluğunun yalnızca bir kısmıyla ilgile nmesidir.

Modüler Uygulama

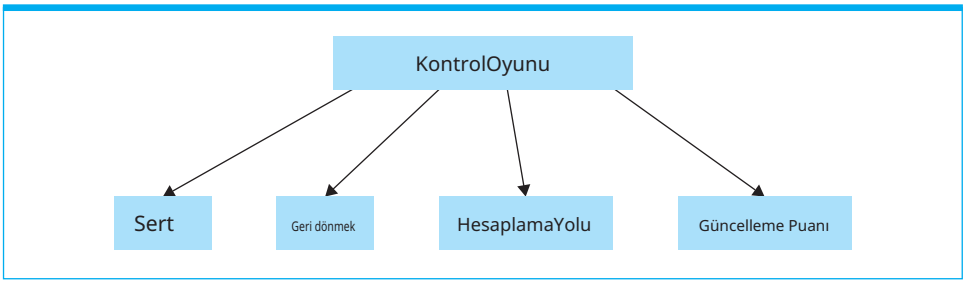
Modüller çeşitli biçimlerde gelir. Zaten gördüğümüz gibi (Bölüm 5 ve 6),zorl ayıcı paradigmada modüller prosedürler olarak görünür. Buna karşılık, nes ne yönelimli paradigma nesneleri temel modüler bileşenler olarak kullanır. Bu ayrımlar önemlidir çünkü yazılım tasarım sürecinin başlangıcındaki teme l hedefi belirler. Hedef, genel görevi bireysel, yönetilebilir süreçler olarak mı temsil etmek yoksa sistemdeki nesneleri tanımlayıp nasıl etkileştiklerini mi anlamaktır?

Örnek vermek gerekirse, basit bir modülerprogramın tenis oyununu sim üle etmek için nasıl geliştirileceğiniimperatif ve nesne yönelimli paradigmala r açısından ele alalım. Imperatif paradigmada,yapılması gereken eylemleri d üşünerek başlarız. Her voleybolun bir oyuncunun topuservis etmesiyle başla dığı için, topun başlangıç hızını ve yönünü hesaplayacakbir `Serve` adlı prose dürü düşünerek başlayabiliriz. Sonra topun yolunubelirlememiz gerekecek. (Ağ ile çarpacak mı? Nerede sekecek?) Bu hesaplamalarıyapmak için `ComputePath` adlı başka bir prosedür oluşturmayı planlayabiliriz.Sonraki adı m, diğer oyuncunun topu geri döndürüp döndüremeyeceğini belirlemekolab ilir ve eğer döndürebiliyorsa, topun yeni hızını ve yönünü hesaplamamızgere kecek. Bu hesaplamaları `Return` adlı bir prosedürde yapmayı planlayabiliriz.

Bu şekilde devam ederek, Şekil 7.3'te gösterilen yapı şeması ile gösterilen modüler yapıya ulaşabiliriz; burada prosedürler dikdörtgenlerle ve prosedür bağımlılıkları (prosedür çağrılarıyla uygulanır) oklarla temsil edilir. Özellikle, şema tüm oyunun ControlGame adlı bir prosedür tarafından izlendiğini ve görevini yerine getirmek için ControlGame'nin Serve, Return, ComputePath ve UpdateScore prosedürlerinin hizmetlerinden yararlandığını belirtmektedir.

Yapı şemasının her prosedürün görevini nasıl yerine getireceğini belirtmed iğini unutmayın. Bunun yerine, yalnızca prosedürleri tanımlar ve prosedürler ar asındaki bağımlılıkları gösterir. Gerçekte, ControlGame görevini yerine getirm ek için önce `Serve` prosedürünü çağırarak, ardından `ComputePath` ve `Return` prosedürlerini tekrar tekrar çağırarak birinin bir hata bildirmesini bekleyebilir v e nihayetinde `UpdateScore` hizmetlerini çağırarak tüm süreci tekrar ederek `Serve`'yi yeniden çağırabilir.

Şekil **7.3** Basit bir yapı şeması



Doclingo https://www.doclingo.ai

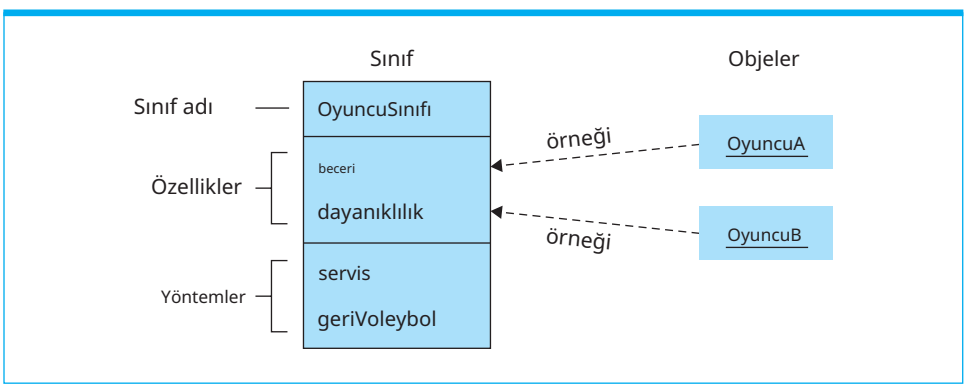
Bu aşamada yalnızca istenen programın çok basit bir taslağını elde ettik, ancak amacımız zaten ortaya konmuştur. Zorunlu paradigma gereği, progra mı gerçekleştirilmesi gereken etkinlikleri dikkate alarak tasarladık ve bu ned enle modüllerin prosedürler olduğu bir tasarım elde ettik.

Şimdi programın tasarımını yeniden gözden geçirelim—bu sefer nesne y önelimli paradigma bağlamında. İlk düşüncemiz, iki oyuncunun `PlayerA` ve `PlayerB` olarak iki nesne ile temsil edilmesi gerektiği olabilir. Bu nesneler ay nı işlevselliğe sahip olacak ancak farklı özelliklere sahip olacaklar. (Her ikisi d e voleybol atabilir ve geri dönebilir, ancak bunu farklı beceri ve güçle yapabil irler.) Böylece, bu nesneler aynı sınıfın örnekleri olacaktır. (Bölüm 6'da bir sın ıf kavramını tanıttığımızı hatırlayın: her nesne ile ilişkilendirilecek prosedürle ri (metotlar olarak adlandırılır) ve özellikleri (örnek değişkenleri olarak adlan dırılır) tanımlayan bir şablon.) Bu sınıf, `PlayerClass` olarak adlandıracağımı z, oyuncunun karşılık gelen eylemlerini simüle eden `serve` ve `return` metotl arını içerecektir. Ayrıca, oyuncunun özelliklerini yansıtan `skill` ve `endurance` gibi nitelikleri de içerecektir. Şu ana kadar tasarımımız Şekil 7.4't e gösterilmektedir. Orada `PlayerA` ve `PlayerB` sınıfın örnekleri olarak görülmektedir. `PlayerClass` ve bu sınıfın `skill` ve`endurance` özelliklerini içerdiğini, ayrıca `serve` ve`returnVolley` yöntemlerini içerdiğini belirtir. (Burada Şekil 7.4'te nesne adlarını sınıf adlarından ayırt etmek için altını çizdiğimizi unutmayın.)

Sonraki adımda, oyuncuların gerçekleştirdiği eylemlerin yasal olup olmadığını belirleyen resmi bir nesneye ihtiyacımız var. Örneğin, servis fileyi aşıp kortun uygun alanına düştü mü? Bu amaçla, Judge adında bir nesne oluşturabiliriz ve bu nesne evaluateServe ve evaluateReturn yöntemlerini içerebilir. Eğer Judge nesnesi bir servisi veya dönüşü kabul edilebilir olarak değerlendirirse, oyun devam eder. Aksi takdirde, Judge sonuçları kaydetmek için Score adında başka bir nesneye mesaj gönderir.

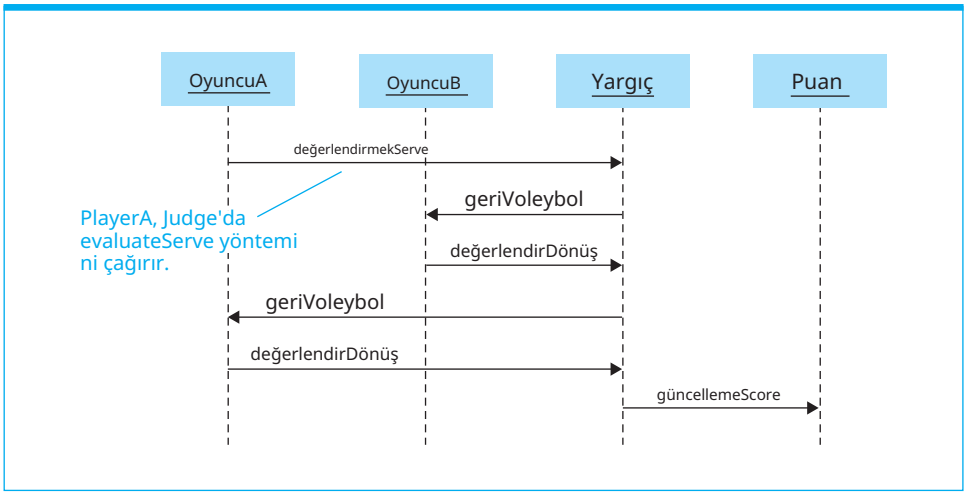
Bu noktada tenis programımızın tasarımı dört nesneden oluşmaktadır: `PlayerA`,`PlayerB`,`Judge` ve `Score`. Tasarımımızı netleştirmek için, Şekil 7.5'te gösterildiği gibi bir voley sırasında meydana gelebilecek olayların sırasını düşünelim; burada yer alan nesneleri dikdörtgenler olarak temsil ettik. Şekil, bu nesneler arasındaki iletişimi PlayerA nesnesi içindeki serve yöntemini çağırmanın sonucu olarak sunmayı amaçlamaktadır. Olaylar, şeklin aşağısına doğru ilerledikçe kronolojik olarak görünmektedir. İlk yatay okla gösterildiği gibi, `PlayerA` servisini `Judge` nesnesine `evaluateServe` yöntemini çağırarak bildirir. Bu

Şekil 7.4 PlayerClass'ın yapısı ve örnekleri



Doclingo https://www.doclingo.ai

Şekil **7.5** OyuncuA'nın servisinden kaynaklanan nesneler arasındaki etkileşim



Hakim daha sonra servisin iyi olduğunu belirler ve `PlayerB`'den geri dönmesini ister `PlayerB`'nin `returnVolley` metodunu çağırarak. Voleybol, Hakim`PlayerA`'nın hata yaptığını belirlediğinde sona erer ve `Score` nesnesinden sonuçları kaydetmesini ister.

İhtimal örneğimizde olduğu gibi, nesne yönelimli programımız bu aşamada oldukça basittir. Ancak, nesne yönelimli paradigmanın nasıl modüler bir tasarıma yol açtığını görmek için yeterince ilerledik; bu tasarımda temel bileşenler nesnelerdir.

Bağlama

Modülerliği yönetilebilir yazılımlar üretmenin bir yolu olarak tanıttık. Fikir, gelecekteki herhangi bir değişikliğin muhtemelen yalnızca birkaç modüle uygulanacağıdır; bu, değişikliği yapan kişininsistemin o kısmına odaklanmasını sağlar, tüm paketle uğraşmak yerine. Bu, elbette,bir modüldeki değişikliklerin sistemdeki diğer modülleri bilmeden etkilemeyeceği varsayımına bağlıdır. Sonuç olarak, bir modüler sistem tasarlarken bir hedef,modüller arasındaki bağımsızlığı maksimize etmek olmalıdır veya diğer bir deyişle,modüller arasındaki bağlantıyı (intermodule coupling olarak bilinir) minimize etmektir. Gerçekten de, bir yazılım sisteminin karmaşıklığını ölçmek için kullanılan bir ölçüt,yazılımın bakım maliyetini tahmin etmenin bir yolu olarak, intermodule couplin g'i ölçmektir.

Modüller arası bağlanma birkaç biçimde gerçekleşir. Bunlardan biri kontrol bağlanmasıdır, bu, bir modülün başka birine yürütme kontrolünü devretmesi durumunda meydana gelir, örneğin bir prosedür çağrısında. Şekil 7.3'teki yapı şeması, prosedürler arasındaki mevcut kontrol bağlanmasını temsil etmektedir. Özellikle, `ControlGame` modülünden `Serve` modülüne giden ok, birincisinin ikincisine kontrol devrettiğini gösterir. Ayrıca, nesneden nesneye geçerken kontrolün yolunu izleyen okların yer aldığı Şekil 7.5'te de kontrol bağlanması temsil edilmektedir.

Modüller arası bağlanmanın bir diğer biçimi veri bağlanmasıdır, bu, modüller arasında veri paylaşımını ifade eder. Eğer iki modül aynı veri öğesi ile etkileşime giriyorsa, o zaman bir modülde yapılan değişiklikler diğerini etkileyebilir ve verinin kendisinin formatındaki değişiklikler her iki modülde de sonuçlar doğurabilir.

Doclingo https://www.doclingo.ai

Prosedürler arasındaki veri bağlanması iki biçimde gerçekleşebilir. Birinci, verilerin bir prosedürden diğerine parametreler biçiminde açıkça geçirilmesidir. Bu tür bir bağlanma, prosedürler arasında veri iletildiğini belirtmek için etiketlenmiş bir ok ile yapı şemasında temsil edilir. Okun yönü, öğenin hangi yönde transfer edildiğini gösterir. Örneğin, Şekil 7.6, prosedürün belirtildiği Şekil 7.3'ün genişletilmiş bir versiyonudur.
`ControlGame`, `Serve` prosedürünü çağırdığında hangi oyuncunun özellikleri nin simüle edileceğini söyleyecek ve `Serve` prosedürü, `Serve` görevini tamamladığında topun hareketini `ControlGame`'ye bildirecektir.

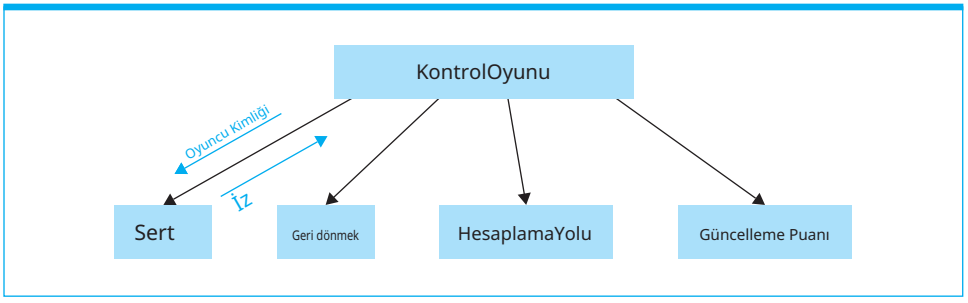
Nesne yönelimli bir tasarımda nesneler arasında benzer veri bağlanması gerçekleşir. Örneğin, `PlayerA` nesnesi `Judge` nesnesine servisinin değerlendirilmesini istediğinde (bkz. Şekil 7.5), hareket bilgilerini `Judge`'ye iletmelidir. Öte yandan, nesne yönelimli paradigmanın faydalarından biri, nesneler arasındaki veri bağlanmasını en aza indirmeye eğilimli olmasıdır. Bunun nedeni, bir nesne içindeki yöntemlerin, nesnenin iç verilerini manipüle eden tüm prosedürleri içermeye eğilimli olmasıdır. Örneğin, `PlayerA` nesnesi, o oyuncunun özellikleri yle ilgili bilgileri ve o bilgiyi gerektiren tüm yöntemleri içerecektir. Dolayısıyla, o bilgiyi diğer nesnelere iletme ihtiyacı yoktur ve böylece nesneler arası veri bağlanması en aza indirilir.

Verileri açıkça parametreler olarak iletmek yerine, veriler, sistem genelinde tüm modüllere otomatik olarak erişilebilen global data biçiminde modüller arasında örtük olarak paylaşılabilir; bu, yalnızca belirli bir modül içinde erişilebilen yerel veri öğelerine karşıdır. Çoğu yüksek seviyeli dil, hem global hem de yerel verilerin uygulanması için yollar sağlar, ancak global verilerin kullanımı dikkatle yapılmalıdır. Sorun, global verilere bağımlı bir modülü değiştirmeye çalışan bir kişinin, söz konusu modülün diğer modüllerle nasıl etkileşimde bulunduğunu tanımlamakta zorlanabilmesidir. Kısacası, global verilerin kullanımı, modülün soyut bir araç olarak faydasını azaltabilir.

Bağlılık

Modüller arasındaki bağımlılığı en aza indirmek kadar önemli olan, her modül içindeki içsel bağları maksimize etmektir. Kohezyon terimi, bu içsel bağlanmayı veya diğer bir deyişle, bir modülün iç parçalarının ilişkili olma derecesini ifade eder. Kohezyonun önemini takdir edebilmek için, bir sistemin başlangıç geliştirmesinin ötesine bakmalı ve tüm yazılım yaşam döngüsünü göz önünde bulundurmalıyız. Bir modülde değişiklik yapma gereği doğarsa, içindeki çeşitli etkinliklerin varlığı, aksi takdirde basit bir süreci karmaşık hale getirebilir. Bu nedenle, düşük modüller arası bağımlılık arayışının yanı sıra, yazılım tasarımcıları yüksek iç modül kohezyonu için çaba gösterirler.

Şekil **7.6** Veri bağımlılığını içeren bir yapı şeması



Doclingo https://www.doclingo.ai

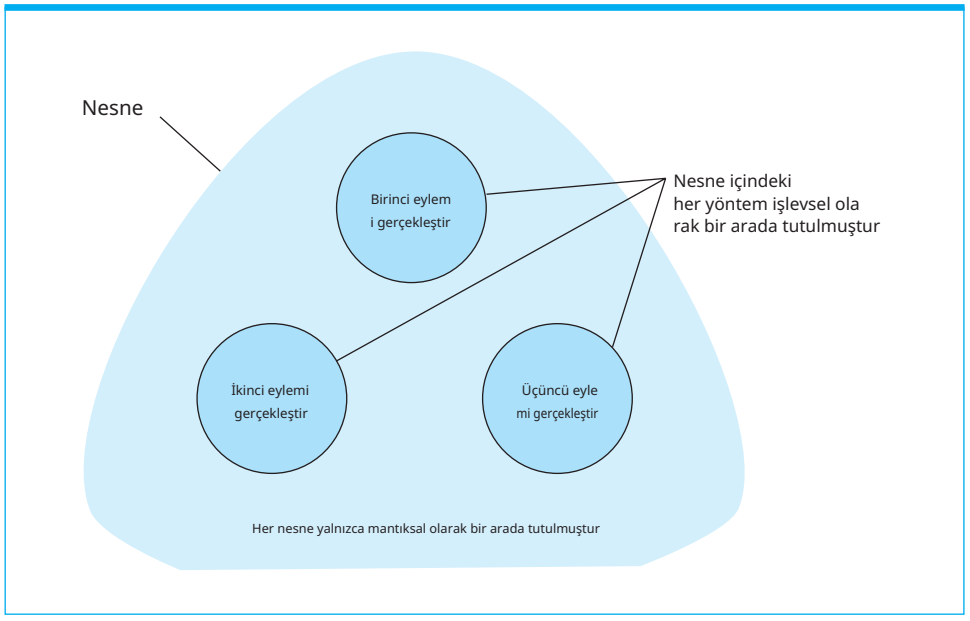
İçindeki etkinliklerin çeşitliliği, aksi takdirde basit bir süreç olacak olanı karmaşık hale getirebilir. Bu ne denle, düşük modüller arası bağımlılık arayışının yanı sıra, yazılım tasarımcıları yüksek iç modül kohez yonu için çaba gösterirler.

Kohezyonun zayıf bir biçimi mantıksal kohezyon olarak bilinir. Bu, bir m odül içindeki içsel unsurların mantıksal olarak benzer nitelikteki etkinlikler g erçekleştirmesi nedeniyle oluşan kohezyondur. Örneğin, bir sistemin dış dün ya ile olan tüm iletişimini gerçekleştiren bir modülü düşünün. Böyle bir mod ülü bir arada tutan "yapıştırıcı", modül içindeki tüm etkinliklerin iletişimle ilgil i olmasıdır. Ancak, iletişimin konuları büyük ölçüde değişebilir. Bazıları veri el de etmeyi ele alırken, diğerleri sonuçları raporlamayı ele alır.

Daha güçlü bir kohezyon biçimi, işlevsel kohezyon olarak bilinir; bu, modülün tü m parçalarının tek bir aktivitenin performansına odaklandığı anlamına gelir. İkna edi ci bir tasarımda, işlevsel kohezyon genellikle alt görevleri diğer modüllerde izole ede rek artırılabilir ve bu modüller daha sonra soyut araçlar olarak kullanılabilir. Bunun, modül `ControlGame`'ün diğer modülleri soyut araçlar olarak kullandı ğı tenis simülasyonu örneğimizde (bkz. tekrar Şekil 7.3) gösterilmektedir; bö ylece oyunu denetlemeye odaklanabilir ve servis verme, geri alma ve skoru k oruma detaylarıyla dikkatinin dağılmasını önleyebilir.

Nesne yönelimli tasarımlarda, tüm nesneler genellikle yalnızca mantıksal olarak koheziftir çünkü bir nesne içindeki yöntemler genellikle gevşek bir şekilde ilişkili akti viteleri gerçekleştirir—tek ortak bağ, bunların aynı nesne tarafından gerçekleştirilen aktiviteler olmasıdır. Örneğin, tenis simülasyonu örneğimizde, her oyuncu nesnesi h em servis yapma hem de topu geri alma yöntemlerini içerir; bu, önemli ölçüde farklı aktivitelerdir. Bu nedenle, böyle bir nesne yalnızca mantıksal olarak kohezif bir mod ül olacaktır. Ancak, yazılım tasarımcıları, bir nesne içindeki her bir bireysel yöntemi iş levsel olarak kohezif hale getirmeye çalışmalıdır. Yani, nesne bütünüyle yalnızca man tıksal olarak kohezif olsa da, bir nesne içindeki her yöntem yalnızca bir işlevsel olarak kohezif görev gerçekleştirmelidir (Şekil 7.7).

Şekil **7.7** Nesne içindeki mantıksal ve işlevsel bütünlük



Doclingo https://www.doclingo.ai

Bilgi Gizleme

İyi modüler tasarımın köşe taşlarından biri, yazılım sisteminin belirli bir bölü müne bilgi kısıtlamasını ifade eden bilgi gizleme kavramında yakalanmıştır. Burada bilgi terimi, bir program biriminin yapısı ve içeriği hakkında herhangi bir bilgiyi kapsayan geniş bir anlamda yorumlanmalıdır. Bu nedenle, verileri, kullanılan veri yapılarının türünü, kodlama sistemlerini, bir modülün iç yapıs al bileşimini, bir prosedürel birimin mantıksal yapısını ve bir modülün iç özell ikleriyle ilgili diğer faktörleri içerir.

Bilgi gizlemenin amacı, modüllerin diğer modüller üzerinde gereksiz bağ ımlılıklara veya etkilere sahip olmasını engellemektir. Aksi takdirde, bir modü lün geçerliliği, diğer modüllerin geliştirilmesindeki hatalar veya yazılım bakı mındaki yanlış yönlendirilmiş çabalar nedeniyle tehlikeye girebilir. Örneğin, bir modül, iç verilerinin diğer modüller tarafından kullanılmasını kısıtlamıyor sa, o veriler diğer modüller tarafından bozulabilir. Ya da, bir modül, diğerinin iç yapısından yararlanacak şekilde tasarlanmışsa, o iç yapı değişirse daha so nra işlev bozukluğu yaşayabilir.

Bilgi gizlemenin iki biçimi olduğunu belirtmek önemlidir—biri bir tasarım hedefi, diğeri ise bir uygulama hedefidir. Bir modül, diğer modüllerin iç bilgi lerine erişim ihtiyacı duymayacak şekilde tasarlanmalıdır ve bir modül, sınırla rını güçlendiren bir şekilde uygulanmalıdır. İlkine örnekler, uyumu maksimiz e etmek ve bağlantıyı minimize etmektir. İkincisine örnekler, yerel değişkenl erin kullanımı, kapsülleme uygulamak ve iyi tanımlanmış kontrol yapıları kull anmaktır.

Son olarak, bilgi gizlemenin soyutlama teması ve soyut araçların kullanımı açısından merkezi olduğunu belirtmeliyiz. Gerçekten de, soyut bir aracın kavra mı, kullanıcısının iç özelliklerini göz ardı edebileceği bir “kara kutu”dur ve bu da kullanıcının mevcut daha büyük uygulamaya odaklanmasını sağlar. Bu anlamd a, bilgi gizleme, soyut aracı, karmaşık ve potansiyel olarak tehlikeli elektronik e kipmanları korumak için kullanılabilecek bir müdahale geçirmez muhafaza gibi mühürleme kavramıyla örtüşmektedir. Her ikisi de kullanıcılarını içteki tehlikele rden korurken, içlerini de kullanıcılarının müdahalesinden korur.

Bileşenler

Daha önce yazılım mühendisliği alanındaki bir engelin, büyük yazılım sistemleri nin inşa edilebileceği önceden üretilmiş “raf ürünleri” yapı taşlarının eksikliği ol duğunu belirtmiştik. Modüler yazılım geliştirme yaklaşımı bu konuda umut vaa t etmektedir. Özellikle, nesne yönelimli programlama paradigması, nesnelerin çevreleriyle net bir şekilde tanımlanmış arayüzlere sahip, tamamlanmış, kendi kendine yeterli birimler oluşturması nedeniyle özellikle faydalı olmaktadır.

Bir nesne veya daha doğru bir ifadeyle bir sınıf, belirli bir rolü yerine getirmek üzere tasarlandı ğında, bu rolü o hizmeti gerektiren herhangi bir programda yerine getirmek için kullanılabilir. Ayrıca, miras alma, tanımların belirli bir uygulamanın ihtiyaçlarına uyacak şekilde özelleştirilmesi gerektiği durumlarda, önceden hazırlanmış nesne tanımlarını rafi ne etmenin bir yolunu sağlar.

Bu nedenle, nesne yönelimli programlama dilleri C++, Java ve C#'nın, progr amcıların belirli rolleri yerine getirmek için nesneleri kolayca uygulayabileceği önceden hazırlanmış "şablonlar" koleksiyonlarıyla birlikte olması şaşırtıcı değild ir. Özellikle, C++ C++ Standart Şablon Kütüphanesi ile ilişkilendirilmiştir,

Doclingo https://www.doclingo.ai

Java programlama ortamı, Java Uygulama Programlama Arayüzü (API) ile birl ikte gelir ve C# programcıları .NET Framework Sınıf Kütüphanesine erişim sa ğlar.

Objelerin ve sınıfların yazılım tasarımı için önceden üretilmiş yapı taşları sağla ma potansiyeline sahip olması, onların ideal olduğu anlamına gelmez. Bir sorun, i nşa etmek için sağladıkları blokların nispeten küçük olmasıdır. Böylece, bir nesne aslında, tanım gereği, yeniden kullanılabilir bir yazılım birimi olan daha genel bir bileşen kavramının özel bir durumudur. Pratikte, çoğu bileşen nesne yönelimli pa radigma üzerine kuruludur ve kendine yeterli bir birim olarak işlev gören bir veya daha fazla nesnenin koleksiyonu biçimindedir.

Bileşenlerin geliştirilmesi ve kullanımı üzerine yapılan araştırmalar, gelen eksel programcı rolünün, yazılım sistemlerini önceden üretilmiş bileşenlerde n inşa eden bir bileşen montajcısı tarafından değiştirildiği, bileşen mimarisi olarak bilinen yeni bir alanın ortaya çıkmasına yol açmıştır. Bu bileşenler, bir çok geliştirme ortamında grafiksel bir arayüzde simgeler olarak görüntülenir . Bileşenlerin iç programlamasıyla ilgilenmek yerine, bir bileşen montajcısını n metodolojisi, önceden tanımlanmış bileşenler koleksiyonundan ilgili bileşe nleri seçmek ve ardından istenen işlevselliği elde etmek için minimum özelle ştirme ile bunları bağlamaktır. Gerçekten de, iyi tasarlanmış bir bileşenin bir özelliği, iç değişiklikler olmadan belirli bir uygulamanın özelliklerini kapsayac ak şekilde genişletilebilmesidir.

Bileşen mimarilerinin verimsiz bir zemin bulduğu bir alan akıllı telefon si stemleridir. Bu cihazların kaynak kısıtlamaları nedeniyle, uygulamalar aslınd a her biri uygulama için bazı belirgin işlevler sağlayan bir dizi işbirliği yapan bileşendir. Örneğin, bir uygulama içindeki her ekran genellikle ayrı bir bileşe ndir. Sahne arkasında, bir bellek kartında bilgi depolamak ve erişmek, bazı s ürekli işlevler (örneğin müzik çalmak) gerçekleştirmek veya bilgiye erişmek iç in başka hizmet bileşenleri bulunabilir.

Gerçek Dünyada Yazılım Mühendisliği

Aşağıdaki senaryo, gerçek dünya yazılım mühendislerinin karşılaştığı sorunlar ın tipik bir örneğidir. XYZ Şirketi, şirketin veri işleme ihtiyaçlarını karşılamak içi n şirket genelinde entegre bir yazılım sistemi geliştirmek ve kurmak üzere bir yazılım mühendisliği firmasıyla anlaşır. XYZ Şirketi tarafından üretilen sistemi n bir parçası olarak, çalışanların şirket genelindeki sisteme erişimini sağlamak için bir PC ağı kullanılır. Böylece her çalışan masasında bir PC bulur. Kısa süre içinde bu PC'ler yalnızca yeni veri yönetim sistemine erişim sağlamakla kalma z, aynı zamanda her çalışanın verimliliğini artırdığı özelleştirilebilir araçlar olar ak da kullanılır. Örneğin, bir çalışan, o çalışanın görevlerini kolaylaştıran bir h esap tablosu programı geliştirebilir. Ne yazık ki, bu tür özelleştirilmiş uygulam alar iyi tasarlanmamış veya kapsamlı bir şekilde test edilmemiş olabilir ve çalı şan tarafından tam olarak anlaşılamayan özellikler içerebilir. Yıllar geçtikçe, b u geçici uygulamaların kullanımı şirketin iç iş süreçlerine entegre hale gelir. D ahası, bu uygulamaları geliştiren çalışanlar terfi edebilir, transfer olabilir veya şirketten ayrılabilir, geride başkalarının anlamadığı bir programı kullanarak b ırakabilir.Sonuç olarak, iyi tasarlanmış, tutarlı bir sistem olarak başlayan şey, kötü tasarlanmış, belgelenmemiş ve hata yapmaya yatkın uygulamaların yam alarına bağımlı hale gelebilir.

Doclingo https://www.doclingo.ai

İnternet. Bu bileşenlerin her biri, kullanıcıya verimli bir şekilde hizmet vermek i çin gerektiğinde ayrı ayrı başlatılır ve durdurulur; ancak, uygulama kesintisiz bi r dizi ekran ve eylem olarak görünür.

Sistem kaynaklarının kullanımını sınırlama motivasyonunun yanı sıra, akı llı telefonların bileşen mimarisi uygulamalar arasında entegrasyonda avantaj lar sağlar. Örneğin, bir akıllı telefonda çalıştırıldığında Facebook (iyi bilinen b ir sosyal ağ sistemi), tüm Facebook arkadaşlarını kişi olarak eklemek için kişil er uygulamasının bileşenlerini kullanabilir. Ayrıca, telefonun işlevlerini yönet en telefon uygulaması, gelen bir aramanın arayanını bulmak için kişiler bileş enlerine de erişebilir. Böylece, bir Facebook arkadaşından gelen bir arama al ındığında, arkadaşın resmi telefonun ekranında görüntülenebilir (son Faceb ook gönderisiyle birlikte).

Sorular & Alıştırmalar

1. Bir roman, bölümler, kısımlar veya girişler gibi birimlerinin arasındak i bağlantı derecesi açısından bir ansiklopediden nasıl farklıdır? Peki y a bütünlük?
2. Bir spor etkinliği genellikle birimlere ayrılır. Örneğin, bir beyzbol oyunu, devrelere ve bir tenis maçı setlere ayrılır. Bu tür "modüller" arasındaki bağlantıyı analiz edin. Bu tür birimler ne anlamda bütünlük gösterir?
3. Bütünlüğü maksimize etme hedefi, bağlantıyı minimize etme ile uyuml u mu? Yani, bütünlük arttıkça, bağlantı doğal olarak azalmaya mı eğilim lidir?
4. Bağlantıyı, bütünlüğü ve bilgi gizlemeyi tanımlayın.
5. Şekil 7.3'teki yapı şemasını, `ControlGame` ve`UpdateScore` modülleri arasındaki veri bağlantısını içerecek şekilde genişletin.
6. Şekil 7.5'e benzer bir diyagram çizin, böylece `PlayerA`'nın servisinin geçersi z sayılması durumunda gerçekleşecek sırayı temsil edin.
7. Geleneksel bir programcı ile bir bileşen montajcısı arasındaki fark ne dir?
8. Çoğu akıllı telefonun bir dizi kişisel organizasyon uygulamasına (takvim ler, kişiler, saatler, sosyal ağlar, e-posta sistemleri, haritalar vb.) sahip o lduğunu varsayarsak, hangi bileşen işlevlerinin kombinasyonlarını fayd alı ve ilginç bulursunuz?

7.5 Ticaret Araçları

Bu bölümde, yazılım geliştirme analiz ve tasarım aşamalarında kullanılan ba zı modelleme teknikleri ve notasyon sistemlerini inceliyoruz.

Bunların birçoğu, zorunlu paradigmanın yazılım mühendisliği disiplininde ha kim olduğu yıllarda geliştirilmiştir. Bunlardan bazıları, nesne yönelimli parad igma bağlamında faydalı roller bulurken, diğerleri, yapı şeması (bkz. tekrar Ş ekil 7.3) gibi, zorunlu paradigmaya özgüdür. Hayatta kalan bazı teknikleri ele alarak başlıyoruz.

Doclingo https://www.doclingo.ai

onların zorunlu köklerini keşfettikten sonra, daha yeni nesne yönelimli araçları ve tasarı m kalıplarının genişleyen rolünü keşfetmeye geçin.

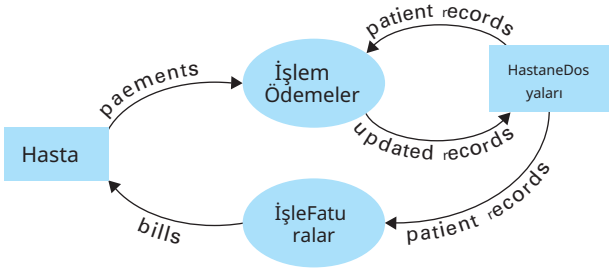
Bazı Eski Arkadaşlar

Zorunlu paradigma, yazılımı prosedürler açısından inşa etmeyi amaçlasa da, bu p rosedürleri tanımlamanın bir yolu, işlenecek verileri dikkate almaktır; prosedürleri n kendilerini değil. Teori, verilerin bir sistemde nasıl hareket ettiğini inceleyerek, v erilerin formatlarının değiştiği veya veri yollarının birleşip ayrıldığı noktaları tanım lamaktır. Bu noktalar, işlemenin gerçekleştiği yerlerdir ve dolayısıyla veri akışı ana lizi, prosedürlerin tanımlanmasına yol açar. Bir veri akışı diyagramı, bu tür veri ak ışı çalışmalarından elde edilen bilgileri temsil etmenin bir yoludur. Bir veri akışı di yagramında, oklar veri yollarını temsil eder, ovalar veri manipülasyonunun gerçek leştiği noktaları temsil eder ve dikdörtgenler veri kaynaklarını ve depolarını temsil eder. Örneğin, Şekil 7.8, bir hastanenin hasta fatura sistemini temsil eden temel bir veri akışı diyagramını göstermektedir. Diyagramın, Ödemeler (hastalardan gel en) ve HastaKayıtları (hastanenin dosyalarından gelen) oval ÖdemeİşlemleriNokt asında birleştiğini ve buradan GüncellenmişKayıtlar hastanenin dosyalarına geri aktığını gösterdiğine dikkat edin.

Veri akış diyagramları, yazılım geliştirme tasarım aşamasında prosedürleri t anımlamaya yardımcı olmanın yanı sıra, analiz aşamasında önerilen sistemi anl amaya çalışırken de faydalıdır. Gerçekten de, veri akış diyagramları oluşturmak , müşteriler ile yazılım mühendisleri arasındaki iletişimi geliştirmek için bir araç olarak hizmet edebilir (çünkü yazılım mühendisi, müşterinin ne istediğini anla maya çalışırken, müşteri de beklentilerini tanımlamakta zorlanır) ve bu nedenle bu diyagramlar, zorunlu paradigmanın popülaritesinin azaldığı durumlarda bil e uygulama bulmaya devam etmektedir.

Yazılım mühendisleri tarafından yıllardır kullanılan bir diğer araç ise veri sö zlüğüdür; bu, bir yazılım sisteminde yer alan veri öğeleri hakkında bilgilerin me rkezi bir deposudur. Bu bilgiler, her öğeyi referans almak için kullanılan tanıml ayıcıyı, her öğedeki geçerli girişleri (Öğe her zaman sayısal mı yoksa belki her z aman alfabetik mi olacak? Bu öğeye atanabilecek değerlerin aralığı ne olacak?), öğenin nerede saklandığını (Öğe bir dosyada mı yoksa bir veritabanında mı sa klanacak ve eğer öyleyse hangisinde?), ve öğenin yazılımda nerede referans alı ndığını (Hangi modüller öğenin bilgisine ihtiyaç duyacak?) içerir.

Şekil **7.8** Basit bir veri akış diyagramı



Doclingo https://www.doclingo.ai

Bir veri sözlüğü oluşturmanın bir amacı, bir yazılım sisteminin paydaşları ile bu paydaşların ihtiyaçlarını gereksinim spesifikasyonuna dönüştürmekle görevli yazılım mühendisi arasındaki iletişimi geliştirmektir. Bu bağlamda, bir veri sözlüğü oluşturmak, parça numaralarının gerçekten sayısal olmadığını analiz aşamasında ortaya çıkmasını sağlamaya yardımcı olur; bu durum, tasarım veya uygulama aşamalarında geç keşfedilmektense. Veri sözlüğü ile ilişkili bir diğer amaç, sistem genelinde bir tutarlılık sağlamaktır. Genellikle, sözlüğü oluşturarak, gereksizlikler ve çelişkiler ortaya çıkar. Örneğin, envanter kayıtlarında `PartNumber` olarak adlandırılan öğe, satış kayıtlarındaki `PartId` ile aynı olabilir. Dahası, personel departmanı bir çalışanı ifade etmek için `Name` terimini kullanabilirken, envanter kayıtları bir parçayı ifade etmek için `Name` terimini içerebilir.

Birleşik Modelleme Dili

Veri akış diyagramları ve veri sözlükleri, nesne yönelimli paradigmanın ortaya çıkmasından çok önce yazılım mühendisliği envanterinde araçlar olarak yer alıyordu ve başlangıçta geliştirildikleri zorunlu paradigmanın popülaritesi azalmış olsa bile, hala faydalı roller bulmaya devam etmiştir. Şimdi, nesne yönelimli paradigma göz önünde bulundurularak geliştirilmiş olan Birleşik Modelleme Dili (UML) olarak bilinen daha modern araçlar koleksiyonuna geçiyoruz. Ancak, bu koleksiyondaki ilk araç, temel paradigma ne olursa olsun faydalıdır çünkü yalnızca önerilen sistemin kullanıcı bakış açısındaki görüntüsünü yakalamaya çalışır. Bu araç, kullanım durumu diyagramı'dır—örneği Şekil 7.9'da görünmektedir.

Bir kullanım durumu diyagramı, önerilen sistemi büyük bir dikdörtgen olarak tasvir eder; burada sistem ile kullanıcıları arasındaki etkileşimler (kullanım durumları olarak adlandırılır) oval olarak ve sistemin kullanıcıları (aktörler olarak adlandırılır) çubuk figürleri olarak temsil edilir (bir aktör bir kişi olmayabilir). Böylece, Şekil 7.9'da önerilen Hastane Kayıt Sistemi'nin hem Doktorlar hem de Hemşireler tarafından Tıbbi Kayıtları Alma amacıyla kullanılacağı gösterilmektedir.

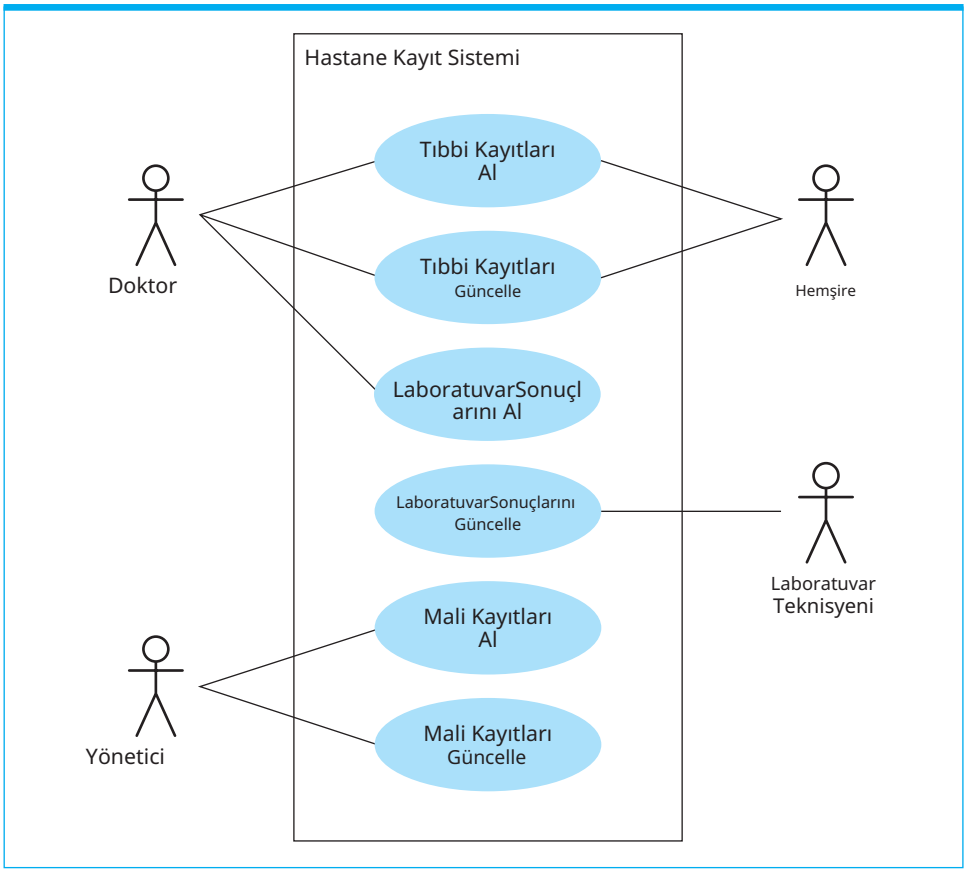
Kullanım durumu diyagramları önerilen bir yazılım sistemine dışarıdan bakarken, UML, bir sistemin iç nesne yönelimli tasarımını temsil etmek için çeşitli araçlar sunar. Bunlardan biri, sınıfların yapısını ve sınıflar arasındaki ilişkileri (UML terimlerinde ilişkiler olarak adlandırılır) temsil etmek için bir notasyon sistemidir. Örneğin, doktorlar, hastalar ve hastane odaları arasındaki ilişkileri düşünelim. Bu varlıkları temsil eden nesnelerin Doktor, Hasta ve

Oda sınıflarından oluşturulduğunu varsayıyoruz.

Şekil 7.10, bu sınıflar arasındaki ilişkilerin bir UML sınıf diyagramında nasıl temsil edilebileceğini göstermektedir. Sınıflar dikdörtgenlerle ve ilişkiler çizgilerle temsil edilir. İlişki çizgileri etiketli veya etiketsiz olabilir. Eğer etiketli iseler, etiketin okunması gereken yönü belirtmek için kalın bir ok ucu kullanılabilir. Örneğin, Şekil 7.10'da "ilgilenir" etiketinin ardından gelen ok ucu, bir doktorun bir hastaya "ilgilenir" olduğunu, bir hastanın bir doktora "ilgilenir" olmadığını gösterir. Bazen ilişki çizgilerine, ilişkilerin her iki yönde okunması için terminoloji sağlamak amacıyla iki etiket verilir. Bu, Şekil 7.10'da Hasta ve Oda sınıfları arasındaki ilişkide örneklendirilmiştir.

Doclingo https://www.doclingo.ai

Şekil **7.9**Basit bir kullanım durumu diyagramı



Sınıflar arasındaki ilişkileri belirtmenin yanı sıra, bir sınıf diyagramı aynı za manda bu ilişkilerin çokluklarını da iletebilir. Yani, bir sınıfın kaç örneğinin başk a bir sınıfın örnekleriyle ilişkilendirilebileceğini gösterebilir. Bu bilgi, ilişki çizgile rinin uçlarında kaydedilir. Özellikle, Şekil 7.10, her hastanın bir odayı işgal edeb ileceğini ve her odanın sıfır veya bir hastayı barındırabileceğini göstermektedir. (Her odanın özel bir oda olduğunu varsayıyoruz.) Rastgele bir negatif olmayan sayıyı belirtmek için bir yıldız kullanılır. Böylece, Şekil 7.10'daki yıldız, her doktor un birçok hastaya bakabileceğini belirtirken, ilişki çizgisinin doktor ucundaki 1, her hastanın yalnızca bir doktor tarafından bakıldığını ifade eder. (Tasarımımız yalnızca birincil doktorların rolünü dikkate alır.)

Tamamlayıcılık açısından, ilişki çokluklarının üç temel biçimde meydana geldiğini belirtmeliyiz: bire-bir ilişkiler, bire-çok ilişkiler,

Şekil **7.10**Basit bir sınıf diyagramı



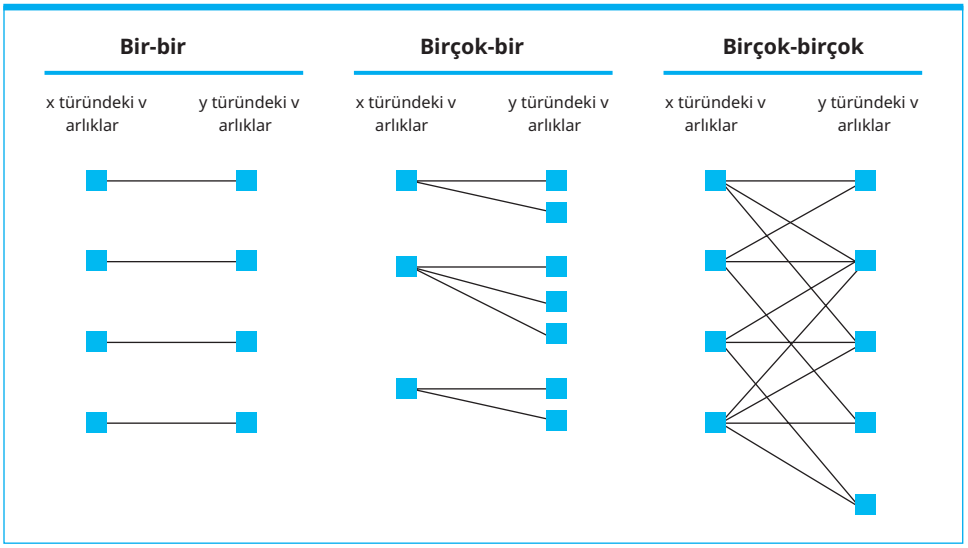
Doclingo https://www.doclingo.ai

ve çoktan çoka ilişkiler, Şekil 7.11'de özetlenmiştir. Bir bire birilişki, her hasta nın yalnızca bir oda ile ilişkilendirilmesi ve her odanın yalnızca bir hasta ile ili şkilendirilmesi ile örneklendirilir. Bir birden çoğa ilişki, bir doktorun birçok h asta ile ilişkilendirilmesi ve her hastanın bir (birincil) doktor ile ilişkilendirilme si ile örneklendirilir. Bir çoktan çoka ilişki, doktor-hasta ilişkisinde danışman doktorları dahil edersek meydana gelir. O zaman her doktor birçok hasta ile ilişkilendirilebilir ve her hasta birçok doktor ile ilişkilendirilebilir.

Nesne yönelimli bir tasarımda, genellikle bir sınıfın başka bir sınıfın daha spesifik bir versiyonunu temsil ettiği durumlar vardır. Bu durumlarda, sonu ncu sınıfın ilk sınıfın bir genellemesi olduğunu söyleriz. UML, genellemeleri t emsil etmek için özel bir notasyon sağlar. Şekil 7.12'de, TıbbiKayıt, CerrahiKa yıt ve OfisZiyaretiKayıt sınıfları arasındaki genellemeleri gösteren bir örnek v erilmiştir. Sınıflar arasındaki ilişkiler, genellemeleri temsil eden boş ok uçları ile gösterilir. Her sınıf, Şekil 7.4'te tanıtılan formatta sınıfın adı, nitelikleri ve y öntemlerini içeren bir dikdörtgen ile temsil edilir.

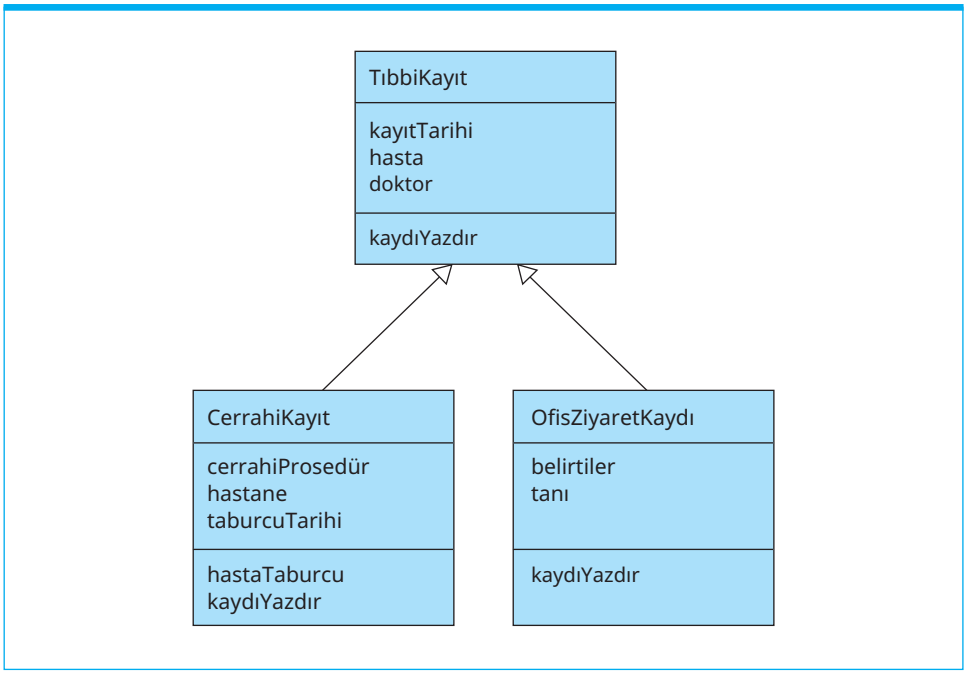
Bu, UML'nin bir sınıfın içsel özelliklerini bir
sınıf diyagramında temsil etme yoludur. Şekil 7.12'de gösterilen bilgi, sınıfın TıbbiKayıt, CerrahiKayıt sınıfının bir genellemesi olduğu gibi OfisZiyaretiKay ıt'nın da bir genellemesidir. Yani, CerrahiKayıt ve OfisZiyaretiKayıt sınıfları, T ıbbiKayıt sınıfının tüm özelliklerini ve ayrıca uygun dikdörtgenleri içinde açıkç a listelenen özellikleri içerir. Böylece, hem CerrahiKayıt hem de OfisZiyareti Kayıt sınıfları hasta, doktor ve kayıt tarihi bilgilerini içerir, ancak CerrahiKayıt sınıfı ayrıca cerrahi işlem, hastane, taburcu tarihi ve bir hastayı taburcu etm e yeteneğini içerirken, OfisZiyaretiKayıt sınıfı semptomlar ve tanı içerir. Üç sı nıf da tıbbi kaydı yazdırma yeteneğine sahiptir. yazdırKayıt metodu

Şekil **7.11** X ve Y türündeki varlıklar arasındaki bir-bir, bir-birçok ve birçok-birçok iliş kileri



Doclingo https://www.doclingo.ai

Şekil **7.12**Genelleştirmeleri tasvir eden bir sınıf diyagramı



CerrahiKayıt ve OfisZiyaretiKayıt, TıbbiKayıt içindeki yazdır-Kayıt metodunu n özelleşmeleridir; her biri kendi sınıfına özgü bilgileri yazdıracaktır.

6. bölümden (Bölüm 6.5) hatırlayacağınız üzere, nesne yönelimli programlam a ortamında genelleştirmeleri uygulamanın doğal bir yolu kalıtım kullanmaktır. Ancak, birçok yazılım mühendisi kalıtımın tüm genelleştirme durumları için u ygun olmadığını belirtmektedir. Bunun nedeni, kalıtımın sınıflar arasında gü çlü bir bağlılık derecesi getirmesidir; bu bağlılık, yazılımın yaşam döngüsünü n ilerleyen dönemlerinde istenmeyen bir durum olabilir. Örneğin, bir sınıf içi ndeki değişiklikler, ondan miras alan tüm sınıflarda otomatik olarak yansıtıld ığından, yazılım bakımı sırasında küçük değişiklikler gibi görünen şeyler bekl enmedik sonuçlara yol açabilir. Örneğin, bir şirket çalışanları için bir rekreas yon tesisi açarsa, bu da rekreasyon tesisine üye olan tüm kişilerin çalışanlar olduğu anlamına gelir. Bu tesis için bir üyelik listesi geliştirmek üzere, bir pro gramcı daha önce tanımlanmış bir
Çalışan sınıfından RekreasyonÜyesi sınıfını oluşturmak için kalıtımı kullanabilir. An cak, şirket daha sonra başarılı olursa ve rekreasyon tesisini çalışanların bağımlıları na veya belki de şirket emeklilerine açmaya karar verirse, o zaman Çalışan sınıfı il e RekreasyonÜyesi sınıfı arasındaki gömülü bağlılık kesilmelidir. Bu nedenle, kalıtı m yalnızca kolaylık için kullanılmamalıdır. Bunun yerine, uygulanan genelleştirme nin değişmez olduğu durumlarla sınırlı olmalıdır.

Sınıf diyagramları, bir programın tasarımının statik özelliklerini temsil ed er. Onlar, yürütme sırasında meydana gelen olayların dizilerini temsil etmez. Bu tür dinamik özellikleri ifade etmek için, UML, etkileşim diyagramları olara k bilinen çeşitli diyagram türleri sunar. Etkileşim diyagramı türlerinden biri, b ireyler (aktörler, tamamlanmış yazılım bileşenleri veya bireysel nesneler gibi) arasındaki iletişimi tasvir eden sıralama diyagramıdır.

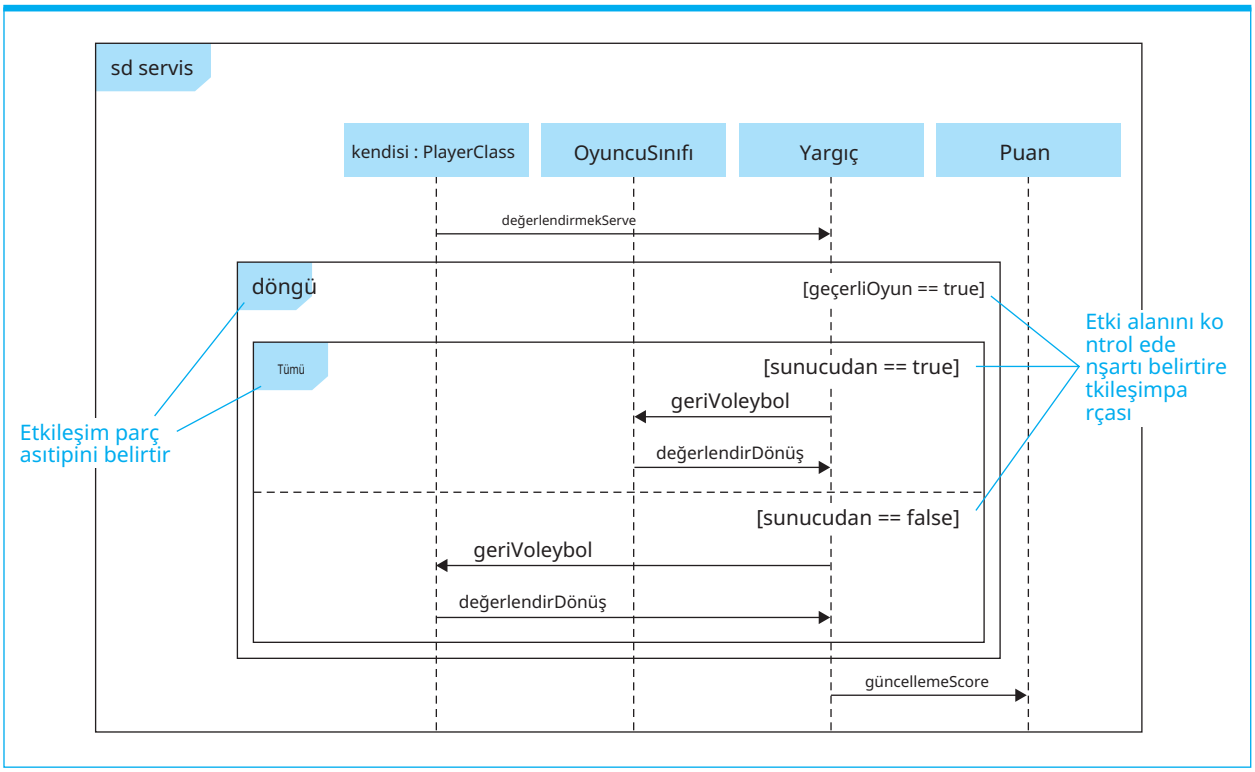
Doclingo https://www.doclingo.ai

bir görevi yerine getirmede yer alan. Bu diyagramlar, aşağıya doğru uzanan ke sik çizgilerle dikdörtgenler olarak bireyleri temsil ettikleri için Şekil 7.5'e benzer. Her dikdörtgen ve kesik çizgisi birlikte bir yaşam hattı olarak adlandırılır. Bireyler arasındaki iletişim, uygun yaşam hattını bağlayan etiketli oklarla te msil edilir; burada etiket, talep edilen eylemi belirtir. Bu oklar, diyagramın üs tten alta doğru okunmasıyla kronolojik olarak görünür. Bir bireyin talep edil en bir görevi tamamladığında ve kontrolü talep eden bireye geri verdiğinde gerçekleşen iletişim, geleneksel bir prosedürden dönüşte olduğu gibi, orijina l yaşam hattına geri bakan etiketlenmemiş bir okla temsil edilir.

Bu nedenle, Şekil 7.5 esasen bir sıralama diyagramıdır. Ancak, Şekil 7.5'in s özdizimi birkaç eksikliğe sahiptir. Bunlardan biri, iki oyuncu arasındaki simetrin in yakalanmasına izin vermemesidir. `PlayerB`'den gelen bir servisle başlayan b ir voleyi temsil etmek için ayrı bir diyagram çizmemiz gerekir, oysa etkileşim sır ası `PlayerA` servis yaptığında çok benzer. Dahası, Şekil 7.5 yalnızca belirli bir v oleyi tasvir ederken, genel bir voleyi sonsuz bir şekilde uzatmak mümkündür. Resmi sıralama diyagramları, bu varyasyonları tek bir diyagramda yakalama k için tekniklere sahiptir ve bunları ayrıntılı olarak incelememize gerek olmas a da, tenis oyunu tasarımımıza dayanan genel bir voleyi gösteren Şekil 7.13't eki resmi sıralama diyagramına kısaca göz atmalıyız.

Ayrıca, Şekil 7.13'ün tüm bir sıralama diyagramının bir dikdörtgenin içine alındığını gö sterdiğini unutmayın (bu dikdörtgene çerçeve denir). Çerçevenin sol üst köşesinde, *sd* (yan i "sıralama diyagramı" anlamına gelir) karakterlerini içeren bir beşgen bulunmaktadır.

Şekil **7.13** Genel bir voleyi tasvir eden bir sıralama diyagramı



Doclingo https://www.doclingo.ai

bir tanımlayıcı tarafından. Bu tanımlayıcı, genel sıralamayı tanımlayan bir isim veya, Şekil 7.13'te olduğu gibi, sıralamayı başlatmak için çağrılan metodun adıdır. Şekil 7.5'in aksine, Şekil 7.13'teki oyuncuları temsil eden dikdörtgenlerin belirli oyuncuları ifade etmediğini, yalnızca "tip" PlayerClass nesnelerini temsil ettiğini unutmayın. Bunlardan biri self olarak adlandırılmıştır, bu da sıralamayı başlatmak için aktif hale getirilen `serve` metodunun ait olduğu nesne anlamına gelir.

Şekil 7.13 ile ilgili diğer bir nokta, iki iç dikdörtgenle ilgilidir.Bunlar etkileşim parçaları olup, bir diyagram içinde alternatif sıralamaları temsil etmek için kullanılır.Şekil 7.13, iki etkileşim parçası içermektedir. Biri "döngü" olarak etiketlenmiş, diğeri "alt" olarak etiketlenmiştir.Bunlar esasen, Bölüm 5.2'deki sahte kodumuzda ilk kez karşılaştığımız while ve if-then-else yapılarıdır."Döngü" etkileşim parçası, sınırları içindeki olayların `Judge` nesnesi `validPlay` değerinin doğru olduğunu belirlediği sürecetekrarlanacağını belirtir. "Alt" etkileşim parçası, `fromServer` değerinin doğru veya yanlış olmasına bağlı olarakalternatiflerinden birinin gerçekleştirilmesi gerektiğini belirtir.

Sonunda, UML'nin bir parçası olmasalar da, nesne yönelimli tasarımları doğrulamada önemli bir rol oynadıkları için CRC (sınıf-sorumluluk-işbirliği) kartlarının rolünü tanıtmak bu noktada uygundur. Bir CRC kartı, bir nesnenin tanımının yazılı olduğu bir kart, örneğin bir indeks kartıdır. CRC kartlarının metodolojisi, yazılım tasarımcısının önerilen sistemdeki her nesne için bir kart üretmesini ve ardından bu kartları sistemin simülasyonunda nesneleri temsil etmek için kullanmasını içerir—belki bir masaüstünde veya her tasarım ekibi üyesinin bir kart tuttuğu ve o kartta tanımlanan nesnenin rolünü oynadığı "tiyatro" deneyinde. Bu tür simülasyonlar (genellikle yapılandırılmış yürüyüşler olarak adlandırılır) tasarımın uygulanmasından önce bir tasarımdaki hataları tanımlamada faydalı bulunmuştur.

Tasarım Kalıpları

Yazılım mühendisleri için giderek daha güçlü bir araç, tasarım kalıplarının artan koleksiyonudur. Bir tasarım kalıbı, yazılım tasarımında tekrarlayan bir problemi çözmek için önceden geliştirilmiş bir modeldir. Örneğin, Adaptör kalıbı, prefabrik modüllerden yazılım inşa ederken sıkça karşılaşılan bir probleme çözüm sunar. Özellikle, bir prefabrik modül, mevcut uygulama ile uyumlu bir arayüze sahip olmayabilir, ancak mevcut problemin çözümü için gerekli işlevselliğe sahip olabilir. Bu tür durumlarda Adaptör kalıbı, orijinal modülün arayüzü ile dış dünya arasında çeviri yapan başka bir modülün içine o modülü "sarmak" için standart bir yaklaşım sunar ve böylece orijinal, prefabrik modül uygulamada kullanılabilir hale gelir.

Diğer bir iyi bilinen tasarım deseni Decorator desenidir. Bu, bir sistemin o anki duruma bağlı olarak aynı aktivitelerin farklı kombinasyonlarını gerçekleştirmesini sağlayan bir tasarım aracıdır. Bu tür sistemler, dikkatli bir tasarım olmadan son derece karmaşık yazılımlara yol açabilecek bir seçenek patlamasına neden olabilir. Ancak, Decorator deseni, bu tür sistemlerin uygulanması için standartlaştırılmış bir yol sunarak yönetilebilir bir çözüm sağlar.

Yazılım mühendisliğinde tekrarlayan sorunların tanımlanması ve bunları çözmek için tasarım desenlerinin oluşturulması ve kataloglanması sürekli bir süreçtir. Ancak, hedef yalnızca tasarım sorunlarına çözümler bulmak değil, yazılım yaşam döngüsünde daha sonra esneklik sağlayan yüksek kaliteli çözümler bulmaktır. Bu nedenle,

Doclingo https://www.doclingo.ai

bağlantıyı en aza indirme ve uyumu maksimize etme gibi iyi tasarım ilkelerinin dikka te alınması, tasarım desenlerinin geliştirilmesinde önemli bir rol oynamaktadır.

Tasarım deseni geliştirmedeki ilerlemenin sonuçları, Oracle tarafından sağl anan Java programlama ortamları ve Microsoft tarafından sağlanan .NET Fram ework gibi günümüz yazılım geliştirme paketlerinde sunulan araçlar kütüphan esinde yansıtılmaktadır. Gerçekten de, bu "araç setlerinde" bulunan birçok "şab lon", esasen tasarım sorunlarına hazır, yüksek kaliteli çözümler sunan tasarım deseni iskeletleridir.

Kapanışta, yazılım mühendisliğinde tasarım kalıplarının ortaya çıkışının, farklı alanların birbirine nasıl katkıda bulunabileceğinin bir örneği olduğunu belirtmeli yiz. Tasarım kalıplarının kökenleri, geleneksel mimaride Christopher Alexander'ın araştırmalarına dayanmaktadır. Amacı, binalar veya bina kompleksleri için yükse k kaliteli mimari tasarımlara katkıda bulunan özellikleri tanımlamak ve ardından bu özellikleri içeren tasarım kalıpları geliştirmekti. Bugün, onun birçok fikri yazılı m tasarımına dahil edilmiştir ve çalışmaları birçok yazılım mühendisi için ilham ka ynağı olmaya devam etmektedir.

Sorular & Alıştırmalar

1. Bir kütüphaneden bir müşterinin bir kitabı çıkardığında meydana gelen veri akışın ı temsil eden bir veri akış diyagramı çizin.
2. Bir kütüphane kayıt sistemi için bir kullanım durumu diyagramı çizin.
3. Seyahat edenler ile kaldıkları oteller arasındaki ilişkiyi temsil eden bir sınıf diyagramı çizin.
4. Bir kişinin bir çalışanın genellemesi olduğunu temsil eden bir sınıf diy agramı çizin. Her birine ait olabilecek bazı nitelikleri ekleyin.
5. Şekil 7.5'i tam bir sıralama diyagramına dönüştürün.
6. Tasarım kalıpları yazılım mühendisliği sürecinde hangi rolü oynar?

7.6 Kalite Güvencesi

Yazılım arızalarının, maliyet aşımının ve kaçırılan son tarihlerinin artışı, yazılı m kalite kontrol yöntemlerinin geliştirilmesini zorunlu kılmaktadır. Bu bölüm de, bu çabada izlenen bazı yönleri ele alıyoruz.

Kalite Güvencesinin Kapsamı

Bilgisayarın ilk yıllarında, kaliteli yazılım üretme sorunu, uygulama sırasında meydana gelen programlama hatalarını ortadan kaldırmaya odaklanıyordu. Bu bölümde, bu yönde kaydedilen ilerlemeleri tartışacağız. Ancak, günümüz de yazılım kalite kontrolünün kapsamı, hata ayıklama sürecinin çok ötesine g eçmektedir; yazılım mühendisliği prosedürlerinin iyileştirilmesi, birçok duru mda sertifikaya yol açan eğitim programlarının geliştirilmesi ve sağlam yazılı m mühendisliğine dayanan standartların oluşturulması gibi alanları içermekt edir. Bu bağlamda,

Doclingo https://www.doclingo.ai

ISO, IEEE ve ACM gibi organizasyonların profesyonelliği artırma ve yazılım ge liştirme şirketlerinde kalite kontrolünü değerlendirmek için standartlar oluşt urma rolünü zaten belirtmiştik. Belirli bir örnek, tasarım, üretim, kurulum ve servis gibi birçok endüstriyel faaliyeti ele alan ISO 9000 serisi standartlarıdır. Diğer bir örnek ise, ISO ve Uluslararası Elektroteknik Komisyonu (IEC) tarafın dan ortaklaşa geliştirilen ISO/IEC 15504 standartlarıdır.

Birçok büyük yazılım yüklenicisi, yazılım geliştirmek için işe aldıkları orga nizasyonların bu standartları karşılamasını artık talep etmektedir. Sonuç olar ak, yazılım geliştirme şirketleri, organizasyon tarafından benimsenen kalite k ontrol sistemlerini denetlemek ve uygulamakla görevli yazılım kalite güvenc esi (SQA) grupları kurmaktadır. Böylece, geleneksel şelale modelinde, SQA gr ubunun tasarım aşaması başlamadan önce yazılım gereksinimleri spesifikas yonunu onaylama veya uygulama başlatılmadan önce tasarımı ve ilgili belgel eri onaylama görevini üstlenmesi gerekecektir.

Bugünün kalite kontrol çabalarının altında yatan birkaç tema var. Bunlardan biri kayıt tutmadır. Geliştirme sürecindeki her adımın gelecekteki referans için doğru bir şekilde belgelenm esi son derece önemlidir. Ancak, bu hedef insan doğasıyla çelişmektedir. Sorun, ilgili bel geleri güncellemeksizin kararlar almak veya kararları değiştirme cazibesidir. Sonuç olar ak, kayıtların yanlış olma ihtimali vardır ve dolayısıyla gelecekteki aşamalarda kullanıml arı yanıltıcı olacaktır. İşte burada CASE araçlarının önemli bir faydası yatmaktadır. Bu ar açlar, diyagramları yeniden çizmek ve veri sözlüklerini güncellemek gibi görevleri manu el yöntemlere göre çok daha kolay hale getirir. Sonuç olarak, güncellemelerin yapılma o lasılığı daha yüksektir ve nihai belgelerin doğru olma olasılığı artar. (Bu örnek, yazılım mühendisliğinin insan doğasının kusurlarıyla başa çıkmak zorunda olduğu birçok duru mdan sadece biridir. Diğerleri, insanların birlikte çalıştığında ortaya çıkan kaçınılmaz kiş ilik çatışmaları, kıskançlıklar ve ego çatışmalarını içerir.) Diğer bir kalite odaklı tema, yazı lım geliştirme projesinde yer alan çeşitli tarafların belirli bir konuyu değerlendirme k için bir araya geldiği gözden geçirme toplantılarıdır. Gözden geçirmeler, yazılım geliştirme süreci boyunca, gereksinim gözden geçirmeleri, t asarım gözden geçirmeleri ve uygulama gözden geçirmeleri şeklinde gerçekleşir. Gerek sinim analizinin erken aşamalarında bir prototip gösterimi olarak ortaya çıkabilirler,

Sistem Tasarım Trajedileri

İyi tasarım disiplinlerine olan ihtiyaç, 1980'lerin ortalarında tıbbi topluluk tarafında n kullanılan bilgisayar tabanlı elektron hızlandırıcı radyasyon terapisi sistemi olan T herac-25'te karşılaşılan sorunlarla örneklendirilmiştir. Makinenin tasarımındaki kus urlar, üçü ölümle sonuçlanan altı radyasyon aşırı doz vakasına katkıda bulunmuştu r. Kusurlar arasında (1) operatörün makine uygun dozaj için ayarlanmadan önce ra dyasyona başlamasına izin veren makinenin arayüzü için kötü bir tasarım ve (2) bel irli güvenlik özelliklerinin yokluğuna neden olan donanım ve yazılım tasarımı arası ndaki kötü koordinasyon bulunmaktadır.

Daha yakın tarihteki vakalarda, kötü tasarım yaygın elektrik kesintilerine, telefon hiz metinin kesilmesine, finansal işlemlerde büyük hatalara, uzay sondalarının kaybına ve İn ternet'in kesintiye uğramasına yol açmıştır. Bu tür sorunlar hakkında daha fazla bilgi edi nebilirsiniz, Risks Forum'u aracılığıyla `http://catless.ncl.ac.uk/Risks`.

Doclingo https://www.doclingo.ai

Yazılım tasarım ekibi üyeleri arasında yapılandırılmış bir yürüyüş olarak veya tasarımın ilgili kısımlarını uygulayan programcılar arasında koordinasyon ola rak. Bu tür incelemeler, tekrarlayan bir temel üzerinde, yanlış anlamaların ö nlenebileceği ve hataların felaket haline gelmeden önce düzeltilebileceği ileti şim kanalları sağlar. İncelemelerin önemi, IEEE 1028 olarak bilinen IEEE Yazılı m İnceleme Standardı'nda özel olarak ele alındığı gerçeğiyle örneklendirilmi ştir.

Bazı incelemeler doğası gereği kritik öneme sahiptir. Bir örnek, bir proje nin paydaşları ile yazılım geliştirme ekibi arasındaki incelemedir; bu incelem ede nihai yazılım gereksinimleri spesifikasyonu onaylanır. Gerçekten de, bu onay, resmi gereksinim analizi aşamasının sona erdiğini işaret eder ve geri k alan geliştirme sürecinin temelini oluşturur. Ancak, tüm incelemeler önemlid ir ve kalite kontrolü açısından, devam eden kayıt bakım sürecinin bir parçası olarak belgelenmelidir.

Yazılım Testi

Yazılım kalite güvencesinin artık tüm geliştirme sürecini kapsayan bir konu olar ak tanındığına rağmen, programların kendilerinin test edilmesi ve doğrulanma sı hala bir araştırma konusudur. Bölüm 5.6'da, algoritmaların doğruluğunu ma tematiksel olarak titiz bir şekilde doğrulama tekniklerini tartıştık, ancak günüm üzde çoğu yazılımın "test" yoluyla "doğrulandığını" sonucuna vardık.
Ne yazık ki, bu tür testler en iyi ihtimalle kesin değildir. Bir yazılım parçasının doğru olduğunu test yoluyla garanti edemeyiz, yeterince test yapmadıkça tüm olası senary oları tüketmiş olmayız. Ancak, basit programlarda bile, potansiyel olarak geçilebilece k milyarlarca farklı yol olabilir. Bu nedenle, karmaşık bir program içindeki tüm olası y olları test etmek imkansız bir görevdir.

Öte yandan, yazılım mühendisleri, sınırlı sayıda test ile yazılımdaki hataları ort aya çıkarma olasılığını artıran test metodolojileri geliştirmiştir. Bunlardan biri, yazı lımdaki hataların genellikle kümelendiği gözlemi üzerine kuruludur. Yani, deneyi mler, büyük bir yazılım sistemindeki az sayıda modülün diğerlerinden daha sorunl u olma eğiliminde olduğunu göstermiştir. Bu nedenle, bu modülleri tanımlayarak ve daha kapsamlı bir şekilde test ederek, sistemin hatalarının daha fazlası keşfedil ebilir; bu, tüm modüllerin tek tip, daha az kapsamlı bir şekilde test edilmesinden daha etkilidir. Bu, ekonomist ve sosyolog Vilfredo Pareto'ya (1848–1923) atıfta bul unarak bilinen Pareto prensibi olarak adlandırılan önermenin bir örneğidir; Paret o, İtalya nüfusunun küçük bir kısmının İtalya'nın zenginliğinin çoğunu kontrol etti ğini gözlemlemiştir. Yazılım mühendisliği alanında, Pareto prensibi, sonuçların ge nellikle çabaların yoğun bir alana uygulanmasıyla en hızlı şekilde artırılabileceğini belirtir.

Başka bir yazılım test metodolojisi olan temel yol testi, yazılımda her bir t alimatın en az bir kez çalıştırılmasını sağlayan bir test verisi seti geliştirmekti r. Bu tür test veri setlerini tanımlamak için grafik teorisi olarak bilinen bir ma tematik alanını kullanan teknikler geliştirilmiştir. Böylece, bir yazılım sistemin deki her yolun test edilmesini sağlamak imkansız olsa da, sistem içindeki her ifadenin test süreci boyunca en az bir kez çalıştırılmasını sağlamak mümkün dür.

Pareto prensibi ve temel yol testi üzerine kurulu teknikler, test edilen yaz ılımın iç yapısına dair bilgiye dayanır. Bu nedenle, cam kutu testi olarak adla ndırılan kategoriye girer—yani yazılım testçisi, yazılımın iç yapısını bilmekted ir ve bu bilgiyi test tasarlarken kullanır.

Doclingo https://www.doclingo.ai

Buna karşılık, yazılımın iç yapısına dair bilgiye dayanmadığı için kara kutu testi olarak adlandırılan bir kategori vardır. Kısacası, kara kutu testi, kullanıcının bakış açısına göre gerçekleştirilir. Kara kutu testinde, yazılımın görevini nasıl yerine getirdiği ile ilgilenilmez, yalnızca yazılımın doğruluk ve zamanlama açısından doğru bir şekilde çalışıp çalışmadığına bakılır.

Kara kutu testine bir örnek, sınır değer analizi olarak adlandırılan teknikt ir; bu teknik, yazılımın benzer şekilde performans göstermesi gereken veri ar alıklarını, eşdeğer sınıflar olarak tanımlamayı ve ardından yazılımı bu aralıkl arın kenarlarına yakın verilerle test etmeyi içerir. Örneğin, yazılım belirli bir a ralıkta giriş değerlerini kabul etmesi gerekiyorsa, yazılım bu aralıktaki en düş ük ve en yüksek değerlerde test edilecektir; ya da yazılım birden fazla etkinli ği koordine etmesi gerekiyorsa, yazılım en büyük olası etkinlikler koleksiyon u üzerinde test edilecektir. Temel teori, eşdeğer sınıfları tanımlayarak, test d urumu sayısının minimize edilebileceğidir çünkü bir eşdeğer sınıf içindeki bir kaç örnekte doğru çalışma, yazılımı tüm sınıf için geçerli kılma eğilimindedir. Ayrıca, bir sınıf içindeki bir hatayı tanımlama şansı en iyi, sınıf kenarlarında v eri kullanarak elde edilir.

Bir kara kutu kategorisine giren başka bir metodoloji, beta testi olarak adlandırılır; burada yazılımın ön sürümü, nihai ürün sürümü pazar için kesinleşmeden önce yazılımın gerçek yaşam durumlarında nasıl performans gösterdiğini öğrenmek amacıyla hedef kitle segmentine verilir. (Geliştiricinin sitesinde yapılan benzer testlere **alpha** testleri denir.) Beta testinin avantajları, geleneksel hata keşfinin çok ötesine uzanır. Pazar stratejilerini geliştirmeye yardımcı olabilecek genel müşteri geri bildirimleri (hem olumlu hem de olumsuz) elde edilir. Ayrıca, beta yazılımının erken dağıtımı, diğer yazılım geliştiricilerinin uyumlu ürünler tasarlamasına yardımcı olur. Örneğin, PC pazarı için yeni bir işletim sistemi durumunda, beta sürümünün dağıtımı, nihai işletim sisteminin sonunda mağaza raflarında yanındaki yardımcı ürünlerle birlikte görünmesini sağlamak için uyumlu yardımcı yazılımların geliştirilmesini teşvik eder. Ayrıca, beta testinin varlığı, pazarda bir beklenti duygusu yaratabilir—bu, pazarlama ve satışları artıran bir atmosferdir.

Sorular & Alıştırmalar

1. SQA grubunun yazılım geliştirme organizasyonundaki rolü nedir?
2. İnsan doğası kalite güvencesine karşı hangi yollarla çalışır?
3. Kaliteyi artırmak için geliştirme sürecinde uygulanan iki temayı belirleyin.
4. Yazılım testinde, başarılı bir test hata bulan mı yoksa bulmayan mı?
5. Diğerlerinden daha kapsamlı test edilmesi gereken bir sistem içindeki modülleri belirlemek için hangi teknikleri önerirsiniz?
6. En fazla 100 girişten oluşan bir listeyi sıralamak için tasarlanmış bir yazılım paketi üzerinde gerçekleştirilecek iyi bir test ne olurdu?

Doclingo https://www.doclingo.ai

7.7 Belgelendirme

Bir yazılım sistemi, insanlar onu kullanmayı ve bakımını öğrenmedikçe pek bir işe yaramaz. Bu nedenle, belgelendirme son bir yazılım paketinin önemli bir parçasıdır ve geliştirilmesi yazılım mühendisliğinde önemli bir konudur.

Yazılım belgelendirmesi üç amaca hizmet eder ve üç belgelendirme kategorisi ne yol açar: kullanıcı belgelendirmesi, sistem belgelendirmesi ve teknik belgelendirme. Kullanıcı belgelendirmesinin amacı, yazılımın özelliklerini açıklamak ve bunların nasıl kullanılacağını tanımlamaktır. Yazılımın kullanıcısı tarafından okunması amaçlanmıştır ve bu nedenle uygulamanın terminolojisiyle ifade edilmiştir.

Bugün, kullanıcı belgelendirmesi önemli bir pazarlama aracı olarak kabul edilmektedir. İyi kullanıcı belgeleri ile iyi tasarlanmış bir kullanıcı arayüzü, bir yazılım paketini erişilebilir hale getirir ve böylece satışlarını artırır. Bunu fark eden birçok yazılım geliştiricisi, ürünlerinin bu kısmını üretmek için teknik yazarlar tutar veya ürünlerinin ön sürümlerini bağımsız yazarlara sağlar, böylece yazılım halka sunulduğunda nasıl yapılır kitapları kitapçılarda mevcut olur.

Kullanıcı belgeleri geleneksel olarak fiziksel bir kitap veya kitapçık şeklinde olur, ancak birçok durumda aynı bilgiler yazılımın kendisinin bir parçası olarak da dahil edilir. Bu, bir kullanıcının yazılımı kullanırken belgeleri referans almasına olanak tanır. Bu durumda, bilgiler bazen yardım paketleri olarak adlandırılan küçük birimlere bölünebilir ve kullanıcı komutlar arasında çok uzun süre beklerse otomatik olarak ekran üzerinde görünebilir.

Sistem belgelemesinin amacı, yazılımın iç yapısını tanımlamaktır, böylece yazılımın yaşam döngüsünün ilerleyen dönemlerinde bakımının yapılabilmesi sağlanır. Sistem belgelemesinin önemli bir bileşeni, sistemdeki tüm programların kaynak versiyonudur. Bu programların okunabilir bir formatta sunulması önemlidir; bu nedenle yazılım mühendisleri, iyi tasarlanmış, yüksek seviyeli programlama dillerinin kullanımını, bir programı notlandırmak için yorum ifadelerinin kullanımını ve her modülün tutarlı bir birim olarak sunulmasına olanak tanıyan modüler bir tasarımı destekler. Aslında, yazılım ürünleri üreten çoğu şirket, çalışanlarının program yazarken uyması gereken kurallar benimsemiştir. Bunlar, yazılı sayfada bir programı düzenlemek için girinti kuralları; değişkenler, sabitler, nesneler ve sınıflar gibi farklı program yapılarının adları arasında bir ayrım oluşturan adlandırma kuralları; ve tüm programların yeterince belgelenmesini sağlamak için belgeleme kuralları içerir. Bu tür kurallar, bir şirketin yazılımında bir bütünlük sağlar ve nihayetinde yazılım bakım sürecini basitleştirir.

Bir sistem belgelendirmesinin bir diğer bileşeni, yazılım gereksinimleri spesifikasyonu ve bu spesifikasyonların tasarım sırasında nasıl elde edildiğini gösteren kayıtlar da dahil olmak üzere tasarım belgelerinin kaydıdır. Bu bilgi, yazılım bakımında faydalıdır çünkü yazılımın neden bu şekilde uygulandığını gösterir—bu bilgi, bakım sırasında yapılan değişikliklerin sistemin bütünlüğünü bozma olasılığını azaltır.

Teknik belgelendirmenin amacı, bir yazılım sisteminin nasıl kurulması ve hizmet verilmesi gerektiğini tanımlamaktır (örneğin, işletim parametrelerini ayarlamak, güncellemeleri yüklemek ve sorunları yazılımın geliştiricisine bildirmek gibi). Yazılımın teknik belgelendirmesi, otomotiv endüstrisinde mekanikçilere sağlanan belgelere benzer. Bu belgeler, arabanın nasıl tasarlandığı ve inşa edildiği hakkında bilgi vermez (sistem belgelendirmesine benzer), ayrıca arabanın nasıl sürüleceği ve ısıtma/soğutma sisteminin nasıl çalıştırılacağı hakkında da açıklama yapmaz.

Doclingo https://www.doclingo.ai

(kullanıcı belgelendirmesine benzer). Bunun yerine, arabanın bileşenlerine nasıl hizmet edileceğini tanımlar—örneğin, şanzımanın nasıl değiştirileceği veya arızalı bir elektrik s orununu nasıl tespit edeceğiniz gibi.

PC alanında teknik belgeler ile kullanıcı belgeleri arasındaki ayrım bulanı ktır çünkü kullanıcı genellikle yazılımı kuran ve hizmet veren kişidir. Ancak, ç ok kullanıcılı ortamlarda ayrım daha nettir. Bu nedenle, teknik belgeler, kend i yetki alanındaki tüm yazılımların bakımından sorumlu olan sistem yöneticisi için hazırlanmıştır ve kullanıcıların yazılım paketlerine soyut araçlar olarak er işmesine olanak tanır.

Sorular & Alıştırmalar

1. Yazılım hangi biçimlerde belgelenebilir?
2. Yazılım yaşam döngüsünün hangi aşamasında (veya aşamalarında) sistem belg eleri hazırlanır?
3. Bir program mı yoksa onun belgeleri mi daha önemlidir?

7.8 İnsan-Makine Arayüzü

Gereksinim analizi sırasında önerilen yazılım sisteminin çevresiyle nasıl etkileşimd e bulunacağını tanımlamak gibi görevlerden birinin olduğunu 7.2. Bölümden hatı rlayın. Bu bölümde, bu etkileşimle ilgili konuları, insanların iletişim kurmasını içer diğinde ele alıyoruz - derin anlamlara sahip bir konu. Sonuçta, insanların bir yazılı m sistemini soyut bir araç olarak kullanmalarına izin verilmelidir. Bu araç, uygulan ması kolay olmalı ve sistem ile insan kullanıcıları arasındaki iletişim hatalarını en a za indirmek (ideal olarak ortadan kaldırmak) için tasarlanmalıdır. Bu, sistemin ara yüzünün, yalnızca yazılım sisteminin kolaylığına değil, insanların rahatlığına göre t asarlanması gerektiği anlamına gelir.

İyi arayüz tasarımının önemi, bir sistemin arayüzünün, kullanıcı üzerinde diğe r sistem özelliklerinden daha güçlü bir izlenim bırakma olasılığıyla daha da vurgul anmaktadır. Sonuçta, bir insan bir sistemi, iç görevlerini ne kadar zekice yerine ge tirdiği açısından değil, kullanılabilirliği açısından değerlendirme eğilimindedir. Bir insanın perspektifinden, iki rakip sistem arasındaki seçim, sistemlerin arayüzle rine dayalı olarak yapılma olasılığı yüksektir. Bu nedenle, bir sistemin arayüzünün tasarımı, bir yazılım mühendisliği projesinin başarısı veya başarısızlığı açısından b elirleyici bir faktör olabilir.

Bu nedenlerle, insan-makine arayüzü, yazılım geliştirme projelerinin ger eksinim aşamasında önemli bir endişe haline gelmiştir ve yazılım mühendisli ğinin büyüyen bir alt alanıdır. Aslında, bazıları insan-makine arayüzlerinin inc elenmesinin kendi başına bir alan olduğunu savunmaktadır.

Bu alandaki araştırmaların bir yararlanıcısı akıllı telefon arayüzüdür. Kullanışlı bir cep boyutundaki cihaz hedefini elde etmek için, geleneksel insan-makine arayüzünü n (tam boyutlu klavye, fare, kaydırma çubukları, menüler) unsurları, dokunmatik ekr anda gerçekleştirilen hareketler, sesli komutlar ve kelimelerin ve ifadelerin gelişmiş otomatik tamamlama özelliklerine sahip sanal klavyeler gibi yeni yaklaşımlarla değiş tirilmektedir. Bu önemli bir ilerlemeyi temsil etse de, çoğu akıllı telefon kullanıcısı da ha fazla yenilik için hala geniş bir alan olduğunu savunacaktır.

Doclingo https://www.doclingo.ai

İnsan-makine arayüzü tasarımı üzerindeki araştırmalar, insanların fizikse l yetenekleriyle uyumlu sistemler tasarlayan ergonomi ve insanların zihinsel yetenekleriyle uyumlu sistemler tasarlayan kognetik alanlarından yoğun bir şekilde yararlanmaktadır. İkisi arasında, ergonomi daha iyi anlaşılmaktadır; bunun büyük bir nedeni, insanların yüzyıllardır makinelerle fiziksel olarak etk ileşimde bulunmuş olmalarıdır. Antik aletler, silahlar ve ulaşım sistemlerinde örnekler bulunmaktadır. Ancak bu tarihin çoğu kendiliğinden açıktır, ergono minin uygulanması bazen sezgisel olmaktan uzak olmuştur. Sıklıkla alıntılan an bir örnek, daktilo klavyesinin (şimdi bilgisayar klavyesi olarak yeniden doğ muş) tasarımıdır; burada tuşlar, erken makinelerde kullanılan mekanik kol si steminin sıkışmaması için bir daktilocunun hızını azaltmak amacıyla kasıtlı ol arak düzenlenmiştir.

Makinelerle zihinsel etkileşim ise, bunun aksine, nispeten yeni bir fenomen dir ve bu nedenle, kognetik, verimli araştırma ve aydınlatıcı içgörüler için daha yüksek bir potansiyel sunmaktadır. Genellikle bulgular, incelikleri açısından ilgi nçtir. Örneğin, insanlar alışkanlıklar geliştirir - bu, yüzeyde verimliliği artırabilec eği için iyi bir özellik olarak görünmektedir. Ancak, alışkanlıklar hatalara da yol açabilir, arayüz tasarımı sorunu kasıtlı olarak ele alsa bile. Bir insanın tipik bir iş letim sistemine bir dosyayı silmesini istemesi sürecini düşünün. İstenmeyen sil meleri önlemek için, çoğu arayüz, kullanıcıdan isteği onaylamasını istemek sur etiyle böyle bir isteğe yanıt verir - belki de “Bu dosyayı gerçekten silmek istiyor musunuz?” gibi bir mesaj aracılığıyla. İlk bakışta, bu onay gereksinimi, istenme yen silme sorununu çözüyormuş gibi görünmektedir. Ancak, sistemi uzun bir s üre kullandıktan sonra, bir insan otomatik olarak “evet” yanıtını verme alışkanlı ğını geliştirir. Böylece, bir dosyayı silme görevi, bir silme komutunu takiben bir soruya düşünceli bir yanıt verme sürecinden oluşan iki adımlı bir süreç olmakta n çıkar.Bunun yerine, bir adımlı bir “sil-evet” süreci haline gelir; bu da, insan ya nlış bir silme isteğinin gönderildiğini fark ettiğinde, isteğin zaten onaylandığı v e silmenin gerçekleştiği anlamına gelir.

Alışkanlıkların oluşumu, bir insanın birkaç uygulama yazılımı paketini kulla nması gerektiğinde sorunlara yol açabilir.Bu tür paketlerin arayüzleri benzer a ma farklı olabilir. Benzer kullanıcı eylemleri, farklı sistem tepkilerine yol açabilir veya benzer sistem tepkileri, farklı kullanıcı eylemleri gerektirebilir. Bu durumla rda, bir uygulamada geliştirilen alışkanlıklar, diğer uygulamalarda hatalara yol açabilir.

İnsan-makine arayüz tasarımında araştırmacıları ilgilendiren bir diğer ins an özelliği, bir insanın dikkatinin dar olmasıdır; bu, konsantrasyon seviyesi ar ttıkça daha da odaklanma eğilimindedir. Bir insan, üzerinde çalıştığı işe daha fazla dalgınlaştıkça, bu odaklanmayı bozmak daha zor hale gelir. 1972'de bir ticari uçak, pilotların iniş takımı problemiyle (aslında, iniş takımı gösterge a mpulünü değiştirme süreciyle) o kadar meşgul olmaları nedeniyle yere çakıl dı; oysa kokpitte uyarılar çalıyor olmasına rağmen.

Daha az kritik örnekler, PC arayüzlerinde rutin olarak ortaya çıkar. Örneğin, ç oğu klavyede, klavyenin “Caps Lock” modunda olduğunu belirtmek için bir “Caps L ock” ışığı bulunur (yani, “Caps Lock” tuşuna basılmıştır). Ancak, tuş yanlışlıkla basıl dığında, bir insan genellikle ışığın durumunu, ekranda garip karakterler görünme ye başlayana kadar fark etmez. O zaman bile, kullanıcı genellikle sorunun nedeni ni anlamadan önce bir süre bu durumla başa çıkmaya çalışır. Bir anlamda, bu şaşı rtıcı değildir; çünkü klavyedeki ışık kullanıcının görüş alanında değildir. Ancak, kull anıcılar genellikle doğrudan yerleştirilen göstergeleri fark etmezler.

Doclingo https://www.doclingo.ai

görüş alanlarında. Örneğin, kullanıcılar bir göreve o kadar dalabilirler ki, ekran üzerindeki imlecin görünümündeki değişiklikleri gözlemlemeyi başarısızlıkla so nuçlandırabilirler, oysa görevleri imleci izlemeyi içerir.

Arayüz tasarımında öngörülmesi gereken bir diğer insan özelliği, zihnin aynı anda birden fazla gerçeklikle başa çıkma kapasitesinin sınırlı olmasıdır. 1956 yılında Psychological Review dergisinde yayımlanan bir makalede, George A. Miller, insan zihninin aynı anda yalnızca yaklaşık yedi detayı işleyebildiğini göster en araştırmaları bildirmiştir. Bu nedenle, bir karar gerektiğinde tüm ilgili bilgileri s unacak şekilde bir arayüz tasarlanması önemlidir; insan kullanıcının hafızasına gü venmek yerine. Özellikle, bir insanın önceki ekran görüntülerinden kesin detayları hatırlamasını gerektirmek kötü bir tasarım olurdu. Dahası, bir arayüz, ekran görü ntüleri arasında geniş bir navigasyon gerektiriyorsa, bir insan labirentte kaybolabi lir. Bu nedenle, ekran görüntülerinin içeriği ve düzeni önemli bir tasarım meselesi haline gelir.

Ergonomi ve kognetik uygulamaları, insan-makine arayüz tasarımına be nzersiz bir tat katarken, bu alan aynı zamanda yazılım mühendisliğinin daha geleneksel konularını da kapsamaktadır. Özellikle, arayüz tasarımında metri k arayışı, yazılım mühendisliğinin daha geleneksel alanlarında olduğu kadar önemlidir. Ölçüme tabi tutulan arayüz özellikleri arasında bir arayüzü öğren mek için gereken süre, arayüz aracılığıyla görevleri yerine getirmek için gere ken süre, kullanıcı-arayüz hatalarının oranı, bir kullanıcının arayüzle olan yet erliliğini uzun süre kullanılmadıktan sonra ne ölçüde koruduğu ve kullanıcıla rın arayüzü ne kadar sevdikleri gibi öznel özellikler bile bulunmaktadır.

GOMS (Toms ile kafiyeli) modeli, 1954 yılında tanıtılmıştır ve insan-makine ara yüzü tasarımı alanında metrikler arayışının temsilcisidir. Modelin temel metodoloj isi, görevleri kullanıcı hedefleri (örneğin, bir metinden bir kelime silmek), operatör ler (örneğin, fare düğmesine tıklamak), yöntemler (örneğin, fare düğmesine çift tı klamak ve silme tuşuna basmak) ve seçim kuralları (örneğin, aynı hedefe ulaşman ın iki yöntemi arasında seçim yapmak) açısından analiz etmektir. Aslında, GOMS kı saltmasının kökeni de buradan gelmektedir—hedefler, operatörler, yöntemler ve seçim kuralları. Kısacası, GOMS, bir arayüzü kullanan bir insanın eylemlerinin tem el adımlar (bir tuşa basmak, fareyi hareket ettirmek, bir karar vermek) dizileri olar ak analiz edilmesine olanak tanıyan bir metodolojidir. Her temel adımın performa nsına belirli bir zaman aralığı atanır ve böylece, bir görevdeki adımlara atanan za manları toplayarak, GOMS, benzer görevleri yerine getirirken her birinin gerektire ceği zaman açısından farklı önerilen arayüzleri karşılaştırmanın bir yolunu sağlar.

Sistemlerin teknik detaylarını anlamak, GOMS gibi, mevcut çalışmamızın amacı değildir. Bizim durumumuzda önemli olan, GOMS'un insan davranışını n özelliklerine (ellerin hareketi, karar verme vb.) dayandığıdır. Aslında, GOMS 'un geliştirilmesi başlangıçta bir psikoloji konusu olarak düşünülmüştür. Bu nedenle, GOMS, insan özelliklerinin insan-makine arayüzü tasarımındaki rolünü yeniden vurgulamaktadır, hatta geleneksel yazılım mühendisliğinden devralınan konularda bile.

İnsan-makine arayüzlerinin tasarımı, yakın gelecekte aktif bir araştırma a lanı olma vaadini taşımaktadır. Bugünün GUI'leri ile ilgili birçok sorun henüz çözülmemiştir ve üç boyutlu arayüzlerin kullanımında ortaya çıkacak çok sayı da ek problem bulunmaktadır. Gerçekten de, bu arayüzlerin sesli ve dokuns al iletişimi üç boyutlu görselle birleştirme vaadi taşıması nedeniyle, potansiy el sorunların kapsamı muazzamdır.

Doclingo https://www.doclingo.ai

Sorular & Alıştırmalar

1. a. İnsan-bilgisayar arayüzü tasarımında ergonominin bir uygulaması nı tanımlayın.
   b. İnsan-bilgisayar arayüzü tasarımında kognetiklerin bir uygulaması nı tanımlayın.
2. Bir akıllı telefonun insan-bilgisayar arayüzündeki belirgin bir fark, ekranın bir kısmını kaydırmak için kullanılan tekniklerdir. Bir masaüstünde, kaydır ma genellikle kaydırma bölgesinin sağ ve alt taraflarında görüntülenen ka ydırma çubukları üzerinde fareyi sürükleyerek gerçekleştirilir. Öte yandan , akıllı telefonda kaydırma çubukları genellikle kullanılmaz. (Kullanıldıkları nda, alttaki ekranın hangi kısmının şu anda görünür olduğunu belirtmek i çin ince çizgiler olarak görünürler.) Kaydırma, bu nedenle, ekranın üzerin de kaydırma dokunuşu hareketiyle gerçekleştirilir.
   a. Ergonomi temelinde, bu farklılığı desteklemek için hangi argümanlar öne sürülebilir?
   b. Kognetikler temelinde, bu farklılığı desteklemek için hangi argümanlar öne sürülebilir?
3. İnsan-makine arayüzü tasarımını daha geleneksel yazılım mühendisli ği alanından ne ayırır?
4. İnsan-makine arayüzü tasarlarken dikkate alınması gereken üç insan özelliğini belirleyin.

7.9 Yazılım Mülkü ve Sorumluluk

Çoğu kişi, bir şirketin veya bireyin kaliteli yazılım geliştirmek için gereken yatırımı geri kaza nmasına ve kâr etmesine izin verilmesi gerektiği konusunda hemfikir olacaktır. Aksi takdird e, toplumumuzun istediği yazılımı üretme görevini üstlenmeye istekli olanların sayısının az olacağı muhtemeldir. Kısacası, yazılım geliştiricilerin ürettikleri yazılım üzerinde bir mülkiye t düzeyine ihtiyaçları vardır.

Böyle bir mülkiyet sağlamaya yönelik yasal çabalar, büyük ölçüde telif ha kkı ve patent hukukunun iyi yerleşmiş ilkelerine dayanan fikri mülkiyet huku ku kategorisine girmektedir. Gerçekten de, bir telif hakkı veya patentin amac ı, bir "ürün" geliştiricisinin o ürünü (veya onun kısımlarını) hedeflenen tarafla ra sunmasına izin vermek ve mülkiyet haklarını korumaktır. Bu nedenle, bir ürünün geliştiricisi (birey veya şirket olsun) ürettiği tüm eserlerde bir telif ha kkı beyanı ekleyerek mülkiyetini iddia edecektir; gereksinim spesifikasyonları , tasarım belgeleri, kaynak kodu, test planları ve nihai ürün içinde görünür bi r yerde. Bir telif hakkı bildirimi, mülkiyeti, eseri kullanmaya yetkili personeli v e diğer kısıtlamaları açıkça tanımlar. Ayrıca, geliştiricinin hakları, yasal teriml erle bir yazılım lisansında resmi olarak ifade edilir.

Bir yazılım lisansı, bir yazılım ürününün sahibi ile kullanıcısı arasında, kullan ıcının ürünü kullanma konusunda belirli izinler veren yasal bir anlaşmadır; bu, mülkiyet haklarını fikri mülkiyete devretmeden yapılır. Bu anlaşmalar, her iki ta rafın haklarını ve yükümlülüklerini ayrıntılı bir şekilde belirtir. Bu nedenle,

Doclingo https://www.doclingo.ai

bir yazılım ürününü yüklemeden ve kullanmadan önce yazılım lisansının şartlar ını dikkatlice okumak ve anlamak önemlidir.

Telif hakları ve yazılım lisans anlaşmaları, yazılımın doğrudan kopyalanması nı ve yetkisiz kullanımını engellemek için yasal yollar sağlasa da, genellikle başk a bir tarafın benzer işlevselliğe sahip bir ürünü bağımsız olarak geliştirmesini ö nlemek için yetersizdir. Yıllar içinde gerçekten devrim niteliğinde bir yazılım ürü nünün geliştiricisinin, icadından tam olarak yararlanamadığı birçok durum olm uştur (iki dikkat çekici örnek, elektronik tablolar ve web tarayıcılarıdır). Bu duru mların çoğunda, başka bir şirket, pazarın baskın bir payını güvence altına alan r ekabetçi bir ürün geliştirmede başarılı olmuştur. Bir rakip tarafından bu tür bir müdahaleyi önlemek için yasal bir yol, patent hukukunda bulunur.

Patent yasaları, bir mucidin bir icattan ticari olarak faydalanmasını sağla mak için oluşturulmuştur. Bir patent almak için mucit, icadın ayrıntılarını açık lamalı ve bunun yeni, faydalı ve benzer geçmişe sahip diğerleri için belirgin o lmadığını göstermelidir (bu, yazılım için oldukça zorlayıcı bir gereklilik olabilir). Eğer bir patent verilirse, mucit, başvuru tarihinden itibaren genellikle yirmi yıl süreyle, başkalarının icadı yapmasını, kullanmasını, satmasını veya ithal e tmesini engelleme hakkına sahip olur.

Patentlerin kullanımının bir dezavantajı, bir patent almak için sürecinpahalı ve zaman alıcı olmasıdır; genellikle birkaç yıl sürmektedir. Bu süre zarfında bir yazılım ürünü eskiyebilir ve patent verilene kadar başvuranın, ürünün başkaları tarafındankullanılmasını engelleme konusunda yalnızca sorgulanabilir bir yetk isi vardır.

Telif haklarını, yazılım lisanslarını ve patentleri tanımanın önemi, yazılım mühendisliği sürecinde son derece önemlidir. Bir yazılım ürünü geliştirirken, yazılım mühendisleri genellikle diğer ürünlerden yazılım dahil etmeyi seçer; bu, tam bir ürün, bileşenlerin bir alt kümesi veya hatta İnternet üzerinden indirilen kaynak kodunun parçaları olabilir. Ancak, bu süreçte fikri mülkiyet haklarına saygı göstermemek büyük sorumluluklar ve sonuçlar doğurabilir. Örneğin, 2004 yılında, çok az bilinen NPT Inc. şirketi, Research In Motion (RIM—BlackBerry akıllı telefonlarının üreticisi) aleyhine, RIM'in e-posta sistemlerine gömülü birkaç ana teknoloji için patent ihlali davasını başarıyla kazandı. Karar, Amerika Birleşik Devletleri'ndeki tüm BlackBerry kullanıcılarına e-posta hizmetlerini askıya alma emrini içeriyordu! RIM, sonunda NPT'ye toplam 612.5 milyon dolar ödemek için bir anlaşmaya vardı ve böylece bir kapanmayı önledi.

Son olarak, sorumluluk meselesini ele almalıyız. Yazılım geliştiricileri, kendileri ni sorumluluktan korumak için genellikle yazılım lisanslarında sorumluluklarının sı nırlamalarını belirten feragatnameler eklerler. "Hiçbir durumda Şirket X, bu yazılı mın kullanımından kaynaklanan herhangi bir zarardan sorumlu olmayacaktır" gibi ifadeler yaygındır. Ancak, mahkemeler, davacının sanığın tarafında ihmal gösterd iği durumlarda feragatnamesini nadiren tanır. Bu nedenle, sorumluluk davaları g enellikle sanığın üretilen ürünle uyumlu bir dikkat seviyesi kullanıp kullanmadığın a odaklanır. Bir kelime işleme sistemi geliştirme durumunda kabul edilebilir olara k değerlendirilebilecek bir dikkat seviyesi, bir nükleer reaktörü kontrol etmek için yazılım geliştirirken ihmal olarak kabul edilebilir. Sonuç olarak, yazılım sorumluluğ u taleplerine karşı en iyi savunmalardan biri, yazılımın geliştirilmesi sırasında sağl am yazılım mühendisliği ilkelerini uygulamak, yazılımın uygulamasıyla uyumlu bir dikkat seviyesi kullanmak ve bu çabaları doğrulayan kayıtlar üretmek ve sürdürm ektir.

Doclingo https://www.doclingo.ai

Sorular & Alıştırmalar

1. Bir telif hakkı bildirisinin gereksinim spesifikasyonları, tasarım belgeleri, kaynak kodu ve nihai ürün üzerindeki önemi nedir?
2. Telif hakkı ve patent yasalarının topluma fayda sağlamak için nasıl tasarlandığına dair hangi yollar vardır?
3. Feragatnamelerin mahkemeler tarafından ne ölçüde tanınmadığı?

Bölüm Gözden Geçirme Problemleri

(Yıldızlı problemler isteğe bağlı bölümlerle ilişkilidir.)

1. Yazılım geliştirme çabalarının daha sonra yazılım bakımında nasıl kazanç sağlayabileceğine bir örnek verin.
2. Evrimsel prototipleme nedir?
3. Belirli yazılım özelliklerini ölçmek için metriklerin eksikliğinin yazılım mühendisliği disiplinini nasıl etkilediğini açıklayın.
4. Bir yazılım sisteminin karmaşıklığını ölçmek için bir metrikin, bir bütün sistemin karmaşıklığının parçalarının karmaşıklıklarının toplamı anlamında birikimli olmasını bekler misiniz? Cevabınızı açıklayın.
5. Bir yazılım sisteminin karmaşıklığını ölçmek için bir metrikin, bir bütün sistemin karmaşıklığının, başlangıçta özellik X ile geliştirilmişse ve daha sonra özellik Y eklenmişse veya başlangıçta özellik Y ile geliştirilmişse ve daha sonra özellik X eklenmişse aynı olmasını bekler misiniz? Cevabınızı açıklayın.
6. Yazılım mühendisliği, elektrik mühendisliği ve makine mühendisliği gibi diğer, daha geleneksel mühendislik alanlarından nasıl farklıdır?
7. a. Yazılım geliştirme için geleneksel şelale modelinin bir dezavantajını tanımlayın.
   b. Geleneksel su perdesi modelinin yazılım geliştirme için bir avantajını tanımlayın.
8. Açık kaynak geliştirme, yukarıdan aşağıya mı yoksa aşağıdan yukarıya mı bir metodolojidir? Cevabınızı açıklayın.
9. Sabitlerin yerine literallerin kullanılmasının yazılım bakımını nasıl basitleştirebileceğini tanımlayın.
10. Kopma ve uyum arasındaki fark nedir? Hangisi en aza indirilmeli ve hangisi en üst düzeye çıkarılmalıdır? Neden?
11. Gündelik hayattan bir nesne seçin ve işlevsel veya mantıksal uyum açısından bileşenlerini analiz edin.
12. Basit bir `goto` ifadesi ile elde edilen iki program birimi arasındaki kopmayı, bir prosedür çağrısı ile elde edilen kopma ile karşılaştırın.
13. 6. Bölümde, parametrelerin prosedürlere değer veya referans ile geçirilebileceğini öğrendik. Hangi yöntem daha karmaşık bir veri kopması sağlar? Cevabınızı açıklayın.
14. Büyük bir yazılım sistemi, tüm veri elemanlarının küresel olacak şekilde tasarlandığı durumlarda bakım sırasında hangi sorunlar ortaya çıkabilir?
15. Nesne yönelimli bir programda, bir örnek değişkeninin public veya private olarak tanımlanması, veri kopması hakkında neyi gösterir? Örnek değişkenlerini private olarak tanımlama tercihinin arkasındaki mantık ne olurdu?

*16. Paralel işleme bağlamında ortaya çıkabilecek bir veri kopması ile ilgili bir problemi tanımlayın.

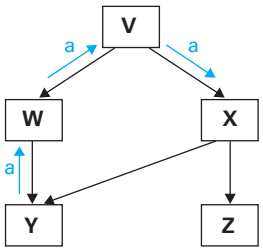
17. Aşağıdaki soruları, ekli yapı şemasına ilişkin olarak yanıtlayın:
   a. Modül Y hangi modüle kontrolgeri döner?
   b. Modül Z hangi modüle kontrol döner?
   c. Modüller W ve X kontrolbağlantısı ile bağlı mı?

Doclingo https://www.doclingo.ai

d. Modüller W ve X veribağlantısı ile m
i bağlı?

e. Hem modül W hem de modül Y tarafından hangi veriler payla
şılmaktadır?

f. Modüller W ve Z nasıl ilişkilidir?



18. Basit bir mağaza için bir envanter/muhaseb
e sisteminin prosedürel yapısını temsil etmek iç
in bir yapı şeması kullanın (belki bir
bir tatil beldesindeki özel müze dükkanı).
Sisteminizde hangi modüller satış vergisi y
asalarındaki değişiklikler nedeniyle değiş
tirilmelidir? Geçmiş müşterilerin kay
dını tutma kararı alınırsa hangi modüllerin
değiştirilmesi gerekecektir ki reklamlar o
nlara gönderilebilsin?

19. Önceki problem için bir nesne-yönelimli çözü
m tasarlamak amacıyla bir sınıf diyagramı kullanın.

20. Bir dergi yayıncıları, dergiler ve aboneler ar
asındaki ilişkileri temsil eden basit bir sını
f diyagramı çizin. Her bir sınıfı temsil eden
kutu içinde yalnızca sınıf adını göstermek
yeterlidir.

21. UML nedir ve ne için kullanılır?
"M." harfine karşılık gelen kelimeyi ayrıntılı olarak
açıklayın.

22. Bir kütüphane kullanıcısının bir kütüphaneyi nasıl kull
andığını gösteren basit bir kullanım durumu diyagra
mı çizin.

23. Bir hizmet şirketinin bir müşteriye fatura gö
nderdiğinde meydana gelecek etkileşim sırasını
temsil eden bir sıralama diyagramı çizin.

24. Bir satış yapıldığında otomatik bir envanter sis
teminde meydana gelen veri akışını göst
eren basit bir veri akış diyagramı çizin.

25. Bir sınıf diyagramında temsil edilen bilgiyi,
bir sıralama diyagramında temsil edilen bi
lgi ile karşılaştırın.

26. Birden çoğa ilişki ile çoktan çoğa ilişki arası
ndaki fark nedir?

27. Bu bölümde bahsedilmeyen birden çoğa iliş
kiye bir örnek verin. Bu bölümde bahsedil
meyen çoktan çoğa ilişkiye bir örnek verin.

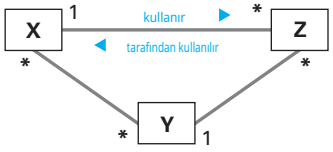
28. Şekil 7.10'daki bilgilere dayanarak, bir
doktor ile bir hasta arasında bir ziyaret sır
asında meydana gelebilecek bir etkileş
im sırasını hayal edin. O sıralamayı t
emsil eden bir sıralama diyagramı çizin.

29. Bir restorandaki sunucular ile müşteriler
arasındaki ilişkileri temsil eden bir sınıf diy
agramı çizin.

30. Dergiler, dergi yayıncıları ve dergi abonel
eri arasındaki ilişkileri temsil eden bir sını
f diyagramı çizin.
Her sınıf için bir dizi örnek değişken
ve yöntem ekleyin.

31. Şekil 7.5'teki "sıralama diyagramını" genişleterek
, `PlayerA` başarılı bir şekilde geri dönerse meyd
ana gelecek etkileşim sırasını gösterin.
`PlayerB`'nin voleybolu, ancak `PlayerB` o voleybolu geri
döndüremiyor.

32. Aşağıdaki soruları, araçlar, kullanıcıları ve
üreticileri arasındaki ilişkileri temsil edenb
eraberindeki sınıf diyagramına dayanara
k yanıtlayın.



a. Hangi sınıflar (X, Y ve Z) araçları,kullanıcıları ve
üreticileri temsil ediyor? Cevabınızıhaklı
çıkarın.

b. Bir aracı birden fazla kullanıcı kullanabilir mi?

c. Bir aracı birden fazla üretici üretebilir mi
?

d. Her kullanıcı, birçok üretici tarafından üretilen
araçları mı kullanıyor?

33. Aşağıdaki her durumda, etkinliğin bir sıra di
yagramı,bir kullanım durumu diyagramı veya bir sın
ıf diyagramı ile ilgili olup olmadığını belirleyin.

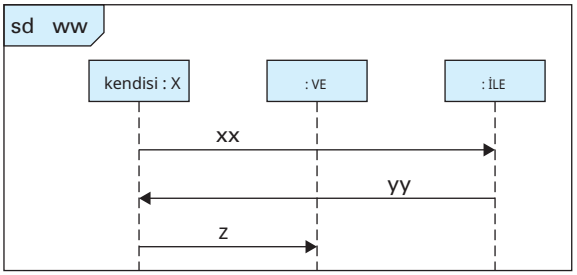
a. Kullanıcıların sistemle nasıl etkileşimde b
ulunacağını temsil eder.

b. Sistemdeki sınıflar arasındaki ilişkiy
i temsil eder.

c. Nesnelerin bir görevi başarmak için nasıl etki
leşimde bulunacağını temsil eder.

Doclingo https://www.doclingo.ai

34. Aşağıdaki sıra diyagramına dayanarak aşa ğıdaki soruları yanıtlayın.



a. ww adında bir metoda sahip olan sınıf hangisidir?
b. xx adında bir metoda sahip olan sınıf hangisidir?
c. Sıra boyunca, "tip" Z'nin nesnesi"tip" Y' nin nesnesiyle doğrudan iletişim kuruy or mu?

35. Bir nesne A'nın nesne B'deki bb metodunu çağırdı ğını, B'nin istenen eylemi gerçekleştirdiğini ve ko ntrolü A'ya geri verdiğini, ardından A'nın nesne B'deki cc metodunu çağırdığını gösteren bir sıralama diyagr amı çizin.

36. Önceki probleme çözümünüzü genişleterek, A'nın y alnızca "continue" değişkeni doğruysa bb metodu nu çağırdığını ve B kontrolü geri verdikten sonra " continue" doğru kaldığı sürece bb'yi çağırmaya de vam ettiğini belirtin.

37. Kamyon ve Otomobil sınıflarının Araç sınıf ının genelleştirmeleri olduğunu gösteren bi r sınıf diyagramı çizin.

38. Şekil 7.12'ye dayanarak, "tip" SurgicalRecord olan bir nesnede hangi ek örnek değişke nler bulunur? "tip" OfisZiyaretKaydı?

39. Kalıtımın sınıf genelleştirmelerini uygulamak için her zaman en iyi yol olmadığını açıklayın.

40. Yazılım mühendisliği dışındaki alanlarda bazı tas arım kalıplarını tanımlayın.

41. Yazılım mühendisliğinde tasarım kalıplarının rolünü özetleyin.

42. Tipik bir yüksek seviyeli programlama dili ndeki kontrol yapılarının (`if-then-else`,`while`, vb.) küçük ölçekli tasarım kalıpları olma derecesi nedir?

43. Aşağıdakilerden hangileri Pareto ilkesini içerir? Cevaplarınızı açıklayın.
a. Bir rahatsız edici kişi partiyiherkes için ma hvedebilir.
b. Her radyo istasyonu, sert rock müz iği, klasik müzik veya sohbet gibi belirli b ir formata odaklanır.
c. Bir seçimde, adayların kampanyalarını ge çmişte oy kullanmış seçmen segmentine odaklamaları akıllıca olur.

44. Yazılım mühendisleri, büyük yazılım sistemlerinin h ata içeriği açısından homojen mi yoksa heterojen mi o lmasını bekliyorlar? Cevabınızı açıklayın.

45. Kara kutu testi ile cam kutu testi arasındaki fark nedir?

46. Yazılım mühendisliği dışındaki alanlarda kara kutu ve cam kutu testine bazı benzetm eler verin.

47. Açık kaynak geliştirme, beta testinden n asıl farklıdır? (Cam kutu testi ile kara kutu t estini dikkate alın.)

48. Varsayalım ki, büyük bir yazılım sistemin e son testten önce kasıtlı olarak 100 hata yerleştirildi.
Ayrıca, bu son test sırasında 200 hata keş fedildi ve düzeltildi; bunlardan 50'si siste mde kasıtlı olarak yerleştirilen gruptan ge ldi. Kalan 50 bilinen hata düzeltildikten sonra, sistemde hala kaç bilinmeyen h ata olduğunu tahmin edersiniz? Nedenini a çıklayın.

49. GOMS nedir?

50. Ergonomi nedir? Kognetik nedir?

51. Akıllı telefonun insan-bilgisayararayüzü ile mas aüstü bilgisayarınki arasındaki bir fark,görünt ü ekranındaki bir görüntünün ölçeğinideğiştir mek için kullanılan tekniktir; bu işlem"zoo ming" (yakınlaştırma) olarak adlandırılır. Masaü stünde,yakınlaştırma genellikle görüntülenen alandanayrı bir kaydırıcıyı sürükleyerek veya b ir menüveya araç çubuğu öğesi kullanarak ger çekleştirilir.Akıllı telefonda ise, yakınlaştırm a, parmakve baş parmakla ekranın dokunulm asıylaaynı anda gerçekleştirilir ve ardından her ikidokunma noktası arasındaki mesafe de ğiştirilir(bu işlem "çift dokunma—yayılma" ile "ya kınlaştırma"veya "çift dokunma—sıkıştırma" ile "uzaklaştırma"olarak adlandırılır).
a. Ergonomi temelinde, bu farkı destekleye n hangi argümanlaröne sürülebilir?
b. Kognetik temelinde, bu farkı destekleyen hangi argümanlaröne sürülebilir?

Doclingo https://www.doclingo.ai

52. Geleneksel telif hakkı yasaları, yazılım geliştiricilerinin y atırımlarınıkorumada hangi yönlerden yetersiz ka lmaktadır?

53. Bir yazılım geliştiricisi, patent almakta h angi yollarla başarısız olabilir?

Sosyal Sorunlar

Aşağıdaki sorular, bilgisayar alanıyla ilişkili etik/sosyal/hukuki sorunlara bir r ehber olarak tasarlanmıştır. Amaç yalnızca bu soruları yanıtlamak değildir. A yrıca neden böyle yanıt verdiğinizi ve gerekçelerinizin bir sorudan diğerine t utarlı olup olmadığını da düşünmelisiniz.

1. a. Mary Analyst, tıbbi kayıtların büyük bir ağa bağlı bir bilgisayarda sakla nacağı bir sistemi uygulama görevine atanmıştır. Ona göre, sistemin güvenlik tasarımı hatalıdır ancak endişeleri mali nedenlerle göz ardı e dilmiştir. Yetersiz olduğunu düşündüğü güvenlik sistemi ile projeye d evam etmesi söylenmiştir. Ne yapmalıdır? Neden?

   b. Diyelim ki Mary Analyst, kendisine söylendiği gibi sistemi uyguladı ve ş imdi tıbbi kayıtların yetkisiz kişiler tarafından gözlemlendiğini fark etti. Ne yapmalıdır? Güvenlik ihlali için ne ölçüde sorumludur?

   c. Diyelim ki Mary Analyst, işverene itaat etmek yerine sistemi uygulamay ı reddeder ve hatalı tasarımı kamuya açıklayarak şirket için mali zorluk lara ve birçok masum çalışanın işini kaybetmesine neden olur. Mary A nalyst'in eylemleri doğru muydu? Eğer Mary Analyst, genel ekibin sad ece bir parçası olduğu için, şirket içinde geçerli bir güvenlik sistemi ge liştirmek için samimi çabaların yapıldığından habersiz olduğunu öğre nirse, bu Mary'nin eylemlerini nasıl değiştirir? (Unutmayın, Mary'nin d uruma bakışı daha öncekiyle aynıdır.)

2. Büyük yazılım sistemleri birçok kişi tarafından geliştirildiğinde, sorumlulukl ar nasıl atanmalıdır? Bir sorumluluk hiyerarşisi var mı? Sorumluluk derecel eri var mı?
3. Büyük, karmaşık yazılım sistemlerinin genellikle birçok birey tarafından gelişti rildiğini gördük; bunların hiçbiri projenin tamamı hakkında tam bir resme sah ip olmayabilir. Bir çalışanın, işlevinin tam bilgisi olmadan bir projeye katkıda b ulunması etik olarak doğru mu?
4. Biri, başarılarının başkaları tarafından nasıl uygulandığı konusunda ne öl çüde sorumludur?
5. Bir bilgisayar profesyoneli ile bir müşteri arasındaki ilişkide, profesyonelin müşterin in isteklerini uygulama sorumluluğu mu vardır yoksa müşterinin isteklerini yönlend irme sorumluluğu mu? Profesyonel, bir müşterinin isteklerinin etik olmayan sonuçl ara yol açabileceğini öngörüyorsa ne olur? Örneğin, müşteri verimlilik uğruna kesti rmeler yapmak isteyebilir, ancak profesyonel bu kestirmelerin yanlış veri kaynağına veya sistemin kötüye kullanılmasına yol açabileceğini öngörebilir.
   Müşteri ısrar ederse, profesyonel sorumluluktan muaf mı olur?

Doclingo https://www.doclingo.ai

6. Eğer teknoloji o kadar hızlı ilerlemeye başlarsa ki yeni icatlar, mucidin ica ttan kâr elde etme zamanı olmadan geçersiz hale gelirse ne olur? İcatçıla rı motive etmek için kâr gerekli mi? Açık kaynak geliştirme başarısı cevab ınızla nasıl ilişkilidir? Ücretsiz kaliteli yazılım sürdürülebilir bir gerçeklik m i?
7. Bilgisayar devrimi, dünyanın enerji sorunlarına katkıda mı bulunuyor yoksa bu sorunları çözmeye mi yardımcı oluyor? Açlık ve yoksulluk gibi diğer büyük ölçekli sorunlar hakkında ne düşünüyors unuz?
8. Teknolojideki ilerlemeler sonsuza dek devam edecek mi? Toplumun tekn olojik bağımlılığını tersine çevirecek bir şey var mı? Teknolojiyi sonsuza d ek ilerleten bir toplumun sonucu ne olurdu?
9. Eğer bir zaman makineniz olsaydı, hangi tarih döneminde yaşamak isterdiniz? Yanınıza almak istediğiniz mevcut teknolojiler var mı? Bir teknoloji diğeri nden ayrılabilir mi? Küresel ısınmaya karşı protesto etmek ama modern t ıbbi tedaviyi kabul etmek gerçekçi mi?
10. Bir akıllı telefondaki birçok uygulama, diğer uygulamalar tarafından sağlanan hizmetlerle otomatik olarak entegre olur. Bu entegrasyon, bir uygulamaya gir ilen bilgileri başka bir uygulama ile paylaşabilir. Bu entegrasyonun faydaları n elerdir? "Çok fazla" entegrasyon ile ilgili endişeler var mı?

Ek Okuma

Alexander, C., S. Ishikawa, ve M. Silverstein. *Bir Desen Dili*. New York: Oxford University Press, 1977.

Beck, K. *Aşırı Programlama Açıklandı: Değişimi Kucakla*, 2. baskı. Boston, MA: Addison-Wesley, 2004.

Bowman, D. A., E. Kruijff, J. J. LaViola, Jr., ve I. Poupyrev. *3D Kullanıcı Arayüzleri Te ori ve Uygulama.*Boston, MA: Addison-Wesley, 2005.

Braude, E. *Yazılım Tasarımı: Programlamadan Mimariliğe*. New York: Wiley, 2004.

Bruegge, B. ve A. Dutoit. *Nesne Yönelimli Yazılım Mühendisliği UML, Desenler ve Java Kullanarak*, 3. baskı. Boston, MA: Addison-Wesley, 2010.

Cockburn, A. *Çevik Yazılım Geliştirme: İşbirlikçi Oyun,* 2. baskı. Boston, MA: Addison-Wesley, 2006.

Fox, C. *Yazılım Mühendisliği Tasarımına Giriş: Süreçler, İlkeler ve Desenler UML2 ile.*Boston, MA: Addison-Wesley, 2007.

Gamma, E., R. Helm, R. Johnson ve J. Vlissides. *Tasarım Desenleri: Yeniden Kullanılab ilir Nesne Yönelimli YazılımınÖgeleri*. Boston, MA: Addison-Wesley, 1995.

Maurer, P. M. *Bileşen Düzeyinde Programlama.*Upper Saddle River, NJ: Prentice-Hall, 2003.

Pfleeger, S. L. ve J. M. Atlee. *Yazılım Mühendisliği: Teori ve Pratik*, 4. baskı. Upper Saddle River, NJ: Prentice-Hall, 2010.

Pilone, D. *UML 2.0 Kapsamında.*Cambridge, MA: O'Reilly Media, 2005.

Doclingo https://www.doclingo.ai

Pressman, R. S. *Yazılım Mühendisliği: Uygulayıcı Yaklaşımı,* 7. baskı. New York: McGraw-Hill, 2009.

Schach, S. R. *Klasik ve Nesne Yönelimli Yazılım Mühendisliği,* 8. baskı. New York: McGraw-Hill, 2010.

Shalloway, A. ve J. R. Trott. *Tasarım Kalıpları Açıklandı,* 2. baskı. Boston, MA: Addison-Wesley, 2005.

Shneiderman, B., C. Plaisant, Cohen, M, ve Jacobs S. *Kullanıcı Arayüzü Tasarımı: Etkili İnsan-Bilgisayar Etkileşimi için Stratejiler*, 5. baskı. Boston, MA: Addison-Wesley, 2009.

Sommerville, I. *Yazılım Mühendisliği,* 8. baskı. Boston, MA: Addison-Wesley, 2006.

Doclingo https://www.doclingo.ai

Bu sayfa kasıtlı olarak boş bırakılmıştır

Doclingo https://www.doclingo.ai

BÖLÜM

Veri Soyutlamaları 8

Bu bölümde, bir bilgisayarın ana belleği tarafından sağlana n hücre hücre organizasyonu dışındaki veri düzenlemeleri nin nasıl simüle edilebileceğini araştırıyoruz—bu konu veri yapıları olarak bilinir. Amaç, verinin kullanıcısının verileri, b ilgisayarın ana bellek organizasyonu açısından düşünmek z orunda kalmadan soyut araçlar olarak erişebilmesini sağla maktır. Çalışmamız, bu tür soyut araçlar oluşturma arzusu nun nesneler ve nesne yönelimli programlama kavramına nasıl yol açtığını gösterecektir.

8.1 Temel Veri Yapıları
Diziler
Listeler, Yığınlar ve Kuyruklar
Ağaçlar

8.2 İlgili Kavramlar
Abstraksiyon Yen idenStatik ile Dinamik Yapılar
Pointernlar

8.3 Veri Uygulama Yapılar
Dizileri Saklama
Listeleri Saklama
Yığınları ve Kuyrukları Saklama
İkili Ağaçları Saklama
Veri Yapılarını Manipüle Etme

8.4 Kısa Bir Vaka Çalışması

8.5 Özelleştirilmiş Veri Türleri
Kullanıcı Tanımlı Veri Türleri
Soyut Veri Türleri

***8.6Sınıflar ve Nesnele**

r*8.7Makinedeki Göst ergeler

**Yıldızlar, isteğe bağlı bölümler için önerileri belirtir.*

Doclingo https://www.doclingo.ai

Veri yapısı kavramını Bölüm 6'da tanıttık; burada yüksek seviyeli programla ma dillerinin programcıların algoritmaları, işlenen verilerin bir bilgisayarın a na belleği tarafından sağlanan hücre hücre düzenlemesinden farklı şekillerd e depolandığı gibi ifade etmelerini sağlayan teknikler sunduğunu öğrendik. Ayrıca, bir programlama dilinin desteklediği veri yapılarının ilkel yapılar olara k bilindiğini de öğrendik. Bu bölümde, bir dilin ilkel yapılarından başka veri y apılarının nasıl inşa edileceği ve manipüle edileceği tekniklerini keşfedeceğiz —bu çalışma bizi geleneksel veri yapılarından nesne yönelimli paradigmaya götürecektir. Bu ilerleme boyunca temel bir tema, soyut araçların inşasıdır.

8.1 Temel Veri Yapıları

Çalışmamıza, gelecekteki bölümlerde örnek olarak hizmet edecek bazı temel veri yapıları tanıtarak başlıyoruz.

Diziler

Bölüm 6.2'de, homojen ve heterojen diziler olarak bilinen veri yapıları hakkın da bilgi edindik. Hatırlayın ki, bir homojen dizi, girişlerinin aynı türde olduğu "dikdörtgen" bir veri bloğudur. Özellikle, iki boyutlu bir homojen dizi, konum ların çift indeksler ile tanımlandığı satırlar ve sütunlardan oluşur—ilk indeks, konumla ilişkili satırı tanımlar, ikinci indeks ise sütunu tanımlar. Bir örnek, bi r satış gücünün üyeleri tarafından yapılan aylık satışları temsil eden sayılarda n oluşan dikdörtgen bir dizi olacaktır—her satırdaki girişler belirli bir üye tar afından yapılan aylık satışları ve her sütundaki girişler ise belirli bir ay için he r üye tarafından yapılan satışları temsil eder. Böylece, üçüncü satır ve birinci sütundaki giriş, Ocak ayında üçüncü satış elemanı tarafından yapılan satışlar ı temsil eder.

Homojen bir diziye karşı, heterojen bir dizi, farklı türlerde olabilecek veri öğelerinin birbloğu olduğunu hatırlayın. Blok içindeki öğelere genellikle bil eşenler denir. Heterojen bir dizinin bir örneği,bir çalışana ait verilerin bloğu olabilir; bu bileşenler çalışanın adı (karakter türünde), yaşı (tam sayı türünde) ve beceri puanı(gerçek türünde) olabilir.

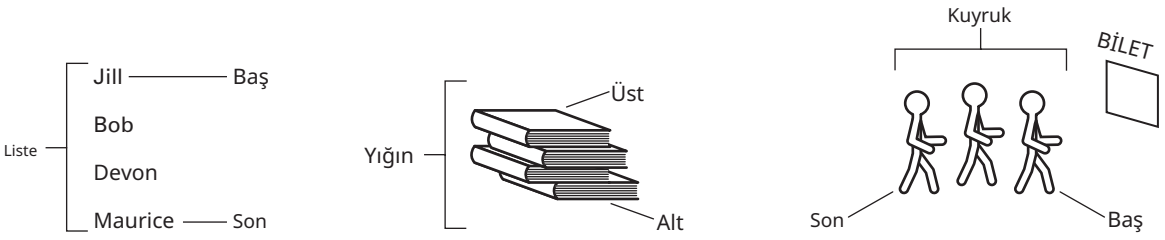
Listeler, Yığınlar ve Kuyruklar

Bir diğer temel veri yapısı liste dir; bu, girişlerinin sıralı olarak düzenlendiği bir koleksiyondur(Şekil 8.1a). Bir listenin başlangıcına listenin başı denir. Bir listeni n diğer ucu ise kuyruğu olarak adlandırılır.

Neredeyse her veri koleksiyonu bir liste olarak düşünülebilir. Örneğin, metin bir semboller listesi olarak,düzyüzeysel bir dizi bir satırlar listesi olarak ve bir CD'y e kaydedilen müzik bir sesler listesi olarak düşünülebilir. Dahageleneksel örnekle r arasında misafir listeleri, alışveriş listeleri, sınıf kayıt listeleri vestok listeleri bulu nmaktadır. Bir listeyle ilişkili aktiviteler duruma bağlı olarak değişir. Bazı durumlar da,bir listeden girişleri kaldırmamız, bir listeye yeni girişler eklememiz,bir listedek i girişleri birer birer "işlememiz", girişlerin düzenini değiştirmemiz gerekebilir.

Doclingo https://www.doclingo.ai

Şekil **8.1** Listeler, yığınlar ve kuyruklar



a. Bir isimler listesi **b.** Bir yığın kitap **c.** Bir kuyruk insan

bir liste, ya da belki belirli bir öğenin bir listede olup olmadığını kontrol etmek. Bu bölümde bu tür işlemleri daha sonra inceleyeceğiz.

Bir listenin girişlerinin erişim şeklini kısıtlayarak, yığınlar ve kuyruklar ola rak bilinen iki özel liste türü elde ederiz. Bir yığın, girişlerin yalnızca baştan e klendiği ve çıkarıldığı bir listedir. Fiziksel kısıtlamaların tüm eklemelerin ve sil melerin üstte gerçekleşmesini zorunlu kıldığı bir kitap yığını örneğidir(Şekil 8 .1b). Günlük terminolojiye göre, bir yığının başına yığın üstü denir. Bir yığının kuyruğuna ise alt veya taban denir. Yığının üstüne yeni bir giriş eklemeye it me denir. Yığının üstünden bir girişi çıkarmaya ise çıkarma denir. Yığına yerl eştirilen son giriş her zaman çıkarılan ilk giriş olacaktır—bu gözlem, bir yığını n son giren, ilk çıkan veya**LIFO** (okunuşu "LIE-foe") yapısı olarak bilinmesine yol açar.

Bu LIFO özelliği, bir yığının, saklandıkları ters sırayla geri alınması gereke n öğeleri depolamak için ideal olduğu anlamına gelir ve bu nedenle bir yığın genellikle geri izleme faaliyetlerinin temelini oluşturur. (Geri izleme terimi, bi r sistemden, sistemin girildiği ters sırayla geri çıkma sürecini ifade eder. Klas ik bir örnek, bir ormandan çıkmak için adımların geri izlenmesi sürecidir.) Ör neğin, bir özyinelemeli süreci desteklemek için gereken temel yapıyı düşünü n. Her yeni aktivasyon başlatıldığında, önceki aktivasyon bir kenara konulma lıdır. Ayrıca, her aktivasyon tamamlandıkça, bir kenara konulan son aktivasy on geri alınmalıdır. Böylece, eğer aktivasyonlar bir yığının üzerine konulursa, her seferinde geri alınması gereken aktivasyon yığının üstünde olacaktır.

A kuyruk, girişlerin yalnızca baştan çıkarıldığı ve yenigirişlerin yalnızca ku yrukta sona eklendiği bir listedir. Bir örnek, bir tiyatroda bilet almak için bekl eyen insanlarınsırasıdır (Şekil 8.1c)—kuyruğun başındaki kişi hizmet alırken y eni gelenler kuyrukta arka tarafa (veya sona) geçer. Daha önce, bir toplu işle me işletim sisteminin, yürütülmeyi bekleyen işleri bir iş kuyruk adı verilen bir kuyrukta sakladığını gördüğümüz 3. Bölümde kuyruk yapısıyla tanıştık. Ora da ayrıca bir kuyruğun ilk giren, ilk çıkan, veya**FIFO** (okunuşu "FIE-foe") yapıs ı olduğunu öğrendik; bu, girişlerin saklandıkları sırayla kuyruktan çıkarıldığı anlamına gelir.

Kuyruklar genellikle bir tamponun temel yapısı olarak kullanılır; bu, 1. Bö lümde tanıtıldığı gibi, geçici veri yerleştirme alanıdır.

Doclingo https://www.doclingo.ai

bir yerden başka bir yere aktarılır. Veriler tampona geldiğinde,kuyrukta en s ona yerleştirilir. Daha sonra, öğeleri nihai varış yerine iletme zamanı geldiğin de,kuyrukta başta bulunan öğelerin sırasına göre iletilir. Böylece, öğeler gel dikleri sırayla iletilir.

Ağaçlar

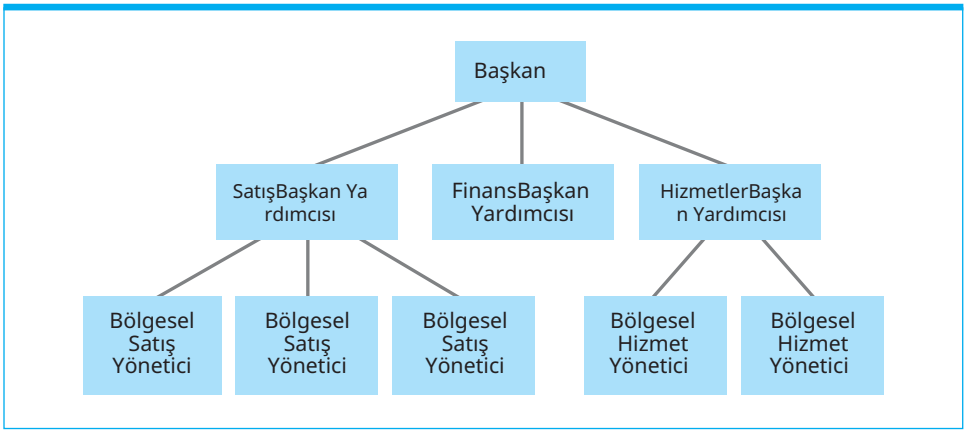
Bir ağaç, girişlerinin bir organizasyon şemasına benzer hiyerarşik bir organiz asyona sahip olduğu bir koleksiyondur (Şekil 8.2). Başkan en üstte temsil edil ir, aşağıya doğru başkan yardımcılarına, ardından bölgesel yöneticilere ve de vamına doğru dallar uzanır. Bu sezgisel ağaç yapısı tanımına bir ek kısıtlama getiriyoruz; bu, (bir organizasyon şeması açısından) şirketteki hiçbir bireyin i ki farklı üst düzey yöneticisine rapor vermemesi gerektiğidir. Yani, organizas yonun farklı dalları daha alt bir seviyede birleşmez.
(Ağaçların, 6. Bölüm'de ayrıştırma ağaçları biçiminde ortaya çıktığı örnekleri zaten gördük.)

Bir ağaçtaki her pozisyona düğüm (Şekil 8.3) denir. En üstteki düğüme kö k düğüm denir (çizimi ters çevirirsek, bu düğüm ağacın tabanını veya kökün ü temsil eder). Diğer uçtaki düğümler terminal düğümler (veya bazen yaprak düğümler) olarak adlandırılır. Kökten bir yaprağa kadar en uzun yoldaki düğ üm sayısına ağacın derinliği denir. Diğer bir deyişle, bir ağacın derinliği içind eki yatay katman sayısıdır.

Bazen ağaç yapılarından bahsederken, her düğümün hemen altındaki düğüm leri doğurduğu gibi düşünürüz. Bu anlamda, genellikle bir düğümün atalarından veya torunlarından bahsederiz. Hemen altındaki torunlarına çocukları ve hemen ü stündeki atasına ebeveyni deriz. Ayrıca, aynı ebeveyni olan düğümler kardeş olara k adlandırılır. Her ebeveynin en fazla iki çocuğa sahip olduğu bir ağaç ikili ağaç ol arak adlandırılır.

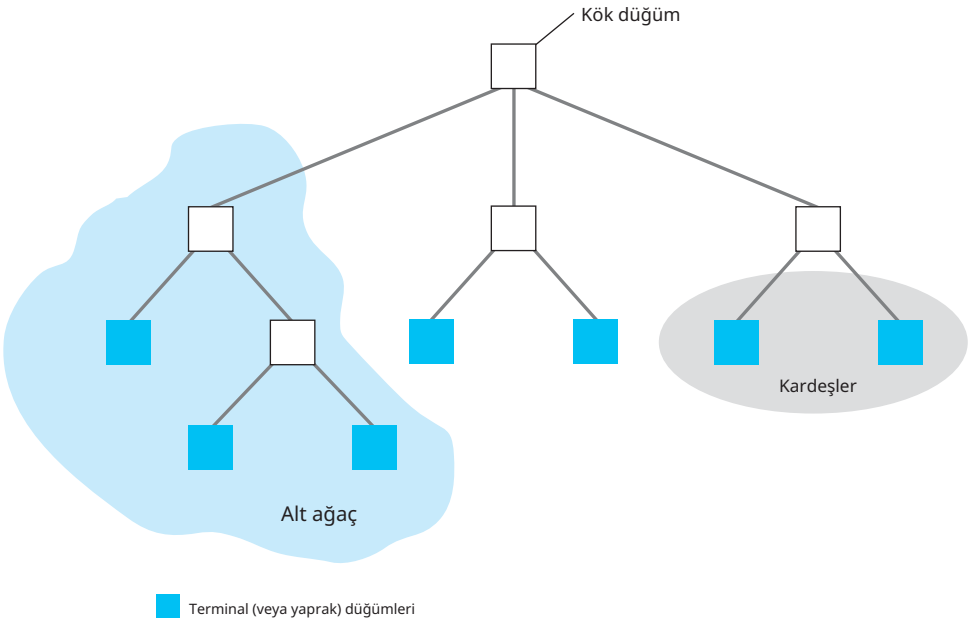
Bir ağaçtaki herhangi bir düğümü seçtiğimizde, o düğümün altında bulunan düğümle rle birlikte bir ağaç yapısına sahip olduğunu görürüz. Bu daha küçük yapılara alt ağaçlar di yoruz. Böylece, her çocuk düğümü, çocuğun ebeveyninin altında bir alt ağacın köküdür. He r böyle alt ağaç, ebeveynin bir dalı olarak adlandırılır. İkili ağaçta, genellikle bir düğümün s ol dalı veya sağ dalı hakkında konuşuruz, bu ağaç yapısının nasıl görüntülendiğine atıfta bu lunarak.

Şekil **8.2** Bir organizasyon şemasına örnek



Doclingo https://www.doclingo.ai

Şekil **8.3**Ağaç terminolojisi



Sorular & Alıştırmalar

1. Aşağıdaki her bir yapının (bilgisayar bilimi dışında) örneklerini verin: liste, yığın, kuyruk ve ağaç.
2. Listeler, yığınlar ve kuyruklar arasındaki farkı özetleyin.
3. A harfi boş bir yığına itildiğinde, ardından B ve C harfleri bu sırayla iti ldiğinde varsayalım. Sonra bir harf yığından çıkarıldığında ve D ile E h arfleri itildiğinde, yığının üstten alta doğru hangi harfleri içereceğini l isteleyin. Eğer bir harf yığından çıkarılırsa, hangi harf alınacaktır?
4. Varsayalım ki A harfi boş bir kuyruğa yerleştirildi, ardından B ve C ha rfleribu sırayla eklendi. Sonra bir harfin kuyruktan çıkarıldığını ve D il e E harflerinineklendiğini varsayalım. Kuyrukta baştan sona doğru ha ngi harflerin olacağınılisteleyin. Şu anda kuyruktan bir harf çıkarılırsa , hangi harf olur?
5. Varsayalım ki bir ağaçta A, B, C ve D adında dört düğüm var. Eğer A ve Ckardeşse ve D'nin e beveyni A ise, hangi düğümler yaprak düğümleridir? Hangi düğümkök düğümdür?

8.2 İlgili Kavramlar

Bu bölümde veri yapıları ile yakından ilişkili üç konuyu izole ediyoruz:abstrak siyon, statik ve dinamik yapılar arasındaki ayrım ve bir işaretçi kavramı.

Doclingo https://www.doclingo.ai

Yine Abstraksiyon

Önceki bölümde sunulan yapılar genellikle veri ile ilişkilidir. Ancak, bir bilgisayarın ana belleği diziler, listeler, yığınlar, kuyruklar ve ağaçlar olarak organize edilmemiştir, bunun yerine adreslenebilir bellek hücrelerinin bir dizisi olarak organize edilmiştir. Bu nedenle, diğer tüm yapılar simüle edilmelidir. Bu simülasyonun nasıl gerçekleştirildiği bu bölümün konusudur. Şimdilik, diziler, listeler, yığınlar, kuyruklar ve ağaçlar gibi organizasyonların, veri kullanıcılarının gerçek veri depolama detaylarından korunabilmesi ve bilgilere daha uygun bir biçimde depolanmış gibi erişebilmesi için oluşturulmuş soyut araçlar olduğunu belirtmekle yetiniyoruz.

Kullanıcı terimi bu bağlamda mutlaka bir insana atıfta bulunmaz. Bunun yerine, kelimenin anlamı o anki bakış açımıza bağlıdır. Eğer bir kişinin bowlin g ligi kayıtlarını tutmak için bir PC kullandığını düşünüyorsak, o zaman kullan ıcı bir insandır. Bu durumda, uygulama yazılımı (belki bir elektronik tablo yaz ılımı paketi) verileri insana uygun soyut bir biçimde sunmaktan sorumlu olac aktır—muhtemelen homojen bir dizi olarak. Eğer bir sunucunun İnternet üz erindeki işleyişini düşünüyorsak, o zaman kullanıcı bir istemci olabilir. Bu dur umda, sunucu verileri istemciye uygun soyut bir biçimde sunmaktan soruml u olacaktır. Eğer bir programın modüler yapısını düşünüyorsak, o zaman kull anıcı verilere erişim gerektiren herhangi bir modül olacaktır. Bu durumda, v erileri içeren modül, verileri diğer modüllere uygun soyut bir biçimde sunma ktan sorumlu olacaktır. Bu senaryoların her birinde ortak nokta, kullanıcının veriye soyut bir araç olarak erişim ayrıcalığına sahip olmasıdır.

Statik ve Dinamik Yapılar

Soyut veri yapıları oluştururken önemli bir ayrım, simüle edilen yapının statik mi yoksa dinamik mi olduğudur; yani yapının şeklinin veya boyutunun zama nla değişip değişmediğidir. Örneğin, soyut araç bir isimler listesi ise, listenin varlığı boyunca sabit bir boyutta kalıp kalmayacağını veya isimler eklendikçe ve silindikçe genişleyip daralacağını düşünmek önemlidir.

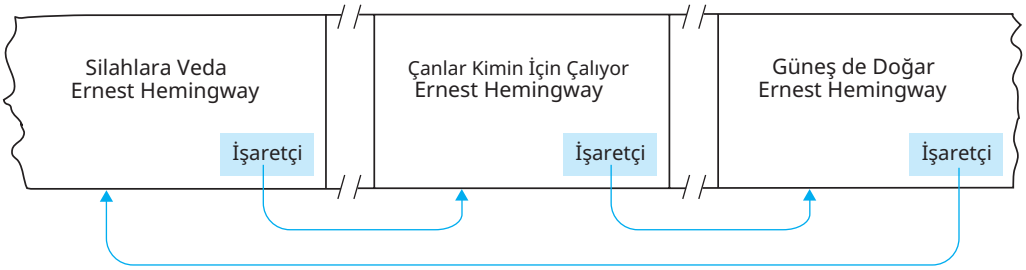
Genel bir kural olarak, statik yapılar dinamik yapılardan daha kolay yönet ilir.Eğer bir yapı statikse, yapının içindeki çeşitli veri öğelerine erişim sağlam ak içinsadece bir yol sunmamız ve belki de belirli yerlerdeki değerleri değiştir mek için bir yolsağlamamız gerekir. Ancak, yapı dinamikse, ekleme ve silme i şlemleriyle birliktebüyüyen bir veri yapısının gerektirdiği bellek alanıyla da il gilenmemiz gerekir.Kötü tasarlanmış bir yapı durumunda, tek bir yeni giriş e klemek, yapının büyük birşekilde yeniden düzenlenmesine neden olabilir ve aşırı büyüme, tüm yapının daha fazlaalanın mevcut olduğu başka bir bellek a lanına aktarılmasını gerektirebilir.

Pointernlar

Bir makinenin ana belleğindeki çeşitli hücrelerin sayısal adreslerle tanımlandığını unutmayın. Sayısal değerler olduğundan, bu adresler kendileri de kodlanabilir ve bellek hücrelerinde saklanabilir. Bir işaretçi, böyle bir kodlanmış adresi içeren bir depolama alanıdır. Veri yapıları durumunda, işaretçiler veri öğelerinin saklandığı yeri kaydetmek için kullanılır. Örneğin, bir veri öğesini bir yerden başka bir yere tekrar tekrar taşımamız gerekiyorsa, bir işaretçi olarak hizmet etmesi için sabit bir yer belirleyebiliriz. Sonra, her seferinde öğeyi taşırken,

Doclingo https://www.doclingo.ai

Şekil 8.4 Başlıklarına göre düzenlenmiş ancak yazarlıklarına göre bağlantılı romanlar



veri için yeni adresi yansıtacak şekilde işaretçiyi güncelleyin. Daha sonra, verinin öğe sine erişmemiz gerektiğinde, onu işaretçi aracılığıyla bulabiliriz. Gerçekten de, işaret çi her zaman veriye "işaret edecektir".

CPU'lar üzerine yaptığımız çalışmada, 2. Bölümde işaretçi kavramıyla zaten karşılaştık. Orada, bir sonraki yürütülecek talimatın adresini tutmak için bir pro gram sayacı adı verilen bir kayıt kullanıldığını bulmuştuk. Böylece, program say acı bir işaretçi rolü oynamaktadır. Aslında, program sayacının bir diğer adı da t alimat işaretçisidir.

İşaretçilerin uygulanmasına bir örnek olarak, bilgisayarın belleğinde başlıkları na göre alfabetik olarak saklanan bir roman listemiz olduğunu varsayalım. Birçok uygulamada kullanışlı olmasına rağmen, bu düzenleme belirli bir yazarın tüm ro manlarını bulmayı zorlaştırır; çünkü romanlar liste boyunca dağılmıştır. Bu sorunu çözmek için, her romanı temsil eden hücreler bloğunda ek bir bellek hücresi ayıra biliriz ve bu hücreyi aynı yazarın bir kitabını temsil eden başka bir bloğa işaretçi ol arak kullanabiliriz. Bu şekilde, ortak yazarlığa sahip romanlar bir döngü içinde ba ğlanabilir (Şekil 8.4). Belirli bir yazarın bir romanını bulduğumuzda, bir kitaptan di ğerine işaretçileri takip ederek diğerlerini bulabiliriz.

Birçok modern programlama dili, işaretçileri temel bir veri tipi olarak içerir. Yani, işaretçilerin beyanı, tahsisi ve manipülasyonu, tamsayılar ve karakter dizil eriyle benzer şekillerde yapılmasına izin verir. Böyle bir dili kullanarak, bir prog ramcı, bir makinenin belleğinde veri için karmaşık ağlar tasarlayabilir; burada i şaretçiler, ilgili öğeleri birbirine bağlamak için kullanılır.

Sorular & Alıştırmalar

1. Veri yapıları, diziler, listeler, yığınlar, kuyruklar ve ağaçlar gibi, hangi anla mda soyutlamalardır?
2. Statik bir veri yapısını içereceğini beklediğiniz bir uygulamayı tanımla yın. Ardından, dinamik bir veri yapısını içereceğini beklediğiniz bir uy gulamayı tanımlayın.
3. İşaretçi kavramının ortaya çıktığı bilgisayar bilimi dışındaki bağlamları t anımlayın.

Doclingo https://www.doclingo.ai

8.3 Veri Yapılarını Uygulama

Şimdi, önceki bölümde tartışılan veri yapıların bir bilgisayarın ana belleğinde nasıl saklanabileceğini düşünelim.

Dizileri Saklama

Dizileri saklama teknikleriyle başlayalım. Bölüm 6'da gördüğümüz gibi, bu ya pılar genellikle yüksek seviyeli programlama dillerinde ilkel yapılar olarak sa ğlanır. Buradaki amacımız, bu tür yapılarla ilgilenen programların, ana bellek te saklanan verileri manipüle eden makine dili programlarına nasıl çevrildiği ni anlamaktır.

Homojen Diziler 24 saatlik sıcaklık ölçümünün bir dizisini saklamak istediğimi zi varsayalım; her biri bir bellek hücresi depolama alanı gerektirir.
Ayrıca, bu okumaları dizideki konumlarıyla tanımlamak istediğimizi varsayalı m. Yani, ilk okumaya veya beşinci okumaya erişebilmek istiyoruz. Kısacası, di ziyi tek boyutlu homojen bir diziymiş gibi manipüle etmek istiyoruz.

Bu hedefe, okumaları ardışık adreslere sahip 24 bellek hücresinin bir dizi sinde saklayarak ulaşabiliriz. O zaman, dizideki ilk hücrenin adresi x ise, belirl i bir sıcaklık okumasının yeri, istenen okumanın indeksinden bir çıkararak ve ardından sonucu x'ye ekleyerek hesaplanabilir. Özellikle, dördüncü okuma a dres x + (41) konumunda olacaktır; bu, Şekil 8.5'te gösterilmiştir.

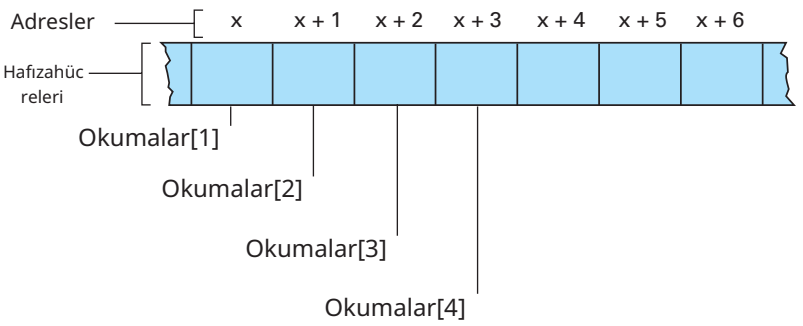
Bu teknik, yüksek seviyeli programlama dillerinin çoğu çevirmeni tarafında n tek boyutlu homojen dizileri uygulamak için kullanılır. Çevirmen, aşağıdaki gi bi bir bildirim ifadesiyle karşılaştığında

```
int Okumalar[24];
```

terimin `Readings`'in 24 tamsayı değerinden oluşan tek boyutlu bir diziye atıfta bulunacağını bildirdiğini, çevirmen 24 ardışık bellek hücresinin ayrılmasını sağl ar. Programın ilerleyen kısımlarında, atama ifadesiyle karşılaştığında

```
Okumalar[4]←67;
```

Şekil **8.5** Bellekte x adresinden başlayan sıcaklık okumalarının dizisi



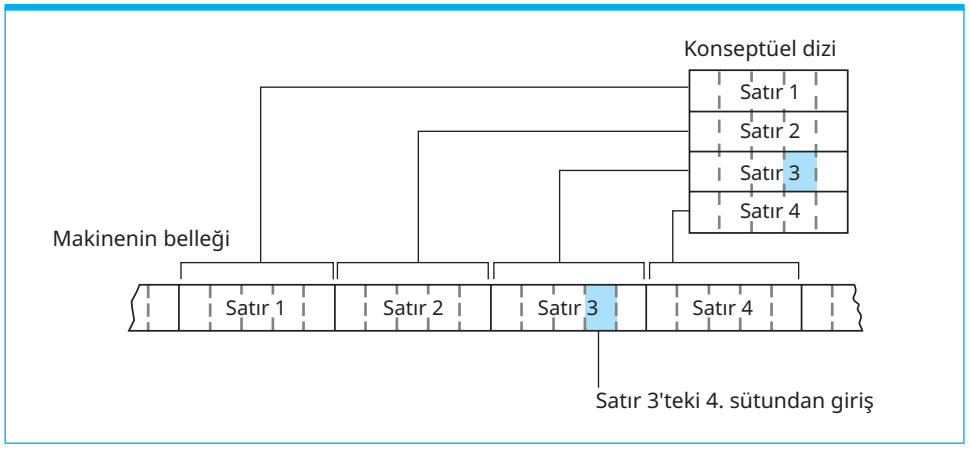
Doclingo https://www.doclingo.ai

67 değerinin dizinin dördüncü girişine Okumalar konulmasını talep eden çe virmen, 67 değerini x adresindeki bellek hücresine yerleştirmek için gereken makine talimatları dizisini oluşturur + (41), burada x, Okumalar ile ilişkili blo ktaki ilk hücrenin adresidir. Bu şekilde, programcı, sıcaklık okumalarının gerç ekten tek boyutlu bir dizide saklanıyormuş gibi programı yazmasına izin veril ir. (Dikkat: C, C++, C# ve Java dillerinde, dizi indeksleri 1 yerine 0'dan başlar, bu nedenle dördüncü okuma Okumalar[3] ile referans alınır. Bu bölümün so nunda Soru/Alıştırma 3'e bakın.) Şimdi, bir şirketin satış gücünün bir haft alık süre içinde yaptığı satışları kaydetmek istediğimizi varsayalım. Bu du rumda, verilerin her bir satırda belirli bir çalışanın yaptığı satışları gösterdiği ve bir sütunda belirli bir günde yapılan tüm satışları temsil ettiği iki boyutlu h omojen bir dizide düzenlendiğini hayal edebiliriz.

Bu ihtiyacı karşılamak için, önce dizinin güncellemeler yapıldıkça boyutunu n değişmediği anlamında statik olduğunu kabul ediyoruz. Bu nedenle, tüm dizi için gereken depolama alanının miktarını hesaplayabiliriz ve o boyutta kesintisi z bellek hücreleri bloğu ayırabiliriz. Sonra, verileri diziye satır satır depoluyoruz. Ayrılmış bloğun ilk hücresinden başlayarak, dizinin ilk satırındaki değerleri ardı şık bellek konumlarına depoluyoruz; bunun ardından bir sonraki satırı, sonra bi r sonraki satırı ve devam ediyoruz (Şekil 8.6). Böyle bir depolama sisteminin, diz inin sütun sütun depolandığı sütun ana sırası ile karşılaştırıldığında satır ana s ırası kullandığı söylenir.

Veriler bu şekilde depolandığında, dizinin üçüncü satırındaki ve dördüncü süt unundaki değeri nasıl bulabileceğimizi düşünelim. Makinenin belleğindeki ayrılmı ş bloğun ilk konumunda olduğumuzu hayal edin. Bu konumdan başlayarak, dizini n ilk satırındaki verileri buluyoruz, ardından ikinci satıra, sonra üçüncü satıra ve d evam ediyoruz. Üçüncü satıra ulaşmak için, hem birinci hem de ikinci satırın ötesi ne geçmemiz gerekiyor. Her satırın beş girişi (haftanın her günü için bir tane, Paz artesi'den Cuma'ya kadar) içerdiğinden, üçüncü satırın ilk girişine ulaşmak için to plamda 10 girişin ötesine geçmemiz gerekiyor. Oradan, satırın dördüncü sütunun daki girişe ulaşmak için başka üç girişin ötesine geçmemiz gerekiyor.
Toplamda, üçüncü satıra ve dördüncü sütuna ulaşmak için, blok başlangıcında n itibaren 13 girişin ötesine geçmemiz gerekiyor.

Şekil **8.6** Dört satır ve beş sütun içeren iki boyutlu bir dizi, satır ana sırasına göre saklanmıştır



Doclingo https://www.doclingo.ai

Ardışık Listelerin Uygulanması

Çoğu yüksek seviyeli programlama dillerinde sağlanan dizileri oluşturma ve manipüle etme için kullanılan ilkel, bitişik listeler oluşturmak ve manipüle etmek için uygun araçlardır. Eğer listenin tüm girişleri aynı ilkel veri türündeyse, o zaman liste sadece bir boyutlu homojen bir dizi olmaktan başka bir şey değildir. Daha karmaşık bir örnek, metinde tartışıldığı gibi, her biri sekiz karakterden uzun olmayan on isimden oluşan bir listedir. Bu durumda, bir programcı, on satır ve sekiz sütunlu karakterlerden oluşan iki boyutlu bir dizi olarak bitişik listeyi oluşturabilir; bu, Şekil 8.6'da temsil edilen yapıyı üretecektir (dizinin satır ana sırasına göre saklandığı varsayılarak).

Birçok yüksek seviyeli dil, bu tür liste uygulamalarını teşvik eden özellikler içerir. Örneğin, yukarıda önerilen iki boyutlu karakter dizisinin `MemberList` olarak adlandırıldığını varsayalım. O zaman, `MemberList[3,5]` ifadesinin üçüncü satır ve beşinci sütundaki tek karakteri ifade ettiği geleneksel notasyona ek olarak, bazı diller `MemberList[3]` ifadesini tüm üçüncü satırı ifade etmek için benimser; bu da listedeki üçüncü girişi temsil eder.

Önceki hesaplama, satır ve sütun pozisyonları açısından referansları gerçek bellek adreslerine dönüştürmek için bir formül elde etmek üzere genelleştirilebilir. Özellikle, c bir dizideki sütun sayısını temsil ederse (bu, her satırdaki giriş sayısıdır), o zaman i'inci satır ve j'inci sütundaki girişin adresi şöyle olacaktır:

$$x + (c \times (i - 1)) + (j - 1)$$

burada x, birinci satır ve birinci sütundaki girişi içeren hücrenin adresidir. Yani, i $-$ 1 satır, her biri *c içerir* girişler, i. satıra ulaşmak ve ardından j $-$ 1 daha giriş, jth girişine ulaşmak için bu satır. Önceki örneğimizde c $=$ $_{5, \text{ben}}$ $=$ 3, ve j $=$ 4, bu yüzden dizi saklanıyorsa adres x ile başlayarak, üçüncü satırdaki, dördüncü sütundaki giriş adres x+ (5×(3 1)()4 1)x= 13. İfade (c×(i1))+(j1) bazen adres polinom olarak adlandırılır.

Bir kez daha, bu, yüksek seviyeli programlama dillerinin çoğu çevirmeni tarafından kullanılan tekniktir. Beyan ifadesi ile karşılaştıklarında

```
int Satışlar[8, 5];
```

terimin Satış olarak, 8 satır ve 5 sütundan oluşan bir iki boyutlu tamsayı dizisini ifade edeceğini beyan ederek, çevirmen 40 ardışık bellek hücresinin ayrılmasını sağlar. Daha sonra, eğer atama ifadesiyle karşılaşırsa

```
Sales[3, 4] ← 5;
```

5 değerinin dizinin üçüncü satır ve dördüncü sütunundaki girişe yerleştirilmesi talep edildiğinde, 5 değerini x+5× (3 1)(41) adresine sahip bellek hücresine yerleştirmek için gereken makine talimatları dizisini oluşturur, burada x, dizi ile ilişkili bloktaki ilk hücrenin adresidir.
Satış. Bu şekilde, programcı programı, sanki satışlar gerçekten iki boyutlu bir dizide saklanıyormuş gibi yazmasına izin verilir.

Doclingo https://www.doclingo.ai

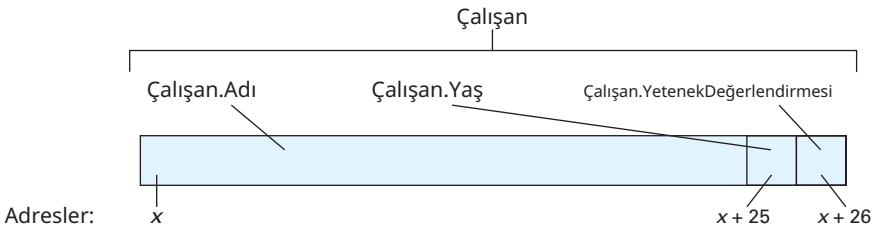
Heterojen Diziler Şimdi, `Name` karakter türünde, `Age` tam sayı türünde ve `SkillRating` re el türünde üç bileşenden oluşan `Employee` adında bir heterojen dizi saklamak iste diğimizi varsayalım. Her bileşenin gerektirdiği bellek hücresi sayısı sabitse, o zaman diziyi bitişik hücreler bloğunda saklayabiliriz. Örneğin, `Name` bileşeninin en fazla 25 hücre gerektirdiğini, `Age` bileşeninin yalnızca bir hücre gerektirdiğini ve `SkillRating` bileşeninin de yalnızca bir hücre gerektirdiğini varsayalım. O zaman , 27 bitişik hücreden oluşan bir blok ayırabilir, ismi ilk 25 hücrede saklayabilir, yaşı 2 6. hücrede saklayabilir ve beceri puanını son hücrede saklayabiliriz (Şekil 8.7a).

Bu düzenleme ile dizinin içindeki farklı bileşenlere erişmek kolay olacaktı r. İlk hücrenin adresi x ise, `Employee.Name` (dizideki `Name` bileşenini ifade e der) ifadesine yapılan her atıf, bu adresi kullanarak erişim sağlayacaktır. Çalışan) adresi x ile başlayan 25 hücreye çevrilecektir ve bir referansto Çalışa n.Yaş (Çalışan içindeki Yaş bileşeni) $x+$ 25 adresindeki hücreye çevrilecektir. Özellikle, bir çevirmen böyle bir ifadeyi bulursa

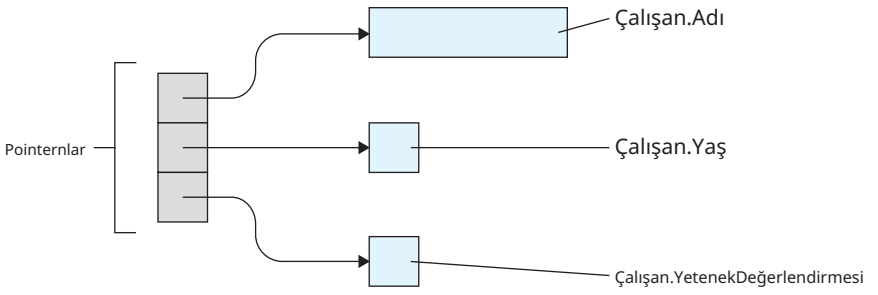
Çalışan.Yaş←`22;`

yüksek seviyeli bir programda, o zaman yalnızca değeri 22'yi $x+$ 25 adresine sahip b ellek hücresine yerleştirmek için gereken makine dili talimatlarını oluşturacaktır.

Şekil **8.7** Heterojen dizinin saklanması Çalışan



a. Sürekli bir blokta saklanan dizi



b. Ayrı konumlarda saklanan dizi bileşenleri

Doclingo https://www.doclingo.ai

Ya da, eğer `EmployeeOfMonth`, adres y'de saklanan benzer bir dizi olarak tanımlanmışsa, şu ifade

```
EmployeeOfMonth← Çalışan;
```

27 hücrenin içeriğini adres x'den başlayarak 27 hücreye kopyalayan bir talim at dizisine dönüşecektir.

Heterojen bir diziyi sürekli bellek hücreleri bloğunda saklamanın bir alter natifi, her bileşeni ayrı bir konumda saklamak ve ardından bunları işaretçiler aracılığıyla birbirine bağlamaktır. Daha kesin olarak, eğer dizi üç bileşen içeri yorsa, o zaman bellekte üç işaretçi saklamak için bir yer buluruz; her biri bile şenlerden birine işaret eder (Şekil 8.7b). Eğer bu işaretçiler adres x'den başla yan bir blokta saklanıyorsa, o zaman ilk bileşen, konum x'de saklanan işaretç iyi takip ederek bulunabilir, ikinci bileşen konum $x+$ 1'deki işaretçiyi takip ed erek bulunabilir ve bu şekilde devam eder.

Bu düzenleme, özellikle dizinin bileşenlerinin boyutunun dinamik olduğu durumlarda faydalıdır. Örneğin, işaretçi sistemini kullanarak, ilk bileşenin b oyutunu yalnızca daha büyük bileşeni tutacak bir alan bulup, ardından uygu n işaretçiyi yeni konuma yönlendirecek şekilde ayarlayarak artırabiliriz. Anca k dizi, ardışık bir blokta saklanıyorsa, tüm dizinin değiştirilmesi gerekecektir.

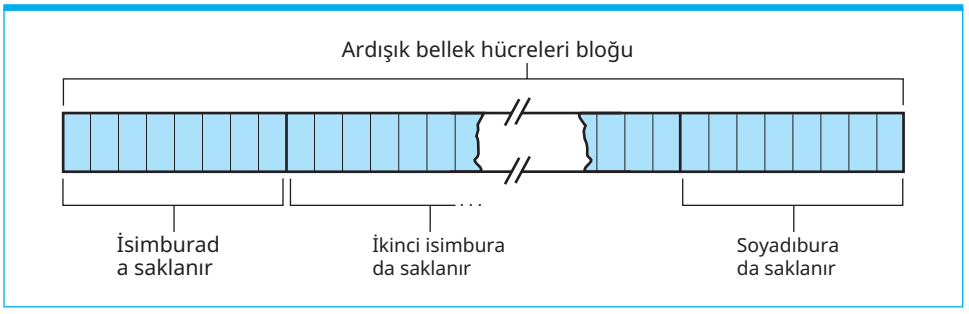
Listeleri Saklama

Şimdi bir bilgisayarın ana belleğinde bir isim listesi saklama tekniklerini ele a lalım. Bir strateji, tüm listeyi ardışık adreslere sahip tek bir bellek hücresi blo ğunda saklamaktır. Her ismin sekiz harften uzun olmadığını varsayarsak, bü yük hücre bloğunu her biri sekiz hücre içeren bir dizi alt bloğa bölebiliriz. Her alt bloğa, her harf için bir hücre kullanarak ASCII kodunu kaydederek bir isi m saklayabiliriz. Eğer isim, kendisine tahsis edilen alt bloktaki tüm hücreleri doldurmuyorsa, kalan hücreleri boşluk için ASCII kodu ile doldurabiliriz. Bu si stemi kullanmak, 10 ismi saklamak için 80 ardışık bellek hücresi gerektirir.

Az önce tanımlanan depolama sistemi Şekil 8.8'de özetlenmiştir. Önemli nokta, tüm listenin tek bir büyük bellek bloğunda saklanmasıdır; ardışık giriş ler birbirini takip eden ardışık bellek hücrelerinde yer alır. Bu tür bir organiza syona ardışık liste denir.

Bir ardışık liste, statik listeleri uygulamak için uygun bir depolama yapısıdır, anca k silme ve isimlerin eklenmesi durumunda dinamik listelerde dezavantajları vardır.

Şekil **8.8** Bellekte ardışık bir liste olarak saklanan isimler



Doclingo https://www.doclingo.ai

İsimlerin eklenmesi, girişlerin zaman alıcı bir şekilde karıştırılmasına yol açabilir. En kötü senaryoda, girişlerin eklenmesi, genişletilmiş liste için yeterince büyük bir hücre bloğu elde etmek amacıyla tüm listenin yeni bir konuma taşınmasını gerektirebilir.

Bu sorunlar, bir listedeki bireysel girişlerin, tek bir büyük ardışık blokta bir arada değil, bellek içinde farklı alanlarda saklanmasına izin verirsek basitleştirilebilir. Açıklamak gerekirse, isimlerin saklandığı örneğimizi yeniden gözden geçirelim (her ismin en fazla sekiz karakter uzunluğunda olduğunu varsayıyoruz). Bu sefer her ismi dokuz ardışık bellek hücresinden oluşan bir blokta saklıyoruz. Bu hücrelerin ilk sekizi ismin kendisini tutmak için kullanılırken, son hücre listedeki bir sonraki isme işaretçi olarak kullanılır. Bu bağlantı sistemine dayanarak, liste, işaretçilerle birbirine bağlı birkaç küçük dokuz hücreli blok arasında dağılabilir. Bu tür bir organizasyona bağlı liste denir.

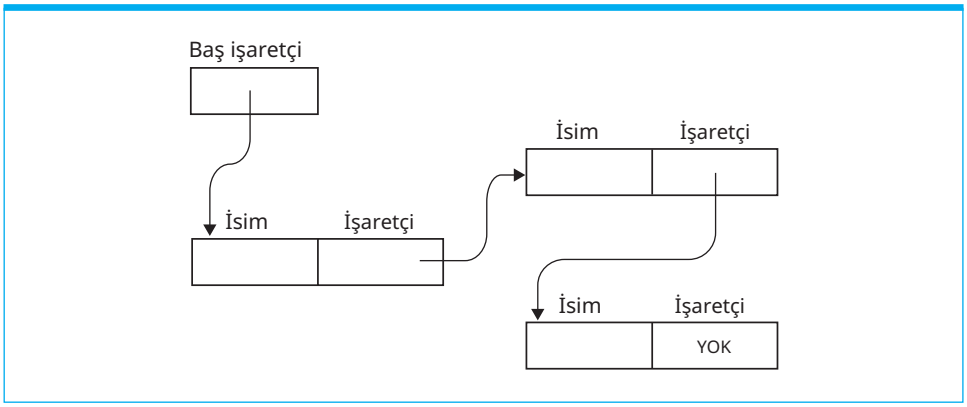
Bağlı bir listenin başlangıcını takip etmek için, ilk girişin adresini kaydettiğimiz başka bir işaretçi ayırıyoruz. Bu işaretçi listenin başlangıcına, yani başına işaret ettiğinden, baş işaretçi olarak adlandırılır.

Bağlı bir listenin sonunu işaretlemek için, NIL işaretçisi (aynı zamanda NULL işaretçisi olarak da bilinir) kullanıyoruz; bu, listenin son girişinin işaretçi hücresine yerleştirilen özel bir bit desenidir ve listenin daha fazla girişi olmadığını belirtir. Örneğin, bir liste girişini adres 0'da saklamamaya karar verirsek, sıfır değeri asla geçerli bir işaretçi değeri olarak görünmeyecek ve bu nedenle NIL işaretçisi olarak kullanılabilir.

Son bağlı liste yapısı, Şekil 8.9'daki diyagramla temsil edilmektedir; burada, liste için kullanılan dağınık bellek bloklarını bireysel dikdörtgenlerle gösteriyoruz. Her dikdörtgen, bileşimini belirtmek için etiketlenmiştir. Her işaretçi, işaretçiden işaretçinin muhatabına giden bir ok ile temsil edilir. Listeyi gezmek, ilk girişi bulmak için baş işaretçiyi takip etmeyi içerir. Oradan, girişlerle saklanan işaretçileri takip ederek bir girişten diğerine atlayarak NIL işaretçisine ulaşana kadar devam ederiz.

Bağlı bir listenin, bitişik bir listeye göre avantajlarını takdir etmek için, bir girişin silinmesi görevini düşünün. Bitişik bir listede bu bir delik oluşturur; bu, silinen girişin ardından gelen girişlerin listenin bitişik kalmasını sağlamak için ileri taşınmasını gerektiği anlamına gelir. Ancak, bağlı liste durumunda, bir giriş tek bir işaretçiyi değiştirerek silinebilir. Bu, daha önce silinen girişe işaret eden işaretçinin, silinen girişin ardından gelen girişe işaret edecek şekilde değiştirilmesiyle yapılır.

Şekil **8.9** Bağlı listenin yapısı



Doclingo https://www.doclingo.ai

Pointernlerle İlgili Bir Sorun

Akış şemalarının karmaşık algoritma tasarımlarına yol açtığı gibi (Bölüm 5) veras tgele kullanılan goto ifadelerinin kötü tasarlanmış programlara yol açtığı gibi (B ölüm 6),disiplinsiz pointer kullanımı gereksiz yere karmaşık vehata yapmaya yat kın veri yapıları üretmektedir. Bu kaosa düzen getirmek için, birçok programlam a dilipointerların esnekliğini kısıtlamaktadır. Örneğin, Java genel formda pointer lara izin vermez.Bunun yerine, yalnızca referanslar olarak adlandırılan kısıtlı bir pointer formuna izin verir.Bir ayrım, bir referansın aritmetik bir işlemle değiştiril emeyeceğidir. Örneğin, bir Java programcısıNext referansını bitişik bir listedeki bir sonraki girişe ilerletmek istiyorsa,şuna eşdeğer bir ifade kullanır:

```
Sonraki liste girişine yönlendir
```

bir C programcısının eşdeğer bir ifade kullanacağı yer

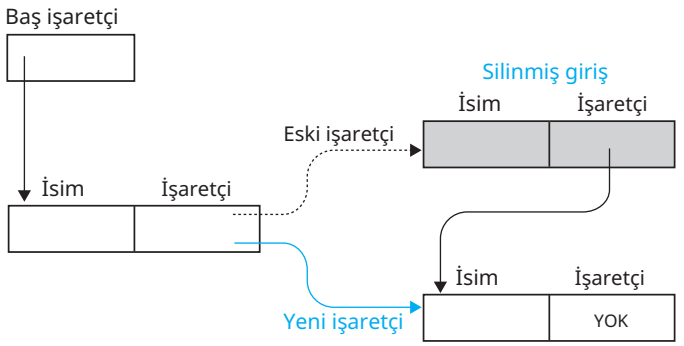
```
Sonraki'ye Sonraki değerini ata  + 1
```

Java ifadesinin temel amacı daha iyi yansıttığını unutmayın. Ayrıca, Java ifadesini yürü tmek için başka bir liste girişi olmalıdır, ancak eğer Sonraki zaten listedeki son girişe işaret ediyorsa, C ifadesi Sonraki'yi listenin dışında bir şeye işaret edecek şekilde son uçlanır—bu, yeni başlayanlar ve hatta deneyimli C programcıları için yaygın bir hatadır.

silinmiş giriş (Şekil 8.10). O zamandan beri, liste geçildiğinde, silinmiş giriş geçilir çünkü artık zincirin bir parçası değildir.

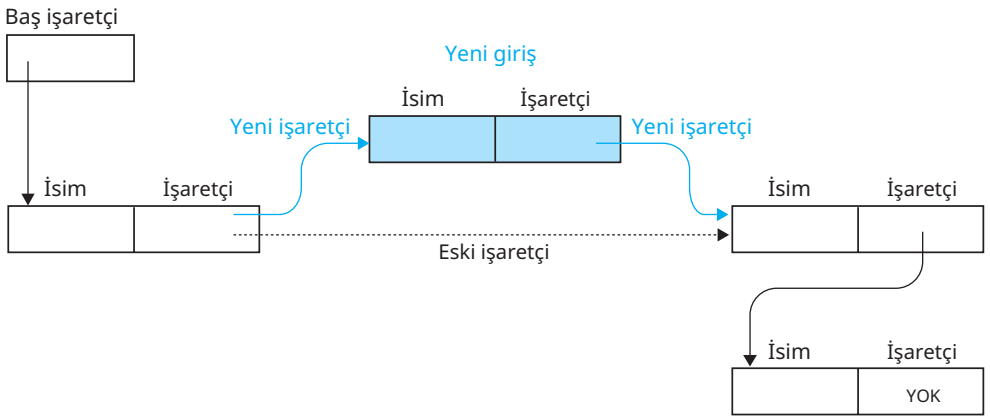
Bağlı bir listeye yeni bir giriş eklemek sadece biraz daha karmaşıktır. Önceli kle,yeni girişi ve bir işaretçiyi tutacak kadar büyük kullanılmamış bir bellek hücr esi bloğu buluruz.Burada yeni girişi saklarız ve işaretçiyi yeni girişten sonra gel mesi gerekenlistedeki girişin adresi ile doldururuz. Son olarak, yeni girişten ön ce gelmesi gereken girişle ilişkili işaretçiyi değiştiririz, böylece yeni girişe işaret eder (Şekil 8.11). Bu değişikliği yaptıktan sonra,yeni giriş her seferinde liste geç ildiğinde doğru yerde bulunacaktır.

Şekil **8.10**Bağlı bir listeden bir girişi silme



Doclingo https://www.doclingo.ai

Şekil **8.11**Bağlı listeye bir giriş ekleme



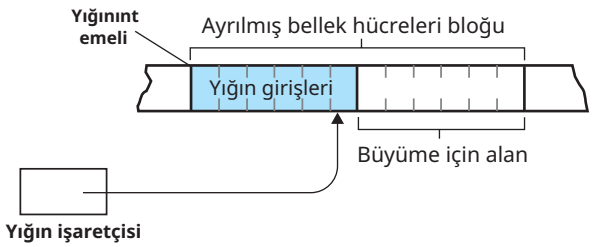
Yığınlar ve Kuyruklar Saklama

Yığınlar ve kuyruklar için, genellikle kesintisiz bir listeye benzer bir organizas yon kullanılır. Bir yığın durumunda, yığının maksimum boyutunu karşılayaca k kadar büyük bir bellek bloğu ayrılır. (Bu bloğun boyutunu belirlemek genel likle kritik bir tasarım kararı olabilir. Eğer çok az alan ayrılırsa, yığın nihayetin de ayrılan depolama alanını aşacaktır; ancak, eğer çok fazla alan ayrılırsa, bel lek alanı israf edilecektir.) Bu bloğun bir ucu yığının tabanı olarak belirlenir. İl k olarak yığına itilecek giriş burada saklanır. Daha sonra her ek giriş, yığının ayrılan bloğun diğer ucuna doğru büyüdükçe, bir öncekine bitişik olarak yerl eştirilir.

Girişlerin itildiği ve çekildiği gözlemlendiğinde, yığının üst kısmının konumu ayrılmış bellek hücreleri bloğu içinde ileri geri hareket edecektir. Bu konumu taki p etmek için, adresi ek bir bellek hücresinde saklanır ve bu hücre yığın işaretçisi olarak bilinir. Yani, yığın işaretçisi yığının üstüne işaret eden bir işaretçidir.

Tam sistem, Şekil 8.12'de gösterildiği gibi, şu şekilde çalışır: Yığına yeni bir gi riş eklemek için, önce yığın işaretçisini boşluğa işaret edecek şekilde ayarlıyoruz.

Şekil **8.12**Bellekte bir yığın

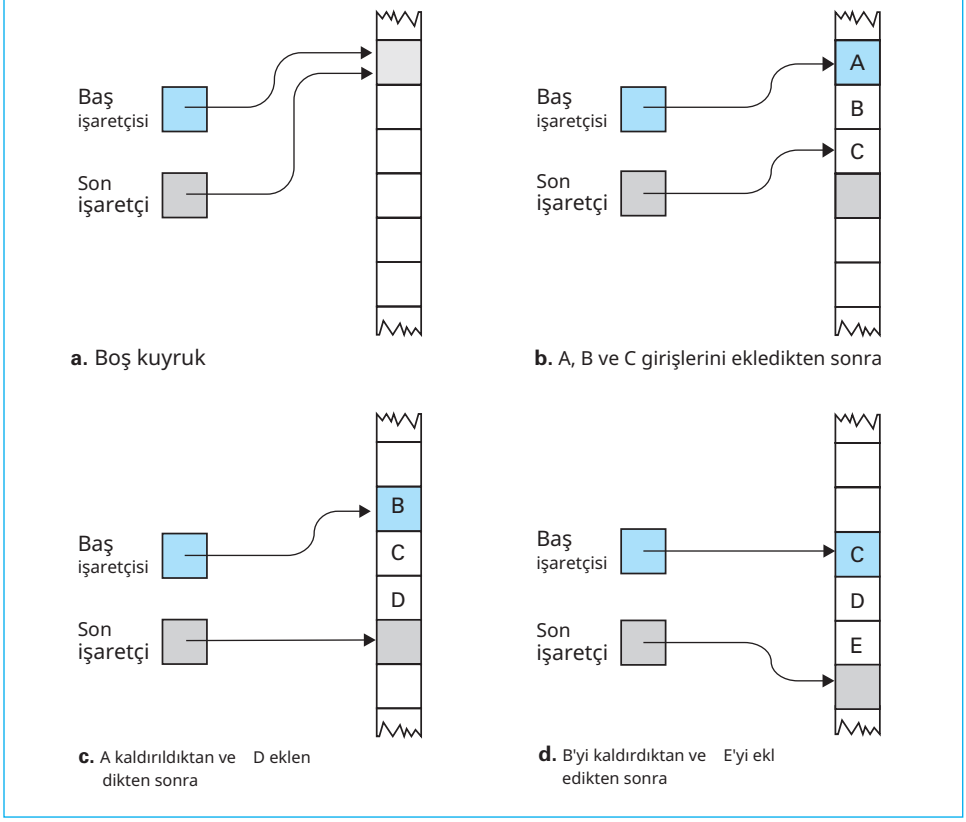


Doclingo https://www.doclingo.ai

yığın üstünün hemen ötesine yerleştirip yeni girişi bu konuma koyun. Bir girişi yı ğından çıkarmak için, yığın işaretçisinin işaret ettiği veriyi okuruz ve ardından yığ ın işaretçisini yığındaki bir sonraki girişe işaret edecek şekilde ayarlarız.

Bir kuyruğun geleneksel uygulanışı, bir yığınınkine benzer. Yine, kuyruğun öngörülen maksimum boyutunu tutacak kadar büyük bir ana bellek bloğu ayırı rız. Ancak, bir kuyruk durumunda, yapının her iki ucunda da işlemler gerçekleşt irmemiz gerektiğinden, bir yığın için yaptığımız gibi yalnızca bir tane yerine iki bellek hücresi ayırırız. Bu işaretçilerden biri, baş işaretçisi olarak adlandırılır ve kuyruğun başını takip eder; diğeri ise kuyruk işaretçisi olarak adlandırılır ve ku yruğun sonunu takip eder. Kuyruk boş olduğunda, bu işaretçilerin her ikisi de a ynı konuma işaret eder (Şekil 8.13). Her seferinde bir giriş kuyruğa eklendiğind e, kuyruk işaretçisinin işaret ettiği konuma yerleştirilir ve ardından kuyruk işare tçisi bir sonraki kullanılmayan konuma işaret edecek şekilde ayarlanır.
Bu şekilde, kuyrukta kuyruk işaretçisi her zaman kuyruğun sonundaki ilk boşluğ a işaret eder.Kuyruktan bir girişi kaldırmak, baş işaretçisinin işaret ettiği girişi çık armayı ve ardından baş işaretçisini kuyruğun bir sonraki girişine işaret edecek şe kilde ayarlamayı içerir.

Şekil **8.13** Baş ve kuyruk işaretçileri ile bir kuyruk uygulaması. Girişler eklendikçe ve kaldı rıldıkça kuyruğun bellekte nasıl ilerlediğine dikkat edin.



a. Boş kuyruk

b. A, B ve C girişlerini ekledikten sonra

c. A kaldırıldıktan ve D eklendikten sonra

d. B'yi kaldırdıktan ve E'yi ekledikten sonra

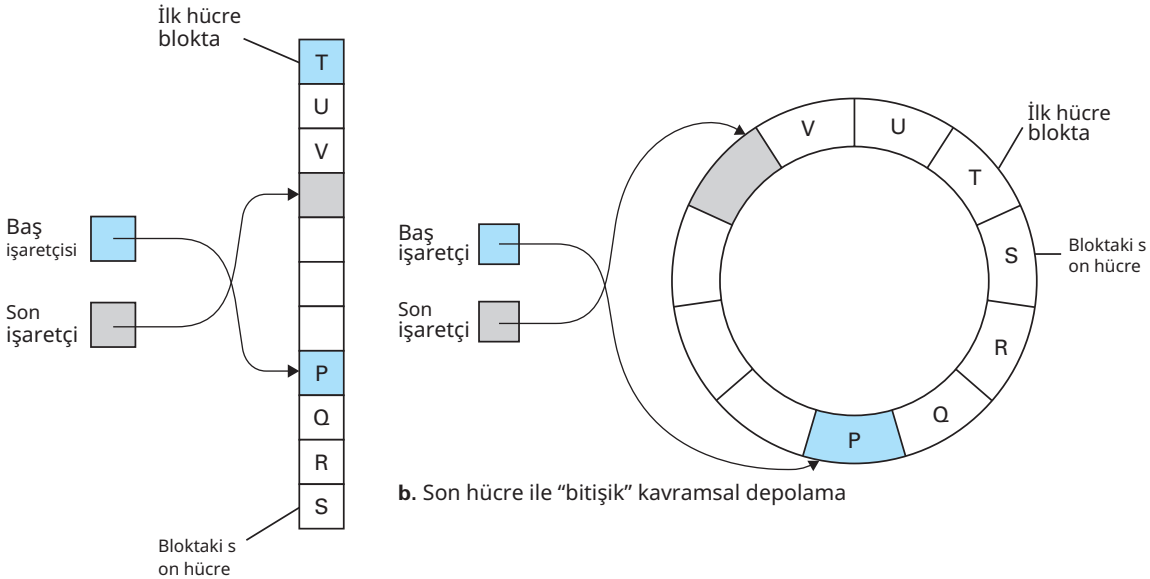
Doclingo https://www.doclingo.ai

Şu ana kadar tanımlanan depolama sistemiyle ilgili bir sorun, girişler ekl endikçe ve kaldırıldıkça, kuyruğun bellekte bir buzul gibi sürünmesidir (bkz. t ekrar Şekil 8.13). Bu nedenle, kuyruğu ayrılmış bellek bloğuna hapsetmek içi n bir mekanizmaya ihtiyacımız var. Çözüm basittir. Kuyruğun bloğun içinde g öç etmesine izin veriyoruz. Ardından, kuyruğun ucu bloğun sonuna ulaştığın da, eklemeye başladığımız ek girişleri bloğun orijinal ucuna geri ekliyoruz; b u noktada boş durumdadır. Benzer şekilde, bloktaki son giriş nihayet kuyruğ un başı haline geldiğinde ve bu giriş kaldırıldığında, baş işaretçisi diğer girişl erin beklediği bloğun başına geri ayarlanır. Bu şekilde, kuyruk, sanki bloğun uçları bir döngü oluşturacak şekilde bağlıymış gibi, bloğun içinde kendisini t akip eder (Şekil 8.14). Sonuç, dairesel kuyruk olarak adlandırılan bir uygulam adır.

İkili Ağaçların Saklanması

Ağaç saklama tekniklerini tartışmak amacıyla, dikkatimizizi ikili ağaçlara kısıtl ıyoruz; hatırlatmak gerekirse, her düğümün en fazla iki çocuğu olan ağaçlar dır. Bu tür ağaçlar genellikle bağlı listelere benzer bir bağlı yapı kullanılarak bellekte saklanır. Ancak, her girişin iki bileşenden (veri ve bir sonraki giriş işa retçisi) oluşması yerine, ikili ağacın her girişi (veya düğümü) üç bileşen içerir: (1) veri, (2) düğümün ilk çocuğuna işaretçi ve (3) düğümün ikinci çocuğuna i şaretçi. Her ne kadar

Şekil **8.14** P'den V'ye kadar harfleri içeren dairesel kuyruk



a. Kuyruk aslında saklandığı gibi

b. Son hücre ile "bitişik" kavramsal depolama

Doclingo https://www.doclingo.ai

Bir makinede sağ veya sol yoktur, ilk işaretçiyi
sol çocuk işaretçisi ve diğer işaretçiyi sağ çocuk işaretçisi olarak adlandırmak f
aydalıdır.Kağıt üzerinde ağacı çizeceğimiz şekilde referans alarak. Böylece ağac
ın her düğümü,Şekil 8.15'te gösterilen formatta kısa, bitişik bir bellek hücreleri
bloğu ile temsil edilir.

Ağacı bellekte depolamak, düğümleri tutacak mevcut bellek hücreleri bloklarını bulmayı ve bu düğümleri istenen ağaç yapısına göre bağlamayı içerir.
Her işaretçi, ilgili düğümün sol veya sağ çocuğuna işaret edecek şekilde ayarlanmalı veya ağaçta o yönde daha fazla düğüm yoksa NIL değerine atanmalıdır.
(Bu, bir terminal düğümünün her iki işaretçisinin de
NIL'e atanmasıyla karakterize edildiği anlamına gelir.) Son olarak, kök
işaretçisi, kök düğümün adresini depoladığımız özel bir bellek konumu ayırıyoruz. Bu kök işaretçisi, ağaca ilk erişimi sağlar.

Bu bağlı depolama sisteminin bir örneği Şekil 8.16'da sunulmuştur; burada kavramsal bir ikili ağaç yapısı ile birlikte, o ağacın bir bilgisayarın belleğinde nasıl görünebileceğine dair bir temsil sergilenmektedir. Dikkat edin ki, düğümlerin ana bellek içindeki gerçek düzeni, kavramsal düzenle oldukça farklı olabilir. Ancak, kök işaretçisini takip ederek, kök düğümü bulabilir ve ardından herhangi bir yolu ağaç boyunca takip ederek, düğümden düğüme uygun işaretçileri izleyerek izleyebilirsiniz.

Bir ikili ağacı bağlı bir yapı olarak depolamanın bir alternatifi,tüm ağaç için te
k bir, bitişik bellek hücreleri bloğu kullanmaktır. Bu yaklaşımı kullanarak,ağacın kö
k düğümünü bloğun ilk hücresinde depoluyoruz. (Basitlik açısından,ağacın her dü
ğümünün yalnızca bir bellek hücresine ihtiyaç duyduğunu varsayıyoruz.)Sonra kö
kün sol çocuğunu ikinci hücrede, kökün sağ çocuğunu üçüncü hücrede depoluyor
uzve genel olarak, hücrede bulunan düğümün sol ve sağ çocuklarınısırasıyla hücr
elerde $2n$ ve $2n+1$ olarak depolamaya devam ediyoruz. Ağaç tarafından kullanılm
ayankonumları temsil eden blok içindeki hücreler, veri yokluğunu gösterenbenzer
siz bir bit deseni ile işaretlenir. Bu teknikle, Şekil 8.16'da gösterilen aynı ağaç,Şekil
8.17'de gösterildiği gibi depolanır. Dikkat edin ki, sistem esasen,düğümleri ağaç
boyunca ardışık olarak daha düşük seviyelerdesektörler olarak depolamaktır; biri
diğerinin ardından gelir. Yani, bloktaki ilk giriş kökdüğüm, ardından kökün çocukl
arı, ardından kökün torunları vebunun gibi devam eder.

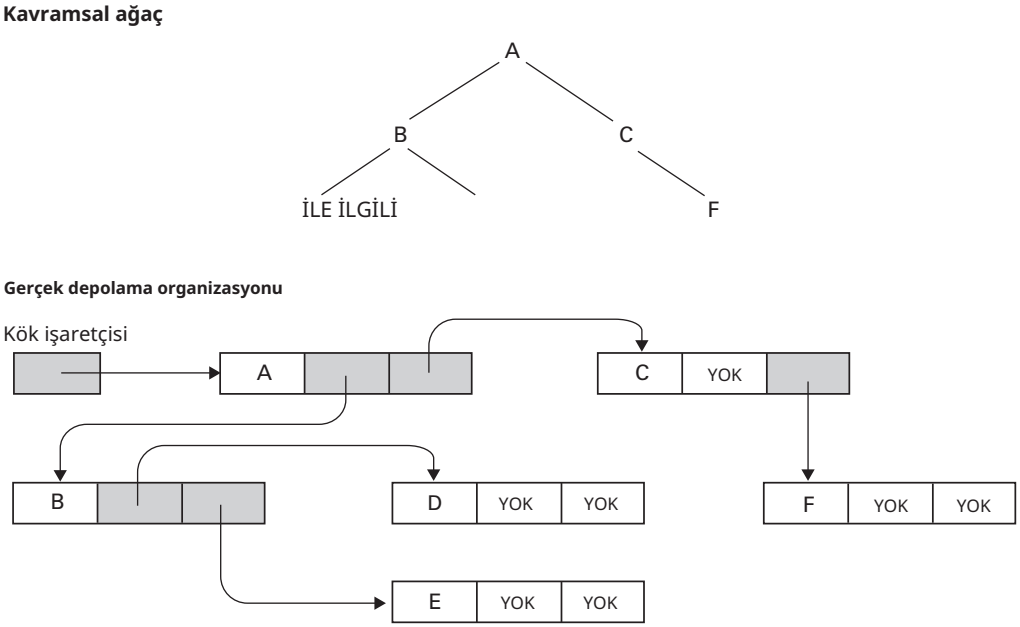
Önceden tanımlanan bağlı yapıya kıyasla, bu alternatif depolamasistemi,
herhangi bir düğümün ebeveynini veya kardeşini bulmak için verimli bir yön
tem sunar. Bir düğümün ebeveyninin yeri, düğümün bloktaki konumunu 2'y
e bölerek ve kalan kısmı atarak bulunabilir (7. konumdaki düğümün ebeveyn
i 3. konumdaki düğüm olacaktır). Bir düğümün kardeşinin yeri, çift sayılı bir k
onumdaki düğümün yerinin üzerine 1 ekleyerek veya tek sayılı bir konumda
ki düğümün yerinden 1 çıkararak bulunabilir. Örneğin, 4. konumdaki düğüm
ün kardeşi 5. konumdaki düğümdür, oysa

Şekil **8.15** İkili ağaçtaki bir düğümün yapısı



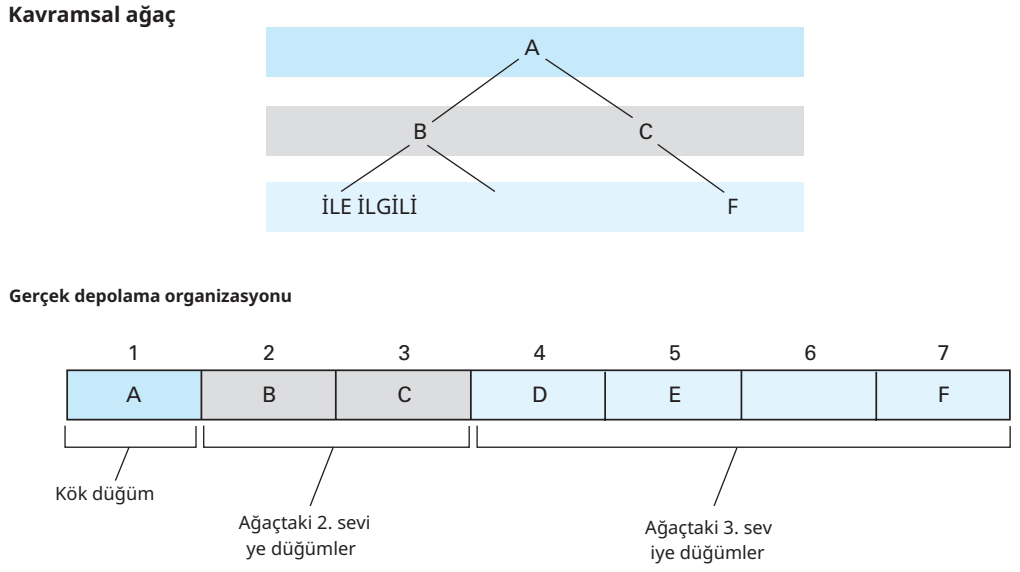
Doclingo https://www.doclingo.ai

Şekil **8.16** Bağlı depolama sistemi kullanarak bir ikili ağacın kavramsal ve gerçek organizasyonu



3. pozisyondaki düğümün kardeşi 2. pozisyondaki düğümdür. Dahası, bu depolamasist emi, ikili ağaç yaklaşık dengeli olduğunda (kök düğümün altındaki her iki alt ağacın aynı derinliğe sahip olduğu anlamında) ve dolu olduğunda (uzun, ince dallara sahip olmadığ ı anlamında) alanı verimli bir şekilde kullanır. Dalların uzun ve ince olmadığı ağaçlar için

Şekil **8.17**İşaretçiler olmadan saklanan bir ağaç



Doclingo https://www.doclingo.ai

Bu özellikler nedeniyle, sistem oldukça verimsiz hale gelebilir, bu da Şekil 8.18'de gösterilmiştir.

Veri Yapılarını Manipüle Etme

Veri yapıların bir bilgisayarın belleğinde nasıl saklandığını, kullanıcının hayal ettiği kavramsal yapı ile aynı olmadığını gördük.
İki boyutlu homojen bir dizi aslında iki boyutlu dikdörtgen bir blok olarak sak lanmaz ve bir liste veya ağaç, aslında büyük bir bellek alanına dağılmış küçük parçalardan oluşabilir.

Bu nedenle, kullanıcının yapıyı soyut bir araç olarak erişebilmesi için, kullanıcıyı gerçek depolama sist eminin karmaşıklıklarından korumalıyız. Bu, kullanıcının verdiği talimatların (ve soyut araç terimleriyle ifad e edilen) gerçek depolama sistemi için uygun adımlara dönüştürülmesi gerektiği anlamına gelir. Homojen diziler durumunda, bunun nasıl yapılabileceğini, satır ve sütun indekslerini bellek hücresi adreslerine dönü ştürmek için bir adres polinomunun kullanılmasıyla gördük. Özellikle, bir programcı tarafından soyut hom ojen bir dizi açısından düşünüldüğünde yazılan ifadenin, ana bellekte doğru değişiklikleri gerçekleştiren ad ımlara nasıl dönüştürülebileceğini gördük. Benzer şekilde,

```
Sales[3, 4] ← 5;
```

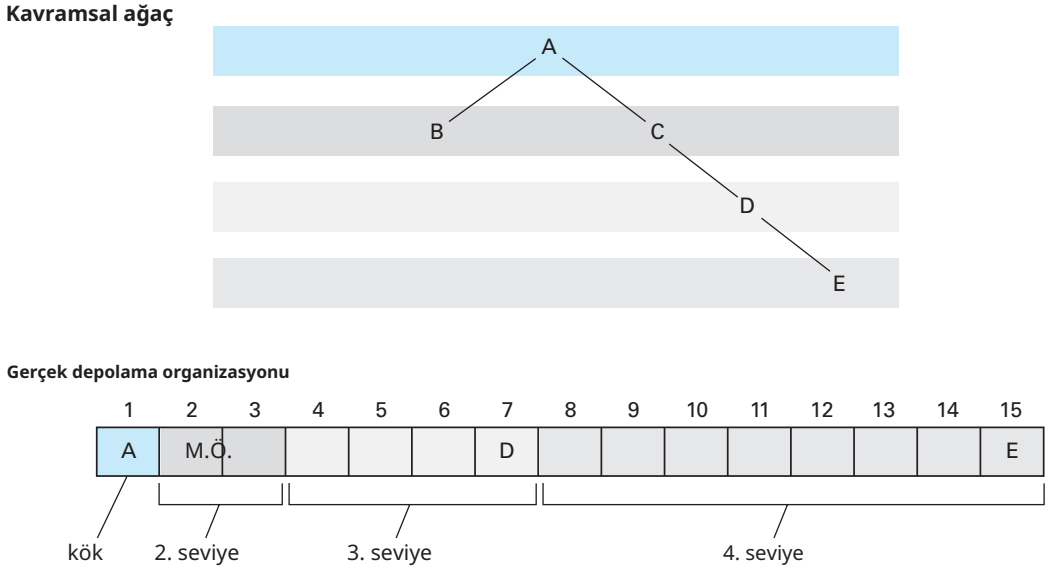
şu tür ifadelerin nasıl işlendiğini de gördük.

```
Employee.Age ← 22;
```

soyut heterojen bir diziye atıfta bulunmak, dizinin gerçekten nasıl saklandığına bağlı olarak uygun hareketlere dönüştürülebilir.

Listeler, yığınlar, kuyruklar ve ağaçlar durumunda, soyut yapı açısından belirtilen talimatlar genelliklekullanıcıyı korurken gerekli görevi yerine getire n prosedürler aracılığıyla uygun hareketlere dönüştürülür.

Şekil **8.18** Bir kavramsal formda ve işaretçiler olmadan saklanacağı şekilde gösterilen seyrek, dengesiz bir ağaç



Doclingo https://www.doclingo.ai

Şekil **8.19**Bağlı bir listeyi yazdırma prosedürü

```
prosedür  PrintList (Liste)
CurrentPointer ← head pointer of List.
iken  (CurrentPointer NIL değil)  yap   (Curr
   entPointer tarafından işaret edilen girişteki ismi yazdır;    CurrentPoin
    ter tarafından işaret edilen Liste girişinin işaretçi hücresindeki d
    eğeri gözlemle     ve CurrentPointer'ı o değer olacak şekilde yeniden
    atayın.)
```

temel depolama sisteminin ayrıntılarından. Örneğin, yeni girişleri bir bağlı lis teye eklemek için `insert` prosedürü sağlanmışsa,J. W. Brown, Fizik 208 ders ine kayıtlı öğrenciler listesine yalnızca aşağıdaki gibi bir prosedür çağrısı ger çekleştirerek eklenebilir:

```
insert("Brown, J.W.", Physics208)
```

Prosedür çağrısının tamamen soyut yapı açısından belirtildiğine dikkat edin —listenin gerçekten nasıl uygulandığı gizlidir.

Detaylı bir örnek olarak, Şekil 8.19, bir bağlı listeyi yazdırmak için printList adı nda bir prosedür sunmaktadır. Bu prosedür, listenin ilk girişinin baş işaretçisi adı verilen bir işaretçi tarafından gösterildiğini ve listedeki her girişin iki parçadan olu ştuğunu varsayar: bir isim ve bir sonraki girişe işaret eden bir işaretçi. Bu prosedü r geliştirildikten sonra, listeyi yazdırmak için gereken adımlarla ilgilenmeden soyu t bir araç olarak bağlı listeyi yazdırmak için kullanılabilir. Örneğin, Ekonomi 301 içi n basılı bir sınıf listesi elde etmek için, bir kullanıcının yalnızca prosedür çağrısını g erçekleştirmesi gerekir.

```
printList(Economics301ClassList)
```

istenen sonuçları elde etmek için. Ayrıca, daha sonra listenin gerçekten nasıl saklandığını değiştirmeye karar verirsek,o zaman yalnızca prosedürün içsel e ylemleri `printList` değiştirilmelidir—kullanıcı, listenin yazdırılmasını aynı p rosedür çağrısıyla talep etmeye devam edecektir.

Sorular & Alıştırmalar

1. Aşağıdaki dizinin ana bellekte nasıl düzenleneceğini gösterinsatır ana sıra sına göre saklandığında.

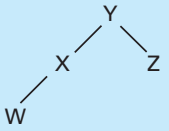
5	3	7
4	2	8
1	9	6

2. İki boyutlu bir dizinin $i$inci satırındaki ve $j$uncu sütunundaki girişi bulmak i çin bir formül verinsütun ana sırasına göre saklandığında, satır ana sırası na göre değil.
3. C, C++, Java ve C# programlama dillerinde, dizilerin indeksleri0'dan başlar, 1'den değil. Böylece, Array adındaki bir dizinin birinci satır, dördüncü sütundaki girişi

Doclingo https://www.doclingo.ai

Array[0][3] ile referans alınır. Bu durumda,çevirici, Array[i][j] bi çimindeki referansları bellek adreslerine dönüştürmek için hangi adre s polinomunu kullanır?

4. Bağlı bir listenin boş olduğunu gösteren koşul nedir?
5. Şekil 8.19'daki prosedürü, belirli bir ad yazdırıldıktan sonra yazdırmayı dur duracak şekilde değiştirin.
6. Bu bölümdeki bir yığın uygulama tekniğine dayanarak, yığının boş old uğunu gösteren koşul nedir?
7. Bir yığının, bir boyutlu bir dizi cinsinden yüksek seviyeli bir dilde nasıl uygulanabileceğini tanımlayın.
8. Bu bölümde açıklandığı gibi, bir kuyruk dairesel bir şekilde uygulandığ ında, kuyruk boşken baş ve kuyruk işaretçileri arasındaki ilişki nedir? K uyruk doluyken ne olur? Bir kuyruğun dolu veya boş olduğunu nasıl te spit edebilirsiniz?
9. Aşağıdaki ağacın, bu bölümde açıklandığı gibi sol ve sağ çocuk işaretçileri k ullanılarak bellekte nasıl göründüğünü temsil eden bir diyagram çizin. Ardı ndan, ağacın bu bölümde açıklanan alternatif depolama sistemi kullanılara k kesintisiz depolamada nasıl görüneceğini gösteren başka bir diyagram çi zin.



8.4 Kısa Bir Vaka Çalışması

Alfabetik sırayla bir isim listesi depolama görevini ele alalım. Bu liste üzerind e gerçekleştirilecek işlemlerin aşağıdakiler olduğunu varsayıyoruz:

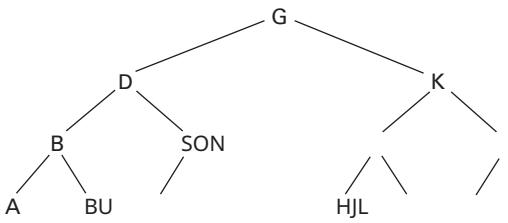
arama bir kaydın varlığını kontrol et,
yazdır listeyi alfabetik sırayla, ve
ekle yeni bir kayıt

Amacımız, bu işlemleri gerçekleştirmek için bir dizi prosedürle birlikte bir depolama sist emi geliştirmektir—böylece tamamlayıcı bir soyut araç üretmiş olacağız.

Listeyi depolamak için seçenekleri değerlendirerek başlıyoruz. Eğer liste bağlı liste modeliyle depolanırsa, listeyi sıralı bir şekilde aramamız gerekecek ; bu süreç, Bölüm 5'te tartıştığımız gibi, liste uzun olduğunda çok verimsiz ol abilir. Bu nedenle, arama prosedürümüz için ikili arama algoritmasını (Bölü m 5.5) kullanmamıza olanak tanıyan bir uygulama arayacağız. Bu algoritmay ı uygulamak için, depolama sistemimizin, listenin giderek daha küçük parçal arının ortadaki girişini bulmamıza izin vermesi gerekir. Çözümümüz, listeyi b ir ikili ağaç olarak depolamaktır. Ortadaki liste girişi kök düğüm olur. Ardınd an, listenin kalan ilk yarısının ortasını kökün sol çocuğu yaparız ve kalan ikinc i yarının ortasını kökün sağ çocuğu yaparız. Listenin her kalan dörtte birinin ortadaki girişleri, kökün çocuklarının çocukları olur ve bu şekilde devam eder . Örneğin, Şekil 8.20'deki ağaç, A, B, C, D, E, F, G, H,

Doclingo https://www.doclingo.ai

Şekil **8.20** A'dan M'ye kadar olan harflerin sıralı bir ağaçta düzenlenmiş hali



I, J, K, L ve M harflerinin listesini temsil etmektedir. (Söz konusu listenin parçası çift sayıda giriş içerdiğinde, ortadaki iki girişten daha büyüğünü ortada olarak kabul ediyoruz.)

Bu şekilde saklanan listeyi aramak için, hedef değeri kök düğümü ile karşıla ştırırız. Eğer ikisi eşitse, aramamız başarılı olmuştur. Eğer eşit değillerse, hedefi n kökten daha küçük veya daha büyük olup olmadığına bağlı olarak kökün sol v eya sağ çocuğuna geçeriz. Orada, aramayı devam ettirmek için gerekli olan list enin kısmının ortasını buluruz. Karşılaştırma ve bir çocuğa geçme işlemi, hedef değeri bulana kadar (yani aramamızın başarılı olduğu anlamına gelir) veya hed ef değeri bulamadan bir NIL işaretçisine ulaşana kadar (yani aramamızın başarı sız olduğu anlamına gelir) devam eder.

Şekil 8.21, bu arama sürecinin bağlı ağaç yapısı durumunda nasıl ifade e dilebileceğini göstermektedir. Bu prosedürün, ikili aramanın orijinal ifadesi o lan Şekil 5.14'teki prosedürün yalnızca bir iyileştirmesi olduğunu unutmayın. Ayrım büyük ölçüde kozmetiktir. Algoritmayı

Şekil **8.21** Liste bağlı bir ikili ağaç olarak uygulanmış olsaydı ikili arama

```
prosedür  Arama(Ağaç, HedefDeğeri)

eğer  (Ağaç'ın kök işaretçisi = NIL)
 o zaman
    (arama işlemini başarısız olarak ilan et)
 aksi takdirde
    (aşağıdaki talimat bloğunu çalıştırın
         uygun durumla ilişkilidir)
     durum 1: HedefDeğer = kök düğümün değeri          (a
           rama işleminin başarılı olduğunu bildir)    dur
     um 2: HedefDeğer < kök düğümün değeri          (Hed
           efDeğer'in kökün sol çocuk işaretçisi tarafından ta
             nımlanan alt ağaçta            olup olmadığını görme
              k için Arama prosedürünü uygula ve
              o aramanın sonucunu bildir)     durum 3:
     HedefDeğer > kök düğümün değeri          (HedefDeğ
           er'in kökün sağ çocuk işaretçisi tarafından tanıml
              anan alt ağaçta             olup olmadığını görmek içi
              n Arama prosedürünü uygula ve              o ara
              manın sonucunu bildir)    ) eğer sonu
```

Doclingo https://www.doclingo.ai

Çöp Toplama

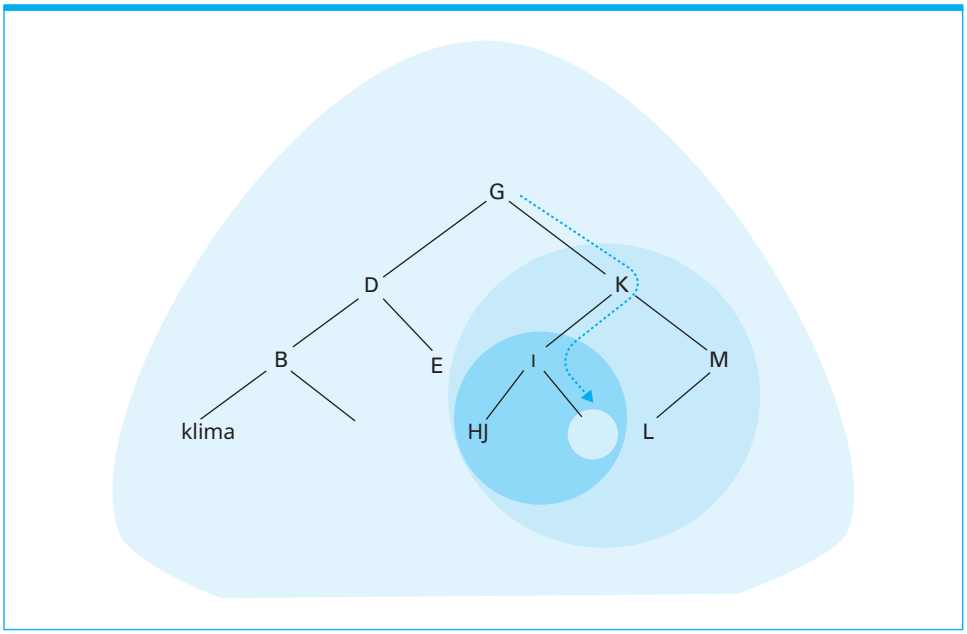
Dinamik veri yapıları büyüdükçe ve küçüldükçe, depolama alanı kullanılır ve serbest bıra kılır. Kullanılmayan depolama alanını gelecekteki kullanım için geri kazanma sürecine ç öp topla-ma denir. Çöp toplama birçok ortamda gereklidir. Bir işletim sistemindeki belle k yöneticisi, bellek alanını tahsis ederken ve geri alırken çöp toplama işlemini gerçekleşti rmelidir. Dosya yöneticisi, dosyalar makinenin kütle depolamasına kaydedilirken ve silini rken çöp toplama işlemini gerçekleştirir. Ayrıca, dispatcher kontrolü altında çalışan herh angi bir süreç, kendi tahsis edilmiş bellek alanında çöp toplama işlemini gerçekleştirmek zorunda kalabilir.

Çöp toplama bazı ince sorunlar içerir. Bağlantılı yapılar durumunda, bir veri öğesine işaretçi her değiştiğinde, çöp toplayıcının işaretçinin başlangıçta işaret ettiği depolama alanını geri alıp almayacağına karar vermesi gerekir. Sorun, bir den fazla işaretçi yolunu içeren iç içe geçmiş veri yapılarında özellikle karmaşık hale gelir. Yanlış çöp toplama rutinleri veri kaybına veya depolama alanının veri msiz kullanımına yol açabilir. Örneğin, çöp toplama depolama alanını geri almay ı başaramazsa, mevcut alan yavaş yavaş azalır; bu olaya bellek sızıntısı denir.

listenin giderek daha küçük segmentlerini arayarak, algoritmayı giderek daha küçük alt ağaçları arama terimleriyle ifade ediyoruz (Şekil 8.22).

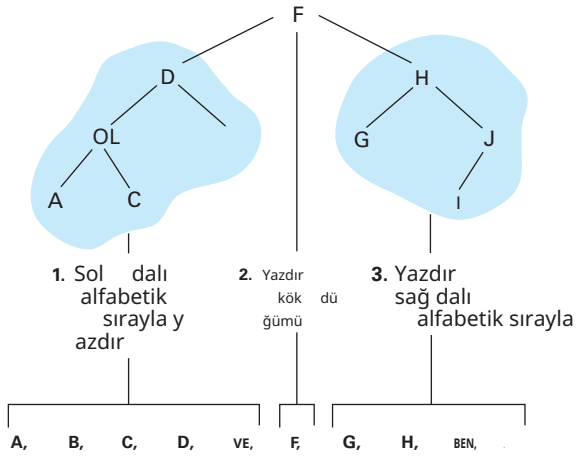
"liste"mizi bir ikili ağaç olarak depoladıktan sonra, listenin alfabetik sırayla yazdırılmasının şimdi zor olacağını düşünebilirsiniz. Ancak, listenin alfabetik sırayla yazdırılması için, yalnızca sol alt ağacı alfabetik sırayla yazdırmamız, kök düğümü yazdırmamız ve ardından sağ alt ağacı alfabetik sırayla yazdırmamız gerekir (Şekil 8.23). Sonuçta, sol alt ağaç tüm

Şekil **8.22** Prosedür tarafından Şekil 8.21'de dikkate alınan giderek daha küçük ağaçlar J harfini ararken



Doclingo https://www.doclingo.ai

Şekil **8.23**Alfabetik sırayla bir arama ağacını yazdırma



kök düğümden daha az olan elemanlar, sağ alt ağaç ise kök düğümden daha büy ük olan tüm elemanları içerir. Şu ana kadar mantığımızın bir taslağı şöyle görünm ektedir:

```
eğer(ağaç boş değilse)
o zaman(sol alt ağacı alfabetik sırayla yazdır;
        kök düğümü yazdır;
        sağ alt ağacı alfabetik sırayla yazdır)
```

Bu taslak, sol alt ağacı ve sağ alt ağacı alfabetik sırayla yazdırma görevlerini içermektedir; her ikisi de esasen orijinal görevimizin daha küçük versiyonlarıdı r. Yani, bir ağacı yazdırma problemini çözmek, alt ağaçları yazdırma daha küçü k görevini içerir; bu da ağaç yazdırma problemimize özyinelemeli bir yaklaşım önerir.

Bu izden yola çıkarak, başlangıç fikrimizi, ağaçlarımızı yazdırmak için tam bir pseudocode prosedürü haline genişletebiliriz; bu, Şekil 8.24'te gösterilmi ştir. Rutine `PrintTree` adını verdik ve ardından sol ve sağ alt ağaçları yazdırm ak için `PrintTree` hizmetlerini talep ettik. Özyinelemeli sürecin sonlanma koş ulunun (boş bir alt ağaca ulaşmak) ulaşılacağından emin olun, çünkü her

Şekil **8.24**İkili ağaçtaki verileri yazdırma prosedürü

```
prosedür  PrintTree (Ağaç)

eğer  (Ağaç boş değilse)
   o zaman  (PrintTree prosedürünü Ağaç'taki          sol
              dal olarak görünen ağaçta uygula;
            Ağaç'ın kök düğümünü yazdır;
                 PrintTree prosedürünü Ağaç'taki       s
            ağ dal olarak görünen ağaçta uygula)
```

Doclingo https://www.doclingo.ai

rutinlerin ardışık aktivasyonu, aktivasyona neden olan ağaçtan daha küçük bir ağaç üze rinde çalışır.

Ağaçta yeni bir giriş ekleme görevi, ilk bakışta göründüğünden daha kola ydır. Sezgileriniz, eklemelerin yeni girişler için yer açmak amacıyla ağacı kes meyi gerektirebileceğini düşündürebilir, ancak aslında eklenen düğüm her z aman yeni bir yaprak olarak eklenebilir, ilgili değere bakılmaksızın. Yeni bir g iriş için uygun yeri bulmak için, o girişi arıyormuş gibi ağaç boyunca aşağıya doğru hareket ederiz. Giriş ağaçta olmadığından, aramamız bir NIL işaretçisi ne yol açacaktır.

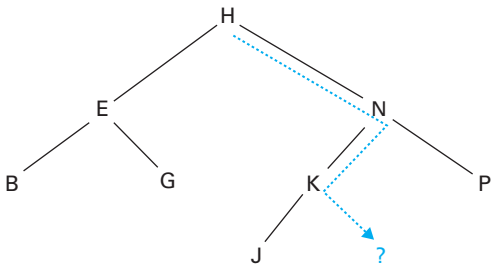
Bu noktada yeni düğüm için uygun konumu bulmuş olacağız (Şekil 8.25). Ger çekten de, bu yeni giriş için bir arama yapılırsa ulaşılacak konum budur.

Bağlantılı ağaç yapısı durumunda bu süreci ifade eden bir prosedür Şekil 8.26'da gösterilmiştir. Ağaçta eklenmekte olan değeri (Yeni Değer olarak adlandırılır) arar ve ardından Yeni Değer içeren yeni bir yaprak düğümünü uygun konuma yerleştirir. Arama sırasında eklenmekte olan girişin ağaçta gerçekten bulunduğu not edilirse, hiçbir ekleme yapılmaz.

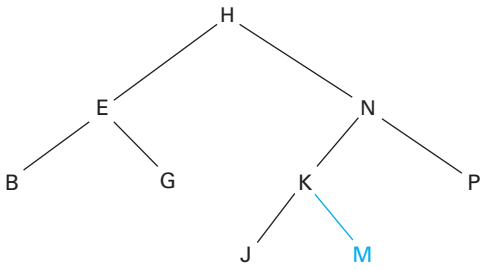
Bir bağlı ikili ağaç yapısından oluşan bir yazılım paketinin,aramak, yazdır mak ve eklemek için prosedürlerimizle birlikte,varsayımsal uygulamamız tar afından soyut bir araç olarak kullanılabilecektam bir paket sağladığını sonuc una varıyoruz. Gerçekten de, doğru bir şekildeuygulandığında, bu paket gerç ek altta yatan depolama yapısı için endişe duymadankullanılabilir. Paketteki prosedürleri kullanarak, kullanıcı bir isimler listesihayal edebilir

Şekil **8.25** M girişinin B, E, G, H, J, K, N, P listesine ağaç olarak eklenmesi

a. Yeni girişin yokluğu tespit edilene kadar arama yapın



b. Bu, yeni girişin eklenmesi gereken konumdur



Doclingo https://www.doclingo.ai

Şekil **8.26**İkili ağaç olarak saklanan bir listede yeni bir giriş ekleme prosedürü

```
prosedür  Ekle(Ağaç, Yeni Değer) eğer  (Ağa

ç'ın kök işaretçisi = NIL) (kök işaretçis
  ini Yeni Değer içeren yeni bir yaprağa          işaret ede
        cek şekilde ayarla)
  aksi takdirde  (aşağıdaki talimat bloğunu çalıştırın
                uygun durumla ilişkilidir)
        durum 1: YeniDeğer = kök düğümün değeri              (Hiç
              bir şey yapma)
        durum 2: YeniDeğer < kök düğümün değeri
              (eğer (kök düğümün sol çocuk işaretçisi = NIL)
                    o zaman  (o işaretçiyi YeniDeğer içeren yeni
                                bir yaprak düğüme işaret edecek şekilde ayarla)
                    aksi takdirde  (YeniDeğer'i sol çocuk işaretçisiyle
                                tanımlanan alt ağa eklemek için
                          Insert prosedürünü uygula)
        durum 3: YeniDeğer > kök düğümün değeri
                    (eğer  (kök düğümün sağ çocuk işaretçisi = NIL)
                  o zaman  (o işaretçiyi YeniDeğer içeren yeni
                              bir yaprak düğüme işaret edecek şekilde ayarla)
                    aksi takdirde  (YeniDeğer'i sağ çocuk işaretçisiyle
                                tanımlanan alt ağa eklemek için
                          Insert prosedürünü uygula)
  ) eğer sonu
```

alfabetik sırada saklanırken, gerçekte "liste" girişlerinin, ikili ağaç olarak bağ lı bellek hücreleri blokları arasında dağılmış olduğunu belirtin.

Sorular & Alıştırmalar

1. Gelecekte arama yapmak için R, S, T, U, V, W, X,Y ve Z listesini depolamak için kull anabileceğiniz bir ikili ağaç çizin.
2. İkili arama algoritmasının, Şekil 8.20'deki ağaçta J girişini ararken izl ediği yolu Şekil 8.21'de belirtin. P girişi için ne olacak?
3. Şekil 8.24'teki özyinelemeli ağaç-yazdırma algoritmasının etkinlik du rumunu temsil eden bir diyagram çizin; K düğümü, Şekil 8.20'deki sır alı ağaç içinde yazdırıldığında.
4. Her düğümün en fazla 26 çocuğa sahip olduğu bir ağaç yapısının, İngilizce dilinde ki kelimelerin doğru yazımını kodlamak için nasıl kullanılabileceğini açıklayın.

8.5 Özelleştirilmiş Veri Türleri

Bölüm 6'da bir veri türü kavramını tanıttık ve tamsayı, reel, karakter ve Bool ean gibi temel türleri tartıştık. Bu veri türleri, çoğu programlama dilinde ilke l veri türleri olarak sağlanmaktadır. Bu bölümde, bir programcının belirli bir uygulamanın ihtiyaçlarına daha yakın bir şekilde kendi veri türlerini tanımla ma yollarını ele alacağız.

Doclingo https://www.doclingo.ai

Kullanıcı Tanımlı Veri Türleri

Bir algoritmayı ifade etmek, programlama dilinde sağlanan ilkel türler dışın da veri türleri mevcut olduğunda genellikle daha kolaydır. Bu nedenle, birço k modern programlama dili, programcıların ilkel türleri yapı taşları olarak k ullanarak ek veri türleri tanımlamasına izin verir. Bu tür "ev yapımı" veri türl erinin en temel örnekleri, esasen tek bir ad altında toplanmış ilkel türlerin bi rleşimleri olan kullanıcı tanımlı veri türleri olarak bilinir.

Açıklamak gerekirse, birçok değişken içeren bir program geliştirmek istediği mizi varsayalım; her biri bir isim, yaş ve beceri derecesinden oluşan aynı heteroje n dizi yapısına sahip. Bir yaklaşım, her bir değişkeni ayrı ayrı heterojen bir dizi ol arak tanımlamak olacaktır (Bölüm 6.2). Ancak, daha iyi bir yaklaşım, heterojen ya pıyı yeni (kullanıcı tanımlı) bir veri türü olarak tanımlamak ve ardından bu yeni tü rü sanki bir ilkel veri türüymüş gibi kullanmaktır.

Bu fikri uygulamak için, aşağıdaki biçimde bir psödo kod ifadesi benimseyebiliriz

```
tip tanımlaEmployeeTypeşu şekilde
{char Name[25];
 int Yaş;
 gerçek Beceri Derecelendirmesi;
}
```

heterojen yapıdan oluşan yeni bir tür tanımlamak için, `EmployeeType` olarak adl andırılan, `Name` (karakter türünde), `Age` (tam sayı türünde) ve `SkillRating` (gerç ek türünde) olarak adlandırılan bileşenleri içeren. Bu yeni veri türü, ilkel bir veri türü gibi değişkenleri tanımlamak için kullanılabilir. Yani, çoğu programlama d ilinin `x` değişkeninin aşağıdaki ifade ile tam sayı olarak tanımlanmasına izin ver diği şekilde

```
int x;
```

Employee1 değişkeni, aşağıdaki ifade ile EmployeeType türünde tanımlanabili r

```
Çalışan Türü Çalışan1;
```

Daha sonra programda, `Employee1` değişkeni, bir çalışanın ismini, yaşını ve b eceri derecesini içeren bir bellek hücreleri bloğuna atıfta bulunacaktır. Blok içindeki bireysel öğeler, gibi ifadelerle referans alınabilir Çalışan1.Adı ve Çalışan1.Yaşı. Böylece, aşağıdaki gibi bir ifade

```
Çalışan1.Yaşı←26;
```

değer 26'yı `Yaş` bileşenine atamak için kullanılabilir Çalışan1 olarak bilinen blok içinde. Ayrıca, ifade

```
Çalışan Türü DistManager, Satış Temsilcisi1, Satış Temsilcisi2;
```

üç değişkeni `DistManager`,`SalesRep1` veSalesRep2'nin `EmployeeType` türünde olduğun u beyan etmek için kullanılabilir, tam olarak şu biçimde bir ifade olarak

```
gerçek Kol, Bel, Boyun;
```

genellikle `Sleeve`,`Waist` ve `Neck` değişkenlerini primitive türü `real` olarak tanımla mak için kullanılır.

Kullanıcı tanımlı bir veri türü ile o türün gerçek bir öğesi arasında ayrım y apmak önemlidir. İkincisine türün bir örneği denir. Kullanıcı tanımlı veri türü , türün örneklerini oluşturmak için kullanılan temelde bir şablondur. O türün tüm örneklerinin sahip olduğu özellikleri tanımlar, ancak kendisi değildir.

Doclingo https://www.doclingo.ai

o türden bir olay oluşturur (tıpkı bir kurabiye kalıbının kurabiyelerin yapıldığı bir şablon olması gibi ama kendisi bir kurabiye değildir). Önceki örnekte, kullanıcı tanımlı veri türü `EmployeeType`, `DistManager`,`SalesRep1` ve `SalesRep2` olarak bilinen o türden üç örnek oluşturmak için kullanıldı.

Soyut Veri Türleri

Kullanıcı tanımlı veri türü kavramı avantajlı olsa da, yeni veri türlerinin tam anlamıyla oluşturulmasını sağlamada yetersiz kalmaktadır. Tam bir veri türü iki bölümden oluşur: (1) önceden belirlenmiş bir depolama sistemi (örneğin, tam sayı türü için iki'nin tamamlayanı sistemi ve gerçel türü için kayan nokta sistemi) ve (2) önceden tanımlanmış işlemlerden oluşan bir koleksiyon (örneğin toplama ve çıkarma). Özellikle, bir programlama dilindeki ilkel veri türleri, ilkel işlemlerle ilişkilidir. Bir programcı bir değişkeni ilkel türde olarak tanımlarsa, programcı bu değişkene ilkel işlemleri uygulamaya başlayabilir.

Ancak, geleneksel kullanıcı tanımlı veri türleri, programcıların yalnızca yeni depolama sistemleri tanımlamasına izin verir. Bu yapılar üzerinde veri ile gerçekleştirilecek işlemleri sağlamazlar. Açıklamak gerekirse, bir program içinde birkaç tamsayı değerinin yığınını oluşturmak ve kullanmak isteseydik, yaklaşımımız her yığını 20 tamsayı değerinden oluşan homojen bir dizi olarak uygulamak olabilir. Yığındaki alt giriş, ilk dizi konumuna yerleştirilecek (itilecek) ve ek yığın girişleri dizide artan yükseklikteki girişlere yerleştirilecektir (bkz. Bölüm 8.3'teki Soru/Alıştırma 7). Ek bir tamsayı değişkeni yığın işaretçisi olarak kullanılacaktır. Bu, bir sonraki yığın girişinin itileceği dizi girişinin indeksini tutacaktır. Böylece her yığın, yığını ve yığın işaretçisi rolünü oynayan bir tamsayıyı içeren homojen bir diziden oluşacaktır.

Bu planı uygulamak için, önce bir kullanıcı tanımlı tür olan `StackType` şeklinde bir ifade ile tanımlayabiliriz

```
.define typeStackType olarak
{int StackEntries[20];
  int StackPointer = 0;
}
```

(C, C++, C# ve Java gibi dillerin izinden giderek, dizinin `StackEntries` indekslerinin 0'dan 19'a kadar olduğunu varsayıyoruz, bu nedenle `StackPointer` değerini 0 olarak başlattık.) Bu bildirimi yaptıktan sonra, `StackOne`,`StackTwo` ve `StackThree` adında yığınlar tanımlayabiliriz.

```
StackType StackOne, StackTwo, StackThree;
```

Bu noktada, `StackOne`,`StackTwo` ve `StackThree` değişkenlerinin her biri, bireysel bir yığın uygulamak için kullanılan benzersiz bir bellek hücresi bloğunu referans alır.

Peki, şimdi `StackOne` üzerine 25 değerini eklemek istersek ne olur? Yığın uygulamasının altında yatan dizi yapısının detaylarından kaçınmak veyığını soyut bir araç olarak kullanmak istiyoruz—belki de push(25, StackOne) benzeri bir prosedür çağrısı yaparak.

Ancak böyle bir ifade, `push` adında uygun bir prosedür tanımlamadığımız sürece mevcut olmayacaktır. Ayrıca yapmak istediğimiz diğer işlemler

Doclingo https://www.doclingo.ai

StackType türündeki değişkenler, yığından girişleri çıkarmayı, yığının boş olu p olmadığını kontrol etmeyi ve yığının dolu olup olmadığını kontrol etmeyi iç erecektir; bunların hepsi ek prosedürlerin tanımlarını gerektirecektir. Kısacas ı, StackType veri türünün tanımı, türle ilişkilendirmek istediğimiz tüm özellikl eri içermemektedir.

Bu sorunu, `define type` ifademizi prosedürleri ve veri tanımlarını içerece k şekilde genişleterek çözebiliriz. Örneğin, `StackType` tanımını şu şekilde ya

zabiliriz:

```
{int StackEntries[20];
  int StackPointer = 0;
  prosedürpush(değer)
    {StackEntries[StackPointer]←value;
     StackPointer←StackPointer1;
    }
  prosedürpop . . .
}
```

Bu, `StackType` türünün `StackEntries` ve `StackPointer` adında değişkenlerle ve `push`, `pop` adında prosedürlerle ilişkilendirildiğini ifade etmek için tasarlanmıştır. (Basitlik adına, `push` prosedürünün çok naif bir versiyonunu dahil ettik. Gerçe kte, prosedür, ek bir giriş eklemeden önce yığının dolu olmadığını sağlamalı dır.)

Bu türün genişletilmiş tanımı ile`StackType`, `StackOne`,`StackTwo` ve `StackThree`'ü yığınlar olarak beyan edebiliriz.

```
StackType StackOne, StackTwo, StackThree;
```

Sonra bu yığınlara aşağıdaki gibi ifadelerle girişler ekleyebiliriz:

```
StackOne.push(25);
```

bu, `StackOne` ile ilişkili `push` prosedürünü 25 değerini gerçek parametre olarak kulla narak yürütmek anlamına gelir.

Kullanıcı tanımlı veri türleri, işlemlerin tanımlarını içerenleresoyut veri tü rleri denir. Dolayısıyla, daha temel kullanıcı tanımlı veri türlerinin aksine,soyu t veri türleri tam veri türleridir ve 1980'lerde Ada gibi dillerdeki görünümü,pr ogramlama dili tasarımında önemli bir ilerlemeyi temsil etmiştir. Bugün, nes ne yönelimli diller,soyut veri türlerinin genişletilmiş versiyonları olan sınıfları sağlamaktadır; bunubir sonraki bölümde göreceğiz.

Sorular & Alıştırmalar

1. Bir veri türü ile o türün bir örneği arasındaki fark nedir?
2. Kullanıcı tanımlı veri türü ile soyut veri türü arasındaki fark nedir?
3. Bir liste uygulamak için bir soyut veri türünü tanımlayın.
4. Kontrol hesaplarını uygulamak için bir soyut veri türünü tanımlayın.

Doclingo https://www.doclingo.ai

8.6 Sınıflar ve Nesneler

6. Bölümde öğrendiğimiz gibi, nesne yönelimli paradigma,her biri diğerleriyle etkileşime gi rerek görevleri yerine getiren nesne adı verilen birimlerden oluşan sistemlere yol açar. Her nesne, diğer nesnelerden aldığı mesajlara yanıt veren bir varlıktır. Nesneler, sınıflar olarak bilinen şablonlarla tanımlanır.

Birçok açıdan, bu sınıflar aslında soyut veri türlerinin tanımlarıdır (örnekl eri nesne olarak adlandırılır). Aslında, popüler nesne yönelimli programlama dillerinde sınıfları tanımlamak için kullanılan ifadeler,önceki bölümde tanıtıla n `define type` ifadesine çarpıcı bir şekilde benzemektedir. Örneğin,Şekil 8.27 , `StackOfIntegers` olarak bilinen bir sınıfın Java ve C# dillerinde nasıl tanım lanabileceğini göstermektedir. (C++ dilindeki eşdeğer sınıf tanımı,aynı yapıy a sahip ancak biraz farklı bir sözdizimine sahiptir.) Bu sınıf ile önceki bölümd esoyut veri türü StackType'ı tanımlamak için kullanılan define type ifadesi ara sındaki benzerliğe dikkat edin. Bu sınıf/tip,`StackEntries` olarak adlandırıla n bir tamsayı dizisi, dizide yığının üstünü tanımlamak için kullanılan bir tams ayı olan `StackPointer` veyığını manipüle etmek için prosedürler içermekte dir.

Bu sınıfı bir şablon olarak kullanarak, `StackOne` adında bir nesne, bir Java veya C# pro gramında aşağıdaki ifade ile oluşturulabilir

```
StackOfIntegers StackOne = new StackOfIntegers();
```

veya bir C++ programında aşağıdaki ifade ile

```
StackOfIntegers StackOne();
```

Daha sonra programlarda, 106 değeri `StackOne` üzerine aşağıdaki ifade ile eklenebilir

```
StackOne.push(106);
```

veya `StackOne` üzerindeki en üstteki giriş alınarak OldValue değişkenine aşa ğıdaki ifade ile yerleştirilebilir

```
OldValue = StackOne.pop();
```

Şekil **8.27**Java ve C# ile uygulanmış bir tamsayı yığını

```
sınıf StackOfIntegers
{private int[] StackEntries = new int[20];
 private int StackPointer = 0;


  public void push(int NewEntry)
  {if (StackPointer < 20)
      StackEntries[StackPointer++] = YeniGiriş;
  }

  genel int pop()
   {if (StackPointer > 0) return StackEntries[--StackPointer];
     else return 0;
}\N
}
```

Doclingo https://www.doclingo.ai

Standart Şablon Kütüphanesi

Bu bölümde tartışılan veri yapıları standart programlamayapıları haline gelmi
ştir—öyle standart ki, birçok programlama ortamı bunlarıprimitifler gibi ele al
maktadır. Bir örnek, Standart Şablon Kütüphanesi (STL) ile zenginleştirilmiş C
++ programlama ortamında bulunmaktadır. STL, popüler veri yapılarını tanım
layan önceden tanımlanmış sınıfların bir koleksiyonunu içerir. Sonuç olarak, S
TL'yi bir C++ programına dahil ederek, programcı bu yapıların ayrıntılı tanımın
ı yapma görevinden kurtulmuş olur. Bunun yerine, sadece bu türdeki tanımla
yıcıları, daha önce tanımladığımız şekilde beyan etmesi yeterlidir.
`StackOne` türünde `StackOfIntegers` olarak Bölüm 8.6'da.

Bu özellikler esasen soyut veri türleri ile ilişkili olanlarla aynıdır. Ancak, sı
nıflar ile soyut veri türleri arasında farklılıklar vardır.
İlk olarak, ikincisinin bir uzantısıdır. Örneğin, Bölüm 6.5'te öğrendiğimiz gibi,
nesne yönelimli diller, sınıfların diğer sınıflardan özellikleri miras almasınave
bireysel nesneleri oluşturulduklarında özelleştiren yapıcılar olarak adlandırıl
anözel yöntemler içermesine izin verir. Ayrıca, sınıflar genelliklefarklı kapsüll
eme dereceleri ile ilişkilidir (Bölüm 6.5), bu daörneklerinin iç özelliklerinin ya
nlış yönlendirilmiş kısayollardan korunmasını sağlar. Ve son olarak, bir sınıfili
şkili prosedürleri gruplamak için bir araç olarak kullanılabilir, bu nedenlebir s
ınıf yalnızca prosedür tanımlarından oluşabilir. Bu anlamda, birsınıfı soyut bi
r tür olarak adlandırabiliriz, soyut veri türü yerine.

Sınıflar ve nesneler kavramlarının, programlarda veri soyutlamalarını temsil e
tme tekniklerinin evriminde bir başka adımı temsil ettiğini sonucuna varıyoruz. As
lında, soyutlamaları uygun bir şekilde tanımlama ve kullanma yeteneği, nesne yö
nelimli programlama paradigmasının popülaritesine yol açmıştır.

Sorular & Alıştırmalar

1. Soyut veri türleri ve sınıflar hangi yönlerden benzer? Hangi yönlerden farklıdırlar?
2. Bir sınıf ile bir nesne arasındaki fark nedir?
3. Bir kuyruk-tamsayı türü nesneleri oluşturmak için bir şablon olarak kullan
ılacak bir sınıfı tanımlayın.

8.7 Makine Dilinde Göstergeler

Bu bölümde göstergeleri tanıttık ve bunların veri yapıları oluşturmadaki kull
anımını gösterdik. Bu bölümde, göstergelerin makine dilinde nasıl işlendiğin
i ele alıyoruz.

Doclingo https://www.doclingo.ai

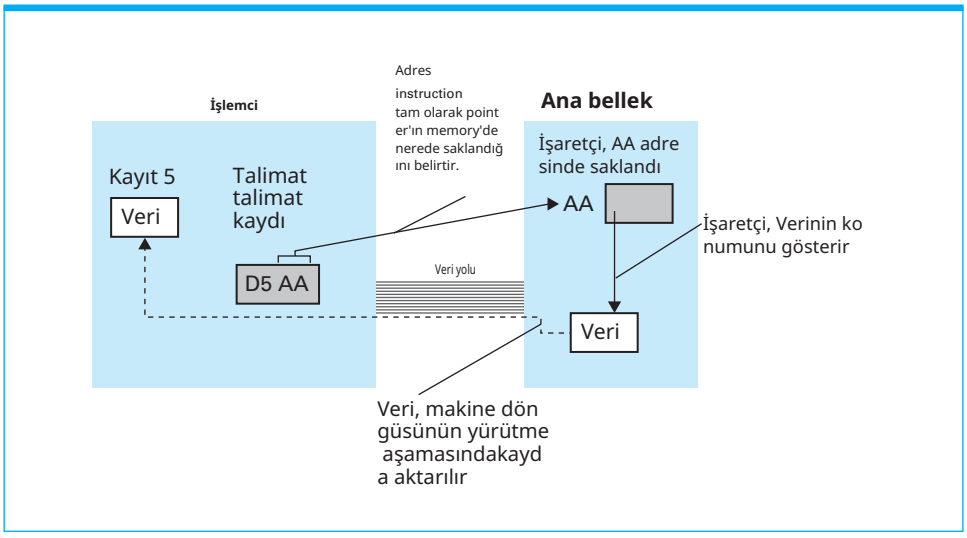
Appendix C'de tanımlanan makine dilinde, yığın üzerindeki bir girişi çıkar mak için bir program yazmak istediğimizi varsayalım. Bu, Şekil 8.12'de açıkla ndığı gibi, o girişi genel amaçlı bir kaydediciye yerleştirmek anlamına gelir. D iğer bir deyişle, yığının üstündeki girişi içeren bellek hücresinin içeriği ile bir kaydediciyi yüklemek istiyoruz. Makine dilimiz, kaydedicileri yüklemek için iki talimat sağlar: biri op-kodu 2, diğeri op-kodu 1 ile. Op-kodu 2 durumunda, o perand alanı yüklenecek veriyi içerir ve op-kodu 1 durumunda, operand alan ı yüklenecek verinin adresini içerir.

İçeriğin ne olacağını bilmiyoruz, bu yüzden amacımıza ulaşmak için op-k odu 2'yi kullanamayız. Ayrıca, adresin ne olacağını bilmediğimiz için op-kodu 1'i de kullanamayız. Sonuçta, yığının üstündeki adres, program çalıştırıldıkça değişecektir. Ancak, yığın işaretçisinin adresini biliyoruz.
Yani, yüklemek istediğimiz verinin adresinin konumunu biliyoruz. O zaman, ihtiyaç duyd uğumuz şey, operandın yüklenmesi gereken verinin adresini içeren bir işaretçi adresini içeren bir kayıt yüklemek için üçüncü bir op-kodu.

Bu hedefe ulaşmak için, Ek C'deki dili D op-kodunu içerecek şekilde geniş letebiliriz. Bu op-koda sahip bir talimat DRXY biçiminde olabilir; bu, R kaydını, adresi XY'de bulunan bellek hücresinin içeriği ile yüklemek anlamına gelir (Ş ekil 8.28). Böylece, yığın işaretçisi AA adresindeki bellek hücresindeyse, D5AA talimatı, yığının en üstündeki verinin 5 kaydına yüklenmesine neden olur.

Ancak bu talimat, pop işlemini tamamlamaz. Yığın işaretçisinden bir çıka rmamız gerekir, böylece yeni yığın üstüne işaret eder. Bu, yükleme talimatın dan sonra, makine dili programımızın yığın işaretçisini bir kayda yüklemesi, ondan bir çıkarması ve sonucu belleğe geri kaydetmesi gerektiği anlamına g elir.

Şekil **8.28** Ek C'deki makine dilini işaretçilerden yararlanacak şekilde genişletme konusund aki ilk denememiz



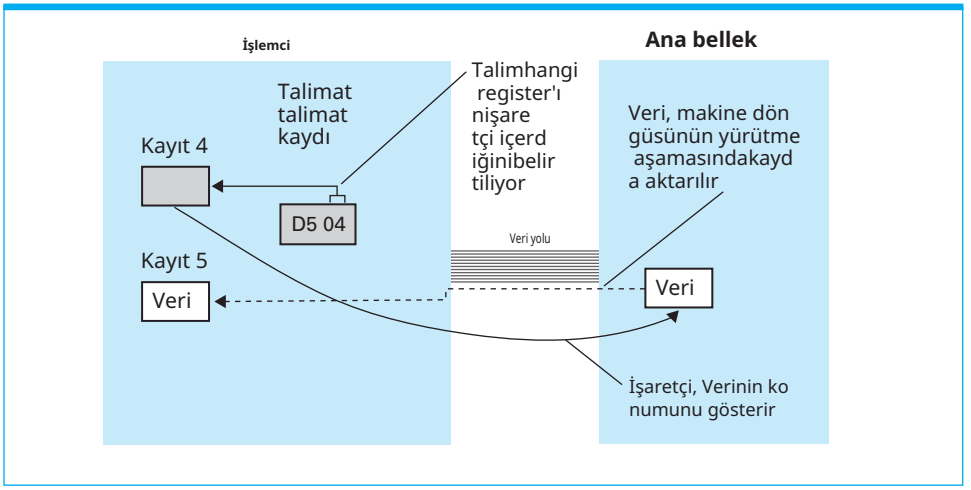
Doclingo https://www.doclingo.ai

Yığın işaretçisini bir bellek hücresi yerine kayıtlardan biri olarak kullanara k,yığın işaretçisinin kayıtlar ve bellek arasında ileri geri hareketini azaltabiliri z. Ancak bu, yüklemekomutunu yeniden tasarlamamız gerektiği anlamına ge lir; böylece işaretçinin anabellekte değil, bir kayıtta olmasını bekleriz. Böylec e, önceki yaklaşımın yerine,D op-koduna sahip bir komut tanımlayabiliriz; bu , R kaydını S kaydı tarafından gösterilenbellek hücresinin içeriği ile yüklemek anlamına gelecektir (Şekil 8.29).
Daha sonra, bir pop işlemi, bu komutun ardından S kaydında saklanan değer den birçıkarma işlemi (veya işlemleri) ile gerçekleştirilebilir.

Bir push işlemini uygulamak için benzer bir komutun gerektiğini unutma yın. Bu nedenle,Ek C'de tanımlanan dili, E op-kodunu tanıtarak daha da geniş letebiliriz; böylece ER0S biçimindeki bir komut,R kaydının içeriğini S kaydı tar afından gösterilen bellek hücresine saklamak anlamına gelecektir. Yine,push işlemini tamamlamak için, bu komutun ardından S kaydındaki değere bir ekl eyen birkomut (veya komutlar) gelmelidir.

Önerdiğimiz bu yeni op-kodlar D ve E, yalnızca makine dillerinin işaretçil eri nasıl manipüle etmek için tasarlandığını göstermekle kalmaz, aynı zaman da orijinal makine dilinde mevcut olmayan bir adresleme tekniğini de göster mektedir. Ek C'de sunulduğu gibi, makine dili bir talimde yer alan verileri tan ımlamak için iki yöntem kullanır. Bunlardan ilki, op-kodu 2 olan bir talimle gö sterilmektedir. Burada, operand alanı açıkça yer alan verileri içermektedir. B una anlık adresleme denir.
Verileri tanımlamanın ikinci yolu, op-kodları 1 ve 3 olan talimlerle gösterilme ktedir. Burada operand alanları, ilgili verilerin adresini içermektedir. Buna d oğrudan adresleme denir. Ancak, önerdiğimiz yeni op-kodlar D ve E, verileri t anımlamanın bir başka biçimini göstermektedir. Bu talimlerin operand alanl arı, verilerin adresinin adresini içermektedir. Buna dolaylı adresleme denir. Üçü de günümüz makine dillerinde yaygındır.

Şekil **8.29** Bir register'ı, bir register'da saklanan bir işaretçi aracılığıyla bulunan bir bellek hücresi nden yükleme



Doclingo https://www.doclingo.ai

Sorular & Alıştırmalar

1. Ek C'de tanımlanan makine dilinin bu bölümün sonunda önerildiği gi bi genişletildiğini varsayalım. Ayrıca, register 8'in DB desenini, DB ad resindeki bellek hücresinin CA desenini ve CA adresindeki hücrenin A 5 desenini içerdiğini varsayalım. Aşağıdaki her bir talimi yürüttükten hemen sonra register 5'te hangi bit deseni olacaktır?
   a. 25A5
   b. 15CA
   c. D508
2. Bu bölümün sonunda açıklanan uzantıları kullanarak, bir pop işlemi gerçekleştiren tam bir makine dili rutini yazın. Yığın yapısının Şekil 8. 12'de gösterildiği gibi uygulandığını varsayın, yığın işaretçisi F kayde dicisindedir ve yığının üstündeki değer 5 kaydedicisine alınacaktır.
3. Bu bölümün sonunda açıklanan uzantıları kullanarak, A0 adresinden başl ayarak beş bitişik bellek hücresinin içeriğini B0 adresinden başlayarak be ş hücreye kopyalayan bir program yazın. Programınızın 00 adresinden ba şladığını varsayın.
4. Bölümde, DR0S biçiminde bir makine talimatı tanıttık. Bu biçimi DRXS olarak genişlettiğimizi varsayalım; bu, "R kaydedicisini S kaydedicisindeki değere artı X değerini gösteren verilerle yükle" anlam ına gelir. Böylece, verilerin işaretçisi, S kaydedicisindeki değeri alarak v e ardından bu değeri X kadar artırarak elde edilir. S kaydedicisindeki de ğer değiştirilmez. (Eğer F kaydedicisi 04 içeriyorsa, DE2F talimatı E kayd edicisini 06 adresindeki bellek hücresinin içeriği ile yükleyecektir. F kay dedicisinin değeri 04 olarak kalacaktır.) Bu talimatın ne gibi avantajları olurdu? S kaydedicisindeki değeri T kaydedicisindeki değerle artırarak gösteren DRTS biçiminde bir talimat hakkında ne dersiniz?

Bölüm Gözden Geçirme Problemleri

(Yıldızlı problemler isteğe bağlı bölümlerle ilişkilidir.)

1. Aşağıdaki dizininsatır ana sırasına ve sütu n ana sırasına görebir makinenin belleğinde n asıl göründüğünü gösteren resimler çizin:

A	B	C	D
EFGH			
IJKL			

2. Altı satırve sekiz sütun içeren homojen bir d izi satır öncelikli olarak saklanmaktadır adres 20'den (onluk) başlayan sıralama. Eğer d izideki heröğe yalnızca bir bellek hücresi gere ktiriyorsa,üçüncü satır ve dördüncü sütund aki öğenin adresi nedir?Her öğe iki bellek h ücresi gerektiriyorsa ne olur?
3. Satır ana sırası yerine sütun ana sırasını varsa yarak Problem 2'yi yeniden çalışın.
4. Birinin geleneksel bir boyutlu homojen dizi kullanar ak dinamik bir liste uygulamaya çalışması durum unda hangi komplikasyonlar ortaya çıkar?

Doclingo https://www.doclingo.ai

5. Üç boyutlu homojen dizileri depolamak için bir yöntem tanımlayın. i. düzlemde, j. satırda ve k. sütunda girişi bulmak için hangi adres polinomunu kullanılır?

6. A, B, C, E, F ve G harflerinin bir bellek hücreleri bloğunda ardışık olarak depolandığını varsayalım. D harfini listeye eklemek için hangi işlemler gereklidir, listenin alfabetik sırasının korunması gerektiğini varsayarsak?

7. Aşağıdaki tablo, bir bilgisayarın ana belleğindeki bazı hücrelerin içeriğini ve her hücrenin temsil edilen adresini göstermektedir. Bazı hücrelerin alfabedeki harfleri içerdiğini ve her bir harf içeren hücrenin ardından boş bir hücre geldiğini unutmayın. Bu boş hücrelere adresler yerleştirin, böylece her harf içeren hücre ve onu takip eden hücre, harflerin alfabetik sırayla göründüğü bir bağlı listede bir giriş oluşturur. (NIL işaretçisi için sıfır kullanın.) Baş işaretçisi hangi adresi içermelidir?

Adres	İçindekiler
11	C
12	
13	G
14	
15	E
16	
17	B
18	
19	U
20	
21	F
22	

8. Aşağıdaki tablo, bir bilgisayarın ana belleğindeki bir bağlı listenin bir kısmını temsil etmektedir. Listede her bir giriş, iki hücreden oluşur: Birincisi alfabedeki bir harfi içerir; ikincisi ise bir sonraki liste girişine işaret eden bir işareti içerir. İşaretçileri değiştirerek N harfinin listede yer almadığında emin olun. Ardından N harfini G harfi ile değiştirin ve yeni harfin alfabedeki doğru yerinde listede görünmesi için işaretçileri değiştirin.

Adres	İçindekiler
30	J
31	38
32	B
33	30
34	X
35	46
36	N
37	40
38	K
39	36
40	P
41	34

9. Aşağıdaki tablo, önceki problemlerle aynı formatı kullanarak bir bağlı listeyi temsil etmektedir. Eğer baş işareti 44 değerini içeriyorsa, liste tarafından hangi isim temsil edilmektedir? İşaretçileri değiştirerek listenin Jean ismini içermesini sağlayın.

Adres	İçindekiler
40	N
41	46
42	I
43	40
44	J
45	50
46	E
47	00
48	M
49	42
50	A
51	40

10. Aşağıdaki rutinlerden hangisi, YeniGiriş'i bağlı listede ÖncekiGiriş'ten hemen sonra doğru bir şekilde ekler? Diğer rutinde ne yanlış?

Rutin 1:

1. ÖncekiGiriş'in işaretçi alanındaki değeri YeniGiriş'in işaretçi alanına kopyalayın.
2. ÖncekiGiriş'in işaretçi alanındaki değeri YeniGiriş'in adresine değiştirin.

Doclingo https://www.doclingo.ai

Rutin 2:

1. ÖncekiGiriş'in işaretçi alanınd aki değeri YeniGiriş'in adresine değişti rin.
2. ÖncekiGiriş'in işaretçi alanındaki değ eri YeniGiriş'in işaretçi alanına kop yalayın.

11. İki bağlı listeyi birleştirmek için bir prosed ür tasarlayın(birini diğerinin önüne koyara k tek bir liste oluşturmak).

12. İki sıralı bitişik listeyi tek bir sıralı bitişiklist eye dönüştürmek için bir prosedür tasarlayı n. Listeler bağlıysa ne olur?

13. Bir bağlı listenin sırasını tersine çevirmek için birprosed ür tasarlayın.

14. a. Bir yığın kullanarak bağlı bir listeyi ters sıradayazdırmak için bir algoritma ta sarlayın.

b. Aynı görevi yığın kullanmadan gerçekleştirmek için bir özyinelemeli prosedür tasarlayın. Özyinelemeli çözümünüzde yığın hangi biçimde hala kullanılmaktadır?

15. Bazen bir bağlı listeye her bir girişe bird en fazlaişaretçi ekleyerek iki farklı sıra veri lir. Aşağıdakitabloda, her harften sonra il k işaretçiyi takip ederekCarol ismini bula cak şekilde doldurun, ancakher harften sonr a ikinci işaretçiyi takip ederekharfleri alfabe tik sırayla bulacaksınız.
Her iki listede temsil edilen baş işaretçisinde hangi değerler bulunmalıdır?

Adres	İçindekiler
60	O
61	
62	
63	C
64	
65	
66	A
67	
68	
69	L
70	
71	
72	R
73	
74	

16. Aşağıdaki tablo, metinde tartışıldığı gibi, bitişik bir bellek hücresi bloğunda saklanan bir yığını temsil etmektedir. Yığının tabanı 10 adresindeyse ve yığın işaretçisi 12 değerini kontrol ediyorsa, bir pop komutuyla hangi değer alınır? Pop işleminden sonra yığın işaretçisinde hangi değer vardır?

Adres	İçindekiler
10	F
11	C
12	A
13	B
14	E

17. Bellek hücrelerinin son içeriklerini gösteren bir tablo çizin. Eğer Problem 16'daki talimat , bir harfi çıkarmak yerine yığın üzerine D harfini itmek olsaydı, yığın işaretçisind eki değer ne olurdu?

18. Yığından alt girişini kaldırmak için bir prosedür t asarlayın, böylece yığının geri kalanı korun ur. Yığını yalnızca itme ve çıkarma işlemleri kulla narak erişmelisiniz. Bu problemi çözmek için han gi yardımcı depolama yapısı kullanılmalıdır?

19. İki yığının içeriğini karşılaştırmak için bir prosed ür tasarlayın.

20. İki yığın verildiğini varsayalım. Eğer bir yı ğından diğerine yalnızca bir seferde girişle ri taşımanıza izin veriliyorsa, orijinal verilerin hangi yeniden düzenlemeleri müm kün olurdu? Üç yığın verildiğinde hangi düze nlemeler mümkün olurdu?

21. Üç yığın verildiğini varsayalım ve yalnızca bir seferde girişleri bir yığından diğerine taşı manıza izin veriliyorsa. Yığınlardan birind e iki bitişik girişi tersine çevirmek için bir al goritma tasarlayın.

22. Uzunlukları değişen isimlerden oluşan bir yığı n oluşturmak istiyoruz. İsimleri bellek alanlarını n ayrı yerlerinde saklamak ve ardından bu isiml ere işaretçilerden oluşan bir yığın oluşturm ak neden avantajlıdır? Yığının kendisinin isimleri içermesine izin vermek yerine?

23. Bir kuyruk, bellekte başı veya kuyruğu yö nünde mi sürünür?

Doclingo https://www.doclingo.ai

24. Yeni girişlerin kendileriyle ilişkili önceliklere sahip olduğu bir "kuyruk" uygulamak istediğinizi varsayalım. Böylece yeni bir giriş, daha düşük öncelikle sahip olanların önüne yerleştirilmelidir. Böyle bir "kuyruk" uygulamak için bir depolama sistemi tanımlayın ve kararlarınızı gerekçelendirin.

25. Bir kuyruktaki girişlerin her birinin bir bellek hücresine ihtiyaç duyduğunu, baş göstergesinin 11 değerini ve kuyruk göstergesinin 17 değerini içerdiğini varsayalım. Bir giriş eklendikten ve iki giriş çıkarıldıktan sonra bu göstergelerin değerleri nedir?

26. a. Aşağıdaki diyagramda gösterilen durumda dairesel bir şekilde uygulanan bir kuyruk olduğunu varsayalım. G ve R harfleri eklendikten, üç harf çıkarıldıktan ve D ve P harfleri eklendikten sonraki yapıyı gösteren bir diyagram çizin.



b. (a) kısmında G, R, D ve P harfleri herhangi bir harf çıkarılmadan önce eklenirse hangi hata meydana gelir?

27. Yüksek seviyeli bir dilde yazılmış bir programda bir kuyruğu uygulamak için bir dizinin nasıl kullanılabileceğini tanımlayın.

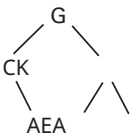
28. İki kuyruk verildiğini ve yalnızca bir girişin bir kuyruktan diğerine baştan kuyruğa taşınmasına izin verildiğini varsayalım. Kuyrukl ardan birinde iki bitişik girişi tersine çevirmek için bir algoritma tasarlayın.

29. Aşağıdaki tablo, bir makinenin belleğinde saklanan bir ağacı temsil etmektedir.Ağacın her düğümü üç hücreden oluşur. İlk hücre,datanın (bir harf) bulunduğu yerdir, ikinci hücre,düğümün sol çocuğunun işaret eden bir işaretçiyive üçüncü hücre ise düğümün sağ çocuğuna işareteden bir işaretçiyi içerir. 0 değeri, bir NIL işaretçisini temsil eder. Eğerkök işaretçisinin değeri 55 ise, ağacın bir resminiçizin.

Adres	İçindekiler
40	G
41	0
42	0
43	X
44	0
45	0
46	J
47	49
48	0
49	M
50	0
51	0
52	F
53	43
54	40
55	W
56	46
57	52

30. Aşağıdaki tablo, bir bilgisayarın ana belleğindeki bir hücre bloğunun içeriğini temsil etmektedir. Bazı hücrelerin alfabedeki harfleri içerdiğini ve bu hücrelerin her birinin ardından iki boş hücre geldiğini unutmayın. Bellek bloğunu, aşağıdaki ağacı temsil edecek şekilde boş hücreleri doldurun. Bir harften sonraki ilk hücreyi, o düğümün solçocuğuna işaretçi olarak ve bir sonraki hücreyisağ çocuğa işaretçi olarak kullanın. NIL işaretçileriiçin 0 kullanın. Kök işaretçisinde hangi değerbulunmalıdır?

Adres	İçindekiler
30	C
31	
32	
33	H
34	
35	
36	K
37	
38	
39	E
40	
41	
42	G
43	
44	
45	P
46	
47	



Doclingo https://www.doclingo.ai

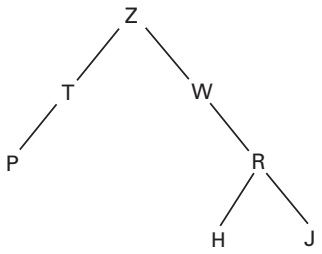
31. Tekrar eden algoritmayı, Şekil 8.21'de temsil edilen tekrar etmeyen bir algoritma ile dizayn edin.

32. Tekrar eden algoritmayı, Şekil 8.24'tetemsil edil en tekrar etmeyen bir algoritma iledizayn edin. Gerekli olabilecek herhangi bir geri izlemekontrolü için bir yığın kullanın.

33. Şekil 8.24'teki tekrar eden ağaç yazdırma algoritmasını uygulayın. Düğüm X yazdırıldığında, algoritmanın iç içe geçmiş aktivasyonlarını (ve her birindeki mevcut konumu) temsil eden bir diyagram çizin.

34. Kök düğümünü aynı tutarak vedata elem anlarının fiziksel konumunu değiştirmed en,Problem 29'un ağacındaki işaretçileri değiştirin,böylece Şekil 8.24'teki ağaç yaz dırmaalgoritması düğümleri alfabetik ol arak yazdırır.

35. Aşağıdaki ikili ağacın,Bölüm 8.3'te açıklandığı gi bi,bir blok sürekli bellek hücresi kullanarakişar etçi olmadan depolandığındabellekte nasıl görü ndüğünü gösteren bir diyagram çizin.



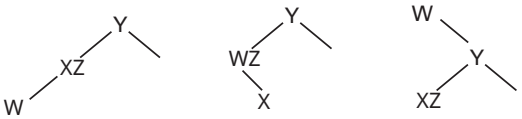
36. Bölüm 8.3'te açıklandığı gibi, bir ikili ağacı temsil eden sürekli hücrelerin sırasıyla A, B, C, D, E, F ve F değerlerini içerdiğini varsayalım. Ağacın bir resmini çizin.

37. Bir listeyi (kavramsal yapı) bir ağaç olarak(gerçek altta yatan yapı) uygulamak isteye bileceğinizbir örnek verin. Bir ağacı (kavra msal yapı)bir liste olarak (gerçek altta yata n yapı)uygulamak isteyebileceğiniz bir ör nek verin.

38. Metinde tartışılan bağlı ağaç yapıları,birinin ebe veynlerden çocuklara doğruağaçta aşağı hareke t etmesine izin verenişaretçiler içeriyordu. Çoc uklardan ebeveynleredoğru ağaçta yukarı harek et etmeyisağlayacak bir işaretçi sistemini tanıml ayın.Kardeşler arasında hareket hakkında ne d ersiniz?

39. Bir satranç oyunu sırasında birtahta konfigürasyon unu temsil etmek içinuygun bir veri yapısını tanımla yın.

40. Aşağıdaki ağaçları tanımlayın, düğümleri Şek il 8.24'teki algoritma tarafından alfabetik sıra yla yazdırılacaktır.



41. Şekil 8.24'teki prosedürü, "listeyi" ters sırayla ya zdıracak şekilde değiştirin.

42. Bir ağaç yapısını tanımlayın, bu yapı bir ailenin so ybilimsel tarihini saklamak için kullanılabilir. Ağaç üzerinde hangi işlemler gerçekleştirilir? Ağ aç bağlı bir yapı olarak uygulanmışsa, her düğü mle hangi işaretçilerin ilişkilendirilmesi gerekir? Yukarıda tanımladığınız işlemleri gerçekleştirme k için prosedürler tasarlayın, ağacın, az önce tan ımladığınız işaretçilerle bağlı bir yapı olarak uygu landığını varsayarak. Depolama sisteminizi k ullanarak, bir kişinin tüm kardeşlerini nasıl bulab ileceğinizi açıklayın.

43. Şekil 8.20'deki şekilde saklanan bir ağaçtan be lirli bir değeri bulmak ve silmek için bir prose dür tasarlayın.

44. Bir ağacın geleneksel uygulamasında, her düğü m, her olası çocuk için ayrı bir işaretçi ile inşa edilir. Bu tür işaretçilerin sayısı bir tasarım kararıd ır ve herhangi bir düğümün sahip olabileceği maksimum çocuk sayısını temsil eder. Eğer bir dü ğüm, işaretçilerin sayısından daha az çocuğa sahips e, bazı işaretçileri basitçe NIL olarak ayarlanır. An cak böyle bir düğüm asla işaretçilerin sayısınd an daha fazla çocuğa sahip olamaz. Bir düğümü n sahip olabileceği çocuk sayısını sınırlamadan bir a ğacın nasıl uygulanabileceğini tanımlayın.

45. Bölüm 8.5'te tanıtılan `define type` saht e kod durumunu kullanarak, bir şirketin çalı şanına ilişkin verileri (isim, adres, iş ataması , maaş ölçeği vb.) temsil eden kullanıcı ta nımlı bir veri türü tanımlayın.

46. Bölüm 8.5'te tanıtılan `define type` saht e kod durumunu kullanarak, bir isimler listesi ni temsil eden soyut bir veri türünün tanımın ı çizin. Özellikle, listeyi içerecek yapı ne olac ak ve listeyi manipüle etmek için hangi pros edürler sağlanacak? (Bunu yapmayacaksınız)

Doclingo https://www.doclingo.ai

prosedürlerin detaylı tanımlarını içermesi gerekir.)

47. 8.5. Bölümde tanıtılan tanımla türü sahte k od (pseudocode) ifadesini kullanarak, bir kuyr uk (queue) temsil eden soyut bir veri türünün tanımını çizin. Ardından, o türün örnekler inin nasıl oluşturulabileceğini ve bu örnekler den nasıl girişlerin eklenip silinebileceğini gö steren sahte kod ifadeleri verin.

48. a. Kullanıcı tanımlı veri türü ile ilkel v eri türü arasındaki fark nedir?
b. Soyut veri tipi ile kullanıcı tanımlı veri tipi arasındaki fark nedir?

49. Bir adres defterini temsil eden soyut bir v eri türündebelirebilecek veri yapıları ve prosedürleri tanımlayın.

50. Bir video oyununda basit bir uzay aracını te msil eden soyut bir veri türündebelirebilecek veri yapıları ve prosedürleri tanımlayın.

*51. Şekil 8.27'yi, sınıfın bir yığın yerine birkuyruk tanı mlayacak şekilde değiştirin.

*52. Bir sınıf, geleneksel soyut veri türünden hangi açıd an daha genel bir yapıya sahiptir?

*53. DR0S veER0S biçimindeki talimatları kulla narak, Bölüm 8.7'nin sonunda açıklandığı gi bi,bir girişi, Şekil 8.12'de gösterildiği gibiuyg ulanan bir yığına itmek içintam bir makine dili rutini yazın.Yığın işaretçisinin F kayde dicisinde olduğunu veitilecek girişin 5 kay dedicisinde olduğunu varsayın.

*54. Her bir bağlı liste girişinin, bir veri bellek hücr esinin ardından geldiğini varsayalım. bir sonraki liste girişine işaretçi. Ayrıca, A0 bellek adresinde bulunan yeni bi r girişin B5 ve C4 konumlarındaki giri şlerin arasına yerleştirileceğini varsayalım. Dizin C'de tanımlanan dili kullanarakve Bölü m 8.7'nin sonunda tanımlanan ek op-k odlar D ve E'yi kullanarak,bir makine dili ru tini yazın.insertion işlemini gerçekleştirme k için.

*55. DR0S biçimindeki bir talimatın, Bölüm 8.7'de tanımlandığı gibi, DRXY biçimindeki bir tali mata göre ne gibi avantajları vardır? Bölüm 8 .7'deki Soru/Alıştırma 4'te tanımlanan DRXS biçiminin, DR0S biçimine göre ne gibi bir a vantajı vardır?

Sosyal Sorunlar

Aşağıdaki sorular, bilgisayar alanıyla ilişkili etik/sosyal/hukuki sorunlara bir r ehber olarak tasarlanmıştır. Amaç yalnızca bu soruları yanıtlamak değildir. A yrıca, neden böyle yanıt verdiğinizi ve gerekçelerinizin bir sorudan diğerine t utarlı olup olmadığını da düşünmelisiniz.

1. Farz edelim ki bir yazılım analisti, belirli bir uygulamada verilerin verimli bir şekil de manipüle edilmesine olanak tanıyan bir veri organizasyonu tasarlıyor. O veri yapısının hakları nasıl korunabilir? Bir veri yapısı bir fikrin ifadesi midir (bir şiir gi bi) ve bu nedenle telif hakkıyla korunur mu yoksa veri yapıları algoritmalar gibi a yrı hukuki boşluklardan mı geçer? Peki ya patent hukuku?
2. Yanlış verilerin, veri olmamasından ne kadar daha kötü olduğu?
3. Birçok uygulama programında, bir yığın (stack) ne kadar büyüyebileceği, m evcut bellek miktarıyla belirlenir. Eğer mevcut alan tüketilirse, yazılım "yığın taşması" gibi bir mesaj üretmek ve sonlandırmak üzere tasarlanmıştır. Çoğ u durumda bu hata asla gerçekleşmez ve kullanıcı bunun farkında olmaz. Böyle bir hata meydana gelirse ve hassas bilgiler kaybolursa kim sorumlu olur? Yazılım geliştiricisi, sorumluluğunu nasıl en aza indirebilir?
4. Bir işaretçi (pointer) sistemine dayanan bir veri yapısında, bir öğenin silinmesi gene llikle bellek hücrelerini silmek yerine bir işaretçiyi değiştirmekten ibarettir. Bu nede nle, bir bağlı liste (linked list) içindeki bir giriş silindiğinde, silinen giriş aslında belle k içinde kalır, ta ki bellek alanı başka veriler tarafından gerekli olana kadar. Silinen v erilerin bu kalıcılığı hangi etik ve güvenlik sorunlarına yol açar?

Doclingo https://www.doclingo.ai

5. Verilerin ve programların bir bilgisayardan diğerine aktarılması kolaydır. Bu nedenle, bir makinede tutulan bilginin birçok makineye aktarılması k olaydır. Buna karşılık, bir insanın bilgiyi başka bir insana aktarması bazen uzun zaman alır. Örneğin, bir insanın başka bir insana yeni bir dil öğret mesi zaman alır. Eğer makinelerin yetenekleri insanlarının yeteneklerini zorlamaya başlarsa, bu bilgi aktarım hızındaki farkın ne gibi sonuçları ola bilir?
6. İşaretçilerin kullanımı, ilişkili verilerin bir bilgisayarın belleğinde, birçok kişinin insan zihninde bilginin nasıl ilişkilendirildiğine inandığı şekilde bağlanmasını sağlar. Bir bilgisayarın belleğindeki bu tür bağlantılar, bir beyindeki bağlantıla ra nasıl benzer? Nasıl farklıdırlar? İnsan zihnini daha yakından taklit eden bilgi sayarlar inşa etmeye çalışmak etik midir?
7. Bilgisayar teknolojisinin yaygınlaşması yeni etik sorunlar mı üretti yoksa sadece önceki etik teorilerin uygulanabilir olduğu yeni bir bağlam mı sağ ladı?
8. Varsayalım ki, bir giriş bilgisayar bilimi kitabının yazarı, metindeki kavra mları göstermek için program örnekleri eklemek istiyor. Ancak, netlik sa ğlamak için birçok örneğin, profesyonel kalitedeki yazılımlarda kullanılac ak olanların basitleştirilmiş versiyonları olması gerekiyor. Yazar, bu örne klerin dikkatsiz okuyucular tarafından kullanılabileceğini ve nihayetinde daha sağlam tekniklerin daha uygun olacağı önemli yazılım uygulamaları na girebileceğini biliyor. Yazar, basitleştirilmiş örnekleri mi kullanmalı, tü m örneklerin sağlam olmasını mı ısrar etmeli, yoksa netlik ve sağlamlığın her ikisini de elde edemediği sürece böyle örnekleri kullanmayı mı redd etmeli?

Ek Okuma

Carrano, F. M. *Veri Soyutlama ve C ile Problem Çözme++: Duvarlar ve Aynalar,* 5. baskı. Boston, MA: Addison-Wesley, 2007.

Carrano, F. M. ve J. Prichard. *Veri Soyutlama ve Java ile Problem Çözme: Duvarlar ve Aynalar,* 2. baskı. Boston, MA: Addison-Wesley, 2006.

Gray, S. *Java'da Veri Yapıları: Soyut Veri Türlerinden Java Koleksiyonları Çerçevesine.* Boston, MA: Addison-Wesley, 2007.

Main, M. *Java Kullanarak Veri Yapıları ve Diğer Nesneler,* 3. baskı. Boston, MA: Addison-Wesley, 2006.

Main, M. ve W. Savitch. *C Kullanarak Veri Yapıları ve Diğer Nesneler++*, 4. baskı. Boston, MA: Addison-Wesley, 2010.

Shaffer, C. A. *Veri Yapıları ve Algoritma Analizine Pratik Giriş*, 2. baskı Upper Saddle River, NJ: Prentice Hall, 2001.

Weiss, M. A. *Java Kullanarak Veri Yapıları ve Problem Çözme*, 3. baskı. Boston, MA: Addison-Wesley, 2006.

Weiss, M. A. *C'de Veri Yapıları ve Algoritma Analizi++,* 3. baskı. Boston, MA: Addison-Wesley, 2007.

Weiss, M. A. *Java'da Veri Yapıları ve Algoritma Analizi,* 2. baskı. Boston, MA: Addison-Wesley, 2007.

Doclingo https://www.doclingo.ai

Bu sayfa kasıtlı olarak boş bırakılmıştır

Doclingo https://www.doclingo.ai

BÖLÜM

Veritabanı Sistemleri 9

Bir veritabanı, büyük bir veri koleksiyonunu soyut bir araca dönüştüren bir sistemdir; bu, kullanıcıların bilgileri arayıp çıkarmasına olanak tanır v e bu işlemi kullanıcı için uygun bir şekilde gerçekleştirir.

Bu bölümde bu konuyu inceliyoruz ve veri madenciliği ile ilgili alanlara yan gezintiler yapıyoruz; bu alan, büyük veri koleksiyonlarındaki gizli kalıpları ortaya çıkarmak için te knikler aramaktadır ve geleneksel dosya yapıları, günüm üzün veritabanı ve veri madenciliği sistemlerinin temel ar açlarını sağlamaktadır.

9.1 Veritabanı Temeller

Veritabanı Sistemlerinin Önemi
Şemaların Rolü
Veritabanı Yönetim Sistemleri
Veritabanı Modelleri

9.2 İlişkisel Model

İlişkisel Tasarım SorunlarıİlişkiselİşlemlerSQL

***9.3Nesne YönelimliVeritabanları**

***9.4VeritabanıBütünlüğünü Koruma**

Commit/Rollback Protokolü
Kilitleme

***9.5Geleneksel Dosya Yapıları**

Sıralı Dosyalar
İndeksli Dosyalar
Hash Dosyaları

9.6 Veri Madenciliği

9.7 Sosyal Etki Veritabanı Teknolojisi

**Yıldızlar, isteğe bağlı bölümler için önerileri belirtir.*

Doclingo https://www.doclingo.ai

Bugünün teknolojisi son derece büyük miktarda veriyi depolama kapasitesine sah iptir, ancak bu veri koleksiyonları, elimizdeki göreve uygun olan belirli bilgi parçal arını çıkaramadığımız sürece işe yaramaz. Bu bölümde veritabanı sistemlerini inc eleyecek ve bu sistemlerin büyük veri kümelerini nasıl yararlı bilgi kaynaklarına d önüştürmek için soyutlama uyguladığını öğreneceğiz. İlgili bir konu olarak, veri k oleksiyonları içindeki kalıpları tanımlama ve keşfetme tekniklerini geliştirmeyi am açlayan hızla genişleyen veri madenciliği alanını araştıracağız. Ayrıca, günümüz ve ritabanı ve veri madenciliği sistemlerinin temelini oluşturan geleneksel dosya yapı larının ilkelerini de inceleyeceğiz.

9.1 Veritabanı Temelleri

Veritabanı terimi, içindeki girişler arasındaki bağlantıların bilgiyi çeşitli bakış açılarından erişilebilir hale getirdiği çok boyutlu bir veri koleksiyonunu ifade eder. Bu, bazen düz dosya olarak adlandırılan geleneksel bir dosya sistemini n (Bölüm 9.5) aksine, tek boyutlu bir depolama sistemi anlamına gelir; yani b ilgiyi tek bir bakış açısıyla sunar.

Bir besteciler ve onların eserleri hakkında bilgi içeren düz bir dosya, besteciy e göre düzenlenmiş eserlerin bir listesini sağlayabilirken, bir veritabanı tek bi r bestecinin tüm eserlerini, belirli bir tür müzik yazan tüm bestecileri ve belki de başka bir bestecinin eserinin varyasyonlarını yazan bestecileri sunabilir.

Veritabanı Sistemlerinin Önemi

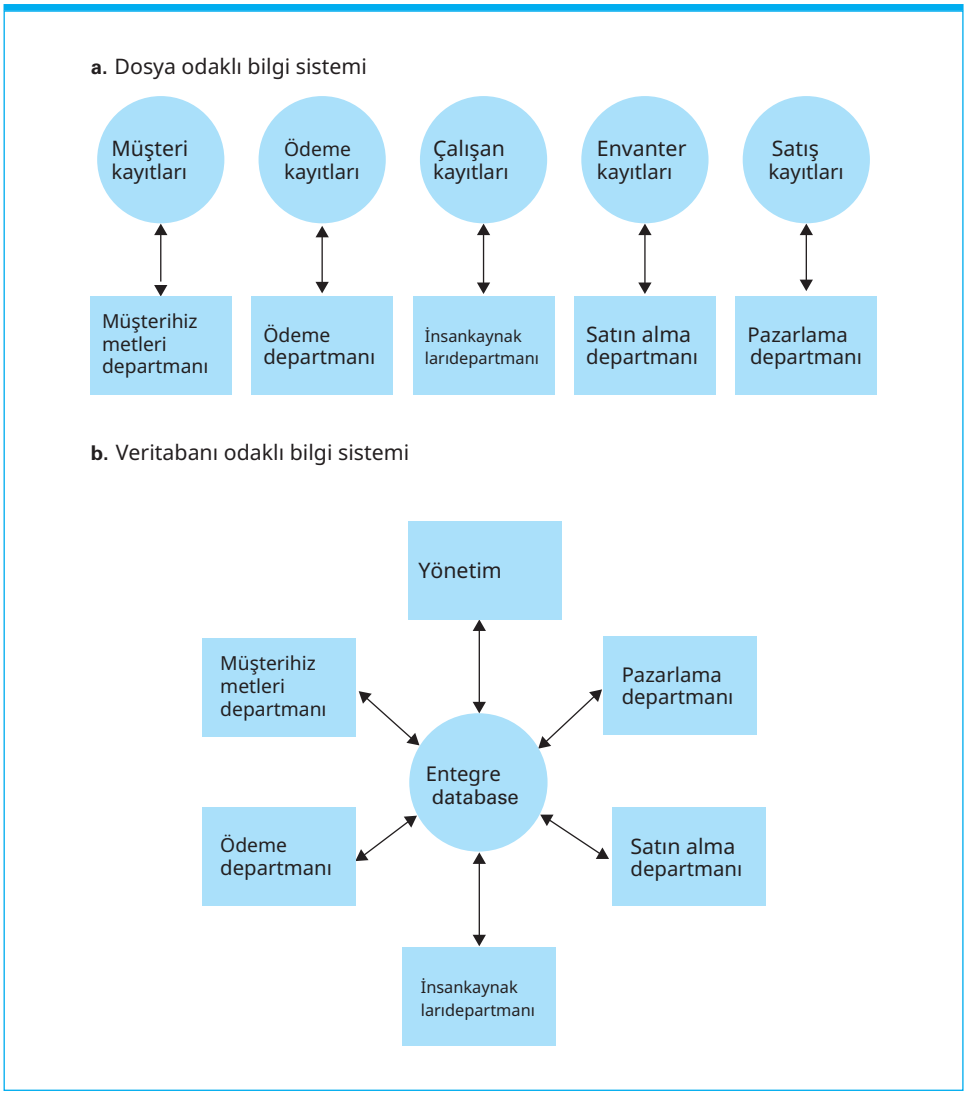
Tarihsel olarak, bilgisayar makineleri bilgi yönetiminde daha geniş kullanımlar buldukça, her uygulama kendi veri koleksiyonu ile ayrı bir sistem olarak uygula nma eğilimindeydi. Bordro, bordro dosyası kullanılarak işleniyordu, personel d epartmanı kendi çalışan kayıtlarını tutuyordu ve envanter envanter dosyası ara cılığıyla yönetiliyordu. Bu, bir organizasyonun ihtiyaç duyduğu bilgilerin çoğun un şirket genelinde kopyalandığı, birçok farklı ama ilgili öğenin ayrı sistemlerde saklandığı anlamına geliyordu. Bu ortamda, veritabanı sistemleri, belirli bir org anizasyon tarafından saklanan ve sürdürülen bilgilerin entegrasyonu için bir ar aç olarak ortaya çıktı (Şekil 9.1). Böyle bir sistemle, aynı satış verileri yeniden st oklama siparişleri üretmek, piyasa trendleri hakkında raporlar oluşturmak, rekl amları ve ürün duyurularını bu tür bilgilere olumlu yanıt verme olasılığı en yüks ek olan müşterilere yönlendirmek ve satış ekibi üyeleri için bonus çekleri oluştu rmak için kullanılabilirdi.

Bütünleşik bilgi havuzları, yönetim kararlarının alınabileceği değerli bir k aynak sağladı, bu bilgilerin anlamlı bir şekilde erişilebilir olduğu varsayılarak. Bunun sonucunda, veritabanı araştırmaları, bir veritabanındaki bilgilerin kar ar verme sürecine nasıl dahil edilebileceğine dair teknikler geliştirmeye odak landı. Bu konuda önemli ilerlemeler kaydedildi. Bugün, veritabanı teknolojisi, veri madenciliği teknikleriyle birleştirildiğinde, bir organizasyonun tüm yönl erini ve çevresini kapsayan devasa veri yığınlarından ilgili bilgileri çıkarmasın a olanak tanıyan önemli bir yönetim aracıdır.

Ayrıca, veritabanı sistemleri, Dünya Çapında Ağ'daki birçok popüler siteyi destekleyen temel teknoloji haline geldi. Google, eBay ve Amazon gibi sitele rin temel teması, istemciler ile veritabanları arasında bir arayüz sağlamaktır.

Doclingo https://www.doclingo.ai

Şekil **9.1** Bir dosya ile bir veritabanı organizasyonu



İstemcinin talebine yanıt vermek için, sunucu bir veritabanını sorgular, sonu çları bir Web sayfası biçiminde düzenler ve o sayfayı istemciye gönderir. Bu t ür Web arayüzleri, veritabanı teknolojisi için yeni bir rol popüler hale getirmi ştir; burada bir veritabanı artık bir şirketin kayıtlarını saklama aracı değil, şirk etin ürünüdür. Gerçekten de, veritabanı teknolojisini Web arayüzleriyle birleş tirerek, İnternet büyük bir dünya çapında bilgi kaynağı haline gelmiştir.

Şemaların Rolü

Veritabanı teknolojisinin yayılmasının dezavantajları arasında, hassas veriler e yetkisiz kişilerin erişim potansiyeli bulunmaktadır. Birinin bir şirketin web s itesinde sipariş vermesi, o kişinin şirketin

Doclingo https://www.doclingo.ai

mali veriler; benzer şekilde, bir şirketin yan haklar departmanındaki bir çalışanın şirketin çalışan kayıtlarına erişim ihtiyacı olabilir ancak şirketin envanterine veya satış kayıtlarına erişim hakkına sahip olmamalıdır. Bu nedenle, veritabanındaki bilgilere erişimi kontrol etme yeteneği, onları paylaşma yeteneği kadar önemlidir.

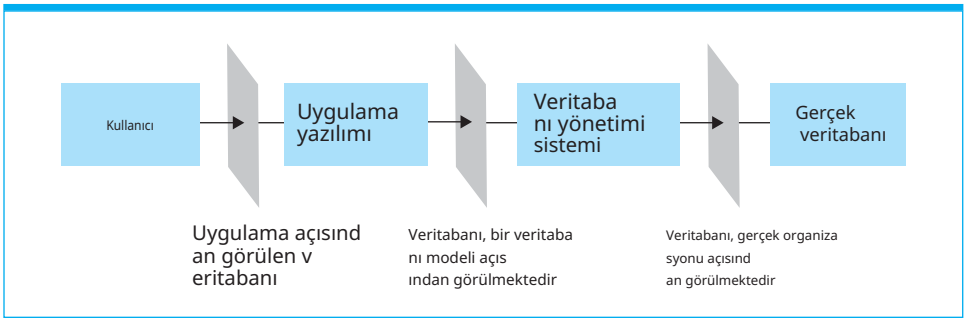
Farklı kullanıcıların bir veritabanındaki farklı bilgilere erişimini sağlamak için,veritabanı sistemleri genellikle şemalar ve alt şemalara dayanır. Bir şema,veritabanı yazılımı tarafından veritabanını sürdürmek için kullanılan tüm veritabanıyapısının bir tanımıdır. Bir alt şema, yalnızca belirli bir kullanıcının ihtiyaçlarıylailgili veritabanının o kısmının tanımıdır. Örneğin, bir üniversite veritabanı içinbir şema, her öğrenci kaydının, öğrencinin akademik kaydının yanı sıra o öğrencininmevcut adresi ve telefon numarası gibi öğeleri içerdiğini gösterecektir. Ayrıca,her öğrenci kaydının, öğrencinin fakülte danışmanının kaydıyla bağlantılı olduğunubelirtecektir. Sırasıyla, her fakülte üyesinin kaydı, kişinin adresini, çalışmageçmişini ve benzeri bilgileri içerecektir. Bu şemaya dayanarak, bir bağlantısistemi sürdürülerek, nihayetinde bir öğrencinin bilgilerini bir fakülte üyesininçalışma geçmişiyle bağlayacaktır.

Üniversitenin öğrenci işleri müdürlüğünün bu bağlantıyı kullanarak öğretim üyeleri hakkında ayrıcalıklı bilgilere erişimini engellemek için, öğrenci işleri müdürlüğünün veritabanına erişimi, öğretim üyesi kayıtlarının istihdam geçmişini içermeyen bir alt şemaya kısıtlanmalıdır. Bu alt şema altında, öğrenci işleri müdürlüğü, belirli bir öğrencinin danışmanının hangi öğretim üyesi olduğunu öğrenebilir, ancak o öğretim üyesi hakkında ek bilgilere erişemez. Buna karşılık, bordro departmanı için alt şema, her öğretim üyesinin istihdam geçmişini sağlayacak, ancak öğrenciler ile danışmanlar arasındaki bağlantıyı içermeyecektir. Böylece bordro departmanı, bir öğretim üyesinin maaşını değiştirebilir, ancak o kişinin danışmanlık yaptığı öğrencilerin isimlerini elde edemez.

Veritabanı Yönetim Sistemleri

Tipik bir veritabanı uygulaması, iki ana katmana gruplandıracağımız birden fazla yazılım katmanını içerir - bir uygulama katmanı ve bir veritabanı yönetim katmanı (Şekil 9.2). Uygulama yazılımı, veritabanının kullanıcısıyla iletişimi yönetir ve oldukça karmaşık olabilir, örneğin

Şekil **9.2** Bir veritabanı uygulamasının kavramsal katmanları



Doclingo https://www.doclingo.ai

Dağıtık Veritabanları

Ağ yeteneklerinin gelişmesiyle birlikte, veritabanı sistemleri, farklı makinelerde bulunan verilerden oluşan dağıtılmış veritabanları olarak bilinen veritabanlarını kapsayacak şekil de büyümüştür. Örneğin, uluslararası bir şirket, yerel sitelerde yerel çalışan kayıtlarını d epolayıp bakımını yapabilir, ancak bu kayıtları bir ağ aracılığıyla bağlayarak tek bir dağıtıl mış veritabanı oluşturabilir.

Dağıtılmış bir veritabanı, parçalanmış ve/veya kopyalanmış veriler içerebilir. İl k durum, veritabanının farklı parçalarının farklı konumlarda depolandığı önceki çal ışan kayıtları örneğiyle gösterilmektedir. İkinci durumda, aynı veritabanı bileşenini n kopyaları farklı konumlarda depolanır. Bu tür bir kopyalama, bilgi alma süresini azaltma aracı olarak gerçekleşebilir. Her iki durum da daha geleneksel merkezi ver itabanlarında mevcut olmayan sorunlar ortaya çıkarır—veritabanının dağıtılmış do ğasını gizlemenin bir yolu veya veritabanının kopyalanmış kısımlarının güncellemel er gerçekleşirken birbirinin kopyası olarak kalmasını sağlamanın bir yolu. Sonuç ol arak, dağıtılmış veritabanlarının incelenmesi, güncel bir araştırma alanıdır.

kullanıcıların bir web sitesi aracılığıyla bir veritabanına eriştiği durumdur. Bu durumda, tü m uygulama katmanı, İnternet genelinde istemcilerden oluşur ve istemcilerin taleplerini ka rşılamak için veritabanını kullanan bir sunucudan oluşur.

Uygulama yazılımının veritabanını doğrudan manipüle etmediğini unutmayın. Veritabanının gerçek manipülasyonu, veritabanıyönetim sistemi (DBYS) tara fından gerçekleştirilir. Uygulama yazılımı, kullanıcının talep ettiği eylemi belir ledikten sonra, sonuçları elde etmek için DBYS'yi soyut bir araç olarak kullanı r. Eğer talep veri eklemek veya silmekse, veritabanını gerçekten değiştiren D BYS'dir. Eğer talep bilgi almaksa, gerekli aramaları gerçekleştiren DBYS'dir.

Uygulama yazılımı ile DBYS arasındaki bu ikilik, birkaç avantaj sağlar. Biri ncisi, soyut araçların inşasına ve kullanımına olanak tanımasıdır; bu, yazılım t asarımında önemli bir basitleştirici kavram olarak tekrar tekrar bulduğumuz bir şeydir. Veritabanının gerçekten nasıl depolandığına dair detaylar DBYS içi nde izole edildiğinde, uygulama yazılımının tasarımı büyük ölçüde basitleştiri lebilir. Örneğin, iyi tasarlanmış bir DBYS ile uygulama yazılımı, veritabanının tek bir makinede mi yoksa bir ağ içinde birçok makineye dağılmış bir dağıtıl mış veritabanı olarak mı depolandığıyla ilgilenmek zorunda değildir. Bunun yerine, DBYS bu sorunlarla ilgilenir ve uygulama yazılımının veritabanına, ver ilerin gerçekten nerede depolandığına dair bir endişe duymadan erişmesine olanak tanır.

Uygulama yazılımının DBMS'ten ayrılmasının ikinci avantajı, böyle bir org anizasyonun veritabanına erişimi kontrol etme aracı sağlamasıdır. DBMS'in v eritabanına erişimi gerçekleştirmesini zorunlu kılarak, DBMS çeşitli alt şemal ar tarafından dayatılan kısıtlamaları uygulayabilir. Özellikle, DBMS iç ihtiyaçla rı için tüm veritabanı şemasını kullanabilir, ancak her kullanıcının kullandığı uygulama yazılımının o kullanıcının alt şemasının tanımladığı sınırlar içinde k almasını talep edebilir.

Doclingo https://www.doclingo.ai

Kullanıcı arayüzünü ve gerçek veri manipülasyonunu iki farklı yazılım katmanına ayırmanın bir diğer nedeni, veri bağımsızlığını sağlamaktır—veritabanının organizasyonunu uygulama yazılımını değiştirmeden değiştirme yeteneği. Örneğin, personel departmanı, her çalışanın kaydına, ilgili çalışanın şirketin yeni sağlık sigortası programına katılmayı seçip seçmediğini belirtmek için ek bir alan eklemek isteyebilir. Eğer uygulama yazılımı doğrudan veritabanıyla ilgileniyorsa, verinin formatındaki böyle bir değişiklik, veritabanıyla ilgilenen tüm uygulama programlarında değişiklikler gerektirebilir. Sonuç olarak, personel departmanı tarafından başlatılan değişiklik, bordro programında ve şirketin bülteni için posta etiketleri yazdırma programında da değişikliklere neden olabilir.

Uygulama yazılımı ile bir VYS arasındaki ayrım, böyle bir yeniden programlama ihtiyacını ortadan kaldırır. Tek bir kullanıcının gerektirdiği veritabanındaki bir değişikliği uygulamak için, yalnızca genel şemayı ve değişiklikle ilgili kullanıcıların alt şemalarını değiştirmek gerekir. Diğer tüm kullanıcıların alt şemaları aynı kalır, bu nedenle değiştirilmemiş alt şemalara dayanan uygulama yazılımları da değiştirilmek zorunda değildir.

Veritabanı Modelleri

Abstraksiyonun içsel karmaşıklıkları gizlemek için nasıl kullanılabileceğini defalarca gördük. Veritabanı yönetim sistemleri bir başka örnek sunar. Bunlar, bir veritabanının iç yapısının karmaşıklıklarını gizler ve veritabanı kullanıcısının veritabanında saklanan bilgilerin daha kullanışlı bir formatta düzenlendiğini hayal etmesine olanak tanır. Özellikle, bir VYS, veritabanının kavramsal görünümü açısından ifade edilen komutları, gerçek veri depolama sisteminin gerektirdiği eylemlere çeviren rutinler içerir. Veritabanının bu kavramsal görünümüne veritabanı modeli denir.

Sonraki bölümlerde hem ilişkisel veritabanı modelini hem de nesne yönelimli veritabanı modelini ele alacağız. İlişkisel veritabanı modeli durumunda, veritabanının kavramsal görünümü, satırlardan ve sütunlardan oluşan bir tablo koleksiyonu olarak tanımlanır. Örneğin, bir şirketin çalışanları hakkında bilgi, her çalışan için bir satır ve ad, adres, çalışan kimlik numarası gibi etiketlenmiş sütunlar içeren bir tablo olarak görülebilir. Bununla birlikte, VYS, uygulama yazılımının tablonun belirli bir satırından belirli girişleri seçmesine veya belki de maaş sütununda bulunan değer aralığını raporlamasına olanak tanıyan rutinler içerecektir—oysa bilgiler aslında satırlarda ve sütunlarda saklanmamaktadır.

Bu rutinler, uygulama yazılımının veritabanına erişmek için kullandığı soyut araçları oluşturur. Daha kesin olarak, uygulama yazılımı genellikle Bölüm 6'da tartışılan genel amaçlı programlama dillerinden birinde yazılır. Bu diller, algoritmik ifadeler için temel bileşenleri sağlar ancak bir veritabanını manipüle etmek için talimatlar içermez. Ancak, bu dillerden birinde yazılmış bir program, DBMS tarafından sağlanan rutinleri önceden yazılmış alt rutinler olarak kullanabilir; bu da dilin yeteneklerini veritabanı modelinin kavramsal görüntüsünü destekleyecek şekilde genişletir.

Daha iyi veritabanı modelleri arayışı devam eden bir süreçtir. Amaç, karmaşık veri sistemlerinin kolayca kavramsallaştırılmasına, bilgi taleplerinin özlü bir şekilde ifade edilmesine ve verimli veritabanı yönetim sistemleri üretmesine olanak tanıyan modeller bulmaktır.

Doclingo https://www.doclingo.ai

Sorular ve Alıştırmalar

1. Bir üretim tesisinde aynı veya benzer envanter bilgilerini farklı şekille rde kullanacak iki departmanı tanımlayın. Ardından, bu iki departma nın alt şemasının nasıl farklı olabileceğini açıklayın.
2. Bir veritabanı modelinin amacı nedir?
3. Uygulama yazılımı ve bir DBMS'in rollerini özetleyin.

9.2 İlişkisel Model

Bu bölümde ilişkisel veritabanı modeline daha yakından bakıyoruz. Bu mode l, verilerin dikdörtgen tablolarda, yani ilişkilerde saklandığını gösterir; bu, bil gilerin hesap tablosu programları tarafından görüntüleme formatına benzer . Örneğin, ilişkisel model, bir firmanın çalışanlarıyla ilgili bilgilerin Şekil 9.3'te gösterilen bir ilişki ile temsil edilmesine olanak tanır.

Bir ilişkideki bir satıra demet denir (bazıları "TOO-pul," bazıları "TU-pul" der). Şekil 9.3'teki ilişkide, demetler belirli bir çalışana ait bilgileri içerir. Bir ilişkide ki sütunlara nitelikler denir çünkü bir sütundaki her giriş, karşılık gelen dem eti temsil eden varlığın bazı özelliklerini veya niteliklerini tanımlar.

İlişkisel Tasarım Sorunları

İlişkisel bir veritabanı tasarlamanın temel adımlarından biri, veritabanını oluşturan ili şkileri tasarlamaktır. Bu basit bir görev gibi görünse de, dikkatsiz tasarımcıları tuzağ a düşürebilecek birçok incelik vardır.

Şekil 9.3'teki ilişkide yer alan bilgilere ek olarak, çalışanların sahip olduğu işler hakkında bilgi eklemek istiyoruz. Her çalışana ait iş geçmişini, iş unvanı (sekreter, ofis yöneticisi, kat süpervizörü), her işe özgü bir iş tanımlama kodu , her işle ilişkili beceri kodu, işin bulunduğu departman ve çalışanın işte bulu nduğu süreyi başlangıç tarihi ve bitiş tarihi olarak içeren nitelikler ile ekleme k isteyebiliriz. (İş, çalışanın mevcut pozisyonunu temsil ediyorsa, bitiş tarihi o larak bir yıldız kullanıyoruz.)

Şekil **9.3** Çalışan bilgilerini içeren bir ilişki

Çalışan No	İsim	Adres	SSN
25	Joe E. Baker	33 Hiçbir Yer St.	111223333
	Cheryl H. Clark	563 Şehir Merkezi Bulvarı.	999009999
	Mr. Jerry Smith	1555 Çember Dr.	111005555
•	•	•	•
•	•	•	•
•	•	•	•

Doclingo https://www.doclingo.ai

Bu soruna bir yaklaşım, Şekil 9.3'teki ilişkiyi genişleterek bu nitelikleri tablodaki ek sütunlar olarak dahil etmektir; bu, Şekil 9.4'te gösterilmiştir. Ancak, sonucun dikkatli incelenmesi birkaç sorunu ortaya koymaktadır. Bunlardan biri, fazlalık nedeniyle verimlilik eksikliğidir. İlişki artık her çalışan için bir demet içermemekte, aksine her çalışanın bir işe atanması için bir demet içermektedir. Eğer bir çalışan, şirkette birkaç işten oluşan bir dizi aracılığıyla ilerlediyse, yeni ilişkideki birkaç demet aynı çalışan hakkında (isim, adres, kimlik numarası ve Sosyal Güvenlik numarası) aynı bilgileri içermelidir. Örneğin, Baker ve Smith hakkında kişisel bilgiler tekrarlanmaktadır çünkü birden fazla işte çalışmışlardır. Dahası, belirli bir pozisyon birçok çalışan tarafından tutulduğunda, o işle ilişkili departman ve uygun beceri kodu, işin atanmasını temsil eden her demette tekrarlanmalıdır. Örneğin, zemin yöneticisi işinin tanımı, birden fazla çalışanın bu pozisyonda bulunması nedeniyle tekrarlanmaktadır.

Uzun ilişkimizin başka, belki de daha ciddi bir sorunu, veritabanından bilgi silmeyi düşündüğümüzde ortaya çıkıyor. Örneğin, Joe E. Baker'ın D7 olarak tanımlanan işi tutan tek çalışan olduğunu varsayalım. Eğer şirketten ayrılırsa ve Şekil 9.4'te temsil edilen veritabanından silinirse, D7 işine dair bilgiyi kaybederiz. Sonuçta, D7 işinin K2 beceri seviyesini gerektirdiğini belirten tek tuple, Joe Baker ile ilgili olan tuple'dır.

Bir tuple'ın yalnızca bir kısmını silme yeteneğinin sorunu çözebileceğini savunabilirsiniz, ancak bu da başka karmaşıklıklar getirecektir. Örneğin, F5 işine ilişkin bilgilerin de kısmi bir tuple'da mı saklanması gerekir, yoksa bu bilgi ilişkide başka bir yerde mi bulunur? Dahası, kısmi tuple'lar kullanma isteği, veritabanı tasarımının geliştirilebileceğinin güçlü bir göstergesidir.

Tüm bu sorunların nedeni, tek bir ilişkide birden fazla kavramı birleştirmiş olmamızdır. Önerildiği gibi, Şekil 9.4'teki genişletilmiş ilişki, doğrudan çalışanlarla (isim, kimlik numarası, adres, Sosyal Güvenlik numarası) ilgili bilgileri, şirketteki mevcut işler hakkında (iş tanımlaması, iş unvanı, departman, beceri kodu) bilgileri ve bilgi içerir.

Şekil **9.4** Bir ilişkideki fazlalık

Çalışan No	İsim	Adres	SSN	İş kimliği	İş Unvanı	Beceri Kodu	Bölüm	Başlangıç Tarihi	Dönem Tarihi
25X15	Joe E. Fırıncı	33 Hiçbir yer St.	111223333	F5	Kat müdürü	FM3	Satış	9-1-2009	9-30-2010
25X15	Joe E. Fırıncı	33 Hiçbir yer St.	111223333	D7	Böl. baş	K2	Satış	10-1-2010	*
34Y70	Cheryl H. Clark	563 Şehir Merkezi Cad.	999009999	F5	Kat müdürü	FM3	Satış	10-1-2009	*
23Y34	G. Jerry Smith	1555 Daire Dr.	111005555	S25X	Sekreter	T5	Personel	3-1-1999	4-30-2010
23Y34	G. Jerry Smith	1555 Daire Dr.	111005555	S26Z	Sekreter	T6	Muhasebe	5-1-2010	*
•	•	•	•	•	•	•	•	•	•
•	•	•	•	•	•	•	•	•	•
•	•	•	•	•	•	•	•	•	•

Doclingo https://www.doclingo.ai

PC'ler için Veritabanı Sistemleri

Kişisel bilgisayarlar, ilkel olanlardan karmaşık olanlara kadar çeşitli uygulamalardakul lanılmaktadır. Noel kartı listelerini saklamak veya bowling ligi kayıtlarınıkorumak gibi ilkel "veritabanı" uygulamalarında, uygulama veri saklama,yazdırma ve sıralama yete neğinden fazlasını gerektirmediği için genellikleveritabanı yazılımı yerine elektronik t ablo sistemleri kullanılmaktadır. Ancak,PC pazarında mevcut gerçek veritabanı sistem leri vardır, bunlardan biriMicrosoft'un Access'idir. Bu, Bölüm 9.2'de tanımlandığı gibi t am bir ilişkiselveritabanı sistemidir ve ayrıca grafik ve rapor oluşturma yazılımıdır. Access, bu metinde sunulan ilkelerin günümüzdeki popüler ürünlerinomurgasını nasıl ol uşturduğuna dair mükemmel bir örnek sunmaktadır.

çalışanlar ve işler arasındaki ilişki ile ilgili (başlangıç tarihi, fesih tarihi). Bu gözlem temelinde, sistemi üç ilişki kullanarak yeniden tasarlayarak sorunlarımızı çözebiliriz—her biri önceki kategorilerden biri için. Orijinal ilişkiyi Şekil 9.3'te (şimdi EMPLOYEE ilişkisi olarak adlandırdığımız) tutabiliriz ve iki yeni ilişki olan JOB ve ASSIGNMENT şeklinde ek bilgileri ekleyebiliriz, bu da Şekil 9.5'teki veritabanını üretir.

Şekil **9.5** Üç ilişkiden oluşan bir çalışan veritabanı

ÇALIŞAN ilişkisi

Çalışan No	İsim	Adres	SSN
25	Joe E. Baker	33 Hiçbir Yer St.	111223333
	Cheryl H. Clark	563 Şehir Merkezi Bulvarı.	999009999
	Mr. Jerry Smith	1555 Çember Dr.	111005555
•	•	•	•
•	•	•	•
•	•	•	•

İŞ ilişkisi

İş kimliği	İş Unvanı	Yetenek Kodu	Bölüm
S25X	Sekreter	T5	Personel
S26Z	Sekreter	T6	Muhasebe
F5	Kat yöneticisi	FM3	Satış
•	•	•	•
•	•	•	•
•	•	•	•

ATAMA ilişkisi

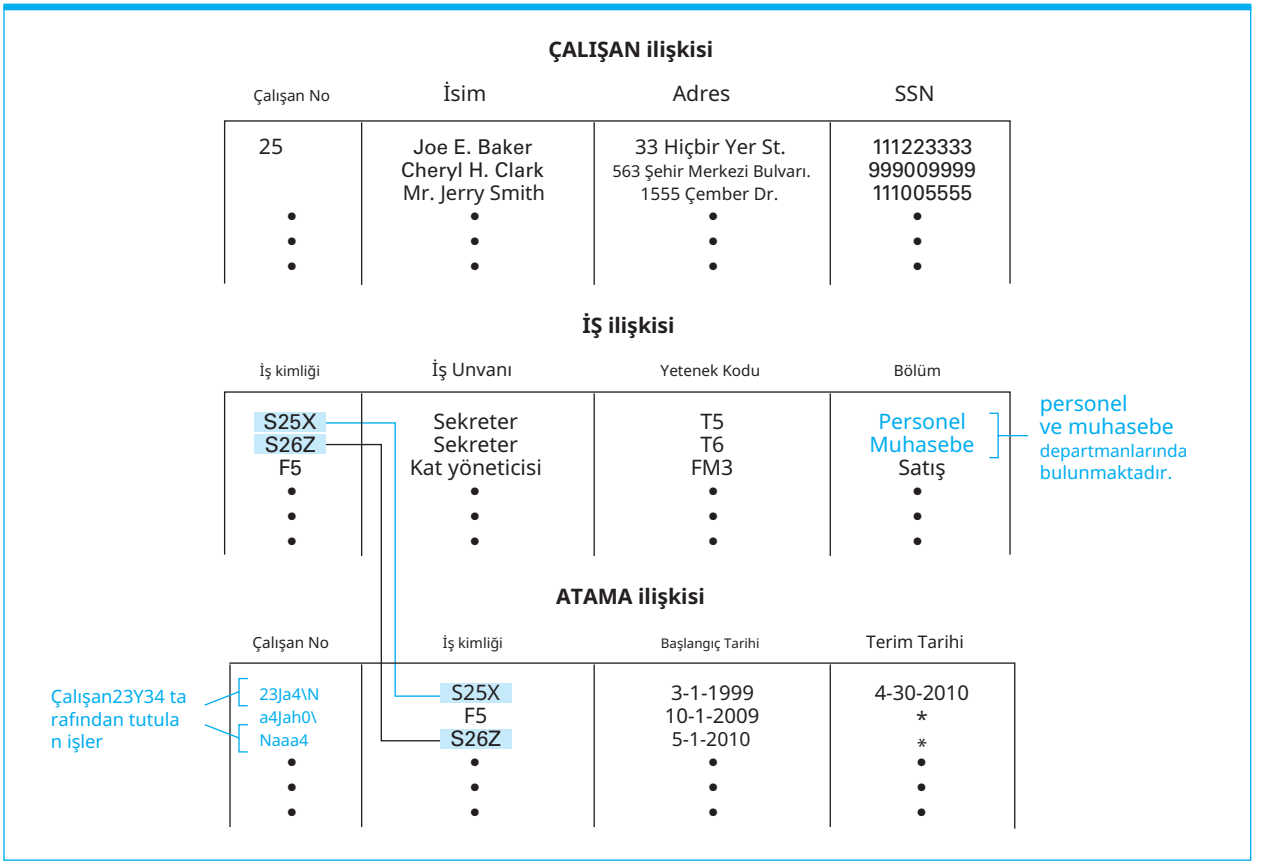
Çalışan No	İş kimliği	Başlangıç Tarihi	Terim Tarihi
23Ja4\N	S25X	3-1-1999	4-30-2010
a4Jah0\	F5	10-1-2009	*
Naaa4	S26Z	5-1-2010	*
•	•	•	•
•	•	•	•
•	•	•	•

Doclingo https://www.doclingo.ai

Bu üç ilişkiden oluşan bir veritabanı, ÇALIŞAN ilişkisi hakkında çalışanlarl a ilgili gerekli bilgileri, mevcut işler hakkında ise bilgi içerir. İŞ ilişkisi ve GÖREV ilişkisi içindeki iş geçmişi hakkında. Farklı ilişkilerden eld e edilen bilgileri birleştirerek ek bilgiye ulaşmak mümkündür. Örneğin, bir ça lışanın kimlik numarasını biliyorsak, o çalışanın çalıştığı departmanları bulma k için önce o çalışanın GÖREV ilişkisi kullanılarak sahip olduğu tüm işleri bul abiliriz ve ardından bu işlerle ilişkili departmanları İŞ ilişkisi aracılığıyla bulab iliriz (Şekil 9.6). Bu tür süreçler aracılığıyla, tek büyük ilişkiden elde edilebilec ek her türlü bilgi, daha önce belirtilen sorunlar olmaksızın üç daha küçük iliş kiden elde edilebilir.

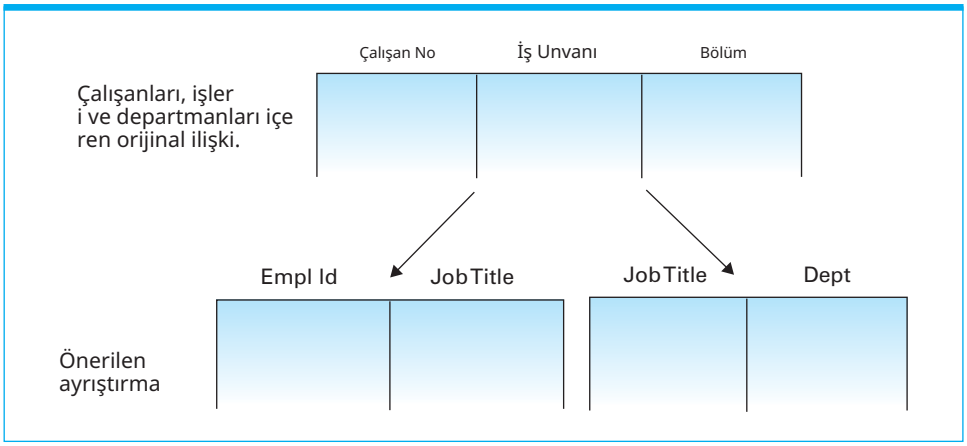
Ne yazık ki, bilgiyi çeşitli ilişkiler halinde bölmek her zaman önceki örnekt eki kadarkolay değildir. Örneğin, Şekil 9.7'deki `EmplId`,`JobTitle` ve `Dept` öz niteliklerine sahip orijinalilişkiyi, iki ilişkiye önerilen ayrıştırma ile karşılaştırın . İlk bakışta, iki ilişki sistemitek ilişki sisteminin aynı bilgileri içeriyormuş gibi görünebilir, ancakaslında öyle değildir. Örneğin, belirli bir çalışanın çalıştığı departmanı bulma sorununu düşünün.Bu, hedef çalışanın kimlik numarasını içeren demeti sorgulayarak vekarşılık gelen departmanı çıkararak tek ilişki si steminde kolayca yapılabilir. Ancak,

Şekil **9.6** Çalışan 23Y34'in çalıştığı departmanları bulma



Doclingo https://www.doclingo.ai

Şekil **9.7** Bir ilişki ve önerilen ayrıştırma



iki ilişkili sistemde, istenen bilgi mutlaka mevcut değildir.
Hedef çalışanın iş unvanını ve böyle bir işe sahip bir departmanı bulabiliriz, a ncak bu, hedef çalışanın o belirli departmanda çalıştığı anlamına gelmez, çü nkü birkaç departman aynı unvana sahip işlere sahip olabilir.

O halde, bazen bir ilişkiyi daha küçük ilişkilere ayırmanın bilgi kaybına ne den olduğunu ve bazen de neden olmadığını görüyoruz. (İkincisine kayıpsız ayrıştırma veya bazen kayıpsız ayrıştırma denir.) Bu tür ilişkisel özellikler öne mli tasarım unsurlarıdır. Amaç, veritabanı tasarımında sorunlara yol açabilec ek ilişkisel özellikleri tanımlamak ve bu sorunlu özellikleri ortadan kaldırmak için bu ilişkileri yeniden düzenlemenin yollarını bulmaktır.

İlişkisel İşlemler

Veri, ilişkisel model açısından nasıl düzenlenebileceğine dair temel bir anlayışa sahip olduğunuzdan, ilişkilerden oluşan bir veritabanından bilginin nasıl çıkarılabileceğini görmek için zamanıdır. İlişkiler üzerinde gerçekleştirmek isteyebileceğimiz bazı işle mlere bir göz atarak başlayalım.

Bazen bir ilişkiden belirli demetleri seçmemiz gerekir. Bir çalışana dair bil gileri almak için, `EMPLOYEE` ilişkisinden uygun tanımlama niteliği değerine sa hip demeti seçmemiz gerekir veya belirli bir departmandaki iş unvanlarının li stesini elde etmek için, o departmanı departman niteliği olarak taşıyan `JOB` il işkisinden demetleri seçmemiz gerekir. Bu tür bir işlemin sonucu, ana ilişkid en seçilen demetlerden oluşan başka bir ilişkidir. Belirli bir çalışana dair bilgi seçmenin sonucu, yalnızca `EMPLOYEE` ilişkisinden bir demet içeren bir ilişki il e sonuçlanır. Belirli bir departmanla ilişkili demetleri seçmenin sonucu, muht emelen `JOB` ilişkisinden birkaç demet içeren bir ilişki ile sonuçlanır.

Kısacası, bir ilişkide gerçekleştirmek isteyebileceğimiz bir işlem, belirli özelliklere sahip demetleri seçmek ve bu seçilen demetleri yeni bir ilişkide yerleştirmektir. Bu iş lemi ifade etmek için, aşağıdaki sözdizimini benimseriz

YENİ← EMPLOYEE'den seç SELECT where EmplId = “34Y70”

Doclingo https://www.doclingo.ai

Bu ifadenin anlamı, NEW adında yeni bir ilişki oluşturmaktır ve bu ilişki, EMPLOYEE ilişkisi içindeki, EmplId niteliği 34Y70 olan (bu durumda yalnızca bir tane olmalıdır) tuple'ları içermektedir (Şekil 9.8).

SELECT işleminin bir ilişkiden satırları çıkardığına karşılık, PROJECT işlemi sütunları çıkarmaktadır. Örneğin, belirli bir departmandaki iş unvanlarını ararken, hedef departmana ait olan JOB ilişkisinden tuple'ları zaten SEÇTİYSEK ve bu tuple'ları NEW1 adında yeni bir ilişkiye yerleştirmişsek, aradığımız liste bu yeni ilişkideki JobTitle sütunudur. PROJECT işlemi, bu sütunu (gerekirse birden fazla sütunu) çıkarmamıza ve sonucu yeni bir ilişkiye yerleştirmemize olanak tanır. Böyle bir işlemi şu şekilde ifade ederiz:

```
NEW2 ← PROJECT JobTitle from NEW1
```

Sonuç, NEW1 ilişkisinin JobTitle sütunundaki tek değer sütununu içeren başka bir yeni ilişki (adına NEW2 denir) oluşturulmasıdır.

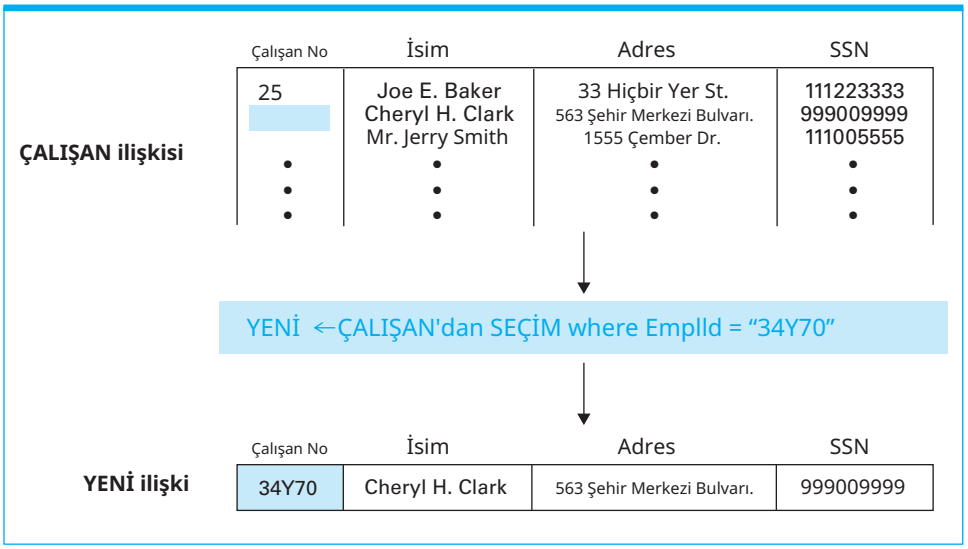
PROJECT işleminin bir başka örneği olarak, ifade

```
MAIL ← PROJECT Name, Address from EMPLOYEE
```

tüm çalışanların isim ve adreslerinin listesine ulaşmak için kullanılabilir. Bu liste, yeni oluşturulan (iki sütunlu) MAIL ilişkisi içindedir (Şekil 9.9).

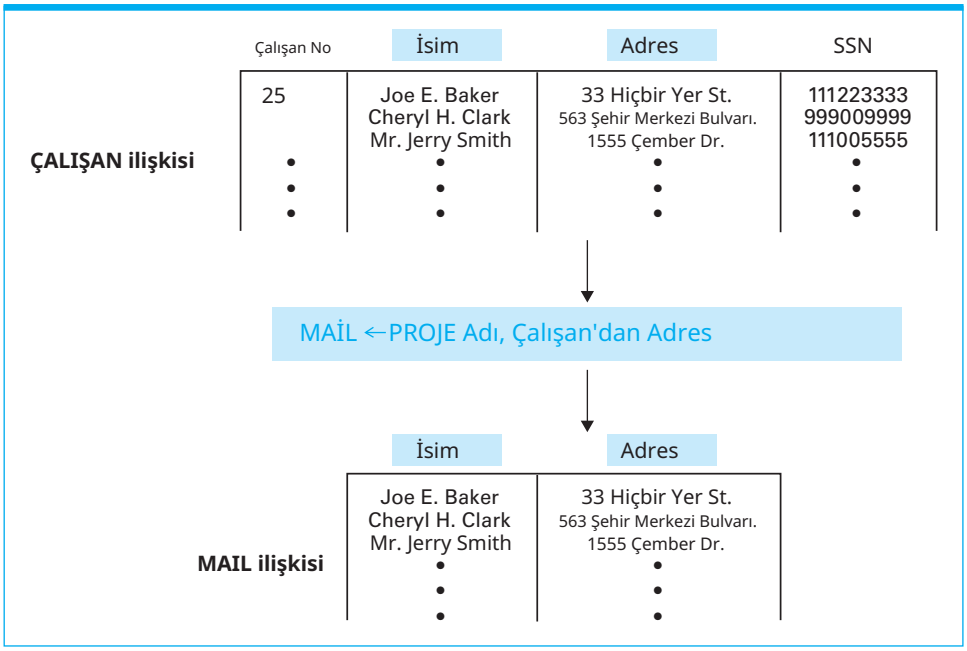
İlişkisel veritabanları ile birlikte kullanılan bir diğer işlem JOIN işlemidir. Farklı ilişkileri birleştirmek için kullanılır.
İki ilişkinin JOIN'i, orijinal ilişkilerin özniteliklerinden oluşan yeni bir ilişki üretir (Şekil 9.10). Bu özniteliklerin adları, orijinal ilişkilerdeki adlarla aynıdır, tek farkla ki her biri, kökeninin ilişkisi ile ön eklenmiştir. (Eğer V ve W özniteliklerini içeren A ilişkisi, X, Y ve Z özniteliklerini içeren B ilişkisi ile JOIN edilirse, son uçta A.V, A.W, B.X, B.Y ve B.Z adında beş öznitelik bulunur.) Bu adlandırma kuralı, yeni ilişkideki özniteliklerin

Şekil **9.8**SEÇİM işlemi



Doclingo https://www.doclingo.ai

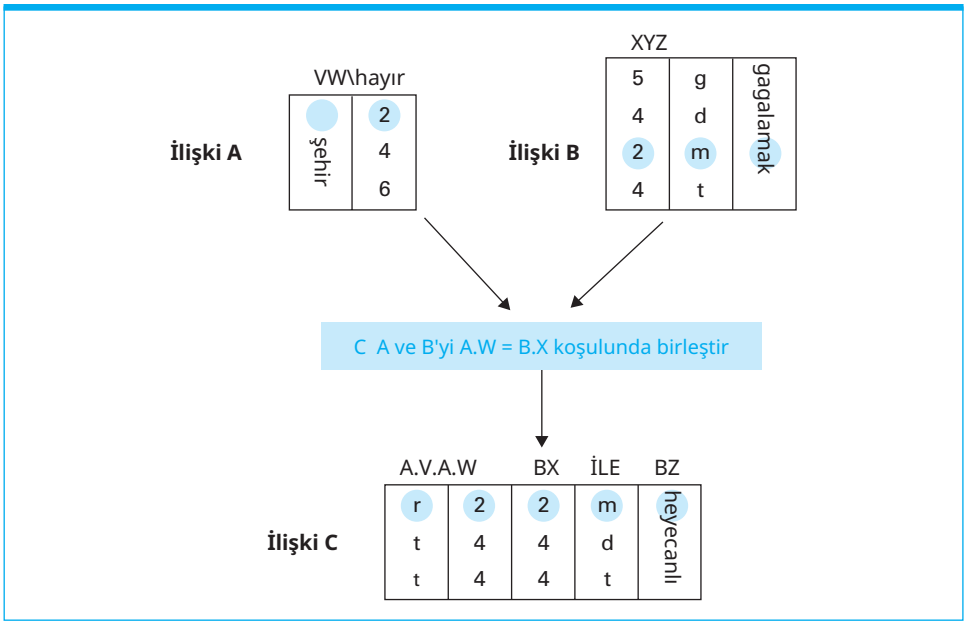
Şekil **9.9**PROJE işlemi



benzersiz adlar, orijinal ilişkilerin ortak öznitelik adlarına sahip olabileceği duru mlarda bile.

Yeni ilişkinin satırları, iki orijinal ilişkiden gelen satırların birleştirilmesiyle üretilir (tekrar Şekil 9.10'a bakın). Hangi satırların gerçekten

Şekil **9.10**JOIN işlemi



Doclingo https://www.doclingo.ai

Yeni ilişkideki demetlerin oluşturulması, JOIN'in inşa edildiği koşul tarafında n belirlenir.Bu koşullardan biri, belirlenen niteliklerin aynı değere sahip olma sıdır. Bu, aslında, Şekil 9.10'da temsil edilen durumdur; buradaifadenin yürüt ülmesinin sonucunu gösteriyoruz.

```
C ← JOIN A and B where A.W = B.X
```

Bu örnekte, A ilişkisinden bir demet, B ilişkisinden bir demetle yalnızca W veX niteliklerinin ikidemette eşit olduğu durumlarda birleştirilmelidir. Böylece, A i lişkisinden (r, 2) demetinin,B ilişkisinden (2, m, q) demetiyle birleştirilmesi, ilk demetteki W niteliğinin değerininikinci demetteki X niteliğinin değeriyle eşit olmasından dolayı sonuçta görünmektedir. Öte yandan,A ilişkisinden (r, 2) d emetinin, B ilişkisinden (5, g, p) demetiyle birleştirilmesinin sonucu,W veX nit eliklerinde ortak değerler paylaşmadıkları için son ilişkide görünmemektedir.

Başka bir örnek olarak, Şekil 9.11, ifadenin yürütülmesinin sonucunu temsil etmektedir.

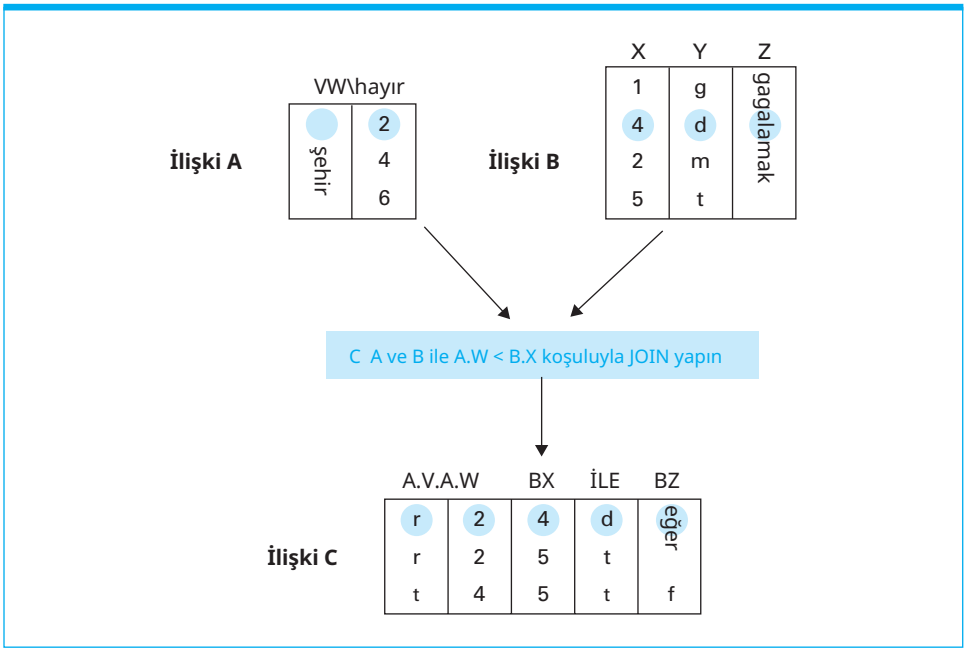
```
C ← JOIN A and B where A.W < B.X
```

Sonuçtaki demetlerin, W ile ilgili olanların tam olarak o olduğunu unutmayın. A, B ile ilgili olarak X niteliğinden daha küçüktür.

Şimdi, Şekil 9.5'teki veritabanı ile JOIN işleminin nasıl kullanılabileceğine bakalım ve her çalışanın çalıştığı departman ile birlikte tüm çalışan kimlik nu maralarının bir listesini elde edelim. İlk gözlemimiz, gerekli bilgilerin birden f azla ilişkide dağıtılmış olduğudur ve bu nedenle bilgiyi geri alma süreci yalnı zca SEÇİM ve PROJE işlemlerini içermelidir. Aslında, ihtiyacımız olan araç, ifad edir

YENİ1← GÖREV ve İŞİ BİRLEŞTİR
GÖREV.JobId = İŞ.JobId olduğu yer

Şekil **9.11**JOIN işleminin bir başka örneği



Doclingo https://www.doclingo.ai

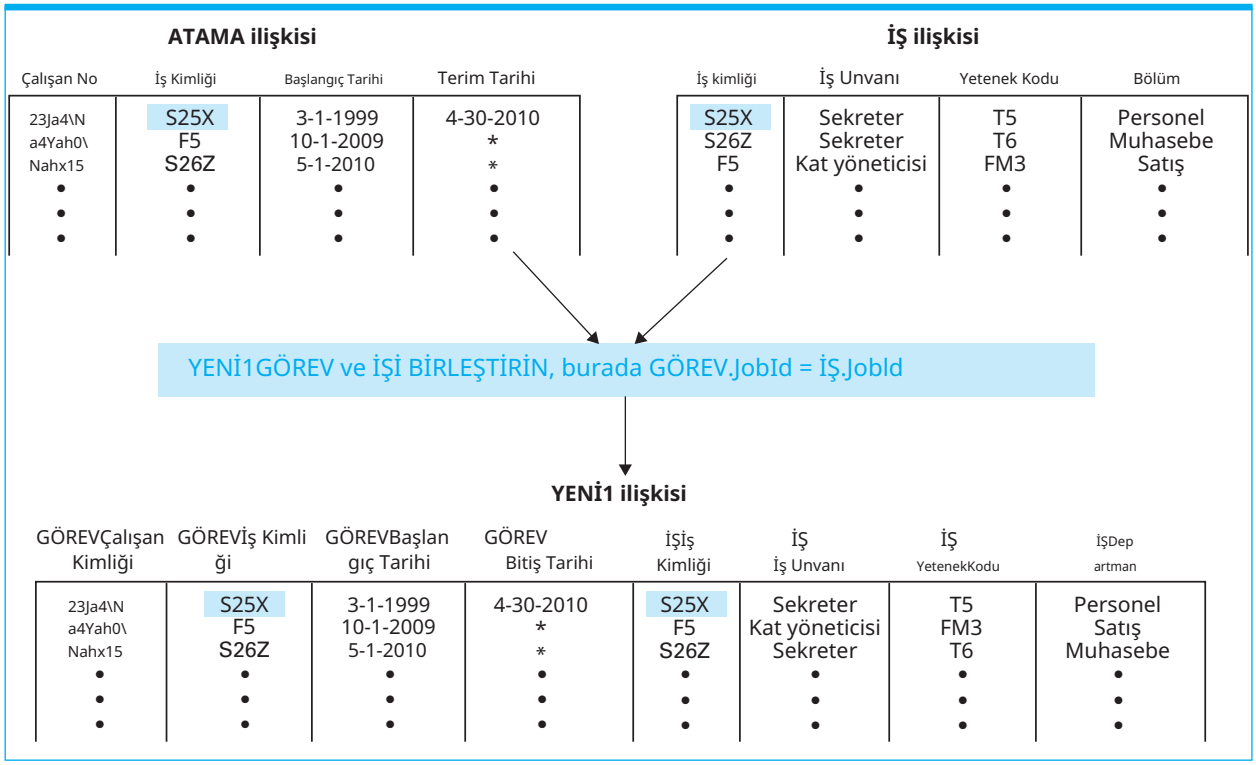
bu ilişkiyi üretir `NEW1`, Şekil 9.12'de gösterildiği gibi. Bu ilişkiden, sorunumuza önce ASSIGNMENT olan o tuple'ları SEÇerek çözüm bulabiliriz. `TermDate` "*" (yani "hala çalışıyor") ile eşitlenir ve ardından PROJEKTE ederek özellikleri `ASSIGNMENT.EmplId` ve `JOB.Dept` alırız. Kısacası, ihtiyaç duyduğumuz bilgi Şekil 9.5'teki veritabanından aşağıdaki sırayı çalıştırarak elde edilebilir

YENİ1b GÖREV ve İŞİ BİRLEŞTİR
GÖREV.JobId = İŞ.JobId
YENİ2b YENİ1'den SEÇ where GÖREV.TermDate = "*"
LİSTEb PROJE GÖREV.EmplId, İŞ.Dept YENİ2'den

SQL

Temel ilişkisel işlemleri tanıttıktan sonra, bir veritabanı sisteminin genel yapı sını yeniden gözden geçirelim.Unutmayın ki bir veritabanı aslında bir kütle d epolama sisteminde saklanır. Uygulama programcısını bu tür sistemlerin det aylarından kurtarmak için, uygulama yazılımının bir veritabanı modeli, örneğ in ilişkisel model açısından yazılmasına olanak tanıyan bir veritabanı yönetim sistemi sağlanır. DBMS, model açısından komutları kabul eder ve bunları ge rçek depolama yapısına göre eylemlere dönüştürür. Bu dönüşüm, DBMS için de uygulama yazılımı tarafından soyut araçlar olarak kullanılan bir dizi rutin t arafından yönetilir. Böylece ilişkisel modele dayalı bir DBMS, SELECT, PROJEC T ve JOIN işlemlerini gerçekleştirmek için rutinler içerecek ve bu rutinler uyg ulama yazılımından çağrılabilecektir. Bu şekilde

Şekil **9.12** JOIN işleminin bir uygulaması



Doclingo https://www.doclingo.ai

uygulama yazılımı, veritabanının aslında ilişkisel modelin basit tablo biçimin de saklandığı gibi yazılabilir.

Günümüzün ilişkisel veritabanı yönetim sistemleri, SELECT, PROJECT ve J OIN işlemlerini ham biçimlerinde gerçekleştirmek için mutlaka rutinler sağla maz. Bunun yerine, bu temel adımların kombinasyonları olabilecek rutinler s unarlar. Bir örnek, çoğu ilişkisel veritabanı sorgu sisteminin belkemiğini oluş turan SQL (Yapılandırılmış Sorgu Dili) dilidir. Örneğin, SQL, birçok veritabanı sunucusu tarafından kullanılan MySQL (okunuşu "My–S–Q–L") ilişkisel veritab anı sisteminin temel dilidir.

SQL'in popülaritesinin bir nedeni, Amerikan Ulusal Standartlar Enstitüsü tarafından standartlaştırılmış olmasıdır. Diğer bir neden ise, ilk olarak IBM tarafından geliştirilip p azarlanmış olması ve bu nedenle yüksek bir görünürlükten faydalanmış olmasıdır. Bu b ölümde, ilişkisel veritabanı sorgularının SQL'de nasıl ifade edildiğini açıklıyoruz.

SQL'de ifade edilen bir sorgunun, emir kipinde bir biçimde ifade edildiğin i bulmak üzereyiz, ancak gerçek şu ki, esasen bir deklaratif ifadedir. Bir SQL i fadesini, gerçekleştirilecek bir dizi aktivite yerine, istenen bilginin bir tanımı olarak okumalıyız. Bunun önemi, SQL'in uygulama programcılarını ilişkileri m anipüle etmek için algoritmalar geliştirme yükünden kurtarmasıdır; yalnızca i stenen bilgiyi tanımlamaları yeterlidir.

SQL ifadesinin ilk örneği için, son sorgumuzu yeniden gözden geçirelimç alışan kimlik numaralarını ve bunlara karşılık gelen departmanları elde etme k için üç aşamalı bir süreç geliştirdiğimiz. SQL'de bu tümsorgu tek bir ifade il e temsil edilebilir

```
ASSIGNMENT'tan EmplI
d, Deptseçin; burada ASS
IGNMENT.JobId = JOB.JobIdve ASSIGNMEN
   T.TermDate = '*'
```

Bu örneğin gösterdiği gibi, her SQL sorgu ifadesi üçkloz içerebilir: bir `select` klozu, bir `from` klozu ve bir `where` klozu. Kabaca konuşursak,bu tür bir ifade, `from` klozunda listelenen tüm ilişkilerin JOIN'ini oluşturma sonucun u talep eden bir istektir,`where` klozundaki koşulları sağlayan o tuple'ları SEÇe rek ve ardından selectklozunda listelenen tuple'ları PROJEKTE ederek. (Termi nolojinin biraz ters döndüğünü unutmayın çünkü `select` klozubir SQL ifade sinde PROJEKTE işlemi için kullanılan nitelikleri tanımlar.)Bazı basit örnekleri göz önünde bulunduralım.

İfade

```
ÇALIŞAN'dan
Ad, Adres'i seçin
```

EMPLOYEE ilişkisi içinde yer alan tüm çalışan isimleri ve adreslerinin bir listesini üretir

Not edin ki bu yalnızca bir PROJEKTE işlemi.

İfade

```
EmplId, Ad, Adres, SSNum seçin
EMPLOYEE'den
      Ad = 'Cheryl H. Clark' olan yer
```

Cheryl H. Clark ile ilişkili olan demet bilgilerini üretir demekteki`EMPLOYEE` ilişkisi. Bu esasen bir SEÇİM işlevidir.

Doclingo https://www.doclingo.ai

İfade

ÇALIŞAN'dan
Ad, Adres'i seçin
Ad = ‘Cheryl H. Clark’ olan yer

Cheryl H. Clark'ın adını ve adresini üretir
`EMPLOYEE` ilişkisi içinde. Bu, SEÇİM ve PROJEKTİF işlemlerinin bir kombinasyonudur.

İfade

EMPLOYEE.Name, ASSIGNMENT.StartDate seçin
EMPLOYEE, ASSIGNMENT'dan
EMPLOYEE.EmplId = ASSIGNMENT.EmplId olan yer

tüm çalışan adlarının ve ilk istihdam tarihlerinin bir listesini üretir.
Bu, `EMPLOYEE` ve ASSIGNMENT ilişkilerini BİRLEŞTİRmenin
ve ardından uygun demetleri ve nitelikleri SEÇmenin ve PROJEKTİF etmenin sonucudur.
`where` ve`select` ifadelerinde tanımlandığı gibi.

SQL'in ilişkilerin yapısını tanımlamak, ilişkiler oluşturmak ve ilişkilerin içe riğini değiştirmek için ifadeleri kapsadığını ve sorgular gerçekleştirdiğini beli rtmekle kapanıyoruz. Örneğin, aşağıdakilerinsert into, delete from, ve `update` ifadelerine örneklerdir.

İfade

ÇALIŞAN'a ekle
değerler (‘42Z12’, ‘Sue A. Burt’, ‘33 Fair St.’,
`‘444661111’)`

verilen değerleri içeren bir ÇALIŞAN ilişkisine bir tuple ekler;

silmek için ÇALIŞAN'dan
Ad = 'G. Jerry Smith' olan kaydı siler

G. Jerry Smith ile ilgili olan demeti ÇALIŞAN ilişkisi içinden kaldırır; ve

ÇALIŞAN'ı günceller
Adres = '1812 Napoleon Ave.'
Ad = 'Joe E. Baker' olan kaydı günceller

Joe E. Baker ile ilişkili olan demetteki adresi değiştirir.

Sorular ve Alıştırmalar

1. Aşağıdaki soruları, Şekil 9.5'teki ÇALIŞAN, İŞ, ve GÖREV ilişkileri teme linde verilen kısmi bilgilere dayanarak yanıtlayın:
   a. Personel departmanında deneyimi olan muhasebe departmanınd aki sekreter kimdir?
   b. Satış departmanındaki kat yöneticisi kimdir?
   c. G. Jerry Smith şu anda hangi işi yapıyor?
2. Şekil 9.5'te sunulan ÇALIŞAN, İŞ, ve GÖREV ilişkilerine dayanarak, pe rsonel departmanındaki tüm iş unvanlarının bir listesini elde etmek i çin bir dizi ilişkisel işlem yazın.

Doclingo https://www.doclingo.ai

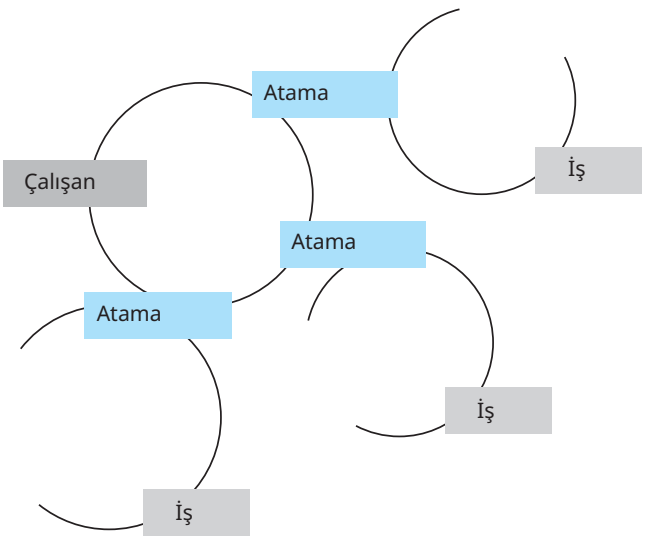
3. Şekil 9.5'te sunulan `EMPLOYEE`, `JOB`, ve`ASSIGNMENT` ilişkilerine day anarak, çalışan isimlerinin yanı sıra çalışanların departmanlarını elde etmek için bir dizi ilişkisel işlem yazın.
4. Sorular 2 ve 3'teki cevaplarınızı SQL'e dönüştürün.
5. İlişkisel model veri bağımsızlığını nasıl sağlar?
6. Bir ilişkisel veritabanındaki farklı ilişkiler nasıl bir araya getirilir?

9.3 Nesne Yönelimli Veritabanları

Başka bir veritabanı modeli, nesne yönelimli paradigma üzerine kuruludur. B u yaklaşım, nesnelerin birbirleriyle bağlantılı olduğu ve ilişkilerini yansıttığı b ir nesne yönelimli veritabanı oluşturur. Örneğin, önceki bölümdeki çalışan v eritabanının nesne yönelimli bir uygulaması, üç sınıf (nesne türü) içerebilir: ÇALIŞAN, İŞ ve GÖREV. ÇALIŞAN sınıfından bir nesne, `EmplId`, İsim, Adres v e`SSNum` gibi girişler içerebilir; İŞ sınıfından bir nesne, `JobId`, İşUnvanı, Yete nekKodu ve `Dept` gibi girişler içerebilir; ve GÖREV sınıfından her nesne, Baş langıçTarihi ve BitişTarihi gibi girişler içerebilir.

Bu tür bir veritabanının kavramsal temsili Şekil 9.13'te gösterilmiştirve çe şitli nesneler arasındaki bağlantılar, ilgili nesneleri birleştiren çizgilerle temsi l edilmektedir. `EMPLOYEE` türünde bir nesneye odaklandığımızda, bu nesneni n, o belirli çalışanın sahip olduğu çeşitli atamaları temsil eden `ASSIGNMENT` t üründe bir nesne koleksiyonuna bağlı olduğunu buluruz. Sırasıyla, bu `ASSIGNMENT` türündeki her nesne, o atama ile ilişkili olan `JOB` türünde bir ne sneye bağlıdır. Böylece bir çalışanın tüm atamaları bulunabilir.

Şekil **9.13** Nesne yönelimli bir veritabanındaki nesneler arasındaki ilişkiler



Doclingo https://www.doclingo.ai

o çalışanın temsil eden nesneden bağlantıları takip ederek bulunur. Aynı şekilde, belirli bir işi yürütmüş olan tüm çalışanlar, o işi temsil eden nesneden bağlantıları takip ederek bulunabilir.

Bir nesne yönelimli veritabanındaki nesneler arasındaki bağlantılar genellikle DBMS tarafından sürdürülür, bu nedenle bu bağlantıların nasıl uygulandığı, uygulama yazılımı yazan programcının bir kaygısı değildir. Bunun yerine, veritabanına yeni bir nesne eklendiğinde, uygulama yazılımı yalnızca bağlanması gereken diğer nesneleri belirtir. DBMS, bu ilişkileri kaydetmek için gerekli olabilecek herhangi bir bağlantı sistemini oluşturur. Özellikle, bir DBMS, belirli bir çalışanın atamalarını temsil eden nesneleri, bağlı bir listeye benzer bir şekilde bağlayabilir.

Bir nesne yönelimli DBMS'in bir diğer görevi, kendisine emanet edilen nesneler için kalıcı depolama sağlamaktır—bu, açık görünebilecek bir gereklilik ancak nesnelerin normalde nasıl ele alındığından doğası gereği farklıdır. Normalde, bir nesne yönelimli program çalıştırıldığında, programın yürütülmesi sırasında oluşturulan nesneler, program sona erdiğinde atılır. Bu anlamda nesneler geçici olarak kabul edilir. Ancak, oluşturulan ve bir veritabanına eklenen nesnelerin, onları oluşturan program sona erdikten sonra kaydedilmesi gerekir. Bu tür nesnelere kalıcı denir. Böylece kalıcı nesneler oluşturmak, normdan önemli bir sapmadır.

Nesne yönelimli veritabanlarının savunucuları, nesne yönelimli yaklaşımın veritabanı tasarımında neden ilişkisel yaklaşımdan daha iyi olduğunu göstermek için birçok argüman sunmaktadır. Bunlardan biri, nesne yönelimli yaklaşımın tüm yazılım sisteminin (uygulama yazılımı, VYS ve veritabanı) aynı paradigma içinde tasarlanmasına olanak tanımasıdır. Bu, tarihsel olarak yaygın olan uygulama yazılımını bir ilişkisel veritabanını sorgulamak için geliştirmek amacıyla bir zorlayıcı programlama dili kullanma pratiği ile çelişmektedir. Böyle bir görevde zorlayıcı ve ilişkisel paradigmalar arasındaki çatışma doğaldır. Bu ayrım, çalışma seviyemizde ince bir ayrıntıdır ancak yıllar içinde birçok yazılım hatasının kaynağı olmuştur. Bizim seviyemizde bile, nesne yönelimli bir veritabanının nesne yönelimli bir uygulama programı ile birleştirilmesinin, sistem boyunca birbirleriyle iletişim kuran nesnelerin homojen bir görüntüsünü ürettiğini takdir edebiliriz. Öte yandan, bir ilişkisel veritabanı zorlayıcı bir uygulama programı ile birleştirildiğinde, iki doğası gereği farklı organizasyonun ortak bir arayüz bulmaya çalıştığı bir görüntü canlanmaktadır.

Nesne yönelimli veritabanlarının ilişkisel muadillerine göre sahip olduğu bir diğer avantajı takdir etmek için, bir ilişkisel veritabanında çalışan isimleri saklama sorununu düşünün. Eğer bir tam isim bir ilişkide tek bir özellik olarak saklanıyorsa, yalnızca soyadlarıyla ilgili sorgular zorlayıcı hale gelir. Ancak, isim bireysel özellikler olarak, örneğin ad, ikinci ad ve soyadı olarak saklanıyorsa, o zaman özellik sayısı sorunlu hale gelir çünkü tüm isimler belirli bir yapıya uymamaktadır—tek bir kültürle sınırlı bir nüfus bile olsa. Nesne yönelimli bir veritabanında, bu sorunlar çalışanın ismini tutan nesne içinde gizlenebilir. Bir çalışanın ismi, ilgili çalışanın ismini çeşitli formatlarda raporlama yeteneğine sahip akıllı bir nesne olarak saklanabilir. Böylece, bu nesnelerin dışından, yalnızca soyadlarıyla, tam isimlerle, kızlık soyadlarıyla veya takma adlarla başa çıkmak aynı derecede kolay olacaktır. Her bir perspektifle ilgili detaylar nesnelerin içinde kapsüllenmiş olacaktır.

Farklı veri formatlarının teknik detaylarını kapsülleme yeteneği, diğer durumlarda da avantajlıdır. İlişkisel bir veritabanında, bir ilişkideki nitelikler veritabanının genel tasarımının bir parçasıdır ve bu nedenle türler

Doclingo https://www.doclingo.ai

bu niteliklerle ilişkili olanlar DBMS'nin tamamını kaplar. (Geçici depolama için değişkenler uygun türde tanımlanmalıdır ve çeşitli türlerdeki verileri manip üle etmek için prosedürler tasarlanmalıdır.) Bu nedenle, bir ilişkisel veritaba nını yeni türlerin (ses ve video) niteliklerini içerecek şekilde genişletmek soru nlu olabilir. Özellikle, veritabanı tasarımı boyunca çeşitli prosedürlerin bu ye ni veri türlerini içerecek şekilde genişletilmesi gerekebilir. Ancak, nesne yöne limli bir tasarımda, bir çalışanın adını temsil eden bir nesneyi almak için kulla nılan aynı prosedür, bir sinema filmini temsil eden bir nesneyi almak için de kullanılabilir çünkü türler arasındaki farklılıklar ilgili nesneler içinde gizlenebi lir. Böylece nesne yönelimli yaklaşım, çoklu ortam veritabanlarının inşasıyla daha uyumlu görünmektedir - bu da zaten büyük bir avantaj sağladığı kanıtl anmaktadır.

Nesne yönelimli paradigmanın veritabanı tasarımına sunduğu bir diğer a vantaj, yalnızca verileri depolamak yerine akıllı nesneleri depolama potansiy elidir. Yani, bir nesne, içeriği ve ilişkileri ile ilgili mesajlara nasıl yanıt vermesi gerektiğini tanımlayan yöntemler içerebilir. Örneğin, Şekil 9.13'teki EMPLOY EE sınıfının her bir nesnesi, nesnedeki bilgileri raporlama ve güncelleme yön temlerinin yanı sıra, o çalışanın iş geçmişini raporlama ve belki de o çalışanın iş atamasını değiştirme yöntemi içerebilir. Benzer şekilde, JOB sınıfından her bir nesne, işin ayrıntılarını raporlama yöntemi ve belki de o belirli işi yapmış olan çalışanları raporlama yöntemi içerebilir. Böylece, bir çalışanın iş geçmişi ni almak için karmaşık bir prosedür oluşturmak zorunda kalmayız. Bunun ye rine, uygun çalışan nesnesine iş geçmişini raporlamasını isteyebiliriz. Bileşen lerinin sorgulara akıllıca yanıt verdiği veritabanları oluşturma yeteneği, daha geleneksel ilişkisel veritabanlarının sunduğundan daha heyecan verici bir ol asılıklar dizisi sunar.

Sorular ve Alıştırmalar

1. Bu bölümde tartışılan çalışan veritabanındaki ASSIGNMENT sınıfında n bir nesne örneğinde hangi yöntemler bulunur?
2. Kalıcı bir nesne nedir?
3. Bir depo envanteri ile ilgili nesne yönelimli bir veritabanında kullanıla bilecek bazı sınıfları ve bunların bazı iç özelliklerini tanımlayın.
4. Nesne yönelimli bir veritabanının ilişkisel bir veritabanına göre sahip olabileceği b ir avantajı tanımlayın.

9.4 Veritabanı Bütünlüğünü Koruma

Kişisel kullanım için ucuz veritabanı yönetim sistemleri nispeten basit sistemlerdir. Genellik le tek bir amaca sahiptirler—kullanıcıyı veritabanı uygulamasının teknik detaylarından koru mak. Bu sistemler tarafından yönetilen veritabanları nispeten küçüktür ve genellikle kaybı veya bozulması durumunda rahatsız edici, felaket niteliğinde olmayan bilgileri içerir.

Doclingo https://www.doclingo.ai

Bir sorun ortaya çıktığında, kullanıcı genellikle hatalı öğeleri doğrudan düzel tebilir veya veritabanını bir yedek kopyadan yeniden yükleyip o kopyayı günc el hale getirmek için gerekli değişiklikleri manuel olarak yapabilir. Bu süreç r ahatsız edici olabilir, ancak rahatsızlığın önlenmesi maliyeti genellikle rahatsı zlığın kendisinden daha büyüktür. Her durumda, rahatsızlık yalnızca birkaç ki şiyle sınırlıdır ve herhangi bir mali kayıp genellikle sınırlıdır.

Bununla birlikte, büyük, çok kullanıcılı, ticari veritabanı sistemleri söz kon usu olduğunda, stake'ler çok daha yüksektir. Yanlış veya kaybolan verilerin maliyeti muazzam olabilir ve yıkıcı sonuçlar doğurabilir. Bu ortamlarda, DBM S'nin ana rolü, veritabanının bütünlüğünü korumak için, bazı nedenlerden d olayı yalnızca kısmen tamamlanan işlemler veya yanlışlıkla etkileşime girebil ecek farklı işlemler gibi sorunlara karşı koruma sağlamaktır. Bu bölümde ele aldığımız DBMS'nin bu rolüdür.

Commit/Rollback Protokolü

Bir banka hesabından diğerine para transferi, bir havayolu rezervasyonunun iptali veya bir öğrencinin bir üniversite dersine kaydı gibi tek bir işlem, verita banı düzeyinde birden fazla adım içerebilir. Örneğin, banka hesapları arasın da para transferi, bir hesapta bakiyenin azaltılmasını ve diğerinde bakiyenin artırılmasını gerektirir.
Böyle adımlar arasında veritabanındaki bilgi tutarsız olabilir.
Gerçekten de, ilk hesap azaltıldıktan sonra ancak diğer hesap artırılmadan ö nce kısa bir süre boyunca fonlar kaybolur. Benzer şekilde, bir yolcunun bir uç uşta koltuğu yeniden atandığında, yolcunun hiç koltuğu olmadığı veya yolcu listesinin gerçekte olduğundan bir yolcu daha fazla göründüğü bir an olabilir .

Büyük veritabanlarının yoğun işlem yüklerine maruz kaldığı durumlarda, ra stgele bir anlık görüntünün veritabanını bir işlemin ortasında bulması oldukça olasıdır. Bu nedenle, bir işlemin yürütülmesi için bir talep veya bir ekipman arız ası, veritabanının tutarsız bir durumda olduğu bir zamanda meydana gelecekti r.

Öncelikle bir arıza sorununu ele alalım. DBMS'in amacı, böyle bir sorunu n veritabanını tutarsız bir durumda dondurmasını sağlamaktır. Bu genellikle, her işlemin faaliyetlerinin kaydını içeren bir günlüğü, manyetik disk gibi kalı cı bir depolama sisteminde tutarak gerçekleştirilir. Bir işlemin veritabanını d eğiştirmesine izin verilmeden önce, gerçekleştirilecek değişiklik önce günlüğ e kaydedilir. Böylece günlük, her işlemin eylemlerinin kalıcı bir kaydını içerir.

Bir işlemin tüm adımlarının günlüğe kaydedildiği noktaya commit point denir. DBMS'in, bu noktada, işlemi yeniden oluşturmak için gerekli bilgilere s ahip olduğu anlamına gelir, eğer bu gerekli olursa. Bu noktada DBMS, işlemi n faaliyetlerinin veritabanında yansıtılmasını garanti etme sorumluluğunu ka bul ettiği anlamında işlemi kabul eder. Bir ekipman arızası durumunda, DBM S, son yedeklemenin yapıldığı tarihten bu yana tamamlanan (commit edilen) işlemleri yeniden oluşturmak için günlüğündeki bilgileri kullanabilir.

Bir işlem, taahhüt noktasına ulaşmadan önce sorunlar ortaya çıkarsa, DBMS, tamamlanamayan kısmen yürütülmüş bir işlemle karşılaşabilir.

Doclingo https://www.doclingo.ai

Bu durumda, günlük, işlemin gerçekten gerçekleştirdiği faaliyetleri geri alma k için kullanılabilir.Örneğin, bir arıza durumunda,DBMS, arıza anında tamaml anmamış (taahhüt edilmemiş) olan işlemleri geri alarak kurtulabilir.

Ancak, işlemlerin geri alınması, yalnızca ekipman arızalarından kurtulma süreciyle sınırlı değildir.Genellikle bir DBMS'nin normal işleyişinin bir parçası dır. Örneğin, bir işlem, ayrıcalıklı bilgilere erişim girişimi nedeniyle tüm adıml arını tamamlamadan sonlandırılabilir veya birbirlerini kullanan veriler için be kleyen rekabet eden işlemler arasında bir çıkmaza girebilir. Bu durumlarda, DBMS, bir işlemi geri almak için günlüğü kullanabilir ve böylece tamamlanm amış işlemler nedeniyle hatalı bir veritabanından kaçınabilir.

DBMS tasarımının hassas doğasını vurgulamak için, geri alma sürecinde gizli sorunların bulunduğunu belirtmeliyiz. Bir işlemin geri alınması, diğer işl emler tarafından kullanılan veritabanı girişlerini etkileyebilir. Örneğin, geri al ınan işlem bir hesap bakiyesini güncellemiş olabilir ve başka bir işlem bu gün cellenmiş değere dayalı faaliyetlerini zaten gerçekleştirmiş olabilir. Bu, bu ek işlemlerin de geri alınması gerektiği anlamına gelebilir ki bu da başka işleml eri olumsuz etkileyebilir. Sonuç, zincirleme geri alma olarak bilinen sorundu r.

Kilitleme

Artık veritabanının başka bir işlemden kaynaklanan bir değişim durumunda çalıştırılan bir işlemi ele alıyoruz; bu durum, işlemler arasında istemeden etki leşime yol açabilir ve hatalı sonuçlar üretebilir. Örneğin, yanlış özet problem i olarak bilinen sorun, bir işlem bir hesaptan diğerine para transferi yaparke n, başka bir işlemin bankadaki toplam mevduatları hesaplamaya çalışması d urumunda ortaya çıkabilir. Bu, transfer adımlarının hangi sırayla gerçekleştir ildiğine bağlı olarak toplamın ya çok büyük ya da çok küçük olmasına neden olabilir. Diğer bir olasılık ise kaybolan güncelleme problemi olarak bilinir; bu , her biri aynı hesaptan bir kesinti yapan iki işlemle örneklendirilebilir. Eğer b ir işlem, diğerinin bakiyeyi okuduğu anda hesabın mevcut bakiyesini okursa ancak henüz yeni bakiyeyi hesaplamadıysa, her iki işlem de kesintilerini aynı başlangıç bakiyesine dayandıracaktır. Bu durumda, kesintilerden birinin etki si veritabanında yansıtılmayacaktır.

Bu tür sorunları çözmek için, bir DBMS, her yeni işlemi, öncekilerin tama mlanmasını bekleyerek sıraya alarak, tek seferde tam olarak çalıştırmaya zor layabilir. Ancak bir işlem genellikle kütle depolama işlemlerinin gerçekleştiril mesini beklerken çok fazla zaman harcar. İşlemlerin yürütülmesini iç içe geçi rerek, bir işlemin beklediği süre, daha önce almış olduğu verileri işlemek için başka bir işlem tarafından kullanılabilir. Bu nedenle, çoğu büyük veritabanı yönetim sistemi, işlemler arasında zaman paylaşımını koordine etmek için bi r zamanlayıcı içerir; bu, çoklu programlama işletim sisteminin süreçlerin iç iç e geçişini koordine etmesine benzer (Bölüm 3.3).

Bu tür anormalliklere karşı, yanlış özet problemi ve kaybolan güncelleme p roblemi gibi, bu zamanlayıcılar, bir veritabanında şu anda bazı işlemler tarafınd an kullanılan öğelerin işaretlendiği bir kilitleme protokolü içermektedir.

Doclingo https://www.doclingo.ai

Bu işaretler kilit olarak adlandırılır; işaretlenmiş öğelere kilitlenmiş denir. İki tür kilit yaygındır—paylaşılan kilitler ve özel kilitler.
Bu kilitler, bir işlemin ihtiyaç duyabileceği veriye erişim türlerine karşılık gelir—paylaşılan erişim ve özel erişim. Eğer bir işlem bir veri öğesini değiştirmeyecekse, o zaman paylaşılan erişim gerektirir, bu da diğer işlemlerin veriyi görüntülemesine izin verildiği anlamına gelir. Ancak, işlem öğeyi değiştirecekse, özel erişime sahip olmalıdır, bu da o veriye erişimi olan tek işlem olması gerektiği anlamına gelir.

Bir kilitleme protokolünde, her seferinde bir işlem bir veri öğesine erişim talep ettiğinde, aynı zamanda DBMS'ye ihtiyaç duyduğu erişim türünü de bildirmelidir. Eğer bir işlem, ya kilitlenmemiş ya da paylaşılan bir kilit ile kilitlenmiş bir öğeye paylaşılan erişim talep ederse, bu erişim verilir ve öğe paylaşılan bir kilit ile işaretlenir.
Ancak, talep edilen öğe zaten özel bir kilit ile işaretlenmişse, ek erişim reddedilir. Eğer bir işlem bir öğeye özel erişim talep ederse, bu talep yalnızca öğenin üzerinde hiçbir kilit yoksa kabul edilir. Bu şekilde, veriyi değiştirecek bir işlem, o veriyi diğer işlemlerden korumak için özel erişim elde ederken, eğer hiçbiri onu değiştirmeyecekse birkaç işlem bir öğeye paylaşılan erişim sağlayabilir. Elbette, bir işlem bir öğe ile işini bitirdiğinde, DBMS'yi bilgilendirir ve ilgili kilit kaldırılır.

Bir işlemin erişim talebinin reddedildiği durumu ele almak için çeşitli algoritmalar kullanılmaktadır. Bu algoritmalardan biri, işlemin yalnızca talep edilen öğe mevcut olana kadar bekletilmesidir. Ancak bu yaklaşım, iki işlemin aynı iki veri öğesine özel erişim gerektirmesi durumunda birbirlerinin ilerlemesini engelleyebileceği için deadlock'a yol açabilir; eğer her biri bir öğeye özel erişim elde eder ve ardından diğerini beklemeye insist ederse. Böyle deadlock'ları önlemek için bazı veritabanı yönetim sistemleri daha eski işlemlere öncelik verir. Yani, eğer daha eski bir işlem, daha genç bir işlem tarafından kilitlenmiş bir öğeye erişim gerektiriyorsa, daha genç işlem tüm veri öğelerini serbest bırakmaya zorlanır ve faaliyetleri geri alınır (log'a dayanarak).
Daha sonra, daha eski işleme talep ettiği öğeye erişim verilir ve daha genç işlem yeniden başlatılmaya zorlanır. Eğer daha genç bir işlem sürekli olarak öncelikli daha eski işlemlerden biri haline gelir. Wound-wait protokolü olarak bilinen bu protokol (eski işlemler genç işlemleri yaralar, genç işlemler eski işlemleri bekler), her işlemin nihayetinde görevini tamamlamasına izin verileceğini garanti eder.

Sorular ve Alıştırmalar

1. Bir işlemin taahhüt noktasına ulaştığı ile ulaşmadığı arasındaki farkı nedir?
2. Bir DBMS, geniş çaplı geri alma işlemlerine karşı nasıl korunabilir?
3. Bir hesabından 100 $ düşüren bir işlem ile aynı hesaptan 200 $ düşüren bir işlemin kontrolsüz bir şekilde iç içe geçmesinin, başlangıç bakiyesi 400 $ varsayıldığında 100 $, 200 $ ve 300 $'lık nihai bakiyeler üretebileceğini gösterin.

Doclingo https://www.doclingo.ai

4. a. Bir veritabanındaki bir öğeye paylaşımlı erişim talep eden bir işlemin olası sonuçlarını özetleyin.
   b. Bir veritabanındaki bir öğeye özel erişim talep eden bir işlemin olası sonuçlarını özetleyin.
5. Bir veritabanı sisteminde işlemler gerçekleştiren işlemler arasında deadlock'a yol açacak bir olaylar dizisini tanımlayın.
6. Cevabınızdaki 5. Sorudaki deadlock'un nasıl kırılabileceğini tanımlayın. Çözümünüz veritabanı yönetim sisteminin günlüğünü kullanmayı gerektirir mi? Cevabınızı açıklayın.

9.5 Geleneksel Dosya Yapıları

Bu bölümde, çok boyutlu veritabanı sistemleri üzerindeki çalışmamızdan sapparak geleneksel dosya yapılarını ele alıyoruz. Bu yapılar, mevcut veritabanı teknolojisinin evrildiği veri depolama ve alma sistemlerinin tarihsel başlangıcını temsil eder. Bu yapılar için geliştirilen birçok teknik (örneğin, indeksleme ve hashing) günümüzün büyük, karmaşık veritabanlarının inşasında önemli aralardır.

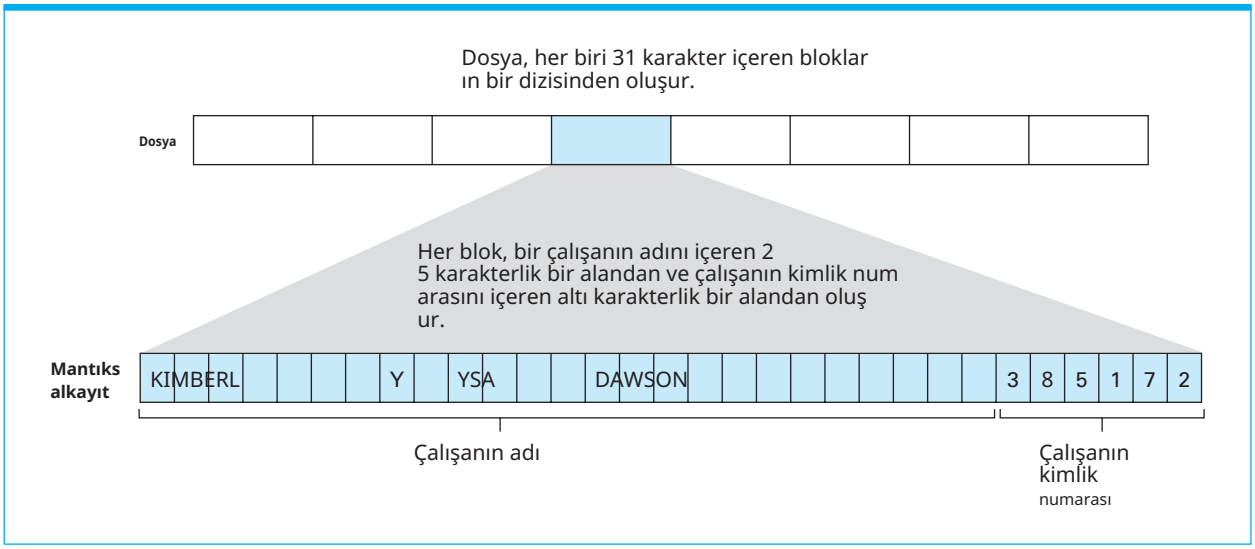
Sıralı Dosyalar

Sıralı bir dosya, dosyanın içindeki bilgilerin tek bir uzun satır halinde düzenlenmiş gibi, başlangıcından sonuna kadar seri bir şekilde erişilen bir dosyadır. Örnekler arasında ses dosyaları, video dosyaları, program içeren dosyalar ve metin belgeleri içeren dosyalar bulunmaktadır. Aslında, tipik bir kişisel bilgisayar kullanıcısı tarafından oluşturulan dosyaların çoğu sıralı dosyalardır. Örneğin, bir elektronik tablo kaydedildiğinde, bilgileri sıralı bir dosya olarak kodlanır ve depolanır; bu dosyadan elektronik tablo uygulama yazılımı elektronik tabloyu yeniden oluşturabilir.

Metin dosyaları, her mantıksal kaydın tek bir sembol olarak kodlandığı ardışık dosyalardırASCII veya Unicode kullanarak, genellikle çalışan kayıtları dosyası gibi daha karmaşık ardışık dosyalar oluşturmak için temel bir araç olarak hizmet eder. Her bir çalışanın bilgilerini bir metin dizesi olarak temsil etmek için tek tip bir format oluşturmak, bilgileri bu formata göre kodlamak ve ardından elde edilen çalışan kayıtlarını bir dizi metin dizesi olarak kaydetmek yeterlidir. Örneğin, her çalışan kaydını 31 karakterden oluşan bir dize olarak girmeyi kabul ederek basit bir çalışan dosyası oluşturulabilir; bu dize, çalışanın adını içeren 25 karakterlik bir alan (25 karakterlik alanı tamamlamak için yeterince boşlukla doldurulmuş) ve ardından çalışanın kimlik numarasını temsil eden 6 karakterlik bir alan içerir.
Son dosya, her 31 karakterlik bloğun tek bir çalışana ait bilgileri temsil ettiği uzun bir kodlanmış karakter dizisi olacaktır(Şekil 9.14). Bilgiler, 31 karakterlik bloklardan oluşan mantıksal kayıtlar açısından dosyadan alınacaktır. Bu blokların her birinde, bireysel alanlar blokların oluşturulduğu tek tip formata göre tanımlanacaktır.

Doclingo https://www.doclingo.ai

Şekil **9.14** Basit bir çalışan dosyasının metin dosyası olarak uygulanma yapısı



Sıralı bir dosyadaki veriler, dosyanın sıralı doğasının korunacak şekilde k ütle depolama alanında kaydedilmelidir. Eğer kütle depolama sistemi kendisi sıralıysa (manyetik bant veya CD durumunda olduğu gibi), bu basit bir iştir. Dosyayı yalnızca depolama ortamının sıralı özelliklerine göre depolama orta mında kaydetmemiz gerekir. Ardından dosyayı işlemek, yalnızca dosyanın iç eriğini bulunduğu sırayla okumak ve işlemekle ilgilidir. Bu, müziğin bir sıralı dosya olarak, sürekli bir spiral iz boyunca sektör sektör depolandığı ses CD'le rini çalma sürecinde izlenen süreçtir.

Ancak manyetik disk depolama durumunda, dosya farklı sektörler üzerin de dağılmış olacaktır ve bu sektörler çeşitli sıralarda geri alınabilir. Doğru sır alamayı korumak için, çoğu işletim sistemi (daha kesin olarak, dosya yönetici si) dosyanın depolandığı sektörlerin bir listesini tutar. Bu liste, dosyanın bulu nduğu diskin dizin sistemi parçası olarak dosyanın bulunduğu aynı diske kay dedilir. Bu liste aracılığıyla, işletim sistemi dosya sıralı olarak depolanmış gibi sektörleri doğru sırayla geri alabilir, oysa dosya aslında diskin çeşitli bölüml erine dağılmıştır.

Bir sıralı dosyayı işlerken, dosyanın sonuna ulaşıldığını tespit etme ihtiya cı doğar. Genel olarak, bir sıralı dosyanın sonuna dosya sonu (EOF) olarak atı fta bulunuruz. EOF'yu tanımlamanın çeşitli yolları vardır. Bunlardan biri, dosy anın sonuna bir özel kayıt, yani bir sentinel yerleştirmektir. Diğeri, işletim sis teminin dizin sistemindeki bilgileri kullanarak bir dosyanın EOF'sini tanımlam aktır. Yani, işletim sistemi dosyayı içeren sektörleri bildiğinden, dosyanın ner ede sona erdiğini de bilir.

Klasik bir örnek, küçük bir şirkette bordro işlemlerini içeren sıralı dosyala rdır. Burada, her biri bir çalışanın maaşı hakkında bilgi içeren mantıksal kayıt lar serisinden oluşan bir sıralı dosya hayal ediyoruz (isim, çalışan kimlik num arası, maaş ölçeği vb.) ve bu kayıtlardan düzenli olarak çekler basılmalıdır. H er çalışan kaydı alındıkça, o çalışanın

Doclingo https://www.doclingo.ai

ücret hesaplanır ve uygun çek üretilir. Böyle bir sıralı dosyanın işlenmesi faaliyeti, aşağıdaki ifade ile örneklendirilmiştir

while(DOSYA SONUNA ulaşılmadı) **do**
(bir sonraki kaydı dosyadan al ve işle)

Sıralı bir dosya içindeki mantıksal kayıtlar anahtar alanıdeğerleri ile tanı mlandığında, dosya genellikle kayıtların anahtarlar tarafından belirlenen sıra da (belki alfabetik veya sayısal) görünecek şekilde düzenlenir. Böyle bir düze nlemebilgilerin dosyada işlenmesi görevini basitleştirir. Örneğin, bordro işle minin her çalışan kaydının o çalışanın zaman çizelgesindeki bilgileri yansıtac ak şekilde güncellenmesini gerektirdiğini varsayalım. Eğer zaman çizelgesi k ayıtlarını içeren dosya ve çalışan kayıtlarını içeren dosya aynı anahtarlara gör e aynı sıradaysa, o zaman bu güncelleme işlemi her iki dosyaya sıralı olarak erişerek gerçekleştirilebilir—bir dosyadan alınan zaman çizelgesini kullanara k diğer dosyadaki karşılık gelen kaydı güncellemek. Bu, dosyalar karşılık gele n sırada olmadığında gerekli olacak tekrar eden arama sürecine göre önemli bir iyileştirmedir. Böylece klasik sıralı dosyaların güncellenmesi genellikle bi rden fazla adımda gerçekleştirilir.Öncelikle, yeni bilgiler (örneğin zaman çizel gelerinin toplanması) bir işlem dosyası olarak bilinen sıralı bir dosyaya kayde dilir ve bu işlem dosyası güncellenecek dosyanın sırasına uyması için sıralanı r,bu dosyaya ana dosya denir. Ardından, ana dosyadaki kayıtlar her iki dosya dan kayıtları sıralı olarak alarak güncellenir.

Bu güncelleme sürecinin hafif bir varyasyonu, iki ardışık dosyanın birleştiril mesi sürecidir; bu işlem, iki orijinalin kayıtlarını içeren yeni bir dosya oluşturur. Girdi dosyalarındaki kayıtların, ortak bir anahtar alanına göre artan sırada d üzenlendiği varsayılmaktadır ve dosyaların, anahtarlarının da artan sırada ol duğu bir çıktı dosyası üretecek şekilde birleştirileceği de varsayılmaktadır. Kl asik birleştirme algoritması Şekil 9.15'te özetlenmiştir. Temel tema, çıktı dosy asını iki girdi dosyası sırasıyla taranırken oluşturmaktır (Şekil 9.16).

İndeksli Dosyalar

Sıralı dosyalar, dosyanın girişlerinin saklandığı sırayla işlenecek verileri depol amak için idealdir. Ancak, dosya içindeki kayıtların öngörülemeyen bir sırada alınması gerektiğinde, bu tür dosyalar verimsizdir. Böyle durumlarda

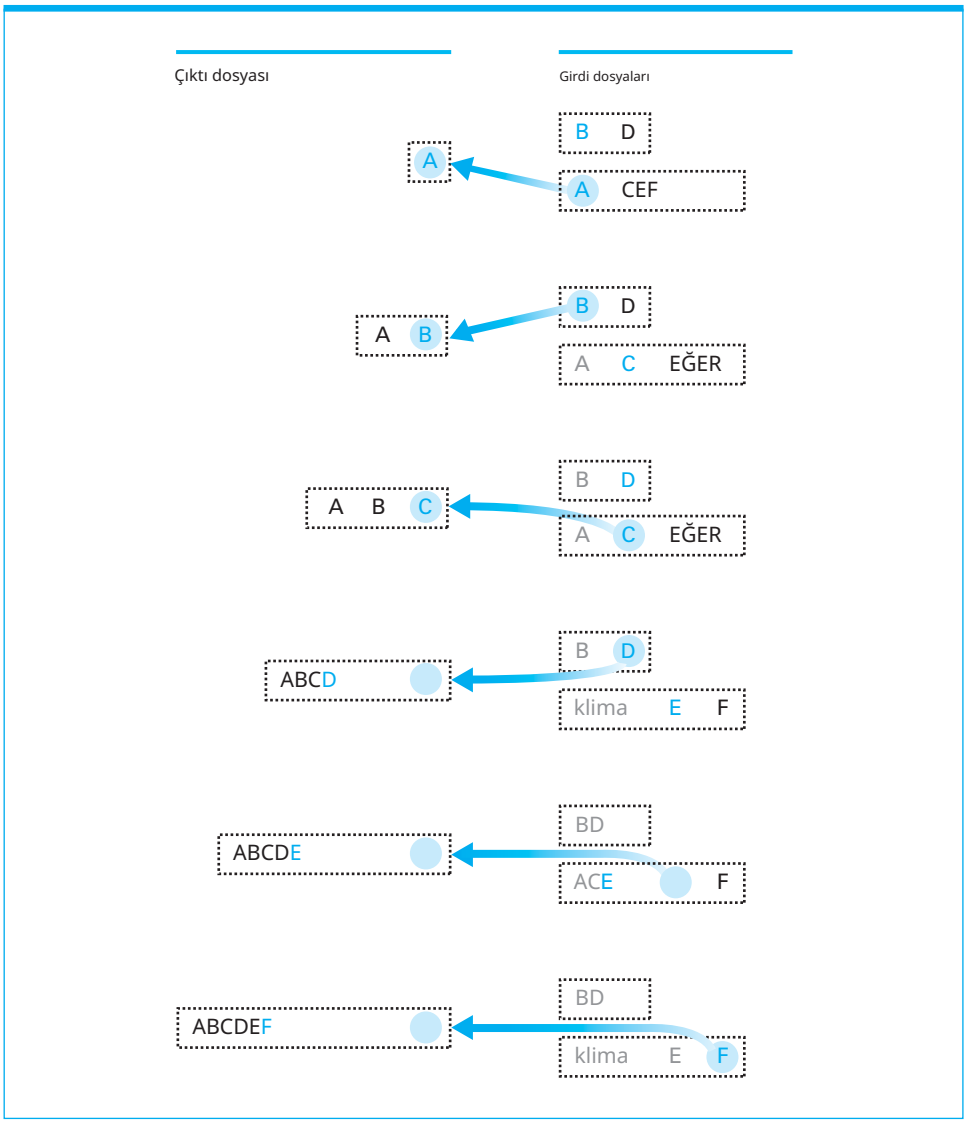
Şekil **9.15** İki ardışık dosyanın birleştirilmesi için bir prosedür

```
prosedür MergeFiles (InputFileA, InputFileB, OutputFile)

if (her iki girdi dosyası EOF'ta ise) o zaman (Durdur, OutputFile boş)
if (InputFileA EOF'ta değilse) o zaman (İlk kaydını mevcut kaydı olarak ilan et)
if (InputFileB EOF'ta değilse) o zaman (İlk kaydını mevcut kaydı olarak ilan et)
while (hiçbir girdi dosyası EOF'ta değilse) yap
   (Mevcut kaydı "daha küçük" anahtar alan değeri ile OutputFile'a koyun;
    eğer (bu mevcut kayıt, ilgili girdi dosyasındaki son kayıt ise)
        o zaman (bu girdi dosyasının EOF'da olduğunu beyan edin)aksi takd
        irde (o girdi dosyasındaki bir sonraki kaydı dosyanın mevcut kaydı olarak beyan edin)
    )
EOF'da olmayan girdi dosyasındaki mevcut kayıt ile başlayarak,
        kalan kayıtları OutputFile'a kopyalayın.
```

Doclingo https://www.doclingo.ai

Şekil **9.16**Birleştirme algoritmasının uygulanması (Harfler, tüm kayıtları temsil etmek için kullanılır. B elirli harf, kaydın anahtar alanının değerini gösterir.)



gereken, istenen mantıksal kaydın konumunu hızlı bir şekilde tanımlamanın bir yolu dur.Popüler bir çözüm, dosya için bir indeks kullanmaktır; bu, bir kitapta konuları bul mak içinbir kitapta kullanılan bir indeksle çok benzer. Böyle bir dosya sistemi indeksli dosya olarak adlandırılır.

Bir dosya için indeks, dosyada saklanan anahtarların bir listesini içerir ve her anahtarın saklandığı kaydın nerede bulunduğunu belirten girişleri içerir. Böylecebelirli bir kaydı bulmak için, indekste tanımlayıcı anahtarı bulur ve ar dındanbu anahtarla ilişkili konumda saklanan bilgi bloğunu alır.

Bir dosyanın indeksi genellikle indeksli dosya ile aynı kütle depolama aletinde ayrı bir dosya olarak saklanır. İndeks genellikle ana belleğe aktarılır

Doclingo https://www.doclingo.ai

dosya işleme başlamadan önce, kayıtlara erişim gerektiğinde kolayca erişilebilir olması için (Şekil 9.17).

Bir indeksli dosyanın klasik bir örneği, çalışan kayıtlarını sürdürme bağlamında ortaya çıkar. Burada, bireysel bir kaydı alırken uzun aramaları önlemek için bir indeks kullanılabilir. Özellikle, eğer çalışan kayıtları dosyası çalışan kimlik numaralarına göre indekslenmişse, o zaman bir çalışanın kaydı, çalışanın kimlik numarası bilindiğinde hızlı bir şekilde alınabilir. Başka bir örnek, bireysel kayıtların nispeten hızlı erişimini sağlamak için bir indeksin kullanıldığı ses CD'lerinde bulunur.

Yıllar içinde temel indeks kavramının sayısız varyasyonu kullanılmıştır. Bir varyasyon, indeksi hiyerarşik bir şekilde inşa eder, böylece indeks katmanlı veya ağaç yapısına sahip olur. Öne çıkan bir örnek, dosya depolamasını düzenlemek için çoğu işletim sistemi tarafından kullanılan hiyerarşik dizin sistemidir. Böyle bir durumda, dizinler veya klasörler, her biri alt indekslerine bağlantılar içeren indekslerin rolünü oynar. Bu perspektiften bakıldığında, tüm dosya sistemi yalnızca büyük bir indeksli dosyadır.

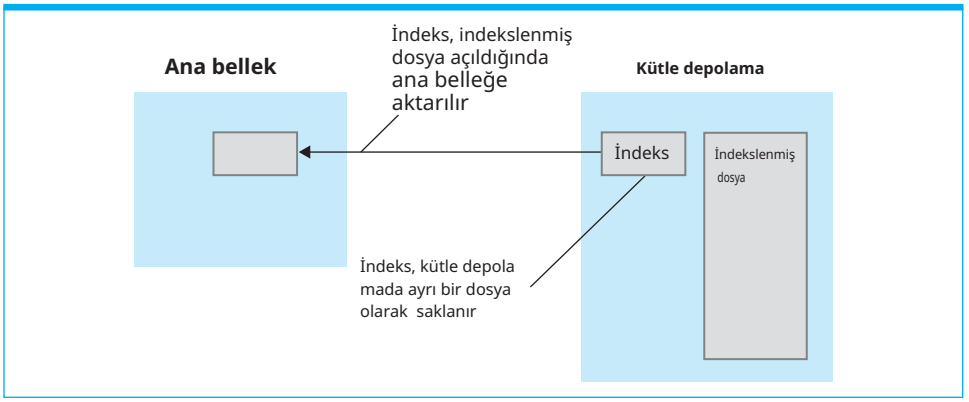
Hash Dosyaları

İndeksleme, bir veri depolama yapısı içindeki kayıtlara nispeten hızlı erişim sağlasa da, bunu indeks bakımının maliyetiyle yapar. **Hashing**, böyle bir yük olmadan benzer bir erişim sağlayan bir tekniktir. İndeksli bir sistemde olduğu gibi, hashing, bir kaydın bir anahtar değeri aracılığıyla bulunmasına olanak tanır. Ancak, anahtarı bir indekste aramak yerine, hashing, kaydın konumunu doğrudan anahtardan belirler.

Bir hash sistemi şu şekilde özetlenebilir: Veri depolama alanı, bucket olarak adlandırılan birkaç bölüme ayrılır; her biri birkaç kayıt tutma kapasitesine sahiptir. Kayıtlar, anahtar değerlerini bucket numaralarına dönüştüren bir algoritmaya göre bucket'lar arasında dağıtılır. (Anahtar değerlerinden bucket numaralarına bu dönüşüme hash fonksiyonu denir.) Her kayıt, bu süreçle belirlenen bucket'ta saklanır. Bu nedenle, depolama yapısına yerleştirilen bir kayıt, önce kaydın tanımlayıcı anahtarına hash fonksiyonunu uygulayarak uygun bucket'ı belirleyerek, ardından o bucket'ın içeriğini alarak ve son olarak alınan veriler arasında istenen kaydı arayarak geri alınabilir.

Hashing, yalnızca kütle depolamadan veri almak için bir araç olarak değil,büyük veri bloklarından ana bellekte depolanan öğeleri almak için de bir araç olarak kullanılır. Hashing, kütle depolamadaki bir depolama yapısına uygulandığında,sonuç bir hash dosyası olarak adlandırılır. Ana bellek içindeki bir depolama yapısına uygulandığında,sonuç genellikle bir hash tablosu olarak adlandırılır.

Şekil **9.17**İndekslenmiş bir dosyanın açılması



Doclingo https://www.doclingo.ai

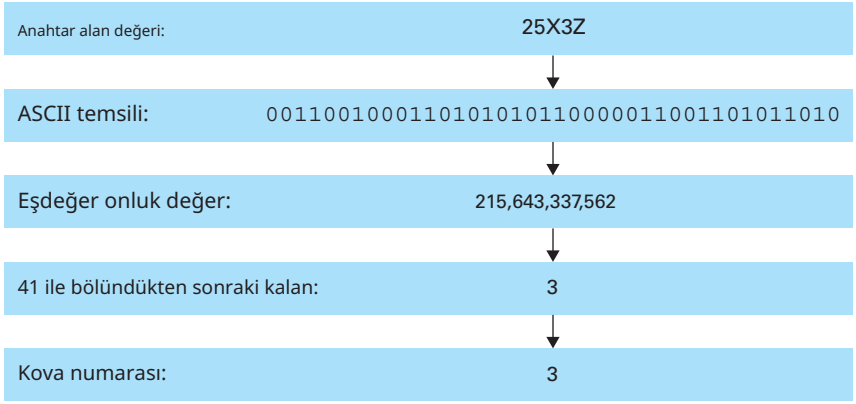
Hashing ile Kimlik Doğrulama

Hashleme, verimli veri depolama sistemleri oluşturmanın çok ötesinde bir anlam taşır. Örneğin, hashleme, İnternet üzerinden iletilen mesajları doğrulamak için bir yöntem olarak kullanılabilir. Temel tema, mesajı gizli bir şekilde hashlemektir. Bu değer daha sonra mesajla birlikte iletilir. Mesajı doğrulamak için alıcı, alın an mesajı (aynı gizli şekilde) hashler ve üretilen değerin orijinal değerle uyuşu p uyuşmadığını onaylar. (Değiştirilmiş bir mesajın aynı değere hashlenme ola sılığının çok küçük olduğu varsayılmaktadır.) Elde edilen değer orijinal değerl e uyuşmuyorsa, mesajın bozulduğu ortaya çıkar. İlgilenenler, kimlik doğrula ma uygulamalarında yaygın olarak kullanılan bir hash fonksiyonu olan MD5 h akkında bilgi için İnternet'te arama yapmayı düşünebilirler.

Hata tespit tekniklerini kimlik doğrulama için hashlemenin bir uygulamas ı olarak düşünmek aydınlatıcıdır. Örneğin, parite bitlerinin kullanımı, bir bit d eseninin hashlenerek ya 0 ya da 1 üretildiği temelde bir hashleme sistemidir. Bu değer daha sonra orijinal desenle birlikte iletilir. Nihayetinde alınan desen bu aynı değere hashlenmiyorsa, desen bozulmuş kabul edilir.

Hashing tekniğini, her kaydın bir şirketteki tek bir çalışan hakkında bilgi i çerdiği klasik çalışan dosyasına uygulayalım. Öncelikle, kova rolünü oynayac ak birkaç mevcut kütle depolama alanı belirliyoruz. Kova sayısı ve her bir kov aların boyutu, daha sonra dikkate alacağımız tasarım kararlarıdır. Şu an için, 41 kova oluşturduğumuzu varsayalım, bunları kova numarası 0, kova numar ası 1, kova numarası 40 olarak adlandırıyoruz. (Neden 41 kova seçtiğimizi, 40 yerine, kısa süre içinde açıklayacağız.) Bir çalışanın kimlik numarasının, ça lışanın kaydını tanımlamak için anahtar olarak kullanılacağını varsayalım. O halde, bir sonraki görevimiz, bu anahtarları kova numaralarına dönüştüre n bir hash fonksiyonu geliştirmektir. Çalışan kimlik "numaraları" 25X3Z veya J 2X35 biçiminde olabilir ve bu nedenle sayısal değildir, ancak bit desenleri ola rak saklanır ve bit desenlerini sayılar olarak yorumlayabiliriz. Bu sayısal yoru mlamayı kullanarak, herhangi bir anahtarı mevcut kova sayısına bölebilir ve kalanı kaydedebiliriz; bu durumda kalan, 0 ile 40 arasındaki bir tam sayı olac aktır.Bu nedenle, bu bölme işleminin kalanı, 41 kovadan birini tanımlamak içi n kullanılabilir (Şekil 9.18).

Şekil **9.18** Anahtar alan değerini 25X3Z bir 41 kovasından birine hashleme



Doclingo https://www.doclingo.ai

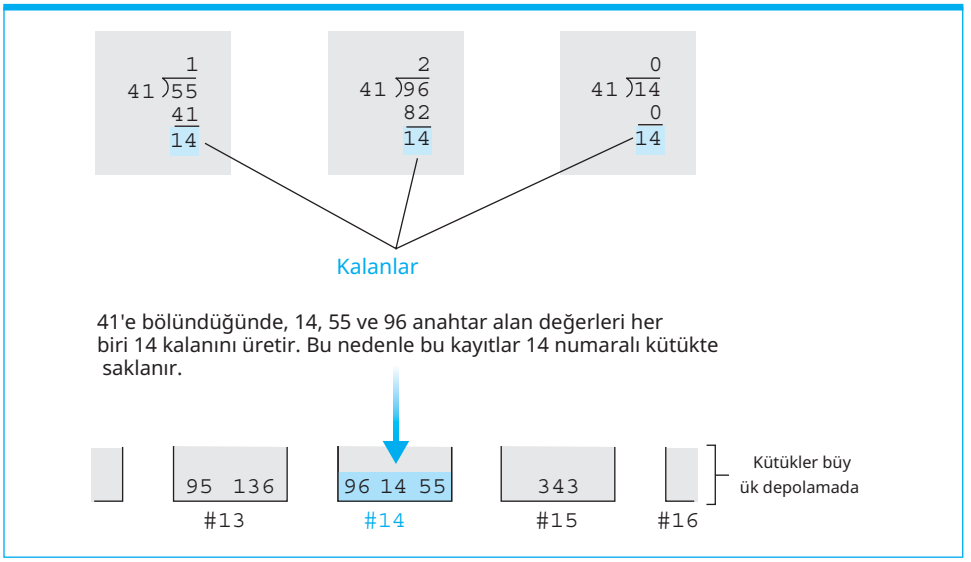
Bu hash fonksiyonunu kullanarak, her kaydı ayrı ayrı dikkate alarak dosyayı oluşturmak için, anahtarına 41'e bölme hash fonksiyonumuzu uygulayarak bir kova numarası elde ediyoruz ve ardından kaydı o kovada saklıyoruz (Şekil 9.19). Daha sonra, bir kaydı geri almak gerektiğinde, yalnızca kaydın anahtarına hash fonksiyonu muzu uygulamamız yeterlidir; bu, uygun kovayı tanımlamak ve ardından o kovada ilgili kay dı aramak için gereklidir.

Bu noktada, depolama alanını 41 kovaya ayırma kararımızı yeniden gözd en geçirelim. Öncelikle, verimli bir hash sistemi elde etmek için, saklanan kay ıtların kovalar arasında eşit şekilde dağıtılması gerektiğini unutmayın. Eğer o rantısız sayıda anahtar aynı kovaya hash olursa (bu olaya kümelenme denir), o zaman orantısız sayıda kayıt tek bir kovada saklanacaktır. Sonuç olarak, o kovadan bir kaydı almak zaman alıcı bir arama gerektirebilir ve bu da hashin g ile elde edilen faydaların kaybolmasına neden olabilir.

Şimdi, depolama alanını 41 yerine 40 kovaya ayırmayı seçseydik, hash fo nksiyonumuz anahtarları 40 değeri ile bölmeyi içerecekti. Ancak, bir pay ve b ir bölenin her ikisi de ortak bir faktöre sahipse, o faktör kalanda da mevcut o lacaktır. Özellikle, hash dosyamızda saklanan girişlerin anahtarları 5'in katlar ı olsaydı (bu aynı zamanda 40'ın bir bölenidir), o zaman 40 ile bölündüğünde 5 faktörü kalanda görünecek ve girişler 0, 5, 10, 15, 20, 25, 30 ve 35 kalanları ile ilişkili olan o kovada kümelenecektir. 2, 4, 8, 10 ve 20'nin katları olan anah tarlar için benzer durumlar meydana gelecektir, çünkü bunlar da 40'ın faktör leridir. Sonuç olarak, depolama alanını 41 kovaya ayırmayı seçiyoruz çünkü 4 1'in bir asal sayı olması, ortak faktörlerin olasılığını ortadan kaldırmış ve dola yısıyla kümelenme şansını azaltmıştır.

Ne yazık ki, kümeleme olasılığı asla tamamen ortadan kaldırılamaz. İyi tasarla nmış bir hash fonksiyonu ile bile, iki anahtarın aynı değere hashlenmesi, dosya ol uşturma sürecinin başlarında bir çarpışma olarak adlandırılan bir olgu ile yüksek olasılıkla gerçekleşir. Nedenini anlamak için, aşağıdaki senaryoyu düşünün.

Şekil 9.19 Bir hashing sisteminin temelleri



Doclingo https://www.doclingo.ai

Diyelim ki, kayıtları 41 kütük arasında rastgele dağıtan bir hash fonksiyo nu bulduk, depolama sistemimiz boş ve yeni kayıtları birer birer ekleyeceğiz. İlk kaydı eklediğimizde, boş bir kütüğe yerleştirilecektir. Ancak, bir sonraki k aydı eklediğimizde, 41 kütükten yalnızca 40'ı hala boş olduğundan, ikinci kay dın boş bir kütüğe yerleştirilme olasılığı yalnızca 40/41'dir. İkinci kaydın boş bir kütüğe yerleştirildiğini varsayarsak, üçüncü kayıt yalnızca 39 boş kütük b ulur ve dolayısıyla bunlardan birine yerleştirilme olasılığı 39/41'dir. Bu süreci devam ettirerek, ilk yedi kaydın boş kütüklere yerleştirildiğini bulursak, sekizi nci kaydın kalan boş kütüklerden birine yerleştirilme olasılığı yalnızca 34/41' dir.

Bu analiz, ilk sekiz kaydın boş kutulara yerleştirilme olasılığını hesaplama mıza olanak tanır - bu, her kaydın boş bir kutuya yerleştirilme olasılıklarının ç arpımıdır, önceki girişlerin bu şekilde yerleştirildiği varsayılarak. Bu olasılık

```
(41/41)(40/41)(39/41)(38/41) . . . (34/41) = .482
```

Önemli olan, sonucun yarımın altında olmasıdır. Yani, 41 kutu arasında kayıt dağıt ımında, sekizinci kayıt depolandığında bir çarpışmanın meydana gelmesi olasılığı, olmamasından daha yüksektir.

Çarpışmaların yüksek olasılığı, ne kadar iyi seçilmiş bir hash fonksiyonu olursa ol sun, herhangi bir hash sisteminin kümelenmeyi göz önünde bulundurarak tasarlan ması gerektiğini göstermektedir. Özellikle, bir kutunun dolup taşması mümkündür. Bu soruna bir yaklaşım, kutuların boyutunun genişlemesine izin vermektir. Diğer bir yaklaşım, kutuların bu amaç için ayrılmış bir taşma alanına dökülmesine izin vermekt ir. Her durumda, kümelenme ve taşan kutuların meydana gelmesi, bir hash dosyasın ın performansını önemli ölçüde düşürebilir.

Araştırmalar, genel bir kural olarak, hash dosyalarının, kayıt sayısının dosyanın topl am kayıt kapasitesine oranı (yük faktörü olarak bilinen bir oran) %50'nin altında kaldığı sürece iyi performans gösterdiğini göstermiştir. Ancak, yük faktörü %75'in üzerine çıkm aya başladığında, sistemin performansı genellikle düşer (kümelenme çirkin yüzünü gös terir, bazı kutuların dolmasına ve muhtemelen taşmasına neden olur). Bu nedenle, bir h ash depolama sistemi genellikle yük faktörü %75 değerine yaklaştığında daha büyük bir kapasite ile yeniden yapılandırılır. Bir hash sisteminin uygulanmasıyla elde edilen kayıt alma verimliliğinin maliyetsiz olmadığını sonucuna varıyoruz.

Sorular ve Alıştırmalar

1. Şekil 9.15'te sunulan birleştirme algoritmasını takip edin, birgiriş dosyasının B ve E'ye eşit anahtar alan değerlerine sahip kayıtlar içerdiğinivarsayarken diğeri nin A, C, D ve F'yi içerdiğini varsayın.
2. Birleştirme algoritması, birleştirme sıralama adı verilen popüler bir sıralamaalgoritm asının kalbidir. Bu algoritmayı keşfedebilir misiniz? (*İpucu:* Herhangi bir boş olmayan dosyabir giriş dosyası koleksiyonu olarak düşünülebilir.)
3. Sıralı olmak, bir dosyanın fiziksel mi yoksa kavramsal mı bir özelliğidir?
4. İndeksli bir dosyadan bir kaydı alırken gereken adımlar nelerdir?
5. Kötü seçilmiş bir hash fonksiyonunun, bir hash depolamasisteminin neredeyse sadece bir sı ralı dosya haline gelmesine nasıl yol açabileceğini açıklayın.

Doclingo https://www.doclingo.ai

6. Bir hash depolama sistemi, metinde sunulan bölme hash fonksiyonu kullanılarak ancak altı depolama kovası ile inşa edilirse ne olur? Aşağı daki anahtar değerleri için, o anahtara sahip kaydın yerleştirildiği kov ayı belirleyin. Ne yanlış gidiyor ve neden?

a. 24	**b.** 30	**c.** 3	**d.** 18	**Ve.** 15
f. 21	**g.** 9	**h.** 39	**Ben.** 27	**j.** 0

7. Bir grup insanın toplanması için kaç kişi gereklidir ki, grubun iki üyesi nin yılın aynı gününde doğum günleri olsun? Bu sorun, bu bölümdek i materyalle nasıl ilişkilidir?

9.6 Veri Madenciliği

Hızla genişleyen ve veritabanı teknolojisi ile yakından ilişkili bir konu,veri made nciliğidir; bu, veri koleksiyonlarındaki kalıpları keşfetmek için teknikler içerir. Ve ri madenciliği,çok sayıda alanda önemli bir araç haline gelmiştir; bunlar arasın da pazarlama, envanter yönetimi,kalite kontrolü, kredi riski yönetimi, dolandırı cılık tespiti ve yatırım analizi bulunmaktadır. Veri madenciliği teknikleri,DNA m oleküllerinde kodlanmış belirli genlerin işlevlerini tanımlamak ve organizmaları n özelliklerinikarakterize etmek gibi, pek olası görünmeyen ortamlarda bile uyg ulamalara sahiptir.

Veri madenciliği faaliyetleri, geleneksel veritabanı sorgulamalarından farklıdır çünkü v eri madenciliği, saklanan gerçeklerin geri alınmasını talep eden geleneksel veritabanı sorg ularının aksine, daha önce bilinmeyen kalıpları tanımlamayı amaçlar. Ayrıca, veri madencili ği, sık güncellemeye tabi olan "çevrimiçi" operasyonel veritabanları yerine, veri ambarı olar ak adlandırılan statik veri koleksiyonları üzerinde uygulanır. Bu ambarlar genellikle veritab anlarının veya veritabanı koleksiyonlarının "anlık görüntüleri"dir. Statik bir sistemde kalıpla rı bulmak, dinamik bir sistemde bulmaktan daha kolay olduğu için, gerçek operasyonel ver itabanlarının yerine kullanılırlar.

Ayrıca, veri madenciliği konusunun yalnızca bilgisayar bilimi alanıyla sınırlı ol madığını, istatistiklere kadar uzanan kolları olduğunu belirtmeliyiz. Aslında, birçok kişi veri madenciliğinin büyük, çeşitli veri koleksiyonları üzerinde istatistiksel anali z yapma girişimlerinden kaynaklandığını savunur; bu nedenle, istatistiklerin bir uy gulamasıdır, bilgisayar bilimi alanı değil.

Veri madenciliğinin iki yaygın biçimi, sınıf tanımı ve sınıf ayrımıdır. Sınıf t anımı, belirli bir veri öğesi grubunu karakterize eden özellikleri tanımlamakla ilgilenirken, sınıf ayrımı iki grubu ayıran özellikleri tanımlamakla ilgilenir. Ör neğin, sınıf tanımı teknikleri, küçük ekonomik araçlar satın alan kişilerin özell iklerini tanımlamak için kullanılırken, sınıf ayrımı teknikleri, ikinci el araçlar içi n alışveriş yapan müşterileri yeni araçlar için alışveriş yapanlardan ayıran öz ellikleri bulmak için kullanılır.

Veri madenciliğinin bir başka biçimi, sınıfları keşfetmeyi amaçlayan küm e analizidir. Bu, zaten tanımlanmış sınıflar içindeki üyelerin özelliklerini keşfe tmeyi amaçlayan sınıf tanımından farklıdır. Daha kesin olarak,küme analizi, g rupların keşfine yol açan veri öğelerinin özelliklerini bulmaya çalışır. Örneğin , belirli bir sinema filmini izleyen kişilerin yaşları hakkında bilgi analiz ederke n, küme analizi müşterinin

Doclingo https://www.doclingo.ai

Biyoinformatik

Veritabanı teknolojisi ve veri madenciliği tekniklerindeki ilerlemeler, biyologlarınş ablonların tanımlanması ve organik bileşenlerin sınıflandırılmasıyla ilgiliaraştırma alanlarında kullanılabilir araçların repertuarını genişletmektedir. Sonuç,biyoloji içi nde bioinformatik adı verilen yeni bir alan ortaya çıkmaktadır. DNA'yıçözme çabal arından doğan bioinformatik, artık proteinleri kataloglama veprotein etkileşimleri nin dizilerini anlama (biyokimyasal yollar olarak adlandırılır)gibi görevleri kapsam aktadır. Genellikle biyolojinin bir parçası olarak kabuledilse de, bioinformatik bilgi sayar biliminin diğer alanları etkileme ve hattaonlara yerleşme örneğidir.

temel iki yaş grubuna ayrılmaktadır - 4 ila 10 yaş grubu ve 25 ila 40 yaş grubu. (Belki de sinema filmi çocukları ve ebeveynlerini çekmiştir?)

Veri madenciliğinin bir başka biçimi, veri grupları arasındaki bağlantıları aramayı içeren ilişki analizidir.İlişki analizi, patates cipsi satın alan müşteriler in aynı zamanda bira ve soda da satın aldığını veya geleneksel hafta içi çalış ma saatlerinde alışveriş yapan kişilerin emeklilik yardımları aldığını ortaya çı karabilir.

Aykırı değer analizi, başka bir veri madenciliği biçimidir. Normla uyumlu olmayan veri öğelerini tanımlamaya çalışır. Aykırı değer analizi, veri toplama hatalarını tanımlamak, bir müşterinin normal satın alma kalıplarından ani sapmaları tespit ederek kredi kartı hırsızlığını belirlemek ve belki de alışılmadık davranışları tanıyarak potansiyel teröristleri belirlemek için kullanılabilir.

Son olarak, ardışık desen analizi olarak adlandırılan veri madenciliği biçimi, zaman içinde davranış kalıplarını tanımlamaya çalışır. Örneğin, ardışık desen analizi, sermaye piyasaları gibi ekonomik sistemlerde veya iklim koşulları gibi çevresel sistemlerde trendleri ortaya çıkarabilir.

Son örneğimizde belirtildiği gibi, veri madenciliğinden elde edilen sonuçlar, gelecek davranışları tahmin etmek için kullanılabilir. Bir varlık, bir sınıfı tanımlayan özelliklere sahipse, o varlık muhtemelen o sınıfın üyeleri gibi davranacaktır. Ancak, birçok veri madenciliği projesi, verilerin daha iyi anlaşılmasına yönelik olup, DNA'nın sırlarını çözmek için veri madenciliğinin kullanımında görüldüğü gibi. Her durumda, data madenciliği uygulamalarının kapsamı potansiyel olarak muazzamdır ve bu nedenle veri madenciliği, önümüzdeki yıllarda aktif bir araştırma alanı olmayı vaat etmektedir.

Veritabanı teknolojisi ve veri madenciliğinin yakın akraba olduğunu ve bu ned enlebirindeki araştırmaların diğerinde sonuçlar doğuracağını unutmayın. Veritab anı teknikleri,data warehouse'ların verileri veri küpleri (verilerin çoklu perspektifl erden görüldüğü—küp terimi çoklu boyutların imajını tasvir etmek için kullanılır) ş eklinde sunmakapasitesine sahip olmasını sağlamak için yaygın olarak kullanılma ktadır. Bununlabirlikte, veri madenciliği araştırmacıları veri küplerini uygulama te knikleriniiyileştirdikçe, bu sonuçlar veritabanı tasarımı alanında fayda sağlayacaktı r.

Sonuç olarak, başarılı veri madenciliğinin bir veri koleksiyonu içindeki şablonların tanımlanmasından çok daha fazlasını kapsadığını kabul etmeliyiz. Bu şablonların önemli mi yoksa sadece tesadüf mü olduğunu belirlemek için akıllı bir yargı uygulanmalıdır. Belirli bir bakkalın yüksek sayıda kazanan piyango bileti satmış olması, piyango bileti almayı planlayan biri için önemli olarak değerlendirilmemelidir, ancak

Doclingo https://www.doclingo.ai

Müşterilerin atıştırmalık gıda satın alırken dondurulmuş akşam yemekleri de satın alma eğiliminde olduklarını keşfetmek, bir bakkal yöneticisi için anlamlı bir bilgi oluşturabilir. Benzer şekilde, veri madenciliği, veri ambarında temsil edilen bireylerin hakları, çıkarılan sonuçların doğruluğu ve kullanımı ile veri madenciliğinin uygunluğu gibi çok sayıda etik sorunu kapsamaktadır.

Sorular ve Alıştırmalar

1. Neden veri madenciliği "çevrimiçi" veritabanlarında yapılmamaktadır?
2. Metinde tanımlanan her veri madenciliği türü tarafından bulunabilecek bir örüntüye ek bir örnek verin.
3. Bir veri küpünün satış verilerini madencilik yaparken hangi farklı bakış açılarını sağlayabileceğini belirleyin.
4. Veri madenciliği, geleneksel veritabanı sorgularından nasıl farklıdır?

9.7 Veritabanı Teknolojisinin Sosyal Etkisi

Veritabanı teknolojisinin gelişimi ile bir zamanlar gizli kayıtlarda gömülü olan bilgiler erişilebilir hale gelmiştir. Birçok durumda, otomatik kütüphane sistemleri bir kullanıcının okuma alışkanlıklarını kolayca erişilebilir hale getirirken, perakendeciler müşterilerinin satın alma kayıtlarını tutmakta ve İnternet arama motorları müşterilerinin taleplerini kaydetmektedir. Bu bilgi, pazarlama firmalarına, kolluk kuvvetlerine, siyasi partilere, işverenlere ve özel bireylere potansiyel olarak sunulmaktadır.

Bu, veritabanı uygulamalarının tamamını etkileyen potansiyel sorunların temsilcisidir. Teknoloji, muazzam miktarda veri toplamak ve farklı veri koleksiyonlarını birleştirip karşılaştırmak için kolaylık sağlamıştır; bu sayede aksi takdirde yığın içinde gömülü kalacak ilişkiler elde edilebilir. Hem olumlu hem de olumsuz sonuçlar muazzamdır. Bu sonuçlar yalnızca akademik tartışmaların konuları değildir - bunlar gerçeklerdir.

Bazı durumlarda veri toplama süreci belirgin bir şekilde ortaya çıkar; diğerlerinde ise ince bir şekilde gerçekleşir. İlk durumun örnekleri, birinin açıkça bilgi vermesi istendiğinde ortaya çıkar. Bu, anketler veya yarışma kayıt formları gibi gönüllü bir şekilde yapılabilir veya hükümet düzenlemeleri tarafından dayatıldığı gibi gönülsüz bir şekilde de olabilir. Bazen gönüllü olup olmadığı, kişinin bakış açısına bağlıdır. Bir kredi için başvururken kişisel bilgi vermek gönüllü mü yoksa gönülsüz mü? Ayrım, krediyi almanın bir kolaylık mı yoksa bir gereklilik mi olduğuna bağlıdır. Artık bazı perakendecilerde kredi kartı kullanmak, imzanızın dijital bir formatta kaydedilmesini gerektiriyor. Yine, bilgiyi sağlamak, durumunuza bağlı olarak ya gönüllü ya da gönülsüzdür.

Daha ince veri toplama durumları, konu ile doğrudan iletişimden kaçınır. Örnekler arasında, kredi kartı sahiplerinin satın alma alışkanlıklarını kaydeden bir kredi şirketi, siteyi ziyaret edenlerin kimliklerini kaydeden web siteleri ve hedeflenen bir kurumun otoparkında park eden araçların plaka numaralarını kaydeden sosyal aktivistler bulunmaktadır. Bu durumlarda, veri toplamanın konusu

Doclingo https://www.doclingo.ai

bilgi toplandığının farkında olmayabilir ve oluşturulan veritabanlarının varlığ ından haberdar olma olasılığı daha düşüktür.

Bazen, temel veri toplama faaliyetleri, birinin sadece düşünmek için dur ması durumunda kendiliğinden anlaşılır hale gelir. Örneğin, bir bakkal, önce den mağazaya kaydolan düzenli müşterilerine indirimler sunabilir. Kayıt süre ci, indirim almak için satın alma sırasında sunulması gereken kimlik kartlarını n verilmesini içerebilir. Sonuç olarak, mağaza, müşterilerin satın alımlarının kaydını derleyebilir; bu kayıt, verilen indirimlerin değerinden çok daha fazlad ır.

Elbette, bu veri toplama patlamasını yönlendiren güç, verinin değeridir ve bu değer, verilerin, aksi takdirde belirsiz kalacak bilgileri ortaya çıkaracak şekillerde b ağlantı kurulmasına olanak tanıyan veritabanı teknolojisindeki ilerlemelerle artırıl maktadır. Örneğin, kredi kartı sahiplerinin satın alma kalıpları sınıflandırılabilir ve çapraz listelenerek muazzam pazarlama değeri olan müşteri profilleri elde edilebi lir. Vücut geliştirme dergileri için abonelik formları, son zamanlarda egzersiz ekip manı satın alanlara gönderilebilirken, köpek itaat dergileri için abonelik formları s on zamanlarda köpek maması satın alanlara yönelik olarak hedeflenebilir. Bilgileri birleştirmenin alternatif yolları bazen çok hayal gücü gerektirir. Refah kayıtları, şartlı tahliye ihlalcilerini bulmak ve yakalamak için suç kayıtlarıyla karşıla ştırılmıştır ve 1984 yılında Amerika Birleşik Devletleri'nde Seçici Hizmet, askeri taslağa k aydolmayı başaramayan vatandaşları tanımlamak için popüler bir dondurma perakend ecisinin eski doğum kaydı listelerini kullanmıştır.

Toplumu veri tabanlarının kötüye kullanımından korumanın birkaç yaklaşı mı vardır. Bunlardan biri yasal çözümler uygulamaktır. Ne yazık ki, bir eyleme k arşı bir yasa çıkarmak, eylemin gerçekleşmesini durdurmaz, sadece eylemi yas adışı hale getirir. Amerika Birleşik Devletleri'nde bunun en iyi örneği, 1974 tarih li Gizlilik Yasasıdır; bu yasanın amacı, vatandaşları hükümet veri tabanlarının kö tüye kullanımından korumaktı. Bu yasada, hükümet ajanslarının veri tabanların ın bildirimini Federal Kayda yayınlaması gerektiği belirtilmiştir, böylece vatanda şlar kişisel bilgilerine erişebilir ve bunları düzeltebilir. Ancak, hükümet ajansları bu maddeye uymakta yavaş kaldı. Bu, mutlaka kötü niyetli bir amacı ifade etme z. Birçok durumda sorun bürokrasi ile ilgiliydi. Ancak, bir bürokrasinin tanımlay amadığı personel veri tabanları oluşturması, güven verici değildir.

Veritabanı istismarını kontrol etmenin başka, belki de daha güçlü bir yaklaşım ı kamuoyudur.Veritabanları, cezalar faydalardan daha ağır basmadığı sürece istis mar edilmeyecektir; ve işletmelerin en çok korktuğu ceza olumsuz kamuoyudur—bu doğrudan kar marjını etkiler. 1990'ların başında, büyük kredi bürolarının pazar lama amaçları için posta listeleri satmasını durduran nihai etken kamuoyuydu. Daha yakın zamanda, Amerika Online (büyük bir internet servis sağlayıcısı) müşteri ile il gili bilgileri tele-pazarlamacılara satma politikasına karşı kamu baskısı altında geri adım attı. Hükümet ajansları bile kamuoyuna boyun eğdi. 1997'de Amerika Birleşik Devletleri' ndeki Sosyal Güvenlik İdaresi, kamuoyunun bilgilerin güvenliği konusunda soru işaretle ri oluşturması üzerine sosyal güvenlik kayıtlarını internet üzerinden erişilebilir hale getir me planını değiştirdi. Bu durumlarda sonuçlar günler içinde elde edildi—hukuki süreçle rle ilişkilendirilen uzun zaman dilimlerinin keskin bir zıttı.

Elbette, birçok durumda veritabanı uygulamaları hem verinin sahibine h em de verinin konusuna faydalıdır, ancak her durumda hafife alınmaması ge reken bir mahremiyet kaybı vardır. Bu tür mahremiyet sorunları, bilgi doğru olduğunda ciddidir, ancak bilgi hatalı olduğunda devasa hale gelir. Kredi not unuzun olumsuz etkilendiğini fark ettiğinizde umutsuzluk hissini hayal edin.

Doclingo https://www.doclingo.ai

yanlış bilgi. Bu yanlış bilginin diğer kurumlarla kolayca paylaşıldığı bir ortam da sorunlarınızın nasıl büyüyeceğini hayal edin.

Gizlilik sorunları, genel olarak ilerleyen teknolojinin ve özellikle veritabanı tek niklerinin büyük bir yan etkisi olmaya devam edecektir. Bu sorunların çözümleri, e ğitimli, dikkatli ve aktif bir vatandaşlık gerektirecektir.

Sorular ve Alıştırmalar

1. Hukuk uygulayıcılarının, suç eğilimleri olan bireyleri tanımlamak ama cıyla veritabanlarına erişim verilmesi gerektiği düşünülebilir mi, oysa bu bireyler henüz bir suç işlemiş olmayabilir?
2. Sigorta şirketlerine, potansiyel tıbbi sorunları olan bireyleri tanımlamak ama cıyla veritabanlarına erişim verilmesi gerektiği düşünülebilir mi, oysa bu bire yler henüz herhangi bir belirti göstermemiş olabilir?
3. Diyelim ki mali açıdan rahat bir durumdasınız. Bu bilginin çeşitli kuruml ar arasında paylaşılması durumunda hangi faydaları elde edebilirsiniz? Aynı bilginin dağıtımından hangi cezaları çekebilirsiniz? Mali açıdan r ahatsız bir durumda olsaydınız ne olurdu?
4. Serbest basının veritabanı kötüye kullanımını kontrol etmedeki rolü nedir? (Örneğin, basın kamuoyunu ne ölçüde etkiler veya kötüye kullanımı ne ölçüde açığa çıkarır?)

Bölüm Gözden Geçirme Problemleri

(Yıldızlı problemler isteğe bağlı bölümlerle ilişkilidir.)

1. Düz bir dosya ile bir veritabanı arasındaki f arkı özetleyin.
2. Veri bağımsızlığı ne anlama geliyor?
3. Bir veritabanı uygulamasında katmanlı yaklaşımda DBMS'nin rolü nedir?
4. Bir şema ile bir alt şema arasındaki fark ne dir?
5. Uygulama yazılımını DBMS'den ayırmanın ik i faydasını belirtin.
6. Bir soyut veri türü (Bölüm 8) ile bir veritaban ı modeli arasındaki benzerlikleri tanımlayın.
7. Aşağıdaki endişelerin veya faaliyetlerin her birinin gerçekleştiği veritabanı sistemi içind eki seviyeyi (kullanıcı, uygulama yazılımı p rogramcısı, DBMS yazılımı tasarımcısı) belirl eyin:
   a. Veriler diskte verimliliği artırmak için nasıl saklanmalıdır?
   b. Uçuş 243'te bir boş koltuk var mı?
   c. Bir ilişkinin kütle depolamada nasıl organize edilmes i gerekir?
   d. Bir kullanıcının bir şifreyi yanlış yazmasın a ne kadar izin verilmelidir, konuşma son a ermeden önce?
   e. PROJE işlemi nasıl uygulanabilir?
8. Aşağıdaki görevlerden hangileri birDBMS tarafından yönetilme ktedir?
   a. Bir kullanıcının veritabanına erişi mininuygun alt şemayla sınırlı olmasını sağlamak.
   b. Veritabanı modelinde belirtilen komutları, güncel veri depolama sistemiyle uyumlu hareketlere dönüştürmek.
   c. Verilerin veritabanında aslında bir ağda kibirçok bilgisayara dağılmış olduğu nu gizlemek.

Doclingo https://www.doclingo.ai

9. Aşağıdaki bilgilerin havayolları, uçuşlar (belirli bir gün için) ve yolcular hakkında bir ilişkisel veritabanında nasıl temsil edileceğini tanımlayın:

Havayolları: Clear Sky, Long Hop ve Tree Top

Clear Sky için uçuşlar: CS205, CS37 ve CS102

Long Hop için uçuşlar: LH67 ve LH89

Tree Top için uçuşlar: TT331 ve TT809Smith,

CS205 (koltuk 12B),CS37 (koltuk 18C) ve LH 89 (koltuk 14A) için rezervasyon yaptırmıştır.

Baker, CS37 (koltuk 18B) veLH89 (koltuk 14B) için rezervasyon yaptırmıştır.

Clark, LH67 (koltuk 5A) veTT331 (koltuk 4B) için rezervasyon yaptırmıştır.

10. SELECT ve PROJECT işlemlerinin bir ilişkiye uygulanma sırasının önemi nedir? Yani, hangi koşullar altında SELECT etmek ve sonra PROJECT etmek, önce PROJECT edip sonra SELECT etmekle aynı sonuçları verecektir?

11. 9.2. Bölümde tanımlandığı gibi JOIN işlemi içindeki "where" ifadesinin gerekli olmadığını gösteren bir argüman verin. (Yani, "where" ifadesini kullanan herhangi bir sorgunun, bir ilişkideki her bir tuple'ı diğerindeki her bir tuple ile birleştiren bir JOIN işlemi kullanılarak yeniden ifade edilebileceğini gösterin.)

12. Aşağıda gösterilen ilişkiler açısından, bu talimatların her birini uyguladıktan sonra RESULT ilişkisinin görünümü nedir:

X ilişkisi

U	V	W
A	Z	5
B	D	3
C	Q	5

Y ilişkisi

R	S
3	J
4	K

a. SONUÇ ← PROJE W'den X
b. SONUÇ ← SEÇ X'den W = 5 olanları
c. SONUÇ ← PROJE S'den Y
d. SONUÇ ← X ve Y'yi birleştir X.W olduğu yerde ⩾ Y. R.

13. SEÇ, PROJE veBİRLEŞTİR komutlarını kullanarak, aşağıdaki sorulara yanıt vermek için bir dizi talimat yazın

parçalar ve bunların üreticileri ile ilgiliaşağıdaki veritabanında:

PART ilişkisi

ParçaAdı	Ağırlık
Somun 2X	1
Somun 2Z	1.5
Somun V5	0.5

ÜRETİCİ ilişkisi

ŞirketAdı	ParçaAdı	Maliyet
Şirket X	Somun 2Z	.03
Şirket X	Somun V5	.01
Şirket Y	Somun 2X	.02
Şirket Y	Somun V5	.01
Şirket Y	Somun 2Z	.04
Şirket Z	Somun V5	.01

a. Hangi şirketler Bolt 2Z üretmektedir?
b. Şirket X tarafından üretilen parçaların ve her parçanın maliyetinin bir listesini alın.
c. Hangi şirketler 1 ağırlığında bir parça üretmektedir?

14. Problem 13'ü SQL kullanarak yanıtlayın.

15. SEÇ, PROJE veBİRLEŞTİR komutlarını kullanarak, aşağıdaki sorulara yanıt vermek için diziler yazın
ÇALIŞAN, İŞ, ve ATAMA ilişkileri
Şekil 9.5'te:
a. Şirketin çalışanlarının isimleri ve adresleri nin bir listesini alın.
b. Personel departmanında çalışmış veya çalışanların isimleri ve adreslerinin bir listesini alın.
c. Personel departmanında çalışanların isimleri ve adreslerinin bir listesini alın.

16. Önceki soruyu SQL kullanarak yanıtlayın.

17. Müzik bestecileri, hayatları ve eserleri hakkında bilgi içeren bir ilişkisel veritabanı tasarlayın. (Şekil 9.4'teki gibi gereksizliklerden kaçının.)

18. Müzik sanatçıları, kayıtları ve kaydettikleri müziklerin bestecileri hakkında bilgi içeren bir ilişkisel veritabanı tasarlayın. (Şekil 9.4'teki gibi gereksizliklerden kaçının.)

Doclingo https://www.doclingo.ai

19. Bilgisayar ekipmanlarının üreticileri ve ürünleri hakkında bilgi içeren bir ilişkisel veritabanı tasarlayın.(Aynı şekilde, Şekil 9.4'teki gibi tekrarları önleyin.)

20. Yayıncılar, dergiler ve aboneler hakkında bilgi içeren bir ilişkisel veritabanı tasarlayın.(Aynı şekilde, Şekil 9.4'teki gibi tekrarları önleyin.)

21. Parçalar, tedarikçiler ve müşteriler hakkında bilgi içeren bir ilişkisel veritabanı tasarlayın. Her parça birkaç tedarikçi tarafından sağlanabilir ve birçok müşteri tarafından sipariş edilebilir. Her tedarikçi birçok parça sağlayabilir ve bir çok müşteriye sahip olabilir. Her müşteri birçok tedarikçiden birçok parça sipariş edebilir; aslında, aynı parça birden fazla tedarikçiden sipariş edilebilir. (Şekil 9.4'teki benzer gereksizliklerden kaçının.)

22. SELECT, PROJECT ve JOIN işlemlerini kullanarak, muhasebe departmanındaki her iş için JobId, StartDate ve TermDate değerlerini almak için bir dizi talimat yazın. Aşağıda tanımlanan ilişkisel veritabanından alınacaktır. Şekil 9.5.

23. Önceki soruyu SQL kullanarak yanıtlayın.

24. SELECT, PROJECT ve JOIN işlemlerini kullanarak, her mevcut çalışanın Name, Address, JobTitle ve Dept değerlerini almak için bir dizi talimat yazın. Aşağıda tanımlanan ilişkisel veritabanından alınacaktır. Şekil 9.5.

25. Önceki soruyu SQL kullanarak yanıtlayın.

26. SELECT, PROJECT ve JOIN işlemlerini kullanarak, her mevcut çalışanın Name ve JobTitle değerlerini almak için bir dizi talimat yazın. Aşağıda tanımlanan ilişkisel veritabanından alınacaktır. Şekil 9.5.

27. Önceki soruyu SQL kullanarak yanıtlayın.

28. Tekil ilişki tarafından sağlanan bilgi ile iki ilişki arasındaki fark nedir?

İsim	Bölüm	Telefon Numarası
Jones\&	Satış	555-2222
Smith\	Satış	555-3333
Baker	Personel	555-4444

ve iki ilişki

İsim	Bölüm
Jones\&	Satış
Smith\	Satış
Baker	Personel

Bölüm	Telefon Numarası
Satış	555-2222
Satış	555-3333
Personel	555-4444

29. Otomobil parçaları ve alt parçaları hakkında bilgi içeren bir ilişkisel veritabanı tasarlayın. Bir parçanın daha küçük parçalar içerebileceğini ve aynı zamanda daha büyük parçalarda yer alabileceğini göz önünde bulundurduğunuzdan emin olun.

30. www.google.com, www.amazon.com veya www.ebay.com gibi popüler bir web sitesi için ve sitenin destekleyici veritabanı olarak önerdiğiniz bir ilişkisel veritabanı tasarlayın.

31. Şekil 9.5'te temsil edilen veritabanına dayanarak, aşağıdaki program segmenti tarafından yanıtlanan soruyu belirtin:

```
TEMP← SEÇİM ASSIGNMENT'tan
    TermDate = "*"
SONUÇ← PROJE JobId, BaşlangıçTarihi
    TEMP'tan
```

32. Önceki sorudaki sorguyuSQL'e çevirin.

33. Şekil 9.5'te temsil edilen veritabanına dayanarak, aşağıdaki program segmenti tarafından yanıtlanan soruyu belirtin:

```
TEMP1← ÇALIŞAN ve ASSIGNMENT ile BİRLEŞTİR
    ÇALIŞAN.EmplId =
    ASSIGNMENT.EmplId
TEMP2← TEMP1'den SEÇ
    TermDate = "*"
SONUÇ← PROJE ad, BaşlangıçTarihi
    TEMP2'den
```

34. Önceki sorudaki sorguyuSQL'e çevirin.

35. Şekil 9.5'te temsil edilen veritabanına dayanarak, aşağıdaki program segmenti tarafından yanıtlanan soruyu belirtin:

```
TEMP1← ÇALIŞAN ve İŞ ile BİRLEŞTİR
    ÇALIŞAN.EmplId = İŞ.EmplId
```

Doclingo https://www.doclingo.ai

```
TEMP2← TEMP1'den SEÇ
   Dept = "SATIŞ"
SONUÇ← TEMP2'den PROJE Adı
```

36. Önceki sorudaki sorguyuSQL'e çevirin.

37. SQL ifadesini çevirin

```
JOB.JobTitle'ı seçin
ASSIGNMENT, JOB'dan
where ASSIGNMENT.JoblId = JOB.JobId
  and ASSIGNMENT.EmplId = "34Y70"
```

SEÇ, PROJE veBİRLEŞTİR işlemlerinin bir dizisine.

38. SQL ifadesini çevirin

```
ASSIGNMENT.StartDate'ı seçin
ASSIGNMENT, ÇALIŞAN'dan
where ASSIGNMENT.EmplId =
      ÇALIŞAN.EmplId
   ve ÇALIŞAN.Ad = "Joe E.
      Fırıncı"
```

SEÇ, PROJE veBİRLEŞTİR işlemlerinin bir dizisine.

39. Aşağıdaki SQL ifadesininProblem 13'te veritabanı üzerindeki etkisini tanımlayın.

```
MANUFACTURER içine ekle
değerler ('Şirket Z', 'Civata 2X', .03)
```

40. Aşağıdaki SQL ifadesininProblem 13'te veritabanı üzerindeki etkisini tanımlayın.

```
GÜNCELLE ÜRETİCİ
   Maliyet = .03
   ŞirketAdı = 'Şirket Y'
      ve ParçaAdı = 'Civata 2X'
```

***41.** Bir bakkalın envanterini korumak için kullanılan nesne yönelimli veritabanında bulmayı bekleyeceğiniz bazı nesneleri tanımlayın. Bu nesnelerin her birinde hangi yöntemleri bulmayı beklersiniz?

***42.** Bir kütüphanenin tutma kayıtlarını korumak için kullanılan nesne yönelimli veritabanında bulmayı bekleyeceğiniz bazı nesneleri tanımlayın. Bu nesnelerin her birinde hangi yöntemleri bulmayı beklersiniz?

***43.** Aşağıdaki T1 ve T2 işlem programı tarafından üretilen yanlış bilgi nedir?

T1, A ve B hesaplarının toplamını hesaplamak için tasarlanmıştır; T2, A hesabından B hesabına $100 transfer etmek için tasarlanmıştır. T1, A hesabının bakiyesini alarak başlar; ardından, T2 transferini gerçekleştirir; ve nihayet, T1 B hesabının bakiyesini alır ve aldığı değerlerin toplamını raporlar.

***44.** Metinde tanımlanan kilitleme protokolünün, Problem 43'te üretilen hatayı nasıl çözeceği ni açıklayın.

***45.** Eğer T1 daha genç işlemse, Problem 43'te ki olaylar dizisi üzerinde wound-wait protokolünün ne etkisi olurdu? Eğer T2 daha genç işlemse?

***46.** Bir işlemin, bakiyesi $200 olan bir hesaba $100 eklemeye çalıştığını ve başka bir işlemin aynı hesaptan $100 çekmeye çalıştığını varsayalım. Bu işlemlerin, $100'lık bir son bakiyeye ulaşacak şekilde nasıl iç içe geçeceğini tanımlayın. Bu işlemlerin, $300'lük bir son bakiyeye ulaşacak şekilde nasıl iç içe geçeceğini tanımlayın.

***47.** Bir işlemin bir veritabanındaki bir öğeye özel erişime sahip olması ile paylaşılan erişime sahip olması arasındaki fark nedir ve bu ayrım neden önemlidir?

***48.** Bölüm 9.4'te tartışılan eşzamanlı işlemlerle ilgili problemler, veritabanı ortamlarıyla sınırlı değildir. Kelime işlemcileri ile bir belgeye erişirken hangi benzer problemler ortaya çıkacaktır? (Eğer bir kelime işlemciye sahip bir PC'niz varsa, kelime işlemcisinin iki aktivasyonu ile aynı belgeye erişmeyi deneyin ve ne olduğunu görün.)

***49.** Varsayalım ki, ardışık bir dosya 50.000 kayıt içermektedir ve bir kaydı sorgulamak için 5 milisaniye gerekmektedir. Dosyanın ortasından bir kayıt alırken ne kadar beklememiz gerektiğini tahmin etmeliyiz?

***50.** Başlangıçta bir girdi dosyasının boş olduğu durumda, Şekil 9.15'teki birleştirme algoritmasında gerçekleştirilen adımları listeleyin.

***51.** Şekil 9.15'teki algoritmayı, her iki girdi dosyasının da aynı anahtar alan değerine sahip bir kayıt içerdiği durumu ele alacak şekilde değiştirin. Bu kayıtların özdeş olduğunu ve yalnızca birinin çıktı dosyasında görünmesi gerektiğini varsayın.

Doclingo https://www.doclingo.ai

*52. Bir diskte saklanan bir dosyanın, iki farklı sıralama ile ardışık bir dosya olarak işlenebi leceği bir sistem tasarlayın.

*53. Bir derginin aboneleri hakkında bilgi içere n bir ardışık dosyanın, temel yapı olarak bir metin dosyası kullanılarak nasıl oluşturulabi leceğini tanımlayın.

*54. Mantıksal kayıtları tutarlı bir boyutta olmaya n bir ardışık dosyanın, bir metin dosyası ola rak nasıl uygulanabileceğine dair bir teknik tasarlayın. Örneğin, bir yazar hakkında bilgi içeren ve o yazarın ese rlerinin bir listesini içeren bir ardışık dosya oluş turmak istediğinizi varsayalım.

*55. Bir indeksli dosyanın, bir hash dosyasına göre ne gibi avantajları vardır? Bir hash dosyasının, bir indeksli dosy aya göre ne gibi avantajları vardır?

*56. Bölüm, geleneksel bir dosya indeksinin bir işl etim sistemi tarafından sürdürülen dosya dizi n sistemi ile paralellikler kurdu. Bir işletim sis teminin dosya dizini, geleneksel bir indeksten hangi yönleriyle farklıdır?

*57. Eğer bir hash dosyası 10 kovaya bölünm üşse, üç rastgele kaydın en az ikisinin aynı kovaya hashlenme olasılığı nedir?

(Hash fonksiyonunun diğerlerine göre hiçbir kova önceliğ i vermediğini varsayın.) Çakışmaların meydana gelme olasıl ığının, meydana gelmeme olasılığından daha yüksek olması için dosyada kaç kayıt saklanmalıdır?

*58. Önceki problemi çözün, dosyanın 10 yerine 100 kova olarak bölündüğünü varsayarak.

*59. Bu bölümde tartışılan bölme tekniğini bir hash fonksiyonu olarak kullanıyorsak ve dos ya depolama alanı 23 kovaya bölünmüşse, an ahtarı ikili değer olarak yorumlandığında 124 tam sayısı olan kaydı bulmak için ha ngi bölümü aramalıyız?

*60. Bir hash dosyasının uygulanmasını, homojen iki boyutlu bir dizinin uygulanmasıyla karşılaştırın. Hash fonksiyonu ve adres polinomunun roll eri nasıl benzerlik gösteriyor?

*61. a. Sıralı bir dosyanın, ind eksli bir dosyaya göre bir avantajını verin.

b. Sıralı bir dosyanın, hash dosyasına göre bir avantajını verin.

c. İndeksli bir dosyanın, sıralı bir dosyaya göre bir avantajını verin.

d. İndeksli bir dosyanın, hash dosyasına göre bir avantajını verin.

e. Hash dosyasının, sıralı bir dosyaya göre bir avantajı vardır.

f. Hash dosyasının, indeksli bir dosyaya göre bir avantajı vardır.

*62. Sıralı bir dosya, bir bağlı listeye hangi yönde n benzer?

Sosyal Sorunlar

İzleyen sorular, bilgisayar alanıyla ilişkili etik/sosyal/hukuki sorunlara bir reh ber olarak tasarlanmıştır. Amaç yalnızca bu soruları yanıtlamak değildir. Ayrı ca, neden böyle yanıt verdiğinizi ve gerekçelerinizin bir sorudan diğerine tut arlı olup olmadığını da düşünmelisiniz.

1. Amerika Birleşik Devletleri'nde, tüm federal mahkumların DNA kayıtları artık suç soruş turmalarında kullanılmak üzere bir veritabanında saklanmaktadır. Bu bilgilerin başka amaçlar için, örneğin tıbbi araştırmalar için, yayımlanması etik olur mu? Olursa, hangi amaçlar için? Olmazsa, neden olmasın? Her durumda artılar ve eksiler nelerdir?
2. Bir üniversitenin öğrencileri hakkında ne ölçüde bilgi yayımlamasına izin verilmelidir? İsimleri ve adresleri hakkında ne olacak? Öğrencileri tanıml amadan not dağılımları hakkında ne olacak? Cevabınız, Soru 1'e verdiğini z cevapla tutarlı mı?
3. Bireyler hakkında veritabanları oluşturulmasıyla ilgili hangi kısıtlamalar uygun dur? Bir hükümetin vatandaşlarıyla ilgili hangi bilgilere sahip olma hakkı vardı r? Bir sigorta şirketinin müşterileriyle ilgili hangi bilgilere sahip olma hakkı var dır? Bir şirketin çalışanlarıyla ilgili hangi bilgilere sahip olma hakkı vardır?

Doclingo https://www.doclingo.ai

Bu ortamlarda kontroller uygulanmalı mı ve uygulanacaksa, nasıl?

4. Bir kredi kartı şirketinin, müşterilerinin satın alma alışkanlıklarını pazarlama firmala rına satması uygun mu? Bir spor araba mail order işinin, posta listesini bir spor ara ba dergisine satması kabul edilebilir mi? Amerika Birleşik Devletleri'ndeki İç Gelir S ervisi'nin, önemli sermaye kazançlarına sahip olan mükelleflerin isimlerini ve adresl erini borsa aracılıklarına satması kabul edilebilir mi? Eğer buna kesin bir evet veya hayır ile cevap veremiyorsanız, kabul edilebilir bir politika olarak ne önerirsiniz?
5. Bir veritabanının tasarımcısı, o veritabanındaki bilgilerin nasıl kullanıldığı kon usunda ne ölçüde sorumludur?
6. Varsayalım ki bir veritabanı, veritabanındaki bilgilere onaylanmamış erişi me izin veriyor. Eğer bu bilgiler elde edilip olumsuz bir şekilde kullanılırs a, veritabanı tasarımcıları bu bilgilere yönelik kötüye kullanımda ne ölçü de sorumluluk paylaşır? Cevabınız, failin veritabanı tasarımındaki hatayı keşfetmek ve yetkisiz bilgilere ulaşmak için gereken çabaya bağlı mı?
7. Veri madenciliğinin yaygınlığı, birçok etik ve gizlilik sorununu gündeme getiriyor. Veri madenciliği, topluluğunuzun genel nüfusu hakkında belirli özellikleri orta ya çıkardığında gizliliğiniz ihlal edilmiş mi olur? Veri madenciliğinin kullanımı, i yi iş uygulamalarını mı yoksa önyargıyı mı teşvik ediyor? Daha fazla bilginin, b ireysel anketlerde açıkça talep edilenden daha fazla çıkarılacağını bilerek vata ndaşları bir nüfus sayımına katılmaya zorlamak ne ölçüde doğrudur? Veri ma denciliği, pazarlama firmalarına, habersiz kitleler üzerinde haksız bir avantaj s ağlar mı? Profil oluşturma ne ölçüde iyi ya da kötü?
8. Bir kişi veya şirketin bireyler hakkında bilgi toplamasına ve saklamasına ne ölçüde izin verilmelidir? Toplanan bilgiler zaten birkaç kaynak arasınd a dağılmış olarak kamuya açıksa ne olur? Bir kişi veya şirketin bu tür bilgi leri koruması ne ölçüde beklenmelidir?
9. Birçok kütüphane, kullanıcıların bilgi ararken bir kütüphanecinin yardımını alabil mesi için bir referans hizmeti sunmaktadır. İnternetin ve veritabanı teknolojisini n bu hizmeti gereksiz hale getirip getirmeyeceği ne olacak? Eğer öyleyse, bu bir i lerleme mi yoksa gerileme mi olur? Eğer değilse, neden olmasın? İnternetin ve v eritabanı teknolojisinin kütüphanelerin varlığını nasıl etkileyeceği?
10. Kendinizi kimlik hırsızlığı olasılığına ne ölçüde maruz hissediyorsunuz? Bu maruziyeti en a za indirmek için hangi adımları atabilirsiniz? Kimlik hırsızlığının kurbanı olursanız nasıl zar ar görebilirsiniz? Kimlik hırsızlığı meydana geldiğinde sorumlu olmalı mısınız?

Ek Okuma

Beg, C. E. ve T. Connolly. *Veritabanı Sistemleri: Tasarım, Uygulama ve Yöneti m için Pratik Bir Yaklaşım*, 4. baskı. Boston, MA: Addison-Wesley, 2005.

Berstein, A., M. Kifer ve P. M. Lewis. *Veritabanı Sistemleri*, 2. baskı. Boston, MA: Addison-Wesley, 2006.

Date, C. J. *Veritabanı Sistemlerine Giriş,* 8. baskı. Boston, MA: Addison-Wesley, 2004.

Doclingo https://www.doclingo.ai

Date, C. J. *Veritabanları, Türler ve İlişkisel Model,* 3. baskı. Boston, MA: Addison-Wesley, 2007.

Elmasri, R. ve S. Navathe. *Veritabanı Sistemlerinin Temelleri*, 6. baskı. Boston, MA: Addison-Wesley, 2011.

Patrick, J. J. *SQL Temelleri*, 3. baskı. Upper Saddle River, NJ: Prentice-Hall, 2009.

Silberschatz, A., H. Korth ve S. Sudarshan. *Veritabanı Sistemleri Kavramları*, 8. baskı. New York: McGraw-Hill, 2009.

Ullman, J. D. ve J. D. Widom. *Veritabanı Sistemlerinde İlk Kurs*, 3. baskı. Upper Saddle River, NJ: Prentice-Hall, 2008.

Doclingo https://www.doclingo.ai

BÖLÜM

Bilgisayar Grafikleri

10

Bu bölümde, hareketli resimlerin ve etkileşimli video oyunl arının üretiminde büyük bir etki yaratan bilgisayar grafikle ri alanını keşfediyoruz. Gerçekten de, bilgisayar grafiklerin deki ilerlemeler, görsel medyayı gerçekliğin kısıtlamalarınd an kurtarıyor ve birçok kişi, bilgisayar tarafından üretilen a nimasyonların yakında geleneksel aktörler, setler ve fotoğr afçılığın yerini alabileceğini savunuyor.

10.1 Bilgisayar Grafiklerinin Kapsamı

10.2 3D'ye Genel Bakış Grafikler

10.3 Modelleme

Bireysel Nesnelerin Modellenmesi
Tüm Sahne Modellenmesi

10.4 Renderlama

Işık-Yüzey EtkileşimiKırp ma, Tarama Dönüşümü veGizli Yüzeylerin KaldırılmasıGö lgeleme
Renderlama-Pipeline Donanımı

***10.5 KüreselAydınlatma ile İlgilenme**

Işın İzlemeRa dyoaktivite

10.6 Animasyon

Animasyon Tem elleriKinematik ve Dina mikAnimasyon Süreci

**Yıldızlar, isteğe bağlı bölümler için önerileri belirtir.*

Doclingo https://www.doclingo.ai

Bilgisayar grafikleri, bilgisayar teknolojisini görsel temsillerin üretilmesi ve manipülasyonu için uygulayan bilgisayar bilimi dalıdır. Metin sunumu, grafik ve tabloların oluşturulması, grafik kullanıcı arayüzlerinin geliştirilmesi, fotoğrafların manipülasyonu, video oyunlarının üretilmesi ve animasyonlu filmlerin yaratılması gibi çok çeşitli konularla ilişkilidir. Ancak, *bilgisayar grafikleri* terimi giderek 3D grafikleri olarak adlandırılan belirli bir alanı ifade etmek için kullanılmaktadır ve bu bölümün çoğu bu konuya odaklanmaktadır. 3D grafikleri tanımlayarak ve bilgisayar grafiklerinin daha geniş yorumlaması içindeki rolünü netleştirerek başlıyoruz.

10.1 Bilgisayar Grafiklerinin Kapsamı

Dijital kameraların ortaya çıkmasıyla birlikte, dijital olarak kodlanmış görüntüleri manipüle etmek için yazılımların popülaritesi hızla artmıştır. Bu yazılım, lekeleri ve korkulan "kırmızı göz"ü kaldırarak fotoğrafları "düzeltmeye" olanak tanır ve farklı fotoğraflardan parçaları kesip yapıştırarak gerçeği yansıtmayacak görüntüler oluşturur.

Benzer teknikler, sinema ve televizyon endüstrilerinde özel efektler yaratmak için sıklıkla uygulanmaktadır. Aslında, bu tür uygulamalar, bu endüstrilerin film gibi analog sistemlerden dijital olarak kodlanmış görüntülere geçişinin ana motivasyon faktörleriydi. Uygulamalar arasında destek tellerinin görünümünü kaldırmak, birden fazla görüntüyü üst üste bindirmek veya bir kamerayla ilk olarak yakalanan eylemi değiştirmek için kullanılan yeni görüntülerin kısa dizilerini üretmek bulunmaktadır.

Dijital fotoğrafları ve hareketli resim karelerini manipüle etmek için yazılımlara ek olarak, basit çizimlerden karmaşık sanata kadar iki boyutlu görüntüler üretmeye yardımcı olan çok çeşitli yardımcı/uygulama yazılımı paketleri bulunmaktadır. (İyi bilinen bir temel örnek, Microsoft'un Paint adlı uygulamasıdır.) Bu programlar en azından kullanıcının noktalar ve çizgiler çizmesine, oval ve dikdörtgen gibi basit geometrik şekiller eklemesine, bölgeleri renklendirmesine ve bir çizimin belirli kısımlarını kesip yapıştırmasına olanak tanır.

Tüm önceki uygulamaların düz iki boyutlu şekiller ve görüntülerin manipülasyonu ile ilgilendiğini unutmayın. Bu nedenle, iki ilgili araştırma alanının örnekleridir: Biri 2D grafikler, diğeri ise görüntü işleme. Ayrım, 2D grafiklerin iki boyutlu şekilleri (daireler, dikdörtgenler, harfler vb.) bir görüntü üretmek için piksel desenlerine dönüştürme görevine odaklanmasıdır; oysa daha sonra yapay zeka çalışmamızda karşılaşacağımız görüntü işleme, bir görüntüdeki pikselleri analiz etmeye odaklanarak görüntüyü geliştirmek veya belki de "anlamak" için kullanılabilecek desenleri tanımlamaya çalışır. Kısacası, 2D grafikler görüntü üretimi ile ilgilenirken, görüntü işleme görüntüleri analiz etmekle ilgilenir.

2D grafiklerde olduğu gibi iki boyutlu şekilleri görüntülere dönüştürmenin aksine, 3D grafikler alanı üç boyutlu şekilleri görüntülere dönüştürmekle ilgilenir. Süreç, üç boyutlu sahnelerin dijital olarak kodlanmış versiyonlarını oluşturmak ve ardından bu sahnelerin görüntülerini üretmek için fotoğrafik süreci simüle etmektir. Tema, geleneksel fotoğrafçılıkla benzerlik gösterir, ancak 3D grafik teknikleriyle "fotoğraflanan" sahne fiziksel bir gerçeklik olarak mevcut değildir, bunun yerine yalnızca bir veri ve algoritma koleksiyonu olarak "mevcuttur". Böylece, 3D grafikler sanal dünyaların "fotoğraflanmasını" içerirken (Şekil 10.1), geleneksel fotoğrafçılık gerçek dünyanın fotoğraflanmasını içerir.

Doclingo https://www.doclingo.ai

Şekil 10.1: 3D grafikler kullanılarak üretilen bir sanal dünyanın “fotoğrafı” (Walt Disney Pictu res/Pixar Animation Studios'tan Toy Story) © Disney/Pixar



3D grafikler kullanılarak bir görüntü oluşturmanın iki ayrı adımı içerdiğin i belirtmek önemlidir. Birincisi, fotoğraflanacak sahnenin oluşturulması, kodl anması, depolanması ve manipülasyonudur. Diğeri ise görüntünün üretilme si sürecidir. İlk adım yaratıcı, sanatsal bir süreçtir; ikincisi ise hesaplama açısı ndan yoğun bir süreçtir. Bu konuları önümüzdeki dört bölümde keşfedeceği z.

3D grafiklerin sanal sahnelerin “fotoğraflarını” üretmesi, etkileşimli video oyunları ve animasyonlu sinema prodüksiyonları gibi gerçekliğin aksiyonları sınırlayacağı durumlarda kullanım için ideal hale getirir. Bir etkileşimli video oyunu, oyun oyuncusunun etkileşimde bulunduğu kodlanmış üç boyutlu bir sanal ortamdan oluşur. Oyuncunun gördüğü görüntüler, 3D grafik teknolojis i aracılığıyla üretilir. Animasyonlu sinema filmleri benzer bir şekilde oluşturul ur, ancak burada sanal ortamla etkileşimde bulunan insan animatördür, nih ai izleyici değil. Kamuya dağıtılan ürün, prodüksiyonun yönetmeni/producen ti tarafından belirlenen iki boyutlu görüntülerin bir dizisidir.

3D grafiklerin animasyondaki kullanımını daha ayrıntılı olarak Bölüm 10. 6'da inceleyeceğiz. Şimdilik, bu uygulamaların 3D grafik teknolojisi ilerledikç e nereye gidebileceğini hayal ederek bu bölümü kapatalım. Bugün, sinema fi lmleri iki boyutlu görüntü dizileri olarak dağıtılmaktadır. Bu bilgileri görüntül eyen projektörler, film makaraları ile analog cihazlardan DVD oynatıcılar ve d üz panel ekranlar kullanan dijital teknolojiye geçiş yapmış olsa da, hala yalnız ca iki boyutlu temsillerle işlemektedirler.

Ancak, gerçekçi üç boyutlu sanal dünyalar yaratma ve manipüle etme yet eneğimiz geliştikçe bunun nasıl değişebileceğini hayal edin. Bu sanal dünyal arı “fotoğrafını çekmek” yerine, bir hareketli resim şeklinde dağıtmak yerine

Doclingo https://www.doclingo.ai

iki boyutlu görüntüler, sanal dünyaları dağıtabiliriz. Potansiyel birizleyici, sad ece hareketli görüntü yerine hareketli görüntü setine erişim alır.Bu üç boyutl u set, video oyunlarının özel amaçlı “oyun kutuları” aracılığıylaizlendiği şekild e “3D grafik projektörü” aracılığıyla görüntülenecektir. Öncelikle“önerilen bir senaryo” izlenebilir ve bu, hareketli görüntünün yönetmen/üretici tarafından hayal edildiği gibi izlenmesini sağlar. Ancak, izleyici aynı zamandavideo oyu nunu andıran bir şekilde sanal setle etkileşimde bulunarak başka senaryolar üretebilir. Olasılıklar geniştir, özellikle de geliştirilen üç boyutlu insan-makine arayüzlerinin potansiyellerini de göz önünde bulundurduğumuzda.

Sorular ve Alıştırmalar

1. Görüntü işleme, 2D grafikler ve 3D grafikler arasındaki farkı özetleyi n.
2. 3D grafikler geleneksel fotoğrafçılıktan nasıl farklıdır?
3. 3D grafik kullanarak bir "fotoğraf" üretmenin iki ana adımı nelerdir ?

10.2 3D Grafiklerin Genel Görünümü

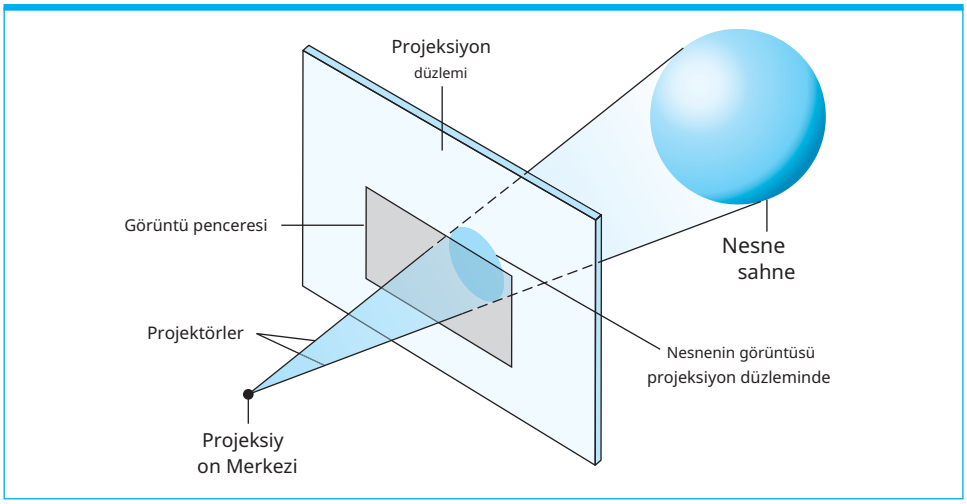
3D grafiklerin incelenmesine, görüntülerin yaratılması ve görüntülenmesi süre cini ele alarak başlayalım—bu süreç üç adımdan oluşur: modelleme, render al ma ve görüntüleme. Modelleme adımı (detaylı olarak Bölüm 10.3'te inceleyece ğiz) geleneksel sinema endüstrisinde bir set tasarlamak ve inşa etmekle benzer lik gösterir, ancak 3D grafik sahnesi dijital olarak kodlanmış veriler ve algoritm alar kullanılarak “inşa” edilir. Dolayısıyla, bilgisayar grafikleri bağlamında üretil en sahne gerçekte asla var olmayabilir.

Bir sonraki adım, sahnedeki nesnelerin belirli bir konumda bir kamera tarafından çekil en bir fotoğrafta nasıl görüneceğini hesaplayarak sahnenin iki boyutlu bir görüntüsünü ür etmektir. Bu adım render alma olarak adlandırılır—Bölüm 10.4 ve 10.5'in konusudur. Rend er alma, analitik geometri matematiğini uygulayarak sahnedeki nesnelerin düz bir yüzeye, projeksiyon düzlemi olarak bilinen bir yüzeye projeksiyonunu hesaplamayı içerir; bu, bir k ameranın bir sahneyi filme projekte etmesine benzer (Şekil 10.2). Uygulanan projeksiyon t ürü perspektif projeksiyonu olup, bu, tüm nesnelerin projeksiyonlar olarak adlandırılan d üz hatlar boyunca, projeksiyon merkezi veya görüş noktası olarak adlandırılan ortak bir n oktadan uzanarak projekte edildiği anlamına gelir. (Bu, projeksiyonların paralel olduğu pa ralel projeksiyon ile karşılaştırıldığında geçerlidir. Perspektif projeksiyonu, insan gözünün gördüğüne benzer bir projeksiyon üretirken, paralel projeksiyon bir nesnenin “gerçek” prof ilini üretir; bu, mühendislik çizimleri bağlamında sıklıkla faydalıdır.)

Projeksiyon düzleminin son görüntünün sınırlarını tanımlayan kısıtlı bölü mü, görüntü penceresi olarak bilinir. Bu, çoğu kameranın vizöründe potansi yel resmin sınırlarını göstermek için görüntülenen dikdörtgene karşılık gelir. Gerçekten de, çoğu kameranın vizörü, kameranın projeksiyon düzleminin yal nızca görüntü penceresini değil, daha fazlasını görmenizi sağlar. (Siz

Doclingo https://www.doclingo.ai

Şekil **10.2** 3D grafik paradigması



"Teyze Martha'nın" başının üstünü vizörde görebilirsiniz, ancak başının üstü resim penceresinin içinde değilse, son resimde görünmeyecektir.)

Görüntü penceresine yansıyan sahne kısmı belirlendikten sonra, nihai gö rüntüdeki her pikselin görünümü hesaplanır. Bu piksel bazında işlem, sahne deki nesnelerin ışıkla nasıl etkileşime girdiğini belirlemeyi gerektirdiğinden h esaplama açısından karmaşık olabilir - parlak ışıkta sert, parlak bir yüzey, dol aylı ışıkta yumuşak, şeffaf bir yüzeyden farklı şekilde işlenmelidir. Bununla bi rlikte, render alma süreci, malzeme bilimi ve fizik gibi birçok alandan yoğun bir şekilde yararlanmaktadır. Dahası, bir nesnenin görünümünü belirlemek g enellikle sahnedeki diğer nesneler hakkında bilgi gerektirir. Nesne başka bir nesnenin gölgesinde olabilir veya nesne, görünümü esasen başka bir nesne nin görünümü olan bir ayna olabilir.

Her pikselin görünümü belirlendiğinde, sonuçlar bir depolama alanında gö rüntünün bit haritası temsili olarak topluca saklanır ve bu alana çerçeve tampo nu denir. Bu tampon, ana belleğin bir alanı olabilir veya grafik uygulamaları içi n özel olarak tasarlanmış donanım durumunda, özel amaçlı bellek devrelerinin bir bloğu olabilir.

Son olarak, çerçeve tamponunda saklanan görüntü ya görüntülenmek üzere gö sterilir ya da daha sonra görüntülemek için daha kalıcı bir depolama alanına aktarılır . Eğer görüntü bir sinema filmi için üretiliyorsa, son sunumdan önce saklanabilir ve b elki de değiştirilir. Ancak, etkileşimli bir video oyunu veya uçuş simülatöründe, görün tüler gerçek zamanlı olarak üretildikçe gösterilmelidir; bu gereklilik, oluşturulan gör üntülerin kalitesini genellikle sınırlamaktadır. Bu nedenle, sinema endüstrisi tarafınd an dağıtılan tam özellikli animasyonlu prodüksiyonların grafik kalitesi, günümüz etkil eşimli video oyunlarının kalitesini aşmaktadır.

3D grafiklere girişimizi tipik bir video oyunu sistemini analiz ederek kapa tıyoruz. Oyun, esasen, oyun oyuncusu tarafından manipüle edilebilen yazılım la birlikte kodlanmış sanal bir dünyadır. Oyuncu bu dünyayı manipüle ettikçe , oyun sistemi sahneyi tekrar tekrar işler ve görüntüyü görüntü tamponunda saklar. Gerçek dünya zaman kısıtlamalarını aşmak için, bu işleme sürecinin ç oğu özel amaçlı donanım tarafından yönetilmektedir.

Doclingo https://www.doclingo.ai

Gerçekten, bu donanımın varlığı bir oyun sistemi ile genel bir kişisel bilgisayar arasındaki ayırt edici bir özelliktir. Son olarak, oyun sistemindeki görüntüleme cihazı, çerçeve tamponunun içeriğini gösterir ve oyuncuya değişen bir sahne ill üzyonu verir.

Sorular ve Alıştırmalar

1. 3D grafikler kullanarak bir görüntü üretme sürecinde yer alan üç adımı öz etleyin.
2. Projeksiyon düzlemi ile görüntü penceresi arasındaki fark nedir?
3. Çerçeve tamponu nedir?

10.3 Modelleme

Bir 3D bilgisayar grafikleri projesi, bir tiyatro sahne prodüksiyonu ile çok benze r bir şekilde başlar - bir set tasarlanmalı ve gerekli aksesuarlar toplanmalı veya inşa edilmelidir. Bilgisayar grafikleri terminolojisinde, set bir sahne olarak adla ndırılır ve aksesuarlar nesneler olarak adlandırılır. Unutmayın ki, bir 3D grafik s ahnesi sanaldır çünkü nesneler, somut fiziksel yapılar yerine dijital olarak kodla nmış modeller olarak "inşa" edilmiştir.

Bu bölümde, nesneleri ve sahneleri "inşa etme" ile ilgili konuları keşfedeceğiz. Bireysel nesnelerin modelleme sorunlarıyla başlayacağız ve bu nesneleri bir sahn e oluşturmak için toplama görevini ele alarak sona ereceğiz.

Bireysel Nesnelerin Modellemesi

Bir sahne prodüksiyonunda, bir aksesuarın gerçeğe ne kadar uyduğu, sahnede nasıl kullanılacağına bağlıdır. Tam bir otomobile ihtiyacımız olmayabilir, telefo nun işlevsel olması gerekmez ve arka plan manzarası düz bir arka planda boya nmış olabilir. Benzer şekilde, bilgisayar grafikleri durumunda, bir nesnenin yazı lım modelinin nesnenin gerçek özelliklerini ne kadar doğru yansıttığı, durumun gereksinimlerine bağlıdır. Ön plandaki nesneleri modellemek için daha fazla a yrıntı gereklidir, arka plandaki nesneler için ise daha az ayrıntı yeterlidir. Ayrıca, sıkı, gerçek zamanlı kısıtlamalar altında olmayan durumlarda daha fa zla ayrıntı üretilebilir.

Bu nedenle, bazı nesne modelleri nispeten basit olabilirken, diğerleri son derece karm aşık olabilir. Genel bir kural olarak, daha hassas modeller daha yüksek kaliteli görüntülerle sonuçlanır, ancak daha uzun işleme süreleri gerektirir. Bunun sonucunda, bilgisayar grafikl eri alanındaki devam eden araştırmalar, son derece ayrıntılı, ancak verimli nesne modelleri oluşturma tekniklerinin geliştirilmesine odaklanmaktadır. Bu araştırmalardan bazıları, nes nenin sahnedeki nihai rolüne bağlı olarak farklı ayrıntı seviyeleri sağlayabilen modeller geli ştirmeyi ele almaktadır; sonuç olarak, değişen bir ortamda kullanılabilecek tek bir nesne m odeli ortaya çıkmaktadır.

Bir nesneyi tanımlamak için gereken bilgiler, nesnenin şeklinin yanı sıra, nesnenin ı şıkla nasıl etkileşime girdiğini belirleyen yüzey özellikleri gibi ek özellikleri de içerir.

Doclingo https://www.doclingo.ai

Şimdi, şekil modelleme görevini ele alalım.

Şekil 3D grafikte bir nesnenin şekli genellikle her biri bir çokgenin şekli olan k üçük düz yüzeylerin bir koleksiyonu olarak tanımlanır düz yamaçlar. Bu çokg enler topluca, tanımlanan nesnenin şeklini yaklaşık olarak temsil eden bir ço kgen ağ oluşturur (Şekil 10.3). Küçük düz yamaçlar kullanarak, yaklaşık değe r istenen kadar hassas hale getirilebilir.

Bir çokgen ağındaki düzlem yamalar genellikle üçgenler olarak seçilirçün kü her üçgen, üç köşesi ile temsil edilebilir; bu, üç boyutluuzayda düz bir yüz eyi tanımlamak için gereken minimum nokta sayısıdır.Her durumda, bir çokg en ağ, düzlem yamalarının köşelerinin bir koleksiyonu olarak temsil edilir.

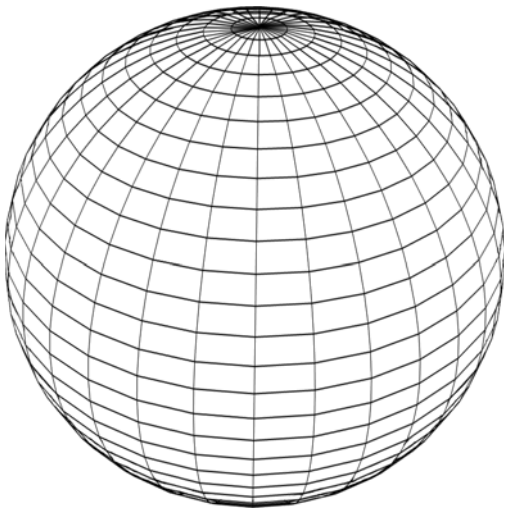
Bir nesnenin çokgen ağ temsilini çeşitli yollarla elde edebiliriz.Bunlardan biri, istenen şeklin kesin bir geometrik tanımı ile başlamaktır,ve ardından bu tanımı kullanarak bir çokgen ağ inşa etmektir. Örneğin, analitikgeometri, orij inde merkezlenmiş bir kürenin (yarıçapı r olan) aşağıdaki denklemi ile tanıml andığını söyler:

$$r^2 = x^2 + y^2 + z^2$$

Bu formüle dayanarak, küredeki enlem ve boylam çizgeleri için denklemler kurabi liriz,bu çizgelerin kesiştiği noktaları belirleyebiliriz ve ardından bu noktaları birçok gen ağın köşeleri olarak kullanabiliriz. Benzer teknikler, diğer gelenekselgeometri k şekillere de uygulanabilir; bu nedenle, daha az maliyetlibilgisayar tarafından üre tilen animasyonlardaki karakterler genellikleküreler, silindirler ve koniler gibi yapı larla bir araya getirilmiş gibi görünür.

Daha genel şekiller, daha sofistike analitik yöntemlerle tanımlanabilir. Bunlardan biri, üç boyutlu uzaydaki eğik bir çizgi parçasını tanımlamak için kullanılan Bezier eğrileri (1970'lerin başında Renault otomobil şirketinde mühendis olarak çalışan Pierre Bezier'den adını almıştır) temelinde geliştirilmiştir.

Şekil **10.3** Bir küre için çokgen ağ



Doclingo https://www.doclingo.ai

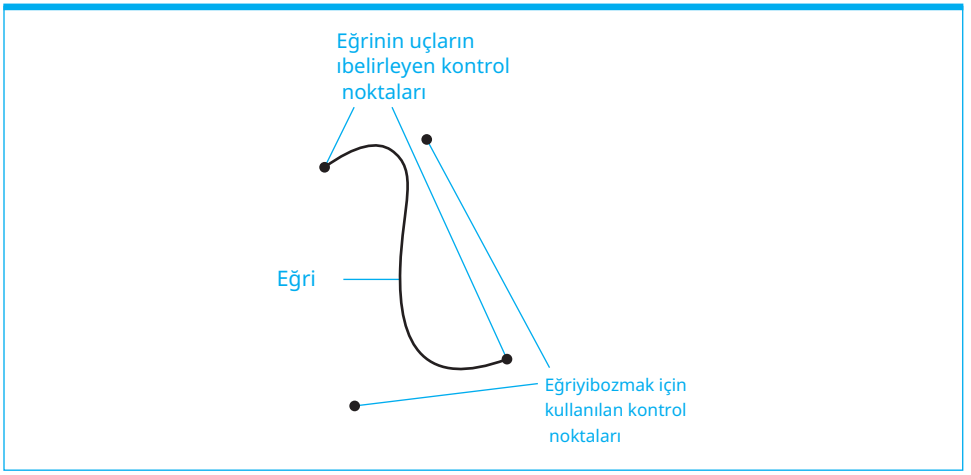
kontrol noktaları olarak adlandırılan sadece birkaç nokta ile tanımlanır—bunlardan ikisi eğri segmentinin uçlarını temsil ederken, diğerleri eğrinin nasıl bozulduğunu gösterir. Örnek olarak, Şekil 10.4 dört kontrol noktası ile tanımlanan bir eğriyi gösterme ktedir. Eğrinin, segment uçlarını tanımlamayan iki kontrol noktasına doğru çeki liyormuş gibi göründüğüne dikkat edin. Bu noktaları hareket ettirerek, eğri far klı şekillere bükülebilir. (Microsoft'un Paint gibi çizim yazılımları kullanarak eğri çizgiler oluştururken böyle tekniklerle karşılaşmış olabilirsiniz.) Burada konuyu derinlemesine incelemeyeceğiz, ancak Bezier'in eğrileri tanımlama teknikleri, B ezier yüzeyleri olarak bilinen üç boyutlu yüzeyleri tanımlamak için genişletilebili r. Bezier yüzeyleri, karmaşık yüzeyler için çokgen ağlar elde etme sürecinde ver imli bir ilk adım olarak kanıtlanmıştır.

Bir şeklin, bir kürenin özlü formülü veya bir Bezier yüzeyini tanımlayan formül ler gibi, kesin tanımını çokgen bir ağ kullanarak bir şeklin yaklaşık bir temsilisine d önüştürmenin neden gerekli olduğunu sorabilirsiniz. Cevap, tüm nesnelerin şekill erini çokgen ağlar ile temsil etmenin, render alma sürecine uniform bir yaklaşım s ağladığıdır—bu özellik, tüm sahnelerin daha verimli bir şekilde render edilmesine olanak tanır. Böylece, geometrik formüller şekillerin kesin tanımlarını sağlasa da, yalnızca çokgen ağlar inşa etmek için araçlar olarak hizmet ederler.

Poligonal bir ağ elde etmenin bir diğer yolu, ağı birzorlayıcı yöntemle inş a etmektir. Bu yaklaşım, bir şeklin zarif matematiksel tekniklerle temsil edilm esini engellediği durumlarda popülerdir. Prosedür, nesnenin fiziksel bir mod elini inşa etmek ve ardından modelin yüzeyinde noktaların konumunu, üç bo yutlu uzaydaki konumunu kaydeden bir kalem cihazıyla yüzeye dokunarak k aydetmektir—bu işleme dijitalleştirme denir. Elde edilen nokta koleksiyonu, şekli tanımlayan bir poligonal ağ elde etmek için köşe noktaları olarak kullan ılabilir.

Ne yazık ki, bazı şekiller o kadar karmaşıktır ki, geometrik modelleme vey a manuel dijitalleştirme ile gerçekçi modeller elde etmek mümkün değildir. Örnekler arasında ağaçlar gibi karmaşık bitki yapıları, dağ sıraları gibi karma şık araziler ve bulutlar, duman veya ateşin alevleri gibi gaz halindeki maddel er bulunmaktadır. Bu durumlarda, istenen şekli otomatik olarak inşa eden pr ogramlar yazarak poligonal ağlar elde edilebilir. Bu tür programlar topluca

Şekil **10.4** Bir Bezier eğrisi



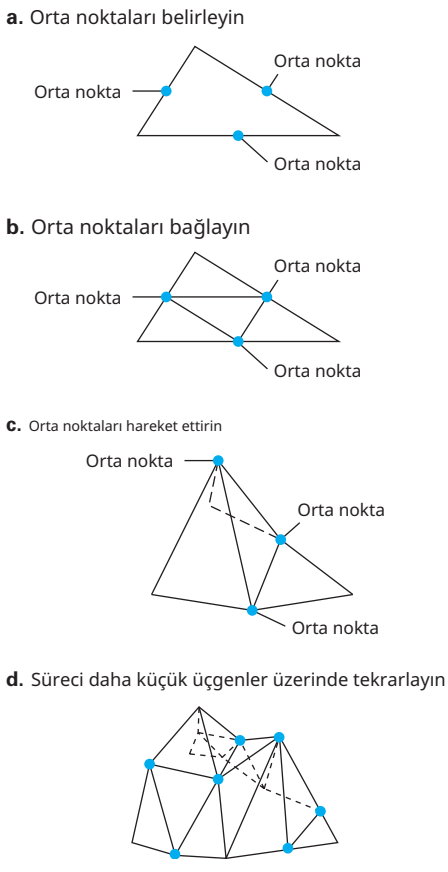
Doclingo https://www.doclingo.ai

prosedürel modellerdir. Başka bir deyişle, bir prosedürel model, istenen yapıyı oluştur mak için bir algoritma uygulayan bir program birimidir.

Örnek olarak, prosedürel modeller, aşağıdaki adımları uygulayarak dağ sıraları oluşturmak için kullanılmıştır: Tek bir üçgenle başlayarak, üçgenin kenarlarının orta noktalarını belirleyin (Şekil 10.5a). Ardından, bu orta noktaları birleştirerek toplamda dört daha küçük üçgen oluşturun (Şekil 10.5b). Şimdi, orijinal üçgenin köşelerini sabi t tutarken, orta noktaları üç boyutlu uzayda hareket ettirin (üçgenin kenar çizgilerini n uzamasına veya kısalmasına izin vererek), böylece üçgen şekillerini bozulmuş hale getirin (Şekil 10.5c). Bu süreci her bir daha küçük üçgenle tekrarlayın (Şekil 10.5d) ve istenen detay elde edilene kadar bu işlemi devam ettirin.

Prosedürel modeller, benzer ancak benzersiz olan birden fazla karmaşık nesne üretm enin etkili bir yolunu sağlar. Örneğin, bir prosedürel model, her biri kendi, ancak benzer, d allanma yapısına sahip çeşitli gerçekçi ağaç nesneleri oluşturmak için kullanılabilir. Bu tür a ğaç modellerine bir yaklaşım, dallanma kurallarını uygulayarak ağaç nesnelerini "büyütme k"tır; bu, bir ayrıştırıcının (Bölüm 6.4) dil bilgisi kurallarından bir ayrıştırma ağacı oluşturma sıyla benzer bir şekilde gerçekleşir. Aslında, bu durumlarda kullanılan dallanma kuralları ge nellikle bir dil bilgisi olarak adlandırılır. Bir dil bilgisi, çam ağaçlarını "büyütmek" için tasarla nabilirken, diğeri meşe ağaçlarını "büyütmek" için tasarlanabilir.

Şekil **10.5** Bir dağ sırası için çokgen ağ oluşturma



Doclingo https://www.doclingo.ai

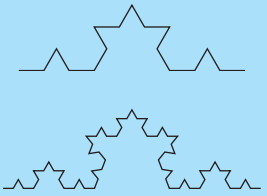
Fraktallar

Metinde tanımlandığı gibi bir prosedürel model aracılığıyla bir dağ sırasının inşası (bkz. Şekil 10.5), fraktalların 3D grafiklerindeki rolüne bir örnektir. Teknik olarak, bir fraktal, "Hausdorff boyutu topolojik boyutundan büyük olan" geometrik bir nesnedir. Bu, sezgisel olarak, nesnenin daha düşük boyutlu bir nesnenin "bir araya getirilmesi" ile oluştuğu anlamına gelir. (Birden fazla paralel çizgi segmentinin "bir araya getirilmesi" ile oluşturulan genişlik illüzyonunu düşünün.) Bir fraktal genellikle, her bir aktivasyonun "bir araya getirdiği" ek (ancak daha küçük) kopyalarla fraktalı inşa etmek için kullanılan desenin bir tekrarlama süreci aracılığıyla oluşur. Ortaya çıkan fraktal, her bölümün büyütüldüğünde kendisinin bir kopyası gibi göründüğü öz benzerlik taşır.

Bir fraktalın geleneksel bir örneği, yapıda düz çizgi segmentlerini tekrar tekrar daha küçük versiyonlarıyla değiştirilerek oluşturulan von Koch kar tanesidir.



Aynı yapının daha küçük versiyonlarıyla değiştirilmesiyle oluşur. Bu, aşağıdaki gibi bir dizi iyileştirmeye yol açar:



Fraktallar genellikle 3D grafikler alanındaki prosedürel modellerin belkemiğidir. Gerçekten de, dağ sıralarının, bitki örtüsünün, bulutların ve dumanın gerçekçi görüntülerini üretmek için kullanılmışlardır.

Prosedürel modellerin bir diğer yapım yöntemi, bir nesnenin temel yapısını büyük bir parçacık koleksiyonu olarak simüle etmektir. Bu tür modellere parçacık sistemleri denir. Genellikle parçacık sistemleri, istenen şekli üretmek için parçacıkları sistemde hareket ettirmek üzere belirli önceden tanımlanmış kurallar uygular; belki de moleküler etkileşimleri andıran bir şekilde. Örneğin, parçacık sistemleri, animasyon tartışmamızda daha sonra göreceğimiz gibi, dalgalanan suyun animasyonunu üretmek için kullanılmıştır. (Bir su kovasını, mermerlerden oluşan bir kova olarak hayal edin. Kova sallandıkça, mermerler etrafa dağılır ve suyun hareketini simüle eder.) Parçacık sistemi uygulamalarına diğer örnekler arasında titreyen ateş alevleri, bulutlar ve kabalık sahneleri bulunmaktadır.

Bir prosedürel modelin çıktısı genellikle istenen nesnenin şeklini yaklaşık olarak veren çokgen bir ağdır. Bazı durumlarda, örneğin üçgenlerden bir dağ sırası oluştururken, ağ oluşturma sürecinin doğal bir sonucudur. Diğer durumlarda, örneğin dallanma kurallarından bir ağaç büyütürken, ağ ek bir son adım olarak inşa edilebilir. Örneğin, parçacık sistemleri durumunda, sistemin dış kenarındaki parçacıklar, nihai çokgen ağın köşeleri için doğal adaylardır.

Doclingo https://www.doclingo.ai

Bir prosedürel model tarafından üretilen ağın ne kadar hassas olduğu, d uruma bağlıdır. Bir sahnedeki arka plandaki bir ağaç için bir prosedürel mod el, ağacın yalnızca temel şeklinin yansıtıldığı kaba bir ağ üretebilirken, ön pla ndaki bir ağaç için bir prosedürel model, bireysel dalları ve yaprakları ayırt e den bir ağ üretebilir.

Yüzey Özellikleri Sadece çokgen bir ağdan oluşan bir model, bir nesnenin yalnızca şek lini yakalar. Çoğu renderleme sistemi, kullanıcı tarafından talep edilen çeşitli yüzey ö zelliklerini simüle etmek için renderleme sürecinde bu tür modelleri zenginleştirme y eteneğine sahiptir. Örneğin, farklı gölgeleme teknikleri uygulayarak (Bölüm 10.4'te t anıtacağız) bir kullanıcı, bir topun çokgen ağının kırmızı pürüzsüz bir top veya yeşil p ürüzlü bir top olarak renderlenmesini belirtebilir. Bazı durumlarda, bu tür bir esnekli k arzu edilir. Ancak, orijinal bir nesnenin sadık bir şekilde renderlenmesini gerektiren durumlarda, modelde nesne hakkında daha spesifik bilgiler yer almalıdır, böylece re nderleme sistemi ne yapması gerektiğini bilecektir.

Bir nesne hakkında, şeklinin yanı sıra bilgi kodlamak için çeşitli teknikler vardır. Örneğin, bir çokgen ağdaki her bir köşe ile birlikte, o noktadaki orijin al nesnenin rengini kodlayabilirsiniz. Bu bilgi, renderleme sırasında orijinal n esnenin görünümünü yeniden oluşturmak için kullanılabilir.

Diğer durumlarda, renk desenleri bir nesnenin yüzeyi ile ilişkilendirilebilirbir süreç aracılığıyla, bu süreç doku haritalama olarak bilinir. Doku haritalama, bir nesnenin yüze yine önceden tanımlanmış bir görüntüyü ilişkilendirdiği için, duvar kağıdı uygulama sür ecine benzer. Bu görüntü, dijital bir fotoğraf, bir sanatçınınresmi veya belki de bilgisaya r tarafından üretilen bir görüntü olabilir. Geleneksel doku görüntülerikırmızı tuğla duva rlar, ahşap dokulu yüzeyler ve mermer cepheleri içerir.

Örneğin, bir taş duvar modellemek istediğimizi varsayalım. Duvarın şekli ni, uzun bir dikdörtgen katı tasvir eden basit bir çokgen ağ ile temsil edebiliri z. Ardından, bu ağ ile taş işçiliğinin iki boyutlu bir görüntüsünü sağlayabiliriz. Daha sonra, render süreci sırasında, bu görüntü dikdörtgen katıya uygulana rak bir taş duvar görünümü oluşturulabilir. Daha kesin olarak, render süreci, duvardaki bir noktanın görünümünü belirlemesi gerektiğinde, sadece işçilik görüntüsündeki karşılık gelen noktanın görünümünü kullanacaktır.

Doku haritalama, nispeten düz yüzeylere uygulandığında en iyi şekilde çalışır. Sonuç, d oku görüntüsünün eğik bir yüzeyi kaplamak için önemli ölçüde bozulması gerektiğinde (bir plaj topunu duvar kağıdı ile kaplamaya çalışmanın sorunlarını hayal edin) veya doku görünt üsü bir nesnenin etrafında tamamen sarıldığında ve doku deseninin kendisiyle karışmadığı bir dikiş oluşturduğunda yapay görünebilir. Yine de, doku haritalamanın doku simüle etme nin verimli bir yolu olduğu kanıtlanmış ve gerçek zamanlı hassasiyet gerektiren durumlarda yaygın olarak kullanılmaktadır—bunun en iyi örneği etkileşimli video oyunlarıdır.

Gerçekçilik Arayışı Gerçekçi görüntülere yol açan nesne modelleri oluşturmak, dev am eden bir araştırma konusudur. Özellikle, cilt, saç, kürk ve tüy gibi canlı karakte rlerle ilişkili malzemeler ilgi çekmektedir. Bu araştırmaların çoğu belirli bir madde ye özgüdür ve hem modelleme hem de render tekniklerini kapsamaktadır.
Örneğin, insan cildinin gerçekçi modellerini elde etmek için bazı araştırmacılar, ışığın epidermal ve dermal cilt katmanlarına ne kadar derinlemesine girdiğini ve bu katma nların içeriğinin cildin görünümünü nasıl etkilediğini dikkate almışlardır.

Doclingo https://www.doclingo.ai

Bir diğer örnek, insan saçının modellenmesini içermektedir. Saç uzaktan gö rülecekse, daha geleneksel modelleme teknikleri yeterli olabilir. Ancak, yakın çe kim görüntüler için gerçekçi görünen saç elde etmek zorlayıcı olabilir. Sorunlar arasında saydam özellikler, dokusal derinlik, dökümlü karakteristikler ve saçın r üzgar gibi dış kuvvetlere nasıl tepki verdiği yer almaktadır. Bu engelleri aşmak i çin bazı uygulamalar, saçın bireysel tellerini modellemeye başvurmuştur—bu, i nsan başındaki saç sayısının yaklaşık 100,000 olabileceği düşünüldüğünde, zorl u bir görevdir. Ancak daha da şaşırtıcı olan, bazı araştırmacıların her bir saç teli nin ölçekli dokusu, renk varyasyonu ve mekanik dinamiklerini ele alan saç mod elleri oluşturmuş olmalarıdır.

Önemli ayrıntıların takip edildiği bir diğer örnek, kumaş modellemesidir. Bu d urumda, dokuma desenlerinin karmaşıklıkları, dokuma türleri arasında uygun do kusal farklılıklar üretmek için kullanılmıştır; örneğin, kadife ile saten arasında ve i pliklerin detaylı özellikleri, örme kumaş verisi ile birleştirilerek, son derece gerçek çi yakın çekim görüntülerine yol açan örme kumaş modelleri oluşturulmuştur. Örneğin, Şrek'in kolundaki detaylara bakın, bu detaylar Şekil 10.6'da görünmekte dir (aynı zamanda, Şrek'in kucağındaki Puss in Boots adlı kedinin şapkasındaki tüy ün gerçekçiliği de). Ayrıca, fizik ve makine mühendisliği bilgisi, ipliklerin gerilmesi ve dokumanın kesilmesi gibi unsurları hesaba katarak, drape edilmiş malzemenin görüntülerini hesaplamak için bireysel ipliklere uygulanmıştır.

Gerçekçi görüntülerin üretilmesi, modelleme ve render süreçlerinde tek nikleri içeren aktif bir araştırma alanıdır.

Şekil **10.6** Dreamworks SKG tarafından üretilen *Shrek2* filminden bir sahne (© Dreamworks/Picture Desk Inc./ Kobal Koleksiyonu)



Doclingo https://www.doclingo.ai

Tipik olarak, ilerleme kaydedildikçe, yeni teknikler ilk olarak gerçek zamanlı k ısıtlamalara tabi olmayan uygulamalara entegre edilir; örneğin, modelleme/r endering süreci ile görüntülerin nihai sunumu arasında önemli bir gecikme olan sinema prodüksiyon stüdyolarının grafik yazılımlarında. (Bu tür ilerleme ler, Disney'in *Toy Story 2* (1999) karakterleri ile orijinal *Toy Story* (1995) karakte rleri arasında yakın karşılaştırmalarla gözlemlenebilir.)
Bir şey için, yeni geliştirilen teknikler, burun ile yüzün geri kalanı arasındaki s ınır gibi yüz özelliklerini temsil eden çokgen ağlar arasındaki dikişleri geliştir mek için uygulandı.) Bu yeni teknikler rafine edildikçe ve sadeleştirildikçe, ge rçek zamanlı uygulamalara girmekte ve bu ortamlardaki grafiklerin kalitesi d e artmaktadır. Gerçekçi bir şekilde sanal dünyalarla gerçek zamanlı etkileşim , gelecekte çok da uzak olmayabilir.

Tüm Sahne Modelleri

Bir sahnedeki nesneler yeterince tanımlandıktan ve dijital olarak kodlandıkta n sonra, her birine sahne içinde bir konum, boyut ve yönlendirme atanır. Bu bilgi koleksiyonu, bir veri yapısı olan bir sahne grafiği oluşturmak için birleşti rilir. Ayrıca, sahne grafiği, ışık kaynaklarını temsil eden özel nesnelere ve ka merayı temsil eden belirli bir nesneye bağlantılar içerir. Burada, kameranın k onumu, yönlendirmesi ve odak özellikleri kaydedilir.

Böylece, bir sahne grafiği geleneksel fotoğrafçılıktaki bir stüdyo kurulumuna benzer. Kamerayı, ışıkları, aksesuarları ve arka plan manzarasını içerir—deklanşör ba sıldığında fotoğrafın görünümüne katkıda bulunacak her şey—hepsi doğru y erlerinde. Fark, geleneksel bir fotoğraf set-up'ının fiziksel nesneleri içermesi, oysa bir sahne grafiğinin nesnelerin dijital olarak kodlanmış temsillerini içer mesidir. Kısacası, sahne grafiği sanal bir dünyayı tanımlar.

Sahne grafiğinin kapsamını vurgulamak için, Şekil 10.1'deki görüntüyü tekr ar düşününve onu üretmek için kullanılan sahne grafiğini hayal edin. Karakterl er, duvar,nevresim, yatak başlığı, Buzz Lightyear'ın (uzay kaşifi) arkasındaki am balaj,pencere çerçevesi, pencereden dışarıdaki ağaçlar ve ışık kaynakları, uygu n detaylarıylabirlikte modellenmiş ve sahne grafiğinde temsil edilmiştir. Aslınd a, başlangıçta tek bir yapıolarak düşünebileceğiniz nesneler, örneğin Woody (k ovboy bebeği), sahne grafiğindebirçok bireysel nesnenin birleşimi olarak temsil edilmiştir.

Kamera pozisyonunun bir sahne içindeki konumunun birçok sonucu vardır. Daha önce bahsettiğimiz gibi, nesnelerin modellenme detayları, nesnenin sahnedeki konumuna depend. Ön plan nesneleri, arka plan nesnelerinden daha fazla detay gerektirir ve ön plan ile arka plan arasındaki ayrım, kamera pozisyonuna bağlıdır. Eğer sahne, tiyatro sahne ayarına benzer bir bağlamda kullanılacaksa, o zaman ön plan ve arka planın rolleri iyi belirlenmiştir ve nesne modelleri buna göre inşa edilebilir. Ancak, eğer bağlam, farklı görüntüler için kameranın pozisyonunun değiştirilmesini gerektiriyorsa, nesne modellerinin sağladığı detay, "fotoğraflar" arasında ayarlanmak zorunda kalabilir. Bu, mevcut araştırma alanlarından biridir. Bir sahnenin, kameranın sahne içinde hareket ederken çokgen ağlarını ve diğer özelliklerini iyileştiren "akıllı" modellerden oluştuğu hayal edilmektedir.

Hareketli bir kamera senaryosunun ilginç bir örneği, bir insan kullanıcını n hayali bir üç boyutlu dünyada hareket etme hissini deneyimlemesine izin v eren sanal gerçeklik sistemlerinde ortaya çıkar. Hayali dünya, bir sahne grafi ği ile temsil edilir ve insan bu ortamı

Doclingo https://www.doclingo.ai

3D Televizyon

Televizyon bağlamında 3D görüntü üretmek için birkaç teknoloji mevcuttur, ancak hepsi aynı stereoskopik görsel etkiden yararlanmaktadır—sol ve sağ gözlere ulaşan iki hafif farklı görüntü,beyin tarafından derinlik olarak yorumlanır. Bunun için en ucuz mekanizmalar, filtre lenslere sahip özel gözlükler gerektirir.Eski renkli lensler (1950'lerde sinemada kullanılan) veya daha modern polarize lensler,bir ekranın tek bir görüntüsünün farklı yönlerini filtreleyerek, farklı gözlere farklı görüntülerin ulaşmasını sağlar.
Daha maliyetli teknoloji, sol ve sağ lensleri sırayla kapatan "aktif" gözlükleri içerir ve bu,sol ve sağ görüntüler arasında hızlı bir şekilde geçiş yapan bir 3D televizyon ile senkronize çalışır.Son olarak, özel gözlük veya başlık gerektirmeyen 3D televizyonlar geliştirilmektedir.Ekranın yüzeyinde sol ve sağ görüntüleri izleyicinin başına hafif farklı açılarda yansıtmak için karmaşık filtreler veya büyütücü lensler kullanır,bu da izleyicinin sol ve sağ gözlerinin farklı görüntüler görmesi anlamına gelir.

insanın hareketlerine göre sahnede hareket eden bir kamera aracılığıyla görür. Aslında, üç boyutun derinlik algısını sağlamak için iki kamera kullanılır: biri insanın sağ gözünü, diğeri insanın sol gözünü temsil eder. Her kameradan elde edilen görüntü, uygun gözün önünde gösterildiğinde, insan üç boyutlu sahnin içinde bulunduğu illüzyonunu elde eder ve deneyime ses ve dokunsal hisler eklendiğinde, illüzyon oldukça gerçekçi hale gelebilir.

Sonuç olarak, bir sahne grafiğinin inşasının 3D grafik sürecinde merkezi bir konumda bulunduğunu belirtmeliyiz. Nihai görüntüyü üretmek için gereken tüm bilgileri içerdiğinden, tamamlanması sanatsal modelleme sürecinin sonunu ve görüntünün işlenmesi için hesaplama yoğun sürecinin başlangıcını işaret eder. Gerçekten de, sahne grafiği inşa edildikten sonra, grafik görevi projeksiyonları hesaplamak, belirli noktalarda yüzey detaylarını belirlemek ve ışık etkilerini simüle etmekten oluşur—bu görevler büyük ölçüde belirli uygulamadan bağımsızdır.

Sorular ve Alıştırmalar

1. Aşağıdakiler, bir düzlem yaması köşelerini temsil eden dört noktadır (geleneksel dikdörtgen koordinat sistemi kullanılarak kodlanmıştır).Yamanın şeklini tanımlayın. (Analitik geometri bilgisi olmayanlar için, her üçlü,odağın köşesinden başlayarak ilgili noktaya nasıl ulaşacağınızı söyler. İlk sayı, sağınızdaki duvar ile zemin arasındaki dikiş boyunca ne kadar yürüyüş yapmanız gerektiğini söyler.İkinci sayı, solunuzdaki duvara paralel bir yönde odaya ne kadar yürüyüş yapmanız gerektiğini söyler.Üçüncü sayı, zeminden ne kadar yükseğe çıkmanız gerektiğini söyler. Eğer bir sayı negatifse, bir hayalet gibi davranmanız ve duvarlar ve zeminler üzerinden geri yürüyüş yapmanız gerektiğini hayal etmelisiniz.)

   $(0, 0, 0)$ $(0, 1, 1)$ $(0, 2, 1)$ $(0, 1, 0)$

Doclingo https://www.doclingo.ai

2. Prosedürel model nedir?
3. Bir parkta bir görüntü üretmek için kullanılan bir sahne grafiğinde bulunabil ecek bazı nesneleri listeleyin.
4. Şekillerin, geometrik denklemlerle daha hassas bir şekilde temsil edilebil eceği halde neden çokgen ağlarla temsil edildiğini açıklayın?
5. Doku haritalama nedir?

10.4 Görselleştirme

Artık, bir sahne grafiğindeki nesnelerin projeksiyon düzlemine yansıtıldığın da nasıl görüneceğini belirlemeyi içeren görselleştirme sürecini ele alma za manı geldi. Görselleştirme görevini yerine getirmenin birkaç yolu vardır. Bu bölüm, günümüzde "tüketici pazarında" en popüler grafik sistemleri (video oyunları, ev bilgisayarları vb.) tarafından kullanılan geleneksel yaklaşımı ele almaktadır. Sonraki bölüm, bu yaklaşıma iki alternatif araştırmaktadır.

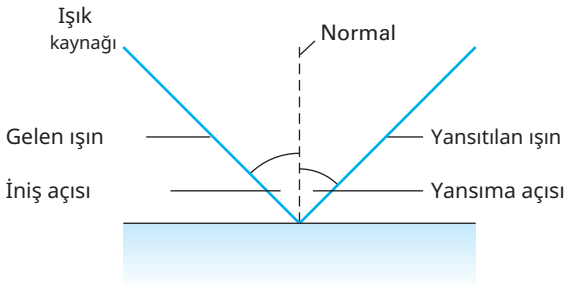
Bir nesnenin görünümü, o nesneden yayılan ışık tarafından belirlendiği nden, ışık ve nesneler arasındaki etkileşim hakkında bazı arka plan bilgileri i le başlayalım. Sonuçta, bir nesnenin görünümünü belirlemek, nihayetinde ı şığın davranışını simüle etme görevi haline gelir.

Işık-Yüzey Etkileşimi

Bir nesnenin malzeme özelliklerine bağlı olarak, yüzeyine çarpan ışık emilebilir, yüze yden yansıyarak yansıtılan ışık olarak geri dönebilir veya yüzeyden geçerek (ve bükül erek) kırılan ışık olarak geçebilir.

Yansıma Düz bir opak yüzeyden yansıyan bir ışık ışınını ele alalım. Işın, düz bir çizgide gelir ve yüzeye bir geliş açısı ile çarpar. Işının yansıdığı a çı, geliş açısıyla aynıdır. Şekil 10.7'de gösterildiği gibi, bu açılar yüzeye dik (ve ya **normal**) bir çizgiye göre ölçülür. (Bir yüzeye normal olan bir çizgi genellikl e "normal" olarak adlandırılır; "Geliş açısı normal ile ölçülür." gibi.) Gelen ışın, yansıyan ışın ve normal hepsi aynı düzlemde yer alır.

Şekil **10.7** Yansıtılan ışık



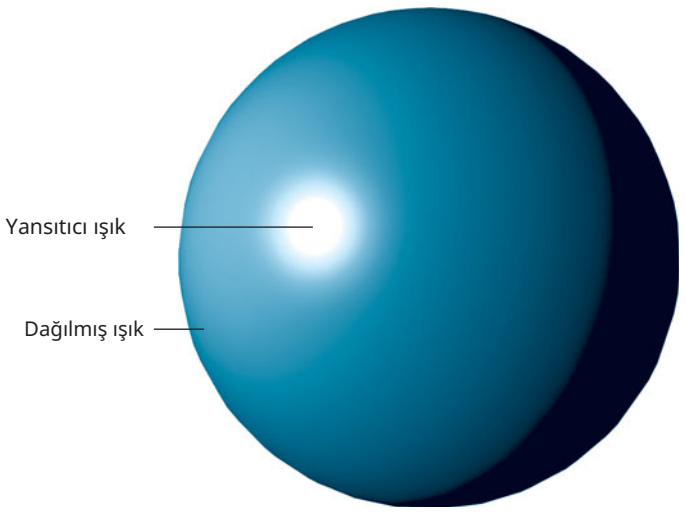
Doclingo https://www.doclingo.ai

Eğer bir yüzey pürüzsüzse, aynı ışık kaynağından gelen paralel ışınlar (ör neğin, aynı alana çarpanlar) yüzeye çarptıklarında esasen aynı yönde yansıya cak ve nesneden paralel ışınlar olarak uzaklaşacaktır. Böyle yansıtılan ışığa s pekülar ışık denir. Spekülar ışığın yalnızca yüzeyin ve ışık kaynağının yöneli minin ışığın izleyicinin yönüne yansımasına neden olduğu durumlarda gözle mlenebileceğini unutmayın. Bu nedenle, genellikle bir yüzeyde parlak vurgul ar olarak görünür. Dahası, spekülar ışığın yüzeyle minimum teması olduğun dan, orijinal ışık kaynağının rengini yansıtma eğilimindedir.

Ancak, yüzeyler nadiren mükemmel pürüzsüzdür ve bu nedenle birçok ış ın, mevcut yüzeyin yöneliminden farklı noktalara çarpabilir. Ayrıca, ışınlar ge nellikle bir yüzeyin hemen sınırını geçer ve yüzey parçacıkları arasında seker ek nihayetinde yansıtılan ışık olarak ayrılır. Sonuç olarak, birçok ışın farklı yö nlere dağılacaktır. Bu dağılmış ışığa difüz ışık denir. Spekülar ışığın aksine, difüz ışık geniş bir yön yelpazesinde görünür. Ve, yüzeyle uzun süreli teması olduğundan, difüz ışık malzemenin emme özelliklerine daha duyarlıdır ve bu nedenle nesnenin rengini yansıtma eğilimindedir.

Şekil 10.8, tek bir ışık kaynağı tarafından aydınlatılan bir topu sunmaktad ır. Top üzerindeki parlak vurgu, yansıtıcı ışıktan kaynaklanmaktadır. Işık kayn ağına bakan yarım kürenin geri kalanı, dağılmış ışık aracılığıyla görülmektedi r. Ana ışık kaynağından uzak olan yarım kürenin, o kaynaktan doğrudan yan sıyan ışıkla görünmediğini unutmayın. Topun bu kısmını görebilme yeteneği, belirli bir kaynak veya yönle ilişkili olmayan "dağınık" veya yayılmış ışık olan ortam ışığına bağlıdır. Ortam ışığıyla aydınlatılan yüzeylerin bölümleri genelli kle tek tip koyu bir renk olarak görünür.

Şekil 10.8 Yansıtıcı ve dağılmış ışık



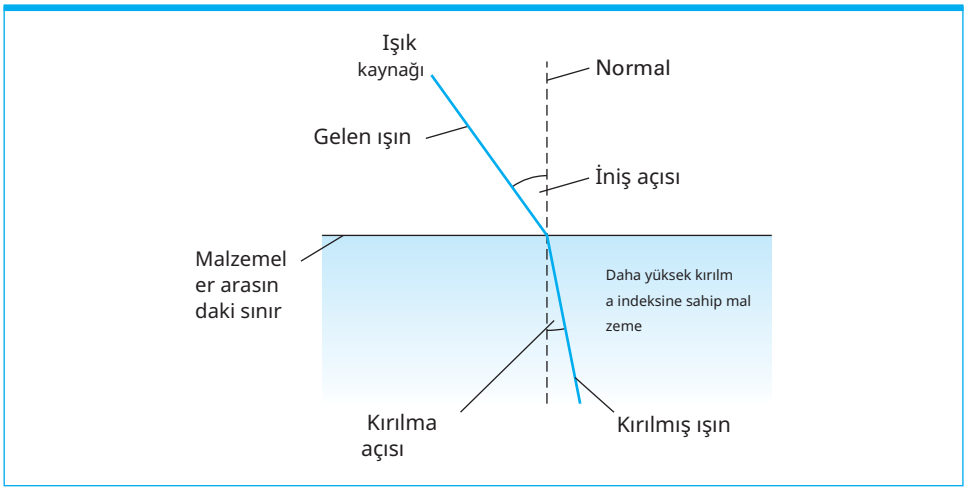
Doclingo https://www.doclingo.ai

Çoğu yüzey hem yansıtıcı hem de dağılmış ışığı yansıtır. Yüzeyin özellikleri, her birinin oranını belirler. Düz yüzeyler, dağılmış ışıktan daha fazla yansıtıcı ışık yansıttıkları için parlak görünürler. Pürüzlü yüzeyler, yansıtıcı ışıktan daha fazla dağılmış ışık yansıttıkları için mat görünürler. Ayrıca, bazı yüzeylerin küçük özellikleri nedeniyle, yansıtıcı ve dağılmış ışık oranı gelen ışığın yönüne bağlı olarak değişir. Bir yönden böyle bir yüzeye çarpan ışık, esasen yansıtıcı ışık olarak yansıtılabilirken, başka bir yönden yüzeye çarpan ışık esasen dağılmış ışık olarak yansıtılabilir. Böylece, yüzeyin görünümü döndürüldüğünde parlaklıktan matlığa kayar. Bu tür yüzeylere, yansıma desenleri simetrik olan izotropik yüzeylerin aksine, anizotropik yüzeyler denir. Anizotropik yüzeylere örnekler, kumaşta, örneğin saten gibi, kumaşın tüyünün yönüne bağlı olarak malzemenin görünümünü değiştirdiği yerlerde bulunur.Bir diğer örnek, bir atletizm sahasının çim yüzeyidir; burada çimin durumu (genellikle çimin kesilme şekliyle belirlenir) ışık ve karanlık şerit desenleri gibi anizotropik görsel etkiler üretir.

Kırılma Şimdi, ışığın opak değil, şeffaf bir nesneye çarptığını düşünün. Bu durumda, ışınlar nesnenin yüzeyinden sekmek yerine nesnenin içinden geçer. Işınlar yüzeyi deldikçe, yönleri değişir—bu olaya kırılma denir (Şekil 10.9). Kırılma derecesi, ilgili malzemelerin kırılma indisi ile belirlenir. Kırılma indisi, malzemenin yoğunluğu ile ilişkilidir. Yoğun malzemeler, daha az yoğun malzemelere göre genellikle daha yüksek bir kırılma indeksine sahiptir. Bir ışık ışını, daha yüksek bir kırılma indeksine sahip bir malzemeye (örneğin, havadan suya geçerken) girdiğinde, giriş noktasında normale doğru bükülür. Daha düşük bir kırılma indeksine sahip bir malzemeye geçerse, normalden uzaklaşır.

Şeffaf nesneleri doğru bir şekilde renderlemek için, render yazılımının ilgili malzemelerin kırılma indekslerini bilmesi gerekir. Ancak bu, tüm hikaye değildir. Render yazılımı ayrıca bir nesnenin yüzeyinin hangi tarafının iç kısmı, hangisinin dış kısmı olduğunu da bilmelidir.

Şekil **10.9** Kırılmış ışık



Doclingo https://www.doclingo.ai

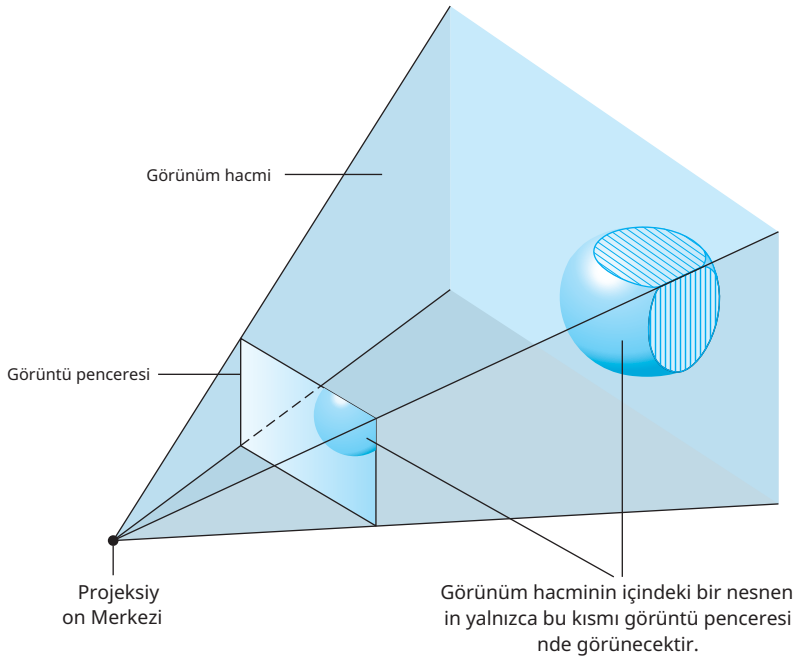
Bir nesnenin iç kısmını dış kısmına karşı ayırt etmek için, ışık nesneye mi giriy or yoksa nesneden mi çıkıyor? Bu bilgiyi elde etme teknikleri bazen oldukça i nceliklidir. Örneğin, her çokgen ağdaki çokgenin köşelerini dışarıdan bakıldı ğında saat yönünün tersine sıralamayı kabul edersek, bu liste, yamanın hang i tarafının nesnenin dışını temsil ettiğini akıllıca kodlar.

Kırpma, Tarama Dönüşümü ve Gizli Yüzeylerin Kaldırılması

Şimdi bir sahne grafiğinden bir görüntü üretme sürecine odaklanalım. Şu anda izl ediğimiz yaklaşım, çoğu etkileşimli video oyunu sistemlerinde kullanılan teknikleri takip etmektir. Bu teknikler topluca, renderlama boru hattı olarak bilinen yerleşi k bir paradigmayı oluşturur. Bu bölümün sonunda bu yaklaşımın bazı artılarını ve eksilerini gözden geçireceğiz ve bir sonraki bölümde iki alternatif keşfedeceğiz. Ş u anda, renderlama boru hattının opak nesnelerle ilgilendiğini ve bu nedenle kırıl manın bir sorun olmadığını belirtmek faydalıdır. Ayrıca, nesneler arasındaki etkile şimleri göz ardı eder, bu nedenle şu anda aynalar ve gölgelerle ilgilenmeyeceğiz.

Renderlama boru hattı, kameranın "görebileceği" nesneleri (veya nesneleri n parçalarını) içeren üç boyutlu bir sahnedeki bölgeyi tanımlayarak başlar. Bu b ölge, görünüm hacmi olarak adlandırılır ve görüntü penceresinin köşelerinden projeksiyon merkezine doğru uzanan düz çizgilerle tanımlanan piramidin içind eki alandır (Şekil 10.10).

Şekil **10.10** Görünüm hacminin içinde yer alan sahne bölgesinin tanımlanması



Doclingo https://www.doclingo.ai

Aliaslama

Şeritli gömleklerin ve kravatların televizyonda garip "parlak" bir görünüme sahip olduğunu hiç fark ettiniz mi? Bu, istenen görüntüdeki bir desenin, görüntüyü oluşturan piksellerin yoğunluğu ile uygunsuz bir şekilde örtüşmesi sonucu ortaya çıkan aliasing adı verilen bir fenomenin sonucudur. Örneğin, istenen görüntünün bir kısmı sırayla siyah ve beyaz şeritlerden oluşuyorsa, ancak tüm piksellerin merkezi yalnızca siyah şeritlere denk geliyorsa, nesne tamamen siyah olarak render edilir. Ancak, nesne hafifçe hareket ederse, tüm piksellerin merkezi beyaz şeritlere denk gelebilir, bu da nesnenin aniden beyaza dönüşeceği anlamına gelir. Bu sinir bozucu etkiyi telafi etmenin çeşitli yolları vardır. Bunlardan biri, her pikseli görüntüdeki küçük bir alanın ortalaması olarak render etmektir, tam bir tek noktanın görünümü yerine.

Görüş hacmi belirlendikten sonra, görüş hacmi ile kesişmeyen nesneleri veya nesne parçalarını dikkate almaktan çıkarmak gerekir.
Sonuçta, sahnenin o kısmının projeksiyonu görüntü penceresinin dışına düşecek ve bu nedenle son görüntüde görünmeyecektir. İlk adım, tamamen görüş hacminin dışında olan tüm nesneleri çıkarmaktır. Bu süreci kolaylaştırmak için, sahne grafiği, sahnenin farklı bölgelerindeki nesnelerin farklı dallarda saklandığı bir ağaç yapısında düzenlenebilir. Bu sayede, sahne grafiğinin büyük bölümleri, ağaçtaki tüm dalları yok sayarak kolayca çıkarılabilir.

Görüş hacmi ile kesişmeyen nesneleri belirledikten ve çıkardıktan sonra, kalan nesneler, her nesnenin görüş hacminin dışına düşen kısmını kesen bir işlem olan kesme (clipping) ile düzenlenir. Daha kesin olarak, kesme, her bir düzlem parçasını görüş hacminin sınırları ile karşılaştırma ve dışarıda kalan parçaları kesme işlemidir. Sonuç, tamamen görüş hacminin içinde yer alan çokgen ağlar (muhtemelen kesilmiş düzlem parçaları) olacaktır.

Renderleme borusundaki bir sonraki adım, kalan düz yamaçlardaki noktaların son görüntüdeki piksel pozisyonlarıyla ilişkilendirilmesini belirlemektir. Bu noktaların yalnızca bunların son resme katkıda bulunacağını anlamak önemlidir. Bir nesnedeki bir detay piksel pozisyonları arasında kalıyorsa, bir piksel tarafından temsil edilmeyecek ve dolayısıyla son görüntüde görünmeyecektir. Bu nedenle, piksel sayıları dijital kamera pazarında yoğun bir şekilde reklam edilmektedir. Daha fazla piksel varsa, küçük detayların bir fotoğrafta yakalanma olasılığı o kadar yüksektir.

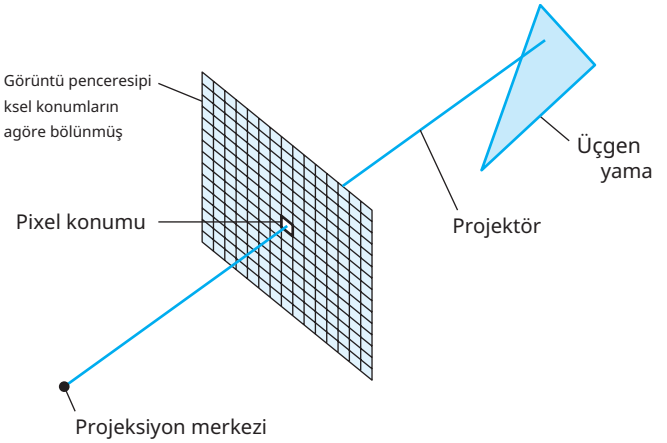
Piksel pozisyonlarını sahnedeki noktalarla ilişkilendirme sürecine tarama dönüşümü denir (çünkü yamaları tarama çizgileri olarak adlandırılan yatay piksel satırlarına dönüştürmeyi içerir) veya rasterizasyon (çünkü bir piksel dizisi raster olarak bilinir) denir. Tarama dönüşümü, projeksiyon merkezinden her piksel pozisyonuna doğru düz çizgiler (projeksiyonlar) uzatarak ve ardından bu projeksiyonların düz yamalarla kesiştiği noktaları belirleyerek gerçekleştirilir. İşte bu noktalar, yamalar üzerinde bir nesnenin görünümünü belirlememiz gereken noktalardır. Gerçekten de, bunlar son görüntüdeki pikseller tarafından temsil edilecek noktalardır.

Doclingo https://www.doclingo.ai

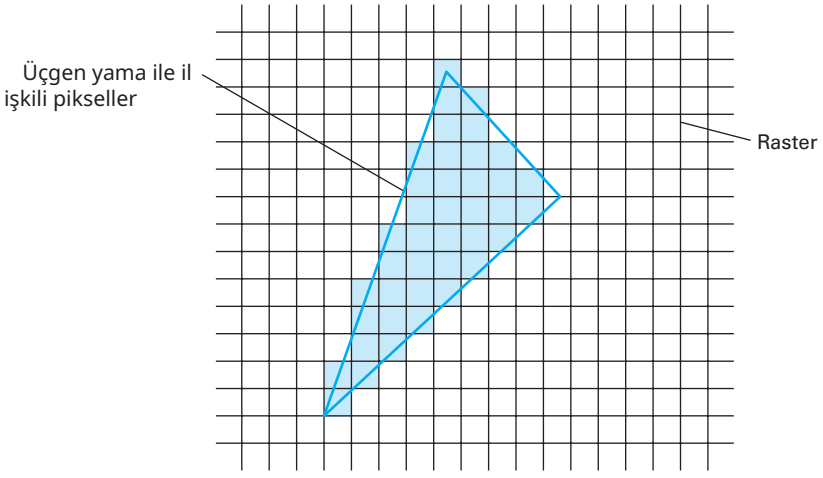
Şekil 10.11, tek bir üçgen yamanın tarama dönüşümünü göstermektedir. Şek lin a kısmı, bir projektörün bir piksel konumunu yamadaki bir nokta ile nasıl ilişki lendirdiğini göstermektedir. b kısmı, tarama dönüşümü ile belirlenen yamanın p iksel görüntüsünü göstermektedir. Tüm piksel dizisi (raster), bir ızgara ile temsil edilir ve üçgenle ilişkili pikseller gölgelenmiştir. Şeklin ayrıca, piksellerin boyutun a göre küçük olan bir şeklin tarama dönüşümünde meydana gelebilecek bozulm ayı da gösterdiğini unutmayın. Bu tür dişli kenarlar, çoğu kişisel bilgisayar ekranı kullanıcıları için tanıdık bir görüntüdür.

Ne yazık ki, bir sahnenin (veya tek bir nesnenin) tarama dönüşümü, tek bir yamanın ta rama dönüşümünden o kadar da basit değildir. Bunun nedeni, birden fazla yamanın söz ko nusu olduğu durumlarda, bir yamanın diğerinin görünümünü engelleyebilmesidir. Böylece, bir projektör düz bir yama ile kesişse bile, yamadaki o nokta son gö rüntüde görünmeyebilir. Noktaları tanımlamak ve atmak

Şekil **10.11** Üçgen bir yamanın tarama dönüşümü



a. Tarama dönüşüm süreci



b. Üçgen yamayı "projeksiyon şekli" olarak gösteren raster

Doclingo https://www.doclingo.ai

Görüşten engellenen bir sahne, gizli yüzeykaldırma olarak adlandırılan süreç tir.

Gizli yüzey kaldırmanın belirli bir versiyonu, arka yüz eliminasyonudur, bu, bir nesnenin "arka tarafını" temsil eden çokgen bir ağdaki yamanın kaldırılmasını içerir. Arka yüz eliminasyonunun görece basit olduğunu unutmayın çünkü bir nesnenin arka tarafındaki yamalar, yönlerinin kameradan uzaklaştığı yamalar olarak tanımlanabilir.

Ancak, tam gizli yüzey kaldırma problemini çözmek, arka yüz eliminasyonu ndançok daha fazlasını gerektirir. Örneğin, bir binanın önünde bir otomobilinb ulunduğu bir sahneyi hayal edin. Hem otomobilden hem de binadan gelen düz yamalar, görüntü penceresinin aynı alanına projekte edilecektir. Üst üstebinen yerlerde, çerçeve tamponunda nihayetinde saklanan pikseldatası, ön plandaki nesnenin (otomobil) görünümünü göstermelidir,arka plandaki nesnenin (bina) değil. Kısacası, bir projektör birden fazla düzyamayla kesişiyorsa, görüntü penc eresine en yakın olan yamadakinokta render edilmelidir.

Bu "ön plan/arrière plan" sorununu çözmenin basit bir yaklaşımı,"ressam algoritması" olarak bilinir,nesneleri sahnedeki mesafelerine göre düzenlem ek ve daha uzak nesneleri önce tarayıp dönüştürmektir,bu sayede daha yakı n nesnelerin tarama sonuçları önceki sonuçları geçersiz kılabilir. Ne yazık ki, ressam algoritması,nesnelerin iç içe geçtiği durumları ele almakta başarısızd ır. Bir ağacın bir kısmı başka bir nesnenin arkasında olabilirken,ağacın diğer bir kısmı o nesnenin önünde olabilir.

"Ön plan/arrière plan" sorununa daha kapsamlı çözümler,tam nesneler y erine bireysel piksellere odaklanarak elde edilir. Popüler bir teknik,"z-buffer" (aynı zamanda derinlik tamponu olarak da bilinir) adı verilen ek bir depolam a alanı kullanır,bu alan görüntüdeki her piksel için bir giriş içerir (veya eşdeğ er olarak, çerçeve tamponundaki her piksel girişi). Z-buffer'daki her pozisyon ,çerçeve tamponundaki karşılık gelen girişle şu anda temsil edilen nesneye,k ameradan nesneye uygun projektör boyunca mesafeyi depolamak için kulla nılır. Böylece, bir z-buffer yardımıyla,"ön plan/arrière plan" sorunu, yalnızca o piksel için veri henüz çerçeve tamponuna yerleştirilmemişse veyaşu anda di kkate alınan nesnedeki nokta, z-buffer'da kaydedilen mesafe bilgisine göre,ö nceki render edilen nesneden daha yakınsa, bir pikselin görünümünü hesapl ayarak ve depolayarak çözülebilir.

Daha kesin olarak, bir z-buffer kullanıldığında, render alma süreci şu şekilde il erleyebilir:Z-buffer'daki tüm girişleri, bir nesnenin render almak için dikkate alına cağı maksimum mesafeyi temsil eden bir değere ayarlayın. Ardından, her seferin de bir düzlem yaması üzerindeki yeni bir nokta render almak için dikkate alındığın da, önce bu noktanın kameradan uzaklığını, mevcut piksel konumuyla ilişkili z-buf fer'daki değerle karşılaştırın. Eğer bu mesafe z-buffer'da bulunan değerden daha azsa, noktanın görünümünü hesaplayın, sonuçları çerçeve tamponuna kaydedin v e z-buffer'daki eski girişi, yeni render edilen noktaya olan mesafe ile değiştirin. (N oktanın mesafesi z-buffer'da bulunan değerden büyükse, hiçbir işlem yapılmasına gerek yoktur çünkü yamanın üzerindeki nokta çok uzakta veya daha önce render edilmiş daha yakın bir nokta tarafından görünümden engellenmiştir.)

Gölgeleme

Bir tarama dönüşümü, nihai görüntüde görünmesi gereken bir düzlem yaması üz erinde bir noktayı tanımladıktan sonra, renderleme görevi, o noktadaki yamayı na sıl görüneceğini belirlemek olur. Bu işleme *gölgeleme* denir. Gölgelemenin

Doclingo https://www.doclingo.ai

kameraya doğru yansıtılan ışığın özelliklerini hesaplamak, söz konusu nokta daki yüzeyin yönüne bağlıdır. Sonuçta, o noktadaki yüzeyin yönü, kameranın gördüğü yansıtıcı, difüz ve ortam ışığının derecesini belirler.

Gölgeleme sorununa basit bir çözüm, düz gölgeleme olarak adlandırılır veher bir noktanın yönünü, düz bir yamanın yönü olarak kullanmaktır—yani, her yamanın üzerindeki yüzeyin düz olduğunu varsaymaktır. Ancak sonuç, n ihai görüntünün Şekil 10.12'de gösterildiği gibi keskin görüneceği, Şekil 10.8' de gösterildiği gibi yuvarlak değil. Bir anlamda, düz gölgeleme, ağın modelle diği nesne yerine çokgen ağın kendisinin bir görüntüsünü üretir.

Daha gerçekçi bir görüntü üretmek için, render alma süreci, bireysel düz yamaların görünümünü düzgün bir şekilde eğrilen bir yüzeye harmanlamalı dır. Bu, her bir render edilen noktada orijinal yüzeyin gerçek yönünü tahmin ederek gerçekleştirilir.

Böyle tahmin şemaları genellikle çokgen ağın köşelerindeki yüzey yönün ü gösteren verilerle başlar. Bu verileri elde etmenin birkaç yolu vardır. Bir yol , orijinal yüzeyin her köşedeki yönünü kodlamak ve bu veriyi modelleme sür ecinin bir parçası olarak çokgen ağa eklemektir. Bu, her köşeye eklenmiş okl ar olan normal vektörler ile bir çokgen ağ üretir. Her normal vektör, orijinal yüzeye dik yönde dışa doğru işaret eder. Sonuç, Şekil 10.13'te gösterildiği gi bi hayal edilebilecek bir çokgen ağdır. (Başka bir yaklaşım, bir köşeye bitişik her yamayı yönünü hesaplamak ve ardından bu yönlerin bir "ortalamasını" k öşedeki yüzeyin yönü olarak tahmin etmek olabilir.)

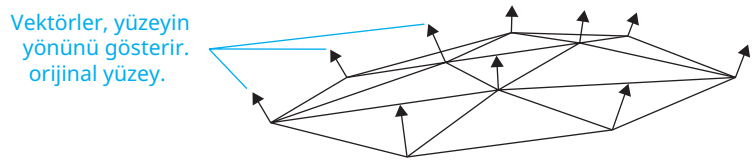
Orijinal yüzeyin bir çokgen ağının köşelerindeki yönelimi nasıl elde edilirse edilsi n, bu veriye dayalı olarak bir düzlem yamanın gölgelendirilmesi için birkaç strateji m evcuttur. Bunlar arasında Gouraud gölgelendirme ve Phong gölgelendirme bulunm aktadır; aralarındaki fark ince bir ayrıntıdır. Her ikisi de bir yamadaki köşelerin yüzey yönelimi hakkında bilgi kullanarak yüzeyi yaklaşık olarak tanımlamaya başlar.

Şekil **10.12** Düz gölgelendirme ile render edildiğinde bir kürenin görünümü



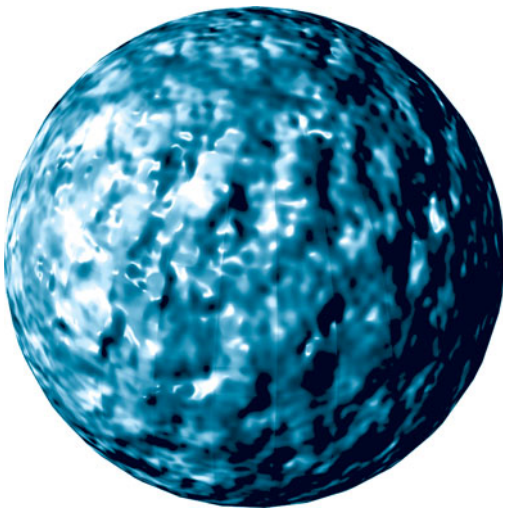
Doclingo https://www.doclingo.ai

Şekil **10.13** Dikdörtgen bir ağın köşelerindeki normal vektörlerle kavramsal görünümü



yamaların sınırları boyunca yönelim. Gouraud gölgelendirmesi, bu bilgiyi kulla narak yamaların sınırları boyunca yüzeyin görünümünü belirler ve nihayetinde, bu sınır görünümünü iç kısımdaki noktaların yüzey görünümünü tahmin etme k için iç içe geçirir. Buna karşılık, Phong gölgelendirmesi, yamaların sınırları bo yunca yüzeyin yönelimini iç içe geçirerek yamaların iç kısmındaki noktalar için y üzey yönelimini tahmin eder ve ancak o zaman görünüm sorularını dikkate alır. (Kısacası, Gouraud gölgelendirmesi yönelim bilgisini renk bilgisine dönüştürür ve ardından renk bilgisini iç içe geçirir. Phong gölgelendirmesi, ilgili noktanın y önelimi tahmin edilene kadar yönelim bilgisini iç içe geçirir ve ardından o yönel im bilgisini renk bilgisine dönüştürür.) Sonuç olarak, Phong gölgelendirmesi, y üzey yönelimindeki değişikliklere daha duyarlı olduğu için bir yamadaki iç kısım da yansıtıcı ışığı tespit etme olasılığı daha yüksektir. (Bu bölümün sonunda Sor u 3'e bakın.) Son olarak, temel gölgelendirme tekniklerinin bir yüzeye doku gör ünümü eklemek için artırılabileceğini belirtmeliyiz.Bir örnek, bump mappi ng olarak adlandırılan, bir yüzeyin görünür yöneliminde küçük değişiklikler üre terek yüzeyin pürüzlü görünmesini sağlama yöntemidir. Daha kesin olarak, bu mp mapping, geleneksel gölgelendirme algoritmaları tarafından uygulanan iç i çe geçirme sürecine bir rastgelelik derecesi ekler, böylece genel yüzeyin dokuy a sahip olduğu izlenimi verir; bu, Şekil 10.14'te gösterilmiştir.

Şekil **10.14** Yüzey haritalama kullanılarak render edildiğinde görünebilecek bir küre



Doclingo https://www.doclingo.ai

Renderlama-Pipeline Donanımı

Daha önce de söylediğimiz gibi, kesme, tarama dönüşümü, gizli-yüzey kaldırma v e gölgeleme işlemleri, renderleme boru hattı olarak bilinen bir dizi olarak topluca görülmektedir. Ayrıca, bu görevleri yerine getirmek için verimli algoritmalar iyi bili nmektedir ve doğrudan elektronik devrelerde uygulanmıştır; bu da VLSI teknoloji si ile miniaturize edilerek tüm renderleme boru hattını otomatik olarak gerçekleşt iren çipler üretmiştir. Bugün, hatta ucuz örnekler bile saniyede milyonlarca düzle m yaması render edebilme kapasitesine sahiptir.

Grafik uygulamaları için tasarlanmış çoğu bilgisayar sistemi, video oyun ma kineleri de dahil olmak üzere, bu cihazları tasarımlarında entegre etmektedir. Daha genel amaçlı bilgisayar sistemleri durumunda, bu teknoloji, bilgisayarın v eriyolu ile özel bir denetleyici olarak bağlanan bir grafik kartı veya grafik adapt örü şeklinde eklenebilir (bkz. Bölüm 2). Bu tür donanım, renderleme sürecini g erçekleştirmek için gereken süreyi önemli ölçüde azaltır.

Render-pipeline donanımı, grafik uygulama yazılımının karmaşıklığını da az altır. Temelde, tüm yazılımın yapması gereken, grafik donanımına sahne grafiği ni sağlamaktır. Donanım daha sonra pipeline adımlarını gerçekleştirir ve sonuç ları çerçeve tamponuna yerleştirir. Böylece, yazılımın perspektifinden, tüm ren der pipeline'ı, donanımı soyut bir araç olarak kullanarak tek bir adıma indirgeni r.

Örneğin, etkileşimli bir video oyununu tekrar ele alalım. Oyunu başlatma k için, oyun yazılımı sahne grafiğini grafik donanımına aktarır. Donanım dah a sonra sahneyi render eder ve görüntüyü çerçeve tamponuna yerleştirir; bu radan otomatik olarak monitör ekranında görüntülenir. Oyun ilerledikçe, oy un yazılımı yalnızca grafik donanımındaki sahne grafiğini güncelleyerek değiş şen oyun durumunu yansıtır ve donanım sahneyi tekrar tekrar render eder; her seferinde güncellenmiş görüntüyü çerçeve tamponuna yerleştirir.

Ancak, farklı grafik donanımlarının yetenekleri ve iletişim özelliklerinin ö nemli ölçüde değiştiğini belirtmeliyiz. Bu nedenle, bir video oyunu gibi bir uy gulama belirli bir grafik platformu için geliştirilmişse, başka bir ortama aktarı lırsa değiştirilmesi gerekecektir. Grafik sistemlerinin spesifik detaylarına olan bu bağımlılığı azaltmak için, grafik donanımı ile uygulama yazılımı arasında aracılık rolü oynayan standart yazılım arayüzleri geliştirilmiştir. Bu arayüzler, standartlaştırılmış komutları belirli bir grafik donanım sistemi için gereken ö zel talimatlara dönüştüren yazılım rutinlerinden oluşur. Örnekler arasında **OpenGL** (Open için kısaltma) bulunmaktadır.

Grafik Kütüphanesi), SiliconGraphics tarafından geliştirilen ve video oyun endüstrisin de yaygın olarak kullanılan bir özel olmayan sistemdir ve **Direct3D,** Microsoft tarafın dan Microsoft Windows ortamlarında kullanılmak üzere geliştirilmiştir.

Sonuç olarak, renderlama boru hattıyla ilişkili tüm avantajların yanı sıra, en önemli dezavantajlardan biri de boru hattının yalnızca yerel aydınlatma modelini uygulamasıdır; bu, boru hattının her nesneyi diğer nesnelerden ba ğımsız olarak renderlaması anlamına gelir.

Yani, yerel aydınlatma modelinde, her nesne, sahnedeki tek nesneymiş gibi ışık kaynaklarıyla ilişkili olarak renderlanır. Sonuç olarak, nesneler arasındaki ışık e tkileşimleri, gölgeler ve yansımalar gibi, yakalanmaz. Bu, nesneler arasındaki et kileşimlerin dikkate alındığı küresel aydınlatma modeline karşıt bir durumdur. Küresel aydınlatma modelini uygulamak için iki tekniği tartışacağız.

Doclingo https://www.doclingo.ai

Bu bölümde küresel aydınlatma modelini uygulamak için teknikler bulunmaktadır. Şu anda, bu tekniklerin mevcut teknolojinin gerçek zamanlı yeteneklerinin dışında olduğunu yalnızca belirtelim.

Ancak, renderlama boru hattı donanımını kullanan sistemlerin bazı küres el aydınlatma efektleri üretemeyeceği anlamına gelmez. Gerçekten de, yerel aydınlatma modelinin getirdiği bazı kısıtlamaları aşmak için akıllı teknikler ge liştirilmiştir. Özellikle, yere düşen gölgeler olarak bilinen düşen gölgelerin g örünümü, gölgeyi atan nesnenin poligonal ağının bir kopyasını yaparak, kop ya ağı düzleştirerek, yere yerleştirerek ve koyu renkle boyayarak yerel aydınl atma modeli bağlamında simüle edilebilir. Diğer bir deyişle, gölge, başka bir nesneymiş gibi modellenir ve ardından geleneksel renderlama boru hattı do nanımı tarafından renderlanarak bir gölge illüzyonu oluşturulabilir. Bu tür te knikler, etkileşimli video oyunları gibi "tüketici düzeyindeki" uygulamalar ve uçuş simülatörleri gibi "profesyonel düzeydeki" uygulamalar arasında popül erdir.

Sorular ve Alıştırmalar

1. Spekülar ışık, difüz ışık ve ortam ışığı arasındaki farkı özetleyin.
2. Kesme ve tarama dönüşümü terimlerini tanımlayın.
3. Gouraud gölgeleme ve Phong gölgelemesi şu şekilde özetlenebilir: Gouraud gölgelemesi, bir nesnenin yüzeyinin bir yamanın sınırları bo yunca yönünü kullanarak yüzeyin görünümünü belirler ve ardından bu görünümleri yamanın iç kısmında belirli noktaların görünümünü belirlemek için iç içe geçirir. Phong gölgelemesi, sınır yönlerini içe do ğru noktaların yönlerini hesaplamak için iç içe geçirir ve ardından bu yönleri belirli noktaların görünümünü belirlemek için kullanır. Bir nes nenin görünümü nasıl farklılık gösterebilir?
4. Renderlama boru hattının önemi nedir?
5. Bir ayna üzerindeki yansımaların yerel aydınlatma modeli kullanılarak nasıl simüle edilebilec eğini tanımlayın.

10.5 Küresel Aydınlatma ile İlgilenme

Araştırmacılar şu anda renderlama boru hattına iki alternatif üzerinde çalışıy orlar; her ikisi de küresel aydınlatma modelini uygular ve böylece geleneksel boru hattındaki yerel aydınlatma modeli kısıtlamalarını aşma potansiyeli su nar. Bu alternatiflerden biri ışın izleme, diğeri ise radyositedir. Her ikisi de titi z ve zaman alıcı süreçlerdir, şimdi göreceğimiz gibi.

Işın İzleme

Işın izleme esasen bir ışın kaynağını bulmak için bir ışın izinin geriye doğru takip e dilmesi sürecidir. Süreç, işlenecek bir pikselin seçilmesi, o pikselin ve projeksiyon merkezinin üzerinden geçen düz çizginin tanımlanmasıyla başlar ve ardından

Doclingo https://www.doclingo.ai

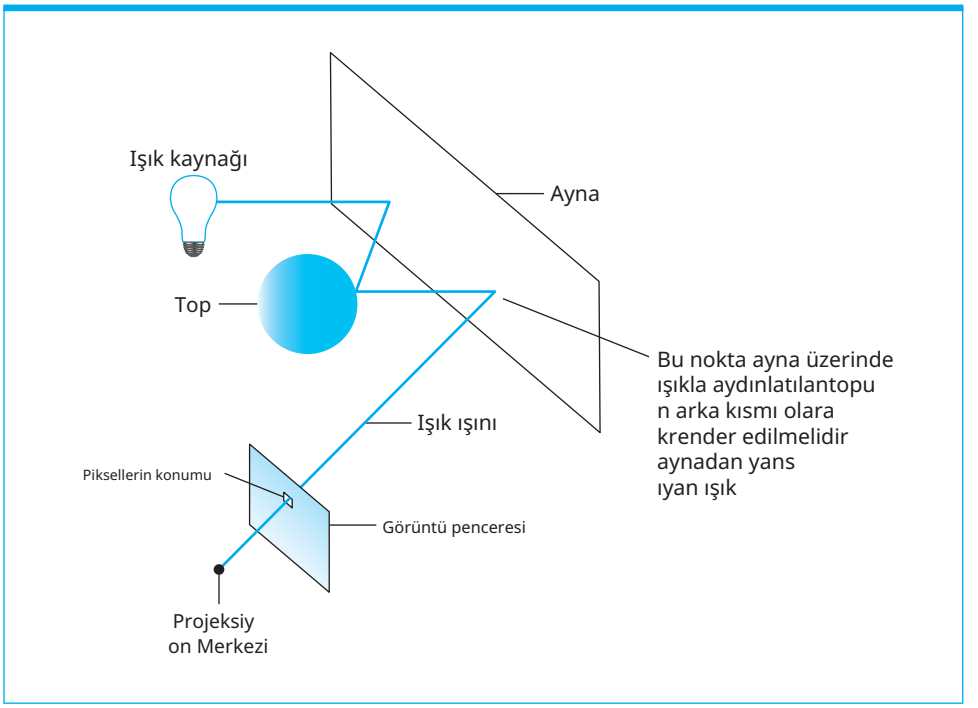
o çizgi boyunca görüntü penceresine çarpan ışın izinin izlenmesiyle devam eder. Bu izleme süreci, çizgiyi sahneye doğru takip etmeyi ve bir nesneyle temas edene kadar devam etmeyi içerir. Eğer o nesne bir ışık kaynağıysa, ışın izleme süreci son a erer ve piksel, ışık kaynağındaki bir nokta olarak işlenir. Aksi takdirde, nesnenin yüzeyinin özellikleri, sürecin geriye doğru izlediği ışın izini üretmek için yansıtılan gelen ışın izinin yönünü belirlemek amacıyla değerlendirilir. Süreç daha sonra o g elen ışın izini geriye doğru takip ederek kaynağını bulur; bu noktada başka bir ışın tanımlanabilir ve izlenebilir.

Bir ışın izleme örneği Şekil 10.15'te gösterilmektedir; burada bir ışının görü ntü penceresi boyunca geriye doğru izlenerek bir aynanın yüzeyine ulaştığını g örüyoruz. Oradan ışın, parlak bir topa izlenir, oradan tekrar aynaya, ve aynada n ışık kaynağına izlenir. Bu izleme sürecinden elde edilen bilgilere dayanarak, g örüntüdeki piksel, aynada yansıtılan ışık kaynağı tarafından aydınlatılan top üz erindeki bir nokta olarak görünmelidir.

Ray tracing'in bir dezavantajı, yalnızca spekülar yansımaları izlemesidir. Bu nedenle, bu yöntemle işlenen tüm nesneler genellikle parlak bir görünüme sahip olur. Bu etkiyi önlemek için, dağıtılmış ray tracing olarak adlandırılan bir ray tracing varyasyonu uygulanabilir. Fark, bir yansıma noktasından geriye doğru tek bir ışın izlemek yerine, dağıtılmış ray tracing'in o noktadan birden fazla ışın izlemesidir; her biri hafifçe different bir yönde uzanır.

Temel ray tracing'e uygulanabilir bir diğer varyasyon, şeffaf nesnelerin söz konusu olduğu durumlarda geçerlidir. Bu durumda, her bir ışın bir yüzeye izlenirken iki etki dikkate alınmalıdır. Biri yansıma, diğeri kırılmadır. Örneğin, Şekil 10.1'de Buzz'ın (uzay kaşifi) kaskının şeffaf görünümünü ve kaskın üst yüzeyine yakın spekülar vurguları not edin. Bu durumda, görev

Şekil **10.15** Ray tracing



Doclingo https://www.doclingo.ai

orijinal ışının izlenmesi iki göreve ayrılır: yansımanın geriye doğru izlenmesi ve kırılmanın geriye doğru izlenmesi.

Ray tracing genellikle her aktivasyonun bir ışın izlemek için kaynağına yönl endirilmesiyle özyinelemeli olarak uygulanır. İlk aktivasyon, ışınını parlak bir op ak yüzeye izleyebilir. O noktada, ışınının gelen bir ışının yansıması olduğunu ta nır, o gelen ışının yönünü hesaplar ve o gelen ışını izlemek için başka bir aktiva syonu çağırır. Bu ikinci aktivasyon, ışınının kaynağını bulmak için benzer bir gör ev gerçekleştirir - bu süreç, hala başka aktivasyonların çağrılmasına neden olab ilir.

Özyinelemeli ışın izlemeyi sonlandırmak için çeşitli koşullar kullanılabilir. İzlenen ışın bir ışık kaynağına ulaşabilir, izlenen ışın bir nesneye çarpmadan sahneden çıkabilir veya aktivasyon sayısı önceden belirlenmiş bir limite ulaşa bilir. Başka bir sonlandırma koşulu, karşılaşılan yüzeylerin emme özelliklerin e dayanabilir. Eğer bir yüzey yüksek derecede emici ise, örneğin koyu mat bi r yüzey, o zaman gelen herhangi bir ışın yüzeyin görünümünde çok az etki y apacak ve ışın izleme durabilir. Birikimli emme benzer bir etki yaratabilir. Ya ni, ışın izleme, birkaç orta derecede emici yüzeyi ziyaret ettikten sonra sona erebilir.

Küresel aydınlatma modeline dayandığı için, ışın izleme geleneksel renderlam a hattındaki birçok kısıtlamayı aşar. Örneğin, gizli yüzeylerin kaldırılması ve gölgel erin tespit edilmesi sorunları, ışın izleme sürecinde doğal olarak çözülür. Ne yazık ki, ışın izlemenin en büyük dezavantajı zaman alıcı olmasıdır. Her yansıma kaynağı na geri izlenirken, gereken hesaplama sayısı muazzam bir şekilde artar - bu, kırıl maları hesaba katarken veya dağıtılmış ışın izlemeyi uygularken daha da karmaşı klaşan bir sorundur. Bu nedenle, ışın izleme "tüketici düzeyindeki" gerçek zamanlı sistemlerde, örneğin etkileşimli video oyunlarında uygulanmaz, bunun yerine ger çek zamanlı hassasiyet gerektirmeyen "profesyonel düzeydeki" uygulamalarda bu lunabilir, örneğin sinema stüdyoları tarafından kullanılan grafik yazılımları.

Radyoaktivite

Geleneksel renderleme hattına alternatiflerden biri radyoaktivitedir. Işın izle me, bireysel ışınları izleyerek nokta nokta bir yaklaşım benimserken,radyoak tivite, düzlem yamalar arasındaki toplam ışık enerjisini dikkate alarak daha b ölgesel bir yaklaşım benimser. Bu yayılan ışık enerjisi esasendifüz ışıktır. Bir nesneden yayılan ışık enerjisi ya nesne tarafından üretilir(bir ışık kaynağı dur umunda olduğu gibi) ya da nesneden yansır. Her nesnenin görünümü,daha sonra diğer nesnelerden aldığı ışık enerjisi dikkate alınarak belirlenir.

Bir nesneden yayılan ışığın diğer bir nesnenin görünümünü ne derece et kilediği, form faktörleri adı verilen parametreler tarafından belirlenir. Sahne deki her yamayla ilişkili benzersiz bir form faktörü vardır. Bu formfaktörleri, yer alan yamalar arasındaki ayrılma mesafesi ve göreceli yönelimler gibi geo metrik ilişkileri dikkate alır. Sahnedeki bir yamayı belirlemek için,sahnedeki d iğer tüm yamalardan alınan ışık enerjisi miktarı, her hesaplama için uygun fo rm faktörü kullanılarak hesaplanır. Sonuçlar, her yama için tek bir renk ve yo ğunluk üretmek üzere birleştirilir. Bu değerler, ardından yan yana olan yama lar arasında,Gouraud gölgeleme benzeri teknikler kullanılarak, keskin bir yüz ey yerine düzgün konturlu bir yüzey elde etmek için iç içe geçirilir.

Doclingo https://www.doclingo.ai

Birçok yamanın dikkate alınması gerektiğinden, radyositi hesaplamak oldukça hesaplama yoğun bir işlemdir. Dolayısıyla, ışın izleme gibi, uygulaması mevcut tüketici pazarındaki gerçek zamanlı grafik sistemlerinin yeteneklerinin dışındadır. Radyositi ile ilgili bir diğer sorun ise, tüm yamalardan oluşan birimlerle çalıştığı için, bireysel noktaların detaylarını yakalayamamasıdır; bu da radyositi ile işlenen tüm yüzeylerin mat bir görünüme sahip olmasına neden olur.

Ancak, radyositi bazı avantajlara sahiptir. Bunlardan biri, radyositi kullanarak sahnelerin görünümünü belirlemenin kameradan bağımsız olmasıdır. Böylece, bir sahne için radyositi hesaplamaları yapıldıktan sonra, sahnenin farklı kamera pozisyonları için hızlı bir şekilde işlenmesi tamamlanabilir. Diğer bir avantajı ise, radyositi ışığın renk sızması gibi birçok ince özelliğini yakalamasıdır; burada bir nesnenin rengi çevresindeki diğer nesnelerin tonunu etkiler. Dolayısıyla, radyositi belirli alanlarda kendine özgü bir yer edinmiştir. Bunlardan biri, mimari tasarım ortamlarında kullanılan grafik yazılımlarıdır. Gerçekten de, önerilen bir binadaki ışık esasen difüz ve ortam ışığından oluştuğundan, yansıtıcı etkiler önemli değildir ve yeni kamera pozisyonlarının verimli bir şekilde işlenebilmesi, bir mimarın farklı odaları farklı açılardan hızlı bir şekilde görebilmesini sağlar.

Sorular ve Alıştırmalar

1. Görüntü penceresinden ışık kaynağına doğru değil, ışık kaynağından görüntü penceresine doğru ışın izleme neden takip edilir?
2. Doğrudan ışın izleme ile dağıtılmış ışın izleme arasındaki fark nedir?
3. Radyoaktivitenin iki dezavantajı nedir?
4. Işın izleme ve radyoaktivite hangi yönlerden benzerdir? Hangi yönlerden farklıdırlar?

10.6 Animasyon

Artık bilgisayar animasyonu konusuna geçiyoruz; bu, hareket sergileyen görüntüleri üretmek ve görüntülemek için bilgisayar teknolojisinin kullanılmasını ifade eder.

Animasyon Temelleri

Öncelikle bazı temel animasyon kavramlarını tanıtarak başlıyoruz.

Kareler Animasyon, hızlı bir şekilde ardışık olarak görüntülenen bir dizi görüntü, yani kareler, ile gerçekleştirilir. Bu kareler, düzenli zaman aralıklarında değişen bir sahnenin görünümünü yakalar ve böylece ardışık sunumları, sahneyi sürekli olarak gözlemleme illüzyonunu yaratır. Sinema endüstrisinde standart görüntüleme hızı saniyede yirmi dört karedir. Yayın videosunda standart ise saniyede altmış karedir (her ne kadar her diğer video karesi, önceki kare ile iç içe geçecek şekilde tasarlandığı için tam detaylı bir görüntü üretmek için).

Doclingo https://www.doclingo.ai

Kineograf

Kineograf, sayfaları hızlı bir şekilde çevrildiğinde hareketi simüle eden ani masyon karelerinin bir kitabıdır. Bu metinden kendi kineografinizi yapabili rsiniz (kenarları karalamalarla doldurmadıysanız). İlk sayfanın kenarına bir nokta koyun ve ardından üçüncü sayfadaki izlenimin konumuna dayanara k, üçüncü sayfada ilk sayfadaki noktadan biraz farklı bir konumda başka bi r nokta koyun. Bu işlemi, kitabın sonuna ulaşana kadar her ardışık tek sayf ada tekrarlayın. Şimdi sayfaları çevirin ve noktanın etrafta dans ettiğini izle yin. İşte! Kendi kineografinizi yaptınız ve belki de animasyon kariyerine ilk adımı attınız! Kinematik bir deney olarak, basit bir nokta yerine bir çubuk fi gürü çizmeyi deneyin ve çubuk figürünün yürüyormuş gibi görünmesini sa ğlayın. Şimdi, dinamiklerle deney yaparak bir su damlasının yere düştüğü görüntüsünü üretin.

tam detaylı bir görüntü oluşturmak için video ayrıca otuz kare/saniye sistemi olarak da sınıflandırılabilir).

Çerçeveler, geleneksel fotoğrafçılık ile üretilebilir veya bilgisayar grafikle ri aracılığıyla yapay olarak oluşturulabilir. Dahası, iki teknik birleştirilebilir. Örneğin, 2D grafik yazılımları genellikle fotoğrafçılıkla elde edilen görüntüleri des tek tellerinin görünümünü kaldırmak, görüntüleri üst üste bindirmek ve bir nesne nin başka bir nesneye dönüşüyormuş gibi görünmesini sağlayan morfing illüzyon unu yaratmak için kullanılır.

Morfing üzerine daha yakından bakmak, animasyon süreci hakkında ilgi nç bilgiler sunar. Morfing etkisini oluşturmak, morfing dizisini kapsayan bir a nahtar çerçeve çiftini tanımlamakla başlar. Biri morfın gerçekleşmesinden ö nceki son görüntüdür; diğeri ise morf gerçekleştiğinde sonraki ilk görüntüdü r. (Geleneksel sinema prodüksiyonunda, bu, morfın gerçekleşmesine kadar olan bir eylem dizisi ile morfın gerçekleşmesinden sonraki bir eylem dizisini “ filmlemek” gerektirir.) Morf öncesindeki çerçevede bulunan kontrol noktaları olarak adlandırılan noktalar ve çizgiler, morf sonrası çerçevedeki benzer öze lliklerle ilişkilendirilir ve morf, kontrol noktalarını rehber olarak kullanarak bi r görüntüyü diğerine kademeli olarak bozan matematiksel teknikler uygulan arak oluşturulur. Bu bozulma sürecinde üretilen görüntüleri kaydederek, orij inal anahtar çerçeveler arasındaki boşluğu dolduran ve morfing illüzyonunu yaratan kısa bir yapay görüntü dizisi elde edilir.

Storyboard Tipik bir animasyon projesi, bir storyboard'un oluşturulmasıyla başlar, bu, sunumun ana noktalarındaki sahnelerin eskizleri şeklinde tam hikayeyi anlatan iki boyutlu görüntülerin bir dizisidir. Storyboard'un nihai rolü, animasyon projesinin 2D veya 3D teknikleri kullanıl arak uygulanmasına bağlıdır. 2D grafikler kullanan bir projede, storyboard g enellikle 1920'lerdeki Disney stüdyolarında olduğu gibi nihai kare setine evril ir. O günlerde, ustalık animatörleri olarak adlandırılan sanatçılar, storyboard' u karakterlerin ve manzaraların görünümünü belirleyen detaylı kareler, yani ana kareler haline getirirlerdi. Yardımcı animatörler daha sonra ana kareler arasındaki boşlukları dolduracak ek kareler çizerlerdi, böylece animasyon

Doclingo https://www.doclingo.ai

Bulandırma

Geleneksel fotoğrafçılık alanında, hızlı hareket eden nesnelerin keskin görüntülerini üretme amacı yla çok çaba harcanmıştır. Animasyon alanında ise tam tersi bir sorun ortaya çıkmaktadır. Eğer bir hareket eden nesneyi tasvir eden bir dizideki her bir kare nesneyi keskin bir görüntü olarak tasvir e diyorsa, o zaman hareket sarsıntılı görünebilir. Ancak, keskin görüntüler, sahne grafiğindeki durağ an nesnelerin bireysel görüntüleri olarak kareler oluşturmanın doğal bir yan ürünüdür. Bu nedenle , animatörler genellikle bir bilgisayar tarafından üretilen karede hareket eden nesnenin görüntüsü nü yapay olarak bozarlar. Bir teknik, süper örnekleme olarak adlandırılan, hareket eden nesnenin y alnızca hafifçe yer değiştirdiği birden fazla görüntü üretmek ve ardından bu görüntüleri tek bir kar e oluşturmak için üst üste bindirmektir. Diğer bir teknik ise, hareket yönünde uzamış gibi görünme si için hareket eden nesnenin şeklini değiştirmektir.

kesintisiz ve pürüzsüz görünecektir. Bu boşluk doldurma sürecine **aralıklama denir.**

Bu sürecin günümüzde kullanılan yöntemle arasındaki temel fark, animatörlerin artık anahtar kareleri çizmek için görüntü işleme ve 2D grafik yazılımları kullanması ve aralıklama sürecinin büyük ölçüde otomatikleştirilmiş olmasıdır; bu nedenle yardı mcı animatörlerin rolü esasen ortadan kalkmıştır.

3D Animasyon Çoğu video oyunu animasyonu ve tam özellikli animasyonlu pr odüksiyonlar artık 3D grafikler kullanılarak oluşturulmaktadır. Bu durumlard a proje, iki boyutlu görüntülerden oluşan bir hikaye tahtası oluşturulmasıyla başlar. Ancak, 2D grafik projelerinde olduğu gibi nihai ürüne dönüşmek yeri ne, hikaye tahtası üç boyutlu sanal dünyanın inşasında bir rehber olarak kull anılır. Bu sanal dünya, içindeki nesneler senaryoya veya video oyununun ilerl eyişine göre hareket ettirildiğinde tekrar tekrar "fotoğraflanır".

Belki burada durup, bir nesnenin bilgisayar tarafından oluşturulmuş bir sahnede h areket etmesinin ne anlama geldiğini netleştirmeliyiz. "Nesne"nin aslında sahne grafiği nde depolanan bir veri kümesi olduğunu unutmayın. Bu koleksiyondaki veriler arasında nesnenin konumunu ve yönünü belirten değerler bulunmaktadır. Dolayısıyla, bir nesne yi "hareket ettirmek", yalnızca bu değerleri değiştirmekle gerçekleştirilir. Bu değişiklikle r yapıldıktan sonra, yeni değerler render sürecinde kullanılacaktır. Sonuç olarak, nesne nihai iki boyutlu görüntüde hareket etmiş gibi görünecektir.

Kinematik ve Dinamik

3D grafik sahnesindeki hareketin ne kadarının otomatikleştirildiği veya bir insan animat örü tarafından kontrol edildiği uygulamalar arasında değişiklik göstermektedir. Amaç, e lbette, tüm süreci otomatikleştirmektir. Bu amaca ulaşmak için, doğal olarak meydana gelen fenomenlerin hareketini tanımlama ve simüle etme yollarını bulmaya yönelik çok sayıda araştırma yapılmıştır. Bu bağlamda, mekanik alanlarından ikisi özellikle faydalı ol muştur.

Bunlardan biri, bir nesnenin hareketini tanımlamak için fizik yasalarını uy gulayarak nesneye etki eden kuvvetlerin etkilerini belirleyen dinamiklerdir. Ö rneğin, bir sahnedeki bir nesneye bir konumun yanı sıra bir hareket yönü, hı z ve kütle atanabilir. Bu öğeler, yerçekiminin veya diğer nesnelerle çarpışmal arın etkilerini belirlemek için kullanılabilir.

Doclingo https://www.doclingo.ai

nesne üzerinde, yazılımın nesnenin bir sonraki kare için doğru konumunu hesaplam asına olanak tanıyacak.

Örnek olarak, bir kapta suyun dalgalanmasını gösteren animasyonlu bir seka ns oluşturma görevini düşünelim. Su temsil etmek için sahne grafiğinde bir parça cık sistemi kullanabiliriz; burada her parçacık, küçük bir su birimini temsil eder. (Su, mermer boyutunda büyük "moleküller"den oluştuğunu düşünün.) Sonra , parçacıkların üzerindeki yerçekimi etkilerini ve kap, yan yana sallanırken pa rçacıklar arasındaki etkileşimi hesaplamak için fizik yasalarını uygulayabiliriz. Bu, her parçacığın konumunu düzenli zaman aralıklarında hesaplamamıza o lanak tanır ve dış parçacıkların konumunu çokgen bir ağın köşeleri olarak kul lanarak, suyun yüzeyini gösteren bir ağ elde edebiliriz. Animasyonumuzu, si mülasyon ilerledikçe bu ağı tekrar tekrar "fotoğraflayarak" elde edebiliriz.

Hareketi simüle etmek için kullanılan diğer mekanik dal, bir nesnenin hareketini nesnenin parçalarının birbirine göre nasıl hareket ettiğini tanımlamakla ilgilenen kin ematiklerdir. Kinematiklerin uygulamaları, kollar ve bacaklar gibi uzuvların hareket e ttirilmesi gerektiğinde, eklemli figürlerin animasyonunda belirgindir. Bu hareketler, kaslar ve yerçekimi tarafından uygulanan bireysel kuvvetlerin etkilerini hesaplamakt an ziyade, eklem hareketi kalıplarını simüle ederek daha kolay bir şekilde modellenir . Böylece, dinamikler zıplayan bir topun yolunu belirlerken tercih edilen teknik olabili rken, animasyonlu bir karakterin kolunun hareketi, omuz, dirsek ve bilek rotasyonlar ını hesaplamak için kinematiklerin uygulanmasıyla belirlenecektir. Bununla birlikte, c anlı karakterlerin animasyonunda çok sayıda araştırma, anatomik sorunlara ve ekle m ile uzuv yapısının hareketi nasıl etkilediğine odaklanmaktadır.

Tipik bir kinetik uygulama yöntemi, bir karakteri temsil etmek için karakt erin tasvir edilen iskelet yapısını simüle eden bir çubuk figürü ile başlamaktır . Figürün her bölümü, o bölümün etrafındaki karakterin yüzeyini temsil eden çokgen bir ağ ile kaplanır ve bitişik ağların birbirleriyle nasıl bağlanacağına dair kurallar belirlenir.

Figür, iskelet yapısındaki eklemleri yeniden konumlandırarak (ya yazılım ya da bir insan animatör tarafından) manipüle edilebilir; bu, bir ip kuklasını manipüle etme şekline benzer. Modelin "iplerinin" bağlandığı noktalar, "artikülasyon değişkenleri" (veya daha yakın zamanda "animasyon değişkenleri") için kısaltma olan avars olar ak adlandırılır.

Pratikte, avars yalnızca iskelet eklem pozisyonlarını kontrol etmekle kalmaz. Ö rneğin, Toy Story'deki Woody adlı kovboy bebeği, yalnızca yüzüyle ilişkili yaklaşık 100 avara sahipti; bu, animatörlerin yüzü duyguları ifade edecek şekilde ayarlama sına ve ağzı konuşulan kelimelere karşılık gelecek şekilde hareket ettirmesine ola nak tanıyordu.

Kinetik uygulamalarında yapılan birçok araştırma, doğal olarak meydana g elen hareketi taklit eden uzuv pozisyonlarının dizilerini otomatik olarak hesapla mak için algoritmalar geliştirmeye yönlendirilmiştir. Bu doğrultuda, gerçekçi yü rüyüş dizileri üreten algoritmalar artık mevcuttur.

Ancak, kinetik temelli animasyonların çoğu, bir karakteri önceden belirlen miş eklem-uzuv pozisyonları dizisi boyunca yönlendirerek hala üretilmektedir. Bu pozisyonlar, bir animatörün yaratıcılığı ile oluşturulabilir veya bir hareket yakalama ile elde edilebilir; bu, bir canlı modelin istenen eylemi gerçekleştiri rken pozisyonlarının kaydedilmesini içerir. Daha kesin olarak, bir insanın vüc udundaki stratejik noktalara yansıtıcı bant uygulandıktan sonra, insan bir be yzbol fırlatırken farklı açılardan fotoğraflanabilir. Ardından,

Doclingo https://www.doclingo.ai

çeşitli fotoğraflardaki bantta, insanın kollarının ve bacaklarının kesin yönleri, atış eylemi ilerledikçe tanımlanabilir ve bu yönler daha sonra bir animasyond aki bir karaktere aktarılabilir.

Animasyon Süreci

Animasyonda araştırmanın nihai amacı, tüm animasyonsürecini otomatikleştirmekti r. Doğru parametreler verildiğinde, istenen animasyon dizisini otomatik olarak ürete cek bir yazılım hayal edilmektedir. Bu yöndeki ilerleme, sinema endüstrisinin artık ka labalıklar, savaş sahneleri ve kaçan hayvanların görüntülerini, sahne grafiği içinde ot omatik olarak hareket eden bireysel sanal "robotlar" aracılığıyla, her birinin kendisin e tahsis edilen görevi yerine getirdiği gerçeğiyle gösterilmektedir.

Fantastik orduların, Orc'ların ve insanların, *Yüzüklerin Efendisi* üçlemesi için çekimleri sırasında ilginç bir durum ortaya çıktı. Ekrandaki her savaşçı, k endi fiziksel özelliklerine sahip ayrı bir "zeki" nesne olarak modellenmişti ve s aldırma veya kaçma eğilimleri veren rastgele atanmış bir kişiliğe sahipti. İkin ci filmdeki Helms Deep savaşı için yapılan test simülasyonlarında, Orc'ların k açma eğilimi çok yüksek ayarlanmıştı ve insan savaşçılarıyla karşılaştıklarınd a sadece kaçtılar. (Bu, sanal figürlerin bir işi çok tehlikeli bulduğu belki de ilk durumdu.)

Elbette, günümüzde birçok animasyon hala insan animatörler tarafından yaratıl maktadır. Ancak, 1920'lerde olduğu gibi iki boyutlu kareleri elle çizmek yerine, bu ani matörler sahne grafiğindeki üç boyutlu sanal nesneleri manipüle etmek için yazılım kullanıyorlar; bu, daha önce kinetik hareket üzerine yaptığımız tartışmada açıklandığ ı gibi ip kuklalarını kontrol etmeye benzer bir şekilde. Bu şekilde, bir animatör, anima syonu üretmek için "fotoğraflanan" sanal sahneler serisi oluşturabilir. Bazı durumlar da, bu teknik yalnızca ana kareler için sahneleri üretmek için kullanılır ve ardından ya zılım, sahneyi otomatik olarak render ederek, sahne grafiğindeki nesneleri bir ana k are sahne pozisyonundan diğerine taşımak için dinamikler ve kinetikler uygulandığı nda ara kareleri üretmek için kullanılır.

Bilgisayar grafikleri alanındaki araştırmalar ilerledikçe ve teknoloji geliştikçe, ani masyon sürecinin daha fazla kısmı kesinlikle otomatik hale gelecektir. İnsan animatö rlerinin, insan aktörlerin ve fiziksel setlerin rolünün bir gün geçersiz hale gelip gelme yeceği henüz belli değil, ancak birçok kişi o günün gelecekte çok uzak olmadığını düş ünüyor. Gerçekten de, 3D grafiklerin, sessiz filmlerden "konuşan filmlere" geçişten ç ok daha fazla sinema endüstrisini etkileme potansiyeli vardır.

Sorular ve Alıştırmalar

1. Bir insanın gördüğü görüntüler, insanın algısında yaklaşık 200 milisa niye boyunca kalma eğilimindedir. Bu tahmine dayanarak, animasyo n oluşturmak için insana saniyede kaç görüntü sunulmalıdır? Bu tah min, sinema endüstrisinde kullanılan saniyedeki kare sayısıyla nasıl k arşılaştırılır?
2. Storyboard nedir?
3. In-betweening nedir?
4. Kinematik ve dinamik terimlerini tanımlayın.

Doclingo https://www.doclingo.ai

Bölüm Gözden Geçirme Problemleri

1. Aşağıdakilerden hangileri 2D grafik uygulam alarıdır ve hangileri 3D grafik uygulamala rıdır?
   a. Dergi sayfalarının düzenini tasarlamakb. M icrosoft Paint kullanarak bir görüntü çizmekc. Bir video oyunu için sanal bir dünyadan görüntü ler üretmek

2. 3D grafikler bağlamında, gelenek sel fotoğrafçılıktan her bir aşağıdaki öğe ye ne karşılık gelir? Cevaplarınızı açıklayın .
   . Film
   b. Vizör içindeki dikdörtgen
   c. Fotoğraflanan sahne

3. Perspektif projeksiyon kullanıldığında, sahnedeki bir küre projeksiyon düzleminde bir daire üretmeye cek şekilde hangi koşullar altında olacaktır?

4. Perspektif projeksiyon kullanıldığında, düz bir do ğru parçasının projeksiyon düzleminde eğri b ir doğru parçası olma durumu mümkün müdür? Cevabınızı gerekçelendirin.

5. Farz edelim ki sekiz ayak uzunluğundaki bir düz direğin bir u cu projeksiyon merkezinden dört ayak uzaklıktadır. Ayrıca, projeksiyon merkezinden direğin bir u cuna giden düz bir doğrunun, projeksiyon düzle minde projeksiyon merkezinden bir ayak uzaklı kta bir noktada kesiştiğini varsayalım. Eğer direk projeksiyon düzlemine paralel ise, direğin pro jeksiyon düzlemindeki görüntüsü ne kadardır?

6. Paralel projeksiyon ile perspektif projek siyon arasındaki farkı açıklayın.

7. Görüntü penceresi ile çerçeve tamponu a rasındaki ilişkiyi açıklayın.

8. 3D grafikleri kullanarak bir sinema fil mi üretmek ile 3D grafikleri kullanarak e tkileşimli bir video oyunu için animasyon ü retmek arasındaki önemli bir fark nedir? Cevabınızı açıklayın.

9. Bir nesnenin 3D grafik sahnesinde kullanılmak üzere o nesnenin modeline dahil edilebilecek b azı özelliklerini tanımlayın. Modelde muhtemel en temsil edilmeyecek bazı özellikleri tanımla yın. Cevabınızı açıklayın.

10. Bir nesnenin fiziksel özelliklerinden bazılarını ta nımlayınmodelin yalnızca çokgen bir ağ içerdi ği durumlardayakalanmayan. (Bu nedenle, y alnızca bir çokgenağ, nesnenin tam bir mode lini oluşturmaz.) Buözelliklerden birinin nesnenin modeline nasıleklenebileceğini açıklayın.

11. Üç boyutlu uzaydaki herhangi dört nokta, bir çokgen ağdaki bir yamanın köşeleri olabilir mi? Cevabınızı açıklayın.

12. Takip eden her koleksiyon, bir çokgenağ daki bir yamanın köşelerini (geleneksel dikdörtgenkoordinat sistemini kullanarak) temsil eder.Ağın şeklini tanımlayın.

    Yama 1: (0, 0, 0) (0, 2, 0)
    (2, 2, 0) (2, 0, 0)
    Yama 2: (0, 0, 0) (1, 1, 1)
    (2, 0, 0)
    Yama 3: (2, 0, 0) (1, 1, 1)
    (2, 2, 0)
    Yama 4: (2, 2, 0) (1, 1, 1)
    (0, 2, 0)
    Yama 5: (0, 2, 0) (1, 1, 1)
    (0, 0, 0)

13. Takip eden her koleksiyon, bir çokgenağ daki bir yamanın köşelerini (geleneksel dikdörtgenkoordinat sistemini kullanarak) temsil eder.Ağın şeklini tanımlayın.

    Yama 1: (0, 0, 0) (0, 4, 0)
    (2, 4, 0) (2, 0, 0)
    Yama 2: (0, 0, 0) (0, 4, 0)
    (1, 4, 1) (1, 0, 1)
    Yama 3: (2, 0, 0) (1, 0, 1)
    (1, 4, 1) (2, 4, 0)
    Yama 4: (0, 0, 0) (1, 0, 1)
    (2, 0, 0)
    Yama 5: (2, 4, 0) (1, 4, 1)
    (0, 4, 0)

14. Bir dikdörtgen katı temsil eden bir çokge nağ tasarlayın. Köşeleri kodlamak içi n gelenekseldikdörtgen koordinat sistem ini kullanın veçözümünüzü temsil eden bir t aslak çizin.

Doclingo https://www.doclingo.ai

15. Sekiz üçgen parça kullanarak,bir yarıçapı b ir olan bir küreninşeklini yaklaşık olarak te msil eden birpoligonal ağ tasarlayın. (Sadec eeşit sekiz parça ile, ağınız bir kürenin çokk aba bir yaklaşımı olacaktır, ancak amaç,bir poligonal ağın ne olduğunu anlamanızıs ergilemek, bir kürenin kesin birtemsilini üretmek değil.) Parçalarınızınköşelerini gele neksel dikdörtgen koordinatsistemini kull anarak temsil edin ve ağınızınbir taslağını çi zin.

16. Aşağıdaki dört nokta neden bir düzlem y aması'nın köşeleri olamaz?

(0, 0, 0) (1, 0, 0)

(0, 1, 0) (0, 0, 1)

17. Noktaların (1, 0, 0), (1, 1, 1) ve (1, 0, 2) bir düzlem yama sı'nın köşeleri olduğunu varsayalım. Aşağıdaki doğru parçalarından hangisi/hangi parçaları yamanın yüz eyine normaldir?

a. (1, 0, 0) ile (1, 1, 0) arasındaki doğru parçası
b. (1, 1, 1) ile (2, 1, 1) arasındaki doğru parçası
c. (1, 0, 2) ile (0, 0, 2) arasındaki doğru parçası
d. (1, 0, 0) ile (1, 1, 1) arasındaki doğru parçası

18. İki "tip" prosedür modelini tanımlayın.

19. Modelleme ve render alma süreçleri a rasında hangisi daha

a. standartlaştırılmış bir görev?

b. hesaplama açısından yoğun bir görev?

c. yaratıcı bir görev?

Cevaplarınızı gerekçelendirin.

20. Aşağıdakilerden hangisi bir sahne grafiğinde temsil edilebilir?

a. Işık kaynakları
b. Cansız nesneler
c. Karakterler/aktörler
d. Kamera

21. Bir sahne grafiğinin oluşturulması, 3D grafikler sü recinde ne anlamda önemli bir adım olarak kabul edilir?

22. Bir sahne grafiğindeki kameranın konum v e yönlerinin değişmesi, hangi komplikasyo nları getirir?

23. Düzlem parçasının yüzeyinin köşeleri (0, 0, 0), (0, 2, 0), (2, 2, 0) ve (2, 0, 0) düzgün ve parlak olduğunu varsayalım. Eğer bir ışın (0, 0, 1) noktasından çıkıyorsa ve yüzeye (1, 1, 0), yansıyan ışın hangi noktalar üzerind en geçecektir?

a. (0, 0, 1)
b. (1, 1, 1)
c. (2, 2, 1)
d. (3, 3, 1)

24. Bir şamandıra, durgun su yüzeyinin on feet y ukarısında bir ışığı destekliyorsa, gözlem ci şamandıradan on beş feet uzakta ve su yüzeyinin beş feet yukarısındaysa, gözlemci ı şığın yansımasını su yüzeyinde nerede gö recektir?

25. Bir balık durgun su yüzeyinin altında yüzüy orsa ve bir gözlemci balığı suyun üstünden i zliyorsa, balık gözlemcinin konumundan ne rede görünecektir?

a. Gerçek konumunun arka planına doğr u yukarıda
b. Gerçek konumda
c. Gerçek konumunun ön planına doğru aşa ğıda

26. (1, 0, 0), (1, 1, 1) ve (1, 0, 2) noktalarının bir d üzlem yaması olduğunu ve köşelerin n esnenin dışından görüldüğünde saat yönün ün tersine sıralandığını varsayalım. Aşağıd aki her durumda, verilen noktadan çıkan bir ışın, yamayı nesnenin dışından mı yoksa içinden mi vuracağını belirtin.

A. (0, 0, 0)
. (2, 0, 0)c.
(2, 1, 1)\v
e. (3, 2, 1)

27. Görüş hacminin dışında bir nesnenin niha i görüntüde hala görünebileceği bir örnek v erin. Cevabınızı açıklayın.

28. Bir z-buferinin içeriğini ve amacını tanı mlayın.

29. Gizli yüzey kaldırma konusundaki tartışma mızda,"ön plan/arka plan" sorununu çöz me prosedürünüe bir z-büfesi yardımıyla tanımladık. Bu prosedürü,Bölüm 5'te tanıtıl an sahte kod kullanarak ifade edin.

30. Bir nesnenin yüzeyinin, her biri bir santime tre genişliğinde, sırayla turuncu ve mavi dikey şeritlerle kaplı olduğunu varsayalım . Eğer nesne, piksel konumlarının

Doclingo https://www.doclingo.ai

iki santimetre aralıklarla yerleştirilmiş nesne , son görüntüde nesnenin olası görün ümleri ne olurdu? Cevabınızı açıklayın.

31. Dokusal haritalama ve çıkıntı haritalama "dokuyu" bir yüzeyle ilişkilendirme yöntemleridir, fakat oldukça farklı tekniklerdir. İkisini karşılaştıran kısa bir paragraf yazın.
32. Dört adımı listeleyin ve her birinin kısa bir tanımını verin.
33. Donanım/yazılım uygulamasının renderl ama boru hattını kullanmanın bazı avantajla rı nelerdir?
34. Bir bilgisayardaki donanım, etkileşimli video oy unları için tasarlandığında, genel amaçlı bir PC'den ne şekilde farklılık gösterir?
35. Geleneksel renderleme borusunun öne mli bir sınırlaması nedir?
36. Yerel aydınlatma modeli ile küresel aydınl atma modeli arasındaki fark nedir?
37. Ray tracing'in geleneksel renderlama hattın a göre ne avantajı var? Hangi dezavantajları var?
38. Dağıtılmış ışın izleme, geleneksel ışın izlem eye göre ne gibi bir avantaja sahiptir? Hangi dezavantajı vardır?
39. Radyoaktivitenin geleneksel renderleme h attına göre ne avantajı var? Ne dezavantajı v ar?
40. Geleneksel ışın izleme ile üretilen bir sahne görüntüsü, radyoaktivite ile üretilen benzer bir görüntü ile karşılaştırıldığında, iki görü ntü nasıl karşılaştırılır?
41. Bir sinema salonunda gösterilecek doksan dakikalık bir animasyon üretmek için kaç k are gerekecektir?
42. Bir parçacık sisteminin, titreyen ateş alevlerinin animasyon unu üretmek için nasıl kullanılabileceğini tanımlayın.
43. Bir sahne içinde hareket eden tek bir nesneyi tasvir eden bir animasyon dizisi oluştururken z-bufe rinin kullanımının nasıl yardımcı olabileceğini açı klayın.
44. Bugünün insan animatörleri ile geçmişteki i nsan animatörleri arasındaki bazı farklılıklar nelerdir?

Sosyal Sorunlar

İzleyen sorular, bilgisayar alanıyla ilişkili etik/sosyal/hukuki sorunlara bir reh ber olarak tasarlanmıştır. Amaç yalnızca bu soruları yanıtlamak değildir. Ayrı ca, neden böyle yanıt verdiğinizi ve gerekçelerinizin bir sorudan diğerine tut arlı olup olmadığını da düşünmelisiniz.

1. Diyelim ki bilgisayar animasyonu, gerçek aktörlerin sinema ve televizyon endüstrilerinde artık gerekli olmadığı bir noktaya ulaştı. Sonuçları ne olu rdu? "Film yıldızları" olmamasının yankı etkisi ne olurdu?
2. Dijital kameraların ve ilgili yazılımların gelişimi ile fotoğrafları değiştirme veya üretme yeteneği genel halkın erişimine sunulmuştur. Bu toplumda ne gibi değişiklikler getirecek? Hangi etik ve yasal sorunlar ortaya çıkabili r?
3. Fotoğraflar ne ölçüde sahiplenilmelidir? Bir kişi fotoğrafını bir web sitesi ne koyarsa ve başka biri o fotoğrafı indirip, konuyu zor bir durumda göst erecek şekilde değiştirip, değiştirilmiş versiyonu yayarsa, fotoğrafın kon usu ne tür bir başvuru yapmalıdır?
4. Şiddet içeren bir video oyunu geliştirmeye yardımcı olan bir programcı, oyun un sonuçlarından ne ölçüde sorumludur? Çocukların video oyunlarına erişimi kısıtlanmalı mı? Eğer öyleyse, nasıl ve kim tarafından? Mahkumlar gibi toplum un diğer grupları hakkında ne düşünülüyor?

Doclingo https://www.doclingo.ai

Ek Okuma

Angel, E. *Etkileşimli Bilgisayar Grafikleri, Aşağıdan Yukarıya Yaklaşım OpenGL Kullanarak*, 5. baskı. Boston, MA: Addison-Wesley, 2009.

Bowman, D. A., E. Kruijff, J. J. LaViola, Jr., ve I. Poupyrev. *3D Kullanıcı Arayüzleri Te ori ve Uygulama.*Boston, MA: Addison-Wesley, 2005.

Hill, Jr., F. L. ve S. Kelley. *OpenGL Kullanarak Bilgisayar Grafikleri*. 3. baskı. Upper Saddle River, NJ: Prentice-Hall, 2007.

McConnell, J. J. *Bilgisayar Grafikleri, Teoriden Uygulamaya*. Sudbury, MA: Jones and Bartlett, 2006.

Parent, R. *Bilgisayar Animasyonu, Algoritmalar ve Teknikler*, 2. baskı. San Francisco, CA: Morgan Kaufmann, 2008.

Doclingo https://www.doclingo.ai

BÖLÜM

Yapay Zeka

11

Bu bölümde yapay zeka olarak bilinen bilgisayar bilimi da lını inceliyoruz. Bu alan nispeten genç olmasına rağmen, uzman satranç oyuncuları, öğrenip akıl yürütebilen bilgisa yarlar ve ortak bir hedefe ulaşmak için faaliyetlerini koordi ne eden makineler gibi bazı şaşırtıcı sonuçlar üretmiştir. Yapay zekada, bugünün bilim kurgu hikayeleri yarının gerç eği olabilir.

11.1Zeka ve Makineler
Akıllı Ajanlar
Araştırma Yöntemleri
Turing Testi

11.2 Algı
Görüntüleri Anlama
Dil İşleme

11.3Akıl Yürütme
Üretim Sistemleri
Arama Ağaçları
Heuristikler

11.4Araştırmanın Ek Alanları
Bilgiyi Temsil Etme veManip üle Etme
Öğrenme
Genetik Algoritmalar

11.5Yapay Sinir Ağları
Temel Özellikl erYapay SinirAğlarını Eğitme
İlişkisel Bellek

11.6Robotik

11.7Sonuçları Düşünmek

Doclingo https://www.doclingo.ai

Yapay Zeka, karmaşık görevleri insan müdahalesi olmadan yerine getirebile n otonom makineler inşa etmeyi amaçlayan bilgisayar bilimi alanıdır. Bu hed ef, makinelerin algılayabilmesi ve akıl yürütebilmesi gerektiğini gerektirir. Bu tür yetenekler, insan zihni için doğal olmasına rağmen makineler için zorlayı cı olan sağduyu aktiviteleri kategorisine girer. Sonuç olarak, bu alandaki çalış malar zorlu olmaya devam etmektedir. Bu bölümde, bu geniş araştırma alan ındaki bazı konuları keşfedeceğiz.

11.1 Zeka ve Makineler

Yapay zeka alanı oldukça geniştir ve psikoloji, nöroloji, matematik, dilbilim ve elektrik ve makine mühendisliği gibi diğer konularla birleşir.Düşüncelerimizi odaklamak için, bir ajan kavramını ve bir ajanın sergileyebileceği zeki davra nış türlerini düşünerek başlıyoruz.Gerçekten de, yapay zeka alanındaki araştı rmaların çoğu, bir ajanın davranışları açısından kategorize edilebilir.

Akıllı Ajanlar

Bir ajan, çevresinden gelen uyarıcılara yanıt veren bir "cihaz"dır. Bir ajanı, bir ro bot gibi bireysel bir makine olarak hayal etmek doğaldır, ancak bir ajan, otono m bir uçak, etkileşimli bir video oyunundaki bir karakter veya İnternet üzerinde n diğer süreçlerle iletişim kuran bir süreç gibi başka biçimler de alabilir (belki bi r istemci, bir sunucu veya bir eş olarak). Çoğu ajanın, çevrelerinden veri aldığı s ensörleri ve çevrelerini etkileyebildiği aktüatörleri vardır. Sensörlere örnek olar ak mikrofonlar, kameralar, mesafe sensörleri ve hava veya toprak örnekleme ci hazları verilebilir. Aktüatörlere örnek olarak ise tekerlekler, bacaklar, kanatlar, kavrayıcılar ve konuşma sentezleyicileri verilebilir.

Yapay zeka alanındaki araştırmaların çoğu, akıllı davranan ajanlar inşa et me bağlamında karakterize edilebilir; bu, ajanın aktüatörlerinin eylemlerinin, sensörleri aracılığıyla alınan verilere rasyonel yanıtlar olması gerektiği anla mına gelir. Bu araştırmayı, bu yanıtların farklı seviyelerini dikkate alarak sınıfl andırabiliriz.

En basit yanıt, yalnızca girdi verilerine önceden belirlenmiş bir yanıt olan refleks eylemidir. Daha "akıllı" davranışlar elde etmek için daha yüksek yanıt seviyeleri gereklidir. Örneğin, bir ajana çevresi hakkında bilgi verip, ajanın ey lemlerini buna göre ayarlamasını isteyebiliriz. Bir beyzbolu fırlatma süreci bü yük ölçüde bir refleks eylemidir, ancak topu nasıl ve nereye fırlatacağınızı bel irlemek, mevcut çevre hakkında bilgi gerektirir. (Birinci ve üçüncü üslerde ko şucular var.) Bu tür gerçek dünya bilgisinin nasıl depolanacağı, güncellenece ği, erişileceği ve nihayetinde karar verme sürecinde nasıl uygulanacağı, yapa y zeka alanında hala zorlu bir problem olmaya devam etmektedir.

Bir hedefe ulaşmak istiyorsak, örneğin bir satranç oyununu kazanmak ve ya kalabalık bir geçitten geçmek gibi, başka bir yanıt düzeyi gereklidir.
Bu tür hedef odaklı davranış, ajanın yanıtının veya yanıtlar dizisinin, kasıtlı olar ak bir eylem planı oluşturmanın veya mevcut seçenekler arasında en iyi eylemi seçmenin sonucu olmasını gerektirir.

Bazı durumlarda, bir ajanın yanıtları zamanla gelişir çünkü ajan öğrenir.

Bu, prosedürel bilgi geliştirme (nasıl öğrenileceği) veya deklaratif bilgi depolama (ne öğrenileceği) biçiminde olabilir. Prosedürel bilgi öğrenimi genellikle, bir ajanın kötü eylemler için cezalandırılarak ve iyi eylemler için ödüllendirilerek uygun eylemleri öğrendiği bir deneme yanılma sürecini içerir.

Doclingo https://www.doclingo.ai

Bu yaklaşımı izleyerek, zamanla, dama ve satranç gibi rekabetçi oyunlarda y eteneklerini geliştiren ajanlar geliştirilmiştir.

Deklaratif bilgi öğrenimi genellikle, bir ajanın bilgi deposundaki "gerçekleri" ge nişletme veya değiştirme biçiminde olur. Örneğin, bir beyzbol oyuncusu, gelec ekteki olaylara rasyonel yanıtların belirlendiği bilgi veritabanını (hala sadece bir out var, ama şimdi birinci ve ikinci base'de koşucular var) tekrar tekrar ayarlam ak zorundadır.

Uygun tepkiler üretmek için, bir ajan "algılamalıdır"sensörleri tarafından alınan uyarıları. Yani, bir ajanın sensörleri tarafından üretilen verilerden bilgi çıkarabilmesi gerekir, diğer bir deyişle, bir ajanın algılayabilmesi gerekir. Ba zı durumlarda bu basit bir süreçtir. Bir jiroskoptan elde edilen sinyaller, tepki leri belirlemek için hesaplamalarla uyumlu formlarda kolayca kodlanabilir. A ncak diğer durumlarda, giriş verilerinden bilgi çıkarmak zordur. Örnekler ara sında konuşmayı ve görüntüleri anlama yer alır.

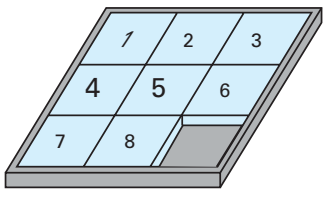
Aynı şekilde, ajanlar tepkilerini aktüatörleriyle uyumlu terimlerle formüle ed ebilmelidir. Bu basit bir süreç olabilir veya bir ajanın tepkileri tam konuşma c ümleleri olarak formüle etmesini gerektirebilir—bu da ajanın konuşma üret mesi gerektiği anlamına gelir. Dolayısıyla, görüntü işleme ve analiz, doğal dil anlama ve konuşma üretimi gibi konular önemli araştırma alanlarıdır.

Burada tanımladığımız ajan nitelikleri, geçmiş ve mevcut araştırma alanları nı temsil etmektedir. Elbette, bunlar birbirinden tamamen bağımsız değildir. Tü m bu niteliklere sahip ajanlar geliştirmek istiyoruz; çevrelerinden aldıkları verile ri anlayan ve ajanın yeteneklerini maksimize etmeyi amaçlayan bir öğrenme sü reci aracılığıyla yeni tepki kalıpları geliştiren ajanlar üretmek. Ancak, çeşitli türd eki rasyonel davranışları izole ederek ve bunları bağımsız olarak takip ederek, a raştırmacılar daha sonra diğer alanlardaki ilerlemelerle birleştirilebilecek bir ba şlangıç noktası elde ederler ve daha zeki ajanlar üretirler.

Bu alt bölümü, Bölüm 11.2 ve 11.3'teki tartışmamıza bir bağlam sağlayac ak bir ajan tanıtarak kapatıyoruz. Ajan, 1'den 8'e kadar numaralandırılmış se kiz kare karonun, üç satır ve üç sütun halinde toplam dokuz karoyu tutabilen bir çerçeve içinde yer aldığı sekiz bulmacayı çözmek için tasarlanmıştır (Şekil 11.1). Çerçevedeki karolar arasında, yanındaki herhangi bir karonun itilebile ceği bir boşluk bulunmaktadır ve bu, çerçevedeki karoların karışmasına olan ak tanır. Sorun, karışık bir bulmacadaki karoları başlangıç pozisyonlarına ger i taşımaktır (Şekil 11.1).

Ajanımız, bir tutucu, bir video kamera ve bir şeyi itme sırasında kaymam ası için lastik uçlu bir parmakla donatılmış bir kutu şeklindedir.

Şekil **11.1** Sekiz bulmacanın çözülmüş konfigürasyonu



Doclingo https://www.doclingo.ai

(Şekil 11.2). Ajan ilk açıldığında, tutucusu bulmacayı istercesine açılıp kapan maya başlar. Karışık bir sekiz bulmacayı tutucuya yerleştirdiğimizde, tutucu bulmacanın üzerine kapanır. Kısa bir süre sonra makinenin parmağı alçalır v e karoları çerçeve içinde itmeye başlar, ta ki karolar başlangıç pozisyonlarına geri dönene kadar. Bu noktada makine bulmacayı serbest bırakır ve kendini kapatır.

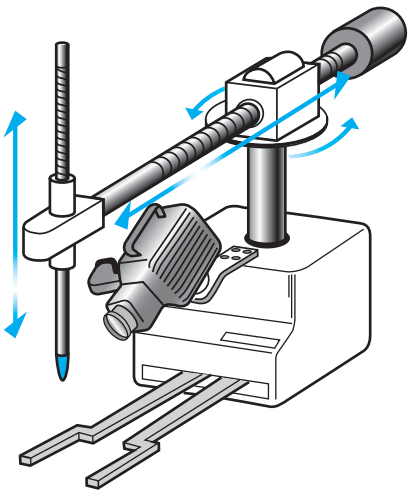
Bu bulmaca çözme makinesi, tanımladığımız iki ajan özelliğini sergileme ktedir. İlk olarak, kameradan aldığı görüntüden mevcut bulmaca durumunu çıkarması gerektiği anlamında algılayabilmelidir. Görüntüleri anlama konusu nu Bölüm 11.2'de ele alacağız. İkincisi, bir hedef elde etmek için bir plan geli ştirmeli ve uygulamalıdır. Bu konuları Bölüm 11.3'te ele alacağız.

Araştırma Yöntemleri

Yapay zeka alanını takdir etmek için, bunun iki yol boyunca sürdürüldüğünü anlamak faydalıdır. Biri, araştırmacıların akıllı davranış sergileyen sistemler g eliştirmeye çalıştığı mühendislik yoludur. Diğeri ise, araştırmacıların hayvan —özellikle insan—zekasının hesaplamalı bir anlayışını geliştirmeye çalıştığı t eorik bir yoldur. Bu ikilik, iki yolun nasıl sürdürüldüğünü dikkate alarak netle ştirilir. Mühendislik yaklaşımı, temel hedef belirli performans hedeflerini kar şılayan bir ürün üretmek olduğu için performansa odaklı bir metodolojiye yo l açar. Teorik yaklaşım ise, zekayı anlama kapasitemizi genişletmek olduğu iç in simülasyona odaklı bir metodolojiye yol açar ve bu nedenle vurgu, dışsal p erformans yerine temel süreç üzerinedir.

Örnek olarak, doğal dil işleme ve dilbilim alanlarını düşünün. Bu alanlar birbirine yakından bağlıdır ve birbirlerinden araştırma yaparak fayda sağlarl ar, ancak temel hedefler farklıdır. Dilbilimciler, insanların dili nasıl işlediğini ö ğrenmekle ilgilenirler ve bu nedenle daha teorik uğraşlara yönelirler.

Şekil **11.2**Bulmaca çözme makinemiz



Doclingo https://www.doclingo.ai

Doğal dil işleme alanındaki araştırmacılar, doğal dili manipüle edebilen maki neler geliştirmekle ilgilenirler ve bu nedenle mühendislik yönüne eğilim göst erirler. Böylece, dilbilimciler simülasyon odaklı bir modda çalışırlar - teorileri test etmek amacıyla sistemler inşa ederler. Buna karşılık, doğal dil işleme ara ştırmacıları performans odaklı bir modda çalışırlar - görevleri yerine getirme k için sistemler inşa ederler. Bu son modda üretilen sistemler (belge çevirme nleri ve makinelerin sözlü komutlara yanıt verdiği sistemler gibi) dilbilimciler tarafından elde edilen bilgiye büyük ölçüde dayanır, ancak genellikle belirli si stemin kısıtlı ortamında işe yarayan "kestirmeler" uygularlar.

Temel bir örnek olarak, dış dünyadan sözlü İngilizce komutları aracılığıyl a talimatlar alan bir işletim sistemi için bir kabuk geliştirme görevini düşünü n. Bu durumda, kabuk (bir ajan) tüm İngilizce dilini düşünmek zorunda değil dir. Daha kesin olarak, kabuk "copy" kelimesinin çeşitli anlamları arasında ay rım yapmak zorunda değildir. (Bir isim mi yoksa bir fiil mi? Plagiarizm anlamı nı mı taşımalı?) Bunun yerine, kabuk yalnızca "copy" kelimesini "rename" ve " delete" gibi diğer komutlardan ayırt etmelidir. Böylece kabuk, görevini yalnız ca girdilerini önceden belirlenmiş ses kalıplarıyla eşleştirerek yerine getirebil ir. Böyle bir sistemin performansı bir mühendis için tatmin edici olabilir, anc ak elde etme şekli bir teorisyen için estetik olarak hoş olmayacaktır.

Turing Testi

Geçmişte, Turing testi (1950'de Alan Turing tarafından önerilmiştir), yapay ze ka alanındaki ilerlemeyi ölçmek için bir kıstas olarak hizmet etmiştir. Bugün Turing testinin önemi azalmış olsa da, yapay zeka folklorunun önemli bir par çası olmaya devam etmektedir. Turing'in önerisi, bir insanın, sorgulayıcı olar ak adlandırdığımız kişinin, bir test konusu ile bir daktilo sistemi aracılığıyla il etişim kurmasına izin vermekti; test konusunun bir insan mı yoksa bir makin e mi olduğu kendisine söylenmeyecekti. Bu ortamda, bir makine, sorgulayıcı nın onu bir insandan ayırt edememesi durumunda akıllı davranıyor olarak ka bul edilecekti. Turing, 2000 yılına kadar makinelerin beş dakikalık bir Turing t estini geçme şansının %30 olacağını öngörmüştü; bu varsayım, şaşırtıcı bir ş ekilde doğru çıktı.

Yapay Zekanın Kökenleri

İnsan davranışını taklit eden makineler inşa etme çabası uzun bir geçmişe sahiptir, an cak birçok kişi modern yapay zeka alanının kökenlerinin 1950'de yattığına katılacaktır. Bu yıl, Alan Turing'in "Hesaplama Makineleri ve Zeka" başlıklı makalesini yayı mladığı yıl olup, bu makalede makinelerin akıllı davranış sergilemek üzere pro gramlanabileceğini önermiştir. Alanın adı—yapay zeka—birkaç yıl sonra, John McCarthy tarafından yazılan ve "yapay zekanın yaz aylarında 1956'da Dartmo uth Koleji'nde incelenmesi" önerisini içeren efsanevi öneride türetilmiştir; bu öneri, "öğrenmenin her yönünün veya zekanın herhangi bir özelliğinin, prensi pte o kadar kesin bir şekilde tanımlanabileceği varsayımını keşfetmek" amacı taşımaktadır.

Doclingo https://www.doclingo.ai

Bir neden, Turing testinin artık zeka için anlamlı bir ölçüt olarak kabul edi lmemesidir ki, korkutucu bir zeka görünümü görece kolaylıkla üretilebilir. İyi bilinen bir örnek, 1960'ların ortalarında Joseph Weizenbaum tarafından geliş tirilen DOCTOR programından kaynaklanmıştır (daha genel bir sistem olan E LIZA'nın bir versiyonu). Bu etkileşimli program, bir psikolojik görüşme gerçe kleştiren bir Rogerian analistin imajını yansıtmak üzere tasarlanmıştır; bilgis ayar analistin rolünü oynarken, kullanıcı hasta rolünü üstlenmiştir.
DOCTOR'un içsel olarak yaptığı tek şey, hastanın yaptığı ifadeleri bazı iyi tanımlan mış kurallara göre yeniden yapılandırmak ve bunları hastaya geri yönlendirmekti. Örneğin, "Bugün yorgunum" ifadesine yanıt olarak, DOCTOR "Bugün neden yorgun olduğunuzu düşünüyorsunuz?" şeklinde yanıt vermiş olabilir. DOCTO R cümle yapısını tanıyamazsa, sadece "Devam et" veya "Bu çok ilginç" gibi bir şeyle yanıt verirdi.

Weizenbaum'un DOCTOR'u geliştirmekteki amacı, doğal dil iletişimini inc elemekle ilgiliydi. Psikoterapi konusu, programın "iletişim kurabileceği" bir o rtam sağlıyordu. Ancak Weizenbaum'un üzüntüsüyle, birkaç psikolog progra mın gerçek psikoterapi için kullanılmasını önerdi. (Rogerian tezi, terapötik ot urum sırasında hastanın, analistin değil, tartışmayı yönlendirmesi gerektiğidi r ve bu nedenle, bir bilgisayarın bir terapist kadar iyi bir tartışma yürütebilec eğini savundular.) Dahası, DOCTOR, anlama imajını o kadar güçlü bir şekilde yansıttı ki, onunla "iletişim kuran" birçok kişi makinenin soru-cevap diyalogu na boyun eğdi. Bir anlamda, DOCTOR Turing testini geçti. Sonuç olarak, hem etik hem de teknik sorunlar gündeme geldi ve Weizenbaum, gelişen teknolo ji dünyasında insan onurunu koruma savunucusu oldu.

Daha yakın tarihli Turing testi "başarıları" arasında, insan kurbanıyla "zeki" diy aloglar kurarak insanı kötü amaçlı yazılım korumasını bırakmaya ikna eden intern et virüsleri bulunmaktadır. Ayrıca, Turing testlerine benzer fenomenler, satranç o ynayan programlar gibi bilgisayar oyunları bağlamında da ortaya çıkmaktadır.
Bu programlar, yalnızca kaba kuvvet tekniklerini (Bölüm 11.3'te tartışacağımı z benzer teknikler) uygulayarak hamleler seçmesine rağmen, bilgisayara kar şı yarışan insanlar genellikle makinenin yaratıcılığa ve hatta bir kişiliğe sahip olduğu hissini yaşarlar. Benzer hisler, fiziksel özellikleri zeki karakteristikler y ansıtan makinelerin yapıldığı robotikte de ortaya çıkmaktadır. Örnekler arası nda, başlarını eğerek veya bir sese tepki olarak kulaklarını kaldırarak sevimli kişilikler yansıtan oyuncak robot köpekler bulunmaktadır.

Sorular ve Alıştırmalar

1. Bir ajanın yapabileceği birkaç "zeki" eylem türünü tanımlayın.

2. Tek bir ışık kaynağı olan karanlık bir odada yerleştirilen bir bitki, ışığa doğru büyür. Bu, zeki bir tepki midir? Bitkinin zekası var mıdır? Peki, o zaman zekanın tanımı nedir?
3. Diyelim ki bir otomat, hangi düğmeye basıldığına bağlı olarak çeşitli ürünler dağıtacak şekilde tasarlanmıştır. Böyle bir makinenin hangi d üğmeye basıldığını "fark ettiğini" söyler misiniz? O zaman, farkındalığ ın tanımı nedir?

Doclingo https://www.doclingo.ai

4. Eğer bir makine Turing testini geçerse, onun zeki olduğunu kabul eder misiniz? Eğer hayırsa, onun zeki göründüğünü kabul eder misiniz?
5. Diyelim ki bir sohbet odasında birisiyle İnternet üzerinden sohbet ettiniz (veya Anlık Mesajlaşma kullandınız) ve on dakika boyunca anlamlı bir tutarlı sohbet gerçekleştirdiniz. Daha sonra bir makineyle konuştuğunuzu öğrendiğinizde, makinenin zeki olduğunu mu sonucuna varırsınız?

   Neden veya neden olmasın?

11.2 Algı

Bir ajanın sensörlerinden gelen girdilere akıllıca yanıt verebilmesi için, o girdi yi anlayabilmesi gerekir. Yani, ajanın algılayabilmesi gerekir. Bu bölümde, al gılamada özellikle zorlayıcı olduğu kanıtlanmış iki araştırma alanını inceliyor uz—görüntüleri ve dili anlama.

Görüntüleri Anlama

Önceki bölümde tanıtılan bulmaca çözme makinesinin ortaya koyduğu soru nları ele alalım. Makinenin kavrayıcısının açılıp kapanması ciddi bir engel teş kil etmemektedir ve bu süreçte bulmacanın kavrayıcıda varlığını tespit etme yeteneği, uygulamamızın çok az hassasiyet gerektirmesi nedeniyle basittir. H atta, kameranın bulmacaya odaklanma sorunu, kavrayıcının bulmacayı belirli bir önceden belirlenmiş konumda görüntüleme için konumlandırılacak şekil de tasarlanmasıyla basitçe halledilebilir. Sonuç olarak, makinenin gerektirdiğ i ilk akıllı davranış, görsel bir ortam aracılığıyla bilgi çıkarmaktır.

Makinemizin bulmacaya bakarken karşılaştığı sorunun yalnızca bir görün tü üretmek ve depolamak olmadığını anlamak önemlidir.
Teknoloji, geleneksel fotoğrafçılık ve televizyon sistemleri gibi yıllardır bunu yapabiliyor. Bunun yerine, sorun görüntüyü anlamak ve bulmacanın mevcut durumunu çıkarmaktır (ve belki de daha sonra karoların hareketini izlemekti r).

Bulmacayı çözme makinemiz durumunda, bulmaca görüntüsünün olası yorumları görece sınırlıdır. Görüntüde her zaman 1'den 8'e kadar olan raka mları içeren iyi organize edilmiş bir desenin bulunduğunu varsayabiliriz. Sor un, bu rakamların düzenini çıkarmaktır. Bunun için, bulmacanın görüntüsün ün bilgisayarın belleğinde bitler cinsinden kodlandığını, her bitin belirli bir pi kselin parlaklık seviyesini temsil ettiğini hayal ediyoruz.
Görüntünün uniform bir boyutunu varsayarak (makine bulmacayı kameranın önünd e önceden belirlenmiş bir konumda tutar), makinemiz, görüntünün farklı bölümlerin i bulmacada kullanılan bireysel rakamların ürettiği bit desenlerinden oluşan öncede n kaydedilmiş şablonlarla karşılaştırarak hangi karonun hangi pozisyonda olduğunu tespit edebilir. Eşleşmeler bulunduğunda, bulmacanın durumu ortaya çıkar.

Görüntüleri tanıma tekniği, optik karakter okuyucularında kullanılan bir yöntemdir. Ancak, okunan sembollerin stil, boyut ve yönelimi için belirli bir u niformite gerektirmesi gibi bir dezavantajı vardır. Özellikle, fiziksel olarak bü yük bir karakterin ürettiği bit deseni, aynı sembolün daha küçük bir versiyon u için şablonla eşleşmez, şekiller aynı olsa bile. Ayrıca, el yazısı materyali işle meye çalışırken sorunların zorluk seviyesinin nasıl arttığını hayal edebilirsiniz .

Doclingo https://www.doclingo.ai

Karakter tanıma sorununa başka bir yaklaşım, sembollerin tam görünüm ünden ziyadegeometrik özellikleri eşleştirmeye dayanmaktadır. Bu tür duru mlarda, rakam 1tek bir dikey çizgi olarak tanımlanabilir, 2 ise alt kısımda yat ay bir düz çizgiylebirleşmiş açık bir eğri çizgi olarak tanımlanabilir ve devam edebilir. Sembolleri tanımabu yöntemi iki adım içerir: birincisi, işlenen görün tüden özellikleri çıkarmaktır,ikincisi ise bu özellikleri bilinen sembollerin özell ikleriyle karşılaştırmaktır. Şablon-eşleştirmeyaklaşımında olduğu gibi, karakt erleri tanıma tekniği de kusursuz değildir. Örneğin,görüntüdeki küçük hatala r, O ile C'yi ayırt etme veya sekiz bulmacasında 3 ile 8'iayırma durumunda ol duğu gibi tamamen farklı geometrik özellikler seti üretebilir.

Bulmaca uygulamamızda şanslıyız çünkü genel üç boyutlu sahnelerin görü ntülerini anlamamız gerekmiyor. Örneğin, tanınması gereken şekillerin (1'den 8'e kadar olan rakamlar) resmin farklı kısımlarında izole edilmiş olduğundan e min olmamızın sağladığı avantajı düşünün; bu, daha genel ortamlarda yaygın o larak görülen üst üste binen görüntüler olarak görünmekten ziyade. Genel bir f otoğrafta, örneğin, bir nesneyi farklı açılardan tanıma problemiyle karşılaşmanı n yanı sıra, nesnenin bazı kısımlarının görünümden gizlenmiş olabileceği gerçe ğiyle de karşılaşılır.

Genel görüntüleri anlama görevi genellikle iki aşamalı bir süreç olarak el e alınır: (1) görüntü işleme, bu görüntünün özelliklerini tanımlamayı ifade ed er ve (2) görüntü analizi, bu özelliklerin ne anlama geldiğini anlama sürecini i fade eder. Bu ikiliği, sembolleri geometrik özellikleri aracılığıyla tanıma bağla mında zaten gözlemledik. O durumda, görüntü işleme, görüntüde bulunan g eometrik özellikleri tanımlama süreciyle temsil edilirken, görüntü analizi bu özelliklerin anlamını tanımlama süreciyle temsil edilmiştir.

Görüntü işleme birçok konuyu içerir. Bunlardan biri, bir görüntüdeki böl geler arasındaki sınırları netleştirmek için matematiksel tekniklerin uyguland ığı kenar iyileştirmedir. Bir anlamda, kenar iyileştirme bir fotoğrafı çizgi resm e dönüştürme girişimidir. Görüntü analizindeki bir diğer etkinlik ise

Güçlü AI ile Zayıf AI

Makinaların zeki davranış sergileyecek şekilde programlanabileceği varsayımı, zayıf AI olar ak bilinir ve günümüzde geniş bir kitle tarafından farklı derecelerde kabul edilmektedir. Ancak, makinelerin zeka ve aslında bilinç sahibi olacak şekilde programlanabil eceği varsayımı, güçlü AI olarak bilinir ve geniş bir tartışma konusudur. Güçlü AI karşıtları, bir makinenin doğası gereği bir insandan farklı olduğunu ve bu nedenle asla sevgi hissedemeyeceğini, doğru ile yanlışı ayırt edemeyeceğini ve kendisi hakkında bir i nsan gibi düşünemeyeceğini savunurlar. Ancak, güçlü AI'nın savunucuları, insan zihninin bir eysel olarak insan olmayan ve bilinçli olmayan küçük bileşenlerden oluştuğunu, ancak bir ar aya geldiğinde bilinçli hale geldiğini iddia ederler. Aynı fenomenin makinelerle neden müm kün olamayacağını savunurlar?

Güçlü AI tartışmasını çözmedeki sorun, zeka ve bilinç gibi niteliklerin do ğrudan tanımlanamayacak içsel özellikler olmasıdır. Alan Turing'in belirttiği gibi, diğer insanlara zeka atfederiz çünkü akıllıca davranırlar—içsel zihinsel durumlarını gözlemleyemesek bile. O zaman, bir makine dışsal bilinç özelli klerini sergilediğinde aynı esnekliği ona vermeye hazır mıyız? Neden veya n eden olmasın?

Doclingo https://www.doclingo.ai

bölge bulma. Bu, bir görüntüde parlaklık, renk veya doku gibi ortak özellikler e sahip olan alanları tanımlama sürecidir. Böyle bir bölge muhtemelen tek bi r nesneye ait olan görüntünün bir bölümünü temsil eder. (Bölge tanıma yete neği, bilgisayarların eski siyah-beyaz hareketli resimlere renk eklemesine ola nak tanır.) Görüntü işleme kapsamındaki bir diğer etkinlik ise düzleştirmedir; bu, görüntüdeki kusurları giderme sürecidir. Düzleştirme, görüntüdeki hatal arın diğer görüntü işleme adımlarını karıştırmasını önler, ancak aşırı düzleşti rme önemli bilgilerin kaybına neden olabilir.

Düzleştirme, kenar iyileştirme ve bölge bulma, bir görüntüdeki çeşitli bil eşenleri tanımlama adımlarıdır. Görüntü analizi, bu bileşenlerin neyi temsil e ttiğini ve nihayetinde görüntünün ne anlama geldiğini belirleme sürecidir. B urada, farklı açılardan kısmen engellenmiş nesneleri tanıma gibi sorunlarla k arşılaşılır. Görüntü analizine bir yaklaşım, görüntünün ne olabileceği hakkınd a bir varsayımla başlamaktır ve ardından görüntüdeki bileşenleri varsayılan nesnelerle ilişkilendirmeye çalışmaktır. Bu, insanların uyguladığı bir yaklaşım gibi görünmektedir. Örneğin, bazen bulanık bir görüş alanında beklenmedi k bir nesneyi tanımakta zorlanırız, ancak nesnenin ne olabileceğine dair bir i pucu aldığımızda, onu kolayca tanıyabiliriz.

Genel görüntü analizi ile ilgili sorunlar büyüktür vebu alanda yapılması gerek en çok fazla araştırma bulunmaktadır. Gerçekten de, görüntü analizi, insan zihni t arafından hızlı ve görünüşte kolay bir şekilde gerçekleştirilen görevlerin makinele rin yeteneklerini nasıl zorlamaya devam ettiğini gösteren alanlardan biridir.

Dil İşleme

Karşılaşılan bir diğer algı problemi, dili anlama meselesidir. Resmi yüksek se viyeli programlama dillerinin makine diline çevrilmesinde elde edilen başarı (Bölüm 6.4), erken araştırmacıların bilgisayarları doğal dili anlamaya program lama yeteneğinin sadece birkaç yıl uzakta olduğuna inanmasına yol açtı. Ger çekten de, programları çevirebilme yeteneği, makinenin çevrilen dili gerçekt en anladığı yanılsamasını yaratır. (Bölüm 6.1'den hatırlayın, Grace Hopper'ın yöneticilerin bilgisayarlara Almanca öğretmeye çalıştığını düşündüğü hikaye sini.)

Bu araştırmacıların anlamadığı şey, resmi programlama dillerinin İngilizc e, Almanca ve Latince gibi doğal dillerden ne kadar farklı olduğunun derinliğ iydi. Programlama dilleri, her ifadenin yalnızca bir dilbilgisel yapıya ve yalnızc a bir anlama sahip olacak şekilde iyi tasarlanmış ilkelere dayanarak inşa edil miştir. Buna karşılık, doğal bir dildeki bir ifade, bağlamına veya hatta iletilme şekline bağlı olarak birden fazla anlama sahip olabilir. Bu nedenle, doğal dili anlamak için insanlar ek bilgiye büyük ölçüde dayanır.

Örneğin, cümleler

> Norman Rockwell insanların resmini yaptı.

ve

> Sindirella bir balo yaptı.

birçok anlamı vardır ve bu anlamlar her birkelimeyi bağımsız olarak ayrıştıra rak veya çevirerek ayırt edilemez. Bunun yerine, bu cümleleri anlamak, ifade nin yapıldığı bağlamı kavrama yeteneğini gerektirir. Diğer durumlarda, bir c ümlenin gerçek anlamı, kelime kelime çevirisiyle aynı değildir. Örneğin,Saa

> t kaç biliyor musun?

Doclingo https://www.doclingo.ai

sıklıkla "Lütfen bana saat kaç olduğunu söyle" anlamına gelir veya eğer konuşmacı uzun bir süre beklemişse,"Çok geç kaldın" anlamına gelebilir.

Bu nedenle, doğal bir dilde bir ifadenin anlamını çözmek, bir dizi analiz seviyesini gerektirir. Bunların ilki sözdizimsel analizdir, bunun ana bileşeni ayrıştırmadır. Burada cümlenin öznesi

Mary, John'a bir doğum günü kartı verdi.

olarak tanınırken,

John, Mary'den bir doğum günü kartı aldı.

cümlesinin öznesi *John olarak bulunur.*

Bir diğer analiz seviyesi anlamsal analizdir. Ayrıştırma sürecinin aksine, her kelimenin dilbilgisel rolünü yalnızca tanımlayan anlamsal analiz,eğer cümledeki her kelimenin anlamsal rolünü tanımlama görevini üstlenir. Anlamsal analiz, tanımlanması gereken şeyler arar,betimlenen eylem, o eylemin ajanı (cümlenin öznesi olup olmaması fark etmeksizin) ve eylemin nesnesi gibi. Anlamsal analiz sayesinde "Mary, John'a bir doğum günü kartı verdi" ve "John , Mary'den bir doğum günü kartı aldı" cümleleri aynı şeyi söylediği olarak tanınır.

Üçüncü bir analiz seviyesi, bağlamsal analizdir. Bu seviyede, cümlenin bağlamı anlama sürecine dahil edilir. Örneğin, cümledeki her kelimenin dilbilgisel rolünü tanımlamak kolaydır.

Yarasa yere düştü.

Hatta düşme eylemini, yarasa *olarak tanımlayarak anlamsal analiz bile yapabiliriz.* Ancak, ifadenin bağlamını dikkate almadıkça,ifadenin anlamı netleşmez. Özellikle, bir beyzbol oyunu bağlamında farklı bir anlamı vardır,mağara keşfi bağlamında farklı bir anlamı vardır. Dahası, bağlamsal seviyede "Saat kaç olduğunu biliyor musun?" sorusunun gerçek anlamı nihayet ortaya çıkacaktır.

Çeşitli analiz seviyelerinin—sentaktik, anlamsal ve bağlamsal—mutlaka bağımsız olmadığını belirtmeliyiz. Cümlenin öznesi

Bozulan sığırlar tehlikeli olabilir.

eğer sığırların kendi başlarına bozulduğunu hayal edersek, sığır (sıfat bozulan ile değiştirilmiş) isimdir. Ancak, özne, bir sorun çıkarıcının eğlencesininbozulanları başlatmak olduğu bağlamda bozulan (nesne sığır ile) fiilimsidir. Böylece cümlenin birden fazla dilbilgisel yapısı vardır—hangisinin doğru olduğu bağlama bağlıdır.

Doğal dil işleme alanındaki bir diğer araştırma alanı, bireysel cümleler yerine bir bütün olarakdokümanlarla ilgilidir. Burada endişe duyulan problemler iki kategoriye ayrılır: bilgi retrievalvebilgi extraction.

Bilgi retrieval, mevcut konu ile ilgili belgeleri tanımlama görevini ifade eder. Bir örnek, kullanıcıların belirli bir konu ile ilgili siteleri bulmaya çalışırken karşılaştığısorundur. Mevcut en iyi uygulama, siteleri anahtar kelimeler için aramaktır, ancak bu genellikleyanlış yönlendirmelerin bir çığını üretir ve "arabalar" yerine "otomobiller" ile ilgili olduğu içinönemli bir siteyi gözden kaçırabilir. Gerekli olan, dikkate alınan sitelerin içeriğini anlayanbir arama mekanizmasıdır. Böyle bir anlayışın elde edilmesinin zorluğu, birçok kişininBölüm 4.3'te tanıtılan anlamlı bir Web üretmek için XML gibi tekniklere yönelmesinin nedenidir.

Doclingo https://www.doclingo.ai

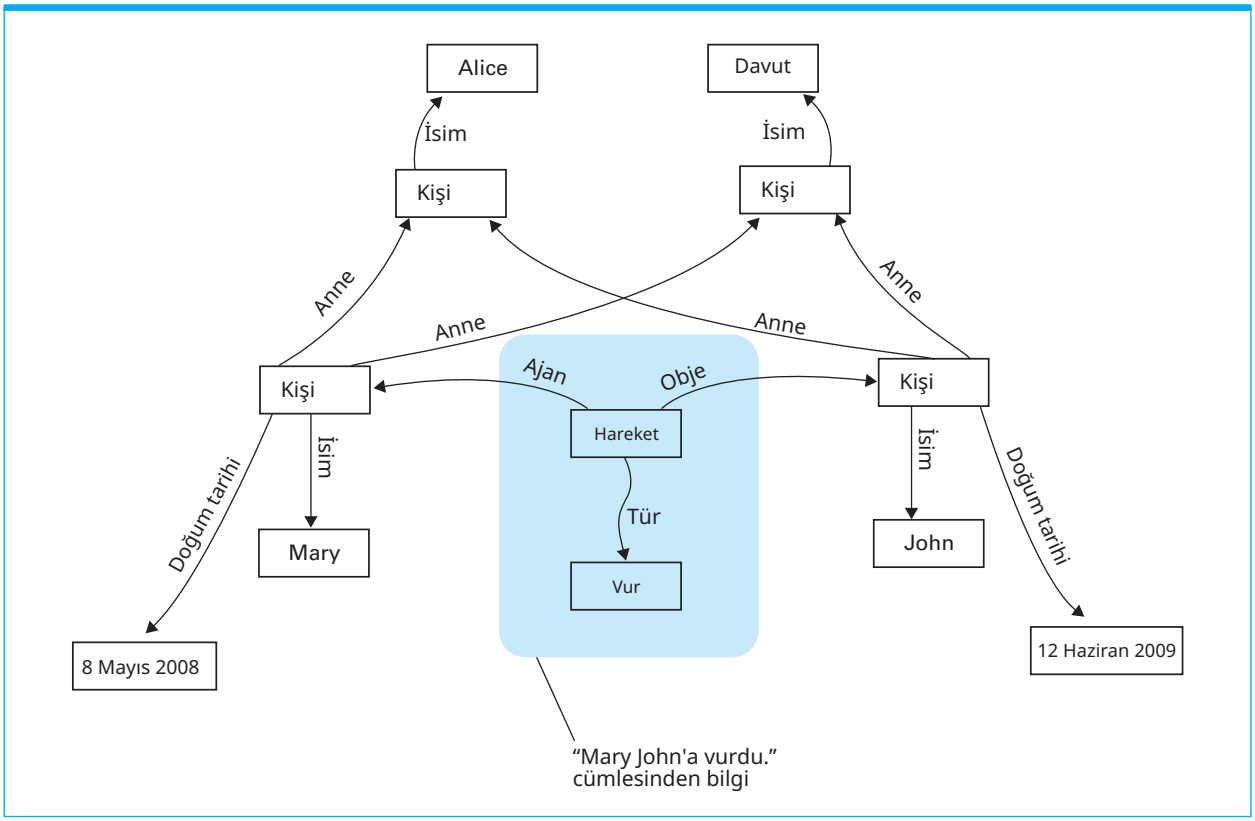
Bilgi extraction, belgelerden bilgiyi çıkarma görevini ifade eder, böylece bu bilgibaşka uygulamalarda faydalı bir form alır. Bu, belirli bir sorunun ceva bını tanımlamayıveya bilgiyi daha sonra soruların cevaplanabileceği bir form da kaydetmeyianlamına gelebilir. Bu tür bir form, çerçeve olarak bilinir; bu, e sasenözelliklerin kaydedildiği bir şablondur. Örneğin, bir gazete okuma siste mi düşünün.Sistem, bir gazete içinde yer alabilecek her tür makale için bir çe rçeve kullanabilir.Eğer sistem, bir makaleyi bir hırsızlık raporu olarak tanımla rsa, hırsızlık çerçevesindekiboşlukları doldurmaya çalışarak devam eder. Bu ç erçeve muhtemelen hırsızlığın adresi,hırsızlığın zamanı ve tarihi, alınan eşyal ar gibi öğeleri talep edecektir. Buna karşılık,eğer sistem, bir makaleyi doğal bir felaket raporu olarak tanımlarsa, doğal felaket çerçevesinidolduracak ve bu da sistemi felaketin türünü, zarar miktarını vb. tanımlamaya yönlendirece ktir.

Bilgi çıkarıcılarının bilgileri kaydettiği bir diğer biçim, bir anlamsal ağ olarak bilinir. Bu, esasen, veri öğeleri arasındaki ilişkileribelirtmek için işaretçilerin kul lanıldığı büyük bir bağlı veri yapısıdır. Şekil 11.3, cümleden elde edilen bilgilerin vurgulandığı bir anlamsal ağın bir kısmını göstermektedir

Mary John'a vurdu.

vurgulanmıştır.

Şekil **11.3** Bir anlamsal ağ



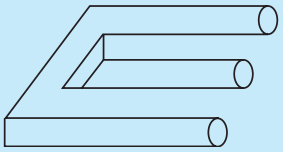
Doclingo https://www.doclingo.ai

Elinizin Avucunda Yapay Zeka

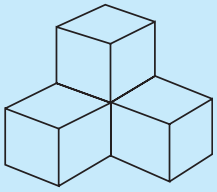
Yapay zeka teknikleri, akıllı telefon uygulamalarında giderek daha fazla görünmeye başlıyor. Ör neğin, Google, görsel bir arama motoru sağlayan Google Goggles adlı bir akıllı telefon uygulam ası geliştirdi. Bir kitabın, simgenin veya tabelanın fotoğrafını akıllı telefonun kamerasıyla çekin v e Goggles, görüntü işleme, görüntü analizi ve metin tanıma gerçekleştirecek ve ardından nesne yi tanımlamak için bir Web araması başlatacaktır. Eğer Fransa'da bulunan bir İngilizce konuşuy orsanız, bir tabelanın, menünün veya diğer metinlerin fotoğrafını çekebilir ve bunun İngilizceye çevrilmesini sağlayabilirsiniz. Goggles'ın ötesinde, Google sesli dil çevirisi üzerinde aktif olarak ç alışıyor. Yakında telefonunuza İngilizce konuşacak ve kelimelerinizin İspanyolca, Çince veya baş ka bir dilde söylenmesini sağlayabileceksiniz. Akıllı telefonlar, yapay zekanın yenilikçi yollarla kul lanılmaya devam etmesiyle kesinlikle daha akıllı hale gelecektir.

Sorular ve Alıştırmalar

1. Bir robotun video sisteminin gereksinimleri, robotun kendi aktivitelerini k ontrol etmek için kullanması durumunda, uzaktan robotu kontrol eden bi r insana iletmekten nasıl farklılık gösterir?
2. Aşağıdaki çizimin saçmalık olduğunu size ne söylüyor? Bu içgörü bir makineye nasıl programlanabilir?



3. Sonraki temsil edilen yığındaki kaç blok var? Bir makine, böyle sorulara doğru bir şekilde yanıt vermek için nasıl programlanabilir?



4. "Hiçbir şey, tam mutluluktan daha iyi değildir" ve "Soğuk bir çorba, hi çbir şeyden daha iyidir" ifadelerinin "Soğuk bir çorba, tam mutlulukta n daha iyidir" anlamına gelmediğini nasıl biliyorsunuz? Bu ayrımı yap abilme yeteneğiniz bir makineye nasıl aktarılabilir?
5. "Onlar yarış atı koşuyorlar." cümlesinin çevrilmesinde yer alan belirsizlikle ri tanımlayın.
6. Aşağıdaki iki cümlenin ayrıştırma sonuçlarını karşılaştırın. Ardından, cümlelerin anl amsal olarak nasıl farklılık gösterdiğini açıklayın.

   Çiftçi tarlada çiti inşa etti.

   Çiftçi kışın çiti inşa etti.

Doclingo https://www.doclingo.ai

7. Şekil 11.3'teki anlamsal ağa dayanarak, Mary ile John arasındaki aile i lişkisi nedir?

11.3 Akıl Yürütme

Şimdi, Bölüm 11.1'de tanıtılan bulmaca çözme makinesini kullanarak, temel akıl y ürütme yeteneklerine sahip ajanlar geliştirme tekniklerini keşfedelim.

Üretim Sistemleri

Bulmacayı çözme makinemiz, görsel görüntüden karoların konumlarını çözd ükten sonra, görevi bulmacayı çözmek için hangi hamlelerin gerektiğini belir lemektir. Bu probleme akla gelebilecek bir yaklaşım, makineyi karoların tüm olası düzenlemeleri için çözümlerle önceden programlamaktır.
O zaman makinenin görevi yalnızca uygun programı seçmek ve yürütmek olacaktır. Ancak, sekiz bulmacanın 100.000'den fazla konfigürasyonu vardır, bu nedenle her bi ri için açık bir çözüm sağlama fikri cazip değildir. Bu nedenle, amacımız makineyi sek iz bulmacaya kendi başına çözümler oluşturacak şekilde programlamaktır. Yani, mak ine temel akıl yürütme faaliyetlerini gerçekleştirecek şekilde programlanmalıdır.

Bir makinede akıl yürütme yeteneklerinin geliştirilmesi, yıllardır araştırm a konusu olmuştur. Bu araştırmanın sonuçlarından biri, ortak özelliklere sahi p büyük bir akıl yürütme problemi sınıfının tanınmasıdır.
Bu ortak özellikler, üretim sistemi olarak bilinen soyut bir varlıkta izole edilm iştir; bu sistem üç ana bileşenden oluşur:

1. Bir dizi durum. Her durum, uygulama ortamında meydana gelebilecek bir durumdu r. Başlangıç durumu, başlangıç (veya ilk) durumu olarak adlandırılır; istenen durum (veya durumlar) ise hedef durum olarak adlandırılır. (Bizim durumumuzda, bir duru m sekiz bulmacanın bir konfigürasyonudur; başlangıç durumu, bulmacanın makine ye verildiği zamanki konfigürasyonudur; hedef durum ise çözülmüş bulmacanın kon figürasyonudur, bu Şekil 11.1'de gösterilmiştir.)
2. *Üretimlerin (kurallar veya hamleler) bir koleksiyonu.* Bir üretim, bir du rumdan diğerine geçmek için uygulama ortamında gerçekleştirilebilec ek bir işlemdir.Her üretim, ön koşullarla ilişkilendirilebilir; yani, bir üre timin uygulanabilmesi için ortamda mevcut olması gereken koşullar ol abilir. (Bizim durumumuzda üretimler, karoların hareketleridir. Bir kar onun her hareketinin, ilgili karonun yanındaki boşluğun mevcut olma ön koşulu vardır.)
3. *Bir kontrol sistemi.* Kontrol sistemi, başlangıç durumundan hedef durumu na geçiş sorununu çözen mantıktan oluşur. Süreçteki her adımda kontrol si stemi, ön koşulları sağlanan üretimlerden hangisinin bir sonraki adımda uy gulanması gerektiğine karar vermelidir. (Sekiz bulmacamızda belirli bir dur um verildiğinde, boşluğun yanında birkaç karo olacağından, dolayısıyla bir kaç uygulanabilir üretim olacaktır. Kontrol sistemi hangi karoyu hareket ett ireceğine karar vermelidir.)

Atanan görevin, bulmaca çözme makinemiz bağlamında bir üretim siste mi olarak formüle edilebileceğini unutmayın. Bu bağlamda kontrol sistemi,

Doclingo https://www.doclingo.ai

bir programın. Bu program, sekiz bulmacanın mevcut durumunu inceler, hedef d uruma ulaşan bir üretim dizisini tanımlar ve bu diziyi uygular. Bu nedenle, sekiz b ulmacayı çözmek için bir kontrol sistemi tasarlamak bizim görevimizdir.

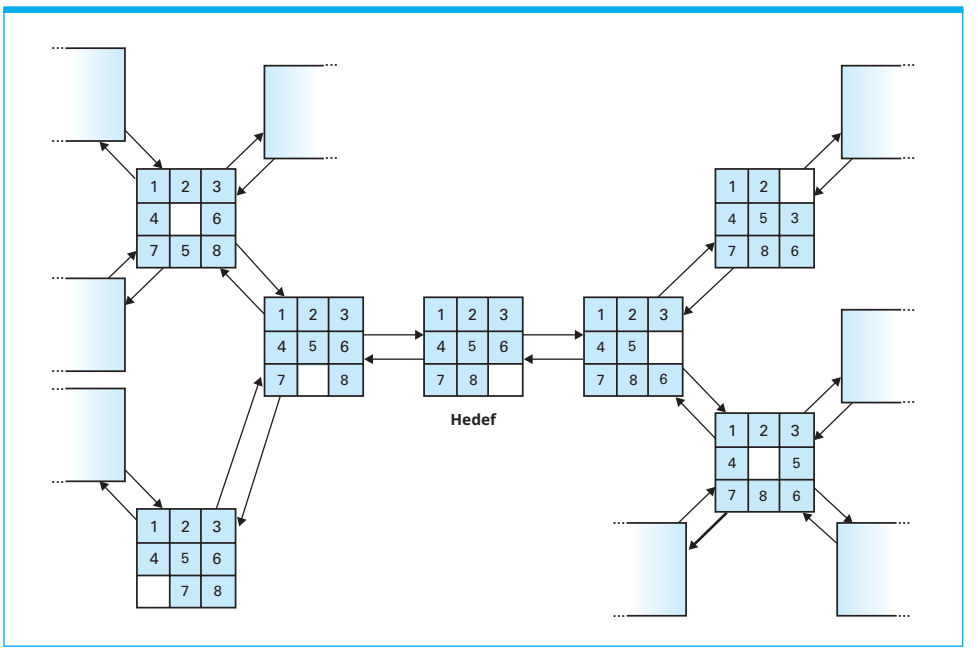
Bir kontrol sistemi geliştirmedeki önemli bir kavram, bir problem alanıdı r; bu, bir üretim sistemindeki tüm durumların, üretimlerin ve ön koşulların k oleksiyonudur. Bir problem alanı genellikle bir durum grafiği biçiminde kavr amsallaştırılır. Burada grafik terimi, matematikçilerin yönlendirilmiş grafik ol arak adlandırdığı bir yapıyı ifade eder; bu, düğüm olarak adlandırılan yerleri n oklarla bağlandığı bir koleksiyondur. Bir durum grafiği, sistemdeki duruml arı temsil eden düğümlerin bir koleksiyonundan oluşur ve sistemin bir duru mdan diğerine geçişini temsil eden oklarla birbirine bağlıdır. İki düğüm, dur um grafiğinde yalnızca, okun başlangıç noktasındaki durumdan okun varış n oktasındaki duruma sistemi dönüştüren bir üretim varsa okla bağlıdır.

Olası durumların sayısının, sekiz bulmacaya önceden programlanmış çöz ümler sunmamızı engellediği gibi, büyüklük problemi de tüm durum grafiği ni açıkça temsil etmemizi engellemektedir.

Bir durum grafiği, dolayısıyla mevcut problemi kavramsallaştırmanın bir yolu dur, ancakbunu tamamen çizmeyi düşünmeyiz. Yine de, Şekil 11.4'te gösteril en sekiz bulmacanınbir durum grafiğinin kısmını düşünmek (ve muhtemelen genişletmek) faydalı olabilir.

Durum grafiği açısından bakıldığında, kontrol sisteminin karşılaştığı prob lem,bir başlangıç durumundan hedef durumuna giden oklar dizisini bulmakt ır. Sonuçta,bu oklar dizisi, orijinal problemi çözen bir üretim dizisini temsil ed er. Böylece,uygulamadan bağımsız olarak, kontrol sisteminin görevi bir duru m grafiği boyuncabir yol bulmak olarak görülebilir. Kontrol sistemlerinin bu evrensel görüşü,üretim sistemleri açısından akıl yürütme gerektiren proble mleri analiz ederek eldeettiğimiz ödüldür. Eğer bir problem bir üretim siste mi açısından karakterize edilebiliyorsa,o zaman çözümü bir yol arayışı açısın dan formüle edilebilir.

Şekil **11.4** Sekiz bulmacanın durum grafiğinin küçük bir kısmı



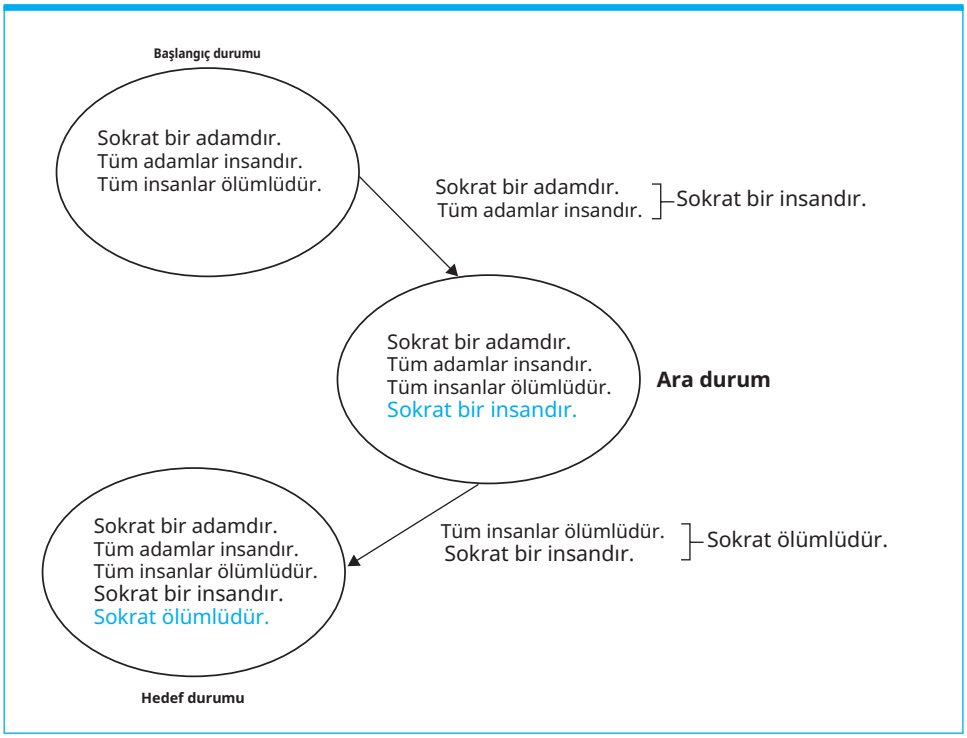
Doclingo https://www.doclingo.ai

Bu noktayı vurgulamak için, diğer görevlerin nasıl üretim sistemleri açısından çerçevelenebileceğini ve böylece kontrol sistemlerinin durum grafikleri boyunca yollar bulma bağlamında nasıl gerçekleştirilebileceğini düşünelim. Yapay zeka alanındaki klasik problemlerden biri, satranç gibi oyunlar oynamaktır. Bu oyunlar, iyi tanımlanmış bir bağlamda orta düzeyde karmaşıklık içerir ve bu nedenle theorileri test etmek için ideal bir ortam sağlar. Satrançta, temel üretim sisteminin durumları olası tahta konfigürasyonlarıdır, üretimler taşların hamleleridir ve kontrol sistemi oyunculardaki (insan veya başka) somutlaşır. Durum grafiğinin başlangıç düğümü, taşların başlangıç pozisyonlarındaki tahtayı temsil eder. Bu düğümden dallanan oklar, bir oyundaki ilk hamleden sonra ulaşılabilecek tahta konfigürasyonlarına yönelir; bu konfigürasyonlardan dallanan oklar, bir sonraki hamleyle ulaşılabilecek konfigürasyonlara yönelir; ve bu şekilde devam eder. Bu formülasyonla, bir satranç oyununu, her birinin kendi seçtiği bir hedef düğümüne giden büyük bir durum grafiği boyunca bir yol bulmaya çalışan iki oyuncu olarak hayal edebiliriz.

Belki de bir üretim sisteminin daha az belirgin bir örneği, verilen gerçeklerde n mantıksal sonuçlar çıkarma problemidir. Bu bağlamdaki üretimler, eski ifadeler den yeni ifadelerin oluşturulmasına izin veren mantık kurallarıdır ki bunlara çıkarı m kuralları denir. Örneğin, "Tüm süper kahramanlar soyludur" ve "Süperman bir s üper kahramandır" ifadeleri birleştirilerek "Süperman soyludur" sonucuna ulaşıla bilir. Böyle bir sistemdeki durumlar, çıkarım sürecinin belirli noktalarında doğru ol duğu bilinen ifade koleksiyonlarından oluşur: Başlangıç durumu, sonuçların çıkarı lacağı temel ifadelerin (genellikle aksiyomlar olarak adlandırılır) koleksiyonudur v e hedef durumu, önerilen sonucu içeren herhangi bir ifade koleksiyonudur.

Bir örnek olarak, Şekil 11.5, "Sokrat ölümlüdür" sonucunun çıkarıldığı ifade ko leksiyonu üzerinden geçilebilecek bir durum grafiğinin kısmını göstermektedir.

Şekil **11.5** Bir üretim sistemi bağlamında çıkarımsal akıl yürütme



Doclingo https://www.doclingo.ai

"Sokrat bir adamdır," "Tüm adamlar insandır," ve "Tüm insanlar ölümlüdür."Burada bilgi birikiminin bir durumdan diğerine geçişini görüyoruz, çünkü akıl yürütme süreci uygun üretimleri uygulayarak ek ifadeler üretmektedir.

Bugün, genellikle mantık programlama dillerinde (Bölüm 6.7) uygulanan bu tür akıl yürütme sistemleri, insan uzmanların aynı durumlarla karşılaştıklarında izleyecekleri neden-sonuç akıl yürütmesini simüle etmek için tasarlanmış yazılım paketleri olan çoğu uzman sistemin belkemiğidir. Örneğin, tıbbi uzman sistemler, hastalıkları teşhis etme veya tedavi geliştirme konusunda yardımcı olmak için kullanılır.

Arama Ağaçları

Bir üretim sistemi bağlamında, bir kontrol sisteminin görevinin, başlangıç düğümünden bir hedefe giden bir yol bulmak için durum grafiğini aramak olduğunu gördük. Bu aramayı gerçekleştirmenin basit bir yöntemi, başlangıç durumundan çıkan her bir ok boyunca ilerlemek ve her durumda varış durumunu kaydetmek, ardından bu yeni durumları terk eden okları izlemek ve sonuçları tekrar kaydetmektir. Hedefe yapılan arama, başlangıç durumundan suya düşen bir boya damlası gibi yayılır. Bu süreç, yeni durumlardan biri bir hedef olana kadar devam eder; bu noktada bir çözüm bulunmuş olur ve kontrol sistemi, başlangıç durumundan hedefe kadar keşfedilen yol boyunca üretimleri uygulamakla yükümlüdür.

Bu stratejinin etkisi, kontrol sistemi tarafından araştırılan durum grafiğinin bir parçasını içeren bir ağaç, arama ağacı, oluşturmaktır.
Araştırma ağacının kök düğümü başlangıç durumudur ve her düğümün çocukları, bir üretim uygulayarak ebeveynden ulaşılabilen durumlardır. Araştırma ağacındaki düğümler arasındaki her yay, tek bir üretimin uygulanmasını temsil eder ve kökten bir yaprağa giden her yol, durum grafiğindeki karşılık gelen durumlar arasındaki bir yolu temsil eder.

Şekil 11.6'da gösterilen konfigürasyondan sekiz bulmacayı çözmek için üretilecek araştırma ağacı Şekil 11.7'de gösterilmiştir. Bu ağacın en soldaki dalı, problemi çözmek için önce 6 taşını yukarı hareket ettirme girişimini temsil eder, ortadaki dal 2 taşını sağa hareket ettirme yaklaşımını temsil eder ve en sağdaki dal 5 taşını aşağı hareket ettirmeyi temsil eder.
Ayrıca, araştırma ağacı, 6 taşını yukarı hareket ettirmeye başlarsak, bir sonraki izin verilen tek üretimin 8 taşını sağa hareket ettirmek olduğunu gösterir. (Aslında, o noktada 6 taşını aşağı da hareket ettirebiliriz ama bu, önceki üretimi tersine çevirir ve dolayısıyla gereksiz bir hamle olur.)

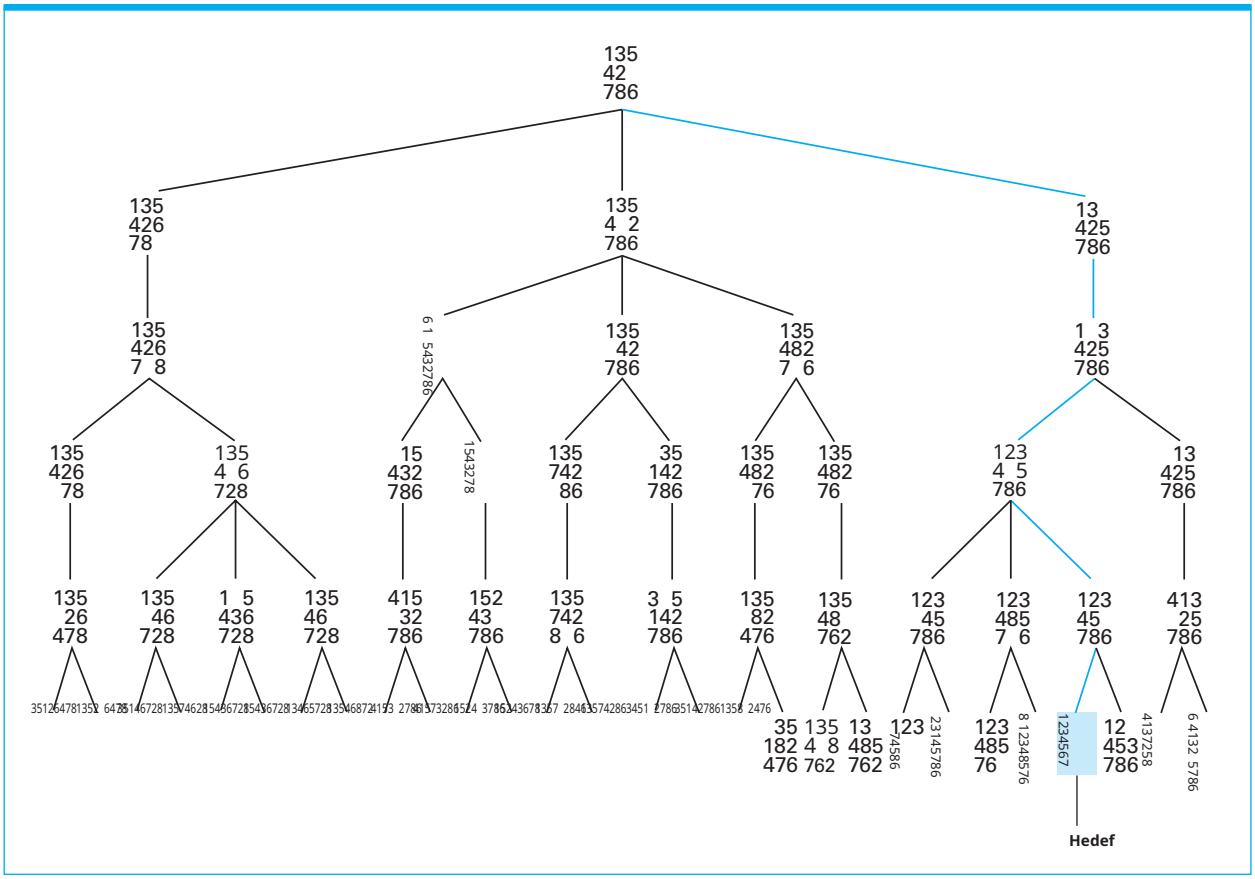
Amaç durumu, Şekil 11.7'deki araştırma ağacının son seviyesinde gerçekleşir. Bu, bir çözüm bulunduğunu gösterdiğinden, kontrol sistemi arama prosedürünü sonlandırabilir ve bulmacayı dış ortamda çözmek için kullanılacak talimat dizisini oluşturmaya başlayabilir. Bu, hedef düğümün konumundan araştırma ağacında yukarı yürümek kadar basit bir süreçtir.

Şekil **11.6** Çözülmemiş sekiz bulmaca

1	3	5
4	2	
7	8	6

Doclingo https://www.doclingo.ai

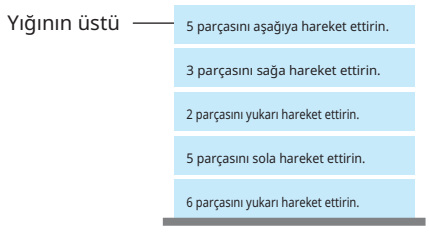
Şekil **11.7** Örnek bir arama ağacı



Ağaç kolları tarafından temsil edilen üretimleri bir yığına iterek karşılaştıkçau ygulamak. Bu tekniği Şekil 11.7'deki arama ağacına uygulamak, Şekil 11.8'de kiüretim yığınını üretir. Kontrol sistemi artık bu yığından çıkarılan talimatları uygulayarak dış dünyadaki bulmacayı çözebilir.

Bir gözlem daha yapmalıyız. 8. Bölümde tartıştığımız ağaçların, ağaç boyunca aşağıya noktalar işaret eden bir gösterge sistemi kullandığını hatırlayın, böylece bir ebeveyn düğümden çocuklarına geçiş yapmamıza olanak tanır. Bir arama durumunda

Şekil **11.8** İleride yürütme için yığılmış üretimler



Doclingo https://www.doclingo.ai

Ağaçta, kontrol sisteminin hedef durumdan başlangıç durumuna doğru yukarı harek et ederken bir çocuktan ebeveyne geçebilmesi gerekir.Bu tür ağaçlar, işaretçi sistem leri aşağıya değil yukarıya işaret edecek şekilde inşa edilir. Yani, her çocuk düğümü e beveynine işaret eden bir işaretçi içerirken, ebeveyn düğümleri çocuklarına işaret ed en işaretçiler içermez. (Bazı uygulamalarda, her iki işaretçi seti de ağaç içinde her iki yönde hareket etmeye izin vermek için kullanılır).

Heuristikler

Şekil 11.7'deki örneğimiz için, yönetilebilir bir arama ağacı üreten bir başlangıç yapılandırması seçtik. Buna karşılık, daha karmaşık bir problemi çözme girişimi nde üretilen arama ağacı çok daha büyük hale gelebilir. Bir satranç oyununda y irmi olası ilk hamle vardır, bu nedenle böyle bir durumda arama ağacının kök d üğümü yirmi çocuğa sahip olacaktır; oysa örneğimizde üç çocuğu vardır.
Ayrıca, bir satranç oyunu kolayca otuz ila otuz beş hamle çiftinden oluşabilir. Se kiz bulmacası durumunda bile, hedef hızlı bir şekilde ulaşılmadığında arama ağ acı oldukça büyük hale gelebilir. Sonuç olarak, tam bir arama ağacı geliştirmek, tüm durum grafiğini temsil etmek kadar pratik olmayabilir.

Bu sorunu aşmanın bir stratejisi, arama ağacının inşa edilme sırasını değ iştirmektir. Ağacı genişlik öncelikli bir şekilde (yani ağacın katman katman in şa edilmesi anlamında) inşa etmek yerine, daha derinlere daha umut verici y olları takip edebiliriz ve bu orijinal seçimler yanlış izler çıkarsa diğer seçenekl eri değerlendirebiliriz. Bu, arama ağacının derinlik öncelikli bir inşasını sonu çlandırır; bu da ağacın yatay katmanlar yerine dikey yollar inşa edilerek oluşt urulması anlamına gelir. Daha kesin olarak, bu yaklaşım genellikle, takip için seçilen dikey yolun en iyi potansiyeli sunduğu gerçeğini tanıyarak en iyi önc elikli inşaat olarak adlandırılır.

En iyi öncelikli yaklaşım, insanların sekiz bulmacayla karşılaştıklarında uygu layacakları stratejiye benzer. Genellikle, genişlik öncelikli yaklaşımda olduğu gi bi, aynı anda birkaç seçeneği takip etmeyiz. Bunun yerine, muhtemelen en um ut verici görünen seçeneği seçer ve önce onu takip ederiz. En umut verici görü neni söyledik. Belirli bir noktada hangi seçeneğin en iyi olduğunu nadiren kesin olarak biliriz. Sadece sezgimizi takip ederiz, bu da elbette bizi yanıltabilir. Yine de, böyle sezgisel bilgilerin kullanımı, her seçeneğin eşit dikkatle ele alındığı ka ba kuvvet yöntemlerine göre insanlara bir avantaj sağlıyormuş gibi görünmekt edir ve bu nedenle sezgisel yöntemlerin otomatik kontrol sistemlerinde uygula nması akıllıca olacaktır.

Bunun için, birkaç durumdan hangisinin en umut verici olduğunu belirle menin bir yoluna ihtiyacımız var. Yaklaşımımız, her durumla ilişkilendirilmiş v e o durumdan en yakın hedefe olan "mesafeyi" ölçmeye çalışan bir niceliksel değer olan bir sezgisel kullanmaktır. Bir anlamda, sezgiselimiz öngörülen m aliyetin bir ölçüsüdür. İki durum arasında bir seçim yapıldığında, daha küçük sezgisel değere sahip olan, hedefe en az maliyetle ulaşılabileceği görünen d urumdur. Bu durum, dolayısıyla, takip etmemiz gereken yönü temsil eder.

Bir sezgisel yöntem iki özelliğe sahip olmalıdır. İlk olarak, ilişkili durumun ul aşılması durumunda çözümde kalan iş miktarının makul bir tahminini oluşturm alıdır. Bu, seçenekler arasında seçim yaparken anlamlı bilgiler sağlayabileceği a nlamına gelir - sezgisel yöntem tarafından sağlanan tahmin ne kadar iyi olursa, bu bilgilere dayanan kararlar o kadar iyi olacaktır. İkinci olarak, sezgisel yönte m hesaplaması kolay olmalıdır. Bu, kullanımının arama sürecine fayda sağlama şansı olduğu anlamına gelir, yük olmaktan ziyade. Eğer hesaplama

Doclingo https://www.doclingo.ai

Davranışa Dayalı Zeka

Yapay zeka konusundaki erken çalışmalar, zekayı simüle etmek için açıkça progra mlar yazma bağlamında yaklaşmıştır. Ancak, birçok kişi bugün insan zekasının kar maşık programların yürütülmesine dayanmadığını, bunun yerine nesiller boyunca evrimleşen basit uyarıcı-tepki fonksiyonları tarafından belirlendiğini savunmaktadı r. Bu "zeka" teorisi, "zeki" uyarıcı-tepki fonksiyonlarının belirli bireylerin hayatta kal masına ve üremesine neden olan davranışların sonucu gibi görünmesi nedeniyle d avranışa dayalı zeka olarak bilinir.

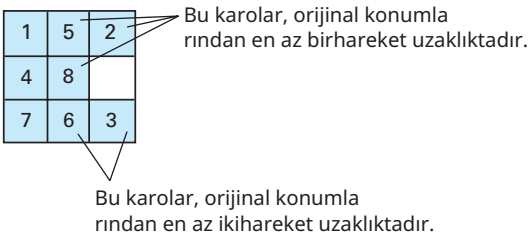
Davranışa dayalı zeka, von Neumann mimarisine dayanan makinelerin hesapl ama becerilerinde insanları kolayca geride bırakmasının ancak sağduyu sergileme kte zorlanmasının nedenleri gibi yapay zeka topluluğundaki birkaç soruya yanıt ver iyor gibi görünüyor. Bu nedenle, davranışa dayalı zeka, yapay zeka araştırmalarınd a büyük bir etki olmayı vaat ediyor. Metinde açıklandığı gibi, davranışa dayalı tekni kler, nöronları istenen şekillerde davranmaya öğretmek için yapay sinir ağları alanı nda, daha geleneksel programlama sürecine alternatif sağlamak için genetik algori tmalar alanında ve makinelerin performansını reaktif stratejilerle artırmak için rob otikte uygulanmıştır.

sezgisel yöntem son derece karmaşıksa, o zaman zamanımızı geniş bir arama yapar ak harcamak daha iyi olabilir.

Sekiz bulmaca durumunda basit bir sezgisel yöntem, hedefe "mesafeyi" t ahmin etmek için yerinden çıkmış karoların sayısını saymak olacaktır - dört k aronun yerinden çıktığı bir durumun, yalnızca iki karonun yerinden çıktığı bir durumdan daha uzak (ve dolayısıyla daha az çekici) olduğu varsayılmaktadır . Ancak, bu sezgisel yöntem karoların ne kadar yerinden çıktığını dikkate alm az. Eğer iki karo, uygun konumlarından uzaktaysa, bulmacanın içinde onları hareket ettirmek için birçok üretim gerekebilir.

O halde, biraz daha iyi bir sezgi, her karonun varış noktasına olan mesaf esini ölçmek ve bu değerleri tek bir miktar elde etmek için eklemektir. Nihai varış noktasına hemen bitişik bir karo, bir mesafe ile ilişkilendirilirken, köşesi nihai varış noktasının karesine dokunan bir karo, iki mesafe ile ilişkilendirilir (çünkü en az bir pozisyon dikey ve bir pozisyon yatay olarak hareket etmesi gerekir). Bu sezgi hesaplaması kolaydır ve bulmacanın mevcut durumundan hedefe dönüşmesi için gereken hamlelerin kabaca bir tahminini üretir. Örne ğin, Şekil 11.9'daki konfigürasyonla ilişkilendirilen sezgi değeri yedidir (çünk ü 2, 5 ve 8 numaralı karolar).

Şekil **11.9** Çözülmemiş sekiz bulmaca



Doclingo https://www.doclingo.ai

Şekil **11.10** Heuristikler kullanan bir kontrol sistemi için bir algoritma

```
Durum grafiğinin başlangıç düğümünü arama ağacının kökü olar
    ak belirleyin ve heuristik değerini kaydedin.
while (hedef düğüme ulaşılmadı) do [Tüm yaprak
    düğümler arasında en küçük heuristik değere sahip en soldaki y
        aprak düğümünü seçin.
     Seçilen düğüme, tek bir üretimle ulaşılabilen düğümleri ç
        ocuk olarak ekleyin.
     Bu yeni düğümlerin her birinin heuristik değerini arama ağacın
        daki düğümün yanına kaydedin.
    ]
Hedef düğümden köke kadar arama ağacını geçin, her geçil
    en yay ile ilişkili üretimi bir yığına iterek.

Orijinal problemi, yığınlardan çıkarıldıkları gibi üretimleri gerçekl
    eştirerek çözün.
```

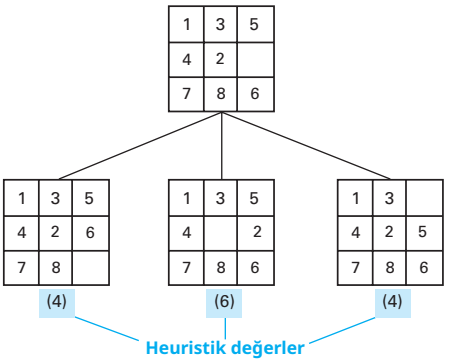
her biri son varış noktalarından bir mesafede iken, 3 ve 6 numaralı karolar evden iki mesafededir). Aslında, bu bulmaca konfigürasyonunu çözülen konfigürasyona dönd ürmek için toplamda yedi hamle gerekmektedir.

Artık sekiz bulmaca için bir sezgisel yöntemimiz olduğuna göre, bir sonraki adım bunu karar verme sürecimize entegre etmektir. Bir insanın bir karar ile karşılaştığın da, hedefe en yakın görünen seçeneği seçme eğiliminde olduğunu hatırlayın. Bu ned enle, arama prosedürümüz, arama ağacındaki her yaprak düğümün sezgisini dikkat e almalı ve en küçük değere sahip yaprak düğümden aramayı sürdürmelidir. Bu, bir arama ağacı geliştirmek ve elde edilen çözümü yürütmek için bir algoritma sunan Şe kil 11.10'da benimsenen stratejidir.

Bu algoritmayı sekiz bulmacaya uygulayalım, Şekil 11.6'daki başlangıç konfigüra syonundan başlayarak. Öncelikle, bu başlangıç durumunu kök düğüm olarak belirliy oruz ve heuristik değerini, yani beşi kaydediyoruz. Ardından, while ifadesinin gövdes inden geçen ilk geçiş, başlangıç durumundan ulaşılabilen üç düğümü eklememizi tal ep ediyor, bu da Şekil 11.11'de gösterilmiştir. Her yaprak düğümün heuristik değerin i parantez içinde düğümün altında kaydettiğimizi unutmayın.

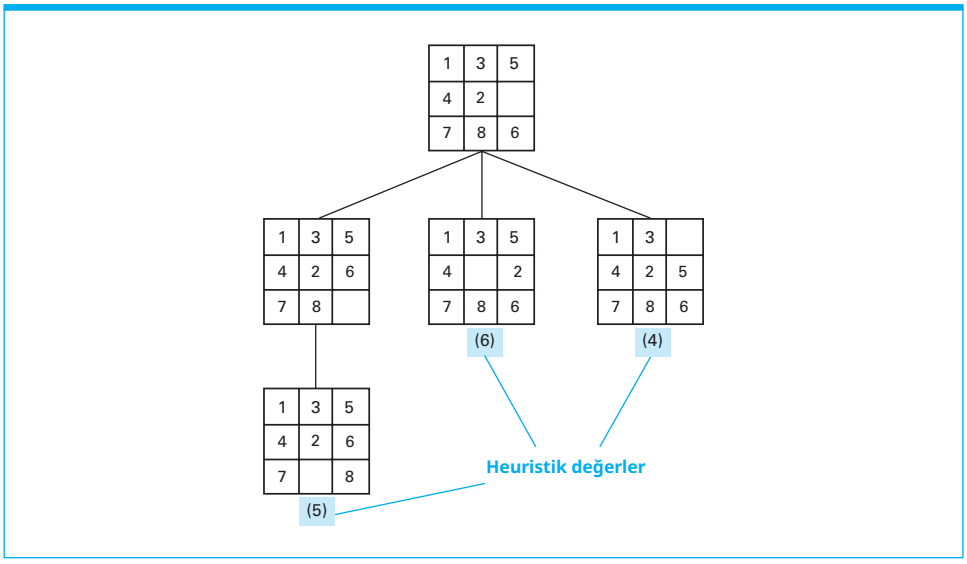
Hedef düğüme ulaşılmadı, bu yüzden tekrar while ifadesinin gövdesinden geç iyoruz, bu sefer aramamızı en soldaki düğümden (“hedef düğüm”) genişletiyoruz.

Şekil **11.11** Heuristik arama sürecimizin başlangıçları



Doclingo https://www.doclingo.ai

Şekil **11.12**İki geçişten sonraki arama ağacı

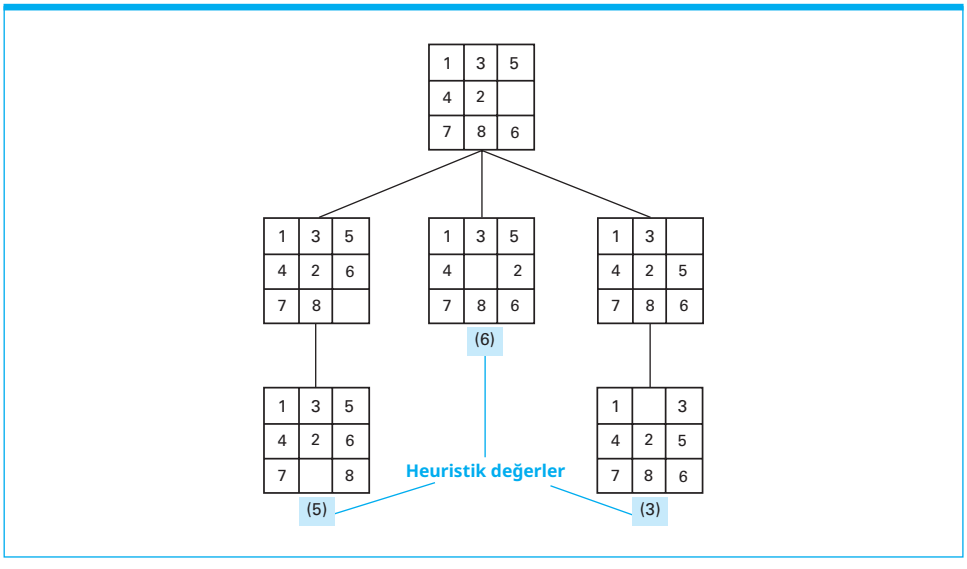


en küçük heuristik değere sahip en soldaki yaprak düğüm"). Bunun ardından, arama ağacı Şekil 11.12'de gösterilen biçimi alır.

En soldaki yaprak düğümün heuristik değeri artık beş, bu da budalın bel ki de peşinden koşulacak iyi bir seçim olmadığını gösteriyor. Algoritma bunu fark eder ve while ifadesi içindeki bir sonraki geçişte, ağacı en sağdaki düğü mden (artık"en küçük heuristik değere sahip en soldaki yaprak düğüm") geni şletmemiziister. Bu şekilde genişletildikten sonra, arama ağacı Şekil 11.13'te ki gibi görünür.

Bu noktada algoritmanın doğru yolda olduğu görülüyor. Çünkü bu son düğümü n heuristikdeğeri yalnızca üç, while ifadesi bize bu yolda devam etmemizisöyler.

Şekil **11.13**Üç geçişten sonraki arama ağacı



Doclingo https://www.doclingo.ai

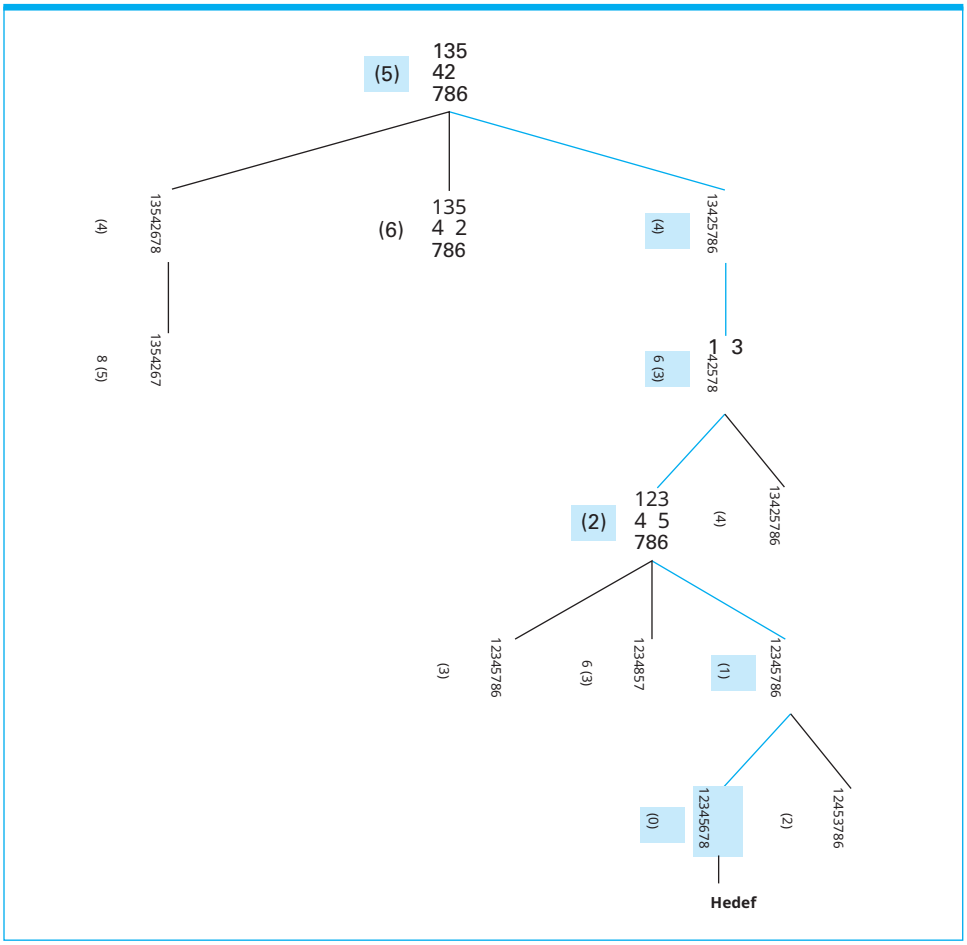
bu yolu takip etmeye devam ediyoruz ve arama hedefe doğru odaklanıyor, Şeki l 11.14'te görünen arama ağacını üretiyor. Bunu Şekil 11.7'deki ağaçla karşılaştı rmak,yeni algoritmanın erken dönemde aldığı geçici yanlış yönelime rağmen,h euristik bilginin arama ağacının boyutunu büyük ölçüde azalttığını ve çok daha verimli bir süreç ürettiğini gösteriyor.

Hedef duruma ulaştıktan sonra, while ifadesi sona erer ve ağaçta hedef dü ğümden köke doğru ilerlemeye başlarız, karşılaştığımız üretimleri bir yığına ite rek devam ederiz. Elde edilen yığın, daha önce Şekil 11.8'de gösterildiği gibi gö rünmektedir.

Son olarak, bu üretimleri yığından çıkarıldıkça yürütmemiz gerektiği belirtiliyo r. Bu noktada, bulmaca çözme makinesinin parmağını indirdiğini ve karoları hare ket ettirmeye başladığını gözlemleyeceğiz.

Heuristik arama ile ilgili son bir yorumda bulunmak gerekiyor. Bu bölümde öner diğimiz algoritma, genellikle en iyi uyum algoritması olarak adlandırılır, tüm uygula malarda en iyi çözümü bulma garantisi yoktur. Örneğin, bir otomobilde Küresel Kon umlama Sistemi (GPS) kullanarak bir şehre giden bir yol ararken, sadece herhangi bi r yolu değil, en kısa yolu bulmak istersiniz. A* algoritması (A yıldız algoritması olarak telaffuz edilir), en iyi uyum algoritmamızın değiştirilmiş bir versiyonudur.

Şekil 11.14 Heuristik sistemimiz tarafından oluşturulan tam arama ağacı



Doclingo https://www.doclingo.ai

İki algoritma arasındaki temel fark, A* algoritmasının bir heuristik değerin y anı sıra, bir sonraki genişletilecek düğümü seçerken her yaprak düğüme ula şmak için katlanılan "birikmiş maliyeti" dikkate almasıdır. (Bir otomobilin GPS 'inde, bu maliyet, GPS'in iç veritabanından elde ettiği kat edilen mesafedir.) B öylece, A* algoritması kararlarını, yalnızca kalan maliyetlerin projeksiyonları na dayanmak yerine, potansiyel tam yolların maliyet tahminlerine dayandırır .

Sorular ve Alıştırmalar

1. Yapay zeka alanında üretim sistemlerinin önemi nedir?
2. Aşağıdaki durumu temsil eden düğüm etrafındaki sekiz bulmacanın durum grafiğinin bir kısmını çizin:

4	1	3
	2	6
7	5	8

3. Aşağıdaki başlangıç durumundan sekiz bulmacayı çözerken bir kontrol sistemi tarafından oluşturulan arama ağacını genişlik öncelikli bir yaklaşım kullanarak çizin:

1	2	3
4	8	5
7	6	

4. Kurşun kalem, kağıt ve genişlik öncelikli yaklaşımı kullanarak aşağıdaki başlan gıç durumundan sekiz bulmacayı çözerken üretilen arama ağacını oluşturmaya çalışın. (Tamamlamak zorunda değilsiniz.) Hangi sorunlarla karşılaşıyorsunuz?

4	3	
2	1	8
7	6	5

5. Sekiz bulmacayı çözmek için kullandığımız sezgisel sistem ile yerel ar azinin sadece dikkate alındığı ve her zaman en dik yükseliş yönünde i lerleyen bir dağcı arasında hangi benzerlik kurulabilir?

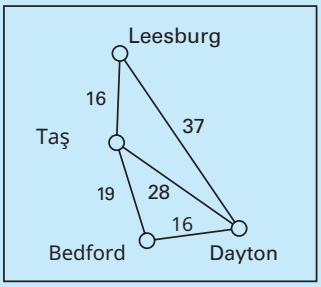
6. Bu bölümde sunulan sezgiyi kullanarak, aşağıdaki sekiz bulmacayı çözme probl eminin 11.10 numaralı Şekil'deki en iyi uyum algoritmasını uygulayın:

1	2	3
4		8
7	6	5

Doclingo https://www.doclingo.ai

7. Sekiz bulmacanın bir durumu için sezgisel değeri hesaplama yöntemi mizi geliştirin, böylece Şekil 11.10'daki arama algoritması, bu bölümd eki örnekte olduğu gibi yanlış bir seçim yapmaz. Sezgiselinizin arama yı yanlış yola sürüklediği bir örnek bulabilir misiniz?

8. Leesburg'dan Bedford'a giden en iyi uyum algoritması (Şekil 11.10) tara fından üretilen arama ağacını çizin. Arama ağacındaki her düğüm harit adaki bir şehir olacaktır. Leesburg için bir düğümle başlayın. Bir düğüm ü genişletirken, yalnızca genişletilen şehre doğrudan bağlı olan şehirler i ekleyin. Her düğümde Bedford'a olan düz hat mesafesini kaydedin ve bunu sezgisel değer olarak kullanın. En iyi uyum algoritması tarafından bulunan çözüm nedir? Bulunan çözüm en kısa yol mu?



Dayton'dan Bedford'a doğru düz hat mesafesi 16
Leesburg 34
Taş 19

9. A* algoritması, en iyi ilk algoritmayı iki önemli şekilde değiştirir. İlk olarak, bir duruma ulaşmanın gerçek maliyetini kaydeder. Bir haritada ki bir rota durumunda, gerçek maliyet kat edilen mesafedir. İkincisi, geniş letilecek bir düğüm seçerken, gerçek maliyet ile sezgisel değerin toplamı en küçük olan düğümü seçer. Bu iki değişiklikten kaynaklanacak Soru 8'in arama ağacını çizin. Her düğümde şehre ulaşmak için kat edilen mesafey i, hedefe ulaşmak için sezgisel değeri ve bunların toplamını kaydedin. Bul unan çözüm nedir? Bulunan çözüm en kısa rota mıdır?

11.4 Ek Araştırma Alanları

Bu bölümde, bilgi yönetimi, öğrenme ve çok karmaşık problemlerle başa çık ma konularını araştırıyoruz; bu konular yapay zeka alanındaki araştırmacıları zorlamaya devam ediyor. Bu faaliyetler, insan zihinleri için kolay gibi görüne n ancak makinelerin yeteneklerini zorlayan yetenekleri içeriyor. Şu anda, "ze ki" ajanlar geliştirmedeki ilerlemenin çoğu, bu sorunlarla doğrudan yüzleşm ekten kaçınarak - belki de akıllı kısayollar uygulayarak veya bir problemin ort aya çıktığı kapsamı sınırlayarak - elde edilmiştir.

Bilgiyi Temsil Etme ve Manipüle Etme

Algı üzerine yaptığımız tartışmada, görüntüleri anlamanın, görüntüdeki nes neler hakkında önemli bir bilgi miktarı gerektirdiğini ve bir cümlenin anlamı nın bağlamına bağlı olabileceğini gördük. Bunlar, genellikle gerçek dünya bil gisi olarak adlandırılan bilgi deposunun oynadığı rolün örnekleridir.

Doclingo https://www.doclingo.ai

insan zihinleri tarafından. Bir şekilde, insanlar büyük miktarda bilgiyi depolar ve bu bilgiden olağanüstü bir verimlilikle yararlanır. Bu yeteneği makinelere vermek, yapay zeka alanında büyük bir zorluktur.

Temel hedef, bilgiyi temsil etme ve depolama yollarını bulmaktır. Bu, daha önce gördüğümüz gibi, bilginin hem açıklayıcı hem de prosedürel formlarda ortaya çıkması gerçeğiyle karmaşıklaşmaktadır. Dolayısıyla, bilgiyi temsil etmek yalnızca gerçeklerin temsil edilmesi değil, çok daha geniş bir yelpazeyi kapsamaktadır. Tüm bilgi formlarını temsil etmek için tek bir şemanın nihayetinde bulunup bulunmayacağı sorgulanabilir.

Ancak sorun, bilgiyi temsil etmek ve depolamakla sınırlı değildir. Bilgi anında zamanda kolayca erişilebilir olmalıdır ve bu erişilebilirliği sağlamak bir zorluktur. Bölüm 11.2'de tanıtılan anlamsal ağlar, genellikle bilgi temsil ve depolama aracı olarak kullanılır, ancak bunlardan bilgi çıkarmak sorunlu olabilir. Örneğin, “Mary John'a vurdu” ifadesinin önemi, Mary ve John'un yaşlarına bağlıdır. (Yaşlar 2 ve 30 mu yoksa tam tersi mi?) Bu bilgi, Şekil 11.3'te önerilen tam anlamsal ağda depolanacaktır, ancak bağlamsal analiz sırasında böyle bir bilgiyi çıkarmak, ağda önemli miktarda arama yapmayı gerektirebilir.

Bilgiye erişimle ilgili bir diğer sorun, mevcut görevle açıkça değil, dolaylı olarak ilişkili olan bilgiyi tanımlamaktır. “Arthur yarışı kazandı mı?” sorusuna basit bir “Hayır” yanıtı vermek yerine, “Hayır, grip oldu ve yarışmaya katılamadı” şeklinde yanıt verebilen bir sistem istiyoruz. Bir sonraki bölümde, bu ilgili bilgi sorununu çözmeye çalışan bir araştırma alanı olan çağrışım belleği kavramını keşfedeceğiz. Ancak, görev yalnızca ilgili bilgiyi geri almak değildir. İlgili bilgi ile alakalı bilgi arasında ayrım yapabilen sistemlere ihtiyacımız var. Örneğin, “Hayır, Ocak'ta doğdu ve kız kardeşinin adı Lisa” gibi bir yanıt, bildirilen bilgi bir şekilde ilişkili olsa bile, önceki soruya değerli bir yanıt olarak kabul edilmeyecektir.

Bilgi çıkarım sistemlerini geliştirmek için başka bir yaklaşım, çıkarım sürecine çeşitli akıl yürütme biçimlerini eklemektir; bu da meta-akıl yürütme olarak adlandırılır—yani akıl yürütme hakkında akıl yürütme. Örneğin, veritabanı aramaları bağlamında kullanılan bir örnek, kapalı dünya varsayımını uygulamaktır; bu, bir ifadenin, mevcut bilgilerden açıkça türetilmediği sürece yanlış olduğu varsayımıdır. Örneğin, veritabanının Nicole Smith'in belirli bir dergiye abone olmadığını sonuçlandırmasına olanak tanıyan kapalı dünya varsayımıdır; oysa veritabanında Nicole hakkında hiç bilgi yoktur. Süreç, Nicole Smith'in abonelik listesinde olmadığını gözlemlemek ve ardından kapalı dünya varsayımını uygulayarak Nicole Smith'in abone olmadığını sonuçlandırmaktır.

Yüzeyde kapalı dünya varsayımı önemsiz görünebilir, ancak görünüşte masum meta-akıl yürütme tekniklerinin ince, istenmeyen etkileri olabileceğini gösteren sonuçları vardır. Örneğin, elimizdeki tek bilginin şu tek ifade olduğunu varsayalım:

Mickey bir fare veya Donald bir ördek.

Bu ifadeden yalnızca Mickey'in gerçekten bir fare olduğunu sonucunu çıkaramayız. Bu nedenle kapalı dünya varsayımı, şu sonuca varmamızı zorunlu kılar:

Mickey bir fare.

Doclingo https://www.doclingo.ai

yanlıştır. Benzer bir şekilde, kapalı dünya varsayımı, ifadeninşu şekilde sonuçla nmamıza zorlar:

Donald bir ördek.

yanlıştır. Böylece, kapalı dünya varsayımı, en az bir ifadenin doğru olması gerektiği halde, her ikisinin de yanlış olduğu çelişkili sonuca yol açmıştır.
Böyle masum görünen meta-akıl yürütme tekniklerinin sonuçlarını anlamak, yapay zeka ve veritabanı alanlarındaki araştırmaların bir hedefidir ve ayrıcaa kıllı sistemlerin geliştirilmesinde yer alan karmaşıklıkları vurgular.

Son olarak, saklanan bilgiyi değişen bir ortamda güncel tutma sorunu, ç erçeve problemi olarak bilinir. Eğer bir akıllı ajan,bilgisini davranışını belirle mek için kullanacaksa, o zaman bu bilgi güncel olmalıdır.
Ancak, akıllı davranışı desteklemek için gereken bilgi miktarı muazzam olabilir ve bu bilgiyi değişen bir ortamda sürdürmek büyük bir çaba gerektirebilir. Karmaşıklaştıran bir faktör, bir ortamda meydana gelen değişikliklerin genellikle dolaylı olarak diğer bilgi öğelerini değiştirmesidir ve böyle dolaylı sonuçları hesaba katmak zordur. Örneğin, eğer bir çiçek vazosu devrilip kırılırsa, durum hakkındaki bilginiz artık vazoda su bulunduğu gerçeğini içermez, oysa suyun dökülmesi, vazonun kırılmasıyla yalnızca dolaylı olarak ilişkilidir.
Bu nedenle, çerçeve problemini çözmek yalnızca çok büyük miktarda bilgiyi verimli bir şekilde depolama ve geri alma yeteneğini gerektirmekle kalmaz, aynı zamanda depolama sisteminin dolaylı sonuçlara uygun bir şekilde tepki vermesini de talep eder.

Öğrenme

Bilgiyi temsil etme ve işleme yeteneğine ek olarak, akıllı ajanlara yeni bilgi edin me yeteneği vermek istiyoruz. Bilgisayar tabanlı bir ajana her zaman yeni bir pr ogram yazarak ve kurarak veya saklanan verilerine açıkça ekleyerek "öğretebili riz", ancak akıllı ajanların kendi başlarına öğrenebilmesini istiyoruz. Ajanların d eğişen ortamlara uyum sağlamasını ve önceden kolayca program yazamayaca ğımız görevleri yerine getirmesini istiyoruz. Ev işleri için tasarlanmış bir robot, y eni mobilyalar, yeni ev aletleri, yeni evcil hayvanlar ve hatta yeni sahiplerle karş ılaşacaktır. Otonom, sürücüsüz bir otomobil, yollardaki sınır çizgilerindeki değiş ikliklere uyum sağlamalıdır. Oyun oynayan ajanlar, yeni stratejiler geliştirebilm eli ve uygulayabilmelidir.

Bilgisayar öğrenme yaklaşımlarını sınıflandırmanın bir yolu, gereken insa n müdahalesi seviyesidir. İlk seviyede, bir kişinin bir görevin adımlarını doğr udan gösterdiği (belki bir dizi bilgisayar işlemi gerçekleştirerek veya bir robo tu bir dizi hareketle fiziksel olarak hareket ettirerek) ve bilgisayarın yalnızca a dımları kaydettiği ödünleme yoluyla öğrenme vardır. Bu öğrenme biçimi, sıkl ıkla meydana gelen komut dizilerinin kaydedildiği ve daha sonra tek bir istek le tekrar oynatıldığı elektronik tablo ve kelime işlemci gibi uygulama progra mlarında yıllardır kullanılmaktadır. Ödünleme yoluyla öğrenmenin ajana çok az sorumluluk yüklediğini unutmayın.

Bir sonraki seviyede, denetimli eğitimle öğrenme bulunmaktadır. Denet imli eğitimde, bir kişi bir dizi örnek için doğru yanıtı belirler ve ardından ajan, bu örneklerden genelleme yaparak yeni durumlara uygulanabilen bir algori tma geliştirir. Örnekler dizisine eğitim seti denir. Denetimli eğitimin tipik uy gulamaları, bir kişinin el yazısını veya sesini tanımayı öğrenmek, gereksiz ve hoş gelen e-postaları ayırt etmeyi öğrenmek ve bir dizi semptomdan bir hast alığı tanımlamayı öğrenmektir.

Doclingo https://www.doclingo.ai

Üçüncü bir seviye, pekiştirme ile öğrenme dir. Pekiştirme ile öğrenmede, ajana bir genel kural verilir ve bu kural, deneme yanılma sırasında bir görevde başarılı olup olmadığını kendisi nin değerlendirmesi için kullanılır. Pekiştirme ile öğrenme, satranç veya dama gibi bir oyunu o ynamayı öğrenmek için iyidir, çünkü başarı veya başarısızlık tanımlanması kolaydır. Denetimli eğitimle karşılaştırıldığında, pekiştirme ile öğrenme, ajanın zamanla davranışı nı geliştirmeyi öğrenirken özerk olarak hareket etmesine olanak tanır.

Öğrenme, tüm olası öğrenme faaliyetlerini kapsayan genel, evrensel bir ilke bulun madığı için zorlu bir araştırma alanı olmaya devam etmektedir. Ancak, ilerleme kaydedil diğine dair birçok örnek vardır. Bunlardan biri, Carnegie Mellon Üniversitesi'nde geliştir ilen ALVINN (Otonom Kara Aracı Sinir Ağı) sistemidir; bu sistem, bir video kamerası kull anarak bir vanayı yönlendirmeyi öğrenmek için bir yerleşik bilgisayar kullanır. Kullanılan yaklaşım denetimli eğitimdi. ALVINN, bir insan sürücüsünden veri topla dı ve bu verileri kendi direksiyon kararlarını ayarlamak için kullandı. Öğrendikçe, n ereye yönlendireceğini tahmin eder, tahminini insan sürücüsünün verileriyle kont rol eder ve ardından insanın direksiyon seçimlerine daha yakın olmak için parame trelerini değiştirirdi. ALVINN, saatte yetmiş mil hızla vanı yönlendirebilecek kadar başarılı oldu ve bu, trafiğin içinde otoyol hızlarında başarılı bir şekilde sürüş yapa n kontrol sistemleri üreten ek araştırmalara yol açtı.

Son olarak, öğrenme ile yakından ilişkili bir olguyu tanımamız gerekiyor: keşif. Ayrım, öğrenmenin "hedef odaklı" olmasıdır, oysa keşif değildir. Keşif t erimi, öğrenmede mevcut olmayan beklenmedik bir anlam taşır. Yabancı bir dil öğrenmek veya araba sürmeyi öğrenmek için yola çıkabiliriz, ancak bu gö revlerin beklediğimizden daha zor olduğunu keşfedebiliriz. Bir kaşif, yalnızca orada ne olduğunu öğrenmek amacıyla büyük bir göl keşfedebilir.

Verimli bir şekilde keşfetme yeteneğine sahip ajanlar geliştirmek, ajanın pota nsiyel olarak verimli "düşünce zincirlerini" tanımlayabilmesini gerektirir. Burada, k eşif büyük ölçüde akıl yürütme yeteneğine ve sezgisel yöntemlerin kullanımına da yanır. Dahası, keşfin birçok potansiyel uygulaması, bir ajanın anlamlı sonuçları ön emsiz olanlardan ayırt edebilmesini gerektirir. Örneğin, bir veri madenciliği ajanı, bulduğu her önemsiz ilişkiyi rapor etmemelidir.

Mantık Programlamada Bilgi

Bilgiyi temsil etme ve depolama konusunda önemli bir endişe, bilginin erişme si gereken sistemle uyumlu bir şekilde yapılmasıdır. Bu bağlamda, mantık pro gramlaması (bkz. Bölüm 6.7) genellikle faydalı olmaktadır. Bu tür sistemlerde bilgi, "mantık" ifadeleriyle temsil edilir, örneğin

Dumbo bir fil.

ve

X bir fil ise X gri olur.

Bu tür ifadeler, çıkarım kurallarının uygulanmasına kolayca erişilebilen nota syon sistemleri kullanılarak temsil edilebilir. Bunun sonucunda, Şekil 11.5'te gör düğümüz gibi, çıkarımsal akıl yürütme dizileri basit bir şekilde uygulanabilir. Böy lece, mantık programlamasında bilginin temsil edilmesi ve depolanması, bilgi çı karım ve uygulama süreciyle iyi bir şekilde entegre edilmiştir. Mantık programla ma sistemlerinin, saklanan bilgi ile uygulaması arasında "dikişsiz" bir sınır sağlad ığını söyleyebiliriz.

Doclingo https://www.doclingo.ai

Bilgisayar keşif sistemlerinde başarı örnekleri arasında, filozof Sir Francis Bacon' dan ismini alan Bacon bulunmaktadır; bu sistem, Ohm'un elektrik yasasını, Kepler'in gezegen hareketinin üçüncü yasasını ve momentumun korunumu yasasını keşfetmiş tir (ya da belki "yeniden keşfetmiştir" demeliyiz). Daha ikna edici olan, kızılötesi spekt ral verileri kullanarak daha önce astronomide bilinmeyen yeni yıldız sınıflarını keşfed en AUTOCLASS sistemidir—bir bilgisayar tarafından yapılan gerçek bir bilimsel keşif.

Genetik Algoritmalar

A* algoritması (önceki bölümde tanıtılmıştır) birçok arama probleminin opti mal çözümünü bulacaktır; ancak bazı problemler, bu tür arama teknikleriyle çözülemeyecek kadar karmaşıktır (işlem mevcut belleği aşar veya makul bir s üre içinde tamamlanamaz). Bu problemler için, bazen birçok deneme çözüm ünün evrimsel bir süreç aracılığıyla keşfedilmesi mümkündür. Bu strateji, ge netik algoritmalar olarak adlandırılan şeyin temelini oluşturur. Özünde, gene tik algoritmalar, rastgele davranış ile üreme teorisinin simülasyonu ve doğal seçilim sürecinin bir kombinasyonu ile bir çözüm keşfedecektir.

Bir genetik algoritma, rastgele bir deneme çözümleri havuzu oluşturarak başlar. Her çözüm sadece bir tahmindir. (Sekiz bulmacası durumunda, bir deneme ç özümü, rastgele bir karoların hareketleri dizisi olabilir.) Her deneme çözümü ne kromozom denir ve kromozomun her bileşenine gen denir (sekiz bulmac ası durumunda tek bir karo hareketi).

Her başlangıç kromozomu rastgele bir tahmin olduğundan, mevcut proble min çözümünü temsil etmesi çok olası değildir. Bu nedenle, genetik algoritma, her kromozomun önceki havuzun iki kromozomunun (ebeveynler) bir yavrusu (çocuk) olduğu yeni bir kromozom havuzu oluşturmak için ilerler. Ebeveynler, h avuzdan rastgele seçilir ve çözüm bulma şansını en iyi sağlayan kromozomlara olasılıksal bir tercih verilir, böylece en uygun olanların hayatta kalma ilkesini ta klit eder. (Hangi kromozomların ebeveynlik için en iyi adaylar olduğunu belirle mek, genetik algoritma sürecindeki en sorunlu adım olabilir.) Her yavru, ebeve ynlerden rastgele bir gen kombinasyonudur. Ayrıca, ortaya çıkan bir yavru baz en rastgele bir şekilde (iki hareketi değiştirmek) mutasyona uğrayabilir. Umarı m, bu süreci tekrar tekrar tekrarlayarak, çok iyi bir çözüm, en iyi olmasa bile, ke şfedilene kadar daha iyi ve daha iyi deneme çözümleri evrimleşecektir.Ne yazık ki, genetik algoritmanın nihayetinde bir çözüm bulacağına dair bir güvence yo ktur, ancak araştırmalar, genetik algoritmaların şaşırtıcı derecede geniş bir kar maşık problem yelpazesini çözmede etkili olabileceğini göstermiştir.

Program geliştirme görevine uygulandığında, genetik algoritma yaklaşımı evrimsel programlama olarak bilinir. Burada amaç, programları açıkça yazarak değil, evrimleşmelerine izin vererek geliştirmektir. Araştırmacılar, işlevsel programlama dillerini kullanarak program-geliştirme sürecine evrimsel programlama tekniklerini uygulamışlardır. Yaklaşım,bir diz i işlev içeren programlar koleksiyonu ile başlamaktır. Bu başlangıç koleksiyo nundaki işlevler, gelecekteki program nesillerinin inşa edileceği "gen havuzu "nu oluşturur. Daha sonra evrimsel sürecin birçok nesil boyunca çalışmasına izin verilir; umulur ki, önceki nesildeki en iyi performans gösterenlerden her nesil üreterek, hedef probleme bir çözüm evrimleşecektir.

Doclingo https://www.doclingo.ai

Sorular & Alıştırmalar

1. Terimden ne kastedilmektedir *gerçek dünya bilgisi,* ve bunun anlamı-yapay zekadaki değişiklik?
2. Bir dergi aboneleri hakkında bir veritabanı genellikle her dergiye ait abonelerin bir listesini içerir, ancak abonelik yapmayanların bir listesi ni içermez. O halde, böyle bir veritabanı bir kişinin belirli bir dergiye abone olmadığını nasıl belirler?
3. Çerçeve problemini özetleyin.
4. Bir bilgisayarı eğitmenin üç yolunu belirleyin. Hangisi doğrudan insa n müdahalesini içermez?
5. Evrimsel teknikler, daha geleneksel problem çözme tekniklerinden n asıl farklıdır?

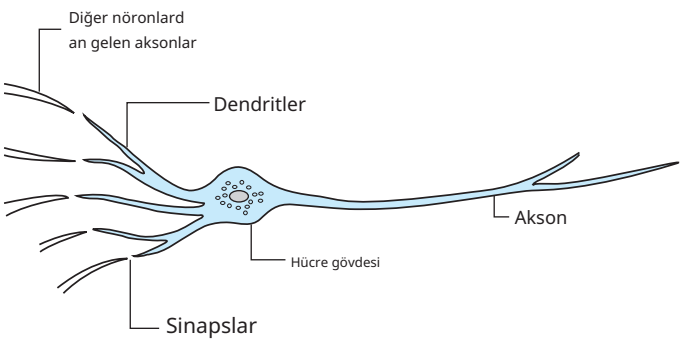
11.5 Yapay Sinir Ağları

Yapay zekada kaydedilen tüm ilerlemelere rağmen, alandaki birçok sorun, gelene ksel algoritmik yaklaşımlar kullanan bilgisayarların yeteneklerini zorlamaya deva m etmektedir. Talimat dizileri, insan zihninin seviyeleriyle karşılaştırılabilir düzeyle rde algılama ve akıl yürütme yeteneğine sahip gibi görünmemektedir. Bu nedenle , birçok araştırmacı doğada gözlemlenen fenomenlerden yararlanan yaklaşımlara yönelmektedir. Böyle bir yaklaşım, önceki bölümde sunulan genetik algoritmalar dır. Diğer bir yaklaşım ise yapay sinir ağlarıdır.

Temel Özellikler

Yapay sinir ağları, canlı biyolojik sistemlerdeki nöron ağlarını taklit eden bir bilgisaya r işleme modeli sağlar. Biyolojik bir nöron, dendritler adı verilen giriş tentakellerine v e akson adı verilen bir çıkış tentakeline sahip tek bir hücredir (Şekil 11.15). Bir hücren in aksonu aracılığıyla iletilen sinyaller, hücrenin inhibe edilmiş veya uyarılmış bir dur umda olup olmadığını yansıtır. Bu durum, hücrenin dendritleri tarafından alınan siny allerin kombinasyonu ile belirlenir. Bu dendritler, diğer hücrelerin aksonlarından sin yalleri, sinapslar olarak bilinen küçük boşluklar aracılığıyla alır. Araştırmalar,

Şekil **11.15** Canlı bir biyolojik sistemde bir nöron



Doclingo https://www.doclingo.ai

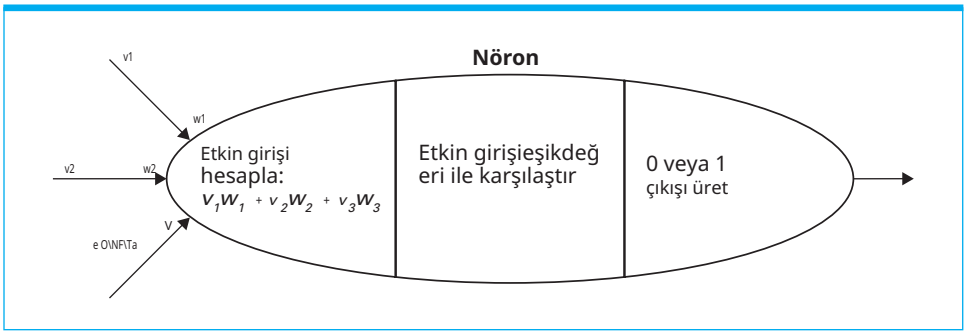
bir sinaps üzerindeki iletkenlik, sinapsın kimyasal bileşimi tarafından kontrol edilir.Yani, belirli bir giriş sinyalinin nöron üzerinde heyecanlandırıcı mı yoksa engelleyici mi bir etki yapacağı,sinapsın kimyasal bileşimi tarafından belirlenir. Bu nedenle, biyolojik bir sinir ağının,nöronlar arasındaki bu kimyasal bağlantıları ayarlayarak öğrendiğine inanılmaktadır.

Yapay sinir ağındaki bir nöron, biyolojik bir nöronun bu temel anlayışını taklit eden bir yazılım birimidir.Girişinin etkili olup olmadığına bağlı olarak 1 veya 0 çıktısı üretir; bu değer, nöronun eşik değeri olarak adlandırılır.Bu etkili giriş, gerçek girişlerin ağırlıklı toplamıdır; Şekil 11.16'da gösterildiği gibi. Bu şekilde, bir nöron,oval ile temsil edilir ve nöronlar arasındaki bağlantılar oklarla gösterilir. Diğer nöronların aksonlarından elde edilen değerler(v_1, v_2 vev_3 ile gösterilir) tasvir edilen nörona giriş olarak kullanılır.Bu değerlere ek olarak, her bağlantı bir ağırlık ile ilişkilendirilmiştir (w_1, w_2 ve w_3 ile gösterilir).Bu giriş değerlerini alan nöron, her birini bağlantı için ilişkili ağırlıkla çarpar ve ardından bu çarpımları toplayarak etkili girişi oluşturur(v_1w_1 +v $_2w_2$+v_3w_3). Bu toplam, nöronun eşik değerini aşarsa, nöron 1 çıktısı üretir(heyecanlı bir durumu simüle eder); aksi takdirde nöron 0 çıktısı üretir (engellenmiş bir durumu simüle eder).

Şekil 11.16'nın izinden giderek, nöronları daireler olarak temsil etme geleneğini benimsiyoruz. Her bir girişin bir nörona bağlandığı yerlerde, o girişle ilişkili ağırlığı kaydediyoruz. Son olarak, nöronun eşik değerini dairenin ortasına yazıyoruz. Örneğin, Şekil 11.17, eşik değeri 1.5 olan ve her bir giriş bağlantısıyla ilişkili 2, 3 ve 1 ağırlığına sahip bir nöronu temsil etmektedir. Bu nedenle, eğer nöron 1, 1 ve 0 girişlerini alırsa, etkili girişi (1)(2)+ (1)(3) + (0)(1)1 olur ve dolayısıyla çıktısı 0'dır. Ancak, eğer nöron 0, 1 ve 1 alırsa, etkili girişi (0)(2)+ (1)(3)+ (1)(1)2 olur ve bu eşik değerini aşar. Bu durumda nöronun çıktısı 1 olacaktır.

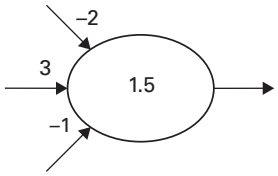
Ağırlığın pozitif veya negatif olabilmesi, ilgili girişin alıcı nöron üzerinde ya inhibe edici ya da heyecanlandırıcı bir etki yaratabileceği anlamına gelir. (Eğer ağırlık negatifse, o giriş pozisyonundaki 1, ağırlıklı toplamı azaltır ve dolayısıyla etkili girişi eşik değerinin altında tutma eğilimindedir. Aksine, pozitif bir ağırlık, ilişkili girişin ağırlıklı toplam üzerinde artırıcı bir etki yaratmasına ve dolayısıyla bu toplamın eşik değerini aşma olasılığını artırmasına neden olur.) Ayrıca, ağırlığın gerçek boyutu, ilgili girişin alıcı nöronu inhibe etme veya heyecanlandırma derecesini kontrol eder. Sonuç olarak, bir yapay sinir ağı bo yunca ağırlıkların değerlerini ayarlayarak, ağı iletilen farklı girişlere önceden belirlenmiş bir şekilde yanıt vermesi için programlayabiliriz.

Şekil**11.16**Bir nöron içindeki aktiviteler



Doclingo https://www.doclingo.ai

Şekil **11.17**Bir nöronun temsili



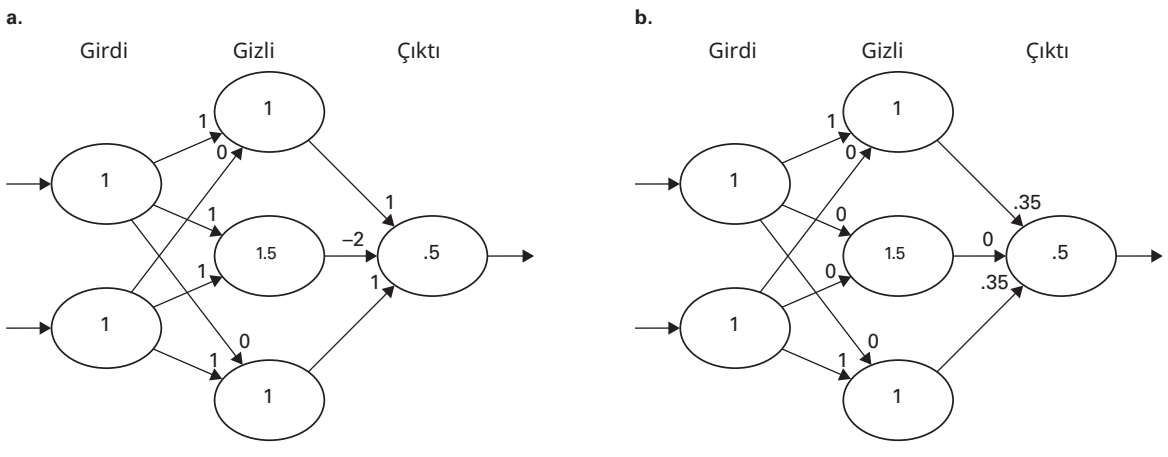
Yapay sinir ağları genellikle birkaç katman topolojisinde düzenlenmiştir. Giriş nöronları ilk katmanda, çıkış nöronları ise son katmandadır. Giriş ve çıkı ş katmanları arasında ek nöron katmanları (gizli katmanlar olarak adlandırılır) bulunabilir. Bir katmandaki her nöron, sonraki katmandaki her nöronla bir birine bağlıdır. Örneğin, Şekil 11.18a'da sunulan basit ağ, iki girişi farklı oldu ğunda 1 çıkışı ve aksi takdirde 0 çıkışı üretmek üzere programlanmıştır. Anca k, ağırlıkları Şekil 11.18b'de gösterilenlere değiştirirsek, her iki girişi de 1 old uğunda 1 ile yanıt veren ve aksi takdirde 0 ile yanıt veren bir ağ elde ederiz.

Şekil 11.18'deki ağ yapılandırmasının gerçek bir biyolojik ağdan çok daha basit olduğunu belirtmeliyiz. İnsan beyni yaklaşık 10^{11} nöron içerir ve her nör on başına yaklaşık 10^{4} sinaps vardır. Gerçekten de, bir biyolojik nöronun den dritleri o kadar çoktur ki, Şekil 11.15'te temsil edilen bireysel tentaküllerden çok lifli bir ağ gibi görünmektedir.

Yapay Sinir Ağlarını Eğitmek

Yapay sinir ağlarının önemli bir özelliği, geleneksel anlamda programlanma mış olmaları, bunun yerine eğitilmeleridir. Yani, bir programcı belirli bir prob lemi çözmek için gereken ağırlıkların değerlerini belirlemez ve ardından bu d eğerleri ağa "takmaz". Bunun yerine, bir yapay sinir

Şekil **11.18**İki farklı programla bir sinir ağı



Doclingo https://www.doclingo.ai

ağ, eğitim setinden gelen girdilerin ağa uygulandığı ve ardından ağırlıkların küçük artışlarla ayarlandığı denetimli eğitim (Bölüm 11.4) aracılığıyla uygun ağırlık değerlerini öğrenir, böylece ağın performansı istenen davranışa yaklaşır.

Genetik algoritma tekniklerinin yapay sinir ağlarını eğitme görevine nasıl uygulandığını not etmek ilginçtir. Özellikle, bir sinir ağını eğitmek için, ağ için bir dizi ağırlık seti rastgele oluşturulabilir—her bir set, genetik algoritma için bir kromozom olarak hizmet edecektir. Ardından, adım adım bir süreçte, ağa her kromozomu temsil eden ağırlıklar atanabilir ve çeşitli girdiler üzerinde test edilebilir.
Bu test süreci sırasında en az hata üreten kromozomlar, bir sonraki nesil için ebeveyn olarak seçilme olasılığı daha yüksek olacak şekilde değerlendirilebilir. Sayısız deneyde bu yaklaşım, nihayetinde başarılı bir ağırlık setine yol açmıştır.

Bir yapay sinir ağını bir problemi çözmek için eğitmenin başarılı olduğu ve belki de geleneksel programlama teknikleriyle bir çözüm sağlamaya çalışmaktan daha verimli olduğu bir örneği ele alalım. Problem, bir robotun video kamerasından aldığı bilgiler aracılığıyla çevresini anlamaya çalışırken karşılaşabileceği bir durumdur. Örneğin, robotun bir odanın beyaz olan duvarları ile siyah olan zeminini ayırt etmesi gerektiğini varsayalım. İlk bakışta, bu kolay bir görev gibi görünebilir: Beyaz pikselleri duvarın bir parçası olarak ve siyah pikselleri zemin olarak sınıflandırmak. Ancak, robot farklı yönlere baktıkça veya odada hareket ettikçe, çeşitli aydınlatma koşulları duvarın bazı durumlarda gri görünmesine neden olabilirken, diğer durumlarda zemin gri görünebilir. Bu nedenle, robotun farklı aydınlatma koşulları altında duvarlar ile zemin arasında ayırt etmeyi öğrenmesi gerekmektedir.

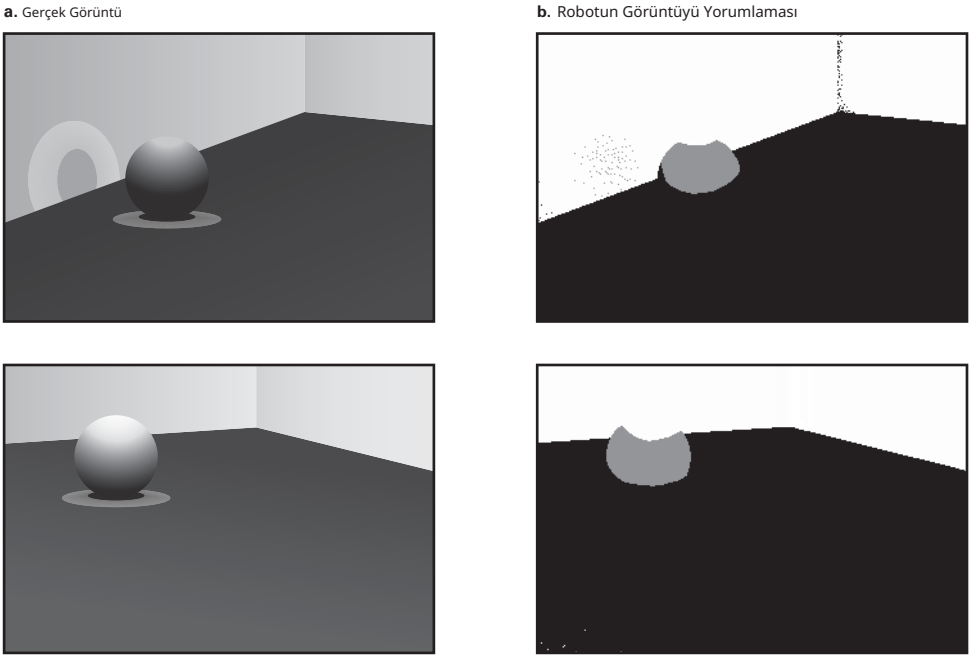
Bunu başarmak için, görüntüdeki bireysel bir pikselin renk özelliklerini belirten değerler ile tüm görüntünün genel parlaklığını belirten bir değerden oluşan bir yapay sinir ağı inşa edebiliriz. Daha sonra, çeşitli aydınlatma koşulları altında duvarların ve zeminlerin parçalarını temsil eden piksel örnekleri ile ağı eğitebiliriz.

Bu teknikleri kullanarak bir yapay sinir ağını eğitmenin sonuçları Şekil 11.19'da gösterilmektedir. İlk sütun orijinal görüntüleri temsil eder; sonraki sütun robotun yorumunu göstermektedir. Üstteki orijinal görüntüdeki duvarların oldukça karanlık olmasına rağmen, robot çoğu ilişkili pikseli beyaz duvar pikselleri olarak doğru bir şekilde tanımlamıştır, ancak alt görüntüdeki zemin hala doğru bir şekilde tanımlanmıştır. (Görüntülerdeki top daha kapsamlı bir deneyin parçasıydı.) Robotun görüntü işleme sisteminin mükemmel olmadığını da fark edeceksiniz. Sinir ağı, bazı duvar piksellerini zemin pikselleri olarak (ve bazı zemin piksellerini de duvar pikselleri olarak) yanlış tanımlamıştır. Bunlar, bir teorinin uygulanmasında genellikle göz önünde bulundurulması gereken gerçekliklerin örnekleridir. Bu durumda, hatalar, robotu çok sayıda duvar pikseli arasında görünen bireysel zemin piksellerini (ve tersini) göz ardı edecek şekilde programlayarak düzeltilebilir.

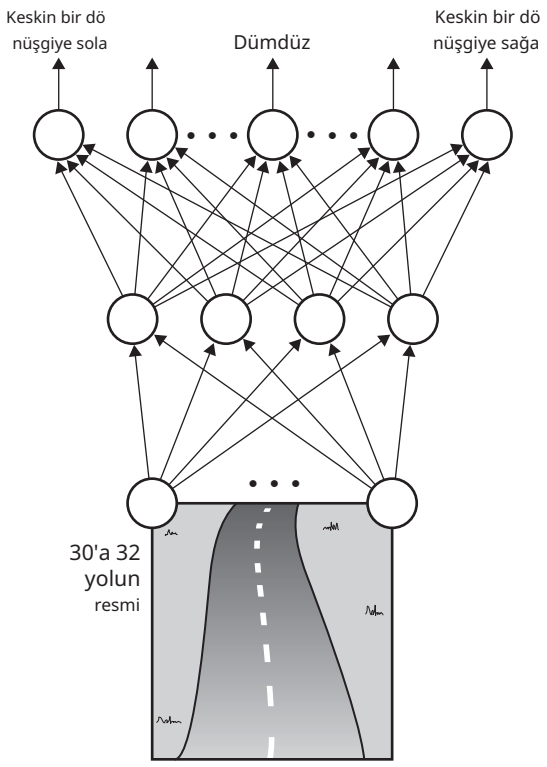
Basit öğrenme problemlerinin ötesinde (piksel sınıflandırması gibi), yapay sinir ağları, önceki bölümde atıfta bulunulan ALVINN projesinin tanıklığıyla, karmaşık akıllı davranışları öğrenmek için kullanılmıştır. Gerçekten de, ALVINN, bileşimi şaşırtıcı derecede basit olan bir yapay sinir ağıydı (Şekil 11.20). Girişi, 30'a 32'lik bir sensör dizisinden elde edilmiştir,

Doclingo https://www.doclingo.ai

Şekil **11.19**Bir görüntüdeki pikselleri sınıflandırmak için bir sinir ağı kullanmanın sonuçları (www.actapress.com'dan ilham alınmıştır)



Şekil **11.20**ALVINN'ın yapısı (Sinir Ağı İçinde Otonom Kara Aracı)



Doclingo https://www.doclingo.ai

her biri, önündeki yolun video görüntüsünün benzersiz bir bölümünü gözle mleyip bulgularını gizli bir katmandaki dört nörona iletti. (Böylece, bu dört n örondan her birinin 960 girişi vardı.) Bu dört nörondan her birinin çıktısı, yön lendirme yönünü gösteren otuz çıkış nöronuna bağlanmıştı. Otuz nöron sıra sının bir ucundaki heyecanlı nöronlar, sola keskin bir dönüşü gösterirken, di ğer ucundaki heyecanlı nöronlar sağa keskin bir dönüşü gösteriyordu.

ALVINN, bir insanın sürüşünü "izleyerek" eğitildi ve kendi direksiyon kararları nı verirken, bu kararları insanın kararlarıyla karşılaştırarak, kararlarını insanınkine daha yakın hale getirmek için ağırlıklarında küçük değişiklikler yaptı. Ancak, ilginç bir yan mesele vardı. ALVINN bu basit teknikle direksiyon kullanmayı öğrenmiş ol sa da, hatalardan nasıl kurtulacağını öğrenmedi. Bu nedenle, insan tarafından top lanan veriler, kurtarma durumlarını da içerecek şekilde yapay olarak zenginleştiril di. (Bu kurtarma eğitimi için başlangıçta düşünülen bir yaklaşım, insanın aracı sav urmasını sağlamak ve böylece ALVINN'in insanın nasıl kurtulduğunu izlemesine v e kendi başına nasıl kurtulacağını öğrenmesine olanak tanımaktı. Ancak, ALVINN, insan ilk savurma prosedürünü gerçekleştirirken devre dışı bırakılmadığı sürece, hem savurmayı hem de kurtulmayı öğrenmişti - bu açıkça istenmeyen bir özellikti.)

İlişkisel Bellek

İnsan zihni, mevcut bir düşünce konusuyla ilişkili bilgileri geri çağırma konus unda muazzam bir yeteneğe sahiptir. Belirli kokuları deneyimlediğimizde, ço cukluğumuza dair anıları kolayca hatırlayabiliriz. Bir arkadaşın sesi, o kişinin görüntüsünü veya belki de güzel anıları canlandırabilir.
Bazı müzikler belirli tatil sezonlarını düşündürebilir. Bunlar, mevcut bilgiyle il işkili veya bağlantılı bilgilerin geri çağrılması olan ilişkilendirici bellek örnekl eridir.

İlişkilendirici belleğe sahip makineler inşa etmek, yıllardır araştırmanın bi r hedefi olmuştur. Bir yaklaşım, yapay sinir ağları tekniklerini uygulamaktır. Örneğin, girdi veya çıktısı olmayan bir ağ oluşturmak için birbirine bağlı birç ok nörondan oluşan bir ağı düşünün. (Bazı tasarımlarda, Hopfield ağları olar ak adlandırılan, her nöronun çıktısı diğer nöronların her birine girdi olarak b ağlanır; diğer durumlarda ise bir nöronun çıktısı yalnızca komşularına bağla nabilir.) Böyle bir sistemde, uyarılmış nöronlar diğer nöronları uyarma eğilim indedir, oysa inhibe edilmiş nöronlar diğerlerini inhibe etme eğilimindedir. S onuç olarak, tüm sistem sürekli bir değişim halinde olabilir veya sistem, uyar ılmış nöronların uyarılmış kalmaya ve inhibe edilmiş nöronların inhibe edilmi ş kalmaya devam ettiği kararlı bir yapı bulabilir. Ağı kararsız bir yapıdan başl atırsak, kararlı bir yapıya doğru dolaşmasını bekleriz. Bir anlamda, kararlı bir yapının bir kısmı verildiğinde, ağ yapıyı tamamlayabilir.

Şimdi heyecanlı bir durumu 1 ile ve engellenmiş bir durumu 0 ile temsil etti ğimizi varsayalım, böylece ağın herhangi bir zamandaki durumu 0'lar ve 1'lerde n oluşan bir yapı olarak hayal edilebilir. O zaman, ağı kararlı bir desene yakın bi r bit desenine ayarlarsak, ağın kararlı desene geçmesini bekleyebiliriz. Diğer bir deyişle, ağ, kendisine verilen desene yakın olan kararlı bit desenini bulabilir.

Doclingo https://www.doclingo.ai

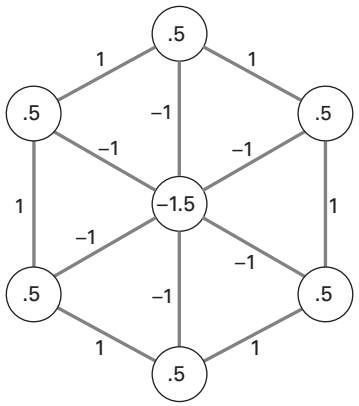
Böylece bazı bitler kokuları kodlamak için kullanılıyorsa ve diğerleri çocukluk anılarını kodlamak için kullanılıyorsa, o zaman koku bitlerini belirli bir kararlı yapılandırmaya göre başlatmak, kalan bitlerin ilişkili çocukluk anısına ulaşma sını sağlayabilir.

Şimdi Şekil 11.21'de gösterilen yapay sinir ağını düşünün.
Yapay sinir ağlarını tasvir etmek için kullanılan geleneklere göre, şekildeki he r daire, dairenin içinde kaydedilen eşik değeri olan bir nöronu temsil eder. O klar yerine, daireleri bağlayan çizgiler, ilgili nöronlar arasındaki iki yönlü bağl antıları temsil eder. Yani, iki nöronu bağlayan bir çizgi, her nöronun çıktısının diğerine bir girdi olarak bağlandığını gösterir. Böylece, merkez nöronun çıkt ısı, çevresindeki her nörona bir girdi olarak bağlanır ve çevresindeki her nör onun çıktısı, merkez nörona ve çevresindeki hemen komşularına bir girdi ola rak bağlanır. İki bağlı nöron, birbirlerinin çıktısına aynı ağırlığı ilişkilendirir. B u ortak ağırlık, nöronları bağlayan çizginin yanında kaydedilir. Böylece, diyag ramın üstündeki nöron, merkez nörondan aldığı girdi ile 1 ağırlığını ve çevre sindeki iki komşusundan aldığı girdilerle de 1 ağırlığını ilişkilendirir. Benzer ş ekilde, merkez nöron, çevresindeki nöronlardan aldığı her bir değerle 1 ağırlı ğını ilişkilendirir.

Ağ, tüm nöronların girişlerine senkronize bir şekilde yanıt verdiği ayrık a dımlarda çalışır. Ağın mevcut konfigürasyonundan bir sonraki konfigürasyon unu belirlemek için, ağ boyunca her nöronun etkili girişlerini belirleriz ve ard ından tüm nöronların aynı anda girişlerine yanıt vermesine izin veririz. Sonuç olarak, tüm ağ, etkili girişleri hesaplama, girişlere yanıt verme, etkili girişleri hesaplama, girişlere yanıt verme vb. koordineli bir diziyi takip eder.

Ağ, en sağdaki iki nöronun inhibe edildiği ve diğer nöronların uyarıldığı b ir başlangıç ile başlatıldığında meydana gelecek olaylar dizisini düşünün.

Şekil **11.21** Bir ilişkisel hafıza uygulayan yapay sinir ağı

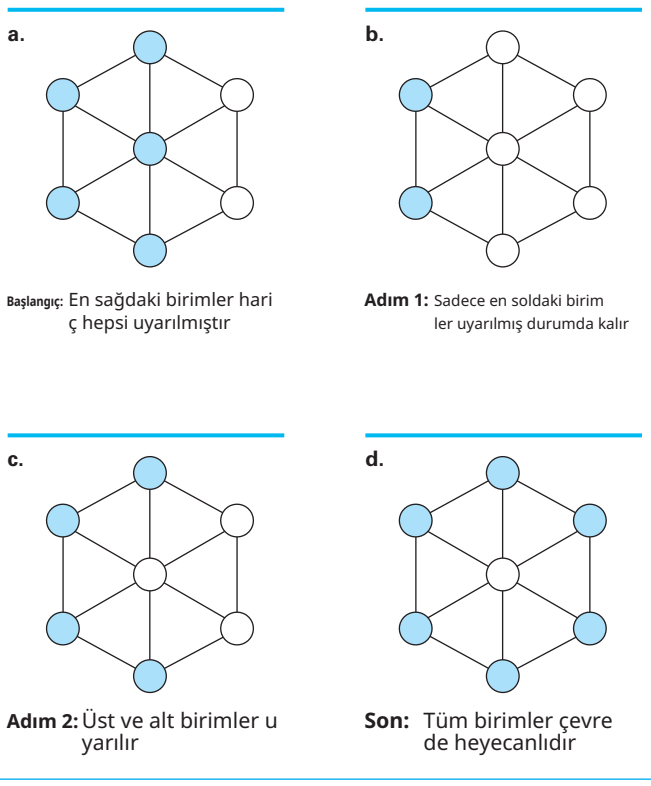


Doclingo https://www.doclingo.ai

(Şekil 11.22a). En soldaki iki nöronun etkili girişleri 1 olacağından, uyarılmış k alacaklardır. Ancak, çevresindeki komşuları 0 etkili girişine sahip olacağından , inhibe olacaklardır. Benzer şekilde, merkez nöronun etkili girişi 4 olacağınd an, o da inhibe olacaktır. Böylece, tüm ağ, yalnızca en soldaki iki nöronun uy arıldığı Şekil 11.22b'de gösterilen konfigürasyona geçecektir. Merkez nöron artık inhibe olacağından, en soldaki nöronların uyarılmış durumu, üst ve alt nöronların tekrar uyarılmasına neden olacaktır. Bu arada, merkez nöron 2 et kili girişine sahip olacağından inhibe kalacaktır. Böylece, ağ, Şekil 11.22c'deki konfigürasyona geçecek ve ardından Şekil 11.22d'deki konfigürasyona yol a çacaktır. (Ağ, yalnızca üstteki dört nöron uyarıldığında bir yanıp sönme feno meninin meydana geleceğini doğrulamak isteyebilirsiniz. Üst nöron uyarılmı ş kalacakken, çevresindeki iki komşusu ve merkez nöron, uyarılmış ve inhibe olmuş arasında geçiş yapacaktır.)

Son olarak, ağın iki kararlı konfigürasyonu olduğunu gözlemleyin: birind e merkez nöron uyarılmış ve diğerleri inhibe edilmiş, diğer konfigürasyonda ise merkez nöron inhibe edilmiş ve diğerleri uyarılmıştır. Ağı, merkez nöron uyarılmış ve diğer nöronlardan en fazla iki tanesi uyarılmış şekilde başlatırsa k, ağ eski kararlı konfigürasyona doğru yol alacaktır.
Ağı, çevredeki en az dört bitişik nöron uyarılmış durumda olacak şekilde başl atırsak, ağ son konfigürasyona doğru yol alacaktır. Böylece

Şekil **11.22** Kararlı bir konfigürasyona giden adımlar

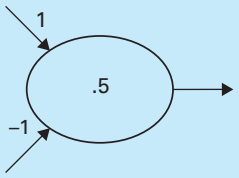


Doclingo https://www.doclingo.ai

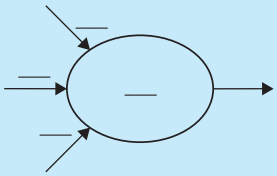
Ağ, önceki kararlı konfigürasyonu, merkez nöronunun ve üçten az çevre nör onunun heyecanlandığı başlangıç desenleri ile ilişkilendiriyor ve sonraki kara rlı konfigürasyonu, dört veya daha fazla çevre nöronunun heyecanlandığı ba şlangıç desenleri ile ilişkilendiriyor. Kısacası, ağ, temel bir çağrışım belleği te msil ediyor.

Sorular & Alıştırmalar

1. Her iki girişi de 1 olan bu nöronun çıktısı nedir?
   0, 0; 0, 1; ve 1, 0 giriş desenleri hakkında ne dersiniz?



2. Aşağıdaki nöronun ağırlıklarını ve eşik değerini ayarlayın, böylece çık tısı yalnızca en az iki girişi 1 olduğunda 1 olsun.



3. Yapay sinir ağını eğitirken meydana gelebilecek bir problemi tanımlayın.
4. Şekil 11.22'deki ağ, tüm nöronları engellenmiş olarak başlatıldığında han gi kararlı konfigürasyona doğru gidecektir?

11.6 Robotik

Robotik, fiziksel, özerk ajanların akıllı bir şekilde davrandığı bir çalışmadır. Tüm ajanlarda olduğu gibi, robotların çevrelerini algılayabilmesi, akıl yürütebilmesi ve hare ket edebilmesi gerekir. Bu nedenle robotik araştırmaları, yapay zekanın tüm alanlarını kap samakta ve mekanik ile elektrik mühendisliğinden yoğun bir şekilde yararlanmaktadır.

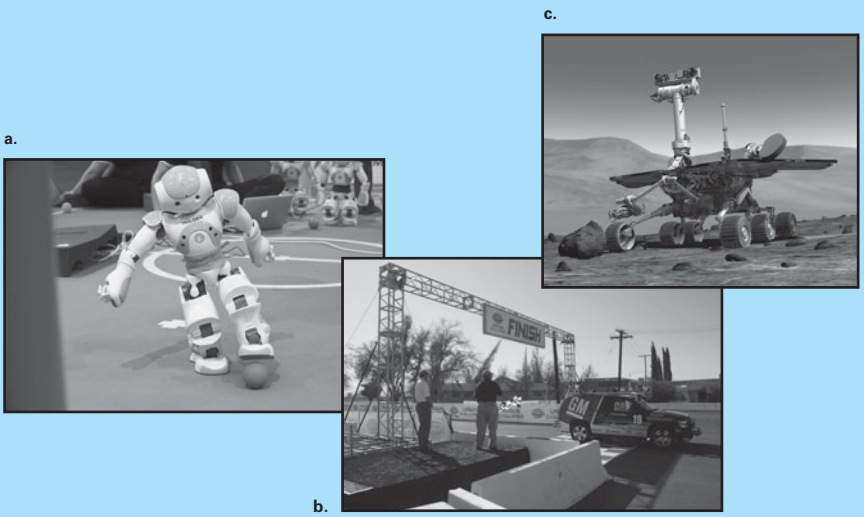
Dünyayla etkileşimde bulunmak için robotların nesneleri manipüle etme ve hareket etme mekanizmalarına ihtiyacı vardır. Robotik alanının ilk günlerinde, bu alan, en çok dirsek, bilek ve eller veya aletlerle mekanik kolların geliştirilmes iyle yakından ilişkiliydi. Araştırmalar, bu tür cihazların nasıl manevra yapabilece ği ile birlikte, konumları ve yönleri hakkında bilginin nasıl sürdürülebileceği ve uygulanabileceği ile de ilgileniyordu. (Gözlerinizi kapatıp parmağınızla burnunu za dokunabiliyorsunuz çünkü beyniniz burnunuzun ve parmağınızın nerede old uğunu kaydediyor.) Zamanla robotların kolları, dokunma hissi ile birlikte daha yetenekli hale geldi ve artık yumurta ve kağıt bardakları başarıyla tutabiliyorlar.

Son zamanlarda, daha hızlı ve daha hafif bilgisayarların geliştirilmesi, hareket edebi len mobil robotlar üzerinde daha fazla araştırmaya yol açtı. Bu hareketliliği sağlamak

Doclingo https://www.doclingo.ai

Robotlar Tarih Yazıyor

a. Bir futbol robotu, 15 Nisan 2010'da RoboCup Almanya Açık Turnuvası'nda bir topa vuruyor, 2010 Doğu Almanya'daki Magdeburg'da (© Jens Schlueter/SFP/gettyimages/ Newscom). **b.** Tartan Racing'in "Boss"u—DARPA tarafından sponsor olunan, araçların kendiliğ inden bir kentsel ortamda sürmesini sağlamak için düzenlenen Urban Challenge yarışmasın ın kazananı (© DARPA). **c.** NASA'nın Rovers'larından biri—Mars yüzeyini keşfeden bir robot je olog (NASA/JPL-Caltech'in izniyle).



robot hareketliliği alanında yaratıcı tasarımların bolluğuna yol açmıştır. Araştırmacılar, balık gibi yüzebilen, yusufçuk gibi uçabilen, çekirge gibi zıplayabilen ve yılan gibi sürüne bilen robotlar geliştirmiştir.

Tekerlekli robotlar, tasarım ve inşa açısından nispeten kolay oldukları için oldukça popülerdir, ancak geçebilecekleri arazi türü açısından sınırlıdırlar. Bu kısıtlamayı aşmak, merdivenleri tırman mak veya kayaların üzerinden yuvarlanmak için tekerlek veya palet kombinasyonları kullanmak, m evcut araştırmanın amacıdır. Örneğin, NASA'nın Mars keşif araçları, kayalık topraklarda hareket et mek için özel olarak tasarlanmış tekerlekler kullanmıştır.

Bacaklı robotlar daha fazla hareket kabiliyeti sunar ancak önemli ölçüde daha ka rmaşıktır. Örneğin, insanlar gibi yürümek üzere tasarlanmış iki bacaklı robotlar, süre kli olarak duruşlarını izlemeli ve ayarlamalıdır; aksi takdirde düşerler. Ancak, bu tür z orluklar aşılabilir; Honda tarafından geliştirilen ve merdivenleri çıkabilen hatta koşab ilen Asimo adlı iki bacaklı insansı robot bunun bir örneğidir.

Manipülatörler ve hareketlilikte büyük ilerlemelere rağmen, çoğu robot hala ç ok özerk değildir. Endüstriyel robot kolları genellikle her görev için katı bir şekilde programlanmıştır ve sensör olmadan çalışır; parçaların kendilerine tam konumlar da verileceğini varsayarlar. NASA'nın Mars keşif araçları ve askeri insansız hava ar açları (İHA'lar) gibi diğer mobil robotlar, zeka için insan operatörlerine güvenmekt edir.

Bu insanlara olan bağımlılığı aşmak, mevcut araştırmanın önemli bir hedefidir. Bir soru, özerk bir robotun çevresi hakkında ne bilmesi gerektiği ve eylemleri ni ne ölçüde önceden planlaması gerektiği ile ilgilidir. Bir

Doclingo https://www.doclingo.ai

yaklaşım, çevrelerinin ayrıntılı kayıtlarını tutan robotlar inşa etmektir; bu kayıtlar, nesnelerin envanterini ve konumlarını içermekte ve bunlarla kesin eylem planları geliştirmektedir. Bu yöndeki araştırmalar, bilgi temsilinin ve depolamanın yanı sıra geliştirilmiş akıl yürütme ve plan geliştirme tekniklerinin ilerlemesine büyük ölçüde bağlıdır.

Alternatif bir yaklaşım, karmaşık kayıtlar tutmak ve ayrıntılı eylem planları oluşturmak için büyük çaba harcamak yerine, dünyayla etkileşimde basit kurallar uygulayan reaktif robotlar geliştirmektir. Reaktif robotik savunucuları, bir araba ile uzun bir yolculuk planlarken insanların her şeyi kapsayan, ayrıntılı planlar yapmadığını savunuyor. Bunun yerine, sadece ana yolları seçerler ve nerede yemek yiyecekleri, hangi çıkışları alacakları ve sapmaları nasıl yönetecekleri gibi ayrıntıları daha sonra düşünürler. Benzer şekilde, kalabalık bir koridorda gezinmesi veya bir binadan diğerine geçmesi gereken bir reaktif robot, önceden son derece ayrıntılı bir plan geliştirmez, bunun yerine karşılaştığı her engeli aşmak için basit kurallar uygular. Bu, tarihin en çok satan robotu olan iRobot Roomba elektrikli süpürgenin benimsediği yaklaşımdır; bu robot, mobilyaların ve diğer engellerin ayrıntılarını hatırlamaya çalışmadan reaktif bir modda bir zemin üzerinde hareket eder. Sonuçta, aile kedisinin muhtemelen bir sonraki sefer aynı yerde olmayacağıdır.

Elbette, tek bir yaklaşımın tüm durumlar için en iyi olduğunu kanıtlaması pek olası değildir. Gerçekten özerk robotlar, muhtemelen birden fazla akıl yürütme ve planlama seviyesini kullanacak, yüksek seviyeli teknikleri büyük hedefler belirlemek ve başarmak için uygularken, daha düşük seviyeli reaktif sistemleri küçük alt hedeflere ulaşmak için kullanacaklardır. Bu tür çok seviyeli akıl yürütmenin bir örneği, 2050 yılına kadar dünya çapında insan futbol takımlarını yenebilecek bir robot takımı geliştirmek için araştırma forumu olarak hizmet veren Robocup yarışmasında bulunur. Burada vurgu, sadece bir topu "tekmeleyebilen" mobil robotlar inşa etmek değil, birbirleriyle işbirliği yaparak ortak bir hedefe ulaşan bir robot takımı tasarlamaktır. Bu robotlar sadece hareket etmekle kalmayıp, eylemleri hakkında akıl yürütmek zorundadırlar, aynı zamanda takım arkadaşlarının ve rakiplerinin eylemleri hakkında da akıl yürütmeleri gerekmektedir.

Robotikteki bir diğer araştırma örneği, evrimsel robotik olarak bilinen alandır; burada evrim teorileri, hem düşük seviyeli reaktif kurallar hem de yüksek seviyeli akıl yürütme için şemalar geliştirmek amacıyla uygulanmaktadır. Burada, çok sayıda nesil boyunca kendi denge veya hareket yöntemlerini edinen cihazlar geliştirmek için en uygun olanın hayatta kalma teorisinin kullanıldığını görüyoruz. Bu alandaki araştırmaların çoğu, bir robotun iç kontrol sistemi (büyük ölçüde yazılım) ile fiziksel yapısı arasında ayrım yapmaktadır. Örneğin, bir yüzme kurbağa robotunun kontrol sistemi, bacakları olan benzer bir robota aktarılmıştır. Ardından, sürünme yeteneği olan bir robot elde etmek için kontrol sistemi içinde evrimsel teknikler uygulanmıştır. Diğer durumlarda, belirli bir görevi yerine getirmek için en uygun sensör pozisyonlarını keşfetmek amacıyla bir robotun fiziksel yapısına evrimsel teknikler uygulanmıştır. Daha zorlu araştırmalar, yazılım kontrol sistemlerini fiziksel yapı ile eşzamanlı olarak evrimleştirme yollarını aramaktadır.

Robotik alanındaki araştırmalardan elde edilen tüm etkileyici sonuçları listelemek, bunaltıcı bir görev olacaktır. Mevcut robotlarımız, kurgusal filmlerde ve romanlarda yer alan güçlü robotlardan çok uzaktır, ancak belirli görevlerde etkileyici başarılar elde etmiştir. Trafikte gidebilen, evcil köpekler gibi davranabilen ve silahları hedefleri ne yönlendirebilen robotlarımız var. Ancak, bu başarıların tadını çıkarırken, yapay bir evcil köpeğe duyduğumuz sevgi ile akıllı silahların muazzam gücünün toplumu zorlayan sosyal ve etik soruları gündeme getirdiğini unutmamalıyız. Geleceğimiz, onu nasıl şekillendirdiğimizdir.

Doclingo https://www.doclingo.ai

Sorular & Alıştırmalar

1. reaktif yaklaşımın robot davranışına etkisi, daha geleneksel “plan temel li” davranıştan nasıl farklılık gösterir?
2. Robotik alanındaki bazı güncel araştırma konuları nelerdir?
3. Robot geliştirme sürecinde evrimsel teorilerin uygulandığı iki seviye nedir?

11.7 Sonuçları Düşünmek

Şüphesiz, yapay zeka alanında kaydedilen ilerlemeler insanlığa fayda sağlam a potansiyeline sahiptir ve potansiyel faydalar tarafından yaratılan heyecana kapılmak kolaydır. Ancak, gelecekte, karşıtlarının faydaları kadar yıkıcı olabil ecek potansiyel tehlikeler de gizlenmektedir. Ayrım genellikle yalnızca birinin bakış açısı veya belki de toplumdaki konumudur - bir kişinin kazancı, diğerin in kaybı olabilir. Bu nedenle, ilerleyen teknolojiyi alternatif bakış açılarıyla de ğerlendirmek için bir an durmak uygun olacaktır.

Bazıları teknolojinin ilerlemesini insanlığa bir hediye olarak görüyor—insan ları sıkıcı, sıradan görevlerden kurtaran ve daha keyifli yaşam tarzlarına kapı aç an bir araç. Ancak diğerleri bu aynı olguyu, vatandaşları istihdamdan mahrum eden ve zenginliği güç sahiplerine yönlendiren bir lanet olarak görüyor. Bu, aslı nda Hindistan'ın özverili insaniyetçi Mahatma Gandhi'nin bir mesajıydı. Gandhi, Hindistan'ın büyük tekstil fabrikalarını köylülerin evlerine yerleştirilen iplik tezg ahlarıyla değiştirmesinin daha iyi olacağını defalarca savundu. Bu şekilde, yalnı zca birkaç kişiyi istihdam eden merkezi kitlesel üretimin, çok sayıda insanın fay dasına olacak dağıtılmış bir kitlesel üretim sistemiyle değiştirileceğini iddia etti.

Tarih, zenginlik ve ayrıcalıkların orantısız dağılımında kökleri olan devrim lerle doludur. Bugünün ilerleyen teknolojisinin bu tür farklılıkları kökleşmesi ne izin verilirse, felaket sonuçlar doğabilir.

Ancak giderek daha zeki makineler inşa etmenin sonuçları, toplumun farklı kesimleri arasındaki güç mücadeleleriyle ilgili olanlardan daha ince—daha tem eldir. Sorunlar, insanlığın öz imajının tam kalbine vuruyor. On dokuzuncu yüzyıl da, toplum Charles Darwin'in evrim teorisi ve insanların daha düşük yaşam for mlarından evrimleşmiş olabileceği düşüncesi karşısında dehşete düştü. Peki, to plum, insanlarla karşılaştırıldığında zihinsel yetenekleri sorgulayan makinelerin saldırısıyla karşılaştığında nasıl tepki verecek?

Geçmişte, teknoloji yavaş gelişti ve bu, kendilik algımızın zeka kavramımızı yeniden ayarlayarak korunmasına olanak tanıdı. Antik atalarımız, on dokuzunc u yüzyılın mekanik cihazlarını doğaüstü bir zekaya sahip olarak yorumlamış ols alardı da, bugün bu makinelere hiç zeka atfetmiyoruz. Ancak, makineler gerçek ten insan zekasını zorlamaya başladığında veya daha olası olarak, makinelerin yetenekleri, bizim uyum sağlama yeteneğimizden daha hızlı bir şekilde ilerleme ye başladığında insanlık nasıl tepki verecek?

İnsanlığın, zekamızı zorlayan makinelere karşı potansiyel tepkisini, yirmi nci yüzyılın ortasında IQ testlerine toplumun tepkisini düşünerek anlayabiliri z. Bu testler, bir çocuğun

Doclingo https://www.doclingo.ai

zekayı belirlemek için kullanılıyordu. Amerika Birleşik Devletleri'ndeki çocuklar, bu testlerdeki performanslarına göre sıklıkla sınıflandırılıyor ve buna göre eğiti m programlarına yönlendiriliyordu. Sonuç olarak, bu testlerde iyi performans g österen çocuklara eğitim fırsatları açılırken, kötü performans gösteren çocuklar ise düzeltici çalışma programlarına yönlendiriliyordu. Kısacası, bireyin zekasını ölçmek için bir ölçek verildiğinde, toplum genellikle ölçeğin alt kısmında yer ala nların yeteneklerini göz ardı etme eğilimindeydi. O halde, makinelerin "entellek tüel" yetenekleri, insanlarla karşılaştırılabilir hale geldiğinde veya en azından öy le görünmeye başladığında toplum bu durumu nasıl ele alırdı? Toplum, makine lerin yeteneklerine "aşağı" olarak görülenleri mi dışlayacaktı? Eğer öyleyse, bu durumun toplumun o üyeleri için sonuçları ne olurdu?
Bir kişinin onuru, onun bir makineyle nasıl karşılaştırıldığına bağlı mı olmalıdır?

Artık insanların entelektüel yeteneklerinin belirli alanlarda makineler tar afından sorgulandığını görmeye başladık. Makineler artık satrançta uzmanla rı yenebiliyor; bilgisayarlı uzman sistemler tıbbi tavsiyeler verebiliyor; ve yatır ım portföylerini yöneten basit programlar genellikle yatırım profesyonellerin den daha iyi performans gösteriyor. Bu tür sistemler, ilgili bireylerin öz imajı nı nasıl etkiliyor? Bir birey, makineler tarafından giderek daha fazla alanda g eride bırakıldıkça, özsaygısı nasıl etkilenecek?

Birçok kişi, makinelerin sahip olduğu zekanın her zaman insanlarınkinde n farklı olacağını savunuyor, çünkü insanlar biyolojik, makineler ise değil. Bu nedenle, makinelerin asla bir insanın karar verme sürecini yeniden üretemey eceğini iddia ediyorlar. Makineler, insanlar gibi aynı kararlara ulaşabilir, anca k bu kararlar insanlar tarafından alınanlarla aynı temele dayanmadan alınac aktır. O halde, farklı türde zekaların varlığı ne kadar geçerlidir ve toplumun i nsan dışı zekanın önerdiği yolları takip etmesi etik olur mu?

Joseph Weizenbaum, kitabında, *Bilgisayar Gücü ve İnsan Aklı,* yapay zek anın kontrolsüz uygulanmasına karşı şu şekilde itiraz ediyor:

> Bilgisayarlar yargı kararları verebilir, bilgisayarlar psikiyatrik yargılarda b ulunabilir. En sabırlı insanın yapabileceğinden çok daha sofistike şekillerd e para atabilirler. Burada önemli olan, bu tür görevlerin onlara verilme mesi gerektiğidir. Bazı durumlarda "doğru" kararlara ulaşabilecekleri bile olabilir - ama her zaman ve mutlaka, hiçbir insanın kabul etmeye ist ekli olması gereken temellere dayanarak.
>
> "Bilgisayarlar ve Zihin" üzerine birçok tartışma olmuştur. Burada vardı ğım sonuç, ilgili meselelerin ne teknolojik ne de matematiksel olduğu; etik olduğu. "Olabilir mi?" ile başlayan sorular sorarak çözülemezler. B ilgisayarların uygulanabilirliğinin sınırları nihayetinde yalnızca yapılm ası gerekenler açısından ifade edilebilir. En temel içgörü olarak ort aya çıkan şey, şu anda bilgisayarları bilge yapmanın hiçbir yoluna sahi p olmadığımız için, bilgisayarlara bilgelik gerektiren görevler vermeme miz gerektiğidir.

Bu bölümün çoğunun bilgisayar bilimi yerine bilim kurgu ile sınırda oldu ğunu savunabilirsiniz. Ancak, çok uzun bir süre önce, birçok kişi "Bilgisayarla r toplumu ele geçirirse ne olur?" sorusunu asla olmayacak bir tutumla geçişti rmişti. Ama birçok açıdan, o gün şimdi geldi.
Eğer bir bilgisayarlı veritabanı yanlış bir şekilde kötü bir kredi notuna, bir suç kayd ına veya aşırı çekim hesabına sahip olduğunuzu bildiriyorsa, bilgisayarın beyanı m ı yoksa masumiyet iddianız mı geçerli olacaktır? Eğer bir arıza meydana gelirse

Doclingo https://www.doclingo.ai

navigasyon sistemi, sisle kaplı bir pistin yanlış bir yerde olduğunu gösteriyor sa,uçak nerede inecek? Kamuoyunun çeşitli siyasi kararlara tepkisini tahmin etmek için bir makine kullanılıyorsa, bir politikacı hangi kararı alır? Bir memu r, "bilgisayar kapalı" olduğu için size kaç kez yardımcı olamadı? O zaman kim (ya da ne) sorumludur? Toplumu makinelere teslim etmedik mi?

Sorular & Alıştırmalar

1. Bugünün nüfusunun ne kadarı, son yüz yılda geliştirilen makineler k aldırılırsa hayatta kalırdı? Peki ya son elli yıl? Yirmi yıl ne olacak? Haya tta kalanlar nerede bulunacak?
2. Hayatınız ne ölçüde makineler tarafından kontrol ediliyor? Hayatınızı etkileyen makineleri kim kontrol ediyor?
3. Günlük kararlarınızı dayandırdığınız bilgiyi nereden alıyorsunuz? Pek i ya büyük kararlarınız? O bilginin doğruluğuna ne kadar güveniyorsu nuz? Neden?

Bölüm Gözden Geçirme Problemleri

(Yıldızlı problemler isteğe bağlı bölümlerle ilişkilidir.)

1. 11.2. Bölümde gösterildiği gibi, insanlarbir sor uyu sormaktan başka bir amaçlakullanabilirler. Bir diğer örnek "Lastiğinizinpatlak olduğunu biliyor mus unuz?" ifadesidir; bu,sormak yerine bilgilendir mek için kullanılır. Rahatlatmak,uyarmak ve eleştirm ek için kullanılan sorularaörnekler verin.
2. Bir soda dağıtım makinesini birajan olar ak analiz edin. Sensörleri nelerdir? Aktüatörl eri nelerdir?Hangi düzeyde bir tepki (refleks , bilgiye dayalı,amaç odaklı) sergiliyor?
3. Aşağıdaki her bir yanıtırefleks, bilgiye da yalı veya hedefe dayalı olarak tanımlayın. Cevaplarınızı gerekçelendirin.
   a. Almanca'dan İngilizce'ye metin çeviren bir bil gisayar programı
   b. Bir termostatın, bir evdeki sıcaklık mevcu t ayarın altına düştüğündeısıtıcıyı açması
   c. Bir pilotun bir uçağı güvenli bir şekilde bir pistte indirmesi
4. Eğer bir araştırmacı insan zihninin belleklem e yeteneklerini incelemek için bilgisayar modeller i kullanıyorsa, bu amaçla geliştirilen programlar makine, makinenin yeteneklerinin en iyisiyle mutlaka ezberlemeli mi? Açıklayın.
5. Bazı açıklayıcı bilgi örnekleri verin. Bazı prosedürel bilgi örnekleri verin.

*6. Nesne yönelimli programlama bağlamında,bi r nesnenin hangi kısımları açıklayıcı bilgiyisakla mak için kullanılır? Hangi kısımlarprosedürel bilg iyi saklamak için kullanılır?

7. Aşağıdaki aktivitelerden hangisinin performansa yöneli k olduğunu ve hangisinin simülasyona yönelik ol duğunu bekliyorsunuz?
   a. Bir otomatik servis sistemi tasarımı(genellikle havaalanlarında terminaller arasında kullanı lır)b. Bir kasırganın yolunu tahmin eden bir mo delin tasarımı
   c. Dünya Çapında Ağ'da saklanan belgeler için dizi nleri türetmek ve sürdürmek amacıyla kullan ılan bir Web arama veritabanının tasarımı
   d. Teorileri test etmek için bir ulusun ekonomisini n modelinin tasarımı
   e. Bir hastanın hayati belirtilerini izlemek için bir programın tasarımı

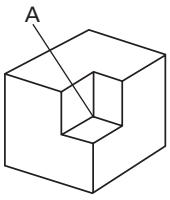
Doclingo https://www.doclingo.ai

8. Bugün, bazı telefon görüşmeleri, arayanl a konuşmak için konuşma ve ses tanıma kullanan otomatik yanıt sistemleri tarafınd an yönetilmektedir. Bu sistemler Turing test ini geçiyor mu? Cevabınızı açıklayın.

9. F, E, L ve T sembollerini ayırt etmek için k ullanılabilecek küçük bir geometri özellikl eri setini tanımlayın.

*10. Karakteristikleri şablonlarla karşılaştırar ak tanımlama tekniği ile Bölüm 1'de tartışıl an hata düzeltme kodları arasındaki benzer likleri tanımlayın.

11. Aşağıdaki çizim çizgisinin "köşe"sinin A ola rak işaretlenip işaretlenmediğine bağlı ol arak iki yorumunu tanımlayın:



12. Aşağıdaki iki cümledeki (sadece bir kelimey le farklı olan) edat cümlelerinin rollerini kar şılaştırın. Bir makine bu tür ayrımları yap mak için nasıl programlanabilir?

Ahırın yanına domuz ahırı inşa edildi.
Domuz ahırını çiftçi inşa etti.

13. Aşağıdaki iki cümlenin ayrıştırma sonuçları nas ıl farklılık gösteriyor? Anlamsal analiz sonuçları nasıl farklılık gösteriyor?

Andrea muhteşem bir gün batımı gördü.
Andrea muhteşem bir gün batımını gördü.

14. Aşağıdaki iki cümlenin ayrıştırma sonuçları nas ıl farklılık gösteriyor? Anlamsal analiz sonuçları nasıl farklılık gösteriyor?

Eğer X < 10 ise X'ten 1 çıkar, aksi takdirde X'e 1 ekle.
Eğer X > 10 ise X'e 1 ekle, aksi takdirde X'ten 1 çıkar.

15. Metinde, doğal dillerin formal programlam a dillerine kıyasla anlama sorunlarını kısac a tartıştık. Doğal dillerin karmaşıklıkları açı sından, "Saat kaç olduğunu biliyor musun? " sorusunun farklı anlamlara sahip olduğu durumları belirleyin.

16. Bir cümlenin bağlamındaki değişiklikle r, cümlenin önemini ve anlamını değiştirebil ir. Şekil 11.3 bağlamında, "Mary John'a vu rdu" cümlesinin önemi, doğum tarihleri 2 000'lerin sonlarında olsaydı nasıl değişirdi? Peki, biri 1980'lerde diğeri 2000'lerin so nlarında olsaydı?

17. Aşağıdaki paragraftaki bilgiyi temsil eden bir anlamsal ağ çizin.

Donna topu Jack'a attı, o da topuorta sahay a vurdu. Ortada yer alan oyuncuonu yakala maya çalıştı, ama top duvardansekti.

18. Bazen bir soruya cevap verme yeteneği, bilginin sınırlarını bilmek kadar, bilgilerin kendisi kadar önemlidir. Örneğin, veritabanları A ve B'nin her ikisi de şirketin sağlık sigortası programınaait ça lışanların tam bir listesini içeriyor,fakat yalnız ca veritabanı A listenin tamolduğunun fark ında. Veritabanı A, kendilistesinde olmayan bir üye hakkındaveritabanı B'nin çıkaramay acağı nesonuç çıkarabilir?

19. Kapalı dünya varsayımının bir çelişkiye neden olduğu bir örnek verin.

20. Kapalı dünya varsayımının yaygın olarak kullanıldığı iki örnek verin.

21. Bir üretim sistemi bağlamında, bir durum grafiği ile bir arama ağacı arasındaki fark nedir?

22. Rubik küpünü çözme görevini bir üretim sistemi açısından analiz edin. (Durumlar, üretimler vb. nedir?)

23. a. Bir arama ağacının ikili bir ağaçolduğunu ve hedefe ulaşmanın sekizüretim gerek tirdiğini varsayalım. Eğerağaç genişlik ön celikli bir şekildeinşa edilirse, hedef d urumuulaşıldığında ağaçta olabileceken büyük düğüm sayısı nedir?

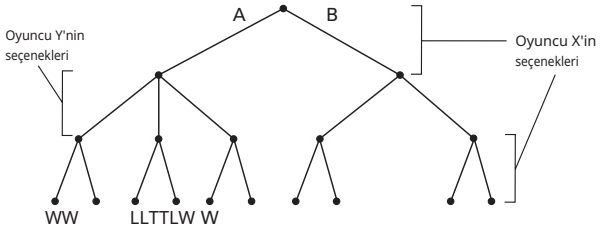
b. Arama sırasında dikkate alınan toplamdüğüm s ayısının, biri başlangıçdurumundan başlayıp diğeri hedeftengeriye doğru arama yapar ak iki aramaaynı anda gerçekleştirilerek nasılaz altılabileceğini açıklayın—iki aramabirbirine ul aşana kadar. (Geriye doğruarama sırasında bu lunan durumlarıkaydeden arama ağacının da bir ikiliağaç olduğunu ve her iki aramanın daaynı hızda ilerlediğini varsayın.)

Doclingo https://www.doclingo.ai

24. Metinde, bir üretim sisteminin genellikle bilin en gerçeklerden sonuçlar çıkarmak için bir tekn ik olarak kullanıldığını belirttik. Sistem durumla rı, akıl yürütme sürecinin her aşamasında doğru olduğu bilinen gerçeklerdir ve üretimler, bilin en gerçekleri manipüle etmek için mantık kurall arıdır. "John uzundur" sonucunun, "John bir bas ketbol oyuncusudur," "Basketbol oyuncuları kısa değildir" ve "John ya kısadır ya da uzundur" gerç eklerinden elde edilmesini sağlayan bazı ma ntık kurallarını belirleyin.

25. Aşağıdaki ağaç, rekabetçi bir oyundaki olası hamleleri t emsil eder ve oyuncu X'in şu anda A hamlesi ile B haml esi arasında bir seçim yapma hakkına sahip olduğunu gösterir. Oyuncu X'in hamlesinin ardından, oyunc u Y bir hamle seçme hakkına sahiptir ve ardından oyunc u X oyunun son hamlesini seçme hakkına sahiptir. Ağa cın yaprak düğümleri, bu sonucun oyuncu X için bir zafer, kayıp veya berabereyi temsil edip etmediğine bağlı olarak W, L veya T ile etiketlenmiştir.

Oyuncu X A hamlesini mi yoksa B hamlesini mi seçmeli? Neden? Rekabetçi bir ortamda bir "üretim" se çmenin, sekiz bulmacası gibi tek kişilik bir o yundan nasıl farklı olduğunu açıklayın.



26. Dama oyununu bir üretim sistemi olarak analiz ed in ve iki durumdan hangisinin hedefe daha yakın olduğunu belirlemek için kullanılabilecek bir sezgis el tanımlayın. Bu ortamda kontrol sistemi, sekiz b ulmacası gibi bir tek kişilik oyunun kontrol sist eminden nasıl farklı olacaktır?

27. Cebirsel ifadelerin basitleştirilmesi ile ilgili problemler, üretim sisteminin bağlamında c ebirsel manipülasyon kurallarını üreti m olarak dikkate alarak çözülebilir. 3/(2x1) 6x(B 1) denklemini x3 biçimine indi rgemeye olanak tanıyan bir dizi cebirse l üretim belirleyin. Böyle cebirsel basitleştir meler yaparken kullanılan bazı kural dışı (yani, sezgisel kurallar) nelerdir?

28. Aşağıdaki başlangıç durumundan sekiz bu lmacayı çözmeye çalışırken üretilen geniş lik öncelikli arama ağacını çizin. Herhangi bir sezgisel bilgi yardımı kullanmadan durumu çözmeye çalışın.

	1	3
4	2	5
7	8	6

29. Problem 28'deki başlangıç durumundan sekiz bulma cayı çözmeye çalışırken, yerinden çıkmış karoların sayı sının bir sezgisel olarak kullanıldığı durumda Şekil 11.1 0'un en iyi uyum algoritması tarafından üretilen aram a ağacını çizin.

30. Aşağıdaki başlangıç durumundan sekiz bulmac ayı çözmeye çalışırken, kullanılan sezgiselin Bölü m 11.3'te geliştirilenle aynı olduğunu varsaya rak Şekil 11.10'un en iyi uyum algoritması tarafı ndan üretilen arama ağacını çizin.

1	2	3
5	7	6
4		8

31. Sekiz bulmacayı çözerken, yerinden çıkmış karolar ın sayısının Bölüm 11.3'te kullanılan sezgisel kad ar iyi bir sezgisel olmamasının nedeni nedir?

32. Listeyi göz önünde bulundururken hangi ya rısını seçeceğinizi belirleme tekniği ile bir sezgisel arama yaparken hangi dalı takip ed eceğinizi belirleme arasındaki fark nedir?

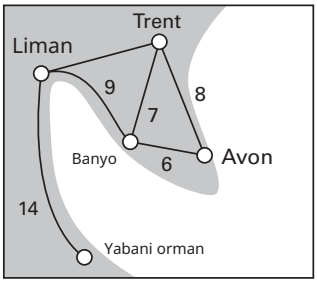
33. Bir üretim sisteminin durum grafiğindeki bir durum, diğer durumlarla karşılaştırıldığında so n derece düşük bir sezgisel değere sahipse v e o durumdan kendisine bir üretim varsa, Şe kil 11.10'daki algoritma o durumu tekrar tekra r göz önünde bulundurarak döngüye girebilir. Sistemdeki herhangi bir üretimi gerçekleştir menin maliyeti en az bir ise, o zaman sezgisel değerin yanı sıra geçilen yol boyunca duruma ulaşmanın maliyetinin toplamı olarak öngörül en maliyeti hesaplayarak bu sonsuz döngü s ürecinin önleneceğini gösterin.

34. Büyük bir yol haritasında iki şehir arasında bir rota ararken hangi sezgiyi kullanıyorsunuz?

35. Şekil 11.10'daki en iyi uyum algoritması ile Trent'ten Wildwood'a giden rotayı bulmak için üretilen aram a ağacını dört seviyeye kadar çizin. Arama ağacınd aki her düğüm haritadaki bir şehir olacaktır. Trent için bir düğümle başlayın. Bir düğümü genişletirken

Doclingo https://www.doclingo.ai

düğüm, genişletilen şehre doğrudan bağlı ola
n sadece şehirleri ekleyin. Her düğümde Wil
dwood'a olan düz mesafeyi kaydedin
ve bunu sezgisel değer olarak kullanın.
En iyi uyum algoritmasının işleme sürecinde bir k
usuru var mı? Varsa, hangi düzeltme gereklidir?



Dik bir çizgi mesafesi
Wildwood'dan

Avon	10
Bath	8
Trent	15
Seaport	13

36. A* algoritması, en iyi ilk algoritmayı iki önemli şekilde değiştirir. İlk olarak, bir duruma ulaşm anın gerçek maliyetini kaydeder. Bir haritadak i bir rota durumunda, gerçek maliyet, kat ed ilen mesafedir. İkinci olarak, genişletilecek bir d üğüm seçerken, gerçek maliyet ile sezgisel de ğerin toplamı en küçük olan düğümü seçe r. Bu iki değişiklikten kaynaklanacak olan Prob lem 35'in arama ağacını çizin. Her düğümd e şehre ulaşmak için kat edilen mesafeyi, hed efe ulaşmak için sezgisel değeri ve bunların t oplamını kaydedin. Dearborn'dan Wildwoo d'a bulunan yol nedir?

37. Bir sezgiselin, bir üretim sisteminde faydalı olabilmesi i çin sahip olması gereken iki özelliği listeleyin.

38. İki kova olduğunu varsayın. Birinin kapasit esi tam olarak üç litre; diğerinin kapasitesi beş litredir. Bir kovadan diğerine su dökebilir, bir kovayı boşaltabilir veya istediğiniz zaman bir kovayı doldurabilirsiniz. Probleminiz, beş lit relik kovaya tam olarak dört litre su koymak tır. Bu problemin nasıl bir üretim sistemi olara k çerçevelenebileceğini tanımlayın.

39. Görevinizin, her biri en fazla on dört ton taşıyabil en iki kamyonun yüklenmesini denetlemek ol duğunu varsayın. Yük, toplam ağırlığı yirmi seki z ton olan çeşitli sandıklardır, ancak her bir s andığın ağırlığı sandıktan sandığa değişir. Her sandığın ağırlığı yan tarafında işaretlenmiştir. İki kamyon arasında sandıkları bölmek için hangi sezgisel yöntemi kullanırsınız?

40. Aşağıdakilerden hangileri meta-akıl yü rütme örnekleridir?

a. Uzun zamandır gitti, bu yüzden uzaklara gitmi ş olmalı.

b. Genellikle yanlış karar verdiğim için ve son iki kararım doğru olduğu için, bir son raki kararımı tersine çevireceğim.

c. Yorgun hissediyorum, bu yüzden muhtemelen n et düşünemiyorum.

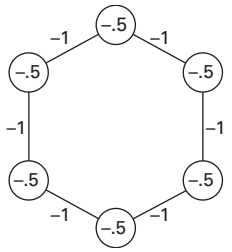
d. Yorgun hissediyorum, bu yüzden bir şekerleme yapmayı düşünüyorum.

41. Bir insanın çerçeve problemini çözme yeteneğinin, kaybolan eşyaları bulmasına nasıl yardımcı olduğunu tanımlayın.

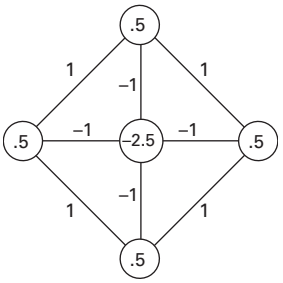
42. a. Taklit yoluyla öğrenme, denetimli eğitim yoluy la öğrenmeye hangi anlamda benzer?

b. Taklit yoluyla öğrenme, denetimli eğitim y oluyla öğrenmeden hangi anlamda farklıdır?

43. Aşağıdaki diyagram, Bölüm 11.5'te tartışıldı ğı gibi, bir ilişkisel bellek için yapay bir sinir ağını temsil etmektedir. Sadece bir nöronla ayrılmış iki nöronun uyarıldığı herhangi bi r desenle hangi deseni ilişkilendirir? Ağ, tü m nöronları inhibe edilmiş olarak başlatılır sa ne olur?



44. Aşağıdaki diyagram, Bölüm 11.5'te tartışıldı ğı gibi, bir ilişkisel bellek için yapay bir sinir ağını temsil etmektedir. En az üç nöronun uyarıldığı ve merkez nöronun inhibe olduğu herhangi bir başlangıç deseniyle hangi kar arlı konfigürasyonu ilişkilendirir? Eğer sadec e birbirine zıt olan iki nöronun uyarıldığı bir başlangıç deseni verilirse ne olur?



45. Bir ilişkilendirme belleği için bir yapay sinir a ğı tasarlayın (Bölüm 11.5'te tartışıldığı gibi)

Doclingo https://www.doclingo.ai

tek bir dikey sinir sütununun uyarıldığı kararlı de senlere doğru hareket etmeye çalışan bir sinir h ücreleri dikdörtgen dizisinden oluşmaktadır.

46. Aşağıdaki 11.18 numaralı yapay sinir ağındaki ağır lıkları ve eşik değerlerini ayarlayın, böylece çık tısı her iki girdi aynı olduğunda (her ikisi de 0 ve ya her ikisi de 1) 1 ve girdiler farklı olduğunda (biri nin 0, diğerinin 1 olduğu durumlarda) 0 olur.

47. $7x+3\ x5$3cebirsel ifadesini *x ifadesine sad eleştirme sürecini temsil eden, Şekil 11.5' e benzer bir diyagram çizin.*

$= -2.$

48. Önceki probleminize verdiğiniz yanıtı genişleter ek, bir kontrol sisteminin problemi çözmeye çalışırken izleyebileceği diğer yolları gösterin.

49. "Polly uçar" sonucuna ulaşırken, "Polly bir papağan," "Bir papağan bir kuştur" ve "Tü m kuşlar uçar" gibi başlangıç gerçeklerinde n yararlanarak, Şekil 11.5'e benzer bir diyagr am çizin.

50. Önceki problemin ifadesinin aksine, devek uşu veya kırık kanatlı bir robin gibi bazı ku şlar uçamaz.
Ancak, "Tüm kuşlar uçar" ifadesinin tüm i stisnalarının açıkça listelendiği bir çıkarım sal akıl yürütme sistemi kurmak mantıklı gör ünmüyor. O halde, biz insanlar belirli bir ku şun uçar mı yoksa uçamaz mı olduğunu nasıl karar veriyoruz?

51. "Yeni vergi yasasını okudum" cümlesinin anl amının bağlama nasıl bağlı olduğunu açıklayın.

52. Bir şehirden diğerine seyahat etme problemi nin nasıl bir üretim sistemi olarak çerçevelen ebileceğini tanımlayın. Haller nelerdir? Üretiml er nelerdir?

53. A, B ve C adında, herhangi bir sırayla (ancak aynı anda değil) ger çekleştirilebilecek üç görev yapmanız gerektiğini varsayalım. Bu problemin nasıl bir üretim sistemi olar ak çerçevelenebileceğini tanımlayın ve du rum grafiğini çizin.

54. Önceki problemin durum grafiği, görev C'nin görev B'den önce gerçekleştirilmesi gerektiğind e nasıl değişir?

55. a. *i* ve *j* pozitif tam sayıları olan (*i, j*) notasyo nunun, "eğer listedeki i. pozisyondaki giri ş, j. pozisyondaki girişten büyükse, iki giriş i yer değiştirin" anlamına geldiği varsayı ldığında, aşağıdaki iki diziden hangisi üç uz unluğundaki bir listeyi sıralamada daha iyi bir iş çıkarır?
$(1, 3)\ (3, 2)$
$(1, 2)\ (2, 3)\ (1, 2)$

b. Bu şekilde yer değiştirme dizilerini te msil ederek, diziler alt dizilere ayrılabilir v e ardından yeni diziler oluşturmak için ye niden birleştirilebilir.
Bu yaklaşımı, on uzunluğundaki listeleri sıralayan bir program geliştirmek için bir genetik algoritmay ı tanımlamak üzere kullanın.

56. Bir grup robotun her bir üyesinin bir çift se nsörle donatılacağını varsayın. Her sensö r, önünde iki metre mesafe içinde bir nesne yi doğrudan tespit edebilir. Her robot, yuv arlak bir çöp kutusu şeklindedir ve her yöne hareket edebilir. Sensörlerin, bir basketb olu düz bir çizgide iten bir robot üretmek içi n nereye yerleştirilmesi gerektiğini belirlem ek üzere bir deney dizisi tasarlayın. Den ey diziniz, evrimsel bir sistemle nasıl karşıl aştırılıyor?

57. Kararları reaktif mi yoksa planlı bir modda m ı alma eğilimindesiniz? Cevabınız, öğle y emeği için ne seçeceğinizi mi yoksa kariy er kararı mı alacağınızı belirlemeniz e bağlı mı?

Sosyal Sorunlar

Aşağıdaki sorular, bilişim alanıyla ilişkili etik/sosyal/hukuki sorunlara bir rehb er olarak tasarlanmıştır. Amaç, bu soruları yalnızca yanıtlamak değildir. Ayrıc a, neden böyle yanıt verdiğinizi ve gerekçelerinizin bir sorudan diğerine tuta rlı olup olmadığını da düşünmelisiniz.

1. Nükleer enerji, genetik mühendislik ve yapay zeka alanındaki araştırmacıla r, çalışmalarının sonuçlarıyla ilgili olarak ne ölçüde sorumlu tutulmalıdır?

Doclingo https://www.doclingo.ai

Bir bilim insanı, araştırmalarının ortaya koyduğu bilgi için sorumlu mudu r? Ortaya çıkan bilgi beklenmedik bir sonuçsa ne olur?

2. Zeka ile simüle edilmiş zeka arasında nasıl bir ayrım yaparsınız? Araların da bir fark olduğuna inanıyor musunuz?
3. Diyelim ki bir bilgisayarlı tıbbi uzman sistemi, tıp camiasında sağlam tavs iyeler verme konusunda bir üne kavuştu. Bir doktor, bu sistemin hastalar ın tedavisiyle ilgili kararlarını değiştirmesine ne ölçüde izin vermelidir? D oktor, uzman sistemin önerdiği tedaviye aykırı bir tedavi uygularsa ve sis tem doğru çıkarsa, doktor tıbbi hatadan mı sorumlu olur? Genel olarak, bir uzman sistemi bir alanda iyi biliniyorsa, bu durum insan uzmanların k endi yargılarını oluşturma yeteneklerini ne ölçüde engelleyebilir, artırma k yerine?
4. Pek çok kişi, bir bilgisayarın eylemlerinin yalnızca nasıl programlandığının sonuçları ol duğunu ve dolayısıyla bir bilgisayarın özgür iradeye sahip olamayacağını savunur. Bu durumda, bir bilgisayarın eylemlerinden sorumlu tutulmaması gerekir. Bir insanın zih ni bir bilgisayar mıdır? İnsanlar doğumda önceden programlanmış mıdır? İnsanlar çev releri tarafından mı programlanır? İnsanlar eylemlerinden sorumlu mudur?
5. Bilim, yapabileceği halde takip etmemesi gereken yollar var mı? Örneğin, insanlarla karşılaştırılabilir algı ve akıl yürütme becerilerine sahip bir ma kine inşa etmek mümkün hale gelirse, böyle bir makinenin inşası uygun olur mu? Böyle bir makinenin varlığı hangi sorunları gündeme getirebilir ? Diğer bilim alanlarındaki ilerlemelerin bugün gündeme getirdiği bazı s orunlar nelerdir?
6. Tarih, bilim insanlarının ve sanatçıların çalışmalarının, zamanlarının siyas i, dini veya diğer sosyal etkilerinden etkilendiği birçok örnekle doludur. B u tür sorunlar mevcut bilimsel çabaları nasıl etkiliyor? Özellikle bilgisayar bilimi hakkında ne söyleyebilirsiniz?
7. Bugün birçok kültür, gelişen teknoloji nedeniyle işleri gereksiz hale gelenlere yeniden eğitim verme konusunda en azından bir miktar sorumluluk alıyor. Te knoloji, yeteneklerimizin giderek daha fazla gereksiz hale gelmesiyle toplum ne yapmalı/yapabilir?
8. Diyelim ki $0.00 tutarında bilgisayar tarafından üretilmiş bir fatura aldınız. Ne yapmalısınız? H içbir şey yapmadığınızı varsayalım ve 30 gün sonra hesabınızda $0.00 borçlu olduğunuzu beli rten ikinci bir bildirim alıyorsunuz. Ne yapmalısınız? Hiçbir şey yapmadığınızı varsayalım ve 30 gün sonra hesabınızda $0.00 borçlu olduğunuzu belirten bir başka bildirim alıyorsunuz ve fat uranın zamanında ödenmemesi durumunda yasal işlem yapılacağına dair bir not eklenmiş. Ki m sorumlu?
9. Kişisel bilgisayarınızla kişilikleri ilişkilendirdiğiniz zamanlar var mı? Bilgisa yarınızın intikamcı veya inatçı göründüğü zamanlar oluyor mu? Bilgisaya rınıza hiç sinirleniyor musunuz? Bilgisayarınıza sinirlenmek ile bilgisayarı nızın sonucu olarak sinirlenmek arasında ne fark var? Bilgisayarınız size hiç sinirleniyor mu? Arabalar, televizyonlar ve tükenmez kalemler gibi di ğer nesnelerle benzer ilişkileriniz var mı?
10. Soru 9'a verdiğiniz yanıtlara dayanarak, insanlar bir varlığın davranışını z eka ve farkındalık varlığı ile ilişkilendirmeye ne ölçüde istekli? İnsanların böyle ilişkiler kurması ne ölçüde gereklidir? Zeki bir varlığın, davranışınd an başka bir şekilde zekasını ortaya koyması mümkün mü?

Doclingo https://www.doclingo.ai

11. Pek çok kişi, Turing testini geçme yeteneğinin bir makinenin zeki olduğu an lamına gelmediğini düşünüyor. Bir argüman, zeki davranışın kendiliğinden zekayı ima etmediğidir. Ancak evrim teorisi, en güçlülerin hayatta kalması üzerine kuruludur ki bu da davranışa dayalı bir testtir. Evrim teorisi, zeki da vranışın zekanın bir öncüsü olduğunu mu ima ediyor? Turing testini geçme yeteneği, makinelerin zeki olma yolunda olduğunu mu gösterir?

12. Tıbbi tedavi, insan vücudunun birçok parçasının artık yapay parçalar veya insa n bağışçılarından alınan parçalarla değiştirilebileceği bir noktaya ilerlemiştir. Bu durumun bir gün beynin parçalarını da kapsayabileceği düşünülebilir. Bu tür yetenekler hangi etik sorunları ortaya çıkarır? Bir hastanın nöronl arı birer birer yapay nöronlarla değiştirildiğinde, hasta aynı kişi olarak ka lır mı? Hasta bir fark hisseder mi? Hasta insan olarak kalır mı?

13. Bir otomobildeki GPS, sürücüye yaklaşan dönüşler ve diğer eylemler hakkın da bildiren dostça bir ses sağlar. Sürücü bir hata yaptığında, otomatik olara k ayarlamalar yapar ve yola geri dönmek için yönlendirme sağlar, gereksiz bir duygu olmadan. Bir GPS'in yeni bir hedefe giderken sürücünün stresini azalttığını düşünüyor musunuz? Bir GPS, strese hangi şekillerde katkıda bul unur?

14. Akıllı telefonunuz sesli sesli dil çevirisi sağlıyorsa, bu özelliği kullanmakta n rahat hisseder misiniz? Doğru anlamı ileteceğine güvenir misiniz? Herh angi bir endişeniz olur mu?

Ek Okuma

Banzhaf, W., P. Nordin, R. E. Deller ve F. D. Francone. *Genetik Programlama: Bir Giriş.*San Francisco, CA: Morgan Kaufmann, 1998.

Lu, J. ve J. Wu. *Çoklu Ajan Robotik Sistemler*. Boca Raton, FL: CRC Press, 2001.

Luger, G. F. *Yapay Zeka: Karmaşık Problem Çözme için Yapılar ve Stratejiler,* 5. baskı. Boston, MA: Addison-Wesley, 2005.

Mitchell, M. *Genetik Algoritmalara Giriş.*Cambridge, MA: MIT Press, 1998.

Negnevitsky, M. *Yapay Zeka: Akıllı Sistemler için Bir Rehber,* 2. baskı. Boston, MA: Addison-Wesley, 2005.

Nilsson, N. *Yapay Zeka: Yeni Bir Sentez.*San Francisco, CA: Morgan Kaufmann, 1998.

Nolfi, S. ve D. Floreano. *Evrimsel Robotik*. Cambridge, MA: MIT Press, 2000.

Rumelhart, D. E. ve J. L. McClelland. *Paralel Dağıtık İşleme.*Cambridge, MA: MIT Press, 1986.

Russell, S. ve P. Norvig. *Yapay Zeka: Modern Bir Yaklaşım,* 3. baskı. Upper Saddle River, NJ: Prentice-Hall, 2009.

Shapiro, L. G. ve G. C. Stockman. *Bilgisayarla Görme*. Englewood Cliffs, NJ: Prentice-Hall, 2001.

Shieber, S. *Turing Testi.*Cambridge, MA: MIT Press, 2004.

Weizenbaum, J. *Bilgisayar Gücü ve İnsan Aklı.*New York: W. H. Freemann, 1979.

Doclingo https://www.doclingo.ai

BÖLÜM

Hesaplama Teorisi

12

Bu bölümde bilgisayar biliminin teorik temellerini ele alacağız. Bir anlamda, bu bölümdeki materyal bilgisayar bilimine gerçek bir bilim statüsü kazandırmaktadır. Doğası gereği biraz soyut olmasına rağmen, bu bilgi birikiminin birçok çok pratik uygulaması vardır. Özellikle, programlama dillerinin gücü ile ilgili çıkarımlarını keşfedeceğiz ve bunun, İnternet üzerinden iletişimde yaygın olarak kullanılan bir açık anahtar şifreleme sistemine nasıl öncülük ettiğini göreceğiz.

12.1 Fonksiyonlar ve Onların Hesaplaması

12.2 Turing Makineleri
Turing Makinesi Temelleri
Church–Turing Tezi

12.3 Evrensel Programlama Dilleri
Temel Dil
Temel Dilde Programlama
Temel Dilin Evrenselliği

12.4 Hesaplanamaz Fonksiyon
Durdurma Problemi
Durdurma Probleminin Çözümsüzlüğü

12.5 Karmaşıklık Problemler
Bir Problemin Karmaşıklığını Ölçmek
PolinomikPolinomik Olmayan ProblemlerNP Problemleri

***12.6Açık Anahtar Şifreleme**
Modüler Notasyon
RSA Açık Anahtar Kriptografisi

**Yıldızlar, isteğe bağlı bölümler için önerileri belirtir.*

Doclingo https://www.doclingo.ai

Bu bölümde, bilgisayarların neleri yapabileceği ve neleri yapamayacağı ile ilgili sor uları ele alıyoruz. Turing makineleri olarak bilinen basit makinelerin, makineler tara fından çözülebilen problemler ile çözülemeyen problemler arasındaki sınırı belirlem ek için nasıl kullanıldığını göreceğiz. Algoritmik sistemlerin güçlerinin ötesinde kala n ve dolayısıyla bugünün ve yarının bilgisayarlarının yeteneklerinin ötesinde olan bi r problem olan durma problemi olarak bilinen belirli bir problemi tanımlayacağız. Ayrıca, makine tarafından çözülebilen problemler arasında bile, çözümleri o ka dar karmaşık olan problemler bulacağız ki, bunlar pratik bir bakış açısından çöz ülemez. Karmaşıklık alanındaki bilginin, bir açık anahtar şifreleme sistemi inşa etmek için nasıl kullanılabileceğini düşünerek kapanış yapıyoruz.

12.1 Fonksiyonlar ve Hesaplamaları

Bu bölümdeki amacımız bilgisayarların yeteneklerini araştırmaktır. Makinelerin ne yapabileceğini ve ne yapamayacağını, makinelerin tam potansiyeline ulaşması için hangi özelliklerin gerektiğini anlamak istiyoruz. Hesaplama fonksiyonları kavramıyl a başlıyoruz.

Matematiksel anlamda bir fonksiyon, olası giriş değerleri kümesi ile çıkış değerleri kümesi arasında her olası girişi tek bir çıkışa atayan bir karşılılıktır. Bir örnek, ölçümleri yarddan metreye dönüştüren fonksiyondur. Yarda yapıl an her ölçümle, aynı mesafenin metre cinsinden ölçülmesi durumunda elde edilecek değeri atar.

Bir diğer örnek, sıralama fonksiyonu olarak adlandırabileceğimiz, her giriş sayısal değerler listesini, giriş listesindeki değerlerle aynı olan ancak artan değer sırasına göre düzenlenmiş bir çıkış listesine atayan bir fonksiyondur. Bir başka örnek ise, girdileri değer çiftleri olan ve çıktıları her giriş çiftinin toplamını temsil eden değer ler olan toplama fonksiyonudur.

Bir fonksiyonun belirli bir girdi için atadığı çıkış değerini belirleme sürecine *fonksiy onu hesaplama* denir. Fonksiyonları hesaplama yeteneği önemlidir çünkü problemleri ç özmek için fonksiyonları hesaplamak suretiyle hareket ederiz. Bir toplama problemini ç özmek için toplama fonksiyonunu hesaplamalıyız; bir listeyi sıralamak için sıralama fonk siyonunu hesaplamalıyız. Dolayısıyla, bilgisayar biliminin temel görevlerinden biri, çözm ek istediğimiz problemlerin altında yatan fonksiyonları hesaplamak için teknikler bulma ktır.

Örneğin, bir fonksiyonun girdileri ve çıktılarının önceden belirlenip bir ta bloda kaydedilebildiği bir sistemi düşünün. Fonksiyonun çıktısı her seferinde

Özyinelemeli Fonksiyon Teorisi

İnsan doğasını en çok cezbeden şey, bir şeyin yapılamayacağı söylenmesidir. Araştırmacılar, algoritmik çözümleri olmayan anlamda çözülemeyen problemleri tanımlamaya başladıkça, diğerleri bu problemleri incelemeye ve karmaşıklıklarını anlamaya çalıştılar. Bugün, bu araştırma alanı, özyinelemeli fonksiyon teorisi olarak bilinen alt konu içinde önemli bir yer t utmaktadır ve bu süper zor problemler hakkında çok şey öğrenilmiştir. Gerçekten de, matemat ikçilerin sonsuzluğun ötesinde "niceliksel" seviyeleri ortaya çıkaran sayı sistemleri geliştirdiği gi bi, özyinelemeli fonksiyon teorisyenleri de algoritmaların yeteneklerinin çok ötesinde var olan problemler içinde birden fazla karmaşıklık seviyesini keşfetmişlerdir.

Doclingo https://www.doclingo.ai

gerekli olduğunda, yalnızca gerekli çıktıyı bulduğumuz tabloda verilen girişi ararız. Böylece, fonksiyonun hesaplanması, tablonun içinde arama yapma sü recine indirgenir. Bu tür sistemler kullanışlıdır ancak güç açısından sınırlıdır ç ünkü birçok fonksiyon tamamen tabular biçimde temsil edilemez. Bir örnek, ölçümleri yard cinsinden eşdeğer ölçümlere dönüştüren fonksiyonu göster meye yönelik bir çabadır. Olası girdi/çıktı çiftlerinin listesinde bir sınır olmadı ğı için, tablo tamamlanmamış olmaya mahkumdur.

Daha güçlü bir hesaplama fonksiyonu yaklaşımı, tüm olası girdi/çıktı kombi nasyonlarını bir tabloda göstermeye çalışmak yerine, bir cebirsel formül tarafın dan sağlanan talimatları izlemektir. Örneğin, cebirsel formülü kullanabiliriz

$$V = P(1 + r)^n$$

yatırımın değerini P hesaplamak için, r yıllık bileşik faiz oranı kazandıktan son ra n yıl boyunca.

Ancak cebirsel formüllerin ifade gücünün de sınırlamaları vardır. Girdi/çıktı ilişkileri, cebirsel manipülasyonlarla tanımlanamayacak kadar karmaşık olan fo nksiyonlar vardır. Örnekler arasında sinüs ve kosinüs gibi trigonometrik fonksi yonlar bulunur. 38 derecenin sinüsünü hesaplamaya zorlandığınızda, uygun üç geni çizebilir, kenarlarını ölçebilir ve istenen oranı hesaplayabilirsiniz; bu süreç, 38 değerinin cebirsel manipülasyonları açısından ifade edilemez. Cep hesap m akineniz de 38 derecenin sinüsünü hesaplama görevinde zorlanır. Gerçekte, 38 derecenin sinüsüne çok iyi bir yaklaşım elde etmek için oldukça sofistike mate matik teknikleri uygulamak zorundadır ve bunu size cevap olarak bildirir.

O halde, artan karmaşıklıkta fonksiyonları düşündüğümüzde, onları hesaplama k için daha güçlü teknikler uygulamak zorunda kaldığımızı görüyoruz. Sorumuz, ka rmaşıklıklarına bakılmaksızın her zaman fonksiyonları hesaplamak için bir sistem b ulup bulamayacağımızdır. Cevap hayırdır. Matematikten çarpıcı bir sonuç, o kadar karmaşık fonksiyonlar olduğu ki, girdi değerlerine dayanarak çıktılarının belirlenme si için iyi tanımlanmış, adım adım bir süreç yoktur. Bu fonksiyonların hesaplanması, herhangi bir algoritmik sistemin yeteneklerinin ötesindedir. Bu fonksiyonlar hesap lanamaz olarak adlandırılırken, girdi değerlerinden algoritmik olarak belirlenebilen çıktı değerlerine sahip fonksiyonlar hesaplanabilir olarak adlandırılır.

Hesaplanabilir ve hesaplanamaz fonksiyonlar arasındaki ayrım, bilgisayar bilim inde önemlidir. Çünkü makineler yalnızca tanımlanan görevleri yerine getirebilirler.

Şekil **12.1** Yard cinsinden ölçümleri metreye dönüştüren işlevi göstermeye yönelik bir dene me

Yards (girdi)	Metre (çıktı)
1	0.9144
2	1.8288
3	2.7432
4	3.6576
5	4.5720
.	.
.	.
.	.

Doclingo https://www.doclingo.ai

algoritmalar tarafından, hesaplanabilir fonksiyonların incelenmesi, makinelerin nihai kapasitelerinin incelenmesidir. Eğer bir makinenin tüm hesaplanabilir fonksiyonlar kümesini hesaplamasına olanak tanıyan yetenekleri tanımlayabilirsek ve ardından bu yeteneklere sahip makineler inşa edebilirsek, inşa ettiğimiz makinelerin mümkün olan en güçlü makineler olduğunu garanti altına almış olacağız. Benzer şekilde, bir problemin çözümünün hesaplanamaz bir fonksiyonun hesaplanmasını gerektirdiğini keşfedersek, o problemin çözümünün makinelerin kapasitelerinin ötesinde olduğunu sonucuna varabiliriz.

Sorular & Alıştırmalar

1. Tamamen tablo biçiminde temsil edilebilecek bazı işlevleri tanımlayın.
2. Girişleriyle ilgili bir cebirselifade olarak tanımlanabilen bazı fonksiyonları belirleyin.
3. Bir cebirsel formülle tanımlanamayan bir fonksiyonu tanımlayın. Yine de fonksiyonunuz hesaplanabilir mi?
4. Antik Yunan matematikçileri, şekiller çizmek için bir cetvel ve pergel kullandılar.Düz bir çizgide orta noktayı bulma, dik açı oluşturma ve eşkenar üçgen çizme teknikleri geliştirdiler. Ancak, "hesaplama sistemlerinin" gerçekleştiremeyeceği bazı "hesaplamalar" nelerdi?

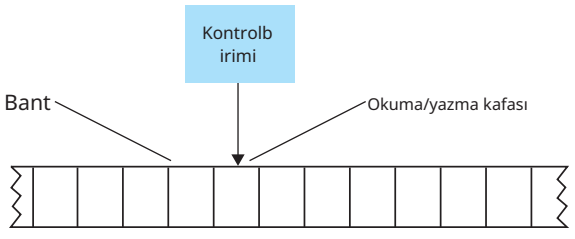
12.2 Turing Makineleri

Makinelerin yeteneklerini ve sınırlamalarını anlamak amacıyla, birçok araştırmacı çeşitli hesaplama cihazlarını önermiş ve incelemiştir. Bunlardan biri, 1936 yılında Alan M. Turing tarafından önerilen Turing makinesidir ve hala algoritmik süreçlerin gücünü incelemek için bir araç olarak kullanılmaktadır.

Turing Makinesi Temelleri

Bir Turing makinesi, bir okuma/yazma kafası aracılığıyla bir bantta sembolleriokuyabilen ve yazabilen bir kontrol biriminden oluşur (Şekil 12.2). Bant, her ikiuçta sonsuz bir şekilde uzanır ve her biri sonlu bir sembol kümesinden herhangibirini içerebilen hücrelere bölünmüştür. Bu küme, makinenin alfabesi olarakadlandırılır.

Şekil **12.2**Bir Turing makinesinin bileşenleri



Doclingo https://www.doclingo.ai

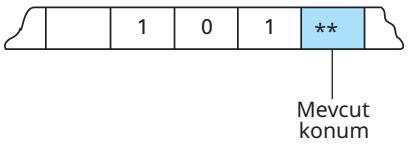
Turing Makinelerinin Kökenleri

Alan Turing, 1930'larda, bugün bildiğimiz makineleri sağlayacak teknolojinin çok öncesinde, bir Turing makinesi kavramını geliştirmiştir. Aslında, Turing'in vizyonu, bir insanın kalem ve kağıtla hesaplamalar yaptığı bir görüntüydü. Turing'in amacı, "hesaplama süreçlerinin" sınırlarının incelenebileceği bir model sağlamaktı. Bu, 1931'de hesaplama sistemlerinin sınırlamalarını ortaya koyan Gödel'in ünlü makalesinin yayınlanmasından kısa bir süre sonraydı ve bu sınırlamaları anlamaya yönelik büyük bir araştırma çabası yönlendiriliyordu. Aynı yıl Turing modelini sunduğunda (1936), Emil Post başka bir model sundu (şimdi Post üretim sistemleri olarak bilinir) ve bu modelin Turing'in aynı yeteneklere sahip olduğu gösterilmiştir. Bu erken araştırmacıların içgörülerinin bir kanıtı olarak, hesaplama sistemlerinin (Turing makineleri ve Post üretim sistemleri gibi) modelleri hala bilgisayar bilimi araştırmalarında değerli araçlar olarak hizmet etmektedir.

Bir Turing makinesinin hesaplaması sırasında, makine sonlu sayıdakoşuldan birinde olmalıdır; bu koşullara durum denir. Bir Turing makinesininhesaplaması, başlangıç durumu olarak adlandırılan özel bir durumda başlar vemakine, durma durumu olarak bilinen başka bir özel duruma ulaştığında sona erer.

Bir Turing makinesinin hesaplaması, makinenin kontrol birimi tarafındanuygulanan bir dizi adımdan oluşur. Her adım, mevcut bant hücresindeki sembolü(okuma-yazma kafası tarafından görülen) gözlemlemek, o hücreye bir sembolyazmak, okuma-yazma kafasını muhtemelen bir hücre sola veya sağa hareketettirmek ve ardından durum değiştirmekten oluşur. Gerçekleştirilecek eylem,makinenin durumu ve mevcut bant hücresinin içeriği temelinde kontrol biriminene yapması gerektiğini söyleyen bir program tarafından belirlenir.

Belirli bir Turing makinesinin bir örneğini ele alalım. Bu amaçla,makinenin şeridini, makinenin alfabesinden semboller kaydedebileceğimiz hücrelere bölünmüş yatay bir şerit olarak temsil ediyoruz. Makinenin okuma/yazma başlığının mevcut konumunu, mevcut şerit hücresinin altına bir etiket koyarak gösteriyoruz. Örneğimizin alfabesi 0, 1 ve * sembollerinden oluşmaktadır. Makinemizin şeridi aşağıdaki gibi görünebilir:

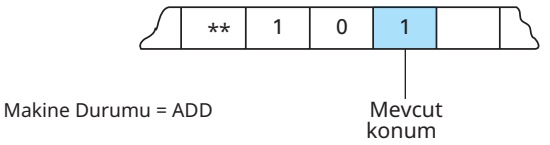


Şeritteki semboller dizisini, yıldızlarla ayrılmış ikili sayıları temsil edecek şekilde yorumlayarak, bu belirli şeridin 5 değerini içerdiğini kabul ediyoruz. Turing makinemiz, şeritteki böyle bir değeri 1 artırmak üzere tasarlanmıştır. Daha kesin olarak, başlangıç konumunun 0 ve 1'lerin bir dizisinin sağ ucunu işaret eden bir yıldızda olduğunu varsayıyor ve bit desenini sola doğru değiştirerek bir sonraki daha büyük tam sayıyı temsil edecek şekilde ilerliyor.

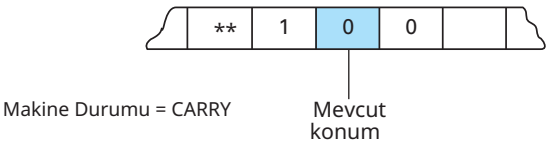
Doclingo https://www.doclingo.ai

Makinemiz için durumlar START, ADD, CARRY, OVERFLOW, RETURN, ve HALT' dır. Bu durumların her birine karşılık gelen eylemler ve mevcut hücrenin içeri ği Şekil 12.3'teki tabloda açıklanmaktadır. Makinenin her zaman START duru munda başladığını varsayıyoruz.

Bu makineyi daha önce resmedilen ve 5 değerini içeren şeride uygulayalı m. Mevcut hücre * içerdiğinde START durumundayken, tablo tarafından *'yı yeniden yazmamız, okuma/yazma başlığını bir hücre sola hareket ettirmemiz ve ADD durumuna girmemiz talimatı verilmektedir. Bunu yaptıktan sonra, m akine şu şekilde tanımlanabilir:



Devam etmek için, mevcut hücre 1 içerdiğinde ADD durumundayken ne yapmamız gerektiğini görmek için tabloya bakıyoruz. Tablo, mevcut hücredeki 1'i 0 ile değiştirmemiz, okuma/yazma baş lığını bir hücre sola hareket ettirmemiz ve CARRY durumuna girmemiz gerektiğini söylüyor. Böylece makinenin konfigürasyonu şu hale gelir:



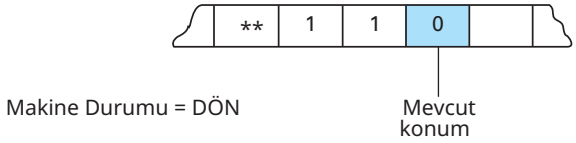
Bir sonraki adımı görmek için tekrar tabloya başvuruyoruz ve CARRY durumunda mevcut hücre 0 içeriyorsa, 0'ı 1 ile değiştirmemiz gerektiğini buluyoruz,

Şekil **12.3** Bir değeri artırmak için bir Turing makinesi

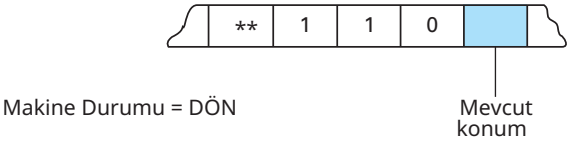
Mevcut durum	Mevcut hücre içerik	Yazılacak değer	Hareket yönü	Girilmesi gereken Yeni durum
BAŞLA	*01*01*(Göz ardı edildi)01*	*	Sol	EKLE
EKLE		1	Sağ	DÖN
EKLE		0	Sol	TAŞI
EKLE		*	Sağ	DUR
TAŞI		1	Sağ	DÖN
TAŞI		0	Sol	TAŞI
TAŞI		1	Sol	TAŞMA
TAŞMA		*	SağS	DÖN
DÖN		0	ağSa	DÖN
DÖN		1	ğHar	DÖN
DÖN		*	eket yok	DUR

Doclingo https://www.doclingo.ai

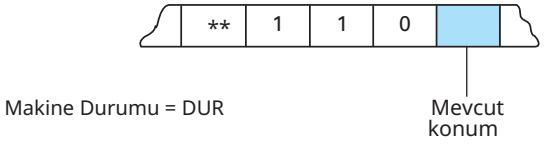
okuma/yazma kafasını bir hücre sağa hareket ettirin ve RETURN durumuna girin. Bunu yaptıktan sonra, makinenin konfigürasyonu aşağıdaki gibidir:



Bu durumdan, tablo bize mevcut hücredeki 0'ı başka bir 0 ile değiştirmemizi, okuma/yazma başlığını bir hücre sağa hareket ettirmemizi ve DÖN durumunda kalmamızı talimatlandırıyor. Sonuç olarak, makinemizi aşağıdaki durumda buluyoruz:



Bu noktada, tablonun mevcut hücredeki yıldızı yeniden yazmamızı ve DUR durdurmamızı söylediğini görüyoruz. Makine, aşağıdaki yapılandırmada durur (şu anda banttaki semboller istenen değer 6'yı temsil eder):



Church-Turing Tezi

Önceki örnekteki Turing makinesi, her bir negatif olmayan tam sayı girdi değerini n çıktısı $n+$ 1'e atayan ardıl fonksiyon olarak bilinen fonksiyonu hesaplamak için kullanılabilir. Tek yapmamız gereken, girdi değerini makinenin bantında ikili formda yerleştirmek, makineyi durana kadar çalıştırmak ve ardından banttan çıktı değerini okumaktır. Bu şekilde bir Turing makinesi tarafından hesaplanabilen bir fonksiyona Turing hesaplanabilir denir.

Turing'in varsayımı, Turing hesaplanabilir fonksiyonların hesaplanabilir fonksiyonlarla aynı olduğuydu. Diğer bir deyişle, Turing makinelerinin hesaplama gücünün herhangi bir algoritmik sistemin gücünü kapsadığını veya eşdeğer olarak, (tablolar ve cebirsel formüller gibi yaklaşımlara karşı) Turing makinesi kavramının, tüm hesaplanabilir fonksiyonların çözümlerinin ifade edilebileceği bir bağlam sağladığını varsaydı. Bugün, bu varsayım genellikle Church–Turing tezi, hem Alan Turing'in hem de Alonzo Church'ün katkılarına atıfta bulunur. Turing'in ilk çalışmalarından bu yana, bu tezi destekleyen çok sayıda kanıt toplanmıştır ve bugün Church–Turing tezi yaygın olarak kabul edilmektedir. Yani, hesaplanabilir fonksiyonlar ve Turing-hesaplanabilir fonksiyonlar aynı olarak kabul edilmektedir.

Bu varsayımın önemi, hesaplama makinelerinin yetenekleri ve sınırlamaları hakkında içgörü sağlamasıdır. Daha kesin olarak, Turing makinelerinin yeteneklerini, diğer hesaplama sistemlerinin güçlerinin karşılaştırılabileceği bir standart olarak belirler. Eğer bir hesaplama sistemi, tüm Turing-hesaplanabilir fonksiyonları hesaplayabiliyorsa, o sistem, herhangi bir hesaplama sisteminin sahip olabileceği kadar güçlü olarak kabul edilir.

Doclingo https://www.doclingo.ai

Sorular & Alıştırmalar

1. Bu bölümde tanımlanan Turing makinesini (Şekil 12.3) uygulayın, aşağıda ki başlangıç durumu ile başlayarak:



2. 0 ve 1'lerden oluşan bir diziyi tek bir 0 ile değiştiren bir Turing makinesi tanımlayın.
3. Şeritteki değeri sıfırdan büyükse azaltan veya sıfırsa değeri değiştirm eyen bir Turing makinesi tanımlayın.
4. Hesaplamanın gerçekleştiği günlük bir durumu tanımlayın. O durumun bir Turi ng makinesine nasıl benzer olduğunu açıklayın?
5. Bazı girdiler için nihayetinde duraklayan ancak diğerleri için asla durmayan bir Turin g makinesini tanımlayın.

12.3 Evrensel Programlama Dilleri

6. Bölümde, yüksek seviyeli programlama dillerinde bulunan çeşitli özellikleri in celedik. Bu bölümde, hesaplanabilirlik bilgimizi uygulayarak bu özelliklerden ha ngilerinin gerçekten gerekli olduğunu belirleyeceğiz. Bugünün yüksek seviyeli dillerindeki çoğu özelliğin, dilin temel gücüne katkıda bulunmaktan çok, yalnızc a kolaylığı artırdığını göreceğiz.

Yaklaşımımız, tüm Turing-hesaplanabilir fonksiyonları (ve dolayısıyla tüm hes aplanabilir fonksiyonları) hesaplamak için programları ifade etmemize olanak tanı yan yeterince zengin basit bir zorlayıcı programlama dilini tanımlamaktır. Dolayısı yla, gelecekte bir programcı bu dil kullanılarak bir problemin çözülemeyeceğini b ulursa, bunun nedeni dilin bir hatası olmayacaktır. Bunun yerine, problemin çözü mü için bir algoritmanın olmamasıdır. Bu özelliğe sahip bir programlama diline e vrensel programlama dili denir.

Evrensel bir dilin karmaşık olması gerekmediğini şaşırtıcı bulabilirsiniz. Gerçekten de, sunmak üzere olduğumuz dil oldukça basit. Buna, evrensel bir programlama dilinin minimal gereksinim setini izole ettiği için Bare Bones a dını vereceğiz.

Temel Dil

Sunumumuza Bare Bones ile başlayalım ve diğer programlama dillerinde bul unan deklaratif durum- ifadelerini göz önünde bulunduralım. Bu ifadeler, pr ogramcıların veri yapıları ve veri türleri (sayısal değerlerin dizileri ve alfabetik karakterlerin dizeleri gibi) açısından düşünme lüksünü sağlar, oysa makine kendisi yalnızca bit desenlerini manipüle eder ve bu desenlerin neyi temsil et tiğini bilmez. Bir makineye yürütme için sunulmadan önce, karmaşık veri türl eri ve yapılarıyla ilgili yüksek seviyeli bir talimatın

Doclingo https://www.doclingo.ai

bit desenlerini manipüle eden makine düzeyindeki talimatlara çevrilmiştir, istenen e ylemleri simüle etmek için.

Kolaylık açısından, bu kalıpları ikili notasyonda temsil edilen sayısal değerler o larak yorumlayabiliriz. Böylece, bir bilgisayar tarafından gerçekleştirilen tüm hesa plamalar, negatif olmayan tam sayıları içeren sayısal hesaplamalar olarak ifade ed ilebilir - hepsi bakış açısına bağlıdır. Dahası, programlama dilleri, programcıların a lgoritmaları bu terimlerle ifade etmelerini gerektirerek basitleştirilebilir (bu, progr amcı üzerinde daha büyük bir yük oluşturacaktır).

Bare Bones'u geliştirirken amacımız mümkün olan en basit dili geliştirmek old uğundan, bu yönde ilerleyeceğiz. Bare Bones'daki tüm değişkenler, kolaylık olmas ı açısından, negatif olmayan tam sayılar olarak yorumladığımız bit desenlerini tem sil edecek şekilde kabul edilecektir. Bu nedenle, şu anda 10 desenine atanmış bir değişkenin iki değerini içerdiği, 101 desenine atanmış bir değişkenin ise beş değe rini içerdiği söylenecektir.

Bu kuralı kullanarak, Bare Bones programındaki tüm değişkenler aynıtipte olduğun dan, dilin değişken adlarının ve bunlarla ilişkili özelliklerininbelirtildiği deklaratif ifadeler e ihtiyacı yoktur. Bare Bones kullanırken, bir programcıbir değişken adı kullanmaya ihti yaç duyduğunda, bunun negatif olmayan bir tam sayıolarak yorumlanan bir bit desenin e atıfta bulunduğunu anlayarak yeni bir değişken adıylabaşlayabilir.

Elbette, Bare Bones dilimiz için bir çevirmenin değişken adlarını diğer terimlerde nayırabilmesi gerekir. Bu, Bare Bones'un sözdizimini tasarlayarak, herhangi bir terim inrolünün yalnızca sözdizimi ile tanımlanabilmesini sağlamakla yapılır. Bu amaçla,de ğişken adlarının İngiliz alfabesinden bir harfle başlaması gerektiğini ve ardındanher hangi bir harf ve rakam (0'dan 9'a kadar) kombinasyonu ile devam edebileceğinibelir liyoruz. Böylece `XYZ`,`B747`,`abcdefghi` ve `X5Y` gibi dizelerdeğişken adı olarak kullanıl abilirken, `2G5`,`%o` ve `x.y` kullanılamaz.

Şimdi Bare Bones'daki prosedürel ifadeleri ele alalım. Üç atama ifadesi v e birkontrol yapısı olan bir döngü vardır. Dil serbest formatta bir dildir, bu ne denle herifade bir noktalı virgül ile sona erer, bu da bir çevirmenin aynı satır da görünenifadeleri ayırmasını kolaylaştırır. Ancak, okunabilirliği artırmak içi n her satırasadece bir ifade yazma politikasını benimseyeceğiz.

Her üç atama ifadesi, ifadedeki değişkenin içeriğinin değiştirilmesini tale p eder. İlk ifade, sıfır değerini bir değişkenle ilişkilendirmemize olanak tanır. Söz dizimi

```
temiz isim;
```

burada `name` herhangi bir değişken adı olabilir.

Diğer atama ifadeleri esasen birbirinin zıttıdır:

```
incrname;
```

ve

```
dekrname;
```

Yine, `name` herhangi bir değişken adını temsil eder. Bu ifadelerden ilki, tanım lanan değişkenle ilişkilendirilmiş değerin bir artırılmasına neden olur. Böylec e, eğer `Y` değişkenine

```
Y dahil;
```

ifadesinden önce beş değeri atanmışsa, o zaman ifadenin yürütülmesinden sonra `Y`'ye atanan değer altı olacaktır.

Doclingo https://www.doclingo.ai

Tam tersine, `decr` ifadesi, tanımlanan değişkenle ilişkilendirilmiş değeri bir azaltmak için kullanılır. Bir istisna, değişkenin zaten sıfırla ilişkilendirilmiş olması durumudur; bu durumda bu ifade değeri değiştirmeden bırakır. Bu nedenle, `Y` ile ilişkili değer beyan öncesinde beştir

```
decr Y;
```

uygulanırsa, dört değeri `Y` ile ilişkilendirilecektir. Ancak, eğer `Y`'nin değeri ifa denin yürütülmesinden önce sıfır olsaydı, değer yürütmeden sonra sıfır olar ak kalacaktır.

Bare Bones yalnızca bir kontrol yapısı sağlar ve bu, bir `while-end` ifade çiftini temsil eder. İfade dizisi

```
while namenot 0 bunu yapar;
  ...
son;
```

(burada `name` herhangi bir değişken adını temsil eder) `while` ve`end` ifadeleri ar asına yerleştirilen herhangi bir ifade veya ifade dizisinin, `name` değişkeninin de ğeri sıfır olmadığı sürece tekrar edilmesine neden olur. Daha kesin olmak gere kirse, program yürütülmesi sırasında bir while-endyapısı ile karşılaşıldığında, t anımlanan değişkenin değeri önce sıfır ile karşılaştırılır. Eğer sıfır ise, yapı atlan ır ve yürütme `end` ifadesinden sonraki ifadeye devam eder. Ancak, değişkenin değeri sıfır değilse, `while-end` yapısı içindeki ifade dizisi yürütülür ve kontrol `while` ifadesine geri döner, ardından karşılaştırma tekrar yapılır. Döngü kontro l yükünün kısmen programcıya yüklendiğini unutmayın; programcı, sonsuz dö ngüden kaçınmak için döngü gövdesi içinde değişkenin değerinin değiştirilmes ini açıkça talep etmelidir. Örneğin,

```
X artı;
X 0 değilken;
   incr Z;
son;
```

sonsuz bir süreçle sonuçlanır çünkü while ifadesine ulaşıldığında, `X` ile ilişkili değer asla sıfır olamaz; oysa

```
Z'yi temizle;
X 0 değilken;
   incr Z;
   decr X;
end;
```

sonuçta, başlangıçta `X` ile ilişkili değeri `Z` değişkenine aktarma etkisiyle sona erer.

Gözlemleyin ki `while` ve`end` ifadeleri, while ifadesinin önce geldiği çiftler halinde görünmelidir. Ancak, bir `while-end` ifade çifti, başka bir `while-end` çifti tarafından tekrarlanan talimatlar içinde görünebilir. Böyle bi r durumda, `while` ve`end` ifadelerinin eşleştirilmesi, programın yazılı biçimin de baştan sona taranarak, her `end` ifadesinin henüz eşleştirilmemiş en yakın önceki `while` ifadesi ile ilişkilendirilmesiyle gerçekleştirilir.

Sözdizimsel olarak gerekli olmamakla birlikte, bu tür yapıların okunabilirliğini artırmak i çin genellikle girinti kullanırız.

Doclingo https://www.doclingo.ai

Kapanış örneği olarak, Şekil 12.4'teki talimat dizisi, X veY ile ilişkili değerleri n çarpımının Z'ye atanmasıyla sonuçlanır, ancak bu, X ile ilişkili olabilecek herh angi bir sıfırdan farklı değerin yok edilmesi yan etkisine sahiptir. (W değişkeni t arafından kontrol edilen while-end yapısı, Y'nin orijinal değerinin geri yüklenm esi etkisine sahiptir.)

Temel Programlama

Temel Programlama dilini sunmadaki amacımızın pratik olanı değil, mümkün o lanı araştırmak olduğunu unutmayın. Temel Programlama, uygulamalı bir orta mda kullanıldığında garip bir hale gelecektir. Öte yandan, bu basit dilin, sade bi r evrensel programlama dili sağlama amacımızı yerine getirdiğini yakında göre ceğiz. Şimdilik, Temel Programlama'nın bazı temel işlemleri ifade etmek için na sıl kullanılabileceğini göstereceğiz.

Öncelikle, atama ifadelerinin bir kombinasyonu ile herhangi bir değerin (he rhangi bir negatif olmayan tam sayının) belirli bir değişkenle ilişkilendirilebilece ğini not ediyoruz. Örneğin, aşağıdaki dizi, önce değişkene sıfır değerini atayara k ve ardından değerini üç kez artırarak değişken X'ye üç değerini atar:

```
temizle X;
arttır X;
arttır X;
arttır X;
```

Programlardaki bir diğer yaygın etkinlik, verileri bir yerden başka bir yer e kopyalamaktır. Bare Bones açısından bu, bir değişkenin değerini başka bir değişkene atayabilmemiz gerektiği anlamına gelir. Bu, önce hedefi temizleye rek ve ardından uygun sayıda artırarak gerçekleştirilebilir. Aslında, zaten dizi nin

```
Z'yi temizle;
X 0 değilken;
   incr Z;
   decr X;
end;
```

X ile ilişkili değeri Z'ye aktarır. Ancak, bu dizinin yan etkisi, X'in orijinal değeri ni yok etmektir. Bunu düzeltmek için, önce konunun değerini başlangıçtan bi r yardımcı değişkene aktarabiliriz.

Şekil **12.4**X× Y hesaplamak için bir Bare Bones programı

```
Z'yi temizle;
X 0 olmadığı sürece devam et;
        W'yi temizle;
         Y 0 olmadığı sürece;
                Z'yi artır;
             artış W;
              azalış Y;
        bitiş;
         W 0 olmadığı sürece;
                Y'yi artır;
                decr W;
         end;
         decr X;
end;
```

Doclingo https://www.doclingo.ai

konum. Daha sonra bu yardımcı değişkeni, konuyu istenen hedefe yerleştirirken orijinal değişkeni geri yüklemek için veri kaynağı olarak kullanıyoruz.Bu şekilde, Bugün 'den Dün 'e hareket, Şekil 12.5'te gösterilen sırayla gerçekleştirilebilir.

Sözdizimini benimsiyoruz

isim1'i isim2'ye kopyala;

(burada `name1` ve`name2` değişken adlarını temsil eder) Şekil 12.5'teki yapı biçimi için bir kısayol notasyonu olarak. Böylece, Bare Boneskendisi açık bir kopyalama talimatına sahip olmasa da, genellikle programları sanki varmış gibi yazarız; bu, böyle gayri resmi programları gerçek Bare Bones programlarına dönüştürmek için, `copy` ifadelerini, programda başka bir yerde kullanılan bir adla çakışmayan bir yardımcı değişkenin adıyla `while-end` yapılarıyla değiştirmemiz gerektiği anlayışıyla yapılır.

Bare Bones'un Evrenselliği

Şimdi, Bare Bones'un evrensel bir programlama dili olduğunu doğrulamak için Church-Turing teoremini uygulayalım. Öncelikle, Bare Bones'da yazılmış herhangi bir programın bir fonksiyonun hesaplamasını yönlendirdiği düşünülebilir. Fonksiyonun girişi, programı çalıştırmadan önce değişkenlere atanan değerlerden oluşur ve fonksiyonun çıktısı, program sona erdiğinde değişkenlerin değerlerinden oluşur.
Fonksiyonu hesaplamak için, programı doğru değişken atamaları ile başlatarak yürütüyoruz ve ardından program sona erdiğinde değişkenlerin değerlerini gözlemliyoruz.

Bu koşullar altında program

```
incr X;
```

aynı fonksiyonun (ardıl fonksiyon) hesaplanmasını yönlendirir; bu, Bölüm 12.2'deki Turing makinesi örneği tarafından hesaplanır. Gerçekten de, `X` ile ilişkili değeri bir artırır. Benzer şekilde, değişkenleri `X` ve
`Y` girdi olarak ve değişkeni `Z` çıktı olarak yorumlarsak, program

```
copy Y to Z;
X 0 değilken;
 incr Z;
 decr X;
end;
```

toplama fonksiyonunun hesaplanmasını yönlendirir.

Şekil **12.5**"Bugünü Yarın'a Kopyala" talimatının Temel Uygulaması

```
Aux'u temizle;
Yarını temizle;
Bugün 0 olmasa da;
      Aux'u artır;
       decr Today;
end;
Aux 0 değilken;
      Bugünü artır;
      Yarını artır;
      Aux'u azalt;
son;
```

Doclingo https://www.doclingo.ai

Araştırmacılar, Bare Bones programlama dilinin tüm Turing hesaplanabilir fonksiyonları hesaplamak için algoritmalar ifade etmekte kullanılabileceğini göstermiştir. Bu durumu Church-Turing tezi ile birleştirmek, herhangi bir hesaplanabilir fonksiyonun Bare Bones dilinde yazılmış bir program tarafından hesaplanabileceği anlamına gelir. Böylece, Bare Bones, bir algoritma bir problemi çözmek için mevcutsa, o problemin bazı Bare Bones programları tarafından çözülebileceği anlamında evrensel bir programlama dilidir. Dolayısıyla, teorik olarak Bare Bones genel amaçlı bir programlama dili olarak hizmet edebilir.

Teorik olarak diyoruz çünkü böyle bir dil kesinlikle Bölüm 6'da tanıtılan yüksek seviyeli diller kadar kullanışlı değildir. Ancak, bu dillerin her biri esasen Bare Bones'un temel özelliklerini içerir. Aslında, bu temel, bu dillerin evrenselliğini sağlar; diğer tüm özellikler ise çeşitli dillerde kolaylık sağlamak için eklenmiştir.

Uygulama programlama ortamında pratik olmasa da, Bare Bones gibi diller teorik bilgisayar biliminde kullanılmaktadır. Örneğin, Ek E'de, Bölüm 5'te gündeme getirilen yinelemeli ve özyinelemeli yapıların eşdeğerliliği ile ilgili sorunu çözmek için Bare Bones'u bir araç olarak kullanıyoruz.Orada, eşdeğerlilik konusundaki şüphemizin aslında haklı olduğunu bu luyoruz.

Sorular & Alıştırmalar

1. İfadenin gösterilmesi X'i ters çevir; (hareketi, değer `x` başlangıç değeri sıfırsa 1'e ve başlangıç değeri sıfırdan farklıysa sıfıra ayarlanabilir) bir Bare Bones program segmenti ile simüle edilebilir.
2. Basit Bare Bones dilimizin, clear ifadesinin dildeki diğer ifadelerin kombinasyonlarıyla değiştirilebileceğini göstererek, gerekli olandan daha fazla durum ifadesi içerdiğini gösterin.
3. `if-then-else` yapısının BareBones kullanılarak simüle edilebileceğini gösterin. Yani, ifadenin eylemini simüle eden Bare Bones'da bir program dizisi yazın.

   eğer X 0 değilse S1, aksi takdirde S2;

   S1 ve S2, keyfi ifade dizilerini temsil eder.
4. Her bir Bare Bones ifadesinin, Ek C'nin makine dili cinsinden ifade edilebileceğini gösterin.(Böylece Bare Bones, böyle bir makine için bir programlama dili olarak kullanılabilir.)
5. Negatif sayılar Bare Bones'ta nasıl ele alınabilir?
6. Aşağıdaki Bare Bones programı tarafından hesaplanan fonksiyonu tanımlayın, fonsiyonun girdi olarak `X` ile temsil edildiğini ve çıktısının `Z` ile temsil edildiğini varsayarak:

```
Z'yi temizle;
X 0 değilken;
   incr Z;
   incr Z;
   decr X;
end;
```

Doclingo https://www.doclingo.ai

12.4 Hesaplanamaz Bir Fonksiyon

Artık Turing hesaplanamaz bir fonksiyonu tanımlıyoruz ve bu nedenle, Church-Turing tezi gereği, genel anlamda hesaplanamaz olduğu geniş çapta kabul edilmektedir. Dolayısıyla, bu, hesaplanması bilgisayarların yeteneklerinin ötesinde olan bir fonksiyondur.

Durdurma Problemi

Açıklayacağımız hesaplanamaz fonksiyon, durdurma problemi olarak biline n bir problemle ilişkilidir,ki bu (gayri resmi bir anlamda) bir programın belirli koşullar altında başlatıldığındasonlanıp sonlanmayacağını önceden tahmin e tmeye çalışmakla ilgilidir. Örneğin, basit bir Bare Bones programını düşünün

```
:while X 0 değilse do;
   artı X;
end;
```

Bu programı X'in başlangıç değeri sıfır olduğunda çalıştırırsak, döngü çalıştırıl mayacak ve programın çalışması hızla sona erecektir. Ancak, programı X'in baş ka bir başlangıç değeri ile çalıştırırsak, döngü sonsuza kadar çalıştırılacak ve bu da sonlanmayan bir sürece yol açacaktır.

Bu durumda, programın yürütülmesinin yalnızca X sıfır değerine atandığında duracağını sonuçlandırmak kolaydır. Ancak, daha karmaşık örneklere geçtikçe, bir programın davranışını tahmin etme görevi daha karmaşık hale gelir. Aslında, bazı durumlarda bu görev imkansızdır, bunu göreceğiz. Ama önce terminolojimizi res mileştirmemiz ve düşüncelerimizi daha kesin bir şekilde odaklamamız gerekiyor.

Örneğimiz, bir programın nihayetinde durup durmayacağının, değişkenleri nin başlangıç değerlerine bağlı olabileceğini göstermiştir. Dolayısıyla, bir progr amın yürütmesinin durup durmayacağını tahmin etmeyi umuyorsak, bu başlan gıç değerleri konusunda kesin olmalıyız. Bu değerler için yapacağımız seçim, ilk başta size garip gelebilir, ama umutsuzluğa kapılmayın. Amacımız, kendine atı fta bulunan bir nesne fikri olan kendine referans adı verilen bir teknikten yarar lanmaktır. Bu hile, "Bu ifade yanlıştır" gibi gayri resmi meraklardan "Tüm kümel erin kümesi kendisini içerir mi?" sorusuyla temsil edilen daha ciddi bir paradok sa kadar matematikte şaşırtıcı sonuçlara yol açmıştır. O halde, yapacağımız şey, "Eğer içeriyorsa, o zaman içermez; ama, eğer içermiyorsa, o zaman içer" benze ri bir akıl yürütme için sahneyi kurmaktır.

Bizim durumumuzda, kendine referans verme, bir programdaki değişkenlere progr amın kendisini temsil eden bir başlangıç değeri atayarak gerçekleştirilecektir. Bu amaçl a, her Bare Bones programının ASCII kullanarak bir karakter başına bir bayt formatında tek bir uzun bit deseni olarak kodlanabileceğini gözlemleyin; bu da daha sonra (biraz b üyük) pozitif bir tam sayının ikili temsili olarak yorumlanabilir. Bu tam sayı değeri, progr amdaki değişkenler için başlangıç değeri olarak atadığımız değerdir.

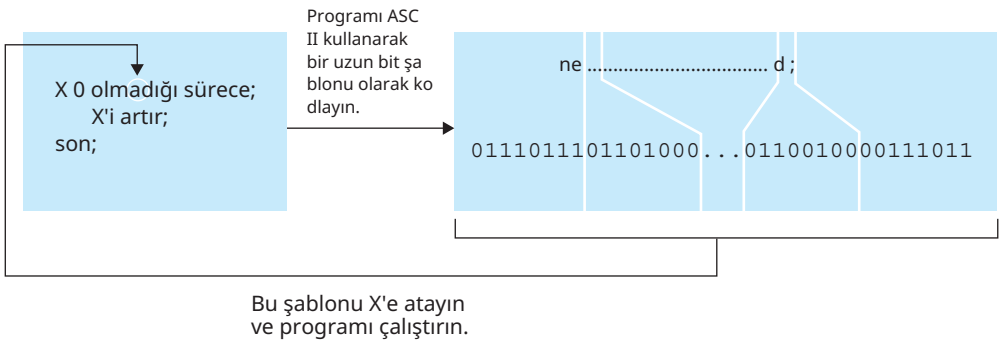
Basit program durumunda bunu yaparsak ne olacağını düşünelim.

```
X 0 değilken;
   incr X;
son;
```

Bu programı, programın kendisini temsil eden tam sayı değeri ile X atayarak başlatırsak ne olacağını bilmek istiyoruz (Şekil 12.6). Bu durumda cevap hem en açıktır. Çünkü X sıfırdan farklı bir değere sahip olacağından, program

Doclingo https://www.doclingo.ai

Şekil **12.6**Kendiliğinden sonlanma için bir programı test etme



döngüde takılı kalacak ve asla sonlanmayacaktır. Öte yandan, programla benzer bir deney yaparsak

```
X'i temizle;
X 0 değilken;
  incr X;
son;
```

program sonlanacaktır çünkü `X` değişkeni, başlangıç değeri ne olursa olsun, `while-end` yapısına ulaşıldığında sıfır değerine sahip olacaktır.

O halde, aşağıdaki tanımı yapalım: Bir Bare Bones programı, tüm değişkenleri programın kendi kodlanmış temsiline başlatıldığında sonlanan bir süreçle sonuçlanıyorsa kendiliğinden sonlanan bir programdır. Gayri resmi olarak, bir program kendisi girdi olarak başlatıldığında yürütmesi sonlanıyorsa kendiliğinden sonlanan bir programdır. İşte, vaat ettiğimiz kendine referans burada.

Bir programın kendiliğinden sona erip ermediğinin muhtemelen programın yazılış amacıyla hiçbir ilgisi yoktur. Bu, her Bare Bones programının ya sahip olduğu ya da sahip olmadığı bir özelliktir. Yani, her Bare Bones programı ya kendiliğinden sona eren ya da sona ermeyen bir programdır.

Artık durma problemini kesin bir şekilde tanımlayabiliriz. Bu, Bare Bones programlarının kendiliğinden sona erip ermediğini belirleme problemidir. Bu sorunun genel olarak yanıtlanması için bir algoritmanın olmadığını göreceğiz. Yani, herhangi bir Bare Bones programı verildiğinde, o programın kendiliğinden sona erip ermediğini belirleyebilen tek bir algoritma yoktur. Dolayısıyla durma probleminin çözümü bilgisayarların yeteneklerinin ötesindedir.

Önceki örneklerimizde durma problemini çözmüş olduğumuz ve şimdi durma probleminin çözülemez olduğunu iddia ettiğimiz gerçeği çelişkili görünebilir, bu yüzden açıklama için duralım. Örneklerimizde kullandığımız gözlemler, o belirli durumlara özgüydü ve tüm durumlarda uygulanabilir değildi. Durma probleminin talep ettiği, herhangi bir Bare Bones programına uygulanabilecek tek bir genel algoritmadır. Belirli bir programın kendiliğinden sona erip ermediğini belirlemek için bazı izole içgörüleri uygulama yeteneğimiz, tüm durumlarda uygulanabilecek tek bir genel yaklaşımın varlığını hiçbir şekilde ima etmez. Kısacası, biz

Doclingo https://www.doclingo.ai

belirli bir durma problemini çözebilen bir makine inşa edebilir,bir durma pro blemini çözmek için kullanabileceğimiz tek bir makine inşa edemeyiz.

Durma Probleminin Çözülemezliği

Şimdi durma probleminin makinelerin yeteneklerinin ötesinde olduğunu gös termek istiyoruz. Amacımız, bu problemi çözmenin, hesaplanamaz bir fonksi yonu hesaplamak için bir algoritma gerektireceğini göstermektir. Söz konus u fonksiyonun girdileri, Bare Bones programlarının kodlanmış versiyonlarıdı r; çıktıları ise 0 ve 1 değerleriyle sınırlıdır. Daha kesin olarak, bir kendiliğinde n sonlanan programı temsil eden bir girdi, 1 çıkış değeri üretirken, kendiliğin den sonlanmayan bir programı temsil eden bir girdi 0 çıkış değeri üretir. Kısalık adına, bu fonksiyona *durma fonksiyonu* olarak atıfta bulunacağız.

Görevimiz durma fonksiyonunun hesaplanamaz olduğunu göstermektir. Y aklaşımımız "çelişki ile kanıt" olarak bilinen tekniktir. Kısacası, bir ifadenin yanlış olduğunu, onun doğru olamayacağını göstererek kanıtlarız. O halde, "durma f onksiyonu hesaplanabilir" ifadesinin doğru olamayacağını gösterelim. Tüm arg ümanımız Şekil 12.7'de özetlenmiştir.

Eğer durdurma fonksiyonu hesaplanabilir ise, (çünkü Bare Bones evrensel bir programlama dilidir) onu hesaplayan bir Bare Bones programı olmalıdır. Başka bir deyişle, eğer girişi kendiliğinden sonlanan bir programın kodlanmı ş versiyonu ise, çıktısı 1 olan ve aksi takdirde çıktısı 0 olan bir Bare Bones pro gramı vardır.

Bu programı uygulamak için hangi değişkenin giriş değişkeni olduğunu t anımlamamız gerekmiyor, bunun yerine yalnızca programın tüm değişkenle rini test edilecek programın kodlanmış temsiline başlatmamız yeterlidir. Çün kü bir giriş değişkeni olmayan bir değişken, başlangıç değeri nihai çıktı değe rini etkilemeyen bir değişkendir. Eğer durdurma fonksiyonu hesaplanabilir is e, o zaman tüm değişkenleri kendiliğinden sonlanan programın kodlanmış v ersiyonuna başlatıldığında çıktısı 1 olan ve aksi takdirde çıktısı 0 olan bir Bar e Bones programı vardır sonucuna varıyoruz.

Programın çıktı değişkeninin adı `X` ise (değilse değişkenleri basitçe yenid en adlandırabiliriz), programı aşağıdaki ifadeleri ekleyerek değiştirebiliriz

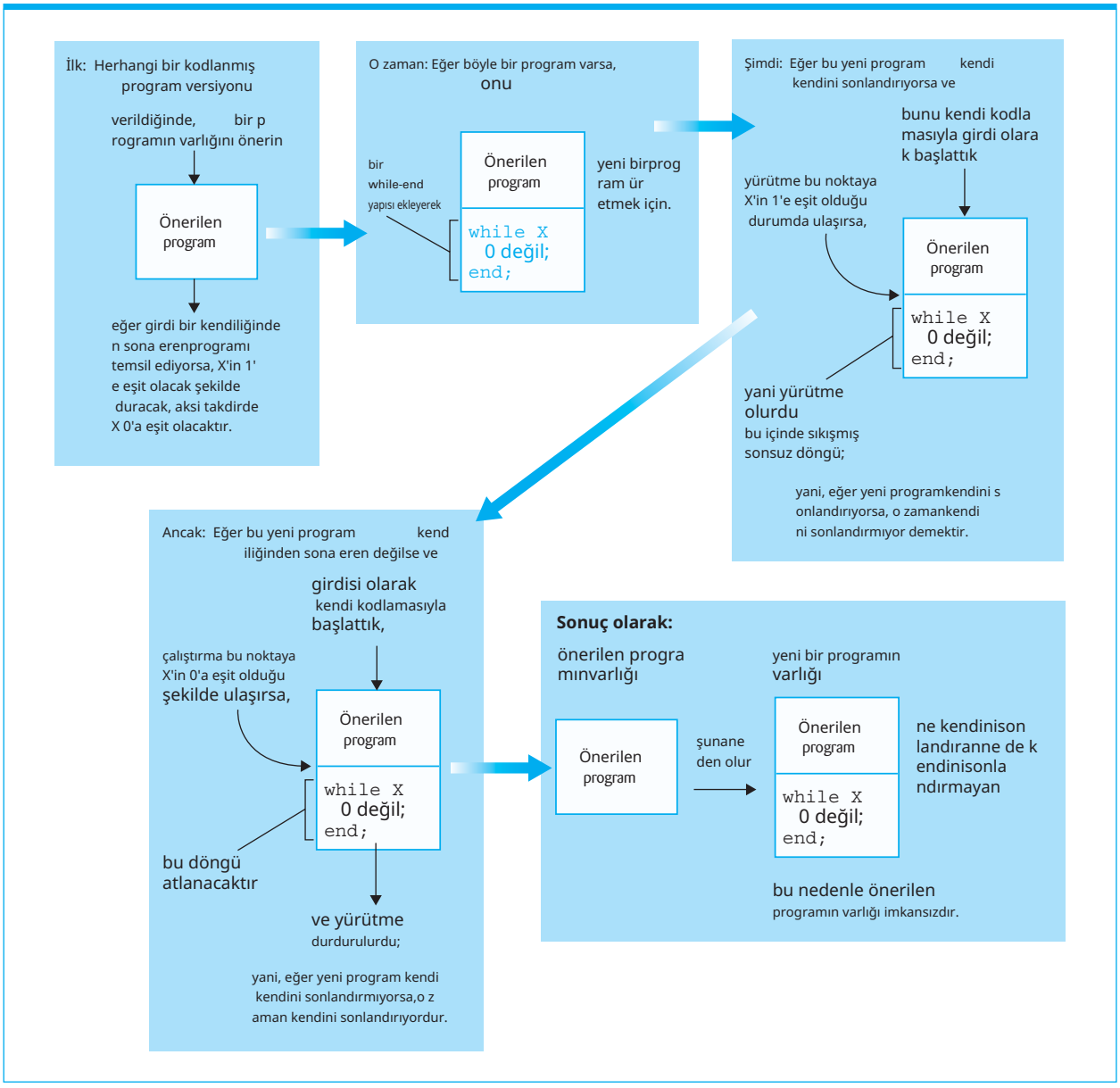
```
X 0 değilken devam et;
son;
```

sonuna ekleyerek yeni bir program oluşturur. Bu yeni program ya kendiliğinden sonlanan ya da sonlanmayan olmalıdır. Ancak, bunun ne ikisi olabileceğini göreceğiz.

Özellikle, eğer bu yeni program kendiliğinden sonlanıyorsa ve onu progr amın kendi kodlanmış temsilini başlatılmış değişkenleri ile çalıştırırsak, o za man yürütmesi eklediğimiz `while` ifadesine ulaştığında `X` değişkeni 1 değeri ni içerecektir. (Bu noktada yeni program, kendiliğinden sonlanan bir progra mın temsilinin girişi olduğunda 1 üreten orijinal programla aynıdır.) Bu nokt ada, programın yürütmesi `while-end` yapısında sonsuza dek takılı kalacaktı r çünkü döngü içinde `X`'nin azaltılması için hiçbir düzenleme yapmadık. Anca k bu, yeni programın kendiliğinden sonlandığı varsayımımızla çelişiyor. Bu n edenle, yeni programın kendiliğinden sonlanmadığı sonucuna varmalıyız.

Doclingo https://www.doclingo.ai

Şekil 12.7Durdurma programının çözümsüzlüğünü kanıtlama



Ancak, eğer yeni program kendiliğinden sonlanmıyorsa ve onu programın kendi kodlanmış temsilini başlatılmış değişkenleri ile çalıştırırsak, eklenen `while` ifadesine `X`'nin 0 değerini alarak ulaşacaktır. (Bu, `while` ifadesinden önc eki ifadelerin, kendiliğinden sonlanmayan bir programın temsilinin girişi olduğ unda 0 çıktısı üreten orijinal programı oluşturmasından kaynaklanır.) Bu duru mda, `while-end` yapısındaki döngü atlanacak ve program duracaktır. Ancak b u, kendiliğinden sonlanan bir programın özelliğidir, bu nedenle yeni programın kendiliğinden sonlandığı sonucuna varmak zorundayız; tıpkı daha önce bunun kendiliğinden sonlanmadığı sonucuna varmak zorunda kaldığımız gibi.

Doclingo https://www.doclingo.ai

Özetle, bir programınbir yandan ya kendini sonlandırması ya da sonlandırmaması gerektiğini ve diğer yandanne kendini sonlandıran ne de sonlandırmayan bir durumla k arşı karşıya olduğumuzu görüyoruz. Sonuç olarak, bu ikileme yol açan varsayımyanlış ol malıdır.

Durma fonksiyonunun hesaplanabilir olmadığını sonucuna varıyoruz ve durm a probleminin çözümü o fonksiyonun hesaplanmasına dayandığı için durma probl eminin çözümünün herhangi bir algoritmik sistemin yeteneklerinin ötesinde oldu ğunu sonucuna varmalıyız. Bu tür problemlere çözümsüz problemler denir.

Kapanışta, az önce tartıştıklarımızı Bölüm 10'daki fikirlerle ilişkilendirmeli yiz. Orada, temel bir soru, hesaplama makinelerinin zeka için gerekli olan gü çleri içerip içermediğiydi. Makinelerin yalnızca algoritmik çözümleri olan pro blemleri çözebileceğini hatırlayın ve şimdi algoritmik çözümleri olmayan pro blemler bulduğumuzu gördük. Soru, insan zihninin yalnızca algoritmik süreçl erin yürütülmesinden daha fazlasını içerip içermediğidir. Eğer içermezse, bu rada tanımladığımız sınırlar da insan düşüncesinin sınırlarıdır. Söylemeye ge rek yok, bu oldukça tartışmalı ve bazen duygusal bir konudur. Örneğin, insa n zihinleri yalnızca programlanmış makinelerden ibaretse, o zaman insanları n özgür iradeye sahip olmadığı sonucuna varılabilir.

Sorular & Alıştırmalar

1. Aşağıdaki Bare Bones programı kendiliğinden sona eriyor mu? Cevabınızı açıklayın.

```
artı X;
decr Y;
```

2. Aşağıdaki Bare Bones programı kendiliğinden sona eriyor mu? Cevabınızı açıklayın.

```
X'i Y'ye kopyala;
incr Y;
Y dahil;
X 0 değilken;
  decr X;
  decr X;
  decr Y;
  decr Y;
son;
decr Y;
Y 0 değilken;
end;
```

3. Aşağıdaki senaryoda ne yanlış?

   Belirli bir toplulukta, herkes kendi evine sahiptir. Topluluğun ev bo yacısı, sadece sahipleri tarafından boyanmayan evleri boyadığını i ddia ediyor.

   (*İpucu:* Ev boyacısının evini kim boyar?)

Doclingo https://www.doclingo.ai

12.5 Problemlerin Karmaşıklığı

12.4. Bölümde, problemlerin çözülebilirliğini inceledik. Bu bölümde, çözülebi lir bir problemin pratik bir çözümü olup olmadığı sorusuyla ilgileniyoruz. Teorik olarak çözülebilir bazı problemlerin, pratik açıdan çözülemez kadar ka rmaşık olduğunu göreceğiz.

Bir Problemin Karmaşıklığını Ölçmek

Algoritma verimliliği konusundaki çalışmamıza, 5.6. Bölümde başladığımız gibi geri d önerek başlıyoruz. Orada, algoritmaları çalıştırmak için gereken zamana göre sınıfla ndırmak için büyük-teta notasyonunu kullandık. Ekleme sıralama algoritmasının (N^2) sınıfında, sıralı arama algoritmasının (n) sınıfında ve ikili arama algoritmasının ($\lg n$) s ınıfında olduğunu bulduk. Şimdi bu sınıflandırma sistemini, problemlerimizin karmaş ıklığını belirlememize yardımcı olmak için kullanıyoruz. Amacımız, hangi problemleri n diğerlerinden daha karmaşık olduğunu ve nihayetinde hangi problemlerin çözüml erinin pratikliğin ötesinde olduğunu belirten bir sınıflandırma sistemi geliştirmektir.

Mevcut çalışmamızın algoritma verimliliği bilgimize dayanmasının nedeni, bi r problemin karmaşıklığını çözümlerinin karmaşıklığı açısından ölçmek istememi zdir. Basit bir problemi, basit bir çözümü olan bir problem olarak kabul ediyoruz; karmaşık bir problem ise basit bir çözümü olmayan bir problemdir. Bir problemi n zor bir çözümü olmasının, problemin kendisinin karmaşık olduğu anlamına gel mediğini unutmayın. Sonuçta, bir problemin birçok çözümü vardır ve bunlardan biri karmaşık olmak zorundadır. Dolayısıyla, bir problemin kendisinin karmaşık o lduğunu sonuçlandırmak için, çözümlerinin hiçbiri basit olmadığını göstermemiz gerekir.

Bilgisayar bilimlerinde, ilgi alanındaki problemler makineler tarafından çöz ülebilenlerdir. Bu problemlerin çözümleri algoritmalar olarak formüle edilir. Böylece, bir problemin karmaşıklığı, o problemi çözen algoritmaların özellikl eriyle belirlenir. Daha kesin olarak, bir problemi çözmek için en basit algorit manın karmaşıklığı, problemin kendisinin karmaşıklığı olarak kabul edilir.

Peki, bir algoritmanın karmaşıklığını nasıl ölçeriz? Ne yazık ki, *karmaşıklı k* terimi farklı yorumlara sahiptir. Bir yorum, algoritmada yer alan karar ver me ve dallanma miktarıyla ilgilidir. Bu açıdan, karmaşık bir algoritma, karma şık ve iç içe geçmiş bir yönler setini içeren bir algoritma olacaktır.
Bu yorum, algoritma keşfi ve temsili ile ilgili konulara ilgi duyan bir yazılım m ühendisi bakış açısıyla uyumlu olabilir, ancak bir makinenin bakış açısından k armaşıklık kavramını yakalamaz. Bir makine, bir sonraki yürütme talimatını s eçerken gerçekten karar vermez, yalnızca makine döngüsünü tekrar tekrar t akip eder ve her seferinde program sayacının gösterdiği talimatı yürütür. So nuç olarak, bir makine karmaşık talimatlar kümesini, basit bir sıralı talimatlar listesini yürütmekte olduğu kadar kolay bir şekilde yürütebilir. Bu nedenle, k armaşıklığın bu yorumu, bir algoritmanın temsilinde karşılaşılan zorluğu ölç me eğilimindedir, algoritmanın kendisinin karmaşıklığını değil.

Bir algoritmanın karmaşıklığını bir makinenin bakış açısından daha doğru bir şekild e yansıtan bir yorum, algoritmayı yürütürken gerçekleştirilmesi gereken adım sayısını ö lçmektir. Bunun, yazılı programda görünen talimat sayısıyla aynı olmadığını unutmayın.

Doclingo https://www.doclingo.ai

Bir döngü, gövdesi tek bir ifadeden oluşan ancak kontrolü gövdenin 100 kez yürütülmesini talep eden bir döngü, yürütüldüğünde 100 ifade ile eşdeğerdi r. Bu nedenle, böyle bir rutin, yazılı formda daha uzun görünen 50 ayrı yazıl mış ifadeler listesinden daha karmaşık olarak kabul edilir. Buradaki nokta, ka rmaşıklığın bu anlamının, bir makinenin bir çözümü yürütmesi için gereken z amanla ilgilidir ve çözümü temsil eden programın boyutuyla değil.

Bu nedenle, bir problemin karmaşık olduğunu, tüm çözümlerinin çok zaman ger ektirmesi durumunda kabul ediyoruz. Bu karmaşıklık tanımına zaman karmaşıklığı denir. Zaman karmaşıklığı kavramıyla, 5.6. bölümde algoritma verimliliği çalışmamız aracılığıyla dolaylı olarak tanıştık. Sonuçta, bir algoritmanın verimliliğini incelemek, a lgoritmanın zaman karmaşıklığını incelemektir; ikisi de birbirinin tersidir. Yani, "daha verimli" eşittir "daha az karmaşık." Bu nedenle, zaman karmaşıklığı açısından, sıralı a rama algoritması (bunun in\) içinde olduğunu bulmuştuk) bir listeyi arama problemi için daha karmaşık bir çözümdür, iken ikili arama algoritması (bunun ($\lg n$) içinde old uğunu bulmuştuk) daha az karmaşık bir çözümdür.

Şimdi algoritmaların karmaşıklığı hakkındaki bilgimizi, problemleri tanımlamak için bir yöntem elde etmek üzere uygulayalım. Bir problemin (zaman) karmaşıklığını ($f(n)$) olarak t anımlıyoruz; burada $f(n)$ n içinde bazı matematiksel ifadelerden oluşmaktadır, eğer proble min çözümü için zaman karmaşıklığı ($f(n)$) içinde olan bir algoritma varsa ve problemin çöz Θümü için başka bir algoritmanın daha düşük bir zaman karmaşıklığı yoksa. Yani, bir probl emin (zaman) karmaşıklığı, en iyi çözümünün (zaman) karmaşıklığı olarak tanımlanır. Ne ya zık ki, bir probleme en iyi çözümü bulmak ve bunun en iyi olduğunu bilmek genellikle kend isi zor bir problemdir. Böyle durumlarda, bir problemin karmaşıklığı hakkında bilinenleri te msil etmek için büyük O notasyonu (büyük oh notasyonu olarak okunur) adı verilen büyük -teta notasyonunun bir varyasyonu kullanılır. Daha kesin olarak, eğer $f(n)$ n içinde bir mate matiksel ifade ise ve bir problem ($f(n)$) içinde bir algoritma ile çözülebiliyorsa, o zaman pro blemin O($f(n)$) içinde olduğunu söyleriz (büyük oh $f(n)$ olarak okunur).Bu nedenle, bir probl emin O(f(n)) içinde olduğunu söylemek, karmaşıklığının ($f(n)$) içinde olduğu bir çözümü old uğu anlamına gelir, ancak daha iyi bir çözümü olabileceği anlamına da gelebilir.

Arama ve sıralama algoritmalarını araştırmamız, bir uzunlukta n liste içinde arama yapma probleminin (liste daha önce sıralandığı bilindiğinde) O(lg n) için de olduğunu, çünkü ikili arama algoritmasının bu problemi çözdüğünü gösteri yor. Dahası, araştırmacılar arama probleminin aslında ($\lg n$) içinde olduğunu gö stermiştir, bu da ikili aramanın bu problem için optimal bir çözüm olduğunu be lirtir. Buna karşılık, bir uzunlukta n listeyi sıralama probleminin (değerlerin oriji nal dağılımı hakkında hiçbir şey bilmediğimizde) O(n^2) içinde olduğunu biliyoru z, çünkü ekleme sıralama algoritması bu problemi çözer. Ancak sıralama probl eminin ($n \lg n$) içinde olduğu bilinmektedir, bu da ekleme sıralama algoritmasını n optimal bir çözüm olmadığını (zaman karmaşıklığı bağlamında) gösterir.

Sıralama problemi için daha iyi bir çözüm örneği birleştirme sıralama algorit masıdır. Yaklaşımı, listenin küçük, sıralı parçalarını birleştirerek daha büyük sıralı parçalar elde etmek ve ardından bunları birleştirerek daha da büyük sıralı parçala r elde etmektir. Her bir birleştirme işlemi, ardışık dosyalar hakkında konuştuğum uzda karşılaştığımız birleştirme algoritmasını uygular (Şekil 9.15). Kolaylık olması açısından, bunu yine Şekil 12.8'de, bu sefer iki listeyi birleştirme bağlamında sunu yoruz. Tam (özyinelemeli) birleştirme sıralama algoritması, `MergeSort`

Doclingo https://www.doclingo.ai

Şekil **12.8** İki listeyi birleştirmek için MergeLists prosedürü

```
prosedür  ListeBirleştir (GirdiListeA, GirdiListeB, ÇıktıListe)

 eğer  (her iki girdi listesi boşsa)  o zaman (Durdur, ÇıktıListe boş kalsın)
 eğer (GirdiListeA boşsa)
   o zaman  (Tükenmiş olduğunu beyan et)
   aksi takdirde  (İlk girişini mevcut girişi olarak beyan et)
eğer (InputListB boşsa)
   o zaman  (Tükenmiş olduğunu beyan et)
   aksi takdirde  (İlk girişini mevcut girişi olarak beyan et)
while  (girdi listelerinden hiçbiri tükenmemişse) do
          (Daha "küçük" olan mevcut girişi OutputList'e koy;
       eğer  (o mevcut giriş, karşılık gelen girdi listesindeki son girişse)
          o zaman  (Girdi listesinin tükenmiş olduğunu belirtin)
          diğer  (Giriş listesindeki bir sonraki girişi listenin mevcut girişi olarak ilan et )
       )
Girdi listesindeki tükenmemiş mevcut girişle başlayarak,
          kalan girişleri ÇıktıListesi'ne kopyalayın.
```

Şekil 12.9'da. Bir listeyi sıralaması istendiğinde, bu prosedür önce listenin iki girişten daha kısa olup olmadığını kontrol eder. Eğer öyleyse, prosedürün görevi tamamlanmıştır. Aksi takdirde, prosedür listeyi iki parçaya böler, bu parçaları sıralaması için diğer prosedür kopyalarına sorar `MergeSort` bu parçaları sıralamak için ve ardından bu sıralı parçaları birleştirerek li stenin son sıralı versiyonunu elde eder.

Bu algoritmanın karmaşıklığını analiz etmek için, önce bir uzunluğu r olan l iste ile bir uzunluğu s olan listeyi birleştirirken yapılması gereken liste girişleri arasındaki karşılaştırma sayısını dikkate alıyoruz. Birleştirme süreci, bir listeden bir girişi diğer listeden bir girişle tekrar tekrar karşılaştırarak ve iki girişten "da ha küçük" olanı çıktı listesine yerleştirerek ilerler. Böylece her karşılaştırma yapı ldığında, dikkate alınması gereken giriş sayısı bir azaltılır. Başlangıçta yalnızca r + s giriş olduğu için, iki listenin birleştirilmesi sürecinin en fazla $r + s$ karşılaştır ma içereceğini sonucuna varıyoruz.

Artık tüm birleştirme sıralama algoritmasını ele alıyoruz. Uzunluğu n ola n bir listeyi sıralama görevini, başlangıçtaki sıralama problemini her biri yakl aşık n/2 uzunluğunda bir listeyi sıralaması istenen iki daha küçük probleme i ndirerek ele alır. Bu iki problem, yaklaşık n/4 uzunluğunda listeleri sıralama problemlerinin toplam dört problemine indirgenir. Bu bölme süreci

Şekil **12.9** Bir prosedür MergeSort olarak uygulanan birleştirme sıralama algoritması

```
prosedür  MergeSort (Liste)

eğer  (Liste birden fazla girişe sahipse)
  o zaman (Liste'nin ilk yarısını sıralamak için MergeSort prosedürünü uygula;
                Liste'nin ikinci yarısını sıralamak için MergeSort prosedürünü uygula;
              Liste'nin birinci ve ikinci yarılarını birleştirmek için MergeLists prosedürünü uygula
                Liste'nin sıralı bir versiyonunu üretmek için)
          )
```

Doclingo https://www.doclingo.ai

Ağaç yapısında özetlenebilir, Şekil 12.10'da her bir düğüm, yinelemeli süreçt eki tek bir problemi temsil eder ve bir düğümün altındaki dallar, ebeveynden türetilen daha küçük problemleri temsil eder. Bu nedenle, ağaçtaki düğüml erde meydana gelen karşılaştırmaların sayısını toplayarak, tüm sıralama sür ecinde meydana gelen toplam karşılaştırma sayısını bulabiliriz.

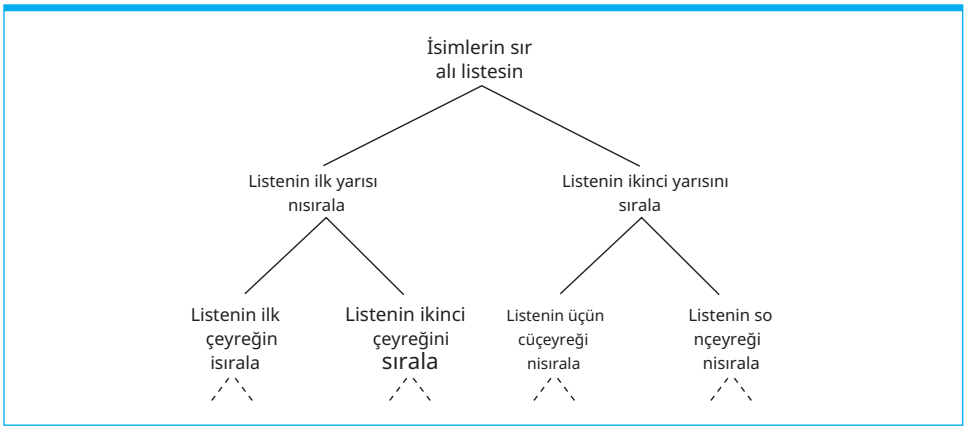
Öncelikle, ağacın her seviyesinde yapılan karşılaştırma sayısını belirleyelim. Ağacın herhangi bir seviyesinde görünen her düğümün, orijinal listenin benze rsiz bir segmentini sıralama görevi olduğunu gözlemleyin. Bu, birleştirme süre ciyle gerçekleştirilir ve bu nedenle, daha önce de belirttiğimiz gibi, liste segmen tindeki giriş sayısından daha fazla karşılaştırma gerektirmez. Dolayısıyla, ağacı n her seviyesi, liste segmentlerindeki toplam giriş sayısından daha fazla karşıla ştırma gerektirmez ve çünkü belirli bir seviyedeki segmentler, orijinal listenin a yrık kısımlarını temsil ettiğinden, bu toplam orijinal listenin uzunluğundan dah a büyük olamaz. Sonuç olarak, ağacın her seviyesi, en fazla n karşılaştırma içeri r. (Elbette en alt seviye, iki uzunluktan daha kısa listeleri sıralamayı içerir, bu da hiç karşılaştırma gerektirmez.)

Şimdi ağaçtaki seviye sayısını belirliyoruz. Bunun için, problemleri daha k üçük problemlere ayırma sürecinin, uzunluğu iki olan listeler elde edilene ka dar devam ettiğini gözlemleyin. Böylece, ağaçtaki seviye sayısı, n değerinden başlayarak ikiye bölme işlemini, sonuç birden büyük olana kadar tekrar tekr ar yapabileceğimiz sayı ile belirlenir; bu da lg n'dir.

Daha kesin olarak, karşılaştırmalar içeren ağaçta $\lceil \lg n \rceil$'den fazla seviye yoktur; burada $\lceil \lg n \rceil$ notasyonu, lg n değerinin bir üst tam sayıya yuvarlanmış halini t emsil eder.

Son olarak, bir uzunluğu n olan listeyi sıralarken, birleştirme sıralama algoritma sı tarafından yapılan toplam karşılaştırma sayısı, ağaçtaki her seviyede yapılan karşıl aştırma sayısının, karşılaştırmaların yapıldığı seviye sayısı ile çarpılmasıyla elde edilir. Bunun n lg n'den büyük olmadığını sonucuna varıyoruz. Çünkü $n \lceil \lg n \rceil$ grafiği, n lg n grafiği ile aynı genel şekle sahiptir; bu nedenle, birleştirme sıralama algoritmasının O(n lg n) sınıfına ait olduğunu sonucuna varıyoruz. Araştırmacıların sıralama proble minin karmaşıklığının (n lg n) olduğunu söyledikleri gerçeği ile birleştirildiğinde, birle Θştirme sıralama algoritmasının sıralama problemi için optimal bir çözüm sunduğun u sonucuna varıyoruz.

Şekil **12.10** Birleştirme sıralama algoritması tarafından üretilen problem hiyerarşisi



Doclingo https://www.doclingo.ai

Alan Karmaşıklığı

Zaman açısından karmaşıklığı ölçmenin bir alternatifi, bunun yerine depolama ala nıgereksinimlerini ölçmektir—bu da alan karmaşıklığı olarak bilinen bir ölçü ile so nuçlanır. Yani,bir problemin alan karmaşıklığı, problemi çözmek için gereken depol ama alanımiktarı ile belirlenir. Metinde, *n* girişe sahip bir listeyi sıralamanın zaman karmaşıklığının O(*n*lg *n*) olduğunu gördük. Aynı problemin alan karmaşıklığıO(*n*+ 1) = 0(*n*) kadar değildir. Sonuçta, *n* girişe sahip bir listeyi eklemesıralaması kullanarak sıralamak, listenin kendisi için alan artı geçiciolarak tek bir girişi depolamak için al an gerektirir. Böylece, daha uzun listeleri sıralamamız istendiğinde, her görev için gereken zamanın, gereken alandan daha hızlı bir şekilde arttığını göreceğiz. Bu aslında yaygın bir fenomendir. Bir problemin alan karmaşıklığı, zaman karm aşıklığından daha hızlı büyümez çünkü alanı kullanmak zaman alır.

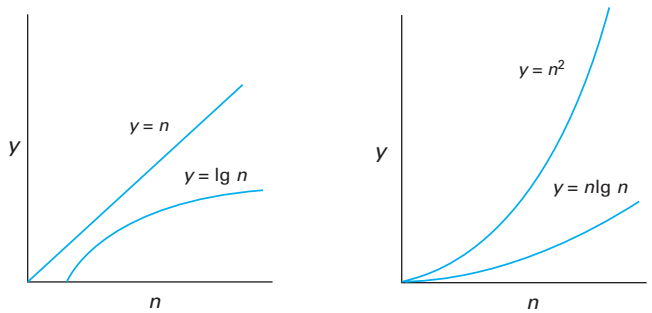
Zaman ve alan karmaşıklığı arasında genellikle bir denge sağlanır. Bazı uygulamalarda, belirli hesaplamaları önceden yapmak ve sonuçları ihtiyaç duyulduğunda hızlıca alınabilecek bir tabloda saklamak avantajlı olabilir. Böyle bir "tablo arama" tekniği, bir sonuca ihtiyaç duyulduğunda elde etmek için gereken zamanı azaltır, ancak tablonun gerektirdiği ek alan pahasına. Öte yand an, veri sıkıştırma genellikle depolama gereksinimlerini azaltmak için kullanılır, ancak veriyi sıkıştırmak ve açmak için gereken ek zaman pahasına.

Polinomik ve Polinomik Olmayan Problemler

Diyelim ki$f(n)$ ve $g(n)$ matematiksel ifadeler. $g(n)$'nin $f(n)$ ile sınırlı olduğunu söylemek, bu ifad eleri daha büyük ve daha büyük n değerlerine uyguladığımızda, $f(n)$ değerinin nihay etinde $g(n)$ değerinden büyük olacağı ve tüm daha büyük n değerleri için $g(n)$'den bü yük kalacağı anlamına gelir. Başka bir deyişle, $g(n)$'nin $f(n)$ ile sınırlı olması, $f(n)$ grafiğ inin "büyük" n değerleri için $g(n)$ grafiğinin üzerinde olacağı anlamına gelir. Örneğin, lg n ifadesi n ifadesi ile sınırlıdır (Şekil 12.11a) ve n lgn ifadesi n^2 ile sınırlıdır (Şekil 12.11b).

Bir problemin polinom problemi olduğunu söyleriz eğer problem $O(f(n))$ içindeyse, burada $f(n)$ ifadesi ya bir polinomdur ya da bir

Şekil 12.11 Matematiksel ifadelerin *n*, lg *n*, *n*lg *n* ve *n*2 grafikleri



a. *n* karşısında lg *n* **b.** n^2 karşısında *n* lg *n*

Doclingo https://www.doclingo.ai

polinom. Tüm polinom problemlerinin kümesi P ile temsil edilir. Önceki araştır malarımız, bir listeyi arama ve bir listeyi sıralama problemlerinin P'ye ait olduğ unu bize söylemektedir.

Bir problemin polinom problemi olduğunu söylemek, problemin çözülmesi için gereke n zaman hakkında bir ifadedir. Genellikle P'deki bir problemin polinom zamanında çözülebi leceğini veya problemin polinom zaman çözümüne sahip olduğunu söyleriz.

P sınıfına ait problemleri tanımlamak, bilgisayar bilimlerinde büyük öne m taşımaktadır çünkü bu, problemlerin pratik çözümlerinin olup olmadığıyla ilgili sorularla yakından ilişkilidir. Gerçekten de, P sınıfının dışındaki problem ler, orta boyutlu girdiler için bile son derece uzun yürütme süreleri ile karakt erize edilir. Örneğin, çözümü 2n\ adım gerektiren bir problemi düşünelim. Ü stel ifade 2n\ herhangi bir polinom tarafından sınırlı değildir—eğer f(^{n}) bir p olinom ise, n değerini artırdıkça, 2^n değerlerinin nihayetinde f(n) değerlerind en daha büyük olduğunu göreceğiz. Bu, karmaşıklığı (2^n) olan bir algoritman ın genellikle daha az verimli olacağı ve dolayısıyla karmaşıklığı ($f(n)$) olan bir algoritm adan daha fazla zaman gerektireceği anlamına gelir. Karmaşıklığı bir üstel if ade ile tanımlanan bir algoritmanın üstel zaman gerektirdiği söylenir.

Özellikle bir örnek olarak, n kişiden oluşan bir gruptan oluşturulabilecek tüm olası alt komitelerin listelenmesi sorununu düşünün. Çünkü burada $2^n - 1$ böyle alt komiteler (bir alt komitenin tamamını oluşturmasına izin veriyoruz grup ama boş kümenin bir alt komite olarak kabul edilmemesi), bu problemi çözen herhangi bir algoritmaen az 2^{n}1 adımına sahip olmalı ve dolayısıyla e n az bu kadar bir karmaşıklığasahip olmalıdır. Ancak, 2^{n}1 ifadesi, üstel bir ifa de olduğundan,hiçbir polinomla sınırlı değildir. Bu nedenle, bu problemin he rhangi bir çözümü,görevlerin seçildiği grubun boyutu arttıkça son derece za man alıcı hale gelir.

Çıktısının boyutu nedeniyle karmaşıklığı büyük olan alt komite problemi mizin aksine, nihai çıktısı yalnızca basit bir evet veya hayır cevabı olan karma şıklıkları büyük olan problemler vardır. Bir örnek, gerçek sayıların eklenmesi yle ilgili ifadelerin doğruluğu hakkında sorulara cevap verme yeteneğini içeri r. Örneğin, "Kendisiyle toplandığında 6 değerini üreten bir gerçek sayı var mı dır?" sorusunun cevabının evet olduğunu kolayca tanıyabiliriz, oysa "Kendisiy le toplandığında 0 olan sıfırdan farklı bir gerçek sayı var mıdır?" sorusunun c evabı hayırdır. Ancak, bu tür sorular daha karmaşık hale geldikçe, onlara cev ap verme yeteneğimiz azalmaya başlar. Eğer kendimizi birçok böyle soruyla karşı karşıya bulursak, bir bilgisayar programına yardım için başvurma eğili minde olabiliriz. Ne yazık ki, bu sorulara cevap verme yeteneğinin üstel zam an gerektirdiği gösterilmiştir, bu nedenle sorular daha karmaşık hale geldikç e bir bilgisayar bile zamanında cevaplar üretemez.

Teorik olarak çözülebilir ancak P içinde olmayan problemlerin bu kadar b üyük zaman karmaşıklıklarına sahip olması, bu problemlerin pratik açıdan es asen çözülemez olduğunu sonucuna varmamıza yol açar. Bilgisayar bilimcile ri bu problemlere çözümsüz demektedir. Bununla birlikte, P sınıfı, çözümsüz problemleri pratik çözümleri olabileceklerden ayıran önemli bir sınırı temsil etmeye başlamıştır. Böylece, P sınıfının anlaşılması bilgisayar bilimi içinde ön emli bir hedef haline gelmiştir.

Doclingo https://www.doclingo.ai

NP Problemleri

Şimdi, seyahat bütçesini aşmadan farklı şehirlerdeki her bir müşterisini ziyar et etmesi gereken bir seyahat satış temsilcisi ile ilgili olan seyahat satış tems ilcisi problemini ele alalım. Problemi, evinden başlayarak, ilgili şehirleri birbi rine bağlayıp tekrar evine dönecek bir yol bulmaktır; bu yolun toplam uzunlu ğu izin verilen milajı aşmamalıdır.

Bu problemin geleneksel çözümü, potansiyel yolları sistematik bir şekilde ele almak, her yolun uzunluğunu milaj sınırı ile karşılaştırmak ve ya kabul edilebilir bi r yol bulana kadar ya da tüm olasılıkları göz önünde bulundurana kadar devam et mektir. Ancak, bu yaklaşım polinom zaman çözümü üretmez. Şehir sayısı arttıkça, test edilmesi gereken yol sayısı herhangi bir polinomdan daha hızlı bir şekilde art ar. Bu nedenle, seyahat satış temsilcisi problemini bu şekilde çözmek, büyük şehir sayıları içeren durumlar için pratik değildir.

Seyahat satış temsilcisi problemini makul bir süre içinde çözmek için dah a hızlı bir algoritma bulmamız gerektiği sonucuna varıyoruz. Eğer tatmin edi ci bir yol varsa ve biz onu ilk seçersek, mevcut algoritmamız hızlı bir şekilde s onlanır. Özellikle, aşağıdaki talimatlar listesi hızlı bir şekilde yürütülebilir ve p roblemi çözme potansiyeline sahiptir:

```
Olası yollardan birini seçin ve toplammesafesini hesaplayın.

Eğer (bu mesafe izin verilen milajdan büyük değilse)
    o zaman (başarıyı ilan edin)
    aksi takdirde (hiçbir şey ilan etmeyin)
```

Ancak, bu talimatlar seti teknik anlamda bir algoritma değildir. İlk talimatı, h angi yolun seçileceğini belirtmediği ve kararın nasıl verileceğini de tanımlam adığı için belirsizdir. Bunun yerine, programı yürüten mekanizmanın kendi b aşına karar vermesi için yaratıcılığına dayanır. Bu tür talimatların belirsiz old uğunu ve bu tür ifadeleri içeren bir "algoritmayı" belirsiz algoritma olarak adl andırıyoruz.

Şehir sayısı arttıkça, önceki belirsiz algoritmayı yürütmek için gereken za man nispeten yavaş artar. Bir yol seçme süreci, yalnızca şehirlerin bir listesin i üretmekten ibarettir ve bu, ilgili şehir sayısına orantılı bir zamanda yapılabil ir. Dahası, seçilen yol boyunca toplam mesafeyi hesaplamak için gereken za man da ziyaret edilecek şehir sayısına orantılıdır ve bu toplamı milaj sınırıyla karşılaştırmak için gereken zaman şehir sayısından bağımsızdır. Dolayısıyla, belirsiz algoritmayı yürütmek için gereken zaman bir polinomla sınırlıdır.

Bu nedenle, seyahat eden satıcı problemini polinom zamanında bir belirsiz algorit ma ile çözmek mümkündür.

Elbette, belirsiz çözümümüz tamamen tatmin edici değildir. Şanslı bir tahmin e dayanır. Ancak, varlığı, belki de seyahat eden satıcı probleminin polinom zamanı nda çalışan bir deterministik çözümü olduğu fikrini önermeye yeter. Bunun doğru olup olmadığı hala açık bir soru. Aslında, seyahat eden satıcı problemi, polinom z amanında çalışan belirsiz çözümleri olduğu bilinen birçok problemden biridir, anc ak henüz hiçbir deterministik polinom zaman çözümü bulunmamıştır.

Doclingo https://www.doclingo.ai

Bu problemlere yönelik belirsiz çözümlerin cezbedici verimliliği, bazılarını bir gün verimli deterministik çözümlerin bulunacağı umudunu taşımaya yönlendiriyor, ancak çoğu, bu problemlerin verimli deterministik algoritmaların yeteneklerinden kaçacak kadar karmaşık olduğuna inanıyor.

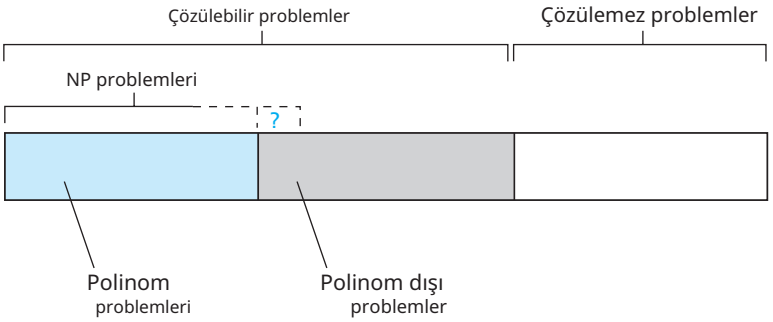
Bir belirsiz algoritma tarafından polinom zamanında çözülebilen bir problem belirsiz polinom problemi veya kısaca NP problemi denir. NP problemleri sınıfı genellikle NP ile gösterilir. P'deki tüm problemlerin aynı zamanda NP'de olduğunu unutmayın, çünkü herhangi bir (deterministik) algoritmaya performansını etkilemeden bir belirsiz talimat eklenebilir.

NP problemlerinin hepsinin P içinde olup olmadığı ise açık bir sorudur, seyahat eden satıcı problemi ile gösterildiği gibi. Bu, günümüzde bilgisayar biliminde en çok bilinen çözülmemiş problemlerden biridir. Çözümü önemli sonuçlar doğurabilir. Örneğin, bir sonraki bölümde, seyahat eden satıcı problemine benzer problemleri çözmek için gereken muazzam zamana dayanan bütünlüğü sağlayan şifreleme sistemlerinin tasarlandığını öğreneceğiz.
Eğer bu tür problemlere verimli çözümlerin var olduğu ortaya çıkarsa, bu şifreleme sistemleri tehlikeye girecektir.

NP sınıfının, aslında P sınıfıyla aynı olup olmadığını çözme çabaları, NP sınıfı içinde NP-tamamlayıcı problemler olarak bilinen bir problem sınıfının keşfine yol açmıştır. Bu problemlerin özelliği, herhangi birinin polinom zaman çözümünün, NP'deki diğer tüm problemler için de polinom zaman çözümü sağlayacak olmasıdır. Yani, eğer bir (belirleyici) algoritma, NP-tamamlayıcı problemlerden birini polinom zaman içinde çözen bir algoritma bulunursa, o algoritma NP'deki diğer herhangi bir problemi de polinom zaman içinde çözmek için genişletilebilir. Bu durumda, NP sınıfı P sınıfıyla aynı olacaktır. Seyahat eden satıcı problemi, NP-tamamlayıcı bir problemin örneğidir.

Özetle, sorunların ya çözülebilir (bir algoritmik çözümü olan) ya da çözülemez (bir algoritmik çözümü olmayan) olarak sınıflandırılabileceğini bulduk; bu, Şekil 12.12'de gösterilmiştir. Ayrıca, çözülebilir sorunlar sınıfı içinde iki alt sınıf bulunmaktadır. Birincisi, pratik çözümleri olan polinom problemlerinin topluluğudur. İkincisi, yalnızca nispeten küçük veya dikkatlice seçilmiş girdiler için pratik çözümleri olan polinom dışı problemlerin topluluğudur. Son olarak, şu ana kadar kesin sınıflandırmadan kaçan gizemli NP problemleri vardır.

Şekil **12.12** Sorun sınıflandırmasının grafik özeti



Doclingo https://www.doclingo.ai

Deterministik ile Belirsiz Arasındaki Fark

Birçok durumda, deterministik ve nondeterministik "algoritma" arasında ince bir çizgi vardır. Ancak, ayrım oldukça net ve önemlidir. Deterministik bir algoritma, algoritmayı yürüten mekanizmanın yaratıcı yeteneklerine dayanmazken, nondeterministik bir "algoritma" dayanabilir. Örneğin, aşağıdaki talimatı karşılaştırın:

> Bir sonraki kavşağa gidin ve ya sağa ya da sola dönün.

ve talimatı

> Bir sonraki kavşağa gidin ve köşede duran kişinin size ne yapmanızı söyle diğine bağlı olarak sağa veya sola dönün.

Her iki durumda da, talimatları izleyen kişinin aldığı eylem, talimatı gerçekten yerine getirmeden önce belirlenmez. Ancak, ilk talimat, talimatları izleyen kişinin kendi yargısına dayalı bir karar vermesini gerektirir ve bu nedenle belirsizdir. İkinci talimat, talimatları izleyen kişiden böyle bir gereklilikte bulunmaz - kişiye her aşamada ne yapması gerektiği söylenir. Eğer birkaç farklı kişi ilk talimatı izlerse, bazıları sağa dönebilirken, diğerleri sola dönebilir. Eğer birkaç kişi ikinci talimatı izler ve aynı bilgiyi alırlarsa, hepsi aynı yöne dönecektir. Burada, deterministik ve belirsiz "algoritmalar" arasında önemli bir ayrım yatmaktadır. Eğer bir deterministik algoritma, aynı girdi verileriyle tekrar tekrar çalıştırılırsa, her seferinde aynı eylemler gerçekleştirilecektir.

Ancak, belirsiz bir "algoritma" aynı koşullar altında tekrarlandığında farklı eylemler üretebilir.

Sorular & Alıştırmalar

1. Bir problemin bir algoritma ile $\Theta_{(2^n)}$ içinde çözülebileceğini varsayalım. Problemin karmaşıklığı hakkında ne sonuç çıkarabiliriz?
2. Bir problemin bir algoritma ile (n^2) ve başka bir algoritma ile (2^n) çözülebileceğini varsayalım. Bir algoritma her zaman diğerinden daha iyi performans gösterir mi?
3. Alice ve Bill'den oluşan bir komiteden oluşturulabilecek tüm alt komiteleri listeleyin. Alice, Bill ve Carol'dan oluşan komiteden oluşturulabilecek tüm alt komiteleri listeleyin.
   Alice, Bill, Carol ve David'den oluşan alt komiteler ne olacak?
4. Bir polinom probleminin örneğini verin. Bir polinom olmayan problemin örneğini verin. Henüz polinom problemi olduğu gösterilmemiş bir NP probleminin örneğini verin.
5. Algoritma X'in karmaşıklığı, algoritma Y'nin karmaşıklığından büyükse, algoritma X'in algoritma Y'den daha zor anlaşılır olması zorunlu mudur? Cevabınızı açıklayın.

Doclingo https://www.doclingo.ai

12.6 Kamu Anahtarı Kriptografisi

Bazı durumlarda, bir problemin çözümünün zor olması, bir yükten ziyade bir varlık haline getirilmiştir. Özellikle ilgi çekici olan, verilen bir tam sayının çarpanlarını bulma problemidir - bunun için henüz verimli bir çözüm bulunmamıştır, eğer böyle bir çözüm varsa. Örneğin, sadece kağıt ve kalemle, 2,173 gibi nispeten küçük değerlerin çarpanlarını bulma görevini zaman alıcı bulabilirsiniz ve eğer söz konusu sayı o kadar büyükse ki, temsili birkaç yüz basamak gerektiriyorsa, modern teknolojinin en iyi çarpan bulma teknikleri kullanılarak uygulanması durumunda bile bu görev çözümsüz hale gelecektir.

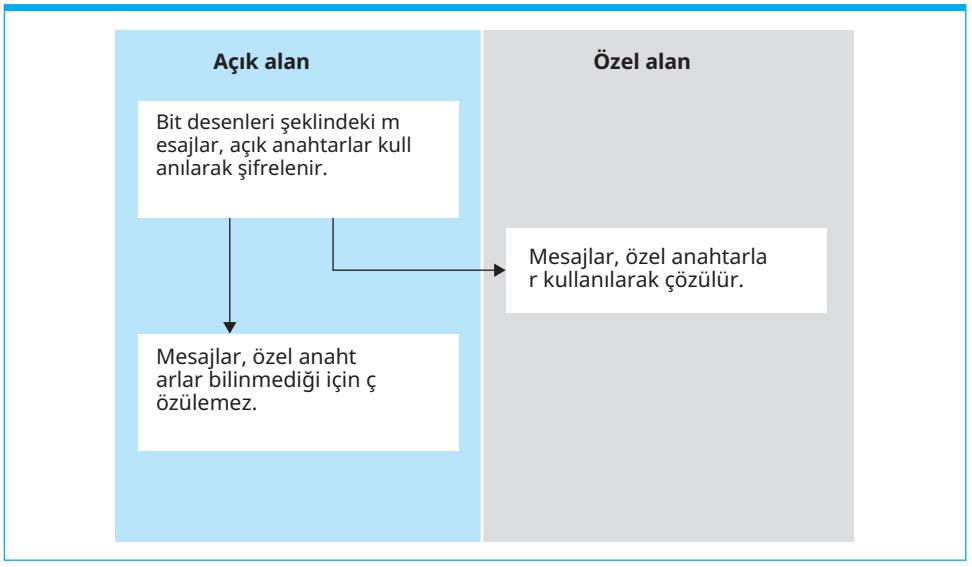
Büyük tam sayıların çarpanlarını belirlemenin etkili bir yolunu bulamamak, uzun zamandır birçok matematikçinin canını sıkmıştır, ancak kriptografi alanında bu, mesajları şifrelemek ve şifre çözmek için popüler bir yöntem üretmek amacıyla uygulanmıştır. Bu yöntem, mucitleri Ron Rivest, Adi Shamir ve Len Adleman'ı onurlandırmak için seçilen bir isim olan RSA algoritması olarak bilinir. Bu, bir dizi değer olan şifreleme anahtarları kullanarak mesajları şifrelemenin ve bu mesajları başka bir dizi değer olan şifre çözme anahtarlarıyla çözmenin bir yoludur. Şifreleme anahtarlarını bilen kişiler mesajları şifreleyebilir, ancak mesajları çözememektedir. Mesajları çözebilen tek kişi, şifre çözme anahtarlarını elinde bulunduran kişidir. Böylece, şifreleme anahtarları sistemin güvenliğini ihlal etmeden geniş bir şekilde dağıtılabilir.

Bu tür kriptografi sistemlerine açık anahtar şifreleme sistemleri denir; bu terim, mesajları şifrelemek için kullanılan anahtarların kamu bilgisi olabileceğini ve sistemin güvenliğini zayıflatmadığını yansıtır. Gerçekten de, şifreleme anahtarları genellikle açık anahtarlar olarak adlandırılırken, şifre çözme anahtarları özel anahtarlar olarak adlandırılır (Şekil 12.13).

Modüler Notasyon

RSA açık anahtar şifreleme sistemini tanımlamak için, x (mod m) notasyonunu kullanmak uygundur; bu, “x mod m” olarak okunur ve x'in m ile bölündüğünde elde edilen kalanı temsil eder. Böylece 9 (mod 7) 2'dir.

Şekil 12.13 Açık anahtar kriptografisi



Doclingo https://www.doclingo.ai

çünkü 9 ÷ 7, 2 kalanını üretir. Benzer şekilde, 24 (mod 7) 3'tür çünkü 24 ÷ 7, 3 kalanını üretir ve 14 (mod 7) 0'dır çünkü 14 ÷ 7, 0 kalanını üretir. x (mod m) eğer x, 0 ile m arasında bir tam sayı ise x'in kendisidir. $m - 1$. Örneğin, 4 (mod 9) 4'tür.

Matematik bize der ki eğer p ve$q$ asal sayılar ve m 0 ile pq (p ve$q$'nun çarpımı) arasında bir tam sayı ise, o zaman

$$1 = m^{k(p-1)(q-1)} \pmod{pq}$$

herhangi bir pozitif tam sayı k için. Bu iddiayı burada gerekçelendirmeyeceğiz, ancak ifadeyi netleştirmek için bir örnek düşünmek faydalıdır. O halde, p ve$q$ sırasıyla 3 ve 5 asal sayıları ve m tam sayısı 4 olsun. O zaman ifade, herhangi bir pozitif tam sayı k için, $m^{k(p1)(q1)}$ değerinin 15'e (3 ve 5'in çarpımı) bölündüğünde 1 kalanı vereceğini iddia etmektedir. Özellikle, eğer k1 ise, o zaman

$$m^{k(p-1)(q-1)} = 4^{1(3-1)(5-1)} = 4^8 = 65{,}536$$

15'e bölündüğünde kalan 1 olan bir sayı. Ayrıca, eğer k $= 2,$

o zaman

$$m^{k(p-1)(q-1)} = 4^{2(3-1)(5-1)} = 4^{16} = 4{,}294{,}967{,}296$$

15'e tekrar bölündüğünde kalan 1 olan bir sayı. Gerçekten de, k için seçilen pozitif tam sayıya bakılmaksızın kalan 1 elde ederiz.

RSA Açık Anahtar Kriptografisi

Artık RSA algoritmasına dayalı bir açık anahtar şifreleme sistemini inşa etmeye ve analiz etmeye hazırız. Öncelikle iki farklı asal sayı p ve$q$ seçiyoruz, bunların çarpımını n ile temsil edeceğiz. Sonra e ve$d$ olmak üzere iki başka pozitif tam sayı seçiyoruz, böylece $e{\times}d$ $(p$k$)(q1)+$ 1, bazı pozitif tam sayılar için k. Bu değerleri e ve$d$ olarak adlandırıyoruz çünkü bunlar sırasıyla şifreleme ve şifre çözme sürecinin bir parçası olacak. (Değerlerin e ve$d$, önceki denklemi sağlamak için seçilebileceği gerçeği, burada takip etmeyeceğimiz başka bir matematiksel gerçektir.)

Böylece beş değer seçmiş olduk: p,q,n,e ve d. Değerler e ve$n$ şifreleme anahtarlarıdır. Değerler d ve$n$ şifre çözme anahtarlarıdır. Değerler p ve$q$ yalnızca şifreleme sistemini inşa etmek için kullanılır.

> Açıklık için belirli bir örneği ele alalım. *p* ve*q* için 7 ve 13 değerlerini seçtiğimizi varsayalım. O zaman *n*7× 1391. Ayrıca, 5 ve 29 değerleri *e* ve*d* için kullanılabilir çünkü 5 ×29145144+ $1 = 2(7-1)(13-1)$ 1 $= 2(p-1)(q-1) +$ 1 gerektiği gibi. Böylece, şifreleme anahtarları n $=$ 91 ve e $=$ 5, ve şifre çözme anahtarları n $=$ 91 ve d $=$ 29. Şifreleme anahtarlarını, olabilecek herkese dağıtıyoruz. mesaj göndermek istiyorsanız, ancak şifre çözme anahtarlarını (ve p ve$q$ değerlerini) kendimize saklıyoruz.

Artık mesajların nasıl şifrelenebileceğini ele alıyoruz. Bu amaçla, bir mesajın şu anda bir bit deseni olarak kodlandığını varsayalım (belki ASCII veya Unicode kullanarak) ve desenin değeri, ikili bir temsil olarak yorumlandığında, n'den daha azdır. (Eğer n'den daha az değilse, mesajı daha küçük parçalara ayırır ve her parçayı ayrı ayrı şifreleriz.)

Doclingo https://www.doclingo.ai

Mesajımızın, ikili bir temsil olarak yorumlandığında, m değerini temsil ett iğini varsayalım. O zaman, mesajın şifreli versiyonu, $c \quad ^{e}(\mathbf{m}\mathrm{mod}n)$ değerinin ikili temsilidir. Yani, şifreli mesaj, m^e değerinin n ile bölünmesinden elde edilen k alanın ikili temsilidir.

> Özellikle, biri 10111 mesajını şifrelemek isterse ve önceki örnekte geliştirilen şifreleme anahtarları n91 ve e5'i kullanırsa, önce 101 11'in 23 için ikili temsil olduğunu kabul eder, ardından $23^e 23^5$ 6,436,343 hesaplar ve nihayet bu değeri n91'e bölerek kalan 4'ü eld e eder. Bu nedenle, mesajın şifreli versiyonu 4'ün ikili temsili olan 100 olacaktır.

Bir mesajı, ikili notasyonda c değerini temsil eden bir mesajı çözmek için, $c^d(\mathrm{mod}n)$ değerini hesaplarız. Yani, c^d değerini hesaplarız, sonucu n ile böleri z ve kalanı alırız. Gerçekten de, bu kalan, orijinal mesajın m değeridir çünkü

$$\begin{aligned} c^d \pmod n &= m^{e\times d} \pmod n \\ &= m^{k(p-1)(q-1)\,+1} \pmod n \\ &= m \times m^{k(p-1)(q-1)} \pmod n \\ &= m \pmod n \\ &= m \end{aligned}$$

Burada $m^{k(p1)(q1)}(\mathrm{mod}n)$ gerçeklerini kullandık. $= m^{k(p-1)(q-1)} \pmod{pq} = 1$ ve$m(\mathrm{mod}n)$ $= m$ (çünkü$m < n$), daha önce iddia edildiği gibi.

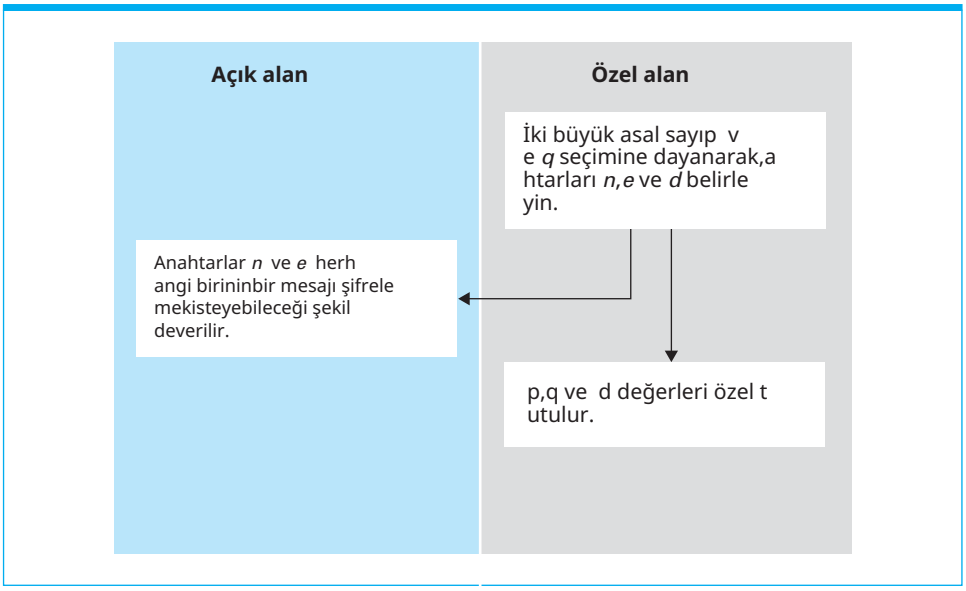
> Önceki örneğe devam edersek, eğer 100 mesajını alırsak, bunu 4 değeri olarak tanırız, $4^d 4$29288,230,376,151,711,744 değerini hes $=$ aplarız ve bu değeri n91 ile böleriz, kalan 23'ü elde ederiz, bu da i kili notasyonda ifade edildiğinde orijinal mesaj 10111'i üretir.

Özetle, bir RSA açık anahtar şifreleme sistemi, p ve$q$ olmak üzere iki asal t am sayı seçilerek oluşturulur, bunlardan n, e ve d değerleri türetilir. n ve$e$ değ erleri mesajları şifrelemek için kullanılır ve bu nedenle açık anahtarlardır. n v ed değerleri mesajları çözmek için kullanılır ve özel anahtarlardır (Şekil 12.14) . Sistemin güzelliği, mesajları şifrelemeyi bilmenin, mesajları çözmeyi sağlam amasıdır. Böylece şifreleme anahtarları n ve$e$ geniş bir şekilde dağıtılabilir. E ğer düşmanlarınız bu anahtarları elde ederse, yine de ele geçirdikleri mesajl arı çözemezler. Sadece şifre çözme anahtarlarını bilen bir kişi mesajları çöze bilir.

Bu sistemin güvenliği, şifreleme anahtarlarını bilmenin n ve e, şifre çözme ana htarlarını n ve d hesaplamaya izin vermediği varsayımına dayanmaktadır. Ancak, tam olar ak bunu yapacak algoritmalar vardır! Bir yaklaşım, n değerini çarpanlarına ayırma k, p ve q değerlerini keşfetmek ve ardından d değerini bulmak için k değerini bulm aktır; böylece $k(p \quad -1)(q-1)+1$, e ile eşit olarak bölünür (bölüm bu durumda d) olacaktır. Öte yandan, bu süreçteki ilk adım zaman alıcı olabil ir—özellikle p ve$q$ değerleri büyük seçilmişse. Aslında, eğer p ve$q$ o kadar büyüktür ki, ikili temsilleri yüzlerce basamağa ihtiyaç duyar, o zaman en iyi bilinen çarpan algoritmaları, p ve$q$'nun kimliklerinin n'den ort aya çıkması için yıllar alacaktır. Bunun sonucunda, şifreli bir mesajın içeriği, a nlamı bozulduktan sonra bile güvenli kalacaktır.

Doclingo https://www.doclingo.ai

Şekil **12.14**RSA genel anahtar şifreleme sisteminin kurulması



Bugüne kadar, RSA kriptografisine dayalı mesajları şifreleme anahtarlarını bilmede n çözmenin verimli bir yolunu bulan kimse olmadı ve bu nedenle RSA algoritmasına day alı açık anahtar şifrelemesi, İnternet üzerinden iletişim kurarken gizlilik elde etmek için yaygın olarak kullanılmaktadır.

Sorular & Alıştırmalar

1. 66,043'ün çarpanlarını bulun. (Bu konuda çok fazla zaman harcamayın. Önemli olan, bunun zaman alıcı olabileceğidir.)
2. Genel anahtarları n $= 91$ ve e $= 5$, mesajı 101'i şifreleyin.
3. Özel anahtarları n $= 91$ ve d $= 29$, mesajı 10'u deşifre edin.
4. Asal sayılara dayalı bir RSAgenel anahtar kriptografi sisteminde deşif releme anahtarları için uygun değeri n ve$d$ bulun. $= 7$ ve q $= 19$ ve şifreleme anahtarı e $= 5.$

Bölüm Gözden Geçirme Problemleri

1. Bir yapının şu şekilde simüle edilebileceğini gösterin:X 0'a eşit olduğu sürece;

...sonu

Bare Bones ile simüle edilebilir.

2. Değişken X, değişken Y'den kü çük veya eşitse, değişken Z'ye 1 yerleştiren bir Bare Bones progra mı yazın; aksi takdirde değişken Z'ye 0 y erleştirin.
3. Değişken Z'ye 2'nin X'inci kuvvetini yerleştire n bir Bare Bones programı yazın.

Doclingo https://www.doclingo.ai

4. Aşağıdaki her durumda, belirtilen etkinliği g erçekleştiren bir programsırası yazın:

   a. X'in değeri çift ise Z'ye 0 atayın;
   aksi takdirde Z'ye 1 atayın.

   b. 0'dan X'ye kadar olan tam sayıların toplamını hesaplayın.

5. Değerini Y ile bölen bir Bare Bones rutini yazın. Herhangi bir kalanı dikkate almay ın; yani, 1'in 2'ye bölümü 0 verir ve 5'in 3'e bölümü 1 verir.

6. Aşağıdaki Bare Bones programı tarafından hesa planan fonksiyonu tanımlayın, fonksiyonun girdilerinin X ve Y ile temsil edildiğini ve çıktısının Z ile temsil edild iğini varsayarak:

```
X'i Z'ye kopyala;
Y'yi Aux'a kopyala;
Aux 0 olmadığı sürece;
   Z'yi azalt;
   Aux'u azalt;
son;
```

7. Aşağıdaki Bare Bones programı tarafından hesa planan fonksiyonu tanımlayın, fonksiyonun girdilerinin X veY ile temsil edildiğini ve çıktısının Z ile temsil edild iğini varsayarak:

```
Z'yi temizle;
X'i Aux1'e kopyala;
Y'yi Aux2'ye kopyala;
Aux1 0 olmadığı sürece;
   Aux2 0 olmadığı sürece;
      Z'yi azalt;
      Aux2'yi azalt;
   son;
   Aux1'i azalt;
son;
```

8. X ve Y değişkenlerinin dışlayıcısını hesaplaya n bir Bare Bones programı yazın, sonucu Z değişkeninde bırakın. X ve Y'nin yalnızca 0 ve 1 tam sayılarıyla başladığını varsayabilirsiniz.

9. Bir BareBones programında talimatların tam sayı değerleriyle etiketlenmesine izin v erirsekve while döngü yapısınıif name not 0 goto label; biçiminde temsil edilenkoş ullu dal ile değiştirirsek,

   burada `name` herhangi bir değişken ve *label* b aşka yerlerde birtalimatı etiketlemek için ku llanılan bir tam sayı değeri ise,yeni dil hala evrense l bir programlama dili olacaktır.

10. Bu bölümde, ifadenin nasıl isim1'i isim2'ye kopyala;
    Bare Bones'ta simüle edilebileceğini gördük. Oifadenin , Bare Bones'taki while döngü yapısı, aşağıdaki biçi mde ifade edilen bir posttest döngüsü ile değiştirilse bile nasıl simüle edilebileceğini gösterin.
    tekrar ... kadar (isim 0'a eşit olana kadar)

11. Bare Bones dilinin, while ifadesi, aşağıda ki biçimde ifade edilen bir posttest döng üsü ile değiştirilse bile evrensel bir dil olara k kalacağını gösterin.
    tekrar ... kadar (isim 0'a eşit olana kadar)

12. Bir kez başlatıldığında, banttaki tek bir hücreden faz lasını kullanmayacak ancak asla durma durumuna ulaş mayacak bir Turing makinesi tasarlayın.

13. Mevcut hücrenin solundaki tüm hücrelere 0 yerleşt iren bir Turing makinesi tasarlayın, bu hücre bir as terisk içeren bir hücreye ulaşana kadar.

14. Bir Turing makinesinin banttaki 0 ve 1'lerden o luşan bir deseninin her iki ucunda asteriskl erle sınırlı olduğunu varsayalım. Bu deseni, maki nenin mevcut hücresinin desenin sağ ucunda ki asterisk olduğu varsayımıyla, bir hücre sola döndüren bir Turing makinesi tasarlayın.

15. Mevcut hücresinin (bir asterisk içeren) v e soldaki ilk asterisk arasında bulduğu 0 v e 1'lerin desenini tersine çeviren bir Turing makinesi tasarlayın.

16. Church–Turing tezini özetleyin.

17. Aşağıdaki Bare Bones programı kendiliği nden sona eriyor mu? Cevabınızı açıklayın.

```
X'i Y'ye kopyala;
incr Y;
Y dahil;
X 0 değilken;
  decr X;
  decr X;
  decr Y;
  decr Y;
son;
decr Y;
Y 0 olmadığı sürece;

  decr Y;
son;
X 0 değilken devam et;
son;
```

Doclingo https://www.doclingo.ai

18. Aşağıdaki Bare Bones programı kendiliğinden sona eriyor mu? Cevabınızı açıklayın.

```
X 0 değilken devam et;
son;
```

19. Aşağıdaki Bare Bones programı kendiliğinden sona eriyor mu? Cevabınızı açıklayın.

```
X 0 değilken;
  decr X;
son;
```

20. Aşağıdaki ifade çiftinin geçerliliğini analiz edin:

Bir sonraki ifade doğrudur.
Önceki ifade yanlıştır.

21. "Bir gemideki aşçı, yalnızca kendisi için pişirmeyenler için pişirir." ifadesinin geçerliliğini analiz edin. (Aşçı için kim pişirir?)

22. Her kişinin ya doğruyu söyleyen ya da yalancı olduğu bir ülkede olduğunuzu varsayın. (Doğruyu söyleyen her zaman doğruyu söyler, yalancı her zaman yalan söyler.) O kişinin doğruyu söyleyip söylemediğini tespit etmenizi sağlayacak tek bir soru nedir?

23. Turing makinelerinin teorik bilgisayar bilimi alanındaki önemini özetleyin.

24. Durdurma probleminin teorik bilgisayar bilimi alanındaki önemini özetleyin.

25. Bir grup insanın belirli bir tarihte doğum günü olan birinin olup olmadığını öğrenmeniz gerektiğini varsayın. Bir yaklaşım, üyeleri birer birer sormak olacaktır. Bu yaklaşımı alırsanız, hangi olayın gerçekleşmesi, böyle bir kişinin olduğunu size söyleyecektir? Hangi olay, böyle bir kişinin olmadığını size söylerdi? Şimdi, en az bir pozitif tam sayının belirli bir özelliğe sahip olup olmadığını öğrenmek istediğinizi varsayalım ve tam sayıları sistematik olarak birer birer test etme yaklaşımını uyguladığınızı düşünelim. Eğer gerçekten bazı tam sayılar bu özelliğe sahipse, bunu nasıl öğrenirsiniz? Anca k, eğer hiçbir tam sayı bu özelliğe sahip değilse, bunu nasıl öğrenirsiniz? Bir varsayımın doğru olup olmadığını test etme görevi, onun yanlış olup olmadığını test etme göreviyle mutlaka simetrik midir?

26. Bir listedeki belirli bir değeri arama problemi polinom bir problem midir?
Cevabınızı gerekçelendirin.

27. Verilen bir pozitif tam sayının asal olup olmadığını belirlemek için bir algoritma tasarlayın. Çözümünüz verimli mi? Çözümünüz polinom mu yoksa polinom olmayan bir çözüm mü?

28. Bir probleme polinom bir çözüm her zaman üstel bir çözümden daha mı iyidir? Cevabınızı açıklayın.

29. Bir problemin polinom bir çözümü olması, her zaman pratik bir süre içinde çözülebileceği anlamına mı gelir? Cevabınızı açıklayın.

30. Charlie Programcı, bir grubu (çift sayıda insan) iki ayrı alt gruba eşit boyutta bölme problemiyle karşı karşıyadır, böylece her alt grubun toplam yaşları arasındaki fark mümkün olduğunca büyük olsun. Tüm olası alt grup çiftlerini oluşturmayı, her çiftin yaş toplamları arasındaki farkı hesaplamayı ve en büyük farka sahip çifti seçmeyi önerir.

Öte yandan, Mary Programcı, orijinal grubun önce yaşa göre sıralanmasını ve ardından sıralanan grubun genç yarısından bir alt grup ve yaşlı yarısından diğer bir alt grup oluşturularak iki alt gruba bölünmesini önerir. Bu çözümlerin her birinin karmaşıklığı nedir? Problem kendisi polinom, NP veya polinom olmayan bir karmaşıklıkta mıdır?

31. Bir listenin tüm olası düzenlemelerini oluşturup ardından istenen düzenlemeye sahip olanı seçme yaklaşımı neden bir listeyi sıralamak için tatmin edici bir yol değildir?

32. Varsayalım ki bir piyango, her biri 1 ile 50 arasında olan dört tam sayı değerini doğru bir şekilde seçmeye dayanmaktadır. Ayrıca, büyük ikramiyenin o kadar büyüdüğünü varsayalım ki, her olası kombinasyon için ayrı bir piyango bileti almak kârlı hale geliyor. Tek bir bileti almak bir saniye sürüyorsa, her kombinasyon için bir bilet almak ne kadar zaman alır? Piyango dört yerine beş numara seçmeyi gerektiriyorsa, zaman gereksinimi nasıl değişir? Bu problemin bu bölümdeki materyalle ne ilgisi var?

Doclingo https://www.doclingo.ai

33. Aşağıdaki algoritma deterministik midir? Cevabınızı açıklayın.

```
prosedürmystery (Sayı)
eğer(Sayı > 5)
   o zaman(cevap "evet")d
   eger (5'ten küçük bir değer seçin ve bu sa
         yıyı cevap olarak verin)
```

34. Aşağıdaki algoritma deterministik midir? Cevabınızı açıklayın.

```
Dümdüz devam et.
Üçüncü kavşakta, köşede duran kişiye sağa
   mı yoksa sola mı dönmeniz gerektiğ
   ini sorun.
O kişinin talimatlarına göre dönün.

İki blok daha sürün ve orada durun.
```

35. Aşağıdaki algoritmadaki belirsizlik noktalarını belirleyin:

```
1 ile arasında üç sayı seçin
   100.
eğer(seçilen sayıların toplamı
     greater than 150)
   o zaman("evet" yanıtını ver)
   aksi takdirde(seçilenlerden birini seçin
         ve o sayıyı
         cevap olarak verin)
```

36. Aşağıdaki algoritmanın polinom veya polinom olmayan zaman karmaşıklığı var mı? Cevabınızı açıklayın.

```
prosedürmystery (ListOfNumbers)ListOfN
umbers'dan bir sayı koleksiyonu seçin.

eğer(o koleksiyondaki sayılar
      125'e ekleniyorsa)
   o zaman("evet" yanıtını ver)
   aksi takdirde(cevap vermeyin)
```

37. Aşağıdaki problemlerden hangileriP sınıfındadır?

a. Karmaşıklığı olan bir problem n^2
b. Karmaşıklığı olan bir problem $3n$
c. Karmaşıklığı olan bir problem n^2+2n
d. Karmaşıklığı olan bir problem $n!$

38. Bir problemin polinom problemi olduğunu ve bir nondeterministic polinom problemi olduğunu belirtmek arasındaki farkı özetleyin.

39. Hem P sınıfında hem de NP sınıfında olan bir probleme örnek verin.

40. Aynı problemi çözmek için iki algoritma verildiğini varsayalım. Bir algoritmanın zaman karmaşıklığı n^4 ve diğerinin zaman karmaşıklığı 4n. Hangi boyutlardaki girdilerde birincisi ikincisinden daha verimlidir?

41. 15 şehrin bulunduğu ve herhangi iki şehrin benzersiz bir yol ile bağlı olduğu bir bağlamda seyahat eden satıcı problemini çözmekle karşı karşıya kaldığımızı varsayalım. Şehirler arasında kaç farklı yol olacaktı? Bir yolun uzunluğunun bir mikrosaniyede hesaplanabileceğini varsayarsak, bu yolların toplam uzunluğunu hesaplamak ne kadar sürerdi?

42. İsimler arasında kaç karşılaştırma yapılırbirleştirme sıralama algoritması (Şekil 12.9ve 12.8) Alice, Bob, Carol,ve David listesine uygulandığında? Eğer listeAlice, Bob, Carol, David ve Elaine isekaç karşılaştırma gereklidir?

43. Şekil 12.12'de temsil edilen her bir kategori için bir problem örneği verin.

44. x2+y2=n biçimindeki denklemler içintam sayı çözümleri bulmak üzere bir algoritmatasarlayın, burada n verilen bir pozitif tam sayıdır.Algoritmanızın zaman karmaşıklığını belirleyin.

45. NP-tamam kategorisine giren bir diğer problem çanta problemi olup, bir listeden hangi sayılarınbelirli bir değerin toplamını oluşturduğunu bulmaproblemidir. Örneğin, 257, 388 ve782 sayıları listedeki girişlerdir

```
642 257 771 388 391 782 304
```

toplamı 1427 olan. Toplamı 1723 olan girişleri bulun. Hangi algoritmayı uyguladınız? O algoritmanın karmaşıklığı nedir?

46. Seyahat eden satıcı problemi ile çanta problemi (Problem 45'e bakın) arasındaki benzerlikleri belirleyin.

47. Bir listeyi sıralamak için aşağıdaki algoritmaya baloncuk sıralama denir. Baloncuk sıralama, n girişli bir listeye uygulandığında kaç karşılaştırma gerektirir?

```
prosedürBubbleSort (Liste)
Sayıcı←1;
iken(Sayıcı < liste içindeki giriş sayısı) yap

   [N ←liste içindeki giriş sayısı;
    iken (N>1) yap
```

Doclingo https://www.doclingo.ai

```
(eğer(N'inci liste girişi, ondan önceki girişten
        küçükse)
   o zaman(N'inci girişi
           önceki girişle yer değiştir);
 N←N    − 1
)
]
```

48. RSA açık anahtar şifrelemesini kullanarak mesajı 110'ü açık anahtarlar n = 91 ve e = 5 ile şifrele.

49. RSA açık anahtar şifrelemesini kullanarakmesajı 111'i özel anahtarlar n = 133ve$d$ = 5 ile deşifrele.

50. RSA algoritmasına dayanan bir açık anahtar şifreleme sisteminin açık anahtarlarını n n = 77 ve e = 7 olduğunu biliyorsanız, özel anahtarlar nelerdir? Bu problemi makul bir süre içindeçözmenizi sağlayan nedir?

51. 107,531'in çarpanlarını bulun. Bu sorun bu bölümle nasıl ilişkilidir?

52. Pozitif tam sayı n 2'den n'nin karekköküne kadar olan tam sayı çarpanları yoksa nesonuç çıkarılabilir? Bu, pozitif bir tam sayının çarpanlarını bulmagörevine dair size ne anlatıyor?

Sosyal Sorunlar

Aşağıdaki sorular, bilişim alanıyla ilişkili etik/sosyal/hukuki sorunlara bir rehber olarak tasarlanmıştır. Amaç, bu soruları yalnızca yanıtlamak değildir. Ayrıca, neden böyle yanıt verdiğinizi ve gerekçelerinizin bir sorudan diğerine tutarlı olup olmadığını da düşünmelisiniz.

1. Bir problemi çözmek için en iyi algoritmanın 100 yıl sürmesi gerekiyorsa, bu problemi çözülmesi mümkün olarak mı değerlendirirsiniz? Neden?

2. Vatandaşların, hükümet ajanslarının izlenmesini engelleyecek şekildemesajları şifreleme hakkına sahip olmaları gerekir mi? Cevabınız "doğru"yasa uygulamasını sağlıyor mu? "Doğru" yasa uygulamasının ne olduğunu kimkarar vermelidir?

3. Eğer insan zihni algoritmik bir cihazsa, Turing'in tezi insanlık açısından ne gibi sonuçlar doğurur? Turing makinelerinin insan zihninin hesaplama yeteneklerini kapsayıp kapsamadığına ne ölçüde inanıyorsunuz?

4. Farklı hesaplama modellerinin (sonlu tablolar, cebirsel formüller, Turing makineleri vb.) farklı hesaplama yeteneklerine sahip olduğunu gördük. Farklı organizmaların hesaplama yeteneklerinde farklılıklar var mı? Farklı insanların hesaplama yeteneklerinde farklılıklar var mı? Eğer öyleyse, daha yüksek yeteneklere sahip insanların bu yetenekleri kullanarak daha yüksek yaşam standartlarına ulaşabilmesi gerekir mi?

5. Bugün çoğu şehrin yol haritalarını sağlayan web siteleri var. Bu siteler belirli adresleri bulmaya yardımcı olur ve küçük mahallelerin düzenini görüntülemek için yakınlaştırma yetenekleri sunar. Bu gerçeklikten yola çıkarak, aşağıdaki kurgusal diziyi düşünün. Diyelim ki bu harita siteleri, benzer yakınlaştırma yeteneklerine sahip uydu fotoğrafları ile geliştirildi. Bu yakınlaştırma yeteneklerinin, bireysel binaların ve çevresindeki manzaranın daha ayrıntılı bir görüntüsünü vermek için artırıldığını varsayalım. Bu görüntülerin, gerçek zamanlı video içerecek şekilde geliştirildiğini varsayalım. Bu noktada, başkaları kendi evinizin içinde 24 saat boyunca sizi izleyebilir. Bu ilerlemenin hangi noktasında gizlilik haklarınız ilk kez ihlal edildi? Bu ilerlemenin hangi noktasında gizlilik haklarımızın ihlal edildiğini düşünüyorsunuz?

Doclingo https://www.doclingo.ai

mevcut casus uydu teknolojisinin ötesinde mi? Bu senaryo ne ölçüde kur gusal?

6. Bir şirket bir şifreleme sistemi geliştirip patentini alırsa, şirketin ulusal hü kümetinin bu sistemi ulusal güvenlik adına istediği gibi kullanma hakkı ol malı mı? Şirketin ulusal hükümetinin, ulusal güvenlik adına şirketin siste min ticari kullanımını kısıtlama hakkı olmalı mı? Şirket çok uluslu bir orga nizasyon ise ne olur?

7. Bir ürün satın aldığınızı ve iç yapısının şifreli olduğunu varsayalım. Temel yapıyı çözme hakkına sahip misiniz? Eğer öyleyse, bu bilgiyi ticari bir şeki lde kullanma hakkına sahip misiniz? Peki ya ticari olmayan bir şekilde? Şif releme gizli bir şifreleme sistemi kullanılarak yapıldıysa ve siz bu sırrı keş fettiyseniz, o sırrı paylaşma hakkına sahip misiniz?
8. Bazı yıllar önce filozof John Dewey (1859–1952) "sorumlu teknoloji" terimi ni tanıttı. "Sorumlu teknoloji" olarak neyi düşündüğünüze dair bazı örnek ler verin. Örneklerinize dayanarak "sorumlu teknoloji" tanımınızı oluştur un. Toplum son 100 yılda "sorumlu teknoloji" uyguladı mı? Bunun sağlan ması için eylem alınmalı mı? Eğer öyleyse, hangi eylemler? Eğer hayırsa, neden?

Ek Okuma

Garey, M. R. ve D. S. Johnson. *Bilgisayarlar ve Çözülmesi Zor Problemler.* New York: W. H. Freemann, 1979.

Hamburger, H. ve D. Richards. *Bilgisayar Bilimi için Mantık ve Dil Modelleri*. Englewood Cliffs, NJ: Prentice-Hall, 2002.

Hofstadter, D. R. *Gödel, Escher, Bach: Sonsuz Altın Örgü.* St. Paul, MN: Bağbozumu, 1980.

Hopcroft, J. E., R. Motwani ve J. D. Ullman. *Otomat Teorisine Giriş, Diller ve Hesaplama*, 3. baskı. Boston, MA: Addison-Wesley, 2007.

Lewis, H. R. ve C. H. Papadimitriou. *Hesaplama Teorisi Unsurları*, 2. baskı. En glewood Cliffs, NJ: Prentice-Hall, 1998.

Rich E. Otomatlar. *Hesaplanabilirlik ve Karmaşıklık: Teori ve Uygulama.* Upper Saddle River, NJ: Prentice-Hall, 2008.

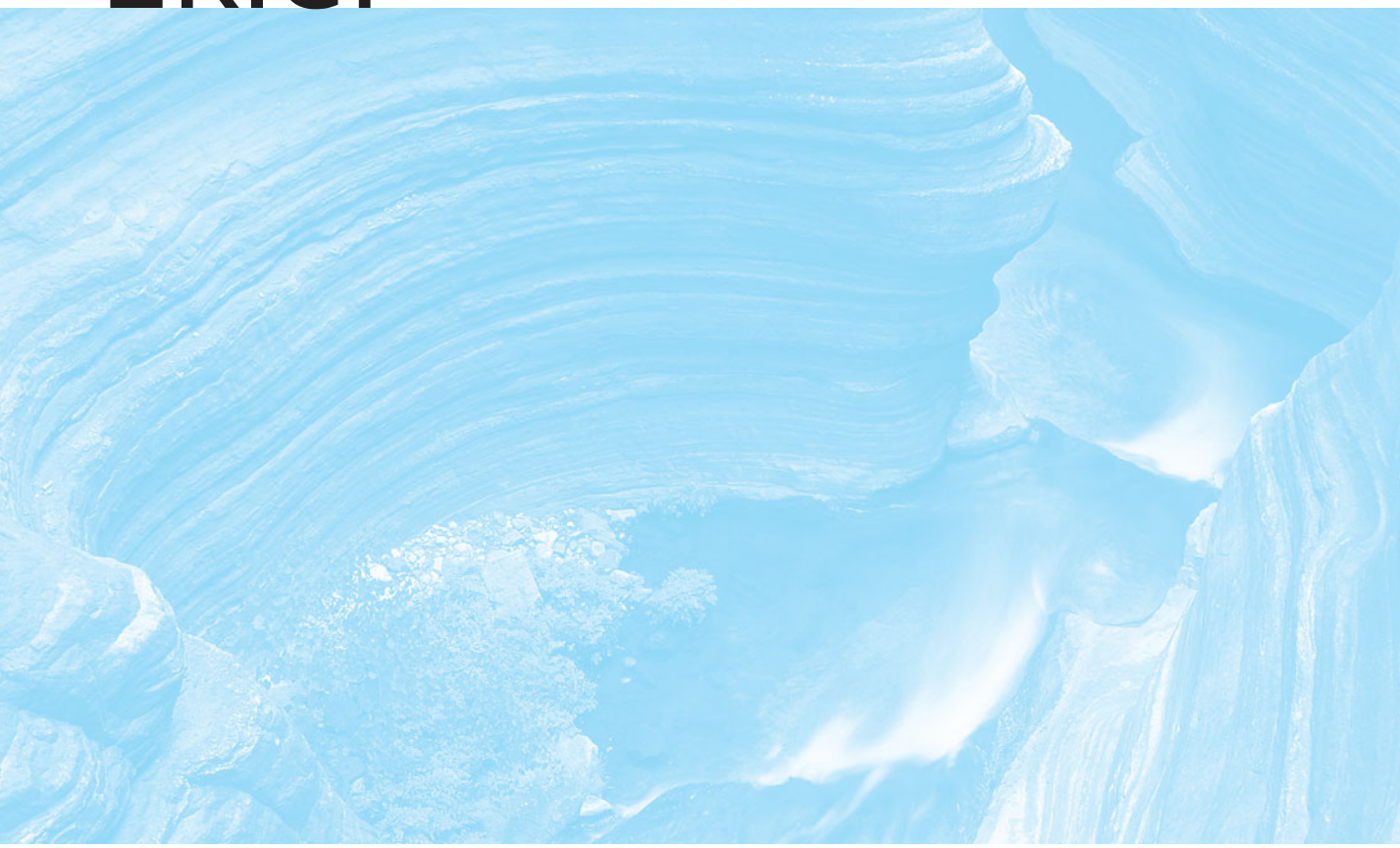
Sipser, M. *Hesaplama Teorisine Giriş.* Boston: PWS, 1996.

Smith, C. ve E. Kinber. *Bilgisayar Bilimi Teorisi: Nazik Bir Giriş.* Englewood Cliffs, NJ: Prentice-Hall, 2001.

Sudkamp, T. A. *Diller ve Makineler: Bilgisayar Biliminin Teorisine Giriş*, 3. baskı. Boston, MA: Addison-Wesley, 2006.

Doclingo https://www.doclingo.ai

Ekler



A ASCII

B İki'ninTamamlayıcı Temsillerini Manipüle Etmek İçin DevrelerC Basit Bir Makine DiliD Yüksek Düzeyli Programlama Dilleri

E Tekrarlıve Özyinelemeli Yapıların EşdeğerliliğiFSorulara ve Alıştırmalara Cevaplar

Doclingo https://www.doclingo.ai

Bu sayfa kasıtlı olarak boş bırakılmıştır

Doclingo https://www.doclingo.ai

ek

A

ASCII

Aşağıda, her bit deseninin soluna 0 eklenerek günümüzde yaygın olarak kull anılan 8 bit desenini oluşturduğu ASCII kodunun kısmi bir listesi verilmiştir. Her 8 bit deseninin onaltılık değeri üçüncü sütunda verilmiştir.

Sembol	ASCII	Altıgen	Sembol	ASCII	Altıgen	Sembol	ASCII	Altıgen
satır sonu	00001010	0A	>	00111110	3E	^	01011110	5E
satır sonu	00001011	0B	?	00111111	3F	_	01011111	5F
boşluk	00100000	20	@	01000000	40	`	01100000	60
!	00100001	21	A	01000001	41	a	01100001	61
”	00100010	22	B	01000010	42	b	01100010	62
#	00100011	23	C	01000011	43	c	01100011	63
$	00100100	24	D	01000100	44	d	01100100	64
%	00100101	25	E	01000101	45	e	01100101	65
&	00100110	26	F	01000110	46	f	01100110	66
’	00100111	27	G	01000111	47	g	01100111	67
(	00101000	28	H	01001000	48	h	01101000	68
)	00101001	29	I	01001001	49	i	01101001	69
*	00101010	2A	J	01001010	4A	j	01101010	6A
+	00101011	2B	K	01001011	4B	k	01101011	6B
’	00101100	2C	L	01001100	4C	l	01101100	6C
-	00101101	2D	M	01001101	4D	m	01101101	6D
.	00101110	2E	N	01001110	4E	n	01101110	6E
/	00111111	2F	O	01001111	4F	o	01101111	6F
0	00110000	30	P	01010000	50	p	01110000	70
1	00110001	31	Q	01010001	51	q	01110001	71
2	00110010	32	R	01010010	52	r	01110010	72
3	00110011	33	S	01010011	53	s	01110011	73
4	00110100	34	T	01010100	54	t	01110100	74
5	00110101	35	U	01010101	55	u	01110101	75
6	00110110	36	V	01010110	56	v	01110110	76
7	00110111	37	W	01010111	57	w	01110111	77
8	00111000	38	X	01011000	58	x	01111000	78
9	00111001	39	Y	01011001	59	y	01111001	79
:	00111010	3A	Z	01011010	5A	z	01111010	7A
;	00111011	3B	[	01011011	5B	{	01111011	7B
<	00111100	3C	\	01011100	5C	|	01111100	7C
=	00111101	3D	]	01011101	5D	}	01111101	7D

Doclingo https://www.doclingo.ai

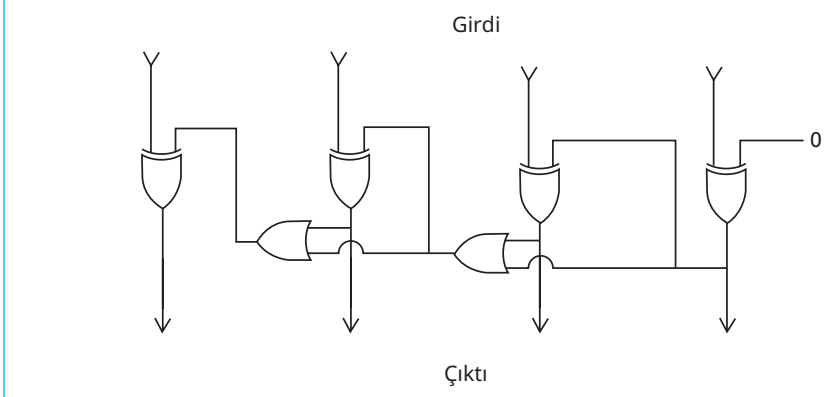
ek

B

İki'nin Tamamlayıcısını Manipüle Etmek İçin Devreler

Bu ek, iki'nin tamamlayıcı notasyonunda temsil edilen değerleri olumsuzlaştırmak ve toplamak için devreler sunmaktadır. Dört bitlik bir iki'nin tamamlayıcı temsilini o değerin negatif temsilini dönüştüren Şekil B.1'deki devre ile başlıyoruz. Örneğin, 3'ün iki'nin tamamlayıcı temsili verildiğinde, devre 3 için temsili üretir. Bunu, metinde sunulan aynı algoritmayı izleyerek yapar. Yani, bir 1 kopyalanana kadar sağdan sola doğru deseni kopyalar ve ardından her kalan biti, girişi çıkışa taşırken tamamlar. En sağdaki XOR kapısının bir girişi 0 olarak sabitlendiğinden, bu kapı yalnızca diğer girişini çıkışa iletecektir. Ancak, bu çıkış da bir sonraki XOR kapısının girişlerinden biri olarak sola iletilir. Eğer bu çıkış 1 ise, bir sonraki XOR kapısı, çıkışa geçerken giriş bitini tamamlayacaktır. Dahası, bu 1 de OR kapısı aracılığıyla sola iletilecek ve bir sonraki kapıyı etkileyecektir.Bu şekilde, çıkışa kopyalanan ilk 1 de sola iletilecek ve burada tüm kalan bitlerin çıkışa taşınırken tamamlanmasına neden olacaktır.

Şekil B.1İki tamamlayıcı desenini iptal eden bir devre



Doclingo https://www.doclingo.ai

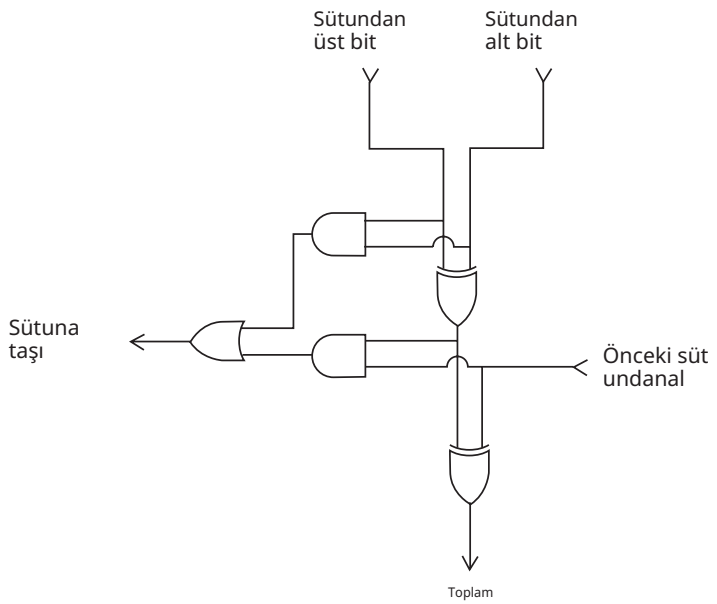
Şimdi, iki değerinin iki tamamlayıcı notasyonunda temsil edildiği toplam a sürecini ele alalım. Özellikle, problemi çözerken

```
  0110
+ 1011
```

her sütun için sağdan sola doğru ilerliyoruz ve her sütun için aynı algoritmay ı uyguluyoruz. Böylece, bu tür bir problemin bir sütununu toplamak için bir d evre elde ettiğimizde, tek sütun devresini tekrar ederek birçok sütunu topla mak için bir devre inşa edebiliriz.

Birden fazla sütun toplama probleminde tek bir sütunu toplamak için alg oritma, mevcut sütundaki iki değeri toplamak, bu toplamı önceki sütundan g elen herhangi bir taşımaya eklemek, bu toplamın en az anlamlı bitini cevaba yazmak ve herhangi bir taşıma değerini bir sonraki sütuna aktarmaktır. Şekil B.2'deki devre bu aynı algoritmayı izler. Üst XOR kapısı, iki giriş bitinin topla mını belirler. Alt XOR kapısı, bu toplamı önceki sütundan taşınan değere ekle r. İki AND kapısı ve OR kapısı, herhangi bir taşıma değerini sola aktarır. Özelli kle, bu sütundaki orijinal iki giriş biti 1 ise veya bu bitlerin toplamı ve taşıma 1 ise, 1'lik bir taşıma üretilir.

Şekil B.2Birden fazla sütunlu toplama probleminde tek bir sütun eklemek için bir devre

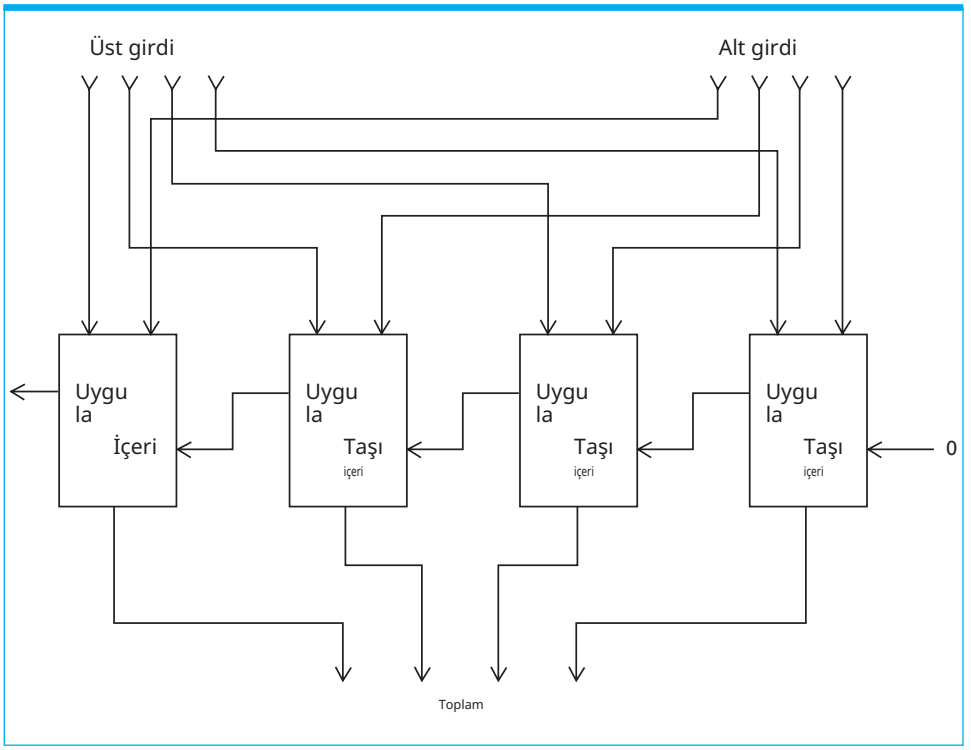


Doclingo https://www.doclingo.ai

Şekil B.3, bu tek sütun devresinin kopyalarının, dört bitlik iki'nin tamamlayıcısı siste minde temsil edilen iki değerin toplamını hesaplayan bir devre üretmek için nasıl kullan ılabileceğini göstermektedir. Her dikdörtgen, tek sütun toplama devresinin bir kopyasın ı temsil eder. En sağdaki dikdörtgene verilen taşıma değerinin her zaman 0 olduğunu u nutmayın çünkü önceki bir sütundan taşıma yoktur. Benzer şekilde, en soldaki dikdörtg enden üretilen taşıma dikkate alınmaz.

Şekil B.3'teki devre, taşıma bilgisi en sağdan en sola doğru yayılması ger ektiğinden *dalgalı toplayıcı* olarak bilinir. Bileşim açısından basit olmasına ra ğmen, bu tür devreler, sütunlar arası yayılmayı minimize eden daha akıllı ver siyonlar, örneğin önceden taşıma toplayıcıdan daha yavaş işlevlerini yerine g etirir. Bu nedenle, Şekil B.3'teki devre, amacımız için yeterli olmasına rağmen , günümüz makinelerinde kullanılan devre değildir.

Şekil B.3Şekil B.2'deki devrenin dört kopyasını kullanarak iki değeri iki'nin tamamlayıcısı notasyonun da toplamak için bir devre



Doclingo https://www.doclingo.ai

ek

C

Basit Bir Makine Dili

Bu ek bölümde basit ama temsilci bir makine dilini sunuyoruz. Makinenin mi marisini açıklayarak başlıyoruz.

Makinenin Mimarisi

Makine, 0'dan F'ye (onaltılık) numaralandırılmış 16 genel amaçlı kayda sahipt ir. Her kayıt bir bayt (sekiz bit) uzunluğundadır. Talimatlar içinde kayıtları tan ımlamak için, her kayda kendine özgü dört bitlik desen atanmıştır; bu desen kaydın numarasını temsil eder. Böylece kayıt 0, 0000 (onaltılık 0) ile tanımlanı rken, kayıt 4, 0100 (onaltılık 4) ile tanımlanır.

Makinenin ana belleğinde 256 hücre bulunmaktadır. Her hücre, 0 ile 255 arasındaki bir tam sayıdan oluşan benzersiz bir adrese atanmıştır. Bu nedenl e, bir adres sekiz bitlik bir desenle 00000000'dan

11111111 (veya 00 ile FF arasındaki bir onaltılık değerle) temsil edilebilir.

Kaydırmalı nokta değerlerinin, Bölüm 1.7'de tartışılan ve Şekil 1.26'da özetlen en sekiz bitlik formatta saklandığı varsayılmaktadır.

Makinenin Dili

Her makine talimi iki bayt uzunluğundadır. İlk 4 bit op-kodu sağlar; son 12 bi t ise işlem alanını oluşturur. Aşağıdaki tablo, her birinin kısa bir tanımıyla birl ikte onaltılık notasyonda talimatları listelemektedir. R, S ve T harfleri, talimatı n belirli uygulamasına bağlı olarak değişen bir kayıt tanımlayıcısını temsil ed en alanlarda onaltılık rakamların yerini almak için kullanılır. X ve Y harfleri ise bir kaydı temsil etmeyen değişken alanlarda onaltılık rakamların yerini alma k için kullanılır.

Op-kodu	Operatör	Açıklama
1	RXY	R kaydını, adresi XY olan bellek hücresinde bulunan bit de senine yükleyin. *Örnek: 14A3, A3 adresinde bulunan bellek hücresinin içeriğinin* 4 kaydına yerleştirilmesine neden olur.
2	RXY	R kaydını, XY bit desenine yükleyin. *Örnek:* 20A3*, A3 değerinin* 0 *kaydına yerleştirilmesine neden olur.*

Doclingo https://www.doclingo.ai

3	RXY	R kaydında bulunan bit desenini, adresi XY olan bellek hücresine kaydedin. *Örnek:* 35B1, *5 kaydının içeriğininB1 adresindeki bellek hücresine yerleştirilmesine neden olur.*
4	0RS	R kaydında bulunan bit desenini S kaydına taşıyın. *Örnek:* 40A4 A kaydının içeriğinin 4 kaydına kopyalanmasına neden olur.
5	RST	S ve T kayıtlarındaki bit desenlerini, sanki kompleman temsilcileriymiş gibi toplayın ve sonucu R kaydında bırakın. *Örnek:* 57262 *ve* 6 *kayıtlarındaki ikili değerlerin toplanmasına ve toplamın* 7 *kaydına yerleştirilmesine neden olur.*
6	RST	S ve T kayıtlarındaki bit desenlerini, sanki kayan nokta notasyonunda değerleri temsil ediyormuş gibi toplayın ve sonucu R kaydında bırakın. *Örnek:* 634E4 *ve E kayıtlarındaki değerlerinkayan nokta değerleri olarak toplanmasına ve sonucun* 3 *kaydına yerleştirilmesine neden olur.*
7	RST	S ve T kayıtlarındaki bit desenlerini OR işlemi yaparaksonucu R kaydına yerleştir. *Örnek: 7CB4 B kaydının içeriği ile 4 kaydının içeriğini OR işlemi yaparak* C kaydına yerleştirir.
8	RST	S ve T kayıtlarındaki bit desenlerini AND işlemi yaparaksonucu R kaydına yerleştir. *Örnek: 8045 4 ve 5 kayıtlarının içeriğini AND işlemi yaparak* 0 kaydına yerleştirir.
9	RST	S ve T kayıtlarındaki bit desenlerini EXCLUSIVE OR işlemi yaparaksonucu R kaydına yerleştir. *Örnek:* 95F3 *F kaydının içeriği ile* 3 *kaydının içeriğini EXCLUSIVE* O*R işlemi yaparak5 kaydına yerleştirir.*
A	R0X	R kaydındaki bit desenini X kez sağa bir bit döndür. Her seferinde düşük sıralı uçta başlayan biti yüksek sıralı uca yerleştir. *Örnek:* A403 *4 kaydının içeriğini dairesel olarak sağa* 3 *bit döndürür.*
B	RXY	R kayıtlarındaki bit deseninin, kayıt numarası 0'daki bit deseni ile eşit olması durumunda, XY adresindeki bellek hücresinde bulunan talimata JUMP yapın.Aksi takdirde, normal yürütme sırasına devam edin. (Atlama, yürütme aşamasında XY'yi program sayacına kopyalayarak gerçekleştirilir.) *Örnek:* B43C önce register 4'ün içeriğini register 0'ün içeriği ile karşılaştıracaktır. Eğer ikisi eşitse, desen 3C program sayacına yerleştirilecektir, böylece bir sonraki çalıştırılacak komut o bellek adresinde bulunan komut olacaktır. Aksi takdirde, hiçbir şey yapılmayacak ve program yürütmesi normal sırasına devam edecektir.
C	000	Yürütmeyi Durdur. *Örnek:* C000 program yürütmesini durduracaktır.

Doclingo https://www.doclingo.ai

ek

D

Yüksek Düzey Programlama Dilleri

Bu ek, Bölüm 6'da örnek olarak kullanılan her bir dilin kısa bir arka planını iç ermektedir.

Orada

Ada dili, Charles Babbage'ın savunucusu ve şair Lord Byron'ın kızı Augusta A da Byron (1815–1851) adını taşımaktadır. ABD Savunma Bakanlığı'nın yazılım geliştirme ihtiyaçları için tek, genel amaçlı bir dil elde etme girişimiyle gelişti rilmiştir. Ada'nın tasarımı sırasında önemli bir vurgu, füzelerin yönlendirilme si sistemleri, binalardaki çevre kontrol sistemleri ve otomobiller ile küçük ev aletleri gibi daha büyük makinelerin bir parçası olarak kullanılan gerçek zam anlı bilgisayar sistemleri için programlama özelliklerini içermeye yönelik olm uştur. Bu nedenle, Ada, paralel işleme ortamlarında etkinlikleri ifade etme öz ellikleri ile uygulama ortamında ortaya çıkabilecek özel durumları (istisnalar olarak adlandırılan) ele almak için uygun teknikler içermektedir. Başlangıçta bir zorlayıcı dil olarak tasarlanmış olmasına rağmen, Ada'nın daha yeni sürü mleri nesne yönelimli paradigmayı benimsemiştir.

Ada dilinin tasarımı, güvenilir yazılımların verimli bir şekilde geliştirilmesine yol açan özelliklere sürekli olarak vurgu yapmıştır; bu, Boeing 777 uçağındaki t üm iç kontrol yazılımlarının Ada dilinde yazılmış olmasıyla örneklendirilmiştir. B u, Ada'nın SPARK dilinin geliştirilmesinde başlangıç noktası olarak kullanılması nın başlıca nedenlerinden biridir; bu durum Bölüm 5'te belirtilmiştir.

C

C dili, 1970'lerin başında Dennis Ritchie tarafından Bell Laboratuvarları'nda geli ştirilmiştir. Başlangıçta sistem yazılımı geliştirmek için bir dil olarak tasarlanmış olmasına rağmen, C, programlama topluluğu arasında popülerlik kazanmış ve Amerikan Ulusal Standartlar Enstitüsü tarafından standartlaştırılmıştır.

C, başlangıçta yalnızca makine dilinden bir adım yukarı olarak düşünülmüştü. Sonuç olarak, sözdizimi, bazı temel öğeleri C'de özel sembollerle temsil eden diğer yüksek seviyeli dillere kıyasla kısadır. Bu kısalık, karmaşık algoritmaları n verimli temsillerine olanak tanır; bu da C'nin popülaritesinin başlıca nedenl erinden biridir. (Genellikle, özlü bir temsil uzun bir temsilden daha okunabilir .)

Doclingo https://www.doclingo.ai

C++

C++ dili, Bjarne Stroustrup tarafından Bell Laboratuvarları'nda C dilinin geliştirilmiş bir versiyonu olarak geliştirilmiştir. Amaç, nesne yönelimli paradigma ile uyumlu bir dil üretmektir. Bugün, C++ yalnızca kendi başına öne çıkan bir nesne yönelimli dil değil, aynı zamanda Java ve C# gibi iki diğer önde gelen nesne yönelimli dilin geliştirilmesi için bir başlangıç noktası olmuştur.

C#

C# dili, Microsoft tarafından .NET Çerçevesi'nde bir araç olarak geliştirilmiştir; bu, Microsoft sistem yazılımlarını çalıştıran makineler için uygulama yazılımları geliştirmek amacıyla kapsamlı bir sistemdir. C# dili, C++ ve Java'ya çok benzemektedir. Aslında, Microsoft'un C#'ı farklı bir dil olarak tanıtmasının nedeni, dil anlamında gerçekten yeni olması değil, farklı bir dil olarak Microsoft'un, diğer dillerle zaten ilişkili olan standartlar veya diğer şirketlerin mülkiyet haklarıkonusunda endişe duymadan dilin belirli özelliklerini özelleştirebilmesidir. Böylece C#'ın yeniliği, .NET Çerçevesi'ni kullanan yazılımlar geliştirmek için öne çıkan bir dil olmasında yatmaktadır. Microsoft'un desteğiyle, C# ve .NET Çerçevesi, yazılım geliştirme dünyasında yıllarca öne çıkan oyuncular olmayı vaat etmektedir.

Fortran

FORTRAN, FORmula TRANslator'ın kısaltmasıdır. Bu dil, geliştirilen ilk yüksek seviyeli dillerden biridir (1957'de duyurulmuştur) ve bilgisayar topluluğu için de geniş kabul gören ilk dillerden biridir. Yıllar içinde resmi tanımı birçok uzantı geçirmiştir; bu da günümüz FORTRAN dilinin orijinalinden çok farklı olduğu anlamına gelmektedir. Gerçekten de, FORTRAN'ın evrimini inceleyerek, programlama dili tasarımındaki araştırmaların etkilerini gözlemleyebilirsiniz. Başlangıçta bir zorunlu dil olarak tasarlanmış olmasına rağmen, FORTRAN'ın daha yeni sürümleri artık birçok nesne yönelimli özelliği kapsamaktadır. FORTRAN, bilimsel topluluk içinde popüler bir dil olmaya devam etmektedir. Özellikle, birçok sayısal analiz ve istatistik paketi FORTRAN'da yazılmakta ve muhtemelen yazılmaya devam edecektir.

Java

Java, 1990'ların başında Sun Microsystems tarafından geliştirilen nesne yönelimli bir dildir. Tasarımcıları, C ve C++'dan yoğun bir şekilde yararlandılar. Java'ya olan heyecan, dilin kendisinden değil, dilin evrensel uygulanabilirliğinden ve Java programlama ortamında mevcut olan çok sayıda önceden tasarlanmış şablondan kaynaklanmaktadır. Evrensel uygulanabilirlik, Java'da yazılmış bir programın geniş bir makine yelpazesinde verimli bir şekilde çalıştırılabileceği anlamına gelir; ve şablonların mevcudiyeti, karmaşık yazılımların görece kolaylıkla geliştirilebileceği anlamına gelir. Örneğin, applet ve servlet gibi şablonlar, Dünya Çapında Ağ için yazılım geliştirmeyi kolaylaştırır.

Doclingo https://www.doclingo.ai

ek

E

Yinelemeli ve Rekürsif Yapıların Eşdeğerliliği

Bu ek bölümde, Bölüm 11'deki Bare Bones dilimizi, Bölüm 4'teki yinelemelive rekürsif yapıların göreli gücü ile ilgili soruyu yanıtlamak için bir araç olarakk ullanıyoruz. Bare Bones'un yalnızca üç atama ifadesi (`clear`,`incr` ve `decr`)ve bir kontrol yapısı (bir while-end ifadesi çiftinden oluşturulmuş) içerdiğinihatır layın. Bu basit dil, bir Turing makinesi ile aynı hesaplama gücüne sahiptir;bu nedenle, Church-Turing tezini kabul edersek, algoritmik bir çözümü olan her sorunun Bare Bones'da ifade edilebilir bir çözümü olduğu sonucuna varabilir iz.

İteratif ve özyinelemeli yapıların karşılaştırmasındaki ilk adım, Bare Bone s'un iteratif yapısını özyinelemeli bir yapıyla değiştirmektir. Bunu, dildeki `while` veend ifadelerini kaldırarak ve bunların yerine bir Bare Bones program ını birimlere ayırma yeteneği ile birlikte bu birimlerden birini programın baş ka bir yerinden çağırma yeteneği sağlamamızla yapıyoruz. Daha kesin olarak , değiştirilmiş dildeki her programın sözdizimsel olarak ayrık bir dizi program biriminden oluşmasını öneriyoruz. Her programın,

```
ANA: başla;
        .
        .
        .
son;
```

(noktalı virgüller diğer Bare Bones ifadelerini temsil eder) ve belki de `MAIN`'e anlamsal olarak bağlı diğe r birimlerin yapısına sahip olmasını varsayıyoruz. Bu birimlerin yapısı

```
birim: başlangıç;
        .
        .
        .
geri dönmek;
```

(burada *`unit`*, değişken adlarıyla aynı sözdizimine sahip birimin adını temsil eder). Bu bölümlenmiş yapının anlamı, programın her zaman `MAIN` biriminin başında çalışmaya başlaması ve o birimin

Doclingo https://www.doclingo.ai

end ifadesine ulaşıldığında durmasıdır. `MAIN` dışındaki program birimleri, koşullu ifad e aracılığıyla prosedürler olarak çağrılabilir.

eğer*isim*not 0 gerçekleştir *birim;*

(burada isim herhangi bir değişken adını ve birim `MAIN` dışındaki program birim adlarından herhangi birini tem sil eder). Ayrıca, MAIN dışındaki birimlerin kendilerini özyinelemeli olarak çağırmalarına izin veriyoruz.

MAIN dışındaki birimlerin kendilerini özyinelemeli olarak çağırmalarına izin veriyoruz.

Bu ek özelliklerle, orijinal Bare Bones'da bulunan `while-end` yapısını sim üle edebiliriz. Örneğin, aşağıdaki biçimde bir Bare Bones programı

```
X 0 değilken;
S;
son;
```

(burada *S* herhangi bir Bare Bones ifadesi dizisini temsil eder) takım yapısı il e değiştirilebilir

```
MAIN:başla;
        eğer X not 0 gerçekleştir unitA;
    son;
unitA: başla;
        S;
        X 0 değilse unitA'yı gerçekleştir;
    return;
```

Sonuç olarak, değiştirilmiş dilin orijinal Bare Bones'un tüm yeteneklerine sahip olduğunu sonuçlandırıyoruz.

Ayrıca, değiştirilmiş dil kullanılarak çözülebilen herhangi bir problemin Bare B ones kullanılarak da çözülebileceği gösterilebilir. Bunu yapmanın bir yolu, değiştir ilmiş dilde ifade edilen herhangi bir algoritmanın orijinal Bare Bones'da nasıl yazıl abileceğini göstermektir. Ancak, bu, özyinelemeli yapıların Bare Bones'un `while-end` yapısıyla nasıl simüle edilebileceğine dair açık bir tanım gerektirir.

Amacımız için, Bölüm 11'de sunulan Church-Turing teoremi üzerine güvenme k daha basittir. Özellikle, Church-Turing teoremi, Bare Bones'un Turing makineleri yle aynı güce sahip olduğu gerçeğiyle birleştiğinde, hiçbir dilin orijinal Bare Bones 'tan daha güçlü olamayacağını belirtir. Bu nedenle, değiştirilmiş dilimizde çözüleb ilen herhangi bir problem, Bare Bones kullanılarak da çözülebilir.

Değiştirilmiş dilin gücünün, orijinal Bare Bones ile aynı olduğunu sonucu na varıyoruz. İki dil arasındaki tek ayrım, birinin yinelemeli bir kontrol yapısı sağlaması, diğerinin ise özyineleme sağlamasıdır.
Böylece, iki kontrol yapısı aslında hesaplama gücü açısından eşdeğerdir.

Doclingo https://www.doclingo.ai

ek

F

Sorular ve Alıştırmalar için Cevaplar

Bölüm 1

Bölüm 1.1

1. Üstteki iki girişten biri ve yalnızca biri 1 olmalıdır ve en düşük giriş 1 ol malıdır.
2. Aşağıdaki girişteki 1, NOT kapısı tarafından 0'a çevrilir ve AND kapısının ç ıkışının 0 olmasına neden olur. Böylece OR kapısına giden her iki giriş de 0'dır (üst girişin flip-flop'ta 0'da tutulduğunu unutmayın), bu nedenle O R kapısının çıkışı 0 olur. Bu, flip-flop'un alt girişinin 0'a döndükten sonra AND kapısının çıkışının 0 kalacağı anlamına gelir.
3. Üst OR kapısının çıkışı 1 olacak, bu da üst NOT kapısının 0 çıkışı üret mesine neden olacaktır. Bu, alt OR kapısının 0 üretmesine neden olac ak ve alt NOT kapısı 1 üretecektir. Bu 1, flip-flop'un çıkışı olarak görülü r ve aynı zamanda üst OR kapısına geri beslenir; burada, flip-flop'un giri şi 0'a döndükten sonra bile o kapının çıkışını 1'de tutar.
4. a. Tüm devre, tek bir VEYA kapısına eşdeğerdir.
   b. Bu tüm devre aynı zamanda tek bir XOR kapısına da eşdeğerdir.
5. a. 6AF2 b. E85517 c. 48
6. a. 01011111110110010111
   b. 0 1 10 00 010 00 0 10 10
   c. 1010101111001101
   d. 0000000100000000

Bölüm 1.2

1. İlk durumda, bellek hücresi numarası 6, 5 değerini içerir. İkinci durumd a ise, 8 değerine sahip olur.
2. Adım 1, yeni değer yazıldığında hücre numarası 3'teki orijinal değeri siler.Sonuç olarak, Adım 2, hücre numarası 3'teki orijinal değeri hücre numarası 2'ye yerleştirmez. Sonuç olarak, her iki hücre de

Doclingo https://www.doclingo.ai

hücre numarası 2'de başlangıçta bulunan değer. Doğru bir prosedür aşağıda ki gibidir:

Adım 1. Hücre numarası 2'nin içeriğini hücre numarası 1'e taşıyın.

Adım 2. Hücre numarası 3'ün içeriğini hücre numarası 2'ye taşıyın.

Adım 3. Hücre numarası 1'in içeriğini hücre numarası 3'e taşıyın.

3. 32768 bit

Bölüm 1.3

1. Verilerin daha hızlı alınması ve daha yüksek transfer hızları
2. Burada hatırlanması gereken nokta, mekanik hareketin yavaşlığınınbilgisay arın iç işleyişinin hızıyla karşılaştırıldığında, okuma/yazmabaşlıklarını hareket ettirmemiz gereken sayıyı en aza indirmemizigerektirdiğidir. Bir yüzeyi tamam en doldurduğumuzda, bir sonraki yüzeyegeçmeden önce her seferinde okuma /yazma başlığını hareket ettirmemizgerekecektir. Bu nedenle hareket sayısı, iki y üzeydeki toplamizleme sayısıyla yaklaşık olarak aynıdır. Ancak, yüzeyler aras ındaokuma/yazma başlıklarını elektronik olarak değiştirerek geçiş yaparsak,h er silindirin doldurulmasından sonra yalnızca okuma/yazma başlıklarınıhareket ettirmemiz gerekecektir.
3. Bu uygulamada, bilgilerin kütle depolamadan rastgele bir şekilde alınması ge rekmektedir; bu, CD'ler ve DVD'ler üzerinde kullanılan spiral sistem bağla mında zaman alıcı olacaktır. (Ayrıca, mevcut teknoloji, bir CD veya DVD üzerind eki verilerin bireysel bölümlerinin güncellenmesine izin vermemektedir.)
4. Depolama alanı fiziksel sektör birimlerinde (gerçekte çoğu durumda sektö r grupları birimlerinde) tahsis edilmektedir. Son fiziksel sektör dolu de ğilse, dosyaya ek metin eklenebilir; bu, dosyaya tahsis edilen depolama alan ını artırmaz. Son fiziksel sektör doluysa, belgeye yapılacak her ekleme için ek fiziksel sektörlerin tahsis edilmesi gerekecektir.
5. Flash sürücüler fiziksel hareket gerektirmediği için daha kısa yanıt sürelerine s ahiptir ve fiziksel aşınmadan etkilenmezler.
6. Bir tampon, verileri geçici olarak tutmak için kullanılan bir veri depolama alanıdır; g enellikle verinin kaynağı ile nihai varış noktası arasındaki tutarsızlıkları absorbe etme aracı olarak kullanılır.

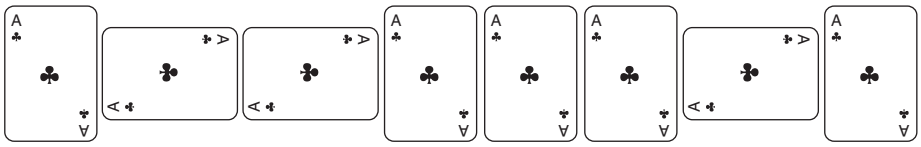
Bölüm 1.4

1. Bilgisayar Bilimleri
2. İki desen aynıdır, tek fark, düşük sıralı uçtan altıncı bitin her zaman büyük harfler için 0 ve küçük harfler için 1 olmasıdır.
3. a.

00100010	01010011	01110100	01101111
01110000	001 00001	00100010	00100000
01000011	01 101000	01100101	01110010
01111001	01101100	001 00000	01110011
01101000	01101111	011 10101	01110100
01100101	01110100	00101110	

Doclingo https://www.doclingo.ai

b. 01000100 01101111 01100101 01110011
00100000 00110010 00100000 00101011
00100000 00110011 00100000 00111101
00100000 001 10101 00111111

4. 

5. a. 5 b. 9 c. 11 d. 6 Ve. 16 f. 18

6. a. 110 b. 1101 c. 1011 d. 10010 Ve. 11011 f. 100

7. 24 bit ile ASCII kullanarak üç sembol depolayabiliriz. Böylece 999'a kadar değerleri de polayabiliriz.Ancak, bitleri ikili rakamlar olarak kullanırsak, 16.777.215'e kadar değ erleri depolayabiliriz.

8. a. 15.15 b. 51.0.128 c. 10.160

9. Geometrik temsiller, bit haritaları olarak kodlanmış görüntülerden d aha fazla ölçek değişikliğine uygundur. Ancak, geometrik temsiller g enellikle bit haritalarının ürettiği aynı fotoğrafik kalitede değildir. Gerçekten de, Bölüm 1.8'de tartışıldığı gibi, bit haritalarının JPEG temsilleri fo toğrafçılıkta çok popülerdir.

10. 44.100 örnekleme hızı ile bir saatlik stereo müzik, 635.040.000 bayt dep olama gerektirecektir. Böylece, kapasitesi 600MB'den biraz daha fazla olan bir CD'yi neredeyse dolduracaktır.

Bölüm 1.5

1. a. 42 b. 33 c. 23 d. 6 Ve. 31

2. a. 100000 b. 1000000 c. 1100000 d. 1111 e. 11011 Ve. 11011

3. a. $3^1/_4$ b. $5^7/_8$ c. $2^1/_2$ d. $6^3/_8$ Ve. $^5/_8$

4. a. 100.1 b. 10.11 c. 1.001 D. 0,0101 Ve. 101.101

5. a. 100111 b. 1011.110 c. 100000 D. 1000,00

Bölüm 1.6

1. a. 3 b. 15 c. −4 d. −6 e. 0 f. −16

2. a. 00000110 b. 1111 1010 c. 11101111
d. 00001101 Ve. 11111111 f. 00000000

3. a. 11111111 b. 10101011 c. 00000100 c. 00000100
d. 00000010 Ve. 00000000 f. 10000000

4. a. 4 bit ile en büyük değer 7 ve en küçük değer 8'dir.
b. 6 bit ile en büyük değer 31 ve en küçük değer 32'dir.
c. 8 bit ile en büyük değer 127 ve en küçük değer 128'dir.

5. A. 0111 (5 2 = 7) B. 0100 (3 1 = 4) C. 1111 (5 (6) = −1)
D. 0001 (2 3 = 1) e. 1000 (6 (2) = −8)

Doclingo https://www.doclingo.ai

6. a. 0111 b. 1011 (taşma) c. 0100 (taşma)
d. 0001 e. 1000 (taşma)

7.

a.	b.	c.	d.	Ve.
0110	0011	0100	0010	0001
0001	1110	1010	0100	1011
0111	0001	1110	0110	1100

8. Hayır. Taşma, kullanılan sistem için çok büyük bir sayıyı depolamaya çalı şıldığında meydana gelir. Pozitif bir değeri negatif bir değere eklerken, sonuç eklenen değerler arasında olmalıdır. Bu nedenle sonuç, hatası z bir şekilde depolanacak kadar küçük olacaktır.

9. a. 6 çünkü 1110 $\rightarrow 14$ $-$
b. -1 çünkü 0111 $\rightarrow 7$ $-$
c. 0 çünkü 1000 $\rightarrow 8$ $-$
d. -6 çünkü 0010 $\rightarrow 2$ -8
Ve. -8 çünkü 0000 $\rightarrow 0$ $-$
f. 1 çünkü 1001 $\rightarrow 9$ -8

10. a. 1101 çünkü 5 $+8$ $= 13 \rightarrow 1101$
b. 0011 çünkü $5+8$ $= 3 \rightarrow 0011$
c. 1011 çünkü 3 $+8$ $= \rightarrow 1011$
d. 1000 çünkü 0 $+8$ $= \rightarrow 1000$
e. 1111 çünkü 7 $+8$ $= 15 \rightarrow 1111$
f. 0000 çünkü $8+8$ $= \rightarrow 0000$

Hayır. Aşırı sekiz notasyonunda saklanabilecek en büyük değer 7'dir, bu da 111 1 ile temsil edilir. Daha büyük bir değeri temsil etmek için en az aşırı 16 (5 bit de senleri kullanır) kullanılmalıdır. Benzer şekilde, 6 aşırı dört notasyonunda temsil edilemez. (Aşırı dört notasyonunda temsil edilebilecek en büyük değer 3'tür.)

Bölüm 1.7

1. a. $^{5}/_{8}$ b. $3^{1}/_{4}$ c. $^{9}/_{32}$ d. $-1^{1}/$ Ve. $-(^{11}/_{64})$

2. a. 01101011 b. 01111010 (kısaltma hatası)
c. 01001100 d. 11101110 e. 111110 0 0 (kısaltma hatası)

3. 01001001 ($^{9}/_{16}$) 00111101 ($^{13}/_{32}$)'den büyüktür. Aşağıda, iki desenin hangisinin daha büy ük bir değeri temsil ettiğini belirlemenin basit bir yolu verilmiştir:

Durum 1. Eğer işaret bitleri farklıysa, daha büyük olan 0 işaret bitine sahip olandır.

Durum 2. Eğer işaret bitleri her ikisi de 0 ise, desenlerin kalan kısımlarını soldan sa ğa doğru tarayınve iki desenin farklı olduğu bir bit pozisyonu bulunana ka dar devam edin. Bu pozisyonda 1 içeren desen,daha büyük değeri tems il eder.

Durum 3. Eğer işaret bitleri her ikisi de 1 ise, desenlerin kalan kısımlarını sold an sağa doğru tarayınve iki desenin farklı olduğu bir bit pozisyonu b ulunana kadar devam edin. Bu pozisyonda 0 içeren desen,daha büyü k değeri temsil eder.

Bu karşılaştırma sürecinin basitliği, üssü kayan nokta sistemlerinde iki 'nin tamamı yerine fazladan notasyonla temsil etmenin nedenlerinden b iridir.

4. En büyük değer $7^{1}/_{2}$ olacaktır, bu da 01111111 deseniyle temsil edilir. En küçük pozitif değer için, iki "doğru" cevap olduğunu savunabilirsiniz.

Doclingo https://www.doclingo.ai

İlk olarak, metinde açıklanan kodlama sürecine bağlı kalırsanız, mantis sanın en anlamlı bitinin 1 olması gerektiği (normalleştirilmiş form o larak adlandırılır) için cevap $^{1}/_{32}$'dir ve bu 00001000 deseniyle temsil edilir . Ancak, çoğu makine 0'a yakın değerler için bu kısıtlamayı getirmez. B öyle bir makine için doğru cevap $^{1}/_{256}$ olup 00000001 ile temsil edilir.

Bölüm 1.8

1. Koşul uzunluğu kodlaması, frekansa bağlı kodlama, göreceli kodlama ve sözlük kodlaması
2. 121321112343535
3. Renkli karikatürler, keskin kenarlarla katı renk bloklarından oluşur. Ayr ıca, kullanılan renk sayısı sınırlıdır.
4. Hayır. Hem GIF hem de JPEG kayıplı sıkıştırma sistemleridir, bu da görüntüdeki de tayların kaybolacağı anlamına gelir.
5. JPEG'nin temel standardı, insan gözünün renk değişikliklerine karşı parlaklık d eğişikliklerine göre daha az hassas olduğunu göz önünde bulundurur. Böylece, görüntü kalitesinde belirgin bir kayıp olmadan renk bilgilerini temsil etme k için kullanılan bit sayısını azaltır.
6. Zamansal maskeleme ve frekans maskeleme
7. Bilgi kodlarken, yaklaşık değerler yapılır. Sayısal veriler söz konusu old uğunda, bu yaklaşık değerler hesaplamalar yapıldığında birikerek hatalı so nuçlara yol açabilir. Ancak, görüntüler ve sesler durumunda yaklaşık değerler o kadar kritik değildir çünkü kodlanmış veriler genellikle yalnızca saklanır, aktarılır ve yeniden üretilir. Ancak, görüntüler veya sesler tekrar tekrar yeni den üretilirse, yeniden kaydedilirse ve sonra yeniden kodlanırsa, bu yaklaşık değerler birikerek nihayetinde değersiz verilere yol açabilir.

Bölüm 1.9

1. b, c ve e
2. Evet. Eğer bir baytta çift sayıda hata oluşursa, parite tekniği bunları te spit etmez.
3. Bu durumda, Hata 1'in a ve d baytlarında hatalar oluşur. Soru 2'nin cevabı aynı kalır.
4. a. 001010111 001101000 101100101
   101110010 101100101 000100000
   001100001 101110010 101100101
   000100000 0011110 01 101101111
   001110101 100111111
   b. 100100010 101001000 101101111
   101110111 100111111 100100010
   000100000 001 000011 001101000
   101100101 101110010 001111001
   101101100 000100000 001100001
   001110011 001101011 101100101
   001100100 100101110
   c. 000110010 100101011 100110011
   000111101 100110101 100101110

Doclingo https://www.doclingo.ai

5. A. YATAK B. TAKSİ c. BAŞLIK
6. Bir çözüm aşağıdaki gibidir:
   A 0 0 0 0 0
   B 1110 0
   C0111 1
   D1001 1

Bölüm 2

Alt Bölüm 2.1

1. Bazı makinelerde bu, önce ilk hücredeki içeriği bir kaydediciye okuma v e ardından kaydediciden hedef hücreye yazma işlemini içeren iki aşamalı bir süreçtir. Çoğu makinede, bu, ara bir kaydedici kullanmadan tek bi r etkinlik olarak gerçekleştirilir.
2. Yazılacak değer, yazılacak hücrenin adresi ve yazma komutu
3. Genel amaçlı kayıtlar, mevcut işlemler için hemen uygulanabilir verileri tutmak için kullanılır; ana bellek, yakın gelecekte ihtiyaç duyulacak verileri tutmak i çin kullanılır; ve kütle depolama, yakın gelecekte muhtemelen ihtiyaç duyulm ayacak verileri tutmak için kullanılır.

Bölüm 2.2

1. "Taşımak" terimi genellikle bir yerden çıkarıp başka bir yere koyma anla mını taşır ve bu da geride bir boşluk bırakır. Bir makine içindeki çoğu d urumda, bu çıkarma işlemi gerçekleşmez. Daha ziyade, taşınan nesne genellikle yeni konuma kopyalanır (veya klonlanır).
2. Göreceli adresleme olarak adlandırılan yaygın bir teknik, nereye atlama k yerine ne kadar uzağa atlayacağınızı belirtmektir. Örneğin, bir tali mat üç talimat ileri atlamak veya iki talimat geri atlamak olabilir. Anca k, bu tür ifadelerin, atlama işleminin başlangıcı ile hedefi arasında daha sonra ek talimatlar eklenirse değiştirilmesi gerektiğini unutmamal ısınız.
3. Bu her iki şekilde de savunulabilir. Talimat, koşullu bir atlama biçimind e ifade edilir. Ancak, 0'ın 0'a eşit olma koşulu her zaman sağlandığı iç in, atlama her zaman hiç koşul belirtilmemiş gibi yapılacaktır. Bu tür tali matlara sahip makineleri sıkça bulacaksınız çünkü verimli bir tasarım sağlarlar. Örneğin, bir makine "Eğer ... atla ..." gibi bir yapıya sahip bir talimatı yürütmek üzere tasarlandıysa, bu talimat biçimi hem ko şullu hem de koşulsuz atlamaları ifade etmek için kullanılabilir.
4. $156C = 0001010101101100$
   $166D = 0001011001101101$
   $5056 = 0101000001010110$
   $306E = 0011000001101110$
   $C000 = 1100000000000000$

Doclingo https://www.doclingo.ai

5. a. Kayıt 6'nın içeriğini bellek hücresi numarası 8A'ya DEPO et.
   b. Kayıt A'nın içeriği kayıt 0'a eşitse DE konumuna ATLAMAK
   .
   c. Kayıt 3 ve C'nin içeriklerini VELE, sonucu kayıt 0'da bırak.
   d. Kayıt F'nin içeriğini kayıt 4'e TAŞI.
6. Talimat 15AB, CPU'nun AB adresindeki bellek hücresinin içeriğinisorg ulamasını gerektirir. Bu değer, bellekten alındığında,belirtilen kay ıt 5'e yerleştirilir. Talimat 25ABböyle bir bellek isteği gerektirmez. Bunu n yerine, AB değerikayıt 5'e yerleştirilir.

7. a. 2356 b. A503 c. 80A5

Bölüm 2.3

1. Onaltılık 34
2. a. 0F b. C3
3. a. 00 b. 01 c. dört kez
4. Durur. Bu, genellikle kendini değiştiren kod olarak adlandırılan bir örnek tir. Yani, program kendini değiştirir. İlk iki talimatın onaltılık C0'ı F8 b ellek konumuna yerleştirdiğini ve sonraki iki talimatın 00'ı F9 konumuna yerleştirdiğini unutmayın. Böylece, makine F8'deki talimata ulaştığında , durdurma talimatı (C000) oraya yerleştirilmiştir.

Bölüm 2.4

1. a. 00001011 b. 10000000 c. 00101101
   d. 11101011 Ve. 11101111 f. f. 11111111
   g. 11100000 h. 01101111 Ben. 11010010
2. 0 011110 0 VE işlemi ile
3. 0 011110 0 XOR işlemi ile
4. a. Sonuç, dizede çift sayıda 1 varsa 0'dır. Aksi takdirde 1'dir.

   b. Sonuç, çift parite için parite bitinin değeridir.
5. Mantıksal XOR işlemi, her iki operandın da 1 olduğu durum hariç topla ma işlemini yansıtır; bu durumda XOR 0 üretirken, toplam 10'dur. (Bu nedenle XOR işlemi, taşıma olmadan bir toplama işlemi olarak düşünül ebilir.)
6. Küçük harfleri büyük harfe dönüştürmek için 11011111 maskesi ile AND kullanın. Büyük harfleri küçük harfe dönüştürmek için 00100000 ile OR kullanın.
7. a. 01001101 b. 11100001 c. 11101111
8. a. 57 b. B8 c. 6F d. 6A
9. 5
10. 0 0110110 iki'nin tamamlayıcısında; 01011110 kayan nokta içinde. Buradaki n okta, değerleri eklemek için kullanılan prosedürün bit desenlerine verilen y oruma bağlı olarak farklı olduğudur.

Doclingo https://www.doclingo.ai

11. Bir çözüm aşağıdaki gibidir:

    12A7 (bellek hücresi A7'nin içeriğini 2 numaralı kayda yükle.)
    2380 (değeri 80 olanı 3 numaralı kayda yükle.)
    7023 (2 ve 3 numaralı kayıtları OR işlemi yaparak sonucu 0 numaralı ka yıtta bırak.)30A7 (0 numaralı kaydın içeriğini bellek hücresi A7'ye kay det.)C000 (DUR.)

12. Bir çözüm aşağıdaki gibidir:

    15E0 (bellek hücresi E0'nın içeriğini 5 numaralı kayda yükle.)
    A502 (5 numaralı kaydın içeriğini 2 bit sola döndür.)
    260F (değeri 0F olanı 6 numaralı kayda yükle.)
    8056 (5 ve 6 numaralı kayıtları AND işlemi yaparak sonucu 0 numaralı k ayıtta bırak.)30E1 (0 numaralı kaydın içeriğini bellek hücresi E1'e kayd et.)C000 (DUR.)

Bölüm 2.5

1. a. 37B5
   b. Bir milyon kat
   c. Hayır. Tipik bir metin sayfası 4000 karakterden daha az içerir. Bu nedenl e, dakikada beş sayfa yazdırma yeteneği, dakikada en fazla 20.000 karakte rlik bir yazdırma hızını gösterir ki bu da bir milyon karakterden çok daha azdır. (Nokta, bir bilgisayarın karakterleri bir yazıcıya yazdırmaktan çok da ha hızlı gönderebilmesidir; bu nedenle yazıcının bilgisayara beklemesini sö ylemenin bir yoluna ihtiyacı vardır.)
2. Disk, bir saniyede 50 devir yapacak, bu da 800 sektörünokuma/yazma k afasının altından bir saniyede geçeceği anlamına geliyor. Her sektör 10 24 bayt içerdiğinden, bitler okuma/yazma kafasının altından yaklaşık6.5 Mbps hızında geçecektir. Bu nedenle, denetleyici ile disk sürücüsü ara sındaki iletişim,diskten okunan verilerle başa çıkabilmek için en az bu hı zda olmalıdır.
3. Unicode'da temsil edilen 300 sayfalık bir roman yaklaşık 2MB veya16.000. 000 bit içermektedir. Bu nedenle, romanın tamamını 54 Mbps hızındaaktarmak için yaklaşık 0.3 saniye gerekecektir.

Bölüm 2.6

1. Boruda, B1B0 (çalıştırılan), 5002 ve belki de B0AA talimatları bulunaca ktır. Eğer kayıt 1'deki değer, kayıt 0'daki değere eşitse, B0 konumuna atl ama gerçekleştirilir ve borudaki talimatlar için harcanan çaba boşa gi der. Öte yandan, bu talimatlar için harcanan çaba ekstra zaman gerekti rmediğinden, zaman kaybı yaşanmaz.
2. Eğer önlem alınmazsa, F8 ve F9 bellek konumlarındaki bilgiler, programı n önceki kısmının bu hücreleri değiştirme şansı olmadan bir talimat olar ak alınır.
3. a. Hücreye 1 eklemeye çalışan CPU, önce hücredeki değeri okuyabilir. B unu takiben diğer CPU hücrenin değerini okur. (Bu noktada her iki CP U da aynı değeri almıştır.) Eğer ilk CPU şimdi ekleme işlemini tamaml ayıp sonucunu hücreye yazmadan önce ikinci CPU çıkarma işlemini t amamlayıp sonucunu yazarsa, hücredeki son değer yalnızca ikinci CPU'nun etkinliğini yansıtır.

Doclingo https://www.doclingo.ai

b. CPU'lar verileri hücreden daha önce olduğu gibi okuyabilir, ancak bu sef er ikinci CPU sonucu birinci CPU'dan önce yazabilir. Böylece yalnızca birin ci CPU'nun etkinliği hücrenin son değerine yansır.

Bölüm 3

Alt Bölüm 3.1

1. Geleneksel bir örnek, bir etkinlik için bilet almak üzere bekleyen in sanların sırasıdır. Bu durumda, "sırayı bozmayı" deneyen birisi olabi lir, bu da FIFO yapısını ihlal eder.
2. Seçenekler (b), (c) ve (e)
3. Gömülü sistemler genellikle belirli görevlere odaklanırken, PC'ler genel amaçlı bilgisayarlardır. Gömülü sistemler, karşılaştırılabilir yaşta PC'lerde n daha sınırlı kaynaklara sahip olabilir, ancak minimal insan müdahalesi ile katı son tarihlerle karşılaşabilirler.
4. Zaman paylaşımı, birden fazla kullanıcının aynı anda bir makineye erişmesini ifade ed er. Çoklu görev, bir kullanıcının aynı anda birden fazla görev gerçekleştirmesini if ade eder.

Alt Bölüm 3.2

1. *Shell:* Makinenin çevresiyle iletişim kurar.

   Dosya yöneticisi: Makinenin kütle depolamasının kullanımını koordine eder.

   Aygıt sürücüleri: Makinenin çevresel cihazlarıyla iletişimi yönetir.

   Bellek yöneticisi: Makinenin ana belleğinin kullanımını koordine eder.

   Planlayıcı: Sistemdeki süreçleri koordine eder.

   Dağıtıcı: Süreçlerin CPU zamanına atanmasını kontrol eder.
2. Çizgi belirsizdir ve ayrım genellikle bakış açısına bağlıdır. Genel olarak, yardımcı yazılımlar temel, evrensel görevleri yerine getirirken, uygulama yazılımları makinenin uygulamasına özgü görevleri yerine getirir.
3. Sanal bellek, ana bellek ile kütle depolama arasında veri ve programları n ileri geri değiştirilmesi süreciyle yaratılan hayali bellek alanıdır.
4. Makine açıldığında, CPU, ROM'da bulunan önyükleme işlemini yürüt meye başlar. Bu önyükleme, işletim sisteminin kütle depolamadan geçi ci ana bellek alanına aktarılması sürecinde CPU'ya rehberlik eder. Bu akt arım tamamlandığında, önyükleme CPU'yu işletim sistemine atlamaya yönlendirir.

Bölüm 3.3

1. Bir program, bir dizi talimattır. Bir süreç, bu talimatları takip etme eyl emidir.
2. CPU mevcut makine döngüsünü tamamlar, mevcut sürecin durumunu k aydeder ve program sayacını önceden belirlenmiş bir değere (kesme i şleyicisinin konumu) ayarlar. Böylece, yürütülen bir sonraki talimat, ke sme işleyicisi içindeki ilk talimat olacaktır.

Doclingo https://www.doclingo.ai

3. Onlara daha yüksek öncelikler verilebilir, böylece dağıtıcı tarafından tercih edilirler. Diğer bir seçenek ise, daha yüksek öncelikli süreçlere daha uzun za man dilimleri vermektir.
4. Eğer her işlem tüm zaman dilimini tüketirse, makine bir saniyede neredeyse 20 işleme tam bir dilim sağlayabilir. Eğer işlemler tüm zaman dilimlerini t üketmezse, bu değer çok daha yüksek olabilir, ancak o zaman bir bağlam de ğiştirme işlemi gerçekleştirmek için gereken zaman daha önemli hale gele bilir (bkz. Soru 5).
5. Makinenin zamanının toplam $^{5000}/_{5001}$ kadar bir kısmı gerçekten işlemleri gerçekleştirme kle harcanacaktır. Ancak, bir işlem bir G/Ç etkinliği talep ettiğinde, zaman dilimi sona erer ken kontrolör talebi yerine getirir. Dolayısıyla, eğer her işlem zaman diliminin yalnızca bir mikro saniyesinden sonra böyle bir talepte bulunursa, makinenin verimliliği $^{1}/_{2}$'ye düşecektir. Yani, makine bağlam değiştirme işlemlerini gerçekleştirmek için harcadığı za man kadar, işlemleri yürütmek için de zaman harcayacaktır.

Bölüm 3.4

1. Bu sistem, kaynağın aynı anda birden fazla işlem tarafından kullanıl madığını garanti eder; ancak, kaynağın kesinlikle sırayla tahsis edilme sini zorunlu kılar. Bir işlem kaynağı kullandıktan ve bıraktıktan sonra, o rijinal işlemin tekrar erişebilmesi için diğer işlemin kaynağı kullanması nı beklemesi gerekir. Bu, ilk işlemin kaynağı hemen ihtiyaç duyması v e diğer işlemin bir süre ihtiyaç duymaması durumunda bile geçerlidir .
2. İki araba tünelin zıt uçlarına aynı anda girerse, diğerinin varlığından hab erdar olmayacaklardır. Işığı açma ve girme süreci, kritik bir bölgenin başka bir örneğidir; bu durumda buna kritik bir süreç diyebiliriz. Bu t erminolojide, hatayı özetleyebiliriz; tünelin zıt uçlarındaki arabalar kriti k süreci aynı anda gerçekleştirebilir.
3. a. Bu, paylaşılmayan kaynağın gerekli olmadığını ve kısmi olarak tahsis edildiğini garanti eder; yani, bir arabaya tüm köprü veya hiçbiri verilm ez.
   b. Bu, paylaşılmayan kaynağın zorla geri alınabileceği anlamına gelir.
   c. Bu, paylaşılmayan kaynağı paylaşılabilir hale getirir, bu da rekabeti ortad an kaldırır.
4. Yönlendirilmiş grafikte kapalı bir döngü oluşturan bir ok dizisi. Bu gözle m üzerine, bazı işletim sistemlerinin deadlock'un varlığını tanımasına ve dolayısıyla uygun düzeltici önlemler almasına olanak tanıyan teknikler geliştirilmiştir.

Bölüm 3.5

1. İsimler ve tarihler, yaygın seçimler oldukları için zayıf adaylar olarak kabul edilir ve bu nedenle şifre tahmincileri için kolay hedeflerdir. Tam kelimeleri n kullanımı da zayıf olarak kabul edilir çünkü şifre tahmincileri, bir sözlük te bulunan kelimeleri denemek için kolayca bir program yazabilirler. A yrıca, yalnızca karakterlerden oluşan şifreler, sınırlı bir karakter setind en oluştuğu için teşvik edilmez.

Doclingo https://www.doclingo.ai

2. Dört, 2 bit kullanılarak oluşturulabilen farklı bit desenlerinin sayısıdır. Daha fazla ayrıcalık seviyesi gerekiyorsa, tasarımcıların farklı seviyeleri temsil etmek için en az 3 bite ihtiyaçları olacak ve bu nedenle muhtemelen toplamda 8 seviye kullanmayı seçeceklerdir. Aynı şekilde, 4'ten daha az ayrıcalık seviyesi için doğal seçim 1 bit ile temsil edilebilen desenlerin sayısı olan 2 olacaktır.
3. Bu süreç, dispatcher'ın her zaman dilimini o işleme vermesi için işletim sistemi programını değiştirebilir.

Bölüm 4

Bölüm 4.1

1. Açık bir ağ, spesifikasyonları ve protokolleri kamuya açık olan bir ağdır, farklı satıcıların uyumlu ürünler üretmesine olanak tanır.
2. Her ikisi de iki otobüsü birleştirerek daha büyük bir otobüs ağı oluşturur. Ancak, bir köprü yalnızca köprünün diğer tarafına gidecek mesajları iletir,\ oysa bir anahtar, her biri köprü gibi davranabilen birden fazla bağlantıya sahiptir.
3. Bir yönlendirici, internet üzerindeki ağlar arasında mesajları yönlendiren bir cihazdır.
4. Peki, bir mail-order işletmesi ve onun müşterileri, bir banka memuru ve bankanın müşterileri, ya da bir eczacı ve onun müşterileri hakkında ne dersiniz?
5. Trafik akışı, sözlü telefon iletişimi ve görgü kurallarıyla ilgili birçok protokol bulunmaktadır.
6. Küme bilişimi genellikle yüksek erişilebilirlik veya yük dengelemesi sağlamak için birden fazla, özel bilgisayar içerir. Grid bilişimi, küme bilişiminden daha gevşek bir şekilde bağlıdır ve başka bir iş yapmadıkları zaman dağıtılmış hesaplamaya katılan makineleri içerebilir.

Bölüm 4.2

1. Tier-1 ve tier-2 İnternet Servis Sağlayıcıları (ISP'ler) İnternet'in iletişim “kökünü” sağlarken, erişim ISP'leri bu köke müşterilerine erişim sağlar.
2. DNS (Alan Adı Sistemi), mnemonik adreslerden IP adreslerine (ve diğer yönde de) çeviri yapmayı sağlayan, İnternet genelindeki ad sunucularının koleksiyonudur.
3. İfade 3.6.9, üç baytlık deseni 000000110000011000001 001'u temsil eder. Bit deseni 0001010100011100, noktalarla ayrılmış onalık notasyonda 21.28 olarak temsil edilecektir.
4. Buna birkaç cevap olabilir. Bir tanesi, her ikisinin de özelden genele doğru ilerlemesidir. Mnemonik formdaki İnternet adresleri, belirli bir makinenin adıyla başlar ve TLD'nin adıyla ilerler. Posta adresleri, bir bireyin adıyla başlar ve şehir, eyalet ve ülke gibi giderek daha büyük bölgelere doğru ilerler. Bu sıra, alanı tanımlayan bit deseniyle başlayan IP adreslerinde tersine döner.
5. Ad sunucuları, mnemonik adresleri IP adreslerine çevirmeye yardımcı olur. Mail sunucuları e-posta mesajlarını gönderir, alır ve depolar. FTP sunucuları dosya aktarım hizmeti sağlar.

Doclingo https://www.doclingo.ai

6. SSH, şifreleme ve kimlik doğrulama sağlar.
7. İlk sunucunun her bir istemciye bireysel mesajlar göndermesi yükünden kurtarır. P2P yaklaşımı bu yükü istemcilere (eşlere) kendilerine kaydırırken, çoklu yayın bu yükü İnternet yönlendiricilerine kaydırır.
8. Dikkate alınması gereken kriterler maliyet, taşınabilirlik, bilgisayarınızı telefon olarak kullanmanın pratikliği, mevcut analog telefonların korunması ihtiyacı, acil durum 911 hizmeti ve ilgili çeşitli sağlayıcıların güvenilirliği ve hizmet alanları olabilir.

Bölüm 4.3

1. Bir URL, esasen Dünya Çapında Ağ'daki bir belgenin adresidir.
   Bir tarayıcı, bir kullanıcının hiper metne erişmesine yardımcı olan bir programdır.
2. Bir işaretleme dili, bir belgede açıklayıcı bilgi eklemek için bir sistemdir.
3. HTML, belirli bir işaretleme dilidir. XML, işaretleme dilleri üretmek için bir standarttır.
4. a. <html>, bir HTML belgesinin başlangıcını işaret eder.
   b. <head> bir belgenin başlığının başlangıcını işaret eder.
   c. </p> bir paragrafın sonunu işaret eder.
   d. başka bir belgeye bağlı bir öğenin sonunu işaret eder.
5. *Müşteri tarafı* ve *sunucu tarafı*, bir etkinliğinmüşterinin bilgisayarında mı yoksa sunucunun bilgisayarında mı gerçekleştirildiğini belirlemek için kullanılan terimlerdir.

Bölüm 4.4

1. Bağlantı katmanı mesajı alır ve onu ağ katmanına iletir.
   Ağ katmanı, mesajın hangi yönde iletilmesi gerektiğini belirlerve mesajı iletilmek üzere bağlantı katmanına geri verir. Yüksek katmanlar yönlendirme için gerekli değildir, ancak gelişmişyönlendiriciler, seçici filtreleme veya katmanlı hizmet kalitesi gibi ekhizmetler sağlamak için taşıma veya uygulama katmanlarını kullanabilir.
2. TCP'nin aksine, UDP, mesajınhedefte alındığını onaylamayan bağlantısız bir protokoldür.
3. Taşıma katmanı, uygulama katmanında hangibirimin gelen bir mesajı alması gerektiğini belirlemek için taşıma protokolü port numaralarını kullanır.
4. Gerçekten hiçbir şey. Herhangi bir ana bilgisayardaki bir programcı, oana bilgisayardaki yazılımı bu tür kayıtları tutacak şekilde değiştirebilir. Bu nedenle, hassas verilerin şifrelenmesi gerekir.

Bölüm 4.5

1. Phishing, kullanıcıların şifrelerini, kredi kartı numaralarını vb. istemek için bir teknik olup, e-posta aracılığıyla kullanıcıların bankası veya kampüs IT departmanı gibi meşru bir varlık olarak kılık değiştirerek gerçekleştirilir.
   Bilgisayarlar oltalama saldırılarına karşı güvenli değildir; kullanıcılar, uygun doğrulama olmadan başkalarına hassas verileri açıklarken sağduyularına güvenmek zorundadır.
2. Bir bölgenin geçidi, paketleri (mesajların parçaları) sadece ileten bir yönlendiricidir. Bu nedenle, geçitteki bir güvenlik duvarı, trafiği içeriğine göre filtreleyemez, yalnızca adres bilgilerine göre filtreleyebilir.

Doclingo https://www.doclingo.ai

3. Şifrelerin kullanımı verileri (ve dolayısıyla bilgileri de) korur. Şifreleme ku llanımı bilgiyi korur.
4. Açık anahtar şifreleme sisteminde, mesajların nasıl şifrelendiğini bilme k, mesajların şifresini çözmeyi sağlamaz.
5. Problemler uluslararası niteliktedir ve bu nedenle tek bir hükümetin y asalarına tabi değildir. Dahası, hukuki çözümler yalnızca zarar göre n taraflara başvuru imkanı sağlar, yaralanmaları önlemez.

Bölüm 5

Bölüm 5.1

1. Bir süreç, bir algoritmayı yürütme etkinliğidir. Bir program, bir algoritma nın temsilidir.
2. Giriş bölümünde müzik çalma, çamaşır makinelerini çalıştırma, modell er oluşturma ve sihir numaraları yapma gibi algoritmaları ve Öklid alg oritmasını alıntıladık. Günlük yaşamda karşılaştığınız birçok "algoritm a", resmi tanımımıza göre algoritma olma özelliğini taşımaz. Uzun böl me algoritması örneği metinde belirtilmiştir. Diğer bir örnek, saat tarafı ndan yürütülen ve her gün kollarını ilerleten ve zillerini çalan algoritma dır.
3. Resmi tanım, adımların sıralı ve belirsiz olmaması gerektiğini gerektirmiyor. Sadece adımların uygulanabilir olması ve bir sona ulaşması gerektiğine d air ipuçları veriyor.
4. Burada iki nokta var. Birincisi, talimatların sonlanmayan bir süreci tanım lamasıdır. Ancak gerçekte, süreç nihayetinde cebinizde hiç bozuk para k almadığı duruma ulaşacaktır. Aslında, bu başlangıç durumu olabilir. B u noktada sorun belirsizliktir. Temsil edilen algoritma, bu durumda ne y apmamız gerektiğini bize söylemiyor.

Bölüm 5.2

1. Bir örnek, maddenin bileşiminde bulunur. Bir seviyede, ilkel maddeler mol eküller olarak kabul edilir, ancak bu parçacıklar aslında atomlardan oluşa n bileşenlerdir ve bu atomlar da elektronlar, protonlar ve nötronlardan oluşur. Bugün, bu “ilkel” maddelerin bileşenler olduğunu biliyoruz.
2. Bir prosedür doğru bir şekilde oluşturulduğunda, prosedürün iç bileşimini yenid en gözden geçirmeden daha büyük program yapıları için bir yapı taşı olarak kull anılabilir.
3. X←girdi daha büyük;
   Y←girdi daha küçük;
   while (Y sıfır değil) do
     (Kalan←X'in Y'ye bölünmesinden sonra kalan;
       X←Y;
       Y←Kalan);
   EBOB←X
4. Tüm diğer ışık renkleri kırmızı, mavi ve green'in birleştirilmesiyle üretil ebilir. Bu nedenle bir televizyon tüpü bu üç basit rengi üretmek üzere tas arlanmıştır.

Doclingo https://www.doclingo.ai

Bölüm 5.3

1. a. eğer (n = 1 veya n = 2)
o zaman (cevap, tek değer n içeren listedir)
aksi takdirde (n'yi 3'e böl, bir bölüm q ve bir kalıntı r elde et.
eğer (r = 0)
o zaman (cevap, q tane 3 içeren listedir)
eğer (r = 1)
o zaman (cevap, (q — 1) tane 3 içeren listedir
ve iki tane 2;)
eğer (r = 2)
o zaman (cevap, q tane 3 ve bir tane 2 içeren listedir)
)

b. Sonuç, 667 üçlü içeren bir liste olacaktır.

c. Muhtemelen bir desen görmeye başlayana kadar küçük girdi değerleriyle denem eler yaptınız.

2. a. Evet. *İpucu:* İlk karoyu ortada yerleştirin, böylece deliği içeren çeyre kten kaçınırken diğer çeyreklerden birer kareyi kaplar. Her çeyrek, orijinal problemin daha küçük bir versiyonunu temsil eder.

b. Tek delikli tahta 2^{2n} içerir — 1 kare, ve her karonun kaptam olarak üç kare.

c. Bu sorunun (a) ve (b) kısımları, bir problemin çözümünü bilmenin başka bir problemi çözmeye nasıl yardımcı olduğuna dair mükemmel bir örnek sunmaktadır. Polya'nın dördüncü aşa masına bakın.

3. "Bu doğru cevaptır."

4. Parçaları bir araya getirmeye çalışmak, aşağıdan yukarıya bir yaklaşım olacaktır . Ancak, bulmacanın kutusuna bakarak resmin nasıl görünmesi gerektiğini görmek, yaklaşımınıza yukarıdan aşağıya bir bileşen ekler.

Bölüm 5.4

1. ifade içindeki testi "hedef değer mevcut girişe eşit değil ve dikkate alı nacak girişler kalmışsa" olarak değiştirin.

2.
```
Z ← 0;
X ← 1;
tekrarla (Z ←Z+X;
          X ← X + 1)
X          = 6)
```

3. Bu, C dilinde bir sorun olduğunu kanıtladı. *do* vewhile anahtar kelimeleri bi rkaç satırla ayrıldığında, bir programın okuyucuları genellikle *while* ifadesinin doğru yorumlanmasında zorlanır. Özellikle, bir *do* ifadesinin sonunda yer alan *while* genellikle bir *while* ifadesinin başlangıcı olarak yorumlanır. Bu nedenle deneyim, ön te st ve son test döngü yapılarını temsil etmek için farklı anahtar kelimeler kullan manın daha iyi olduğunu söyler.

4.

Cheryl	Alice	Alice
Gen	Cheryl	Brenda
Alice	Gen	Cheryl
Brenda	Brenda	Gen

Doclingo https://www.doclingo.ai

5. Zaman kaybıdır, listenin içindeki aynı girişin üzerinde pivot yerleştirmekte ısr ar etmek. Örneğin, önerilen değişikliği yapın ve ardından tüm girişlerin aynı olduğu bir liste üzerinde yeni programı deneyin.
6. prosedür sıralama (Liste)
   N←1;
   while (N Liste'nin uzunluğundan küçük olduğu sürece) yap
   (J←N+1;
   while (J Liste'nin uzunluğundan büyük olmadığı sürece) yap(
   eğer (J pozisyonundaki giriş N pozisyonundaki girişten küçükse)
   o zaman (iki girişi değiştir);
   J←J+1)
   N←N1)
7. Aşağıdaki verimsiz bir çözümdür. Bunu daha verimli hale getirebilir misiniz?

   prosedür sıralama (Liste)
   N←Listenin uzunluğu;
   iken (N 1'den büyük) yap
   (J←Listenin uzunluğu;
   iken (J 1'den büyük) yap(eğer (J pozisy
   onundaki giriş, J pozisyonundaki girişten daha küçükse
   − 1)
   o zaman (iki girişi değiştir);
   J →J − 1)
   N ← N − 1)

Bölüm 5.5

1. İlk düşünülen isim Henry, sonraki isim Larry ve\son isim Joe olacaktır.
2. 8, 17
3. 1, 2, 3, 3, 2, 1
4. Sonlandırma koşulu “N 3'ten büyük veya eşit” (veya “N 3'ten küçük de ğil”) şeklindedir. Bu, ek aktivasyonların oluşturulmadığı koşuldur.

Bölüm 5.6

1. Eğer makine bir saniyede 100 ismi sıralayabiliyorsa, bir saniyede 1/4 (10,0 − 00 100) karşılaştırma yapabilir. Bu, her karşılaştırmanın yaklaşık 0.0004 saniye sürdüğü anlamına gelir. Dolayısıyla, 1000 ismin sıralanması [ort alama olarak 1/4 (1,000,0001000) karşılaştırma gerektirir] yaklaşık 100 sani ye veya 1 2/3 dakika alır.
2. İkili arama $\Theta(\lg n)$ sınıfına aittir, sıralı arama $\Theta(n)$ sınıfına aittir, ve ekleme sıralaması $\Theta(n^2)$ sınıfına aittir.
3. Θ(lgn) sınıfı en verimlisidir, ardından Θ(n),Θ(n2) ve Θ(n3) gelir.
4. Hayır. Cevap doğru değil, ancak doğru gibi görünebilir. Gerçek şu ki, ü ç karttan ikisi her iki tarafta da aynıdır. Dolayısıyla, böyle bir kart seçme olasılığı iki üçte birdir.

Doclingo https://www.doclingo.ai

5. Hayır. Eğer pay, paydadan küçükse, örneğin $^3/_7$, verilen cevap 1'dir, oys a 0 olmalıdır.
6. Hayır. Eğer X'in değeri sıfır ve Y'nin değeri sıfırdan farklıysa, verilen cevap do ğru olmayacaktır.
7. Her seferinde sonlandırma testi yapıldığında, "Toplam $= 1 + 2 + ... + K$ veKN'ye eşit veya daha küçük" ifadesi doğrudur. Bunu sonlandırma koşulu "K N*'ye eşit veya daha büyük" ile birleştirmek, iste nensonucu "Toplam 1* + 2 + ... + N.*" üretir. Çünkü* K *sıfırdan başlat ılır veher döngüde bir artırılır, bu nedenle değeri nihayetinde* N*'ye ula şmalıdır.*
8. Ne yazık ki, hayır. Donanım ve yazılım tasarımının kontrolü dışındaki pr oblemler,örneğin mekanik arızalar ve elektrik sorunları, hesaplamaları et kileyebilir.

Bölüm 6

Bölüm 6.1

1. Üçüncü nesil bir dildeki bir program, adımlarının makinenin özellikleri,ö rneğin kayıtlar ve bellek hücresi adresleri gibi terimlerle belirtilmediği anlamında makine bağımsızdır. Öte yandan, aritmetik taşma ve kesme h atalarının hâlâ meydana geleceği anlamında makine bağımlıdır.
2. Temel ayrım, bir derleyicinin kaynak programdaki her bir talimatı tek bir makine talimatına çevirmesi, oysa bir derleyicinin genellikle tek bir kayn ak program talimatının eşdeğerini elde etmek için birçok makine dili tali matı üretmesidir.
3. Açıklayıcı paradigma, çözülmesi gereken problemin bir tanımını geliştirm eye dayanır. Fonksiyonel paradigma, programcıyı problemin çözüm ünü daha küçük problemlerin çözümleri açısından tanımlamaya zorlar. Nesne yönelimli paradigma, problemin çevresindeki bileşenleri tanım lamaya vurgu yapar.
4. Üçüncü nesil diller, programın problemin çevresi açısından daha fazla if ade edilmesine ve daha önceki nesil dillerden daha az bilgisayar jargon uyla ifade edilmesine olanak tanır.

Bölüm 6.2

1. Açıklayıcı bir sabit kullanmak, programın erişilebilirliğini artırabilir.
2. Açıklayıcı bir ifade terminolojiyi tanımlar; zorlayıcı bir ifade bir algorit madaki adımları tanımlar.
3. Tamsayı, reel, karakter ve Boolean
4. `if-then-else` ve`while` döngü yapıları çok yaygındır.
5. Bir dizinin tüm bileşenleri aynı türdendir.

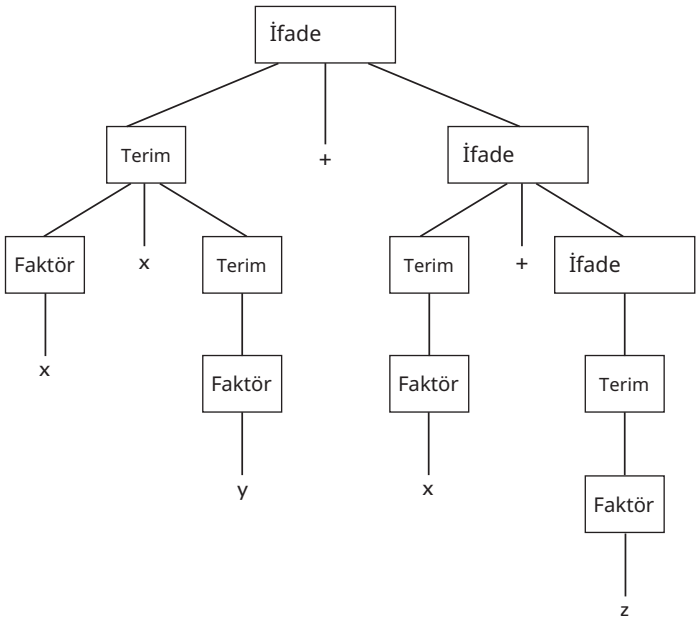
Bölüm 6.3

1. Bir değişkenin kapsamı, o değişkenin erişilebilir olduğu programın aralı ğıdır.

Doclingo https://www.doclingo.ai

2. Bir fonksiyon, fonksiyonun adıyla ilişkili bir değeri döndüren bir prosedü rdür.
3. Çünkü durum budur. G/Ç işlemleri aslında makinenin işletim sistemind eki rutinlere yapılan çağrılardır.
4. Bir resmi parametre, bir prosedür içindeki bir tanımlayıcıdır. Prosedür ç ağrıldığında, prosedüre geçirilen gerçek parametre için bir yer tutucu görevi görür.
5. Bir prosedür bir eylemi gerçekleştirmek için tasarlanmıştır, oysa bir fonksiyo n bir değer üretmek için tasarlanmıştır. Bu nedenle, bir prosedürün adı gerçekl eştirdiği eylemi yansıtıyorsa ve bir fonksiyonun adı döndürdüğü değeri yansıtıyorsa p rogram daha okunabilir hale gelir.

Bölüm 6.4

1. *Lehçe analizi:* token'ları tanımlama süreci.
   Parçalama: programın dilbilgisel yapısını tanıma süreci.
   Kod üretimi: nesne programındaki talimatları üretme süreci.
2. Bir sembol tablosu, ayrıştırıcının programın deklaratif ifadelerinden e lde ettiği bilgilerin kaydıdır.
3. Sözdizim diyagramlarında, oval içinde yer alan terimler terminaldir. Daha fazla tanım gerektiren terimler dikdörtgenler içinde yer alır ve "nonterminal" olarak adlandırılır.
4. 

5. Chacha yapısına uyan dizeler, aşağıdakilerden bir veya daha fazlasını iç ermektedir:
   ileri geri cha cha cha
   geri ileri cha cha cha
   sağa sallan cha cha cha
   sola sallan cha cha cha

Doclingo https://www.doclingo.ai

Bölüm 6.5

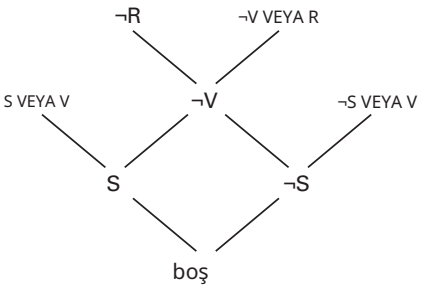
1. Bir sınıf, bir nesnenin tanımıdır.
2. Muhtemelen `MeteorClass`, çeşitli meteorların inşa edileceği sınıf ol acaktır. `LaserClass` içinde, lazerin nişan aldığı yönü belirten `AimDirection` adında bir örnek değişkeni bulunabilir. Bu değişken m uhtemelen fire,turnRight ve `turnLeft` yöntemleri tarafından kulla nılacaktır.
3. Çalışan sınıfı, bir çalışanın adı, adresi, hizmet yılı gibi özellikleri içere bilir. Tam Zamanlı Çalışan sınıfı emeklilik yardımları ile ilgili özellikleri içe rebilir. Yarı Zamanlı Çalışan sınıfı ise haftada çalışılan saatler, saatlik ücr et gibi özellikleri içerebilir.
4. Bir yapıcı, bir sınıfın bir örneği oluşturulduğunda yürütülen özel b ir yöntemdir.
5. Bir sınıftaki bazı öğeler, diğer program birimlerinin bu öğelere doğruda n erişimini engellemek için özel olarak belirlenmiştir. Bir öğe özel is e, o öğeyi değiştirme sonuçları sınıfın içiyle sınırlı olmalıdır.

Bölüm 6.6

1. Liste, eşzamanlı süreçlerin yürütülmesini başlatma teknikleri ni ve süreçler arası iletişim uygulama tekniklerini içerecektir.
2. Biri yükü süreçlere koymak, diğeri yükü verilere koymaktır. İkincisi, gör evi programda tek bir noktada yoğunlaştırma avantajına sahiptir.
3. Bunlar hava durumu tahmini, hava trafiği kontrolü, karmaşık sistemlerin simüla syonu (nükleer tepkimelerden yaya trafiğine kadar), bilgisayar ağları ve verita banı bakımı gibi alanları içerir.

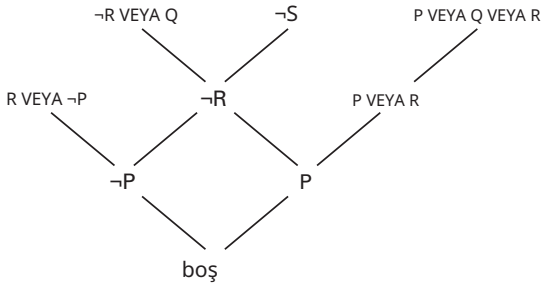
Bölüm 6.7

1. R, T ve V. Örneğin, R'nin bir sonuç olduğunu, onun olumsuzluğunu koleksiyona ekleyerek ve çözümlemenin boş ifadeye yol açabileceğini göstererek gös terebiliriz, burada gösterildiği gibi:



Doclingo https://www.doclingo.ai

2. Hayır. Koleksiyon tutarsızdır, çünkü çözümleme boş ifadeye yol açabilir, burada gösterildiği gibi:



3. anne(X, Y) :- ebeveyn(X, Y), dişi(X).
baba(X, Y) :- ebeveyn(X, Y), erkek(X).

4. Prolog, carol 'un kendi kardeşi olduğunu sonuçlandıracaktır. Bu sorunu çözmek için, kurala X'nin Y'ye eşit olamayacağı gerçeği eklenmelidir, bu da Prolog'da X \Y olarak yazılır. Böylece kuralın geliştirilmiş bir versiyonu kardeş (X, Y) :X \Y, ebeveyn(Z, X), ebeveyn(Z, Y) olacaktır.

   X'in Y'nin kardeşi olduğunu söyleyen, eğer X ve Y eşit değilse ve ortak bir ebeveynl eri varsa. Aşağıdaki versiyon, X ve Y'nin yalnızca her ikisinin de ortak ebeveynleri varsa kardeş olduğunu ısrarla belirtir:

```
kardeş (X, Y) :      – X \= Y, Z \= W
                       ebeveyn (Z, X), ebeveyn (Z, Y),
                       ebeveyn (W, X), ebeveyn (W, Y).
```

Bölüm 7

Bölüm 7.1

1. Uzun bir atama ifadeleri dizisi, program tasarımı bağlamında birkaç iç i çe geçmiş if ifadeleri kadar karmaşık değildir.
2. Belirli bir kullanım süresinden sonra bulunan hata sayısı ne olacak? Buradaki bir sorun, bu değerin önceden ölçülememesidir.
3. Buradaki nokta, yazılım özelliklerinin nasıl ölçülebileceğini düşünmektir. Bir yazılım parçasındaki hata sayısını tahmin etmenin bir yaklaşımı, yazı lım tasarlandığında kasıtlı olarak bazı hatalar yerleştirmektir.
   Daha sonra, yazılımın sözde hata ayıklaması yapıldıktan sonra, orijinal hatalar dan kaç tanesinin hala mevcut olduğunu kontrol edin. Örneğin, yazılıma kasıtlı olarak yedi hata yerleştirirseniz ve hata ayıklama sonrasında beşinin kaldırıl dığını bulursanız, o zaman yazılımdaki toplam hataların yalnızca $^{5}/_{7}$ kadarının kal dırıldığını varsayabilirsiniz.
4. Olası cevaplar arasında metriklerin keşfi, önceden üretilmiş bileşenlerin gelişti rilmesi, CASE araçlarının geliştirilmesi ve standartlara doğru bir hareket bul unmaktadır. Diğer bir konu ise, Bölüm 7.5'te daha sonra ele alınacak olan U ML gibi modelleme ve notasyon sistemlerinin geliştirilmesidir.

Doclingo https://www.doclingo.ai

Bölüm 7.2

1. Küçük çabalar, geliştirme sırasında bakımda muazzam kazançlar sağla yabilir.
2. Gereksinim analizi aşaması, önerilen sistemin neyi başarması gerektiğ ine odaklanır. Tasarım aşaması, sistemin hedeflerine nasıl ulaşacağın a odaklanır. Uygulama aşaması, sistemin gerçek inşasına odaklanır. Test aşaması, sistemin amacına uygun olarak çalıştığından emin olma ya odaklanır.
3. Yazılım gereksinimleri spesifikasyonu, bir müşteri ile bir yazılım mühen dislik firması arasında geliştirilecek yazılımın gereksinimlerini ve spesifikas yonlarını belirten yazılı bir anlaşmadır.

Bölüm 7.3

1. Geleneksel şelale yaklaşımı, gereksinim analizi, tasarım, uygula ma ve test aşamalarının lineer bir şekilde gerçekleştirilmesini zorunlu kılar. Daha yeni modeller, daha rahat bir deneme-yanılma yaklaşımına i zin verir.
2. Artık kademeli model, yinelemeli model ve XP hakkında ne dersiniz?
3. Geleneksel evrimsel prototipleme, yazılımı geliştiren organizasyon için de gerçekleştirilirken, açık kaynak geliştirme bir organizasyonla sınırlı de ğildir. Açık kaynak geliştirme durumunda, geliştirmeyi denetleyen kişi, h angi iyileştirmelerin rapor edileceğini belirlemek zorunda değildir; o ysa geleneksel evrimsel prototipleme durumunda, yazılım geliştirm eyi yöneten kişi, belirli iyileştirme görevlerine personel atar.
4. Bu, düşünmeniz gereken bir konu. Eğer bir yazılım geliştirme şirketin de bir yönetici olsaydınız, şirketiniz tarafından satılacak yazılımın geliş tirilmesi için açık kaynak metodolojisini benimseyebilir miydiniz?

Bölüm 7.4

1. Bir romanın bölümleri birbirine bağlıdır, oysa bir ansiklopedideki bölümler büy ük ölçüde bağımsızdır. Bu nedenle, bir romanın bölümleri arasında bir ansiklop edinin bölümleri arasında olduğundan daha fazla bağlantı vardır. Ancak, bir ansi klopedideki bölümler muhtemelen bir romandaki bölümlerden daha yüksek bir bütünlük seviyesine sahiptir.
2. Biriken puan, veri bağlantısına bir örnek olacaktır. Var olabilecek di ğer "bağlantılar" arasında yorgunluk, momentum, rakibin stratejisi hak kında edinilen bilgi ve belki de öz güven yer alır. Birçok sporda, birimler in bütünlüğü, eylemi sonlandırarak ve bir sonraki birimi taze bir başlan gıçtan başlatarak artırılır. Örneğin, beyzbolda her inning, takımın önc eki inningi yüklü üslerle bitirmiş olsa bile, herhangi bir üs koşucusu olma dan başlar. Diğer durumlarda, birimler ayrı ayrı puanlanır; tenis gibi, her set diğer setlere bakılmaksızın kazanılır veya kaybedilir.

Doclingo https://www.doclingo.ai

3. Bu zor bir durum. Bir bakış açısına göre, her şeyi tek bir modüle yerleşt irerek başlayabiliriz. Bu, çok az bir bütünlük ve hiç bağlantı olmamasıyla so nuçlanır. Daha sonra bu tek modülü daha küçük parçalara ayırmaya başla rsak, sonuç bağlantıda bir artış olacaktır. Bu nedenle, bütünlüğü artırmanın bağlantıyı artırma eğiliminde olduğunu söyleyebiliriz.

   Öte yandan, ele alınan problemin doğal olarak üç çok uyumlu modüle b ölündüğünü varsayalım, bunlara A, B ve C diyeceğiz. Eğer orijinal tas arımımız bu doğal bölünmeyi göz önünde bulundurmuyorsa (örne ğin, görev A'nın yarısı görev B'nin yarısıyla birleştirilmiş olabilir, vb.), uyu mun düşük ve bağlantının yüksek olmasını bekleriz. Bu durumda, A, B ve C görevlerini ayrı modüllere ayırarak sistemi yeniden tasarla mak, muhtemelen modüller arası bağlantıyı azaltırken, modül içi uyu mu artıracaktır.

4. Bağlantı, modüller arasındaki bağlantıdır. Uyum, bir modül içindeki ba ğlılıktır. Bilgi gizleme, bilgi paylaşımının kısıtlanmasıdır.

5. Muhtemelen, `ControlGame`'in `UpdateScore`'e voleyi kimin kazandığını bildirmesi ge rektiğini gösteren bir ok eklemelisiniz ve diğer yönde, `UpdateScore`'in kontrolü `ControlGame`'e geri verdiğinde mevcut durumu (örneğin "set bitti" veya "m aç bitti") bildireceğini gösteren başka bir ok eklemelisiniz.

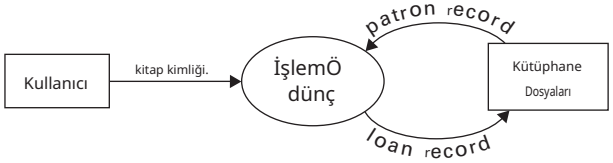
6. Şekil 7.5'teki tüm yatay okları, ilk ve son hariç silin.
   Yani, hakem PlayerA'nın servis atışını değerlendirmeli ve doğrudan Score'e upd ateScore mesajını göndermelidir. (Bu, elbette, ikinci bir servis şansını göz ardı ede r. Program tasarımını çift hata olasılığını dikkate alacak şekilde nasıl değiştirebilirsi niz?)

7. Geleneksel bir programcı, Bölüm 6'da tanıtılan ifadeler gibi ifadelerle progra mlar yazar. Bir bileşen montajcısı, bileşenler olarak adlandırılan önceden üretil miş blokları birleştirerek programlar oluşturur.

8. Bu sorunun birçok yanıtı vardır. Bir kombinasyon, takvimin bir saatte alarm ayarlayarak kullanıcıyı yaklaşan bir randevudan haberdar etme sidir. Ayrıca, takvim uygulaması, randevun adresine yönlendirmeleri s ağlamak için bir harita uygulamasının bileşenlerini kullanabilir.

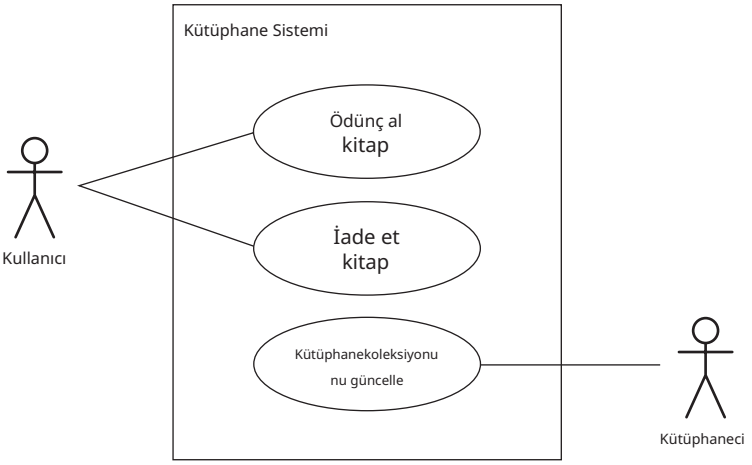
Bölüm 7.5

1. Şemanızın veri akışıyla (kitapların hareketiyle değil) ilgilendiğinden emin olun. A şağıdaki şema, kitap tanımlamalarının (kullanıcılardan) ve kullanıcı kayıtlarının (kütüphane dosyalarından) birleştirilerek kütüphane dosyalarında saklanan ödünç kayıtlarını oluşturduğunu göstermektedir.



Doclingo https://www.doclingo.ai

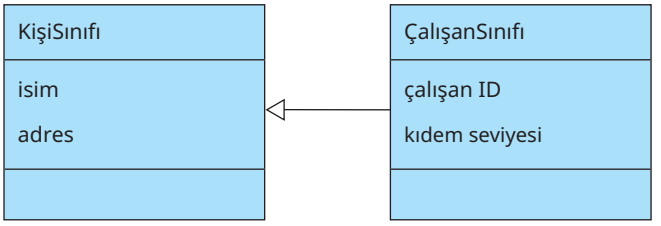
2.



3.



4.



5. Şeklin etrafında bir dikdörtgen çizin ve sol üst köşeye Şekil 7.13'teki gibi bir “sd” etiketi ekleyin.
6. Tasarım kalıpları, tekrarlayan yazılım temalarını uygulamak için standartlaştırılmış, iyi geliştirilmiş yaklaşımlar sunar.

Bölüm 7.6

1. SQA (yazılım kalite güvencesi) grubu, organizasyon tarafından benimsenen kalite kontrol sistemlerini denetler ve uygular.
2. İnsanların bir projede aldıkları adımları (kararlar, eylemler vb.) kaydetme eğilimleri yoktur.(Kişilik çatışmaları, kıskançlıklar ve ego çatışmaları gibi sorunlar da vardır.)
3. Kayıt tutma ve gözden geçirme.
4. Yazılım testinin amacı hataları bulmaktır. Bu anlamda, bir hatayı ortaya çıkarmayan bir test başarısızlıktır.
5. Modüllerdeki dallanma miktarını dikkate almak bir yol olabilir. Örneğin, birçok döngü ve if-then-else ifadeleri içeren bir prosedürel modül, basit bir mantıksal yapıya sahip bir modülden daha fazla hata yapma eğiliminde olacaktır.
6. Sınır değeri analizi, yazılımı 100 girişli bir liste üzerinde ve hiç girişi olmayan bir liste üzerinde test etmenizi önerecektir. Ayrıca, zaten doğru sıralanmış bir liste ile de bir test yapabilirsiniz.

Doclingo https://www.doclingo.ai

Bölüm 7.7

1. Belgeler, kullanıcı belgeleri, sistem belgeleri ve teknik belgeler biçimind e olabilir. Kullanım kılavuzlarında, kaynak programda yorumlar ve iyi yazı lmış kod biçiminde, programın kendisinin bir terminalde yazdığı etkile şimli mesajlar aracılığıyla, veri sözlükleri aracılığıyla ve yapı şemaları, sını f diyagramları, veri akış diyagramları ve varlık-ilişki diyagramları gibi tas arım belgeleri biçiminde görünebilir.

2. Hem geliştirme hem de değiştirme aşamalarında. Nokta, değişikliklerin ori jinal program kadar ayrıntılı bir şekilde belgelenmesi gerektiğidir. (Yazılı mın kullanım aşamasında belgelenmesi de doğrudur. Örneğin, sistemin bir kullanıcısı sorunlar keşfedebilir; bu sorunlar, düzeltmek yerine, sist em kullanıcı kılavuzunun gelecekteki baskılarında yalnızca rapor edilir.) Ayrıca, "nasıl yapılır" kitapları genellikle yazılım uzun bir süre kullanıldıkt an sonra üretilir.)
3. Bu konuda farklı insanların farklı görüşleri olacaktır. Bazıları programın t üm projenin noktası olduğunu ve dolayısıyla doğal olarak daha önemli ol duğunu savunacaktır. Diğerleri ise, bir program belgelenmemişse hiçbir değeri olmadığını, çünkü bir programı anlayamazsanız, onu kullanamaz veya değiştiremezsiniz. Ayrıca, iyi bir belgelerle, programı oluşturma görev i "kolayca" yeniden yaratılabilir.

Bölüm 7.8

1. a. Bir ekranın eğimini veya bir farenin şeklini ayarlama yeteneği hakkında ne dersi niz? Akıllı telefonlarda, bir fare yerine dokunmatik ekranların kullanımı veya girişi sa ğlamak için telefonun eğilmesi hakkında ne dersiniz?
   b. Ekrandaki bir pencerenin düzeni, araç çubuklarının, kaydırma asansörlerinin ve açılır menülerin tasarımı dahil olmak üzere? Bir akıllı telefonda, kamerayı ilgi alanındaki nesnelere yönlendirmek, insanların düşünme şekliyle uyu mlu değil mi?
2. a. Bir akıllı telefonda fare (veya hatta bir kalem) kullanmak pratik ve uygun olmayacaktır. Ayrıca, ekranın küçülmüş boyutu, ekranın ger eksiz unsurlarının sınırlı bir alana sıkıştırılmasını gerektirir. Bu nedenl e, kaydırma çubukları genellikle atlanır. Varsa, kaydırma çubukları i nce çizgiler olarak gösterilir.
   b. Ekranda kaydırma hareketi, düşünme şeklimize doğal bir jesttir. Masa da kağıtları veya diğer eşyaları kaydırarak hareket ettirebiliriz. Bunun, bir masaüstü bilgisayardaki kaydırma çubuklarının kullanımından dah a doğal olduğu iddia edilebilir. Gerçekten de kaydırma çubuğu bekle nildiği gibi hareket eder, ancak kaydırılan alan ters yönde hareket eder. Bil gisayar kullanmamış bir kullanıcı için bu davranış sezgisel olmayabilir.
3. "İnsan özelliklerinin rolü" şeklinde cevap verebilirsiniz. Diğer bir iyi ceva p, arayüz tasarımının bir yazılım sisteminin içsel özelliklerinden ziya de dışsal özelliklerine odaklandığı olacaktır.
4. Metinde tartışılan üç şey alışkanlıkların oluşumu, dikkat daralması ve sın ırlı çoklu işlem yetenekleridir. Başka şeyler hayal edebilir misiniz? Va rsayımlarda bulunma eğilimi hakkında ne dersiniz?

Doclingo https://www.doclingo.ai

Bölüm 7.9

1. Telif hakkı bildirimi, eserin mülkiyetini iddia eder ve eseri kullanmaya y etkili personeli tanımlar. Gereksinim spesifikasyonları, tasarım belgeleri, kaynak kodu ve nihai ürün dahil tüm eserler genellikle üretim için önem li bir yatırım gerektirir. Bir birey veya şirket, mülkiyet haklarının saklandı ğından ve tüm fikri mülkiyetin istenmeyen taraflarca kullanılmadığından emin olmak için adımlar atmalıdır.
2. Telif hakkı ve patent yasaları, yeni ürünlerin yaratıcılarını kamuya sun maya teşvik ettikleri için topluma fayda sağlar.Bu tür bir koruma olmad an, şirketler yeni ürünlere büyük yatırımlar yapmaktan çekineceklerdir.
3. Bir feragatname, bir şirketi ihmalden korumaz.

Bölüm 8

Bölüm 8.1

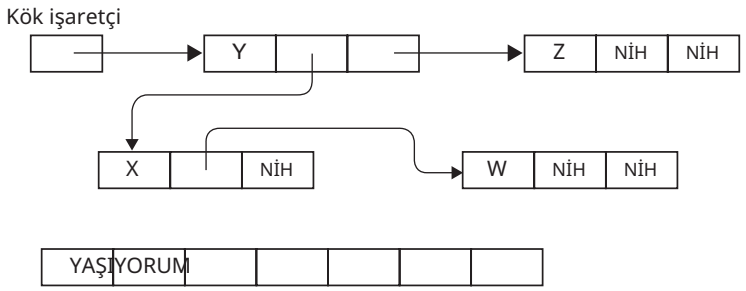
1. Liste: Bir spor takımının üyelerinin listesi.
   Yığın: Bir kafeteryadaki tepsi yığını.
   Kuyruk: Bir kafeteryadaki sıra.
   Ağaç: Birçok hükümetin organizasyon şeması.
2. Yığınlar ve kuyruklar, özel türde listeler olarak düşünülebilir. Genel bir l istede, girişler herhangi bir konumda eklenip çıkarılabilir. Bir yığın dur umunda, girişler yalnızca baştan eklenip çıkarılabilir. Bir kuyruk duru munda, girişler yalnızca kuyrukta eklenebilir ve girişler yalnızca baştan çıkarılabilir.
3. Yığındaki harfler üstten alta doğru E, D, B ve A olacaktır. Eğer bir harf yığ ından çıkarılırsa, bu harf E olacaktır.
4. Kuyruktaki harfler baştan sona B, C, D ve E olacaktır. Eğer bir harf kuyruktan çıkarılacak olursa, bu harf B olacaktır.
5. Yaprak (veya terminal) düğümler D ve C'dir. B, diğer tüm düğümlerin ebeveynleri olduğu için kök düğüm olmalıdır.

Bölüm 8.2

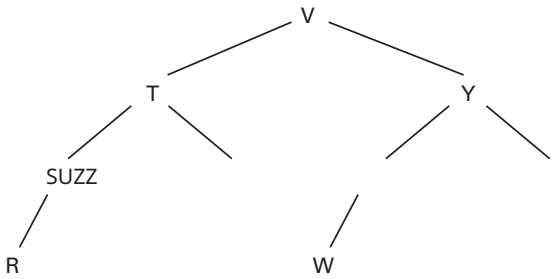
1. Bir bilgisayarın ana belleğindeki veriler aslında bireysel olarak adreslenebilir bellek hücrelerinde saklanır. Verilerin kullanıcılarına daha erişilebilir hale g etirilmesi için diziler, listeler ve ağaçlar gibi yapılar simüle edilir.
2. Bir dama oyunu oynamak için bir program yazsaydınız, dama tahta sını temsil eden veri yapısı muhtemelen statik bir yapı olurdu çünkü ta htanın boyutu oyun sırasında değişmez. Ancak, bir domino oyunu oyn amak için bir program yazsaydınız, masada inşa edilen domino dese nini temsil eden veri yapısı muhtemelen dinamik bir yapı olurdu çün kü bu desenin boyutu değişir ve önceden belirlenemez.
3. Bir telefon rehberi esasen insanlara ait işaretleyicilerin (telefon numaral arının) bir koleksiyonudur. Bir suç mahallinde bırakılan ipuçları, (belki de şifrelenmiş) failin işaretleyicileridir.

Doclingo https://www.doclingo.ai

Bölüm 8.3

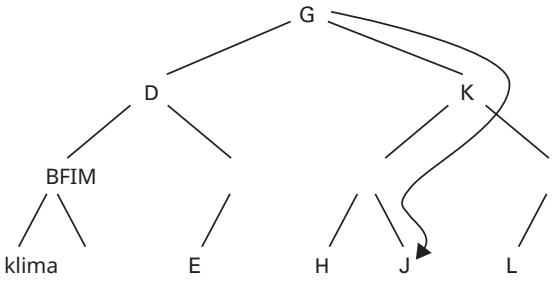
1. 5 3 7 4 2 8 1 9 6
2. Eğer$R$ matrisin satır sayısı ise, formül $R(J$ $-1)+$ (ile ilgili $1)$.
3. $(c\ i)\ j$
4. Baş işaretleyici NIL değerini içerir.
5. Son←son basılacak isim
   Tamamlandı←`false`
   Geçerli İşaretleyici ←baş işaretleyici;
   while (Geçerli İşaretleyici NIL değil ve Tamamlandı = `false`) do
   (Geçerli İşaretçinin işaret ettiği girişi yazdırın,
   if (az önce yazdırılan ad = Son)
   sonra (Tamamlandı ←doğru)
   Mevcut İşaretçi ←Mevcut İşaretçi tarafından işaret edil
   en girişteki işaretçi hücresindeki değer)
6. Yığın işaretçisi, yığının tabanının hemen altındaki hücreye işaret ed er.
7. Yığını tek boyutlu bir dizi olarak ve yığın işaretçisini tam sayı türünde bir değişken olarak temsil edin. Ardından, bu yığın işaretçisini, yığının te pe konumunu dizideki kesin bellek adresi yerine kaydetmek için ku llanın.
8. Hem boş hem de dolu koşullar, eşit baş ve kuyruk işaretçileri ile gös terilir. Bu nedenle, iki koşul arasında ayrım yapmak için ek bilgi gereklidi r.
9. 

Bölüm 8.4

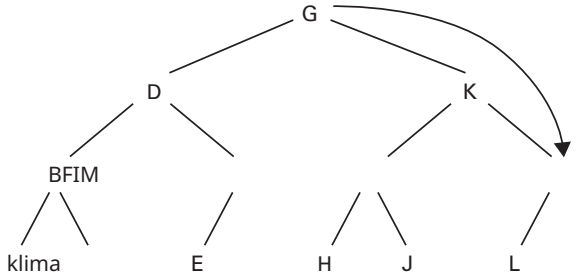
1. 

Doclingo https://www.doclingo.ai

2. J'yi ararken:



P'yi ararken:



3.

```
prosedür  PrintTree(Ağaç)

Eğer (Ağaç'ın kök işaretçisi NIL değilse)
    o zaman  (PrintTree prosedürünü
                Ağaç'taki sol dalda görünen
                ağaç için uygula;
              Ağaç'ın kök düğümünü yazdır;
              PrintTree prosedürünü Ağaç'taki sağ dalda görünen
                ağaç için uygula.)
```

```
prosedür  PrintTree(Ağaç)

Eğer (Ağaç'ın kök işaretçisi NIL değilse)
    o zaman  (PrintTree prosedürünü
                Ağaç'taki sol dalda görünen
                ağaç için uygula;
              Ağaç'ın kök düğümünü yazdır;
              PrintTree prosedürünü Ağaç'taki sağ dalda görünen
                ağaç için uygula.)
```

Burada, K yazdırıldığında

4. Her düğümde, her çocuk işaretçisi alfabedeki benzersiz bir harfi temsil etme
k için kullanılabilir. Bir kelime, kelimenin yazılışını temsil eden işaretçiler dizisi
boyunca ağaçta aşağıya doğru bir yol ile temsil edilebilir. Bir düğüm, d
oğru yazılmış bir kelimenin sonunu temsil ediyorsa özel bir şekilde işaretl
enebilir.

Bölüm 8.5

1. Bir tür, bir şablondur; o türün bir örneği, o şablondan oluşturulmuş gerçek bir varlıktır.
Bir benzetme olarak, köpek bir hayvan türüdür, oysa Lassie ve
Rex o türün örnekleridir.
2. Kullanıcı tanımlı bir veri türü, veri organizasyonunun bir tanımıdır, oys
a soyut bir veri türü, veriyi işlemek için işlemleri içerir.
3. Burada belirtilmesi gereken bir nokta, listeyi sürekli bir liste veya bağlı
bir liste olarak uygulama seçeneğinizin olduğudur. Seçiminiz,

Doclingo https://www.doclingo.ai

yeni girişler ekleme, eski girişleri silme ve ilgi alanındaki girişleri bulma pr osedürlerinin yapısını etkileyecektir. Ancak, bu seçim, soyut veri türünün bi r örneğinin kullanıcısına görünmemelidir.

4. Soyut veri türü, en azından hesap bakiyesini saklamak için bir veri yapı sının tanımını ve bir çek ile para yatırma ve para çekme işlemleri için prosedürleri içerecektir.

Bölüm 8.6

1. Hem soyut veri türleri hem de sınıflar, bir türün örneklerini oluşt urmak için şablonlardır. Ancak, sınıflar, miras ile ilişkili olduklarından daha genel olup yalnızcaprosedürlerin bir koleksiyonunu tanımla yabilirler.
2. Bir sınıf, nesnelerin inşa edildiği bir şablondur.
3. Sınıf, giriş ekleme, giriş kaldırma, kuyruğun dolu olup olmadığını test etm e ve kuyruğun boş olup olmadığını test etme prosedürleri ile birlikte bir dairesel kuyruk içerebilir.

Bölüm 8.7

1. a. A5 b. A5 c. CA
2. D50F, 2EFF, 5FFE
3. 2EA0, 2FB0, 2101, 20B5, D50E, E50F, 5EE1, 5FF1, BF14, B008, C000
4. Her bir kaydın iki bellek hücresinden (bir veri hücresi ve ardından bir sonra ki kayda işaret eden bir gösterici) oluştuğu bağlı bir listeyi gezerken, veri al mak için DR0S biçiminde bir komut kullanılabilir ve bir sonraki kayda işaret e den göstericiyi almak için DR1S kullanılabilir. DRTS biçimi kullanılırsa, o zama n referans alınan tam bellek hücresi, T kaydındaki değeri değiştirerek aya rlanabilir.

Bölüm 9

Bölüm 9.1

1. Satın alma departmanı, daha fazla ham madde siparişi vermek için e nvanter kayıtlarına ilgi duyarken, muhasebe departmanı bilgiyi defterleri dengelemek için ihtiyaç duyacaktır.
2. Bir veritabanı modeli, veritabanının uygulamalarla daha uyumlu olan organi zasyonel bir perspektifini sağlar. Böylece, bir veritabanı modelinin tanıml anması, veritabanının soyut bir araç olarak kullanılmasına izin vermek için atılacak ilk adımdır.
3. Uygulama yazılımı, kullanıcının isteklerini uygulamanın terminolojisin den veritabanı yönetim sistemi tarafından desteklenen veritabanı model ine uyumlu terminolojiye çevirir. Veritabanı yönetim sistemi, bu istekler i gerçek veritabanında eylemlere dönüştürür.

Doclingo https://www.doclingo.ai

Bölüm 9.2

1. a. G. Jerry Smith b. Cheryl H. Clark c. S26Z
2. Bir çözüm şudur

```
TEMP←İŞTEN SEÇ
                    nerede Dept = "PERSONEL"
LIST←TEMP'den JobTitle PROJE
```

Bazı sistemlerde bu, personel departmanında kaç kez meydana geldiğin e bağlı olarak, iş unvanının tekrarlandığı bir liste ile sonuçlanır. Yani, lis temiz sekreter unvanının birçok kez geçmesini içerebilir. Ancak, PROJE i şlemini tasarlamak daha yaygındır, böylece sonuç ilişkisi içinden tekra r eden tuple'ları kaldırır.

3. Bir çözüm şudur

```
TEMP1←İŞ ve GÖREVİ BİRLEŞTİR
                    nerede JOB.JobId = GÖREV.JobId
TEMP2←TEMP1'den SEÇ
                    TermDate = "*"
TEMP3←ÇALIŞAN ve TEMP2'yi BİRLEŞTİR
                    ÇALIŞAN.EmplId = TEMP2.EmplId
  SONUÇ←PROJE Adı, Departman TEMP3'ten
```

4. iş unvanını seç
   İŞ'ten
   Departman = "PERSONEL"

```
ÇALIŞAN.Adı, İŞ.Departmanını seç
   İŞ, GÖREV ve ÇALIŞAN'dan
   (İş.Job = GÖREV.JobId) ve
      (GÖREV.EmplId = ÇALIŞAN.EmplID)
       ve (ATAMA.TermDate = "*")
```

5. Modelin kendisi veri bağımsızlığı sağlamaz. Bu, veri yönetim sisteminin bir özelliğidir. Veri bağımsızlığı, veri yönetim sistemine uygulama yazılı mına tutarlı bir ilişkisel organizasyon sunma yeteneği vererek elde edi lir, gerçekteki organizasyon değişse bile.

6. Ortak nitelikler aracılığıyla. Örneğin, bu bölümdeki ÇALIŞAN ilişkisi, `EmplId` niteliği aracılığıyla ATAMA ilişkisine bağlıdır ve ATAMA ilişkisi, J OB ilişkisine JobId niteliği ile bağlıdır.
İlişkileri bu şekilde bağlamak için kullanılan niteliklere bazen bağlantı nitelikleri denir.

Bölüm 9.3

1. Başlangıç Tarihi ve TermDate atama ve alma yöntemleri olabilir. Hiz metteki toplam süreyi raporlamak için başka bir yöntem sağlanabilir.

2. Sürekli bir nesne, süresiz olarak saklanan bir nesnedir.
3. Bir yaklaşım, envanterdeki her ürün türü için bir nesne oluşturmaktır. Bu nesnelerin her biri, ürününün toplam envanterini, ürünün maliyetin i ve ürün için bekleyen siparişlere bağlantıları sürdürebilir.

Doclingo https://www.doclingo.ai

4. Bu bölümün başında belirtildiği gibi, nesne yönelimli veritabanları bileşik veri türlerini ilişkisel veritabanlarından daha kolay bir şekilde işliyor gibi görünmektedir. Ayrıca, nesnelerin soruları yanıtlamada aktif bir rol üstlenen yöntemler içer ebileceği gerçeği, nesne yönelimli veritabanlarının, yalnızca verileri tu tan ilişkisel veritabanlarına göre bir avantaj sağlamasını vaat ediyor.

Bölüm 9.4

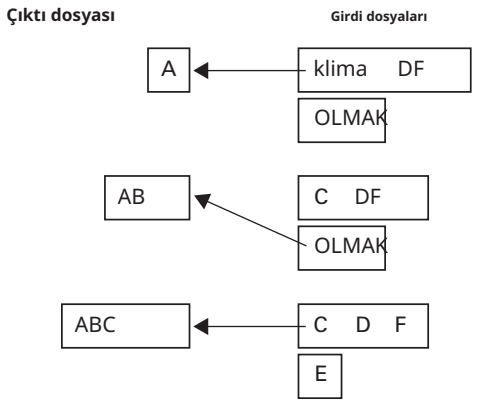
1. Bir işlem, taahhüt noktasına ulaştığında, veritabanı yönetim sistemi, işle min tamamının veritabanında gerçekleştirilmesi sorumluluğunu üstleni r. Taahhüt noktasına ulaşmamış bir işlem bu tür bir güvenceye sahip d eğildir. Sorunlar ortaya çıkarsa, yeniden sunulması gerekebilir.

2. Bir yaklaşım, tüm mevcut işlemlerin tam olarak tamamlanabilmesi için işlemleri b ir anlığına birbirine karıştırmayı durdurmak olacaktır. Bu, gelecekteki bir zincirleme geri alma işleminin sona ereceği bir nokta belirleyecektir.
3. İşlemler birer birer yürütülürse, 100 dolarlık bir bakiye oluşur. İlk işle m, ikinci işlem orijinal bakiyeyi aldıktan sonra ve o ikinci işlem yeni bak iyesini saklamadan önce yürütülürse, 200 dolarlık bir bakiye oluşur. İlk işlem orijinal bakiyeyi aldıktan sonra ve ilk işlem yeni bakiyesini saklam adan önce ikinci işlem yürütülürse, 300 dolarlık bir bakiye oluşur.

4. a. Eğer başka bir işlem özel erişime sahip değilse, paylaşılan erişim verilecektir.
   b. Eğer başka bir işlem zaten bir tür erişime sahipse, veritabanı yönetim sistemi genellikle yeni işlemi bekletir veya diğer işlemleri geri alarak y eni işleme erişim verebilir.
5. Her iki işlemin de farklı öğelere özel erişim sağladığı ve ardından diğerine erişim t alep ettiği durumlarda bir deadlock meydana gelecektir.
6. Önceki deadlock, işlemlerden birinin geri alınması (log kullanılarak) ve diğ er işleme daha önce birincisi tarafından tutulan veri öğesine erişim veril mesiyle ortadan kaldırılabilir.

Bölüm 9.5

1. Bu başlangıç aşamalarında yönlendirilmelisiniz:



Doclingo https://www.doclingo.ai

2. Fikrin ilk olarak saklanacak dosyayı, her biri bir kayıt içeren birçok ayrı dosyaya bölmek olduğu. Sonra, bir kayıt dosyalarını çiftler halinde gruplandırın ve her çifte birleştirme algoritmasını uygulayın. Bu, her biri iki kayıt içeren yarı s ayıda dosya ile sonuçlanır. Ayrıca, bu iki kayıt dosyasının her biri sıralıdır. Onları çiftler halinde gruplandırabiliriz ve yine çiftlere birleştirme algoritm asını uygulayabiliriz. Yine daha az ama daha büyük dosyalarla karşı karşıya yız, her biri sıralıdır. Bu şekilde devam ederek, nihayetinde tüm orijinal kayıt ları içeren ancak sıralı bir dosya ile kalırız. (Bu süreçte herhangi bir aşamad a tek sayıda dosya oluşursa, tek olanı bir kenara koyup bir sonraki aşama da daha büyük dosyalardan biriyle eşleştirmemiz yeterlidir.)
3. Eğer dosya bantta veya CD'de saklanıyorsa, fiziksel organizasyonu muhtemelensıralıdır. Ancak, dosya manyetik diskte saklanıyorsa, o zaman muhtemelen diskin çeşitli sektörlerine dağılmıştır ve dosyanın sıralı doğası, dosyanın saklandığı sektörlerin kaydedildiği bir işaretçi sist emi veya bir liste biçiminde desteklenen kavramsal bir özelliktir.
4. Öncelikle dosyanın indeksinde hedef anahtarı bulun. Oradan, hedef kaydın konumunu elde edin. Ardından, o konumdaki kaydı alın.
5. Kötü seçilmiş bir hash algoritması, normalden daha fazla kümelenmeye veböylece da ha fazla taşmaya yol açar. Her bir kütle depolama bölümündenkaynaklanan taş ma, bağlı liste olarak organize edildiğinden, taşma kayıtlarıüzerinde arama yapmak e sasen sıralı bir dosyada arama yapmaktır.
6. Bölüm atamaları şu şekildedir:
   a.0b.0c.3d.0e.3
   f. 3 g. 3 h. 3 Ben. 3 j. 0

   Böylece tüm kayıtlar 0 ve 3 numaralı kovalarına hashlenir, 1, 2, 4 ve 5numaralı kovalar boş kalır. Buradaki sorun, kullanılan kova sayısının (6) veanahtar değerlerinin 3 orta k faktörüne sahip olmasıdır. (Bu anahtar değerlerini7 kova kullanarak yeniden hashle meyi deneyebilir ve ne tür bir iyileşme bulduğunuzugörebilirsiniz.)
7. Buradaki nokta, temelde gruptaki insanları 365 kategoriden birine y erleştirmek için bir hash algoritması uyguladığımızdır. Hash algoritma sı, elbette, birinin doğum gününün hesaplanmasıdır. Şaşırtıcı olan şey, e n az iki doğum gününün aynı olma olasılığının lehine olduğu için yal nızca yirmi üç kişinin gerekli olmasıdır. Hashlenmiş bir dosya açısından, bu, 365 mevcut depolama alanına kayıtları hashlerken, yalnızca yirmi üç kaydın girilmesinden sonra kümelenmenin muhtemel olduğunu gös terir.

Bölüm 9.6

1. Dinamik verilerde desenler aramak sorunludur.
2. Sınıf tanımı—Belirli bir derginin abonelerinin özelliklerini tanımlayın.
   Sınıf ayrımı—İki derginin aboneleri arasında ayırt edici özellikleri tanımla yın.
   Küme analizi—Benzer aboneleri çekme eğiliminde olan dergileri tanımlayın.
   İlişki analizi—Çeşitli dergilere abone olanlar ile farklı satın alma alışkanlıkla rı arasındaki bağlantıları tanımlayın.
   Aykırı değer analizi—Bir dergiye abone olan ve normal abonelerin profilini ka rşılamayan aboneleri tanımlayın.
   Sıralı desen analizi—Dergi aboneliğindeki eğilimleri tanımlayın.

Doclingo https://www.doclingo.ai

3. Veri küpü, satış verilerinin ay bazında, coğrafi bölge bazında, ürün sınıfı bazınd a vb. olarak görüntülenmesine olanak tanıyabilir.
4. Geleneksel veritabanı sorguları, veritabanında saklanan gerçekleri alır. Veri m adenciliği, gerçekler arasında desenler arar.

Bölüm 9.7

1. Buradaki nokta, bu soruya verdiğiniz yanıtı bir sonraki ile karşılaştırma ktır. İkisi de temelde aynı soruyu farklı bağlamlarda gündeme getiriyor.
2. Önceki soruya bakın.
3. Almadığınız fırsatlar için duyurular veya reklamlar alabilirsiniz, ancak ay nı zamanda talep konusu olabilir veya suçun hedefi haline gelebilirsiniz.
4. Buradaki nokta, özgür bir basının kamuoyunu kötüye kullanımlar veya potansiyel köt üye kullanımlar hakkında bilgilendirebileceği ve böylece kamuoyunu harekete geçirebileceğ idir. Metinde belirtilen çoğu durumda, kamuoyunu bilgilendirerek düzeltici eylemi başlatan özgü r bir basındı.

Bölüm 10

Bölüm 10.1

1. Görüntü işleme, iki boyutlu görüntülerin analiz edilmesiyle ilgilenirken, 2D gr afikler iki boyutlu şekilleri görüntülere dönüştürmekle ilgilenir ve 3D grafikle r üç boyutlu sahneleri görüntülere dönüştürmekle ilgilenir.
2. Geleneksel fotoğrafçılık, gerçek sahnelerin görüntülerini üretirken, 3D graf ikler sanal sahnelerin görüntülerini üretir.
3. İlki sanal sahneyi "inşa etmek". İkincisi ise görüntüyü yakalamaktır.

Bölüm 10.2

1. Adımlar modelleme (sahneyi inşa etme), render alma (bir resim üretme) ve görünt üleme (resmi görüntüleme) şeklindedir.
2. Görüntü penceresi, görüntüyü oluşturan projeksiyon düzleminin bir kısmıdır.
3. Bir çerçeve tamponu, bir görüntünün kodlanmış versiyonunu içeren bir bellek alanıdır.

Bölüm 10.3

1. Bu bir rombustur (sıkıştırılmış kare).
2. Bir prosedürel model, bir nesnenin inşasını yönlendiren bir program seg mentidir.
3. Liste, çimle kaplı zemin, taş yürüyüş yolu, bir gazebo, ağaçlar, çalılar, bulut lar, güneş ve aktörleri içerebilir. Buradaki nokta, bir sahne grafiğinin ka psamını vurgulamaktır—çok fazla ayrıntı içerebilir.
4. Tüm nesneleri çokgen ağlarla temsil etmek, render alma sürecine standart bir yaklaşım sağlar.(Çoğu durumda, render alma, nesneleri render etmekten zi yade düz yama parçalarını render etme görevi olarak ele alınır.)
5. Dokuların haritalanması, iki boyutlu bir görüntüyü bir nesnenin yüzeyiyle i lişkilendirme aracıdır.

Doclingo https://www.doclingo.ai

Bölüm 10.4

1. Speküler ışık, bir yüzeyden "doğrudan" yansıyan ışıktır. Difüz ışık, bir yüz eyden "dağılan" ışıktır. Ortam ışığı, kesin bir kaynağı olmayan ışıktır.
2. Klipleme, görünüm hacminin içinde yer almayan nesneleri (ve nesneleri n parçalarını) atma sürecidir.
3. Diyelim ki bir vurgunun bir yamada ortada görünmesi gerekiyor. O vurgu nun nedeni, yamadaki o noktadaki belirli bir yüzey yönelimidir. Gouraud gölgelemesi yalnızca yamaların sınırları boyunca yüzey yöneli mlerini dikkate aldığı için vurguyu kaçırır. Ancak, Phong gölgelemesi ya maların iç kısmındaki yüzey yönelimlerini belirlemeye çalıştığı için vurgu yu tespit edebilir.
4. Render alma hattı, render alımına standart bir yaklaşım sağlar; bu da ni hayetinde daha verimli render alma sistemlerine yol açar. Özellikle, render alma hattı firmware'de uygulanabilir, bu da render alma sürecinin, görevin geleneksel yazılım aracılığıyla uygulanmasından daha hızlı bir şekilde ge rçekleştirilebileceği anlamına gelir.
5. Bu sorunun amacı, belirli bir önceden belirlenmiş cevap üretmek yeri ne, yerel ve küresel aydınlatma modelleri arasındaki ayrımları düşünm enizi sağlamaktır. Önerdiğiniz potansiyel çözümler, aynayı şeffaf olarak düşünerek yansıtılacak nesnelerin uygun şekilde değiştirilmiş kopyalar ını aynanın arkasına yerleştirmek veya aynadaki görüntüleri bir tür gölge olarak ele almak olabilir.

Bölüm 10.5

1. Sadece nihayetinde görüntü penceresine ulaşan ışınlarla ilgileniyoruz. Işık kaynağından başlarsak, hangi ışınları takip etmemiz gerektiğini bilemeyiz.
2. Dağıtılmış ışın izleme, birden fazla ışın izleyerek geleneksel ışın izlem enin ürettiği doğal parlak görünümü önlemeye çalışır.
3. Radyozite zaman alıcıdır ve yansıtıcı etkileri yakalamakta başarısızdır.
4. Hem ışın izleme hem de radyozite küresel bir aydınlatma modeli uygular ve her ikisi de hesaplama açısından yoğundur. Ancak, ışın izleme genellikle parlak gör ünümlü yüzeyler üretirken, radyozite mat görünümlü yüzeylere yol açar.

Bölüm 10.6

1. Kesin bir cevap yoktur. Bir görüntü 200 milisaniye boyunca kalırsa ve s aniyede beş kare projekte edersek, her kare bir sonraki kare projekt e edildiğinde kaybolmuş olacaktır. Bu muhtemelen uzun süre izlenmesi rahatsız edici olacak titreşen bir görüntü ile sonuçlanır, ancak yine de a nimasyon etkisi yaratır. (Aslında, daha yavaş hızlar kaba animasyon üret ebilir.) Saniyede beş kare hızının, yirmi dört karelik sinema standardının oldukça altında olduğunu unutmayın.
2. Bir storyboard, istenen animasyon dizisinin "resimsel taslağı"dır.
3. Aradaki geçiş, ana kareler arasındaki boşlukları dolduran karelerin oluşturulmas ı sürecidir.

Doclingo https://www.doclingo.ai

4. Dinamizm, hareketi kuvvetlerin sonucu olarak analiz eden mekanik d alıdır. Kinematik ise hareketi, hareketi oluşturan kuvvetlere bakma ksızın analiz eden mekanik dalıdır.

Bölüm 11

Bölüm 11.1

1. Bölümde tanıtılanlar arasında refleks eylemler, gerçek dünya bilgisine da yalı eylemler, hedef arayış eylemleri, öğrenme ve algı bulunmaktadır.
2. Buradaki amacımız bu konuya kesin bir cevap vermek değil, zekanın varlığı üzer indeki tartışmanın ne kadar hassas olduğunu göstermek için kullanmaktır.
3. Çoğumuz muhtemelen hayır deriz, ancak benzer bir atmosferde aynı ür ünleri bir insanın dağıttığını varsaysak, farkı açıklayamıyor olsak bil e, bir bilinç mevcut olacaktır.
4. Doğru veya yanlış bir cevap yoktur. Çoğu kişi, makinenin en azından zek i göründüğünde hemfikirdir.
5. Doğru veya yanlış bir cevap yoktur. Chat botların, bir kişinin sohbetini takli t etmek için tasarlanmış programlar olduğu ve anlamlı bir sohbeti bile kısa bir s üre boyunca sürdürmekte zorluk çektiği belirtilmelidir. Chat botlar kolayca makineler olarak tanımlanabilir.

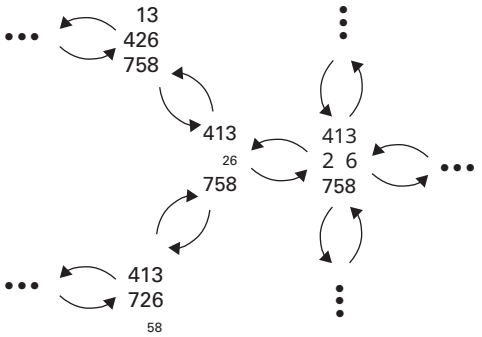
Bölüm 11.2

1. Uzaktan kumanda durumunda, sistem yalnızca resmi iletmek zorundadır, ancak resmi manevra yapmak için kullanmak için robotun resmi "anlaması" gerekmektedir.
2. Çizimin bir bölümündeki olası yorumlar, diğer bölümdekilerle eşleşm iyor.Bu içgörüyü bir programa yerleştirmek için, çeşitli çizgi kesişimleri içinizin verilen yorumları izole edebilir ve ardından uyumlu yorumlar setinibulmaya çalışan bir program yazabilirsiniz (her kesişim için bir t ane).Aslında, durup düşündüğünüzde, bu muhtemelen kendi duyularını zındişi değerlendirmeye çalışırken yaptığı şeydir. Duyularınız olası yo rumlarıbir araya getirmeye çalışırken gözlerinizin çizimin iki ucu ar asında gidipgeldiğini fark ettiniz mi? (Bu konu ilginizi çekiyorsa, D. A. H uffman, M. B.Clowes ve D. Waltz gibi insanların çalışmalarını okumak i steyeceksiniz.)
3. Yığın içinde dört blok var ama yalnızca üçü görünür. Buradaki nokta, bu görünüşte basit kavramı anlamanın önemli bir "zeka" miktarı gere ktirmesidir.
4. İlginç, değil mi? Anlamdaki bu ince ayrımlar, doğal dil anlama alanında önemli sorunlar ortaya çıkarmaktadır.
5. Cümle, onların ne tür atlar olduğunu mu tanımlıyor, yoksa bazı insanların ne yaptığını mı anlatıyor?
6. Ayrıştırma süreci, özdeş yapılar üretir, ancak anlamsal analiz, ilk cümle deki edat cümlesinin çitin nerede inşa edildiğini, ikinci cümledeki ifadeni n ise çitin ne zaman inşa edildiğini söylediğini tanır.
7. Onlar kardeşler.

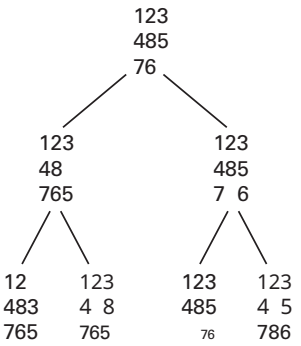
Doclingo https://www.doclingo.ai

Bölüm 11.3

1. Üretim sistemleri, çeşitli sorunlara tek tip bir yaklaşım sağlar. Yani, görünüşte farklı olan tüm sorunlar, üretim sistemleri terimleriyl e yeniden formüle edildiğinde, bir durum grafiği boyunca bir yol bulma problemi haline gelir.

2. 

3. Ağaç dört hamle derinliğinde. Üst kısım şu şekilde görünmektedir:



4. Görev, hem çok fazla kağıt hem de çok fazla zaman gerektiriyor.

5. Sekiz bulmacayı çözmek için kullandığımız sezgisel sistem, dağcılarınki gibi, anlık durumu analiz etmeye dayanmaktadır. Bu kısa görüşlülük, algoritmamızın bu bölümdeki örnekte başlangıçta yanlış bir yol boy unca ilerlemesine izin verdi; tıpkı bir dağcının her zaman yalnızca yerel a raziye dayanarak bir rota çizerken başını belaya sokabileceği gibi. (Bu benzetme, yerel veya anlık bilgilere dayanan sezgisel sistemlerin genellikle tepe tır manma sistemleri olarak adlandırılmasına neden olur.)

6. Sistem, 5, 6 ve 8 numaralı karoları saat yönünde veya saat yönünün tersine döndürür hedef duruma ulaşana kadar.

7. Buradaki sorun, sezgisel şemamızın, deliği yerinden çıkmış karolarla bitişik tutmanın değerini g öz ardı etmesidir. Eğer delik, doğru konumda olan karolarla çevriliyse, bu karolardan bazıları, d oğru yerlerini arayan karolar hareket ettirilmeden önce yerinden çıkarılmalıdır. Bu nedenle, de liği çevreleyen tüm bu karoların aslında doğru olduğunu düşünmek yanlıştır. Bu hatayı düzeltme k için, önce doğru konumda olan ancak yanlış konumda olan karoların deliği engellediği bir kar onun, doğru konumundan uzaklaştırılması ve daha sonra geri getirilmesi gerektiğini gözl emleyebiliriz. Böylece, delik ile en yakın yanlış konumda olan karo arasındaki yolda buluna n her doğru konumda olan karo, kalan çözümde en az iki hamleye karşılık gelir.

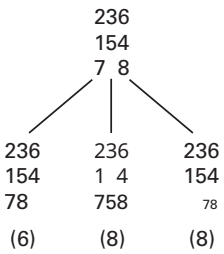
Doclingo https://www.doclingo.ai

Bu nedenle, öngörülen maliyet hesaplamamızı şu şekilde değiştirebiliri z:

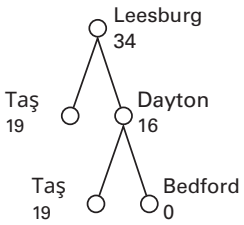
Öncelikle, öngörülen maliyeti daha önceki gibi hesaplayın. Ancak, eğer d elik tamamen yanlış konumda olan karolardan izoleyse, delik ile yanlış ko numda olan bir karo arasındaki en kısa yolu bulun, bu yoldaki karo sayısı nı iki ile çarpın ve elde edilen değeri önceki öngörülen maliyete ekleyin.

Bu sistemle, Şekil 11.10'daki yaprak düğümlerinin tahmin edilen maliyetleri 6, 6, ve 4'tür (soldan sağa), bu nedenle doğru dal başlangıçta izlenir.

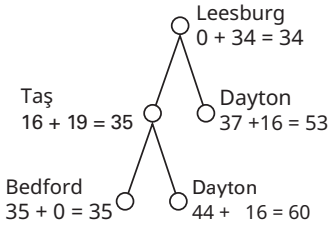
Yeni sistemimiz kusursuz değil. Örneğin, aşağıdaki konfigürasyonu düşünün. Çözüm, 5 kar osunu aşağı kaydırmak, üstteki iki satırı saat yönünde döndürmek ve bu karolar doğru ola na kadar devam etmek, 5 karosunu yukarı geri hareket ettirmek ve nihayetinde 8 karosun u doğru konumuna taşımaktır. Ancak, yeni sezgisel sistemimiz, 8 karosunu hareket ettirer ek başlamamızı istiyor, çünkü bu başlangıç hareketiyle elde edilen durumun tahmin edilen m aliyeti diğer seçeneklerle karşılaştırıldığında yalnızca 6'dır; diğer seçeneklerin maliyetleri 8'dir.



8. En iyi uyum algoritması tarafından bulunan çözüm, Leesburg'dan Dayton'a v e ardından Bedford'a giden yoldur. Bu yol en kısa rota değildir.



9. Bulunan çözüm, Leesburg'dan Stone'a ve ardından Bedford'a giden yoldur. Bu yol en kısa rotadır.



Bölüm 11.4

1. Gerçek dünya bilgisi, bir insanın anlamak ve akıl yürütmek için kul landığı çevre hakkında bilgiye denir. Bu bilgiyi temsil etme,

Doclingo https://www.doclingo.ai

saklama ve hatırlama yöntemleri geliştirmek, yapay zeka araştırmalarının önemli bir hedefidir.

2. Kapalı dünya varsayımını kullanır.
3. Çerçeve problemi, bir makinenin bilgi deposunu olaylar meydana geldikçe doğru bir şekilde güncelleme problemidir. Görev, birçok olayın dolaylı sonuçları olmasınd an dolayı karmaşıklaşmaktadır.
4. Ödünleme, denetimli eğitim ve pekiştirme. Pekiştirme, doğrudan insan müdahalesini içermez.
5. Geleneksel teknikler tek bir bilgisayar sistemine dayanır. Evrimsel teknik ler, "iyi" bir sistemin keşfedilebileceği birden fazla deneme sisteminin nesillerini içerir.

Bölüm 11.5

1. Tüm desenler, 1 çıktısı üreten 1, 0 deseni hariç 0 çıktısı üretir.
2. Her bir girdi için 1 ağırlığı atayın ve birime 1.5 eşik değeri atayın.
3. Metinde tanımlanan büyük bir sorun, eğitim sürecinin dalgalanma göste rebilmesi ve aynı ayarlamaları tekrar tekrar yapmasıdır.
4. Ağ, merkez nöronun uyarıldığı ve diğer tüm nöronların inhibe edildiği ko nfigürasyona doğru dolaşacaktır.

Bölüm 11.6

1. Tam bir eylem planı geliştirmek yerine, reaktif yaklaşım, seçenekler ortay a çıktıkça beklemek ve kararlar almaktır.
2. Buradaki nokta, robotik alanının ne kadar geniş olduğunu düşünmenizd ir. Bu, yapay zekanın tüm kapsamını ve diğer alanlardaki birçok konuyu kapsamaktadır. Amaç, çevreleriyle akıllıca etkileşimde bulunabilen ge rçekten özerk makineler geliştirmektir.
3. İç kontrol ve fiziksel yapı.

Bölüm 11.7

1. Doğru veya yanlış cevap yoktur.
2. Doğru veya yanlış cevap yoktur.
3. Doğru veya yanlış cevap yoktur.

Bölüm 12

Bölüm 12.1

1. AND, OR ve XOR boolean işlemleri hakkında ne dersiniz? Aslında, bu fo nksiyonları tanıtırken Bölüm 1'de tablolar kullandık.
2. Bir kredi ödemesinin hesaplanması, bir dairenin alanı veya bir arabanın kilometr e verisi.
3. Matematikçiler bu tür fonksiyonlara transandantal fonksiyonlar derler. Örnekler logaritmik ve trigonometrik fonksiyonları içerir. Bu özel örnekler hala hesaplanabilir ancak cebirsel yöntemlerle değil. Örneğin,

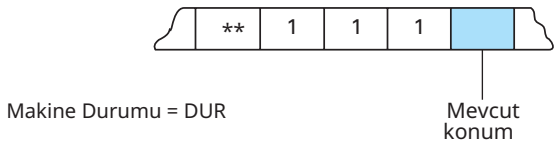
Doclingo https://www.doclingo.ai

trigonometri fonksiyonları, ilgili üçgeni çizerek, kenarlarını ölçerek ve an cak o zaman bölme cebirsel işlemiyle hesaplanabilir.

4. Bir örnek, bir açıyı üçe bölme problemidir. Yani, verilen bir açının üçte biri büyüklüğünde bir açı inşa edemediler.
Önemli olan, Yunanların cetvel ve pergel hesaplama sisteminin sınırlam aları olan bir sistemin başka bir örneği olduğudur.

Bölüm 12.2

1. Sonuç aşağıdaki diyagramdır:



2.

Mevcut durum	Mevcut hücre içerik	Değer için yaz	Yön için hareket et	Yeni durum girmek için
BAŞLA	*	*	sol	DURUM 1
DURUM 1	0	0	sol	DURUM 2
DURUM 1	1	0	sol	DURUM 2
DURUM 1	*	0	sol	DURUM 2
DURUM 2	0	*	sağ	DURUM 3
DURUM 2	1	*	sağ	DURUM 3
DURUM 2	*	*	sağ	DURUM 3
DURUM 3	0	0	sağ	DUR
DURUM 3	1	0	sağ	DUR

3.

Mevcut durum	Mevcut hücre içerik	Değer için yaz	Yön için hareket et	Yeni durum girmek için
BAŞLA	*	*	sol	ÇIKAR
ÇIKAR	0	1	sol	ÖDÜNÇ AL
ÇIKAR	1	0	sol	ÖDÜNÇ YOK
ÖDÜNÇ AL	0	1	sol	ÖDÜNÇ AL
ÖDÜNÇ AL	1	0	sol	ÖDÜNÇ YOK
ÖDÜNÇ AL	*	*	sağ	SIFIR
ÖDÜNÇ YOK	0	0	sol	ÖDÜNÇ YOK
ÖDÜNÇ YOK	1	1	sol	ÖDÜNÇ YOK
ÖDÜNÇ YOK	*	*	sağ	İADE ET
SIFIR	0	0	sağ	SIFIR
SIFIR	1	0	sağ	SIFIR
SIFIR	*	*	hareket yok	DUR
İADE ET	0	0	sağ	İADE ET
İADE ET	1	1	sağ	İADE ET
İADE ET	*	*	hareket yok	DUR

Doclingo https://www.doclingo.ai

4. Buradaki nokta, Turing makinesinin "hesaplamak" anlamını yakala masi gerektiğidir. Yani, hesaplamanın gerçekleştiği her durumda, bir Turing makinesinin bileşenleri ve faaliyetleri mevcut olmalıdır. Örn eğin, bir kişinin gelir vergisi hesaplaması belirli bir ölçüde hesaplama y apmaktadır. Hesaplama makinesi kişi ve şerit, değerlerin kaydedildiği kağıt ile temsil edilmektedir.

5. Aşağıdaki tabloyla tanımlanan makine, çift bir girdi ile başlatıldığında durur, ancak tek bir girdi ile başlatıldığında asla durmaz:

Mevcut durum	Hücre içerik	Değer için yaz	Yön için hareket et	Yeni durum girmek için
BAŞLA	*	*	sol	DURUM 1
DURUM 1	0	0	sağ	DUR
DURUM 1	1	1	hareket yok	DURUM 1
DURUM 1	*	*	hareket yok	DURUM 1

Bölüm 12.3

1. temizle AUX;
   arttır AUX;
   X 0 olmadığı sürece;
   temizle X;
   temizle AUX;
   son;
   iken AUX 0 değilse;
   `incr X;`
   temizle AUX;
   son;

2. X 0 değilken;
   `decr X;`
   son;

3. X'i AUX'a kopyala;
   iken AUX 0 değilse;
   S1
   temizle AUX;
   son;
   X'i AUX'a kopyala;
   `invert AUX;` (Soru #1'e bak)
   iken AUX 0 değilse;
   S2
   temizle AUX;
   son;
   iken X 0 değilse;
   temizle AUX;
   temizle X;
   son;

Doclingo https://www.doclingo.ai

4. Eğer X'in 40 adresindeki bellek hücresini ifade ettiğini ve her pro-gram segmentinin 0 0 konumundan başladığını varsayıyorsak, aşağıdaki dönüşüm tablosuna sahibiz:

	Adres	İçerikler
X'i temizle;	00	20
	01	00
	02	30
	03	40

	Adres	İçerikler
X artı;	00	11
	01	40
	02	20
	03	01
	04	50
	05	01
	06	30
	07	40

	Adres	İçerikler
`decr X;`	00	20
	01	00
	02	23
	03	00
	04	11
	05	40
	06	22
	07	01
	08	B1
	09	10
	0A	40
	0B	03
	0C	50
	0D	02
	0E	B1
	0F	06
	10	33
	11	40

	Adres	İçerikler
X değilken	00	20
0 yap;	01	00
.	02	11
.	03	40
.	04	B1
son;	05	WZ
	..	
	..	
	..	
	WX	B0
	BİZ	00

5. Gerçek bir makinede olduğu gibi, negatif sayılar birkodlama sistemi ar acılığıyla işlenebilir. Örneğin, her dizideki en sağdaki bit bir işaret olarak k ullanılabilir, ancak kalan bitler değerin büyüklüğünü temsil etmek için kull anılabilir.
6. Fonksiyon 2 ile çarpmadır.

Doclingo https://www.doclingo.ai

Bölüm 12.4

1. Evet. Aslında, bu program değişkenlerinin başlangıç değerlerine bakılma ksızın durur ve bu nedenle değişkenleri programın kodlanmış temsilini b aşlatıldığında durması gerekir.
2. Program yalnızca X'in başlangıç değeri 1 ile bitiyorsa durur. Bir noktalı vi rgülün ASCII temsili 00111011 olduğundan, programın kodlanmış versiyonu 1 ile bitmelidir. Bu nedenle program kendiliğinden sona ermektedir.
3. Buradaki nokta, mantığın durma probleminin algoritmik bir çözümü olm adığına dair argümanımızla aynı olduğudur. Eğer ev boyacısı kendi e vini boyarsa, o zaman o boyamaz ve tam tersi de geçerlidir.

Bölüm 12.5

1. Sadece problemin karmaşıklığının $\Theta(2^n)$ olduğunu sonucuna varabiliriz. Eğer pr oblemin çözümü için "en iyi algoritmanın" $\Theta(2^n)$ ait olduğunu gösterebilirsek, p roblemin $\Theta(2^n)$ ait olduğunu sonucuna varabiliriz.
2. Hayır. Genel bir kural olarak, $\Theta(n^2)$ içindeki algoritma, içindeki algoritmadan daha iyi performans gösterecektir. $\Theta(2^n)$, ancak küçük girdi değerleri için üstel bir algoritma genellikle bir polin om algoritmasından daha iyi performans gösterir.Aslında, uygulama yalnızca k üçük girdiler içerdiğinde üstel algoritmaların bazen polinom olanlara tercih edil diği doğrudur.
3. Önemli olan, alt komitelerin sayısının üstel olarak artmasıdır ve bu no ktadan itibaren, tüm olasılıkları listeleme işi zahmetli bir görev haline ge lir.
4. Polinom problemleri sınıfı içinde, ekleme sıralaması gibi polinom algori tmalarıyla çözülebilen sıralama problemi bulunmaktadır.

   Polinom olmayan problemler sınıfı içinde, verilen bir ana komiteden ol uşturulabilecek tüm alt komiteleri listeleme görevi bulunmaktadır.

   Herhangi bir polinom problemi bir NP problemidir. Seyahat Eden S atıcı problemi, polinom problemi olduğu gösterilmemiş bir NP problemi ne örnektir.
5. Hayır. *karmaşıklık* terimini kullanmamız, bir algoritmayı yürütmek için gereken za manı ifade eder - algoritmanın anlaşılması ne kadar zor olursa olsun.

Bölüm 12.6

1. $_{211}$ 313 $= 66043$
2. Mesaj 101, 5'in ikili temsilidir. 5^e $= 5^5 = 15625.\ 15625$ (91'e karşı) $= 64$, bu da ikili notasyonda 1000000'dır. Böylece, 1000000 mesajın şifrelenmiş versiyonudur.
3. Mesaj 10, 2'nin ikili temsilidir. 2^d $= 2^{29} = 536870912.$ 536870912 (91'e karşı) $= 32$, bu da ikili notasyonda 100000'dir. Böylece, 100000 mesajın şifre çözülmüş versiyonudur.
4. $n = p\ q$ $= 7$ 19 $=$ 133. d'yi bulmak için pozitif bir tam sayı değeri k'ya ihtiyacımız var. o$k(p$ $- 1)(q - 1)$ 1 $= k(6\ 18)\ 1$ $= 108k+$ 1 eşit olarak bölünebilir $e = 5$. Değerler k $= 1$ ve k $= 2$ tatmin edici değil, ama k $= 3$ üretir $108k1$ $= 325$, 5'e tam bölünebilir. Bölüm 65, d değeridir.

Doclingo https://www.doclingo.ai

indeks

:(Prolog if sembolü), 289
* (yıldız), 255
/* (yorum), 259
** (üs alma), 255
// (yorum), 259
+ (birleştirme), 255
:(atama operatörü), 255
/ (eğik çizgi), 255
− (çıkarma), 2552D gr
afikler, 4263D ani
masyon, 4543D grafi
kler, 426–428animasyon
, 452–456modelleme,
430–438genel bakış,
428–430paradigma,
429
rendering, 439–449
3D televizyon, 4383G
telefon ağı, 1564G telefon
ağı, 156A* algoritması, 482

–483Abaküs, 4

Soyut veri türleri, 369–370
Soyut araç, 11, 24, 314
Soyutlama, 11–12, 24, 314, 346
Erişim, Microsoft veritabanı
sistemi, 391
İnternet Servis Sağlayı
cısı (ISP), 150Erişim noktası (AP),
141Erişim süresi, 30
ACM. *Bakınız* Bilgisayar
Makineleri
Aktif Sunucu Sayfaları (ASP), 166
Ajanlar, 318
Gerçek parametre, 262, 263
Ada dili, 230, 256, 283
Adaptör deseni, 323
Uyarlanabilir sözlük
kodlama, 59
ADD talimatları, 82
Toplama
ikili, 44–45
iki'nin tamamlayıcısınot
asyonu, 49–50Adr
es, bellek hücresinin, 27Adres p
olinomu, 350
Adleman, Leonard, 178
Yönetici, 130Gelişmiş
Risk Makineleri
(ARM), 78
Ajanlar, 462–464T
oplu tür, 252–253Çevik yön
temler, 308
Aiken, Howard, 6
Alexander, Christopher, 324
Algoritma analizi, 223–226Algor
itmalar, 5
A*, 482–483
soyut doğası, 190ikili ar
ama, 214–220,
225–226
karmaşıklığı, 527–535
kavramı, 188–191
tasarım, 194
belirleyici, 535
d keşif süreci, 198–203
e verimlilik, 222–226, 527
resmi tanımı, 188–189
genetik, 488, 490
ekleme sıralaması, 209–212, 223–225
tekrarlayan yapılar, 204–213
birleştirme sıralam
ası, 528–530belirsiz, 533,
535temsil, 191–198rolü
, 2–4
RSA, 536
biliminin, 10–11
sıralı arama, 204–205
onaylama, 226–230
Aliasing, 443
Alpha testi, 327AL
VINN (Otonom KaraAracı Sini
r Ağı içinde),
487, 492–494
Amazon, 384
Ortam ışığı, 440B
elirsiz dil bilgisi, 272–273Ame
rikan Elektrik Mühendisleri Ens
titüsü, 305
Amerikan Ulusal Standartlar Enstitüsü
(ANSI), 35, 37, 243
American Online, 417
Amerikan Standart Kodu
Bilgi Değişimi
(ASCII), 35–36
Analog telefon adaptörü, 156
Analog vs. Dijital, 46
Analitik Makine, 4–5, 8
VE, 20, 21, 22, 90–92
Animasyon, 452–456
Anizotropik yüzey, 441
Anonim FTP, 155
ANSI. *Bak* Amerikan Ulusal
Standartlar Enstitüsü (ANSI)
Antisiber işgali Tüketici
Koruma Yasası, 181
Antivirüs yazılımı, 176–177

Doclingo https://www.doclingo.ai

Apple Bilgisayar, 7U
ygulama katmanı, 168, 169, 171U
ygulama yazılımı, 115, 116, 387–388
Mimari
alternatif, 100–101b
ileşen, 315–316bilgis
ayar, 74–76İnter
net, 149–151işleti
m sistemi, 114–121
von Neumann, 97
Aristoteles, 15
Aritmetik işlemler, 92–93, 255
Aritmetik kaydırmalar, 92
Aritmetik/lojik talimatlar, 90–94
Aritmetik/lojik birimi, 74, 79, 87
ARM. *Bak* Gelişmiş Risk Makineler (ARM)A
RM hisseleri, 78, 108AR
M tabanlı işlemci, 78, 85, 108
Diziler, 252, 253, 342h
eterojen, 252–253, 342, 351–352
homojen, 342, 348–350
saklama, 348–352
Yapay zeka, 461–508
davranışa dayalı, 479
sonuçları, 500–502
akıllı ajanlar, 462–464
dil işleme, 469–471
sinir ağları, 489–497
kökenleri, 465
algı, 467–473akıl yürüt
me, 473–484araştırma metodolojileri, 464–465
araştırma, 484–489rob
otik, 497–500akıllı
telefonlarda, 472güçl
ü ve zayıf, 468Turing testi
, 465–466Yapay sinir ağları, 489–497
Montajcılar, 241
Montaj dili, 241
İddialar, 228–229
Atama ifadeleri, 193, 254–256, 278
İlişkilendirme analizi, 41
5Bilgisayar Mühendisliği Derneği (ACM), 301, 302, 325
İlişkiler, 318, 319–320İlişkis
el bellek, 494–497
AT&T, 230
Atanasoff, John, 6
Atanasoff-Berry makinesi, 6Öz
nitelikler, 389
Ses sıkıştırma, 62
Ses kodlama, 40
Denetim yazılımı, 130
Kimlik doğrulama, 179, 411
Avarlar, 455
Ortalama durum analizi, 223
Aksiyom, 228

Babbage, Charles, 4–5, 6, 8
Arka yüz eliminasyonu, 445
Geri izleme, 343
Band genişliği, 99
Bardeen, John, 7
Yığın tabanı, 343Temel
durum, 221
Onluk sistem, 42
İkilik sistem, 42
Temel yol testi, 326
Toplu işleme, 111
BD'ler (Blu-ray diskleri), 33
Davranışa dayalı zeka, 479
Karşılaştırma, 85
Berkeley'nin Açık Altyapısı Ağ Hesaplama için (BOINC), 148
Berners-Lee, Tim, 8, 158
Berry, Clifford, 6
En iyi öncelikli arama, 478
Beta testi, 327
Bezier eğrileri, 431–432
Bezier yüzeyleri, 432
Büyük O notasyonu, 528
Büyük-teta notasyonu, 226
İkili toplama, 44–45
İkili notasyon, 37–38, 42–43
İkili arama algoritması, 214–220, 225–226
İkili sistem, 42–47İkili
ağaç, 344, 357–360, 365–367
Biyoenformatik, 415
BIOS (Temel Giriş/Çıkış Sistemi), 119
Bit haritası, 38–39, 91
Bitler, 20, 24
bilgiyi temsil etme olarak, 35–42
Saniye başına bit (bps), 62, 98
Kara kutu, 314
Kara kutu testi, 327Blu-r
ay diskler, 33Bulan
ıklaştırma, 454
Döngünün gövdesi, 205
Boole, George, 20
Boolean veri türü, 251Bool
ean ifadesi, 269Boolean işle
mleri, 20–21Başlatıcı, 120

Başlatma, 120–121
Başlatma, 120–121
Yığın, alt, 343Aşağıdan y
ukarı metodolojisi, 202Sınır değer
i analizi, 327
Bourne shell, 117
Dal, 344
Brattain, Walter, 7
Genişlik öncelikli arama, 478
Köprü, 144
Parlaklık bileşeni, 39
Geniş bant, 99
Tarayıcı, 159Kova, has
hing, 410Tampon, 34–35, 343–344

Bump mapping, 447
Veri yolu, 75
Veri yolu topolojisi, 140, 141, 142, 144
Byron, Augusta Ada (Ada Lovelace), 5, 8, 553
Bayt, 26–27
Bayt kodu, 271

C dili, 248, 255, 257, 258, 262–263, 267
C kabuğu, 117

Doclingo https://www.doclingo.ai

C# dili, 248, 249, 255, 256, 257, 258, 271
C++ dil, 248–249, 255, 256, 257, 258, 314–315
C++ Standart ŞablonKütüphane si, 314–315Önbellek b
elleği, 76CAD. *Bak* Bilgisa
yar desteklidizayn (CAD)

Çağrı, prosedür, 260
Devekuşu yazımı, 197
Etobur, 180
Taşıyıcı Duyma, Çoklu ErişimÇarpı şma Önleme ile (CSMA/CA), 142–143
Taşıyıcı Duyma, Çoklu ErişimÇarpı şma Tespiti ile (CSMA/CD), 141–142
Zincirleme geri alma, 404
Durum ifadesi, 257–258CAS
E. *Bak* Bilgisayar destekliyazıl ım mühendisliği (CASE)CASE ar
açları, 301–302
CD-DA (kompakt disk-dijital audio), 32, 66
Hücre, bellek, 26–27P
rojeksiyon merkezi, 428M
erkezi işlem birimi (CPU),74–75, 84, 85, 120\ çift çe kirdekli, 100
çok çekirdekli, 100
CERN, 160
CERT. *Bakınız* Bilgisayar Acil Müdahale Ekibi (CERT)
Sertifika, 179
Yetki veren otoriteler (CAs), 179
CFE (Ortak Firmware Çevre), 119
CGI (Ortak Geçit Arayüzü), 166
Karakter veri türü, 251K
arakter tanıma, 467–468Karakt
er temelli etik, 15
Checkbyte, 64–65
Kontrol toplamla
rı, 65Ağaçtaki çocuklar, 34
4Çipler, 7, 24
Renklilik, 39
Kromozom, 488
Church, Alonzo, 515
Church-Turing tezi, 515, 520–521
Döngüler
parmak arası terlik, 22–24
yenile, 28
Dairesel kuyruk, 357Dair
esel kaydırma, 92CISC
. *Bak* Karmaşık komutset bilgisayarı (CI SC)Sınıf, 247–248, 276–279
, 371–372
aralarındaki ilişkiler, 318
yapıcı ile, 280
Sınıf tanımı, 414
Sınıf diyagramı, 318, 319, 321
Sınıf ayrımı, 414
Klaus formu, 286–287
İstemci, 146
Müşteri/sunucu modeli, 146–147
Müşteri tarafı etkinlikleri, 165–166
Kırpma, 443
Saat, 85
Saat hızları, 85Ka
palı ağ, 140Kapalı dü
nya varsayımı, 485–486
Bulut bilişim, 149
Küme analizi, 414–415
Küme bilişimi, 148
Kümeleme, 411
COBOL, 241
Kod üretimi, 274Kod
üreteci, 268Kod opti
mizasyonu, 274Zorlama
, 273
Kognetik, 330, 331
Kohesyon, 312–313
Çarpışma, 411
Çarpışma önleme protokolleri, 142–143
Renk sızıntısı, 452
Colossus, 6–7
Sütun ana sırası, 349Kom
ut kabuğu, 117Yoru
mlar, 249, 259–260Ticari
off-the-shelf(COTS) yazılı mı, 304
Taahhüt noktası, 403
Taahhüt/geri alma protokolü, 403–404
Commodore, 7
Cihazlar arası il etişim, 94–99işlem ara sı, 146–148paralel, 9 8
seri, 98
İletişim hataları, 63–67İletişim m
edyası, 98İletişim oranları, 98
–99İletişim Yardımı içinHukuk U
ygulama Yasası
(CALEA), 181
Komple disk (CD), 31–32, 66
Derleme, tam zamanında, 271
Derleyiciler, 242
Tamamlayıcı, 48K
armaşık komut setibilgisa yarı (CISC), 77–78Karmaşı
klık, problem, 527–535Bileşen mi
marisi, 315–316
Bileşen derleyici, 315Bileşenler, 314–316, 342Hesaplanabilir fon
ksiyonlar, 511–512Fonksiyonların hes
aplanması, 510–512
Bilgisayar mimarisi, 74–76
Bilgisayar Acil Durum Yanıt Ekibi (CERT), 174Bilgisayar D
olandırıcılığı ve Kötüye KullanımY asası, 179–180
Bilgisayar grafikleri, 425–4603D grafikleri, 426–430ani
masyon, 452–456glob
al aydınlatma ve, 449–452mod
elleme, 430–438
rendering, 439–449
kapsamı, 426–428B
ilgisayar ağları, 113–114Bilgisay
ar gücü, 85
Bilgisayar bilimi, 2, 10–11
sosyal sonuçları, 13–15
Bilgisayar destekli tasarım (CAD), 39
Bilgisayar destekli yazılım mühendisliği (CASE), 301

Doclingo https://www.doclingo.ai

Bilgisayar biliminin tarihi, 4–10
Eşzamanlı işleme, 283–285
Koşullu atlamalar, 79–80Tı kanıklık kontrolü, 172So nuç temelli etik, 14–15
Sabitler, 253–254
Yapıcılar, 279–280
Bağlam değişimi, 123
Bağlamsal analiz, 470
Sürekli listeler, 350, 352–353
Sözleşme temelli etik, 15
Kontrol bağlantısı, 311
Kontrol noktaları, 432, 453
Kontrol ifadeleri, 256–259
Kontrol sistemi, 473
Kontrol birimi, 74, 79–80
Kontrolör, 95–96
Telif hakkı, 333
Ülke kodu TLD'leri, 152
Modüller arası bağlantı, 311–312
CPU. *Bak* Merkezi işlem birimi (CPU)
CRC (sınıf-sorumluluk-işbirliği) kartları, 323
Kritik bölge, 127
Çapraz platform yazılımı, 243K riptografi, 536–539CSMA /CA. *Bak* Taşıyıcı Duyusu,Çoklu Erişim ileÇarpışma Önl eme (CSMA/CA)
CSMA/CD. *Bak* Taşıyıcı Duyusu, Çoklu Erişim ileÇarpış ma Tespiti (CSMA/CD)
İnternet alan adı kaçakçılığı, 181
Döngüsel fazlalık kontrolleri (CRC), 65
Silindir, 30

Darwin, Charles, 500
Veri
  küresel, 312
  programlara karşı, 88
Veri toplama, 416–417
Veri sıkıştırma, 58–63
Veri bağlantısı, 311–312
Veri küpleri, 415
Veri sözlüğü, 317–318
Veri bağımsızlığı, 388
Veri madenciliği, 414–416
Veri yapıları, 252–253
  diziler, 342
  uygulama, 348–361listeler, yığınlar ve kuyruklar, 342–344
  manipülasyon, 360–361
  statik ve dinamik, 346
  ağaçlar, 344–345
Veri transferi, 78–79
Veri transfer hızları, 98–99
Veri türleri, 249–252
  soyut, 369–370
  özelleştirilmiş, 367–370
  kullanıcı tanımlı, 368–369
Veri ambarı, 414
Veritabanı
  kavramsal katmanlar, 386 t anımı, 384dağıtık, 38 7dosyaya karşı, 385
  temeller, 384–389
  bütünlük, sürdürme, 402–406
  nesne yönelimli, 400–402
  ilişkisel tasarım, 389–393
  şemalar, 385–386
  veritabanıteknolojisinin sosyal etkisi, 416–418
Veritabanı yönetim sistemi (DBMS), 386–388, 397, 402
Veritabanı modelleri, 388
  ilişkisel, 389–399
Veritabanı sistemleri, 384–385, 391
Veri akış diyagramı, 317
DBMS. *Bak* Veritabanı yönetim sistemi (DBMS)
Deadlock, 127–129
Hata ayıklama, 240
Açıklayıcı bilgi, 462Açıklayıcı paradi gma, 245Açıklayıcı programlama , 286–291
Açıklayıcı ifadeler, 249, 273, 278
Decorator deseni, 323
Açık anahtarları çözme, 536
Tür tanım ifadesi, 369, 371
Dejenere durum, 221
Hizmet reddi (DoS) saldırıları, 175
Ağaç derinliği, 344Deri nlik öncelikli arama, 478Ta sarım desenleri, 323–324Yazı lım yaşam döngüsünün tasarım a şaması, 304–305
Masaüstü bilgisayarlar, 7–8
Belirleyici algoritma, 535
Aygıt sürücüsü, 118
Çevirmeli bağlantı, 151
Sözlük, 59
Sözlük kodlaması, 59–60
Fark Motoru, 4, 6
Farklı kodlama, 58–59
Dağınık ışık, 440
Dijital kameralar, 426
Dijital imzalar, 179
Dijital teknoloji, 46
Dijitalleştirme, 432
Dijkstra, E. W., 135
Doğrudan adresleme, 374
Doğrudan bellek erişimi (DMA), 96–97
Directzd, 448
Yönlendirilmiş grafik, 474
Dizin, 118
Dizin yolu, 118
Feragatname, 333
Disk, biçimlendirme, 30
Disk depolama sistemi, 29–31
Dağıtıcı, 119, 122–123
Dağıtılmış veritabanları, 387
Dağıtılmış ışın izleme, 450
Dağıtılmış sistemler, 148–149
DMA. *Bak* Doğrudan bellek erişimi (DMA)
DNS. *Bak* Alan adı sistemi (DNS)
DNS sorgulama, 153
DOKTOR, 466
Dokümantasyon, 328–329
Alan, 152
Alan adı, 152–153
Alan adı sistemi (DNS), 153

Doclingo https://www.doclingo.ai

Noktalı ondalık notasyon, 152
Çift hassasiyetli kayan
punkt, 56
DRAM, 28
Çizim yazılımı, 39, 432
Gölge düşürme, 449
DSL (Dijital Abone Hattı),
98, 151
İkili çekirdekli CPU'lar, 100
Görev temelli etik, 15DVD'le
r (Dijital Çok Amaçlı Diskler),
32–33
Dinamik veri yapıları, 346
Dinamik sözlük
kodlama, 59
Dinamik bellek, 28Dina
mikler, 454–456

eBay, 384
Eckert, J. Presper, 7, 77
Kenar iyileştirme, 469
Edison, Thomas, 77, 305
Bir neurondan etkili girdi, 490

Verimlilik, algoritma, 222–226
EFI (Genişletilebilir Donanım
Arayüzü), 119
Sekiz bulmacası, 463–464, 467–468,
473–474, 476–480
Elektromekanik makine, 5
Elektronik devreler, 31
Elektronik İletişimGizlilik Yas
ası (ECPA), 180
ELIZA, 466
Elektronik posta (e-posta),
154–155
Gömülü sistemler, 114
Kapsülleme, 282
Şifreleme anahtarları, 536
Şifreleme, 177–179, 536–539
Son sistemler, 150
Dosya sonu (EOF), 407
ENIAC, 7
Eşdeğer sınıf, 327Ergo
nomi, 330, 331Hatalı
bilgi, 417–418Hata düzeltme ko
dları, 65–66
Ethernet, 140, 142, 144
Etik, 14–15

Öklid, 2
Öklid algoritması, 2, 3
Çift parite, 64
Olay odaklı yazılım
sistemleri, 266
Evrimsel programlama, 488Evrims
el prototipleme, 307Evrimsel ro
botik, 499Aşırı 16 notasyonu
, 51–52Aşırı sekiz notasyon
u, 51Aşırı notasyon, 51–52
Özel kilit, 405Uzman sist
emler, 476Üs alanı,
54Genişletilebilir İşar
etleme Dili

(XML), 162–165
Aşırı programlama (XP), 308

Federal Register, 417
Alanlar, 34, 253F
IFO. *Bak* İlk-giren, il
k-çıkan (FIFO)Dos
ya yöneticisi, 117–118, 129Dos
ya sunucusu, 147
Dosya yapıları, 406–414Dosy
a aktarım protokolü
(FTP), 155
Dosyalar, 33
veritabanlarına karşı, 385
hash, 410–413
indeksli, 408–410s
ıralı, 406–408depola
ma ve geri alma, 33–35Güven
lik Duvarı, 175–176
FireWire, 95, 97, 98–99
Bellenim, 119
Firmware güncellemesi, 121
İlk-giren, ilk-çıkan (FIFO),
111, 343
Sabitleme formatlı diller, 268
Flash sürücüler, 33
Flash bellek, 33
FlashROM, 119
Düz dosya, 384
Düz gölgeleme, 446
Flip-flop, 22–24
Float, 250
Kaydırmalı nokta notasyonu, 38,
53–55

Akış kontrolü, 172
Akış şemaları, 194, 208
Tommy Çiçekleri, 6–7
Klasör, 118
For döngüsü yapısı, 258
Forking, 127
Form faktörleri, 451
Resmi Dil, 242
Resmi Mantık, 245
Resmi parametreler, 262, 263
Biçimlendirme, 30
FORTRAN, 241, 249
Yönlendirme, 173Y
önlendirme tablosu, 146F
raktallar, 434
Kesirler
ikili sistemde, 45–46
saklama, 53–58
Kare, 322
Kare tamponu, 429
Çerçeve problemi, 486
Kareler, 452–453, 471Se
rbest format dilleri,
268–269
Frekans maskesi, 62Frek
ansa bağlıkodlama, 58
FTP. Dosya T
ransferi Protokolü (FT
P)
FTPS, 177
Fonksiyonel bütünlük, 313Fo
nksiyonel paradigma, 245–247Fon
ksiyonlar, 266–267, 510–512hesa
planabilir, 511–512hesa
planamaz, 522–526

Gandhi, Mahatma, 500
Çöp toplama, 364
Gates, 21–24
Geçit, 146GB
(gigabayt), 28
Gbps, 62, 98
Gen, 488
Genelleme, 320–321Gen
el amaçlı kayıtlar,
74–75
Programlama dilleri nesilleri,
240–242Genetik a
lgoritmalar, 488, 492

Doclingo https://www.doclingo.ai

GIF (Grafik Değişim Formatı), 60
Cam kutu testi, 326–327
Küresel veriler, 312
Küresel aydınlatma modeli, 448–452
Küresel Konumlama Sistemi (GPS), 9
Küresel değişkenler, 261
Hedef durum, 473
Gödel, Kurt, 4, 10
GOMS modeli, 331
Google, 8, 9, 384
Google Goggles, 472
Goto ifadeleri, 256, 354Gouraud gölgeleme, 446–447Dilbilgisi, 269
Grafiksel kullanıcı arayüzü (GUI), 116–117, 331
Grafikler. *Bak* Bilgisayar grafikleri
Grafik adaptörü, 448
Grafik kartı, 448
Ağ hesaplama, 148
GUI. *Bak* Grafiksel kullanıcı arayüzü (GUI)

Durdurma problemi, 522–526
Hamming mesafesi, 65–66
Ellerin sıkışması, 97
Donanım, 2Hash dosyaları, 410–413Hash fonksiyonu, 410Hash tablosu, 410Hashleme, 410, 411Liste başı, 342Baş işaretçisi, 353, 356Başlık, prosedürün, 260–261
Heathkit, 7
Heterojen diziler, 252–253, 342, 351–352
Heuristikler, 478–483
Onaltılık notasyon, 24–26Gizli terminal problemi, 142, 143
Gizli yüzey kaldırma, 445
Yüksek erişilebilirlik sistemleri, 148
Yüksek dereceli uç, 27
Hollerith, Herman, 5
Homojen diziler, 342, 348–350
Atlama sayısı, 173
Hopper, Grace, 242, 469
Ana bilgisayarlar, 150
Sıcak noktalar, 151
HTML. *Bak* Hiper Metin İşaretleme Dili (HTML)
HTTP. *Bak* Hiper Metin Aktarım Protokolü (HTTP)
HTTPS, 177
Merkez, 141
Huffman kodları, 58
Huffman, David A., 58
İnsan-makine arayüzü, 329–332
Hiper bağlantılar, 158
Hiper medya, 158Hiper metin, 158Hiper Metin İşaretleme Dili (HTML), 160–162
Hiper Metin Aktarım Protokolü (HTTP), 159

IBM, 5, 8
Tanımlayıcılar, 241IDE'ler. *Entegre* geliştirme ortamları (IDE'ler)IEEE. *Elektrik ve Elektronikmühendisleri Enstitüsü (IEEE) için*Bakınız
IEEE 802, 144
YazılımGözden Geçirme için IEEE Standardı(IEEE 1028), 326Eğer-şu durumda, 256–257,269–270
Görüntü analizi, 468
Görüntü işleme, 426, 468–469
Görüntü penceresi, 428–429
Görüntüler
  sıkıştırılması, 60–62
  tanınması, 467–469
  temsil edilmesi, 38–39
  Web sayfalarında, 162
IMAP (İnternet Posta Erişim Protokolü), 154
Ödünleme, 486
Anlık adresleme, 374
Emirsel paradigma, 244, 246–247, 317
Emirsel ifadeler, 249, 273Uygulama aşaması, yazılım yaşam döngüsü, 305–306Arada geçiş, 454
İnsidans açısı, 439Tamamlanmamışlık teoremi, 4, 10Tutarsız ifadeler, 287–288
Yanlış özet problem, 404
Artımlı model, 307Girinti, 269
İndeksli dosyalar, 408–410
İndeksler, 252
Dolaylı adresleme, 374
Çıkarım kuralları, 286, 475–476
Bilgi çıkarımı, 470–471
Bilgi gizleme, 314
Bilgi temsili, 35–42
Bilgi retrieval, 470Kalıtım, 281–282, 321Girdi/Çıktı (I/O), 79Girdi/çıktı talimatları, 79, 96
Ekleme sıralama algoritması,209–212, 223–225Sınıfın örneği, 248, 278Veri tipinin örneği, 368Örnek değişkeni, 277Elektrik ve Elektronik Mühendisleri Enstitüsü(IEEE), 302, 305, 325Radyo Mühendisleri Enstitüsü Mühendisleri, 305
Talimat işaretçisi, 347
Talimat kaydedici, 83
Tam sayılar, 250
  depolama, 47–53Entegre devre, 7Entegre geliştirmeenvironmentları (IDE'ler), 302
Intel, 78, 97
Intel işlemcileri, 78, 85Fikri mülkiyet, 13–14, 332
Akıllı ajanlar, 462–464

Doclingo https://www.doclingo.ai

Etkileşim diyagramları, 321–323
Etkileşim parçaları, 323
Etkileşimli işleme, 112
Arayüz tasarımı, 329–332
Modüller arası bağlantı, 311–312
Uluslararası Mahkeme Adalet, 179
Uluslararası ElektroteknikKomisyonu (IEC), 325Uluslararası StandardizasyonÖrgütü (ISO), 36, 38, 243, 302, 325
İnternet, genel terim, 145İnternet, the, 8, 140, 145, 149–158
adresleme, 152–153uygulamalar, 153–158mimari, 149–151Atanmış İsimler veNumaralar için İnternetKurumu (ICANN), 15
2İnternet Posta Erişim Protokolü (IMAP), 154
İnternet Protokolü (IP), 171, 173
İnternet protokolleri, 167–173
İnternet radyosu, 157
İnternet Servis Sağlayıcısı (ISP), 149–150
İnternet yazılımı, 167–171
Internet2, 151
Yorumlayıcı, 242
Süreçler arası iletişim, 146–148
Kesme işleyicisi, 123
Kesintiler, 123, 124
Çözülmesi zor problemler, 532
İntranet, 150
G/Ç talimatları, 79
G/Ç istekleri, 124
IP adresi, 152
IPv4, 173
IPv6, 173
IQ testleri, 500–501
ISO. *Bakınız* Uluslararası StandardizasyonÖrgütü (ISO)ISO 9000 serisi, 325
ISO/OEC 15504, 325
Izotropik yüzey, 441
ISP. *Bakınız* İnternet Servis Sağlayıcısı (ISP)
İteratif model, 307
İteratif yapılar, 204–213

Jacquard, Joseph, 5
Jacquard dokuma tezgahı, 5
Java, 248, 249, 255, 257, 258, 283, 315
uygulaması, 271
işaretçiler, 354
Java Uygulama Programcı Arayüzü (API), 315
JavaServer Sayfaları (JSP), 166
JCL. İş Kontrolü için bakınız Dili (JCL)
İş, 110
İş kontrol dili (JCL), 111
İş kuyruğu, 111
İşler, Steve, 7
JOIN işlemi, 394–399
Ortak Fotoğraf Uzmanları Grubu, 61
JPEG, 61
JSP. *JavaServer Sayfaları (JSP) için bakınızATLA (JUMP) talimatları, 79–80,* 84–85
Just-in-time derlemesi, 271

KB (kilobayt), 28
Kbps, 62, 98, 99
Kernel, 117
Anahtar, 34
Anahtar alan, 34Anahtar çerçeve, 453–454Anahtar kelime, 269
Kilby, Jack, 7
Bir süreci öldür, 128Kilobayt, 28
Kinematik, 454–456
Kineograf, 453
Sırt çantası problemi, 542
Bilgi
deklare, 462prosedürel, 462–463gerçek dünya, 484–485temsil etme veişleme, 484–486
Korn shell, 117
Dil uygulaması, 268–275
Dil işleme, 469–471
Son giren, ilk çıkan (LIFO), 343
Gecikme süresi, 30
Yaprak düğüm, 344Öğrenme, 486–488En az anlamlı bit, 27Sol çocuk işaretçisi, 358Hukuki çareler
veri toplama için, 417ağ güvenliği için, 179–181
Leibniz, Gottfried Wilhelm, 4
Lempel, Abraham, 59
Lempel-Ziv-Welsh (LZW) kodlaması, 59–60
Leonardo da Vinci, 77
Sözcüksel analizör, 268, 274Sorumluluk, 333
Lisans anlaşmaları, 332–333
Hayat hattı, 322
LIFO. *Bak* Son-giren, ilk-çıkan (LIFO)
Işık
ortam, 440
difüz, 440
yansıyan, 439–441kırılan, 441–442spektral, 440Aydınlatma modelleri, 448–452Işık-yüzey etkileşimi, 439–442
Doğru normali, 439Dilbilim, 464–465Bağlantı katmanı, 168, 169, 170Bağlı liste, 353–355, 361
Linux, 110, 116, 307
Liste, 342–343
bitişik, 350, 352–353
bağlı, 353–355, 361
arama, 214–220
sıralama, 209–212
saklama, 352–355, 362–367
Açık, 253–254
Yük dengeleme, 113, 148
Yük faktörü, 413
LOAD talimatı, 78–79, 82, 96
Yerel alan ağı (LAN), 140

Doclingo https://www.doclingo.ai

Yerel aydınlatma modeli, 448
Yerel değişken, 261
Kilitleme, 404–405
Mantık işlemleri, 90–92M
antık programlama, 245,
288–291, 487
Mantıksal uyum, 313
Mantıksal çıkarım, 286–288
Mantıksal kayıt, 34
Mantıksal kaydırm
a, 92Giriş prosedürü, 13
0Uzun bölme algoritması, 2Dö
ngü, 205–209
Döngü değişmez, 22
9Kayıpsız sıkıştırma, 58Kayı
psız ayrıştırma, 393Kayıplı sıkı
ştırma, 58Kaybolan gün
celleme sorunu, 404Düşük
dereceli uç, 27
Parlaklık, 39

Mac OS, 110
Makine döngüsü, 83–88
Makine bağımsızlığı,
242–244
Makineden bağımsız, 241Ma
kine talimatı, 77–80Makine dil
i, 77–83,
240–241
işaretçiler, 372–374evr
ensel, 271Many
etik disk, 29, 407Manyeti
k bant, 31Mail sunu
cusu, 154Ana bell
ek, 26–29Kötü amaçlı y
azılım, 174
MAN. *Bak* Metropol Alan
Ağı
Mantissa alanı, 54
Çoktan-çoğa ilişki, 320
Mark I, 5–6, 7, 228
İşaretleme dili, 164
Mars Keşif Roverları, 114
Maske, 91
Maskeleme, 91
Kütle depolama, 29–35
Mauchly, John, 7
MB (megabayt), 28
Mbps, 62, 98, 99
Hafıza, 75
ilişkisel, 494–497
önbellek, 76
kapasite, 28
değerleri bölme, 79
DRAM, 28
dinamik, 28
flash, 33
ana, 26–29or
ganizasyon, 26–27
RAM, 28
ROM, 120
SDRAM, 28
sanal, 119
Hafıza hücreleri, 26–28Hafıza sız
ıntısı, 364Hafıza yöneticisi, 1
18–119Hafıza eşlemeli G/Ç, 96Birleştirme
sıralama algoritması,
528–530
Meta-akıl yürütme, 485
Yöntemler, 247, 277
Metrikler, 300
Metropol alan ağı
(MAN), 140
Microişlemciler, 74
Microsoft, 8, 110, 114, 116,
265, 271
Microsoft Access, 391
Microsoft Windows, 110,
114–115, 117, 125, 265
MIDI. *Bak* Müzik AletiDijital Aray
üz (MIDI)
Miller, George A., 331
MIMD mimarisi, 101MIM
E (Çok Amaçlı İnternetPosta Uza
ntıları), 154Miniatürleş
me, 9
Mobil İnternet Cihazları
(MID), 74
Modelleme, 430–438
Modem, 98, 151
Modüler notasyon, 536–537
Modüler programlama, 309–311
Modülerlik, 308–316
uyum, 312–313
bileşenler, 314–316
bağlantı, 311–312
bilgi gizleme, 314
Modül, 309
Mondrian, Piet, 220
Monitör, 285
Değişim, 453E
n önemli bit, 27Anakart,
74Hareket, 3D g
rafiklerde,
454–456
Hareket yakalama, 455–456
Hareketli Resimler Uzman Grubu
(MPEG), 62
MP3, 62
MPEG. *Bak* Hareketli Resimler
Uzman Grubu (MPEG)
MS-DOS, 117
Çok noktaya yayın, 157
Çok çekirdekli CPU, 100
Çok çekirdekli işletim
sistemleri, 128
Çoklama, 99Çok i
şlemcili makineler, 101Çoklu pr
ogramlama, 112,
123, 124
Çok Amaçlı İnternet Posta
Uzantıları (MIME), 154Ç
oklu Görev, 112
Müzik Aleti DijitalArayüzü (MIDI
), 40Karşılıklı dışlama, 1
27İsim sunucuları, 153NA

SA Mars gezginleri, 4
98Doğal dil işleme,
464–465
Doğal diller, 242.NET
Ortak Ara Dili, 271

.NET Çerçevesi, 324.N
et Çerçeve Sınıf Kütüp
hanesi, 315
Ağ katmanı, 168, 169, 170
Ağ yazılımı, 167–171
Ağlar/ağ oluşturma,
113–114, 139
sınıflandırmalar, 140–141
birleştirme, 143–146
iletişim, 146–148
temeller, 140–149
protokoller, 141–143

Doclingo https://www.doclingo.ai

güvenlik, 173–181
topoloji, 140–141
Sinir ağları, 489–497
Nöron, 479, 489–491, 494–497
NIL işaretçisi, 353
Düğüm, 344, 474
Hesaplanamaz fonksiyon, 522–526
Belirsiz algoritma, 533, 535
Belirsiz polinom(NP) problemi, 533–534Kaybı olmayan a yrıştırma, 393Polinom dışı pro blemler, 531–532
Sonlu olmayan, 270
Sonlu olmayan genişletmeler, 56
Normal vektör, 446
Normalleştirilmiş form, 55
NOT, 21, 22
Novell Inc., 140
NP problemleri, 533–534N
P-tamamlayıcı problemler, 534
NPT Inc., 333
NULL işaretçisi, 353
Sayısal değerler, 36–38
N-unicast, 157

Nesne, 247–248, 276–279, 371–372
modelleme, 430–437k
alıcı, 401görüntül
eme, 439–449Nesne pr
ogramı, 268Nesne yönel
imli veritabanı, 400–402
Nesne yönelimli diller, 260Nesne y önelimli paradigma, 247, 310
Nesne yönelimli programlama (OOP), 247–248, 276–283
sınıflar, 276–279k
urucular, 279–280kapsü
lleme, 282miras alm
a, 281–282, 321nesneler, 27
6–279çok biçimlili
k, 282program yapısı, 279Tekil parite, 64
Çevrimdışı, 29
Birden çoğa ilişkiler, 319–320
Birden bire ilişkiler, 319–320
Çevrimiçi, 29
OOP. *Bak* Nesne yönelimli programlama (OOP)O
p-kodu, 80, 82, 96, 373Açı
k Firmware, 119Açık a
ğ, 140Açık Sistem Ba
ğlantısı (OSI), 171
OpenGL (Açık GrafikKütüp hanesi), 448
Açık kaynak geliştirme, 307–308
Operand alanı, 80–82
İşletim sistemi, 109–137
mimari, 114–121
bileşenleri, 116–119
koordinasyon, 122–124
tanımı, 110
tarihi, 110–114çok çeki
rdekli, 128kaynak t
ahsisi, 125–129
güvenlik, 130–132b
aşlangıç, 119–121O
peratör önceliği, 255Optik kar
akter okuyucu, 467Optik sistemle
r, 31–33VE, 20–22, 23, 90–92
Oracle, 249, 324
OSI. *Açık SistemBağlantıs ı (OSI)Aykırı değer analizi, 415Taşma, 50*

Aşırı yükleme, 255–256

Paketler, 170
Sayfa, bellek, 119Say
falama, 119
Paint, Microsoft, 426
Resimleyici algoritması, 445
Palm OS, 114
Paralel iletişim, 98
Paralel işleme, 101, 283–285
Paralel projeksiyon, 428
Parametreler, 196, 262–266
gerçek, 262, 263
resmi, 262, 263
referansla geçildi, 264, 265
değerle geçildi, 263, 264
Parantezler, 255
Parantezli notasyon, 263
Pareto, Vilfredo, 326
Pareto ilkesi, 326
Eşitlik bitleri, 63–65
Ayrıştırma ağacı, 270–271, 272
Ayrıştırıcı, 268, 273
Parçacık sistemi, 434
Pascal, Blaise, 4
Pascal yazımı, 197
Referansla geçildi, 264, 265De
ğerle geçildi, 263, 264Şifre
, 131
Patentler, 333
Eşler arası (P2P) modeli, 147–148
Pentium mikroişlemcisi, 228
Algılama, 467–473
Perl, 250
Kalıcı nesne, 401Kişis
el bilgisayar (PC), 8, 391Perspekt
if projeksiyonu, 428
Phishing, 175
Phong gölgeleme, 446–447
PHP, 166, 250
Fiziksel kayıt, 34
Pipelining, 100
Pikseller, 38
Planar yama, 431
Platon, 15
Cep PC, 114Göst
ergeler, 346–347, 354, 359, 372–374
Polya, G., 198
Poligonal ağ, 431, 432, 433, 434–435
Polimorfizm, 282
Polinom problemi, 531–532
Pop, yığın işlemi, 343
POP3 (Posta Ofisi Protokolü sürüm 3), 154
Liman, 95
Port numaraları, 171
Post, Emil, 513

Doclingo https://www.doclingo.ai

Post üretim sistemi, 513
Post koşulları, 229
PostScript, 39
Posttest döngüsü, 208Operatörlerin önceliği, 25
5Doğruluk kanıtında ön koşullar, 228Öneme, 288–289Pretest döngüsü, 208Pretty Good Privacy, 178Temel veri türleri, 251Temeller, 191–192, 193Yazdırma sunucusu, 146
Gizlilik Yasası, 417
Gizlilik hakları, 180Özel anahtarlar, 177, 536Ayrıcalık seviyeleri, 132Ayrıcalıklı talimat, 132Problem karmaşıklığı, 527–535Problem çözme, 198–203Problem alanı, 474
Prosedürel bilgi, 462–463Prosedürel model, 432–433, 434–435
Prosedürel paradigma, 244
Prosedürel birimler, 260–267
Prosedürler, 260–261, 317
Prosedür başlığı, 260–261
Süreç, 122
Süreç durumu, 122
Süreç geçişi, 123
Süreç tablosu, 122
Süreçler, 122
  rekabeti yönetme arasında, 125–129
  öldürme, 128
  başlatma/durdurma, 123–124
Üretim, 473
Üretim sistemi, 473–476
Program, 2
Program sayacı, 83
Program yürütme, 83–90
Programcı, 305
Programlama, 2Programlama kavramları, 248–260
  atama ifadeleri, 254–256
  yorumlar, 259–260
  sabitler, 253–254kontrol ifadeleri, 256–259data yapısı, 252–253data türleri, 249–252literal değerler, 253–254değişkenler, 249–252Programlama dilleri, 192eşzamanlı işleme ve, 283–285
  kültürler, 257
  declarative programming ve, 286–291
  erken nesiller, 240–242
  geçmişi, 240–248
  uygulama, 268–275
  betik dilleri, 250
  sözdizimi, 269–270
  evrensel, 516–521Programlama paradigmaları, 244–248
Programlar, 190
  veri karşısında, 88
  doğrulama, 226–230PROJE işlemi, 394, 395, 398–399
Projeksiyon düzlemi, 428Projeksiyon cihazları, 428
Prolog, 288–291
Telif hakkı olan ağ, 140Protokoller, 141–143

İnternet, 167–173Prototipleme, 307Proxy sunucu, 176Sahte kod, 192–197Açık anahtarlar, 177, 536Açık anahtar şifrelemesi, 177–179, 536–539
Delikli kartlar, 5Push, yığın işlemi, 343
Python, 269

Kalite güvencesi, 324–327
Kuyruk, 111, 343–344, 355–357

Radyo Dükkanı, 7
Radyo ışını, 451–452
Taban noktası, 45
RAM. *Bak* Rastgele erişim belleği (RAM)
Rastgele erişim belleği (RAM), 28
Hızlı prototipleme, 307Rasterleştirme, 443Rasyonel birleşik süreç (RUP), 307
Işın izleme, 449–452
Salt okunur bellek (ROM), 120
Okuma/yazma başlıkları, 29–30
Hazır, işlem, 122Gerçek veri türü, 250Gerçekçilik, 435–437Gerçek zamanlı işleme, 112Gerçek dünya bilgisi, 484–485
Akıl yürütme, 473–484
Kayıt, 252–253
Özyineleme, 221
Özyinelemeli fonksiyon teorisi, 510
Özyinelemeli ışın izleme, 451
Özyinelemeli yapılar, 214–222
Azaltılmış komut seti bilgisayarı (RISC), 77–78
Yansıma, 439–441
Kırılma, 441–442Yenileme devresi, 28Bölge bulma, 469Kayıt birimi, 74Kayıtlar, 74–75Kayıt memurları, 152Güçlendirme, 487Veri tabanındaki ilişki, 289İlişkisel veritabanı modeli, 389–399
İlişkisel tasarım, 389–393İlişkisel işlemler, 393–397
İlişkiler, 389
Göreceli kodlama, 58–59Renderlama, 428, 439–449Renderlama boru hattı, 442–444, 448–449
Tekrar döngüsü, 208, 209
Tekrar edici, 144
Gereksinim analizi, 304
Research in Motion (RIM), 333
Ayrılmış kelimeler, 269

Doclingo https://www.doclingo.ai

Çözünürlük, 286–288
Çözümleyici, 286
Kaynak tahsisi, 125–129
Yazılım geliştirmede
Gözden geçirmeler, 325–326
RGB kodlama, 38
Sağ çocuk işaretçisi, 358
RISC. *Bak* Azaltılmış komut set bilgisayarı (RISC)
Riskler Forumu, 325
Ritchie, Dennis, 553
Rivest, Ron, 178, 536
Robocup, 498
Robotik, 497–500
Geri alma, 404
ROM (salt okunur bellek), 120
Kök düğüm, 344
Kök işaretçisi, 358
Rossum, Guido von, 269
Dönüş, 92
Dönüş gecikmesi, 30
Yuvarlama hatası, 55–57
Yönlendirici, 145–146
Yönlendirme, 173
Satır ana sırası, 349
RSA algoritması, 536, 537–539
Koşu uzunluğu kodlaması, 58
RUP. *Bak* Rasyonel Birleşik Süreç (RUP)

Ölçekleme, 113
Taramalı dönüşüm, 443–445
Sahne, 430
Sahne grafiği, 437–438, 442, 448
Zamanlayıcı, 119
Şema, 385–386
Değişkenin kapsamı, 261
Betik, 250
Betik dilleri, 250
SD (Güvenli Dijital) bellek kartları, 33
SDHC (Yüksek Kapasite) bellek kartları, 33
SDRAM, 28
SDXC (Genişletilmiş Kapasite) bellek kartları, 33
Arama motoru, 8, 165
Arama süreci, 214–220
Araştırma ağaçları, 476–478
Sektör, 30
Güvenli Kabuk (SSH), 155
Güvenli Soket Katmanı (SSL), 177
Güvenlik
ağ, 173–181
işletim sistemi, 130–132
Arama süresi, 30
SEÇİM işlemi, 393–394, 398–399
Kendine atıf, 522Kend
ini sonlandırma, 523, 525Anla
msal analiz, 470Anlamsal a
ğ, 471Anlamsal Web,
165Anlam, 192

Semaphore, 125–127
Gözcü, 407
Sıra diyagramı, 321–323Sıralı dosyalar, 406–408Sıralı d
esen analizi, 415Sıralı arama algo
ritması, 204–205
Seri iletişim, 98
Sunucu, 146
Sunucu tarafı etkinlikleri, 165–166
Gölgeleme, 445–447
Shamir, Adi, 178, 536
Şekil, modelleme, 431–435Pay
laşılan kilit, 405
Kabuklar, 116, 117
Kaydırma işlemleri, 92
Shockley, William, 7
Kardeşler, 344
İşaret biti, 48
SIMD mimarisi, 101Te
k Hassasiyetli Kayan No
kta, 56
SISD mimarisi, 101Akıll
ı telefon, 9, 113, 156–157, 285, 302, 329, 472
Düzgünleştirme, 469SMTP (Basit Posta Aktarım P
rotokolü), 154
Dinleme yazılımı, 130–131, 174–175
Sosyal Güvenlik kayıtları, 417Yum
uşak telefonlar, 156
Yazılım, 2
uygulama, 115, 116
sınıflandırma, 114–115
çapraz platform, 243
event-tabanlı, 266
akıllı telefon, 285
sistem, 115
test etme, 326–327
araç, 115–116
doğrulama, 226–230
Yazılım analisti, 305–306
Yazılım geliştirme paketleri, 275
Yazılım mühendisliği, 299–302
bilgisayar destekli, 301
dökümantasyon, 328–329
metodolojiler, 306–308
modülerlik ve, 308–316
kalite güvencesi, 324–327
gerçek dünya, 315
standartlar, 302
araçlar, 316–324
Yazılım lisansı, 332–333
Yazılım yaşam döngüsü, 302–306
tasarım, 304–305
uygulama, 305–306
gereksinim analizi, 304
test, 306
Yazılım kalite güvencesi (SQA) grupları, 325
Yazılım gereksinimleri belirlemesi, 304
Yazılım doğrulama, 226–230
Ses, temsili, 40
Kaynak program, 268
Web sayfasının kaynak versiyonu, 160
Alan karmaşıklığı, 531
Spam, 175
Spam filtreleri, 176
SPARK, 230
Özel amaçlı kayıtlar, 74–75
Yansıyan ışık, 440
Sahtecilik, 176
Spooling, 128–129
Casus yazılım, 174–175
SQL, 397–399
SSH. *Bakınız* Güvenli Kabuk (SSH)
SSL. *Bakınız* Güvenli Soketler Katmanı (SSL)

Doclingo https://www.doclingo.ai

Yığın, 343, 355–357Yı
ğın işaretçisi, 355–356Pa
ydaşlar, 304Standa
rt Şablon Kütüphanesi
(STL), 372
Yıldız topolojisi, 141Başl
angıç durumu, 473
Durum, 473
Durum grafiği, 474
Statik veri yapıları, 346
Durum kelimesi, 97
Adım adım ayrıştırma, 202
Stibitz, George, 5
Depolama
ikili ağaçların, 357–360
bitin, 24
kesirlerin, 53–58tam
sayıların, 47–53listel
erin, 352–355, 362–367yığın v
e kuyrukların,
355–357
STORE op-kodu, 96Sa
klanan program kavramı,
75–76, 77
Storyboard, 453–454
Akış, 24
Akışkan ses, 157
Güçlü AI, 468
Güçlü tipli diller, 273
Yapı, 252–253
Yapı şeması, 309Yapıla
ndırılmış programlama, 256Yapıla
ndırılmış Sorgu Dili
(SQL), 397–399
Yapılandırılmış yürüyüşler, 323
Alt alan, 153
Alt program, 195
Alt yordam, 195
Alt şema, 386
Alt ağaçlar, 344
Sun Microsystems, 249, 271
Süper kullanıcı, 130
Denetimli eğitim, 486
Yüzey modelleme, 435
Anahtar, 144, 145
Anahtar ifadesi, 257–258
Sembol tablosu, 273
Sözdizimsel analiz, 470
Sözdizimi, 192, 269–270
Sözdizimi diyagramları, 269
Sistem yöneticisi, 113
Sistem analisti, 305
Sistem belgeleri, 328
Sistem yazılımı, 115
Çip üzerindeki sistem yaklaşımı, 100

Etiket, bir işaretleme dilinde, 160
Liste, 342
Son gösterici, 356
Görev Yöneticisi, 125
TB (terabayt), 28
TCP. *Bak* İletim Kontrol
Protokolü (TCP)
TCP/IP protokolü, 140, 171–173
Teknik belgeler,
328–329
Teknolojik iler
leme, 500
Telnet, 155
Şablon, 314–315, 324Zamansal
maskeleme, 62Terminal, bir söz
dizimi diagramında, 270Termi
nal düğümü, 344S
onlandırma koşulu,

206–207, 212
Test et ve ayarla talimatı, 126T
est etme, yazılım, 326–327T
est aşaması, yazılım yaşamdön
güsü, 306
Metin, temsili, 35–36Metin d
üzenleyici, 36
Metin dosyası, 36, 406–407
Doku haritalama, 435
Therac-25, 325
Üçüncü nesil programlama
dilleri, 241–244
İş parçacığı, 283–284
Üç bit fazlası sistemi, 52
Eşik değeri, 490
Verim, 100
Atıl prototipleme, 307
Seviye-1 ISS'leri, 150
Seviye-2 ISS'leri, 150
TIFF (Etiketli Resim Dosyası
Formatı), 61–62
Zaman karmaşıklığı, 528
Zaman paylaşımı, 112
Zaman dilimi, 123
Jeton, 268
Yığın üstü, 343Yukarı
dan aşağıya metodoloji, 202Üst dü
zey alan adları (TLD'ler), 152
Torvalds, Linus, 116, 307
Parça, 29
Geleneksel geliştirme aşaması, yazılı
m yaşam döngüsü,
303–306
Eğitim seti, 486
Aktarım hızı, 30
Transistör, 7
Çeviri süreci, 268–274
Çevirmen, 242
İletim Kontrol Protokolü
(TCP), 171–172
Şeffaf nesne, 450–451Taşıma
katmanı, 168–171Seyaha
t eden satıcı problemi,
533–534
Ağaç, 344–345
ikili, 357–360, 365–367
arama, 476–478
Truva atı, 174
TrueType, 39
Kesme hatası, 55–57İlişkid
eki demet, 389
Turing, Alan, 465, 513, 515
Turing hesaplanabilir, 515
Turing makineleri, 512–516T
uring testi, 465–466İki'
nin tamamlayıcı notasyonu,
38, 47–50
Tür dönüştür
me, 273Tür yükseltm

e, 273UDP. Kullanıcı Dat
agramProtokol
ü (UDP)UML. Birleşik Model
lemeDili (UML)
Koşulsuz atlamalar, 79–80
Unicode, 36
Birleşme, 288
Birleşik Modelleme Dili
(UML), 318–323
Birleşik süreç, 307
Eşit Kaynak Bulucu
(URL), 159–160

Doclingo https://www.doclingo.ai

Evrensel makine
dilleri, 271
Evrensel programlamadi
li, 516–521Evrense
l seri veri yolu (USB), 95,
97, 98–99
UNIX, 110, 117
İnsansız hava araçları(UAV'ler),
498
Çözülemeyen problemler, 52
6Kentsel Meydan Okum
a, 498URL. *Bak* Evrensel Kaynak
Bulucu (URL)
USA PATRIOT Yasası, 180
USB. *Bak* Evrensel seri data
yolu (USB)
Kullanım durumu diyagramı, 318, 319
Kullanım durumları, 318
Kullanıcı Datagram Protokolü (UDP),
171–172
Kullanıcı belgeleri, 328, 329
Kullanıcı arayüzü, 116–117, 329–332
Kullanıcı tanımlı veri türü, 368–369
Faydacılık, 14–15
Yardımcı yazılım, 115–116

Vakum tüpü, 6
Değişken uzunlukta talimatlar, 78
Değişken
atanma, 278
global, 261
örnek, 277ye
rel, 261ka
psamı, 261
VBScript, 250
Vektörler, normal, 446
Çok büyük ölçekli entegrasyon
(VLSI), 24
Video sıkıştırma, 62
Video oyunları, 429–430, 448, 454
Görüş noktası, 428
Görüş hacmi, 442
Sanal bellek, 119
Erdem etiği, 15
Virüs, 174
Visual Basic, 250, 265
VoIP (İnternet Üzerinden Ses
Protokolü), 155–157V
on Koch kar tanesi, 434Von
Neumann mimarisi, 97Von Neuma
nn darboğazı, 97
Von Neumann, John, 77
VxWORKS, 114

W3. *Dünya Çapında Ağ'a b
akınızW3C. Dünya Çapında A
ğ'a bakınızKonsorsiy
umu (W3C)Bekleme, işl
em, 122WAN. Geniş a
lan ağı (WAN)'a b
akınızŞelale modeli, 30
6Zayıf AI, 468*
Dokuma tezgahı, 5We
b. Dünya Çapında Ağ'a bakınız
Web mail, 166
Web sayfaları, 158, 160–162, 163
Web sunucusu, 159, 166
Web siteleri, 158
Ağırlık, bir yapay
neuron'da, 490
Weizenbaum, Joseph, 501
Galce, Terry, 59
While döngüsü, 208, 229
While ifadesi, 206, 207, 212,
256–257
Geniş alan ağı (WAN), 140
WiFi, 142, 143
Pencere yöneticisi, 117
Windows, 110, 114–115, 117,
125, 265
Windows CE, 114
Kablosuz telefon, 156Kelim
e işlemci, 36Dünya Ça
pında Ağ, 8, 158–167Dünya Çapı
nda Ağ Konsorsiyumu
(W3C), 160
Solucan, 174
En kötü durum analizi, 223–226Y
ara bekleme protokolü, 405
Wozniak, Stephen, 7
WWW. *Bak* Dünya Çapında Ağ

X11, 117
XML. *Bak* Genişletilebilir İşar
etleme Dili (XML)
XOR (özel veya), 20, 21, 22,
90–92
XP. *Bak* Aşırı pr
ogramlama (XP)

Yahoo, 8

Z-büfesi, 445
Ziv, Jacob, 59
Bölgelendirilmiş bit kaydı, 30

Doclingo https://www.doclingo.ai